

MBSE-DEVSECOPS Primer

A Case Study

A case study: Jim, Bob and Tom take Tidewater from SysML v1 to SysML v2 in GitLab in twelve weeks • October 2026

MBSE-DEVSECOPS Primer

A case study: Jim, Bob and Tom take Tidewater from SysML v1 to SysML v2 in GitLab in twelve weeks

The agile kit

First edition, October 2026

Tidewater, its people, its prime contractor and every number in this book are fictional and notional. Any resemblance to a real program, organization or person is coincidental. Nothing here describes a real system.

The commands, scripts, templates and pipeline in this book are real: they ship with the agile kit (core Perl, Git for Windows, Vim) and every artifact quoted was produced by running them. Rebuild the evidence with perl sim/tidewater-sim.pl and this book with perl sim/primer.pl.

Typeset with XeLaTeX in TeX Gyre Pagella, TeX Gyre Heros and DejaVu Sans Mono.

For every engineer who was told to go digital
and was not told why.

Bottom line up front

54% of the work the team finished in the first twelve weeks is actual systems engineering, and the share rises sprint by sprint. The rest is a one-time investment in tools and learning: 73% of sprint 1, down to 18% of sprint 6. The figure for the whole eighteen months follows in the next edition, when the later sprints have run.

If you read nothing else in this book, read this page. It is written for the people who have to approve the change, and for the engineers who have to live with it.

Where the team's work goes

Every story on the team's board carries one of three labels. *Engineering* is work on the system itself: requirements, structure, behavior, interfaces, verification. *Platform* is the DevSecOps machinery that checks and publishes that work: the GitLab project, the pipeline, the runner. *Learning* is time the team spends learning the new tools, budgeted on purpose so it does not happen on nights and weekends.

Table 1: Task points closed in each sprint, by kind of work. One point is one ideal person-day.

Sprint	Task points done	Systems engineering	DevSecOps platform	Learning
1	16.5	27%	48%	24%
2	19	32%	53%	16%
3	15	53%	30%	17%
4	17	74%	18%	9%
5	16.5	64%	30%	6%
6	14	82%	14%	4%
Sprints 1-6	98	**54%**	33%	13%

These are *task* points: every finished task counts, including the finished tasks of a story that carried over, so a sprint's figure can be higher than its story points done (Section 13.8, Two ways to count "done" (p. 205)).

Where all the time goes, meetings included

Points cover the work on the board. The agile meetings come on top: a fifteen-minute stand-up each day, a kickoff, a refinement session, and a review with a retrospective at the end of each sprint. Together they take about 7 hours per person per sprint, or 9% of an eighty-hour sprint.

Table 2: All paid hours of the three-person team in the first twelve weeks.

Activity, sprints 1-6	Share of all paid hours
Systems engineering	29%
DevSecOps platform	18%
Learning	7%
Agile meetings	9%
Not on the board: holidays and leave, the lead's management duties, interruptions	37%

What this means

- **The overhead is real, visible and temporary.** Platform and learning work dominate the first two sprints, because the team is building the shop and learning to use it. By sprint 6 most of the board is engineering again. A plan that hides this cost is a plan that fails in week three.
- **The meetings cost less than a tenth of the team's time,** and they replace status meetings, review meetings and the slide-building that used to come before every milestone.
- **The engineering that remains is faster and checked.** Every change is checked automatically and reviewed before it is accepted. Problems the team used to find at the next milestone review are found the same day. The measured results – requirements traced, model items ported, review turnaround, problems caught early – are in Chapter 33, The twelve-week scorecard (p. 520), and the briefing the team gave management at the end of the twelve weeks is in Chapter 35, The briefing (p. 532).
- **What management must provide** is the learning time in this table, a runner machine, the tool approvals and a GitLab tier decision, asked for in writing in sprint 2 (Chapter 37, Writing it down: DoD-style memos from Vim (p. 545)).

Note: Where these numbers come from

The tables are computed from the team's plan (`sim/tidewater-plan.pl`) by `perl sim/tidewater-split.pl`, and the book cites the resulting file, `docs/tidewater/work-split.json`, rather than typing the numbers. When the simulation's results replace the plan in that file, these tables follow. The method is described in Chapter 46, Systems engineering work, not pipeline work (p. 636); you can apply it to your own project's backlog in Chapter 50, Your project in twelve weeks: a playbook (p. 672).

Contents

Preface **xxxii**

I Where they start **1**

1 Thirty years of documents **2**
1.1 A Tuesday in 2019 . . . 2
1.2 The document set . . . 3
1.3 What worked . . . 4
1.4 What it cost . . . 5
1.5 The habits worth keeping . . . 6
1.6 What the spreadsheets were really doing . . . 7
1.7 A year in the document era . . . 8
1.8 Why the document era lasted . . . 8
1.9 Exercises . . . 9

2 The Cameo years **10**
2.1 "Go MBSE" . . . 10
2.2 What the team built . . . 11
2.3 Measuring the spider web . . . 12
2.4 A tour of drop 1, package by package . . . 15
2.5 Why it did not scale . . . 16
2.6 What the team believed, and what was true . . . 18
2.7 Reviewing a drop by screenshot . . . 19
2.8 Who owns what, after the contract change . . . 20
2.9 What a model is, and what a diagram is . . . 20
2.10 Exercises . . . 21

3 The directive **22**
3.1 The memo . . . 22
3.2 Reading it carefully . . . 23
3.3 The meeting with the program manager . . . 24
3.4 The team meeting . . . 24
3.5 The prime's view . . . 27
3.6 Defining measurable progress . . . 28
3.7 Jim's plan in one page . . . 30
3.8 The email Jim sent that night . . . 31
3.9 Week zero . . . 31

3.10 What management needed to hear, and when . . . 32
3.11 Exercises . . . 33

4 The Vee, kept: the systems-engineering V in MBSE DevSecOps 34
4.1 Jim's whiteboard . . . 34
4.2 The Vee in one paragraph . . . 35
4.3 What DevSecOps changes: three Vees, not one . . . 35
4.4 The left arm: the model's layers . . . 36
4.5 The right arm: checks at every level . . . 37
4.6 The horizontal lines: verify, in the same merge request . . . 38
4.7 The gates: technical reviews as baselines . . . 38
4.8 Change control between the reviews . . . 39
4.9 The Vee, then and now . . . 40
4.10 What stays the same, and what changes . . . 40
4.11 Where Tidewater sits on the Vee . . . 41
4.12 Variants you may meet . . . 41
4.13 Digital thread and digital twin . . . 41
4.14 Pitfalls . . . 43
4.15 You can now . . . 44
4.16 Applying this to your project . . . 44

5 DevSecOps in systems-engineering words 46
5.1 Tom's Saturday . . . 46
5.2 What the word means . . . 47
5.3 Eight ideas, in systems engineering terms . . . 47
5.4 The translation table . . . 50
5.5 One change, end to end . . . 52
5.6 Security for a model team, in practice . . . 53
5.7 Reading a pipeline log, before you ever see one . . . 54
5.8 What changes for each person . . . 55
5.9 Exercises . . . 56

6 SysML v2 for SysML v1 modelers 57
6.1 Bob reads the converter output . . . 57
6.2 Why text, and why now . . . 59
6.3 Five ideas that cover most of SysML v2 . . . 59
6.4 Side by side: the solar panel . . . 62
6.5 Side by side: a requirement and its satisfy . . . 62
6.6 Where the converter stops . . . 64
6.7 Writing what the converter could not . . . 65
6.8 Bob's Wednesday, again . . . 67
6.9 Three rules for reading SysML v2 text . . . 68
6.10 Names, identifiers and the way back to v1 . . . 69
6.11 The standard library . . . 70
6.12 A first session with the text, in Vim . . . 70
6.13 Habits from SysML v1 to unlearn . . . 70
6.14 Reading the converter report, column by column . . . 71
6.15 Exercises . . . 72

7 Requirements a pipeline can check: the recipe **73**
7.1 Jim's red pen . . . 73
7.2 The recipe in one sentence . . . 74
7.3 The parts, one at a time . . . 75
7.4 The worked example, decomposed . . . 79
7.5 What the checker looks for . . . 82
7.6 Tidewater's requirements, read by the recipe . . . 83
7.7 Rewriting to the recipe . . . 85
7.8 Who may change the words . . . 87
7.9 The recipe in SysML v2 . . . 87
7.10 The recipe in the pipeline . . . 89
7.11 Habits . . . 90
7.12 Exercises . . . 90
7.13 Applying this to your project . . . 91

8 SysML v2 by example: a workbook **92**
8.1 How to use this workbook . . . 92
8.2 Packages, imports and visibility . . . 94
8.3 Names, documentation and comments . . . 95
8.4 Parts: definitions, usages, multiplicity . . . 96
8.5 Redefinition and subsetting . . . 98
8.6 Ports and interfaces . . . 99
8.7 Connections and flows . . . 100
8.8 Attributes, values and units . . . 101
8.9 Expressions: the small language inside constraints . . . 103
8.10 Requirements: definitions, usages, subjects and constraints . . . 104
8.11 Satisfy, verify and derive . . . 105
8.12 Verification cases . . . 106
8.13 States . . . 107
8.14 Actions, successions, forks and flows . . . 109
8.15 Calculations, constraints and analysis cases . . . 110
8.16 Allocations . . . 111
8.17 Metadata . . . 112
8.18 Use cases . . . 114
8.19 Variants . . . 114
8.20 Views and viewpoints . . . 115
8.21 Reading the official library . . . 116
8.22 Reading a converted file, top to bottom . . . 117
8.23 Putting it together: a model the pipeline accepts . . . 119
8.24 Your port worklist . . . 121
8.25 What the workbook leaves out . . . 123
8.26 Applying this to your project . . . 123

II Week zero **124**

9 The survival kit **126**
9.1 Monday, 08:00 . . . 126

9.2 What is on the laptop . . . 127
9.3 The terminal . . . 128
9.4 Files, folders and paths . . . 129
9.5 The commands you need . . . 130
9.6 Getting the kit . . . 135
9.7 Setting up git . . . 136
9.8 Proving who you are to GitLab: SSH keys and tokens . . . 137
9.9 Setting up Vim . . . 138
9.10 Shell shortcuts . . . 139
9.11 GitLab in the browser . . . 140
9.12 When something goes wrong . . . 140
9.13 The day-one checklist . . . 141
9.14 Applying this to your project . . . 142
9.15 Exercises . . . 142

10 Vim for models **143**
10.1 The one thing they brought . . . 143
10.2 What the kit adds . . . 144
10.3 Reading a model . . . 145
10.4 The TODO list as a jump list . . . 145
10.5 Closing a TODO line: a worked edit . . . 146
10.6 Checking from Vim . . . 148
10.7 Tags: jumping by name and by DOORS id . . . 150
10.8 Seeing the model . . . 150
10.9 Merge conflicts in Vim . . . 151
10.10 Commit messages, issues and merge requests in Vim . . . 153
10.11 The terminal inside Vim . . . 155
10.12 The stand-up and the journal from Vim . . . 155
10.13 A card for the monitor . . . 155
10.14 Applying this to your project . . . 156
10.15 Exercises . . . 156

11 Seeing the model: diagrams from text **158**
11.1 “Where are my diagrams?” . . . 159
11.2 Views are outputs, not inputs . . . 159
11.3 Drawing them from the shell . . . 160
11.4 The five views, read . . . 161
11.5 IDEF0 plates from the shell . . . 164
11.6 Drawing them from Vim . . . 165
11.7 The live viewer beside Vim . . . 166
11.8 Comparing two versions . . . 166
11.9 Plates for briefings . . . 167
11.10 The model site: views for management . . . 167
11.11 The trees as text . . . 168
11.12 Views as measures . . . 168
11.13 What a view cannot show . . . 168
11.14 When a view will not draw . . . 169
11.15 Applying this to your project . . . 169

12 Git from zero, for systems engineers **171**
12.1 Tuesday afternoon, a throwaway folder . . . 171
12.2 Version control in configuration management words . . . 172
12.3 Three places a change lives . . . 173
12.4 A practice repository . . . 173
12.5 The first commit . . . 174
12.6 Changing a file: the diff . . . 175
12.7 Commit messages a CCB would accept . . . 176
12.8 Undoing things . . . 177
12.9 Branches . . . 178
12.10 Merging . . . 179
12.11 Merge conflicts . . . 180
12.12 Putting work aside: stash . . . 183
12.13 Remotes: clone, push and pull . . . 184
12.14 Tags: baselines . . . 186
12.15 Reading history: log, show and blame . . . 187
12.16 Files git should ignore . . . 188
12.17 The rules the team adopted . . . 188
12.18 Branching strategies . . . 189
12.19 The vendor branch . . . 191
12.20 Applying this to your project . . . 194
12.21 Exercises . . . 195

13 The operating model **196**
13.1 Sunday evening, the whiteboard . . . 196
13.2 Why an operating model, and why so small . . . 197
13.3 The five levels . . . 198
13.4 The Tidewater tree . . . 201
13.5 Where each level lives . . . 202
13.6 The cadence . . . 203
13.7 Capacity . . . 204
13.8 Two ways to count “done” . . . 205
13.9 Roles and decisions . . . 206
13.10 Definitions of Ready and Done . . . 208
13.11 The decisions the team wrote down . . . 210
13.12 What the operating model is not . . . 211
13.13 Applying this to your project . . . 211
13.14 Exercises . . . 212

14 Writing stories and tasks for model work **214**
14.1 The Monday the backlog was empty . . . 215
14.2 Stories, tasks and where each lives . . . 215
14.3 A story in systems-engineering words . . . 215
14.4 Writing the story issue . . . 216
14.5 Acceptance criteria as pipeline checks . . . 217
14.6 Writing the task issues . . . 217
14.7 Try it: a story and its first task . . . 218
14.8 Splitting patterns for model work . . . 219

14.9 Sizing: points, poker and the first sprints . . . 220
14.10 The learning budget . . . 220
14.11 The Definition of Ready, applied . . . 221
14.12 A worked example: from an epic to its tasks . . . 221
14.13 The story in the journal . . . 222
14.14 Common mistakes, and their fixes . . . 223
14.15 When a story carries over . . . 223
14.16 A backlog to start from . . . 224
14.17 Applying this to your project . . . 224

15 Setting up GitLab 226
15.1 Tom's first request . . . 226
15.2 What GitLab is, in systems engineering terms . . . 227
15.3 The decisions behind the settings . . . 228
15.4 Settings as a file: project.conf . . . 231
15.5 Applying the file . . . 235
15.6 The first commit and the two long-lived branches . . . 236
15.7 Proving the protections work . . . 237
15.8 Code owners: the signature authority matrix . . . 239
15.9 Templates: the forms . . . 241
15.10 Wiki, README and CHANGELOG . . . 242
15.11 Signing in: SSH keys and tokens . . . 242
15.12 What the enterprise administrators must do, and what the team does . . . 243
15.13 Exercises . . . 243
15.14 Applying this to your project . . . 244

16 The model repository: what goes where 245
16.1 Why the layout is decided at drop 1 . . . 246
16.2 Layers: dependencies point one way . . . 246
16.3 The layout map . . . 247
16.4 The whole repository . . . 248
16.5 The files at the top, one by one . . . 249
16.6 What never goes in the repository . . . 250
16.7 The rules that make the layout work . . . 251
16.8 Decisions and conventions on file . . . 251
16.9 Compared with a reference layout . . . 252
16.10 When one repository is not enough . . . 252
16.11 Applying this to your project . . . 253

17 The DevSecOps tools that earn their keep 255
17.1 The problem with "DevSecOps platform" . . . 255
17.2 The list at a glance . . . 256
17.3 Sprint 1: the work and the gate . . . 257
17.4 Sprint 2: making the merge request say what is wrong . . . 259
17.5 Sprint 3: the "Sec" . . . 261
17.6 Sprint 4: the model as a website . . . 263
17.7 Gates, and what they cost . . . 263
17.8 Sprint 5: deliveries and releases . . . 264

17.9 Sprint 6: showing the flow . . . 265
17.10 Tokens: which one, where . . . 266
17.11 What the team deliberately left off . . . 267
17.12 The cost side . . . 267
17.13 Exercises . . . 268
17.14 Applying this to your project . . . 268

18 A local AI assistant on the laptop 269
18.1 What a local model is, in plain words . . . 270
18.2 Why local, and not a cloud chatbot . . . 271
18.3 Approval comes first . . . 271
18.4 Two ways to host it: every laptop, or one shared server . . . 273
18.5 Installing on a laptop, without admin rights and offline . . . 273
18.6 A one-day bake-off before you standardize . . . 274
18.7 The SysML v2 caveat . . . 275
18.8 The rules . . . 276
18.9 Using it from Git Bash and from Vim . . . 277
18.10 Worked uses, best fit first . . . 278
18.11 What the team took from it . . . 286
18.12 Applying this to your project . . . 287

19 CI/CD from zero 288
19.1 Tom's question . . . 288
19.2 CI: continuous integration, in systems engineering words . . . 289
19.3 CD: continuous delivery . . . 290
19.4 The anatomy of a pipeline . . . 290
19.5 What starts a pipeline . . . 292
19.6 Every job starts from nothing . . . 292
19.7 Reading a pipeline . . . 293
19.8 Running it on your own machine first . . . 296
19.9 Gates . . . 297
19.10 Shift left: catch it at the merge, not at integration . . . 297
19.11 The Sec in DevSecOps . . . 298
19.12 Growing a pipeline one job at a time . . . 298
19.13 The minimum pipeline for any SysML v2 text repository . . . 299
19.14 Explaining a red pipeline to a manager . . . 300
19.15 Exercises . . . 300
19.16 Applying this to your project . . . 301

20 The pipeline, job by job 302
20.1 Reading YAML in ten minutes . . . 302
20.2 The shape of the file . . . 303
20.3 The header: stages, variables, workflow . . . 303
20.4 The conditional include and the defaults . . . 304
20.5 Stage check: the gate . . . 306
20.6 Stage trace: DOORS against the model . . . 310
20.7 Stage docs: generated documents are current . . . 313
20.8 Stage delta: what the prime changed . . . 313

20.9 Stage release: the baseline audit . . . 315
20.10 Stage publish: the model site . . . 316
20.11 Stage report: the weekly export . . . 317
20.12 The whole run, and what it costs . . . 318
20.13 Changing the pipeline . . . 319
20.14 The complete file . . . 319
20.15 Exercises . . . 346
20.16 Applying this to your project . . . 346

21 Runners: where the pipeline runs 347
21.1 GitLab runs nothing . . . 347
21.2 Who a runner serves: scopes . . . 348
21.3 Hosted or your own: the decision . . . 348
21.4 What to put in the request . . . 349
21.5 How a runner runs a job: executors . . . 349
21.6 The lesson of the first week: PowerShell . . . 350
21.7 Registering the runner . . . 352
21.8 The service account: least privilege . . . 353
21.9 Tags, pending and stuck . . . 353
21.10 Protected runners . . . 354
21.11 Keeping the runner healthy . . . 354
21.12 The Docker alternative . . . 355
21.13 The laptop is a runner too . . . 355
21.14 Exercises . . . 355
21.15 Applying this to your project . . . 356

22 Reviewing a model change 357
22.1 Slide 37 . . . 358
22.2 What the pipeline already checked . . . 358
22.3 The reviewer's question . . . 358
22.4 A review, start to finish . . . 360
22.5 Suggestions: the redline you can apply . . . 362
22.6 Different changes, different questions . . . 363
22.7 Reviewing a prime drop merge . . . 364
22.8 Writing findings that teach . . . 364
22.9 Approve, ask for changes, or ask to split . . . 365
22.10 Turnaround . . . 365
22.11 Applying this to your project . . . 366

23 Stand-ups, the journal and the board 367
23.1 Two records, one plan . . . 367
23.2 Everything is a ledger . . . 368
23.3 Setting up the journal . . . 369
23.4 The stand-up file . . . 370
23.5 What the journal tells you . . . 372
23.6 Estimates that change . . . 373
23.7 A story from end to end . . . 374
23.8 The board and the journal, day to day . . . 374

23.9 The reports . . . 375
23.10 Exercises . . . 376
23.11 Applying this to your project . . . 376

24 The rhythm: stand-ups, poker, refinement, kickoff, demo, retro 377
24.1 Why meetings at all . . . 377
24.2 The daily stand-up: terse, written, linted . . . 378
24.3 Planning poker . . . 382
24.4 Refinement and the Definition of Ready . . . 384
24.5 Kickoff and planning . . . 386
24.6 The demo . . . 386
24.7 The retrospective . . . 387
24.8 Telling management every week . . . 387
24.9 Exercises . . . 388
24.10 Applying this to your project . . . 389

25 Talking about progress: helps for engineers who would rather not 390
25.1 Why engineers go quiet . . . 390
25.2 The pattern: outcome, evidence, next or ask . . . 391
25.3 The ladder: one sentence, thirty seconds, five minutes . . . 392
25.4 Tools that draft the words . . . 393
25.5 Know your audience: the Tidewater stakeholder map . . . 398
25.6 Demos that land . . . 398
25.7 Bad news, said so it builds trust . . . 400
25.8 Measuring whether it is working . . . 401
25.9 Exercises . . . 401
25.10 Applying this to your project . . . 402

III Twelve weeks 403

26 Sprint 1: A GitLab project the team can work in 404
26.1 The question, and the mood . . . 404
26.2 Week zero, briefly . . . 405
26.3 The kickoff: Monday 5 October, 09:00 . . . 406
26.4 Week one . . . 408
26.5 Week two . . . 413
26.6 The numbers at the end of sprint 1 . . . 420
26.7 What Jim told management . . . 420
26.8 Tools adopted this sprint, and the pain each removed . . . 421
26.9 What went wrong, and why it was fine . . . 421
26.10 You can now... . . . 422
26.11 Exercises . . . 422
26.12 Applying this to your project . . . 422

27 Sprint 2: Drop 1 on develop, every change gated 424
27.1 The question, and the mood . . . 424
27.2 The kickoff: Monday 19 October . . . 425

27.3 Week one . . . 427
27.4 Week two . . . 431
27.5 The numbers at the end of sprint 2 . . . 435
27.6 What Jim told management . . . 436
27.7 Tools adopted this sprint, and the pain each removed . . . 437
27.8 What went wrong, and what the team did about it . . . 437
27.9 You can now... . . . 437
27.10 Exercises . . . 438
27.11 Applying this to your project . . . 438

28 Sprint 3: the first red X, DOORS in the model, and a wait nobody named 440
28.1 The question this sprint answered . . . 441
28.2 Kickoff: Monday 2 November . . . 441
28.3 Week 1 . . . 442
28.4 Week 2 . . . 450
28.5 What the numbers say . . . 455
28.6 Tools adopted this sprint, and the pain each one removed . . . 456
28.7 What Jim told management . . . 456
28.8 You can now . . . 457
28.9 Applying this to your project . . . 457

29 Sprint 4: the model site, a cycle broken, and the demo that landed 459
29.1 The question this sprint answered . . . 460
29.2 Kickoff: Monday 16 November . . . 460
29.3 Week 1 . . . 461
29.4 Week 2 . . . 467
29.5 What the numbers say . . . 472
29.6 Tools adopted this sprint, and the pain each one removed . . . 472
29.7 What Jim told management . . . 473
29.8 You can now . . . 473
29.9 Applying this to your project . . . 474

30 Sprint 5: Grading the requirements, and a trace that defends itself 475
30.1 The question this sprint answered . . . 475
30.2 Kickoff: a carried story, a small learning budget, mostly engineering . . . 476
30.3 Week 1: clutter out, a ratchet in . . . 477
30.4 The recipe: grading the requirements . . . 482
30.5 Week 2: metadata, a dry run, a tabletop and a catch . . . 485
30.6 Reading the sprint's own numbers . . . 488
30.7 What went wrong, and what the team did about it . . . 489
30.8 The review: numbers someone asked for . . . 490
30.9 The retrospective . . . 492
30.10 What Jim told management . . . 492
30.11 Tools adopted this sprint, and the pain each removed . . . 493
30.12 You can now... . . . 493
30.13 Exercises . . . 494
30.14 Applying this to your project . . . 494

31 Sprint 6: Behavior by hand, a table for the program, and baseline-0 **495**
31.1 The question this sprint answered . . . 495
31.2 Kickoff: a short sprint with a big ending . . . 496
31.3 Week 1: the quiet days and the busy one . . . 497
31.4 Week 2: two of three, and the ending . . . 502
31.5 Thursday 24 December: baseline-0 . . . 505
31.6 The carryover . . . 507
31.7 Reading the sprint's own numbers . . . 507
31.8 What went wrong, and what the team did about it . . . 508
31.9 The review and the retrospective . . . 508
31.10 Twelve weeks, in the measures agreed on day one . . . 509
31.11 What Jim told management . . . 509
31.12 Tools adopted this sprint, and the pain each removed . . . 509
31.13 You can now... . . . 510
31.14 Exercises . . . 510
31.15 Applying this to your project . . . 511

IV Making the case **512**

32 Measuring with GitLab analytics **513**
32.1 Two kinds of numbers . . . 514
32.2 The views GitLab offers . . . 514
32.3 The Free-tier route: gitlab.pl analytics . . . 515
32.4 Reading the five sections . . . 516
32.5 The trend across the twelve weeks . . . 517
32.6 Small numbers and other traps . . . 517
32.7 What goes in the briefing . . . 518
32.8 How this relates to the kit's other measures . . . 518
32.9 You can now . . . 518
32.10 Applying this to your project . . . 519

33 The twelve-week scorecard **520**
33.1 The whole scorecard on one page . . . 520
33.2 Conversion debt: the TODO burn-down . . . 521
33.3 Flow: lead time, review and gating . . . 522
33.4 Trace: computed every time, and flat . . . 522
33.5 Discrepancy reports . . . 523
33.6 The spider web: better, not gone . . . 523
33.7 Pipeline health and baselines . . . 524
33.8 Delivery and morale . . . 524
33.9 Measures beyond the twelve . . . 524
33.10 What the scorecard does not say . . . 525
33.11 You can now . . . 525
33.12 Applying this to your project . . . 525

34 Proving the port is complete and correct **527**
34.1 Complete, and correct . . . 528

34.2 Evidence 1: the converter accounts for every element . . . 528
34.3 Evidence 2: the TODO burn-down reaches zero . . . 528
34.4 Evidence 3: the trace agrees with DOORS . . . 529
34.5 Evidence 4: a sampled side-by-side review . . . 529
34.6 Evidence 5: the views say the same thing . . . 529
34.7 Evidence 6: the baseline rebuilds . . . 530
34.8 Where Tidewater stood at twelve weeks . . . 530
34.9 You can now . . . 531
34.10 Applying this to your project . . . 531

35 The briefing **532**
35.1 Preparing it . . . 532
35.2 The briefing, as given . . . 533
35.3 Slide 1: where we started . . . 534
35.4 Slide 2: what we built . . . 534
35.5 Slide 3: the scorecard, sprint by sprint . . . 534
35.6 Slide 4: before and after . . . 535
35.7 Slide 5: the demonstration . . . 535
35.8 Slide 6: next semester, and the ask . . . 536
35.9 What management asked, and how the numbers answered . . . 536
35.10 What the briefing left out . . . 536
35.11 You can now . . . 537
35.12 Applying this to your project . . . 537

36 Telling management **538**
36.1 Three rhythms . . . 538
36.2 The weekly quad . . . 539
36.3 The weekly note . . . 543
36.4 The sprint review report . . . 543
36.5 How the reports changed management's questions . . . 544
36.6 You can now . . . 544
36.7 Applying this to your project . . . 544

37 Writing it down: DoD-style memos from Vim **545**
37.1 Why memos still matter . . . 545
37.2 The format, and how Markdown maps to it . . . 546
37.3 The five templates . . . 546
37.4 Writing one . . . 548
37.5 Memos under review, like model changes . . . 551
37.6 Tidewater's memos . . . 552
37.7 The progress memo, step by step . . . 553
37.8 Which record for what: memo, ADR, issue, wiki . . . 554
37.9 What md2memo cannot express . . . 554
37.10 Exercises . . . 555
37.11 Applying this to your project . . . 556

38 How the team changed its mind **557**
38.1 The pulse, sprint by sprint . . . 558

38.2 The objections, revisited . . . 559
38.3 What actually changed minds . . . 561
38.4 You can now . . . 562
38.5 Applying this to your project . . . 562

39 A day in the life, week twelve **563**
39.1 09:00 – the stand-up, and the night's failure . . . 564
39.2 10:00 – a draft that the pipeline sends back . . . 565
39.3 14:05 – reviewing Tom's change . . . 566
39.4 15:30 – the day closes . . . 567
39.5 The same Thursday, 2023, the Cameo way (notional) . . . 568
39.6 You can now . . . 569
39.7 Applying this to your project . . . 569

40 Documents still matter: review stages for generated documents **570**
40.1 The scale . . . 571
40.2 The stages as GitLab labels . . . 571
40.3 Sprint 6: two documents through the stages . . . 572
40.4 What the pipeline can check, and what it cannot . . . 572
40.5 You can now . . . 573
40.6 Applying this to your project . . . 573

41 For the lead: leading a reluctant team **574**
41.1 What Jim was handed . . . 574
41.2 Make it safe to be slow and wrong . . . 575
41.3 Protect the time to learn . . . 576
41.4 Pair at the terminal . . . 577
41.5 Pick the first wins on purpose . . . 577
41.6 Watch the workload . . . 578
41.7 Manage up with dated plans . . . 579
41.8 Say no with a date . . . 580
41.9 Know when to slow down . . . 580
41.10 Hand over ownership, on purpose . . . 581
41.11 The lead is a beginner too . . . 582
41.12 The lead's fortnight . . . 582
41.13 Exercises . . . 583
41.14 Applying this to your project . . . 584

V Living with it **585**

42 Administration **586**
42.1 The Friday the nightly pipeline went red . . . 586
42.2 The administration calendar . . . 587
42.3 Members and access reviews . . . 588
42.4 Tokens and rotation . . . 590
42.5 The runner . . . 592
42.6 Protection audits . . . 596

42.7 Backups and restore tests . . . 597
42.8 Audit events . . . 598
42.9 Upgrades . . . 599
42.10 License tiers . . . 599
42.11 Applying this to your project . . . 600
42.12 Exercises . . . 601

43 Maintenance through the life cycle 602
43.1 The eighteen months at a glance . . . 603
43.2 The drop intake runbook . . . 604
43.3 Baselines and releases . . . 607
43.4 Hotfixes . . . 608
43.5 The change board in GitLab . . . 609
43.6 Discrepancy reports, upstream and back . . . 609
43.7 The life cycle beyond the port . . . 610
43.8 Retirement and archival . . . 610
43.9 Applying this to your project . . . 611
43.10 You can now... . . . 611

44 Verification as a closed loop 613
44.1 The loop . . . 613
44.2 One results format . . . 614
44.3 Sprint 6: the VCRM in the pipeline . . . 614
44.4 The testing transition, under pressure . . . 615
44.5 Higher fidelity: model, software and hardware in the loop . . . 616
44.6 Closing the loop on failures . . . 618
44.7 You can now . . . 618
44.8 Applying this to your project . . . 618

45 The prime's model changes: receive, assess, disposition 619
45.1 Contract first . . . 620
45.2 Receive: what arrived, and is it what was sent? . . . 622
45.3 Archive: keep it exactly as delivered . . . 623
45.4 Compare: what changed, element by element . . . 624
45.5 Assess: what does it touch in our model? . . . 624
45.6 Disposition: one issue per significant change . . . 626
45.7 Implement, and report back . . . 628
45.8 Two strategies at the v1-to-v2 boundary . . . 629
45.9 The prime's Jenkins: a delivery interface . . . 630
45.10 Cautions . . . 634
45.11 Applying this to your project . . . 634

46 Systems engineering work, not pipeline work 636
46.1 The line . . . 636
46.2 The balance, sprint by sprint . . . 637
46.3 The Tidewater catalog . . . 638
46.4 What platform work looks like, for contrast . . . 640
46.5 A generic catalog for your project . . . 641

46.6 Splitting an engineering story into tasks . . . 643
46.7 Applying this to your project . . . 643

47 Keeping SysML v2 from becoming a spider web 645
47.1 How tangled was v1, and how tangled is v2? . . . 645
47.2 Twenty practices . . . 646
47.3 Why the web grows, and how the guardrails stop it . . . 649
47.4 You can now . . . 650
47.5 Applying this to your project . . . 650

48 Security: protect the environment, engineer it into the system 651
48.1 Why the second half gets forgotten . . . 651
48.2 Half one: protecting the engineering environment . . . 652
48.3 Half two: engineering security into the system . . . 654
48.4 Process . . . 655
48.5 A worked example: the security gate . . . 655
48.6 Signing a baseline tag . . . 658
48.7 The spillage tabletop . . . 659
48.8 The twenty-six points at a glance . . . 659
48.9 Applying this to your project . . . 660

49 Scaling up: segments, blocks and a ladder of fidelity 662
49.1 A system of systems . . . 663
49.2 Integrated baselines . . . 664
49.3 The work plan, shared across segments . . . 665
49.4 The spine: requirements, functions, logical, physical . . . 665
49.5 Hardware: blocks, not one big Vee . . . 666
49.6 Software: sprints with a pipeline . . . 667
49.7 Verification: a ladder of fidelity . . . 668
49.8 The digital twin is a calibrated simulation . . . 669
49.9 Standards to plan around . . . 669
49.10 Where it breaks: the handoffs . . . 670
49.11 Exercises . . . 670
49.12 You can now . . . 671
49.13 Applying this to your project . . . 671

50 Your project in twelve weeks: a playbook 672
50.1 The shape of the twelve weeks . . . 673
50.2 Week zero, with the kit . . . 674
50.3 Sprint by sprint: what to watch . . . 675
50.4 Selling the plan in week zero . . . 676
50.5 The decisions that are yours . . . 676
50.6 The learning budget . . . 677
50.7 Your backlog . . . 678
50.8 What to measure . . . 679
50.9 What to copy from Tidewater, and what not . . . 680
50.10 When things go differently . . . 681
50.11 The first Monday . . . 682

50.12 Applying this to your project . . . 682

51 Pitfalls and anti-patterns 684
51.1 How to read this chapter . . . 684
51.2 Pitfalls in the model . . . 685
51.3 Pitfalls in the change process . . . 687
51.4 Pitfalls in the pipeline . . . 688
51.5 Pitfalls in the measures . . . 690
51.6 Pitfalls with people . . . 691
51.7 A monthly look in the mirror . . . 692
51.8 Applying this to your project . . . 693
51.9 Exercises . . . 693

Appendices 695

A Command reference 696
A.1 git . . . 696
A.2 gitlab.pl . . . 698
A.3 The kit's journal: daily.pl and scrum.pl . . . 701
A.4 The converters: xmi2sysml.pl and reqif2sysml.pl . . . 703
A.5 The model tools: model.pl and sysml.pl . . . 704
A.6 The pipeline's helpers . . . 705
A.7 Week zero, words, memos and requirement text . . . 705
A.8 Status metrics . . . 706
A.9 The kit itself . . . 706
A.10 Vim . . . 707

B Quick reference: Vim for MBSE DevSecOps 709
B.1 Survival . . . 709
B.2 SysML v2 files (`*.sysml`) . . . 709
B.3 The quickfix list: from error to line . . . 710
B.4 Merge conflicts . . . 710
B.5 Git from Vim . . . 711
B.6 Issue and merge request Markdown . . . 711
B.7 Journal and stand-up keys . . . 711

C Quick reference: git, gitflow and the GitLab tools 713
C.1 git, every day . . . 713
C.2 Gitflow, by branch type . . . 713
C.3 gitlab.pl at a glance . . . 714
C.4 Quick actions . . . 715
C.5 References and Markdown . . . 715
C.6 Pipeline: triggers and rules . . . 715
C.7 Variables and tokens . . . 716
C.8 Where things are in GitLab . . . 717

D Quick reference: SysML v1 to v2 719
D.1 What maps to what . . . 719

D.2 The TODO lines and their hand fixes . . . 720
D.3 Converter options . . . 721
D.4 Reading the v1 delta . . . 722
D.5 The drop cycle . . . 722
D.6 Trace measures . . . 723

E Rescue: when something goes wrong 724
E.1 Three facts that make mistakes cheap . . . 724
E.2 The practice set-up . . . 725
E.3 "I committed to develop by mistake" . . . 725
E.4 "I want my last commit back as edits" . . . 726
E.5 "I am editing on the wrong branch" . . . 726
E.6 "I lost a commit" . . . 726
E.7 "The merge is full of conflicts and I want out" . . . 727
E.8 "I merged, and it was the wrong thing" . . . 727
E.9 "A change on develop is wrong" . . . 728
E.10 "The pipeline is red" . . . 728
E.11 "I changed a file I should not have" . . . 729
E.12 "My clone is a mess" . . . 729
E.13 "I pushed a secret" . . . 729
E.14 "Vim says a swap file exists" . . . 730
E.15 "I broke a practice lesson" . . . 730
E.16 When to ask for help . . . 730

F Model error catalog 731
F.1 How to read a message . . . 731
F.2 The practice model . . . 732
F.3 Grammar check messages . . . 733
F.4 Lint messages . . . 734
F.5 Outline and level-tag messages . . . 735
F.6 Pipeline job messages . . . 736
F.7 Requirement recipe findings . . . 737
F.8 When the message is not here . . . 738

G Modeling style guide 739
G.1 Names . . . 739
G.2 Files and folders . . . 740
G.3 Inside a file . . . 740
G.4 Metadata, not stereotypes . . . 741
G.5 Relationships . . . 741
G.6 Requirement text . . . 742
G.7 TODO lines . . . 742
G.8 Commits and merge requests . . . 742
G.9 Branches, tags and issues . . . 742
G.10 Changing this guide . . . 743

H Quick reference: saying it so they hear it 745
H.1 The pattern . . . 745

H.2 Five stand-up shapes . . . 745
H.3 The ladder . . . 745
H.4 The demo in six lines . . . 745
H.5 The four numbers, every review . . . 746
H.6 Jargon to plain . . . 746
H.7 Bad news . . . 746
H.8 Tools . . . 746

I Templates and GitLab Markdown, field by field **747**
I.1 How GitLab uses templates . . . 747
I.2 GitLab Markdown in fifteen features . . . 748
I.3 Issue templates . . . 748
I.4 Merge request templates . . . 754
I.5 Repository files . . . 757
I.6 Wiki pages . . . 764
I.7 Filled-in samples . . . 774
I.8 Applying this to your project . . . 787
I.9 Exercises . . . 787

J Checklists **789**
J.1 Week zero . . . 789
J.2 Project set-up . . . 790
J.3 The sprint . . . 790
J.4 A change . . . 792
J.5 A prime drop . . . 792
J.6 A baseline and a hotfix . . . 793
J.7 Administration . . . 794
J.8 Before you call the port done . . . 795

K Glossary **796**
K.1 What you already do, and what GitLab calls it . . . 796
K.2 DevSecOps and GitLab words . . . 798
K.3 SysML v1 and SysML v2 . . . 799
K.4 Agile words . . . 800
K.5 Terms of this book and its kit . . . 801

L A twelve-week study plan per role **803**
L.1 Where the hours come from . . . 803
L.2 Week zero: everyone . . . 804
L.3 Jim, the lead . . . 804
L.4 Bob, the modeler . . . 805
L.5 Tom, the pipeline owner . . . 806
L.6 Check yourself . . . 807

M Further reading **809**
M.1 SysML v2 and the language underneath . . . 809
M.2 Systems engineering . . . 810
M.3 Git, GitLab and DevSecOps . . . 810

M.4 Security frameworks named in this book . . . 810
M.5 Agile, for systems engineers . . . 810
M.6 Vim . . . 810
M.7 The kit's own documents . . . 811

N References: DoD and Microsoft guidance 812
N.1 DoD software and DevSecOps guidance . . . 812
N.2 Acquisition policy . . . 813
N.3 Microsoft: Azure Government for DoD . . . 813

O Answers to the exercises 815
O.1 Part I: where they start . . . 815
O.2 The SysML v2 workbook . . . 823
O.3 Part II: week zero . . . 835
O.4 Part III: twelve weeks . . . 851
O.5 Part IV: making the case . . . 858
O.6 Part V: living with it . . . 864
O.7 Appendices . . . 872

List of Figures

1 From what you know to what you do not. The book climbs this ladder in order; each rung leans on the one below it. xxxiv

4.1 The Vee in MBSE DevSecOps: each element and the GitLab mechanism that carries it 35
4.2 One picture: the Vee and its reviews, two-week sprints, the DevSecOps loop, the model in GitLab as the spine, the digital thread and the digital twin 42

11.1 The part tree of drop 1: ObservationNetwork and what it is made of 161
11.2 The requirement trace of drop 1: satisfy on one side, verify on the other, gaps marked 162
11.3 Interconnection views of drop 1: the parts inside each block and the connections between their ports 163
11.4 Package dependencies of drop 1 in the converter's flat layout: the dashed arrows are the cycles 164
11.5 IDEF0 plate O0: the observation cycle's six steps with their inputs, outputs and mechanisms 165

47.1 Package dependencies of drop 1 in the converter's flat layout: the dashed arrows close the cycles 647

49.1 Three segments, three cadences. The stations move in hardware blocks, telemetry in radio and firmware cycles, the data center in sprints. The dashed lines are the integrated baselines where all of them are frozen and tested together. 663
49.2 Overlapping blocks. Block 2 (drifting buoys) starts its design while Block 1 (moored buoys and shore stations) is still in verification; Block 3 (partner data sharing) follows. Sprints run underneath all of them. 667
49.3 The ladder of fidelity. Model checks run on every change; sea trials happen a few times a year. Each rung's measured data calibrates the rung below. 668

List of Tables

1 Task points closed in each sprint, by kind of work. One point is one ideal person-day. iii
2 All paid hours of the three-person team in the first twelve weeks. iv

3 The eighteen months at a glance (the twelve weeks are sprints 1 to 6) xxxvii
4 The prime contractor's drops xxxviii

1.1 The document-era artifact set (typical; Tidewater's names) 3
1.2 The document-era baseline (notional, reconstructed from the last review cycle) . . 5
1.3 Where the time went in one requirement change (notional) 5
1.4 A document-era year, reconstructed (effort notional) 8

2.1 What a prime drop contains 11
2.2 The spider web, measured from drop 1 12
2.3 Drop 1 converted, file by file 15
2.4 The Cameo years: belief and fact 18
2.5 The government's review of one Cameo delivery (notional) 19
2.6 Ownership after the 2025 contract change 20

3.1 The directive, read as a specification 23
3.2 The twelve-week scorecard, as agreed before sprint 1 28
3.3 The twelve-week plan, as sent to the program manager 30
3.4 The twelve-week reporting rhythm 32

4.1 The left arm of the Vee as folders in the model. 36
4.2 Each Vee element in the document era and in MBSE DevSecOps. 40

5.1 Systems engineering to DevSecOps, the translation table 50
5.2 Threats to a model repository and the controls the team used 53

6.1 Definitions and usages, from SysML v1 to SysML v2 60
6.2 What the converter carries (from the converter's own mapping table) 64
6.3 Drop 1's conversion debt, by kind 64
6.4 Reading converted SysML v2 in plain Vim 70
6.5 The converter report's four columns 71

7.1 The eighteen parts of the requirement recipe 74
7.2 The four operative verbs 76
7.3 Requirement tags 79
7.4 The worked example, part by part 79
7.5 What the checker reports 83
7.6 Where the recipe's parts live in SysML v2 87

8.1 Tidewater's drop-1 TODO lines and where the workbook teaches each fix 122

9.1 The survival kit: what is on the laptop 127
9.2 Five keys that make the terminal pleasant 129
9.3 The same folder in Windows and in Git Bash 129
9.4 The commands you need, on one page 134
9.5 The five git settings, and why 136
9.6 Where things are in GitLab's web pages 140
9.7 When something goes wrong 140

10.1 What the kit sets in a SysML buffer 144
10.2 Reading a converted model 145
10.3 Moving through the quickfix list 146
10.4 Model checks from Vim 148
10.5 Seeing the model from Vim 150
10.6 The terminal from inside Vim 155
10.7 The stand-up keys used most 155
10.8 The model workbench on one card 155

11.1 The five views 159
11.2 The view keys in a SysML buffer 166
11.3 The view summaries as measures 168
11.4 When a view will not draw 169

12.1 Configuration management to git 172
12.2 Commit messages 177
12.3 Three branching strategies compared 189

13.1 The five levels of work, with their size limits 198
13.2 Splitting stories that would not fit in a sprint 200
13.3 The Tidewater project tree: three tomes, fourteen epics 201
13.4 The levels in GitLab, by tier 203
13.5 The two-week cadence 204
13.6 Sprint 1 capacity 205
13.7 Who does what (R responsible, A accountable, C consulted, I informed) 206
13.8 The decisions behind the operating model 210

14.1 The two levels the team writes every sprint 215
14.2 INVEST, translated 215
14.3 Three stories and their acceptance criteria 216
14.4 Acceptance criteria a machine can check 217
14.5 The kinds of task in model work 218
14.6 Tidewater's learning budget (share of each sprint's task points done, Section 13.8, Two ways to count "done" (p. 205)) 220
14.7 Epic E6 as stories 221
14.8 Mistakes in the first six sprints 223

15.1 GitLab objects and what they replace 227
15.2 Members and roles in tidewater/model 230

15.3 Protected branches . . . 230
15.4 The issue and merge request templates . . . 241
15.5 Who sets up what . . . 243

16.1 Tidewater's layers, bottom to top . . . 246
16.2 What never goes in . . . 250
16.3 Rules and the checks that keep them . . . 251
16.4 A reference layout and Tidewater's, compared . . . 252

17.1 The GitLab tools the team uses, by tier and adoption sprint . . . 256
17.2 The ISSM's questions and where the answers live . . . 262
17.3 What the gates cost (check GitLab's current pricing page; tiers and quotas change) 264
17.4 GitLab tokens and what each is for . . . 266
17.5 Features the team did not use, and why . . . 267

18.1 Cloud chatbot versus local model . . . 271
18.2 Model families from US and allied developers (check the Ollama library for current tags and memory needs) . . . 272
18.3 Laptop installs versus a shared GPU server . . . 273
18.4 Bake-off scoresheet (one row per task, per model, per person) . . . 275

19.1 What starts a pipeline . . . 292
19.2 Artifacts, cache and variables . . . 293
19.3 How the pipeline grew . . . 298

20.1 Which jobs run for which event (checked with `gitlab.pl ci --list` for each event) 319

21.1 Runner scopes . . . 348
21.2 Hosted runners against your own . . . 348
21.3 Runner executors . . . 349
21.4 Runner care . . . 354

22.1 What to ask first, by kind of change . . . 363
22.2 The three outcomes of a review . . . 365

23.1 The hierarchy, and where each level is recorded . . . 368
23.2 The stand-up verbs used most . . . 370
23.3 The tasks of one story . . . 374

24.1 The two-week rhythm . . . 378

25.1 Stand-up statuses, before and after . . . 392
25.2 The ladder . . . 393
25.3 talk.pl . . . 393
25.4 Who wants which number . . . 398
25.5 Demo roles . . . 399
25.6 Bad-news scripts . . . 400

26.1 The pulse survey at the sprint 1 retrospective (1 = strongly disagree, 5 = strongly agree) . . . 405

26.2 The scorecard at the end of sprint 1 . . . 420
26.3 What sprint 1 brought in . . . 421

27.1 The pulse survey at the sprint 2 retrospective . . . 425
27.2 The scorecard at the end of sprint 2 . . . 435
27.3 What sprint 2 brought in . . . 437

28.1 Reading a failing job log . . . 446
28.2 Sprint 3 pulse survey (mean 2.2) . . . 454
28.3 Sprint 3 against sprint 2 . . . 456
28.4 Tools adopted in sprint 3 . . . 456

29.1 Anatomy of a discrepancy report . . . 465
29.2 The model site in sprint 4, page by page . . . 467
29.3 Sprint 4 pulse survey (mean 3) . . . 471
29.4 Sprint 4 against sprint 3 . . . 472
29.5 Tools adopted in sprint 4 . . . 473

30.1 Sprint 5's tools . . . 493

31.1 The scorecard at the end of each sprint (sprint 1's TODO count is before the drop-1 import) . . . 509
31.2 Sprint 6's tools . . . 510

32.1 GitLab's analytics views, as a systems engineer reads them. . . . 514
32.2 The four DORA measures, translated for a model repository. . . . 516
32.3 Flow measures by sprint, from the scorecard files. . . . 517

33.1 The twelve-week scorecard, sprint 1 against sprint 6. "Before" values are notional, from the program's last document-era review cycle. . . . 521
33.2 TODO lines left on develop, from the pipeline's todo job. . . . 522
33.3 Trace measures at the end of sprint 6, identical to sprint 2. . . . 522
33.4 Spider-web measures of the v2 model, from the pipeline's complexity job. . . . 523
33.5 Delivery and morale by sprint, from the journal and the retrospectives. . . . 524

34.1 Port acceptance evidence, interim status after twelve weeks. . . . 530

35.1 Questions at the December briefing. . . . 536

36.1 The reporting rhythm agreed in week zero. . . . 539
36.2 The quad's quadrants, the template and the kit. . . . 539
36.3 The quad's four-letter words. . . . 539

37.1 Markdown to memorandum . . . 546
37.2 The memo templates . . . 547
37.3 The Tidewater memos . . . 552
37.4 Where to write it down . . . 554

38.1 The pulse survey's mean, from each sprint's morale.json. . . . 558
38.2 The five pulse questions, sprint 1 and sprint 6. . . . 558

39.1 One Thursday, three years apart. The left column is notional; the right column is the evidence above. . . . 568

40.1 The Document Review Stages, from the program's grading chart. . . . 571
40.2 Mechanical checks and human judgment, stage by stage. . . . 572

41.1 Who paired with whom . . . 577
41.2 No, with a date . . . 580
41.3 What the lead gave away . . . 581
41.4 Jim's two weeks . . . 583

42.1 The administration calendar . . . 587
42.2 GitLab roles as Tidewater uses them . . . 588
42.3 Kinds of token and where Tidewater uses each . . . 590
42.4 What a git clone does and does not protect . . . 597
42.5 Features by tier, and what Tidewater did on each . . . 599

43.1 The prime's drops, from their manifests . . . 603
43.2 From the plan: sprints 7 to 39 . . . 604
43.3 The change board's records, before and after . . . 609
43.4 The life cycle and the pipeline's emphasis . . . 610

44.1 The five stages and Tidewater's mapping. Only stage 1 is evidence; stages 2 to 5 are the plan. . . . 616
44.2 Increasing fidelity, and where each level runs. . . . 617

45.1 The four dispositions a change request can get . . . 626
45.2 Mirroring the whole model versus translating only the interface surface . . . 629
45.3 Jenkins and GitLab, side by side . . . 630

46.1 Requirements stories . . . 638
46.2 Architecture stories . . . 638
46.3 Behavior stories . . . 639
46.4 Analysis stories . . . 639
46.5 Verification stories . . . 640
46.6 Change and sustainment stories . . . 640
46.7 Some of Tidewater's platform stories . . . 641

47.1 The spider web measured, v1 against v2. v1 numbers from the drop's own measures; v2 from the complexity job. The cycle row compares two different counts of the same tangle (Chapter 11, Seeing the model: diagrams from text (p. 158) explains the three ways to count it). . . . 646
47.2 Structure practices. . . . 647
47.3 Reuse practices. . . . 647
47.4 Traceability practices. . . . 648
47.5 View practices. . . . 648
47.6 Semantics practices. . . . 648
47.7 Process practices. . . . 649

48.1 The twenty-six security points for Tidewater . . . 659

49.1 Tidewater's segments and their cadences. . . . 663
49.2 The levels of the work plan in this book and in the reference. . . . 665
49.3 Architecture views generated from the model, with Tidewater's content. . . . 666
49.4 Reviews and the evidence each takes from the model. . . . 667
49.5 Standards to plan around. Check current revisions and your contract. . . . 669

50.1 A generic twelve-week plan, from week zero to the first baseline . . . 673
50.2 Decisions to make locally, and what should drive them . . . 676
50.3 Copy the shape, adapt the details . . . 681

A.1 The git commands a model engineer uses . . . 696
A.2 gitlab.pl global options . . . 698
A.3 Set-up commands . . . 698
A.4 Pipeline, release and record commands . . . 700
A.5 The gitflow steps . . . 700
A.6 daily.pl commands by time of day . . . 701
A.7 The daily.pl commands Tidewater uses . . . 702
A.8 xmi2sysml.pl options . . . 703
A.9 reqif2sysml.pl options . . . 703
A.10 model.pl commands . . . 704
A.11 Week zero, drafting and checking words . . . 706
A.12 Vim keys that run the tools . . . 707

B.1 Survival Vim . . . 709
B.2 The kit's SysML keys . . . 709
B.3 Quickfix . . . 710
B.4 Conflicts in Vim . . . 710
B.5 Git from Vim . . . 711
B.6 Markdown for GitLab, in Vim . . . 711
B.7 Journal keys . . . 711

C.1 git every day . . . 713
C.2 Gitflow branches . . . 713
C.3 gitlab.pl at a glance . . . 714
C.4 Quick actions . . . 715
C.5 References and GitLab Markdown . . . 715
C.6 Pipeline sources . . . 716
C.7 Pipeline keywords . . . 716
C.8 Variables and tokens . . . 716
C.9 Where things are . . . 717

D.1 The converter's mapping (from lib/Xmi.pm) . . . 719
D.2 TODO lines and their fixes . . . 720
D.3 Converter options . . . 721
D.4 Delta marks . . . 722
D.5 The drop cycle . . . 722
D.6 Trace measures . . . 723

F.1 Where model errors come from . . . 732

F.2 Messages from ci/job.pl 736
F.3 Common recipe findings 738

G.1 Names in the model 739
G.2 Names outside the model 742

H.1 Outcome, evidence, next or ask 745
H.2 Write the one-sentence version first; say each rung out loud once 745
H.3 Jargon to plain 746
H.4 Fact, cause in one clause, next with a date 746

I.1 The GitLab Markdown this project uses 748

K.1 Systems engineering terms and their GitLab counterparts 796
K.2 DevSecOps and GitLab words 798
K.3 SysML v1 and v2 terms 800
K.4 Agile words 801
K.5 Terms of this book 801

L.1 Learning hours per person per sprint, from the plan (printed by the block above) . 804
L.2 The lead's twelve weeks 805
L.3 The modeler's twelve weeks 806
L.4 The pipeline owner's twelve weeks 807

Preface

From what you know →

You know

how a program document opens: purpose, scope, applicable documents, then the content.

→ Here it is called

the preface: who the book is for, the cast, the calendar, and how to read and type along.

→ What is new

every command in this book has been run, and you are expected to type it too.

This book is a parachute drop.

Picture three systems engineers standing at the door of an aircraft. They have spent their careers writing specifications, reconciling spreadsheets and briefing chiefs. A few years ago someone handed them a modeling tool and told them to put everything they knew into it. They did, by hand, one box and one line at a time. Now the jump light is on again. Below them is a place called "MBSE DevSecOps," and they have been told to land there, set up camp, and show progress within twelve weeks. They have no map of that place. They have no software background. They have never written a pipeline, opened a merge request or resolved a merge conflict. They have the clothes on their backs, and one tool they trust: the Vim editor, which all three of them have used for years to write notes, edit requirement lists and clean up exported text files.

This book is written for people in that position. It follows Jim, Bob and Tom, the three government engineers responsible for the Tidewater system model, through their first twelve weeks. It shows what they did each day, what they typed, what broke, what they argued about and what they measured. It also explains, as plainly as we can, why each step is worth taking. By the end of the twelve weeks the team has a SysML v2 model under configuration control in GitLab, an automated pipeline that checks every change, a trace report that used to take two days and now takes seconds, and a management briefing built from measurements instead of opinions. They also, somewhat to their own surprise, like working this way. The book tries to show how that change of mind happened, without pretending it was easy.

Note: What you have, what you need

What you have: a Windows laptop, the Vim editor, years of systems engineering experience, and a model in a tool you were never comfortable with.

What you need from this preface: a picture of the people, the system and the calendar, so

the rest of the book has somewhere to hang. Nothing to install yet; that starts in Chapter 9, The survival kit (p. 126).

Who this book is for

You will get the most from this book if you are a systems engineer who:

- knows what a requirement, an interface, a verification method and a baseline are;
- has worked with documents (a SEMP, an SRS, ICDs), spreadsheets (requirement matrices, trace tables, action item lists) and briefings (design reviews, CCB packages);
- has seen or used a SysML v1 tool such as Cameo Systems Modeler, even if you did not enjoy it;
- has been told that the program is "moving to DevSecOps" and is not yet sure what that means for you.

You do not need to know how to program. You will type commands into a terminal, edit text files and read the output of automated checks, and the book explains every one of those things the first time it appears. If you already know git, GitLab or SysML v2, you can skim the chapters that teach them and spend your time on the case study chapters in Part III and the chapters about making the case to management in Part IV.

The book is also for the people around such a team: the program manager who issued the directive and wants to know what to expect, the information system security manager (ISSM) who has to approve the new tools, and the contractor who delivers the models. Each of them appears in the story, and each chapter notes what they need to do and when.

How to read this book

The book has five parts.

- **Part I, Where they start**, tells the team's history: decades of documents, then the Cameo years, then the directive that started this story. It shows that the systems-engineering Vee stays (Chapter 4, The Vee, kept: the systems-engineering V in MBSE DevSecOps (p. 34)), explains DevSecOps in systems engineering words, introduces SysML v2 to people who know SysML v1, and ends with a workbook you type. Read it first even if you are impatient. The objections raised in Part I are the objections you will hear (or raise) yourself, and the rest of the book answers them one at a time.
- **Part II, Week zero**, sets everything up, starting with what you already use: the survival kit on the laptop, Vim for models, diagrams generated from the model, then git from zero, the operating model and stories, GitLab, the DevSecOps tools worth turning on, CI/CD and the pipeline, and the daily rhythm. Part II is the part you type along with.
- **Part III, Twelve weeks**, is the case study proper: six two-week sprints, one chapter each, with the real issues, merge requests, pipeline logs, stand-up notes and metrics from the run.
- **Part IV, Making the case**, assembles the twelve-week scorecard, walks through the management briefing slide by slide, and revisits each objection from Part I against the evidence.

- **Part V, Living with it**, covers what comes after the first twelve weeks: administration, maintenance through the life cycle (every prime contractor delivery, every baseline, the remaining months of the project), scaling up to several segments and hardware blocks (Chapter 49, Scaling up: segments, blocks and a ladder of fidelity (p. 662)), and the mistakes to avoid.

The appendices hold a command reference, every template, checklists, a glossary that translates between systems engineering, DevSecOps and SysML v2, and answers to the exercises.

From what you know to what you do not

The order is deliberate. Each chapter starts from something you already do, says what it is called here, and only then shows what is new; a box headed "From what you know" opens every chapter to make that step explicit. The chapters climb the same ladder the team climbed: first the things you know, then the things that are quick to learn because they are your own work in a new form, and only then the things that are genuinely new.

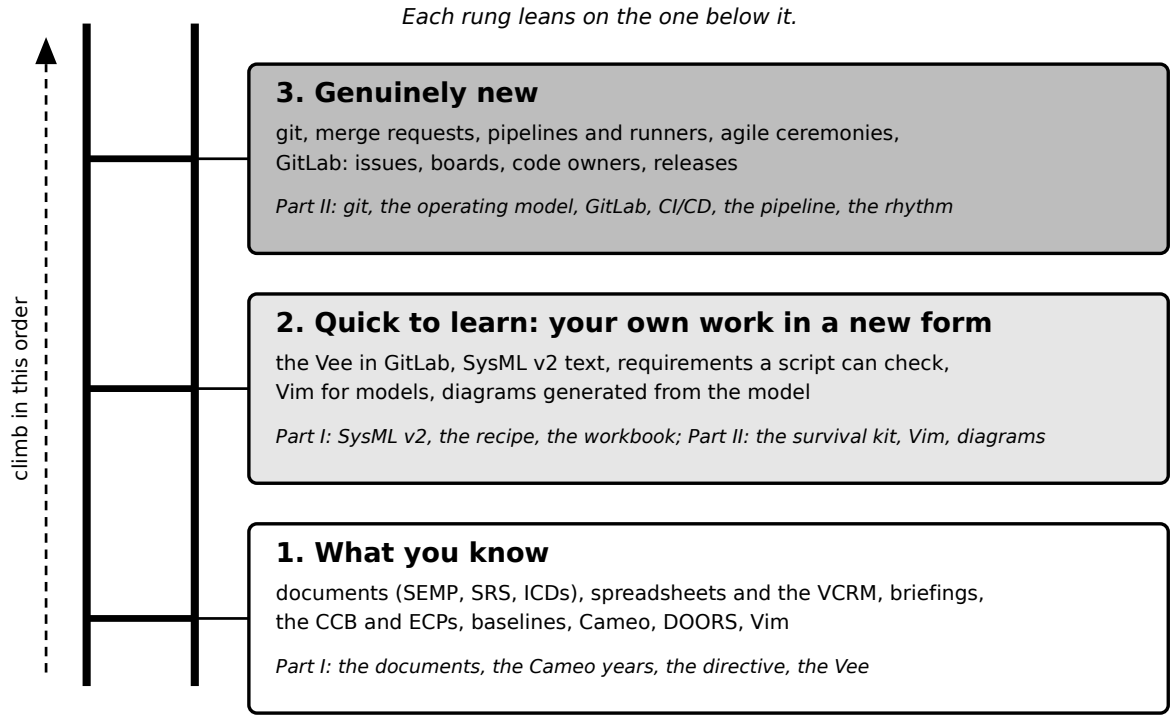


Figure 1: From what you know to what you do not. The book climbs this ladder in order; each rung leans on the one below it.

If a chapter feels too far up the ladder, go down one rung: the study plan (Appendix L, A twelve-week study plan per role (p. 803)) lists the chapters in ladder order for each role.

Type along

This book is meant to be read with a terminal open. Every command shown in a code block is real and has been run against the material that ships with the agile kit this book comes from.

When you see

```
perl "$KIT/sim/tidewater-v1.pl" --drop 1 drop1
```

type it, press Enter, and compare what you see with what the book shows. (The commands in this book are not only shown; they are run: a test in the kit runs every command a reader is told to type, chapter by chapter, against the same material, before the book is printed.) If they differ, that is useful information: either your setup differs (and Chapter 9, The survival kit (p. 126) will help you find out how), or the material has changed since this book was generated. Either way you will learn more from the difference than from reading.

Commands are written for the Git Bash terminal that comes with Git for Windows, because that is what Jim, Bob and Tom have on their government laptops. They also work in a Linux terminal. Paths use forward slashes. When a command is long it is broken across lines with a trailing backslash, which the shell joins back together.

Vim commands are written the way Vim's own help writes them: `:w` means type a colon, then `w`, then press Enter; `\mf` means press the backslash key, then `m`, then `f` in normal mode. The kit's Vim help is always one command away: `:help agile`.

Tip: The exercises

Most chapters end with exercises. Some ask you to type and observe; some ask you to decide something and defend the decision. Answers are in Appendix O, Answers to the exercises (p. 815). The decision exercises have more than one defensible answer; the answers appendix gives the one the team chose and why.

Conventions used in this book

- `Fixed-width text` is something you type or something a program prints: a command, a file name, a line of SysML.
- A code block that starts with a command is meant to be typed. In a block whose lines start with `$ `, type what follows each `$ ` (not the `$` itself); the other lines are what the command prints.
- `$KIT` stands for the folder the agile kit is in (`~/projects/agile` on Tidewater's laptops); Chapter 9, The survival kit (p. 126) shows how to set it once, the way the pipeline sets it. File paths without `$KIT` are relative to the folder you are in unless they start with `/` or `~`. `~/w2s` is the practice folder the book uses; use any folder you like.
- **Bold** marks a term the first time it is defined. The glossary in Appendix K, Glossary (p. 796) has all of them.
- Boxes are labeled by kind. *Note* and *tip* boxes add detail; *warning* boxes mark a mistake the team made or nearly made; *sidebar* boxes step back from the story; *objection* boxes record a skeptic's argument in the speaker's words, followed by the answer and where the evidence is; *exercise* boxes are for you.
- Quotations from Jim, Bob and Tom come from the team's notes, retrospectives and stand-up answers, lightly edited.

- Numbers from the document era are reconstructions and are labeled *notional*. Numbers from the twelve weeks come from the records the tools kept, and each table says where it came from.

The system: Tidewater

Tidewater is a fictional coastal weather-observation network, invented for this book. Nothing in it describes a real system, a real program or a real organization, and every number in it is notional.

Tidewater has three segments:

- **Stations.** Moored buoys offshore and shore stations on masts. Each station measures wind, air pressure and air temperature; buoys also measure waves. A buoy runs on its own solar panel and battery and sends its observations over a radio link; when the link is down it keeps them and sends them later.
- **The data center.** It ingests every observation, checks its quality, archives it with a quality flag, and publishes a marine bulletin on a fixed schedule for forecasters and mariners.
- **Operations.** A maintenance depot, a service vessel with a deck crane, and a pool of spare parts that keep the stations running.

The system is small on purpose. A real program's model has thousands of elements; Tidewater's v1 model has a few hundred. Small enough to read in an afternoon, large enough to have every problem a real model has: requirements that nothing satisfies, requirements that nothing verifies, text in the requirements database that no longer matches the text in the model, diagrams nobody updates, and packages so tangled that a change in one place surprises someone in another.

The cast

Jim is the MBSE lead and the person responsible for the Tidewater system model on the government side. Jim owns the big picture: what the model must contain, which work comes first, what "done" means, and what gets reported upward. In this story Jim is also the one who received the directive to adopt DevSecOps, which makes Jim the person who has to explain it to the other two without fully understanding it yet. Jim has twenty-six years in systems engineering, most of them writing and reviewing specifications.

Bob is the MBSE engineer. Bob built most of the Cameo model during the document port and knows where every block came from, including the ones that should not be there. Bob is the most skeptical member of the team about the new direction, and the one whose skepticism is most useful, because Bob asks precise questions: "Who approves a change?", "Where is the baseline?", "What happens when the contractor's model changes?"

Tom is the configuration management engineer, and in this story becomes the team's DevSecOps engineer. Tom volunteered to build the pipeline, mostly because somebody had to and Tom has run the program's CM database for a decade. Tom has never written a pipeline, never administered a GitLab project, and never written a line of code beyond spreadsheet formulas and Vim macros. Tom likes procedures, checklists and things that can be audited.

Around the team:

- **The program manager** issued the directive and will judge the results at the end of the twelve weeks. The program manager is not hostile, but needs to show the program's leadership that the money spent on tools and training produces something measurable.
- **The deputy program manager** chairs the configuration control board (CCB) and cares about one question above all: can a change be traced from request to approval to implementation?
- **The ISSM** (information system security manager) must approve any new tool or data flow on the program's network. The ISSM is careful, busy and, for three days in week six, unreachable.
- **The prime contractor**, Northwind Marine Systems (fictional), builds Tidewater and maintains the authoritative SysML v1 model in Cameo under a contract data requirement (CDRL A003, "System Model"). The prime delivers the model to the government as "drops": an XMI export of the model, a ReqIF export of the DOORS requirements, a CSV copy of the requirements and a manifest with checksums.
- **The prime's modeler** is the contractor engineer the team talks to about the content of each drop.
- **The enterprise GitLab administrators** run the organization's GitLab server. They create groups and projects on request, register runners and enforce the organization's settings. They are helpful and slow.

Note: A note on pronouns

The book refers to every person by name or by role, and uses "they" when a pronoun is needed.

The calendar

The project in this book lasts eighteen months: thirty-nine two-week sprints from Monday 5 October 2026 to Friday 31 March 2028. The first twelve weeks, sprints 1 to 6, are the subject of Part III. The rest of the project is summarized in Chapter 43, Maintenance through the life cycle (p. 602).

Table 3: The eighteen months at a glance (the twelve weeks are sprints 1 to 6)

Sprint	Dates	What happens
1	5 to 16 Oct 2026	GitLab project created; the first pipeline; drop 1 received and assessed
2	19 to 30 Oct 2026	drop 1 imported through the vendor branch; every change gated
3	2 to 13 Nov 2026	Network and Types cleaned up; DOORS requirements imported; the hardest week for morale
4	16 to 27 Nov 2026	Stations cleaned up; gaps reported to the prime; the model site shown to management
5	30 Nov to 11 Dec 2026	Data center and operations cleaned up; release procedure rehearsed
6	14 to 25 Dec 2026	behavior work starts; interim baseline; the management briefing
7 to 20	Dec 2026 to Jul 2027	the port completed; drops 2 and 3 absorbed; baseline 1 accepted

Sprint	Dates	What happens
17 to 39	May 2027 to Mar 2028	maintenance against drops 4 to 7; baselines 2 and 3

The prime delivers seven drops over the eighteen months:

Table 4: The prime contractor's drops

Drop	Version	Delivered	What changed upstream
1	v1.0	Fri 2 Oct 2026	the model at contract start
2	v1.1	Fri 8 Jan 2027	a humidity sensor and its requirement; one requirement text corrected
3	v1.2	Fri 2 Apr 2027	a storm mode in the buoy's state machine; a fuse; clutter removed
4	v1.3	Fri 2 Jul 2027	drifting buoys; salinity in the observation record; a missing requirement added
5	v1.4	Fri 1 Oct 2027	a changed energy constraint; an activity renamed
6	v1.5	Fri 7 Jan 2028	data sharing with partner agencies; the legacy import package removed
7	v1.6	Fri 3 Mar 2028	missing verifications added; editorial fixes

Federal holidays fall inside the twelve weeks on 12 October, 11 November, 26 November and 25 December, and each team member takes some leave. The plans in Part III account for both, because a plan that ignores the calendar is the first thing a skeptic will pick apart.

What "done" looks like after twelve weeks

So that you know where the story is going, here is the state of things at the end of sprint 6, in the terms the program manager asked for:

- every change to the model goes through a merge request, is checked automatically by a pipeline, and is approved by a code owner other than its author; nothing reaches the integration branch any other way;
- the model's conversion debt, the number of places where the converter could not carry a SysML v1 construct into SysML v2, is measured on every change and has fallen sprint by sprint;
- the trace between the DOORS requirements and the model is computed on every change, including the text drift between them that nobody had ever checked systematically;
- every problem in the contractor's model is tracked as a discrepancy report with an owner, a due drop and a closing condition;
- an interim baseline is tagged, reproducible and published with release notes;
- the team can show all of the above in a browser, live, in a management review, instead of in screenshots pasted into slides.

None of these things require the team to become software developers. They require the team to do what systems engineers already know how to do (control configuration, trace requirements, review changes, verify) with tools that make those things cheap instead of expensive. That is the argument of this book, and the twelve weeks are the evidence.

PART I

Where they start

CHAPTER

1

Thirty years of documents

From what you know →

You know

the SEMP, the SRS, the SSDD, the ICDs, the VCRM spreadsheet and the milestone briefing. Thirty years of them.

→ Here it is called

the document era, described honestly, strengths included.

→ What is new

nothing yet. This chapter names the costs you already pay, so later chapters can show what replaces them.

Note: What you have, what you need

What you have: a career's worth of habits for controlling documents, reviewing changes and briefing decisions.

What you need from this chapter: a clear list of which of those habits are worth keeping (most of them) and which costs they hid. Nothing to type yet.

1.1 A Tuesday in 2019

The request arrived at 4:10 on a Tuesday afternoon, the way such requests always arrived: as a one-line email from the deputy program manager, copied to the program manager, with the subject line "Quick question."

> *Jim – before Thursday's program review, can you tell me which Tidewater requirements don't have a verification method yet? Just a list is fine. Thanks.*

It was not a quick question, and everybody on the distribution knew it. The answer lived in four places. The System Requirements Specification (SRS), revision C, held the requirement text and a section 4 "Qualification provisions" table. The Verification Cross-Reference Matrix

(VCRM) was an Excel workbook with one row per requirement and a column for the method (test, analysis, inspection, demonstration). The prime contractor kept the authoritative requirement text in DOORS and sent a CSV export every month. And the action item tracker, a different spreadsheet, held the open items from the last design review, several of which said things like "Contractor to propose verification approach for SRS 3.2.7."

Jim forwarded the email to Bob with a single line ("Can you pull this together?") and Bob spent Wednesday doing what Bob had done many times before. Bob opened the latest DOORS CSV in Excel, sorted it by object identifier, and pasted it next to the VCRM. A lookup formula matched the identifiers. Forty rows did not match, because the VCRM still used the paragraph numbers from revision B and DOORS used the new object identifiers. Bob fixed those by hand from the cross-reference table in the SRS appendix, found three requirements in DOORS that were not in the VCRM at all, found two in the VCRM that DOORS had deleted, checked the action item tracker for the ones with open actions, and by 3 p.m. on Wednesday had a list of eleven requirements with no verification method. Jim reformatted it as a slide, the deputy program manager presented it on Thursday, and the program review moved on to the next topic.

Two weeks later the prime delivered revision D of the VCRM. Four of the eleven had been fixed. Nobody updated the slide, and nobody ran the comparison again until the next "quick question."

> **Case study: Why this scene matters**
>
> Nothing in the scene is a failure. Every person did their job well. Bob's answer was correct on Wednesday afternoon. The trouble is that it was correct *only* on Wednesday afternoon, it cost a skilled engineer a day and a half, and the work that produced it left no trace that anyone could re-run. Keep that last point in mind; it is the whole argument of Chapter 5, DevSecOps in systems-engineering words (p. 46) in one sentence.

1.2 The document set

For most of Jim's career, systems engineering on a government program meant producing, reviewing and controlling a set of documents. The names vary by organization; Tidewater's set was typical:

Table 1.1: The document-era artifact set (typical; Tidewater's names)

Document	What it held	Who changed it
SEMP (Systems Engineering Management Plan)	how the program does systems engineering	the government, rarely
ConOps (Concept of Operations)	who uses the system and how, in prose	the government, once
SRS (System Requirements Specification)	the "shall" statements, numbered paragraphs	the prime, through the CCB
SSDD (System/Subsystem Design Description)	the architecture: segments, subsystems, block diagrams as pictures	the prime
ICDs (Interface Control Documents)	each interface: messages, fields, timing, connectors	the prime, both sides sign
VCRM (Verification Cross-Reference Matrix)	each requirement's verification method and event	the prime, a spreadsheet

Document	What it held	Who changed it
RTM (Requirements Traceability Matrix)	parent-child links between requirement levels	the prime, a spreadsheet or DOORS
Risk register, action item tracker	open risks and actions	the government, spreadsheets
Design review packages	slides summarizing all of the above	everybody, for each review

Each document had an owner, a revision letter, a change history page and a signature block. Changes moved through a formal process: somebody wrote an engineering change proposal (ECP) or a document change request, the request went into the queue for the monthly configuration control board (CCB), the board approved, rejected or deferred it, and the owner incorporated approved changes into the next revision. Between revisions, changes were tracked as "redlines": a marked-up copy of the document, usually a PDF with comments, sometimes a Word file with tracked changes, occasionally a printout with handwriting.

The spreadsheets lived alongside the documents and did the work the documents could not. The VCRM was a spreadsheet because a matrix is a spreadsheet. The RTM was a spreadsheet because parent-child links between a few hundred requirements are rows. The action item tracker was a spreadsheet because everything that did not fit anywhere else was a spreadsheet.

The briefings sat on top. Every review, every status meeting and every decision produced slides. The slides summarized the documents and spreadsheets, and for the people in the room the slides *were* the system: few decision makers read the SRS, but everybody saw the chart.

1.3 What worked

It is fashionable to sneer at document-based systems engineering. This book will not. The document era got a great deal right, and almost everything it got right survives into the way of working this book describes.

Everybody could read the documents. A Word document or a PDF needs no special software, no license and no training. An operator, a test engineer, a contracting officer and a general can all open the SRS. This is a real virtue, and the team's fear of losing it is one of the objections in Chapter 3, The directive (p. 22).

The documents were the contract. The CDRL told the prime to deliver an SRS, and the government accepted or rejected it. Signatures meant something. When a dispute arose, the signed revision settled it.

Changes were reviewed. Nothing went into a controlled document without a second pair of eyes and a board decision. The review culture of the CCB, the habit of asking "who approved this, and when?", is exactly what DevSecOps calls *change control*, and it is the habit Tom will build the pipeline around.

Baselines existed. "SRS revision C" named a fixed, known set of requirements. Everyone could point at it.

Traceability was taken seriously. The RTM and VCRM were tedious to maintain, but the program knew that every requirement needed a parent and a verification, and it checked, at least at each review.

Tip: Keep this list

Hold on to these five virtues: readable by anyone, contractually meaningful, reviewed, baselined, traced. When someone tells you DevSecOps throws away good systems engineering practice, ask which of the five it throws away. In the chapters that follow, each one is kept, and each one gets cheaper.

1.4 What it cost

The costs of the document era were real but hidden, because they were paid in engineers' time rather than in a budget line. Nobody ever wrote "1.5 days to answer the deputy's question" on a timecard; it was just Wednesday.

To make those costs visible, Jim later reconstructed four of them from the program's last document-era review cycle. They are reconstructions, not measurements, and this book labels them as notional wherever they appear. They are the "before" column of the scorecard in Chapter 33, The twelve-week scorecard (p. 520).

Table 1.2: The document-era baseline (notional, reconstructed from the last review cycle)

Activity in the document era	Typical effort or elapsed time (notional)
A requirement change, from change request to incorporated revision	23 working days
"Which requirements are unverified?", answered from the VCRM, DOORS export and SRS	2 working days
A model or design review: screenshots in a 60-slide briefing, prepared, reviewed and presented	3 weeks
Finding text that differs between DOORS and the documents (or the model)	never done systematically

Where did the 23 working days of a requirement change go? Almost none of it was engineering. Jim's reconstruction of one change, a clarification to a data-latency requirement, looked like this:

Table 1.3: Where the time went in one requirement change (notional)

Step	Working days (notional)
Engineer drafts the change request in the CR template	1
Change request waits for the next monthly CCB agenda	11
CCB meeting; approved with a comment	1
Comment resolved by email; revised wording approved by the chair	3
Prime incorporates the change into the next SRS revision draft	4
Government reviews the draft revision, signs	3

Most of the elapsed time is waiting: for a meeting, for a revision, for a signature. The meeting itself took a day, and the engineering, deciding what the requirement should say, took less than that. The process was designed for a world where making a copy of a document was expensive and distributing it was slow, so changes were batched into monthly meetings and quarterly revisions. Batching is efficient when each batch costs a lot. It is wasteful when each change could be reviewed, approved and incorporated on the day it is proposed.

The other hidden costs were less visible still.

Every answer was a one-off. Bob's Wednesday produced a correct list and a slide. It did not produce anything that could be run again. The next time someone asked, Bob started from scratch, because the spreadsheet formulas were in a copy of the VCRM that was now two revisions out of date.

Consistency was nobody's job. The SRS, the VCRM, the RTM, the DOORS database and the ICDs each held part of the truth, and each was maintained by a different person on a different schedule. When they disagreed, which they always did a little, the disagreement was found by accident, usually by someone preparing for a review. Requirement text in DOORS that differed from the text in the SRS was a known problem with no known size.

The briefing became the truth. Because decision makers saw slides, the slides were where decisions were made, and the slides were copies of copies. A block diagram drawn for a 2017 review would reappear, slightly edited, in 2019, long after the architecture had changed. Nobody could say which picture was current.

Configuration management meant file names. The program's shared drive held files named `SRS_RevC_final.docx`, `SRS_RevC_final_v2.docx`, `SRS_RevC_final_v2_JIM_comments.docx` and, memorably, `SRS_RevC_FINAL_use_this_one.docx`. Tom, who ran the CM database, could tell you which file was the signed revision. Nobody else could, and the CM database did not know about the working copies at all.

Knowledge lived in people. Bob knew why the VCRM's row 214 had a strange formula. Jim knew which ICD paragraph had been "temporarily" waived in 2016. When people moved on, the knowledge went with them.

Objection: "The documents worked. We delivered systems with them for thirty years."

That is true, and nothing in this book disputes it.

Answer

The question is not whether documents can work but what they cost and what they make impossible. The document process worked by spending skilled engineers' time on copying, reconciling and waiting. It made it impossible to ask the same question twice and get a fresh answer cheaply. The chapters in Part III measure what the same questions cost in the new way; Chapter 33, The twelve-week scorecard (p. 520) puts the two side by side.

1.5 The habits worth keeping

Before the team moves on, it is worth naming the document-era habits that will survive the transition, because the team's discouragement in Chapter 3, The directive (p. 22) comes partly from believing that everything they know is being thrown out. It is not. Here is the translation the book will use, one habit at a time. Chapter 5, DevSecOps in systems-engineering words (p. 46) gives the full table.

- **Change requests** become *merge requests*: a proposed change, with a description, reviewed and approved before it is incorporated. The difference is that a merge request contains the actual change, not a description of it, and can be approved the day it is written.
- **The CCB's approval** becomes *a code owner's approval*: a named person, responsible for an

area, approves changes to that area. The board still exists for decisions that need a board; it no longer has to touch every editorial fix.

- **Baselines** become *tags*: a name fixed to an exact, retrievable state of the model. Revision C becomes `baseline-1`.
- **The VCRM and RTM** become *reports computed from the model*: instead of maintaining a matrix by hand, the team writes the trace into the model and lets a program compute the matrix whenever anyone asks.
- **The design review package** becomes *a published site*: the current model, its diagrams and its reports, always generated from the baseline, never pasted.
- **Redlines** become *diffs*: a precise, line-by-line record of what changed, produced automatically.
- **The CM database** becomes *the repository's history*: every change, who made it, when, why, and who approved it, kept automatically.

None of these translations is perfect, and the book points out where they break. But the shape of the work is recognizably the same. Systems engineering still means knowing what the system must do, how it is put together, how each requirement will be shown to be met, and how changes are controlled. What changes is how much of that work is done by hand.

1.6 What the spreadsheets were really doing

Every program has a spreadsheet that nobody is allowed to touch. On Tidewater it was the VCRM, and Tom, who had seen every revision of it, once described it as "the most important database on the program, stored in the least reliable format we have."

Spreadsheets did three jobs on the program, and it is worth separating them, because each job has a better home in the new way of working.

The spreadsheet as a database. The VCRM, the RTM, the interface list and the action item tracker were all tables of records with identifiers, attributes and relationships between them. That is a database. Spreadsheets are a poor database: nothing stops two rows from having the same identifier, a formula from referring to the wrong column after someone inserts one, or a copy of the file from drifting away from the original. In the new way, the records live in the model (requirements, satisfy and verify relationships) or in GitLab (issues), both of which enforce identity and keep history.

The spreadsheet as a calculator. Bob's lookup formulas, the trace coverage percentages and the "requirements without a verification method" count were calculations over the records. Each calculation was written by hand, in one copy of one file, and lost when the file was replaced. In the new way, calculations are small programs that run in the pipeline against the current model, every time. The trace report in Chapter 20, The pipeline, job by job (p. 302) is Bob's Wednesday, written once and run thousands of times.

The spreadsheet as a report. The colored cells, the filtered views and the pivot tables pasted into slides were how the records reached decision makers. In the new way, reports are generated from the model and published (Chapter 17, The DevSecOps tools that earn their keep (p. 255)); nobody formats a cell.

Case study: The VCRM's row 214

Row 214 of the VCRM held a formula that combined three columns and a lookup into a hidden sheet. It computed whether a requirement's verification event had been scheduled. Bob wrote it in 2017. In 2021 someone sorted the sheet without including the hidden columns, and for four months row 214's formula pointed at the wrong requirement. Nobody noticed until a test readiness review. The formula was right; the format was the problem. A calculation that lives in a cell is only as safe as every edit anyone ever makes to that file.

1.7 A year in the document era

To see the pattern of the old way, it helps to look at a whole year rather than a single Tuesday. Here is Tidewater's systems engineering calendar for a typical year before the Cameo port, reconstructed from the team's old meeting invitations. The effort figures are notional.

Table 1.4: A document-era year, reconstructed (effort notional)

When	Event	What the team did	Effort (notional)
monthly	CCB	assemble change requests, write the agenda, record decisions, update the change log	2 days per month
monthly	DOORS export from the prime	reconcile the export against the VCRM and SRS	1 to 2 days
quarterly	SRS / ICD revision	review the prime's revision against approved changes, comment, sign	1 to 2 weeks
quarterly	program management review	build the SE section of the briefing (status, risks, TBD/TBR counts)	3 to 5 days
twice a year	technical review (SRR, PDR, CDR, TRR ...)	review packages, comment matrices, action items	3 to 6 weeks
ad hoc	"quick questions"	one-off answers from the spreadsheets	about a day each, several a month

Add it up and roughly a third of the team's year went into reconciling, reformatting and re-answering: work that produced no new engineering knowledge, only fresh copies of old knowledge in a new format. Nobody on the team thought of it that way at the time, because each piece of it was necessary given how the information was stored. The waste was in the storage, not in the people.

That third of a year is the budget the new way of working draws on. When Chapter 3, The directive (p. 22) asks what the team can stop doing, and when Chapter 33, The twelve-week scorecard (p. 520) compares effort before and after, this table is where the comparison starts.

1.8 Why the document era lasted

If the costs were this large, why did the document era last thirty years? It is worth answering honestly, because the same forces will push back against the new way.

- **The costs were invisible.** Time spent reconciling spreadsheets was never charged to "reconciling spreadsheets." It was charged to systems engineering. Nobody could see it, so nobody could argue to reduce it.

- **The contract required documents.** CDRLs named document deliverables with data item descriptions that assumed a document. Changing what the prime delivered meant changing the contract.
- **Everyone knew how.** A new engineer could be productive with Word and Excel on the first day. Any new way had to compete with zero training cost.
- **Failures were rare and quiet.** Most of the time, the reconciliations caught the inconsistencies before they mattered. When they did not, the failure appeared months later as a test problem or a rework, far from its cause.
- **Nothing better was available to this team.** Until SysML v2's textual notation, the alternatives were diagram tools that, as Chapter 2, The Cameo years (p. 10) shows, kept most of the costs and added new ones.

The last point matters most. The team's resistance to the directive is not stubbornness; it is the reasonable conclusion of people who were offered an alternative once before and found it worse. The new way has to be better on the measures they care about, and it has to show that early.

1.9 Exercises

Exercise 1.1: Count the copies

Pick one requirement on a program you know. List every place its text appears (documents, spreadsheets, databases, slides, emails). For each, write who maintains it and when it was last updated. How many of those places agree, word for word, today? How would you find out without reading them all?

Exercise 1.2: Reconstruct a change

Take one recent change to a controlled document on your program. Reconstruct its timeline as in the table "Where the time went in one requirement change". Separate the time spent deciding from the time spent waiting. What fraction was waiting?

Exercise 1.3: The five virtues

For each of the five virtues of the document era (readable by anyone, contractually meaningful, reviewed, baselined, traced), write one sentence on how your current process achieves it and one on what it costs. Keep the page; you will revisit it after Chapter 33, The twelve-week scorecard (p. 520).

CHAPTER

2

The Cameo years

From what you know →

You know

Cameo: its diagrams, its containment tree, the XMI export and the DOORS link.

→ Here it is called

SysML v1, and the spider web it became.

→ What is new

measuring the spider web (diagrams, duplicates, cycles, relationships per block) instead of only feeling it.

Note: What you have, what you need

What you have: a SysML v1 model in Cameo that you built by hand, that the prime contractor now maintains, and that nobody on the team enjoys opening.

What you need from this chapter: an honest, measured picture of that model, and an understanding of why SysML v1 in a diagram tool did not scale for this team. You will run two commands to measure it yourself.

2.1 "Go MBSE"

The first directive came in the spring of 2022, and it was shorter than the one in Chapter 3, The directive (p. 22). The program's leadership had attended a conference, seen a demonstration of model-based systems engineering, and decided that Tidewater would "transition to MBSE." Licenses for Cameo Systems Modeler arrived in June. Jim, Bob and Tom attended a three-day vendor training course in July. The course taught them which buttons to press to create a block, a port, a requirement and a diagram. It did not teach them what to put in the model, how to organize it, how to review it, or how it would stay consistent with the documents and DOORS. Those questions were left, as the course instructor put it, "to your organization's modeling guidelines." The organization did not have modeling guidelines.

So the team did the only thing that made sense to them: they ported the documents. Every

block diagram in the SSDD became a block definition diagram. Every interface table in the ICDs became an internal block diagram with ports and connectors. Every "shall" in the SRS became a requirement element. Every row in the VCRM became a verify relationship. The work took most of two years, done between other duties, by clicking.

> *I drew the PowerSystem IBD four times. Once from the SSDD figure, once when the ICD changed, once when Cameo crashed and I lost the layout, and once for the CDR briefing because the real one didn't fit on a slide.*
>
> — Bob, recalling 2023

In 2025 the program changed the contract. The prime contractor, Northwind Marine Systems, took over maintenance of the Cameo model under a new contract data requirement, CDRL A003 "System Model." From then on the prime made the changes and delivered the model to the government every few months; the government reviewed it. The team's role shifted from building the model to checking it, which in practice meant opening Cameo, clicking through diagrams, taking screenshots and pasting them into review comments.

2.2 What the team built

The prime's first delivery under this book's timeline, drop 1 (version 1.0, delivered Friday 2 October 2026), is the model as the team built it and the prime inherited it. It is an XMI file: Cameo's export format, an XML file in which every model element is a tagged record with a long, unique identifier. You can generate the drop yourself:

```
mkdir -p ~/w2s
cd ~/w2s
perl "$KIT/sim/tidewater-v1.pl" --drop 1 drop1
```

(`$KIT` is the folder the kit is in; Chapter 9, The survival kit (p. 126) sets it up, and `~/w2s` is the practice folder the book uses.) The command writes four files into `~/w2s/drop1`, exactly as the prime delivers them:

Table 2.1: What a prime drop contains

File	What it is
`tidewater.xmi`	the Cameo model, exported as Eclipse UML2 XMI (what Cameo calls "Export To > Eclipse UML2 (v5.x) XMI File")
`tidewater.reqif`	the DOORS requirement modules, exported in the standard ReqIF exchange format
`tidewater.csv`	the same requirements as a DOORS CSV export, for people who want a spreadsheet
`MANIFEST.txt`	the delivery record: version, date, CDRL, the prime's build (Jenkins job and commit) and tools, and a SHA-256 checksum for each file

Here is one block, the solar panel on a buoy, as it appears in the XMI. You do not need to read XML fluently; just notice the shape.

```
<packagedElement xmi:type="uml:Class" xmi:id="_22_1_7f3a0c1_a89dbce342d7" name="SolarPanel">
  <ownedComment xmi:type="uml:Comment" xmi:id="_22_1_7f3a0c1_fd71123c374d" body="Generates
  power.">
    <annotatedElement xmi:idref="_22_1_7f3a0c1_a89dbce342d7"/>
  </ownedComment>
  <ownedAttribute xmi:type="uml:Property" xmi:id="_22_1_7f3a0c1_8190460a888e" name="ratedPower"
                  type="_22_1_7f3a0c1_dd4fe0cc913f">
    <defaultValue xmi:type="uml:LiteralReal" xmi:id="_22_1_7f3a0c1_06d6ee446155" value="40"/>
  </ownedAttribute>
  <ownedAttribute xmi:type="uml:Port" xmi:id="_22_1_7f3a0c1_911b930881e1" name="out"
                  aggregation="composite" type="_22_1_7f3a0c1_dba6f2e0627e"/>
</packagedElement>
...
<sysml:Block xmi:id="_22_1_7f3a0c1_2629e9a46496" base_Class="_22_1_7f3a0c1_a89dbce342d7"/>
<Tidewater_Profile:L5Assembly xmi:id="_22_1_7f3a0c1_0efc9f0503d9"
base_Class="_22_1_7f3a0c1_a89dbce342d7"/>
<Tidewater_Profile:Procured xmi:id="_22_1_7f3a0c1_6f592009da70"
base_Class="_22_1_7f3a0c1_a89dbce342d7"
                            partNumber="SP-40" supplier="Notional Solar Ltd."/>
```

Three things are worth noticing, because each one comes back later.

First, the solar panel is not one record but several, scattered across the file. The block itself is a UML class near the middle. The fact that it *is* a SysML block is a separate record near the end (`sysml:Block`), pointing back at the class by its identifier. The team's own "assembly level" tag and the supplier data are two more records from the team's custom profile. To know everything about the solar panel you have to collect all four.

Second, the type of the `ratedPower` property is not written as a name. It is an identifier, `_22_1_7f3a0c1_dd4fe0cc913f`, that you have to look up elsewhere in the file to discover it means `Power`. The same is true of the port's type. The file is written for machines, not for people.

Third, the diagrams are not here at all. The XMI holds the model's elements and relationships; the diagrams that people actually looked at, with their boxes, lines and layout, live in Cameo's own project file and appear in the export only as a list of names. When the team "reviewed the model," they reviewed the diagrams, and the diagrams are precisely the part that cannot be exported, compared or checked.

2.3 Measuring the spider web

"The model looks like a giant spider web" is Bob's phrase, and it was meant as a complaint, not a measurement. Jim's first act as the person responsible for the model was to turn the complaint into numbers, because numbers can be tracked and complaints cannot. The measures are simple counts from the drop-1 XMI:

Table 2.2: The spider web, measured from drop 1

Measure	Drop 1	What it tells you
Model elements (records with an identifier)	819	the size of the export, including comments and stereotype applications

Measure	Drop 1	What it tells you
Packages	20	how the model is organized
Blocks / requirements	40 / 30	the parts of the system and the "shall" statements
Diagrams	47	the pictures people review; 11 of them are stale copies or briefing snapshots
Relationships	124	satisfy, verify, trace, derive, allocate, connectors, generalizations, associations
Relationship ends per block, maximum	17 (ObservingStation)	how tangled the most-connected block is
Relationship ends per block, mean	1.4	how tangled a typical block is
Package dependency cycles	2	pairs of packages that each depend on the other
Unused blocks	4 (three clutter blocks, plus the context block, unused by design)	blocks nothing refers to
Duplicate or stale blocks	2	copies left behind by the document port

Two ways of counting elements. The 819 above counts every record in the XMI file that carries an identifier, including comments and stereotype applications, because that is what makes the file big. The converter counts differently: it counts the model elements it walks, and its own report on drop 1 ends with this line (columns: in, mapped, folded, TODO):

docs/tidewater/sprints/01/drop1-xmi2sysml-report.txt

```
total                          560     444      65     51
```

Both numbers are right; they answer different questions. Elsewhere in this book, "elements" means the converter's count unless a table says otherwise. The relationship and per-block counts in the table are Jim's own counts of relationship ends in the XMI; the converter's report and the generator's spider-web summary count only the relationship kinds they handle, so their numbers are a little lower.

A few of these deserve explanation.

Diagrams. The model has 47 diagrams for 40 blocks. Some of that is legitimate: a block definition diagram per segment, an internal block diagram per assembly that has connections, an activity diagram, a state machine, a sequence diagram, use cases, a requirements diagram and two matrices. The rest is what happens when diagrams are the main way to look at a model and nobody is allowed to delete one in case someone needs it. Their names tell the story:

```
bdd Network                    ibd ObservationNetwork          act ObserveCoast
bdd Stations                   ibd ObservationNetwork v2       act ObserveCoast - swimlanes
bdd Stations OLD               ibd ObservingStation            stm BuoyModes
bdd Stations (Jim edits)       ibd PowerSystem                 stm BuoyModes OLD
bdd Everything                 ibd PowerSystem (copy)          sd DailyUpload
bdd Legacy Import              ibd Copy of SensorSuite         sd DailyUpload (as briefed)
```

```
par PowerSystem                  Trace matrix (all)            Briefing chart 2024-03 - do not
edit
par PowerSystem draft            Package overview              PDR backup slide 12
... and 23 more (47 in all)
```

Package dependency cycles. A package depends on another when something in it refers to something in the other: a part typed by a block over there, a trace relationship pointing over there. A cycle means two packages each depend on the other, so neither can be understood, reviewed or changed alone. In drop 1 the cycles are: Stations with DataCenter, and Network with Operations. Each one began as a reasonable shortcut during the document port (a trace link drawn from a station block straight to a data center requirement, because that was where the ICD paragraph pointed) and nobody untangled it afterwards. (The SysML v2 tools count the same tangle in their own ways; Section 11.4, The five views, read (p. 161) explains why the numbers differ.)

The most-connected block. ObservingStation carries 17 relationship ends. Most of them are trace relationships the team drew from the block to every requirement that mentioned it in any document. Some of those are real satisfy relationships under another name; some duplicate a satisfy that already exists; some are simply wrong. In a diagram, those links are the strands of the web.

The Legacy Import package. When the team first ported the documents, they imported a spreadsheet of requirements and blocks with Cameo's Excel import. The import created elements the team later rebuilt by hand, and the originals were never deleted. They sit in a package called "Legacy Import": a `Battery_old`, a `Copy of SensorSuite`, a `TBD Component` that nothing uses, and a requirement-like class called "Requirement from slide 14" with no identifier. Every review since 2023 has had a comment saying "clean up Legacy Import." It is still there.

You can measure the spider web yourself. The converter's report lists every element type and what happens to it:

```
perl "$KIT/bin/xmi2sysml.pl" --report --out drop1-v2 drop1/tidewater.xmi
```

These lines from the report summarize the conversion:

```
total                            560     444      65     51
ported vs remaining (metric A filter uml:Class|Port|Property|Connector|Activity): 206 of 206
ported, 0 remaining
stereotypes not converted: L1Network 1, L2Segment 3, L3Station 10, L4Subsystem 8, L5Assembly 13,
L6Component 1, Procured 3
unmapped (TODO in the output): 12
self-check: ok
```

Read the `total` line as four counts: 560 elements came in; 444 were mapped to SysML v2; 65 were folded into something else (a connector end becomes part of its connector, a multiplicity becomes part of its property); and 51 fall under a TODO. The 51 are fewer things than they sound: the three lifelines, three messages and six message events of one sequence diagram, for

instance, all sit under a single TODO for that diagram. The `unmapped` line says they produce 12 TODO lines. Add one TODO line for each of the 39 custom stereotypes listed above it, and one note about the constraint expression, and the converted files contain 52 TODO lines.

Chapter 6, SysML v2 for SysML v1 modelers (p. 57) explains the rest of the report. For now, the TODO lines are what matter. They are the places where the model contains something the converter could not carry into SysML v2. They are the conversion debt, and counting them will become the team's main measure of progress. Notice also the `ported vs remaining` line: every block, port, property, connector and activity in the model, 206 of 206, came across. The debt is real but bounded.

2.4 A tour of drop 1, package by package

The converter writes one file per top-level package of the Cameo model, so the converted drop doubles as a map of the model's organization. Counting what is in each file gives a first look at where the size, the debt and the tangles are. These are real counts from the converted drop 1.

Table 2.3: Drop 1 converted, file by file

File	Lines	Holds	TODO lines	Imports
`stations/Stations.sysml`	190	25 part defs: segment, stations, power, sensors, telemetry, hull, mooring, components	29	Analysis, DataCenter, Interfaces, Requirements, Types
`requirements/Requirements.sysml`	188	30 requirements, 29 satisfy, the derivations and traces	0	DataCenter, Network, Stations, RequirementDerivation
`verification/Verification.sysml`	60	6 verification defs (the v1 test cases)	0	Requirements
`behavior/Behavior.sysml`	55	8 action defs: the observation cycle	4	Interfaces
`datacenter/DataCenter.sysml`	51	5 part defs: ingest, quality control, archive, publisher	5	Interfaces, Stations
`network/Network.sysml`	47	5 part defs, actors, 4 use cases, the context	3	DataCenter, Operations, Stations
`interfaces/Interfaces.sysml`	40	4 port defs, 3 item defs	0	Types
`operations/Operations.sysml`	30	5 part defs: depot, vessel, crane, spares	5	Network
`legacy-import/Legacy_Import.sysml`	27	4 part defs left from the document port	0	Interfaces, Stations, Types
`types/Types.sysml`	22	value types, units, an enumeration	3	none
`allocations/Allocations.sysml`	14	7 allocations of activities to blocks	0	Behavior, DataCenter, Stations
`analysis/Analysis.sysml`	13	the energy balance constraint	2	Types
`tidewater/Tidewater.sysml`	4	the custom profile's placeholder	1	none

Three things jump out of that table, and the team noticed all of them in the first week.

The size is small. The whole model is about 740 lines of text. Bob had spent two years building it, and it fits in a long afternoon's reading. The point is not that two years were wasted; the content took that long to decide. The point is that once decided, the content is compact, and the work of keeping it current should be proportionate.

The debt is concentrated. Twenty-nine of the 52 TODO lines are in one file, `Stations.sysml`, and almost all of them are level stereotypes. That told Jim where the first weeks' effort would go, and that it would go quickly.

The tangles are written down. Look at the last column. `Stations` imports `DataCenter`, and `DataCenter` imports `Stations`. `Network` imports `Operations`, and `Operations` imports `Network`. Those are the two package dependency cycles from the spider-web table, and in the text they are not hidden in a model browser: they are two pairs of `import` lines, at the top of four files, that anyone can read. `Stations` also imports `Requirements`, because a dozen trace links were drawn from station blocks straight to requirements. In Cameo, finding these took a dependency matrix and an afternoon. In text, they are on the first screen of each file.

Bob's reaction to that last point is worth recording:

> *I always knew the stations and the data center were tangled. I could never show anyone. Now it's four lines.*
>
> — Bob, sprint 1

2.5 Why it did not scale

The team's experience with Cameo is common, and it is worth being precise about why, because "we hate the tool" is not an argument that management will accept, and it is not quite true either. Cameo is a capable tool. The problems the team ran into come from the combination of SysML v1, a diagram-first way of working and a team that was never given the practices that make a shared model work.

2.5.1 The diagram is not the model, but everyone treats it as if it were

In SysML v1 tools, the model and the diagrams are separate things: a block exists once in the model and can appear on many diagrams. That separation is correct and powerful. In practice it confused everyone. Reviewers looked at diagrams, so a relationship that existed in the model but not on any diagram was invisible, and a stale diagram that nobody had opened since 2023 still looked like part of the design to anyone who found it. When Bob redrew the PowerSystem IBD for a briefing, the briefing version had a different layout and fewer ports, and for the people in the room it was the PowerSystem.

2.5.2 Nothing could be compared

The question a reviewer most needs answered is: *what changed since last time?* In the document era the answer was the redline. In Cameo there was no good answer. Cameo has a model-comparison feature, but it compares two projects element by element in a tree that runs to thousands of rows, and the team could not turn its output into anything a reviewer could read. The XMI export changes in ways unrelated to the model (element order, generated identifiers on some exports, tool-version headers), so comparing two XMI files with a text comparison tool

shows thousands of meaningless differences. When drop 2 arrived, the team's review consisted of opening both versions side by side and clicking through the diagrams looking for differences by eye.

2.5.3 Nothing could be merged

A shared model needs a way for two people to work at once and combine their work. Cameo's answer is a server (Teamwork Cloud) that locks elements while someone edits them. The program never bought the server, so the team passed the project file around by email, and only one person could edit at a time. Bob owned the model in practice because Bob had the file. When the prime took over, the file moved to the prime, and the government lost the ability to change it at all.

2.5.4 Nothing was checked automatically

Cameo has a validation feature with rules, and the team ran it once, during the training course. It produced eight hundred warnings, most of them about diagram formatting, and nobody ran it again. The questions the team cared about (is every requirement satisfied, is every requirement verified, does the requirement text in the model match DOORS, does every assembly carry its level tag) were never encoded anywhere. They were answered, when they were answered, by Bob clicking through matrices before a review.

2.5.5 Seats, licenses and one machine

Cameo is licensed per seat. The program bought three. A fourth engineer who wanted to look at the model had to ask for screenshots. The model lived on the machines where Cameo was installed. The data center's engineers, the test engineers and the program office never saw the model, only pictures of it.

2.5.6 The team never had a workflow, only a tool

This is the deepest reason, and it is not Cameo's fault. The team was given a tool and told to use it. Nobody gave them a way of working with it: how changes are proposed, reviewed, approved and recorded; how the model is baselined; how it is checked; how it is kept consistent with DOORS. Without those practices, the model behaved exactly like the documents had, except that it was harder to read and nobody outside the team could open it. It inherited the document era's costs and added new ones.

> **Case study: The scaling problem, stated plainly**
>
> SysML v1 in a diagram tool scales with the number of people who can click. Changes are made one at a time, by one person, in one tool, and reviewed by looking at pictures. A team of three can keep up with a small model that way. It cannot keep up with a model that changes every few months from a contractor, has to stay consistent with a requirements database, and has to be reviewed by people who do not own a license.

2.6 What the team believed, and what was true

By 2026 the team had drawn a conclusion from the Cameo years: *modeling does not work for us.* It is worth separating that conclusion into its parts, because some parts are true and some are not.

Table 2.4: The Cameo years: belief and fact

What the team believed	What was actually true
"Modeling is slower than documents."	Point-and-click modeling of content that already existed in documents is slower than the documents. Modeling as the primary record, with automated checks, is a different activity.
"The model is a spider web."	Measurably true: duplicates, stale diagrams, package cycles and tangled trace links. The web was built by the port, and can be untangled.
"Nobody uses the model."	True: nobody outside the team could open it. That is a distribution problem, not a modeling problem.
"We can't tell what changed in a drop."	True with XMI and screenshots. Not true with a text model in version control (see Chapter 6, SysML v2 for SysML v1 modelers (p. 57) and Chapter 12, Git from zero, for systems engineers (p. 171)).
"The contractor owns the model now; we just review it."	True contractually. The government can still keep its own derived copy, check it, and report discrepancies; that is the plan in Chapter 13, The operating model (p. 196).
"We are engineers, not programmers."	True, and this book does not ask anyone to become a programmer. Text files, reviews and checklists are engineering tools.

Objection: "We already tried modeling. It took two years and gave us a spider web."

Bob, at the first team meeting about the directive in Chapter 3, The directive (p. 22).

Answer

The two years went into porting documents by hand into a tool with no workflow. The spider web is measured above, and the measures are the starting point of the twelve weeks. Chapter 27, Sprint 2: Drop 1 on develop, every change gated (p. 424) through Chapter 30, Sprint 5: Grading the requirements, and a trace that defends itself (p. 475) show the web being untangled package by package, with each tangle reported to the prime as a discrepancy, and Chapter 33, The twelve-week scorecard (p. 520) shows the same measures at the end.

Objection: "If we couldn't make Cameo work, why would SysML v2 be different?"

Tom, same meeting.

Answer

Because the problems in this chapter are about comparison, merging, automated checking and distribution, and those are exactly what a text model in version control does well. The language matters less than the fact that SysML v2 has a standard text notation. Chapter

6, SysML v2 for SysML v1 modelers (p. 57) shows the same Tidewater elements in both forms; Chapter 12, Git from zero, for systems engineers (p. 171) shows what comparison and merging look like for text.

2.7 Reviewing a drop by screenshot

The clearest picture of why the Cameo years were painful is the government's review of a prime delivery in 2025, the first year the prime maintained the model. It is worth walking through, because Chapter 27, Sprint 2: Drop 1 on develop, every change gated (p. 424) repeats the same job the new way and the contrast is the core of the case.

The prime delivered the Cameo project file and a delivery letter listing "significant changes" in prose: "Updated PowerSystem to reflect ECP-0117. Added maintenance states. Updated requirement allocations per CDR action items." The government had thirty days to accept or reject the delivery.

Bob opened the new project in Cameo next to the previous one, on two monitors. For two weeks, Bob clicked through every diagram in both versions, looking for differences by eye. Each difference went into a comment matrix, a spreadsheet with columns for the diagram, the element, the change observed, whether it matched the delivery letter, and the government's comment. To make the comments understandable to the prime, Bob took screenshots of both versions and pasted them into a slide deck, one slide per finding.

Table 2.5: The government's review of one Cameo delivery (notional)

Review activity, 2025 delivery	Effort (notional)
comparing diagrams by eye, two versions side by side	7 working days
screenshots and the comment slides (61 slides)	3 working days
comment matrix (48 comments)	2 working days
review meeting with the prime	1 day
following up the prime's responses	2 working days over the next month

Three things about this review deserve attention.

It could not be complete. Bob compared diagrams, not the model. A change to an element that appeared on no diagram, or a relationship added in the model browser, was invisible. Bob knew this and said so in the review memo: "Review limited to changes visible on diagrams."

It could not be repeated. The comparison lived in Bob's head and in screenshots. When the prime delivered again three months later, the review started from zero.

It produced no record the model could use. The 48 comments lived in a spreadsheet. Whether each was resolved in the next delivery was checked, again, by eye.

In Chapter 27, Sprint 2: Drop 1 on develop, every change gated (p. 424), the same job, reviewing what changed in a drop, is a report generated in seconds from the two XMI files, element by element, including changes that no diagram shows. Chapter 43, Maintenance through the life cycle (p. 602) repeats it for six more drops.

2.8 Who owns what, after the contract change

The 2025 contract change matters to everything in this book, so it is worth stating precisely.

Table 2.6: Ownership after the 2025 contract change

Item	Owner	What the government can do
the SysML v1 model in Cameo	the prime (CDRL A003)	receive each drop, review it, accept or reject it, report discrepancies
the DOORS requirements database	the prime, with government approval of changes through the CCB	receive exports, review, approve changes through the CCB
the government's derived SysML v2 model	the government team (this book)	everything: edit, check, baseline, publish
discrepancy reports against the prime's model	the government raises; the prime resolves	track each to the drop that fixes it

The directive in Chapter 3, The directive (p. 22) keeps this split: the prime's v1 model "remains the contractual baseline," and the government maintains a *derived* v2 model. That word, derived, shapes every technical decision in Part II. The government cannot fix a problem in the prime's model; it can only report it and wait for a later drop. But it can fix the problem in its own v2 model immediately, record that it did, and check, when the next drop arrives, whether the prime fixed it too. Chapter 13, The operating model (p. 196) shows how a vendor branch makes that possible without the government's fixes being overwritten by each new drop.

2.9 What a model is, and what a diagram is

One last idea from the Cameo years, because the team confused it for three years and it matters for everything after. A *model* is a set of elements and relationships: a block named `PowerSystem` that has a part `solar` typed by the block `SolarPanel`, a satisfy relationship from `PowerSystem` to requirement TW-SYS-011, and so on. A *diagram* is a picture of some of those elements and relationships, chosen and laid out by a person for a purpose.

Diagrams are views. They are excellent for explaining, for seeing shape, for spotting a missing connection at a glance. They are poor as a record, because a picture shows only what someone chose to draw, and because the layout (where the boxes are, how the lines bend) carries no engineering meaning but takes most of the effort.

In the Cameo years, the team treated diagrams as the record. In the new way of working, the text is the record, and diagrams are generated from it whenever someone needs a picture. The kit draws part trees, trace diagrams, interface diagrams and package diagrams from the text (Chapter 17, The DevSecOps tools that earn their keep (p. 255)), and a viewer redraws them live beside Vim as the text changes (Chapter 10, Vim for models (p. 143)). The diagrams still exist. They have just stopped being the thing that has to be kept consistent by hand.

2.10 Exercises

Exercise 2.1: Measure it yourself

Generate drop 1 with `perl sim/tidewater-v1.pl --drop 1 /tmp/drop1`. Open `/tmp/drop1/tidewater.xmi` in Vim. Use `/name="Battery` and `n` to find every element whose name starts with "Battery". How many are there? Which one is the real battery? How did you decide?

Exercise 2.2: Read the manifest

Open `/tmp/drop1/MANIFEST.txt`. Then, in the terminal, run `sha256sum /tmp/drop1/tidewater.xmi`. Does the checksum match the manifest? What would it mean if it did not, and who should you call?

Exercise 2.3: Belief and fact

Write your own version of the table "The Cameo years: belief and fact" for a model on your program. Which of the beliefs are about the language, which about the tool, and which about the way the team was asked to work?

CHAPTER

3

The directive

From what you know →

You know

a directive that arrives with a date and no resources, and the meeting where the team works out what it really means.

→ Here it is called

the DevSecOps directive and the twelve-week scorecard.

→ What is new

agreeing in week zero on what will be measured, so success is a number, not an opinion.

Note: What you have, what you need

What you have: a memo telling you to adopt something you have never done, a skeptical team and twelve weeks.

What you need from this chapter: a precise reading of what the memo asks, the team's objections written down so none of them get lost, and a definition of "measurable progress" that everyone, including management, agrees to before the work starts.

3.1 The memo

The memo arrived on a Thursday in late September 2026, attached to an email from the program manager with the subject line "FYSA – DevSecOps for SE artifacts – action required." It was two pages long. The first page is reproduced here because the team read it so many times that Bob could later quote it from memory.

```
MEMORANDUM FOR TIDEWATER SYSTEMS ENGINEERING

SUBJECT: Adoption of DevSecOps Practices for Systems Engineering Artifacts

1. Effective FY27, program systems engineering artifacts, including the system model, will be
```

```
   developed and maintained using DevSecOps practices on the enterprise DevSecOps platform
   (GitLab).

2. The Tidewater system model will be transitioned from SysML v1 to SysML v2 textual notation and
   placed under configuration control in the enterprise GitLab instance. The prime contractor's
   SysML v1 deliveries under CDRL A003 remain the contractual baseline until further notice; the
   government will maintain a derived SysML v2 model and keep it current with each delivery.

3. Systems engineering leads will demonstrate:
   a. automated quality checks on every model change;
   b. traceability from DOORS requirements to the model, computed rather than hand-maintained;
   c. peer review and approval of every change, with a complete audit record;
   d. reproducible baselines;
   e. measurable progress reported at regular intervals.

4. The Tidewater MBSE lead will brief initial results at the December program management review.

5. Tools and data flows require ISSM approval before use with program data.
```

The second page listed training resources (a link to the enterprise platform's documentation and a recorded webinar), a point of contact in the enterprise DevSecOps office, and a sentence that Tom underlined twice: "Teams are encouraged to start small and iterate."

3.2 Reading it carefully

Jim read the memo three times before showing it to anyone. On the third reading Jim did what twenty-six years of requirements work had trained Jim to do: treat it as a specification, and separate what it requires from what it assumes and what it leaves open.

Table 3.1: The directive, read as a specification

The memo says	What it actually requires	What it leaves open
model "developed and maintained using DevSecOps practices"	changes made, reviewed, checked and recorded through the platform	which practices; how much is enough
"transitioned ... to SysML v2 textual notation"	a SysML v2 text model exists and is the government's working copy	how faithful the port must be; what to do with what the converter cannot carry
prime's v1 "remains the contractual baseline"	the government cannot change the authoritative model	how the derived v2 model stays current with each drop
"automated quality checks on every model change"	a pipeline that runs on every change and blocks bad ones	which checks
"traceability ... computed rather than hand-maintained"	a trace report produced by a program from the model and DOORS	which measures
"peer review and approval of every change, with a complete audit record"	merge requests, approvals, protected branches	who approves what
"reproducible baselines"	tags that can be rebuilt identically	how often

The memo says	What it actually requires	What it leaves open
“measurable progress reported at regular intervals”	numbers, every sprint	which numbers
“brief initial results at the December PMR”	a briefing in about twelve weeks	what “initial results” must show
“ISSM approval before use with program data”	the toolchain must be approved	how long that takes

The right-hand column is long. That is not a complaint about the memo. A directive from leadership is supposed to say *what* and leave *how* to the people doing the work. But it meant that most of the important decisions were now Jim’s, and that the team would have to make them without experience. Jim wrote on the printout: “We decide the ‘how’. Write every decision down, with the reason. If we’re wrong, we’ll at least know why.”

That note became a habit, and it is why this book can explain the thought process behind each decision: the team recorded them, first in the wiki and later in merge request descriptions.

3.3 The meeting with the program manager

Jim asked for thirty minutes with the program manager the next morning and brought two questions.

The first was: *what does success look like in December?* The program manager’s answer was frank. “Leadership has spent money on the platform and on licenses. They want to see that programs are using it and that it does something. I need a story I can tell with numbers. I don’t need perfection. I need progress I can measure, and I need your team not to hate it.”

The second question was: *what are we allowed to stop doing?* This mattered more than it sounded. A team of three could not port a model, learn a new way of working and keep doing every document-era activity at the same time. After some discussion, the program manager agreed to three things:

- the monthly screenshot-based model review package would be replaced by whatever the team could publish from the new system, starting when it was ready;
- the government’s VCRM spreadsheet would be frozen at its current revision and replaced by the computed trace report once the trace report existed;
- the team would spend no time on Cameo beyond reading each drop, for the duration of the twelve weeks.

Tip: Ask what you can stop doing

Every change program adds work. Few of them say what to remove. Ask, early, in writing. The three things Jim negotiated freed roughly a fifth of the team’s time, and that time is what paid for the learning curve.

3.4 The team meeting

Jim scheduled the team meeting for Monday afternoon, 28 September, in the small conference room with the broken projector. Jim printed the memo three times, handed out copies, and let

Bob and Tom read it in silence.

Bob finished first and put the memo face down on the table.

> *So we're doing it again. Three years ago it was "go MBSE." We spent two years clicking every box in the SSDD into Cameo. Now it's "go DevSecOps," and we get to do it all again in a different tool. What happens in three years?*
>
> — Bob

Tom read the memo twice, underlined "start small and iterate," and asked a question that sounded practical and was actually the deepest objection in the room.

> *I run CM. I know what a baseline is, I know what a CCB is, I know what an audit is. What I don't know is what any of these words mean. Pipeline. Merge request. I've never written code. Why can't we just do systems engineering the way we know how to do it, and put the documents in GitLab if they want them in GitLab?*
>
> — Tom

Jim did not have good answers yet, and said so. What Jim did instead turned out to be the most useful thing anyone did in the first week: Jim went to the whiteboard and wrote down every objection, in the speaker's words, without arguing with any of them. "If we can't answer these with evidence by December," Jim said, "then we'll put that in the briefing too."

The whiteboard photo went into the team's notes that afternoon. The objections are reproduced here, each with a pointer to the part of the book where it is answered. They are worth reading slowly; if you are in the team's position, most of them are yours.

Objection: "We already did this once. Modeling took two years and gave us a spider web."

Bob.

Answer

The spider web is measured in Chapter 2, The Cameo years (p. 10) and untangled a strand at a time: one package loop broken in Chapter 29, Sprint 4: the model site, a cycle broken, and the demo that landed (p. 459), the Legacy Import clutter removed and the trace spaghetti pruned in Chapter 30, Sprint 5: Grading the requirements, and a trace that defends itself (p. 475). The last loop, between Stations and DataCenter, is still open after twelve weeks and stays on the scorecard (Chapter 47, Keeping SysML v2 from becoming a spider web (p. 645)). The two years went into porting documents by hand into a tool without a workflow; this time the content already exists, a converter carries most of it, and the workflow is the point. Chapter 33, The twelve-week scorecard (p. 520) compares the two efforts.

Objection: "Why can't we just do systems engineering the old way?"

Tom.

Answer

You can keep doing systems engineering; nothing in this book changes what a requirement,

a baseline or a review is. What changes is how much of it is done by hand. Chapter 5, DevSecOps in systems-engineering words (p. 46) translates every DevSecOps practice into the systems engineering practice it implements, and Chapter 38, How the team changed its mind (p. 557) revisits this objection against twelve weeks of evidence.

Objection: "We're systems engineers, not programmers. We don't know how to code."

Tom and Bob.

Answer

Nothing in the twelve weeks requires programming. The team edits text files in Vim (which they already use), types a small number of commands (listed in full in Chapter 9, The survival kit (p. 126) and Chapter 12, Git from zero, for systems engineers (p. 171)) and reads reports. The pipeline is a configuration file of about a hundred lines that calls tools other people wrote; Chapter 20, The pipeline, job by job (p. 302) explains every line.

Objection: "The contractor owns the model. Anything we do is a copy that will go stale."

Bob.

Answer

It would, if keeping it current were done by hand. Chapter 13, The operating model (p. 196) sets up a vendor branch that holds exactly what the prime delivered, converted, so each new drop is merged into the government's model like any other change, with the conflicts shown precisely. Chapter 43, Maintenance through the life cycle (p. 602) shows six more drops absorbed this way over the following fifteen months.

Objection: "Nobody outside our team will be able to read SysML v2 text."

Jim, playing devil's advocate.

Answer

Fewer people could read the Cameo model, because only three people had licenses. The v2 model publishes itself as a website with generated diagrams and reports on every baseline (Chapter 17, The DevSecOps tools that earn their keep (p. 255), Chapter 29, Sprint 4: the model site, a cycle broken, and the demo that landed (p. 459)), readable in a browser by anyone with access. The text is for the people who maintain the model; the site is for everybody else.

Objection: "The ISSM will never approve this in time."

Tom.

Answer

The toolchain is small (Git for Windows, Perl and Vim, already approved on the laptops, plus the enterprise GitLab, already approved for the enterprise) and Chapter 27, Sprint 2:

Drop 1 on develop, every change gated (p. 424) starts the ISSM conversation in week three with a written system description. It does cause a three-day block in Chapter 28, Sprint 3: the first red X, DOORS in the model, and a wait nobody named (p. 440), and the plan absorbs it.

Objection: "We'll spend twelve weeks learning tools and have nothing to show."

Bob.

Answer

The plan in Chapter 13, The operating model (p. 196) delivers something measurable every sprint, starting in Chapter 26, Sprint 1: A GitLab project the team can work in (p. 404) with a working project and a first pipeline, run locally on drop 1 until the runner arrives in Chapter 27, Sprint 2: Drop 1 on develop, every change gated (p. 424). The scorecard below is defined before the work starts, so progress is measured against targets the team set, not against hope.

Objection: "Management will just move the goalposts."

Tom, quietly, at the end of the meeting.

Answer

Possibly. The defense is the scorecard: agreed in writing in week one, measured automatically, reported every sprint. Chapter 35, The briefing (p. 532) shows the December briefing built from it.

Objection: "In three years there'll be another directive."

Bob.

Answer

Very likely. What survives a change of tools is the habit of keeping the system's description as reviewed, versioned, checked text with a computed trace. That habit is tool-independent; Chapter 51, Pitfalls and anti-patterns (p. 684) discusses how not to lock it into GitLab either.

3.5 The prime's view

Jim called the prime's modeler on Friday 2 October, the afternoon drop 1 arrived, to explain what the government was about to do with it. Jim expected resistance. The prime's modeler was cautiously relieved.

> *We've been getting comment matrices with screenshots for a year. Half the time we can't tell which diagram the screenshot came from. If you can tell us exactly which element is wrong, by its ID, we can fix it in the next drop.*
>
> — the prime's modeler, 2 October

The call settled three working agreements that the rest of the book relies on:

- **The prime keeps delivering what the CDRL requires**, unchanged: XMI, ReqIF, CSV and a manifest with checksums. The government will not ask the prime to change tools or formats.
- **The government reports problems as discrepancy reports**, one per problem, each naming the element by its v1 identifier (the `xmi:id` that the converter keeps on every TODO line and that the drop-to-drop comparison reports), with the evidence and the drop by which the government would like it fixed.
- **The prime keeps element identifiers stable** between drops, which Cameo does by default as long as elements are edited rather than deleted and recreated. This one agreement is what makes the government's drop-to-drop comparison and merge possible (Chapter 43, Maintenance through the life cycle (p. 602)).

Tip: Talk to the contractor early

The government's derived model depends on the contractor's deliveries in ways the contract does not spell out: stable identifiers, consistent export settings, a manifest with checksums. A friendly call before the first drop is processed costs an hour and avoids months of friction. Write down what was agreed and send it to the contracting officer's representative.

3.6 Defining measurable progress

The most important thing the team produced in the week before sprint 1 was not a tool or a file. It was a single page that defined what "measurable progress" would mean, agreed by Jim, the program manager and the deputy program manager. The page has three columns that matter: the measure, how it is computed (automatically, from the system, never by hand), and the target for week twelve. A fourth column holds the document-era baseline where one exists, so the comparison in December is fair.

The rule Jim set for choosing measures was simple: *every measure must be computed by a program from the repository or GitLab, and every measure must be something a skeptic in the room would accept as meaningful.* No "percentage complete" judged by eye, no counts of meetings held.

Table 3.2: The twelve-week scorecard, as agreed before sprint 1

#	Measure	How it is computed	Week-12 target	Document-era baseline (notional)
1	Conversion debt: TODO lines left by the converter	counted by the pipeline's `todo` job on every change	all structure TODOs closed; behavior started	not applicable
2	Change lead time	median hours from merge request opened to merged, from GitLab	under 3 working days	23 working days
3	Gated changes	share of changes to the integration branch that passed the pipeline and were approved by a code owner other than the author	100%	not measured
4	Trace answer time	wall-clock time of the pipeline's `trace` job	under one minute	2 working days

#	Measure	How it is computed	Week-12 target	Document-era baseline (notional)
5	Trace measures	requirements linked (C), orphans (F), unverified (G), completeness (I), text drift	computed on every change; every gap has an owner	computed at reviews only
6	Discrepancies reported to the prime	discrepancy report issues opened, each with a due drop	every gap found is reported	ad hoc emails
7	Spider web in the v2 model	package dependency cycles, duplicate/stale blocks, max relationship ends per block	no cycles, no duplicates	see Chapter 2, The Cameo years (p. 10)
8	Pipeline health	share of pipelines passing on the integration branch; longest time from a failure to the next pass	above 90%; restore within one working day	not applicable
9	Reproducible baselines	baseline tags whose rebuild matches the recorded checksum	interim baseline tagged and reproducible	revision letters, not reproducible
10	Review turnaround	median hours from merge request opened to first approval	under 1 working day	3 weeks per review package
11	Delivery against plan	story points done / committed per sprint, from the journal	80% or better	not measured
12	Team morale	five-question pulse survey (1 to 5) at each retrospective	3.5 or better, rising	not measured

A few choices on this page deserve explanation, because they reveal the team's thinking.

Conversion debt is the primary progress measure. The converter carries most of the v1 model into SysML v2 automatically, but leaves a `// TODO` comment wherever it meets something it cannot carry (see Chapter 6, SysML v2 for SysML v1 modelers (p. 57)). Those lines are a precise, countable, shrinking list of work. Unlike "percent complete," a TODO count cannot be argued with: either the line is there or it is not. Jim chose it because it is the measure Bob, the strongest skeptic, would accept.

Lead time and review turnaround have document-era baselines. These are the measures where the new way should be dramatically better, and where management will look first. The team was careful to use the same definition on both sides (from request to incorporation) and to label the old numbers as notional reconstructions.

Gated changes must be 100%. This is the one measure with no tolerance. The memo asks for review and approval of *every* change. If even one change reaches the integration branch without a passing pipeline and an approval, the audit record has a hole. GitLab's protected branches make this a setting rather than a promise, as Chapter 15, Setting up GitLab (p. 226) shows.

Morale is measured. Including it was the program manager's idea, and it surprised the team. "If your team hates this in December," the program manager said, "the numbers won't matter, because it won't last." The pulse survey is five questions, answered anonymously at each retrospective on a scale of 1 (strongly disagree) to 5 (strongly agree):

1. I understand why we are working this way.
2. I can do my work in the new tools without help.

3. The checks catch problems I would have missed.
4. I can see our progress.
5. I would choose to keep working this way.

Nothing is measured by hand. Every measure on the page comes from the repository, GitLab's records or the team's journal, and the pipeline or a script computes it. This was a deliberate break from the document era, where every status number was a one-off, like Bob's Wednesday list in Chapter 1, Thirty years of documents (p. 2).

Warning: Agree on the measures before you start

If you define your measures after the work, you will be accused (correctly) of choosing the ones that look good. Define them first, get them signed, and report all of them every sprint, including the ones that look bad. Several of the team's measures looked bad in sprint 3. Reporting them anyway is what made the December briefing credible.

3.7 Jim's plan in one page

With the scorecard agreed, Jim sketched the twelve weeks as six two-week sprints, each with a goal that produced something measurable. The detailed plan, and the reasoning behind the sprint length and the size rules for work, is in Chapter 13, The operating model (p. 196); this is the outline that went to the program manager on Friday 2 October, the same day the prime delivered drop 1:

Table 3.3: The twelve-week plan, as sent to the program manager

Sprint	Goal	Evidence at the end
1	A GitLab project the team can work in; drop 1 received and assessed	project, protected branches, a first pipeline drafted and run locally on drop 1 (gating every change starts in sprint 2, when the runner is online), TODO inventory
2	Drop 1 on the integration branch through the vendor branch; every change gated	vendor branch, gated merges, TODO burn-down starts
3	Network and Types cleaned up; DOORS imported	trace report on every change; first drift found
4	Stations cleaned up; gaps reported to the prime	discrepancy reports; the model site
5	Data center and operations cleaned up; release rehearsed	structure TODOs closed; release procedure
6	Behavior work started; interim baseline; the briefing	baseline-0, release notes, the December briefing

At the bottom of the page Jim wrote one more line, which the program manager initialed: "We will report every measure every sprint, including the bad ones."

3.8 The email Jim sent that night

After the team meeting, Jim went home and wrote an email to Bob and Tom. Jim almost did not send it, and later said it was the most important thing written in the twelve weeks. It is reproduced here because it set the tone for everything that followed.

```
From: Jim
To: Bob; Tom
Subject: About today

I don't have answers to most of what you said today, and I'm not going to pretend I do.

Here is what I can promise:

1. We will not port anything by hand that a program can port for us. If we find
   ourselves clicking boxes again, we stop and rethink.
2. Every decision we make about how to work goes in writing with the reason, so we can
   change it when we're wrong.
3. Every objection on the whiteboard gets an answer with evidence by December, or it goes
   in the briefing as an open issue. I'll brief it myself.
4. We measure everything the same way every sprint, including the bad numbers.
5. If after twelve weeks this is worse than what we had, I will say so to the PM.

What I'm asking:

1. Try it for real for twelve weeks. Not halfway.
2. Tell me every day what's in your way. That's what the stand-up is for.
3. Tom, take the pipeline. Bob, take the model. I'll take the plan, the prime and
   management, and I'll do reviews.

Neither of you has to become a programmer. I checked. The tools are written; we have to
learn to use them, the way we learned DOORS.

-- Jim
```

Bob replied the next morning with one line: “Twelve weeks. Then we talk.” Tom did not reply, but on Wednesday Tom had already requested a GitLab group from the enterprise administrators, three days before Jim had planned to.

Tip: What the email does

The email does four things a change leader has to do: it admits what is not known, commits to specific behaviors, asks for specific behaviors in return, and sets an exit condition. It does not argue with the objections, and it does not promise that the new way will be better. It promises that the team will find out honestly.

3.9 Week zero

The week between the memo and sprint 1 (29 September to 2 October 2026) was not part of any sprint, and Jim used it to prepare the ground. Nothing in it required DevSecOps knowledge; all of it was ordinary program work. The full checklist is in Appendix J, Checklists (p. 789); the main items were:

- **Request access early.** Tom asked the enterprise GitLab administrators for a group (`tidewater`), a project (`tidewater/model`), three accounts with the right roles, and a runner, the machine that will execute the pipeline. The administrators' standard turnaround was five working days, so the request had to go in before sprint 1, not during it.
- **Book the ISSM.** Jim asked for an hour with the ISSM in the second week of sprint 2 to present the toolchain, knowing that approvals take time and that sprint 3 would need them.
- **Confirm the laptops.** Git for Windows (which includes Git Bash and Perl), Vim and the kit were already on the program's approved software list. Tom checked each laptop's versions so nobody would discover a missing tool on day one.
- **Agree the scorecard.** The scorecard above was agreed by Thursday 1 October, signed by the program manager and the deputy program manager.
- **Receive drop 1.** The prime delivered drop 1 on Friday 2 October. Jim logged the delivery against CDRL A003 that afternoon and did nothing else with it until Monday. The receipt is the first story of Chapter 26, Sprint 1: A GitLab project the team can work in (p. 404).
- **Set the calendar.** Fifteen-minute stand-ups at 09:00 every working day; sprint planning on the first Monday of each sprint; review and retrospective on the last Friday. Holidays and leave were entered before planning, so sprint capacities would be honest.

Warning: Access requests are on the critical path

The longest lead time in the first two weeks was not learning git or writing a pipeline. It was waiting for accounts, a project and a runner from the enterprise administrators. Submit those requests before you do anything else, even before you are sure exactly what you need. You can adjust a project's settings later; you cannot get back the week you spent waiting.

3.10 What management needed to hear, and when

The program manager's request ("a story I can tell with numbers") shaped the reporting rhythm for the twelve weeks. Jim agreed to three kinds of report, each built from the scorecard:

Table 3.4: The twelve-week reporting rhythm

Report	When	Content	Built from
sprint review	every second Friday	what was finished, demonstrated live; scorecard for the sprint	GitLab, the journal, the pipeline
weekly note	every Friday in between	one paragraph: progress, risks, help needed	the journal's weekly quad
December PMR briefing	end of sprint 6	the twelve-week scorecard, before and after, the live demo	everything above; see Chapter 35, The briefing (p. 532)

No report would contain a number that had been typed by hand. That rule was hard to keep in sprint 1, when there was almost nothing to measure, and Jim kept it anyway: the first sprint review reported a scorecard with most cells marked "not yet measurable" and an explanation of when each would become measurable. The program manager later said that page was what convinced them the team was serious.

3.11 Exercises

Exercise 3.1: Read your own directive

Take a directive, policy or tasking memo you have received. Build the three-column table from "The directive, read as a specification": what it says, what it requires, what it leaves open. Who decides the open items?

Exercise 3.2: Write down the objections

Hold a thirty-minute meeting with your team about a change you have been asked to make. Write every objection on a whiteboard in the speaker's words, without arguing. Afterwards, for each, write where and how you will answer it, and with what evidence.

Exercise 3.3: Pick your measures

Choose five measures for your own first twelve weeks. For each, write how it will be computed without human judgment, what the target is, and what the old-way baseline was. Which of your measures would your most skeptical colleague reject, and why?

CHAPTER

4

The Vee, kept: the systems-engineering V in MBSE DevSecOps

From what you know →

You know

the Vee. Decompose on the left arm, from stakeholder needs to design; integrate, verify and validate on the right arm; tie each level on the left to its check on the right; hold a technical review at each gate.

→ Here it is called

still the Vee. The left arm is the model's layers, changed by merge requests. The right arm is the pipeline's checks plus the test events. The horizontal lines are `verify` relationships. The reviews are baselines: protected tags with a review package the pipeline generates.

→ What is new

the cadence. The program still walks one big Vee across its reviews, but every merge request now walks a small one in minutes, and every baseline walks a medium one. Nothing on the right arm waits for a milestone.

This chapter is CORE: it is the frame for everything that follows. Read it before Chapter 5, DevSecOps in systems-engineering words (p. 46), which translates the vocabulary; this chapter explains the shape.

4.1 Jim's whiteboard

The second time the team met about the directive (Chapter 3, The directive (p. 22)), Bob walked to the whiteboard, drew a large V and put the marker down.

> *Every program I've been on runs on this. Requirements down the left, test up the right, a review at every corner. The memo says "continuous delivery". Where does the V go? Do we throw it away?*
>
> — Bob, the week before sprint 1

It was the right question, and the honest answer took Jim most of the afternoon to work out: the V stays. What changes is how often you walk it, and who does the walking on the right-hand side. In the document era the team walked the right arm by hand, a few times a year, in the weeks before a review. In the new way a machine walks the lowest rungs of the right arm on every change, and people spend their time on the rungs only people can climb.

This chapter is that afternoon, written down.

4.2 The Vee in one paragraph

The systems-engineering Vee draws the life of a system as a letter V. Down the **left arm** the team decomposes: stakeholder needs become system requirements, requirements become an architecture, the architecture becomes a detailed design. At the **bottom** something gets built. Up the **right arm** the team integrates the pieces and checks them, level by level, against what the left arm said: the design against its specification, the integrated system against its requirements, the delivered system against the stakeholders' needs. The **horizontal lines** across the V are the promise that every statement on the left has a check on the right. The **gates** on the arms are the technical reviews – SRR, SDR or PDR, CDR, TRR, PRR, and the configuration audits – where a board looks at the evidence and decides whether the program moves on.

Nothing in that paragraph mentions documents, tools or a calendar. That is why the Vee survives the move.

4.3 What DevSecOps changes: three Vees, not one

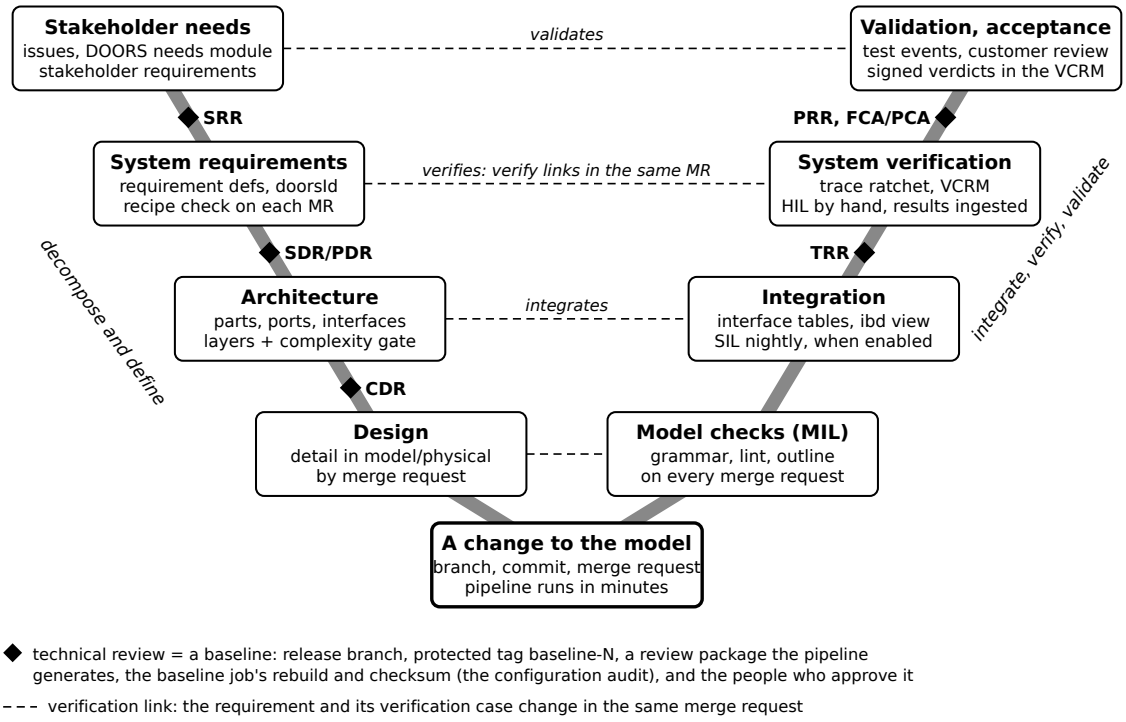


Figure 4.1: The Vee in MBSE DevSecOps: each element and the GitLab mechanism that carries it

In the document era there was one Vee per program, walked once, with reviews months apart. In MBSE DevSecOps the same shape repeats at three time scales:

- **The program's Vee**, across the technical reviews. It is the one you already know, and it still takes years. The reviews still happen and a board still decides; what arrives at each review is different (Section 4.7, The gates: technical reviews as baselines (p. 38)).
- **A baseline's Vee**, release by release. Between reviews the team cuts baselines (Tidewater's are `baseline-0`, `baseline-1` ...). Each one goes down the left arm (what changed), through the build (the release branch), and up the right arm (the baseline audit, the verification matrix, the people who approve it).
- **A merge request's Vee**, in minutes. One change goes down the left arm (Bob edits a requirement and its satisfy relationship), is built (the merge request), and comes up the right arm (grammar, lint, the trace check, the verification matrix, a code owner's review) before it lands. The lowest rungs of the right arm are walked hundreds of times between two reviews instead of once.

The middle of the figure is where the shift happens: *a change to the model* sits at the bottom of the V, and the model checks are its first rung up. Bob's worry was that continuous delivery would flatten the V into a line. It does the opposite: it makes the V small enough to walk every day.

4.4 The left arm: the model's layers

On the left arm the document era had a specification tree: the SRS, the SSDD, the ICDs, the design documents, each with its own revision letter. In the new way the same content lives in one model, arranged in layers that mirror the left arm (Chapter 16, The model repository: what goes where (p. 245)):

Table 4.1: The left arm of the Vee as folders in the model.

Left arm level	Where it lives in the model	How it changes
Stakeholder needs	`model/requirements/stakeholder/` (from the DOORS needs module)	DOORS export, imported by the vendor branch
System requirements	`model/requirements/system/`, each with its `doorsId`	merge request; the recipe check grades the wording (Chapter 7, Requirements a pipeline can check: the recipe (p. 73))
Architecture	`model/logical/` and the parts, ports and interfaces of `model/physical/`	merge request; the layers and complexity checks guard the structure
Design	detail inside `model/physical/`	merge request, by the package's code owner

The arm is still walked top to bottom: a stakeholder need is still decomposed into requirements before anyone designs. What changed is the unit of work. A decomposition step is a story, its pieces are tasks of under a week, and each lands as a merge request that a second engineer reviews (Chapter 13, The operating model (p. 196), Chapter 22, Reviewing a model change (p. 357)).

4.5 The right arm: checks at every level

The right arm is where DevSecOps changes the most. In the document era every rung was a manual event: a desk check, an integration test, a review of a verification matrix someone had assembled by hand. Now each rung has an owner, and the lowest rungs are machines.

- **Model checks, the first rung (MIL, model in the loop).** On every merge request the pipeline checks the grammar, resolves every name, checks the outline, the layers and the complexity, compares the trace with DOORS, regenerates the verification matrix and grades the requirement wording. Minutes, not weeks.
- **Integration.** Interfaces are checked when they change: the interface tables and the connection view are generated from the model (Chapter 11, Seeing the model: diagrams from text (p. 158)), and an interface change needs the systems lead's approval through CODEOWNERS. Where a program has a simulation, a nightly software-in-the-loop job runs it (SIL).
- **System verification.** The trace ratchet keeps every requirement's verification case in place, the verification matrix (VCRM) shows each requirement's latest verdict, and a lab bench runs hardware-in-the-loop tests (HIL) when a person starts them (Chapter 44, Verification as a closed loop (p. 613)).
- **Validation and acceptance.** Test events and the customer's review: people and real conditions. The pipeline's job here is evidence, not judgement: the signed verdicts and the matrix go into the baseline's release.

You can see the rungs the pipeline walks for each kind of event. Start the practice project from lesson 11 (a copy of the team's model repository on an offline GitLab), then ask the pipeline what it would run for a merge request:

```
. ../env.sh > /dev/null
perl "$KIT/bin/gitlab.pl" ci --list --event merge_request_event --target develop | grep -v
skipped
```

Ten jobs: the first rung of the right arm, on every change. Now the same question for a baseline tag:

```
perl "$KIT/bin/gitlab.pl" ci --list --event tag --tag baseline-1 | grep -v skipped
```

The same ten checks run again, and three more join them: `baseline` (the configuration audit), `publish-package` and `publish-release` (the review package). That is the baseline's Vee coming up the right arm. Finally, the nightly schedule with a simulation and a lab bench switched on:

```
perl "$KIT/bin/gitlab.pl" ci --list --event schedule --branch develop --var SIL_ENABLED=true
--var HIL_ENABLED=true | grep -v skipped
```

`sil-regression` runs by itself every night; `hil-test` waits, `manual`, for a person who has checked the bench. Tidewater has neither switched on, because the government team owns no simulation

and no lab; the template carries them for the day a program does (Chapter 44, Verification as a closed loop (p. 613)).

4.6 The horizontal lines: verify, in the same merge request

The horizontal lines are the Vee's most important promise and its most neglected one. In the document era the promise was a verification cross-reference matrix, assembled in a spreadsheet before each review, out of date the day after. Here a horizontal line is a `verify` relationship in the model: a verification case that names the requirement it verifies. Two rules keep the line from breaking:

1. A requirement and its verification case change in **the same merge request**. The reviewer sees both.
2. The **trace check** compares the model with DOORS on every merge request and fails when a change *adds* a gap: a requirement that loses its satisfy (an orphan) or its verify (unverified). Gaps that were already there are reported, not blocked: they are someone else's to fix, often the prime's (Chapter 20, The pipeline, job by job (p. 302)).

Break a horizontal line on purpose and watch the check catch it. On a scratch branch, delete one `verify` and run the trace job as a merge request into `develop` would:

```
git switch -q -c try-vee origin/develop
sed -i '/verify TW_SYS_003;/d' model/verification/Verification.sysml
git commit -qam "drop one verify"
CI_PIPELINE_SOURCE=merge_request_event CI_MERGE_REQUEST_TARGET_BRANCH_NAME=develop perl ci/job.pl
trace 2>&1 | tail -1
git switch -q develop && git branch -q -D try-vee
```

The nine unverified requirements that were already there did not block anyone. The tenth, the one this change created, does, and the message names it. In the document era that line would have broken silently, and the review in three months' time would have found it, if anyone looked.

4.7 The gates: technical reviews as baselines

The technical reviews keep their names, their boards and their decisions. Three things about them change.

- **What a review looks at.** A review used to look at a document set assembled for it. Now it looks at a **baseline**: a protected tag, `baseline-N`, on a commit of the model. Everything the board reads – the generated documents, the verification matrix, the trace report, the toolchain list – is produced from that tag by the pipeline, so it describes exactly what was tagged.
- **How the baseline is audited.** The configuration audit used to be a person comparing a document list with a library. Now the `baseline` job rebuilds the generated documents from the tagged model and compares a checksum with the one recorded in the tag. If the tag's documents were edited by hand, or the model changed without its documents, the audit fails.

- **Who decides.** Nobody new. The board still decides; in GitLab its decision is an approval on the release merge request, by the people CODEOWNERS names, with the change-board concurrence written as a comment (Chapter 31, Sprint 6: Behavior by hand, a table for the program, and baseline-0 (p. 495)).

Cut a baseline by hand and audit it. The tag records the checksum of the generated documents; the `baseline` job rebuilds them and checks:

```
h=$(git ls-tree -r HEAD -- docs | sha256sum | cut -c1-64)
git tag -a baseline-try -m "baseline-try" -m "sha256=$h"
CI_COMMIT_TAG=baseline-try perl ci/job.pl baseline | grep "rebuilds identically"
git tag -d baseline-try > /dev/null
```

Now tag a commit whose model changed without its generated documents, the way a hurried hand edit would:

```
git switch -q -c try-audit
sed -i 's/part def Thermometer\b/part def AirThermometer/' model/physical/stations/Stations.sysml
git commit -qam "rename a part without regenerating the documents"
h=$(git ls-tree -r HEAD -- docs | sha256sum | cut -c1-64)
git tag -a baseline-bad -m "baseline-bad" -m "sha256=$h"
CI_COMMIT_TAG=baseline-bad perl ci/job.pl baseline 2>&1 | tail -1 || echo "exit status $?"
git checkout -q -- . && git switch -q develop && git branch -q -D try-audit && git tag -d
baseline-bad > /dev/null
```

The audit refuses the baseline. A board looking at that tag would have been reading documents that no longer described the model; the machine noticed before the board met.

4.8 Change control between the reviews

Between two reviews the document era ran change control through engineering change proposals, a change board meeting every few weeks, and a revision letter on each document. In the new way:

- A **change proposal** is a merge request. Its description says what and why, cites the requirement or the prime's change request, and links its issue (Chapter 17, The DevSecOps tools that earn their keep (p. 255)).
- **Board approval** is a code owner's approval, enforced by GitLab: an interface change cannot merge without the systems lead, a requirement change without its owner (Chapter 22, Reviewing a model change (p. 357)).
- The **revision letter** is the commit, and the release of record is the baseline tag.
- Changes the government does not own – the prime's – arrive as drops, are assessed by element id and dispositioned by the board before anyone edits the model (Chapter 45, The prime's model changes: receive, assess, disposition (p. 619)).

The board meets less often for routine changes and more productively for the ones that matter, because the routine ones have been checked and approved in the merge request.

4.9 The Vee, then and now

Table 4.2: Each Vee element in the document era and in MBSE DevSecOps.

Vee element	Document era	MBSE DevSecOps
Stakeholder needs	a needs document, a DOORS module	the DOORS module, imported into `model/requirements/stakeholder/`
System requirements	the SRS, revised before each review	requirement definitions in the model, changed by merge request, wording graded on each change
Architecture and design	block diagrams in Cameo, SSDD, ICDs	parts, ports and interfaces as text; the diagrams and interface tables generated from them
Build	the design is drawn and documented	a change is made to the model on a branch
Unit and model checks	desk checks before a review	grammar, lint, outline, layers on every merge request
Integration	interface review meetings	generated interface tables; the systems lead approves interface changes; SIL where it exists
System verification	a VCRM assembled by hand before TRR	the trace ratchet and a VCRM regenerated on every change; HIL results ingested
Validation, acceptance	test events, customer acceptance	the same, with the evidence attached to the baseline's release
Horizontal lines	a spreadsheet cross-reference	`verify` relationships, changed in the same merge request as the requirement
Technical review	a document set assembled for the review	a baseline tag and the review package the pipeline generates from it
Configuration audit	a person compares lists	the baseline job rebuilds and checks the checksum
Change control	ECP, CCB meeting, revision letters	merge request, code-owner approval, commit and tag

4.10 What stays the same, and what changes

What stays the same:

- The shape: decompose, build, integrate and check, and the promise that every statement has a check.
- The reviews, their names, their entry and exit criteria, and the board that decides.
- The roles: who owns the requirements, who owns each part of the design, who approves.
- Validation by people, in real conditions. No pipeline validates a system.

What changes:

- **Cadence.** The right arm's lowest rungs run on every change, not once before a review.
- **Evidence.** What a review reads is generated from a tag, so it cannot drift from the model.
- **Who walks which rung.** Machines walk the repetitive checks; people spend their time on design, judgement and the tests only people can run.

- **When a problem is found.** A broken horizontal line is found the day it breaks, by the person who broke it.

4.11 Where Tidewater sits on the Vee

Tidewater is not a new program. Its buoys and shore stations are fielded, and the program is in production and sustainment, long past CDR: on the program's Vee it sits near the top of the right arm, with the prime maintaining the design. That shapes the team's whole twelve weeks:

- **The port is a small Vee of its own, on the model.** On the left, the team planned the port: what the v2 model must hold, in what layers, with what conventions (Chapter 26, Sprint 1: A GitLab project the team can work in (p. 404), Chapter 27, Sprint 2: Drop 1 on develop, every change gated (p. 424)). At the bottom, the conversion and the hand work that closed the TODO lines. On the right, the checks that prove the port is complete and correct (Chapter 34, Proving the port is complete and correct (p. 527)), the interim baseline, and the management briefing, which is the port's validation: did it give the program what it needed (Chapter 31, Sprint 6: Behavior by hand, a table for the program, and baseline-0 (p. 495))?
- **Each prime drop is a change Vee.** The prime's change requests come down the left arm (what changed, by element id), the impact report finds where the change lands in the government's model, the vendor merge builds it, and the checks and the board's disposition take it up the right arm (Chapter 45, The prime's model changes: receive, assess, disposition (p. 619), Chapter 43, Maintenance through the life cycle (p. 602)).
- **The government's role is mostly the right arm.** The prime designs; the government verifies, accepts and keeps the authoritative view of verification. That is why the trace ratchet and the verification matrix matter more to Jim's team than any design tool.

A team at the start of a program sits at the top of the left arm instead. Everything in this chapter still applies; the stories are about decomposition rather than conversion.

4.12 Variants you may meet

- **Incremental Vee.** The system is delivered in increments, each walking its own Vee, often sharing the upper left arm. In GitLab each increment is a release line: a release branch per delivered configuration, fixes landing on `develop` first (Chapter 43, Maintenance through the life cycle (p. 602)).
- **Iterative or spiral.** The upper left arm is walked several times as the requirements mature. Each pass is a set of sprints ending in a baseline; the baselines are the spiral's turns.
- **Dual Vee.** An architecture Vee for the system and an entity Vee for each element, walked by different teams. In GitLab the entity Vees are separate projects (one per subsystem) feeding the system project through tagged, published interface packages; this is the federation described in Chapter 16, The model repository: what goes where (p. 245).

4.13 Digital thread and digital twin

Two more terms travel with MBSE, and management will use both. This section is CORE for understanding and LATER for the twelve weeks: the team builds the thread from the first sprint,

and nobody builds a twin in the first year.

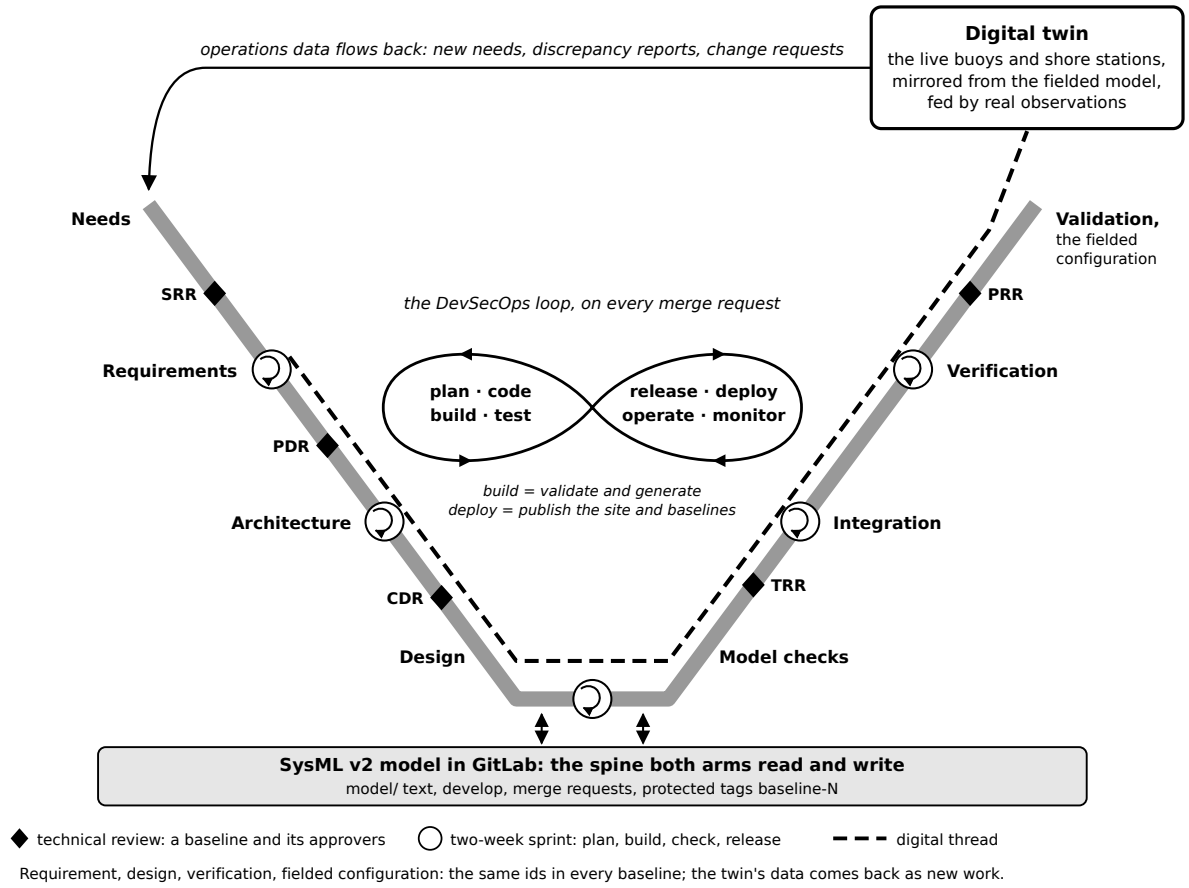


Figure 4.2: One picture: the Vee and its reviews, two-week sprints, the DevSecOps loop, the model in GitLab as the spine, the digital thread and the digital twin

Definitions

digital thread
: the unbroken chain of identifiers that connects a need to the requirements derived from it, to the design elements that satisfy them, to the verification cases and their verdicts, to the configuration that was fielded, so that anyone can follow any element forward or backward through the life cycle.

digital twin
: a live digital counterpart of a fielded system, built from the model and kept in step with the real thing by its operational data, used to predict, diagnose and decide.

In plain words: the **thread** is a promise about the *records* – every element has one id that every later record uses, and every baseline keeps the ids. The **twin** is a *working copy of the system* that runs beside the real one.

The figure puts them together. The model in GitLab is the spine; both arms of the Vee read and write it. The thread runs through it from requirement to design to verification to the fielded

configuration, kept in every baseline. The twin sits beyond the top of the right arm, built from the fielded configuration and fed by real data. Its findings come back along the top to the left arm as new needs, discrepancy reports and change requests, which start the next small Vee.

What Tidewater's twin would be. A model of the live observation network: each buoy and shore station from the v2 model's part tree, with its power system and sensors, fed by the real observation and health streams the stations already send. It could predict when a buoy's battery will not last the winter, show which stations a storm will take offline, or test a new transmission schedule before the prime changes it.

What the kit builds: the thread.

- One stable id per element, carried from the prime's Cameo model through every drop (the prime agreed to stable ids in week zero) and into the v2 text, with a map from each v1 id to its v2 element (Chapter 45, The prime's model changes: receive, assess, disposition (p. 619)).
- `doorsId` on every requirement, so DOORS and the model meet on one key.
- `satisfy` and `verify` relationships, checked on every merge request by the trace ratchet.
- A verification matrix that ties each requirement to its cases and their latest verdicts.
- Baselines that freeze all of it under one tag, audited by checksum.

What the kit does not build: the twin. A twin needs live data feeds from the fielded system, a simulation that runs the model's behavior, and a synchronization layer that keeps the copy in step with the real configuration. None of that is in this kit, and none of it is a twelve-week job.

What a team needs before a twin is worth building.

1. A thread that holds: stable ids, baselines that rebuild, a trace with no unexplained gaps. A twin built on a broken thread mirrors the wrong system.
2. Behavior and parameters in the model, not only structure: states, flows, the energy balance with real values (Chapter 8, SysML v2 by example: a workbook (p. 92)).
3. Access to operational data, approved for the purpose, with its own data rights and markings.
4. A question worth the cost: one decision the twin would change, with a number attached.

Tidewater's team has the first by sprint 6 and part of the second; the third and fourth are a program decision. When management asks "are we building a digital twin?", the honest answer is: "We are building the thread a twin would need, and here is what it would take after that."

4.14 Pitfalls

- **Calling sprints "phases".** A sprint is a timebox, not a stage of the Vee. A sprint can hold work from both arms: one story decomposes a requirement while another closes a verification gap. Teams that map sprints to Vee phases rebuild the waterfall at a two-week scale.
- **Skipping the right arm because the pipeline is green.** The pipeline walks the lowest rungs. A green pipeline says the model is well formed and its trace is intact; it does not say the system works. Integration, test events and validation still need people, plans and dates (Chapter 44, Verification as a closed loop (p. 613)).

- **Reviews with nothing to review.** If the review package is not generated from the tag, the board reads something the model has already moved past. Generate it, tag it, audit it.
- **Treating the twin as the goal.** A twin is the last step of a long chain. Start with the thread.

Exercise 4.1: Map your program's Vee

Draw your program's Vee and mark where it is today. For each rung of the right arm, write who walks it now (a machine, a person, nobody) and how often. Circle the rung where a machine would save the most time.

Exercise 4.2: Find the three Vees

In the practice project, name one example of each Vee: a merge request's (which change, which checks), a baseline's (which tag, which jobs), the program's (which review would read that baseline). Use `gitlab.pl ci --list` with the right `--event` to show the checks.

Exercise 4.3: Break and mend a horizontal line

On a scratch branch, remove one `satisfy` relationship instead of a `verify`, run the trace job as a merge request, and read the message. Then explain to a program manager, in two sentences, why the check blocked this change but not the nine unverified requirements already in the model.

Exercise 4.4: The thread before the twin

Using the four prerequisites in Section 4.13, Digital thread and digital twin (p. 41), write five lines for your own program: which you have, which you lack, and one decision a twin would change.

4.15 You can now

- Show where each element of the systems-engineering Vee lives in an MBSE DevSecOps project, and say what changed and what did not.
- Explain the three Vees – the merge request's, the baseline's and the program's – and show each with the pipeline.
- Demonstrate that a broken horizontal line is caught on the day it breaks, and that a baseline's documents are audited before a board reads them.
- Tell a digital thread from a digital twin, and say what has to exist before a twin is worth building.

4.16 Applying this to your project

Copy the mapping in Section 4.9, The Vee, then and now (p. 40) and fill in your own program's names for each element: your review names, your requirement modules, your test events. Keep the shape and the reviews; change the cadence of the right arm first, starting with the rung where checks are most repetitive. Decide locally which reviews read baselines from the first day and which keep their current package until a pilot proves the generated one. Build the thread

before anyone mentions a twin: stable ids from your prime, `doorsId` on every requirement, the trace check on every merge request.

CHAPTER

5

DevSecOps in systems-engineering words

From what you know →

You know

configuration management, peer review, the CCB, configuration audits, baselines and CDRL deliveries.

→ Here it is called

version control, merge requests, code-owner approval, the pipeline, protected tags and the vendor branch.

→ What is new

all of it happens on every change, automatically, in minutes, instead of at milestones.

Note: What you have, what you need

What you have: a working knowledge of configuration management, change control, reviews, audits and verification, built over a career.

What you need from this chapter: the realization that DevSecOps is mostly those same disciplines, made cheap by keeping everything as text and letting machines do the repetitive checking. You will finish with a translation table you can keep beside you for the rest of the book.

5.1 Tom's Saturday

Tom spent the Saturday after the team meeting reading. The enterprise DevSecOps office's recorded webinar was ninety minutes long, and Tom watched all of it with a notepad, writing down every word that sounded like jargon. The list filled two pages: *pipeline, CI/CD, merge request, branch, commit, shift left, artifact, runner, YAML, SAST, secret detection, protected branch, code owner, tag, release, infrastructure as code, GitOps, trunk, feature flag, observability.*

On Sunday Tom tried something that turned out to be the key to the whole twelve weeks. Next to each word, Tom wrote the configuration management or systems engineering term that seemed closest. Some were easy: a *tag* was obviously a baseline label. Some took a while: a *pipeline*

turned out to be something like a configuration audit, run automatically every time anything changed. Some had no equivalent at all, and those went on a separate list of things to ask about. By Sunday evening the second list had six words on it and the first had thirty.

On Monday Tom showed the list to Jim and Bob. Bob read it twice and said, "So it's CM. It's just CM that runs itself." That was not quite right, but it was close enough to start, and it was the first time anyone on the team had said something about DevSecOps that was not a complaint.

This chapter is Tom's list, cleaned up and explained.

5.2 What the word means

DevSecOps is a compound of three words from the software world:

- **Dev**, development: making changes to the thing being built. For this team, the thing being built is the system model, so "development" means changing the model.
- **Sec**, security: protecting the thing and the process that builds it. For a model, this means controlling who can change what, keeping secrets (passwords, tokens) out of files, checking that what the contractor delivered is what arrived, and recording who did what.
- **Ops**, operations: running the thing and keeping it running. For a model, this means publishing it so people can use it, keeping it current with each contractor delivery, and keeping the tools that check it working.

The word was coined because software organizations used to separate those three jobs into different teams with different goals, and the hand-offs between them were slow and error-prone. Developers threw code "over the wall" to operations; security reviewed it at the end, if at all. DevSecOps means doing all three continuously, as one activity, with the checks built into the process instead of bolted on at the end.

Systems engineers will recognize the problem immediately, because the document era had the same walls. Engineers wrote documents; CM controlled them; reviews happened at milestones; security looked at the system at the end. The fix is the same too: build the checks into every change, so nothing has to wait for a milestone to be found.

Definitions: The one-sentence version

DevSecOps for a system model
every change to the model is made as text, recorded, reviewed by a person, checked by a machine and published, every time, with security controls applied throughout instead of at the end.

5.3 Eight ideas, in systems engineering terms

The DevSecOps literature is large and full of slogans. For a team maintaining a system model, eight ideas carry almost all the value. Each one is explained below in systems engineering terms, with what it costs and what it saves.

5.3.1 1. Everything is text, and the text is under version control

In the document era, the authoritative record was a set of Word files, spreadsheets and a Cameo project. None of them could be compared reliably, because they were binary files whose contents only their own tools could read. In DevSecOps, the authoritative record is plain text: the SysML v2 model, the pipeline configuration, the issue templates, even the project settings. Plain text can be read by any editor (Vim, in this team's case), compared line by line by a program, and stored in a *version control system* that records every change.

The version control system this book uses is **git**. Think of git as a configuration management database that records, for every change: exactly which lines changed, who changed them, when, and the reason they gave. Unlike the CM database Tom ran for a decade, git records the change itself, not a description of it, and it does so automatically every time someone saves their work into it. Chapter 12, Git from zero, for systems engineers (p. 171) teaches git from zero.

What it costs: the model has to be text. That is why the memo says SysML v2 *textual notation*, and why Chapter 6, SysML v2 for SysML v1 modelers (p. 57) matters.

What it saves: every comparison question ("what changed since the last baseline?", "who changed this requirement and why?", "what did the model look like on 1 November?") becomes a command that answers in seconds.

5.3.2 2. Small batches

The document era batched changes: a month of change requests went to one CCB, a quarter of approved changes went into one revision. Batching was rational when each revision cost weeks of editing and distribution. With text under version control, a change costs almost nothing to record and distribute, so batching only adds waiting.

DevSecOps works in small batches: a change is proposed, reviewed and incorporated on its own, often the same day. The team's size rules in Chapter 13, The operating model (p. 196) (a task is less than a week of work) exist to keep batches small.

What it costs: discipline about splitting work into small pieces, and the habit of reviewing small things often instead of large things rarely.

What it saves: waiting, which Chapter 1, Thirty years of documents (p. 2) showed was most of the 23 days of a requirement change. Small changes are also easier to review well, because a reviewer can actually read all of a ten-line change.

5.3.3 3. Every change is reviewed and approved, by a named person

This is the CCB, made continuous. In GitLab, a proposed change is a **merge request**: a request to merge (combine) a set of changes into the shared model. The merge request shows the exact lines changed, carries a description of why, and cannot be merged until the people responsible for that part of the model approve it. Those people are named in a file called `CODEOWNERS`, so the approval rule is itself under configuration control.

What it costs: every change needs a second person's attention. On a three-person team this matters, and Chapter 13, The operating model (p. 196) sets the rules so nobody approves their own work and nobody waits long.

What it saves: the monthly CCB queue for routine changes. The board still decides what needs a board (interface changes with the prime, requirement changes in DOORS); it no longer has to

see a corrected typo.

5.3.4 4. Machines check what machines can check, on every change

Before a merge request can be merged, a **pipeline** runs: a fixed list of automated checks, defined in a text file in the repository, executed by a server every time the model changes. If any required check fails, the merge is blocked.

This is the idea that converted Tom. A pipeline is a configuration audit that runs on every change instead of once a year. It asks questions like: does every file follow the SysML v2 grammar? Does every assembly carry its level tag? Is every requirement satisfied by something and verified by something? Does the requirement text in the model match the text in DOORS? Are the generated documents up to date with the model? Has anyone pasted a password into a file? Each question is a short program, written once and run forever. Chapter 20, The pipeline, job by job (p. 302) walks through every check.

The software world calls this "shifting left": moving checks earlier in the process (to the left on a timeline), where problems are cheap to fix. A grammar error found by the pipeline ten minutes after Bob typed it costs Bob ten minutes. The same error found at a design review costs a review comment, an action item and a re-review.

What it costs: someone has to set up the pipeline (Tom, in this story) and keep it working.

What it saves: almost all of the manual checking that was done before reviews, and all of the problems that manual checking missed.

5.3.5 5. Security is built in, not added at the end

For a model, "security" mostly means controlling access and protecting the integrity of the record:

- **who can change what**: members, roles and *protected branches*, which make it impossible (not just against the rules) to change the integrated model without a merge request;
- **integrity of what arrives**: every contractor drop carries a checksum, and the intake process checks it before anything else happens;
- **no secrets in files**: an automated scan rejects passwords and access tokens pasted into the repository;
- **a complete audit record**: every change, approval and pipeline result is kept, and cannot be quietly edited;
- **least privilege for tools**: the server that runs the pipeline has only the access it needs.

In the document era, security for engineering data meant marking the documents correctly and storing them in the right place. Those obligations remain (and Chapter 15, Setting up GitLab (p. 226) covers marking), but DevSecOps adds controls on the process itself, enforced by the platform rather than by procedure.

5.3.6 6. The model is published continuously

Every time the integrated model changes, the pipeline regenerates the human-readable products (documents, diagrams, trace reports) and publishes them as a website. Nobody pastes screen-

shots; nobody wonders which picture is current. This is what software people call *continuous delivery*, and for a model it means the review package is always ready.

This idea answers the strongest document-era virtue: *everybody could read the documents.* With a published site, everybody can still read them, in a browser, without a license, and what they read is generated from the current model.

5.3.7 7. Measure, and use the measures

Because every change, review and check is recorded by machines, the process measures itself. Lead time, review turnaround, pipeline pass rate and the number of problems found are byproducts, not extra work. The team's scorecard in Chapter 3, The directive (p. 22) is built entirely from such byproducts.

5.3.8 8. Improve in small steps, without blame

Each sprint ends with a retrospective: what worked, what did not, what to change next sprint. When something breaks (a pipeline fails, a merge goes wrong), the question is "what in the process let this happen?", not "who did this?". Systems engineers know this as lessons learned; the difference is that it happens every two weeks instead of at the end of a program, when nobody can use the lessons.

5.4 The translation table

Here is Tom's list, cleaned up. Keep it nearby. The glossary in Appendix K, Glossary (p. 796) has the complete version, including SysML v2 terms.

Table 5.1: Systems engineering to DevSecOps, the translation table

Systems engineering / CM term	DevSecOps / GitLab term	What is the same	What is different
configuration item	repository (or a folder in one)	a controlled body of content	everything in it is versioned automatically
configuration status accounting	git history; GitLab activity	who changed what, when, why	recorded automatically, for every line
baseline (revision C)	tag (`baseline-1`)	a fixed, named state	exact and reproducible; created in seconds
change request, ECP	merge request	a proposed change, reviewed before incorporation	contains the change itself, not a description
redline	diff	what changed	precise, line by line, generated
CCB approval	code owner approval; approval rule	named authority approves	per change, per area, recorded
CCB chair's rules	protected branch settings	nobody bypasses the process	enforced by the platform
configuration audit (FCA/PCA)	pipeline	checks the configuration against rules	runs on every change
verification procedure	pipeline job	a defined check with pass/fail	automated, repeatable
test report	job log; test report in the merge request	evidence of the check	kept with the change
action item	issue	a tracked piece of work with an owner	linked to the changes that close it

Systems engineering / CM term	DevSecOps / GitLab term	What is the same	What is different
IMS milestone	milestone	a date by which work is due	holds the issues due then
work package	epic, story, task (labels and issues)	a decomposition of work	see Chapter 13, The operating model (p. 196)
CDRL delivery	prime drop on the vendor branch	the contractor's deliverable	converted and merged like any change
deficiency report to the contractor	discrepancy report issue (owner::prime)	a problem the contractor must fix	tracked to the drop that fixes it
design review package	published model site (GitLab Pages)	the current design for reviewers	generated, always current
document revision letter	release (with release notes)	a published, approved version	built from a tag
CM library, shared drive	GitLab project	where controlled items live	access controlled, audited
marking and handling	protected project, members, labels, banners	handling rules	enforced by roles
lessons learned	retrospective	improve the process	every sprint

And the six words that had no equivalent on Tom's Sunday list, with what the team decided they meant:

- **branch**: a separate line of work, where changes can be made without affecting the integrated model until they are reviewed and merged. Closest equivalent: a working copy of a document with tracked changes, except that any number of people can have one at once and git combines them afterwards. Chapter 12, Git from zero, for systems engineers (p. 171).
- **commit**: one recorded change, with a message saying why. Closest equivalent: one entry in the change history page of a document, except that it holds the change itself.
- **runner**: the computer that executes the pipeline's checks. Closest equivalent: the person who used to run the checks before a review, except that it is a machine and never gets tired. Chapter 17, The DevSecOps tools that earn their keep (p. 255).
- **YAML**: the text format the pipeline is written in. It is a way of writing lists and key-value pairs in plain text. Chapter 20, The pipeline, job by job (p. 302) shows the whole file.
- **artifact**: a file produced by a pipeline job and kept with its results (a report, a generated document). Closest equivalent: an attachment to a test report.
- **merge conflict**: what happens when two changes touch the same lines and git cannot combine them automatically. Closest equivalent: two people redlining the same paragraph differently. A person has to decide. Chapter 10, Vim for models (p. 143) shows how.

Case study: What DevSecOps is not

DevSecOps is not "agile" by another name, although it is usually practiced together with short iterations. It is not about writing software; this team writes none. It is not a product you buy: GitLab is a platform that makes the practices easy, but the practices work with other platforms. And it is not about moving fast and breaking things. For a system model, the point is to move *safely*, with every change checked and approved, at a pace set by the work instead of by the meeting calendar.

5.5 One change, end to end

To make the ideas concrete, here is one small change of the kind the team made in Chapter 28, Sprint 3: the first red X, DOORS in the model, and a wait nobody named (p. 440), followed from start to finish. It is shown in outline here, with an illustrative issue number; Part III shows the real issues, commands and output.

The converter left a TODO line in the `Network` package where the v1 model had a custom "level" stereotype on the top block:

```
part def ObservationNetwork {
    // TODO stereotype Tidewater_Profile:L1Network on this element (xmi:id ...) not converted
    doc /* Every station, the data center and the field crew that keeps them running. */
```

The government's convention (set up in Chapter 27, Sprint 2: Drop 1 on develop, every change gated (p. 424)) is to write the level as a metadata annotation, `@L1Network;`.

1. **The work is an issue.** A task issue in GitLab says "Close the level-tag TODOs in Network", is linked to its story, assigned to Bob, and estimated at one day. (Action item, with an owner and an estimate.)
2. **Bob starts a branch.** One command creates a separate line of work named after the issue. (A working copy.)
3. **Bob edits the file in Vim**, replacing the TODO line with `@L1Network;` and adding the import of the conventions package at the top. (The edit.)
4. **Bob checks locally**, running the same grammar and lint checks the pipeline will run, from inside Vim. (Self-check before submitting.)
5. **Bob commits** with a message: "Close the L1Network level TODO in Network (#14)". (Change history entry.)
6. **Bob pushes** the branch to GitLab and opens a **merge request** whose description says what changed, why, and "Closes #14". (Change request.)
7. **The pipeline runs** on the merge request: grammar, lint, outline, TODO count, trace, docs, secrets. All pass. The TODO count shows one fewer than the integrated model. (Configuration audit.)
8. **Jim reviews**: reads the two-line diff and the pipeline results, and approves. Jim is the code owner for the `model/physical/network/` folder. (CCB approval, by the responsible person.)
9. **Bob merges.** The change joins the integrated model on the `develop` branch; the issue closes automatically. (Incorporation.)
10. **The pipeline runs again** on `develop`, regenerates the docs and diagrams, and publishes the model site. (Distribution of the new revision.)

Elapsed time for a change like this is typically an hour or two, most of it waiting for the reviewer to finish a meeting; engineering time is about fifteen minutes. Chapter 28, Sprint 3: the first red X, DOORS in the model, and a wait nobody named (p. 440) has the real timestamps. Every step left a record: the issue, the branch, the commit, the merge request, the pipeline log, the approval, the merge. If the deputy program manager asks in six months who changed the Network package's level tag, and why, and who approved it, the answer takes one click.

Compare that with the 23 working days in Chapter 1, Thirty years of documents (p. 2). The engineering content is the same. What vanished is the waiting and the copying.

Objection: "Fifteen minutes for a one-line change is still slower than just typing it into Cameo."

Bob, on reading an early draft of this example.

Answer

True for the typing. But the Cameo edit had no review, no check, no record anyone could find, and no published result. The comparison that matters is with the *controlled* change: the change request, the CCB, the revision. Against that, fifteen minutes and two hours elapsed is a different world. And for a truly trivial change, Chapter 51, Pitfalls and anti-patterns (p. 684) discusses batching several into one merge request.

5.6 Security for a model team, in practice

"Sec" was the part of DevSecOps that worried Tom most, because Tom had spent years on the receiving end of security audits and knew how much paperwork they produced. It helped to make the threats concrete. For a team maintaining a system model, the realistic threats are not exotic. Here are the ones the team listed in Chapter 27, Sprint 2: Drop 1 on develop, every change gated (p. 424), with the control for each.

Table 5.2: Threats to a model repository and the controls the team used

What could go wrong	Example	Control in the new way
an unauthorized change to the model	someone edits the integrated model directly, without review	protected branches: nobody can push to `main` or `develop`; changes arrive only through approved merge requests
a change approved by the wrong person	the author approves their own change	approval rules and CODEOWNERS; GitLab blocks self-approval
a tampered or corrupted delivery	a drop altered in transit or storage	SHA-256 checksums in the prime's manifest, checked before import; drops stored in the package registry
a secret leaked into the repository	a password or access token pasted into a file or commit message	secret detection in the pipeline; tokens kept in masked CI/CD variables, never in files
the history rewritten to hide a change	someone force-pushes over the record	force push disabled on protected branches; protected tags cannot be moved or deleted
access outlives need	a departed engineer keeps access	quarterly access review of project members (Chapter 42, Administration (p. 586))
the pipeline's machine is compromised	the runner is used to reach other systems	a dedicated runner with only the access it needs, patched on a schedule (Chapter 42, Administration (p. 586))

What could go wrong	Example	Control in the new way
marked information in the wrong place	content above the repository's approved level	marking rules, banners, and the project kept on approved infrastructure (Chapter 15, Setting up GitLab (p. 226))

Most of these controls are *settings* rather than procedures. That is the important difference from the document era. A procedure says "do not change the baseline without approval," and depends on everyone following it. A protected branch makes the unapproved change impossible. Tom, who had spent years writing CM procedures that people ignored, found this the most persuasive thing in DevSecOps: "I can stop writing rules and start setting them."

Note: Security reviews still happen

Settings do not replace the ISSM. The ISSM still has to approve the toolchain, the data flows and the controls, and the team still owes the ISSM documentation. What changes is that the documentation describes settings that can be inspected, rather than procedures that have to be trusted. Chapter 28, Sprint 3: the first red X, DOORS in the model, and a wait nobody named (p. 440) and Chapter 30, Sprint 5: Grading the requirements, and a trace that defends itself (p. 475) show the ISSM review.

5.7 Reading a pipeline log, before you ever see one

The first time a pipeline fails on your change, you will be looking at a log: the text output of the job that failed. Logs look intimidating, but they have a simple structure, and knowing it in advance removes most of the fear. Here is the shape of a failed grammar job, of the kind Bob saw in Chapter 28, Sprint 3: the first red X, DOORS in the model, and a wait nobody named (p. 440). The error line is the grammar checker's real output for a level tag written without its closing semicolon; the runner lines around it are abbreviated:

```
Running with gitlab-runner (shell executor) on tidewater-runner-1
Fetching changes...
Checking out 3f1a9c2e as detached HEAD (ref is refs/merge-requests/12/head)...
Executing "step_script" stage of the job script
$ perl ci/job.pl grammar
$ perl "$KIT/tools/sysml/sysml.pl" check model/analysis/Analysis.sysml ...
model/verification/Verification.sysml
model/physical/network/Network.sysml:5:9: error: unexpected 'doc'; expected '.', ':', '::', ';',
'about', 'typed', '{'
sysml: 17 file(s), 1 error(s)
$ perl "$KIT/tools/gitlab/ci-reports.pl" grammar --junit grammar.xml --codequality
grammar-cq.json grammar.txt ...
ci-reports grammar: 17 check(s), 1 failed, 1 finding(s); grammar_files=17 grammar_errors=1
ERROR: Job failed: exit status 1
```

Read it from the bottom up:

1. **The last line** says the job failed and that the command returned a non-zero exit status. Every command reports success (zero) or failure (anything else); the pipeline stops a job at the first failing command.
2. **The lines above it** are the output of the commands the job ran. The last of them (`ci-reports.pl`) only turns the result into the report GitLab shows on the merge request; the check itself is the `sysml.pl` line, and its output is where the problem is. Here: one error, in `model/physical/network/Network.sysml`, line 5, column 9. The checker met the word `doc` where it expected the semicolon that should have ended the line above (`@L1Network` written without its `;`). Errors are reported where the checker noticed them, which is often just after the real mistake.
3. **The lines starting with `$`** are the commands the job ran. The first, `perl ci/job.pl grammar`, is the whole job: the one line in the pipeline file. Run exactly that in your clone and you get the same check on your own laptop, which is how you fix the problem without guessing (Chapter 20, The pipeline, job by job (p. 302), Chapter 10, Vim for models (p. 143)).
4. **The lines at the top** say which machine ran the job and which version of the model it checked. You will rarely need them, except when a job works on your laptop and fails on the runner.

That is all. A pipeline log is a record of a check that someone wrote down so the machine could run it. The skill of reading one is the skill of reading a test report, which every systems engineer already has.

5.8 What changes for each person

The translation table maps practices; it is also worth mapping people. On a three-person team, each person's job changes in a different way.

Jim, the MBSE lead, stops assembling status by hand and starts reading it from the system. Jim's main new activities are reviewing and approving merge requests in the areas Jim owns, keeping the backlog of stories in order, deciding what goes to the prime as a discrepancy, and presenting the scorecard. Jim's main lost activity is the quarterly briefing build.

Bob, the MBSE engineer, stops drawing and starts writing. Bob's main new activities are editing the model as text, closing TODOs, writing what the converter could not, and resolving merge conflicts when a new drop touches something the government changed. Bob's lost activity is the side-by-side diagram comparison.

Tom, the CM engineer turned DevSecOps engineer, stops maintaining the CM database by hand and starts maintaining the system that maintains it. Tom's main new activities are the GitLab project settings, the pipeline, the runner, the drop intake and the baselines. Tom's lost activity is the change log, which git now keeps.

All three still do systems engineering. None of them writes software.

5.9 Exercises

Exercise 5.1: Make your own list

Do what Tom did. Write down every DevSecOps term you have heard on your program. Next to each, write the closest systems engineering or CM term. Mark the ones with no equivalent. Compare your list with the translation table.

Exercise 5.2: Which idea first?

Of the eight ideas in this chapter, which one would save your team the most time in the next month? Which one would meet the most resistance? Are they the same idea?

Exercise 5.3: Follow a change

Take a real change your team made recently. Write it out as the ten steps of "One change, end to end." Which steps existed in your current process? Which were missing? Where did the time go?

CHAPTER

6

SysML v2 for SysML v1 modelers

From what you know →

You know

SysML v1 in Cameo: blocks, ports, activities and requirement diagrams.

→ Here it is called

SysML v2 textual notation: part defs, port defs, action defs and requirement defs.

→ What is new

the model is plain text you can read, compare and review line by line, and the converter shows exactly what it carried and what it left as a TODO.

Note: What you have, what you need

What you have: you know SysML v1: blocks, parts, ports, connectors, requirements, satisfy and verify, activities and state machines. You learned them as boxes and lines on diagrams.

What you need from this chapter: to recognize the same things written as text in SysML v2, to read the converter's output with confidence, and to understand exactly what the converter leaves for a person to do. You will type a few commands and check a few files with the grammar checker.

6.1 Bob reads the converter output

On Monday 5 October, the first day of sprint 1, Bob ran the converter on drop 1 for the first time. Bob had been dreading it. The Cameo model had taken two years to build, and Bob expected the conversion to be either useless or incomprehensible.

To follow along you need drop 1. Section 9.5, The commands you need (p. 130) shows how to set up a practice folder; in short:

```
mkdir -p ~/w2s
cd ~/w2s
```

```
perl "$KIT/sim/tidewater-v1.pl" --drop 1 drop1
```

Then Bob's command, from `~/w2s`:

```
perl "$KIT/bin/xmi2sysml.pl" --report --out drop1-v2 drop1/tidewater.xmi
```

The command ran in under a second and wrote a folder of text files, one per top-level package of the Cameo model. Bob opened the stations file in Vim:

```
vim drop1-v2/stations/Stations.sysml
```

and scrolled to the power system. Bob looked at it for a long time, then called Jim over.

```
    part def PowerSystem {
        // TODO stereotype Tidewater_Profile:L4Subsystem on this element (xmi:id ...) not
        converted
        doc /* Solar generation, storage and regulation for a buoy. */
        part solar : SolarPanel;
        part battery : Battery;
        part regulator : ChargeRegulator;
        attribute averageLoad : Power;
        port supplyOut : PowerIF;
        connection charge connect solar.'out' to regulator.'in';
        connection store connect regulator.toBattery to battery.terminal;
        part balance : EnergyBalance;
        bind balance.generation = solar.ratedPower;
        bind balance.storage = battery.capacity;
        bind balance.load = averageLoad;
    }
```

> *That's the IBD. That's the whole IBD, the one I drew four times. It's twelve lines. I can read it. I can read it faster than I could read the diagram.*
>
> — Bob, sprint 1, day 1

It is worth being careful about what Bob was and was not saying. Bob was not saying text is always better than diagrams. Diagrams are better for seeing shape, and the kit can draw them from the text (see Chapter 10, Vim for models (p. 143) and Chapter 17, The DevSecOps tools that earn their keep (p. 255)). Bob was saying that *as a record*, something to review, compare, check and keep, twelve lines of text are better than a diagram whose meaning is spread across a layout, a property panel and a model tree. That observation, on day one, was the first crack in Bob's skepticism.

This chapter teaches you to read what Bob read.

6.2 Why text, and why now

SysML v1 was defined as an extension of UML, and like UML it had no standard text notation. The model was whatever the tool stored; the exchange format, XMI, was XML written for machines (you saw it in Chapter 2, The Cameo years (p. 10)). Every tool vendor stored models differently, and moving a model between tools was unreliable.

SysML v2 is a new language, standardized by the Object Management Group, with two notations defined together: a graphical notation (diagrams) and a **textual notation** (the text you are about to read). Both describe the same model, and the textual notation is complete: anything you can model, you can write as text. The language also comes with a standard API for tools to exchange models.

For this team, the textual notation is what makes DevSecOps possible. Text can be:

- **compared**: line by line, by git, showing exactly what changed between two versions (Cameo could not do this usefully, see Chapter 2, The Cameo years (p. 10));
- **merged**: two people's changes to different lines combined automatically, and conflicts shown precisely when they touch the same line;
- **checked**: by small programs that read the text (the grammar checker, the linter, the trace report);
- **edited in any editor**: including Vim, which the whole team already uses;
- **reviewed**: in a merge request, as a diff, by anyone with a browser.

Note: The grammar checker

The kit includes a SysML v2 syntax checker built from the official SysML v2 grammar files. Every v2 snippet in this book has been checked with it:

```
perl "$KIT/tools/sysml/sysml.pl" check drop1-v2/stations/Stations.sysml
```

A file that follows the grammar prints `sysml: 1 file(s), 0 error(s)`. A file that does not prints each error with its line number, in a format Vim can jump to (Chapter 10, Vim for models (p. 143)).

6.3 Five ideas that cover most of SysML v2

SysML v2 has a large specification, but a model like Tidewater uses a small part of it. Five ideas cover almost everything in this book.

6.3.1 Idea 1: definitions and usages

The single most important idea in SysML v2 is the separation between a **definition** (a kind of thing) and a **usage** (a use of that kind of thing in a particular place). You already know this idea from SysML v1, under different names:

Table 6.1: Definitions and usages, from SysML v1 to SysML v2

SysML v1	SysML v2 definition	SysML v2 usage
block	`part def SolarPanel`	`part solar : SolarPanel;` (a part property typed by the block)
value type	`attribute def Power`	`attribute ratedPower : Power;`
interface block (port type)	`port def PowerIF`	`port supplyOut : PowerIF;`
signal	`item def Observation Record`	`item record : ObservationRecord;`
activity	`action def SampleSensors`	`action sample : SampleSensors;` (a call behavior action)
state machine	`state def BuoyModes`	`exhibit state modes : BuoyModes;`
constraint block	`constraint def Energy Balance`	`constraint balance : EnergyBalance;` (or `part`, as the converter writes)
requirement	`requirement def`	`requirement TW_SYS_011 { … }`
test case	`verification def Test DataFlow`	`verification testDataFlow : Test DataFlow;`

The keyword `def` marks a definition. A line without `def`, with a colon, is a usage: "a part named `solar`, of kind `SolarPanel`." That is exactly a SysML v1 part property whose type is the `Solar Panel` block. The difference is that SysML v2 applies the same pattern to everything, consistently.

6.3.2 Idea 2: everything lives in a package, and packages import each other

A SysML v2 file contains packages, and packages contain definitions and usages:

```
package Stations {
    private import Analysis::*;
    private import Interfaces::*;
    private import Types::*;
    doc /* Everything that goes out to a site: moored buoys and shore stations. */
    part def StationSegment { ... }
}
```

The `import` lines say which other packages this one uses names from. `Interfaces::*` means "every name in the Interfaces package." `private` means the imported names are visible inside this package but not passed on to packages that import this one. Imports are how SysML v2 makes dependencies between packages explicit, and they are exactly what you need to see the package dependency cycles from Chapter 2, The Cameo years (p. 10): if `Stations` imports `Data Center` and `DataCenter` imports `Stations`, the cycle is written right there in the text.

6.3.3 Idea 3: the body of a definition is its structure

Inside the braces of a `part def` go its features: parts, attributes, ports and connections. Compare the converter's output for `PowerSystem` (above) with what you would have drawn in v1:

- the `part` lines are the part properties, the boxes inside the IBD frame;
- the `attribute` line is a value property;
- the `port` line is a port on the frame;

- each `connection … connect A to B` line is a connector between two ports, written with dotted paths (`solar.'out'` means "the port named `out` on the part named `solar`");
- each `bind A = B` line is a binding connector, the kind used in parametric diagrams to tie a constraint parameter to a value property.

Notice `'out'` and `'in'` in quotes. `out` and `in` are SysML v2 keywords (they mark directions), so a name that happens to be a keyword must be quoted. The converter does this automatically. You will see the same with `'send'` in the behavior package.

6.3.4 Idea 4: documentation is part of the model

```
    doc /* Solar generation, storage and regulation for a buoy. */
```

A `doc` comment is attached to the element it sits in and is part of the model, not a note on a diagram. The converter turns every v1 comment into a `doc`. Plain comments, which the model ignores, start with `//`; the converter uses them for its `TODO` lines, which is why the TODO lines can be counted and removed without changing the model's meaning.

6.3.5 Idea 5: metadata replaces stereotypes

In SysML v1, the team extended the language with a profile: custom stereotypes like `L5Assembly` (the level of a block in the part tree) and `Procured` (a part bought from a supplier, with tagged values for the supplier and part number). In SysML v2 the equivalent is **metadata**: a `metadata def` declares the annotation, and `@Name` applies it:

```
package Conventions {
    doc /* Government modeling conventions for the Tidewater v2 model. */
    metadata def L1Network { doc /* The whole network. */ }
    metadata def L2Segment { doc /* A segment. */ }
}
package Network {
    private import Conventions::*;
    part def ObservationNetwork {
        @L1Network;
        doc /* Every station, the data center and the field crew that keeps them running. */
    }
}
```

The converter does not know the team's custom profile, so it cannot translate the stereotypes; it leaves a TODO line on every element that carried one. Closing those TODOs is the team's first big piece of work in Chapter 27, Sprint 2: Drop 1 on develop, every change gated (p. 424) and Chapter 28, Sprint 3: the first red X, DOORS in the model, and a wait nobody named (p. 440), and it is mechanical: write the `metadata def` once, then replace each TODO line with the matching `@Level;` line.

6.4 Side by side: the solar panel

Here is the solar panel from Chapter 2, The Cameo years (p. 10) again, in both forms. On the left of the comparison (first block) the v1 XMI, on the right (second block) what the converter writes.

```
<packagedElement xmi:type="uml:Class" xmi:id="_22_1_7f3a0c1_a89dbce342d7" name="SolarPanel">
  <ownedComment ... body="Generates power."> ... </ownedComment>
  <ownedAttribute xmi:type="uml:Property" ... name="ratedPower"
  type="_22_1_7f3a0c1_dd4fe0cc913f">
    <defaultValue xmi:type="uml:LiteralReal" ... value="40"/>
  </ownedAttribute>
  <ownedAttribute xmi:type="uml:Port" ... name="out" aggregation="composite"
  type="_22_1_7f3a0c1_dba6f2e0627e"/>
</packagedElement>
<sysml:Block ... base_Class="_22_1_7f3a0c1_a89dbce342d7"/>
<Tidewater_Profile:L5Assembly ... base_Class="_22_1_7f3a0c1_a89dbce342d7"/>
<Tidewater_Profile:Procured ... base_Class="_22_1_7f3a0c1_a89dbce342d7" partNumber="SP-40"
supplier="Notional Solar Ltd."/>
```

```
    part def SolarPanel {
        // TODO stereotype Tidewater_Profile:L5Assembly on this element (xmi:id ...) not
        converted
        // TODO stereotype Tidewater_Profile:Procured on this element (xmi:id ...) not converted
        doc /* Generates power. */
        attribute ratedPower : Power = 40;
        port 'out' : PowerIF;
    }
```

Everything the XMI says about the solar panel is in those seven lines. The type identifiers have become names (`Power`, `PowerIF`). The `sysml:Block` record has become the word `part def`. The default value is `= 40`. The two custom stereotypes are TODO lines that name exactly what was not carried and give the v1 identifier, so a person can find the original. After the team's conventions are in place, the finished version reads:

```
    part def SolarPanel {
        @L5Assembly;
        @Procured { supplier = "Notional Solar Ltd."; partNumber = "SP-40"; }
        doc /* Generates power. */
        attribute ratedPower : Power = 40;
        port 'out' : PowerIF;
    }
```

That version was checked with the grammar checker (inside a small package that declares `Power`, `PowerIF` and the two metadata definitions), as was every snippet in this chapter.

6.5 Side by side: a requirement and its satisfy

In v1, a requirement is a UML class with a stereotype application that holds its identifier and text, and a satisfy relationship is a UML abstraction with another stereotype application:

```
<packagedElement xmi:type="uml:Class" xmi:id="_22_1_7f3a0c1_f832863d9d67" name="Own power"/>
...
<packagedElement xmi:type="uml:Abstraction" xmi:id="_22_1_7f3a0c1_2dc66f99c741"
                 client="_22_1_7f3a0c1_53498550400d" supplier="_22_1_7f3a0c1_f832863d9d67"/>
...
<MD_Customization_for_Requirements__additional_stereotypes:performanceRequirement
    ... base_Class="_22_1_7f3a0c1_f832863d9d67" Id="TW-SYS-011"
    Text="Each buoy shall run from its own solar panel and battery." risk="Low"
    verifyMethod="Analysis"/>
<sysml:Satisfy xmi:id="_22_1_7f3a0c1_090cc58cfa58"
base_Abstraction="_22_1_7f3a0c1_2dc66f99c741"/>
```

Reading that takes work: the abstraction's `client` is the `PowerSystem` block (by identifier), its `supplier` is the requirement (by identifier), and only the stereotype application at the bottom says it is a satisfy. The converter writes:

```
requirement TW_SYS_011 { doc /* Each buoy shall run from its own solar panel and battery. */
attribute doorsId = "TW-SYS-011"; attribute title = "Own power"; attribute verificationMethod =
"Analysis"; attribute risk = "Low"; }
...
satisfy TW_SYS_011 by PowerSystem;
```

The requirement's name is its DOORS identifier with the hyphens changed to underscores (`TW_SYS_011`), because a SysML v2 name cannot contain a hyphen without quotes. The identifier itself is kept, exactly, as an attribute named `doorsId`, and that attribute is how the trace report matches the model against DOORS (Chapter 20, The pipeline, job by job (p. 302)). The v1 name ("Own power") becomes the `title`. The satisfy relationship is one line that says what it means.

Derivation (the v1 `deriveReqt` relationship) becomes a connection using the standard library's derivation definitions:

```
    #derivation connection {
        end #original ::> TW_STK_006;
        end #derive ::> TW_SYS_011;
    }
```

Verification lives inside a `verification def` (the v1 test case), in its objective:

```
    verification def TestStationObservations {
        doc /* Sensors and transmission on a buoy and a shore station. */
        subject station : ObservingStation;
        objective {
            verify TW_SYS_001;
        }
    }
```

6.6 Where the converter stops

The converter (`bin/xmi2sysml.pl`, in the kit) maps every SysML v1 construct it understands into SysML v2, and for everything else it writes a `// TODO` line at the place the construct belonged, naming the v1 type and identifier. It never drops anything silently. This design choice is what makes the port measurable: the TODO lines *are* the remaining work.

Here is what the converter carries and what it leaves, in general:

Table 6.2: What the converter carries (from the converter's own mapping table)

SysML v1	Converter output
Package, Model	`package`, one file per top-level package
Block, Class, Actor	`part def`
InterfaceBlock (or a class used only as a port type)	`port def`
ConstraintBlock	`constraint def` with its parameters; the expression becomes a TODO
Signal	`item def`
ValueType, DataType	`attribute def` (primitive types map to `Scalar Values`); units become a comment and a TODO
Enumeration	`enum def`
Activity	`action def` with parameters, call actions and control flows as `first … then …`
UseCase	`use case def` (skeleton)
Requirement (and Cameo's requirement kinds)	requirement with `doc`, `doorsId`, `title`, `verificationMethod`
TestCase	`verification def`
part, reference, value and flow properties	`part`, `ref part`, `attribute`, `in`/`out` features
ports (flow, proxy, full)	`port`
Connector, BindingConnector	`connection … connect … to …`, `bind … = …`
Generalization	`:>` (specialization)
Satisfy, Verify, DeriveReqt	`satisfy`, `verify` in an objective, `#derivation connection`
Trace, Refine, other dependencies	`dependency from … to …`
Allocate	`allocate … to …`
Comment	`doc`
State machine, interaction, include/extend, fork/join/decision nodes, pins, object flows, constraint expressions, custom stereotypes, instance specifications	`// TODO` line
Diagrams and tool extensions	counted and skipped

For drop 1, the TODO lines break down like this:

Table 6.3: Drop 1's conversion debt, by kind

TODO kind	Lines	What a person must do
custom level stereotype (L1Network ... L6Component)	36	write the conventions package once; replace each line with `@Level;`
custom Procured stereotype	3	replace with `@Procured { supplier = …; partNumber = …; }`

TODO kind	Lines	What a person must do
unit (InstanceSpecification)	3	define the units, or specialize ISQ quantity types
the profile itself	1	replaced by the government conventions package
state machine (BuoyModes)	1	write a `state def` by hand from the v1 diagram
interaction (DailyUpload)	1	write the message sequence by hand, or decide to keep it as a view only
use case include	1	write `include use case`
fork node, input pin, object flow, control flow from the fork	4	write `fork`, `flow` and the extra successions by hand
constraint expression	2	write the expression in the `constraint def` body
total	**52**	

That table is the shape of the port. The 39 stereotype lines are mechanical and fast, so the TODO count will fall quickly in the first sprints. The last nine are judgment calls that need the v1 diagrams open beside the text, and they take longer. Chapter 13, The operating model (p. 196) turns the table into stories and tasks.

6.7 Writing what the converter could not

Three of the hand-written replacements are worth seeing now, because they show SysML v2 constructs the converter never writes.

6.7.1 The buoy's state machine

The v1 model has a state machine with four states and seven transitions on the moored buoy. The converter leaves:

```
        // TODO uml:StateMachine 'BuoyModes' (xmi:id ...): no SysML v2 mapping in this converter
```

Bob wrote the replacement in Chapter 31, Sprint 6: Behavior by hand, a table for the program, and baseline-0 (p. 495) from the v1 diagram. A version that passes the grammar check:

```
package ModesExample {
    state def BuoyModes {
        entry; then observing;
        state observing;
        state degraded;
        state maintenance;
        state recovered;
        transition observing_to_degraded first observing accept LowBattery then degraded;
        transition degraded_to_observing first degraded accept Charged then observing;
        transition to_maintenance first observing accept CrewOnSite then maintenance;
        transition back_to_service first maintenance accept CrewDeparts then observing;
        transition lifted first maintenance accept LiftedAboard then recovered;
```

```
    }
    attribute def LowBattery;
    attribute def Charged;
    attribute def CrewOnSite;
    attribute def CrewDeparts;
    attribute def LiftedAboard;
}
```

Each transition reads as a sentence: *first* in this state, *accept* this event, *then* go to that state. The v1 diagram also had a transition from `degraded` to `maintenance`; a complete version adds it the same way. Note what this means for the future: when the prime adds a "Storm" state to the v1 state machine in drop 3, the converter's output will not change at all, because the converter writes only the TODO line. The change is invisible in the converted text. Chapter 43, Maintenance through the life cycle (p. 602) shows how the team catches it anyway, by comparing the v1 drops directly.

6.7.2 The energy balance

The v1 constraint block's expression was written in English-like text, which v1 tools store but do not check:

```
<ownedRule xmi:type="uml:Constraint" ... name="balance">
  <specification xmi:type="uml:OpaqueExpression" ...><body>generation * sunHours &gt;= load *
  24</body>
  <language>English</language></specification>
</ownedRule>
```

Look closely: the expression uses `sunHours`, but the constraint block has no parameter named `sunHours`. The v1 tool never noticed, because an "opaque expression" is just a string. In SysML v2 the expression is part of the language, so the hand-written version has to declare everything it uses:

```
    constraint def EnergyBalance {
        in generation : Real;
        in sunHours : Real;
        in load : Real;
        generation * sunHours >= load * 24
    }
```

Writing it by hand surfaced a modeling error that had been in the v1 model for three years. This happened again and again during the port, and it became one of the team's favorite arguments in Chapter 38, How the team changed its mind (p. 557): *the act of writing the model as text found mistakes the diagrams had hidden.* Each such finding became a discrepancy report to the prime.

6.7.3 The observation cycle's fork

The v1 activity `ObserveCoast` has a fork after the quality check, so that archiving and health logging can proceed in parallel. The converter carried the call actions and the simple control flows, and left TODOs for the fork, the pin and the object flow. A hand-written version:

```
    action def ObserveCoast {
        out bulletin : Bulletin;
        action sample : SampleSensors;
        action check : CheckQuality;
        action store : ArchiveObservations;
        action logHealth { in report : HealthReport; }
        action publish : PublishBulletin;
        fork afterCheck;
        first start then sample;
        first sample then check;
        first check then afterCheck;
        first afterCheck then store;
        first afterCheck then logHealth;
        first store then publish;
        flow publish.bulletin to bulletin;
    }
```

(The real activity also has `send` and `ingest` steps; they are left out here to keep the example short.)

6.8 Bob's Wednesday, again

On the third day of sprint 1, Bob ran the requirements converter's trace against the converted drop 1. The command reads the DOORS export from the drop and the converted model, and compares them requirement by requirement:

```
perl "$KIT/bin/reqif2sysml.pl" --report --model drop1-v2 drop1/tidewater.reqif
```

The last lines of its report:

```
C requirements linked (doorsId in the model)     29 of 30
F orphan requirements (nothing satisfies)               2
G unverified requirements (no verify)                   9
I trace completeness (satisfied and verified)        66.7%
doorsIds in the model but not in this export            1
text differs from DOORS                                 1
not in the model: TW-SYS-016
orphans: TW-SYS-012 TW-SYS-016
unverified: TW-STK-001 TW-STK-002 TW-STK-003 TW-STK-004 TW-STK-005 TW-STK-006 TW-SYS-011
TW-SYS-016 TW-SEG-002
drift: TW-SYS-006
  TW-SYS-006 DOORS: The data center shall publish a marine bulletin twice a day, on a fixed
  schedule.
  TW-SYS-006 model: The data center shall publish a marine bulletin on a fixed schedule.
```

It ran in about a seventh of a second. Bob read it twice, then went back to the 2019 email in Chapter 1, Thirty years of documents (p. 2) and checked: the question "which requirements

don't have a verification method?" was answered by the `unverified` line, and the answer included things Bob's spreadsheet method would never have found.

Line by line:

- **C, linked: 29 of 30.** One DOORS requirement, TW-SYS-016 (a common time reference), does not appear in the model at all. Nobody had noticed in three years.
- **F, orphans: 2.** Nothing in the model satisfies TW-SYS-012 (stations serviceable from the vessel) or TW-SYS-016. TW-SYS-012 has a *trace* from the mast block, which looks like coverage on a diagram but is not a satisfy.
- **G, unverified: 9.** Six of the nine are stakeholder needs, which are verified through the system requirements derived from them; that is a modeling convention to agree, not a defect. The other three (TW-SYS-011, TW-SYS-016, TW-SEG-002) are real gaps.
- **I, completeness: 66.7%.** The share of requirements that are both satisfied and verified.
- **In the model but not in DOORS: 1.** TW-SEG-009 (a lowerable mast) exists in Cameo but not in DOORS. Either the prime added it to the model and forgot DOORS, or DOORS deleted it and the model kept it.
- **Text drift: 1.** TW-SYS-006 says "twice a day" in DOORS and not in the model. Which one is right is a question for the CCB; that they differ is now a fact that anyone can see, on every run.

Each of those findings becomes a discrepancy report to the prime in Chapter 29, Sprint 4: the model site, a cycle broken, and the demo that landed (p. 459), and the run itself becomes the pipeline's `trace` job in Chapter 28, Sprint 3: the first red X, DOORS in the model, and a wait nobody named (p. 440). Bob summed it up at the stand-up the next morning:

> *Y: ran the trace against drop 1 – found the drift in two minutes, it used to take a week, and we never found this one. T: TODO inventory by package. B: none.*
>
> — Bob, stand-up, 8 October

6.9 Three rules for reading SysML v2 text

When you open a converted file for the first time, these three rules will carry you most of the way.

1. **Find the `def`s first.** Each `part def`, `port def`, `action def`, `requirement` and `verification def` is a thing in the system's vocabulary. Read the names and the `doc` lines before reading any bodies. That gives you the map.
2. **Read every line inside a body as a sentence.** `part solar : SolarPanel;` reads "has a part, solar, which is a SolarPanel." `connection charge connect solar.'out' to regulator.'in';` reads "a connection named charge joins the solar panel's out port to the regulator's in port." `satisfy TW_SYS_011 by PowerSystem;` reads "PowerSystem satisfies TW-SYS-011."
3. **Treat every `// TODO` as an open action item.** It names what the converter could not carry and where the original is. Until it is closed, the model is incomplete at exactly that spot, and nowhere else.

Objection: "Engineers outside the team won't read this."

Jim, again in the devil's advocate role.

Answer

Many will not, and they do not have to. The text is the record that the maintainers edit, review and check. Everyone else reads what is generated from it: the published site with diagrams, trees and trace reports (Chapter 17, The DevSecOps tools that earn their keep (p. 255)). In the Cameo era, the record was readable only by three license holders, and everyone else saw screenshots. Now the record is readable by anyone with an editor, and the generated views by anyone with a browser.

Objection: "The converter will get things wrong, and we won't notice."

Bob.

Answer

The converter reports what it mapped, folded and left as TODO, element type by element type, and the pipeline runs the grammar checker and a structure check on every change (Chapter 20, The pipeline, job by job (p. 302)). The trace report compares every requirement against DOORS. What remains is semantic review by a person, which is what the merge request is for. Chapter 26, Sprint 1: A GitLab project the team can work in (p. 404) shows the team's assessment of the converter's output, package by package, before anything was merged.

6.10 Names, identifiers and the way back to v1

A port has to keep a thread back to the original, so that every element in the v2 model can be traced to the v1 element it came from, and every requirement to DOORS. The converter keeps three such threads.

- **DOORS identifiers.** Every requirement keeps its DOORS identifier as `attribute doors Id = "TW-SYS-011";`. Its SysML v2 name (`TW_SYS_011`) is derived from the identifier, but the attribute is what the trace report reads, so renaming a requirement never breaks the trace.
- **v1 identifiers on every TODO.** Each TODO line carries the `xmi:id` of the v1 element it stands for. When Bob writes the state machine by hand, the TODO line being replaced names the exact v1 element it stood for; that identifier goes into the merge request description, so the record shows which v1 element each hand-written part replaced.
- **Stable names.** The converter names v2 elements after their v1 names (with spaces and keywords quoted: `'serial number'`, `'out'`). Because the prime's tool keeps an element's identifier stable from drop to drop, and the converter is deterministic, converting the same element in two drops produces the same text. That property is what lets git compare and merge successive drops, as Chapter 13, The operating model (p. 196) explains.

6.11 The standard library

SysML v2 comes with a standard library of definitions that every model can use. The converter uses a few of them:

- `ScalarValues::Real`, `ScalarValues::Integer`, `ScalarValues::String`, `ScalarValues::Boolean` for the v1 primitive types;
- `RequirementDerivation` for the derivation connections;
- the ISQ (International System of Quantities) packages for physical quantities, which the team uses when closing the unit TODOs (a `Power` attribute should eventually specialize the ISQ power quantity rather than a bare `Real`).

You do not need to read the library. You do need to know it exists, because the grammar checker and the structure check accept names from it without a declaration in your files, and because when you see `ScalarValues::` or `ISQ::` in the model, the definition is in the standard, not in Tidewater.

6.12 A first session with the text, in Vim

Chapter 10, Vim for models (p. 143) covers the kit's Vim support in full. Here is enough to read the converter's output comfortably on day one, using only plain Vim commands the team already knew:

Table 6.4: Reading converted SysML v2 in plain Vim

To do this	Type
open every converted file at once	`vim /tmp/drop1-v2/*/*.sysml`
go to the next file / previous file	`:next` / `:prev`
find the next definition	`/def` then `n` for the next
jump from an opening brace to its closing brace	`%`
fold everything between braces, to see only the definitions	`:set foldmethod=syntax` then `zM` (open with `zo`)
list every TODO in every file, as a jump list	`:vimgrep /TODO/ /tmp/drop1-v2/**/*.sysml` then `:copen`
jump to the next / previous TODO in that list	`:cnext` / `:cprev`
find every use of a name under the cursor	`*`

The `:vimgrep` line is the one Bob used most in the first week. It turns the 52 TODO lines into a list Vim can step through, one at a time, with the file opened at the right line each time. Bob called it "the action item list I always wanted."

With the kit's Vim files loaded (see Chapter 9, The survival kit (p. 126)), `.sysml` files also get syntax highlighting, and `:make` runs the grammar checker and lint and puts every error into the same jump list.

6.13 Habits from SysML v1 to unlearn

Some habits that served the team well in Cameo work against them in text. The team found these the hard way; listing them here may save you some of the trouble.

- **Drawing to think.** In Cameo, the way to design was to draw. In text, sketch on paper or a whiteboard if that helps, then write. Do not try to keep a hand-drawn diagram "in sync" with the text; generate diagrams from the text when you need them.
- **One giant change before anyone sees it.** In Cameo, Bob would work for a week and then show the result. In the new way, a change that takes more than a day or two to review is too big; split it (Chapter 13, The operating model (p. 196)). Small changes are reviewed better and merged sooner.
- **Fixing the layout.** A surprising share of Cameo time went into moving boxes so lines did not cross. In text there is no layout. Whitespace and order in the file are for readability, and the team agreed a simple convention (definitions in the order of the part tree, one blank line between definitions) and stopped thinking about it.
- **Copying an element to make a variant.** Cameo makes copying easy, which is how `Battery_old` and `Copy of SensorSuite` came to exist. In SysML v2, a variant is a specialization (`part def B :> A`) or a redefinition, and the original stays the single source.
- **Trace links to everything that mentions a block.** The `ObservingStation` block carries 17 relationship ends because the team drew a trace to every requirement that mentioned it in any document. A trace link should mean something specific (satisfy, verify, derive, refine). If it does not, leave it out; the trace report (Chapter 20, The pipeline, job by job (p. 302)) computes the useful relationships.
- **The model as a private workspace.** In Cameo, the model lived on Bob's machine. In the new way, the model is a shared record; every change is visible to the team the moment its merge request opens. That feels exposed at first. It is also what makes help possible.

6.14 Reading the converter report, column by column

The first table in the converter's report has a row for every element type in the XMI and four columns. Learning to read it takes five minutes and saves hours.

Table 6.5: The converter report's four columns

Column	Meaning	What to look for
`in`	how many elements of this type the XMI holds	surprises: types you did not know the model used
`mapped`	how many became a SysML v2 element of their own	should equal `in` for blocks, ports, properties, requirements
`folded`	how many were absorbed into another element (a connector end into its connector, a multiplicity into its property, a generalization into `:>`)	nothing to do; folded is fine
`todo`	how many fall under a TODO line	the types to plan work for

Below that table, the report lists the stereotype applications (including the custom ones it could not convert), the `ported vs remaining` count (the elements that the status metrics in the kit call "in scope": classes, ports, properties, connectors and activities), what the output contains by SysML v2 keyword, and every TODO with its v1 identifier. When the prime delivers a new drop, comparing two of these reports is the quickest first look at what changed in size and debt, before the detailed comparison in Chapter 43, Maintenance through the life cycle (p. 602).

6.15 Exercises

Exercise 6.1: Convert and count

Run `perl sim/tidewater-v1.pl --drop 1 /tmp/drop1` and then `perl bin/xmi2sysml.pl --report --out /tmp/drop1-v2 /tmp/drop1/tidewater.xmi`. In the terminal, count the TODO lines with `grep -r "// TODO" /tmp/drop1-v2 | wc -l`. Does your count match the table "Drop 1's conversion debt, by kind"?

Exercise 6.2: Read a package

Open `/tmp/drop1-v2/datacenter/DataCenter.sysml` in Vim. Without looking at any diagram, draw the data center's internal block diagram on paper from the text: the parts, the ports and the connections. Then compare with the v1 diagram list in Chapter 2, The Cameo years (p. 10). Which was faster to understand?

Exercise 6.3: Find a hidden error

The energy balance expression used a parameter that was never declared. Search the drop-1 XMI for other opaque expressions (`/OpaqueExpression` in Vim). Are there others? What would you write in a discrepancy report to the prime about the one you found?

Exercise 6.4: Write a state machine

Add the missing transition from `degraded` to `maintenance` (event `CrewOnSite`) to the `Buoy Modes` example, save it as `/tmp/modes.sysml`, and check it with `perl tools/sysml/sysml.pl check /tmp/modes.sysml`.

CHAPTER

7

Requirements a pipeline can check: the recipe

From what you know →

You know

how to write a "shall" statement, and the review comments that come back on weak ones.

→ Here it is called

the requirement recipe and the recipe checker.

→ What is new

a script grades every requirement on every change, so the comments arrive before the review.

Note: What you have, what you need

What you have: decades of writing and reviewing requirements. You know a bad one when you see it: "the system shall provide adequate performance" makes every reviewer reach for a red pen. You also know that red pens run out of ink. In a 400-requirement specification, the twentieth weak verb gets through because the reviewer is tired.

What you need from this chapter: one explicit pattern for a good requirement statement, so that "good" stops being a matter of taste; the vocabulary to talk about each part of a statement; and a tool that reads every requirement against the pattern on every change, so the red pen never runs out. You will run the tool on Tidewater and on the recipe's own worked example.

This is a core chapter: read it in the first four weeks, and type the commands. It sits here, between the SysML v2 introduction and the setup chapters, because the requirement statement is where systems engineering and the pipeline first meet. A pipeline cannot judge an architecture. It can judge a sentence.

7.1 Jim's red pen

Jim kept a one-page sheet pinned above the desk for most of the document era. It was a requirements recipe: one sentence pattern, eighteen numbered parts, six of them required, and a list of

verbs nobody could verify. Jim had used it for years to review specifications by hand. A review meant printing the SRS, reading every paragraph with the sheet beside it, and marking up the margins. A careful review of a 300-requirement specification took Jim about three days.

In the Cameo years the sheet did not help much. The requirements were in DOORS, synchronized into the model, and the review happened on screen, one requirement at a time in a property panel. Jim stopped using the sheet; there was no margin to write in.

In sprint 5 Jim pinned it back up, for a different reason. The team had a pipeline that read every model file on every change, and the trace job had already shown that the pipeline could find what tired reviewers missed. Jim asked a simple question at refinement: could the pipeline read the requirement text against the recipe?

> *I've been doing this with a pen for twenty years. If a script can do the first pass, I can spend my three days on the requirements that are actually wrong, instead of the ones that just say "provide".*
>
> — Jim, sprint 5 refinement

The answer was yes, with a qualification this chapter is careful about. A script can check the *form* of a requirement: is there exactly one operative verb, is the target verb concrete, is there a measurable threshold, is the referenced document identified down to the section. It cannot check whether the requirement is the right requirement. The recipe makes the form checkable, so that the human review time goes to the substance.

7.2 The recipe in one sentence

The recipe is the user's own requirement-writing standard, developed from industry practice for requirement specifications. Its premise is simple: if every requirement follows one sentence pattern, then every requirement meets the static criteria of a good requirement by construction: it is verifiable, atomic and clear.

The recipe applies to every requirement in a project's requirement specifications. One paragraph holds one requirement and the statements that belong to it. A requirement is a single sentence that applies functional or performance criteria to the specification target. These single-sentence structures stand alone as paragraphs, or as elements of larger documents and databases.

Here is the pattern. The numbers in brackets are the part numbers used throughout this chapter.

```
(U)[1] While[2] / upon[3] ____[4]____, the ____[5]____, for the ____[6]____ function,
shall/will/should/may[7] ____[8]____ ________[9]________ with/such[10] that
________[11]________ in accordance with ________[12]________ (____[13]____)
____[14]____, except ________[15]________. Note:
________________[16]________________ (____[17]____) ____[18]____
```

Table 7.1: The eighteen parts of the requirement recipe

Part	Name	Required
1	Classification	Yes

Part	Name	Required
2	State	
3	Trigger	
4	Pre-condition(s)	
5	Specification target	Yes
6	Allocated function	
7	Operative verb	Yes
8	Target verb	Yes
9	Precise object(s)	Yes
10	Modifier	
11	Modification(s)	
12	Specification	
13	Document number	
14	Section number	
15	Exception(s)	
16	Informational statement(s)	
17	Requirement ID	Yes
18	Requirement tags	

Six parts are required: the classification, the specification target, the operative verb, the target verb, the precise objects and the requirement ID. Everything else appears when the requirement needs it. A short requirement is not a bad requirement: "(U) The data logger shall store 72 hours of observation records. (TW-SEG-011)" has all six required parts and nothing else, and it is a perfectly good requirement.

The pattern reads like a form with blanks. That is deliberate. The recipe is not prose style advice; it is a structure, and structures can be checked.

7.3 The parts, one at a time

7.3.1 Classification (part 1)

A classification portion mark is required in most defense-related requirement specifications. The portion mark for the requirement statement comes at the beginning of the paragraph. Portion marks include (U), (U//FOUO), (S), (S//NF), (S//FRD) and so on. The project's security classification guide (SCG) and its portion-marking instructions govern the details. Typical guidance is that every paragraph of a document is portion marked, and since a single requirement sits in its own paragraph, every requirement carries its own mark.

Note: FOUO is gone

The recipe's examples use "(U//FOUO)", For Official Use Only, which is how many existing specifications are marked. In current US government practice FOUO has been replaced by Controlled Unclassified Information: CUI, with its own categories and banner rules. When you rewrite or port an older specification, follow your program's CUI guide for the replacement marking rather than copying FOUO forward. Tidewater is fictional and unmarked; its rewrites below use "(U)" only to show where the mark goes.

The tool you will use in this chapter treats the classification as optional by default, because a notional project like Tidewater carries none, and `--require classification` makes it required.

A real program turns that option on in its pipeline on day one.

7.3.2 State, trigger and pre-conditions (parts 2, 3 and 4)

Some requirements, functional or performance, apply only in a specific context. There are two ways to define the context: a state and a trigger.

- A **state** is a specific operating mode or status of the element. Common state words are "while" and "when".
- A **trigger** is an event that causes the action, or that applies the threshold the requirement specifies. Common trigger words are "upon" and "every".

The words after "while" or "upon" are the pre-conditions. It is not necessary to put pre-conditions at the beginning of the statement, but placing them first gives the reader the context before the requirement itself, and the recipe recommends putting all of them at the head of the statement.

In Tidewater terms: "While in observing mode" is a state; "upon each scheduled transmission time" is a trigger.

7.3.3 Specification target and allocated function (parts 5 and 6)

As a rule, the **specification target** is always the subject of the specification itself, at least for performance and functional requirements. A requirement found in the "data processor" specification always has "data processor" as its specification target. If a requirement in the telemetry unit specification has "the data center" as its subject, it is in the wrong specification.

The **allocated function** is optional. It applies the requirement to a structural or functional component of the specification target. If the data processor consists of the "data collector", "data extractor" and "message passer" top-level functions, stating that the required function is allocated to one of them gives greater resolution to the design space.

The only time the specification target may be something other than the subject of the specification is for tertiary requirements on a related entity: the developer who builds the specification target, or the tester who evaluates it. "The developer shall deliver the source code..." in a software specification is the classic case.

In SysML v2 the specification target becomes the requirement's `subject`, and the allocated function becomes an `allocate` relationship or the subject's feature. We come back to that in Section 7.9, The recipe in SysML v2 (p. 87).

7.3.4 Operative and target verbs (parts 7 and 8)

The core of a requirement statement is a pair of verbs: an operative verb and a target verb. There are four main operative verbs in requirements language.

Table 7.2: The four operative verbs

Operative verb	Meaning
shall	The strongest verb. It applies a requisite function or performance threshold to the system. In most contracts every "shall" in a project specification has legal force.

Operative verb	Meaning
will	A weaker verb. Usage varies, but it typically states something about an external actor, mirroring a "shall" for that actor.
should	A design or implementation guideline: how the system should behave or how a function should be implemented. It is not a requirement.
may	Gives resolution to the available design space. Often an informational statement below a "shall" or "should".

There may be others. The specification, or a parent document it references, defines the available operative verbs and their precise meanings.

The **target verb** is the action or performance the system must meet. Target verbs must be specific and verifiable. Some target verbs are common and yet rarely or never verifiable: **provide, have, be, be capable of, establish**. Use specific, concrete verbs that belong to the engineering domain instead: send, store, measure, report, detect, compute, display, isolate.

The kit's checker adds a few verbs of its own to the recipe's list, because they fail the same test in practice: support, allow, enable, handle, ensure, facilitate, accommodate and permit. It also treats "be able to" as a weak verb: "shall be able to read" is a promise about capability, and the requirement should state the action itself, "shall read", with its threshold.

7.3.5 Precise objects (part 9)

The precise objects say what the operative and target verbs act on. They give definition to what the specification target is or does. As in ordinary language, the domain of the object is a function of the verb. If the verb is "store", the precise object is what is stored, and it must be something the specification target can store. If the verb is "maintain", the precise object is a measurable state: altitude, temperature, quantity, state of charge.

7.3.6 Modifiers and modifications (parts 10 and 11)

Modifiers apply specificity and constraint to the verb and its objects. A statement can have any number of them, as prepositional phrases following the noun-verb-object triple. Almost every preposition in the statement is, by definition, a modification. Strictly speaking the state, trigger and exception are modifiers too; this part refers to the modifiers that are not states, triggers or exceptions.

The available modifiers follow from the noun-verb-object structure. If a component "shall hand over bus control", a prepositional phrase should follow, most likely starting with "to", naming the component that receives bus control. And just as the first component must be capable of handing over bus control, the object of the preposition must be capable of receiving it and operating accordingly. A modifier is a claim about two things, not one.

Thresholds live here. "Within", "less than", "above", "at least", "every" all introduce a performance threshold, and a performance verb without one is a requirement nobody can fail.

7.3.7 Specification reference (parts 12, 13 and 14)

Often a requirement is levied on a specification target in one document, but its technical details are found in another. A software item required to use floating-point arithmetic would not define

every case, exception and behavior of floating-point operations; its specification would reference IEEE 754, the standard for binary floating-point arithmetic, over and over.

Whenever an external specification is referenced within a requirement, best practice is to include the document number (unless it is part of the title) and the specific section or sections that apply. If the requirement is about multiplying floating-point numbers, the reference names the sections that define multiplication. It is also acceptable to put the document number in the references list rather than in every requirement that cites it.

Used sparingly, the reference can also levy some or all of an external specification's requirements on the current target. If an interface specification exists, the specifications of the components at either end may each carry a single statement applying the interface specification's requirements to the component.

7.3.8 Exceptions (part 15)

Exceptions modify where the requirement applies, for example when the system is in an error state or off-nominal condition. Requirements must be true all the time, so a requirement must carry the statements that define precisely when it applies. Exceptions are the cut-outs from the state or trigger: they establish the sub-state in which the requirement applies.

The position of an exception is more variable than any other part. Depending on how the sentence flows, it can come at the end of the statement (as in the pattern), as a modifier (exceptions are prepositional too), or as part of the state or trigger.

7.3.9 Informational statements (part 16)

Informational statements sit beside the requirement statement to give additional detail or constraint outside the statement itself. Sometimes they keep the requirement sentence grammatically clean: requirement statements have a specific language, but they must also function as proper English. An informational statement is not part of the requirement and carries no obligation. It can carry design constraints or hints for the verification method.

7.3.10 Requirement identifier (part 17)

Every requirement has a project-unique identifier that labels it independently of all other requirements. Most identifiers combine a number with an acronym that shows where in the system hierarchy the requirement sits: TW-SYS-002 is Tidewater's second system requirement, NC_112358 a navigation computer requirement.

The identifier is what makes a requirement findable in a database, traceable in a model and citable in a merge request. In the SysML v2 port it becomes the `doorsId` attribute, and it is how the trace job matches the model to DOORS (Section 6.10, Names, identifiers and the way back to v1 (p. 69)).

7.3.11 Requirement tags (part 18)

A tag is an optional short indicator that draws special attention to a requirement.

Table 7.3: Requirement tags

Tag	Meaning	Details
SC	Safety-critical	The requirement specifies a safety-critical subsystem, element or function. Treatment of SC requirements follows specific processes in the Quality, Safety and Mission Assurance Plan (QSMAP).
SR	Safety-related	The requirement can affect a safety-critical subsystem, element or function without causing a safety-critical issue directly.
MC	Mission-critical	The requirement is mission-critical, subject to the additional rigor established in the QSMAP or a similar document.
RMF	Risk Mitigation Framework	The requirement has specific cybersecurity equity under the RMF.[1]

Tagging safety-critical and mission-critical requirements is common practice, so they are easy to find and can be held to the additional rigor those requirements must meet. In SysML v2 a tag becomes metadata (`@MC;`), which a pipeline can count and a view can filter on.

7.4 The worked example, decomposed

The recipe comes with a worked example. In a hypothetical project, the following requirement is found in the Navigation Computer CIDS (D12340005):

```
(U//FOUO) While in normal flight mode, except during the navigational upset condition, the
navigation computer
shall send the guidance solution message to the main computer within 30 msec of the 10 Hz clock
signal in
accordance with the Navigation Computer IRS (D12340000) Section 5.1.4. Note: The 10 Hz clock
signal driver is
either directly or indirectly sampled from the main CCA clock source. (NC_112358) [MC]
```

Here it is taken apart, element by element.

Table 7.4: The worked example, part by part

Element	Source text	Evaluation
Classification	“(U//FOUO)”	Unclassified, but labeled For Official Use Only. Care must be taken when extracting this requirement from its specification and storing or sending it through other media, because it is controlled unclassified information.
State indicator	“While”	The system, subsystem or component is in a specific state as the predicate for performing the required function.

[1] In NIST usage, RMF stands for the Risk *Management* Framework (NIST SP 800-37). Use your program’s expansion; the tag means the same thing either way: this requirement has cybersecurity equity.

Element	Source text	Evaluation
Trigger indicator	none	There is no trigger keyword, but an element of the modification defines a trigger.
Pre-conditions	1. “in normal flight mode”; 2. “the 10 Hz clock signal”	1. The requirement applies only while the system is in normal flight mode. The mode need not be defined in this requirement, but it must be defined or referenced in the specification, the Navigation Computer CIDS. 2. A specific trigger, the 10 Hz clock signal, a standard clock signal defined elsewhere in the specification. It appears alongside the modification for grammar and readability.
Specification target	“navigation computer”	The specification is the Navigation Computer CIDS, so the target of its functional and performance requirements is always the navigation computer. The specification may also levy workmanship, security and other requirements on the developer, tester and so on.
Allocated function	none	No allocated function is defined. The allocation of the “send” function will be derived in a lower-level specification descending from the CIDS, or left as design space for the developer to define in the design documentation.
Operative verb	“shall”	The strongest operative verb: complete requisite functionality or performance. In most contracts every “shall” in a project specification has legal force.
Target verb	“send”	Specific and verifiable, given the right objects. The target verb must be something the target can do, and a navigation computer can send a message. “Send” leads us to expect the what and the where: what is sent, and to whom?
Precise object	“guidance solution message”	The navigation computer sends the guidance solution message. The message is identified elsewhere in this specification or a related one (we find later that it is referenced in the Navigation Computer IRS).

Element	Source text	Evaluation
Modifier	1. “to”; 2. “within”	1. “To” introduces the destination. Modifiers are not universally required, but this requirement would be incomplete without the destination. 2. “Within” signals a performance threshold. Words such as within, less than and above all indicate thresholds.
Modifications	1. “the main computer”; 2. “30 msec”	1. The destination of the message, the main computer. 2. The maximum allowable latency for sending the message, coupled with “within”.
Specification	“Navigation Computer IRS”	Additional information about this message passing between the navigation computer and the main computer is in the Navigation Computer IRS.
Document number	“D12340000”	The IRS’s document number. If the project uses an integrated data environment or library system, the number makes the document quick to locate.
Section number	“Section 5.1.4”	The specific subsection of the IRS where the information is found. This makes it far easier for an analyst to line up the family of specifications.
Exceptions	“during the navigational upset condition”	Placed at the front, like a pre-condition, purely for readability. The requirement applies while the computer is in normal flight mode but the upset condition has not been asserted: a very precise state.
Informational statements	“The 10 Hz clock signal driver is either directly or indirectly sampled from the main CCA clock source.”	Not part of the requirement. Related information, design-space constraints or verification elements. Here it says the clock signal is measured against the clock source on the circuit card, which will shape the verification method.
Requirement identifier	“NC_112358”	Project-unique. An acronym for the specification target (NC) and a number (112358) separated by an underscore; searchable in a requirements database, with all its links.
Requirement tags	“MC”	Mission-critical. Quality and mission assurance use the tag to segregate and evaluate requirements under QSMA procedures.

Now let a program read it. The kit’s checker, `tools/sysml/req-recipe.pl`, takes a plain text file

with one requirement per paragraph and decomposes each one the same way. Type this into an empty folder; the first command writes the example into a file:

```
cat > nav.txt <<'EOF'
(U//FOUO) While in normal flight mode, except during the navigational upset condition, the
navigation computer shall send the guidance solution message to the main computer within 30 msec
of the 10 Hz clock signal in accordance with the Navigation Computer IRS (D12340000) Section
5.1.4. Note: The 10 Hz clock signal driver is either directly or indirectly sampled from the main
CCA clock source. (NC_112358) [MC]
EOF
perl "$KIT/tools/sysml/req-recipe.pl" --require classification nav.txt
```

The output lists every part it found, then the summary line every run ends with:

```
NC_112358  score 100  nav.txt:1
    Classification              U//FOUO
    State                       While in normal flight mode
    Preconditions               in normal flight mode
    Specification Target        navigation computer
    Operative Verb              shall
    Target Verb                 send
    Precise Object(s)           the guidance solution message
    Modifier(s)                 to; within
    Modification(s)             the main computer; 30 msec of the 10 Hz clock signal
    Specification               Navigation Computer IRS
    Document Number             D12340000
    Section Number              5.1.4
    Exception(s)                during the navigational upset condition
    Informational Statement(s)  The 10 Hz clock signal driver is either directly or indirectly
    sampled ...
    Requirement ID              NC_112358
    Requirement Tags            MC

requirements=1 findings=0 missing=0
```

Compare it with the table. The program found the same parts a careful human found. It did not find the trigger hidden inside the modification (“of the 10 Hz clock signal”), and it does not need to: the requirement has a threshold and a context, and the score is 100. The decomposition is a reading aid. The findings are the point, and this requirement has none.

Tip: Read the summary line first

Every run of the checker ends with `requirements=N findings=F missing=M`. The pipeline reads that line; so should you. N is how many requirements were read, F how many problems were found, M how many required parts were missing. A score per requirement (100 minus a penalty per problem) helps you sort, but the counts are what you report.

7.5 What the checker looks for

Here is a deliberately weak requirement, the kind every specification has a dozen of:

```
cat > weak.txt <<'EOF'
The data center shall provide a bulletin. (TW-X-001)
EOF
perl "$KIT/tools/sysml/req-recipe.pl" --strict weak.txt
```

`--strict` makes the checker exit with status 1 when it finds anything, which is how a pipeline job fails. The finding is the recipe's first rule:

```
    major  weak target verb 'provide' (the recipe): rarely verifiable -- name the concrete action
    (send, store, measure, report ...)
```

The findings the checker reports, and their weight:

Table 7.5: What the checker reports

Finding	Severity	Example
A required part is missing	major	no requirement ID; no target verb
A weak target verb	major	provide, have, be, establish, be capable of, be able to, support, enable, ensure
More than one operative verb, or more than one sentence	major	"... shall measure wind ... and each buoy shall also measure waves."
Passive voice	minor	"Past observations shall be kept ..."
A performance verb with no measurable threshold	minor	"shall report its operating mode" (how often? to whom?)
A reference without a document number or section	minor	"in accordance with the interface control document"
Vague words	minor	as appropriate, etc., and/or, user-friendly, fast, timely, adequate, robust, minimize
"must" instead of "shall", or will/should/may as the operative verb	info	"The logger must store ..."
A tag the project does not define	minor	[XYZ]

The checker parses by heuristics: it finds the first operative verb, reads the clauses before it as state, trigger, exception, allocated function and target, and reads the words after it as target verb, objects and modifiers. A human reads a sentence better. That is fine: the checker does not replace the reviewer, it makes sure the reviewer never has to spend attention on the mechanical problems.

7.6 Tidewater's requirements, read by the recipe

Now the real thing. Make drop 1, the prime's first delivery, and grade its DOORS export:

```
perl "$KIT/sim/tidewater-v1.pl" --drop 1 drop1 > /dev/null
perl "$KIT/tools/sysml/req-recipe.pl" drop1/tidewater.reqif > recipe-doors.txt
tail -1 recipe-doors.txt
```

```
requirements=30 findings=15 missing=0
```

Thirty requirements, fifteen findings, no missing parts. Every requirement has a target, a "shall", a verb, an object and an ID; the problems are all in how the requirements are worded. To see which requirements, list the ones that scored below 100:

```
grep -E '^TW-.*score [0-9]+ ' recipe-doors.txt | grep -v 'score 100'
```

```
TW-STK-001  score 85  drop1/tidewater.reqif:1  (Coastal coverage)
TW-STK-002  score 84  drop1/tidewater.reqif:1  (Timely observations)
TW-STK-003  score 85  drop1/tidewater.reqif:1  (Public access)
TW-STK-004  score 85  drop1/tidewater.reqif:1  (Serviceable in the field)
TW-STK-005  score 92  drop1/tidewater.reqif:1  (Observation history)
TW-SYS-001  score 77  drop1/tidewater.reqif:1  (Measure conditions)
TW-SYS-002  score 92  drop1/tidewater.reqif:1  (Scheduled transmission)
TW-SYS-008  score 92  drop1/tidewater.reqif:1  (Observation interface)
TW-SYS-009  score 92  drop1/tidewater.reqif:1  (Health reporting)
TW-SYS-012  score 85  drop1/tidewater.reqif:1  (Vessel access)
TW-SYS-014  score 92  drop1/tidewater.reqif:1  (Operating mode)
TW-SEG-003  score 92  drop1/tidewater.reqif:1  (State of charge)
TW-SEG-004  score 92  drop1/tidewater.reqif:1  (Retry)
```

Thirteen requirements, fifteen findings: two requirements have two findings each. Jim read them the way Jim used to read a marked-up SRS, looking for the pattern rather than the instances.

The stakeholder needs are written as wishes. TW-STK-001 says the network "shall provide" observations; TW-STK-003 and TW-STK-004 say mariners and field crews "shall be able to" read and service. That is normal for stakeholder needs, which state what users want in their own words. The recipe still applies, because a need that cannot be verified cannot be traced to anything that can. The fix is not to make the needs technical; it is to make each one name something observable: what is measured, how often, where.

One system requirement is two requirements. TW-SYS-001 reads "Each station shall measure wind, air pressure and air temperature, and each buoy shall also measure waves." Two "shall"s, two subjects, two requirements. The checker reports it as a major finding and scores it lowest, 77. Two requirements in one sentence break traceability: a verification case can pass the first half and fail the second, and the trace cannot say which.

```
perl "$KIT/tools/sysml/req-recipe.pl" --only TW-SYS-001 drop1/tidewater.reqif
```

The interface requirements have no thresholds. TW-SYS-002, -008, -009 and -014 and the segment requirements TW-SEG-003 and -004 say "transmit", "send", "report" and "retry" with no number: no latency, no rate, no limit. These are the findings that matter most for verification, because a test needs a pass criterion, and a requirement with no threshold leaves the test engineer to invent one.

One requirement is a quality, not an action. TW-SYS-012 says "Every station shall be serviceable from the service vessel." "Be" is the weakest verb on the list. "Serviceable" is a judgement; the requirement does not say what the crew must be able to reach, from where, or under what conditions.

> **Case study: The drift shows up again**
>
> Grade the converted model instead of the DOORS export and you get thirty requirements and seventeen findings, two more than DOORS. One of the two is TW-SYS-006, the planted drift from Section 6.8, Bob's Wednesday, again (p. 67): the DOORS text says the bulletin is published "twice a day", the model's copy says only "on a fixed schedule", so the model's version has lost its threshold and earns a finding the DOORS version does not. The other is TW-SEG-009, which exists only in the model. The recipe check and the trace check are independent, and they found the same problem from two directions.

7.7 Rewriting to the recipe

Jim rewrote five of the requirements to show what the recipe asks for. The values are notional, like everything in Tidewater. Each rewrite sits in `examples/tidewater-v1/recipe-rewrites.txt` with the as-delivered text and its finding as comments:

examples/tidewater-v1/recipe-rewrites.txt

```
# Five Tidewater requirements rewritten to the requirement recipe (tools/sysml/req-recipe.pl
grades them).
# Values are notional, like everything in Tidewater. Lines starting with # are comments; one
requirement per
# paragraph. Check it:   perl tools/sysml/req-recipe.pl --require classification
examples/tidewater-v1/recipe-rewrites.txt

# TW-STK-001 as delivered: "The network shall provide marine weather observations for the coastal
region it covers."
# Finding: weak target verb 'provide'. The rewrite names the action, the objects, where and how
often.

(U) The observation network shall measure wind speed, wind direction, air pressure and air
temperature at every station in its assigned coastal region at least once every 10 minutes. Note:
Wave height and period are measured at moored buoys only (TW-SYS-001). (TW-STK-001)

# TW-SYS-002 as delivered: "Each station shall transmit its observations to the data center on a
fixed schedule."
# Finding: 'transmit' with no measurable threshold. The rewrite adds the state, the trigger, the
latency, the
# interface reference and the exception.

(U) While in observing mode, upon each scheduled transmission time, the telemetry unit shall send
the observation record to the data center within 60 s in accordance with the Observation
Interface ICD (TW-ICD-001) Section 3.2, except during maintenance mode. Note: Each station's
transmission schedule is set in its station configuration. (TW-SYS-002) [MC]

# TW-SYS-008 as delivered: "Stations shall send observations to the data center over the
observation interface."
# Finding: 'send' with no threshold; and it repeats TW-SYS-002. The rewrite makes it the
interface-conformance
```

```
# requirement it was meant to be.

(U) The telemetry unit shall format each observation record sent to the data center in accordance
with the Observation Interface ICD (TW-ICD-001) Section 3.1. (TW-SYS-008) [MC]

# TW-SYS-012 as delivered: "Every station shall be serviceable from the service vessel."
# Finding: weak target verb 'be' ("serviceable" cannot be tested). The rewrite says what the crew
must be able
# to reach, from where, and when it does not apply.

(U) While moored alongside the service vessel, the moored buoy, for the maintenance function,
shall present its sensor mast and power system access panels within reach of the vessel's deck
crane in accordance with the Service Vessel Interface Description (TW-SVI-001) Section 2.4,
except in sea states above 4. Note: The deck crane reaches 6 m from the vessel's side.
(TW-SYS-012) [SR]

# TW-SEG-003 as delivered: "The power system shall report the battery state of charge."
# Finding: 'report' with no threshold, and no destination. The rewrite adds both, and an
accuracy.

(U) While in observing mode, the power system shall report the battery state of charge to the
data logger every 10 minutes with an accuracy of 5 % of rated capacity in accordance with the
Health Interface ICD (TW-ICD-002) Section 4.3, except during maintenance mode. (TW-SEG-003)
```

Look at TW-SYS-002 before and after:

```
Before: Each station shall transmit its observations to the data center on a fixed schedule.

After:  (U) While in observing mode, upon each scheduled transmission time, the telemetry unit
shall send the
        observation record to the data center within 60 s in accordance with the Observation
        Interface ICD
        (TW-ICD-001) Section 3.2, except during maintenance mode. Note: Each station's
        transmission schedule is
        set in its station configuration. (TW-SYS-002) [MC]
```

The rewrite added a state (observing mode), a trigger (each scheduled transmission time), a threshold (within 60 s), a reference down to the section (TW-ICD-001 Section 3.2) and an exception (maintenance mode). It also changed the specification target from “each station” to “the telemetry unit”, because the telemetry unit is the thing that sends. And it is now testable: a test engineer can write a pass criterion for it without inventing anything.

All five pass the strict check, with classification required:

```
perl "$KIT/tools/sysml/req-recipe.pl" --require classification --strict
"$KIT/examples/tidewater-v1/recipe-rewrites.txt" | tail -1
```

Warning: Longer is not the goal

The rewrites are longer than the originals, and that is not the point. The point is that every word is now doing a job: naming a state, a threshold, a reference. A long requirement full of vague words ("in a timely and appropriate manner as required") scores worse than a short one. If a rewrite gets long because it is trying to say two things, split it, as TW-SYS-001 must be split.

7.8 Who may change the words

Here the recipe meets the contract. Tidewater's requirements live in DOORS, and DOORS belongs to the prime. The government team cannot fix TW-STK-001 by editing its copy in the model, for the same reason it never edits the prime's Cameo model: the words are the prime's deliverable, and changing them unilaterally would make the government's copy disagree with the contract baseline (Section 6.6, Where the converter stops (p. 64)).

So in Tidewater, recipe findings on DOORS text become a discrepancy report to the prime. In sprint 5 Jim's story "Grade our requirements with the recipe" ended with DR-6, "weak verbs and missing parts in the DOORS text", which listed the thirteen requirements, the finding for each, and the five rewrites as suggested wording. The prime dispositions it like any other DR: accept the wording, propose its own, or explain why the original stands. When a later drop delivers new text, the checker grades it again, and the DR closes when the findings do.

Two consequences are worth stating plainly, because they are where teams go wrong:

- **Fix the source, not the copy.** If the model's copy of a requirement is edited to satisfy the recipe and DOORS is not, the drift check reports a new drift, and the team has created the problem it was trying to remove.
- **Requirements the government owns can be fixed directly.** If your program's stakeholder needs live in the government's own repository, the recipe findings on them become ordinary stories, fixed by merge request, and the recipe job can be made a gate for those files.

7.9 The recipe in SysML v2

A requirement in SysML v2 text is more than its sentence. The sentence stays, as the requirement's documentation, because the sentence is the contractual text. Around it, SysML v2 gives the recipe's parts a home the model can use:

Table 7.6: Where the recipe's parts live in SysML v2

Recipe part	SysML v2
Classification	an attribute, or a marking metadata (`@Marking`) on the requirement or its package
State, trigger, pre-conditions	an `assume constraint`: the conditions under which the requirement applies
Specification target	the requirement's `subject`
Allocated function	an `allocate` relationship, or a feature of the subject
Operative verb, target verb, objects	the documentation text (the sentence itself)
Modifications with a threshold	an attribute with a unit, and a `require constraint` that compares against it

Recipe part	SysML v2
Specification reference	the documentation text; optionally a dependency to the referenced document's element
Requirement ID	the `doorsId` attribute
Requirement tags	metadata (`@MC;`)

Here is TW-SYS-002 written that way. The requirement definition captures the *kind* of requirement (an observation latency limit on a telemetry unit), and the requirement usage gives the specific limit and carries the recipe sentence:

```
cat > Recipe.sysml <<'EOF'
package RecipeExample {
    private import ISQ::*;
    private import SI::*;

    metadata def MC { doc /* The recipe's tag: mission-critical. */ }
    part def TelemetryUnit { attribute latency : DurationValue; }

    requirement def ObservationLatency {
        subject telemetry : TelemetryUnit;
        attribute maxLatency : DurationValue;
        require constraint { telemetry.latency <= maxLatency }
    }

    requirement tw_sys_002 : ObservationLatency {
        @MC;
        attribute doorsId = "TW-SYS-002";
        attribute classification = "U";
        attribute :>> maxLatency = 60 [s];
        doc /* (U) While in observing mode, upon each scheduled transmission time, the telemetry
        unit shall send the
             * observation record to the data center within 60 s in accordance with the
             Observation Interface ICD
             * (TW-ICD-001) Section 3.2, except during maintenance mode. (TW-SYS-002) */
    }
}
EOF
perl "$KIT/tools/sysml/sysml.pl" check Recipe.sysml
```

The grammar checker accepts it. Now the recipe checker, which reads `.sysml` files directly, finding the requirement's documentation, its `doorsId`, the classification attribute and the tag metadata:

```
perl "$KIT/tools/sysml/req-recipe.pl" --require classification Recipe.sysml
```

```
TW-SYS-002  score 100  Recipe.sysml:14
    Classification             U
    State                      While in observing mode
    Trigger                    upon each scheduled transmission time
    ...
```

```
    Requirement ID              TW-SYS-002
    Requirement Tags            MC

requirements=1 findings=0 missing=0
```

Why bother with the constraint when the sentence already says "within 60 s"? Because the sentence is for people and the constraint is for the model. The constraint makes the threshold a value the model holds, with a unit, that an analysis can read, that a verification case can cite, and that a pipeline can compare against a design value. When the prime changes the threshold to 45 s, the change shows up in the model as one changed attribute, which the delta report can find, instead of as a changed word inside a paragraph.

Note: Tidewater does not do this yet, on purpose

Tidewater's converted requirements carry only the sentence and the `doorsId`, because that is what the DOORS export gives the converter, and because the DOORS text is the prime's. Adding subjects and constraints to the prime's requirements would be a government edit to the prime's deliverable. The pattern above is what the team does for requirements it owns, and what it proposed to the prime, in the model exchange agreement, as the target form for future drops. Treat it as the direction of travel, not week-one work.

7.10 The recipe in the pipeline

The model repository's pipeline runs the checker on every change, in the `recipe` job. It reads the model's requirement files (but not the DOORS conversion, which the trace job compares separately), writes a text report, and turns the findings into a Code Quality report so they appear on the merge request:

tools/gitlab/templates/gitlab-ci.yml

```
      metrics: vcrm.metrics

# RECIPE -- every requirement statement graded against the requirement recipe
(tools/sysml/req-recipe.pl): missing
# parts, weak verbs ("provide", "support"), passive voice, vague words. Reports only: the DOORS
text is the prime's,
# so findings go to the prime as a discrepancy report. Findings show in the MR's Code Quality
widget.
recipe:
  stage: trace
  <<: *model_rules
  script:
    - perl ci/job.pl recipe
  artifacts:
    when: always
    paths:
      - recipe.txt
    reports:
```

The job reports and never fails, because in Tidewater the findings are the prime's to fix. A program whose requirements are its own can make it a gate in two steps: first a ratchet (fail only

when a merge request adds a finding, compared with `develop`), then, once the count reaches zero, `--strict`.

You can produce the same reports by hand. The Code Quality format is what GitLab shows in the merge request widget; JUnit is what it shows in the Tests tab:

```
perl "$KIT/tools/sysml/req-recipe.pl" --format junit -o recipe-junit.xml drop1/tidewater.reqif
perl "$KIT/tools/sysml/req-recipe.pl" --format codequality -o recipe-cq.json
drop1/tidewater.reqif
grep -o 'tests="[0-9]*" failures="[0-9]*"' recipe-junit.xml
```

Thirty tests, thirteen failures: each requirement is one test, and a requirement with any finding fails. That is the view management sees in the Tests tab: “13 of 30 requirements have wording problems”, which is a sentence anyone can act on.

7.11 Habits

Checklist: Before you file or approve a requirement

- ☐ One sentence, one “shall”
- ☐ The subject is the thing the specification is about
- ☐ A concrete target verb: not provide, have, be, establish, be able to, support
- ☐ The object is something the subject can act on
- ☐ A threshold with a unit, when the verb is about performance
- ☐ References carry the document number and the section
- ☐ Exceptions stated, not implied
- ☐ Informational notes marked “Note:” and kept out of the obligation
- ☐ An ID, and the tags the project uses
- ☐ The checker’s summary line, before and after your change

7.12 Exercises

Exercise 7.1: Decompose by hand, then check

Take TW-SEG-003 as delivered (“The power system shall report the battery state of charge.”) and fill in the eighteen-part table by hand. Which required parts are present? Which optional parts would a test engineer need? Then run the checker with `--only TW-SEG-003` on the drop-1 DOORS export and compare.

Exercise 7.2: Split TW-SYS-001

Rewrite TW-SYS-001 as two requirements that each pass `--strict`. Put them in a text file, one per paragraph, with IDs TW-SYS-001a and TW-SYS-001b, and run the checker on it. What did you have to decide that the original left open?

Exercise 7.3: A need that can be traced

TW-STK-003 says mariners "shall be able to read the latest bulletin and observations from a public service." Rewrite it so it is verifiable without turning it into a system requirement. Hint: what would the program office accept as evidence that the need is met?

Exercise 7.4: Add a constraint

Extend `Recipe.sysml` with a second requirement, TW-SEG-003, as a `requirement def` with the power system as subject and a reporting period attribute with a unit, and a usage carrying the rewritten sentence from `recipe-rewrites.txt`. Grammar-check it and grade it.

Exercise 7.5: Write the DR

Draft the description of DR-6 for the prime: the thirteen requirements, the finding for each in one line, and the suggested wording for the five that have it. Keep it to one screen. Which findings would you ask the prime to fix first, and why?

7.13 Applying this to your project

- **Agree the recipe before you grade anything.** Put the pattern, the weak-verb list and the tag list in your conventions (`docs/conventions/` in a repository made by `kickoff.pl scaffold`). If your program already has a requirements style guide, map its rules onto the eighteen parts; the checker's `--tag` option adds your tags.
- **Grade the baseline first, as a report.** Run the checker on the requirements as they are today and publish the summary line. That number is your starting point, not a failure.
- **Fix at the source.** Requirements someone else owns go back to them as a discrepancy report; requirements you own get stories.
- **Ratchet, then gate.** Fail a merge request only when it adds a finding. Make the job strict when the count reaches zero.
- **Turn classification on.** On a real program, run with `--require classification` from the first day, so an unmarked requirement can never be merged.

CHAPTER

8

SysML v2 by example: a workbook

From what you know →

You know

drawing blocks, ports and activities on Cameo diagrams.

→ Here it is called

writing part, port, action and requirement definitions as text.

→ What is new

you type the model, and the grammar check tells you within a second whether it is valid.

Note: What you have, what you need

What you have: you can read SysML v2 text now (Chapter 6, SysML v2 for SysML v1 modelers (p. 57)). You know what a part def, a port and a satisfy relationship are, because you drew them for years in SysML v1.

What you need from this chapter: to *write* them. Every construct the Tidewater team writes, one at a time: what it is, what it was in SysML v1, a short Tidewater example you type in and check, what to notice, and an exercise. By the end you will build a small model that passes the same checks the pipeline runs.

This is a core chapter, and it is a workbook, not a reference. Do it with a terminal open. Budget two sessions: the first half (packages to verification cases) in week 2, the second half (states to views) in week 3. The plan in Chapter 13, The operating model (p. 196) gives you that time as learning stories; take it.

8.1 How to use this workbook

Every section has the same shape:

1. A sentence on what the construct is for, and what it was called in SysML v1.
2. A snippet to type. Each one is complete: a package you can check on its own. Type it rather than pasting it; typing is how the keywords get into your fingers.

3. The check. Every snippet goes through the same grammar checker the pipeline's `grammar` job runs.
4. What to notice: the two or three things that trip v1 modelers.
5. An exercise that changes the snippet.

Work in an empty folder. The snippets are written with `cat > file <<'EOF'`, which writes everything up to the `EOF` line into the file; you can equally open the file in Vim and type the lines between. `$KIT` is where the kit lives on your laptop (the survival kit, Chapter 9, The survival kit (p. 126), set it up).

8.1.1 The check loop

Before the first construct, learn what a mistake looks like, because you will make many. Here is a v1 habit written as v2 text, a "block":

```
cat > typo.sysml <<'EOF'
package Stations {
    block Hull { part ballast : Ballast; }
}
EOF
perl "$KIT/tools/sysml/sysml.pl" check typo.sysml
```

```
typo.sysml:2:11: error: unexpected 'Hull'; expected ':', '::>', ':=', ':>', ':>>', ';', '=',
'=>', ...
sysml: 1 file(s), 1 error(s)
```

Read it the way you would read a review comment. The file and position come first: line 2, column 11, which is the word `Hull`. The checker read `block` as a name it did not know and expected a punctuation mark after it; it found another name instead. The fix is the v2 keyword: `part def Hull`. The last line is the summary, and the exit status is 1, which is how a pipeline job knows it failed.

The same check runs from Vim. In a `.sysml` buffer `:make` runs the checker (or the project's lint, inside a model repository) and fills the quickfix list, so `:cnext` takes you from error to error (Chapter 10, Vim for models (p. 143)). Here it is headless, the way the book's tests run it, writing the quickfix list to a file so you can see it:

```
edit typo.sysml
silent make
call writefile(map(getqflist(), {_, e -> bufname(e.bufnr) . ':' . e.lnum . ': ' . e.text}),
'qf.txt')
```

```
cat qf.txt
```

Two rules for reading errors, which save hours:

- **Fix the first error first.** One missing brace can produce ten errors below it. Fix the first, check again.
- **Look one line up.** A missing `;` is reported where the next statement starts, which is usually the line after the real mistake.

8.1.2 Your own model, for comparison

Every section below ends by finding the same construct in Tidewater's real converted model, so you can see the textbook form next to what the converter wrote from the prime's Cameo model. Make the drop and convert it once now:

```
perl "$KIT/sim/tidewater-v1.pl" --drop 1 drop1 > /dev/null
perl "$KIT/bin/xmi2sysml.pl" --out tw drop1/tidewater.xmi
ls tw
```

The converter writes one folder per top-level package of the Cameo model: `tw/stations/Stations.sysml`, `tw/interfaces/Interfaces.sysml` and so on. Keep this folder; the sections below read it.

Exercise 8.1: Break it three ways

In `typo.sysml`, fix `block` to `part def` and check that it passes. Then make three mistakes one at a time and read each error: delete the `;` after `Ballast`, delete the closing `}` of the package, and misspell `part` as `prat`. Which error message points at the line that is actually wrong?

8.2 Packages, imports and visibility

A **package** is a namespace: a folder for names. In SysML v1 it was the Package in the containment tree. In v2 text it is the outermost thing in almost every file, and Tidewater has one top-level package per file.

```
cat > 01-packages.sysml <<'EOF'
package Stations {
    doc /* Everything that goes out to a site. */
    private import Types::*;
    public import Interfaces::ObservationPort;
    private part def Ballast;
    part def Hull { part ballast : Ballast; }
}
package Types { attribute def StationMode; }
package Interfaces { port def ObservationPort; }
EOF
perl "$KIT/tools/sysml/sysml.pl" check 01-packages.sysml
```

What to notice:

- **`import Types::*;`** brings every name in `Types` into scope, so you can write `StationMode` instead of `Types::StationMode`. `Interfaces::ObservationPort` imports one name. Prefer the narrow import in files you write: it makes dependencies visible in a merge request diff (Chapter 47, Keeping SysML v2 from becoming a spider web (p. 645)).
- **`private` and `public`** say whether a name is visible to whoever imports *this* package. `private import` keeps `Types` out of what `Stations` offers to others; `public import` passes `ObservationPort` on. Default to private. A chain of public imports is how a model quietly becomes a spider web.
- **`private part def Ballast`** is a part def nobody outside `Stations` can see. Use it for details that are nobody else's business.
- **`doc /* … */`** is documentation that belongs to the element, not a comment. It shows in generated documents. `//` comments do not.

8.2.1 In the Tidewater drop

```
grep -n "import" tw/stations/Stations.sysml
```

Every converted package starts with a list of `private import …::*` lines: wildcards, one per package it uses, including quoted names such as `Requirements::'System Requirements'::Functions::*` for packages whose v1 names had spaces. The converter imports widely because it cannot know which names each file really needs. The government team leaves these as delivered (hand-editing them would conflict with every future drop) and writes narrow imports only in the files it owns (Chapter 47, Keeping SysML v2 from becoming a spider web (p. 645)).

8.2.2 A common mistake

The missing semicolon after an import is the most common first error. Notice where it is reported:

```
printf 'package S {\n    private import Types::*\n    part def Hull;\n}\n' > imp.sysml
perl "$KIT/tools/sysml/sysml.pl" check imp.sysml
```

The mistake is on line 2; the error is on line 3, where the checker found `part` while still waiting for the `;`. Look one line up.

Exercise 8.2: Narrow the imports

Add a part def `Mooring` to `Stations` that uses `StationMode` as an attribute type. Change the wildcard import of `Types` to a narrow one. Check. What happens if you make the import `public` and import `Stations::*` from a fourth package: which names can that package see?

8.3 Names, documentation and comments

Three kinds of text can sit in a model file, and they are not the same. In SysML v1 the difference was hidden in the tool: the documentation field of a block, a comment shape on a diagram, a

note nobody exported. In text it is visible, and it matters, because generated documents and reviews treat the three differently.

```
cat > 03-names.sysml <<'EOF'
package 'Data Center' {
    doc /* The services that turn observations into an archive and a bulletin. */
    // A line comment: for the next reader of the file, not part of the model.
    comment about IngestService /* Duplicates are dropped here, not in quality control. */
    part def IngestService {
        doc /* Receives and de-duplicates observations. */
        port 'in';
        attribute 'records per hour' : ScalarValues::Integer;
    }
    part def QualityControl;
}
EOF
perl "$KIT/tools/sysml/sysml.pl" check 03-names.sysml
```

What to notice:

- **`doc /* … */`** is the element's documentation. It is model content: generated documents print it, the recipe checker reads requirement text from it, and a merge request that changes it is a model change.
- **`// …`** is a note to whoever edits the file next. It is not model content. The converter's TODO lines are `//` comments on purpose: they are instructions for a person, and they disappear when the person does the work.
- **`comment about X /* … */`** is a model comment attached to an element: v1's comment shape with an anchor line. Use it for review notes that should travel with the model; prefer `doc` for what the element *is*.
- **Quoted names** (`'Data Center'`, `'in'`, `'records per hour'`) let a name contain spaces or be a keyword. The converter uses them to keep v1 names unchanged. In files you write, prefer plain names: `DataCenter`, `recordsPerHour`. The conventions most SysML v2 projects use, and Tidewater's `docs/conventions/naming.md`, are UpperCamelCase for definitions and lowerCamelCase for usages.

Exercise 8.3: Document a decision

Add a `comment about QualityControl` recording why quality control flags observations instead of deleting them, and a `doc` that says what quality control is. Check. Then ask: which of the two would you want in the generated document a forecaster reads, and which only in the review?

8.4 Parts: definitions, usages, multiplicity

A **part def** is a kind of thing; a **part** is a use of it inside something else. In SysML v1 these were the Block and the part property typed by it. The v2 vocabulary says what v1 left implicit: definitions are reusable, usages are where they are used (Section 6.3, Five ideas that cover most of SysML v2 (p. 59)).

```
cat > 02-parts.sysml <<'EOF'
package Parts {
    part def SensorSuite;
    part def Battery;
    part def Cell;
    part def ObservingStation {
        part sensors : SensorSuite;
        part battery : Battery { part cells : Cell[4]; }
    }
    part def MooredBuoy :> ObservingStation {
        part spareCells : Cell[0..2];
    }
    part def StationSegment {
        part buoys : MooredBuoy[1..*];
    }
    part network : StationSegment;
}
EOF
perl "$KIT/tools/sysml/sysml.pl" check 02-parts.sysml
```

What to notice:

- **`:` means "is typed by".** `part sensors : SensorSuite;` reads "sensors, a SensorSuite".
- **`:>` means "specializes".** `MooredBuoy :> ObservingStation` is v1's generalization arrow: a moored buoy is an observing station and has everything one has, plus its own parts.
- **Multiplicity in brackets.** `[4]` exactly four, `[0..2]` up to two, `[1..*]` one or more.
- **A usage can have a body.** `part battery : Battery { part cells : Cell[4]; }` says this station's battery has four cells, without changing the `Battery` definition. In v1 that needed a new block or a redefinition in a property panel.
- **A usage at package level** (`part network : StationSegment;`) is a specific thing, not a kind. Tidewater uses these for the system context.

8.4.1 In the Tidewater drop

```
grep -n -A4 "part def MooredBuoy" tw/stations/Stations.sysml
```

The converter turned v1's generalization into `:>` and every part property into a part usage. The second line is a converter TODO: the v1 level stereotype on the block, which the team replaces with `@L3Station;` (Section 8.17, Metadata (p. 112), and practice lesson 10).

8.4.2 A common mistake

Leaving out the colon, which reads naturally in English ("part sensors SensorSuite"):

```
printf 'package P {\n    part def SensorSuite;\n    part def Station { part sensors SensorSuite;
}\n}\n' > colon.sysml
perl "$KIT/tools/sysml/sysml.pl" check colon.sysml
```

Exercise 8.4: A shore station

Add `part def ShoreStation :> ObservingStation` with a part `mast` of a new part def `Mast`, and give `StationSegment` a part `shoreStations` with multiplicity one or more. Check. Then draw the part tree in your head: how many levels deep is a cell in a buoy in the segment?

8.5 Redefinition and subsetting

When one definition specializes another (`:>`), it can narrow what it inherited. **Subsetting** (`:>` on a usage) says a feature is one of the inherited feature's values; **redefinition** (`:>>`) replaces an inherited feature, for example with a new default. In SysML v1 this was the redefinition and subsetting fields in a property's specification dialog, which few modelers ever opened. In v2 it is two characters, and you will see `:>>` in every Tidewater requirement that gives a limit a value.

```
cat > 02b-redefinition.sysml <<'EOF'
package Redefinition {
    private import ISQ::*;
    private import SI::*;
    part def Sensor { attribute samplePeriod : DurationValue; }
    part def SensorSuite {
        part sensors : Sensor[1..*];
        attribute samplePeriod : DurationValue = 10 [min];
    }
    part def Anemometer :> Sensor;
    part def BuoySensorSuite :> SensorSuite {
        part anemometer : Anemometer :> sensors;
        attribute :>> samplePeriod = 1 [min];
    }
}
EOF
perl "$KIT/tools/sysml/sysml.pl" check 02b-redefinition.sysml
```

What to notice:

- **`part anemometer : Anemometer :> sensors;`** says the buoy's anemometer is *one of* its sensors. The suite still has its general `sensors` collection; the anemometer is a named member of it.
- **`attribute :>> samplePeriod = 1 [min];`** replaces the inherited ten-minute default with one minute for buoys. Without `:>>` you would have declared a second, unrelated attribute with the same name, which lint reports as a duplicate.
- **Read the three symbols aloud** until they are automatic: `:` "is a", `:>` "is a kind of / is one of", `:>>` "replaces".

Exercise 8.5: A shore-station suite

Write `ShoreSensorSuite :> SensorSuite` with a `barometer : Barometer :> sensors` and a sample period of five minutes. Check. Then try leaving out the `:>>` and run the file through lint in a copy of the `lintdemo` project from Section 8.8, Attributes, values and units (p. 101). What does lint say?

8.6 Ports and interfaces

A **port def** says what can cross a boundary; an **interface def** says how two ports connect. In SysML v1 these were proxy ports typed by interface blocks with flow properties, and connectors on an IBD. v2 keeps the idea and drops the ceremony.

```
cat > 03-ports.sysml <<'EOF'
package Ports {
    item def ObservationRecord;
    port def ObservationPort { out item observation : ObservationRecord; }
    interface def ObservationLink {
        end transmitter : ObservationPort;
        end receiver : ~ObservationPort;
    }
    part def TelemetryUnit { port obsOut : ObservationPort; }
    part def IngestService { port obsIn : ~ObservationPort; }
    part def Network {
        part telemetry : TelemetryUnit;
        part ingest : IngestService;
        interface link : ObservationLink connect transmitter ::> telemetry.obsOut to receiver ::>
        ingest.obsIn;
    }
}
EOF
perl "$KIT/tools/sysml/sysml.pl" check 03-ports.sysml
```

What to notice:

- **Direction lives in the port def.** `out item observation` says observations leave through this port.
- **~ conjugates.** `~ObservationPort` is the same port seen from the other side: the `out` becomes `in`. One port def serves both ends; in v1 you either made two interface blocks or ticked "conjugated" in a dialog.
- **The interface usage names its ends.** `connect transmitter ::> telemetry.obsOut to receiver ::> ingest.obsIn` binds each end of the interface def to a real port. `::>` reads "is bound to" (strictly, references).
- **The dot is a path.** `telemetry.obsOut` is the `obsOut` port of the `telemetry` part.

8.6.1 In the Tidewater drop

```
grep -n -A3 "port def ObservationIF" tw/interfaces/Interfaces.sysml
```

The v1 interface block `ObservationIF`, with its flow property, became a port def with an `out item`. The v1 name kept its "IF" suffix: the converter preserves names so the prime's v1 ids and names still line up with the v2 text.

8.6.2 A common mistake

Drawing the connector the v1 way, with an arrow:

```
printf 'package P {\n    part def A { port p; }\n    part def N { part a : A; part b : A;
connection a.p -> b.p; }\n}\n' > arrow.sysml
perl "$KIT/tools/sysml/sysml.pl" check arrow.sysml
```

A connection needs the word `connect` and the word `to`: `connection connect a.p to b.p;`. The checker stops at the first dot because, without `connect`, it reads `a` as the connection's name.

Exercise 8.6: A health interface

Add a `HealthReport` item, a `HealthPort` port def with `out item health : HealthReport`, a `healthOut` port on the telemetry unit, a conjugated `healthIn` on the ingest service, and a second interface usage connecting them. Check.

8.7 Connections and flows

A **connection** joins two parts or ports; a **flow** says something moves along it. In v1 these were connectors and item flows on an IBD. In Tidewater the converter wrote connections for every v1 connector.

```
cat > 04-flows.sysml <<'EOF'
package Flows {
    item def ObservationRecord;
    part def DataLogger { out item record : ObservationRecord; }
    part def TelemetryUnit { in item record : ObservationRecord; }
    part def Station {
        part logger : DataLogger;
        part telemetry : TelemetryUnit;
        connection toTelemetry connect logger to telemetry;
        flow of ObservationRecord from logger.record to telemetry.record;
    }
}
EOF
perl "$KIT/tools/sysml/sysml.pl" check 04-flows.sysml
```

What to notice:

- **An `item def` is a thing that moves** (a record, a message, a fuel load). It replaces v1's signals and the types of flow properties.
- **`connection NAME connect A to B;`** joins two features. The name is optional but worth giving; a named connection can be traced, allocated and found in a diff.
- **`flow of T from A to B;`** is the item flow, with its type and both ends. Flows also connect actions (see Section 8.14, Actions, successions, forks and flows (p. 109)); the same keyword covers what v1 split between item flows and object flows.

8.7.1 In the Tidewater drop

```
grep -n "connection" tw/datacenter/DataCenter.sysml
```

Every v1 connector became a `connection … connect … to …`, named when the v1 connector had a name (`toQc`, `toArchive`). Some names are quoted, `ingest.'out'`, because `out` and `in` are SysML v2 keywords: a quoted name is how v2 uses a keyword as a name.

Exercise 8.7: Two flows

Add a `HealthReport` item that also flows from the logger to the telemetry unit. Do you need a second connection, or can two flows share the parts? Check your answer with the grammar checker, then argue it with a teammate: which is clearer to a reviewer?

8.8 Attributes, values and units

An **attribute** is a value a part has: a mass, a power rating, a mode. In v1 these were value properties typed by value types with units. In v2 the units come from the standard library's International System of Quantities (ISQ) and SI packages, so a power is a `PowerValue` and "40 W" is `40 [W]`.

```
cat > 05-values.sysml <<'EOF'
package Values {
    private import ISQ::*;
    private import SI::*;
    attribute def Power :> PowerValue;
    attribute def Energy :> EnergyValue;
    enum def StationMode { enum observing; enum degraded; enum maintenance; enum recovered; }
    part def SolarPanel { attribute ratedPower : Power = 40 [W]; }
    part def Battery {
        attribute capacity : Energy;
        attribute mass : MassValue = 12 [kg];
    }
    part def Station { attribute mode : StationMode = StationMode::observing; }
}
EOF
perl "$KIT/tools/sysml/sysml.pl" check 05-values.sysml
```

What to notice:

- **`attribute def Power :> PowerValue;`** makes a project value type that *is* an ISQ power. This is what replaced Tidewater's converter TODO for the v1 unit instance specifications (Section 6.6, Where the converter stops (p. 64)).
- **`= 40 [W]`** gives a default value with a unit. The brackets hold the unit; `W`, `kg`, `s` and `h` come from `SI`.
- **`enum def`** is v1's enumeration, and `StationMode::observing` names one literal.

8.8.1 What the grammar check does not see

The grammar checker reads the *shape* of the text. It does not know whether `PowerValue` exists. Misspell it, and the grammar check still passes:

```
sed 's/PowerValue;/SolarPowerValue;/' 05-values.sysml > 05-misspelled.sysml
perl "$KIT/tools/sysml/sysml.pl" check 05-misspelled.sysml
```

Inside a model repository, the project's lint resolves names, and it catches what the grammar missed. Make a one-file project and lint it:

```
mkdir -p lintdemo/model
cp 05-misspelled.sysml lintdemo/model/Values.sysml
printf 'title = Lint demo\n' > lintdemo/sysml.conf
perl "$KIT/tools/sysml/model.pl" --root lintdemo lint
```

```
lintdemo/model/Values.sysml:4:28: error: unresolved supertype 'SolarPowerValue'
lint: 1 file(s), 1 error(s), 0 warning(s)
elements=3 errors=1 warnings=0
```

This is why the pipeline runs both jobs: `grammar` for shape, `lint` for names. It is also why an AI assistant's output must go through both (Chapter 18, A local AI assistant on the laptop (p. 269)): an invented library name is perfectly good grammar.

8.8.2 In the Tidewater drop

```
grep -n -A2 "attribute def Power" tw/types/Types.sysml
```

The converter could not know which ISQ quantity a v1 value type meant, so it made `Power` a plain `Real` and left a comment telling a person to finish the job. That person writes `attribute def Power :> ISQ::PowerValue;`, exactly the form you typed above, and the unit TODO lines below it go away.

8.8.3 A common mistake

Writing the unit without brackets:

```
printf 'package V {\n    private import SI::*;\n    part def SolarPanel { attribute ratedPower =
40 W; }\n}\n' > unit.sysml
perl "$KIT/tools/sysml/sysml.pl" check unit.sysml
```

The unit is an expression in square brackets: `40 [W]`.

Exercise 8.8: A power budget's inputs

Give `Battery` a `stateOfCharge` attribute as a percentage (hint: the library has a `Real` you can constrain, or look for a ratio type, see Section 8.21, Reading the official library (p. 116)), and give `Station` an `averageLoad : Power`. Check with both the grammar checker and, in

a copy of `lintdemo`, the lint.

8.9 Expressions: the small language inside constraints

Constraints, calculations, default values and requirement limits are all **expressions**: arithmetic, comparisons, Boolean logic and navigation through the model. In SysML v1 they were opaque strings in OCL or plain English that no tool read, which is why Tidewater's converter could only quote them in a TODO line. In v2 they are part of the language, and the grammar checks them.

```
cat > 05b-expressions.sysml <<'EOF'
package Expressions {
    private import ISQ::*;
    private import SI::*;
    private import ScalarValues::*;
    part def Battery {
        attribute capacity : EnergyValue = 400 [W*h];
        attribute stateOfCharge : Real = 0.8;
        attribute usable : EnergyValue = capacity * stateOfCharge;
        attribute low : Boolean = stateOfCharge < 0.2;
        attribute mode : String = if low ? "degraded" else "observing";
    }
    part def Station {
        part batteries : Battery[2];
        attribute total : EnergyValue = sum(batteries.usable);
        constraint enough { total >= 500 [W*h] and batteries.stateOfCharge->forAll { in s; s >
        0.1 } }
    }
}
EOF
perl "$KIT/tools/sysml/sysml.pl" check 05b-expressions.sysml
```

What to notice:

- **Units compose.** `[W*h]` is watt-hours; the multiplication is inside the brackets.
- **Arithmetic and comparison** read as you would write them: `capacity * stateOfCharge`, `stateOfCharge < 0.2`. Boolean operators are words: `and`, `or`, `not`, `implies`.
- **`if c ? a else b`** is the conditional expression.
- **Navigation with dots reaches through multiplicity.** `batteries.usable` is the usable energy of *each* battery, and `sum(…)` adds them up. `->forAll { in s; s > 0.1 }` applies a condition to every value.
- **The grammar checks the shape, not the arithmetic.** It will not tell you that a power was compared with an energy; an execution engine, or a reviewer, will.

Exercise 8.9: A reserve

Add an attribute `reserve` to `Station`: the total usable energy minus one day of a 15 W load (`15 [W] * 24 [h]`), and a constraint that the reserve is positive. Check. Then write the same rule as one English sentence a reviewer could check by hand, and compare the two.

8.10 Requirements: definitions, usages, subjects and constraints

A **requirement def** is a kind of requirement; a **requirement** usage is a specific one. In SysML v1 a requirement was a stereotyped class with an id and a text. In v2 it can also say *what it constrains* (its subject) and *how* (its constraints), so the model can check it. The recipe chapter shows how the recipe's parts map onto this (Section 7.9, The recipe in SysML v2 (p. 87)).

```
cat > 06-requirements.sysml <<'EOF'
package Reqs {
    private import ISQ::*;
    private import SI::*;
    part def TelemetryUnit { attribute latency : DurationValue; }
    requirement def ObservationLatency {
        doc /* The telemetry unit sends each record within its latency limit. */
        subject telemetry : TelemetryUnit;
        attribute maxLatency : DurationValue;
        assume constraint { true }
        require constraint { telemetry.latency <= maxLatency }
    }
    requirement tw_sys_002 : ObservationLatency {
        attribute doorsId = "TW-SYS-002";
        attribute :>> maxLatency = 60 [s];
    }
    requirement tw_stk_002 {
        attribute doorsId = "TW-STK-002";
        doc /* Forecasters shall receive observations in time to use them in the next bulletin.
        */
    }
}
EOF
perl "$KIT/tools/sysml/sysml.pl" check 06-requirements.sysml
```

What to notice:

- **`subject`** is what the requirement is about: the recipe's specification target.
- **`require constraint { … }`** is the measurable part, a Boolean expression over the subject's values. **`assume constraint`** is the condition under which it applies (the recipe's state and pre-conditions); `true` here means always.
- **`attribute :>> maxLatency = 60 [s];`** redefines (`:>>`) the def's attribute with a value. One def, many usages, each with its own limit.
- **Tidewater's converted requirements look like `tw_stk_002`:** a doorsId and the text, no subject or constraint, because that is what DOORS carries. Both forms are legal; the first is the direction of travel.

8.10.1 In the Tidewater drop

```
grep -n "requirement TW_SYS_002" tw/requirements/Requirements.sysml
```

The converted requirement is one line: the name made from the DOORS id (dashes become underscores), the text as `doc`, and the DOORS attributes. No subject and no constraint, because the v1 model had none. This is the form your trace and recipe checks read today.

8.10.2 A common mistake

Writing the requirement sentence directly in the body, as you would in a document:

```
printf 'package R {\n    requirement r { The telemetry unit shall send the record. }\n}\n' >
req.sysml
perl "$KIT/tools/sysml/sysml.pl" check req.sysml
```

Text goes inside `doc /* … */`. Everything else in a body is model, and the checker reads it as model.

Exercise 8.10: The battery requirement

Write a requirement def `StateOfChargeReporting` with the power system as subject and a `reportingPeriod : DurationValue`, and a usage `tw_seg_003` with doorsId `TW-SEG-003` and a period of 10 minutes (`10 [min]`). Check.

8.11 Satisfy, verify and derive

These are the trace relationships: which design element **satisfies** a requirement, which verification case **verifies** it, and which requirement is **derived** from which. In v1 they were stereotyped dependencies drawn on requirement diagrams and maintained in matrices. In v2 they are statements in the text.

```
cat > 07-trace.sysml <<'EOF'
package Trace {
    private import RequirementDerivation::*;
    requirement tw_stk_002 { doc /* Timely observations. */ }
    requirement tw_sys_002 { doc /* Scheduled transmission. */ }
    part def TelemetryUnit;
    part def Station { part telemetry : TelemetryUnit; }
    satisfy tw_sys_002 by Station::telemetry;
    #derivation connection {
        end #original ::> tw_stk_002;
        end #derive ::> tw_sys_002;
    }
}
EOF
perl "$KIT/tools/sysml/sysml.pl" check 07-trace.sysml
```

What to notice:

- **`satisfy R by X;`** names the requirement and the element that satisfies it. Satisfy at the level that actually carries the responsibility, not lower; a hundred fine-grained satisfies nobody reviews is how the spider web grows back (Chapter 47, Keeping SysML v2 from becoming a spider web (p. 645)).
- **Derivation** is a connection from the standard library's `RequirementDerivation` package, with metadata (`#derivation`, `#original`, `#derive`) marking which end is which. This is

exactly what the converter writes for v1's deriveReqt, so you will read it in every Tidewater drop.
- **`verify`** lives inside a verification case's objective; the next section shows it.
- **Write each relationship once,** in the element that owns the responsibility. The matrices, the trace report and the VCRM are generated from these statements; never maintain the reverse direction by hand.

8.11.1 In the Tidewater drop

```
grep -n "satisfy TW_SYS_00[12]" tw/requirements/Requirements.sysml
grep -n -A3 "#derivation connection" tw/requirements/Requirements.sysml | head -4
```

The prime drew satisfies and derives in Cameo; the converter wrote each one as a statement, all in the `Requirements` package. Count them: that list *is* the v1 trace matrix, now reviewable line by line.

8.11.2 A common mistake

Forgetting the `by`:

```
printf 'package T {\n    requirement r;\n    part def Station;\n    satisfy r Station;\n}\n' >
sat.sysml
perl "$KIT/tools/sysml/sysml.pl" check sat.sysml
```

Exercise 8.11: Trace a new requirement

Add `tw_sys_013` ("store and forward") derived from `tw_stk_002` and satisfied by a new `DataLogger` part of `Station`. Check. Then predict what the pipeline's trace job would report for `tw_sys_013` (hint: is anything verifying it?).

8.12 Verification cases

A **verification def** describes how requirements are verified: the subject under test, the objective naming the requirements, and the verdict. In v1 a test case was an activity stereotyped TestCase with verify dependencies to requirements.

```
cat > 08-verification.sysml <<'EOF'
package Verification {
    private import VerificationCases::*;
    requirement tw_sys_002 { doc /* Scheduled transmission. */ }
    part def Station;
    verification def TestStationObservations {
        doc /* Sensors and transmission on a buoy and a shore station. */
        subject station : Station;
        objective { verify tw_sys_002; }
        return verdict : VerdictKind;
    }
```

```
    verification testShoreStation : TestStationObservations;
}
EOF
perl "$KIT/tools/sysml/sysml.pl" check 08-verification.sysml
```

What to notice:

- **`objective { verify R; }`** is where the verify relationship lives. One case can verify several requirements; list them all in its objective.
- **`return verdict : VerdictKind;`** uses the library's verdict (pass, fail, inconclusive, error). The VCRM job reads verdicts from the results file, not from the model (Section 8.21, Reading the official library (p. 116) shows where VerdictKind lives).
- **The verification method** (test, analysis, inspection, demonstration) is metadata or an attribute in Tidewater (`verificationMethod` from DOORS). The recipe's parts and the method together are what a test engineer needs.

8.12.1 In the Tidewater drop

```
grep -n -A8 "verification def TestStationObservations" tw/verification/Verification.sysml
```

One converted verification case verifies four requirements; the v1 test case had four verify dependencies, and the converter gathered them into the objective. There is no `subject` yet: v1 test cases did not say what they tested, so that is part of the team's verification work (Chapter 44, Verification as a closed loop (p. 613)).

8.12.2 A common mistake

Putting `verify` straight into the body:

```
printf 'package V {\n    requirement r;\n    verification def T { verify r; }\n}\n' >
verify.sysml
perl "$KIT/tools/sysml/sysml.pl" check verify.sysml
```

`verify` belongs inside `objective { … }`.

Exercise 8.12: Inspect the vessel access

TW-SYS-012 is verified by inspection. Write `InspectVesselAccess` with the service vessel as subject and an objective verifying `tw_sys_012`. Where would you record that it is an inspection, so the VCRM can show the method?

8.13 States

A **state def** describes modes and the transitions between them. In v1 this was a state machine owned by a block. It is the construct Tidewater's converter could not carry (Section 6.7, Writing what the converter could not (p. 65)), so the team writes it by hand.

```
cat > 09-states.sysml <<'EOF'
package Modes {
    attribute def LowBattery;
    attribute def CrewOnSite;
    state def BuoyModes {
        entry; then observing;
        state observing;
        state degraded;
        state maintenance;
        transition first observing accept LowBattery then degraded;
        transition first degraded accept CrewOnSite then maintenance;
        transition first maintenance then observing;
    }
    part def MooredBuoy { exhibit state modes : BuoyModes; }
}
EOF
perl "$KIT/tools/sysml/sysml.pl" check 09-states.sysml
```

What to notice:

- **`entry; then observing;`** is the initial state: where the machine starts.
- **`transition first A accept E then B;`** reads "from A, on receiving E, go to B". A transition without `accept` happens when the source state's behavior completes.
- **The triggers are types.** `LowBattery` is an attribute def here; in a fuller model it would be an item or signal the power system sends.
- **`exhibit state`** is how a part has a state machine. v1 owned the state machine inside the block; v2 says the part *exhibits* it.

8.13.1 In the Tidewater drop

```
grep -n "StateMachine" tw/stations/Stations.sysml
```

This is the converter's limit: a whole v1 state machine becomes one TODO line inside `MooredBuoy`. The snippet you typed is the start of its replacement; the full version, with the Storm mode drop 3 adds, is Bob's sprint 6 story (Section 6.7, Writing what the converter could not (p. 65)).

8.13.2 A common mistake

The v1 arrow again:

```
printf 'package M {\n    state def BuoyModes {\n        state observing; state degraded;\n
transition observing -> degraded;\n    }\n}\n' > state.sysml
perl "$KIT/tools/sysml/sysml.pl" check state.sysml
```

A transition reads as a sentence: `transition first observing then degraded;`, with `accept SIGNAL` between the two when an event triggers it.

Exercise 8.13: The Storm mode

Drop 3 adds a Storm mode: from observing on a storm warning, and back to observing when the warning ends. Add it, with two attribute defs for the triggers. Check. Then look at drop 3's delta report in Chapter 45, The prime's model changes: receive, assess, disposition (p. 619) and compare your model with what the prime drew.

8.14 Actions, successions, forks and flows

An **action def** is a function: inputs, outputs and the steps inside it. In v1 this was an activity with actions, pins, control flows and object flows. v2 keeps the semantics in a much smaller notation.

```
cat > 10-actions.sysml <<'EOF'
package Cycle {
    item def ObservationRecord;
    item def Bulletin;
    action def SampleSensors { out record : ObservationRecord; }
    action def CheckQuality { in record : ObservationRecord; out checked : ObservationRecord; }
    action def ArchiveObservations { in record : ObservationRecord; }
    action def PublishBulletin { in record : ObservationRecord; out bulletin : Bulletin; }
    action def ObserveCoast {
        out bulletin : Bulletin;
        action sample : SampleSensors;
        action check : CheckQuality;
        action store : ArchiveObservations;
        action publish : PublishBulletin;
        flow sample.record to check.record;
        first start then sample;
        first sample then check;
        first check then afterCheck;
        fork afterCheck;
        first afterCheck then store;
        first afterCheck then publish;
        flow check.checked to store.record;
        flow check.checked to publish.record;
    }
}
EOF
perl "$KIT/tools/sysml/sysml.pl" check 10-actions.sysml
```

What to notice:

- **Parameters are directed features.** `in record`, `out checked`: v1's pins and parameter nodes.
- **`first A then B;`** is a succession, v1's control flow. `start` is the built-in beginning of the action.
- **`fork afterCheck;`** is a fork node: after it, `store` and `publish` both run. A `join` node collects them back.
- **`flow A.x to B.y;`** is v1's object flow, between the output of one step and the input of another.

- This is the shape of Tidewater's ObserveCoast, and the IDEF0 plates are drawn from it (Chapter 11, Seeing the model: diagrams from text (p. 158)).

8.14.1 In the Tidewater drop

```
grep -n "first\|TODO" tw/behavior/Behavior.sysml | head -12
```

The converter carried the control flows as `first … then …` successions and the call actions as action usages, and left TODO lines for the fork, the pin and the object flow: exactly the three things your snippet wrote by hand.

8.14.2 A common mistake

```
printf 'package C {\n    action def A { action s; action c; first s -> c; }\n}\n' > succ.sysml
perl "$KIT/tools/sysml/sysml.pl" check succ.sysml
```

Successions use `then`, never an arrow.

Exercise 8.14: Join and finish

Add a `join` after `store` and `publish`, and a final step `logBulletin` that runs after the join. Route the bulletin out of `ObserveCoast` with a flow from `publish.bulletin`. Check.

8.15 Calculations, constraints and analysis cases

A **calc def** computes a value; a **constraint def** is a condition that must hold; an **analysis def** is a question asked of a subject, answered by calculation. In v1 these were constraint blocks, parametric diagrams and binding connectors, usually the least-maintained part of a Cameo model because the diagrams were so laborious. In v2 they are short.

```
cat > 11-analysis.sysml <<'EOF'
package Analysis {
    private import ISQ::*;
    private import SI::*;
    calc def DailyEnergy {
        in power : PowerValue;
        in hours : Real;
        return energy : EnergyValue = power * hours * (1 [h]);
    }
    constraint def EnergyBalance {
        in generation : PowerValue;
        in sunHours : Real;
        in load : PowerValue;
        generation * sunHours >= load * 24
    }
    part def PowerSystem {
        attribute generation : PowerValue;
        attribute load : PowerValue;
        constraint balance : EnergyBalance {
```

```
            in generation = PowerSystem::generation;
            in sunHours = 6.0;
            in load = PowerSystem::load;
        }
    }
    analysis def EnergyMargin {
        subject power : PowerSystem;
        objective { doc /* How much daily energy is left after the load. */ }
        return margin : EnergyValue;
    }
}
EOF
perl "$KIT/tools/sysml/sysml.pl" check 11-analysis.sysml
```

What to notice:

- **`return`** names a calc's result; the expression after = computes it.
- **A constraint def's body is a Boolean expression**, here Tidewater's energy balance, the expression the converter could only carry as a TODO line.
- **`constraint balance : EnergyBalance { in generation = …; }`** is v1's binding connectors, written as assignments of the constraint's parameters to the part's attributes.
- **An analysis def** frames the question; the kit does not evaluate it. Evaluating calcs and constraints needs an execution engine (the OMG pilot implementation, or a commercial tool), which Tidewater leaves for later (Chapter 17, The DevSecOps tools that earn their keep (p. 255)).

8.15.1 In the Tidewater drop

```
grep -n "TODO" tw/analysis/Analysis.sysml
```

The v1 constraint block's parameters came across; its expression did not, because v1 stored it as opaque English text. The TODO line quotes it, so the person writing the v2 constraint has it in front of them. Drop 5 changes that expression, and the TODO line changes with it: the drop-5 conflict in Chapter 45, The prime's model changes: receive, assess, disposition (p. 619).

Exercise 8.15: Use the calc

Write a calc usage inside `PowerSystem` that computes the daily load energy with `Daily Energy`, and rewrite the constraint to compare against it. Check. Which form would a reviewer find easier to verify by hand?

8.16 Allocations

An **allocation** says a function is performed by a part. In v1 it was the allocate dependency, often drawn in swimlanes. In Tidewater the converter carried every v1 allocation, at the definition level.

```
cat > 12-allocations.sysml <<'EOF'
package Allocations {
```

```
    action def SampleSensors;
    part def SensorSuite;
    allocate SampleSensors to SensorSuite;
    part def Station {
        part sensors : SensorSuite;
        action sample : SampleSensors;
        allocate sample to sensors;
    }
}
EOF
perl "$KIT/tools/sysml/sysml.pl" check 12-allocations.sysml
```

What to notice:

- **Definition-level** (`allocate SampleSensors to SensorSuite;`) says every sensor suite performs sampling. **Usage-level** (`allocate sample to sensors;` inside `Station`) says this station's sensors perform this station's sample step. Pick one level per project and say which in your conventions; mixing them is a classic source of the web.
- The IDEF0 plates read allocations as **mechanisms**: the part that performs a function (Chapter 11, Seeing the model: diagrams from text (p. 158)).

8.16.1 In the Tidewater drop

```
grep -n "allocate" tw/allocations/Allocations.sysml
```

All six v1 allocations, at the definition level, in their own package. This is what the IDEF0 plates read as mechanisms.

8.16.2 A common mistake

```
printf 'package A {\n    action def S; part def P;\n    allocate S -> P;\n}\n' > alloc.sysml
perl "$KIT/tools/sysml/sysml.pl" check alloc.sysml
```

```
allocate S to P;
```

Exercise 8.16: Allocate the cycle

Allocate `CheckQuality` to a `QualityControl` part def and `ArchiveObservations` to an `Archive` part def, at the definition level. Check. Then decide: should `PublishBulletin` be allocated to the data center or to the publisher inside it? Write down why.

8.17 Metadata

Metadata labels elements with project-specific information: level tags, markings, procurement data, the recipe's tags. In v1 these were stereotypes from a profile, the source of most of Tidewater's converter TODOs (Section 6.3, Five ideas that cover most of SysML v2 (p. 59)). In v2 a project defines a small set of metadata defs in one package and the pipeline checks their use.

```
cat > 13-metadata.sysml <<'EOF'
package Conventions {
    private import ScalarValues::*;
    metadata def L3Station { doc /* A deliverable station within a segment. */ }
    metadata def Procured {
        attribute supplier : String;
        attribute partNumber : String;
    }
}
package Stations {
    private import Conventions::*;
    part def DataLogger {
        @Procured { supplier = "Notional Logger Co."; partNumber = "LG-100"; }
    }
    part def ShoreStation { @L3Station; }
    #L3Station part def MooredBuoy;
}
EOF
perl "$KIT/tools/sysml/sysml.pl" check 13-metadata.sysml
```

What to notice:

- **`@Name;`** inside a body applies metadata; `@Name { attr = value; }` applies it with values.
- **`#Name`** in front of a declaration is the short prefix form of the same thing.
- **Level tags are checked.** In a model repository, `model.pl check` verifies every part def carries the right `@L` tag for its depth in the part tree (Chapter 10, Vim for models (p. 143), and practice lesson 10).
- **Keep the set small.** A metadata def per real need, defined once in `Conventions`, is the v2 answer to v1's stereotype sprawl.

8.17.1 In the Tidewater drop

```
grep -c "TODO stereotype" -r tw | grep -v ':0$'
```

Every v1 stereotype application became a TODO line, most of them level stereotypes in `Stations`. Replacing them with `@L` metadata is the bulk of the structure port in sprints 2 to 5, and the reason the pipeline's TODO count falls the way it does.

8.17.2 A common mistake

The v1 guillemets:

```
printf 'package S {\n    <<L3Station>> part def ShoreStation;\n}\n' > stereo.sysml
perl "$KIT/tools/sysml/sysml.pl" check stereo.sysml
```

`#L3Station part def ShoreStation;` or `@L3Station;` inside the body.

Exercise 8.17: A marking

Add a metadata def `Marking` with a `level : String` attribute and mark the `Stations` package `@Marking { level = "U"; }` (metadata can apply to a package too). Check. In a real program, who decides the value?

8.18 Use cases

A **use case def** describes what an actor achieves with the system. In v1, use case diagrams with actors and include/extend relationships. Tidewater's converter carried the use cases as skeletons and left the include as a TODO.

```
cat > 14-usecases.sysml <<'EOF'
package UseCases {
    part def ObservationNetwork;
    use case def QueryArchive { doc /* Look up observations by station and time. */ }
    use case def ReviewObservations {
        subject network : ObservationNetwork;
        actor forecaster;
        include use case queryArchive : QueryArchive;
    }
}
EOF
perl "$KIT/tools/sysml/sysml.pl" check 14-usecases.sysml
```

What to notice:

- **`subject`** is the system the use case is about; **`actor`** declares who takes part.
- **`include use case`** is v1's include relationship, written inside the including use case. This is exactly the hand fix for Tidewater's include TODO.

8.18.1 In the Tidewater drop

```
grep -n "use case def\|TODO uml:Include" tw/network/Network.sysml
```

The use cases came across as skeletons, with the include as a TODO line inside `ReviewObservations`. One line of the snippet above closes it.

Exercise 8.18: Read a bulletin

Add `ReadBulletin` with a mariner actor and the network as subject. Should it include `QueryArchive`? Decide, then check.

8.19 Variants

A **variation** declares alternatives; a **variant** is one of them. In v1, variants were usually separate copies of blocks or whole branches of the model, a major source of v1 complexity. Tidewater's telemetry unit can use a cellular or a satellite modem.

```
cat > 15-variants.sysml <<'EOF'
package Variants {
    part def Modem;
    part def CellularModem :> Modem;
    part def SatelliteModem :> Modem;
    part def TelemetryUnit {
        variation part modem : Modem {
            variant part cellular : CellularModem;
            variant part satellite : SatelliteModem;
        }
    }
    part shoreTelemetry : TelemetryUnit { part :>> modem = modem::cellular; }
}
EOF
perl "$KIT/tools/sysml/sysml.pl" check 15-variants.sysml
```

What to notice:

- **`variation part modem : Modem { variant … }`** says the modem is one of the listed variants.
- **A configuration chooses** by redefining the variation: the shore telemetry unit uses the cellular modem.
- The grammar check accepts the choice; whether `modem::cellular` names a real variant is a semantic question for a full validator. This is advanced: Tidewater uses it in the later telemetry trade study (Chapter 46, Systems engineering work, not pipeline work (p. 636)), not in the first twelve weeks.

Exercise 8.19: A hybrid

Add a `HybridModem` variant and a `buoyTelemetry` usage that picks the satellite modem. Check. What would the trade study need from each variant to compare them?

8.20 Views and viewpoints

A **viewpoint** states a stakeholder's concern; a **view** exposes the part of the model that answers it. In v1 the diagrams *were* the views, drawn by hand and maintained by hand. In v2 a view is defined in the text, and drawings are generated from it (Chapter 11, Seeing the model: diagrams from text (p. 158)).

```
cat > 16-views.sysml <<'EOF'
package Views {
    part def Network { part stations; part dataCenter; }
    part network : Network;
    viewpoint def ManagementViewpoint {
        doc /* What the program office needs to see: the segments and their status. */
    }
    view def PartTreeView {
        satisfy requirement vp : ManagementViewpoint;
    }
    view networkTree : PartTreeView {
```

```
        expose network::**;
    }
}
EOF
perl "$KIT/tools/sysml/sysml.pl" check 16-views.sysml
```

What to notice:

- **A viewpoint is a requirement on a view**: the view def `satisfy`s it.
- **The expose statement,** `expose network::**;`, puts the network and everything inside it into the view.
- Tidewater's drawings (part tree, trace, interconnection, packages, IDEF0) are generated by the kit's drawers, not yet from view usages. Defining views in the text is how a program records *which* views each stakeholder gets, which keeps view sprawl in check (Chapter 47, Keeping SysML v2 from becoming a spider web (p. 645)).

Exercise 8.20: A test view

Write a `TestViewpoint` for the test organization ("which requirements each verification case covers") and a view def and usage that satisfy it. What would you expose?

8.21 Reading the official library

Every snippet above imported something from the standard library: `ISQ`, `SI`, `VerificationCases`, `RequirementDerivation`, `ScalarValues`. The library is plain SysML v2 text, and the kit carries the official release under `tools/sysml/vendor`. When you want to know whether a type exists, or what it is called, read the library rather than guessing:

```
LIB="$KIT/tools/sysml/vendor/sysml-v2-release/sysml.library"
ls "$LIB"
ls "$LIB/Domain Libraries/Quantities and Units"
```

Find where a value type is defined:

```
LIB="$KIT/tools/sysml/vendor/sysml-v2-release/sysml.library"
grep -rn "attribute def PowerValue\|attribute def DurationValue" "$LIB/Domain
Libraries/Quantities and Units"
```

And the verdict a verification case returns:

```
LIB="$KIT/tools/sysml/vendor/sysml-v2-release/sysml.library"
grep -n "enum def VerdictKind" -A 8 "$LIB/Systems Library/VerificationCases.sysml"
```

What to notice:

- **Three folders.** `Kernel Libraries` (the foundations: scalar values, base types), `Systems Library` (parts, ports, requirements, verification cases, states, views), `Domain Libraries` (quantities and units, analysis, metadata, requirement derivation).
- **`ISQ` re-exports the quantity packages.** `ISQ.sysml` publicly imports `ISQBase`, `ISQMechanics`, `ISQThermodynamics` and the rest, which is why `private import ISQ::*;` was enough for `PowerValue` (mechanics) and `EnergyValue` (thermodynamics).
- **The library is the spec's own example.** Reading how `VerificationCases` is written teaches more about verification cases than any summary. From Vim, `:SysTags` builds a tags file that includes the library, so `Ctrl-]` on a library name jumps to its definition (Chapter 10, Vim for models (p. 143)).

Exercise 8.21: Find three types

Find, in the library, the value types for a temperature, a length and an electric current, and the unit symbols for degrees Celsius and metres. Use them in a `Thermometer` part def with an attribute `range` and a default value. Check with the grammar checker and with lint.

8.22 Reading a converted file, top to bottom

You have written each construct once. Now read a whole file the converter wrote, the way Bob did on day one and the way every reviewer of a drop merge request has to. The stations file is the biggest in drop 1. Start with its head:

```
sed -n '1,20p' tw/stations/Stations.sysml
```

```
// generated by xmi2sysml.pl from tidewater.xmi -- review before committing; TODO lines need a
person
package Stations {
    private import Analysis::*;
    private import DataCenter::*;
    private import Interfaces::*;
    private import Requirements::'System Requirements'::Functions::*;
    ...
    private import Types::*;
    doc /* Everything that goes out to a site: moored buoys and shore stations. */
    part def StationSegment {
        // TODO stereotype Tidewater_Profile:L2Segment on this element (xmi:id
        _22_1_7f3a0c1_a16b5d7b3d55) not converted
        doc /* All the observing stations of the network. */
        part buoys : MooredBuoy[1..*];
        part shoreStations : ShoreStation[1..*];
        port obsOut : ObservationIF;
        port healthOut : HealthIF;
        connection connect obsOut to buoys.obsOut;
        connection connect obsOut to shoreStations.obsOut;
    }
```

Read it in five passes, each looking for one thing. This is the habit that makes a 190-line file reviewable in ten minutes.

1. **The header line.** "generated by xmi2sysml.pl ... TODO lines need a person". A file with this line came from the converter; on the vendor branch it is never edited by hand, and on `develop` the government's edits to it are the TODO closures and nothing else (Section 6.6, Where the converter stops (p. 64)).
2. **The imports.** `Stations` imports `DataCenter`, and (you will find) `DataCenter` imports `Stations`. That pair is one of the two package dependency cycles in the v1 spider web (Chapter 2, The Cameo years (p. 10)); the cycle-break story in sprint 4 removes it.
3. **The definitions and their bodies.** Each `part def` reads as a parts list, its ports, then its connections: the block's BDD and IBD in one place (Section 8.4, Parts: definitions, usages, multiplicity (p. 96), Section 8.6, Ports and interfaces (p. 99)).
4. **The TODO lines.** One per v1 construct the converter could not carry, each naming the v1 type and its xmi:id, so a person can find it in Cameo. In this file almost all of them are level stereotypes.
5. **The documentation.** Every `doc` should say what the element *is*. Some say more than they should.

Look at the power system to see the last point:

```
grep -n -A7 "part def PowerSystem" tw/stations/Stations.sysml
```

```
49:    part def PowerSystem {
50-        // TODO stereotype Tidewater_Profile:L4Subsystem on this element (xmi:id ...) not
converted
51-        doc /* Solar generation, storage and regulation for a buoy. */
52-        doc
53-            /*
54-             * Per SSDD section 4.3 the power budget is TBD pending the 2024 trade study. Do
not change without CCB approval.
55-             */
```

Two documentation blocks. The first describes the element. The second is document prose pasted into Cameo during the port from documents: a reference to a section of a design document, a TBD from a trade study two years ago, and a process instruction. It is exactly the kind of clutter that made the v1 model a spider web, and it is now visible in a text diff instead of hidden in a property panel. It is also the prime's text: the government does not delete it from the converted file; it raises a discrepancy report, and the prime removes it in a later drop. If it were your own model, the decision would be yours: move the reference to a dependency, the TBD to an issue, and the process rule to `docs/conventions/`, and keep `doc` for what the element is.

Exercise 8.22: Review a file in five passes

Pick `tw/datacenter/DataCenter.sysml` and do the five passes. Write down: the packages it imports and which of them import it back, the number of TODO lines and their kinds, and any `doc` that says something other than what the element is. Which of your findings would go to the prime as a discrepancy report, and which are the government's to fix?

8.23 Putting it together: a model the pipeline accepts

The snippets above are each a package on its own. A real model is several packages in several files, under a `model/` folder, with a `sysml.conf` that tells the kit's model tool where the trees start. Here is a small Tidewater in seven files that passes every check the pipeline's `check` stage runs.

Make the project and its conventions:

```
mkdir -p mini/model
cat > mini/sysml.conf <<'EOF'
title      = Mini Tidewater
noun_root  = ObservationNetwork
verb_root  = ObserveCoast
level_tags = L1Network, L2Segment, L3Station
EOF
cat > mini/model/Conventions.sysml <<'EOF'
package Conventions {
    metadata def L1Network { doc /* The whole network. */ }
    metadata def L2Segment { doc /* A segment. */ }
    metadata def L3Station { doc /* A station or service. */ }
}
EOF
cat > mini/model/Interfaces.sysml <<'EOF'
package Interfaces {
    item def ObservationRecord { doc /* One timed set of measurements. */ }
    port def ObservationPort { out item observation : ObservationRecord; }
}
EOF
```

The structure, with level tags and one connection:

```
cat > mini/model/Network.sysml <<'EOF'
package Network {
    private import Conventions::*;
    private import Interfaces::*;
    part def ObservationNetwork {
        @L1Network;
        part stations : StationSegment;
        part dataCenter : DataSegment;
        connection link connect stations.obsOut to dataCenter.obsIn;
    }
    part def StationSegment {
        @L2Segment;
        part buoy : MooredBuoy;
        part shore : ShoreStation;
        port obsOut : ObservationPort;
    }
    part def DataSegment {
        @L2Segment;
        part ingest : IngestService;
        part archive : Archive;
        port obsIn : ~ObservationPort;
    }
    part def MooredBuoy { @L3Station; }
    part def ShoreStation { @L3Station; }
```

```
    part def IngestService { @L3Station; }
    part def Archive { @L3Station; }
}
EOF
```

The behavior, the requirements with their satisfies, and the verification cases:

```
cat > mini/model/Behavior.sysml <<'EOF'
package Behavior {
    private import Interfaces::*;
    action def Sample { out record : ObservationRecord; }
    action def Ingest { in record : ObservationRecord; }
    action def ObserveCoast {
        action sample : Sample;
        action ingest : Ingest;
        flow sample.record to ingest.record;
        first sample then ingest;
    }
}
EOF
cat > mini/model/Requirements.sysml <<'EOF'
package Requirements {
    private import Network::*;
    requirement tw_sys_002 {
        attribute doorsId = "TW-SYS-002";
        doc /* Each station shall transmit its observations to the data center on a fixed
        schedule. */
    }
    requirement tw_sys_003 {
        attribute doorsId = "TW-SYS-003";
        doc /* The data center shall ingest every observation it receives. */
    }
    satisfy tw_sys_002 by StationSegment;
    satisfy tw_sys_003 by DataSegment;
}
EOF
cat > mini/model/Verification.sysml <<'EOF'
package Verification {
    private import Network::*;
    private import Requirements::*;
    verification def TestStationObservations {
        subject segment : StationSegment;
        objective { verify tw_sys_002; }
    }
    verification def TestDataFlow {
        subject segment : DataSegment;
        objective { verify tw_sys_003; }
    }
}
EOF
```

Now run the four checks the pipeline runs on every merge request: grammar, lint, the outline and level-tag check, and the trace. The first command moves into the project folder; stay there for the rest of the section:

```
cd mini
perl "$KIT/tools/sysml/sysml.pl" check model/*.sysml
perl "$KIT/tools/sysml/model.pl" lint
```

```
perl "$KIT/tools/sysml/model.pl" check
```

```
perl "$KIT/tools/sysml/model.pl" trace
```

And see the part tree you wrote, drawn as text and as a picture:

```
perl "$KIT/tools/sysml/model.pl" nouns
perl "$KIT/tools/sysml/model.pl" draw tree -o tree.svg
```

```
ObservationNetwork
├── StationSegment
│   ├── MooredBuoy
│   └── ShoreStation
└── DataSegment
    ├── IngestService
    └── Archive
```

Tip: The outline rule bit Bob too

The first time Bob built this model, `model.pl check` reported one violation: the `Verification` package had a single member. The kit's outline rule wants two to nine children under every parent, the same rule of thumb a document outline follows (a section with one subsection is a sign the outline is wrong). Bob added the second verification case, which the model needed anyway. When the rule fires, ask what the outline is telling you before you reach for a waiver.

Exercise 8.23: Grow the mini model

Add the moored buoy's state machine from Section 8.13, States (p. 107) to `Network.sysml` (with `exhibit state` on `MooredBuoy`), add a third requirement satisfied by `Archive` and a verification case for it, and rerun all four checks. Then break the trace on purpose: remove the new verification case and run `model.pl trace` again. What does it report, and what would the pipeline's trace job do on a merge request?

8.24 Your port worklist

The workbook's constructs are not an abstract syllabus. Every converter TODO line in Tidewater's drop 1 is one of them, waiting to be written. Go back up to the folder where you converted the drop, and count the TODO lines by kind:

```
cd ..
grep -rhoE "// TODO (uml:[A-Za-z]+|stereotype [A-Za-z_]+:[A-Za-z0-9]+|constraint expression)" tw
\
  | sed 's|// TODO ||' | sort | uniq -c | sort -rn
```

```
     13 stereotype Tidewater_Profile:L5Assembly
     10 stereotype Tidewater_Profile:L3Station
      8 stereotype Tidewater_Profile:L4Subsystem
      3 uml:InstanceSpecification
      3 stereotype Tidewater_Profile:Procured
      3 stereotype Tidewater_Profile:L2Segment
      1 uml:StateMachine
      ...
```

And total them:

```
grep -rh "// TODO" tw --include=*.sysml | wc -l | tr -d ' '
```

That number is the port's starting burn-down: the pipeline's `todo` job counts the same lines on every change (Chapter 20, The pipeline, job by job (p. 302)). Here is where each kind is taught:

Table 8.1: Tidewater's drop-1 TODO lines and where the workbook teaches each fix

TODO kind in the drop	How many	Workbook section	Sprint the team did it
stereotype L1Network ... L6Component (level tags)	36	Section 8.17, Metadata (p. 112)	2 to 5
stereotype Procured	3	Section 8.17, Metadata (p. 112)	4
uml:InstanceSpecification (units)	3	Section 8.8, Attributes, values and units (p. 101)	3
uml:Include	1	Section 8.18, Use cases (p. 114)	3
uml:StateMachine	1	Section 8.13, States (p. 107)	6
uml:ForkNode, uml:InputPin, uml:ObjectFlow, uml:ControlFlow	4	Section 8.14, Actions, successions, forks and flows (p. 109)	6
uml:Constraint, constraint expression	2	Section 8.15, Calculations, constraints and analysis cases (p. 110)	later
uml:Interaction	1	(messages and occurrences, not in this workbook)	later
uml:Profile	1	replaced by the Conventions package, Section 8.17, Metadata (p. 112)	2

Read the table the way Jim read it at the sprint 1 estimate: two thirds of the port is one mechanical construct (level tags), which goes fast once one person has done it twice; the rest is a handful of real modeling problems, each a story of its own. That shape is typical of a v1 conversion, and it is why the burn-down falls steeply and then flattens.

Exercise 8.24: Your first TODO

Open `tw/network/Network.sysml` in Vim, find the level-tag TODO on `ObservationNetwork` with `/\/\/ TODO`, and write down the one line that replaces it and the import the package needs. (Practice lesson 10 has you do exactly this and checks it.)

8.25 What the workbook leaves out

The checks you ran are real, and they are what the pipeline runs, but be clear about what they prove:

- **The grammar checker** proves the text is well formed SysML v2, against the official grammar.
- **`model.pl lint`** resolves names and imports, finds duplicates and checks a set of structural rules.
- **`model.pl check`** applies the outline rule and the level tags.
- **None of them** evaluates constraints, checks that a redefinition is type-compatible in every case, or proves the model *means* what you intended. Full semantic validation needs the OMG pilot implementation (`model.pl validate`, with Java) or a commercial tool; meaning needs a reviewer (Chapter 22, Reviewing a model change (p. 357)).

The workbook also left out constructs Tidewater does not use yet: individuals and snapshots, occurrences and messages in detail, trade studies with full analysis cases, and the textual notation for diagrams. The library and the specification's examples, under `tools/sysml/vendor/sysml-v2-release/examples`, show all of them.

8.26 Applying this to your project

- **Give everyone the workbook time.** Two sessions per person, as learning stories with points, in the first three weeks. A team that never typed a part def will not review one well.
- **Keep the snippets.** The folder you worked in is a personal phrasebook. Copy from it; it is faster than the specification.
- **Check everything twice.** Grammar for shape, lint for names, in Vim with `:make` before you commit and in the pipeline after.
- **Start from your converted model.** The best exercises are your own converter TODO lines: each one is a construct from this workbook waiting to be written (Section 6.6, Where the converter stops (p. 64)).
- **Write your conventions down.** Which level of allocation, which metadata, which import style: one page in `docs/conventions/`, agreed once, checked by the pipeline where it can be.

PART II

Week zero

Part I ended on Friday 2 October 2026 with a signed scorecard, a one-page plan, an email that promised honesty, and a zip file from the prime contractor that nobody had opened yet. Part II is everything that has to be true before the first sprint can start on Monday, and everything a reader needs to know to follow the twelve weeks in Part III with a terminal open.

It is the part you type along with. Jim, Bob and Tom did not learn these things in a classroom. They learned them in the gaps of real work: Tom reading about protected branches at the kitchen table on a Saturday, Bob trying git commands in a throwaway folder on a Tuesday afternoon until the words "merge conflict" stopped being frightening, Jim rewriting the Definition of Done three times because the first two versions were documents nobody would read. The chapters here put those lessons in the order a newcomer needs them, which is not the order the team learned them.

Survival kit

What you have

a Windows laptop with Git for Windows and Vim; the agile kit checked out; a SysML v1 model you exported from Cameo (or, for this book, the Tidewater drops); years of systems engineering judgment.

What you need by the end of Part II: a working terminal and a set of commands you trust; enough git to make a branch, commit, push and resolve a conflict; Vim set up to edit and check SysML v2 text; the operating model (who does what, how big each piece of work may be, when the team meets); a GitLab project configured from a file; the DevSecOps tools worth turning on; a pipeline you can read job by job; and the daily rhythm of stand-ups, poker, refinement, kickoff, demo and retrospective.

The chapters build on each other:

- Chapter 13, The operating model (p. 196) describes how the work is organized: the project, tomes, epics, stories and tasks, with the size rule for each; the two-week cadence; who decides what. It comes first because every later chapter refers to it, and because it is the part a systems engineer will find most familiar. It is a work breakdown structure and a schedule, with a few new words.
- Chapter 9, The survival kit (p. 126) sets up the laptop. It is written for someone who has never opened a terminal on purpose.
- Chapter 12, Git from zero, for systems engineers (p. 171) teaches version control from zero in a practice repository, with every command shown and its real output, and each idea tied to the configuration management practice it replaces.
- Chapter 10, Vim for models (p. 143) turns the editor you already know into a model workbench: syntax, checks, jump lists, tags, merge conflicts and Markdown for GitLab.

- Chapter 15, Setting up GitLab (p. 226), Chapter 17, The DevSecOps tools that earn their keep (p. 255), Chapter 19, CI/CD from zero (p. 288), Chapter 20, The pipeline, job by job (p. 302), Chapter 23, Stand-ups, the journal and the board (p. 367) and Chapter 24, The rhythm: stand-ups, poker, refinement, kickoff, demo, retro (p. 377) set up the shared platform, explain continuous integration, walk through the pipeline file, and establish the daily rhythm.

Tip: If you are short of time

Read Chapter 13, The operating model (p. 196) and Chapter 9, The survival kit (p. 126) in full. Work through Chapter 12, Git from zero, for systems engineers (p. 171) with your hands on the keyboard; it is the chapter people most regret skipping. Skim Chapter 10, Vim for models (p. 143) and come back to each section when Part III first needs it.

A word about pace. None of this is hard, but much of it is new at once, and new things in bulk feel hard. The team found that an hour a day of deliberate practice in week zero was worth more than a full day of reading. Bob's rule, written on a sticky note on the monitor, was: *type it, break it, fix it, then read about it*. The exercises at the end of each chapter are built on that rule.

CHAPTER

9

The survival kit

From what you know →

You know

your laptop, Windows, the shared drive, and Vim.

→ Here it is called

the survival kit: Git Bash, Perl, the kit and the doctor check.

→ What is new

a terminal you type commands into, and a check that tells you exactly what is missing.

Survival kit

What you have

a government Windows 11 laptop with Git for Windows and Vim on it (both on the approved software list); a network account; a login to the enterprise GitLab server that the administrators created last week; and the agile kit, which Tom copied from the enterprise GitLab.

What you need

to open a terminal and not be afraid of it; about fifteen commands you can type without looking them up; git configured with your name; a key or token so GitLab knows who you are; Vim set up with the kit; and a list of what to do when something goes wrong. Budget half a day. The team spent most of Monday morning of sprint 1 on this, together, and it was the best half day of the sprint.

9.1 Monday, 08:00

Tom arrived early on the first day of sprint 1 with a printed page. It was a checklist, of course; Tom had run the program's configuration management database for ten years and trusted checklists the way other people trust coffee. At the top it said *Survival kit: every box ticked before stand-up*. There were nineteen boxes.

Bob read it over Tom's shoulder. "What's a terminal?"

"The black window," Tom said. "Git Bash. It came with Git for Windows. You've had it on your laptop for two years."

"I've never opened it."

"Neither had I, until Saturday."

That exchange is the starting point of this chapter. If you have opened a terminal and typed commands into it, you can skim the next three sections. If you have not, read them slowly and type everything. The terminal is the one new tool in this book you cannot avoid. Everything else (git, GitLab, the converters, the checks) is driven from it, and once it stops being strange, everything else gets much easier.

9.2 What is on the laptop

Every tool the team used in the twelve weeks was either already on the government laptop or arrived as plain files inside the kit. Nothing needed an administrator to install. That mattered: an installation request would have taken two weeks and an ISSM review, and the team had neither.

Table 9.1: The survival kit: what is on the laptop

Tool	What it is	How to check it	What the team uses it for
Git for Windows	a bundle: git, the Git Bash terminal, Perl, Vim, SSH, curl, Tcl/Tk and the usual Unix commands	`git --version`	everything below comes with it
Git Bash	the terminal window	it is the window you type into	running every command in this book
git	the version control program	`git --version`	branches, commits, merges, pushing to GitLab
Perl	a scripting language; the kit's tools are written in it	`perl -v`	running the kit, the converters and the checks
Vim	the editor	`vim --version`	editing SysML v2, Markdown, the journal
ssh, ssh-keygen	secure connections and keys	`ssh -V`	proving your identity to GitLab
curl	a program that talks to web servers	`curl --version`	the kit's GitLab client uses it
wish	Tcl/Tk, a small toolkit for windows	`which wish`	the live model viewer beside Vim
the agile kit	a folder of Perl scripts, Vim files and documentation	`perl "$KIT/agile.pl" help`	converters, checks, journal, GitLab tools
a web browser	Edge or Firefox		GitLab's web pages: issues, merge requests, pipelines

On Tom's laptop in October 2026 the versions were:

```
$ git --version
git version 2.55.0.windows.1
$ perl -v | sed -n 2p
This is perl 5, version 42, subversion 2 (v5.42.2) built for x86_64-cygwin-thread-multi
```

```
$ vim --version | head -1
VIM - Vi IMproved 9.2 (2026 Feb 14, compiled Jun 26 2026 07:32:44)
```

Your versions will differ. Anything from the last few years works; the kit is written to run on old and new versions alike.

Note: Why Perl?

The kit's tools are Perl scripts because Perl ships inside Git for Windows, which is already approved almost everywhere. That one fact removed an entire category of approval from the team's critical path. You will not write Perl in this book. You will run Perl scripts, the way you run any program: `perl` followed by the script's name and its arguments.

9.3 The terminal

A **terminal** is a window where you type instructions to the computer, one line at a time, and it types back. The program that reads your line and carries it out is called the **shell**; in Git Bash the shell is called *bash*. The words "terminal," "shell," "command line" and "Git Bash" are used almost interchangeably, including in this book.

A systems engineering analogy: a terminal is a procedure executed step by step, where each step is one line, you can see exactly what happened after every step, and the whole record stays on the screen. A graphical tool hides the steps behind clicks. The terminal shows them. That is why everything in DevSecOps is driven from it: a sequence of typed commands can be written down, reviewed, repeated exactly and, eventually, run by a machine.

To open Git Bash: Start menu, type "Git Bash", press Enter. You see something like this:

```
bob@TWLAPTOP03 MINGW64 ~
$
```

That is the **prompt**. It tells you who you are (`bob`), which computer you are on (`TWLAPTOP03`), and which folder you are in (`~`, which means your home folder). The `$` means "ready for a command." In this book, the `$` at the start of a line means "type what follows"; do not type the `$` itself.

Type `pwd` and press Enter:

```
$ pwd
/c/Users/bob
```

`pwd` means *print working directory*: which folder you are in. Every command runs "in" a folder, and many commands act on files relative to it.

A command line has three kinds of parts:

```
ls -l drop1
```

- `ls` is the **command**: the program to run.
- `-l` is an **option**: it changes how the command behaves. Options start with `-` (one letter) or `--` (a word): `-l` and `--report` are both options.
- `drop1` is an **argument**: what the command acts on, here a folder.

Five keys make the terminal pleasant:

Table 9.2: Five keys that make the terminal pleasant

Key	What it does
Up arrow	brings back the previous command, to run it again or edit it
Tab	completes a file or folder name: type `dr` then Tab and it becomes `drop1/`
Ctrl-C	stops the running command (it does not copy, in a terminal)
Shift-Insert, or right-click then Paste	pastes
`q`	leaves a screen that is showing a long file or help page (the "pager")

Warning: Ctrl-C does not copy

In Windows, Ctrl-C copies. In a terminal, Ctrl-C interrupts whatever is running. To copy text from Git Bash, select it with the mouse (it is copied as soon as you release the button); to paste, use Shift-Insert or right-click. This catches everyone once. When it caught Jim, it stopped a long-running conversion halfway, which was harmless; the command was simply run again.

9.4 Files, folders and paths

A **path** is the address of a file or folder. Git Bash writes paths with forward slashes, the way Unix computers do, and shows the Windows drive `C:` as `/c`:

Table 9.3: The same folder in Windows and in Git Bash

Windows writes	Git Bash writes
`C:\Users\bob\projects\agile`	`/c/Users/bob/projects/agile`
`C:\Users\bob` (your home folder)	`~` or `/c/Users/bob`

A path that starts with `/` or `~` is **absolute**: it means the same place wherever you are. A path that does not is **relative**: it is relative to the folder you are in. `drop1/MANIFEST.txt` means "the file `MANIFEST.txt` in the folder `drop1` inside the folder I am in now." Two special names help: `.` is the current folder and `..` is the folder above it.

Three habits save trouble:

- **Avoid spaces in names you create.** `drop1` and `prime-drop-v1.0` are easy to type; `Prime Drop v1.0` must be quoted every time (`"Prime Drop v1.0"`), and forgetting the quotes produces confusing errors. The kit and GitLab handle spaces correctly; your fingers will not.

- **Names starting with a dot are hidden.** `.git`, `.vimrc` and `.gitlab-ci.yml` are ordinary files and folders that `ls` does not show unless you ask with `ls -a`. Many of the most important files in this book start with a dot.
- **Line endings differ.** Windows ends each line of a text file with two invisible characters (carriage return and line feed, written CRLF); Unix and the kit use one (LF). Most tools cope with both, but not all. The prime's manifest files arrive with Windows line endings; the command `file` tells you which you have:

```
$ file drop1/MANIFEST.txt
drop1/MANIFEST.txt: ASCII text, with CRLF line terminators
```

Section 9.7, Setting up git (p. 136) shows the one git setting that stops line endings from changing silently.

9.5 The commands you need

Here are the commands the team used every week, each with a real example on the first prime drop. To follow along, create the drop the way the book does throughout, by asking the kit's generator for it (the real prime would deliver a zip file; the result is the same four files):

The kit lives in the folder `~/projects/agile` (Section 9.6, Getting the kit (p. 135) shows how to get it). Commands in this book call that folder `$KIT`, the same name the pipeline uses for it, so tell the shell once where it is, and make a folder to practise in:

```
export KIT=~/projects/agile
mkdir -p ~/w2s
cd ~/w2s
```

(`export` sets a **variable**, a name the shell replaces with its value wherever you write `$KIT`. It lasts until the window closes; Section 9.10, Shell shortcuts (p. 139) puts it in `~/.bashrc` so every new window knows it.) Then ask for the drop and convert it:

```
perl "$KIT/sim/tidewater-v1.pl" --drop 1 drop1
perl "$KIT/bin/xmi2sysml.pl" --out drop1-v2 drop1/tidewater.xmi
```

The first line writes the four files the prime would deliver; the second runs the converter, which turns the v1 model into SysML v2 text in a folder of its own. Part I showed what it produces (Chapter 6, SysML v2 for SysML v1 modelers (p. 57)). Now the commands.

9.5.1 Where am I, and what is here: pwd, ls, cd

```
$ pwd
/c/Users/bob/w2s
$ ls
drop1
```

```
drop1-v2
$ ls drop1
MANIFEST.txt
tidewater.csv
tidewater.reqif
tidewater.xmi
$ ls -l drop1
total 200
-rw-r--r-- 1 bob 197121    795 Oct  6 08:03 MANIFEST.txt
-rw-r--r-- 1 bob 197121   4624 Oct  6 08:03 tidewater.csv
-rw-r--r-- 1 bob 197121  66759 Oct  6 08:03 tidewater.reqif
-rw-r--r-- 1 bob 197121 122711 Oct  6 08:03 tidewater.xmi
```

`ls -l` shows the long form: permissions, owner, size in bytes, date and name. The XMI file is 122,711 bytes, about 120 KB. `cd FOLDER` moves into a folder; `cd ..` moves up one; `cd` alone goes home; `cd -` goes back to where you were.

9.5.2 Make, copy, move and remove: mkdir, cp, mv, rm

```
$ mkdir -p work/notes
$ cp drop1/MANIFEST.txt work/notes/
$ ls work/notes
MANIFEST.txt
$ mv work/notes/MANIFEST.txt work/notes/drop1-manifest.txt
$ ls work/notes
drop1-manifest.txt
$ rm work/notes/drop1-manifest.txt
```

`mkdir -p` makes a folder, including any missing folders above it. `cp` copies; `cp -r` copies a folder and everything in it. `mv` moves or renames. `rm` removes a file; `rm -r` removes a folder and everything in it.

Warning: rm has no recycle bin

A file removed with `rm` is gone. There is no undo and no recycle bin. Before you type `rm -r`, type `ls` with the same path and read what it would remove. Inside a git repository you are much safer, because anything that was committed can be brought back (Chapter 12, Git from zero, for systems engineers (p. 171)); that is one of the first comforts of version control.

9.5.3 Read a file: cat, less, head, tail

```
$ cat drop1/MANIFEST.txt
Tidewater system model -- prime drop v1.0 (drop 1)
CDRL: A003 System Model
Contractor: Northwind Marine Systems (fictional)
Delivered: 2026-10-02
Built by: Jenkins https://jenkins.nwm.example.invalid/job/tidewater-model-delivery/137/ (build
137)
Prime repository commit: 2c38530dd8cbd15849071d9e3c4a476cd1235229
Tools: Cameo Systems Modeler 2022x Refresh2; DOORS 9.7 ReqIF export; Teamwork Cloud 2024x
Delivery: CDRL file transfer to government approved storage
```

```
Changes: Initial delivery: the model at contract start.
Files (sha256):
  af17989626f37e8ec57e2891d9cfbb34bf190d46d9b07345a5bed3887cc8d9ad  tidewater.xmi
  d22edcbffe71f554f2b380ec2f97ca1d1165dd9c0892f11073bb3ec8f84bc577  tidewater.reqif
  4a70d12ea15f7f618295cf7967a5e7e094f4aa57474ffe845ee090e062862c58  tidewater.csv
$ head -3 drop1/tidewater.csv
"Object Identifier","Absolute Number","Object Level","Object Heading","Object Text","Verification
Method","Owner","Name"
"TW-STK-001","1","2","","The network shall provide marine weather observations for the coastal
region it covers.","","Network Office","Coastal coverage"
"TW-STK-002","2","2","","Forecasters shall receive observations in time to use them in the next
bulletin.","","Network Office","Timely observations"
```

The manifest says who built the drop (the prime's Jenkins server, build 137), from which commit of the prime's own repository, with which tools, and how it was delivered: the provenance a receiver checks before trusting the files (Section 45.9, The prime's Jenkins: a delivery interface (p. 630) explains the prime's side). `cat` prints a whole file; use it for short files. `less FILE` shows a long file one screen at a time: Space for the next page, `b` for back, `/word` to search, `q` to quit. `head -N` and `tail -N` show the first or last N lines.

9.5.4 Search: grep

`grep` finds lines that contain a piece of text. It is the command the team used more than any other, because a model in text can be searched like any other text:

```
$ grep -n TW-SYS-006 drop1/tidewater.csv
14:"TW-SYS-006","12","3","","The data center shall publish a marine bulletin twice a day, on a
fixed schedule.","Demonstration","Data Services","Bulletin"
$ grep -c TW- drop1/tidewater.csv
30
$ grep -rn "TODO uml:StateMachine" drop1-v2
drop1-v2/stations/Stations.sysml:42:        // TODO uml:StateMachine 'BuoyModes' (xmi:id
_22_1_7f3a0c1_7a13eb20b93a): no SysML v2 mapping in this converter
```

`-n` prints line numbers, `-c` counts matching lines instead of printing them, `-r` searches every file in a folder and the folders below it, and `-i` ignores upper and lower case. The quotes around `"TODO uml:StateMachine"` keep the two words together as one piece of text; without them, grep would look for `TODO` in a file called `uml:StateMachine`, fail, and print a confusing error. When in doubt, quote.

Compare the first line with what Bob used to do in the document era to find one requirement's text in the delivered CSV: open Excel, wait, use Find, wait. It is not that grep is faster (although it is); it is that the grep line can be written down, sent to someone else, and run again next quarter on the next drop, unchanged.

9.5.5 Find files and count lines: find, wc

```
$ find drop1-v2 -name '*.sysml' | wc -l
13
$ grep -r '// TODO' drop1-v2 | wc -l
```

```
52
$ wc -l drop1/tidewater.xmi
1195 drop1/tidewater.xmi
```

`find FOLDER -name 'PATTERN'` lists files whose names match; * in a pattern means "anything," and the quotes stop the shell from expanding it early. `wc -l` counts lines.

The second line is worth stopping on. It counts every line in the converted model that contains `// TODO`, which is the converter's mark for "I could not carry this; a person must." That number, 52, is the team's **conversion debt** for drop 1, the primary progress measure on the scorecard (Section 3.6, Defining measurable progress (p. 28)). It was computed with one line of shell, and from sprint 2 on the pipeline computes it the same way on every change.

9.5.6 Pipes and redirection: | and >

Two symbols let commands work together. The **pipe**, `|`, sends one command's output into the next command as its input: `find … | wc -l` means "list the files, then count the lines of that list." **Redirection**, `>`, sends a command's output into a file instead of the screen:

```
$ grep -r '// TODO' drop1-v2 > work/todo.txt
$ wc -l work/todo.txt
52 work/todo.txt
```

`>` replaces the file if it exists; `>>` adds to the end of it. A line of commands joined by pipes is called a **pipeline** in the shell. It is the same idea, on a small scale, as the GitLab pipeline in Chapter 20, The pipeline, job by job (p. 302): small steps, each doing one thing, connected so that the output of one is the input of the next.

9.5.7 Check a delivery: sha256sum

A **checksum** is a short fingerprint computed from a file's exact contents. Change one byte of the file and the fingerprint changes completely. The prime's manifest lists a SHA-256 checksum (the kind used throughout the government) for each delivered file, so the receiver can prove that what arrived is exactly what was sent:

```
$ sha256sum drop1/tidewater.xmi
af17989626f37e8ec57e2891d9cfbb34bf190d46d9b07345a5bed3887cc8d9ad *drop1/tidewater.xmi
$ cd drop1; grep 'tidewater\.' MANIFEST.txt | sha256sum -c; cd ..
tidewater.xmi: OK
tidewater.reqif: OK
tidewater.csv: OK
```

The first command computes the fingerprint of one file; compare it by eye with the manifest's first line. The second does the comparison for you: `grep` picks the three checksum lines out of the manifest, and `sha256sum -c` checks each file against its listed checksum. In the document era, receipt inspection of a CDRL delivery was a signature on a transmittal form. This is receipt

inspection that actually inspects. It is the first task of the first story Jim did in sprint 1 (Chapter 26, Sprint 1: A GitLab project the team can work in (p. 404)).

Note: On Linux, strip the carriage returns

Git for Windows' `sha256sum` reads the manifest's Windows line endings without complaint. On a Linux machine (or in WSL) the same command fails with `No such file or directory` for each file, because the invisible carriage return becomes part of the file name. Remove the carriage returns and the leading spaces first:

```
cd drop1
grep 'tidewater\.' MANIFEST.txt | tr -d '\r' | sed 's/^ *//' | sha256sum -c
```

Use this form in anything that may run on Linux, such as a pipeline job (Chapter 20, The pipeline, job by job (p. 302)); it works everywhere.

9.5.8 Ask for help: –help, man and the kit's help

Almost every command explains itself:

```
grep --help | less
git help commit
perl "$KIT/agile.pl" help xmi2sysml
```

The last one is the kit's own help, which is also available inside Vim as `:help agile` (Section 9.9, Setting up Vim (p. 138)):

```
$ perl "$KIT/agile.pl" help xmi2sysml
xmi2sysml.pl
    perl bin/xmi2sysml.pl --report FILE.xmi
    perl bin/xmi2sysml.pl --out DIR [--layout '{seg}/{pkg}.sysml'] [--check] FILE.xmi
    perl bin/xmi2sysml.pl --stdout FILE.xmi

A SysML v1 XMI export (Cameo / MagicDraw) to SysML v2 text, one file per
top-level package (DIR/SEGMENT/PACKAGE.sysml). --report: elements in by
xmi:type, ported vs remaining (metric A), stereotypes, the unmapped list.
```

Table 9.4: The commands you need, on one page

Command	Does	Example
`pwd`	prints the current folder	`pwd`
`ls`, `ls -l`, `ls -a`	lists files (long form; including hidden)	`ls -l drop1`
`cd`	changes folder	`cd drop1`, `cd ..`, `cd`
`mkdir -p`	makes a folder (and its parents)	`mkdir -p work/notes`
`cp`, `cp -r`	copies a file (a folder)	`cp drop1/MANIFEST.txt work/`
`mv`	moves or renames	`mv a.txt b.txt`

Command	Does	Example
`rm`, `rm -r`	removes a file (a folder); no undo	`rm work/todo.txt`
`cat`	prints a short file	`cat drop1/MANIFEST.txt`
`less`	pages through a long file (`q` quits)	`less drop1/tidewater.xmi`
`head -N`, `tail -N`	first or last N lines	`head -3 drop1/tidewater.csv`
`grep -n`, `-c`, `-r`, `-i`	finds lines containing text	`grep -rn "TODO" drop1-v2`
`find … -name`	finds files by name	`find drop1-v2 -name '*.sysml'`
`wc -l`	counts lines	`wc -l work/todo.txt`
`sha256sum`, `sha256sum -c`	computes or checks checksums	`sha256sum drop1/tidewater.xmi`
`file`	says what kind of file it is (and its line endings)	`file drop1/MANIFEST.txt`
`|`, `>`, `>>`	pipe; write to a file; add to a file	`grep -r TODO model | wc -l`

9.6 Getting the kit

The agile kit is the folder of tools this book uses. On Tidewater, Tom cloned it from the enterprise GitLab, where the program keeps an approved copy:

```
mkdir -p ~/projects
cd ~/projects
git clone https://gitlab.example.com/tools/agile.git
cd agile
```

(`git clone` copies a repository from GitLab to your laptop; Chapter 12, Git from zero, for systems engineers (p. 171) explains it.) Most teams get the kit the other way, as the slim MBSE zip that `START-MBSE.md` describes: the tools, templates, Vim files and practice lessons this book uses, without the kit's own development files. Unzip it into `C:\Users\you\projects` and it makes the same folder, `~/projects/agile`. The book works with either form, and always calls that folder `$KIT`.

Check that it works on your machine. The kit's `doctor` checks Perl, git and Vim, and runs a small self-test: two of the kit's own test suites, and the converter and grammar checker on the Tidewater sample drop:

```
$ perl "$KIT/tools/gitlab/kickoff.pl" doctor | grep -E 'self-test|grammar'
PASS  kit self-test tests/prelude.t: # all 223 passed
PASS  kit self-test tests/ledger.t: # all 109 passed
PASS  converter and grammar check: xmi2sysml: 560 elements in, 206 of 206 in scope ported, 12
TODO, 13 file(s), self-check ok
```

"All 223 passed" is a better answer to "does the tool work here?" than any installation guide: the checks ran on this laptop, with this Perl. The last line converted a sample drop and checked every file it wrote against the SysML v2 grammar. If a self-test line fails, stop and find out why

before going further; Section 9.12, When something goes wrong (p. 140) lists the usual causes. The team found one laptop with an old Git for Windows this way, on day one rather than in the middle of a sprint.

Tip: Full kit only: run the whole test set once

If you cloned the full kit rather than the slim zip, run every test suite once on the first day: `perl "$KIT/tests/run.pl"`. It takes a few minutes and prints one line per suite. The slim zip carries only the suites `doctor` runs.

Run `doctor` again without the `grep` once the rest of this chapter is done: it checks the whole laptop. It changes nothing; every line is PASS, WARN or FAIL, and every problem comes with the exact command that fixes it:

```
perl "$KIT/tools/gitlab/kickoff.pl" doctor
```

It checks Perl and the modules the kit uses, git and the settings below, Vim and the kit's Vim setup, curl, a short self-test of the converters, your SSH key, and whether GitLab answers. Run it on every laptop in week zero, and again whenever something odd happens.

9.7 Setting up git

Git records who made every change. Before your first change, tell it who you are, and set four other options the team agreed on. These settings are **global**: they apply to every repository on your laptop, and you set them once.

```
$ git config --global user.name "Bob"
$ git config --global user.email bob@example.invalid
$ git config --global init.defaultBranch main
$ git config --global core.editor vim
$ git config --global core.autocrlf false
$ git config --global --list
user.name=Bob
user.email=bob@example.invalid
init.defaultbranch=main
core.editor=vim
core.autocrlf=false
```

Table 9.5: The five git settings, and why

Setting	Why
`user.name`, `user.email`	Every commit records them. Use your real name and your work email, the same one as your GitLab account, so GitLab links your commits to you.
`init.defaultBranch main`	New repositories start with a branch called `main`, matching GitLab's default. Older git versions call it `master`, which causes confusion when the two meet.

Setting	Why
`core.editor vim`	When git needs you to write a message (a commit message, a merge message), it opens Vim.
`core.autocrlf false`	Git does not change line endings when files go in or out. The kit's files use LF; letting git rewrite them changes checksums and can make a baseline impossible to reproduce.

Warning: core.autocrlf is not optional on Tidewater

Git for Windows' installer offers to convert line endings automatically, and many laptops have it turned on. With it on, a file's bytes on the laptop differ from its bytes in the repository. Most of the time nobody notices. Then a baseline's checksum does not match on rebuild, and it takes a day to find out why. Tom found this on one of the three laptops in week zero. Set it to `false`, and leave line endings to the repository's own `.gitattributes` file, which the kit's templates provide.

9.8 Proving who you are to GitLab: SSH keys and tokens

When your laptop sends changes to GitLab (a **push**) or fetches them (a **pull** or **clone**), GitLab has to know who you are. There are two ways, and your enterprise administrators decide which you may use.

9.8.1 An SSH key

An **SSH key** is a pair of files: a private key, which stays on your laptop and is never shown to anyone, and a public key, which you give to GitLab. Anything signed by the private key can be checked with the public one. The analogy is a signature card on file at a bank: the bank holds a sample of your signature, and you hold the pen.

Make a key pair with one command (the `-C` text is just a label so you can recognize the key later):

```
$ ssh-keygen -t ed25519 -C "bob@example.invalid"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/c/Users/bob/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /c/Users/bob/.ssh/id_ed25519
Your public key has been saved in /c/Users/bob/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:K0dPmCKtUHGgAxhkNkqSrSfQOVbaJH5p7OqoeoCdXxg bob@example.invalid
```

Press Enter to accept the file name. When asked for a passphrase, type one: it protects the private key if the laptop is lost. Then print the public key and copy it:

```
cat ~/.ssh/id_ed25519.pub
```

It is one line starting `ssh-ed25519 AAAA…`. In GitLab's web page, open your avatar (top left), then *Preferences*, then *SSH Keys*, paste the line into the *Key* box, set an expiry date, and choose *Add key*. Test it:

```
ssh -T git@gitlab.example.com
```

GitLab answers `Welcome to GitLab, @bob!` and the connection closes. That is all it should do. (This is the one step in the chapter that needs your real GitLab server: the practice lessons and every other example here work offline.)

9.8.2 A personal access token

Some organizations block SSH and allow only HTTPS (the protocol web pages use). Then you prove your identity with a **personal access token**, a long random password created in GitLab for one purpose, with an expiry date. In GitLab: avatar, *Preferences*, *Access tokens*, *Add new token*. Give it a name ("Bob's laptop"), an expiry date (the enterprise maximum, often 90 days), and only the **scopes** (permissions) it needs: `read_repository` and `write_repository` for git. The kit's GitLab client (`gitlab.pl`, Chapter 15, Setting up GitLab (p. 226)) also needs `api`.

The first time git asks for a password over HTTPS, give your GitLab user name and paste the token as the password. Git for Windows includes a credential manager that stores it in Windows' own protected store, so you are not asked again until it expires.

Warning: Never put a token or a private key in a file you might commit

A token is a password. A private key is a password. Never paste either into a text file, a script, a Vim buffer you might save, a chat or an email. If one leaks, revoke it in GitLab at once (the same page that created it) and make a new one. The pipeline's secret detection job (Chapter 17, The DevSecOps tools that earn their keep (p. 255)) looks for exactly this mistake in every change, and the kit's token handling reads tokens from an environment variable, never from a file. The team's rule, from the ISSM's first review: *a secret that has been typed into anything other than its one proper place is a secret that has leaked*.

9.9 Setting up Vim

You already use Vim. The kit adds file types, syntax colors, commands and help for the files this book uses: SysML v2 models, stand-up notes, the journal, chat pastes. Turn it on by adding one line to your Vim settings file.

Open your Vim settings file (in Git Bash it is `~/.vimrc`; create it if it does not exist):

```
vim ~/.vimrc
```

and add these lines (adjust the path if you put the kit somewhere else):

```
syntax on
filetype plugin indent on
source ~/projects/agile/vim/scrum.vim
```

Save and quit (`:wq`), then start Vim again. Check that it worked, in Vim:

```
:help agile
```

The kit's help opens: the same text as `perl agile.pl help`, in Vim's own help format, with links you follow with Ctrl-] and come back from with Ctrl-T. Close it with `:q`. Now open a converted model file and check its type:

```
vim ~/w2s/drop1-v2/stations/Stations.sysml
```

```
:edit drop1-v2/stations/Stations.sysml
:set filetype?
```

Vim answers `filetype=sysml`. The kit's SysML settings are active: four-space indentation, `//` comments, folding by braces, and the `:Sys` commands and `\m` keys that Chapter 10, Vim for models (p. 143) teaches. Type `:Sys` and press Tab to see the commands cycle through: `:SysLint`, `:SysCheck`, `:SysSyntax`, `:SysTrace` and the others.

9.10 Shell shortcuts

Typing `perl ~/projects/agile/bin/daily.pl` every morning gets old. An **alias** is a short name for a longer command. Add these lines to `~/.bashrc` (Git Bash reads it every time it starts):

```
export KIT=~/projects/agile     # the kit, by the name the book and the pipeline use
alias daily='perl ~/projects/agile/bin/daily.pl'
alias agile='perl ~/projects/agile/agile.pl'
alias model='perl ~/projects/agile/tools/sysml/model.pl'
alias gitlab='perl ~/projects/agile/bin/gitlab.pl'
```

Close Git Bash and open it again: `echo $KIT` prints the kit's folder, and `daily status` now means the long form. The book always shows the long form, so you can see what is being run; use the short form in your own work.

9.11 GitLab in the browser

Most of the team's day happens in the terminal and in Vim, but GitLab's web pages are where the shared work is seen: issues, merge requests, pipelines, the board, the wiki. Log in to your organization's GitLab (for Tidewater, `https://gitlab.example.com`) with your network account.

On a project's page, the left sidebar holds everything this book uses. GitLab moves its menus around between versions, so look for the words rather than a position:

Table 9.6: Where things are in GitLab's web pages

Look for	What is there	First used in
Issues, *Issue boards*	stories, tasks, discrepancy reports; the board	Chapter 15, Setting up GitLab (p. 226)
Merge requests	proposed changes, reviews, approvals	Chapter 12, Git from zero, for systems engineers (p. 171)
Pipelines, *Jobs* (under *Build*)	the automated checks on every change	Chapter 19, CI/CD from zero (p. 288)
Repository, *Branches*, *Tags*, *Commits* (under *Code*)	the files and their history	Chapter 12, Git from zero, for systems engineers (p. 171)
Wiki	Definition of Ready and Done, glossary, runbooks	Chapter 13, The operating model (p. 196)
Milestones (under *Plan*)	the sprints	Chapter 15, Setting up GitLab (p. 226)
Pages, *Environments*, *Releases*, *Package registry* (under *Deploy*)	the model site, baselines, prime drops	Chapter 17, The DevSecOps tools that earn their keep (p. 255)
Settings	the project's rules (maintainers only)	Chapter 15, Setting up GitLab (p. 226)

One setting to change on day one: your notification level. By default GitLab emails you about everything in every project you belong to. Avatar, *Preferences*, *Notifications*: set the global level to *Participate* (you get email only for things you are involved in or mentioned in). Bob's inbox received 140 GitLab emails in the first two days before Tom showed Bob this.

9.12 When something goes wrong

Everything below happened to someone on the team in the first two weeks.

Table 9.7: When something goes wrong

You see	It means	Do this
`bash: notacommand: command not found`	the shell does not know that command: a typo, or the program is not installed	check the spelling; `which NAME` shows where a program is, if anywhere
`cat: drop1/nosuchfile.txt: No such file or directory`	the path is wrong for the folder you are in	`pwd` and `ls` to see where you are and what is there; use Tab to complete names

You see	It means	Do this
`fatal: not a git repository (or any of the parent directories): .git`	you ran a git command outside a repository	`cd` into the repository's folder
`Permission denied (publickey).`	GitLab did not accept your SSH key	check the public key is in your GitLab profile; `ssh -T git@gitlab.example.com` to test
`remote: HTTP Basic: Access denied`	your token is wrong or expired	make a new token; git will ask for it again
a screen of text and nothing responds to typing	you are in the pager (`less`)	press `q`
Vim and you cannot get out	you are in Vim	press Esc, then type `:q!` and Enter (quits without saving)
a command seems frozen	it is still running, or waiting for input	wait, or press Ctrl-C to stop it
`perl: warning: Setting locale failed`	a harmless language setting warning	ignore it, or add `export LC_ALL=C.UTF-8` to `~/.bashrc`
a checksum fails on a file you did not change	line endings were converted	`git config --global core.autocrlf false`, then check out the file again

9.13 The day-one checklist

This is Tom's page, as it ended up after the team had used it. Every box was ticked by 10:30 on Monday 5 October.

Checklist: Survival kit: every box ticked before you start

- ☐ Git Bash opens; `pwd` prints your home folder
- ☐ `git --version`, `perl -v`, `vim --version` all print a version
- ☐ the kit is in `~/projects/agile` (the slim zip or a clone)
- ☐ `echo $KIT` prints the kit's folder in a new Git Bash window (`~/.bashrc` exports it)
- ☐ `perl $KIT/tools/gitlab/kickoff.pl doctor` prints no FAIL lines
- ☐ `git config --global --list` shows your name, email, `init.defaultbranch=main`, `core.editor=vim`, `core.autocrlf=false`
- ☐ an SSH key exists and `ssh -T git@gitlab.example.com` says Welcome (or a token works over HTTPS)
- ☐ the key or token has an expiry date, and the date is in your calendar
- ☐ `~/.vimrc` sources the kit's `vim/scrum.vim`; `:help agile` opens in Vim
- ☐ a `.sysml` file opens in Vim with `filetype=sysml`
- ☐ aliases for `daily`, `agile`, `model` and `gitlab` work in a new Git Bash window
- ☐ you can log in to GitLab in the browser and see the `tidewater/model` project
- ☐ GitLab notifications are set to *Participate*
- ☐ you can make drop 1 and convert it, and count its TODO lines (52)

9.14 Applying this to your project

Copy: the checklist, the five git settings, the rule about secrets, and the habit of proving a tool works by running its own tests on your machine.

Adapt: the tool list to what your organization has approved. The essential capabilities are a terminal, git, a scripting runtime for the converters and checks, an editor you are fluent in, and a browser. If your laptops have Python approved and not Perl, the ideas in this book carry over; the commands do not.

Decide locally, before sprint 1: SSH or HTTPS tokens (ask your GitLab administrators and your ISSM together, once); where the approved copy of your tools lives and who updates it; and who on the team is the person others ask when the terminal does something strange. On Tidewater it was Tom, by default; name yours on purpose.

9.15 Exercises

Exercise 9.1: Find a requirement

Using only `grep` on drop 1's `tidewater.csv`, find every requirement whose verification method is Inspection. How many are there? Then find which converted `.sysml` file mentions `TW-SYS-012`.

Exercise 9.2: Count the debt by package

Using `grep -c` (which counts per file when given several files), print the number of `// TODO` lines in each converted file of drop 1. Which package holds most of the conversion debt? Why might that be? (Hint: look at what kind of TODO most of them are.)

Exercise 9.3: Break a checksum

Copy drop 1 to a new folder, change one character in the copy of `tidewater.csv` with Vim, and run the manifest check again. What does `sha256sum -c` print? What would you do if this happened to a real delivery?

Exercise 9.4: Make each mistake on purpose

Make three of the mistakes in Section 9.12, When something goes wrong (p. 140) deliberately (a misspelled command, a wrong path, a git command outside a repository) and read each error message aloud. The goal is to make the messages familiar while nothing is at stake.

CHAPTER

10

Vim for models

From what you know →

You know

Vim: opening files, searching, substituting and saving.

→ Here it is called

Vim for models: the kit's SysML keys, :make and the quickfix list.

→ What is new

the editor runs the model checks and jumps you to each error.

Survival kit

What you have

years of Vim. You can move, search, change, yank and put without thinking; you have a `.vimrc` with a few lines you copied from someone in 2009; and the kit is now sourced from it (Section 9.9, Setting up Vim (p. 138)).

What you need

to read a converted model comfortably; to turn the converter's TODO lines into a jump list and close them one by one; to run the model checks from inside Vim and land on each error; to jump to any definition or DOORS id by name; to resolve a merge conflict without leaving the editor; and to write commit messages, issue descriptions and merge request descriptions in Vim instead of a web text box.

10.1 The one thing they brought

When the directive arrived, the team's inventory of relevant skills was short. None of the three had written code. None had used git. But all three had used Vim for years, for the most unglamorous reasons: Jim to clean up DOORS exports before pasting them into Excel, Bob to fix the XML of a Cameo export that would not import, Tom to edit the CM database's load files with macros nobody else understood. In the Cameo years, Vim was the tool they used when the real

tools failed.

The parachute drop turned that around. In the new way the model *is* text, and the best tool any of them had for text was already in their hands. Bob put it this way at the end of sprint 2: "For three years the model was the one thing I couldn't edit in Vim. Now it's the only thing I edit in Vim."

This chapter is not a Vim tutorial. It assumes you can already edit text in Vim and shows what the kit adds, and which ordinary Vim features matter most for model work. Every command shown was run against the converted Tidewater drop.

10.2 What the kit adds

Sourcing the kit's `vim/scrum.vim` from your `.vimrc` (Section 9.9, Setting up Vim (p. 138)) does four things:

- **File types.** Files ending `.sysml` become type `sysml`; stand-up notes, the journal, pasted chat, drill sheets and IDEF0 models get their own types too. Each type has syntax colors and settings.
- **Commands and keys.** `:Sys…` commands and `\m` keys in SysML buffers; `:S…` commands and `\s` keys for the stand-up and the journal (Chapter 23, Stand-ups, the journal and the board (p. 367)); `:Idef…` for IDEF0 models.
- **A compiler.** `:make` in a SysML buffer runs the kit's model checks and loads every error into Vim's quickfix list.
- **Help.** `:help agile` and every topic under it, including `:help agile-vim-sysml` for this chapter's commands.

In a SysML buffer the kit sets these options (you can see each with `:set OPTION?`):

Table 10.1: What the kit sets in a SysML buffer

Option	Value	Effect
`filetype`	`sysml`	syntax colors for keywords, names, strings, comments
`shiftwidth`, `expandtab`	4, on	indentation is four spaces, never tabs (the converter's style)
`commentstring`, `comments`	`// %s`	`gq` and comment formatting know SysML comments
`iskeyword`	adds `-`	`TW-SYS-006` is one word for `*`, `w` and Ctrl-]
`foldmethod`	`syntax`	every `{ … }` block can be folded
`tags`	`./tags;,tags`	Ctrl-] finds the project's tags file from any subfolder
`makeprg`	the kit's `model.pl` for this project	`:make lint`, `:make check` run the model checks

The **leader** key is backslash unless you have changed it, so `\ml` means backslash, then `m`, then `l`, typed in normal mode. (Strictly, the SysML keys use the *local* leader, which is also backslash by default.)

10.3 Reading a model

A converted model is a folder of files, one per package. Open them all at once from the model's top folder:

```
cd ~/w2s
vim drop1-v2/*/*.sysml
```

(If you no longer have the converted drop from Section 9.5, The commands you need (p. 130), make it again in `~/w2s` first:)

```
mkdir -p ~/w2s
cd ~/w2s
perl "$KIT/sim/tidewater-v1.pl" --drop 1 drop1
perl "$KIT/bin/xmi2sysml.pl" --out drop1-v2 drop1/tidewater.xmi
```

Then move around with commands you mostly know:

Table 10.2: Reading a converted model

To	Type
go to the next file / previous file	`:next` / `:prev` (or `:n` / `:N`)
list the open files and jump to one	`:ls`, then `:b NUMBER` or `:b Stat` and Tab
see two files side by side	`:vsplit drop1-v2/datacenter/DataCenter.sysml`; Ctrl-W w moves between them
fold every block, to see only the definitions	`zM`; open one fold with `zo`, all with `zR`, toggle with `za`
find the next definition of any kind	`/def ` then `n`
jump from a brace to its partner	`%`
find every use of the name under the cursor	`*` (next), `#` (previous)
highlight all matches of the last search	`:set hlsearch` (`:noh` clears it)

Folding is the one the team found most useful. `Stations.sysml` is 180 lines; `zM` folds it to the list of its definitions, one line each, which reads like a block definition diagram's list of blocks. Open the one you care about with `zo`. Bob's habit, from the first week: open a file, `zM`, read the outline, then `zo` into the part that matters.

10.4 The TODO list as a jump list

Vim's **quickfix list** is a list of positions in files (file, line, column, message) that Vim can step through, opening each file at the right line. Compilers fill it with errors; `:vimgrep` fills it with search results. For model work it is the most valuable feature Vim has, because it turns any list of places into an action-item list where every item opens itself.

The converter marks everything it could not carry with a `// TODO` comment. Make them a jump list:

```
:vimgrep /\/\/ TODO/ drop1-v2/**/*.sysml
:copen
```

`:vimgrep /PATTERN/ FILES` searches every file and fills the quickfix list; `**` means "this folder and every folder below it." The pattern is `// TODO` with each slash escaped as `\/`, because `/` ends a Vim pattern. `:copen` opens the list in a window at the bottom. On drop 1 it holds 52 entries, the conversion debt.

Warning: Search for "// TODO", not "TODO"

`:vimgrep /TODO/` finds 65 lines, not 52. Every converted file starts with a header comment ("review before committing; TODO lines need a person") that contains the word TODO but is not a TODO line. The pipeline's count uses the same `// TODO` pattern as this chapter. If your count and the pipeline's disagree, check the pattern first.

Now work through the list:

Table 10.3: Moving through the quickfix list

To	Type
open the entry under the cursor in the list	Enter
go to the next / previous entry	`:cnext` / `:cprev` (`:cn` / `:cp`)
go to the first / last	`:cfirst` / `:clast`
close / reopen the list window	`:cclose` / `:copen`
see older lists (after running another search or check)	`:colder` / `:cnewer`

Map `:cnext` to a key you like. Bob used `]q` and `[q` (from a plugin Bob already had), Tom mapped F8. The kit leaves the choice to you.

10.5 Closing a TODO line: a worked edit

Thirty-six of drop 1's 52 TODO lines are the same kind: a stereotype from the v1 project profile (`Tidewater_Profile:L3Station` and its siblings) that the converter could not carry, because SysML v2 has no stereotypes. In SysML v2 the equivalent is metadata (Chapter 6, SysML v2 for SysML v1 modelers (p. 57)): a `metadata def` declared once, applied to an element with `@`. The government's plan was to declare the six level tags once, in a package of its own in the lowest layer of the model, `model/library/conventions/`, and to replace each TODO line with the matching tag.

To follow along, make a model repository the way week zero does (Chapter 15, Setting up GitLab (p. 226)): the kit's scaffold, with Tidewater's level names, then drop 1 converted into it with the layout map that puts each package in its layer (Chapter 16, The model repository: what goes where (p. 245)):

```
perl "$KIT/tools/gitlab/kickoff.pl" scaffold tidewater-model --levels
L1Network,L2Segment,L3Station,L4Subsystem,L5Assembly,L6Component
cd tidewater-model
```

```
perl "$KIT/sim/tidewater-v1.pl" --drop 1 drops/v1.0
perl "$KIT/bin/xmi2sysml.pl" --out model --layout-map "$KIT/examples/tidewater-v1/layout.map"
drops/v1.0/tidewater.xmi
```

First, the declarations. They live in `model/library/conventions/Conventions.sysml`, which the scaffold writes from the level names it was given; on Tidewater Bob wrote the first version by hand in Vim, and it reads:

```
package Conventions {
    doc /* Metadata definitions used across the model. */
    metadata def L1Network { doc /* Level 1 of the part tree. */ }
    metadata def L2Segment { doc /* Level 2 of the part tree. */ }
    metadata def L3Station { doc /* Level 3 of the part tree. */ }
    metadata def L4Subsystem { doc /* Level 4 of the part tree. */ }
    metadata def L5Assembly { doc /* Level 5 of the part tree. */ }
    metadata def L6Component { doc /* Level 6 of the part tree. */ }
}
```

Then each TODO line. One at a time, in the Network package, the line

```
        // TODO stereotype Tidewater_Profile:L1Network on this element (xmi:id
        _22_1_7f3a0c1_70e9bef7d9f2) not converted
```

becomes

```
        @L1Network;
```

That is `cc` and typing, for one line. For thirty-six lines, use a substitution. In the file you are editing:

```
:%s/\/\/ TODO stereotype Tidewater_Profile:\(L\d\w*\) on this element.*$/@\1;/
```

Read it in pieces: `%s` substitutes on every line of the file; the pattern matches the TODO comment, capturing the level name (`L` followed by a digit and more word characters) inside `\(` and `\)`; `.*$` swallows the rest of the line; the replacement is `@`, the captured name (`\1`) and a semicolon. The indentation in front of the comment is not part of the match, so it stays. Add `c` at the end (`…/@\1;/c`) to confirm each change one by one; the team always did, the first time in each file.

In the repository just made, the whole edit for the Network package, including the import line described next, is four Vim commands (practice lesson 10, `perl $KIT/agile.pl practice 10`, walks through the same edit with a checker that tells you when it is right):

```
:edit model/physical/network/Network.sysml
:%s/\/\/ TODO stereotype Tidewater_Profile:\(L\d\w*\) on this element.*$/@\1;/
:/^package Network {/put ='    private import Conventions::*;'
:write
```

Finally the package must be able to see the new definitions. Add one import line at the top of the package, next to the converter's imports:

```
package Network {
    private import Conventions::*;
    private import DataCenter::*;
```

Check the result with the grammar checker, from Vim (next section) or the terminal:

```
$ perl "$KIT/tools/sysml/sysml.pl" check model/library/conventions/Conventions.sysml
model/physical/network/Network.sysml
sysml: 2 file(s), 0 error(s)
```

One TODO line closed is one fewer in the pipeline's count. Two weeks of such edits, in small merge requests, is how the structural TODOs of drop 1 were closed in sprints 2 to 5.

Warning: A misspelled tag is not caught by lint

While testing this chapter's edit, a deliberately misspelled tag (`@L1Netwrk;`) passed both the grammar check and the kit's lint. The grammar check checks form, not names, and lint resolves the *types* of parts and attributes but not metadata names. The outline check (`\mk`, Section 10.6, Checking from Vim (p. 148)) does compare level tags with the part tree, but only tags it recognizes, so it reported nothing either. Use the substitution (which copies the name from the converter's own TODO line, so it cannot be misspelled), or Ctrl-N completion, rather than typing tag names. A wrongly *placed* tag is caught: putting `@L2Segment;` on `ObservationNetwork` makes the outline check report `ObservationNetwork: tagged L2, first reached at L1`.

10.6 Checking from Vim

The kit runs the same checks the pipeline runs (Chapter 20, The pipeline, job by job (p. 302)), from inside Vim, with every finding loaded into the quickfix list. Run them before you commit; then the pipeline never tells you anything you did not already know.

Table 10.4: Model checks from Vim

Key	Command	Checks	Scope
`\mx`	`:SysSyntax`	the official SysML v2 grammar (`sysml.pl check`)	this file
`\ml`	`:SysLint`	grammar, unresolved types, imports, duplicates (`model.pl lint`)	the whole model

Key	Command	Checks	Scope
`\mk`	`:SysCheck`	the outline rule (2 to 9 children per parent) and level tags (`model.pl check`)	the whole model
`\mg`	`:SysGate`	the security gate: markings, trust zones, threats (Chapter 17, The DevSecOps tools that earn their keep (p. 255))	the whole model
`\mv`	`:SysValidate`	a full compile with the SysML v2 Pilot implementation, if Java is installed	the whole model

Each saves the file first, runs the check, and opens the quickfix window if there are findings. `:make lint` and `:make check` do the same as `\ml` and `\mk`.

Here is a real finding. Change the type of the shore station's mast from `Mast` to `Mastt` (a typo) and press `\ml`. Lint prints:

```
model/physical/stations/Stations.sysml:47:21: error: unresolved type 'Mastt'
lint: 14 file(s), 1 error(s), 0 warning(s)
elements=148 errors=1 warnings=0
```

and the quickfix list holds one entry: `model/physical/stations/Stations.sysml:47:21`, error, `unresolved type 'Mastt'`. Enter on it puts the cursor on the `M` of `Mastt`, line 47, column 21. Fix it, press `\ml` again, and the list is empty. The last line, `elements=148 errors=0 warnings=0`, is the line the pipeline's lint job reads; anything other than zero errors and zero warnings fails the job.

A syntax error (here a missing semicolon after `part mast : Mast`) looks like this from `\mx`:

```
model/physical/stations/Stations.sysml:48:5: error: unexpected '}'; expected ',', '.', ':', '::',
'::>', ':=', ':>', ':>>', ...
sysml: 1 file(s), 1 error(s)
```

Notice that the error is reported on line 48, the line *after* the mistake: the parser only discovers the semicolon is missing when it meets the closing brace. Grammar errors often point just past the real problem. Look one line up.

The outline check, `\mk`, reports something different: not errors in what is written, but shape. On the raw drop 1 conversion it reports eighteen violations, such as `25 package Stations: members` (a package with 25 direct members, more than the nine the outline rule allows) and `1 part def Battery: parts` (a definition with only one part). Each violation goes into the quickfix list at the line of the part def or package it is about (a folder's at the first line of its first file), so `:cnext` walks the shape findings the same way it walks errors. These are not mistakes. They are the spider web of Chapter 2, The Cameo years (p. 10) measured in another way, and closing them is part of the port's later stories (Chapter 30, Sprint 5: Grading the requirements, and a trace that defends itself (p. 475)).

10.7 Tags: jumping by name and by DOORS id

A **tags file** is an index of where every name is defined. With one, Ctrl-] on any name jumps to its definition in whichever file holds it, and Ctrl-T jumps back. You may have used tags for code; the kit builds them for models.

Press `\mt` (`:SysTags`) in any SysML buffer:

```
tags: wrote 217 tags to tags (in vim: Ctrl-] on a name, Ctrl-T to go back)
```

Now put the cursor on `ObservingStation` anywhere and press Ctrl-]; Vim opens `model/physical/stations/Stations.sysml` at the definition. Press Ctrl-T to come back. The tags file also indexes requirements by their **DOORS identifier**, so Ctrl-] on `TW-SYS-006`, in a requirement, a satisfy line, an issue description you are editing in Vim or a pasted chat, opens the requirement. You can also type the jump directly:

```
:tag TW-SYS-006
```

(Because the kit adds `-` to `iskeyword`, the whole id is one word for Ctrl-] and `*`.)

Rebuild the tags with `\mt` after a merge brings in new definitions. The file is called `tags` at the top of the project; list it in the model repository's `.gitignore`, since everyone builds their own.

10.8 Seeing the model

Text is the source, but people also need to see the structure. The kit can show it without leaving Vim:

Table 10.5: Seeing the model from Vim

Key	Command	Shows
`\mn`	`:SysNouns`	the part tree, in a split
`\mb`	`:SysVerbs`	the function (action) tree, in a split
`\mr`	`:SysTrace`	regenerates the trace matrix and opens `docs/Traceability.md`
`\md`	`:SysDocs`	regenerates everything in `docs/`
`\mw`, `\mq`, `\mi`, `\mp`	`:SysDraw tree`, `trace`, `ibd`, `pkg`	an SVG drawing in the current folder
`\mW`	`:SysView`	the live viewer window beside Vim
`\mf`, `\mo`		show the name under the cursor in the viewer; open the file and line last clicked in it

The part tree of the converted drop 1, from `\mn` (the first lines):

```
Observation Network
├── Station Segment
│   ├── Moored Buoy
```

```
|   |   ├── Sensor Suite
|   |   |   ├── Anemometer
|   |   |   ├── Barometer
|   |   |   └── Thermometer
|   |   ├── Data Logger
|   |   ├── Telemetry Unit
|   |   |   ├── Modem
|   |   |   └── Antenna
|   |   ├── Hull
|   |   |   ├── Sensor Mast
|   |   |   └── Ballast
```

This needs a `sysml.conf` file at the project's top that names the roots of the trees (`noun_root = ObservationNetwork`, `verb_root = ObserveCoast`); the model repository has one from the first commit (Chapter 15, Setting up GitLab (p. 226)).

`\mW` starts the **live viewer**, a separate window (built with the Tcl/Tk toolkit that ships in Git for Windows) that draws the part tree, trace, internal block diagram or package diagram and redraws whenever you save. Put it on the second monitor. `\mf` on a name highlights that element in the drawing; clicking an element in the drawing and pressing `\mo` in Vim opens the file at its line. It is the closest thing to Cameo's diagrams the new way has, with the difference that the drawing is always generated from the text and never edited.

Note: The trace from Vim and the trace in the pipeline

`\mr` runs the kit's model-side trace, which reads only the model. On drop 1 it reports six requirements "not satisfied," and only one of them, TW-SYS-012, really is: it treats a requirement satisfied by a port or an attribute (TW-SYS-008, satisfied by `ObservingStation.obsOut`, and likewise TW-SYS-009, TW-SYS-010, TW-SYS-014 and TW-SEG-003) as unsatisfied. The pipeline's trace job uses the DOORS export as well and counts it correctly. Use `\mr` for a quick look while editing, and the pipeline's report as the number of record.

10.9 Merge conflicts in Vim

This section belongs with git, which the next chapters teach: Section 12.11, Merge conflicts (p. 180) explains what a merge conflict is and where one comes from. If you are reading in order, skim it now and come back after Chapter 12, Git from zero, for systems engineers (p. 171); if you already know git, here is how to resolve a conflict quickly in Vim.

First, one setting. Tell git to show the common ancestor in conflicts as well as the two sides:

```
git config --global merge.conflictstyle diff3
```

With it, the vendor-branch conflict that Section 12.19, The vendor branch (p. 191) walks through looks like this:

```
package Stations {
    part def Buoy {
```

```
<<<<<<< HEAD
        @L3Station;
||||||| 7e52f56
        // TODO stereotype Tidewater_Profile:L3Station on this element (xmi:id
        _22_1_7f3a0c1_0215) not converted
=======
        // TODO stereotype Tidewater_Profile:L2Segment on this element (xmi:id
        _22_1_7f3a0c1_0215) not converted
>>>>>>> vendor/prime
        doc /* A moored buoy. */
```

The middle section, between `|||||||` and `=======`, is the line as it was before either side changed it. Now the conflict explains itself: the team replaced the L3 TODO with a tag; the prime changed the level from L3 to L2. The right resolution is the team's form with the prime's level, `@L2Segment;`. Without the middle section you would have to work that out from the history. For vendor merges, where every conflict is "our edit versus their change," the team found the ancestor indispensable.

Resolving in plain Vim. Open the file and find the markers:

```
/^[<|=>]\{7\}
```

That pattern matches any line starting with seven of `<`, `|`, `=` or `>`, so `n` steps through every marker in the file. At each conflict, edit the lines into the right result and delete the marker lines with `dd`. When no markers are left (the search finds nothing), save, then in the terminal `git add FILE` and `git commit`.

Resolving with vimdiff. For a file with several conflicts, git can open Vim with each version side by side:

```
git config --global merge.tool vimdiff
git mergetool
```

Vim opens four windows: LOCAL (yours), BASE (the ancestor) and REMOTE (theirs) across the top, and MERGED (the file you are fixing) below. In the MERGED window, `]c` and `[c` jump to the next and previous difference, and

```
:diffget LOCAL
:diffget REMOTE
:diffget BASE
```

copy that side's version of the current difference into MERGED. Edit by hand when neither side is right. `:wqa` saves and closes everything, and git marks the file resolved. (`git mergetool` leaves `.orig` backup files; add `*.orig` to the model repository's `.gitignore`, or run `git config --global mergetool.keepBackup false`.)

Tip: Check before you commit a resolution

A resolved conflict is a hand edit, so check it like one: `\mx` on the file and `\ml` on the model before `git commit`. A stray `=======` line left behind by a hurried resolution is a grammar error, and `\mx` catches it before anyone else sees it.

10.10 Commit messages, issues and merge requests in Vim

Everything you write for GitLab is plain text, and you can write all of it in Vim. Commits arrive in Chapter 12, Git from zero, for systems engineers (p. 171) and merge requests in Chapter 15, Setting up GitLab (p. 226); this section shows how Vim handles the text they need, so you have it ready when they come.

Commit messages. With `core.editor` set to `vim` (Section 9.7, Setting up git (p. 136)), `git commit` opens Vim with Vim's built-in `gitcommit` file type: the subject line turns a warning color past 50 characters, and the second line is flagged if it is not blank. Write the message (Section 12.7, Commit messages a CCB would accept (p. 176)), `:wq`.

Issue and merge request descriptions. GitLab's descriptions and comments are **GitLab-flavored Markdown**, plain text with light markup: `#` headings, `-` lists, `- [ ]` checkboxes, `|` tables, three backticks around code. The model project keeps templates for each kind of issue and merge request in `.gitlab/issue_templates/` and `.gitlab/merge_request_templates/`. Here is the story template the team filled in for every story:

GitLab Markdown: tools/gitlab/templates/.gitlab/issue_templates/Story.md

```
<!-- Story: something you can show at the sprint review, like a small deliverable.
     Story: a slice of value that fits in ONE sprint, made of tasks of less than a week each.
     Title: a verb and an object, e.g. "Close the Stations TODO lines". The journal id is
     TW-<this issue's number>. -->

## Why

<!-- The outcome for the program in one or two sentences: who gains what. -->

## What "done" looks like

<!-- Acceptance criteria as a checklist. Each line can be checked by someone who did not do the
work. -->
- [ ]
- [ ] The pipeline is green on the MR (grammar, lint 0/0, outline, docs current)
- [ ] The trace job shows no new orphan, unverified or drifted requirement

## Tasks

<!-- One task issue per line, each under 5 points (one point = one ideal day). Create them with
the Task template
     and reference them here; they close through their merge requests. -->
| Task | Points | Who |
|---|---|---|
| # | | @ |

## Notes
```

```
<!-- Links, decisions, the v1 elements involved (xmi:id), the DOORS ids involved. -->

<details><summary>Definition of Ready (check before the sprint takes it)</summary>

- [ ] Fits in one sprint and has an epic (~"epic::...") and a tome (~"tome::...")
- [ ] Acceptance criteria above are testable
- [ ] Tasks are cut, each at most 5 points
- [ ] Dependencies on the prime or another story are linked (`/relate #N`, or "blocked by")

</details>

/label ~"type::story" ~"workflow::ready" ~"priority::2"
/milestone %"Sprint "
/assign @
/weight
```

Copy the template, fill it in with Vim, and create the issue from the file with the kit's GitLab client (Chapter 15, Setting up GitLab (p. 226)):

```
cp .gitlab/issue_templates/Story.md /tmp/story.md
vim /tmp/story.md
perl "$KIT/bin/gitlab.pl" issue new "Close the Network TODOs" \
    --template /tmp/story.md --label type::story --milestone "Sprint 3"
```

`gitlab.pl mr new … --template FILE` does the same for a merge request description. Bob preferred this to GitLab's web text box from the first week: Vim's editing, spell checking (`:setlocal spell`), and no lost text when the browser session times out.

For Markdown files, set the type and teach Vim's Markdown syntax to color code blocks:

```
:set filetype=markdown
let g:markdown_fenced_languages = ['sysml', 'sh', 'yaml', 'perl']
```

(Put the `let` line in your `.vimrc`.) Then a SysML snippet inside three backticks with `sysml` after them is colored as SysML inside the Markdown.

Pulling command output into what you are writing. `:r !COMMAND` runs a command and inserts its output below the cursor. In a merge request description:

```
:r !git log --oneline develop..HEAD
:r !git diff --stat develop
```

inserts the list of commits on your branch, or the files changed, exactly. The reviewer sees facts, not a summary.

10.11 The terminal inside Vim

You do not have to leave Vim to run git or the kit.

Table 10.6: The terminal from inside Vim

To	Type
run one command and see its output	`:!git status`
run a command on the current file	`:!git diff %` (`%` is the current file's name)
open a terminal in a Vim window	`:terminal` (Ctrl-W N to scroll its output; `exit` to close)
insert a command's output into the buffer	`:r !git log --oneline -5`
filter lines through a command	select lines, then `!sort`

Tom's working layout, by sprint 3: Vim full screen, the model file on the left, `:terminal` on the right for git and the pipeline runner, the quickfix list at the bottom. Jim kept a separate Git Bash window; Bob switched between the two as the task required. There is no right answer.

10.12 The stand-up and the journal from Vim

The kit was built for running a team's daily rhythm from Vim, and the same Vim that edits the model runs the stand-up. Jim, as the person who runs the meeting, used these from the first day of sprint 1; Chapter 23, Stand-ups, the journal and the board (p. 367) and Chapter 24, The rhythm: stand-ups, poker, refinement, kickoff, demo, retro (p. 377) show them in use.

Table 10.7: The stand-up keys used most

Key	Command	Does
`\sw`	`:STown`	the meeting layout: pasted chat on the left, the lint table and the replies on the right
`\sp`		paste the meeting chat and check everyone's status lines
`\sy`		copy the replies to the clipboard, to paste back into the chat
`\sn`	`:SNew`	today's stand-up file
`\sc`	`:SCompile`	compile the stand-up into the journal

`:help agile-vim-keys` lists them all.

10.13 A card for the monitor

Table 10.8: The model workbench on one card

In a SysML buffer	Does
`\mx`	grammar check, this file
`\ml`	lint the model; errors to quickfix
`\mk`	outline and level tags
`\mt`	rebuild tags; then Ctrl-] / Ctrl-T
`\mn`, `\mb`	part tree, function tree
`\mr`	trace (model side)
`\mw`, `\mq`, `\mi`, `\mp`	draw tree, trace, IBD, packages
`\mW`, `\mf`, `\mo`	live viewer; focus; jump back

In a SysML buffer	Does
`:vimgrep /\/\/ TODO/ model/**/*.sysml`	the conversion debt as a jump list
`:copen`, `:cnext`, `:cprev`	the jump list
`/^[<|=>]\{7\}`	find conflict markers
`:diffget LOCAL`, `REMOTE`, `BASE`	take a side in `git mergetool`
`:r !git log --oneline develop..HEAD`	your branch's commits, into the text
`zM`, `zo`, `zR`	fold all, open one, open all

10.14 Applying this to your project

Copy: the quickfix habit (every list of places becomes a jump list); running the pipeline's checks from the editor before committing; tags for names and requirement ids; `merge.conflictstyle diff3`; writing GitLab text in the editor.

Adapt: if your team does not use Vim, the same ideas exist in other editors (a problems list, go-to-definition, a merge editor). What matters is that the editor runs the same checks as the pipeline, from the same commands, so the pipeline's verdict is never a surprise.

Decide locally: key mappings for the quickfix list; whether generated files (tags, drawings) are committed or ignored; and a shared `.vimrc` fragment for the team's conventions, kept in the model repository so new members get it with their first clone.

10.15 Exercises

Exercise 10.1: The debt as a jump list

Convert drop 1 and load its TODO lines into the quickfix list. Step through the first ten. Group the 52 lines by kind (level stereotype, procured part, unit, activity construct, other): how many of each? Which kind would you close first, and why?

Exercise 10.2: Close the level tags in one package

In a converted copy of drop 1, close every level-tag TODO line in `Stations.sysml` with one substitution (confirm each), add the `Conventions` package and the import, and check with `\mx` and `\ml`. How many TODO lines are left in the file, and what kinds are they?

Exercise 10.3: Break it three ways

In the same copy, make three mistakes: a misspelled type, a missing semicolon, and a level tag on the wrong level. Which check finds each one, and on which line does it report it?

Exercise 10.4: Resolve with the ancestor

Recreate the vendor-branch conflict from Section 12.19, The vendor branch (p. 191) with `merge.conflictstyle diff3` set, and resolve it once by hand with the marker search and once with `git mergetool`. Which did you prefer for a single conflict? For ten?

Exercise 10.5: Write a merge request description in Vim

Copy the Default merge request template from the model project's templates, fill it in for the change you made in the substitution exercise, and use `:r !git diff --stat` to insert the files changed. Could a reviewer approve your change from the description and the diff alone?

CHAPTER

11

Seeing the model: diagrams from text

From what you know →

You know

Cameo diagrams you drew and laid out by hand.

→ Here it is called

views generated from the text: part tree, trace, interconnection, packages and IDEF0 plates.

→ What is new

nobody draws them. They are regenerated from the model on every change and never go stale.

Survival kit

What you have

three years of Cameo diagrams. You think in block definition diagrams and internal block diagrams; you know what a model looks like because you have drawn it, box by box, line by line, and moved the boxes so the lines would not cross.

What you need

the same pictures, without drawing them. A part tree, a requirement trace, the connections inside each block, the package dependencies, and IDEF0 plates of what the system does – each produced from the text in seconds, on your laptop, in Vim, and on the model site for management, always current.

Note: Core path

This chapter is CORE. You draw the views from week two; the model site that shows them to management arrives in sprint 4 (Section 17.6, Sprint 4: the model as a website (p. 263)).

11.1 “Where are my diagrams?”

On the Tuesday of sprint 2, Bob opened the converted Stations package in Vim, scrolled through ninety lines of text, and said the thing all three of them had been thinking since the directive: “Where are my diagrams?”

It is the right question, and the most important objection a Cameo modeler has. For three years the diagrams *were* the model. Reviews were diagrams in a briefing. A change was a box moved on a diagram. When the team talked about the power system they pointed at the IBD on the wall.

The answer this chapter gives is that the diagrams are still there, and better: they are drawn for you, from the text, every time the text changes. Nobody spends an afternoon moving boxes so lines do not cross, and nobody ever looks at a diagram that is three months out of date. What you lose is the ability to put a box somewhere the model does not say it is. That turns out to be the point.

Objection: “A generated diagram will never look as good as one I drew.”

Answer

True, and it does not need to. The drawn diagram was beautiful on the day it was finished and wrong a month later; the team counted eleven stale diagrams in drop 1 (Section 2.3, Measuring the spider web (p. 12)). A generated view is never wrong about what the model says. When the layout matters for a briefing, the plates command puts views in a titled frame (Section 11.9, Plates for briefings (p. 167)), and you can still draw a slide by hand from a current view – but you start from the truth.

11.2 Views are outputs, not inputs

In SysML v1 under Cameo, a diagram is a stored object with its own position data, separate from the elements it shows. Two people can disagree about where a box goes and both be “right.” In SysML v2 text, the model is the text alone. A diagram is a *view*: a function from the text to a picture. Same text, same picture, every time.

The kit’s view tools follow three rules:

- **Every view comes from the text**, through one command, with no hand positioning.
- **Every view is deterministic**: the same model gives the same bytes, so a view can be committed, compared and reviewed like any other file.
- **Every view also reports what it found**: the trace view counts requirements with gaps, the package view counts dependency cycles, the connection view counts boundary crossings. A view is also a check.

The five views you will use:

Table 11.1: The five views

View	Command	Shows	Cameo’s nearest
Part tree	`model.pl draw tree`	the composition hierarchy from the root part, each part at its level	block definition diagram (composition)

View	Command	Shows	Cameo's nearest
Trace	`model.pl draw trace`	requirements, what satisfies each, what verifies each, the gaps	requirement diagram, satisfy/verify matrix
Interconnection	`model.pl draw ibd`	the parts inside each block and the connections between their ports	internal block diagram
Packages	`model.pl draw pkg`	packages and their dependencies, with cycles marked	package diagram
IDEF0	`model.pl idef0`	the function tree as IDEF0 plates: inputs, controls, outputs, mechanisms	activity diagram (sort of)

11.3 Drawing them from the shell

Practice lesson 10's sandbox holds Tidewater's drop 1 converted into a model folder with its `sysml.conf`. Draw the part tree first:

```
perl "$KIT/tools/sysml/model.pl" draw tree -o tree.svg
```

The command prints one summary line – how many files it read, how many part definitions it found, how many trees and nodes it drew – and writes `tree.svg`. Open it in a web browser (Edge on the laptop opens SVG files directly: `start tree.svg` from Git Bash). The root is the `noun_root` named in `sysml.conf`, `ObservationNetwork`.

Now the other three structural views. Each prints its own summary, and the summaries are worth reading:

```
perl "$KIT/tools/sysml/model.pl" draw trace -o trace.svg
perl "$KIT/tools/sysml/model.pl" draw ibd -o ibd.svg
perl "$KIT/tools/sysml/model.pl" draw pkg -o pkg.svg
```

The trace view reports how many leaf requirements it drew and how many have gaps – the same planted gaps the trace job reports (Section 6.8, Bob's Wednesday, again (p. 67)). The interconnection view reports diagrams and connections. The package view prints a line for each package dependency cycle, then a summary ending in the count of cycles. That count is the spider web of chapter 3, measured. (Your sandbox prints a smaller count than the drop-1 figure below: the sandbox already has the layered folders, the government's conventions package and the DOORS modules, which change which packages depend on which. The figure is drop 1 exactly as converted.)

Note: The package view and markings

The package view was built to check security markings too: every package marked, nothing more sensitive flowing into something less. A converted model has no markings yet, so `draw pkg` says so on its first line and draws packages, dependencies and cycles only. The security gate (`model.pl gate`) still insists on markings; that comes later, in the security epic (Chapter 48, Security: protect the environment, engineer it into the system (p. 651)).

11.4 The five views, read

Here is each view of Tidewater's drop 1 as converted, before the government had changed a line. These are the committed renderings in `docs/tidewater/views/`, made by `sim/tidewater-views.pl` with exactly the commands above.

11.4.1 The part tree

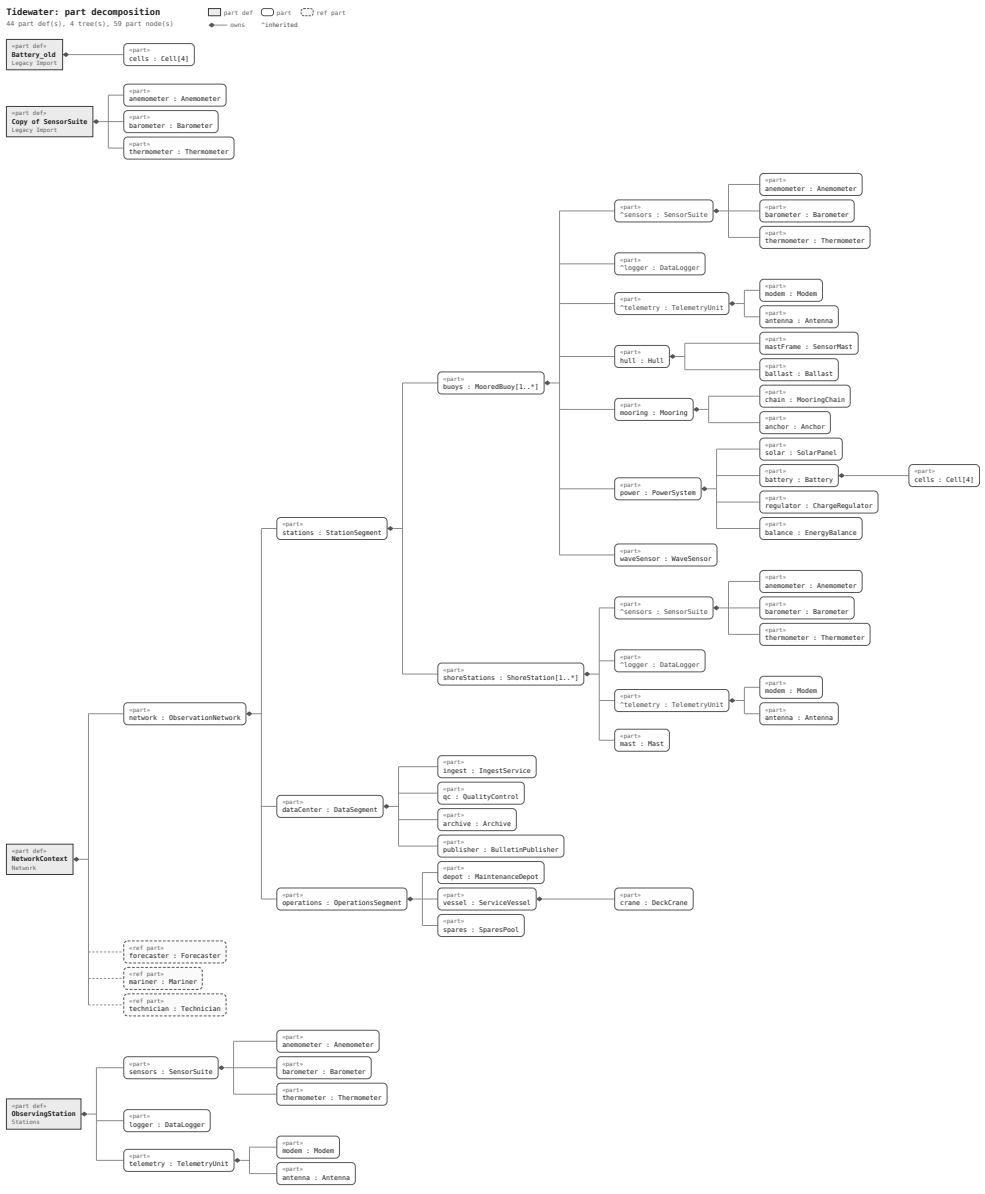


Figure 11.1: The part tree of drop 1: ObservationNetwork and what it is made of

Read it from the left. Each box is a part definition; each line is "is made of." The level tags (`@L2 Segment`, `@L3Station` ...) are what the team adds in the structure epic; in drop 1 they are still TODO lines, so the tree has no level shading yet. What to look for: a part that appears in two

places (the duplicates of the Legacy Import package), a branch that is much deeper than its neighbors, a part with nine or more children.

11.4.2 The trace

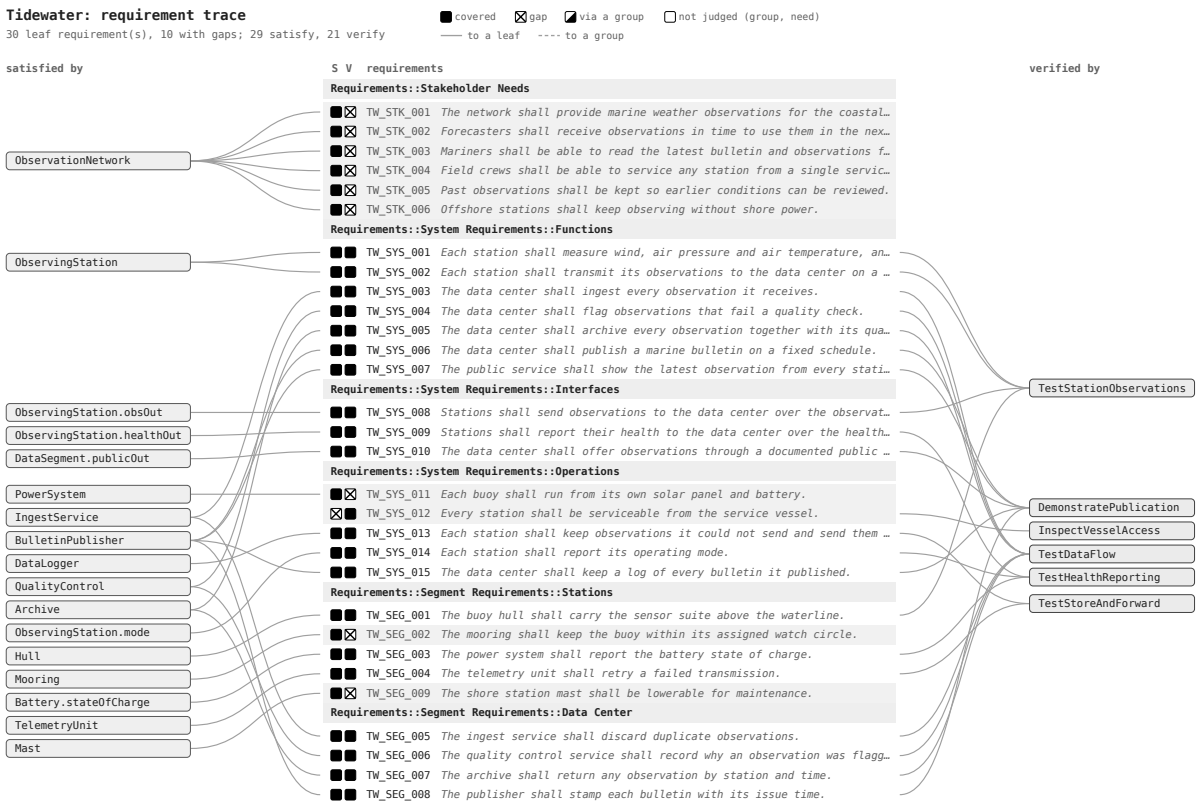


Figure 11.2: The requirement trace of drop 1: satisfy on one side, verify on the other, gaps marked

Each row is a requirement, by DOORS id. To its left, what satisfies it; to its right, what verifies it. A requirement with nothing on one side is a gap and is marked. Drop 1's gaps are the ones the team reported to the prime as DR-2 to DR-4 (Chapter 29, Sprint 4: the model site, a cycle broken, and the demo that landed (p. 459)). This view is the one Jim showed the program office in sprint 4, because it answers their first question – what is not covered? – without anyone reading text.

11.4.3 Interconnection

One diagram per block that has internal connections. Ports sit on the edge of their part; connections join them. When the security epic adds trust zones, this view also marks every connection that crosses from one zone to another – the attack surface.

11.4.4 Packages

Each box is a package; each arrow is "depends on" (an import, a type it uses, a relationship that reaches into it). Arrows that close a cycle are drawn dashed. Network depends on Stations,

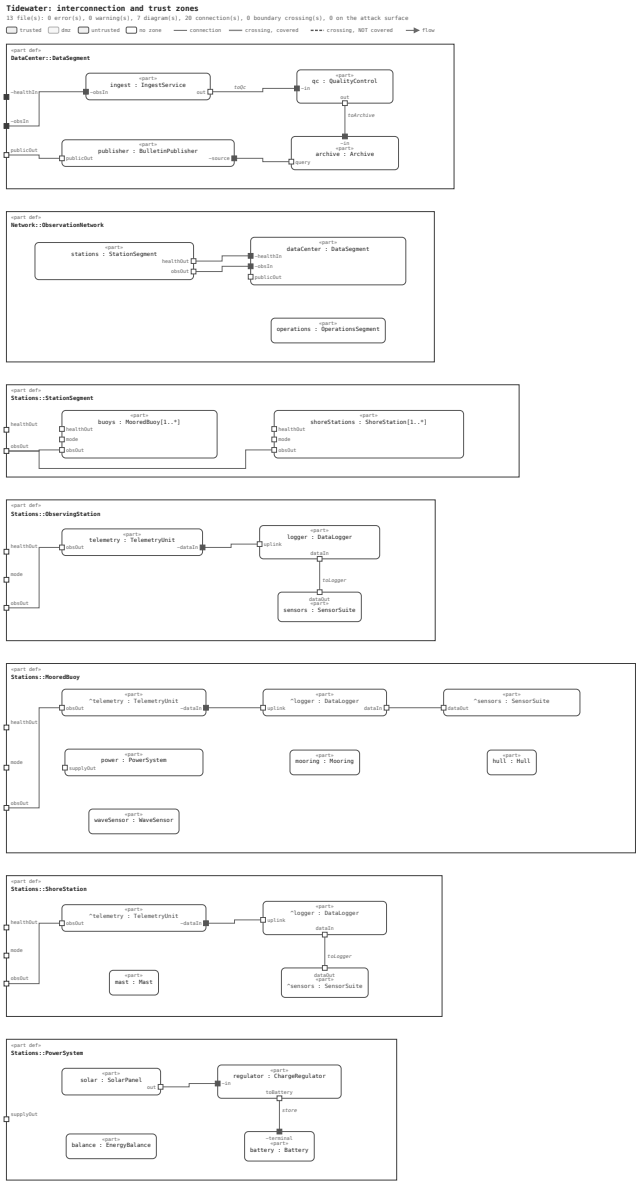


Figure 11.3: Interconnection views of drop 1: the parts inside each block and the connections between their ports

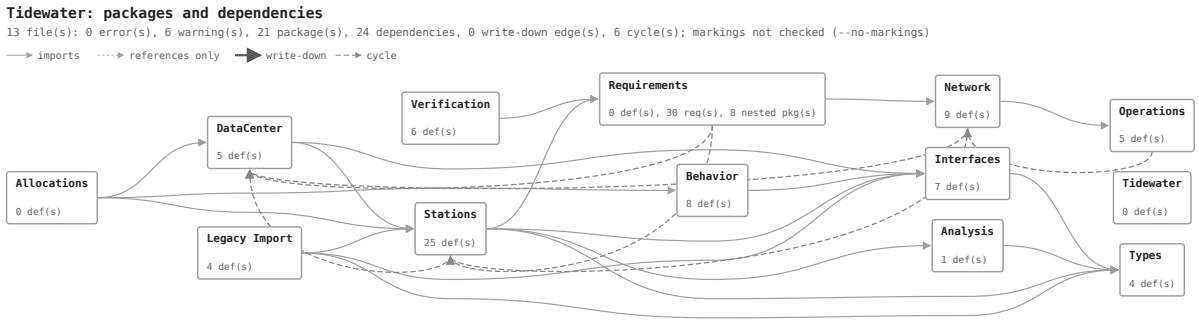


Figure 11.4: Package dependencies of drop 1 in the converter's flat layout: the dashed arrows are the cycles

which depends on DataCenter, which depends back on Network; the drawing marks each arrow that closes such a loop.

Three ways to count the same tangle. The drawer counts dashed arrows, and which arrow it dashes depends on the order it visits the packages, so the same model gives a different number in a different folder layout: six on drop 1 in the converter's flat layout (this figure), four in Tidewater's layered layout (your sandbox, and the model site). The pipeline's `complexity` job counts something steadier: groups of packages that are tangled together, however many arrows tie them. On drop 1 that is one group of five packages. Jim's count in Chapter 2, The Cameo years (p. 10) was a third measure: pairs of packages in the v1 model that each depend on the other, two. All three are the spider web; the team tracks the `complexity` job's number, because it does not move when someone renames a folder.

The cleanup stories are judged by this picture: "Break the Network to Operations cycle" in sprint 4 removed one loop; the Stations and DataCenter loop was still there at the end of sprint 6 (Chapter 47, Keeping SysML v2 from becoming a spider web (p. 645)).

11.4.5 IDEF0

The verbs of the model, not the nouns. The kit reads the function tree under the `verb_root` named in `sysml.conf` (`ObserveCoast`, one cycle of the network) and draws it as IDEF0 plates: a context plate (O-0) with the whole function as one box, and a decomposition plate (O0) with its steps – Sample, Transmit, Ingest, Check, Archive, Publish – each with its inputs on the left, controls on top, output on the right and mechanism underneath. The mechanisms come from the model's `allocate` statements: Sample is performed by the Sensor Suite, Transmit by the Telemetry Unit.

Systems engineers who came up through structured analysis read an IDEF0 plate faster than any SysML diagram, and program offices often still ask for them. Here they cost nothing.

11.5 IDEF0 plates from the shell

The IDEF0 plates need two more lines in `sysml.conf`: the name of the IDEF0 model and its letter. Without them the kit uses the defaults of its first example (a model named KillChain, letter K), which is not what you want on Tidewater's plates:

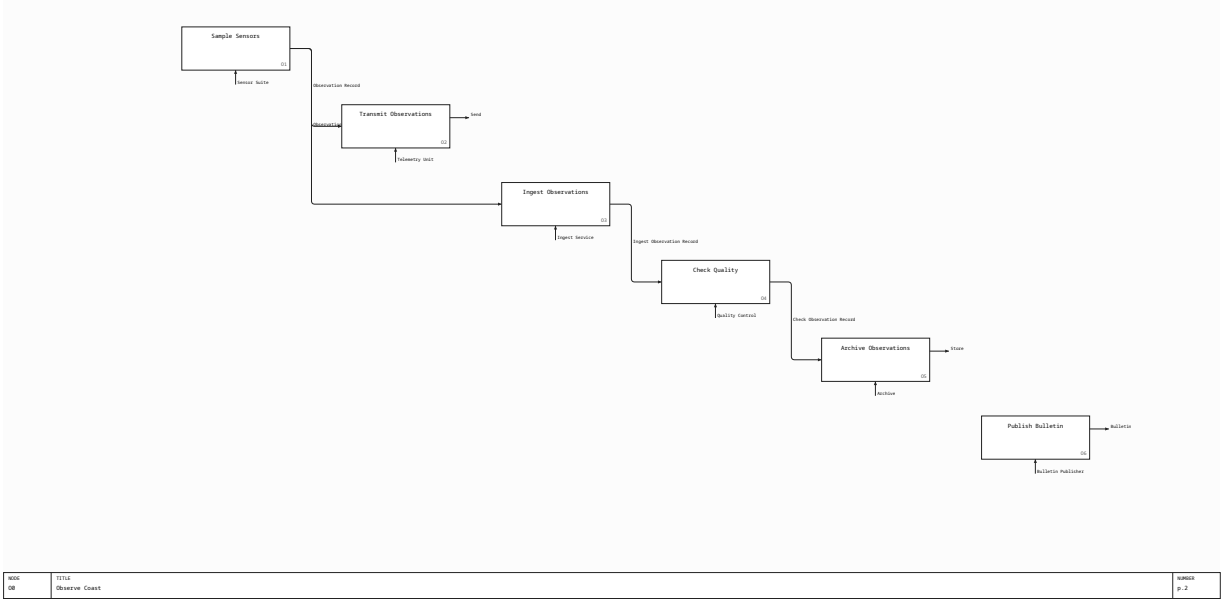


Figure 11.5: IDEF0 plate O0: the observation cycle's six steps with their inputs, outputs and mechanisms

```
printf 'idef0_name   = ObserveCoast\nmodel_letter = O\nidef0_models = auto\n' >> sysml.conf
perl "$KIT/tools/sysml/model.pl" idef0
ls docs/idef0/svg
```

`idef0_models = auto` tells the kit how to cut the function tree into IDEF0 models. When the top-level steps are themselves decomposed, each becomes its own IDEF0 model; when they are all leaves, as in Tidewater, the whole tree is one model and the steps are the boxes of its A0 plate. The command writes `docs/idef0/plates.html` (all plates on one page), the plates as SVG under `docs/idef0/svg/`, and an `INTERFACES` file listing the hand-offs between steps.

Warning: Converted behavior is thin

A v1 activity converted to v2 carries its steps and their parameters, not its object flows. The IDEF0 bridge fills the gap by feeding each step's inputs from the nearest earlier step whose output has the right type, and says so in the generated notes. That is good enough to see the shape of the function; it is not a substitute for writing the flows by hand, which is the behavior epic's job (Chapter 31, Sprint 6: Behavior by hand, a table for the program, and baseline-0 (p. 495)).

11.6 Drawing them from Vim

In a SysML buffer the kit's Vim support has one key per view. Each saves the buffer, runs the same `model.pl draw` command for the project the file belongs to, shows the summary, and writes the SVG in Vim's current directory:

Table 11.2: The view keys in a SysML buffer

Key	Command	View
`\mw`	`:SysDraw tree`	part tree
`\mq`	`:SysDraw trace`	trace
`\mi`	`:SysDraw ibd`	interconnection
`\mp`	`:SysDraw pkg`	packages
`\mn`	`:SysNouns`	the part tree as text, in a scratch window
`\mb`	`:SysVerbs`	the function tree as text
`\mW`	`:SysView`	the live viewer (next section)

Try one headless, the way the test does it:

```
edit model/physical/stations/Stations.sysml
SysDraw tree
```

```
ls -l tree.svg
```

If you set `g:sysml_svg_viewer` in your `.vimrc` (for example `let g:sysml_svg_viewer = 'start ""'` on Windows), each draw also opens the picture. The text trees from `\mn` and `\mb` need no viewer at all: they open in a split, and they are the fastest way to answer "what is under the Power System?" while you are editing.

11.7 The live viewer beside Vim

For editing sessions, the kit has a live viewer: `bin/sysml-view.tcl`, a small window drawn with the Tk toolkit that ships with Git for Windows. It shows one view of the project at a time and redraws whenever you save a `.sysml` file. Start it from Vim with `\mW`, or from Git Bash:

```
wish "$KIT/bin/sysml-view.tcl" .        # from the project's top folder
```

It talks to Vim in both directions:

- **`\mf` in Vim** (focus) tells the viewer to center on the definition under the cursor.
- **Clicking a box in the viewer** records where that element is defined; **`\mo` in Vim** (open) jumps there.

Bob kept it open on a second monitor from sprint 3 on. "It's the part of Cameo I missed," Bob said at the retro, "without the part I hated." The viewer draws, it never edits.

11.8 Comparing two versions

A view of one version answers "what is it?" The question in a review is "what changed?" `model.pl diff` draws the views of two versions with the differences colored, and writes a plain change

list beside them. Make a small change in the sandbox – tag the station segment with its level, the first thing practice lesson 10 asks for – commit it, and compare the commit with the one before: (a commit is a recorded version of the files; Chapter 12, Git from zero, for systems engineers (p. 171), the next chapter, explains it – here you only need the two commands below)

```
perl -0pi -e 's{// TODO stereotype Tidewater_Profile:L2Segment[^\n]*}{\@L2Segment;}; s{(package
Stations \{\n)}{$1    private import Conventions::*;\n}' model/physical/stations/Stations.sysml
git commit -qam "Tag StationSegment with its level"
perl "$KIT/tools/sysml/model.pl" diff --git HEAD~1..HEAD -o diff
cat diff-changes.txt
```

(The first line of that block is a scripted edit so the test can run; you would make the change in Vim.) The command writes `diff-tree.svg`, `diff-trace.svg`, `diff-packages.svg` and `diff-ibd.svg` with added, removed and changed elements colored, and `diff-changes.txt` with one line per change. Its summary also says whether the change opened any new trace gap – the question a reviewer cares most about (Chapter 22, Reviewing a model change (p. 357)).

11.9 Plates for briefings

A briefing wants a picture with a title block: what it is, which baseline, which date, which sheet. `model.pl plates` puts the model's views into ISO-style drawing plates with a title block filled from the model and `sysml.conf`:

```
perl "$KIT/tools/sysml/model.pl" plates --auto -o plates
```

The plates are what Jim printed for the sprint 6 briefing (Chapter 35, The briefing (p. 532)): the part tree and the package view, each on one sheet, titled and dated, with the baseline tag in the corner.

11.10 The model site: views for management

From sprint 4, the pipeline's `pages` job (Section 20.10, Stage publish: the model site (p. 316)) draws all four views and copies the IDEF0 plates into the model site on every merge to `develop`, and the nightly schedule redraws them. The site's front page shows each view with the date and commit it was drawn from. Management never asks for a diagram again; they open the site.

Two choices the team made about the site, worth copying:

- **The site shows the views, not the text.** Management does not need to read SysML v2. Every page leads with a picture or a count, and links to the text for whoever wants it.
- **A view that cannot be drawn says so.** If a draw fails, the job leaves a note on the site instead of silently dropping the picture. A missing picture that nobody notices is worse than a red pipeline.

11.11 The trees as text

Not every question needs a picture. "What is under the Moored Buoy?" is answered fastest by the part tree as text, which prints in the terminal (or, from Vim, in a split with `\mn`):

```
perl "$KIT/tools/sysml/model.pl" nouns --plain | head -14
```

`--plain` turns names like `MooredBuoy` into words ("Moored Buoy"), which is how Jim pasted the tree into the first status note. The function tree prints the same way with `verbs`:

```
perl "$KIT/tools/sysml/model.pl" verbs
```

The text trees are also the easiest thing to put in an issue or a merge request description: paste them inside a code block and GitLab keeps the tree drawing characters aligned.

11.12 Views as measures

Every view prints a summary line, and those lines are measures the team tracked from sprint to sprint. They are cheap – each takes a second to compute – and each one answers a question the program office asks:

Table 11.3: The view summaries as measures

View	Summary measure	The question it answers	Which way is good
Part tree	part definitions, trees, nodes	how big is the structure, and is it one tree?	one tree; growth only when the prime adds parts
Trace	leaf requirements, how many with gaps	what is not covered?	gaps going down
Interconnection	diagrams, connections, boundary crossings	where do the parts talk to each other?	crossings that each trace to a security requirement
Packages	packages, dependencies, cycles	is the spider web getting better?	cycles going to zero
IDEF0	models, activities, links	what does the system do, step by step?	every step with a mechanism

The spider-web number on every sprint review slide from sprint 4 is the `complexity` job's count of tangled package groups, the scorecard's spider-web line: 2 after the DOORS import in sprint 3, 1 once sprint 4 broke the Network and Operations loop, and still 1 at the end of sprint 6, because Stations and DataCenter still depend on each other. The package view is the picture behind that number, and the cleanup stories change it in front of management's eyes.

11.13 What a view cannot show

A view is honest about what the model says, which means it shows nothing the model does not say. Four limits the team ran into:

- **Layout is automatic.** Boxes go where the layout puts them. A view of a large block can be wide; the answer is a narrower view (one block, one package), not hand placement.
- **Converted behavior is thin.** Until the behavior epic writes flows by hand, activities show steps without object flows, and state machines are TODO lines that no view draws.
- **The model tool's own trace has a known gap.** `model.pl trace` (and `\mr` in Vim) counts a requirement satisfied by a *port* or an *attribute* as unsatisfied. On Tidewater that misreports five requirements. The pipeline's trace job, which compares the model with DOORS, does not have this gap; use its numbers as the record.
- **A view shows one version.** "What changed?" is the diff (Section 11.8, Comparing two versions (p. 166)), not two views side by side by eye.

11.14 When a view will not draw

Table 11.4: When a view will not draw

Message or symptom	Cause	Fix
`no model/ directory here or above`	you are not inside the project	`cd` to the project's top folder, or pass `--root DIR`
the tree is empty or starts at the wrong part	`noun_root` in `sysml.conf` names the wrong part definition	set `noun_root` to the top part definition's name
IDEF0 plates named K and KillChain	`idef0_name` and `model_letter` are not set	add both to `sysml.conf` (Section 11.5, IDEF0 plates from the shell (p. 164))
`draw pkg` reports errors about markings	some packages are marked and some are not	mark them all, or none until the security epic
`wish: command not found`	the live viewer's Tk is not on the path	run it from Git Bash, where `wish` is installed with Git for Windows
a draw in the pipeline is missing from the site	the job could not draw it; the site has a note in its place	read the `pages` job's log for that view

11.15 Applying this to your project

- **Copy:** draw the views from the first week, put them on the model site, and treat the view summaries (gaps, cycles, crossings) as measures.
- **Adapt:** set `noun_root`, `verb_root`, `idef0_name` and `model_letter` in your `sysml.conf`; choose `idef0_models` to match how deep your function tree is.
- **Decide locally:** which views go in which briefing, and whether plates replace your program's drawing templates. Expect the first briefing with generated views to look plainer than the old slides; by the second, the program office will notice they are never out of date.

Exercise 11.1: Find a cycle

Draw the package view of the practice-10 sandbox. Pick one dependency cycle from the printed warnings, open the two files it names in Vim, and find the import or reference that closes the loop. Which package should own the shared element so the cycle disappears?

Exercise 11.2: Read a plate

Open `docs/idef0/plates.html` after running `model.pl idef0`. For the Check step, name its input, its output and its mechanism, and say which `allocate` statement in the model supplied the mechanism.

You can now draw the part tree, trace, interconnection and package views from the shell and from Vim, generate IDEF0 plates, run the live viewer beside Vim, compare two versions as colored views, and find the spider web in a picture.

CHAPTER

12

Git from zero, for systems engineers

From what you know →

You know

configuration management: versions, baselines, change records, who changed what and when.

→ Here it is called

git: commits, branches, merges, tags and the log.

→ What is new

every change is recorded as you make it, a branch lets you work without disturbing anyone, and merging is routine, not a crisis.

Survival kit

What you have

a terminal you can use (Chapter 9, The survival kit (p. 126)), git configured with your name, and decades of configuration management experience: baselines, change requests, redlines, revision letters, the CM library.

What you need

the git commands the team used every day, each one tied to the CM idea it replaces, practiced in a throwaway repository until they are boring. Then the commands for working with GitLab (clone, push, pull), and an understanding of branching strategies good enough to defend Tidewater's choice, or to make a different one for your own project.

12.1 Tuesday afternoon, a throwaway folder

On the Tuesday of sprint 1, after stand-up, Bob blocked out the afternoon in the team calendar with the title "Breaking git on purpose." Tom had given Bob a one-page list of commands on Monday. Bob had read it and understood none of it in any way that would survive contact with a real model.

So Bob made a folder called `practice`, put a copy of one small SysML file in it, and spent three hours doing everything on Tom's list, then doing it wrong on purpose to see what happened. Bob made two branches change the same line and watched git refuse to merge them. Bob deleted a file, panicked, and got it back. Bob made a commit with a typo in the message and fixed it. At 16:30 Bob wrote in the team chat:

> *It's a CM library where every change record contains the actual change, and you can't lose anything unless you really try. Why didn't anyone tell us that?*
>
> — Bob, team chat, 6 October 2026

This chapter is Bob's afternoon, written down. Every command was run on a Git for Windows laptop; the output is what git printed, with only the commit identifiers (which will differ on your machine) left as they came out. Make your own `practice` folder and type along. Nothing you do in it can affect anything else.

12.2 Version control in configuration management words

Git is a version control system: a program that keeps every version of every file in a folder, with a record of who changed what, when and why. The folder, together with that history, is a **repository** (often "repo").

You already know version control. The program's CM library was version control, done by hand: controlled items, revision letters, change records, a librarian who checked documents in and out. Git does the same job with three differences that change everything:

1. **Every change record contains the change itself**, line by line, not a description of it. You never have to compare revision B and revision C by eye; git shows exactly which lines differ.
2. **Every person has the whole library**, history included, on their own laptop. You can work without a network, and you can look at any past version instantly.
3. **Combining parallel work is automatic** wherever the changes do not touch the same lines. Two people can change the same file at the same time, and git merges their changes. Only when both changed the same lines does a person have to decide.

Table 12.1: Configuration management to git

Configuration management	Git	Notes
CM library	repository	the folder plus its complete history
controlled item	a file in the repository	every file is controlled; there is no "uncontrolled draft" inside
change record (who, when, why, what)	commit	a snapshot of the changed files with author, date and message
revision identifier	commit hash (`3bcc7d4`)	a fingerprint of the content, unique to that commit
the change package being assembled	the staging area	the changes you have chosen to include in the next commit
a working copy checked out for editing	the working folder	the files you edit

Configuration management	Git	Notes
an alternative line of development	branch	a named line of commits; cheap to make, cheap to delete
incorporating a change	merge	combining one branch's commits into another
baseline	tag	a permanent name on one commit (`baseline-0`)
the central library server	remote (GitLab)	the shared copy everyone pushes to and pulls from
check out a copy to your desk	clone	copy the whole repository, history included
check in to the library	push	send your commits to the remote
get the latest from the library	pull	fetch others' commits and merge them into yours
redline	diff	the exact lines removed and added

12.3 Three places a change lives

The single idea that makes git make sense is that a change passes through three places:

```
working folder  --git add-->  staging area  --git commit-->  repository history
(you edit here)               (the package                   (permanent, with
                               for the next commit)           author, date, message)
```

You edit files in the **working folder** with Vim, as you always have. When a set of edits belongs together, you **stage** them with `git add`, which puts them into the package for the next commit. When the package is right, you **commit** it with a message saying why, and it becomes a permanent part of the history.

Why the middle step? Because a person's edits in an afternoon often belong to more than one change. Bob might fix a TODO line in `Stations.sysml` and, while there, notice a typo in `Network.sysml`. Those are two changes with two reasons; the staging area lets Bob commit them separately, each with its own message, so the history says exactly what was done and why. Tom's analogy, which stuck: the staging area is the change package on the CM librarian's desk before it is signed.

12.4 A practice repository

Start from your home folder and make a new repository:

```
$ cd ~
$ git init practice
Initialized empty Git repository in C:/Users/bob/practice/.git/
$ cd practice
$ git status
On branch main

No commits yet
```

```
nothing to commit (create/copy files and use "git add" to track)
```

`git init` made a folder `practice` with a hidden folder `.git` inside it. The `.git` folder *is* the repository: the whole history lives there. Never edit anything inside it by hand. Everything else in `practice` is your working folder.

`git status` is the command you will type more than any other. It tells you which branch you are on and what state your files are in. Type it whenever you are unsure, before and after every other command, until you no longer need to.

12.5 The first commit

Make a folder and a small SysML file. Use Vim (`vim model/stations/Battery.sysml` after `mkdir -p model/stations`) and type:

```
package Battery {
    part def Battery {
        doc /* Stores power for night and cloud. */
        attribute capacity;
    }
}
```

If you would rather paste than type, this makes the same folder and file:

```
mkdir -p model/stations
cat > model/stations/Battery.sysml <<'END'
package Battery {
    part def Battery {
        doc /* Stores power for night and cloud. */
        attribute capacity;
    }
}
END
```

Now ask git what it sees:

```
$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        model/

nothing added to commit but untracked files present (use "git add" to track)
```

Untracked means git sees a file it has never been told to control. Stage it, and look again:

```
$ git add model/stations/Battery.sysml
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   model/stations/Battery.sysml
```

The file is staged: it is in the package for the next commit. Commit it, with a message:

```
$ git commit -m "Add the Battery package"
[main (root-commit) 07f494b] Add the Battery package
 1 file changed, 6 insertions(+)
 create mode 100644 model/stations/Battery.sysml
$ git log
commit 07f494b7da90a0b4d1d6af9c85bff0f9ab7e1413
Author: Bob <bob@example.invalid>
Date:   Tue Oct 6 07:56:20 2026 -0500

    Add the Battery package
```

That is a change record. It says who (from your git settings), when, why (the message), and, though `git log` does not show it here, exactly what. The long string after `commit` is the **commit hash**, a fingerprint computed from the commit's content, author, date and parent. Any change to any of them would produce a different hash, which is what makes git's history tamper-evident. People usually quote only the first seven characters (`07f494b`); git understands the short form.

12.6 Changing a file: the diff

Edit the file: change "Stores power" to "Stores solar power." Type it in Vim, or give the change on Vim's command line, which is how this book's own test of the chapter makes it:

```
:edit model/stations/Battery.sysml
:%s/Stores power/Stores solar power/
:write
```

Then:

```
$ git status --short
 M model/stations/Battery.sysml
$ git diff
diff --git a/model/stations/Battery.sysml b/model/stations/Battery.sysml
index 5d33039..a163ee7 100644
```

```
--- a/model/stations/Battery.sysml
+++ b/model/stations/Battery.sysml
@@ -1,6 +1,6 @@
 package Battery {
     part def Battery {
-        doc /* Stores power for night and cloud. */
+        doc /* Stores solar power for night and cloud. */
         attribute capacity;
     }
 }
```

`git status --short` is the compact form: `M` means modified (the column it is in says whether the change is staged; here it is not yet). `git diff` shows the **diff**, the redline: lines starting with `-` were removed, lines starting with `+` were added, and the lines around them are context. The `@@ -1,6 +1,6 @@` line says which lines of the old and new file are shown.

This is the moment the document-era habit of "compare revision B and C by eye" dies. Every change, in every review, from now on, is shown this way: exactly the lines that changed, nothing else. When Bob showed the deputy program manager a diff in sprint 2, the deputy's first words were "That's a redline." It is.

Commit it:

```
$ git add -A
$ git commit -m "Battery: say where the power comes from"
[main 9f0add1] Battery: say where the power comes from
 1 file changed, 1 insertion(+), 1 deletion(-)
$ git log --oneline
9f0add1 Battery: say where the power comes from
07f494b Add the Battery package
```

`git add -A` stages every change in the working folder at once; use it when you know everything you changed belongs in this commit. `git log --oneline` shows one line per commit, newest first.

12.7 Commit messages a CCB would accept

The commit message is the "reason for change" field of the change record. In six months someone will run `git log` on a file to find out why a line is the way it is, and the message is all they will have. The team's convention:

- **A subject line of at most 72 characters**, saying what the commit does, starting with the area: `Stations: close the level-tag TODOs`. No full stop at the end.
- **A blank line, then a body** when the reason is not obvious from the subject: why the change was made, what was considered and rejected, what the reviewer should look at.
- **A reference to the task**: `Part of #23` or `Closes #23`, where 23 is the GitLab issue number. GitLab turns these into links, and `Closes` closes the issue when the change is merged (Chapter 15, Setting up GitLab (p. 226)).

Table 12.2: Commit messages

Poor	Better
`fix`	`Stations: type Battery.capacity as an energy value`
`changes per Bob`	`Network: replace the include TODO with an include use case (Closes #31)`
`WIP`	(do not commit work in progress to a shared branch; see Section 12.12, Putting work aside: stash (p. 183))
`updated model`	`Requirements: import DOORS module System Requirements (Part of #18)`

Because `core.editor` is set to Vim (Section 9.7, Setting up git (p. 136)), typing `git commit` without `-m` opens Vim with an empty message. Write the subject on the first line, leave the second blank, write the body below, then `:wq`. Git commits when Vim exits. If you quit without saving (`:q!`), git cancels the commit. For anything with a body, this is easier than `-m`.

12.8 Undoing things

Fear of breaking something is the biggest obstacle for newcomers, so learn the undo commands early. Which one to use depends on how far the change has travelled.

A change you have not staged, that you want to throw away. Make a throwaway edit, then throw it away:

```
:edit model/stations/Battery.sysml
:%s/and cloud/and fog/
:write
```

```
$ git status --short
 M model/stations/Battery.sysml
$ git restore model/stations/Battery.sysml
$ git status --short
```

`git restore FILE` puts the file back the way it was at the last commit. This one cannot be undone: the edits are gone. (They were never committed, so git never had them.)

A change you have staged, that you want to take back out of the package. Add an owner line under the first line, stage it, then take it back out of the package:

```
:edit model/stations/Battery.sysml
:1put ='    // owner: Bob'
:write
```

```
$ git add model/stations/Battery.sysml
$ git status --short
M  model/stations/Battery.sysml
$ git restore --staged model/stations/Battery.sysml
```

```
$ git status --short
 M model/stations/Battery.sysml
```

`git restore --staged FILE` un-stages it: the edit stays in the working folder, it is just no longer in the package. (Notice the `M` move from the first column to the second.) Older guides show `git rm --cached` or `git reset HEAD FILE` for this; `git restore --staged` is the modern form.

The last commit, before you have pushed it, has a typo in the message or is missing a file:

```
$ git commit -am "Batery: note the owner"
[main 7d3134b] Batery: note the owner
 1 file changed, 1 insertion(+)
$ git commit --amend -m "Battery: note the owner"
[main 1541dfd] Battery: note the owner
 Date: Tue Oct 6 07:57:16 2026 -0500
 1 file changed, 1 insertion(+)
```

`--amend` replaces the last commit with a corrected one (note the new hash). Use it only before you push: once others have the commit, replacing it rewrites shared history, which the protected branches in GitLab refuse anyway.

A commit that has been pushed, that turned out to be wrong:

```
$ git revert --no-edit HEAD
[main d93cd21] Revert "Battery: note the owner"
 Date: Tue Oct 6 07:57:24 2026 -0500
 1 file changed, 1 deletion(-)
$ git log --oneline
d93cd21 Revert "Battery: note the owner"
1541dfd Battery: note the owner
9f0add1 Battery: say where the power comes from
07f494b Add the Battery package
```

`git revert COMMIT` makes a *new* commit that undoes an old one. History is not rewritten; it grows. That is exactly how a CM system should behave: the wrong change and its correction are both on the record. `HEAD` means "the commit I am on now," and `HEAD~1` the one before it.

Tip: Committed means safe

Once something is committed, it is very hard to lose. Even a branch you delete keeps its commits for weeks in git's internal log (`git reflog` lists them). The things you can lose are the things you never committed. That is the practical argument for committing small and often.

12.9 Branches

A **branch** is a named line of commits. Every repository starts with one (`main`). Making another is instant, and it lets you make changes without touching anyone else's work until you choose

to combine them. On Tidewater, every task that changes the model is done on its own branch, named for its GitLab issue: `feature/23-network-levels`.

```
$ git switch -c feature/7-battery-units
Switched to a new branch 'feature/7-battery-units'
```

`git switch -c NAME` creates a branch and moves onto it (`git checkout -b NAME` is the older form, and does the same). Edit the file so the capacity has a type (`attribute capacity : ISQ::EnergyValue;`) and commit:

```
:edit model/stations/Battery.sysml
:%s/attribute capacity;/attribute capacity : ISQ::EnergyValue;/
:write
```

```
$ git commit -am "Battery: type the capacity as energy"
[feature/7-battery-units 38fecbe] Battery: type the capacity as energy
 1 file changed, 1 insertion(+), 1 deletion(-)
$ git log --oneline --all --graph
* 38fecbe Battery: type the capacity as energy
* d93cd21 Revert "Battery: note the owner"
* 1541dfd Battery: note the owner
* 9f0add1 Battery: say where the power comes from
* 07f494b Add the Battery package
```

(`commit -a` stages every modified, already-tracked file and commits in one step. It does not pick up new files.)

`git log --oneline --all --graph` draws every branch's history as a graph. Make it a habit; it is the map of the repository. Now switch back to `main` and look at the file:

```
$ git switch main
Switched to branch 'main'
$ cat model/stations/Battery.sysml
package Battery {
    part def Battery {
        doc /* Stores solar power for night and cloud. */
        attribute capacity;
    }
}
```

The typed capacity is not there. It exists only on the branch. This is the point of branches: `main` is untouched until the change is reviewed and merged.

12.10 Merging

To incorporate the branch's work, merge it into the branch you are on:

```
$ git merge --no-ff feature/7-battery-units -m "Merge branch 'feature/7-battery-units' into
'main'"
Merge made by the 'ort' strategy.
 model/stations/Battery.sysml | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
$ git log --oneline --all --graph
*   84f88cb Merge branch 'feature/7-battery-units' into 'main'
|\
| * 38fecbe Battery: type the capacity as energy
|/
* d93cd21 Revert "Battery: note the owner"
* 1541dfd Battery: note the owner
* 9f0add1 Battery: say where the power comes from
* 07f494b Add the Battery package
$ git branch -d feature/7-battery-units
Deleted branch feature/7-battery-units (was 38fecbe).
```

The graph shows the branch going off and coming back. The **merge commit** (`84f88cb`) records the incorporation itself: when the change was incorporated, by whom, and (in GitLab) through which merge request and with whose approval. Deleting the branch afterwards removes only the name; its commits stay in the history.

`--no-ff` ("no fast-forward") forces git to make a merge commit even when it could simply move `main` forward to the branch's last commit. Without it, the graph would be a straight line and the fact that these commits arrived together, as one reviewed change, would be lost. Tidewater always merges this way; GitLab does it for you when the project's merge method is set to "merge commit" (decision D-3 in Section 13.11, The decisions the team wrote down (p. 210)).

On Tidewater you will almost never type `git merge` into a shared branch yourself. Merges into `develop` and `main` happen in GitLab, through a merge request, after the pipeline passes and a code owner approves (Chapter 15, Setting up GitLab (p. 226)). You practice it here so that you know what GitLab is doing for you.

12.11 Merge conflicts

A **merge conflict** happens when two branches changed the same lines differently, and git cannot know which to keep. It is two people redlining the same sentence: a person has to decide. Make one on purpose. On a branch, change the battery's documentation one way:

```
$ git switch -c feature/8-doc-bob
Switched to a new branch 'feature/8-doc-bob'
```

Edit the line to `Stores solar power for the night.` (from Vim's command line, as below, or by hand) and commit:

```
:edit model/stations/Battery.sysml
:%s/for night and cloud/for the night/
:write
```

```
$ git commit -am "Battery doc: night only"
[feature/8-doc-bob b456f01] Battery doc: night only
 1 file changed, 1 insertion(+), 1 deletion(-)
```

Then go back to `main` and change the same line another way (`Stores solar power for night, cloud and storms.`), commit, and merge the branch in:

```
$ git switch main
Switched to branch 'main'
```

```
:edit model/stations/Battery.sysml
:%s/for night and cloud/for night, cloud and storms/
:write
```

```
$ git commit -am "Battery doc: add storms"
[main 8c8f29e] Battery doc: add storms
 1 file changed, 1 insertion(+), 1 deletion(-)
$ git merge feature/8-doc-bob
Auto-merging model/stations/Battery.sysml
CONFLICT (content): Merge conflict in model/stations/Battery.sysml
Automatic merge failed; fix conflicts and then commit the result.
```

When `git merge` stops like this it reports an error: the merge is half done, and git waits for you. Ask for the state:

```
$ git status
On branch main
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
        both modified:   model/stations/Battery.sysml

no changes added to commit (use "git add" and/or "git commit -a")
```

Git stopped half way and wrote both versions into the file, between **conflict markers**:

```
$ cat model/stations/Battery.sysml
package Battery {
    part def Battery {
<<<<<<< HEAD
        doc /* Stores solar power for night, cloud and storms. */
||||||| e65550b
```

```
        doc /* Stores solar power for night and cloud. */
=======
        doc /* Stores solar power for the night. */
>>>>>>> feature/8-doc-bob
        attribute capacity : ISQ::EnergyValue;
    }
}
```

Between `<<<<<<< HEAD` and `=======` is the version on the branch you are on (`main`); between `=======` and `>>>>>>> feature/8-doc-bob` is the incoming version. The middle section, from `|||||||` (with the common ancestor's commit id) to `=======`, is the original line both sides started from; it appears because of the `merge.conflictstyle diff3` setting from Section 10.9, Merge conflicts in Vim (p. 151), and without that setting it is simply absent. Everything outside the markers merged cleanly (notice that the typed capacity from the earlier merge is there). Resolve it by editing the file into what it should be, with the markers removed. Here the right answer combines both:

```
        doc /* Stores solar power for the night, cloud and storms. */
```

In Vim, delete the lines from `<<<<<<<` to `>>>>>>>` and type the combined line in their place; or let Vim's command line do it, which is also how this book's test of the chapter does it:

```
:edit model/stations/Battery.sysml
:/^<<<<<<</,/^>>>>>>>/delete
:/part def Battery {/put ='        doc /* Stores solar power for the night, cloud and storms. */'
:write
```

Then tell git the conflict is resolved and finish the merge:

```
$ git add model/stations/Battery.sysml
$ git commit --no-edit
[main 5324b76] Merge branch 'feature/8-doc-bob'
$ git log --oneline --all --graph
*   5324b76 Merge branch 'feature/8-doc-bob'
|\
| * b456f01 Battery doc: night only
* | 8c8f29e Battery doc: add storms
|/
*   84f88cb Merge branch 'feature/7-battery-units' into 'main'
|\
| * 38fecbe Battery: type the capacity as energy
|/
* d93cd21 Revert "Battery: note the owner"
* 1541dfd Battery: note the owner
* 9f0add1 Battery: say where the power comes from
* 07f494b Add the Battery package
```

If you start a merge and want to back out of it, `git merge --abort` returns everything to how it was before.

Note: Conflicts are information

Bob's first conflict produced a moment of panic, then a realization. In the Cameo years, two people changing the same element produced no warning at all: the second save silently won, and the first person's work vanished until someone noticed, sometimes months later. A conflict is git refusing to lose anyone's work. It is the system working. The Vim chapter (Section 10.9, Merge conflicts in Vim (p. 151)) showed how to resolve conflicts quickly in Vim; now that you know what a conflict is, go back to that section; Chapter 43, Maintenance through the life cycle (p. 602) shows how the team handled the conflicts that each prime drop produced.

12.12 Putting work aside: stash

Sometimes you are in the middle of an edit and need a clean working folder for a few minutes: to look at another branch, or to pull a colleague's change. You do not want to commit half-finished work. Make an edit (here, adding gales to the documentation), then **stash** it:

```
:edit model/stations/Battery.sysml
:%s/cloud and storms/cloud, storms and gales/
:write
```

```
$ git status --short
 M model/stations/Battery.sysml
$ git stash
Saved working directory and index state WIP on main: 5324b76 Merge branch 'feature/8-doc-bob'
$ git status --short
$ git stash list
stash@{0}: WIP on main: 5324b76 Merge branch 'feature/8-doc-bob'
$ git stash pop
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   model/stations/Battery.sysml

no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@{0} (7b0c8a5ad89eed7f1a2ffc6ae4ef0277d8175e52)
```

`git stash` shelves your uncommitted changes and leaves a clean working folder. `git stash list` shows the shelf. `git stash pop` puts the most recent shelved changes back and removes them from the shelf; `git stash apply` puts them back and keeps them on the shelf. `git stash clear` empties the shelf (carefully: that is a real delete). Stash is for minutes, not days. If work will take longer, commit it on its own branch.

12.13 Remotes: clone, push and pull

So far everything has been on one laptop. A **remote** is another copy of the repository that you exchange commits with. On Tidewater the remote is the GitLab project; here, a second folder plays GitLab's part, so you can practice without a server. (A **bare** repository is one with no working folder, only history: what a server keeps.)

```
$ cd ~
$ git init --bare gitlab-model.git
Initialized empty Git repository in C:/Users/bob/gitlab-model.git/
$ cd practice
$ git remote add origin ~/gitlab-model.git
$ git push -u origin main
To C:/Users/bob/gitlab-model.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

`git remote add origin URL` names the remote `origin` (the conventional name). `git push -u origin main` sends the `main` branch there; `-u` remembers the pairing, so later a plain `git push` or `git pull` knows where to go.

Now be Tom. Clone the "server" into a second folder, as Tom would on another laptop, and make a change:

```
$ cd ~
$ git clone gitlab-model.git tom-clone
Cloning into 'tom-clone'...
done.
$ cd tom-clone
$ git log --oneline -3
5324b76 Merge branch 'feature/8-doc-bob'
8c8f29e Battery doc: add storms
b456f01 Battery doc: night only
```

The clone has the whole history. In the clone, set Tom's name and email without `--global`, so these commits are Tom's; then add a README and push it:

```
$ git config user.name Tom
$ git config user.email tom@example.invalid
$ echo "# Tidewater practice model" > README.md
$ git add README.md
$ git commit -m "Add a README"
[main c4f173d] Add a README
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
$ git push
To C:/Users/bob/gitlab-model.git
   5324b76..c4f173d  main -> main
```

Back in Bob's folder, bring in Tom's work:

```
$ cd ~/practice
$ git pull
From C:/Users/bob/gitlab-model
   5324b76..c4f173d  main       -> origin/main
Updating 5324b76..c4f173d
Fast-forward
 README.md | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
```

`git pull` is two steps: `git fetch` (download the remote's new commits) and then a merge of them into your branch. Here Bob had no new commits of their own, so git simply moved Bob's `main` forward: a **fast-forward**.

12.13.1 When the push is rejected

What if both have committed? Tom adds a line to the README and pushes it:

```
$ cd ~/tom-clone
$ echo "Practice only; the real model is in GitLab." >> README.md
$ git commit -am "README: say what this repository is"
[main 4b74c6c] README: say what this repository is
 1 file changed, 1 insertion(+)
$ git push
To C:/Users/bob/gitlab-model.git
   c4f173d..4b74c6c  main -> main
$ cd ~/practice
```

Bob, who has not pulled since, commits the gales edit still in the working folder from Section 12.12, Putting work aside: stash (p. 183), and pushes:

```
$ git commit -am "Battery: gales too"
[main eaa4350] Battery: gales too
 1 file changed, 1 insertion(+), 1 deletion(-)
$ git push
To C:/Users/bob/gitlab-model.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'C:/Users/bob/gitlab-model.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
```

Pull, which merges Tom's commit into Bob's, then push again:

```
$ git pull --no-rebase --no-edit
From C:/Users/bob/gitlab-model
```

```
   c4f173d..4b74c6c  main       -> origin/main
Merge made by the 'ort' strategy.
 README.md | 1 +
 1 file changed, 1 insertion(+)
$ git push
To C:/Users/bob/gitlab-model.git
   4b74c6c..0a2f2f6  main -> main
```

The server refused Bob's push because accepting it would have discarded Tom's commit. That refusal is a safety feature, not an error to be worked around. The fix is always: pull, resolve any conflicts, push again.

On Tidewater this situation almost never arises, because nobody pushes to a shared branch. Each person pushes only their own feature branches, and shared branches change only through merge requests. Your local copy of `develop` only ever moves forward, by pulling. One more setting makes that safe:

```
git config --global pull.ff only
```

With it, `git pull` refuses to make a merge commit on your behalf; it only fast-forwards. If it ever refuses, you committed on a branch you should not have, and you want to know.

12.14 Tags: baselines

A **tag** is a permanent name on one commit. On Tidewater, tags are baselines: `baseline-0`, `baseline-1`. An **annotated** tag also records who made it, when, and a message:

```
$ git tag -a baseline-0 -m "Interim baseline for the practice repo"
$ git show baseline-0 --stat
tag baseline-0
Tagger: Bob <bob@example.invalid>
Date:   Tue Oct 6 08:00:52 2026 -0500

Interim baseline for the practice repo

commit 0a2f2f63a2cb7f06d238a5b7ed4214d42b2c46ec
Merge: eaa4350 4b74c6c
Author: Bob <bob@example.invalid>
Date:   Tue Oct 6 08:00:40 2026 -0500

    Merge branch 'main' of C:/Users/bob/gitlab-model

 README.md | 1 +
 1 file changed, 1 insertion(+)
$ git push origin baseline-0
To C:/Users/bob/gitlab-model.git
 * [new tag]         baseline-0 -> baseline-0
```

Tags are not pushed with `git push` alone; push them by name. Anyone can then check out exactly that baseline (`git switch --detach baseline-0`), years later, and get the identical files.

Compare the document era's "Revision C," which named a document but not the exact state of everything that document depended on.

In GitLab, baseline tags are **protected** (Chapter 15, Setting up GitLab (p. 226)): only maintainers can create them, and nobody can move or delete them. That turns a convention into a control.

12.15 Reading history: log, show and blame

Three commands answer the questions a CM auditor asks.

What changed, and when? `git log`, with options. `git log --oneline --all --graph` for the map; `git log --oneline -- FILE` for one file's history:

```
$ git log --oneline --follow -- model/stations/Battery.sysml
eaa4350 Battery: gales too
8c8f29e Battery doc: add storms
b456f01 Battery doc: night only
38fecbe Battery: type the capacity as energy
d93cd21 Revert "Battery: note the owner"
1541dfd Battery: note the owner
9f0add1 Battery: say where the power comes from
07f494b Add the Battery package
```

What exactly was in that change? `git show COMMIT` prints the change record and its diff; with `--stat`, only a summary:

```
$ git show --stat HEAD~1
commit eaa43503467763c98c60e8177cf03495f036d9d9
Author: Bob <bob@example.invalid>
Date:   Tue Oct 6 08:00:32 2026 -0500

    Battery: gales too

 model/stations/Battery.sysml | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
```

Who last changed each line, and in which change? `git blame FILE`:

```
$ git blame model/stations/Battery.sysml
^07f494b (Bob 2026-10-06 07:56:20 -0500 1) package Battery {
^07f494b (Bob 2026-10-06 07:56:20 -0500 2)     part def Battery {
eaa43503 (Bob 2026-10-06 08:00:32 -0500 3)         doc /* Stores solar power for the night,
cloud, storms and gales. */
38fecbee (Bob 2026-10-06 07:57:36 -0500 4)         attribute capacity : ISQ::EnergyValue;
^07f494b (Bob 2026-10-06 07:56:20 -0500 5)     }
^07f494b (Bob 2026-10-06 07:56:20 -0500 6) }
```

Each line shows the commit that last touched it. Line 3 came from the last change, the gales; line 4 from the typed capacity. Bob's reaction, the first time: "In Cameo, nobody could tell me who changed a block or why. This tells me for every line."

12.16 Files git should ignore

Some files belong in the working folder but not in the repository: editor swap files, generated drawings you can rebuild, scratch output. List them in a file called `.gitignore` at the top of the repository, one pattern per line:

```
$ mkdir -p docs
$ echo '<svg/>' > docs/tree.svg
$ echo "docs/*.svg" > .gitignore
$ git status --short
?? .gitignore
```

The `docs/tree.svg` made first stands for a generated drawing; git no longer reports it, because it matches the pattern. (`??` means untracked.) Commit the `.gitignore` itself. The team's model repository ignores Vim's swap files, the tags file, merge backups and the local pipeline runner's logs, and deliberately does *not* ignore the generated `docs/` folder, because the team decided to commit generated documentation so that reviewers see it change (Chapter 20, The pipeline, job by job (p. 302)).

12.17 The rules the team adopted

By the end of sprint 1 the team had settled on eight rules. They are on the wiki's Gitflow page and on a card taped to each monitor.

1. **Never commit on `main` or `develop`.** Every change starts on a feature branch. GitLab enforces this for pushes; the rule exists so you do not make the local mess that a rejected push leaves behind.
2. **One task, one branch, one merge request.** The branch is named for the task's issue: `feature/23-network-levels`.
3. **Start from the latest `develop`.** `git switch develop`, `git pull`, then `git switch -c feature/…`.
4. **Commit small and often**, each commit with a message that says why (Section 12.7, Commit messages a CCB would accept (p. 176)).
5. **Push at least at the end of every day.** Your branch on GitLab is your backup, and others can see your work.
6. **Never rewrite history that has been pushed.** No force push; GitLab's protections refuse it on shared branches, and the team agreed not to do it on feature branches either.
7. **Never commit secrets or files you did not mean to.** Look at `git status` before every commit.
8. **When unsure, stop and type `git status`**, then `git log --oneline --all --graph`. Most confusion ends there.

The day-to-day sequence for one task, start to merge request, looks like this. Chapter 15, Setting up GitLab (p. 226) adds the GitLab side, and shows the kit's `gitlab.pl flow` commands, which do these steps for you and fill in the merge request:

```
git switch develop
git pull
git switch -c feature/23-network-levels
vim model/physical/network/Network.sysml   # do the work
git add model/physical/network/Network.sysml
git commit                                 # Vim opens; write the message; :wq
git push -u origin feature/23-network-levels
```

When you push a new branch, GitLab's reply includes a link: `To create a merge request for feature/23-network-levels, visit: https://gitlab.example.com/tidewater/model/-/merge_requests/new?…`. Open it, and you are in Chapter 15, Setting up GitLab (p. 226).

12.18 Branching strategies

A **branching strategy** is a team's rule for which long-lived branches exist and how changes move between them. It is the part of git that most resembles a CM plan, and the one where your project may reasonably choose differently from Tidewater. There are three common strategies.

GitHub flow is the simplest. There is one long-lived branch, `main`. Every change is a short-lived branch from `main`, reviewed in a merge request (GitHub calls it a pull request), checked by automated tests, and merged back into `main`. `main` is always the current state, and anything on it may be published at any time. It suits small teams that publish continuously and never need to maintain an older released version.

Git flow (or gitflow) is the most structured. There are two long-lived branches: `develop`, where integrated work accumulates, and `main`, which holds only released versions, each tagged. Features branch from `develop` and merge back into it. When a release is due, a `release/` branch is cut from `develop`, stabilized (only fixes, release notes, version numbers), then merged into `main` and tagged, and merged back into `develop`. Urgent fixes to a released version are `hotfix/` branches from `main`, merged into both `main` and `develop`.

GitLab flow sits between the two and comes in two variants. In the *environment* variant, `main` is the integration branch and further branches represent places the work is deployed (`pre-production`, `production`); changes are promoted from one to the next by merge requests, and the testing at each step widens. In the *release* variant, `main` is the integration branch and a `release/` branch is cut for each released version as late as possible, receiving only serious fixes. Both variants follow an **upstream-first** rule: a fix always lands on `main` first and is then carried to the release or environment branch, never the other way round, so no fix is ever lost.

Table 12.3: Three branching strategies compared

	GitHub flow	GitLab flow	Git flow
Long-lived branches	`main`	`main` plus environment or release branches	`main` (releases) and `develop` (integration)
Where daily work merges	`main`	`main`	`develop`
How a release is made	any commit on `main`; tag it	promote by MR to the next environment, or cut a release branch late	release branch from `develop`, stabilize, merge to `main`, tag, merge back

	GitHub flow	GitLab flow	Git flow
Fixes to a released version	none (you release again)	upstream first: fix on `main`, carry to the release branch	hotfix branch from `main`, merged to `main` and `develop`
Complexity	low	medium	high
Suits	continuous publishing; one current version	staged deployment, or a few supported releases	formal, numbered releases that others depend on

12.18.1 Why Tidewater chose git flow

Jim, Bob and Tom argued about this for most of an afternoon in week zero, and the decision (D-2 in Section 13.11, The decisions the team wrote down (p. 210)) rests on three facts about Tidewater:

1. **Tidewater has formal baselines.** Analyses, contracts and the program's other teams cite a specific baseline of the system model, and the CCB approves each one. Git flow's `main` is exactly the list of approved baselines: every commit on `main` is a baseline merge, and every baseline is a protected tag on `main`. The deputy program manager understood that in one sentence.
2. **Daily work must never wait for a baseline.** Preparing `baseline-1` took a week of review and release notes. With git flow, that happened on a release branch while Bob and Tom kept merging into `develop`. In GitHub flow, the team would have had to freeze `main` for a week, or publish a baseline that included half-reviewed work.
3. **A released baseline may need a correction.** In sprint 24 the program office found a wrong satisfy relationship in `baseline-1`, which another program was using. A hotfix branch from `main` corrected it and the fix reached `develop` too (Chapter 43, Maintenance through the life cycle (p. 602)).

The team made two adjustments to textbook git flow. `develop` is GitLab's **default branch**, the one merge requests target unless told otherwise and the one where "Closes #N" closes issues, so that day-to-day work flows to the right place without anyone thinking about it. And a seventh kind of branch, `vendor/prime`, holds the prime's deliveries, which is the next section.

12.18.2 When your project should choose differently

Choose **GitLab flow (release variant)** if your model is published continuously (a model site everyone reads) and formal baselines are rare, or if you want one integration branch and find `develop` plus `main` confusing. Most of Tidewater's practices carry over unchanged: release branches become baselines, and the vendor branch works the same.

Choose **GitHub flow** if your model has no formal releases at all: an internal model used only by its own team, or a short study. It is the easiest to teach and the hardest to get wrong.

Choose **git flow** if, like Tidewater, people outside the team depend on specific approved versions, and those versions sometimes need corrections while new work continues.

Whatever you choose, write it down with its reason (Section 13.11, The decisions the team wrote down (p. 210)), protect the long-lived branches in GitLab, and keep the rule that nothing reaches a long-lived branch except through a reviewed merge request.

12.19 The vendor branch

The prime contractor delivers a new v1 model every quarter. The team converts each delivery to SysML v2 and then edits the converted text: closing TODO lines, removing clutter, writing what the converter could not. The question that kept Bob awake in week zero was: *when drop 2 arrives, what happens to all our edits?*

Converting drop 2 produces fresh text with every TODO back in place. Copying it over the team's edited files would erase three months of work. Hand-merging thirteen files every quarter would be the Cameo years again. The answer is a **vendor branch**: a branch that holds the deliveries exactly as converted, never edited by hand, and is merged into `develop` like any other branch. Git then does a **three-way merge**: it compares the prime's old and new version and applies the prime's changes to the team's edited version, leaving the team's edits alone wherever the prime did not touch the same lines.

You can see it work in a practice repository of its own. Make a skeleton:

```
$ cd ~
$ git init vendor-practice
Initialized empty Git repository in C:/Users/bob/vendor-practice/.git/
$ cd vendor-practice
$ echo "# Practice: the vendor branch" > README.md
$ git add README.md
$ git commit -m "Project skeleton"
[main (root-commit) 9243b02] Project skeleton
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
```

Then a vendor branch holding the first delivery's converted file (pasted here; on Tidewater the converter writes it):

```
git switch -c vendor/prime
mkdir -p model
cat > model/Stations.sysml <<'END'
package Stations {
    part def Buoy {
        // TODO stereotype Tidewater_Profile:L3Station on this element (xmi:id
        _22_1_7f3a0c1_0215) not converted
        doc /* A moored buoy. */
        part hull : Hull;
    }
    part def Hull;
}
END
```

```
$ git add -A
$ git commit -m "Import prime drop v1.0"
[vendor/prime 38c8272] Import prime drop v1.0
 1 file changed, 8 insertions(+)
 create mode 100644 model/Stations.sysml
$ git switch main
```

```
Switched to branch 'main'
$ git switch -c develop
Switched to a new branch 'develop'
$ git merge --no-ff vendor/prime -m "Merge branch 'vendor/prime' into 'develop'"
Merge made by the 'ort' strategy.
 model/Stations.sysml | 8 ++++++++
 1 file changed, 8 insertions(+)
 create mode 100644 model/Stations.sysml
```

The delivered file has a TODO line where the converter could not carry a level stereotype. On `develop`, the team closes it (replacing the TODO line with `@L3Station;`) and commits:

```
:edit model/Stations.sysml
:%s/^ *\/\/ TODO stereotype Tidewater_Profile:L3Station.*/       @L3Station;/
:write
```

```
$ git commit -am "Stations: Buoy is a level-3 station"
[develop c779858] Stations: Buoy is a level-3 station
 1 file changed, 1 insertion(+), 1 deletion(-)
```

Now drop 2 arrives. The prime added a mooring. Its converted text still has the TODO line, because the converter cannot carry the stereotype now any more than before. It goes onto the vendor branch exactly as converted:

```
git switch vendor/prime
cat > model/Stations.sysml <<'END'
package Stations {
    part def Buoy {
        // TODO stereotype Tidewater_Profile:L3Station on this element (xmi:id
        _22_1_7f3a0c1_0215) not converted
        doc /* A moored buoy. */
        part hull : Hull;
        part mooring : Mooring;
    }
    part def Hull;
    part def Mooring;
}
END
```

```
$ git commit -am "Import prime drop v1.1"
[vendor/prime 7e52f56] Import prime drop v1.1
 1 file changed, 2 insertions(+)
$ git switch develop
Switched to branch 'develop'
$ git merge --no-ff vendor/prime -m "Merge branch 'vendor/prime' into 'develop'"
Auto-merging model/Stations.sysml
Merge made by the 'ort' strategy.
 model/Stations.sysml | 2 ++
 1 file changed, 2 insertions(+)
```

```
$ cat model/Stations.sysml
package Stations {
    part def Buoy {
        @L3Station;
        doc /* A moored buoy. */
        part hull : Hull;
        part mooring : Mooring;
    }
    part def Hull;
    part def Mooring;
}
$ git log --oneline --graph --all
*   56f1116 Merge branch 'vendor/prime' into 'develop'
|\
| * 7e52f56 Import prime drop v1.1
* | c779858 Stations: Buoy is a level-3 station
* | 4223c6e Merge branch 'vendor/prime' into 'develop'
|\|
| * 38c8272 Import prime drop v1.0
|/
* 9243b02 Project skeleton
```

The merged file has both: the team's `@L3Station;` and the prime's new mooring. Git knew the TODO line had not changed between v1.0 and v1.1 on the prime's side, so the team's replacement of it stands. Bob stared at this for a long time.

When the prime *does* change a line the team also changed, git stops and asks. In drop 1.2 of the practice repository the prime changed the stereotype on the very element the team had tagged:

```
git switch vendor/prime
cat > model/Stations.sysml <<'END'
package Stations {
    part def Buoy {
        // TODO stereotype Tidewater_Profile:L2Segment on this element (xmi:id
        _22_1_7f3a0c1_0215) not converted
        doc /* A moored buoy. */
        part hull : Hull;
        part mooring : Mooring;
    }
    part def Hull;
    part def Mooring;
}
END
git commit -am "Import prime drop v1.2"
git switch develop
```

```
$ git merge --no-ff vendor/prime -m "Merge branch 'vendor/prime' into 'develop'"
Auto-merging model/Stations.sysml
CONFLICT (content): Merge conflict in model/Stations.sysml
Automatic merge failed; fix conflicts and then commit the result.
```

```
$ cat model/Stations.sysml
package Stations {
```

```
    part def Buoy {
<<<<<<< HEAD
        @L3Station;
||||||| 6b3d303
        // TODO stereotype Tidewater_Profile:L3Station on this element (xmi:id
        _22_1_7f3a0c1_0215) not converted
=======
        // TODO stereotype Tidewater_Profile:L2Segment on this element (xmi:id
        _22_1_7f3a0c1_0215) not converted
>>>>>>> vendor/prime
        doc /* A moored buoy. */
        part hull : Hull;
        part mooring : Mooring;
    }
    part def Hull;
    part def Mooring;
}
```

This conflict is exactly right. The prime reclassified the buoy, and the team's tag is now out of date. A person must look and decide (here: `@L2Segment;`). The conflict is not a failure of the method; it is the method telling the team precisely where the prime's change and the team's work collide. On Tidewater, each drop's merge produced a handful of these, and resolving them was a task in the drop's story (Chapter 43, Maintenance through the life cycle (p. 602)).

Three things make the vendor branch work, and each became a rule:

- **The converter's output must be stable.** The same v1 element must produce the same lines, in the same file, every time. The prime agreed to keep element identifiers stable between drops (Part I, Section 3.5, The prime's view (p. 27)), and the converter is deterministic.
- **Nobody edits the vendor branch by hand.** It holds only what the converter produced from what the prime delivered. Only Tom can push to it, enforced in GitLab.
- **The team's own additions go in files the converter never writes** (for Tidewater, `model/library/conventions/` and other government-owned files), or in place of TODO lines, so that conflicts appear only where they mean something.

12.20 Applying this to your project

Copy: the practice repository exercise, for everyone, in the first week. The eight rules. Merge commits, never squash, if your metrics or audits rely on history. Annotated, protected tags for baselines. The vendor branch, if a contractor keeps delivering a model you must keep editing.

Adapt: the branching strategy. Use the comparison table and the three facts that decided Tidewater's choice; your facts may point elsewhere. Branch naming: any scheme works if it contains the issue number.

Decide locally: SSH or HTTPS; whether feature branches may ever be force-pushed (Tidewater said no, to keep the rule simple); who may push to the vendor branch; and which files are generated and committed versus ignored.

12.21 Exercises

Exercise 12.1: The practice afternoon

Make a `practice` repository and do every command in this chapter, in order, with your own small SysML file. Then do each of these on purpose and recover: restore a file you broke; un-stage a file you staged; amend a commit message; revert a commit; abort a merge.

Exercise 12.2: A conflict with three sides

In your practice repository, make two branches from `main` that each change the same line differently, and a third change on `main` itself. Merge both branches into `main` one after the other. How many conflicts do you get? Resolve them so that the final line makes sense, and look at the result with `git log --oneline --all --graph`.

Exercise 12.3: The auditor's questions

In your practice repository, answer with one command each: who last changed line 3 of your SysML file, and in which commit? What did that commit change? What was the message of the commit three before the latest?

Exercise 12.4: A vendor branch of your own

Reproduce the vendor-branch session in Section 12.19, The vendor branch (p. 191). Then make a fourth "drop" in which the prime deletes the `Hull` part definition, after the team has added a `doc` comment to `Hull` on `develop`. What does git do with a change on one side and a deletion on the other? What should the team do?

Exercise 12.5: Choose a strategy

For a project you know, list the facts that matter for a branching strategy: are there formal baselines? Do others depend on specific versions? Do released versions get corrected? How often is the model published? Which strategy do the facts point to, and what would you write as its decision record?

CHAPTER

13

The operating model

From what you know →

You know

the integrated master schedule, phases, milestones and the weekly status meeting.

→ Here it is called

the operating model: project, tomes, epics, stories and tasks, in two-week sprints.

→ What is new

work is cut into pieces small enough to finish in days, and progress is counted, not reported.

Survival kit

What you have

a directive, a signed scorecard, a twelve-week outline, three people and a contractor who delivers a model every quarter.

What you need

a way to cut the work into pieces of a known size, a calendar the team keeps, a clear answer to "who decides?", and a written list of the decisions that shape everything else. That is the operating model.

13.1 Sunday evening, the whiteboard

On the Sunday before sprint 1, Jim stood in front of the whiteboard in the team room with a marker and a question the program manager had asked on Thursday: "When you say twelve weeks, twelve weeks of *what*?"

The honest answer, at that moment, was "twelve weeks of figuring it out." Jim knew that would not survive the first review. In the document era the answer would have been a work breakdown structure, an integrated master schedule and a systems engineering management plan: three documents, a month to write, approved by people who would never read them again. Jim did not have a month, and did not want three more documents. What Jim wanted was something

the team could carry in their heads, check every morning, and change on a Wednesday without a CCB.

By the time Bob and Tom arrived on Monday morning, the board held five boxes stacked like a parts tree, each with a size limit written next to it, a two-week calendar with the meetings marked, and three names with arrows to the things each would own. Bob looked at it for a while and said, "So it's a WBS and a schedule."

"It's a WBS and a schedule," Jim agreed. "With size limits that we actually enforce, and a schedule that repeats every two weeks instead of running for three years."

That whiteboard is this chapter.

13.2 Why an operating model, and why so small

An **operating model** is the answer to four questions: how is the work divided, when does the team meet and plan, who decides what, and which decisions have already been made. Every systems engineering organization has one, whether it writes it down or not. In the document era the written version was the SEMP, and the real version was the set of habits the team had built over twenty years: who you called before you changed an ICD, which spreadsheet held the truth this month, which review package was actually read.

DevSecOps does not abolish any of that. It changes three things about it.

First, **the pieces of work are small and of a known size**. In the document era a work package was sized to the contract: "System Specification, revision C" might be six months of effort with one milestone at the end. Nobody knew whether it was on track until the review. In the new way, no piece of work the team commits to is larger than two weeks, and most are a few days. Progress becomes visible because things finish.

Second, **the calendar is a heartbeat, not a timeline**. Instead of an integrated master schedule that runs for years with a few dozen milestones, the team runs the same two-week cycle again and again: plan, work, show, reflect. The long-range plan still exists (Tidewater's runs for thirty-nine sprints), but it is built out of these repeating cycles, and it is re-planned at every one.

Third, **the process is enforced by the platform, not by memory**. "Every change is reviewed" stops being a sentence in a SEMP and becomes a setting on a protected branch in GitLab (Chapter 15, Setting up GitLab (p. 226)). "Nobody changes a baseline" becomes a protected tag. The operating model says what the rule is; the tools make it true.

> **Case study: The SEMP the team did not write**
>
> Tom asked, in the first week, whether the team should update the SEMP to describe the new way of working. Jim's answer was to write a two-page "operating model" page in the GitLab wiki instead, and to put a one-paragraph pointer to it in the next SEMP revision. The wiki page is under version control like everything else, and it changed eleven times in the first twelve weeks. A SEMP revision takes a CCB. The team would have spent the twelve weeks either violating its own SEMP or waiting for approval to change it.

13.3 The five levels

Tidewater's work is divided into five levels. Each has a size limit, and the limit is the point: a level is defined by how long its pieces are allowed to take, not by what kind of work they hold.

Table 13.1: The five levels of work, with their size limits

Level	Size limit	Tidewater example	Where it lives	Who owns it
Project	years	"Tidewater SysML v2 port", 18 months (39 sprints)	the journal (one per project)	Jim, with the program manager
Tome	under 1 year (2 semesters, 26 sprints)	T2 "Port Tidewater v1 to SysML v2", sprints 1 to 20	the journal (`t:` tag); GitLab label `tome::port`	Jim
Epic	under 1 semester (13 sprints)	E6 "Structure: close the structure TODOs", sprints 2 to 8	the journal (`e:` tag); GitLab label `epic::structure`	Jim
Story	under 1 sprint (2 weeks)	"Close the Network TODOs", sprint 3	the journal (an item); a GitLab issue labeled `type::story`	the person assigned
Task	under 1 week (5 points or fewer)	"Replace the level stereotype TODOs in Network.sysml with @L tags", 1 point	GitLab only: an issue labeled `type::task`, closed by a merge request	the person assigned

A **project** is the whole effort, with a start, an end and a purpose a manager can state in a sentence. Tidewater's is "port the Tidewater system model from SysML v1 to SysML v2 in GitLab, then keep it current with the prime's deliveries." A project lasts years. It is the level the program manager funds and the level the program's leadership hears about.

A **tome** is a major body of work inside the project, the size of a large contract line item or a major phase. The word is unusual on purpose (it is the agile kit's word, and it means "a big book"): it does not collide with any term the team already uses, so nobody confuses a tome with a contractual "phase" or "increment." A tome must finish within a year. Tidewater has three.

An **epic** is a body of work inside a tome that delivers one recognizable capability or outcome, the size of a work package in an old WBS. An epic must finish within a semester, thirteen sprints. Tidewater has fourteen.

A **story** is a slice of work that produces something the team can show at the end of a sprint: a package with its TODO lines closed, a pipeline job that blocks a bad change, a set of discrepancy reports sent to the prime. A story must finish within one sprint. Stories are what the team plans with, commits to, and demonstrates.

A **task** is one step of a story that one person can finish in a week or less, usually a day or two: convert this file, write that job, review the other merge request. Tasks are what the team does each day. They are tracked in GitLab, where the work happens, and each model-changing task ends in exactly one merge request.

13.3.1 What a point is

Size is measured in **points**. On Tidewater one point is one **ideal person-day**: a day of focused work on the task by the person who will do it, with no meetings, no interruptions and no waiting.

Nobody has ideal days, and that is fine. Points are not hours on a timecard. They are a unit for comparing pieces of work with each other and with what the team has finished before.

Why ideal days rather than hours? Three reasons the team found persuasive:

- An estimate in hours invites false precision. Nobody knows whether converting a package will take 6 or 9 hours. Most people can tell whether it is a one-day job or a three-day job.
- Hours get compared with timecards, and then the conversation becomes about the timecard, not the work.
- Ideal days are easy to sanity-check against the calendar. A person has about eight points of capacity in a ten-day sprint once meetings, email and interruptions are taken out (see Section 13.7, Capacity (p. 204)).

The size limits translate directly: a task is at most 5 points (a person's working week), and a story is at most what fits in one sprint for the people working on it, which on Tidewater rarely exceeds 6 points.

Note: Why the agile kit says "task" for what Tidewater calls a story

The kit's own documentation calls a journal item a *task* and gives it the limit "under one sprint." That is the same thing Tidewater calls a *story*. Tidewater added a smaller level underneath, the one-week task, and keeps it only in GitLab. If you read the kit's help (`:help agile`) alongside this book, translate the kit's "task" as Tidewater's "story." The reason for the split is in Section 13.5, Where each level lives (p. 202).

13.3.2 Why each limit is where it is

The limits are not arbitrary. Each one is the longest a piece of work can run before the team loses the ability to see whether it is going well.

A task under one week keeps every merge request small enough to review in under an hour. A merge request that changes forty lines can be read line by line; one that changes four thousand gets a glance and an approval, which is the review-by-screenshot habit of the Cameo years (Section 2.7, Reviewing a drop by screenshot (p. 19)) in a new form. It also means the pipeline gives feedback within days of the work starting, not at the end of a month.

A story under one sprint means every story either finishes and is demonstrated at the sprint review, or visibly does not. There is no "eighty percent done" in a sprint review; there is done, and there is carried over. A story that is carried over is a signal, and the team looks at why (see Section 13.3.3, Splitting work that is too big (p. 200)).

An epic under one semester keeps the medium-range plan honest. A semester is about the horizon over which a team can predict its own work with any confidence. An epic that would run longer is really two outcomes, and splitting it gives the program manager two points at which to say "that one is done."

A tome under one year matches the program's planning and funding cycle. The program plans and budgets by fiscal year; a tome that crosses two fiscal years becomes hard to fund and harder to report.

A project of years is whatever the mission needs. The limits below it are what make a long project tractable: a project of thirty-nine sprints is thirty-nine short, visible, re-planned cycles,

not one long one.

Objection: "Real systems engineering work doesn't come in one-week pieces. An ICD takes months."

Answer

An ICD took months because it was one deliverable reviewed at the end. The work inside it was always made of smaller pieces: agree the message list, define each message, reconcile the units, review with the other segment. Each of those is a task or a story. What changes is that each piece is finished, reviewed and merged on its own, so the ICD is always in a known state instead of being "in work" for months. In Tidewater the equivalent of the ICD is the `Interfaces` package; it was ported, cleaned up and reviewed in five stories over three sprints, and was never in an unknown state. See Chapter 28, Sprint 3: the first red X, DOORS in the model, and a wait nobody named (p. 440).

13.3.3 Splitting work that is too big

The skill the team practiced most in the first twelve weeks was not git or SysML. It was splitting. Every planning session found at least one story that was too big, and the team learned a handful of ways to cut it.

Table 13.2: Splitting stories that would not fit in a sprint

Too big	Why it is too big	Split it by	Resulting stories
Port the Stations package	25 members, 18 TODO lines, two reviewers	sub-area	Stations level tags; Stations procured parts; Power system and parametric
Set up the pipeline	unknown amount of learning	first useful slice	Pipeline runs grammar and lint on every MR; then outline and TODO jobs; then trace
Import the requirements	three DOORS modules, drift to investigate	output vs investigation	Import the DOORS modules; Report drift and gaps to the prime
Write the security plan	the ISSM's review cycle is outside the team	what the team controls	Draft the toolchain description; ISSM review meeting; Answer the ISSM's comments
Behavior TODOs	four different constructs	construct	State machine; fork, pin and object flow; constraint; interaction

Three rules of thumb came out of those sessions:

1. **Split by result, not by activity.** "Analyze the Stations package" and "Fix the Stations package" are not a good split, because the first produces nothing anyone can see. "Stations level tags" and "Stations procured parts" each end in a merged change.
2. **Pull out the waiting.** Anything that depends on someone outside the team (the ISSM, the enterprise GitLab administrators, the prime) becomes its own story or task, so the rest of the work does not wait with it.
3. **When in doubt, do the smallest useful thing first.** The first pipeline story was two jobs. It was the most valuable story in sprint 1, because it proved the whole chain (branch, merge

request, pipeline, approval, merge) worked end to end. Everything after it was adding jobs to a chain that already worked.

13.4 The Tidewater tree

Here is the whole project as Jim laid it out in week zero. The sprint spans are the plan; Part III and Chapter 43, Maintenance through the life cycle (p. 602) show how reality compared.

Table 13.3: The Tidewater project tree: three tomes, fourteen epics

Tome	Epic	Sprints	What it delivers
T1 Stand up MBSE DevSecOps (S1 to S8)	E1 GitLab project and gitflow	1 to 5	the project, protections, branch rules, templates
T1	E2 Pipeline and quality gates	1 to 6	grammar, lint, outline, TODO, trace, docs and security jobs
T1	E3 Team practices and onboarding	2 to 6	laptops, working agreements, the rhythm, training
T1	E4 Toolchain security and authorization to use	2 to 8	the ISSM's approval of the runner, repository and data flows
T2 Port Tidewater v1 to SysML v2 (S1 to S20)	E5 Drop 1 intake and first import	1 to 2	drop 1 checked, converted and on the integration branch
T2	E6 Structure: close the structure TODOs	2 to 8	every structural TODO closed, level tags, clutter removed
T2	E7 Requirements and trace	3 to 10	DOORS imported, trace on every change, discrepancy reports
T2	E8 Behavior and analysis	6 to 13	state machines, interactions, flows and constraints written by hand
T2	E9 Drops 2 and 3 during the port	8 to 15	two prime deliveries absorbed while the port continues
T2	E10 Baseline 1 and acceptance	14 to 20	the first accepted baseline
T3 Maintain against prime drops (S17 to S39)	E11 Drop intake automation	17 to 22	the intake runbook automated
T3	E12 Drops 4 and 5	20 to 29	two deliveries absorbed; a rename and a constraint change
T3	E13 Drops 6 and 7	33 to 39	the last two deliveries; clutter removed upstream
T3	E14 Sustainment operations	27 to 39	monthly status, CCB, runner upgrades, backups, baselines 2 and 3

Check the limits against the table: no tome runs more than 26 sprints (T3 runs 23), and no epic more than 13 (E14 runs exactly 13). The tomes overlap. That is deliberate: maintenance begins before the port is finished, because the prime's third drop arrives in sprint 14 whether the port is done or not.

Below the epics are stories. Here is sprint 1, the first plan the team ever made in this form, exactly as the kickoff recorded it (story ids are journal ids; the # numbers under each story are its GitLab task issues):

docs/tidewater/sprints/01/plan.txt

```
Sprint 1 kickoff, 2026-10-05 09:00 (90 minutes)

Goal: A GitLab project the team can work in; drop 1 received and assessed
```

```
Capacity: 18.9 points (Jim 5, Bob 8, Tom 8 per full sprint, less holidays and leave)
Committed: 16.5 points in 7 stories (87% of capacity): engineering 4.5, platform 8.0 (48%),
learning 4.0 (24%)

TW-1     E3    4.0 pts  learning     Learn the tools: survival kit, git, Vim, the operating model
           #8    1.0 pt  jim  2026-10-05 .. 2026-10-06  Practice lessons 10-11; the operating
           model chapter
           #9    1.5 pt  bob  2026-10-05 .. 2026-10-06  Practice lessons 10-11; the survival kit,
           git and Vim chapters
           #10   1.5 pt  tom  2026-10-05 .. 2026-10-06  Practice lessons 10-11; the survival kit,
           git and Vim chapters
TW-2     E3    3.0 pts  platform     Laptops ready: Git for Windows, Vim, the kit
           #11   1.0 pt  jim  2026-10-07 .. 2026-10-08  Set up Jim's laptop; kickoff.pl doctor
           #12   1.0 pt  bob  2026-10-06 .. 2026-10-08  Set up Bob's laptop; kickoff.pl doctor
           #13   1.0 pt  tom  2026-10-06 .. 2026-10-08  Set up Tom's laptop; kickoff.pl doctor
TW-3     E1    3.0 pts  platform     Create the GitLab group and model project
           #14   1.0 pt  tom  2026-10-08 .. 2026-10-09  Apply project.conf with gitlab.pl setup;
           request the runner
           #15   1.0 pt  tom  2026-10-09 .. 2026-10-13  Prove a direct push to develop is refused
           #16   1.0 pt  tom  2026-10-13 .. 2026-10-14  Commit CODEOWNERS, the templates and
           .gitignore
TW-4     E2    1.5 pts  platform     Draft the first pipeline and run it locally
           #17   0.5 pt  tom  2026-10-14 .. 2026-10-15  Draft .gitlab-ci.yml: grammar, lint,
           outline
           #18   1.0 pt  tom  2026-10-15 .. 2026-10-16  Run the draft locally on drop 1 with
           gitlab.pl ci
TW-5     E5    1.5 pts  engineering  Receive drop 1 and record it
           #19   1.0 pt  jim  2026-10-09 .. 2026-10-13  Log the CDRL delivery and check the
           sha256 of every file
           #20   0.5 pt  jim  2026-10-14 .. 2026-10-14  Open the Prime Drop issue with its
           checklist
TW-6     E5    3.0 pts  engineering  Convert drop 1 and assess it
           #21   1.0 pt  bob  2026-10-08 .. 2026-10-09  Run both converters with --report on drop
           1
           #22   1.0 pt  bob  2026-10-09 .. 2026-10-13  Write the TODO inventory by package
           #23   1.0 pt  bob  2026-10-13 .. 2026-10-14  Estimate the port backlog with the team
TW-7     E3    0.5 pts  platform     Working agreements: Definition of Ready and Done
           #24   0.5 pt  jim  2026-10-15 .. 2026-10-15  Write the Definition of Ready and Done in
           the wiki
```

Seven stories and 17 tasks, 16.5 points. Two of the seven are not engineering at all: a learning story (practice lessons and the first chapters of this book, for all three) and a laptop story (Git for Windows, Vim and the kit on each machine, checked with `kickoff.pl doctor`). They are on the board because they take real time; a plan that hides them is a plan that slips. Every task is between half a point and a point and a half, well under the five-point limit. That is typical of a first sprint, when everything is new and the team does not yet trust its estimates; by sprint 4, three- and four-point tasks appear as people learn what a day of work in the new tools looks like.

13.5 Where each level lives

Each level lives in the place where the people who use it already look.

Stories, epics and tomes live in the team's journal, the plain-text ledger the agile kit keeps

for each project (Chapter 23, Stand-ups, the journal and the board (p. 367)). Each story is one journal item, with an id like `TW-12` (the number is the story's GitLab issue number, so the two always match), a size in points, an epic (`e:`) and a tome (`t:`). The journal is where the sprint's commitment, capacity, done and carried-over work are recorded, and where velocity, burn-down and the weekly report come from. It is the part of the operating model Jim owns and reports from.

Tasks live only in GitLab. Each task is a GitLab issue, labeled `type::task`, linked to its story's issue ("Part of #12"), and, if it changes the model, closed automatically by the merge request that does the work. That is where Bob and Tom spend their day.

Why not put tasks in the journal too? Because the kit follows a rule that turned out to suit the team well: the architect's record holds what the architect is accountable for, and the team's day-to-day breakdown stays in the team's own tool. Jim is accountable for stories finishing in the sprint they were committed to; Jim does not need, and should not manage, which of Bob's three tasks is done by Wednesday. GitLab already records that, in detail, for anyone who wants to look.

The mapping to GitLab objects depends on which tier of GitLab the organization licenses. Tidewater's enterprise GitLab is the Free tier (GitLab's name for its unpaid edition, which in an enterprise installation is still supported by the administrators), so the team used labels and milestones for the upper levels:

Table 13.4: The levels in GitLab, by tier

Level	GitLab Free (what Tidewater used)	GitLab Premium or Ultimate (if you have it)
Project	a group (`tidewater`) holding the model project	the same
Tome	a scoped label `tome::port`	a parent epic (Ultimate supports multi-level epics)
Epic	a scoped label `epic::structure`	a group epic, with child issues and a roadmap
Sprint	a milestone "Sprint 3" with start and due dates	an iteration in an iteration cadence
Story	an issue labeled `type::story`	the same, attached to its epic
Task	an issue labeled `type::task`, related to its story	a child task (a GitLab "task" work item) or the same issue

A **scoped label** is a GitLab label whose name contains `::`. An issue can carry only one label from each scope, so adding `workflow::review` to an issue automatically removes `workflow::in-progress`. That one feature is what makes labels usable as fields: an issue's type, epic, tome and workflow state are each exactly one value. Chapter 15, Setting up GitLab (p. 226) creates them from a file.

13.6 The cadence

Every sprint on Tidewater is ten working days, Monday of week one to Friday of week two, and every sprint has the same shape. The team's calendar invitations were created once, in week zero, and never changed.

Table 13.5: The two-week cadence

Day	When	Event	Length	Who	Produces
Day 1 (Monday)	09:00	Sprint kickoff and planning	90 minutes	team; program manager invited for the first 15	the sprint goal, capacity, committed stories, task breakdown, poker estimates
Every day	09:00	Stand-up	15 minutes	team	three terse lines per person (yesterday, today, blockers); blockers become actions
Every Friday	15:00	Weekly note	20 minutes (Jim alone)	Jim	the weekly quad page to the program manager
Wednesday of week 2	10:00	Backlog refinement	60 minutes	team	next sprint's stories meet the Definition of Ready; poker estimates
Day 10 (Friday)	10:00	Sprint review (demo)	60 minutes	team, program manager, deputy program manager; prime's modeler when a drop is involved	live demonstration from GitLab; scorecard; feedback
Day 10 (Friday)	11:00	Retrospective	60 minutes	team only	start, stop, continue; the pulse survey; one improvement for next sprint

Chapter 24, The rhythm: stand-ups, poker, refinement, kickoff, demo, retro (p. 377) describes each event in detail, with the real chat pastes, poker rounds and retrospective notes from the twelve weeks. Here the point is the shape: about 7 hours of meetings per person per sprint (averaged over the first six sprints, from the meeting records), out of eighty hours. In the document era, Bob counted the team's recurring meetings in a typical fortnight (status meetings, working groups, review dry-runs, the CCB) and got eleven hours, not counting the review packages themselves.

Why two weeks? The team considered one week and three. One week was too short to finish anything that involved the ISSM or the prime, and planning every week would have eaten a day a month. Three weeks was too long to wait for feedback in the first quarter, when everything was new. Two weeks gave six sprints in the twelve weeks the directive allowed, which meant six reviews at which the program manager could see progress, and six retrospectives at which the team could fix what was not working.

Holidays do not move the cadence. When a holiday falls in a sprint (12 October in sprint 1, 11 November in sprint 3, 26 November in sprint 4, 25 December in sprint 6), the sprint keeps its dates and loses capacity. The events on the holiday move to the nearest working day. A cadence that shifts with every holiday stops being a cadence.

13.7 Capacity

Capacity is how many points the team can expect to finish in a sprint, worked out before planning from the calendar, not from hope.

Each person has a standing capacity per full sprint, agreed in week zero:

- **Jim: 5 points.** Jim is the MBSE lead and spends roughly half of every sprint on things that are not stories: the program manager, the prime, the CCB, reviews of Bob’s and Tom’s merge requests, the weekly note. Five points is Jim’s honest remainder.
- **Bob: 8 points.** Ten working days, minus about two days for meetings, reviews of others’ work, email and the unexpected.
- **Tom: 8 points.** The same.

Then, before each planning session, the calendar is applied. A holiday removes a day for everyone; leave removes it for one person. Capacity is prorated by working days available out of ten. Sprint 1 had one holiday (Monday 12 October), so everyone had nine days:

Table 13.6: Sprint 1 capacity

Person	Standing capacity	Days available	Sprint 1 capacity
Jim	5	9 of 10	4.5
Bob	8	9 of 10	7.2
Tom	8	9 of 10	7.2
Team	21		the sum

The kickoff record shows the result, and the commitment the team made against it:

docs/tidewater/sprints/01/kickoff.txt

```
Capacity: 18.9 points (Jim 5, Bob 8, Tom 8 per full sprint, less holidays and leave)
Committed: 16.5 points in 7 stories (87% of capacity): engineering 4.5, platform 8.0 (48%),
learning 4.0 (24%)
```

The team then commits to between 85 and 95 percent of capacity. The margin is not padding; it is room for the unplanned work that every sprint contains (a pipeline that breaks, a question from the prime, a review that takes longer than it should). Sprint 1’s commitment, in the record above, is inside that band.

Capacity is recorded in the journal with the stand-up verb `cap` on the first day of the sprint. The kit’s metrics compare each sprint’s load (committed divided by capacity) against two thresholds: above 110 percent is overload, below 70 percent is under-use. Both raise a signal in the weekly report (Chapter 23, Stand-ups, the journal and the board (p. 367)).

Warning: The temptation in sprint 1

On the Friday before sprint 1, Jim’s first draft plan held 26 points: everything the team wanted done by the first review. Tom did the capacity arithmetic on the whiteboard and the draft went to 16.5, with a learning story and a laptop story that the first draft had left out. Jim later said that cutting those points was the single most useful thing done in week zero, because it meant sprint 1 finished. A first sprint that finishes everything it committed to is worth far more to a skeptical team than one that attempts everything and finishes half.

13.8 Two ways to count “done”

The book uses two measures of finished work, and says which one every time:

- **Story points done** is the journal's measure. A story counts only when all of its tasks are done; a story with one open task counts zero, and its remaining points carry over. This is the number behind velocity, predictability and the sprint report: sprint 4 finished 12 of 17 story points, because the Stations story carried over.
- **Task points done** is GitLab's measure: every task closed in the sprint counts, including the finished tasks of a story that carried over. It is the number behind the work mix (engineering, platform, learning) in the Bottom line up front (p. iii) and the scorecard: sprint 4 closed 17 task points.

Neither is wrong. Story points done tells management what was *delivered*; task points done tells the team where the *effort* went. Mixing them in one sentence is the mistake.

13.9 Roles and decisions

Three people do the work. Their roles were assigned in Jim's email (Section 3.8, The email Jim sent that night (p. 31)) and written into the project's settings, so that GitLab enforces them.

Jim, MBSE lead. Owns the model's content and direction: what must be in it, which work comes first (the order of the backlog), what "done" means. Owns tomes and epics, plans each sprint with the team, approves merge requests as a code owner, creates baselines, reports to the program manager, and talks to the prime about the content of each drop. In GitLab, a maintainer.

Bob, MBSE engineer. Does most of the model work: converts, cleans up, writes what the converter cannot, reviews Tom's changes to the model. Owns the model's structure in day-to-day terms. In GitLab, a developer: Bob can push feature branches and open merge requests but cannot push to protected branches or create baseline tags.

Tom, configuration management and DevSecOps engineer. Owns the pipeline, the runner, the project settings, the vendor branch where prime drops land, and the mechanics of releases and baselines. Reviews Bob's changes to the pipeline and project files. In GitLab, a maintainer.

Around them:

Table 13.7: Who does what (R responsible, A accountable, C consulted, I informed)

Activity	Jim	Bob	Tom	Program manager	Deputy PM (CCB chair)	ISSM	Prime	GitLab admins
Set project priorities and the backlog order	A/R	C	C	C	I			
Plan and commit a sprint	A	R	R	I				
Change the model (a feature branch and MR)	C	R	R					
Approve a model change (code owner)	R	R	C					
Change the pipeline or project settings	C	C	A/R			C		C
Create a baseline (tag and release)	A	C	R	I	C		I	
Accept a prime drop	A	R	R	I	I		C	

Activity	Jim	Bob	Tom	Program manager	Deputy PM (CCB chair)	ISSM	Prime	GitLab admins
Raise a discrepancy report to the prime	A/R	R	C	I			I	
Approve the toolchain for use	C		R	I		A		C
Provide accounts, runner registration			R					A
Report progress	A/R	C	C	I	I			

A **RACI** table like this one is familiar from the document era; what is new is that most of its cells are enforced. Bob cannot create a baseline because the protected-tag setting allows only maintainers. A change cannot reach the integration branch without a code owner's approval because the protected-branch setting requires it. The ISSM's "A" on toolchain approval is not enforced by GitLab, which is why it is a story with its own tasks (Chapter 27, Sprint 2: Drop 1 on develop, every change gated (p. 424)).

13.9.1 What happened to the CCB

The deputy program manager's first question, when Jim briefed the operating model, was: "Where does my CCB fit?"

The answer the team worked out, and the deputy accepted after the sprint 2 review, has three parts:

1. **Day-to-day model changes do not go to the CCB.** They are made on feature branches, reviewed in merge requests, checked by the pipeline and approved by a code owner. The approval, the reviewer's comments, the pipeline results and the change itself are recorded together in GitLab, permanently. That record is more complete than any CCB minutes the program ever kept.
2. **Baselines go to the CCB.** A baseline (a protected tag such as `baseline-1`, built from a release branch merged into `main`) is what analyses, contracts and other programs cite. Creating one requires Jim's approval in GitLab and the deputy's concurrence, recorded as a comment on the release merge request. The CCB reviews the release notes, which are generated from the merge requests since the last baseline, not written by hand.
3. **Disagreements go to the CCB.** If a reviewer and an author cannot agree in a merge request, or a change would affect another program's interfaces, the merge request is labeled for the CCB and waits. In twelve weeks this happened once.

The deputy's question ("can a change be traced from request to approval to implementation?") is answered by a chain of links that GitLab maintains automatically: the issue that asked for the change, the merge request that made it (which says "Closes #N"), the pipeline that checked it, the approval, and the merge commit on `develop`. Section 5.5, One change, end to end (p. 52) follows one change along that chain.

13.10 Definitions of Ready and Done

Two short checklists govern the edges of every sprint. The **Definition of Ready** says when a story may be pulled into a sprint; the **Definition of Done** says when a story may be called finished. Both live in the project's wiki, under version control, and both changed several times in the twelve weeks as the team learned what mattered.

The Definition of Ready exists because the most common reason a story was carried over in the document era was that nobody knew what it meant when work started. The Definition of Done exists because "done" meant something different to each of the three people until it was written down. Jim's first draft of each was a page long. The versions that worked are below; the cut-down versions are what the team actually checked at refinement and review.

GitLab Markdown: tools/gitlab/templates/wiki/Definition of Ready.md

```
# Definition of Ready

The entry criteria for a sprint: the checklist a story must meet before the team commits to it,
so nobody starts
work that is not understood. (Like the entry criteria of a review: if they are not met, the item
waits.)

A story can be pulled into a sprint (moved to %"Sprint N" and ~"workflow::ready") when:

- [ ] It fits in one sprint, and every task under it is at most 5 points (one point = one ideal
person-day).
- [ ] It carries exactly one ~"type::story", one ~"tome::..." and one ~"epic::..." label.
- [ ] The acceptance criteria are written as a checklist someone else can verify.
- [ ] The v1 elements (xmi:id) and DOORS ids it touches are named.
- [ ] Dependencies are linked: on the prime (a DR or a drop), or on another story (`/relate #N`,
*blocked by*).
- [ ] Jim has ordered it on the backlog board.

A **task** is ready when it names its story ("Part of #N"), has an estimate (`/estimate`) and an
assignee.

> [!tip]
> If a story does not fit in a sprint, split it by package, by requirement module, or by "convert
/ close TODOs /
> review". Never carry a story on purpose.
```

GitLab Markdown: tools/gitlab/templates/wiki/Definition of Done.md

```
# Definition of Done

"Done" means the same thing for everyone, and someone other than the author can check it. Think
of it as the closure
criteria of an action item, written once for every item of that kind.

## A task is done when

- [ ] **Its merge request is merged into `develop`**, the master copy. (The MR description says
`Closes #task`, so the
      task closes by itself.)
```

```
- [ ] **The pipeline passed.** These are the automated audit checks, each in plain words:

  | Check | What it means |
  |---|---|
  | `grammar` | every model file is valid SysML v2 text |
  | `lint` **0/0** | every name points at something that exists; zero errors, zero warnings |
  | `outline` | every level of the part tree has 2 to 9 children, and each part carries its level
  tag |
  | `secrets` | no password, token or key was committed |
  | `docs` | the generated documents match the model |

- [ ] **It adds no new `// TODO` line**, unless a task exists for it. (A TODO line is something
the converter could not
      translate; a person must close it.)
- [ ] **The owner approved it.** For a requirement change that is Jim; for structure, Bob or Jim;
for the pipeline, Tom.

## A story is done when

- [ ] Every task under it is closed.
- [ ] Its acceptance criteria are checked, by someone other than the author.
- [ ] The **trace** check shows no *new* gap: no new requirement without something that satisfies
it (an *orphan*), no
      new requirement without a verification (*unverified*), and no new difference between DOORS
      and the model
      (*drift*). A gap that is the prime's to fix has a discrepancy report.
- [ ] It was shown at the sprint review, in the model or on the model's web site.

## A drop intake is done when

- [ ] The drop is on the vendor branch `vendor/prime`, filed in the package registry, and merged
into `develop`.
- [ ] Every merge conflict and every *manual* line of the delta report is resolved, with a task
for each.
- [ ] Discrepancy reports this drop fixes are closed; new ones are open and sent to the prime.

## Writing the change's reason (the commit message)

A commit message is the "reason for change" block. Write a short subject saying what the change
does, a blank line,
why, and then trailers that GitLab and the changelog read:

```text
Close the BuoyModes TODO with a state definition

The converter does not translate state machines. Written from the v1
diagram "stm BuoyModes" (drop v1.0, xmi:id _22_1_7f3a0c1_9c2e...).

Changelog: added
Issue: #23
Reviewed-by: Jim <jim@example.invalid>
```

> [!tip]
> If you cannot meet one of these, say so in the MR and ask. A known, written exception is fine;
a silent one is not.
```

Tip: Keep them short enough to check aloud

If a definition cannot be read aloud at the review in under a minute, it will not be checked. The team's rule: every line must be something a person can answer yes or no by looking at GitLab, not by remembering.

13.11 The decisions the team wrote down

The last part of the operating model is a short list of decisions that shape everything else, each with its reason and the condition under which the team would revisit it. Jim kept them as a numbered list on the wiki's Home page and added to it whenever the team argued about something for more than ten minutes. These ten were made in week zero and sprint 1; each is explained fully in the chapter shown.

Table 13.8: The decisions behind the operating model

#	Decision	Why	Revisit if
D-1	The v2 model is SysML v2 textual notation in a git repository	text can be compared, merged, checked and reviewed line by line; see Chapter 6, SysML v2 for SysML v1 modelers (p. 57)	the program mandates a SysML v2 tool with its own repository
D-2	Gitflow branching: `develop` is the default (integration) branch; `main` holds baselines only	baselines are released to other programs; day-to-day work must never wait on a release; see Section 12.18, Branching strategies (p. 189)	releases stop mattering (then switch to GitLab flow)
D-3	Every merge is a merge commit; no squash, no fast-forward	the history of who changed what feeds the metrics and the audit trail	never
D-4	DOORS stays authoritative for requirement text; the model holds a converted copy	the prime and the program manage requirements in DOORS; the trace job reports drift	the program moves requirements into the model
D-5	Prime drops land on a vendor branch, converted and never hand-edited, and are merged into `develop`	git can then merge the prime's changes with the team's own edits; see Chapter 12, Git from zero, for systems engineers (p. 171) and Chapter 43, Maintenance through the life cycle (p. 602)	the prime delivers SysML v2 directly
D-6	The team never edits the v1 model; problems in it go to the prime as discrepancy reports	the prime owns the v1 model under the contract	the contract changes ownership
D-7	Conversion debt (TODO lines left by the converter) is the primary port progress measure	it is computed, countable and indisputable; see Section 3.6, Defining measurable progress (p. 28)	the TODO count reaches zero
D-8	Tasks are at most 5 points; stories fit one sprint; tasks live in GitLab, stories in the journal	small merge requests are reviewed properly; see Section 13.3.2, Why each limit is where it is (p. 199)	the team grows past about eight people
D-9	The pipeline runs on a Windows shell runner with Git for Windows' Perl	the same tools as the laptops; nothing new to approve; see Chapter 17, The DevSecOps tools that earn their keep (p. 255)	the enterprise offers an approved Linux runner with Perl

#	Decision	Why	Revisit if
D-10	Everything that can be a file is a file under version control: settings, templates, pipeline, wiki, decisions	it can be reviewed, rebuilt and audited	never

A **decision record** need not be long. The team used this template in the wiki, one per decision, and the table above is their index:

```
## D-5 Prime drops land on a vendor branch

**Decided:** 2026-10-07, Jim (with Bob and Tom)
**Context:** The prime keeps changing the v1 model and delivers it every quarter. We
will edit the converted v2 text (close TODOs, remove clutter). We need each new
drop's changes without losing ours.
**Decision:** Convert each drop as delivered onto `vendor/prime` (never hand-edited)
and merge that branch into `develop` through a merge request.
**Consequences:** Git does a three-way merge; conflicts appear only where both
sides changed the same lines. Converter output paths must stay stable.
**Revisit if:** the prime delivers SysML v2 directly.
```

Case study: Writing it down is the point

Bob, who argued hardest against several of these decisions, said at the sprint 3 retrospective that the decision list was the thing that made the arguments bearable. "When I lost an argument, it was written down with the reason and the condition for changing it. I didn't have to keep re-fighting it; I had to watch for the condition." Two of the ten were revisited in the eighteen months. Neither was reversed.

13.12 What the operating model is not

It is not Scrum, SAFe or any other branded method, although it borrows freely from them, and the team did not attend a certification course. Jim's view was that the program needed a way of working that three systems engineers could explain to a program manager in five minutes, not a method that needed a coach.

It is not a replacement for the program's systems engineering processes. Requirements are still managed in DOORS. The CCB still meets. The SEMP still exists, with a pointer to the wiki. What changes is the unit of work (small), the rhythm (two weeks, repeated), and the enforcement (by the platform).

It is not fixed. The operating model page changed eleven times in twelve weeks, every change through a merge request like any other file. The retrospective at the end of each sprint is where changes to it are proposed.

13.13 Applying this to your project

Copy these as they are:

- **The size limits.** Task under a week, story under a sprint, epic under a semester, tome under a year. They work for any team of two to eight people doing model work.
- **Points as ideal person-days**, with standing capacity per person and prorating for holidays and leave.
- **The cadence**: one planning day, a daily fifteen-minute stand-up, a mid-sprint refinement, a review and a retrospective at the end. Book it all in the calendar once.
- **Writing decisions down** with reason and revisit condition.

Adapt these to your situation:

- **The number of tomes and epics.** Draw your own tree. Start from the outcomes your program manager will recognize, then check each against the size limits and split what does not fit.
- **The RACI around the team.** Your CCB, security approver and contractor relationships will differ. The question to answer for each is: which of their decisions can be enforced by a GitLab setting, and which must be a story?
- **The GitLab mapping.** If your organization licenses Premium or Ultimate, use epics and iterations; the operating model does not change.

Decide these locally, before sprint 1:

- Each person's standing capacity. Be honest about how much of the lead's time goes to things that are not stories.
- Who approves what, and who may create a baseline.
- Your decision list. Ten decisions is a good start; do not try to decide everything in advance.

13.14 Exercises

Exercise 13.1: Size the tasks

Take a work package from your own program (a document section, an interface definition, a test procedure) and split it into stories that each fit in one sprint and tasks that each fit in one week. Give each task a size in ideal person-days. Which split did you find hardest, and which of the three rules of thumb in Section 13.3.3, Splitting work that is too big (p. 200) helped?

Exercise 13.2: Split a story

The story "Close the Behavior TODOs" covers a state machine, an interaction, a fork with pins and object flows, and a constraint expression. Bob estimated it at 11 points. Split it into stories of at most 6 points each, by the rule "split by result." Name each resulting story so that its end state is something that can be demonstrated.

Exercise 13.3: Compute a sprint's capacity

Sprint 4 runs 16 to 27 November 2026. Thanksgiving (26 November) is a holiday, and Jim takes leave on Friday 27 November. Using the standing capacities above, compute each

person's capacity and the team's, and the range of points the team should commit to.

Exercise 13.4: Write a decision record

Write a decision record, in the template's format, for one decision your own team has argued about more than once (a naming convention, a review rule, a tool choice). Include the condition under which you would revisit it.

Exercise 13.5: Enforce a RACI cell

Pick three cells of your own team's RACI. For each, say whether a GitLab setting could enforce it (which one?), or whether it must be handled as a story with tasks. Use Chapter 15, Setting up GitLab (p. 226) once you have read it to check your answers.

CHAPTER

14

Writing stories and tasks for model work

From what you know →

You know

action items and work packages.

→ Here it is called

stories and tasks, each with acceptance criteria.

→ What is new

every story fits in two weeks and every task in one, and "done" is something a pipeline can check.

Survival kit

What you have

years of work breakdown structures, action item lists and IMS lines. You know how to break a program into deliverables, and you know how a schedule line that says "Model update – 40% complete" can stay at 40% for three months.

What you need

to write the small pieces of work this process runs on – a story that fits one sprint and ends in something you can show, tasks under a week that each end in a merged change – and to write their "done" so that a machine can check most of it. You also need to budget the time it takes to learn all of this, before it gets taken from your evenings.

Note: Core path

This chapter is CORE: you write stories and tasks from the first sprint planning on. The splitting rules and the size limits themselves are in Section 13.3.3, Splitting work that is too big (p. 200) and Section 13.3.2, Why each limit is where it is (p. 199); this chapter is about writing the issues.

14.1 The Monday the backlog was empty

Jim's first sprint planning, on 5 October, was supposed to take ninety minutes. It took three hours, and most of it went on a single question Bob asked about the first card on the board: "What does 'Port the Stations package' mean when it's done?"

Nobody could say. In the document years, a line like that meant "Bob is working on Stations," and it would be done when Bob said it was done, which would be at the review. In the Cameo years it meant the same thing, with a model instead of a document. The line had no edge. It could not be finished, only abandoned.

By the end of the three hours the team had rewritten that one line into four stories, and two of them still did not fit the sprint. The rest of this chapter is what they learned that afternoon and in the five plannings after it, written down so you can get there in less than three hours.

14.2 Stories, tasks and where each lives

Recall the levels from Section 13.3, The five levels (p. 198): a project holds tomes, a tome holds epics, an epic holds stories, a story holds tasks. Two of those levels are written by the team every sprint, and they live in different places:

Table 14.1: The two levels the team writes every sprint

Level	Size	Lives in	Written by	Ends in
Story	less than one sprint	a GitLab issue labelled `type::story`, and one line in the kit's journal (TW-<issue number>)	the person who owns it, at refinement or kickoff	something shown at the sprint review
Task	at most 5 points (a working week)	a GitLab issue labelled `type::task`, linked to its story	the person doing it, at kickoff	one merged change, or one recorded result

The journal tracks stories because the architect's view of the program is stories, epics and tomes; the hours and the one-day steps belong to the team (Section 13.5, Where each level lives (p. 202)). GitLab holds both, because the tasks are where the merge requests are.

14.3 A story in systems-engineering words

Software teams test a story against the acronym INVEST. Translated into the words of a systems engineer, the six letters are six questions you already ask of a work package:

Table 14.2: INVEST, translated

Letter	Software word	The question in SE words	A Tidewater "no"
I	Independent	Can this be done without waiting for another open item?	"Close the Stations TODOs" while the layout decision is still open
N	Negotiable	Is the *outcome* fixed and the method left to the person doing it?	"Use sed to replace the level TODO lines"
V	Valuable	Would the program office care if it were done?	"Reformat Interfaces.sysml"

Letter	Software word	The question in SE words	A Tidewater “no”
E	Estimable	Do we know enough to size it, even roughly?	“Fix whatever drop 2 breaks” before drop 2 arrives
S	Small	Does it fit in one sprint with room to spare?	“Port the Stations package” (25 members, 18 TODO lines)
T	Testable	Could someone who did not do it check that it is done?	“Improve the trace”

The last one is the one that matters most and the one engineers find hardest. “Improve the trace” is a direction, not a result. “No requirement in the Stations segment without a verify, checked by the trace job” is a result.

14.4 Writing the story issue

Every story is a GitLab issue made from the Story template (`.gitlab/issue_templates/Story.md`, Section I.3.1, Story (p. 748)). The template has four parts, and each one answers a question the program office will ask.

The title: a verb and an object. “Close the Stations level tags.” “Grade our requirements with the recipe.” “Data exchange table for the program review.” Not “Stations,” not “Recipe work,” not “Support the review.” A title with a verb can be done; a title that is a noun is a topic.

Why: one or two sentences, for the program. Who gains what. “The part tree shows every Stations element at its level, so the management site’s tree view is readable and the outline check can count them.” If you cannot write this line, the story is probably platform work disguised as engineering, or it is not worth doing this sprint.

What done looks like: a checklist someone else can tick. This is the acceptance criteria. Each line must be checkable by a person who did not do the work. The next section is about writing these lines so that a machine ticks most of them.

Tasks: one line per task issue, each under five points. The tasks are written at kickoff by the people who will do them, not by Jim the week before.

Here are three of Tidewater’s real stories from the first twelve weeks, as they were filed, with their acceptance criteria (these come straight from `sim/tidewater-plan.pl`, the plan the simulation runs):

Table 14.3: Three stories and their acceptance criteria

Story (sprint)	Done looks like
Laptops ready: Git for Windows, Vim, the kit (S1)	`kickoff.pl doctor` passes on all three laptops
Close the Network TODO lines (S3)	No TODO lines in `Network.sysml`; the pipeline is green
The trace job becomes a gate (S5)	The trace job fails an MR that adds an orphan, an unverified requirement or drift

Every one of those lines can be checked by running something. None of them says “reviewed by Jim” or “looks good,” because the review is part of every merge request already (Chapter 22, Reviewing a model change (p. 357)).

14.5 Acceptance criteria as pipeline checks

The habit that changed the team's plannings most was a simple one: write each acceptance criterion as something a job in the pipeline reports. Then "done" is not an opinion, and nobody has to remember to check.

Table 14.4: Acceptance criteria a machine can check

Instead of	Write	Who checks it
Stations is cleaned up	No `// TODO` lines in `model/physical/stations/`	the `todo` job's count for that folder
Requirements are traced	The trace job shows no new orphan, unverified or drifted requirement	the `trace` job (a ratchet from sprint 5)
The model is correct	Lint 0 errors 0 warnings; the grammar check passes	the `lint` and `grammar` jobs
The documents are updated	`docs/` regenerated; the docs job is green	the `docs` job
The spider web is smaller	One package dependency cycle fewer in the package view	`model.pl draw pkg` (Chapter 11, Seeing the model: diagrams from text (p. 158))
Requirements are better written	The recipe report shows no finding for the Stations requirements	the `recipe` job (Chapter 7, Requirements a pipeline can check: the recipe (p. 73))
Every Stations requirement is verified	The VCRM shows 100% verify coverage for Stations	the `vcrm` job

Not every criterion can be a job. "The program office has seen the site" is a meeting. "DR-1 to DR-4 are sent to the prime through the contract channel" is a transmittal record. Those are fine: write them as a fact that anyone can confirm from a record ("the review notes list the program office attendees"; "the DR issues carry the transmittal date"). The rule is not "only machine checks"; the rule is "no opinions."

Tip: Write the criteria before the estimate

Planning poker (Section 24.3, Planning poker (p. 382)) goes faster when the criteria are already on the card. Half of Tidewater's estimate disagreements in sprint 1 were really disagreements about what done meant. In the poker for "Convert drop 1 and assess it" Bob voted 8 and Jim voted 3; once the criteria were written ("both converter reports attached, the TODO inventory by package on the wiki, the backlog estimated"), the second round was 3, 3, 3.

14.6 Writing the task issues

A task is the smallest piece anyone tracks: at most five points, done by one person, ending in one merged change or one recorded result. Tasks are GitLab issues of type `task`, made from the Task template, with three things the story does not have:

- **"Part of #N"** on the first line of the description, naming the story, and a `/relate #N` quick action, so the story's page lists its tasks.

- **An estimate** with `/estimate 2d`. The team uses days because a point is an ideal day (Section 13.3.1, What a point is (p. 198)).
- **The merge request closes the task**, not the story. The MR description says `Closes #42` for the task; the story closes when its last task does (Jim closes it at the review if nobody did).

Tasks for model work fall into a few recognizable kinds. Knowing them makes kickoff faster:

Table 14.5: The kinds of task in model work

Kind	Example from Tidewater	Typical size	Ends in
Close TODO lines	Level tags in Network	1 to 3	an MR that removes TODO lines
Hand-write a construct	Write the BuoyModes state def; exhibit it on MooredBuoy	2 to 4	an MR with new SysML v2 text
Pipeline job	Add the trace job (report only) and the secrets job	1 to 3	an MR to `.gitlab-ci.yml` and `ci/job.pl`
Prime drop step	Import drop 1 onto vendor/prime	1 to 2	the vendor commit and the merge MR
Decision	Decide the layered layout; the layout map	1	an ADR in `docs/adr/`
Record or report	Open DR-1 to DR-4	1 to 2	issues sent to the prime
Meeting with an outcome	Agree the delivery interface with the prime	0.5 to 1	notes and a decision on the wiki
Learning	Practice lessons 10-11; the survival kit, git and Vim chapters	0.5 to 2	a lesson checked as done

If a task does not end in one of those things, it is not a task yet. "Look at the Stations package" ends in nothing. "Write the TODO inventory by package" ends in a wiki page.

14.7 Try it: a story and its first task

Practice lesson 11's sandbox holds an offline GitLab with the Tidewater project and its templates. Make a story from the Story template and a task under it, then look at both. First, step into the clone and load the lesson's environment (it defines `gl` as a short name for `gitlab.pl`):

```
. ./env.sh > /dev/null
cd work
gl issue new "Close the Stations level tags" --template Story --label type::story --milestone
"Sprint 1"
```

The story is issue #2 (the lesson already filed #1). Now the task, with its link and estimate given as quick actions in a note – the same text you would type in the comment box on the web page (the block loads the environment again with `. ../env.sh` only because the book's test runs each block in a fresh shell; in your terminal once is enough):

```
. ../env.sh > /dev/null
gl issue new "Level tags in Stations" --type task --label type::task --milestone "Sprint 1"
gl issue note 3 "Part of #2
/estimate 2d
```

```
/relate #2"
gl issue show 3
```

The last command shows the task with the two system notes the quick actions left behind ("changed time estimate to 2d", "marked this issue as related to #2"). Open the story with `gl issue show 2` and fill in its "Why" and "What done looks like" from the template's comments; in the real GitLab you would do that in the web editor or, as the team did, in Vim (Section 10.10, Commit messages, issues and merge requests in Vim (p. 153)).

Exercise 14.1: Rewrite three criteria

Rewrite these acceptance criteria so a pipeline job or a record can check them: (a) "The Behavior package is finished"; (b) "Traceability is improved for the segment requirements"; (c) "The prime has been told about the problems we found."

14.8 Splitting patterns for model work

Section 13.3.3, Splitting work that is too big (p. 200) gives the three rules (split by result, pull out the waiting, smallest useful thing first). Model work has its own natural seams, and these are the ones the team used most:

1. **By package.** The converter writes one file per package, and packages are owned by people (Chapter 16, The model repository: what goes where (p. 245)). "Close the structure TODOs" became one story per segment folder: Network, Stations, DataCenter, Operations.
2. **By TODO kind.** The converter's TODO lines name their kind (stereotype, state machine, interaction, include, fork, constraint). Level tags are quick and mechanical; a state machine is a design task. Never put both in one story: the quick ones finish and the hard one carries over, and the whole story shows as late.
3. **By requirement group.** For trace and verification work, split by the DOORS module or the system requirement heading: "Stations verification cases," "Data center verification cases."
4. **By interface.** An interface touches two owners. One story per interface keeps the reviews short and the right people on them.
5. **By drop change.** When a prime drop arrives, its change requests (NWM-CR-... numbers in the drop's change list) are natural stories: one per significant change, each with its disposition (Chapter 45, The prime's model changes: receive, assess, disposition (p. 619)).
6. **Investigation apart from result.** "Find out why drop 3's Storm state does not appear in v2" is a one-day spike; "Add the Storm state by hand" is the task that follows.

Case study: The Stations story that would not fit

Bob's first try at the Stations work was one story of 9 points: level tags, procured parts, the power system's parametric and the state machine. Planning poker came back 5, 8, 13. Splitting it by TODO kind gave four stories: "Close the Stations level tags" (5, sprint 4, with its last two points carried into sprint 5), "Procured parts as metadata" (sprint 5, together with the DataCenter tags), the parametric (later, in the behavior and analysis epic), and "BuoyModes state machine by hand" (3, sprint 6). The level tags were the first visible dent

> in the TODO burn-down that management saw.

14.9 Sizing: points, poker and the first sprints

Sizes come from planning poker (Section 24.3, Planning poker (p. 382)): each person votes a number of points, the outliers explain themselves, and the team votes again. A few things the team learned about sizing model work:

- **The first time is always bigger.** The first package with level tags took Bob three points; the third took one. Size the first of a kind generously and expect the next ones to shrink. Velocity in the first two sprints is not a forecast of anything.
- **Waiting is not effort.** A task blocked for three days on the enterprise GitLab administrators is not a five-point task. Size the effort, mark the wait (`/label ~workflow::blocked`), and say it at stand-up as an ask.
- **Half points are fine, quarter points are not.** A half-day task exists (send the DR, hold the meeting). Anything smaller is part of another task.
- **Engineering, platform and learning are labelled, not hidden.** Every story carries `work::engineering`, `work::platform` or `work::learning`. The scorecard shows the mix sprint by sprint, so management can see the platform work shrinking and the engineering growing.

14.10 The learning budget

No line in the first plan Jim drafted said "learn git." The first plan held 26 points against a capacity of about 19, and every point was work. The team would have learned git anyway – at night, at home, or by failing in front of each other in merge requests.

The plan that ran instead budgets the learning as stories, labelled `work::learning`, with real points:

Table 14.6: Tidewater's learning budget (share of each sprint's task points done, Section 13.8, Two ways to count "done" (p. 205))

Sprint	Learning share	What it covers
1	24%	practice lessons 10 and 11; the survival kit, git, Vim and operating-model chapters
2	16%	practice lessons 12 and 13; Vim for models; the diagrams chapter; the SysML v2 workbook, first half
3 and 4	17% and 9%	the workbook, second half; talking about progress; reviewing a model change
5 and 6	6% and 4%	the requirement recipe; memos and briefings

Sprint 1's learning story is a good model to copy. Its title is a verb and an object ("Learn the tools: survival kit, git, Vim, the operating model"); its done is checkable ("each of us finished practice lessons 10 and 11"; "each of us read the primer's Part II chapters for our role"); and its tasks are per person, sized for that person (Jim 1, Bob 1.5, Tom 1.5), because the three do not start from the same place.

Objection: "Management will see 20% of the sprint spent on training and cut it."

Answer

Management sees it either way. Unbudgeted learning shows up as missed commitments, carried-over stories and three tired engineers. Budgeted learning shows up as a line that shrinks every sprint while the engineering line grows – which is exactly the picture Jim showed at the sprint 6 briefing (Chapter 35, The briefing (p. 532)). A visible, shrinking cost is a much easier conversation than an invisible one.

The same goes for the other setup work a new team forgets: laptops (one point each, plus the days waiting on the software request), the request for enablers memo, the conventions documents, the first demo rehearsal. If it takes time, it is a task. If it is a task, it is on the board.

14.11 The Definition of Ready, applied

A story is ready to bring into a sprint when the team can start it on Monday without a meeting. Tidewater's Definition of Ready is short (Section 13.10, Definitions of Ready and Done (p. 208)); in practice the refinement session checks five things for every candidate story:

Checklist: Ready for a sprint

- ☐ The title is a verb and an object.
- ☐ "Why" is written, in the program's words.
- ☐ Every acceptance criterion is checkable by a job or a record.
- ☐ Nothing it needs is waiting outside the team, or the waiting is its own task.
- ☐ It has been through planning poker and fits the sprint with room to spare.

Stories that pass get the `workflow::ready` label. Stories that do not stay in the backlog, and the conversation about why is usually the most useful part of refinement.

14.12 A worked example: from an epic to its tasks

Here is one epic taken all the way down, as the team did it over sprints 2 to 6. The epic is E6, *Structure: close the structure TODOs*. Its outcome, from the epic issue: every structural TODO line in the converted model closed, the level tags in place, the clutter of the document port removed – so the part tree and the outline check are true.

The stories. At the sprint 2 refinement Jim and Bob listed what "every structural TODO closed" meant, package by package and TODO kind by TODO kind, and wrote one story per result:

Table 14.7: Epic E6 as stories

Sprint	Story	Owner	Why it is its own story
2	Level conventions for the part tree	Bob	everything else needs the level tags to exist first
3	Close the Network TODO lines	Bob	one package, two kinds of TODO (levels, an include)
4	Close the Stations level tags	Bob	the biggest package; levels only

Sprint	Story	Owner	Why it is its own story
4	Units and value types	Bob	a different kind of TODO (units), a different file
4	Break the Network to Operations cycle	Tom	a spider-web result, judged by the package view
5	Procured parts as metadata; DataCenter level tags	Bob	the second TODO kind in Stations, plus a small package
5	Untangle the Legacy Import package	Tom	clutter removal, with a DR to the prime
6	Level tags in Operations	Tom	the last package with level TODOs

No story mixes a quick kind of TODO with a slow one; no story spans two packages unless one of them is tiny; each has an owner; and the dependency (conventions first) is a story of its own at the start.

One story, written out. Here is "Close the Network TODO lines" as Bob filed it from the Story template:

```
## Why

The Network package is the top of the part tree. With its level tags and the use case include in
place, the tree view
on the model site starts at the right level, and the outline check counts Network correctly.

## What "done" looks like

- [ ] No `// TODO` lines in `model/physical/network/Network.sysml` (the todo job's per-file count
is 0)
- [ ] The pipeline is green on the MR (grammar, lint 0/0, outline, docs current)
- [ ] The trace job shows no new orphan, unverified or drifted requirement

## Tasks

| Task | Points | Who |
|---|---|---|
| #59 Level tags in Network | 1 | @bob |
| #60 The include use case in ReviewObservations | 2 | @bob |
```

The tasks. Each task issue says "Part of" the story, carries its estimate, and is closed by one merge request. The include task was two points, not one, because nobody had written a SysML v2 `include use case` before; Bob's merge request description says so, and links the workbook section Bob used (Chapter 8, SysML v2 by example: a workbook (p. 92)). The next include, in a later sprint, took half a day.

14.13 The story in the journal

The kit's journal keeps one line per story, not per task (Chapter 23, Stand-ups, the journal and the board (p. 367)). When a story is committed at kickoff, the stand-up file for that day gets two lines; when its last task closes, one more:

```
new TW-51 3 "Close the Network TODO lines" p:2 e:"E6 Structure" t:"T2 Port" o:Bob
commit TW-51 Bob
```

```
...
done TW-51
```

The id is TW- and the GitLab story issue number, so anyone can go from the journal to the issue and back. Points in the journal are the story's total (the sum of its tasks); the journal is what produces the sprint's committed, done and carried-over numbers, the velocity, and the quad (Section 24.8, Telling management every week (p. 387)).

14.14 Common mistakes, and their fixes

Every one of these happened on Tidewater in the first six sprints:

Table 14.8: Mistakes in the first six sprints

Mistake	What it looked like	Fix
A topic, not a result	"Stations" as a story title	a verb and an object: "Close the Stations level tags"
An activity, not a result	"Analyze the trace gaps"	name what the analysis produces: "Write up the drift and the trace gaps"
Criteria as opinions	"Reviewed and approved by Jim"	the review happens on every MR anyway; write what the review is checking for
Hidden waiting	"Runner online" sized at 4 points, 3 of them waiting on the administrators	size the effort; make the wait its own task and label it blocked
Two kinds in one story	level tags and the state machine in one Stations story	split by TODO kind (Section 14.8, Splitting patterns for model work (p. 219))
A task over five points	"Port the requirements" at 8	it is a story; split it into tasks
Learning as overtime	no learning stories in the first plan	budget it (Section 14.10, The learning budget (p. 220))
A story nobody owns	"Team: close the TODOs"	one owner per story; tasks can go to others

14.15 When a story carries over

A story that is not done at the end of the sprint is **carried over**: its remaining points move to the next sprint in the journal (`carry TW-67`: the Stations story in sprint 4, Chapter 29, Sprint 4: the model site, a cycle broken, and the demo that landed (p. 459)), and the story issue moves to the next milestone. Carryover is not failure; it is information. The question at the retrospective is always *why*:

- **It was too big.** Split the rest before the next kickoff; do not carry the same oversized story twice.
- **It waited on someone outside the team.** Was the waiting its own task? If not, next time it will be.
- **Something more urgent came in.** Was it a real interrupt? The journal records unplanned work separately, so the pattern is visible over sprints.

In the first six sprints the plan carried over one story on purpose: "BuoyModes state machine by hand," started in sprint 6 and finished in sprint 7. Bob knew at kickoff that it would not finish, because the first hand-written state machine was a learning task as much as a modeling one. Writing that down at kickoff turned a surprise into a plan.

14.16 A backlog to start from

For a new port, the kit has a generic starting backlog, `tools/gitlab/templates/backlog.csv`, derived from Tidewater's plan with the Tidewater-specific words taken out: one row per task, with its tome, epic and story, its size, the role that usually does it, a sprint hint and an acceptance criterion. The first rows:

tools/gitlab/templates/backlog.csv

```
tome,epic,story,task,points,owner_role,sprint_hint,acceptance
T1 Stand up MBSE DevSecOps,E1 GitLab project and gitflow,Create the group and the model
project,Apply the project file with gitlab.pl setup; request a runner,1,CM engineer,1,The project
exists with the settings in project.conf
T1 Stand up MBSE DevSecOps,E1 GitLab project and gitflow,Create the group and the model
project,Prove a direct push to the integration branch is refused,1,CM engineer,1,The push is
rejected by the server
T1 Stand up MBSE DevSecOps,E1 GitLab project and gitflow,Create the group and the model
project,"Commit CODEOWNERS, the templates and .gitignore",1,CM engineer,1,The files are on the
integration branch through a merge request
T1 Stand up MBSE DevSecOps,E3 Team practices and onboarding,Laptops ready,Install Git for Windows
and Vim; run kickoff.pl doctor (one per person),1,each engineer,1,kickoff.pl doctor shows no FAIL
T1 Stand up MBSE DevSecOps,E3 Team practices and onboarding,Learn the tools,"Practice lessons,
the survival kit, git and Vim chapters (one per person)",1.5,each engineer,1,Lessons done;
questions written in the wiki
T1 Stand up MBSE DevSecOps,E3 Team practices and onboarding,Working agreements,Write the
Definition of Ready and Done in the wiki,0.5,MBSE lead,1,Both pages on the wiki
T1 Stand up MBSE DevSecOps,E1 GitLab project and gitflow,Record the decisions,Write the week-zero
decisions as ADR files under docs/adr,1.5,MBSE lead,2,docs/adr holds one file per decision
```

Load it into a spreadsheet, delete what does not apply, rename the epics for your system, and file the first two sprints as GitLab issues. It is a starting list, not a plan: the plan comes out of your own refinement sessions.

14.17 Applying this to your project

- **Copy:** the Story and Task templates, the habit of acceptance criteria as pipeline checks, the three work labels, and a learning budget that starts near a fifth of capacity and falls.
- **Adapt:** the task kinds to your own model's TODO kinds and drop rhythm; the size limits to your sprint length.
- **Decide locally:** who writes stories (Tidewater: the owner, at refinement), whether tasks live only in GitLab (Tidewater: yes), and how much learning time your management will accept on paper – then ask for it in the request for enablers (Chapter 37, Writing it down: DoD-style memos from Vim (p. 545)).

Exercise 14.2: Split a story

"Port the Requirements package" is too big for one sprint. Using the patterns in Section 14.8, Splitting patterns for model work (p. 219), split it into stories that each fit one sprint, and write one acceptance criterion for each that a pipeline job could check.

Exercise 14.3: Budget your own learning

Your team of four starts next month with no git experience. Draft the learning stories for the first two sprints: titles, done criteria, and the points per person.

You can now write a story issue with a title, a why and checkable criteria; write task issues linked to it with an estimate; recognize the kinds of task in model work; split stories along the model's seams; and budget learning as work.

CHAPTER

15

Setting up GitLab

From what you know →

You know

the CM plan, the CCB charter, and the list of who may approve what.

→ Here it is called

the GitLab project: members, protected branches, CODEOWNERS, labels and milestones.

→ What is new

the rules are settings GitLab enforces, written once in a project file.

Survival kit

What you have

a laptop with Git, Perl and Vim (Chapter 9, The survival kit (p. 126)), enough git to clone, branch and commit (Chapter 12, Git from zero, for systems engineers (p. 171)), and a directive that says "GitLab".

What you need

a GitLab project that works the way your configuration management plan says it should, before the first model file goes into it. By the end of this chapter you will have the project, its members, its protected branches and tags, its labels, its sprints, its templates and its code owners, all written down in one text file that you can review, re-apply and audit.

15.1 Tom's first request

On Thursday 8 October, once the laptops were ready, Tom opened a ticket with the enterprise GitLab administrators (task #14 in the sprint 1 plan). It was Tom's first real task in the new way of working, and it was not technical at all: it was a form. Tom had written hundreds of forms like it for the program's configuration management database. The request asked for three things:

- a **group** named `tidewater`, with Jim as its owner;

- the right for Jim and Tom to create **projects** inside the group;
- a **runner** registered to the group, on a Windows machine the team could log into, with Git for Windows installed (what a runner is, and why the team needed its own, is the subject of Chapter 19, CI/CD from zero (p. 288)).

The administrators answered on Thursday: the group existed, the runner would take another week. Tom's stand-up status that Tuesday was the first one the team's lint flagged as blocked (Chapter 24, The rhythm: stand-ups, poker, refinement, kickoff, demo, retro (p. 377) shows it). That delay is typical, and it is why week zero in Chapter 3, The directive (p. 22) put access requests first. Everything else in this chapter can be done by the team itself, in an afternoon, once the group exists.

Warning: The administrators are on the critical path

In every organization the authors have seen, the slowest step of adopting GitLab is not learning it. It is getting the group, the runner and the network rules approved. Ask on day one, ask for everything at once, and put the request in your stand-up as a blocker until it is done, so management sees the wait.

15.2 What GitLab is, in systems engineering terms

GitLab is a web application that keeps git repositories and the work around them. If you have read Chapter 12, Git from zero, for systems engineers (p. 171), you know that a git repository is a folder with its complete history: every version of every file, who changed it, when, and why. Git by itself lives on your laptop. GitLab is the shared place where the team's copy of the repository lives, plus everything a configuration management office would want around it: who may change what, who must approve, what was checked, what was released.

The vocabulary is new; the ideas are not. The table below maps the GitLab objects this chapter creates to the things a systems engineer already knows. The glossary in Appendix K, Glossary (p. 796) has the full list.

Table 15.1: GitLab objects and what they replace

GitLab object	What it is	The systems engineering thing it replaces
Group	a folder of projects with shared members and settings	the program's CM library, one per program
Project	one repository plus its issues, merge requests, pipelines and wiki	one controlled item: the system model, with its own change history
Member and role	a person with an access level: guest, reporter, developer, maintainer, owner	the access list on the CM library; who may draft, who may approve
Branch	a named line of work inside the repository	a working copy, a draft revision
Protected branch	a branch nobody may push to directly	the controlled baseline library: changes only through the board
Tag	a permanent name for one exact version	a baseline, a revision letter

GitLab object	What it is	The systems engineering thing it replaces
Protected tag	a tag nobody may move or delete	a released baseline under configuration control
Merge request (MR)	a proposed change from one branch into another, with discussion and checks	an engineering change proposal, small enough to review in an hour
Code owners	who must approve changes to which files	the signature authority matrix
Pipeline	automated checks run on every proposed change	the configuration audit and the peer review checklist, done by machine
Issue	a unit of tracked work	an action item, a work package, a discrepancy report
Label	a tag on an issue, such as `type::story`	a column in the action item spreadsheet
Milestone	a dated container of issues	a schedule milestone; for this team, a sprint
Board	issues shown as cards in columns by label	the status wall, the whiteboard of sticky notes
Wiki	pages of text kept with the project	the SEMP appendices, the team's working agreements

Two of those rows deserve a moment, because they are where most of the value is and where most of the confusion starts.

A **protected branch** is GitLab's answer to the question Bob asks first about anything: "Who approves a change?" On a protected branch, nobody pushes directly; not Jim, not Tom, not the administrators. Every change arrives as a merge request, which runs the pipeline and collects the approvals the rules require. This is the configuration control board, scaled down to one change and made impossible to bypass by accident.

A **merge request** is the engineering change proposal you know, with three differences. It is small (an hour of review, not a binder). It shows the exact lines that change, side by side with the old ones, so nobody has to reconstruct the change from prose. And it carries its own audit: the pipeline's results are attached to it before anyone is asked to approve.

15.3 The decisions behind the settings

Before Tom clicked anything, the team spent an hour on Tuesday 6 October deciding how the project should behave. This section records those decisions and the reasoning, because the reasoning is what transfers to your project; the settings themselves are one file.

15.3.1 One project for the model, not one per segment

The team considered a project per package (stations, data center, operations) and rejected it. The packages import each other; a change to an interface touches two of them at once; splitting them would make every interface change two merge requests that must land together. One project per *controlled item* is the rule that held: the Tidewater model is one controlled item, so it is one project. The journal (the team's sprint ledger, Chapter 23, Stand-ups, the journal and the board (p. 367)) is a different item with a different audience, so it lives in its own repository.

15.3.2 Gitflow, with develop as the default branch

Chapter 12, Git from zero, for systems engineers (p. 171) introduced branches. A **branching strategy** is the team's rule for which branches exist and how work moves between them. The three common ones are:

- **GitHub flow**: one long-lived branch (`main`); every change is a short feature branch merged into it. Simplest.
- **GitLab flow**: `main` plus environment or release branches downstream of it; fixes go into `main` first and are promoted ("upstream first").
- **Gitflow**: two long-lived branches. `develop` is where work is integrated; `main` holds only released versions. Release branches stabilize a version before it goes to `main`; hotfix branches repair a released version and go back into both.

The team chose gitflow, for reasons that are about systems engineering, not software:

- **A baseline is a release.** The program needs a small number of formal baselines, each reviewed and accepted. In gitflow, `main` contains only those; anyone who opens `main` sees the last accepted baseline and nothing else.
- **Integration happens continuously on `develop`.** The day-to-day state of the model, checked on every change, is `develop`. That is what the team works against and what the model site shows.
- **The prime's drops need a branch of their own.** Gitflow already thinks in terms of several long-lived branches, so adding `vendor/prime` (which holds exactly what the prime delivered, converted) is natural. Chapter 13, The operating model (p. 196) and Chapter 43, Maintenance through the life cycle (p. 602) explain the vendor branch in full.

Because work lands on `develop`, the team made `develop` the project's **default branch**. That matters more than it sounds: GitLab closes an issue automatically when a merge request that says "Closes #12" is merged *into the default branch*. With `main` as default, no issue would ever close by itself.

Note: When to choose something simpler

If your program has no formal baselines and no contractor deliveries to absorb, GitLab flow (one `main`, short feature branches, tags for versions) is simpler and loses little. The rule of thumb: use gitflow when "released" and "current" must be different things that people can open separately.

15.3.3 Merge commits, never squash

GitLab can combine a merge request's commits into one ("squash") or keep them and add a merge commit. The tutorial material the team read treated squash as a matter of taste. For a model it is not: the team's metrics (lead time, review turnaround, who changed what) are computed from the merge history, and squash hides it. The project merges with a merge commit every time.

15.3.4 Pipelines must succeed; discussions must be resolved

These two switches turn the pipeline and the review from advice into rules. If the pipeline fails, the merge button is disabled. If a reviewer opens a thread (“this connection points the wrong way”), the merge waits until the thread is resolved. Bob, who spent years chasing comment matrices with no disposition column, liked the second one immediately.

15.3.5 Who may do what

Table 15.2: Members and roles in tidewater/model

Person	Role	Why
Jim	maintainer	owns the model’s content, approves requirement changes, creates baselines
Bob	developer	writes most of the model changes; can merge into develop once approved
Tom	maintainer	administers the pipeline, the runner, the vendor branch and the tags

Bob asked why Bob, who builds most of the model, was not a maintainer. The answer is the separation of duties the team already practiced with documents: the author does not approve the author’s own work and does not cut the baseline. A developer can do everything the daily work needs; the two maintainers hold the keys to `main` and the tags. Least privilege is not an insult; it is what the ISSM will ask about first (Chapter 17, The DevSecOps tools that earn their keep (p. 255), Chapter 42, Administration (p. 586)).

15.3.6 Which branches are protected, and how

Table 15.3: Protected branches

Branch	Push	Merge	Force push	Code owner approval
`main`	nobody	maintainers	no	yes
`develop`	nobody	developers	no	yes
`release/*`	maintainers	maintainers	no	no
`hotfix/*`	maintainers	maintainers	no	no
`vendor/prime`	maintainers	nobody	no	no

`vendor/prime` is the unusual row. Nothing is ever merged *into* it, and only a maintainer (Tom) pushes to it, because its only content is the converter’s output for each prime drop, committed by the drop intake procedure. Any human edit there would corrupt the comparison between drops.

Feature branches (`feature/12-close-stations-todos`) are not protected: they are the engineers’ drafts.

15.3.7 Protected tags

Tags named `baseline-*` may be created only by maintainers, and once created they cannot be moved or deleted. A baseline is a promise to the program that a particular version existed and was accepted. A tag that could move would be a promise that could be quietly broken.

15.4 Settings as a file: project.conf

Every setting above can be clicked in GitLab's web pages under Settings. The team decided not to click them. Tom's argument, from a decade of CM audits: "If the settings live only in a web page, nobody can review a change to them, and nobody can prove what they were last March." So the settings live in a text file, `project.conf`, that is committed, reviewed in merge requests like any other change, and applied by a command.

Here is the beginning of the file the team used. Read it as you would read a SEMP appendix: every line is a decision from the previous section.

tools/gitlab/templates/project.conf

```
# project.conf -- the Tidewater model project, as `gitlab.pl setup` applies it
(tools/gitlab/templates/project.conf).
#
#   perl bin/gitlab.pl --url https://gitlab.example.com --token-env GITLAB_TOKEN setup
tools/gitlab/templates/project.conf
#   perl bin/gitlab.pl --local data/tidewater/gitlab setup tools/gitlab/templates/project.conf
  (the offline emulator)
#
# Every setting here is something a maintainer would otherwise click through in Settings. Keeping
it in a file means the
# project can be rebuilt, reviewed in a merge request, and compared with what the server actually
has. Re-running setup
# is safe: it creates what is missing and updates what differs.
# Tidewater is fictional; the people are Jim (MBSE lead, responsible for the model), Bob (MBSE
engineer) and Tom
# (configuration management engineer, who also runs the pipeline). None of them had done
DevSecOps before this year.

[project]
path = tidewater/model
default_branch = develop                                    ; gitflow: develop is where work lands,
so issues close there
merge_method = merge                                        ; a merge commit for every MR: the
metrics read merge history
squash_option = never                                       ; keep each commit; squash would hide
who changed what
only_allow_merge_if_pipeline_succeeds = true
only_allow_merge_if_all_discussions_are_resolved = true
remove_source_branch_after_merge = true
visibility = private

[members]
jim = maintainer                                            ; MBSE lead: owns tomes, epics,
baselines and releases
bob = developer                                             ; MBSE engineer: ports and maintains
the model
tom = maintainer                                            ; CM: pipeline, runners, vendor branch,
tags

[branches]
main = push:none merge:maintainer force:no codeowners:yes         ; released baselines only,
through release/hotfix MRs
develop = push:none merge:developer force:no codeowners:yes       ; integration; every change
arrives by MR
```

```
release/* = push:maintainer merge:maintainer force:no
hotfix/* = push:maintainer merge:maintainer force:no
vendor/prime = push:maintainer merge:none force:no                ; raw converter output of each
prime drop, never hand-edited

[tags]
baseline-* = create:maintainer                              ; a baseline is permanent: nobody can
move or delete it
```

A few notes on the format, which is deliberately plain:

- Sections are in square brackets. Lines are `key = value`. A semicolon after a space starts a comment, which is how each line carries its reason.
- `[members]` uses GitLab's role names. `[branches]` uses `push:`, `merge:`, `force:` and `codeowners:` fields in the order of the protected-branches table.
- The file never holds a password or a token. Those belong in the variables GitLab protects (Chapter 17, The DevSecOps tools that earn their keep (p. 255)).

The rest of the file defines labels, sprints, the approval rule, two CI/CD variables and the two pipeline schedules:

tools/gitlab/templates/project.conf

```
[labels]
; type -- what kind of work item it is (scoped: an issue has exactly one type)
type::epic = #5843AD | A collection of stories finished within one semester (13 sprints)
type::story = #428BCA | A slice of value of several tasks that fits in one sprint
type::task = #69D100 | One piece of work under a story, less than a week (5 points or fewer)
type::defect = #D9534F | Something in the government v2 model or pipeline is wrong
type::discrepancy = #AD4363 | A problem in the prime's v1 model or DOORS, reported to the prime
(Discrepancy Report)
type::drop = #00B140 | Intake of one prime model drop (XMI + ReqIF + CSV under CDRL A003)
type::spike = #8E44AD | Time-boxed investigation that ends in a decision, not a model change
; tome -- the year-scale grouping (< 2 semesters)
tome::T1-stand-up = #36454F | T1 Stand up MBSE DevSecOps (sprints 1-8)
tome::T2-port = #36454F | T2 Port Tidewater v1 to SysML v2 (sprints 1-20)
tome::T3-maintain = #36454F | T3 Maintain against prime drops (sprints 17-39)

epic::E1-gitlab = #6699CC | E1 GitLab project and gitflow
epic::E2-pipeline = #6699CC | E2 Pipeline and quality gates
epic::E3-practices = #6699CC | E3 Team practices and onboarding
epic::E4-toolchain-security = #6699CC | E4 Toolchain security and authorization to use
epic::E5-drop-1 = #6699CC | E5 Drop 1 intake and first import
epic::E6-structure = #6699CC | E6 Structure: close the structure TODOs
epic::E7-requirements = #6699CC | E7 Requirements and trace
epic::E8-behavior = #6699CC | E8 Behavior and analysis
epic::E9-drops-2-3 = #6699CC | E9 Drops 2 and 3 during the port
epic::E10-baseline-1 = #6699CC | E10 Baseline 1 and acceptance
epic::E11-intake-automation = #6699CC | E11 Drop intake automation
epic::E12-drops-4-5 = #6699CC | E12 Drops 4 and 5
epic::E13-drops-6-7 = #6699CC | E13 Drops 6 and 7
epic::E14-sustainment = #6699CC | E14 Sustainment operations
; workflow -- where the item is on the board (scoped: moving it replaces the old one)
workflow::ready = #A8D695 | Meets the Definition of Ready; can be pulled into a sprint
workflow::in-progress = #F0AD4E | Someone is working it
```

```
workflow::review = #F0AD4E | An MR is open and waiting for review or approval
workflow::blocked = #D9534F | Cannot move; the blocker is in the latest comment
workflow::done = #5CB85C | Meets the Definition of Done
; priority, severity, owner, phase
priority::1 = #CC0033 | Do first
priority::2 = #FC9403 | Normal
priority::3 = #C0C0C0 | When there is room
severity::1 = #CC0033 | Wrong model content that would mislead an analysis
severity::2 = #FC9403 | Wrong or missing content with a workaround
severity::3 = #C0C0C0 | Cosmetic
owner::gov = #1F3A5F | The government team fixes it
owner::prime = #B8860B | The prime fixes it in a later drop
phase::port = #7F8C8D | Found or done during the port
phase::maintenance = #7F8C8D | Found or done after baseline 1
phase::integration = #7F8C8D | Found when parts were brought together
; area -- which model package it touches (one per top-level package; CODEOWNERS uses the same
split)
area::network = #E0E0E0 | model/physical/network
area::stations = #E0E0E0 | model/physical/stations
area::datacenter = #E0E0E0 | model/physical/datacenter
area::operations = #E0E0E0 | model/physical/operations
area::interfaces = #E0E0E0 | model/library/Interfaces.sysml
area::interface = #E0E0E0 | A defect at an interface between packages (status metric H counts
these)
area::types = #E0E0E0 | model/library/Types.sysml
area::analysis = #E0E0E0 | model/analysis
area::behavior = #E0E0E0 | model/logical
area::allocations = #E0E0E0 | model/physical/allocation
area::requirements = #E0E0E0 | model/requirements (the three DOORS modules and the Cameo copy)
area::verification = #E0E0E0 | model/verification and verification/*.csv (results)
area::legacy = #E0E0E0 | model/legacy (clutter from the document port; removed upstream in drop
6)
area::pipeline = #E0E0E0 | .gitlab-ci.yml, ci/job.pl, runners, the kit
area::gov = #E0E0E0 | government additions (model/library/conventions,
model/logical/GovBehavior.sysml)
; doc -- the review stage of a document issue: the Document Review Stages scale, Grey
(collection) to Green (releasable), with percent and grade bands; scoped
doc::grey = #A0A0A0 | Grey, Collection (1-24%, G- to G+): a loose collection of content, started
doc::blue = #4A90D9 | Blue, Compliant (25-45%, F- to F+): the outline is complete
doc::pink = #E89AC7 | Pink, Complete (46-63%, D- to D+): content stable and consistent throughout
doc::red = #D9534F | Red, Compelling (64-78%, C- to C+): graphics, tables, data, spelling,
acronyms and grammar finalized
doc::gold = #D4AF37 | Gold, Comprehensive (79-90%, B- to B+): sound, coherent and actionable
doc::white = #F5F5F5 | White, Copy Ready (91-99%, A- to A+): ready for hard-copy review
doc::green = #5CB85C | Green, Done (100%): releasable
; prime changes -- each significant item of a prime drop's delta, dispositioned by Jim and the
deputy PM (the CCB)
prime-change = #B8860B | One significant change the prime made in a drop (cite the prime's CR id,
NWM-CR-nnnn)
prime-tbd = #B8860B | Work that rests on an assumption about the prime's next drop; closed when
the drop lands
disposition::accept = #5CB85C | Accepted as delivered
disposition::accept-with-impact = #F0AD4E | Accepted; it adds government work (tasks linked),
cost and schedule noted
disposition::reject-clarify = #D9534F | Not accepted yet: a DR asks the prime for the rationale
disposition::no-impact = #C0C0C0 | Cosmetic: nothing for the government model to do
work::engineering = #1F77B4 | Systems engineering with the model: the reason the platform exists
work::platform = #7F7F7F | The platform: GitLab, the pipeline, runners, tokens, templates
```

```
work::learning = #2CA02C | Time to learn the tools and the way of working (planned, not stolen
from evenings)
ai::assisted = #8E44AD | an MR whose first draft came from the local SysML assistant (measured:
first-pipeline pass rate)

[milestones]
sprints = 2026-10-05 39 2                                ; Sprint 1 .. Sprint 39, two weeks
each, 18 months

[approvals]
Model review = 1 jim bob                                 ; one of Jim or Bob approves every MR
(Premium; enforced locally)

[variables]
KIT = /opt/agile                                         ; where the runner has the kit checked
out (the jobs call $KIT/...)
DOCKER_RUNNER = false                                    ; "true" only on a Docker-executor
runner: pulls in GitLab's Secret-Detection template

[schedules]
; Description = CRON | REF | TIME ZONE | KEY=VALUE ...   (CI/CD > Schedules; the pipeline sees
$CI_PIPELINE_SOURCE == "schedule")
; Cron is minute hour day-of-month month day-of-week, read in the time zone given (US daylight
saving included).
; The schedule's owner is whoever runs setup: it runs as them, so use a maintainer who will stay
on the project.
Nightly full check = 0 2 * * 1-5 | develop | America/New_York | FULL=1   ; weeknights 02:00
Eastern: every job, uncached
Weekly status = 0 6 * * 5 | develop | America/New_York                  ; Friday 06:00 Eastern:
export + status metrics for the weekly report
```

15.4.1 Labels: scoped, and few

GitLab labels can be **scoped**: a label written `type::story` belongs to the scope `type`, and an issue can carry only one label per scope. Adding `type::task` to an issue removes `type::story`. Scoped labels turn a pile of tags into columns of a spreadsheet, each with exactly one value:

- `type::` what kind of item: epic, story, task, defect, discrepancy, drop, spike;
- `tome::` and `epic::` which part of the plan it belongs to (Chapter 13, The operating model (p. 196) explains the hierarchy);
- `workflow::` where it is on the board: ready, in-progress, review, blocked, done;
- `priority::`, `severity::`, `owner::`, `phase::`, `area::` for sorting and for the metrics;
- `work::` engineering, platform or learning: the split the bottom line of this book is built on (Bottom line up front (p. iii));
- `doc::` the review stage of a document the model generates, grey to green (Chapter 40, Documents still matter: review stages for generated documents (p. 570));
- `disposition::` and `prime-change`, `prime-tbd` for the prime's drops (Chapter 45, The prime's model changes: receive, assess, disposition (p. 619)).

The team kept the list short on purpose. Every label must answer a question someone actually asks. `area::` mirrors the model's packages and the code owner sections, so "show me everything open on the stations" is one click. `owner::prime` marks a discrepancy the prime must fix, so Jim's pre-drop review of what the prime owes is one filtered list.

15.4.2 Milestones are sprints

The line `sprints = 2026-10-05 39 2` creates thirty-nine milestones named "Sprint 1" to "Sprint 39", each two weeks long, starting on Monday 5 October 2026, each due on the Friday of its second week. Every story and task is put in the milestone of the sprint that commits to it. A milestone in GitLab is then exactly a sprint backlog: open it and you see what the team promised, what is done and what is not.

GitLab Premium has a separate feature, **iterations**, designed for sprints, with automatic cadences and burndown charts. The team's license was the Free tier for the first year, so milestones did the job. Chapter 17, The DevSecOps tools that earn their keep (p. 255) lists which features need which tier.

15.4.3 The approval rule and the variable

`Model review = 1 jim bob` says that every merge request needs one approval from Jim or Bob. On GitLab Premium this becomes a real approval rule that blocks the merge; on Free, the code owner approval on the protected branches does most of the same work. The `KIT` variable tells every pipeline job where the agile kit is checked out on the runner; Chapter 20, The pipeline, job by job (p. 302) uses it in every job. `DOCKER_RUNNER` says whether the runner can run Docker containers (Chapter 21, Runners: where the pipeline runs (p. 347) explains why the team's cannot). The `[schedules]` section creates the nightly full check and the weekly status export as cron schedules, each run as the person who ran `setup` (Chapter 17, The DevSecOps tools that earn their keep (p. 255)).

15.5 Applying the file

The command that applies `project.conf` is part of the kit. Run it from the model repository, where the scaffold (Chapter 50, Your project in twelve weeks: a playbook (p. 672)) put the file in `.gitlab/`:

```
perl "$KIT/bin/gitlab.pl" --url https://gitlab.example.com --token-env GITLAB_TOKEN setup
.gitlab/project.conf
```

It talks to GitLab through its web API (the same thing the web pages use underneath), authenticated by a personal access token that Tom created in Tom's own GitLab profile and kept in an environment variable, never in a file. The command creates the project if it does not exist, then makes every setting, member, label, milestone, protection and approval match the file.

You do not need a GitLab server to try it. The kit includes an emulator: a small, offline stand-in for GitLab that keeps its state in a folder, enforces protected branches with real git hooks, and runs pipelines on your machine. Everything in this book that talks to "GitLab" can be pointed at the emulator with `--local DIR` instead of `--url`. Type along in a practice folder (`$KIT` is your copy of the kit, set once in Chapter 9, The survival kit (p. 126)):

```
mkdir -p ~/tw && cd ~/tw
sed "s#^KIT = /opt/agile#KIT = $KIT#" $KIT/tools/gitlab/templates/project.conf > project.conf
```

```
for u in jim bob tom; do perl $KIT/bin/gitlab.pl --local gl user $u --email $u@example.invalid;
done
perl $KIT/bin/gitlab.pl --local gl --as tom setup project.conf
```

The first lines of the output:

```
GitLab: schedule Nightly full check skipped: develop does not exist yet; re-run setup after flow
init
GitLab: schedule Weekly status skipped: develop does not exist yet; re-run setup after flow init
created project tidewater/model
settings: default_branch=develop, merge_method=merge,
only_allow_merge_if_all_discussions_are_resolved=true,
only_allow_merge_if_pipeline_succeeds=true, remove_source_branch_after_merge=true,
squash_option=never
member jim maintainer
member bob developer
member tom maintainer
variable KIT
variable DOCKER_RUNNER
label type::epic
label type::story
label type::task
```

and the last ones:

```
milestone Sprint 39
protected branch main (push:none merge:maintainer force:no codeowners:yes)
protected branch develop (push:none merge:developer force:no codeowners:yes)
protected branch release/* (push:maintainer merge:maintainer force:no)
protected branch hotfix/* (push:maintainer merge:maintainer force:no)
protected branch vendor/prime (push:maintainer merge:none force:no)
protected tag baseline-*
approval rule Model review (1)
schedule Nightly full check skipped (develop does not exist yet)
schedule Weekly status skipped (develop does not exist yet)
```

The whole run took about a quarter of a second. Clicking the same settings through the web pages took Tom most of an afternoon the first time, on a scratch project, and Tom missed two of them.

The two schedules were skipped, not lost: a schedule runs the pipeline on a branch, and `develop` does not exist until the next section creates it. Setup is safe to re-run: it creates what is missing, updates what differs and leaves the rest alone, so the second run, after `develop` exists, adds the schedules.

15.6 The first commit and the two long-lived branches

A brand-new project has no branches at all; protection rules wait for branches to appear. Tom cloned the empty project, added the pipeline file and its job script, the templates, the code owners

file and the ignore list, made the first commit, and ran the kit's gitflow helper, which pushes `main` and creates `develop` from it. Then Tom ran setup a second time, for the schedules, and asked it what still differed from the file:

```
git clone gl/repos/tidewater/model.git tom && cd tom
cp $KIT/tools/gitlab/templates/gitlab-ci.yml .gitlab-ci.yml
cp -r $KIT/tools/gitlab/templates/.gitlab $KIT/tools/gitlab/templates/ci .
cp $KIT/tools/gitlab/templates/CODEOWNERS $KIT/tools/gitlab/templates/README.md \
   $KIT/tools/gitlab/templates/CHANGELOG.md $KIT/tools/gitlab/templates/.gitignore .
git add -A
git commit -m "Project skeleton: pipeline, templates, CODEOWNERS"
perl $KIT/bin/gitlab.pl --local ../gl --project tidewater/model --as tom flow init
perl $KIT/bin/gitlab.pl --local ../gl --as tom setup ../project.conf | grep schedule
perl $KIT/bin/gitlab.pl --local ../gl --project tidewater/model setup --diff ../project.conf
```

```
develop is ready (from main)
schedule Nightly full check (0 2 * * 1-5, America/New_York, develop)
schedule Weekly status (0 6 * * 5, America/New_York, develop)
no differences: the project matches ../project.conf
```

`setup --diff` reads and never changes anything. It compares the file with the live project and prints only what differs, one line per difference, and exits with status 1 when anything does. Tom put it on a monthly reminder: a setting someone changed by hand in the web pages shows up there, and the fix is a merge request to `project.conf` or a re-run of setup, never a second hand edit (Chapter 42, Administration (p. 586)).

The pipeline ran on that first push and failed: the `lint` job could not find a `model/` folder, because there was no model yet. Tom's first red pipeline arrived before the first line of SysML. It was a useful lesson about what a pipeline is (a set of checks that assume the thing they check exists), and Chapter 19, CI/CD from zero (p. 288) returns to it. In the type-along, `perl $KIT/bin/gitlab.pl --local ../gl --project tidewater/model pipeline list` shows it.

Tip: The shorter way

`tools/gitlab/kickoff.pl scaffold` does the copying above in one command, and `kickoff.pl start` runs the whole week-zero sequence, setup included (Chapter 9, The survival kit (p. 126)). This section does it by hand once, so you know what those commands do.

15.7 Proving the protections work

Tom did not trust a setting until it had refused something. The team wrote down three tests, ran them, and kept the output as evidence for the ISSM.

A direct push to develop is refused. Tom committed a one-line change and tried to push it straight to `develop`. The `|| echo` keeps your shell open and shows the refusal's status:

```
echo "One more line." >> README.md
git commit -qam "A direct change"
```

```
git push origin HEAD:develop || echo "exit status $?"
git reset -q --hard HEAD~1          # take the local commit back; nothing reached the server
```

```
remote: GitLab: You are not allowed to push code to protected branches on this project.
 ! [remote rejected] HEAD -> develop (pre-receive hook declined)
error: failed to push some refs to '.../tidewater/model.git'
exit status 1
```

A merge with a failing pipeline is refused, even when approved. Bob's first merge request (the drop 1 import, Chapter 27, Sprint 2: Drop 1 on develop, every change gated (p. 424) tells that story) had a failing check. Jim approved it anyway, to see what would happen, and tried to merge. In the type-along, a small README change on a feature branch plays that merge request; its pipeline fails for the same reason the first one did, because there is no model yet:

```
G="perl $KIT/bin/gitlab.pl --local ../gl --project tidewater/model"
$G --as bob flow start feature 1-readme
echo "The Tidewater system model, in SysML v2 text." >> README.md
git commit -qam "README: what the model is"
$G --as bob flow finish --title "README: what the model is"
$G --as jim mr approve 1
$G --as jim mr merge 1 || echo "exit status $?"
```

```
on feature/1-readme
!1 feature/1-readme -> develop: README: what the model is
pipeline #4 failed
!1 approved by jim
gitlab.pl: pipeline must succeed (pipeline #4 failed) (405 Method Not Allowed)
exit status 1
```

The "405 Method Not Allowed" is what GitLab's real API answers in the same situation; the emulator copies it so that scripts written against the emulator behave the same way against the server.

A baseline tag cannot be moved. That test waited until sprint 6, when the first baseline existed (Chapter 31, Sprint 6: Behavior by hand, a table for the program, and baseline-0 (p. 495)).

> **Case study: What the ISSM saw**
>
> The ISSM had approved document repositories for years by reading their access lists. The three refusals above, with the date and the command that produced them, were the first time a configuration control claim arrived with its own evidence. "I can test that myself," the ISSM said, and did. Chapter 17, The DevSecOps tools that earn their keep (p. 255) has the rest of the security story.

15.8 Code owners: the signature authority matrix

The document era had a signature page: the requirements chapter needed the requirements lead's signature, the interface chapter needed both segment leads'. GitLab has the same idea in a file named `CODEOWNERS`, kept in the repository and therefore reviewed and versioned like everything else.

tools/gitlab/templates/CODEOWNERS

```
# CODEOWNERS -- who must approve a change to which part of the Tidewater model repo.
# GitLab reads this file from the MR's TARGET branch (CODEOWNERS, docs/CODEOWNERS or
.gitlab/CODEOWNERS, first found).
# With "code owner approval" on a protected branch (project.conf [branches] codeowners:yes), an
MR that touches a path
# below needs an approval from one of that path's owners. The last matching pattern in a section
wins.
#
# Sections group the rules; each section is checked on its own, so one MR can need an approval
from several sections.
# [Name][2] would require two approvals from that section; ^[Name] makes a section optional
(advisory only).

# Every section names at least two owners: GitLab never lets an author approve their own MR, so a
section whose only
# owner wrote the change could never be satisfied. The model folders are the layers of the layout
map
# (examples/tidewater-v1/layout.map): library -> requirements -> logical -> physical -> analysis
-> verification.
[Pipeline and repository] @tom @jim
/.gitlab-ci.yml
/ci/
/.ci/
/.gitlab/
/.gitignore
/.kit-version
/CODEOWNERS             @tom @jim
/sysml.conf             @tom @bob

# Decisions: Jim records them, Bob (or Tom) reviews.
[Decisions] @jim @bob
/docs/adr/

[Library and conventions] @bob @jim
/model/library/

# Interfaces are where segments meet, so an interface change needs TWO approvals: the systems
lead's (Jim decides every
# interface; Jim does not author interface changes) and one of the people who build on both sides
(Bob or Tom,
# whoever did not write the change). Two sections, each must be satisfied.
[Systems lead] @jim
/model/library/Interfaces.sysml

[Interfaces] @bob @tom
/model/library/Interfaces.sysml

# Requirements belong to the government: Jim approves every change to them, from either converter
(Bob when Jim wrote it).
```

```
[Requirements and verification] @jim @bob
/model/requirements/
/model/verification/
/verification/

[Logical and physical] @bob @jim
/model/logical/
/model/physical/
/model/legacy/

[Analysis] @jim @bob
/model/analysis/

# Raw prime drops arrive through vendor/prime; Tom imports them, Jim accepts them.
[Prime drops] @tom @jim
/imports/

# Generated docs: whoever changed the model regenerates them; advisory only.
^[Generated docs] @bob @tom
/docs/
```

How to read it:

- Each line pairs a path (a file or a folder, starting from the top of the repository) with the people who must approve changes to it, written `@jim`, `@bob`, `@tom`.
- Lines are grouped in **sections**, the names in square brackets. A merge request that touches files in two sections needs an approval from each section.
- A section that starts with `^` is advisory: its owners are asked, not required.
- Within a section, the last matching line wins.

The team's split follows responsibility, not seniority. Jim is the first owner of requirements and verification, because those are the government's commitments to the program; Bob or Jim approve structure and behavior; Tom is the first owner of the pipeline file, the job script and the templates, because a careless change there disables the checks for everyone; the prime's drops (`/imports/`) need Tom (who imports them) or Jim (who accepts them).

Every section names at least two owners, and that is not generosity. GitLab never lets the author of a merge request approve it. A section whose only owner wrote the change could never be satisfied, and the merge request would wait forever. Jim's requirement changes are approved by Bob; Tom's pipeline changes by Jim.

With "code owner approval" switched on for `develop` and `main` (the `codeowners:yes` field in `project.conf`), GitLab enforces this file on every merge request. On the Free tier the file still works as a list of suggested reviewers; enforcement is a Premium feature. The emulator enforces it, so the team practiced the real rule from day one.

Tip: Change the file, not the setting

When someone joins, leaves or goes on long leave, do not loosen the protection: open a merge request that changes the owners, and another to change them back. The history of who could approve what, and when, is in the file's history.

15.9 Templates: the forms

Templates are Markdown files in the repository, under `.gitlab/issue_templates/` and `.gitlab/merge_request_templates/`. When someone creates an issue or a merge request in the web page, a drop-down offers the templates; picking one fills the description. The team's templates are the forms of the new way of working:

Table 15.4: The issue and merge request templates

Template	Used for	What it asks for
Story	a slice of value that fits in one sprint	why, acceptance criteria, the task table, the Definition of Ready
Task	one piece of work under a week	the parent story, the steps, the one observable result
Defect	something wrong in the government model or pipeline	where, what, expected, evidence
Discrepancy Report	something wrong in the prime's v1 model	the v1 element by `xmi:id`, the evidence, the drop that should fix it
Prime Drop	intake of one prime delivery	the manifest, checksums, the intake checklist
Epic	a semester-scale group of stories	the outcome, the stories, the exit criteria
Spike	a time-boxed question	the question, the time box, the decision it must produce
Default (MR)	most merge requests	what and why, the model changes, the checklist
Prime Drop (MR)	the merge of a drop into develop	the v1 delta, conflicts, rework tasks
Release (MR)	a release branch into main	the baseline contents and the checks
Hotfix (MR)	a fix to a released baseline	what was wrong, the fix, the back-merge

Each template is GitLab-flavored Markdown with HTML comments as instructions (`<!-- like this -->`, invisible once saved) and **quick actions** at the bottom: lines that start with a slash, such as `/label ~"type::story"` or `/milestone %"Sprint 3"`, which GitLab executes when the description is saved and then removes from the text. The story template, for example, ends with:

```
/label ~"type::story" ~"workflow::ready" ~"priority::2"
/milestone %"Sprint "
/assign @
/weight
```

The incomplete lines are blanks to fill in: the sprint number after `Sprint `, a user name after `@`, a number after `/weight`. Fill them before you save. On GitLab, an unfinished quick action is silently ignored, so a story saved with `/milestone %"Sprint "` simply has no sprint, which is a quiet way to lose work from a sprint backlog. The kit's emulator behaves the same way, so you can see it happen:

```
G="perl $KIT/bin/gitlab.pl --local ../gl --project tidewater/model"
$G --as jim issue new "Try the story template" --template Story
$G issue show 1 | grep milestone
```

The labels from the template's first quick action are there; the milestone is `-`, nothing. Tom's rule after that: open every new story once after saving it and check the sprint.

Appendix I, Templates and GitLab Markdown, field by field (p. 747) reproduces every template, and Chapter 17, The DevSecOps tools that earn their keep (p. 255) has filled-in examples of each Markdown field.

15.10 Wiki, README and CHANGELOG

Three more places hold text, each with a different job:

- The **README** at the top of the repository is the front door: what the model is, how it is organized, how to run the checks. GitLab shows it on the project's home page.
- The **CHANGELOG** is the human-readable list of what changed in each baseline. The pipeline copies a baseline's section into its release notes (Chapter 20, The pipeline, job by job (p. 302)).
- The **wiki** holds the team's working agreements: the Definition of Ready, the Definition of Done, the gitflow rules, the drop intake runbook, the baseline procedure, onboarding, administration and the glossary. The team seeded it from `tools/gitlab/templates/wiki/` in sprint 1 (Chapter 26, Sprint 1: A GitLab project the team can work in (p. 404)).

A wiki in GitLab is itself a git repository, so its pages have history too. The team's rule: anything a newcomer must read is in the wiki; anything the pipeline needs is in the repository.

15.11 Signing in: SSH keys and tokens

Each person needs a way to prove who they are to the server, from the laptop's terminal. There are two, and the team used both.

An SSH key is a pair of files: a private key that never leaves the laptop and a public key you paste into your GitLab profile. Git then authenticates every clone, fetch and push with it. In Git Bash:

```
ssh-keygen -t ed25519 -C "jim laptop, tidewater"
cat ~/.ssh/id_ed25519.pub          # copy this line into GitLab: your avatar > Preferences > SSH
Keys
ssh -T git@gitlab.example.com      # should greet you by user name
```

A personal access token is a long password-like string that a script presents to GitLab's API. The kit's `gitlab.pl` uses one, read from an environment variable. Create it under your avatar > Preferences > Access tokens, give it the narrowest scope that works (`api` for setup, `read_api` for reports), give it an expiry date, and put it in your shell's startup file rather than in any file inside a repository:

```
echo 'export GITLAB_TOKEN=...' >> ~/.bashrc      # the real value; never commit it
```

Warning: A token is a password

The pipeline's `secrets` job looks for GitLab tokens (they start with `glpat-`) in every file of every merge request, because sooner or later someone pastes one into a file. If it happens: revoke the token in GitLab first, then remove it from the branch. Deleting the line is not enough; the token is in the history. Chapter 42, Administration (p. 586) covers rotation.

15.12 What the enterprise administrators must do, and what the team does

Table 15.5: Who sets up what

Task	Who	When
Create the group; make the MBSE lead its owner	enterprise admins	week zero
Register a runner for the group (Windows shell runner with Git for Windows)	enterprise admins with Tom	sprint 1
Allow the runner to reach the GitLab server and nothing else	network and ISSM	sprint 1 to 3
Create the project and apply `project.conf`	Tom	sprint 1
Commit the pipeline, templates, CODEOWNERS	Tom	sprint 1
Seed the wiki	Jim	sprint 1
Create epics and the first stories	Jim	sprint 1 planning
Keep `project.conf` and CODEOWNERS current	Tom, by merge request	always
Review members and tokens	Jim and Tom	every quarter (Chapter 42, Administration (p. 586))

15.13 Exercises

Exercise 15.1: Build the project

Using the emulator, create the three users and apply `project.conf` as shown in Section 15.5, Applying the file (p. 235). Then run `gitlab.pl --local gl --project tidewater/model issue list` and `gitlab.pl help setup`. How many labels and how many milestones did setup create? Where in the file is each one defined?

Exercise 15.2: Make it refuse you

Clone the project, commit something, and try to push it straight to `develop`, then to `main`. Then create a tag named `baseline-0` as Bob (`--as bob`) with `gitlab.pl tag`. Write down each refusal and explain which line of `project.conf` caused it.

Exercise 15.3: Redraw the signature page

Take the signature page of a document your team controls today. Write the CODEOWNERS file that would enforce the same signatures if the document were a folder of text files. Which signatures had no clear owner?

Exercise 15.4: Defend a branching strategy

Your program has no contractor drops and releases a new model version every month without a formal review. Which branching strategy would you choose, and which three lines of `project.conf` would you change?

15.14 Applying this to your project

- **Copy**: the idea of settings as a reviewed text file; protected integration and release branches; protected baseline tags; merge commits; "pipeline must succeed" and "discussions resolved"; code owners drawn from your signature authority; a short list of scoped labels; milestones as sprints.
- **Adapt**: the branching strategy (gitflow if released and current must be separate, GitLab flow otherwise); the roles (match your separation of duties); the code owner sections (match your organization, not this team's); the label values (keep `type::` and `workflow::`, change the rest).
- **Decide locally**: which GitLab tier you have and therefore which rules are enforced or merely advisory; who in your enterprise creates groups and runners, and how long they take; where your tokens live.

CHAPTER

16

The model repository: what goes where

From what you know →

You know

a document tree and a numbering scheme for specifications and ICDs.

→ Here it is called

the model repository and its layered folders.

→ What is new

the layout itself says which layer may depend on which, and every prime drop lands in the same place.

Survival kit

What you have

a shared drive with folders named "SRS," "ICDs (current)," "ICDs (OLD - do not use)" and "Bob's working copy," and a Cameo project file that holds everything in one blob.

What you need

one repository whose folders mean something – where the model lives and in what layers, where the prime's deliveries land and why nobody edits them, where decisions and conventions are written down, where the pipeline's own files live – and a layout that every prime drop lands in the same way, so merges keep working for eighteen months.

Note: Core path

This chapter is CORE: the layout is decided in sprint 2, at the first import, and is very expensive to change later. The section on federation (Section 16.10, When one repository is not enough (p. 252)) is LATER.

16.1 Why the layout is decided at drop 1

Most decisions in this book can be revisited. The folder layout of the model is the exception, and the reason is the vendor branch (Section 12.19, The vendor branch (p. 191)). Every prime drop is converted and committed to `vendor/prime`, then merged into `develop`. Git can merge the new drop into the government's edited copy only when each package lands at the *same path* every time. Move `Stations.sysml` from one folder to another after drop 1, and drop 2's Stations arrives at the old path as a brand-new file while the edited one sits elsewhere. Every government edit to Stations then has to be redone by hand.

So in sprint 2 Bob had one decision to make before the first import, and a day budgeted for it: "Decide the layered layout; the layout map." This chapter is that day, written down.

16.2 Layers: dependencies point one way

A model's packages depend on each other. The spider web of the Cameo years (Section 2.3, Measuring the spider web (p. 12)) was dependencies pointing every way: Network used Stations, Stations used DataCenter, DataCenter used Network. Nothing could be understood, or changed, on its own.

The cure is old and simple: **layers**. Put the packages in an order, and allow a package to depend only on packages in its own layer or the layers below it. Tidewater's order, from the bottom:

Table 16.1: Tidewater's layers, bottom to top

Layer	Folder	What lives there	May depend on
Library	`model/library/`	types and units, interface and port definitions, the conventions (level tags, metadata)	nothing in the model
Requirements	`model/requirements/`	the DOORS modules (stakeholder, system, segment) and the requirement copy the Cameo model kept	library
Logical	`model/logical/`	what the network does: the observation cycle and its steps	library, requirements
Physical	`model/physical/`	what it is made of, one folder per segment, and the allocations of functions to parts	library, requirements, logical
Analysis	`model/analysis/`	constraint and calculation definitions (the energy balance)	everything above it in this table
Verification	`model/verification/`	the verification cases	everything above it

The order is written in `sysml.conf` (the `layers` key), and from sprint 5 the pipeline's complexity check fails any change that makes a package import from a layer above it (Chapter 47, Keeping SysML v2 from becoming a spider web (p. 645)).

> **Case study: Why requirements sit below design**
>
> A new reader often expects requirements at the top, because they come first in the life cycle. In the dependency order they sit near the bottom: the design *satisfies* the requirements, so the design depends on them, not the other way round. A requirement that imports a part

> definition is a requirement that knows its own solution. When the layering check flagged one in sprint 5, it turned out to be exactly that: a segment requirement written around the prime's chosen modem.

16.3 The layout map

The converters decide where each package lands. By default they write one folder per top-level package – `model/stations/Stations.sysml`, `model/types/Types.sysml` – which is fine for a first look and wrong for a project: every package at the same level, no layers. Given a **layout map**, they write each package where the map says, and they do it the same way for every drop.

Tidewater's map is `examples/tidewater-v1/layout.map` in the kit:

examples/tidewater-v1/layout.map

```
# layout.map -- where each top-level package of a Tidewater drop goes in the government's model
repo.
# Used by both converters, for every drop:
#   perl bin/xmi2sysml.pl   --out model --layout-map examples/tidewater-v1/layout.map
tidewater.xmi
#   perl bin/reqif2sysml.pl --out model --layout-map examples/tidewater-v1/layout.map
tidewater.reqif
# The same map keeps every package at the same path from drop to drop, so a new drop merges 3-way
on the
# vendor branch. A package the prime adds later is not listed: it lands in unsorted/ and the
report names it.
#
#   PACKAGE = folder/                -> folder/<Package>.sysml
#   PACKAGE = folder/File.sysml      -> exactly that file

# library: types, interfaces, and what the Cameo model had outside any package
Types        = library/
Interfaces   = library/
Tidewater    = library/

# requirements: the three DOORS modules (reqif2sysml), and the copy the Cameo model keeps
(xmi2sysml)
Stakeholder Needs    = requirements/stakeholder/
System Requirements  = requirements/system/
Segment Requirements = requirements/segment/
Requirements         = requirements/model/

# logical: what the network does
Behavior     = logical/

# physical: what it is made of, one folder per segment, and what performs what
Network      = physical/network/
Stations     = physical/stations/
DataCenter   = physical/datacenter/
Operations   = physical/operations/
Allocations  = physical/allocation/

# analysis and verification
Analysis     = analysis/
Verification = verification/
```

```
# clutter from the document port: Discrepancy Reports ask the prime to delete it (gone in drop 6)
Legacy Import = legacy/

# anything the prime adds that is not listed above
* = unsorted/
```

Three features of the map matter:

- **One line per package**, by the name the package has in the prime's model. `Stations = physical/stations/` puts `Stations` in `model/physical/stations/Stations.sysml`.
- **The same map for both converters.** The XMI converter writes the model's packages; the ReqIF converter writes the DOORS modules (`Stakeholder Needs = requirements/stakeholder/`).
- **A catch-all line.** `* = unsorted/` sends any package the prime adds in a later drop to `model/unsorted/`, and the converter's report names it. A new package arriving in `unsorted/` is a signal to decide where it belongs and add its line to the map – before the first government edit to it.

Convert drop 1 with the map, in an empty folder, and look at what lands where:

```
perl "$KIT/bin/xmi2sysml.pl" --out model --layout-map "$KIT/examples/tidewater-v1/layout.map"
"$KIT/examples/tidewater-v1/tidewater.xmi"
perl "$KIT/bin/reqif2sysml.pl" --out model --layout-map "$KIT/examples/tidewater-v1/layout.map"
"$KIT/examples/tidewater-v1/tidewater.reqif"
find model -name '*.sysml' | sort
```

Sixteen files: fifteen in the six layers, and one in `legacy/`, the clutter the team removes in sprint 5. Every later drop, converted with the same map, lands in the same files.

Warning: Change the map only with a plan

Changing a line of the map moves a package for every future drop. If you must (a package was put in the wrong layer), do it in one merge request that moves the file in `develop` *and* updates the map *and* re-imports the current drop on the vendor branch with the new map, so both sides of the next merge agree on the path. Treat it as an architecture decision and write an ADR (Section 16.8, Decisions and conventions on file (p. 251)).

16.4 The whole repository

The model is one folder of the repository. The rest holds what the pipeline, the reviewers and the next person on the team need. The kickoff script makes the skeleton (Chapter 15, Setting up GitLab (p. 226)); here it is, made in a scratch folder, with drop 1 converted into it:

```
perl "$KIT/tools/gitlab/kickoff.pl" scaffold tidewater-model --project tidewater/model --title
Tidewater > /dev/null
cd tidewater-model
```

```
perl "$KIT/bin/xmi2sysml.pl" --out model --layout-map "$KIT/examples/tidewater-v1/layout.map"
"$KIT/examples/tidewater-v1/tidewater.xmi" > /dev/null
perl "$KIT/bin/reqif2sysml.pl" --out model --layout-map "$KIT/examples/tidewater-v1/layout.map"
"$KIT/examples/tidewater-v1/tidewater.reqif" > /dev/null
git ls-files | sed 's#/[^/]*$#/#' | sort -u
```

(The last line lists the folders git already tracks; the converted model is not committed yet.) Put together, the repository looks like this:

```
tidewater-model/
  .gitlab-ci.yml           the pipeline (one commented file in the first twelve weeks)
  .gitlab/                 issue and merge request templates; project.conf (the project's GitLab
  settings)
  ci/job.pl                what each pipeline job does, in Perl, the same under PowerShell, Git
  Bash or Docker
  .ci/                     the pipeline's ratchet files (outline-max ...): numbers that may only
  go down
  CODEOWNERS               who must approve a change to which folder
  README.md CHANGELOG.md   what this is; what changed in each baseline
  sysml.conf               the model's roots, level tags, layers, IDEF0 names
  .kit-version             the version of the agile kit the pipeline must run (from sprint 7)
  model/                   THE MODEL, in layers (library, requirements, logical, physical,
  analysis, verification)
  imports/v1.0/ ...        each prime drop exactly as delivered, with its MANIFEST (arrives via
  vendor/prime)
  verification/            recorded verification results: results.csv, one schema for every
  verdict
  docs/                    generated documents (model.pl docs) and the hand-written ones below
    adr/                   architecture decision records, one decision per file
    conventions/           naming, markings, the review checklist
    memos/                 memos to management, under review like the model
    onboarding.md          the first day for a new team member
    model-exchange-agreement.md   the agreement with the prime about drops
```

16.5 The files at the top, one by one

The top of the repository is a handful of files that each answer one question. Look at three of them in the scaffold.

`sysml.conf` – what the model's roots are. The kit's model tools read it to know where the part tree starts, where the function tree starts, what the level tags are called, and (later) the layer order and the IDEF0 names:

```
cat sysml.conf
```

The scaffold writes generic level tag names (`L1System` ...). Tidewater renamed them to its own (`L1Network, L2Segment, L3Station, L4Subsystem, L5Assembly, L6Component`) in the conventions story of sprint 2, so the tags read naturally in the model. Change them before the first level tag is written, not after.

`CODEOWNERS` – who approves what. Each section names a folder and the people who must approve a change to it. Because the folders are the layers, ownership follows the architecture:

```
grep -n "^\[\|^/model" CODEOWNERS
```

The interface definitions get two sections, so a change to them needs two approvals – the systems lead's and one modeler's (Section 15.8, Code owners: the signature authority matrix (p. 239)).

`.gitignore` – what never goes in. Every file a pipeline job writes into the project folder (logs, reports, the Pages site, exports) is listed, so a local run of the pipeline never leaves junk to commit by accident, along with the editor's swap files:

```
grep -c . .gitignore
grep -n "public\|swp" .gitignore
```

The rest of the top: `.gitlab-ci.yml` is the pipeline (Chapter 20, The pipeline, job by job (p. 302)); `ci/job.pl` does each job's work in Perl so the same pipeline runs under PowerShell on the Windows runner, Git Bash on a laptop, or a Docker image (Section 21.6, The lesson of the first week: PowerShell (p. 350)); `.gitlab/` holds the issue and merge request templates and `project.conf`, the project's GitLab settings as a file; `README.md` says what the repository is and how to start; `CHANGELOG.md` gets one section per baseline.

16.6 What never goes in the repository

A repository is copied to every laptop, backed up, mirrored and read by everyone with access. Some things do not belong in it, however convenient:

Table 16.2: What never goes in

Never commit	Why	Where it goes instead
tokens, passwords, keys	anyone with read access has them; deleting the commit does not delete them	masked CI/CD variables; the person's own credential store
the prime's `.mdzip` Cameo project	large, binary, proprietary, and nobody can review a change to it	the approved storage the CDRL names; the drop's XMI and ReqIF go in `imports/`
pipeline outputs (reports, logs, `public/`)	they belong to the pipeline run that made them	job artifacts, kept by GitLab
personal working notes	they become everyone's, forever	your own notes, or an issue comment
files with a higher marking than the repository	spillage	the system accredited for them; rehearse what to do (Chapter 48, Security: protect the environment, engineer it into the system (p. 651))

16.7 The rules that make the layout work

A layout is only as good as the rules everyone follows inside it. Tidewater's rules, each with the check that keeps it true:

Table 16.3: Rules and the checks that keep them

Rule	Why	Kept true by
Layers import downward only	a change at the top cannot ripple into the bottom	the complexity job's layering check (sprint 5)
One folder per segment under `physical/`, owned by its CODEOWNERS section	ownership matches structure; reviews go to the right person	CODEOWNERS approval on `develop`
Converter files are changed only to close a TODO line or by an accepted DR	the prime's content stays the prime's; drops keep merging	review checklist; `--impact` lists hand edits per drop
`imports/` is never edited	the record of what was received, and when	it arrives only through `vendor/prime`, which only Tom can push
Generated documents in `docs/` are regenerated, never hand-edited	they always match the model	the `docs` job fails when `docs/` is stale
Decisions live in `docs/adr/`	a new person learns why before working around it	review checklist
No secrets anywhere	the repository is copied, backed up and read by many	the `secrets` job

16.8 Decisions and conventions on file

The scaffold gives every new model repository a small `docs/` folder written for engineers new to all this:

- **`docs/adr/`** holds architecture decision records: one file per decision, numbered, each with its context, the decision and its consequences. Tidewater's first ten (D-1 to D-10, from week zero: gitflow, merge commits, the vendor branch, the layered layout ...) were written in sprint 2. When someone asks "why is it like this?", the answer is a file name.
- **`docs/conventions/`** holds the naming conventions, the markings guide the security officer fills in, and the review checklist every reviewer uses (Chapter 22, Reviewing a model change (p. 357)).
- **`docs/onboarding.md`** is the first day for a new team member: run the doctor, do practice lessons 10 to 13, read these chapters.
- **`docs/model-exchange-agreement.md`** is the agreement with the prime: what a drop contains, how often, stable ids, change notes, the return path for discrepancy reports (Chapter 45, The prime's model changes: receive, assess, disposition (p. 619)).

Read one of the decision records the scaffold wrote:

```
cat docs/adr/0002-prime-boundary.md | head -20
```

16.9 Compared with a reference layout

Teams that have done this before publish reference layouts, and one the user of this book brought in is worth comparing line by line, because Tidewater departs from it in four places, each for a reason.

Table 16.4: A reference layout and Tidewater's, compared

Reference layout	Tidewater	Why Tidewater differs
`model/` in layers: library, requirements, logical, physical, analysis, verification	the same layers, via the layout map	– (the same idea)
`model/external/prime/`: only the interface surface of the prime's model, generated by script	the prime's whole model, mirrored on the vendor branch and edited to close TODO lines	the port *is* the goal: the government needs the whole system model in v2. A subsystem team consuming a prime's model should use the reference's way (Chapter 45, The prime's model changes: receive, assess, disposition (p. 619)); Tidewater does it for DOORS
`tools/` with Python checks, generators and their tests, in the repository	the agile kit, pinned by version (`.kit-version`); only `ci/job.pl` is in the repository	nothing to install on a locked-down laptop; the kit's tools have their own tests
`reports/` never committed; documents rebuilt in CI	`docs/` committed, with a gate that fails when stale	reviewers see the rendered change in the merge request; the baseline tag's checksum covers `docs/`
`prime-inputs/` as a restricted folder or separate repository	`imports/` on the vendor branch	one repository is simpler for three people; the separate restricted repository is right when the prime's data has different access rules
`docs/adr/`, `docs/conventions/`, `onboarding.md`, `model-exchange-agreement.md`	the same	–
`test/` harnesses and `results/` with signed-off manual verdicts	`verification/results.csv`, one schema	Tidewater's tests are run by the prime's lab; the government records results (Chapter 44, Verification as a closed loop (p. 613))

Neither column is "right." The reference is built for a team that owns its design and its tests; Tidewater is a government team porting and maintaining a model the prime owns. Choose the column whose reasons match your situation.

16.10 When one repository is not enough

This section is LATER: three people and one model fit one repository comfortably for the whole eighteen months.

A model outgrows one repository when more than one team changes it, or when other teams consume parts of it. The way to split is along the seams the layers already drew, into a GitLab group of repositories:

```
program-x/                 a GitLab group
  mbse-library             the library layer + the pinned tool version, released by tag
  mbse-system              requirements, logical, system-level physical
  mbse-comms, mbse-power   one repository per subsystem; each imports the library and the system
  interfaces
  mbse-tools               shared tooling, published to the package registry
  prime-inputs             restricted: the prime's drops, as received
```

Each repository pins the versions of what it imports (a tag of the library, a tag of the system interfaces). The system repository's pipeline pulls the tagged releases of the subsystems and runs the integrated checks. It is the same layering, enforced across repositories instead of folders – which is why getting the layers right in one repository first is the best preparation for ever needing more than one.

Warning: Do not federate early

Splitting a model across repositories before more than one team needs it buys version pinning, release coordination and cross-repository merges for no gain. Tidewater stayed in one repository for all thirty-nine sprints.

16.11 Applying this to your project

- **Copy:** the layers, the layout map used for every drop, the catch-all `unsorted/` line, the `docs/` folder with ADRs and conventions, and the rule that converter files change only to close TODO lines.
- **Adapt:** the layer names to your domain (a spacecraft might add a separate `operations/` layer); the folders under `physical/` to your segments or subsystems; the map's lines to your prime's package names.
- **Decide locally, before the first import:** whether the prime's model is mirrored whole (Tidewater) or only its interface surface (the reference), whether its drops live on a vendor branch or in a restricted repository, and whether generated documents are committed.

Exercise 16.1: Place a new package

Drop 4 adds a package named `Drifters` that the layout map does not list. Where does it land, how do you find out, and what three things do you change, in one merge request, before anyone edits it?

Exercise 16.2: Spot the upward import

`model/requirements/segment/Segment_Requirements.sysml` contains `private import Stations::*;`. Which rule does it break, why is it a smell, and what do you do about it given that the file came from the prime?

You can now explain why the layout is fixed at the first import, write a layout map, convert a drop into layers, name what goes in each folder of the repository and why, compare your layout with a reference, and say when a model needs more than one repository.

CHAPTER

17

The DevSecOps tools that earn their keep

From what you know →

You know

the tools the program bought, and the ones nobody used.

→ Here it is called

the DevSecOps tools in GitLab, each with its tier and its value.

→ What is new

each tool is adopted only when the team feels the problem it removes.

Survival kit

What you have

a GitLab project with its protections, labels, sprints and templates (Chapter 15, Setting up GitLab (p. 226)).

What you need

to know which of GitLab's many features to turn on, in what order, and why each one is worth the trouble for three systems engineers with no software background. This chapter is the list, with the value of each tool stated in the terms management and the ISSM will ask about, the tier it needs, how to set it up, and when the Tidewater team adopted it. It also lists the features the team deliberately left off.

17.1 The problem with "DevSecOps platform"

GitLab's menus list more than a hundred features: security scanners for code and containers, dependency inventories, package registries, environments, feature flags, error tracking, incident management, compliance frameworks, value stream dashboards. Most of them exist for software teams shipping software. A team of three systems engineers porting a model needs a fraction of them, and turning on the rest has a cost: alerts nobody reads, jobs that fail for reasons nobody understands, and a growing sense that the tool is in charge.

The team's rule, which Tom wrote on the whiteboard in the first week and which survived the twelve weeks:

> *A tool goes on when a problem we already have costs more than the tool does. Not before.*
>
> — Tom, sprint 1 planning

That rule produced the adoption order in this chapter: one or two tools a sprint, each introduced as a story with a stated value, each measured afterwards. The order matters as much as the list. A tool turned on before the team feels the problem it solves is noise; the same tool turned on the week after the team felt the problem is a relief.

17.2 The list at a glance

Table 17.1: The GitLab tools the team uses, by tier and adoption sprint

Tool	Tier	What it gives the team	Adopted
Issues, labels, milestones, boards	Free	the action item list, the sprint backlog and the status wall in one place	sprint 1
Protected branches and tags	Free	nobody changes the integration branch or a baseline except through review	sprint 1
Merge requests	Free	every change proposed, discussed, checked and approved in one page	sprint 1
Code owners	Free (enforced: Premium)	the signature authority matrix, applied per file	sprint 1
CI/CD pipeline	Free	the configuration audit on every change, in minutes	sprint 1
Test report widget (JUnit)	Free	the merge request says which check failed, on which file	sprint 2
Code Quality widget	Free	the merge request lists the findings this change added or fixed	sprint 2
Metrics report widget	Premium	the merge request shows how the TODO count and lint numbers moved	sprint 2
Wiki	Free	working agreements, runbooks and the glossary, versioned	sprint 2
Secret Detection	Free	tokens and keys are caught before they reach the integration branch	sprint 3
GitLab Pages	Free	the model as a website for people who will never open a repository	sprint 4
Pipeline schedules	Free	the nightly check and the weekly status export run by themselves	sprint 4
Generic package registry	Free	each prime drop kept, checksummed, exactly as delivered	sprint 5
Releases	Free	each baseline published with its notes and evidence	sprint 5
Time tracking	Free	estimates and time spent on issues, for the comparison with the old way	sprint 5
Environments	Free	the model site's deployments listed with the commit each one came from	sprint 6
Value stream analytics	Free (limited) / Premium	how long work spends in each stage, from issue to merge	sprint 6
Approval rules	Premium	"one of Jim or Bob approves every change", enforced	when licensed

Tool	Tier	What it gives the team	Adopted
Push rules	Premium	the server rejects commits without the right author e-mail or with large files	when licensed
Merge request and iteration burndown	Premium	sprint burndown charts without a spreadsheet	when licensed
DORA metrics	Ultimate	deployment frequency, lead time, change failure rate, time to restore	not used

The rest of this chapter takes them in adoption order, then lists what the team left off.

17.3 Sprint 1: the work and the gate

17.3.1 Issues, labels, milestones and boards

In the document era, the team's work lived in four places: an action item spreadsheet on a share drive, the integrated master schedule, e-mail, and Jim's notebook. The first tool replaced three of them. Every piece of work starts as an issue; the issue's labels say what kind of work it is and where it sits in the plan; its milestone says which sprint promised it; the board shows all of it as cards.

Value. One list, one status, linked to the change that does the work. When a merge request says `Closes #12`, issue 12 closes when the change merges, with a link from the issue to the exact lines that did it. Nobody has to update a status column.

Set-up. The labels and milestones come from `project.conf` (Chapter 15, Setting up GitLab (p. 226)). The board is two clicks: Plan > Issue boards, create a board scoped to the current milestone, add one list per `workflow::` label. Because the labels are scoped, dragging a card from "in progress" to "review" swaps the label.

Here is the board legend the team kept in the wiki:

GitLab Markdown: tools/gitlab/samples/14-board-and-labels.md

```
# The board and the labels

## Board: "Sprint" (Plan > Issue boards)

Scope: milestone = the current sprint. Lists, left to right, are scoped labels, so moving a card
swaps the label:

| Open | ~"workflow::ready" | ~"workflow::in-progress" | ~"workflow::review" |
~"workflow::blocked" | Closed |
|---|---|---|---|---|---|
| not yet ready | can be pulled | someone is on it | an MR is open | see the last comment | done
|

Swimlanes (Premium): by epic. On Free, a second board "Epics" with one list per ~"epic::…" label.

## Board: "Prime" (discrepancy reports)

Scope: label ~"owner::prime". Lists: ~"severity::1", ~"severity::2", ~"severity::3". Jim reviews
it before each drop.
```

```
## Label legend

| Label | Meaning |
|---|---|
| ~"type::story" | fits in one sprint; several tasks |
| ~"type::task" | under a week (≤ 5 points); one MR |
| ~"type::epic" | under a semester; several stories (Free tier stand-in for a group epic) |
| ~"type::drop" | intake of one prime drop |
| ~"type::discrepancy" | a DR to the prime; closed by the drop that fixes it |
| ~"type::defect" | wrong in the government model or the pipeline |
| ~"tome::port" | year-scale grouping: the port |
| ~"epic::structure" | semester-scale grouping: the structure port |
| ~"owner::prime" / ~"owner::gov" | who fixes it |
| ~"area::stations" | which package; matches a CODEOWNERS section |
| ~"area::interface" | a defect between packages (status metric H) |
| ~"phase::port" | found during the port |
| ~"priority::1" | do first |
```

Quick actions are what make issues fast to drive from a keyboard. Any comment can carry lines that start with a slash; GitLab runs them and records each as a system note. Bob's first task in sprint 1, driven entirely from comments in the emulator. Type it along in the sandbox of practice lesson 11 (`perl $KIT/agile.pl practice 11` builds it; `gl` is the lesson's short name for `perl $KIT/bin/gitlab.pl` against its offline GitLab):

```
. ./env.sh > /dev/null
cd work
gl --as jim issue new "Run both converters with --report" --type task --label type::task
--milestone "Sprint 1" --assign bob
gl --as bob issue note 2 '/estimate 1d
/relate #1
/label ~"workflow::in-progress"'
gl --as bob issue note 2 'Both reports attached to the story. 52 TODO lines, 29 of them in
Stations.
/spend 5h
/label ~"workflow::done"
/close'
gl issue show 2
```

An excerpt of what `issue show` printed:

```
#2  closed  Run both converters with --report
  type: task   labels: ~type::task ~workflow::done   milestone: Sprint 1   assignees: @bob
  2026-10-05T09:40:00Z * @bob: changed time estimate to 1d
  2026-10-05T09:40:00Z * @bob: marked this issue as related to #1
  2026-10-05T09:40:00Z * @bob: added ~"workflow::in-progress" label
  2026-10-06T15:00:00Z * @bob: added 5h of time spent
  2026-10-06T15:00:00Z * @bob: removed ~"workflow::in-progress" label
  2026-10-06T15:00:00Z * @bob: added ~"workflow::done" label
  2026-10-06T15:00:00Z * @bob: closed
  2026-10-06T15:00:00Z @bob: Both reports attached to the story. 52 TODO lines, 29 of them in
  Stations.
```

Every line with a * is a system note: GitLab's record of what changed, by whom, when. That record is what the document-era action item log never had. Nobody wrote it; it wrote itself.

17.3.2 Protected branches, merge requests and code owners

Chapter 15, Setting up GitLab (p. 226) set these up. Their value, in one sentence each, for the management briefing:

- **Protected branches**: the model the program relies on cannot be changed by accident or in a hurry.
- **Merge requests**: every change has a reason, a reviewer and an audit trail, in the same place as the change.
- **Code owners**: the person accountable for a part of the model sees and approves every change to it.

17.3.3 The pipeline

The first pipeline had three jobs: the grammar check, the structure lint and the outline check. Chapter 19, CI/CD from zero (p. 288) explains what a pipeline is from zero; Chapter 20, The pipeline, job by job (p. 302) walks through every job. Its value is the single biggest change in the twelve weeks: every proposed change is checked the same way, in minutes, before anyone is asked to review it.

17.4 Sprint 2: making the merge request say what is wrong

By the middle of sprint 2, Bob had a complaint that turned out to be the most useful complaint of the quarter:

> *When the pipeline is red I have to open the job, scroll through the log, find the line, and then find the file. The old comment matrix at least told me the page number.*
>
> — Bob, sprint 2, day 4

GitLab can show a check's findings directly on the merge request page, in **widgets**, if the job hands GitLab a report file in a format it understands. The kit's `tools/gitlab/ci-reports.pl` turns the model checkers' output into three such formats.

17.4.1 Test reports (JUnit)

JUnit is a file format invented for software tests, which GitLab reads to fill the merge request's test widget: how many checks ran, which failed, and why. For the model, each file the grammar checker reads is one "test", and the lint's zero-errors rule is another. When Bob broke a part type on purpose in a scratch branch (`Hull` typed as `Hul`), the lint job wrote this report. Do the same in the lesson's sandbox, on a scratch branch you then throw away:

```
. ../env.sh > /dev/null
git switch -q -c scratch/broken-type
sed -i 's/part hull : Hull;/part hull : Hul;/' model/physical/stations/Stations.sysml
```

```
perl ci/job.pl lint || echo "exit status $?"
cat lint.xml
```

```
<?xml version="1.0" encoding="UTF-8"?>
<testsuites name="sysml-lint" tests="2" failures="2">
  <testsuite name="lint" tests="2" failures="2">
    <testcase classname="lint" name="model/physical/stations/Stations.sysml"
    file="model/physical/stations/Stations.sysml">
      <failure message="1 error(s), first at line 38: unresolved type 'Hul'"
      type="lint">model/physical/stations/Stations.sysml:38: error: unresolved type
      'Hul'</failure>
    </testcase>
    <testcase classname="lint" name="lint 0/0">
      <failure message="errors=1 warnings=0" type="lint">errors=1 warnings=0</failure>
    </testcase>
  </testsuite>
</testsuites>
```

Nobody reads that file. GitLab does, and the merge request shows "2 failed: model/physical/stations/Stations.sysml, lint 0/0" with the message beside each. That is the page number Bob asked for.

17.4.2 Code Quality

The **Code Quality** widget lists findings by file and line and, more usefully, compares them with the target branch: it shows the findings this merge request *added* and the ones it *fixed*. The same broken type produced:

```
cat lint-cq.json
git checkout -q -- . && git switch -q - && git branch -q -D scratch/broken-type    # throw the
scratch branch away
```

```
[
  {"description": "unresolved type 'Hul'", "check_name": "sysml-lint", "fingerprint":
  "abcf3b0d0373aa7fcf8d21a533cb4d02", "severity": "major", "location": {"path":
  "model/physical/stations/Stations.sysml", "lines": {"begin": 38}}}
]
```

The **fingerprint** is how GitLab recognizes the same finding in two pipelines: it is computed from the file and the message, so a finding that is still there after a change is "unchanged", not "new".

The team used Code Quality for something its designers did not intend: the port burn-down. The `todo` job reports every `// TODO` line the converter left as an informational finding. A merge request that closes TODO lines shows them in the widget as *fixed*; one that adds TODO lines (a careless re-import) shows them as *new*. Bob's satisfaction at seeing "29 fixed" on the Stations merge request in sprint 4 is recorded in Chapter 29, Sprint 4: the model site, a cycle broken, and the demo that landed (p. 459).

17.4.3 Metrics reports

A **metrics report** is a list of `name value` lines; GitLab Premium shows how each value changed against the target branch. The `todo` job's metrics for the raw drop 1 conversion, one line per layer of the model (Chapter 16, The model repository: what goes where (p. 245)):

```
perl ci/job.pl todo > /dev/null
cat todo.metrics
```

```
todo_lines 52
todo_lines_analysis 2
todo_lines_library 4
todo_lines_logical 4
todo_lines_physical 42
```

The per-file count is one `grep` away when you want it: `grep -c "// TODO" model/physical/stations/Stations.sysml` prints 29.

On the Free tier the file is still kept as an artifact, and the team's scripts read it for the scorecard (Chapter 33, The twelve-week scorecard (p. 520)).

17.4.4 The wiki

Sprint 2 also put the wiki to work: the Definition of Ready and Done, the gitflow rules and the glossary moved from a Word document on the share drive into wiki pages, seeded from `tools/gitlab/templates/wiki/`. The value is modest but real: the agreements live next to the work, with history, and a newcomer finds them from the project's sidebar.

17.5 Sprint 3: the “Sec”

17.5.1 Secret Detection

A **secret** is anything that grants access: a GitLab token, a cloud key, a private key, a password. Secrets leak into repositories in mundane ways: a script tested with a real token, a configuration file copied from someone's laptop. Once committed, a secret is in the history forever, copied into every clone.

GitLab's **Secret Detection** is a Free-tier scanner, added to a pipeline with an `include:` of GitLab's template. The scanner runs inside a Docker container, so it needs a runner with the Docker executor. The team's runner is a Windows *shell* runner, where that job would wait “pending” forever and, with “pipelines must succeed”, block every merge request. So the include is conditional on a project variable, `DOCKER_RUNNER`, which `project.conf` sets to `false`:

tools/gitlab/templates/gitlab-ci.yml

```
# GitLab's own secret scanner (Free tier) runs its analyzer in a Docker image, so it needs a
runner with the Docker
# executor. The team's runner is a Windows *shell* runner: there the scanner's job would sit
"pending" forever and, with
```

```
# "pipelines must succeed", block every MR. So the include is conditional: it is pulled in only
when the project
# variable DOCKER_RUNNER is "true" (project.conf [variables] sets it to "false"). The `secrets`
job below is the
# always-on check, runnable on any runner and by `gitlab.pl ci` on a laptop (which lists the
include as "runs on
# GitLab only").
include:
  - template: Security/Secret-Detection.gitlab-ci.yml
    rules:
      - if: '$DOCKER_RUNNER == "true"'
```

The always-on check is the pipeline's own `secrets` job, a few lines of `git grep` for the most common patterns, which runs on any runner and on a laptop (Chapter 20, The pipeline, job by job (p. 302)). When the enterprise offers a Docker runner, setting `DOCKER_RUNNER` to `true` adds GitLab's scanner on top.

Value: the ISSM's first question about any repository is "how do you know nobody committed a credential?" The answer is now a job that runs on every change, with results the ISSM can read.

17.5.2 Why the ISSM blocked the team for three days

The team's security plan for the toolchain went to the ISSM in sprint 3. The ISSM's questions are worth listing, because yours will ask the same ones, and every answer is a GitLab setting or a fact about the pipeline:

Table 17.2: The ISSM's questions and where the answers live

The ISSM asked	The answer, and where it lives
Who can change the model?	members and roles; protected branches (Chapter 15, Setting up GitLab (p. 226))
Who can approve a change?	CODEOWNERS and the approval rule, in the repository
What runs on the runner, as which account?	the pipeline file; the runner's service account has no rights outside its work folder (Chapter 42, Administration (p. 586))
Can a pipeline reach the internet?	no: the runner reaches the GitLab server only; the kit is a local checkout
Where are credentials kept?	masked, protected CI/CD variables; never in the repository; Secret Detection on every change
How long are logs and artifacts kept?	the project's artifact expiry; the audit events log (Chapter 42, Administration (p. 586))
How is a baseline protected?	protected tags; the `baseline` job's rebuild check (Chapter 20, The pipeline, job by job (p. 302))
What happens if the server is lost?	the group backup, and every laptop holds a full clone of the history

The block was not a disagreement. The ISSM was out for three days in week six, and the approval waited. Chapter 28, Sprint 3: the first red X, DOORS in the model, and a wait nobody named (p. 440) tells how the team kept working on everything that did not need the approval.

17.6 Sprint 4: the model as a website

17.6.1 GitLab Pages

Bob's objection in Chapter 3, The directive (p. 22) was that nobody outside the team would read SysML v2 text. Bob was right, and the answer is not to make them read it. **GitLab Pages** publishes a folder of files from a pipeline job as a website. The team's `pages` job builds that folder from the model on every merge into `develop`: the generated documents (the part tree, the function tree, the traceability report), the four diagrams drawn by the kit's view tools (part tree, trace, interconnection, packages) and the IDEF0 plates (Chapter 11, Seeing the model: diagrams from text (p. 158)), the requirement trace against DOORS, the verification matrix, the TODO burn-down and the complexity trend. Chapter 20, The pipeline, job by job (p. 302) shows the job line by line.

Value: the program office, the deputy program manager, the prime's modeler and the review boards read the model's current state in a browser, without a license, without a repository, without asking the team for screenshots. The site was first shown at the sprint 4 review, and the deputy program manager's first question ("Is this today's?") was answered by the commit hash and date on the page.

Pages works on the Free tier. Access can be limited to project members, which is what the team did.

17.6.2 Pipeline schedules

A **schedule** runs a pipeline at set times, like a cron job or a Windows scheduled task. The team set two, under Build > Pipeline schedules:

- **nightly**, 02:00 on weekdays (cron `0 2 * * 1-5`), on `develop`: the full set of checks, so a problem caused by a change outside the repository (a new version of the kit on the runner, an expired certificate) shows up in the morning, not in the middle of someone's merge request;
- **weekly**, Fridays 06:00 (cron `0 6 * * 5`): the `status` job that exports issues, merge requests and pipeline results for the status metrics (Chapter 20, The pipeline, job by job (p. 302), Chapter 36, Telling management (p. 538)).

A schedule is written as a **cron expression**: five fields for minute, hour, day of the month, month and day of the week. `0 2 * * 1-5` reads "minute 0, hour 2, any day of the month, any month, Monday to Friday". The team keeps its schedules in `project.conf` under `[schedules]`, so `gitlab.pl setup` creates them and they are reviewed like any other setting; GitLab's web page (Build > Pipeline schedules) shows the same thing with a "next run" column.

One property of schedules surprises everyone once: **a schedule runs as its owner**, with the owner's permissions. If the person who created it leaves the project, the schedule stops running, silently. The team made Tom the owner of both schedules and put "take ownership of the schedules" on the off-boarding checklist (Chapter 42, Administration (p. 586)).

17.7 Gates, and what they cost

A **gate** is a check that must pass before work moves on. Systems engineers know gates as milestone entry and exit criteria; the difference here is that the gate is enforced by software, on every

change, not by a board twice a year. Tidewater's gates are of two kinds:

- **pipeline gates**: jobs that must pass (grammar, lint zero over zero, the outline ratchet, current docs, secrets);
- **merge gates**: project settings that must be satisfied before the merge button works (pipeline succeeded, code owner approval, discussions resolved, protected branches and tags).

What they cost is a question management asks early, and the answer has three parts:

Table 17.3: What the gates cost (check GitLab's current pricing page; tiers and quotas change)

Item	Cost
The agile kit and every check it runs	nothing: it is a folder of Perl scripts
Pipelines on your own runner (self-managed GitLab or your own runner on GitLab.com)	no per-minute charge; you pay for the machine
Pipelines on GitLab.com's hosted runners	compute minutes: a free monthly quota, then paid
Pipeline gates, protected branches and tags, discussions resolved	Free tier
Approval rules, enforced code owner approval, push rules, merge trains	Premium
Security policies, compliance frameworks and pipelines	Ultimate

Tidewater runs on its own runner, so the pipelines themselves cost nothing beyond the runner machine. The gates that need Premium were practiced on the emulator and requested for the second year. Prices and tier boundaries change often: check GitLab's pricing page and your organization's license before promising a feature to anyone.

17.8 Sprint 5: deliveries and releases

17.8.1 The generic package registry: keeping the prime's drops

Each prime drop is a contract deliverable. The government must be able to produce exactly what was delivered, on which date, years later. GitLab's **package registry** stores files under a name and a version; the "generic" kind takes any file. The `drop-package` job uploads each drop's XMI, ReqIF, CSV and manifest under `prime-drop/v1.0`, `prime-drop/v1.1` and so on, authenticated by the job's own short-lived token. The emulator has the same commands; in the lesson sandbox, file the drop the lesson ships with:

```
. ../env.sh > /dev/null
gl package put prime-drop v1.0 ../drops/v1.0/tidewater.xmi
gl package list
```

```
prime-drop/v1.0/tidewater.xmi  122711 bytes  sha256 af179896...
prime-drop/v1.0  tidewater.xmi
```

Value: the deliverable and the model it became are kept together, with checksums, retrievable by version. The receiving inspection the contracting officer's representative used to do by e-mail now has a record.

17.8.2 Releases and baselines

A **release** in GitLab is a page attached to a tag: notes, links, the evidence of what was checked. For the team, a release is the published face of a baseline. When a `baseline-*` tag passes its rebuild check, the `publish-package` job files the baseline itself in the package registry as `tidewater-model/<tag>` (the model and its generated docs as one archive, the verification matrix, the toolchain inventory and a checksum list), and the `publish-release` job creates the release: that baseline's section of the CHANGELOG as the notes, links to those package files as the release's assets, and the sprint milestones it closes. A downstream team that wants "the model as of baseline 1" downloads one versioned archive; nobody e-mails a zip. The team rehearsed the whole procedure on a scratch project in sprint 5 and used it for real for the interim baseline in sprint 6 (Chapter 31, Sprint 6: Behavior by hand, a table for the program, and baseline-0 (p. 495)).

GitLab also collects **release evidence**: a snapshot, taken when the release is created, of the issues and milestones it names and the test reports of the tag's pipeline. For a baseline, that snapshot is the configuration status accounting record: what was in it, what was checked, when.

17.8.3 Time tracking

Issues and merge requests accept `/estimate` and `/spend` quick actions. The team used them for one reason: the twelve-week comparison with the document era needed effort numbers, and estimating after the fact is guessing. They did not use time tracking to judge individuals, and said so in the working agreements.

17.9 Sprint 6: showing the flow

17.9.1 Environments

An **environment** is GitLab's name for a place a pipeline deploys to. For a software team that is a test server or production. For the model team there is one: the model site. Naming the `pages` job's environment `model-site` gives a page (Operate > Environments) listing every deployment of the site with the commit and merge request it came from. When the deputy program manager asked "what changed on the site since Tuesday?", the answer was a list.

17.9.2 Value stream analytics

Value stream analytics measures how long work spends in each stage: issue created, first commit, merge request opened, merged. The Free tier shows a limited project-level view; Premium adds custom stages. The team used it once, for the management briefing, as a cross-check on the numbers its own scripts computed from the exported merge request history (Chapter 33, The twelve-week scorecard (p. 520)).

The same numbers are available on any tier from the kit, because they are computed from data every tier's API exposes: `gitlab.pl analytics --by sprint` prints the value stream stages, the four DORA measures translated for a model repository, pipeline success rates and durations, merge request throughput and issue flow, and `--csv DIR` writes them for a spreadsheet. Chapter

32, Measuring with GitLab analytics (p. 513) explains each number and which ones belong in a briefing. On the lesson's sandbox, with only a few days of history, most of them are still empty:

```
. ../env.sh > /dev/null
gl analytics --by sprint | head -12
```

17.10 Tokens: which one, where

Tokens confused the whole team, so the team made a table. A token is a password for a program rather than a person. GitLab has several kinds, and choosing the right one is most of security in practice:

Table 17.4: GitLab tokens and what each is for

Token	Who or what holds it	Lifetime	The team used it for
Personal access token	one person	until its expiry date (set one)	`gitlab.pl setup` and ad hoc scripts on a laptop
Project access token	a bot user tied to one project	until its expiry date	the weekly `status` export (`read_api` only), stored as a masked variable
Deploy token	a machine outside the project, limited to named scopes	until its expiry date	LATER: the prime's Jenkins uploads drops with `write_package_registry` only (`gitlab.pl token deploy new`, Chapter 45, The prime's model changes: receive, assess, disposition (p. 619))
CI job token (`CI_JOB_TOKEN`)	one running job	only while the job runs	uploading drops to the package registry
Runner authentication token	the runner	until the runner is re-registered	registering the Windows runner (Tom and the admins)
Trigger token	a system outside GitLab that starts a pipeline	until revoked	LATER: the prime's Jenkins starts the intake pipeline after an upload (`gitlab.pl trigger new`)

The rules the team kept: prefer the job token (it expires by itself); give every other token an expiry date and the narrowest scope; store tokens in masked, protected CI/CD variables or in your shell's environment, never in a file in a repository; rotate them every 90 days (Chapter 42, Administration (p. 586)).

17.10.1 Protected and masked variables

CI/CD variables (Settings > CI/CD > Variables) are values the pipeline can read as environment variables. Two flags matter:

- **Masked**: GitLab replaces the value with `[MASKED]` if a job prints it in a log.
- **Protected**: the value is only given to pipelines running on protected branches and tags. A pipeline on someone's feature branch never sees it.

`STATUS_TOKEN`, the read-only token the weekly export uses, is both. `KIT`, the path of the kit on the runner, is neither: it is not a secret, and every job needs it.

17.11 What the team deliberately left off

Table 17.5: Features the team did not use, and why

Feature	Why not, for now
SAST (static application security testing)	scans source code for vulnerabilities; the repository holds a model, not code. Revisit if the team writes tools
Dependency scanning	lists vulnerable software libraries; the model has none, and the kit is a separate, reviewed checkout
Container scanning	scans Docker images; the runner uses no containers
GitLab Requirements Management (Ultimate)	would create a second requirements database; DOORS stays authoritative, and the trace job compares against it
Merge trains (Premium)	queue merges to keep a busy branch green; three people merging a few times a day do not need a queue
Auto DevOps	builds and deploys software by convention; it has nothing to build
Feature flags, error tracking, incident management	for running software in production
Compliance frameworks and pipelines (Ultimate)	enforce a pipeline across many projects; one project does not need it yet

None of these is a bad feature. Each was left off because the problem it solves was not one the team had. Two of them (SAST, if the team ever writes its own scripts into the repository, and compliance pipelines, if the organization puts several model repositories on the same rules) are on the list to revisit at the end of each semester (Chapter 42, Administration (p. 586)).

17.12 The cost side

Every tool above has a cost, and the briefing in Chapter 35, The briefing (p. 532) states it honestly:

- **Runner time**: the pipeline takes well under a minute of checks per merge request on the team's runner (Chapter 20, The pipeline, job by job (p. 302) has the job timings); the nightly and weekly schedules add a few minutes a week.
- **Attention**: each widget is one more thing on the merge request page. The team removed none, but the rule was that a widget nobody looked at for a sprint would be switched off.
- **Licenses**: everything adopted in the twelve weeks runs on the Free tier, except the metrics widget and enforced code owner approvals, which the team practiced on the emulator and requested for the second year.

17.13 Exercises

Exercise 17.1: Apply the rule

List three problems your team has today with its documents or its model. For each, name the GitLab tool from this chapter that addresses it, and the problem it would create if you turned it on before anyone felt the original problem.

Exercise 17.2: Read a widget

Create a branch in your practice project, break one part type as Bob did, and run the pipeline locally (`gitlab.pl ci --event merge_request_event --target develop`). Open `lint.xml` and `lint-cq.json`. Which line of each would GitLab show on the merge request?

Exercise 17.3: Choose the token

For each job, choose a token from the token table and justify it: (a) a laptop script that lists open issues every morning; (b) the pipeline job that uploads a drop; (c) a scheduled export to a reporting server outside GitLab.

Exercise 17.4: Argue for one left-off tool

Pick one feature from the "left off" table and write the paragraph that would convince Tom to turn it on. What evidence would you need?

17.14 Applying this to your project

- **Copy**: the adoption rule (a tool goes on when a felt problem costs more than the tool); issues with scoped labels and milestones; protected branches and merge requests; a three-job pipeline on day one; report widgets as soon as people complain about reading logs; Secret Detection; Pages for everyone who will not open a repository; the package registry for contract deliverables; releases for baselines.
- **Adapt**: the order (if your ISSM asks for secret scanning on day one, do it on day one); the tiers (check what your license has before promising a feature); the schedules (nightly may be weekly for a slow-moving model).
- **Decide locally**: which tokens your organization allows; who may create project access tokens; how long artifacts and logs are kept; whether the model site may be visible beyond the project's members.

CHAPTER

18

A local AI assistant on the laptop

From what you know →

You know

asking a colleague to look over a draft.

→ Here it is called

a local AI model on the laptop.

→ What is new

a draft in seconds, never trusted, checked by the same gates as anything you write.

Survival kit

What you have

a model repository, a pipeline that checks every change (Chapter 20, The pipeline, job by job (p. 302)), Vim (Chapter 10, Vim for models (p. 143)), and a laptop with no discrete graphics card and somewhere between 16 and 32 GB of memory.

What you need

to know whether a language model running on that laptop (or on a shared server down the hall) can take some of the typing off the team – and how to use one without letting it put a single unchecked line into the model. This chapter is the approval path, the two ways to host a model, a one-day bake-off to pick one, the SysML v2 caveat every reader must understand, the rules, and worked Tidewater uses in the order they pay off.

Every chapter so far has been about tools that are *deterministic*: the grammar check gives the same answer for the same file every time, and so does lint, and so does the pipeline. This chapter is about a tool that is not. A language model writes plausible text. Sometimes the text is right. Sometimes it is wrong in a way that looks exactly like right. The whole value of the chapter is in how the team keeps the first kind and catches the second, and the answer is the same one the rest of the book gives: the gates decide, not the author – and the model is just another author.

Nothing in the Tidewater process needs an AI assistant. The kit runs without one, the pipeline

runs without one, the twelve weeks in Part III were planned without one. Treat everything here as optional, and adopt it the way the team adopted every other tool (Chapter 17, The DevSecOps tools that earn their keep (p. 255)): when a problem you already have costs more than the tool does, and not before.

18.1 What a local model is, in plain words

A *language model* is a large file of numbers – a few gigabytes – that was built by feeding a program an enormous amount of text and adjusting the numbers until the program got good at one thing: given the beginning of a piece of text, guess the next word. Do that guess thousands of times in a row and you get a paragraph, an answer, a block of code. That is all it does. It has no model of Tidewater, no copy of the SysML v2 specification, no idea whether the text it produced will parse. It has a very good statistical sense of what text of that kind usually looks like.

A *local* model is one that runs on a machine you control: the laptop, or a server on the program's network. The program that runs it is called a *runtime*; the one this chapter uses is **Ollama**, a free runtime that downloads models, keeps them on disk, and answers questions about them on the laptop's own network port (11434). You type a question, the runtime feeds it to the model, the model's answer comes back as text. Nothing leaves the machine.

The systems-engineering analogy that works best: a local model is a **junior analyst with a photographic memory of other people's documents and no access to yours**. Fast, tireless, eager, frequently right about general things, confidently wrong about specific ones, and never the signature on anything. You would not let a new hire's first draft go to the CCB without review. The same rule applies here, and the pipeline enforces it.

Definitions

model
: the file of numbers that writes text (a few GB); one model, many questions

runtime
: the program that loads the model and answers questions (Ollama)

prompt
: the text you give the model – your question plus whatever context you paste in

context window
: how much text the model can consider at once (prompt plus answer); tens of thousands of words for current models

parameters
: the count of numbers in the model ("4B" = 4 billion); more usually means better and slower

mixture of experts
: a model built so only part of it works on each word; big-model quality at a lower running cost

grounding
: pasting the facts the model must use (spec excerpts, your names, good examples) into the prompt

hallucination
: fluent, confident text that is false – an invented library name, a keyword that does not

exist

18.2 Why local, and not a cloud chatbot

Everyone on the team has seen a cloud chatbot. The question Jim got first, from Bob, was why not just use that.

Table 18.1: Cloud chatbot versus local model

Question	Cloud chatbot	Local model
Where does the prompt go?	to a company's servers, over the internet	nowhere: it stays on the machine
Can it see Tidewater files?	only what you paste – and pasting is the problem	only what you paste, and the paste stays local
Works offline / on a closed network?	no	yes, once the model file is on the machine
Approval path	a cloud service authorization for the data's marking	a software intake request, like any other install
Quality	usually higher (very large models)	lower, especially on a laptop CPU
Speed	fast	a few words a second on a CPU; faster on a GPU server
Cost	per seat or per use	the hardware you already have, or one server

The deciding row is the first one. Model text, interface details and requirement wording are program information. On most programs, pasting them into a public chatbot is simply not allowed, whatever the chatbot's terms say, and the kit's own AI hook (`lib/Ai.pm`) repeats the same rule in its header: *only use a service your organisation allows for the data's marking*. A local model changes the question from "may this data leave the building?" (usually no) to "may this software run on this machine?" (a question the software intake process already knows how to answer).

That is the whole case for local. It is not that local models are better – they are not. It is that they are the version of the tool the team can actually get approved.

18.3 Approval comes first

Before anyone types `ollama pull`, two things need approval, separately: the **runtime** (Ollama is an executable, installed like any other program) and each **model** (a multi-gigabyte data file with its own origin, license and training data). Approving one does not approve the other, and approving one model does not approve the next.

18.3.1 Model provenance is a policy question, not a technical one

The SysML-tuned model the kit documents in `docs/OLLAMA.md`, `Babitdor/Qwen3-4B-SysMLv2`, is a community fine-tune published by a single individual, with no stated license and no stated training data – and it is built on **Qwen3**, a base model of Chinese origin. Many organizations' policies restrict the *origin* of AI models regardless of whether they run offline: the concern is the provenance of the numbers themselves, not where the prompts go. Read your organization's AI

policy before pulling **any** model, and expect origin to be one of the first questions the approver asks.

If origin is restricted, the families to look at come from US and allied developers. Each publishes several sizes, and the Ollama library lists them; this book does not quote sizes, tags or benchmark scores, because they change faster than books do and none of them could be verified on the machine this chapter was written on:

Table 18.2: Model families from US and allied developers (check the Ollama library for current tags and memory needs)

Family	Developer	Notes for the request
gpt-oss	OpenAI	open-weight models; check the license and sizes on the Ollama library page
Gemma	Google	several sizes; check the license terms on the model page
Llama	Meta	widely used; check the license's use restrictions
Granite	IBM	positioned for enterprise use; check the license
Phi	Microsoft	small models aimed at modest hardware
Mistral, Devstral	Mistral AI (France)	Devstral is aimed at code and tooling work

None of these was trained to write SysML v2 text, and neither, in any depth, was the Qwen3 fine-tune (Section 18.7, The SysML v2 caveat (p. 275)). That matters less than it sounds, because the best uses for the team are not writing `.sysml` at all (Section 18.10, Worked uses, best fit first (p. 278)).

18.3.2 What to put in the request

Checklist

- ☐ What: the runtime (Ollama, version, download source) and each model (name, tag, digest from `ollama list`, publisher, base model and its origin, license, size on disk)
- ☐ Where: which machines (named laptops, or one shared server) and who has access
- ☐ Data: what goes into prompts (model text, requirement text, pipeline logs) and its marking; a statement that prompts and answers stay on the machine
- ☐ Network: whether the machine needs internet (only to download), and the offline route if it does not have it
- ☐ Use: drafting and explaining only; every output passes the same review and pipeline gates as hand-written work; a human commits it
- ☐ Measure: the bake-off results (Section 18.6, A one-day bake-off before you standardize (p. 274)) and what will be measured afterwards
- ☐ Removal: how to uninstall and delete the model files (`ollama rm`, and the models folder)

Send it through the same channel as any software request, and copy the ISSM. The ISSM's questions will be about the runtime's network behavior (it listens on `localhost:11434`; confirm it is not exposed beyond the machine), the model file's integrity (the digest), and the data rule. Have the answers in the request and the review is short.

18.4 Two ways to host it: every laptop, or one shared server

The typical government laptop has **no discrete GPU** and **16 to 32 GB of memory**. That rules out large models on the laptop and makes small ones slow: on a CPU, a small model produces a few words a second. Two hosting options follow from that, and the trade-off is worth deciding deliberately:

Table 18.3: Laptop installs versus a shared GPU server

	Ollama on each laptop	**One approved GPU server on the program network**
Accreditation	one approval, but installed on every laptop; each install is a change to a managed machine	one system to accredit, once; laptops need nothing but a browser or `curl`
Model size	small models only, or mixture-of-experts models that run acceptably on a CPU	larger models; much faster answers
Works when	the laptop is on, even disconnected	the laptop is on the program network
Data path	prompt never leaves the laptop	prompt crosses the program network to the server (still on the program network)
Administration	every user updates their own runtime and models	one administrator; one approved model list
Cost	none beyond the laptops	a server with a GPU, and someone to run it

For a three-person team, the laptop route is quicker to start and the server route is easier to live with. A shared server is often *easier to accredit* than installs on every laptop, because it is one system with one boundary and one owner, and it allows bigger models that are noticeably better at the hard tasks. If the program already runs a GPU server for analysis or simulation, ask there first.

Pointing the tools at a server instead of the laptop is one setting. Ollama serves the same API in both places, so the kit's hook (`ai_url` in `scrum.conf`, Section 18.10.8, 8. The stand-up summary through the kit's AI hook (p. 285)) just names the server's address instead of `localhost`.

18.5 Installing on a laptop, without admin rights and offline

The steps are the ones in `docs/OLLAMA.md`; this section repeats them so the book stands alone. Requirements, from Ollama's Windows documentation: Windows 10 22H2 or newer, about 4 GB of disk for Ollama itself plus the model's size. A GPU is optional; without one Ollama runs on the CPU, more slowly.

1. Download `OllamaSetup.exe` from `https://ollama.com/download`.
2. Run it. It does **not** need Administrator: it installs into your home folder and adds a tray icon.
3. Check it from Git Bash:

```
ollama --version
curl -s http://localhost:11434/api/version     # the local service answers on port 11434
```

No installer allowed? Ollama also ships a standalone `ollama-windows-amd64.zip` (command line only): unzip it into a folder in your home directory, run `ollama serve` in one Git Bash window and use `ollama` from another.

Models go to `C:\Users\<you>\.ollama\models`. To keep them elsewhere (another drive, outside OneDrive), set the user environment variable `OLLAMA_MODELS` and restart Ollama.

Getting a model – once the model is approved:

```
ollama pull Babitdor/Qwen3-4B-SysMLv2      # 2.5 GB, once (only if your policy allows this
model's origin)
ollama list                                # NAME ... ID b8a8d69fdeec ... SIZE 2.5 GB
ollama show Babitdor/Qwen3-4B-SysMLv2      # parameters, context length, template
```

Check that the ID in `ollama list` matches the digest in the approval. If it does not, the model on the registry has changed since it was approved: stop, and re-approve before using it. The same rule applies to whichever model you are approved for.

When the work laptop has no internet, bring the model in by the approved offline route:

1. On an approved machine with internet, install Ollama and pull the model.
2. Copy the whole `models` folder (with its `manifests` and `blobs` subfolders) to approved removable media, and record a checksum of the archive:

```
cd ~/.ollama && tar -cf qwen3-4b-sysmlv2.tar models && sha256sum qwen3-4b-sysmlv2.tar
```

1. On the work laptop, check the checksum, unpack it into `%USERPROFILE%\.ollama\` (or wherever `OLLAMA_MODELS` points), restart Ollama, and run `ollama list` to confirm the ID.

To remove it: `ollama rm` the model, then uninstall Ollama from Settings, "Add or remove programs", and delete the models folder yourself if you moved it.

18.6 A one-day bake-off before you standardize

Do not pick a model by reading about it. Pick it by giving two or three approved candidates the team's *own* tasks for one day and scoring the results the same way the team scores its own work: did it pass the gates, and how long did a person spend turning the draft into something mergeable?

Five Tidewater tasks make a good bake-off, because they cover the range from the model's best fit to its worst:

1. **Explain a TODO.** Paste a converter TODO line and its v1 XMI fragment; ask what the v1 construct is and what the v2 equivalent would be. Score: correct explanation (Bob judges), minutes to read.

2. **Draft a state def.** Ask for the v2 text of the BuoyModes state machine. Score: passes `sysml.pl check` (yes/no, how many tries), passes lint inside the model, minutes of hand fixing.
3. **Write a `ci/job.pl` helper.** Ask for a small Perl function the pipeline needs (for example, count the TODO lines per package). Score: `perl -c` passes, it runs on the drop-1 conversion, the output is right, minutes.
4. **Draft a Discrepancy Report.** Give the drift between DOORS and the model for TW-SYS-006; ask for a DR to the prime in the template's fields. Score: facts correct, tone usable, minutes of editing.
5. **Explain a pipeline log.** Paste a failing lint job log; ask what failed and what to change. Score: correct cause, correct fix, minutes.

Each person runs each task once per candidate, with the same prompt (write the prompts down first; they are part of the result). Score into one sheet:

Table 18.4: Bake-off scoresheet (one row per task, per model, per person)

Task	Model	Passes gate	Tries	Minutes to mergeable	Correct?	Notes
1 Explain a TODO	candidate A	n/a	1			
2 Draft a state def	candidate A	grammar / lint				
3 ci/job.pl helper	candidate A	perl -c / runs				
4 Draft a DR	candidate A	n/a				
5 Explain a log	candidate A	n/a				
1 Explain a TODO	candidate B	n/a				

Add a baseline row for each task: the same person doing it *without* the model. If the model's "minutes to mergeable" does not beat the baseline on at least two of the five tasks, the honest result is "not yet", and that is a perfectly good result to report. Keep the sheet; it goes in the approval request, and it is the first data point of the measurement the rules below require.

18.7 The SysML v2 caveat

Here is the most important paragraph in the chapter. **Every general-purpose model is weak at SysML v2 textual syntax**, because there was very little SysML v2 text on the internet when these models were trained. The language is new, the textual notation newer, and most published SysML is v1 diagrams. So the model has seen a great deal of UML, a great deal of SysML v1 described in prose, and a great deal of code in languages that look a little like SysML v2 – and it blends them. The result is text that is **plausible but invalid**: v1 vocabulary in v2 clothing (`state machine`, `region`, `block`), keywords borrowed from programming languages, and library names that sound exactly right and do not exist. A SysML-tuned model reduces this; it does not remove it.

Two mitigations, and both are needed:

- **Grounding.** Paste the facts into the prompt: the relevant excerpt of the specification (the textual notation for the construct you want), the model library names it may use (the kit

ships the full list in `tools/sysml/sysml-library-names.txt`), and one or two *good* examples from your own repository. A crude but effective retrieval step is `grep`: find the existing text closest to what you want and paste it. The model is much better at "do it like this" than at "do it from memory".

- **The validate stage.** Everything the model writes goes through `sysml.pl check` and `model.pl lint`, exactly like hand-written text, before a human even reviews it. The worked examples below show both gates catching real mistakes.

The consequence for how the team uses the assistant is the ordering of the next section: the best uses are the ones *around* the model – small Perl helpers, pipeline YAML, report scripts, requirement and DR wording, explanations – where the model's training is strong and the result is easy to test. Writing `.sysml` is the weaker fit, and comes last.

18.8 The rules

Warning: The rules for AI-assisted work

Never trusted. A model's output is a draft from an author with no accountability. It is read, checked and owned by a person before it goes anywhere.

Same gates. AI-drafted text goes through the same branch, merge request, pipeline and code-owner approval as everything else. No gate is relaxed for it, and no gate is added that hand-written work skips.

A human owns the commit. The person who commits it is the author of record. "The model wrote it" is never an answer in review.

Label it. A merge request that contains AI-drafted text carries the scoped label `ai::assisted` (add it under `[labels]` in `project.conf`; it is not in the template's set). The label is how the team measures.

Measure before claiming value. Use the bake-off sheet's columns – gate passes, tries, minutes to mergeable – on labeled merge requests. Report what the numbers say, including "it did not help with X".

Data rule. Prompts carry only what the machine is approved for. A local model does not change the marking of the text you paste into it.

Two practical limits belong next to the rules. A **4-billion-parameter model on a laptop CPU** is slow (a few words a second; a state def takes a minute or two) and has a short attention span: long prompts with many instructions get partly ignored. And every answer can differ from the last one for the same prompt – run it twice and you may get two drafts. Neither is a reason not to use it; both are reasons to keep prompts short and specific.

Prompt patterns that work, in the team's experience writing this chapter's examples:

- **One construct at a time.** "The state def for BuoyModes", not "fix the Stations package".
- **Give it the three things it needs:** the v1 XMI fragment, the converter's TODO line, and the surrounding v2 text the answer must fit (the package's imports and the owning part def).

- **Ask for text only.** "SysML v2 text only, no explanation" – then ask for the explanation separately if you want one.
- **Name the names.** "Use the attribute defs Power and Energy from package Types" beats letting it guess.
- **Paste the error back.** When the gate fails, paste the real error message and the draft and ask for a fix. Two rounds of that is the limit; after that, write it by hand.

18.9 Using it from Git Bash and from Vim

The commands in this section need Ollama and an approved model on your laptop, so the book's own test does not run them; the checks of the drafts they produce, later in the chapter, were run for real. From Git Bash, a one-shot question:

```
ollama run Babitdor/Qwen3-4B-SysMLv2 "SysML v2 text only: a port def TrackPort with an out
attribute track : String"
```

Or an interactive session (`ollama run MODEL` with no prompt; type `/bye` to leave). Qwen3-based models can print a `<think>…</think>` section before the answer; ignore it, or turn it off with `/set nothink` inside a session.

To keep a prompt you will reuse, write it in a file – prompts are worth versioning like anything else – and feed it in:

```
ollama run Babitdor/Qwen3-4B-SysMLv2 < prompts/state-def.txt > draft.sysml
perl "$KIT/tools/sysml/sysml.pl" check draft.sysml
```

From Vim, three forms cover nearly everything. All three were checked in Vim with a stand-in `ollama` command; the model name is whichever one you were approved for:

```
" 1. Send the visual selection to the model; the answer is shown below the command line
:'<,'>w !ollama run Babitdor/Qwen3-4B-SysMLv2

" 2. Send the selection, and put the answer in a scratch buffer you can edit and check
:'<,'>w! ~/prompt.txt
:new | setlocal buftype=nofile | r !ollama run Babitdor/Qwen3-4B-SysMLv2 < ~/prompt.txt

" 3. Insert an answer under the cursor (a one-line question)
:r !ollama run Babitdor/Qwen3-4B-SysMLv2 "SysML v2 text only: a port def TrackPort with an out
attribute track : String"
```

Form 2 is the one to use for model text: the draft lands in a throwaway buffer, not in the model file. Save the buffer as a `.sysml` file in a scratch folder, press `\mx` (grammar-check this file, Chapter 10, Vim for models (p. 143)), fix, and only then copy the passing text into the real file – where `\ml` (lint the model) checks it in context.

18.10 Worked uses, best fit first

Every assistant answer below is **illustrative**: the machine this book was written on could not run the model (its Ollama was too old for Qwen3-based models, and no model was pulled), so the answers are written to show the kind of text such a model produces, including its typical mistakes. Every *check* below is **real**: each draft was saved to a file and run through the kit's actual gates, and the output shown is the gates' actual output.

18.10.1 1. A helper script for the pipeline (best fit)

Small Perl helpers are where a model is strongest: Perl is all over its training data, the task is self-contained, and the result is easy to test. Bob wanted the TODO count *per package*, to decide which package to port next.

The prompt:

```
Write a short core-Perl script todo-by-package.pl MODEL_DIR that counts lines starting with
"// TODO" (after leading spaces) in every .sysml file under MODEL_DIR, grouped by the first
folder
under MODEL_DIR, and prints one line per folder, most TODOs first, then a total line. Core Perl
only,
use strict and warnings, File::Find is fine. No explanation.
```

The draft (illustrative):

```
#!/usr/bin/perl
# todo-by-package.pl MODEL_DIR -- count the converter's // TODO lines per package folder, worst
first
use strict;
use warnings;
use File::Find;

my $dir = shift or die "usage: todo-by-package.pl MODEL_DIR\n";
my %n;
find(sub {
    return unless /\.sysml\z/;
    (my $rel = $File::Find::name) =~ s{^\Q$dir\E/?}{};
    my ($pkg) = $rel =~ m{^([^/]+)/} or return;
    open my $fh, '<', $_ or die "cannot read $File::Find::name: $!\n";
    while (<$fh>) { $n{$pkg}++ if m{^\s*// TODO} }
}, $dir);
my $total = 0;
for my $p (sort { $n{$b} <=> $n{$a} || $a cmp $b } keys %n) {
    printf "%-16s %3d\n", $p, $n{$p};
    $total += $n{$p};
}
printf "%-16s %3d\n", 'total', $total;
```

The real checks: `perl -c` and a run on the real drop-1 conversion.

```
$ perl -c todo-by-package.pl
todo-by-package.pl syntax OK
$ perl todo-by-package.pl model
stations          29
datacenter         5
operations         5
behavior           4
network            3
types              3
analysis           2
tidewater          1
total             52
```

Fifty-two TODO lines, the same total the converter's report and Part I give for drop 1, so the script agrees with an independent count – that comparison is the test. It took one prompt and two minutes of reading. This is the kind of task the bake-off's third row measures, and it is the kind that usually wins.

18.10.2 2. Explaining a failing pipeline log

The second-best fit: a log is text, the model has seen a great many of them, and the explanation is checked by whether the fix works. Here is a real lint failure (the draft that caused it appears in Section 18.10.6, 6. SysML v2 text: the EnergyBalance constraint (p. 283)):

```
model/analysis/Analysis.sysml:8:25: error: unresolved type 'ISQ::SolarPowerValue'
model/analysis/Analysis.sysml:9:22: error: unresolved type 'ISQ::BatteryEnergyValue'
lint: 14 file(s), 2 error(s), 0 warning(s)
elements=148 errors=2 warnings=0
```

An explanation of the kind a model gives (illustrative):

```
The lint job failed because two attribute types in Analysis.sysml, lines 8 and 9, refer to
ISQ::SolarPowerValue and ISQ::BatteryEnergyValue, which are not defined in the model or in the
standard library. Use types that exist: either the model's own attribute defs (Power and Energy
in package Types) or library types such as ISQ::PowerValue and ISQ::EnergyValue.
```

That one is right, and it is checkable: `grep` the library list (`tools/sysml/sysml-library-names.txt` has `PowerValue` and `EnergyValue`, and no `SolarPowerValue`). The habit to build: when the explanation names a fix, confirm the fix exists before you make it.

18.10.3 3. Text for people: an MR description, a DR, a retro summary

Prose is a strong fit, and the review is the same as for any draft: facts first, then tone. Give the model the facts in the prompt – it must not supply them.

A Discrepancy Report draft (illustrative), from the prompt "Draft a discrepancy report to the prime contractor in these fields: Title, Model element, DOORS object, What we found, Evidence,

Requested action. Facts: TW-SYS-006 text differs: DOORS says the bulletin is published twice a day on a fixed schedule; the Cameo model's requirement text says only on a fixed schedule. Found by the trace job's drift check.":

```
**Title:** Requirement text drift, TW-SYS-006 (Bulletin)

**Model element:** requirement TW-SYS-006 "Bulletin", package System Requirements / Functions
**DOORS object:** TW-SYS-006, module System Requirements

**What we found:** The requirement text in the Cameo model does not match DOORS. DOORS states the
data center shall publish a marine bulletin *twice a day*, on a fixed schedule; the model's text
omits "twice a day".

**Evidence:** trace report from pipeline job `trace` (drift: TW-SYS-006), drop v1.0.

**Requested action:** Update the model's requirement text to match DOORS (DOORS is authoritative)
in the next drop, or confirm that DOORS should change.
```

Every fact in it came from the prompt; the model supplied the structure and the sentences. Jim's edit was one word. Notice what the reviewer checks: that it did not *add* a fact (a date, a paragraph number, a severity) that was not given. Models add plausible details; in a DR to a contractor, an invented detail is worse than a missing one.

The same pattern works for a merge request description ("here is the diff summary and the issue number; fill the Default template") and for a retrospective summary ("here are the sticky notes, verbatim; group them into start, stop, continue without changing any wording"). In both, paste the facts, ask for structure, check that nothing was added.

18.10.4 4. Explaining a converter TODO and its v1 construct

Useful for learning, especially in the first weeks when the team is still matching v1 vocabulary to v2. The prompt gives the TODO line and the XMI fragment it came from – both real, from drop 1:

```
// TODO uml:StateMachine 'BuoyModes' (xmi:id _22_1_7f3a0c1_7a13eb20b93a): no SysML v2 mapping in
this converter
```

examples/tidewater-v1/tidewater.xmi

```
        <ownedBehavior xmi:type="uml:StateMachine" xmi:id="_22_1_7f3a0c1_7a13eb20b93a"
        name="BuoyModes">
          <region xmi:type="uml:Region" xmi:id="_22_1_7f3a0c1_f5b2c26f3e15">
            <subvertex xmi:type="uml:Pseudostate" xmi:id="_22_1_7f3a0c1_5c26d0884fcc"
            kind="initial"/>
            <subvertex xmi:type="uml:State" xmi:id="_22_1_7f3a0c1_9a1f982f3529"
            name="Observing"/>
            <subvertex xmi:type="uml:State" xmi:id="_22_1_7f3a0c1_239bb5b533d5" name="Degraded"/>
            <subvertex xmi:type="uml:State" xmi:id="_22_1_7f3a0c1_085b57f4bd53"
            name="Maintenance"/>
```

```
      <subvertex xmi:type="uml:State" xmi:id="_22_1_7f3a0c1_90f9eb5f51cd"
      name="Recovered"/>
      <transition xmi:type="uml:Transition" xmi:id="_22_1_7f3a0c1_3ac7dba8cf09"
      source="_22_1_7f3a0c1_5c26d0884fcc" target="_22_1_7f3a0c1_9a1f982f3529"/>
      <transition xmi:type="uml:Transition" xmi:id="_22_1_7f3a0c1_18494c86076e"
      source="_22_1_7f3a0c1_9a1f982f3529" target="_22_1_7f3a0c1_239bb5b533d5">
        <trigger xmi:type="uml:Trigger" xmi:id="_22_1_7f3a0c1_914894f45b79"
        name="lowBattery"/>
      </transition>
      <transition xmi:type="uml:Transition" xmi:id="_22_1_7f3a0c1_ffbafb629d95"
      source="_22_1_7f3a0c1_239bb5b533d5" target="_22_1_7f3a0c1_9a1f982f3529">
        <trigger xmi:type="uml:Trigger" xmi:id="_22_1_7f3a0c1_f7de241dcfd0"
        name="charged"/>
      </transition>
      <transition xmi:type="uml:Transition" xmi:id="_22_1_7f3a0c1_7ba773be01e6"
      source="_22_1_7f3a0c1_9a1f982f3529" target="_22_1_7f3a0c1_085b57f4bd53">
        <trigger xmi:type="uml:Trigger" xmi:id="_22_1_7f3a0c1_042ecdab7487"
        name="crewOnSite"/>
      </transition>
      <transition xmi:type="uml:Transition" xmi:id="_22_1_7f3a0c1_d0fb7b7514a3"
      source="_22_1_7f3a0c1_239bb5b533d5" target="_22_1_7f3a0c1_085b57f4bd53">
        <trigger xmi:type="uml:Trigger" xmi:id="_22_1_7f3a0c1_66acf0ced679"
        name="crewOnSite"/>
      </transition>
      <transition xmi:type="uml:Transition" xmi:id="_22_1_7f3a0c1_e4fcee824aee"
      source="_22_1_7f3a0c1_085b57f4bd53" target="_22_1_7f3a0c1_9a1f982f3529">
        <trigger xmi:type="uml:Trigger" xmi:id="_22_1_7f3a0c1_394a0a86571a"
        name="crewDeparts"/>
      </transition>
      <transition xmi:type="uml:Transition" xmi:id="_22_1_7f3a0c1_29753e3cc9fb"
      source="_22_1_7f3a0c1_085b57f4bd53" target="_22_1_7f3a0c1_90f9eb5f51cd">
        <trigger xmi:type="uml:Trigger" xmi:id="_22_1_7f3a0c1_82a0ba8b2a36"
        name="liftedAboard"/>
      </transition>
    </region>
  </ownedBehavior>
```

An explanation of the kind a model gives (illustrative): *"This is a UML state machine owned by the MooredBuoy block. It has one region with an initial pseudostate and four states – Observing, Degraded, Maintenance, Recovered – and transitions triggered by named events such as lowBattery and charged. In SysML v2 the equivalent is a state def with states and transitions, exhibited by the part def; the triggers become accepted items or attributes, and the initial pseudostate becomes an entry action followed by the first state."*

That is correct and is exactly the mental bridge a v1 modeler needs. Compare it with Chapter 6, SysML v2 for SysML v1 modelers (p. 57), where the same mapping is shown by hand. The explanation is cheap to check – every claim is visible in the fragment above.

18.10.5 5. SysML v2 text, the weaker fit: the BuoyModes state def

Now the hard part: three drafts of model text, each through the real gates. The first draft (illustrative) is what an under-grounded model typically writes – the v1 vocabulary leaks straight through:

```
package BuoyModesDraft {
    state machine BuoyModes {
        region main {
            initial -> Observing;
            state Observing;
            state Degraded;
            state Maintenance;
            state Recovered;
            Observing -> Degraded : lowBattery;
            Degraded -> Observing : charged;
        }
    }
}
```

The real grammar check:

```
$ perl "$KIT/tools/sysml/sysml.pl" check buoymodes-draft1.sysml
buoymodes-draft1.sysml:2:19: error: unexpected 'BuoyModes'; expected ':', '::>', ':=', ':>',
':>>', ';', '=', '=>', ...
sysml: 1 file(s), 1 error(s)
```

Exit status 1: in the pipeline this is a red `grammar` job. Read the error the way Chapter 10, Vim for models (p. 143) taught: line 2, column 19. The parser read `state machine` as "a state usage named *machine*" and then choked on the next word. There is no `state machine` in SysML v2, and no `region`, no `initial ->`, no `source -> target : trigger` – all four are UML. Pasting the error back may get a better second draft; on a 4B model it often gets a different wrong one. Bob wrote it by hand, from Chapter 6, SysML v2 for SysML v1 modelers (p. 57)'s pattern:

```
package BuoyModesDraft {
    attribute def LowBattery;
    attribute def Charged;
    attribute def CrewOnSite;
    attribute def CrewDeparts;
    attribute def LiftedAboard;

    state def BuoyModes {
        doc /* The buoy's operating modes, from the v1 state machine BuoyModes. */
        entry; then Observing;
        state Observing;
        state Degraded;
        state Maintenance;
        state Recovered;
        transition lowBattery first Observing accept LowBattery then Degraded;
        transition charged first Degraded accept Charged then Observing;
        transition crewArrives first Observing accept CrewOnSite then Maintenance;
        transition crewArrivesDegraded first Degraded accept CrewOnSite then Maintenance;
        transition crewLeaves first Maintenance accept CrewDeparts then Observing;
        transition recovered first Maintenance accept LiftedAboard then Recovered;
    }
}
```

```
$ perl "$KIT/tools/sysml/sysml.pl" check buoymodes-fixed.sysml
sysml: 1 file(s), 0 error(s)
```

Then in context: the TODO line in `model/physical/stations/Stations.sysml` replaced by `exhibit state modes : BuoyModes;` inside MooredBuoy, and the state def and its five trigger attribute defs added to the package. The real checks on the whole drop-1 model afterwards:

```
$ perl "$KIT/tools/sysml/model.pl" lint
lint: 14 file(s), 0 error(s), 0 warning(s)
elements=148 errors=0 warnings=0
$ perl "$KIT/tools/sysml/sysml.pl" check model/physical/stations/Stations.sysml
sysml: 1 file(s), 0 error(s)
```

The lesson is not "the model failed". The draft still saved time: the five state names and the six transitions were right, and copying them was faster than reading the XMI. The *syntax* was wrong, the gate said so in one line, and a person who knew the v2 pattern fixed it. That is the realistic shape of AI help on `.sysml` today.

18.10.6 6. SysML v2 text: the EnergyBalance constraint

The converter left two TODO lines in `Analysis.sysml`, one of them carrying the v1 expression in English: `generation * sunHours >= load * 24`. A grounded prompt ("here is the constraint def, here is the expression, write the v2 constraint body; parameters are inputs") gets close. This draft (illustrative) shows the *other* classic mistake – invented library names that sound exactly right:

```
package EnergyDraft {
    constraint def EnergyBalance {
        in generation : ISQ::SolarPowerValue;
        in storage : ISQ::BatteryEnergyValue;
        in load : ISQ::PowerValue;
        in sunHours : ScalarValues::Real;
        generation * sunHours >= load * 24
    }
}
```

The grammar check passes it – the syntax is perfectly good:

```
$ perl "$KIT/tools/sysml/sysml.pl" check energy-invented.sysml
sysml: 1 file(s), 0 error(s)
```

This is why the pipeline has two gates and not one. Spliced into the model, lint catches what grammar cannot: the names resolve to nothing.

```
$ perl "$KIT/tools/sysml/model.pl" lint
model/analysis/Analysis.sysml:8:25: error: unresolved type 'ISQ::SolarPowerValue'
model/analysis/Analysis.sysml:9:22: error: unresolved type 'ISQ::BatteryEnergyValue'
lint: 14 file(s), 2 error(s), 0 warning(s)
elements=148 errors=2 warnings=0
```

There is a quieter lesson in the fix. `ISQ::PowerValue` and `ISQ::EnergyValue` exist, and swapping them in makes lint pass – but the rest of Tidewater uses the model's own attribute defs `Power` and `Energy` (package Types), and the PowerSystem's bind lines connect this constraint to parts typed with them. The right fix is the model's own types, which is a judgment no gate makes for you:

```
    constraint def EnergyBalance {
        doc /* Daily energy in must cover daily energy out, with the battery as the buffer. */
        in generation : Power;
        in storage : Energy;
        in load : Power;
        in sunHours : ScalarValues::Real;
        generation * sunHours >= load * 24
    }
```

Real result, in the model: `lint: 14 file(s), 0 error(s), 0 warning(s)` and the grammar check clean. Note too that the v1 expression mentions `sunHours`, which the v1 constraint block never declared as a parameter; the draft added it, correctly, and the reviewer has to notice that the model *added a parameter* – a design decision worth a line in the merge request, and a question for the prime's modeler.

18.10.7 7. SysML v2 text: fork and flow in ObserveCoast

The behavior package has four TODO lines in ObserveCoast: a fork node after the quality check, an input pin, an object flow and a control flow. A draft for the fork and the flow (illustrative), grounded with the action def's real text:

```
        fork afterCheck;
        first check then afterCheck;
        first afterCheck then store;
        first afterCheck then 'log health';
        flow publish.bulletin to bulletin;
```

Replacing the fork TODO, the `first check then store;` line, and the two edge TODOs with those lines, the real checks on the model:

```
$ perl "$KIT/tools/sysml/sysml.pl" check model/logical/Behavior.sysml
sysml: 1 file(s), 0 error(s)
$ perl "$KIT/tools/sysml/model.pl" lint
```

```
lint: 14 file(s), 0 error(s), 0 warning(s)
elements=148 errors=0 warnings=0
```

That draft passed first time – the construct is close to what the model has seen in other languages. The input pin TODO stays for a later task, and the helper from example 1 shows the effect of the three fixes together on a scratch copy of the model:

```
$ perl todo-by-package.pl model        # after BuoyModes, EnergyBalance, fork and flow
stations          28
datacenter         5
operations         5
network            3
types              3
behavior           1
tidewater          1
total             46
```

From 52 to 46. Passing the gates is necessary, not sufficient: the reviewer still asks whether the fork is where the v1 activity put it, whether `'log health'` should run in parallel with archiving, and whether the flow goes to the right parameter. Those are engineering questions, and they are why a person owns the commit.

18.10.8 8. The stand-up summary through the kit's AI hook

The kit can hand the day's stand-up data to an AI and put the answer in the status mail. `lib/Ai.pm` has three backends: `paste` (the default: it writes the prompt to a file, you paste it wherever you are allowed to), `mock` (tests), and `curl` (an OpenAI-style chat endpoint – which is what Ollama serves). In the journal project's `scrum.conf`:

```
ai_backend = curl
ai_url     = http://localhost:11434/v1/chat/completions
ai_model   = Babitdor/Qwen3-4B-SysMLv2
```

Then `perl bin/daily.pl ai` (Vim: `:SAi`) sends the day's digest – answers, flags, the sprint's numbers – and parses the reply into five fixed sections: HIGHLIGHTS, RISKS, STUCK, QUESTIONS FOR LEADS, PROGRESS. On a shared server, `ai_url` names the server instead of `localhost`.

Two cautions. A SysML-tuned model was tuned for SysML, not for status writing; a general model from the approved list will usually write better summaries, and `ai_model` picks which. And the numbers never come from the AI: the kit computes every metric from the journal, and the AI only words the summary around them. If its wording contradicts a number, the number wins and the wording is edited.

18.11 What the team took from it

Case study: Bob's first try

Bob's first prompt was "convert the Stations package to SysML v2". The answer was four screens of confident text that did not parse, used `block` throughout, and invented a package import. Bob's verdict, after the grammar check listed the errors, was the one most engineers reach: *"It's a fast typist that doesn't know the language."* The second prompt – one state def, with the XMI fragment and the surrounding text pasted in – produced the draft in example 5: wrong syntax, right content, fixed in five minutes. The third prompt asked for the helper script in example 1 and was right the first time. The team's ranking of uses came out of that afternoon, and it matched what this chapter recommends.

The ranking, for the record: helper scripts and pipeline glue first, explanations second, prose drafts third, `.sysml` drafts last – and every one of them through the same gates. A team that starts at the bottom of that list concludes AI is useless; a team that starts at the top concludes it is a useful junior analyst. Both are looking at the same tool.

Exercise 18.1: Write the approval request

Fill in the request checklist in Section 18.3, Approval comes first (p. 271) for your own organization: one runtime, one model you would choose from the families table, the laptops or server, the data rule. Find out which office approves model origin.

Exercise 18.2: Run a three-task bake-off

Pick three of the five bake-off tasks. Write the prompts down first. Run each with one approved model and without one. Fill the scoresheet: gate passes, tries, minutes to mergeable. Does the model beat the baseline on any task?

Exercise 18.3: Make the gates catch a hallucination

Take the invented-name EnergyBalance draft from Section 18.10.6, 6. SysML v2 text: the EnergyBalance constraint (p. 283), splice it into a scratch copy of the drop-1 conversion, and run `sysml.pl check` and `model.pl lint`. Which gate catches it, and why can the other one not? Fix it with the model's own types and run both again.

Exercise 18.4: The scratch-buffer loop in Vim

Using form 2 from Section 18.9, Using it from Git Bash and from Vim (p. 277), send a TODO line and its XMI fragment to a model (or, without one, to `cat` in place of `ollama run …` to practice the keystrokes), save the scratch buffer as a `.sysml` file, and press `\mx`. Time the whole loop.

Exercise 18.5: Check a draft for added facts

Ask a model (or a colleague playing one) to draft the TW-SYS-006 DR from the facts in example 3. Underline every fact in the draft that was not in the prompt. How many were there, and which would have embarrassed you in front of the prime?

18.12 Applying this to your project

- **Copy:** the rules box, word for word; the `ai::assisted` label; the bake-off scoresheet; the scratch-buffer loop; the order of uses (tooling, explanations, prose, then model text).
- **Adapt:** the model list to your organization's approved list and its origin policy; the hosting choice to what your program already runs (a shared GPU server, if one exists, is usually the better start); the bake-off tasks to five real tasks from your own backlog.
- **Decide locally:** whether AI-assisted work is allowed at all on your network and for your data's marking; who approves models; whether the team needs it yet – the honest answer after a bake-off may be "not yet", and that is a result.
- **Never adapt:** the gates. AI-drafted text gets exactly the checks hand-written text gets. If a team ever proposes relaxing a gate "because the AI usually gets that right", the bake-off sheet is the answer.

CHAPTER

19

CI/CD from zero

From what you know →

You know

the configuration audit and the peer-review checklist, run before a milestone.

→ Here it is called

continuous integration and continuous delivery (CI/CD).

→ What is new

the audit and the checklist run on every change, in minutes, and the products are rebuilt every time.

Survival kit

What you have

a GitLab project (Chapter 15, Setting up GitLab (p. 226)), git on your laptop (Chapter 12, Git from zero, for systems engineers (p. 171)), and a sense from Chapter 17, The DevSecOps tools that earn their keep (p. 255) that "the pipeline" is the most important tool in the box.

What you need

to understand what continuous integration and continuous delivery are, in words you already use; to know the parts of a pipeline by name; to read a job log, passing and failing; to run the pipeline on your own laptop before you push; and to explain a red pipeline to a manager. No programming. The next chapter, Chapter 20, The pipeline, job by job (p. 302), then walks through the team's real pipeline file line by line.

19.1 Tom's question

On the Wednesday of week one, Tom stood at the whiteboard with a marker and asked the question that the whole team had been too embarrassed to ask:

Everybody keeps saying CI/CD. I run configuration management for a living. Is this some-

thing I already do with a new name, or something new?

— Tom, sprint 1, day 3

The honest answer is: both. Continuous integration and continuous delivery are practices that systems engineering already has, done at a speed and a regularity that only a machine can manage. This chapter takes the two halves in turn, then the vocabulary, then the skill of reading what a pipeline tells you.

19.2 CI: continuous integration, in systems engineering words

Integration is a word systems engineers own. It is the moment the parts come together and you find out whether they fit. In the document era, integration of the *descriptions* of the system (the specification, the interface documents, the requirement matrix) happened at milestone reviews: every few months, someone gathered the latest version of each document, read them side by side, and wrote down where they disagreed. Between reviews, each document drifted on its own. The drift was found late, in a conference room, and fixed by hand.

Continuous integration means that every change is integrated into the shared description the same day it is made, and is checked automatically when it is. In the model's terms:

- every change is made on a short branch and proposed as a merge request against `develop`, the shared model;
- before anyone reviews it, a machine runs the same set of checks on the *combined* result (the change merged into the latest shared model), in minutes;
- if a check fails, the change does not merge until it is fixed;
- changes are small enough (one task, under a week of work, usually a day) that a failure points at a handful of lines, not a binder.

Compare that with two things Tom knew well:

- A **configuration audit** checks that the configuration item matches its documentation. Done by people, it happens at a few milestones a year. CI runs a configuration audit on every change.
- A **peer review checklist** asks the same questions of every document: are the requirements numbered, does every interface have two ends, is every term defined. Done by people, it depends on who has time. CI runs the checklist the same way every time.

So CI is the configuration audit and the peer review checklist, run by a machine on every change, before I sign.

— Tom, writing on the whiteboard

That sentence is the definition this book uses.

Note: What "integrate" does not mean here

Continuous integration of the model does not mean the hardware is integrated, or that the model is "complete". It means the shared description is always in a state that passed its checks. That is a smaller claim, and a much more useful one than the document era could

make: on any given day, the model on `develop` is consistent with itself, by the rules the team wrote down.

19.3 CD: continuous delivery

Delivery is also a word systems engineers own: the contract data requirement says what must be delivered, in what form, when. In the document era, producing a deliverable was a project: export the diagrams, regenerate the tables, assemble the document, check the cross-references, convert to PDF, route for signature. It took days, and so it happened only when someone demanded it.

Continuous delivery means the products of the model are produced by the pipeline from the model, never by hand, and are always ready to hand over. For the Tidewater team the products are:

- the generated documents: the part tree, the function tree, the traceability report;
- the diagrams the kit's view tools draw from the text;
- the requirement trace against DOORS;
- the model website (GitLab Pages, Chapter 17, The DevSecOps tools that earn their keep (p. 255));
- for each baseline, the release notes and the evidence that the baseline rebuilds identically.

None of these is ever edited by hand. If one is wrong, the model is wrong, and the fix is a change to the model.

The "D" is sometimes read as **deployment**: putting new software into service. For a model team there is almost nothing to deploy. The only deployment in this book is publishing the model website after each merge, and publishing a release when a baseline is tagged. When people at your organization say "CI/CD", they usually mean the whole idea; this book does too.

19.4 The anatomy of a pipeline

A **pipeline** is the set of automated checks and production steps that GitLab runs for one event (a push, a merge request, a tag, a schedule). It is described in one file at the top of the repository, `.gitlab-ci.yml`, written in a text format called YAML (indentation matters; Chapter 20, The pipeline, job by job (p. 302) explains the few rules you need).

Here is the vocabulary, each word with its Tidewater example.

Definitions: The parts of a pipeline

pipeline
: one run of the checks for one event. Example: the run GitLab starts when Bob opens a merge request that closes the Stations level-tag TODOs.

stage
: a group of jobs that run together; stages run in order, and a stage starts only if the previous one passed. Example: `check` (grammar, lint, outline, todo, secrets), then `trace` (trace, vcrm), then `docs`.

job
one check or one production step; it runs a short script and passes or fails. Example: `lint`, which runs the kit's model linter and fails unless it finds zero errors and zero warnings.

script
the commands a job runs, the same commands you would type in a terminal. Example: `perl ci/job.pl lint`, which runs the linter and the report writer (Chapter 21, Runners: where the pipeline runs (p. 347) explains why every job is one such line).

runner
the computer that runs the jobs. Example: a Windows machine with Git for Windows and the agile kit checked out, registered to the group by the enterprise administrators.

rules
when a job runs. Example: `delta` runs only on the branch that merges a prime drop; `baseline` only when a `baseline-*` tag is created.

artifact
a file a job keeps after it finishes, downloadable from the job page and passed to later jobs. Example: `lint.txt`, the linter's full output.

report
an artifact in a format GitLab understands and shows on the merge request page. Example: `lint.xml`, which fills the test widget.

variable
a named value the jobs can read. Example: `KIT`, the path of the kit on the runner.

cache
files kept between pipelines to save time. Example: none in this pipeline; the jobs are fast enough.

allow_failure
a job whose failure shows yellow and does not block the merge. Example: the first version of `trace`, because most trace gaps are the prime's to fix (it later became a ratchet instead, Chapter 20, The pipeline, job by job (p. 302)).

manual job
a job that waits for someone to press its play button. Example: none yet.

environment
a named place a job deploys to, with a history of deployments. Example: `model-site`, the model website.

A picture helps. The merge request pipeline for an ordinary model change looks like this in GitLab's pipeline graph, one column per stage:

```
check                 trace            docs
-----------------     -------------    --------
[ok] grammar      -->  [ok] trace    -->  [ok] docs
[ok] lint              [ok] vcrm
[ok] outline           [ok] recipe
[ok] todo
[ok] secrets
[ok] complexity
```

All six `check` jobs run at the same time (if the runner can run several at once; the team's runner

ran them one after another, which made no practical difference at these durations). The `trace` stage starts when all of them pass, and `docs` when the three trace-stage jobs pass. After a merge into `develop`, a fourth stage, `publish`, rebuilds the model site (Chapter 20, The pipeline, job by job (p. 302)).

19.5 What starts a pipeline

Table 19.1: What starts a pipeline

Event	GitLab's name for it	What it is for, here
Someone pushes commits to a branch with no open merge request	`push`	a quick check of a draft; also every merge into `develop` or `main`
A merge request is opened or its branch is pushed again	`merge_request_event`	the gate: the checks that must pass before the change may merge
A tag is created	a tag pipeline (`CI_COMMIT_TAG` is set)	the baseline audit and the release
A schedule fires	`schedule`	the nightly check and the weekly status export
Someone presses "Run pipeline"	`web`	a manual re-run, for example after the runner was repaired

Two refinements matter.

One pipeline per change, not two. When a branch has an open merge request, a push to it would normally start both a branch pipeline and a merge request pipeline. The team's file has a `workflow:` section that suppresses the branch pipeline in that case (Chapter 20, The pipeline, job by job (p. 302)). Fewer pipelines, less confusion about which result counts.

Schedules are cron. A schedule fires at times written as a cron expression (`0 2 * * 1-5` is 02:00 on weekdays), on a branch you choose, and runs the pipeline as the schedule's owner. The team has two: nightly checks and the weekly status export (Chapter 17, The DevSecOps tools that earn their keep (p. 255), Chapter 20, The pipeline, job by job (p. 302)).

Merged results. The merge request pipeline runs on the result of merging the change into the current `develop`, not on the branch alone. If Bob's branch was started on Monday and Tom merged a change to an interface on Tuesday, Bob's Wednesday pipeline checks Bob's change *together with* Tom's. That is the integration in continuous integration. GitLab calls this a merged results pipeline; the kit's emulator does the same.

19.6 Every job starts from nothing

This is the idea that surprises everyone the first time, and it is the source of most "it works on my machine" puzzles. Each job runs in a **fresh environment**: the runner gets a clean copy of the repository at the commit being checked, runs the job's script, keeps the artifacts the job declared, and throws everything else away. The next job, even in the same pipeline, starts again from a clean copy.

Consequences:

- A file one job writes is not there for the next job, unless the first job declares it as an **artifact**. Later stages download earlier stages' artifacts by default; a job can name the jobs it needs (`needs:` or `dependencies:`) to download only theirs.
- Anything that must be the same for every job (the path of the kit, for example) is a **variable**, not something a job sets up and the next one inherits.
- Something slow that does not change often (downloading tools, for software teams) can be kept in a **cache** between pipelines. A cache is a speed-up, never a correctness requirement: a job must still work if the cache is empty. The team's jobs take seconds and use no cache.
- Your laptop is *not* fresh: it has files you created, settings you forgot, tools nobody else has. That is why the local run below runs each job's script in a controlled way, and why a pipeline is a better witness than "it worked for me".

Table 19.2: Artifacts, cache and variables

Mechanism	Keeps what	Between	Use it for
Artifact	files a job declares	jobs of the same pipeline, and for download afterwards	outputs: reports, logs, the website
Cache	files a job declares as cache	pipelines	speed-ups that are safe to lose
Variable	a named value	every job	settings and secrets

19.7 Reading a pipeline

A pipeline answers three questions, in this order: did it pass, which job failed, and why. GitLab shows the first on the merge request (a green check mark or a red cross next to the pipeline), the second on the pipeline page (the graph above, with each job's status), and the third in the failed job's **log**: everything the job printed, line by line, with each command shown before its output.

19.7.1 A passing job, line by line

Here is the real log of the `lint` job on the team's import of drop 1, run on a laptop. Run it yourself in the sandbox of practice lesson 11 (`perl $KIT/agile.pl practice 11` builds it), which holds the same import; GitLab's web page shows the same lines with timestamps and a few lines of runner preamble:

```
. ../env.sh > /dev/null
perl ci/job.pl lint
```

```
$ perl ci/job.pl lint
$ /usr/bin/perl /opt/agile/tools/sysml/model.pl lint
lint: 17 file(s), 0 error(s), 0 warning(s)
elements=148 errors=0 warnings=0
$ /usr/bin/perl /opt/agile/tools/gitlab/ci-reports.pl lint --junit lint.xml --codequality
lint-cq.json --metrics lint.metrics lint.txt
ci-reports lint: 1 check(s), 0 failed, 0 finding(s); model_elements=148 lint_errors=0
lint_warnings=0
```

(The path of the kit, `/opt/agile` here, is wherever the runner's `KIT` variable points.) Read it as a transcript:

- Lines starting with `$` are commands; every other line is what the previous command printed.
- The first line is the job's whole script, `perl ci/job.pl lint`. Everything after it was printed by that script, which shows each command it runs before running it (Chapter 21, Runners: where the pipeline runs (p. 347) explains why every job is one such line).
- Before running anything, the script checks that the `KIT` variable is set and points at the kit. If not, the job stops with a sentence that says what to fix: `KIT is not set: add it under Settings > CI/CD > Variables (the kit's path on the runner)`.
- The linter checked 17 files and 148 model elements and found nothing wrong.
- The report writer turned that output into the files GitLab shows on the merge request (Chapter 17, The DevSecOps tools that earn their keep (p. 255)).
- Then the script applies the gate: the linter's last line must end in `errors=0 warnings=0`. It printed nothing more, because nothing was wrong. Silence after the last command is success.

19.7.2 A failing job, line by line

On a scratch branch, Bob typed the part type `Hull` as `Hul` and ran the same job:

```
git switch -q -c scratch/hul
sed -i 's/part hull : Hull;/part hull : Hul;/' model/physical/stations/Stations.sysml
perl ci/job.pl lint || echo "exit status $?"
```

```
$ perl ci/job.pl lint
$ /usr/bin/perl /opt/agile/tools/sysml/model.pl lint
model/physical/stations/Stations.sysml:38:21: error: unresolved type 'Hul'
lint: 17 file(s), 1 error(s), 0 warning(s)
elements=148 errors=1 warnings=0
$ /usr/bin/perl /opt/agile/tools/gitlab/ci-reports.pl lint --junit lint.xml --codequality
lint-cq.json --metrics lint.metrics lint.txt
ci-reports lint: 2 check(s), 2 failed, 1 finding(s); model_elements=148 lint_errors=1
lint_warnings=0
lint is not 0/0 (elements=148 errors=1 warnings=0)
```

The line that matters is the third: `model/physical/stations/Stations.sysml:38:21: error: unresolved type 'Hul'`. Every checker in the kit writes problems in the same shape: **file, line, column, severity, message**. Line 38, column 21, of the Stations file names a type that does not exist. The last line, `lint is not 0/0`, is the gate saying why the job failed. Everything between is supporting detail.

Tip: The habit

When a pipeline is red: open the failed job, search the log for `error:`, read the first one, go to that file and line. Fix the first error first; later errors are often caused by it.

19.7.3 When the line number points one line too late

The grammar checker reads the text the way a parser does: token by token, until something cannot follow what came before. A missing semicolon is therefore reported where the parser noticed, which is the *next* line:

```
        part hull : Hull
        part mooring : Mooring;
```

```
git checkout -q -- .
sed -i 's/part hull : Hull;/part hull : Hull/' model/physical/stations/Stations.sysml
perl $KIT/tools/sysml/sysml.pl check model/physical/stations/Stations.sysml || echo "exit status
$?"
```

```
model/physical/stations/Stations.sysml:39:9: error: unexpected 'part'; expected ',', '.', ':',
'::', '::>', ':=', ':>', ':>>', ...
sysml: 1 file(s), 1 error(s)
```

Line 39 is the `part mooring` line, which is perfectly fine; the mistake is the end of line 38. When a grammar error seems to point at an innocent line, look at the end of the line above. The list after "expected" is what the parser would have accepted instead; a semicolon is not in the visible part of the list, which is the parser's way of being unhelpful. Bob kept a sticky note on the monitor: *grammar error: look one line up*.

19.7.4 From the log to the line, in Vim

Reading line numbers off a web page and typing them into the editor is the document era's page-number hunt. Vim does it for you. The kit sets up Vim's **quickfix** list for SysML: inside the model repository, open any `.sysml` file and type

```
:compiler sysml
:make lint
```

Here is the same thing run on the scratch branch with the `Hul` mistake, ending with `:clist`, which prints the list:

```
:!git checkout -q -- . && sed -i 's/part hull : Hull;/part hull : Hul;/'
model/physical/stations/Stations.sysml
:edit model/physical/stations/Stations.sysml
:compiler sysml
:silent make lint
:redir! >> /dev/stdout
:clist
:redir END
```

Vim runs the same linter the pipeline runs, reads its `file:line:col: error: message` lines, and jumps to the first problem. `:cnext` goes to the next one, `:copen` shows the list. `:make check` does the same for the outline check. Outside a model repository, `:make` checks the current file's grammar. Chapter 10, Vim for models (p. 143) covers the rest of Vim for models; the point here is that the pipeline and your editor speak the same format, so fixing a red pipeline is the same motion as fixing a typo.

19.8 Running it on your own machine first

A pipeline that fails on the server costs a few minutes and a red mark on the merge request. A pipeline that fails on your laptop before you push costs seconds and nobody sees it. The kit's `gitlab.pl ci` reads the same `.gitlab-ci.yml`, decides which jobs GitLab would run for a given event using the same rules, and runs them in the current repository.

First ask what would run. Throw the scratch branch away, then list:

```
git checkout -q -- . && git switch -q - && git branch -q -D scratch/hul
perl $KIT/bin/gitlab.pl ci --event merge_request_event --target develop --var KIT=$KIT --list
```

On Bob's branch with the drop 1 import, the answer was:

```
grammar                  check      on_success .gitlab-ci.yml  rules: rule 4 matched
lint                     check      on_success .gitlab-ci.yml  rules: rule 4 matched
outline                  check      on_success .gitlab-ci.yml  rules: rule 4 matched
todo                     check      on_success .gitlab-ci.yml  rules: rule 4 matched
secrets                  check      on_success .gitlab-ci.yml  rules: rule 4 matched
complexity               check      on_success .gitlab-ci.yml  rules: rule 4 matched
trace                    trace      on_success .gitlab-ci.yml  rules: rule 4 matched
vcrm                     trace      on_success .gitlab-ci.yml  rules: rule 4 matched
recipe                   trace      on_success .gitlab-ci.yml  rules: rule 4 matched
docs                     docs       on_success .gitlab-ci.yml  rules: rule 4 matched
include: (conditional: rules), template: Security/Secret-Detection.gitlab-ci.yml -
skipped    -               include: runs on GitLab only
delta                    -          skipped    .gitlab-ci.yml  rules: no rule matched
drop-package             -          skipped    .gitlab-ci.yml  rules: no rule matched
intake                   -          skipped    .gitlab-ci.yml  rules: no rule matched
baseline                 -          skipped    .gitlab-ci.yml  rules: no rule matched
publish-package          -          skipped    .gitlab-ci.yml  rules: no rule matched
publish-release          -          skipped    .gitlab-ci.yml  rules: no rule matched
sil-regression           -          skipped    .gitlab-ci.yml  rules: no rule matched
hil-test                 -          skipped    .gitlab-ci.yml  rules: no rule matched
pages                    -          skipped    .gitlab-ci.yml  rules: no rule matched
status                   -          skipped    .gitlab-ci.yml  rules: no rule matched
pipeline: 10 jobs
```

Ten jobs would run; the others are for other events (a drop branch, a delivery from the prime, a tag, the default branch, the schedule, and two simulation jobs that wait for a test harness the program does not have yet), and the list says which rule excluded each. The third column names the file each job came from, which matters once a pipeline is split into several files with `include:`

`local` (Chapter 20, The pipeline, job by job (p. 302)). GitLab's own template for secret scanning is listed but not run: it only exists on the server, and only with a Docker runner (Chapter 21, Runners: where the pipeline runs (p. 347)).

Then run them, dropping `--list`. Each job's log goes to a folder you name with `--logs`, and a failing job's last lines are printed under it:

```
perl $KIT/bin/gitlab.pl ci --event merge_request_event --target develop --var KIT=$KIT --logs
../ci-logs
```

Chapter 20, The pipeline, job by job (p. 302) shows the full run, including how the `outline` job lives with the violations the raw drop 1 conversion inherited.

Note: The CI/CD editor on the server

GitLab has the same idea built in: Build > Pipeline editor. It checks the YAML as you type, shows the pipeline as a graph ("Visualize"), validates it against a chosen branch ("Validate"), and shows the "full configuration" with every `include:` and `extends:` expanded. Use it when you change the pipeline file. Use `gitlab.pl ci` when you change the model, because it actually runs the checks.

19.9 Gates

A **gate** is a check that must pass before work moves on: a milestone entry criterion, enforced by software on every change. A pipeline job that blocks the merge when it fails is a gate (lint's zero over zero); a job that is allowed to fail is a warning light, not a gate (the first version of `trace`); a ratchet is a gate on change, not on state (`outline`, and `trace` later; Chapter 20, The pipeline, job by job (p. 302)). Project settings add gates of their own: the pipeline must succeed, a code owner must approve, discussions must be resolved. Deciding which checks are gates and which are warnings is one of the most consequential decisions in a pipeline, and it is a systems engineering decision, not a software one: block on what must never reach the shared model, warn on what someone else must fix. Most gates cost nothing; some settings need a paid GitLab tier, and hosted runners charge compute minutes (Section 17.7, Gates, and what they cost (p. 263)).

19.10 Shift left: catch it at the merge, not at integration

"Shift left" is DevSecOps jargon for a systems engineering truism: a defect found earlier is cheaper to fix. Draw the life cycle as a line from left (idea) to right (operations); every check you move to the left finds problems when they cost less.

For a model, the leftmost possible check is the merge request. An interface whose two ends disagree (a port typed one way on the stations side and another on the data center side) costs a few minutes to fix when the pipeline flags it on the merge request that introduced it. Found at an integration review three months later, it costs a meeting, a discrepancy report, two document revisions and the review's credibility. The status metrics in `docs/STATUS-METRICS.html` count exactly this, as metric H: interface mismatches caught at merge versus those that reached

integration. For Tidewater the second number stayed at zero; Chapter 33, The twelve-week scorecard (p. 520) has the counts.

The pipeline is the team's safety net, not its judge. Three habits make that true:

- **Never merge red.** The project setting enforces it, but the habit matters more: a red pipeline on `develop` means everybody's next merge request inherits the failure.
- **Fix forward, quickly.** If something red reaches `develop` (it happened once, in sprint 6, when the runner's disk filled up, Chapter 31, Sprint 6: Behavior by hand, a table for the program, and baseline-0 (p. 495)), the first priority of whoever is free is to make it green again. The scorecard measures the time from the first red to the next green.
- **Fail fast.** Cheap checks (grammar, lint) run first, so a typo fails in seconds instead of after the slow jobs.

19.11 The Sec in DevSecOps

Security in a pipeline is mostly unglamorous, and it is cheap precisely because it is automated:

- **Secrets never in the repository.** The `secrets` job looks for tokens and keys in every change (Chapter 17, The DevSecOps tools that earn their keep (p. 255)).
- **Secrets in protected, masked variables.** The weekly export's token is given only to pipelines on protected branches, and never printed in a log.
- **A least-privilege runner.** The runner's account can read the kit and write its own work folder; it cannot reach anything but the GitLab server, and it holds no credentials of its own beyond its registration (Chapter 42, Administration (p. 586)).
- **Protected branches and tags.** Nobody, including the team, changes the shared model or a baseline outside a reviewed, checked merge request.
- **Verifiable baselines.** Each baseline tag records a checksum of the generated documents; the `baseline` job proves the tagged model rebuilds them identically. Tampering with a baseline would require changing a protected tag, which nobody can do.

None of these needed a security specialist. Each is one setting or one job, and each answers a question the ISSM asks of every system (Chapter 17, The DevSecOps tools that earn their keep (p. 255) lists the questions).

19.12 Growing a pipeline one job at a time

The team's pipeline did not start with fifteen jobs. It grew the way Tom's rule from Chapter 17, The DevSecOps tools that earn their keep (p. 255) says tools should: one job each time a problem repeated.

Table 19.3: How the pipeline grew

Sprint	Job added	The problem it answered
1	grammar, lint, outline	"we can't tell whether a change broke the model"
2	todo, docs, report widgets	"how much is left?", "the generated docs are stale", "I can't find the error in the log"

Sprint	Job added	The problem it answered
3	secrets, trace	“the ISSM asked about credentials”, “which requirements are unlinked today?”
4	pages, schedules	“management wants to see the model”, “something broke overnight and nobody noticed”
5	recipe, the trace ratchet, drop-package, baseline, publish-release	“the DOORS text is vague”, “a trace gap slipped in”, “where is the exact delivery?”, “prove the baseline”
6	vcrm, delta (rehearsed for drop 2)	“which requirements are verified, and how?”, “what did the prime change that the converter can’t show?”
later	complexity, intake, publish-package, the SIL/HIL templates	“is the spider web growing back?”, “a drop arrived from the prime’s Jenkins”, “give other teams the baseline”

Each job has a cost: runner time, a log someone must read when it fails, false alarms. The team reviewed the pipeline at each retrospective with two questions: which job caught something real this sprint, and which job failed for a reason that was not a real problem? A job that never catches anything and sometimes cries wolf is a candidate for removal.

19.13 The minimum pipeline for any SysML v2 text repository

If you take one thing from this chapter to your own project, take this. It needs a runner with Perl, the agile kit checked out at the path in the `KIT` variable, and the kit’s job script copied into the repository as `ci/job.pl` (from `tools/gitlab/templates/ci/job.pl`). Each script line is a single `perl` command, so the same file works on a Windows runner whose shell is PowerShell, in Git Bash and in a Docker image (Chapter 21, Runners: where the pipeline runs (p. 347) tells how the team learned that the hard way):

```
stages:
  - check

grammar:
  stage: check
  script:
    - perl ci/job.pl grammar

lint:
  stage: check
  script:
    - perl ci/job.pl lint
```

Try it on the lesson’s model, as if it were your project’s only pipeline file:

```
cat > ../minimum.yml <<'EOF2'
stages:
```

```
  - check

grammar:
  stage: check
  script:
    - perl ci/job.pl grammar

lint:
  stage: check
  script:
    - perl ci/job.pl lint
EOF2
perl $KIT/bin/gitlab.pl ci --file ../minimum.yml --event push --branch develop --var KIT=$KIT
```

Two jobs: every file is valid SysML v2, and every name in the model resolves. With "pipelines must succeed" switched on, that alone makes the model's integration branch trustworthy in a way no document set ever was. Everything else in Chapter 20, The pipeline, job by job (p. 302) is added on top, one problem at a time.

One subtlety. Jobs with no `rules:` run in *branch* pipelines (every push), not in merge request pipelines: ask `gitlab.pl ci --event merge_request_event --list` and it answers "not added to merge request pipelines" for both. That is GitLab's default, and for the minimum pipeline it is fine: the branch pipeline on the merge request's source branch is the one GitLab checks before allowing the merge. The team's full pipeline uses `rules:` and a `workflow:` section so that the gate is a merged results pipeline instead (Chapter 20, The pipeline, job by job (p. 302)).

19.14 Explaining a red pipeline to a manager

Sooner or later the program manager will see a red mark and ask what it means. The answer the team gave, which worked:

> *A red pipeline means the checks caught a problem in a proposed change before it reached the shared model. That is the system working. The change will not merge until it is fixed, usually the same day. A red pipeline on the shared model itself would be a problem, and we measure how long it lasts: so far, once, for less than a day.*
>
> — Jim, to the deputy program manager, sprint 4 review

19.15 Exercises

Exercise 19.1: Break it and watch it fail

In your practice repository, create a branch, change one part's type to a name that does not exist, and run `gitlab.pl ci --event merge_request_event --target develop --var KIT=$KIT --job lint`. Find the error line in the log. Then fix it with `:make lint` in Vim and run the job again.

Exercise 19.2: One line up

Delete a semicolon at the end of a `part` line and run the grammar checker on that file. Which line does the error name? Explain the difference to a colleague in one sentence.

Exercise 19.3: Add a job

Add a job to the minimum pipeline that counts `// TODO` lines and prints the count, and never fails. Run it locally. What would you need to add so the count appears on the merge request page? (Hint: Chapter 17, The DevSecOps tools that earn their keep (p. 255).)

Exercise 19.4: Allowed to fail

Make the `lint` job of the minimum pipeline `allow_failure: true` and break the model again. What does the pipeline status become? Write two sentences on why this would be a bad idea for `lint`, and two on why the team first made `trace` allowed to fail and later replaced that with a ratchet.

Exercise 19.5: The manager's question

Write the three sentences you would say to your program manager about a red pipeline on a merge request, and the three you would say about a red pipeline on the integration branch.

19.16 Applying this to your project

- **Copy**: the definitions (CI = the configuration audit and peer review checklist on every change; CD = every product generated from the model, always ready); never merge red; run the pipeline locally before pushing; the two-job minimum pipeline on day one.
- **Adapt**: the jobs to your checks (your organization may have its own modeling rules; each one that a program can check is a job); the triggers (a slower project may need only merge request and tag pipelines).
- **Decide locally**: where the runner lives and who maintains it; which checks block a merge and which only warn; how you will explain red and green to your management, before the first red appears.

CHAPTER

20

The pipeline, job by job

From what you know →

You know

the checklist a reviewer works through before signing off.

→ Here it is called

the pipeline: jobs such as grammar, lint, trace and docs.

→ What is new

each checklist item is a script, and a red job stops the change.

Survival kit

What you have

the idea of a pipeline (Chapter 19, CI/CD from zero (p. 288)): stages, jobs, rules, artifacts, reports, a runner.

What you need

to be able to read, change and defend every line of the team's real pipeline file. This chapter takes `.gitlab-ci.yml` from the top, explains the few YAML rules you need, then goes job by job: what each job checks, the systems engineering question it answers, when it runs, what it keeps, what its output looks like on Tidewater's drop 1, and how it fails. The complete file is reproduced at the end.

20.1 Reading YAML in ten minutes

The pipeline file is written in **YAML**, a text format for nested lists and named values. You need six rules.

1. **Indentation is structure.** Two spaces deeper means "inside". Never use a tab in a YAML file; in Vim, `:setlocal expandtab` makes the Tab key insert spaces.
2. **`name: value` is a named value.** `stage: check` gives the job's `stage` the value `check`.

3. **A line starting with `-` is a list item.** Under `script:`, each `-` line is one command.
4. **A `#` starts a comment** (to the end of the line). The team's file is about half comments, on purpose.
5. **Quote a value that contains `:` (colon, space).** Otherwise YAML reads it as another `name: value`. The kit's local runner refuses such lines with a message that says to quote them; GitLab refuses them too.
6. **`&name` marks a block to reuse; `*name` reuses it; `<<: *name` copies its contents in.** The file uses this once, for the rules every model job shares.

That is all. There are no loops and no variables in YAML itself; anything that needs logic lives in a script the job runs.

20.2 The shape of the file

The file has two kinds of top-level entries: a few reserved names that configure the whole pipeline (`stages`, `variables`, `workflow`, `include`, `default`), and everything else, which is a job. A name that starts with a dot (`.check`) is a **hidden job**: a template other jobs copy, never run on its own.

The team settled on one principle after a lesson described in Chapter 21, Runners: where the pipeline runs (p. 347): **the YAML says when and where; the script says what.** Every job's `script:` is a single line,

```
  script:
    - perl ci/job.pl lint
```

and the commands that job runs live in `ci/job.pl`, a short Perl script kept in the repository next to the pipeline file. The reason is portability: the Windows runner's default shell is PowerShell, where the shell commands in a typical pipeline (`if [ … ]`, `$(find …)`, `$KIT`) do not work, while `perl ci/job.pl lint` means the same thing in PowerShell, in Git Bash, on Linux and in a Docker container. A second benefit is that a person can run any job by hand with the same line, and see exactly what the pipeline sees.

So when this chapter says "the job runs the linter", it means: the YAML entry for `lint` says when the job runs and what files it keeps; `ci/job.pl lint` runs the linter, writes the report files and decides pass or fail. The log of each job starts with the `perl ci/job.pl NAME` line and then shows the commands `job.pl` runs and their output, in the same transcript form as Chapter 19, CI/CD from zero (p. 288).

20.3 The header: stages, variables, workflow

```
stages:
  - check
  - trace
  - docs
  - delta
  - release
  - publish
```

```
  - report

variables:
  GIT_DEPTH: "0"
```

The seven stages run in that order. `check` is the gate; `trace` reports requirement gaps; `docs` checks that the generated documents are current; `delta` runs only when a prime drop arrives; `release` only on baseline tags; `publish` only on the default branch; `report` only on the weekly schedule.

`GIT_DEPTH: "0"` asks the runner for the full history of the repository. By default GitLab fetches only the most recent commits to save time. The team's jobs read history: `trace` and `delta` compare imported drops, the burn-down counts TODO lines across merges, the baseline check reads the tag message. A shallow copy would make them wrong in confusing ways.

```
workflow:
  rules:
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'
    - if: '$CI_COMMIT_BRANCH && $CI_OPEN_MERGE_REQUESTS && $CI_PIPELINE_SOURCE == "push"'
      when: never
    - if: '$CI_COMMIT_BRANCH'
    - if: '$CI_COMMIT_TAG'
    - if: '$CI_PIPELINE_SOURCE == "schedule"'
```

`workflow: rules:` decides whether a pipeline starts at all. GitLab reads the rules top to bottom and stops at the first that matches:

1. a merge request event: yes, run (the gate);
2. a push to a branch that already has an open merge request: no (the merge request pipeline covers it; without this rule every push would produce two pipelines);
3. any other push to a branch (`develop`, `main`, `vendor/prime`, a draft branch with no MR yet): yes;
4. a tag: yes;
5. a schedule: yes.

The `$CI_…` names are **predefined variables** GitLab sets for every pipeline: `CI_PIPELINE_SOURCE` says what started it, `CI_COMMIT_BRANCH` is the branch (empty for tags and merge requests), `CI_COMMIT_TAG` the tag, `CI_OPEN_MERGE_REQUESTS` lists open merge requests for the branch. A bare `$NAME` in a rule is true when the variable is set and not empty.

20.4 The conditional include and the defaults

```
include:
  - template: Security/Secret-Detection.gitlab-ci.yml
    rules:
      - if: '$DOCKER_RUNNER == "true"'
```

`include:` pulls in YAML from elsewhere, here one of GitLab's built-in templates: its secret scanner. The scanner runs in a Docker container, so it can only work on a runner that has Docker. The

team's runner does not (Chapter 21, Runners: where the pipeline runs (p. 347) explains why), and a job waiting for a runner that never comes stays "pending" forever, which with "pipelines must succeed" would block every merge request. So the include is conditional on a project variable, `DOCKER_RUNNER`, set to `false` in `project.conf`. The pipeline's own `secrets` job (below) is the always-on check. The kit's local runner lists the include as "runs on GitLab only".

```
default:
  tags:
    - windows
    - mbse
```

`default:` sets things every job inherits. The team's default is the runner tags: every job asks for a runner tagged `windows` and `mbse`, which is the team's Windows shell runner (Chapter 21, Runners: where the pipeline runs (p. 347)). On a Docker runner these tags change, and nothing else does. The check that `KIT` is set and points at the kit lives in `ci/job.pl`, so a misconfigured runner fails with a sentence that says what to fix rather than with a "file not found" deep in a job.

```
.model_rules: &model_rules
  rules:
    - if: '$CI_PIPELINE_SOURCE == "schedule" && $FULL != "1"'
      when: never
    - if: '$CI_PIPELINE_SOURCE == "trigger"'
      when: never
    - if: '$CI_COMMIT_BRANCH == "vendor/prime"'
      when: never
    - when: on_success

.check:
  stage: check
  <<: *model_rules
```

`.model_rules` is the rule shared by every job that checks the model, read top to bottom:

1. never on a schedule unless that schedule sets the variable `FULL` to `1` (the nightly schedule does, and runs every check; the weekly status schedule does not, so it only exports reports);
2. never in a pipeline the prime's delivery system starts (a *trigger*, Chapter 45, The prime's model changes: receive, assess, disposition (p. 619)): that pipeline only takes in the files;
3. never on `vendor/prime`, the branch that holds each drop exactly as delivered and converted. The checks run where the drop is *merged*, on its merge request into `develop`; on the raw branch they would stay red for structure only the prime can change;
4. otherwise, always.

`&model_rules` names the block; `<<: *model_rules` copies it into `.check`, the hidden job the six check-stage jobs `extend`. Writing the rule once means changing it once.

All the job commands in this chapter can be typed in the sandbox of practice lesson 11, which holds the drop 1 import in the team's layout (`perl $KIT/agile.pl practice 11` builds it):

```
. ../env.sh > /dev/null
echo "KIT is $KIT"
```

20.5 Stage check: the gate

The six `check` jobs are the configuration audit. Each must pass (except where noted) for a merge request to merge.

20.5.1 grammar: is every file valid SysML v2?

```
grammar:
  extends: .check
  script:
    - perl ci/job.pl grammar
  artifacts:
    when: always
    paths: [grammar.txt]
    reports:
      junit: grammar.xml
      codequality: grammar-cq.json
```

The question it answers: is every `.sysml` file in `model/` a valid sentence in the SysML v2 language? The document-era equivalent is the editor's first read: is the thing even well-formed?

What it runs: the kit's grammar checker over every model file, which is built from the official SysML v2 grammar (Chapter 6, SysML v2 for SysML v1 modelers (p. 57)), then `ci-reports.pl grammar` to write one JUnit test case per file and a Code Quality finding per error.

```
perl ci/job.pl grammar
```

On drop 1: `sysml: 17 file(s), 0 error(s)`, in about two seconds. The converter's output is always grammatical; this job earns its keep on hand edits, where a missing semicolon is the classic mistake (Chapter 19, CI/CD from zero (p. 288) shows the "one line up" error).

`artifacts: when: always` keeps the reports even when the job fails, which is exactly when you need them.

20.5.2 lint: does every name resolve?

```
lint:
  extends: .check
  script:
    - perl ci/job.pl lint
  artifacts:
    when: always
    paths: [lint.txt]
    reports:
      junit: lint.xml
```

```
      codequality: lint-cq.json
      metrics: lint.metrics
```

The question it answers: is the model internally consistent? Does every part's type exist, does every import point at a real package, is anything declared twice? The document-era equivalent is the cross-reference check: does every "see section 3.2.1" point at a section that exists?

What it runs: `model.pl lint`, which ends with one summary line, then the report writer, then the gate: the last line must end `errors=0 warnings=0`. The team called it "zero over zero to merge", and it is the single most important rule in the file.

On drop 1:

```
perl ci/job.pl lint
```

```
lint: 17 file(s), 0 error(s), 0 warning(s)
elements=148 errors=0 warnings=0
```

`elements=148` counts the model elements the linter understood. Chapter 19, CI/CD from zero (p. 288) shows the failing case (`unresolved type 'Hul'`), and Chapter 17, The DevSecOps tools that earn their keep (p. 255) shows the JUnit and Code Quality files it produced.

20.5.3 outline: is the model's structure readable?

The question it answers: does every level of the part tree have between 2 and 9 children, and does every part definition carry the level tag (`@L1Network` down to `@L6Component`) that says where it sits? This is the kit's version of a rule every systems engineer knows from specification trees: fewer than two children is not a breakdown, more than nine is not readable, and every item knows its level.

What it runs: `model.pl check`, which reports level-tag mismatches and outline violations. On the drop 1 import, in the team's layered layout (Chapter 16, The model repository: what goes where (p. 245)):

```
perl $KIT/tools/sysml/model.pl check | head -14
```

```
== Level tags ==
@L tag mismatches         0

== Outline (2..9 children per parent) ==
violations                20
    1  directory entries  (model/analysis/)
    1  package Analysis: members  (model/analysis/Analysis.sysml)
    1  directory entries  (model/legacy/)
    1  part def Battery_old: parts  (model/legacy/Legacy_Import.sysml)
    1  directory entries  (model/library/conventions/)
    1  directory entries  (model/logical/)
```

```
  1  directory entries  (model/physical/allocation/)
 25  package Stations: members  (model/physical/stations/Stations.sysml)
  1  part def Battery: parts  (model/physical/stations/Stations.sysml)
 32  package Requirements: members  (model/requirements/model/Requirements.sysml)
```

(an excerpt; there are 20 lines). Each line is a count and the place it was counted: the Stations package has 25 members, the requirements copy 32, `Battery` one part, and many layer folders hold a single file.

Two kinds of violation are mixed here. The folder lines are an artifact of the layout: a layer with one file in it today (`model/logical/` holds only Behavior) will fill as the model grows, or the team can decide one file is right for it. The package lines are about the model itself: the 25-member Stations package, the one-part `Battery`, the duplicate in the legacy package. The first kind is cheap to accept; the second is the prime's structure, which the government should not silently rearrange.

The level tags show zero mismatches only because there are no level tags yet: the converter turned every v1 stereotype into a TODO line. They appear as the team closes those TODOs (Chapter 28, Sprint 3: the first red X, DOORS in the model, and a wait nobody named (p. 440), Chapter 29, Sprint 4: the model site, a cycle broken, and the demo that landed (p. 459)).

20.5.4 The ratchet: why outline does not fail on day one

Twenty violations on the first day posed a real dilemma, and the team's resolution is one of the most transferable ideas in this book.

Option one: make the job fail on any violation. Then every merge request fails until all 20 are fixed, including the ones only the prime can fix (a 25-member package is the prime's structure; the government's copy should not silently diverge from it). Nobody could merge anything. The gate would be switched off within a week.

Option two: make the job allowed to fail. Then it is yellow forever, everybody learns to ignore it, and new violations creep in unnoticed.

Option three, which the team chose: a **ratchet**. A file in the repository, `.ci/outline-max`, holds the number of violations currently allowed: 20 on the day drop 1 was imported into the lesson's copy (the team's own count, with its own conventions file, is in Chapter 27, Sprint 2: Drop 1 on develop, every change gated (p. 424)). The job fails if a change *raises* the count above that number, and tells the author to *lower* the number if the change fixed some. The number can only go down, one reviewed merge request at a time, until it reaches zero and the rule is absolute.

```
cat .ci/outline-max
perl ci/job.pl outline | tail -1
```

```
20
outline violations: 20 (allowed: 20)
```

A ratchet turns an inherited problem into a measured, shrinking one without blocking the work, and it works for any rule you cannot satisfy on day one: unverified requirements, missing documentation, lint warnings in legacy content. The violations that only the prime can fix become

discrepancy reports (Chapter 29, Sprint 4: the model site, a cycle broken, and the demo that landed (p. 459)); the ones the government can fix become tasks.

Tip: Lowering the ratchet is part of the change

When your merge request fixes outline violations, the job says `fewer violations than allowed: lower .ci/outline-max to N in this MR`. Do it in the same merge request. The reviewer checks that the number went down; Tom's rule was that nobody ever raises it without a written reason in the merge request and Jim's approval.

20.5.5 todo: the port burn-down

The question it answers: how much conversion debt is left? Every `// TODO` line the converter wrote is a v1 construct it could not carry into SysML v2 (Chapter 6, SysML v2 for SysML v1 modelers (p. 57)), and closing them is the port.

What it runs: a search for `// TODO` in `model/`, a count, and the report writer, which emits each TODO line as an informational Code Quality finding (so merge requests show the TODOs they closed as "fixed") and a metrics file. It never fails: the burn-down is information, not a gate.

On drop 1:

```
perl ci/job.pl todo
```

```
TODO lines remaining: 52
ci-reports todo: 1 check(s), 0 failed, 52 finding(s); todo_lines=52 todo_lines_analysis=2
todo_lines_library=4 todo_lines_logical=4 todo_lines_physical=42
```

Fifty-two lines, forty-two of them in the physical layer and twenty-nine of those in Stations (`grep -c "// TODO" model/physical/stations/Stations.sysml`). That number is the first row of the scorecard (Chapter 3, The directive (p. 22)) and the spine of the sprint plans in Part III.

20.5.6 secrets: no credentials in the repository

The question it answers: did this change commit a token, a cloud key or a private key?

What it runs: a search of every tracked file for three patterns: GitLab personal tokens (`glpat-` followed by twenty characters), cloud access keys, and the header line of a private key file. If it finds one, it fails with the instruction "remove it, rotate it, then rewrite the branch before it merges". The order matters: a committed secret must be revoked first, because it is already in the history of the branch and in every clone of it.

The pattern list is short on purpose. It catches the common accidents with almost no false alarms. GitLab's own scanner, when a Docker runner is available, catches many more.

```
perl ci/job.pl secrets
```

20.5.7 complexity: is the spider web growing back?

The question it answers: are the packages getting more tangled? The v1 model became a spider web one unreviewed link at a time (Chapter 2, The Cameo years (p. 10)); this job measures the web on every change so it cannot grow back unnoticed (Chapter 47, Keeping SysML v2 from becoming a spider web (p. 645)).

What it runs: for every package, its **fan-in** (how many other packages depend on it) and **fan-out** (how many it depends on), the package dependency **cycles**, any dependency that goes up the layer order set in `sysml.conf`, and the number of relationship ends on each part definition. On a merge request it is a ratchet like `outline`: any new cycle or upward dependency fails, and fan-out or relationship ends fail when they get worse than on the target branch *and* are above the limits in `sysml.conf` (7 and 10 by default), so a new project's first parts and links pass; elsewhere it reports.

```
perl ci/job.pl complexity | grep -A8 "^cycles:"
```

```
cycles: DataCenter <-> Stations; Network <-> Operations
upward-layer dependencies: none

relationship ends per part def (top 5):
  ObservingStation           19
  ObservationNetwork         8
  BulletinPublisher          6
  IngestService              5
```

The cycles are the spider web's, inherited from the prime's model; `ObservingStation`, with its trace spaghetti, tops the relationship list. (This sandbox shows two tangled pairs; on the real project the job counted 2 tangled groups after the sprint 3 DOORS import.) Breaking the cycles is engineering work, not pipeline work: the team broke the Network and Operations loop in sprint 4 (Chapter 29, Sprint 4: the model site, a cycle broken, and the demo that landed (p. 459)), and the Stations and DataCenter loop was still open at the end of sprint 6 (1 group left).

20.6 Stage trace: DOORS against the model

```
trace:
  stage: trace
  <<: *model_rules
  script:
    - perl ci/job.pl trace
  artifacts:
    when: always
    paths: [trace.txt]
    reports:
      junit: trace.xml
      metrics: trace.metrics
```

The question it answers: for the requirements in the newest imported DOORS export, which are linked to the model (metric C), which have nothing satisfying them (orphans, F), which

have nothing verifying them (G), what share is complete (I), and which have different text in DOORS and in the model (drift)? In the document era this was two days of spreadsheet work before each review (Chapter 1, Thirty years of documents (p. 2)). Here it takes a fraction of a second, on every change.

What it runs: `reqif2sysml.pl --report --model` against the latest `imports/*/tidewater.reqif`, then the report writer (one JUnit case per gap, so the merge request's test widget lists them). The model side of the comparison leaves out the requirement packages the job's own DOORS import generated (they carry a header saying so); otherwise DOORS would be compared with a copy of itself.

On drop 1, against the converted model (before the DOORS modules themselves were imported, see the warning below):

```
C requirements linked (doorsId in the model)     29 of 30
F orphan requirements (nothing satisfies)               2
G unverified requirements (no verify)                   9
I trace completeness (satisfied and verified)        66.7%
doorsIds in the model but not in this export            1
text differs from DOORS                                 1
not in the model: TW-SYS-016
orphans: TW-SYS-012 TW-SYS-016
unverified: TW-STK-001 TW-STK-002 TW-STK-003 TW-STK-004 TW-STK-005 TW-STK-006 TW-SYS-011
TW-SYS-016 TW-SEG-002
drift: TW-SYS-006
  TW-SYS-006 DOORS: The data center shall publish a marine bulletin twice a day, on a fixed
  schedule.
  TW-SYS-006 model: The data center shall publish a marine bulletin on a fixed schedule.
```

The command took 0.17 seconds. Bob's reaction to the drift line is in Chapter 26, Sprint 1: A GitLab project the team can work in (p. 404).

```
perl ci/job.pl trace | tail -2
```

20.6.1 Gate or warning? The trace ratchet

Gaps like these posed the same dilemma as the outline violations, with a twist: most of them are the *prime's* to fix. TW-SYS-012 has nothing satisfying it because the prime's v1 model has no satisfy relationship for it; the government's copy should not invent one. Blocking the government's merge requests on the prime's gaps would punish the wrong people.

The first version of the job was therefore `allow_failure: true`: it failed when there were gaps, showed yellow, and blocked nothing. The team soon noticed the cost of a permanent yellow light: nobody looks at it, so a merge request that *removed* a satisfy relationship by mistake would also show yellow, exactly as before, and pass.

The job became a **ratchet**, like `outline`. On a merge request it traces the model twice: once as the change leaves it, once as it was where the branch left the target branch (the merge base). It fails only if the change *adds* a gap: an orphan, an unverified requirement or a drifted text that was not there before. On `develop` and on schedules it reports and passes. The end of its log on the practice repository:

```
trace gaps: 2 orphan(s), 9 unverified, 1 drifted
not a merge request: reported only
```

and, on a merge request that changes nothing in the trace, by `no new trace gaps`, and on one that makes things worse by `this change adds trace gaps:` … naming the measure and its before and after. The prime's gaps stay visible in the widget, with a discrepancy report each (Chapter 29, Sprint 4: the model site, a cycle broken, and the demo that landed (p. 459)); the government's own mistakes cannot get in.

Tip: Two kinds of ratchet

The outline ratchet keeps the allowed number in a file someone lowers by hand. The trace ratchet compares with the target branch automatically, so it needs no file. Use the first when the number should only ever go down by a deliberate decision; use the second when "no worse than before" is the whole rule.

20.6.2 vcrm: the verification cross-reference matrix

The second job in the stage builds the **VCRM**, the verification cross-reference matrix every systems engineer has maintained in a spreadsheet: for each requirement, its verification method, the verification cases that verify it, and their latest results. The model supplies the first three (the `verify` relationships, Chapter 6, SysML v2 for SysML v1 modelers (p. 57)); results come from files under `verification/` in a fixed format (requirement, case, verdict, commit, environment, timestamp, signed by), filled by the prime's test results in later drops and by people for analysis and inspection verdicts.

On drop 1 there are no results yet, and the job says so:

```
perl ci/job.pl vcrm
```

```
VCRM: 30 requirements, 0 verified, 0 failed, 21 not run, 9 without a case (vcrm.html, vcrm.csv)
```

It writes the matrix as a web page and a CSV for Excel, never fails (missing cases are the trace ratchet's job), and its page joins the model site. In the document era the VCRM was a spreadsheet edited by hand before each review; here it is regenerated on every change, from the model, and nobody edits it.

20.6.3 recipe: are the requirement statements well written?

The third job in the stage grades every requirement statement against the requirement recipe (Chapter 7, Requirements a pipeline can check: the recipe (p. 73)): a classification, a subject, one operative verb, a verifiable target verb, a threshold, an ID. It never fails: the DOORS text is the prime's, so its findings become a discrepancy report, not a blocked merge request. They show in the merge request's Code Quality widget, and the summary line feeds the scorecard.

```
perl ci/job.pl recipe | tail -1
```

Warning: Watch what you compare

The trace compares DOORS with *the model*. When the team imported the DOORS modules themselves into the repository as SysML (Chapter 28, Sprint 3: the first red X, DOORS in the model, and a wait nobody named (p. 440)), the first trace run over all of `model/` found every DOORS id in the DOORS-generated packages and reported `C 30 of 30` and `text differs from DOORS 0`: DOORS agreeing with itself. The job now leaves those packages out, by the header the import writes into them. The lesson stands for any measure: if your trace suddenly looks perfect, check what it is comparing before you brief it.

20.7 Stage docs: generated documents are current

The question it answers: are the documents generated from the model (the part tree, the function tree, the IDEF0 plates, the traceability report, all under `docs/`) the ones this version of the model produces?

What it runs: `model.pl docs`, then a check that git sees no difference in `docs/`. If the author changed the model and forgot to regenerate the documents, they differ, and the job fails with `docs/ is stale: run model.pl docs and commit`. The fix is two commands and a commit.

Why commit generated files at all? Because the baseline check below hashes them, and because a reviewer reading a merge request sees the effect of the model change on the documents (a part moved in the tree, a requirement newly traced) in the same diff. Committed, generated, checked: never edited by hand.

20.8 Stage delta: what the prime changed

```
delta:
  stage: delta
  rules:
    - if: '$CI_MERGE_REQUEST_SOURCE_BRANCH_NAME =~ /^feature\/drop-/'
    - if: '$CI_COMMIT_BRANCH == "vendor/prime"'
  script:
    - perl ci/job.pl delta
  artifacts:
    paths: [delta.txt]
```

The question it answers: between the two newest prime drops, which v1 elements were added, removed or changed, and which of those changes can the converter carry into v2 automatically, and which must be reworked by hand?

When it runs: on the `vendor/prime` branch (where each drop's converted output is committed) and on the merge request that brings a drop into `develop` (branches named `feature/drop-…`). The `=~ /^feature\/drop-/` is a pattern match: "the branch name starts with `feature/drop-`".

What it runs: `xmi2sysml.pl --delta OLD NEW` on the two newest `imports/*/tidewater.xmi`, with the project's layout map so the paths are the repository's. On drop 1 alone it says "first drop: nothing to compare". With drop 2 (delivered 8 January 2027, Chapter 43, Maintenance through the life cycle (p. 602)) it printed what you get by generating drop 2 yourself and comparing:

```
perl $KIT/sim/tidewater-v1.pl --drop 2 ../drops/v1.1 > /dev/null
perl $KIT/bin/xmi2sysml.pl --layout-map $KIT/examples/tidewater-v1/layout.map \
     --delta ../drops/v1.0/tidewater.xmi ../drops/v1.1/tidewater.xmi | head -24
```

```
xmi delta: tidewater.xmi (v1.0) -> tidewater.xmi (v1.1)
elements 560 -> 567: 7 added, 0 removed, 1 changed
carried by the converter (re-convert the drop and merge it): 8
manual (the converter does not carry these; rework the v2 text by hand): 0

added:
  + uml:Property           Stations::SensorSuite::hygrometer
      carried: physical/stations/Stations.sysml
  + uml:Class              Stations::Hygrometer
      carried: physical/stations/Stations.sysml
  + uml:Class              Requirements::System Requirements::Functions::Humidity
      carried: requirements/model/Requirements.sysml
  + uml:Abstraction        Verification::(Verify TestStationObservations -> Humidity)
      carried: verification/Verification.sysml

changed:
  ~ uml:Class              Requirements::System Requirements::Functions::Bulletin -- stereotype
  functionalRequirement: Text changed
      carried: requirements/model/Requirements.sysml
```

(an excerpt). Everything in drop 2 is carried; the "changed" line is the prime fixing the drift the team reported. Later drops have manual items (a new state in a state machine, which the converter does not carry), and those become tasks. This job is the reason the team rehearsed drop intake in sprint 6, before drop 2 arrived.

20.8.1 drop-package: keeping the delivery

On `vendor/prime` only, this job uploads each file of the newest drop to the project's generic package registry under `prime-drop/<version>/`, authenticated by the job's own token (`CI_JOB_TOKEN`, valid only while the job runs). On a laptop there is no registry, and the job says so and passes:

```
perl ci/job.pl drop-package
```

20.8.2 intake: a delivery from the prime's build system

The one job a *trigger* pipeline runs. When the prime's own build system (a Jenkins server, Chapter 45, The prime's model changes: receive, assess, disposition (p. 619)) uploads a drop to the

package registry and starts a pipeline with the variable `DROP_VERSION`, this job fetches the files, checks every checksum against the drop's manifest, keeps the drop as received (it never expires) and opens the Prime Drop issue that starts the vendor import. Tidewater used it from drop 4; until then drops arrived by file transfer and Tom ran the import by hand.

20.9 Stage release: the baseline audit

20.9.1 baseline: does the tagged model rebuild its documents?

The question it answers: is this baseline what it claims to be? This is the physical configuration audit, automated.

When it runs: only when a tag starting `baseline-` is created.

What it runs: regenerate `docs/` from the tagged model and check nothing changed; compute a SHA-256 checksum over the list of every file under `docs/` with its content hash; and compare it with the `sha256=` line written into the tag's message when the tag was created. If the documents do not rebuild, or the checksum differs, the job fails and the release is never published.

The job also writes `toolchain.txt`, an inventory of what produced the baseline, and keeps it, with the baseline's VCRM, as an artifact that never expires (`expire_in: never`): formal evidence for the life of the program.

A rehearsal tag, in the lesson's sandbox (the tag stays local; delete it after):

```
H=$(git ls-tree -r HEAD -- docs | sha256sum | cut -d' ' -f1)
git tag -a baseline-0 -m "Interim baseline 0
sha256=$H"
perl $KIT/bin/gitlab.pl ci --event tag --tag baseline-0 --var KIT=$KIT --var
CI_DEFAULT_BRANCH=develop --logs ../tag-logs
grep "rebuilds identically" ../tag-logs/baseline.log
head -8 toolchain.txt
```

The `baseline` job's last line, and the start of the inventory:

```
baseline-0 rebuilds identically
(sha256=1de3d676b74905a085651b79e158b084dbb622ca4c18c97794eba5a0a819dbcd)
baseline: baseline-0
commit: 130765dec690b480bf2dc6b9b157729137a03608
kit: 0e059c7
perl 5.34.0
git version 2.34.1
os: linux
runner: unknown
image: none (shell runner)
```

(`kit:` names the kit version that ran the checks; on the runner it is a released kit version, which is exactly what the inventory is there to prove. The checksum and commit differ on your machine.)

20.9.2 publish-package: the baseline as a package

Also on `baseline-*` tags, this job files the baseline itself in the package registry for downstream users (other programs, the test range, the prime): `tidewater-model/<tag>/` holding the model and its generated docs as one archive, the VCRM, the toolchain inventory and a `SHA256SUMS` list. On a laptop it leaves the files in a folder instead:

```
tidewater-model baseline-0: tidewater-model-baseline-0.tar.gz vcrm.csv toolchain.txt SHA256SUMS
no package registry here: the files are in package-upload/tidewater-model/baseline-0 (a local
run)
```

You never type the tag command by hand on the real project: `gitlab.pl flow released` creates the tag with the checksum after the release merge request merges (Chapter 31, Sprint 6: Behavior by hand, a table for the program, and baseline-0 (p. 495), Chapter 43, Maintenance through the life cycle (p. 602)).

20.9.3 publish-release: the release page

Once `baseline` passes, this job copies the baseline's section from `CHANGELOG.md` into release notes, adds the verification summary, and GitLab's `release:` keyword creates the release page for the tag with those notes, links to the package files as the release's assets, and the sprint milestones it closes. The rehearsal had no CHANGELOG section for `baseline-0`, and the notes said so honestly:

```
cat release-notes.md
git tag -d baseline-0
```

`needs: [baseline]` makes the release wait for the audit; if the audit fails, there is no release.

20.10 Stage publish: the model site

```
pages:
  stage: publish
  rules:
    - if: '$CI_PIPELINE_SOURCE == "schedule" && $FULL != "1"'
      when: never
    - if: '$CI_PIPELINE_SOURCE == "trigger"'
      when: never
    - if: '$CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH'
  environment:
    name: model-site
    url: $CI_PAGES_URL
  script:
    - perl ci/job.pl pages
  artifacts:
    paths: [public]
```

The question it answers: what does the model look like today, for someone who will never open the repository?

When it runs: on every push to the default branch (`develop`), which in practice means after every merge, and on the nightly schedule; never on the weekly export or a delivery trigger.

What it runs: copies the generated documents into `public/docs/`, draws the four views (part tree, trace, interconnection and packages) into `public/views/`, copies the IDEF0 plates into `public/idef0/`, writes the trace report, the VCRM, the complexity trend and a TODO burn-down table (one row per merge, computed from the history), and writes an index page. GitLab publishes the `public/` folder because the job is named `pages`; that name is the whole configuration. The `environment:` lines record each publication as a deployment of `model-site`.

```
perl ci/job.pl pages | tail -1
ls public public/views
```

The package view deserves a note. The kit's package drawer normally checks a security marking on every package (Chapter 48, Security: protect the environment, engineer it into the system (p. 651)); the converted Tidewater model has none yet, so the drawer draws packages, dependencies and cycles only, and says it did not check markings. Its count of cycles (four on drop 1 in the layered layout, six in the converter's flat layout) is higher than the `complexity` job's, because the drawer counts each arrow that closes a loop, in the order it visits the folders, while the job counts groups of packages tangled together. Both are the spider web, seen at two levels of detail (Section 11.4, The five views, read (p. 161)).

20.11 Stage report: the weekly export

```
status:
  stage: report
  rules:
    - if: '$CI_PIPELINE_SOURCE == "schedule" && $FULL != "1"'
  script:
    - perl ci/job.pl status
  artifacts:
    paths: [exports/]
```

The question it answers: what does GitLab know this week, in a form the status metrics can read?

When it runs: only from the weekly pipeline schedule (the one without `FULL=1`): every Friday at 06:00 Eastern, cron expression `0 6 * * 5`, on `develop`, set in `project.conf` under `[schedules]` (Chapter 15, Setting up GitLab (p. 226)). A schedule runs the pipeline as the person who owns the schedule, with that person's permissions; if the owner leaves the project, the schedule stops (Chapter 42, Administration (p. 586) has the hand-over step).

What it runs: `gitlab.pl export`, authenticated with `STATUS_TOKEN`, a read-only project access token stored as a masked, protected CI/CD variable, which writes `ci_jobs.csv`, `mrs.csv`, `issues.csv`, `defects.csv` and one tracker file per sprint into `exports/`. If the repository holds a `metrics.json`, it also runs the status metrics collector. On the lesson's emulator the same export, by hand:

```
. ../env.sh > /dev/null           # gl is a shell function: each new shell (or test block) needs
it again
gl export ../exp
head -3 ../exp/ci_jobs.csv
```

```
ci_jobs 54, defects 0, issues 2, mrs 1, tracker 1 -> ../exp
ts,job,branch,result,minutes
2026-10-06T15:46:53Z,grammar,main,pass,0.02
2026-10-06T15:46:54Z,lint,main,pass,0.01
```

(the counts and times depend on how far you went in the lesson). Chapter 36, Telling management (p. 538) shows how the weekly export turns into the Friday status.

The second schedule, **nightly** (`0 2 * * 1-5`, weekdays at 02:00, on `develop`), runs the ordinary push pipeline, so a problem caused by something outside the repository (a kit update on the runner, an expired certificate) shows up in the morning.

20.12 The whole run, and what it costs

The merge request pipeline on the drop 1 import, run with the kit's local runner on a laptop:

```
perl $KIT/bin/gitlab.pl ci --event merge_request_event --target develop --var KIT=$KIT --logs
../mr-logs
```

```
grammar                  check      success            2.8s
lint                     check      success            2.3s
outline                  check      success            0.4s
todo                     check      success            0.1s
secrets                  check      success            0.1s
complexity               check      success            0.1s
trace                    trace      success            0.6s
vcrm                     trace      success            0.1s
recipe                   trace      success            0.4s
docs                     docs       success            0.8s
pipeline: success (10 jobs)
```

About eight seconds of checks. On the GitLab server, each job also pays the runner's start-up cost (fetching the repository, preparing the shell), typically a few seconds to half a minute per job depending on the runner, so a merge request pipeline takes a minute or two from push to result. In the document era the same audit was a review meeting.

Table 20.1: Which jobs run for which event (checked with `gitlab.pl ci --list` for each event)

Event	Jobs that run	Purpose
Merge request	grammar, lint, outline, todo, secrets, complexity, trace, vcrm, recipe, docs	the gate
Push to develop (after a merge)	the same ten, plus pages	integration check and the model site
Push to vendor/prime	delta and drop-package only	drop intake: the raw drop is filed, not judged
Merge request from feature/drop-...	the ten, plus delta	reviewing a drop's merge
Tag baseline-*	the ten, plus baseline, publish-package and publish-release	the baseline audit and release
Trigger from the prime's build (DROP_VERSION set)	intake	taking in a delivery
Weekly schedule	status	the export for the status metrics
Nightly schedule (FULL=1, on develop)	the ten, plus pages	early warning

Warning: Local runs leave files behind

When you run the pipeline on your laptop, the jobs write their reports (`lint.txt`, `lint.xml`, `lint-cq.json`, `lint.metrics` and the rest) into your working copy, because that is where they run. On the server they are thrown away with the job's environment. Without care, the next `git add -A` commits them. The model repository's `.gitignore` lists them; the template the team started from (`tools/gitlab/templates/.gitignore`) already does: `*.xml`, `*-cq.json`, `*.metrics`, the jobs' `.txt` outputs, `public/` and the package and log folders, with an exception for the manifests under `imports/`. Check with `git status` after a local run: it should show nothing new.

20.13 Changing the pipeline

The pipeline file is protected like everything else: CODEOWNERS makes Tom its owner, so every change to it is a merge request Tom approves, and the pipeline runs on the merge request *with the changed file*. That last point is a trap and a help. A change that breaks the pipeline file fails on its own merge request and never reaches `develop`. But a change that weakens a check (a deleted line, an added `allow_failure`) passes its own pipeline easily. The reviewer of a pipeline change must read the diff of the YAML as carefully as a reviewer of a requirement change reads the "shall".

Before pushing a change to the pipeline, check it locally (`gitlab.pl ci --list` for every event in the table above) and in GitLab's pipeline editor (Build > Pipeline editor > Validate).

20.14 The complete file

tools/gitlab/templates/gitlab-ci.yml

```
# .gitlab-ci.yml -- the Tidewater government model pipeline
(tools/gitlab/templates/gitlab-ci.yml).
#
# In configuration-management terms this file is the CONFIGURATION AUDIT, written down once and
run by the server on
# every proposed change (every merge request), on every baseline tag, nightly and once a week for
the status report.
# A job is one audit check; a stage is a group of checks that run together; the stages run in the
order listed below.
#
# Copy it to the model repo as .gitlab-ci.yml, with tools/gitlab/templates/ci/job.pl as ci/job.pl.
Every job's script
# is one line, `perl ci/job.pl NAME`: the job's logic lives in that Perl script, so it runs the
same under the Windows
# runner's PowerShell, cmd, Git Bash, a Linux runner, the Docker image (tools/gitlab/docker) and
`gitlab.pl ci` on a
# laptop. ci/job.pl calls the agile kit through the CI/CD variable KIT (the path of the kit's
checkout on the runner;
# `gitlab.pl setup` sets it from project.conf [variables]). Nothing is installed: Perl and Git
are all a job needs.
#
# Run it on your own machine before you push, with the same rules GitLab applies:
#   perl $KIT/bin/gitlab.pl ci --event merge_request_event --branch feature/12-stations --target
develop --var KIT=$KIT
#   perl $KIT/bin/gitlab.pl ci --list            (which jobs would run, and why the others are
skipped)
#
# Stages, in order:
#   check    the gate: grammar, structure lint, outline (a ratchet), secrets, and the TODO count
(the port burn-down)
#   trace    DOORS against the model (a ratchet on merge requests); the VCRM; SIL/HIL when a
harness exists (disabled)
#   docs     generated docs must be committed and current
#   delta    on drop intake only: the delivery checked (intake, on a trigger), what the prime
changed in v1 (delta),
#            and the drop filed in the package registry
#   release  on baseline tags only: the baseline rebuilds byte for byte, then the GitLab release
is created
#   publish  on the default branch only: the model's web site (GitLab Pages) for readers who
never open the repo
#   report   on the weekly schedule only: export issues, MRs and jobs for the status metrics
#
# Reports: the check jobs attach JUnit (the MR's test widget: which check failed and why), Code
Quality (the MR's
# findings widget: which errors, warnings or TODO lines this change added or fixed) and metrics
(Premium: how each
# number moved) -- all written by $KIT/tools/gitlab/ci-reports.pl from the checkers' own output,
called by ci/job.pl.
#
# YAML note: a script line that contains ": " (colon, space) must be in quotes, or YAML reads it
as a map. With
# ci/job.pl there are none left.

stages:
  - check
  - trace
  - docs
```

```
  - delta
  - release
  - publish
  - report

variables:
  GIT_DEPTH: "0"            # the full history: trace reads the merge base, delta reads earlier
  imports, pages the merges

# One pipeline per change, never two: an MR gets a merge request pipeline; a branch with an open
MR does not also get a
# branch pipeline. Branches without an MR (develop, main, vendor/prime), tags and the schedules
still get theirs.
workflow:
  rules:
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'
    - if: '$CI_COMMIT_BRANCH && $CI_OPEN_MERGE_REQUESTS && $CI_PIPELINE_SOURCE == "push"'
      when: never
    - if: '$CI_COMMIT_BRANCH'
    - if: '$CI_COMMIT_TAG'
    - if: '$CI_PIPELINE_SOURCE == "schedule"'

# GitLab's own secret scanner (Free tier) runs its analyzer in a Docker image, so it needs a
runner with the Docker
# executor. The team's runner is a Windows *shell* runner: there the scanner's job would sit
"pending" forever and, with
# "pipelines must succeed", block every MR. So the include is conditional: it is pulled in only
when the project
# variable DOCKER_RUNNER is "true" (project.conf [variables] sets it to "false"). The `secrets`
job below is the
# always-on check, runnable on any runner and by `gitlab.pl ci` on a laptop (which lists the
include as "runs on
# GitLab only").
include:
  - template: Security/Secret-Detection.gitlab-ci.yml
    rules:
      - if: '$DOCKER_RUNNER == "true"'

# Runner tags pick the runner: the team's Windows shell runner is registered with the tags
windows and mbse. On a
# Docker runner (tools/gitlab/docker), drop these tags or use that runner's own.
default:
  tags:
    - windows
    - mbse

# The rules every model job shares: run on changes and on the nightly full check (FULL=1), never
on the weekly status
# schedule (that one only reports) or a delivery trigger (that one only takes the drop in).
.model_rules: &model_rules
  rules:
    - if: '$CI_PIPELINE_SOURCE == "schedule" && $FULL != "1"'
      when: never
    - if: '$CI_PIPELINE_SOURCE == "trigger"'
      when: never
    # vendor/prime holds the prime's drop as delivered, converted and untouched: the model checks
    run where it is
    # merged (feature/drop-VERSION and its MR), not on the raw branch, which would stay red (no
    docs, no recorded
```

```
    # outline count) for structure only the prime can change. The delta and drop-package jobs
    still run there.
    - if: '$CI_COMMIT_BRANCH == "vendor/prime"'
      when: never
    - when: on_success

# A hidden job: the check-stage jobs extend it. Reports are kept even when the check fails (that
is when they matter).
.check:
  stage: check
  <<: *model_rules

# GRAMMAR -- every .sysml file against the official SysML v2 grammar (tools/sysml/sysml.pl, built
from the vendored
# KerML/SysML release). A syntax error anywhere stops the merge. One JUnit test case per file.
grammar:
  extends: .check
  script:
    - perl ci/job.pl grammar
  artifacts:
    when: always
    paths:
      - grammar.txt
    reports:
      junit: grammar.xml
      codequality: grammar-cq.json

# LINT -- the model's structure: names resolve, packages import what they use. The last line is
# elements=N errors=E warnings=W; the gate is 0 errors and 0 warnings ("0/0 to merge").
lint:
  extends: .check
  script:
    - perl ci/job.pl lint
  artifacts:
    when: always
    paths:
      - lint.txt
    reports:
      junit: lint.xml
      codequality: lint-cq.json
      metrics: lint.metrics

# OUTLINE -- 2 to 9 children at every level, and each part def carries its level tag (@L1Network
... @L6Component).
# A RATCHET, not a wall: the converted v1 model starts with violations it inherited from the
prime's structure (a
# 25-member package, parts with one child), and most can only be fixed upstream, by a discrepancy
report to the prime.
# So the gate is "no new violations": .ci/outline-max holds the highest count allowed; a change
that adds a violation
# fails, a change that removes one is told to lower the number (and the reviewer checks that it
did). When the count
# reaches 0 the file reads 0 and the rule is absolute. A new project (kickoff.pl scaffold) starts
with room: its
# skeleton's one-file folders plus one for each layer folder still to come, with comment lines
saying when to tighten
# it (every layer folder holds two files, or the first baseline at the latest); the job reads
only the first line.
outline:
```

```
  extends: .check
  script:
    - perl ci/job.pl outline
  artifacts:
    when: always
    paths:
      - outline.txt
    reports:
      junit: outline.xml
      codequality: outline-cq.json
      metrics: outline.metrics

# TODO -- the port burn-down: every // TODO line the converter left. Never fails. Each TODO is an
"info" finding, so the
# MR widget shows the TODO lines a change closed (and any it added).
todo:
  extends: .check
  script:
    - perl ci/job.pl todo
  artifacts:
    paths:
      - todo.txt
    reports:
      codequality: todo-cq.json
      metrics: todo.metrics

# SECRETS -- tokens, cloud keys and private keys must never reach the repo (an MR is the last
place to catch it).
secrets:
  extends: .check
  script:
    - perl ci/job.pl secrets

# COMPLEXITY -- the spider-web guardrails: per package fan-in and fan-out, dependency cycles,
dependencies that go up the
# layers named in sysml.conf (`layers = ...`, lowest first), duplicate names, and relationship
ends per part def (the
# spaghetti measure). A RATCHET on merge requests: it fails only when a change makes a measure
worse than on the target
# branch, or adds a wildcard import to a government file. Findings show in the MR's Code Quality
widget.
complexity:
  extends: .check
  script:
    - perl ci/job.pl complexity
  artifacts:
    when: always
    paths:
      - complexity.txt
    reports:
      codequality: complexity-cq.json
      metrics: complexity.metrics

# TRACE -- DOORS against the model, from the newest imported drop: linked (C), orphans (F),
unverified (G), text drift.
# A RATCHET on merge requests: the job also traces the model as it was at the merge base and
fails only when this change
# adds a gap (removes a satisfy, drops a verify, edits a requirement's text away from DOORS).
Gaps the prime must fix
```

```
# stay visible in the widget without blocking unrelated work. On develop and schedules it reports
and passes.
trace:
  stage: trace
  <<: *model_rules
  script:
    - perl ci/job.pl trace
  artifacts:
    when: always
    paths:
      - trace.txt
    reports:
      junit: trace.xml
      metrics: trace.metrics

# VCRM -- the verification cross-reference matrix: every requirement, its verification method,
the cases that verify it
# (verify relationships in the model) and their newest verdicts from verification/*.csv (one
schema: requirement, case,
# verdict, commit, environment, timestamp, signed_by -- the prime's test results ride in each
drop from drop 4 on;
# analysis and inspection verdicts are entered and signed by a person). Reports, never blocks:
the trace ratchet gates
# missing cases. vcrm.html is also on the Pages site.
vcrm:
  stage: trace
  <<: *model_rules
  script:
    - perl ci/job.pl vcrm
  artifacts:
    when: always
    paths:
      - vcrm.html
      - vcrm.csv
    reports:
      junit: vcrm.xml
      metrics: vcrm.metrics

# RECIPE -- every requirement statement graded against the requirement recipe
(tools/sysml/req-recipe.pl): missing
# parts, weak verbs ("provide", "support"), passive voice, vague words. Reports only: the DOORS
text is the prime's,
# so findings go to the prime as a discrepancy report. Findings show in the MR's Code Quality
widget.
recipe:
  stage: trace
  <<: *model_rules
  script:
    - perl ci/job.pl recipe
  artifacts:
    when: always
    paths:
      - recipe.txt
    reports:
      codequality: recipe-cq.json
      metrics: recipe.metrics

# DOCS -- generated docs (docs/Nouns.md, Verbs.md, idef0/, Traceability.md) are committed with
the model that made them.
```

```
# Stale docs fail here: run model.pl docs and commit the result.
docs:
  stage: docs
  <<: *model_rules
  script:
    - perl ci/job.pl docs

# DELTA -- drop intake: the v1 delta between the two newest imports. Runs on the vendor branch
and on the drop merge MR,
# so the reviewer sees what the prime changed, including what the converter cannot carry (state
machines, interactions,
# constraint expressions), which has to be reworked by hand.
delta:
  stage: delta
  rules:
    - if: '$CI_MERGE_REQUEST_SOURCE_BRANCH_NAME =~ /^feature\/drop-/'
    - if: '$CI_COMMIT_BRANCH == "vendor/prime"'
  script:
    - perl ci/job.pl delta
  artifacts:
    paths:
      - delta.txt

# DROP-PACKAGE -- file each prime drop, exactly as delivered, in the project's Generic Package
Registry
# (Deploy > Package registry > prime-drop): the CDRL deliverable kept with the model it became,
downloadable by version.
# The job token authenticates; nothing to configure. Skipped outside GitLab (no registry on a
laptop).
drop-package:
  stage: delta
  rules:
    - if: '$CI_COMMIT_BRANCH == "vendor/prime"'
  script:
    - perl ci/job.pl drop-package

# INTAKE -- a delivery from the prime's Jenkins: its build uploads the drop to the Generic
Package Registry (prime-drop/
# <version>, with a deploy token that can do nothing else) and fires a pipeline trigger with
DROP_VERSION. This job then
# fetches every file, checks each sha256 against the MANIFEST the prime's build wrote, keeps the
drop as received
# (formal evidence: never expires) and opens the Prime Drop issue that starts the vendor import.
A green Jenkins build
# on the prime's side is evidence, not acceptance: the government's own gates run on the import
MR.
intake:
  stage: delta
  rules:
    - if: '$CI_PIPELINE_SOURCE == "trigger" && $DROP_VERSION'
  script:
    - perl ci/job.pl intake
  artifacts:
    expire_in: never
    paths:
      - intake.txt
      - intake/
```

```
# BASELINE -- a baseline tag must rebuild: regenerate the docs, confirm nothing changed, and
check the hash in the tag
# message: sha256 over `git ls-tree -r <commit> -- docs` (mode, blob id and path of every tracked
file under docs/),
# the same text `gitlab.pl flow released` hashed when it wrote the tag. It also writes
toolchain.txt: the kit version,
# Perl, Git, OS, runner and image that produced the baseline (an inventory for the audit). This
is the physical
# configuration audit of the baseline, done by machine.
baseline:
  stage: release
  rules:
    - if: '$CI_COMMIT_TAG =~ /^baseline-/'
  script:
    - perl ci/job.pl baseline
  artifacts:
    expire_in: never         # formal evidence: the baseline's audit, toolchain and VCRM are kept
    for the life of the program
    paths:
      - toolchain.txt
      - vcrm.html
      - vcrm.csv
    reports:
      dotenv: release.env    # RELEASE_MILESTONE: the sprint this baseline closes, for the
      release below

# PUBLISH-PACKAGE -- the baseline as a package for downstream users (other programs, the test
range, the prime): the
# model and its docs at the tag (git archive), the VCRM, the toolchain and SHA256SUMS, as
tidewater-model/<tag> in the
# Generic Package Registry. CI_JOB_TOKEN authenticates the upload. The release below links to
these files.
publish-package:
  stage: release
  needs:
    - baseline
  rules:
    - if: '$CI_COMMIT_TAG =~ /^baseline-/'
  script:
    - perl ci/job.pl publish-package
  artifacts:
    expire_in: never
    paths:
      - package-upload/

# PUBLISH-RELEASE -- once the baseline audit passes, create the GitLab release for the tag
(Deploy > Releases), with the
# baseline's CHANGELOG section, its verification summary and the toolchain as notes, links to the
package files, and
# the sprint milestone it closes (GitLab then collects the release evidence: the milestone's
issues and the tag
# pipeline's test report, as a JSON snapshot). The `release:` keyword is run by GitLab's
release-cli after the script.
publish-release:
  stage: release
  needs:
    - baseline
    - publish-package
  rules:
```

```
    - if: '$CI_COMMIT_TAG =~ /^baseline-/'
  script:
    - perl ci/job.pl publish-release
  release:
    tag_name: $CI_COMMIT_TAG
    name: 'Tidewater model $CI_COMMIT_TAG'
    description: release-notes.md
    milestones:
      - $RELEASE_MILESTONE
    assets:
      links:
        - name: 'Model and docs (tar.gz)'
          url: '${CI_API_V4_URL}/projects/${CI_PROJECT_ID}/packages/generic/tidewater-model/${CI_
          COMMIT_TAG}/tidewater-model-${CI_COMMIT_TAG}.tar.gz'
          link_type: package
        - name: 'VCRM (csv)'
          url: '${CI_API_V4_URL}/projects/${CI_PROJECT_ID}/packages/generic/tidewater-model/${CI_
          COMMIT_TAG}/vcrm.csv'
          link_type: other
        - name: 'Toolchain'
          url: '${CI_API_V4_URL}/projects/${CI_PROJECT_ID}/packages/generic/tidewater-model/${CI_
          COMMIT_TAG}/toolchain.txt'
          link_type: other
        - name: 'SHA256SUMS'
          url: '${CI_API_V4_URL}/projects/${CI_PROJECT_ID}/packages/generic/tidewater-model/${CI_
          COMMIT_TAG}/SHA256SUMS'
          link_type: other

# SIL / HIL -- software-in-the-loop regression (nightly) and hardware-in-the-loop tests (on a
bench, by hand), for when
# the program has a simulation or a lab to drive. DISABLED: each runs only when its variable is
"true" (SIL_ENABLED,
# HIL_ENABLED), so this file stays valid where there is no harness. Each job runs the harness
named by SIL_CMD /
# HIL_CMD and files its verdicts in the same results schema (verification/results-sil.csv /
-hil.csv), so the VCRM
# shows them next to the prime's results. The HIL job shows the GitLab features a lab needs: a
runner tag that only the
# bench's runner has, a resource_group (one test on the bench at a time), when: manual (a person
starts it, with the
# bench ready), and an environment (the bench's deployment history).
sil-regression:
  stage: trace
  rules:
    - if: '$SIL_ENABLED == "true" && $CI_PIPELINE_SOURCE == "schedule"'
  script:
    - perl ci/job.pl sil-regression
  artifacts:
    expire_in: never
    paths:
      - sil.txt
      - verification/results-sil.csv

hil-test:
  stage: trace
  tags:
    - hil-bench
  resource_group: hil-bench
  rules:
```

```
    - if: '$HIL_ENABLED == "true" && $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH'
      when: manual
  environment:
    name: hil-lab
  script:
    - perl ci/job.pl hil-test
  artifacts:
    expire_in: never
    paths:
      - hil.txt
      - verification/results-hil.csv

# PAGES -- the model's web site, rebuilt on every merge to the default branch and nightly: the
generated docs, the tree /
# trace / package views, the requirement trace and the port burn-down. For the program office,
the prime and review
# boards: people who need the model's state and will never clone a repository. GitLab serves
public/ at $CI_PAGES_URL.
pages:
  stage: publish
  rules:
    - if: '$CI_PIPELINE_SOURCE == "schedule" && $FULL != "1"'
      when: never
    - if: '$CI_PIPELINE_SOURCE == "trigger"'
      when: never
    - if: '$CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH'
  environment:
    name: model-site
    url: $CI_PAGES_URL
  script:
    - perl ci/job.pl pages
  artifacts:
    paths:
      - public

# STATUS -- weekly, from a pipeline schedule (Build > Pipeline schedules, Fridays 06:00): export
what GitLab knows --
# issues, MRs, jobs -- in the status-metrics formats, and collect this week's metrics if the repo
has a metrics.json.
# STATUS_TOKEN is a masked CI/CD variable holding a read_api project access token.
status:
  stage: report
  rules:
    - if: '$CI_PIPELINE_SOURCE == "schedule" && $FULL != "1"'
  script:
    - perl ci/job.pl status
  artifacts:
    paths:
      - exports/
```

And the script every job calls, `ci/job.pl` (copied into the model repository from `tools/gitlab/templates/ci/job.pl`):

tools/gitlab/templates/ci/job.pl

```
#!/usr/bin/perl
```

```
# ci/job.pl -- every job of the model pipeline, in core Perl, so .gitlab-ci.yml only ever says
`perl ci/job.pl NAME`.
#
# Why a script and not shell lines in the YAML: GitLab Runner's Windows shell executor runs
PowerShell, where $KIT is
# $env:KIT and `$(find ...)`, `[ -f ... ]`, `| xargs`, `&&` and quoting all differ from sh. One
Perl script behaves the
# same under PowerShell, cmd, Git Bash, a Linux runner, the Docker image and `gitlab.pl ci` on a
laptop. The YAML keeps
# what GitLab needs to know (when a job runs, what it keeps, which reports it attaches); this
file keeps what it does.
#
#   perl ci/job.pl grammar | lint | outline | todo | secrets | trace | vcrm | docs | delta |
drop-package | intake | baseline
#                  | publish-package | publish-release | pages | status | sil-regression |
hil-test
#
# Every job needs the CI/CD variable KIT (the agile kit's checkout on the runner) and writes its
log and report files
# in the project directory: NAME.txt (the checker's own output), NAME.xml (JUnit), NAME-cq.json
(Code Quality),
# NAME.metrics (metrics), via $KIT/tools/gitlab/ci-reports.pl. Exit status: 0 pass, 1 the check
failed, 2 the job
# cannot run (KIT missing, bad usage).
#
# Copied into the model repository as ci/job.pl (from tools/gitlab/templates/ci/job.pl). Core
Perl only.
package CIJob;                                  # a package, so tests can `require` this file and
call its functions
use strict;
use warnings;
use File::Find ();
use File::Path qw(make_path);
use File::Copy qw(copy);
use Digest::SHA ();

my %JOBS = map { $_ => 1 } qw(grammar lint outline todo secrets trace vcrm recipe complexity docs
delta drop-package intake baseline publish-package
                                publish-release pages status sil-regression hil-test);
our $KIT = $ENV{KIT} // '';
# The toolchain is pinned like the model: .kit-version names the kit release the model was
checked with. A runner with a
# different kit fails every job, so a tool upgrade reaches the model only through a reviewed MR
that bumps the pin.
sub kit_version { my $v = $ENV{KIT_VERSION}; return $v if defined $v && length $v;
    my $o = `git -C "$KIT" describe --tags --always 2>&1`; $o =~ s/\s+$//; $? ? 'unknown' : $o }
sub main {
    my $job = shift // '';
    if (!$JOBS{$job}) { print STDERR "usage: perl ci/job.pl " . join(' | ', sort keys %JOBS) .
    "\n"; return 2 }
    if ($KIT eq '') { print STDERR "KIT is not set: add it under Settings > CI/CD > Variables
    (the kit's path on the runner)\n"; return 2 }
    if (!-f "$KIT/tools/sysml/model.pl") { print STDERR "KIT=$KIT does not hold the agile kit (no
    tools/sysml/model.pl)\n"; return 2 }
    $| = 1;
    if (open my $pin, '<', '.kit-version') {
        my $want = <$pin> // ''; close $pin;
        $want =~ s/^\s+|\s+$//g;
        my $have = kit_version();
```

```
        if (length $want && $want ne $have) {
            print STDERR "the kit on this runner is $have, but .kit-version pins $want:\n"
                       . "  bump .kit-version in a merge request (toolchain changes are reviewed
                       like model changes), or put the runner back on $want\n";
            return 2;
        }
    }
    our %run;
    $run{$job}->() ? 1 : 0;
}

# ---------------------------------------------------------------- helpers
sub slurp { my $f = shift; open my $fh, '<:raw', $f or return undef; local $/; my $t = <$fh>;
close $fh; $t // '' }
sub spew { my ($f, $t) = @_; open my $fh, '>:raw', $f or die "cannot write $f: $!\n"; print $fh
$t; close $fh }
our $QUIET = 0;                                 # local $QUIET = 1: run() writes the log file only,
not the job log
sub run {                                       # run(LOGFILE|undef, cmd...) -> exit code; output
(stdout+stderr) to the log and the job log
    my ($log, @cmd) = @_;
    print '$ ', join(' ', map { /\s/ ? "\"$_\"" : $_ } @cmd), "\n" unless $QUIET;
    return system(@cmd) >> 8 unless defined $log;
    open my $save_out, '>&', \*STDOUT or die "cannot save STDOUT: $!\n";
    open my $save_err, '>&', \*STDERR or die "cannot save STDERR: $!\n";
    open STDOUT, '>', $log or die "cannot write $log: $!\n";
    open STDERR, '>&', \*STDOUT or die "cannot redirect STDERR: $!\n";
    my $rc = system(@cmd);
    open STDOUT, '>&', $save_out or die "cannot restore STDOUT: $!\n";
    open STDERR, '>&', $save_err or die "cannot restore STDERR: $!\n";
    print slurp($log) // '' unless $QUIET;
    $rc == -1 ? 127 : $rc >> 8;
}
sub out {                                       # out(cmd...) -> its standard output (a temp file,
not a pipe: list-form pipes are not portable)
    my @cmd = @_;
    require File::Temp;
    my ($fh, $tmp) = File::Temp::tempfile('ci-job-XXXXXX', TMPDIR => 1, UNLINK => 1);
    close $fh;
    open my $save, '>&', \*STDOUT or die "cannot save STDOUT: $!\n";
    open STDOUT, '>', $tmp or die "cannot write $tmp: $!\n";
    system(@cmd);
    open STDOUT, '>&', $save or die "cannot restore STDOUT: $!\n";
    my $t = slurp($tmp) // '';
    unlink $tmp;
    $t;
}
sub perl_tool { my ($log, $tool, @args) = @_; run($log, $^X, "$KIT/$tool", @args) }
sub reports { my ($mode, @args) = @_; perl_tool(undef, 'tools/gitlab/ci-reports.pl', $mode,
@args) }
sub sysml_files { my @f; File::Find::find({ no_chdir => 1, wanted => sub { push @f,
$File::Find::name if /\.sysml$/ && -f } }, 'model') if -d 'model'; sort @f }
sub vkey { my $v = shift; join '.', map { /^\d+$/ ? sprintf('%06d', $_) : $_ } split /[.\-]/, $v
}
sub imports {                                   # the imported prime drops, oldest first
(imports/v1.0, imports/v1.1, ... in version order)
    opendir my $dh, 'imports' or return ();
    my @d = grep { !/^\./ && -d "imports/$_" } readdir $dh;
    closedir $dh;
```

```
    map { "imports/$_" } sort { vkey($a) cmp vkey($b) } @d;
}
sub ls { my $d = shift; opendir(my $dh, $d) or return (); my @e = sort grep { !/^\./ } readdir
$dh; closedir $dh; @e }
sub docs_hash { Digest::SHA::sha256_hex(out('git', 'ls-tree', '-r', $_[0] // 'HEAD', '--',
'docs')) }   # the same text GitFlow hashes
sub sha256_file { my $f = shift; my $d = Digest::SHA->new(256); open my $fh, '<:raw', $f or
return ''; $d->addfile($fh); close $fh; $d->hexdigest }
sub last_line { my $t = shift // ''; my @l = grep { /\S/ } split /\n/, $t; $l[-1] // '' }

# ------------------------------------------------------------------ the jobs
our %run;
$run{grammar} = sub {                          # every .sysml file against the SysML v2 grammar;
one JUnit case per file
    my @f = sysml_files();
    if (!@f) { spew('grammar.txt', "no .sysml files under model/\n"); print "no .sysml files
    under model/ yet\n"; return 0 }
    my $rc = perl_tool('grammar.txt', 'tools/sysml/sysml.pl', 'check', @f);
    reports('grammar', '--junit', 'grammar.xml', '--codequality', 'grammar-cq.json',
    'grammar.txt', @f);
    $rc;
};
$run{lint} = sub {                             # structure: the gate is 0 errors and 0 warnings
    perl_tool('lint.txt', 'tools/sysml/model.pl', 'lint');
    reports('lint', '--junit', 'lint.xml', '--codequality', 'lint-cq.json', '--metrics',
    'lint.metrics', 'lint.txt');
    my $l = last_line(slurp('lint.txt'));
    return 0 if $l =~ / errors=0 warnings=0$/;
    print "lint is not 0/0 ($l)\n";
    1;
};
sub outline_junit_inherited {                  # findings the allowance already covers: skipped
tests, not failures (a green MR shows no red)
    my $x = slurp('outline.xml') // return;
    my $n = $x =~ s{<failure message="([^"]*)"[^>]*>.*?</failure>}{<skipped message="inherited,
    within .ci/outline-max: $1"/>}gs;
    return unless $n;
    $x =~ s{(<testsuites?\b[^>]*?) failures="(\d+)"}{my ($h, $f) = ($1, $2); "$h failures=\"0\"
    skipped=\"$f\""}ge;
    spew('outline.xml', $x);
}
$run{outline} = sub {                          # the 2..9 rule and level tags, as a ratchet
against .ci/outline-max
    perl_tool('outline.txt', 'tools/sysml/model.pl', 'check');
    reports('outline', '--junit', 'outline.xml', '--codequality', 'outline-cq.json', '--metrics',
    'outline.metrics', 'outline.txt');
    my $t = slurp('outline.txt');
    my ($v) = $t =~ /^violations\s+(\d+)/m;
    my ($m) = $t =~ /^\@L tag mismatches\s+(\d+)/m;
    my $n = ($v // 0) + ($m // 0);
    my $file = slurp('.ci/outline-max') // '';
    my $max = $file =~ /^\s*(\d+)/ ? $1 : 0;   # the first line; comment lines after it (a new
    scaffold's room) are for people
    print "outline violations: $n (allowed: $max)\n";
    # vendor/prime holds the prime's structure as delivered, and a drop branch merges it:
    measured, not ratcheted (the
    # drop's merge request records the new count in .ci/outline-max; Discrepancy Reports ask the
    prime to fix the rest)
```

```
    if (($ENV{CI_COMMIT_BRANCH} // '') =~ m{^(?:vendor/prime$|feature/drop-)} ||
    ($ENV{CI_MERGE_REQUEST_SOURCE_BRANCH_NAME} // '') =~ m{^feature/drop-}) {   # the MR and the
    branch's own pushes
        outline_junit_inherited();
        print "a prime drop: reported, not ratcheted\n";
        return 0;
    }
    if (($ENV{CI_MERGE_REQUEST_SOURCE_BRANCH_NAME} // '') =~ m{^release/} ||
    ($ENV{CI_COMMIT_BRANCH} // '') =~ m{^release/}) {   # as complexity: develop already checked
    it
        outline_junit_inherited();
        print "a release: its changes were checked on develop; reported only\n";
        return 0;
    }
    if ($n > $max) { print "this change adds outline violations ($n > $max): fix them, do not
    raise .ci/outline-max\n"; return 1 }
    if ($n < $max) {                       # a scaffold's allowance keeps room for new one-file
    folders until its comment says to tighten it
        print $file =~ /^#/m ? "fewer violations than allowed: room left for new folders (see the
        comment in .ci/outline-max for when to tighten it to $n)\n"
                             : "fewer violations than allowed: lower .ci/outline-max to $n in
                             this MR\n";
    }
    outline_junit_inherited();
    0;
};
$run{todo} = sub {                             # the port burn-down: every // TODO line the
converter left; never fails
    my @hits;
    for my $f (sysml_files()) { my $n = 0; for my $l (split /\n/, slurp($f)) { $n++; push @hits,
    "$f:$n:$l" if index($l, '// TODO') >= 0 } }
    spew('todo.txt', join('', map { "$_\n" } @hits));
    print "TODO lines remaining: " . scalar(@hits) . "\n";
    reports('todo', '--codequality', 'todo-cq.json', '--metrics', 'todo.metrics', -d 'model' ?
    'model' : 'todo.txt');
    0;
};
$run{secrets} = sub {                          # tokens, cloud keys and private keys must never
reach the repository
    my @pat = (qr/glpat-[0-9A-Za-z_\-]{20}/, qr/AKIA[0-9A-Z]{16}/, qr/-----BEGIN [A-Z ]*PRIVATE
    KEY-----/);
    my @bad;
    for my $f (grep { length } split /\n/, out('git', 'ls-files')) {
        next if $f eq '.gitlab-ci.yml' || $f eq 'ci/job.pl';
        my $t = slurp($f) // next;
        next if $t =~ /\x00/;                  # binary
        my $n = 0;
        for my $l (split /\n/, $t) { $n++; for my $p (@pat) { push @bad, "$f:$n: matches $p" if
        $l =~ $p } }
    }
    if (@bad) { print "$_\n" for @bad; print "possible secret committed: remove it, rotate it,
    then rewrite the branch before it merges\n"; return 1 }
    print "no secrets found in " . scalar(grep { length } split /\n/, out('git', 'ls-files')) . "
    tracked files\n";
    0;
};
sub is_doors {                                 # a file reqif2sysml wrote from the DOORS export
(whatever folder the layout puts it in)
```

```
    my $f = shift; open my $fh, '<', $f or return 0; my $l = <$fh> // ''; close $fh; $l =~ m{^//
    generated by reqif2sysml\.pl} ? 1 : 0 }
sub model_side {                            # the model without the DOORS export's own
conversion (by its header): what trace
    my ($root) = @_;                        # compares DOORS against -- comparing DOORS with
    its own copy would always agree
    require File::Temp;
    my $d = File::Temp::tempdir('ci-model-XXXXXX', TMPDIR => 1, CLEANUP => 1);
    File::Find::find({ no_chdir => 1, wanted => sub {
        return unless -f && /\.sysml$/;
        (my $r = $File::Find::name) =~ s{^\Q$root\E/?}{};
        return if is_doors($File::Find::name);
        (my $dir = "$d/$r") =~ s{/[^/]+$}{};
        make_path($dir); copy($File::Find::name, "$d/$r");
    } }, $root) if -d $root;
    $d;
}
sub trace_counts {                          # (report text) -> { orphans, unverified, drift,
linked, ids => { kind => {id=>1} } }
    my $t = shift // q{};
    my %c = (orphans => 0, unverified => 0, drift => 0, linked => 0);
    $c{orphans} = $1 if $t =~ /^F orphan requirements[^\n]*?(\d+)\s*$/m;
    $c{unverified} = $1 if $t =~ /^G unverified requirements[^\n]*?(\d+)\s*$/m;
    $c{drift} = $1 if $t =~ /^text differs from DOORS\s+(\d+)\s*$/m;
    ($c{linked}, $c{total}) = ($1, $2) if $t =~ /^C requirements linked[^\n]*?(\d+) of
    (\d+)\s*$/m;
    for my $k (qw(orphans unverified drift)) { my ($l) = $t =~ /^$k: ([^\n]*)$/m; $c{ids}{$k} = {
    map { $_ => 1 } grep { !/^(?:none|-)$/ } split q{ }, $l // q{} } }
    \%c;
}
sub trace_junit_inherited {                 # gaps this change did not add: skipped tests, not
failures; $new = { orphans => [ids], ... } stays failed
    my ($why, $new) = @_;
    my $x = slurp('trace.xml') // return;
    my %keep; for my $k (keys %{ $new // {} }) { $keep{"trace $k"}{$_} = 1 for @{ $new->{$k} } }
    my %left;
    $x =~ s{(<testcase classname="([^"]*)" name="([^"]*)"[^>]*>\s*)<failure
    message="([^"]*)"[^>]*>.*?</failure>}{
        my ($head, $suite, $id, $msg) = ($1, $2, $3, $4);
        if ($keep{$suite}{$id}) { $left{$suite}++; $& } else { "$head<skipped message=\"$why:
        $msg\"/>" } }gse;
    $x =~ s{(<testsuite name="([^"]*)" tests="\d+") failures="(\d+)"}{my $f = $left{$2} // 0; "$1
    failures=\"$f\" skipped=\"" . ($3 - $f) . "\""}ge;
    my $all = 0; $all += $_ for values %left;
    $x =~ s{(<testsuites name="[^"]*" tests="\d+") failures="\d+"}{$1 failures="$all"};
    spew('trace.xml', $x);
}
$run{trace} = sub {                         # DOORS against the model, from the newest imported
drop; a RATCHET on merge requests
    my ($d) = reverse imports();
    my ($reqif) = $d ? grep { -f } map { "$d/$_" } grep { /\.reqifz?$/i } ls($d) : ();
    if (!$reqif) { spew('trace.txt', "no prime drop imported yet\n"); print "no prime drop
    imported yet\n"; return 0 }
    perl_tool('trace.txt', 'bin/reqif2sysml.pl', '--report', '--model', model_side('model'),
    $reqif);
    reports('trace', '--junit', 'trace.xml', '--metrics', 'trace.metrics', 'trace.txt');
    my $now = trace_counts(slurp('trace.txt'));
    printf "trace gaps: %d orphan(s), %d unverified, %d drifted\n", @$now{qw(orphans unverified
    drift)};
```

```
    if (($ENV{TRACE_MODE} // '') eq 'report') { trace_junit_inherited('reported, not gated
    (TRACE_MODE=report)'); print "TRACE_MODE=report: gaps are reported, never fatal\n"; return 0
    }
    if (($ENV{CI_PIPELINE_SOURCE} // '') ne 'merge_request_event') {
    trace_junit_inherited('reported, not gated (not a merge request)'); print "not a merge
    request: reported only\n"; return 0 }
    # the ratchet: the same DOORS export against the model as it was where this branch left the
    target branch
    my $base = $ENV{CI_MERGE_REQUEST_DIFF_BASE_SHA} // '';
    $base = (out('git', 'merge-base', 'HEAD', "origin/$ENV{CI_MERGE_REQUEST_TARGET_BRANCH_NAME}")
    =~ /([0-9a-f]{40})/)[0] // '' if $base eq '' && $ENV{CI_MERGE_REQUEST_TARGET_BRANCH_NAME};
    if ($base eq '') { trace_junit_inherited('reported, not gated (no merge base)'); print "no
    merge base known: reported only\n"; return 0 }
    require File::Temp;
    my $wt = File::Temp::tempdir('ci-trace-XXXXXX', TMPDIR => 1, CLEANUP => 1);
    rmdir $wt;
    run(undef, 'git', 'worktree', 'add', '-q', '--detach', $wt, $base) and do {
    trace_junit_inherited('reported, not gated'); print "cannot check out the merge base:
    reported only\n"; return 0 };
    my $was = -d "$wt/model" ? do { local $QUIET = 1; perl_tool(q{trace-base.txt},
    'bin/reqif2sysml.pl', '--report', '--model', model_side("$wt/model"), $reqif) } : -1;
    run(undef, 'git', 'worktree', 'remove', '--force', $wt);
    my $old = $was == -1 ? undef : trace_counts(slurp(q{trace-base.txt}));
    # the target branch's numbers in one labelled line (its full report is the artifact
    trace-base.txt, not this log)
    print !$old ? "target branch: no model folder (nothing to compare)\n"
        : !$old->{linked} ? sprintf("target branch: 0 of %d linked (no model yet)\n",
        $old->{total} // 0)
        : sprintf("target branch: %d of %d linked; %d orphan(s), %d unverified, %d drifted (full
        report: trace-base.txt)\n", @$old{qw(linked total orphans unverified drift)});
    # like with like: a target branch that has no model side yet (a release MR into main before
    the first baseline, or a
    # branch older than the first import) has nothing to compare -- every gap would look new.
    Report only.
    if (!$old || !$old->{linked}) { trace_junit_inherited('reported, not gated (the target branch
    has no model)'); print "the target branch has no model to compare with: reported only\n";
    return 0 }
    # a gap is new only if this requirement did not have it on the target branch (sets of ids,
    not counts)
    my %new = map { my $k = $_; ($k => [ sort grep { !$old->{ids}{$k}{$_} } keys %{
    $now->{ids}{$k} } ]) } qw(orphans unverified drift);
    my @worse = grep { @{ $new{$_} } } qw(orphans unverified drift);
    trace_junit_inherited('inherited from the target branch', \%new);
    if (@worse) { print "this change adds trace gaps: " . join(q{, }, map { "$_ $old->{$_} ->
    $now->{$_} (new: @{ $new{$_} })" } @worse) . " (see trace.txt)\n"; return 1 }
    print "no new trace gaps (target branch: " . join(', ', map { "$_ $old->{$_}" } qw(orphans
    unverified drift)) . ")\n" unless $was == -1;
    0;
};
sub csv_rows {                                # a small CSV reader: quoted fields, "" escapes; ->
list of hashes keyed by the header
    my $t = slurp(shift) // return ();
    my @rows;
    for my $line (grep { /\S/ } split /\r?\n/, $t) {
        my @f; while ($line =~ /\G(?:"((?:[^"]|"")*)"|([^,]*))(?:,|$)/g) { push @f, defined $1 ?
        $1 =~ s/""/"/gr : $2; last if pos($line) >= length $line }
        push @rows, \@f;
    }
    my $h = shift @rows or return ();
```

```
    map { my $r = $_; +{ map { ($h->[$_] => $r->[$_] // '') } 0 .. $#$h } } @rows;
}
sub vcrm_data {                                 # the VCRM: every requirement, its method, its
cases (verify), their latest verdicts
    my (%req, %name2id, %cases);
    for my $f (sysml_files()) {
        my $t = slurp($f);
        while ($t =~ /\brequirement\s+(?:def\s+)?(\w+)\s*\{([^{}]*)\}/g) {
            my ($name, $body) = ($1, $2);
            my ($id) = $body =~ /doorsId\s*=\s*"([^"]+)"/ or next;
            $name2id{$name} = $id;
            my ($vm) = $body =~ /verificationMethod\s*=\s*"([^"]+)"/;
            my ($doc) = $body =~ m{doc\s*/\*\s*(.*?)\s*\*/}s;
            my $r = $req{$id} //= { id => $id, method => '', text => '' };
            $r->{in_doors} = 1 if is_doors($f);
            $r->{method} = $vm if $vm && (!$r->{method} || is_doors($f));      # DOORS is
            authoritative (D-4)
            $r->{text} = $doc if defined $doc && (!$r->{text} || is_doors($f));
        }
    }
    for my $f (sysml_files()) {
        my $t = slurp($f);
        while ($t =~ /\bverification\s+def\s+(\w+)\s*\{/g) {
            my $case = $1;
            my $depth = 1; my $i = pos($t); my $start = $i;
            while ($depth && $i < length $t) { my $c = substr($t, $i++, 1); $depth++ if $c eq
            '{'; $depth-- if $c eq '}' }
            my $body = substr($t, $start, $i - $start);
            for my $n ($body =~ /\bverify\s+(?:requirement\s+)?(\w+)\s*;/g) { my $id =
            $name2id{$n} // next; $cases{$id}{$case} = 1 }
        }
    }
    my %res;                                    # (requirement, case) -> the newest row, from every
    verification/*.csv
    if (opendir my $dh, 'verification') {
        for my $f (sort grep { /\.csv$/ } readdir $dh) {
            for my $r (csv_rows("verification/$f")) {
                next unless $r->{requirement} && $r->{case};
                my $k = "$r->{requirement}\0$r->{case}";
                $res{$k} = { %$r, source => "verification/$f" } if !$res{$k} || ($r->{timestamp}
                // '') ge ($res{$k}{timestamp} // '');
                $cases{ $r->{requirement} }{ $r->{case} } //= 1 if $req{ $r->{requirement} };
            }
        }
        closedir $dh;
    }
    my @rows;
    my $doors = grep { $_->{in_doors} } values %req;  # with DOORS imported, the VCRM lists
    DOORS's requirements (the baseline, D-4)
    for my $id (sort grep { !$doors || $req{$_}{in_doors} } keys %req) {
        my @c = sort keys %{ $cases{$id} // {} };
        my @v = map { $res{"$id\0$_"} } @c;
        my $status = !@c ? 'no case'
                   : (grep { $_ && $_->{verdict} eq 'failed' } @v) ? 'failed'
                   : (@v && !grep { !$_ || $_->{verdict} ne 'passed' } @v) ? 'verified' : 'not
                   run';
        push @rows, { %{ $req{$id} }, cases => \@c, results => \@v, status => $status };
    }
    \@rows;
```

```
}
sub xmlq { my $t = shift // ''; $t =~ s/&/&amp;/g; $t =~ s/</&lt;/g; $t =~ s/>/&gt;/g; $t =~
s/"/&quot;/g; $t }
$run{vcrm} = sub {                          # the verification cross-reference matrix, from the
model and verification/*.csv
    my $rows = vcrm_data();
    my %n; $n{ $_->{status} }++ for @$rows;
    my $tot = @$rows;
    my $withres = grep { grep { $_ } @{ $_->{results} } } @$rows;
    my $csv = "requirement,method,cases,status,verdicts,evidence\n";
    my $html = "<!DOCTYPE html>\n<html lang=\"en\"><head><meta
    charset=\"utf-8\"><title>VCRM</title>\n<style>body{font:14px sans-serif;margin:1em}"
             . "table{border-collapse:collapse}td,th{border:1px solid #999;padding:2px
             6px;text-align:left}.failed{background:#f3c6c6}.verified{background:#cfe8cf}</style>
             </head><body>\n"
             . "<h1>Verification cross-reference matrix</h1>\n<p>" . ($ENV{CI_COMMIT_SHORT_SHA} ?
             "Commit $ENV{CI_COMMIT_SHORT_SHA}. " : '')
             . "$tot requirements: " . join(', ', map { "$_ " . ($n{$_} // 0) } 'verified',
             'failed', 'not run', 'no case') . ".</p>\n"
             . "<table><tr><th>Requirement</th><th>Method</th><th>Cases</th><th>Status</th><th>Ve
             rdicts
             (newest)</th></tr>\n";
    my $junit = '';
    for my $r (@$rows) {
        my $verd = join('; ', map { my ($c, $v) = ($r->{cases}[$_], $r->{results}[$_]); $v ? "$c
        $v->{verdict} ($v->{environment}, $v->{timestamp}" . ($v->{signed_by} ? ",
        $v->{signed_by}" : '') . ')' : "$c no result" } 0 .. $#{ $r->{cases} });
        my $ev = join(' ', sort { $a cmp $b } keys %{ { map { ($_->{source} => 1) } grep { $_ }
        @{ $r->{results} } } });
        $csv .= join(',', map { my $x = $_; $x =~ /[",]/ ? '"' . ($x =~ s/"/""/gr) . '"' : $x }
        $r->{id}, $r->{method}, join(' ', @{ $r->{cases} }), $r->{status}, $verd, $ev) . "\n";
        (my $cls = $r->{status}) =~ s/\s+/-/g;
        $html .= "<tr class=\"$cls\"><td>" . xmlq($r->{id}) . "</td><td>" . xmlq($r->{method}) .
        "</td><td>" . xmlq(join(', ', @{ $r->{cases} })) . "</td><td>" . xmlq($r->{status}) .
        "</td><td>" . xmlq($verd) . "</td></tr>\n";
        $junit .= qq(  <testcase classname="vcrm" name=") . xmlq($r->{id}) . '"'
                . ($r->{status} eq 'failed' ? ">\n    <failure message=\"" . xmlq($verd) .
                "\"/>\n  </testcase>\n"
                 : $r->{status} eq 'verified' ? "/>\n" : ">\n    <skipped message=\"" .
                 xmlq($r->{status} eq 'no case' ? 'no verification case' : 'no result yet') .
                 "\"/>\n  </testcase>\n");
    }
    $html .= "</table>\n</body></html>\n";
    spew('vcrm.csv', $csv);
    spew('vcrm.html', $html);
    spew('vcrm.xml', qq(<?xml version="1.0" encoding="UTF-8"?>\n<testsuites>\n<testsuite
    name="vcrm" tests="$tot" failures=") . ($n{failed} // 0) . qq(" skipped=") . (($n{'not run'}
    // 0) + ($n{'no case'} // 0)) . qq(">\n$junit</testsuite>\n</testsuites>\n));
    spew('vcrm.metrics', sprintf("vcrm_requirements %d\nvcrm_verified %d\nvcrm_verified_pct
    %.1f\nvcrm_failed %d\nvcrm_cases_with_result %d\n",
                                 $tot, $n{verified} // 0, $tot ? 100 * ($n{verified} // 0) / $tot
                                 : 0, $n{failed} // 0, $withres));
    printf "VCRM: %d requirements, %d verified, %d failed, %d not run, %d without a case
    (vcrm.html, vcrm.csv)\n",
        $tot, map { $n{$_} // 0 } 'verified', 'failed', 'not run', 'no case';
    0;                                      # reports; the trace ratchet is the gate for
    missing cases
};
```

```
sub harness {                                # sil-regression / hil-test: run the configured
harness, keep its results in the one schema
    my ($kind, $var) = @_;
    my $cmd = $ENV{$var} // '';
    if ($cmd eq '') { print "$kind: no harness configured (set $var to the command that runs it;
    it must write results-$kind.csv\n"
                            . "with the columns
                            requirement,case,verdict,commit,environment,timestamp,signed_by)\n";
                            return 0 }
    my $rc = run("$kind.txt", split ' ', $cmd);
    if (-f "results-$kind.csv") { make_path('verification'); copy("results-$kind.csv",
    "verification/results-$kind.csv"); $run{vcrm}->() }
    $rc;
}
$run{'sil-regression'} = sub { harness('sil', 'SIL_CMD') };
$run{'hil-test'} = sub { harness('hil', 'HIL_CMD') };
$run{intake} = sub {                         # a drop the prime's Jenkins delivered: fetch it,
check it against its MANIFEST
    my $ver = $ENV{DROP_VERSION} // '';
    if ($ver !~ /^[\w.+-]+$/) { print "intake needs DROP_VERSION (the trigger's variable), got
    '$ver'\n"; return 1 }
    my $dir = "intake/$ver";
    make_path($dir);
    my $fetch = sub {                        # one file of prime-drop/<ver>: from the package
    registry, or a folder (laptop, emulator)
        my $f = shift;
        if ($ENV{INTAKE_DROP_DIR}) { return copy("$ENV{INTAKE_DROP_DIR}/$f", "$dir/$f") ? 0 : 1 }
        if ($ENV{GITLAB_LOCAL_REGISTRY}) {    # the local GitLab emulator: its Generic Package
        Registry is a folder, NAME/VERSION/FILE
            my $src = "$ENV{GITLAB_LOCAL_REGISTRY}/prime-drop/$ver/$f";
            return -f $src && copy($src, "$dir/$f") ? 0 : 1;
        }
        if (!$ENV{CI_API_V4_URL} || !$ENV{CI_JOB_TOKEN}) { print "no package registry and no
        INTAKE_DROP_DIR: nothing to fetch\n"; return 1 }
        run(undef, 'curl', '--fail', '--silent', '--show-error', '--header', "JOB-TOKEN:
        $ENV{CI_JOB_TOKEN}", '--output', "$dir/$f",
            "$ENV{CI_API_V4_URL}/projects/$ENV{CI_PROJECT_ID}/packages/generic/prime-drop/$ver/$f
            ");
    };
    if ($fetch->('MANIFEST.txt')) { print "prime-drop $ver: no MANIFEST.txt\n"; return 1 }
    my $man = slurp("$dir/MANIFEST.txt"); $man =~ s/\r\n/\n/g;
    my @want = $man =~ /^\s+([0-9a-f]{64})\s+(\S+)\s*$/mg;
    my ($bad, $report) = (0, "prime-drop $ver\n");
    $report .= "$_\n" for grep { /^(Built by|Prime repository commit|Tools|Delivered):/ } split
    /\n/, $man;
    while (my ($sum, $f) = splice @want, 0, 2) {
        if ($fetch->($f)) { $report .= "MISSING  $f\n"; $bad++; next }
        my $got = sha256_file("$dir/$f");
        if ($got eq $sum) { $report .= "ok       $f  sha256 $sum\n" } else { $report .= "MISMATCH
        $f  manifest $sum, received $got\n"; $bad++ }
    }
    $report .= $bad ? "REJECTED: $bad file(s) missing or altered -- do not import; tell the
    prime\n"
                    : "ACCEPTED for import: the drop is exactly what the prime's build
                    recorded\n";
    spew('intake.txt', $report);
    print $report;
    return 1 if $bad;
```

```
    if ($ENV{INTAKE_TOKEN} && $ENV{CI_SERVER_URL}) {   # open the Prime Drop issue that starts
    the vendor import (a project access token, masked)
        return perl_tool(undef, 'bin/gitlab.pl', '--url', $ENV{CI_SERVER_URL}, '--token-env',
        'INTAKE_TOKEN', '--project', $ENV{CI_PROJECT_PATH},
                         'issue', 'new', "Prime drop $ver", '--template', 'Prime Drop',
                         '--label', 'type::drop', '--label', 'owner::gov');
    }
    print "INTAKE_TOKEN is not set: open the Prime Drop issue by hand (the drop is verified)\n";
    0;
};
# ------------------------------------------------------------------ complexity: the spider-web
guardrails
# The dependency graph of the model's root packages: an edge P -> Q when P imports Q, or names an
element Q defines (a
# type, a relationship end) -- unique names only. fan-out(P) = distinct packages P depends on,
fan-in(P) = distinct
# packages that depend on P, cycles = strongly connected groups of more than one package. Per
part def: relationship
# ends (satisfy, verify, allocate, dependency, connect lines naming it) plus incoming type
references -- the spaghetti
# measure. Layers (sysml.conf `layers = A, B; C; ...`, lowest first): a package may depend only
on its own layer or below.
my $DEFKW = qr/(?:part|port|item|action|attribute|enum|interface|connection|constraint|state|calc
|metadata|occurrence|verification|analysis|use\s+case|requirement|allocation|flow)\s+def/;
sub sysml_conf { my %c; for (split /\n/, slurp('sysml.conf') // '') { next if /^\s*#/; $c{$1} =
$2 if /^\s*([\w.]+)\s*=\s*(.*?)\s*(?:#.*)?$/ } \%c }
sub _gov_file { my $t = shift // ''; $t =~ m{^// Government-owned} ? 1 : 0 }   #
government-authored files say so in their first line
sub model_graph {                           # (\%files: path => text) -> { pkgs => {P => {file,
gov}}, edges => {P => {Q => 1}}, rel => {part => n}, defs => {name => P}, wild => [[P, Q, file]]
}
    my ($files) = @_;
    my (%pkg, %defs, %dup, %body);
    for my $f (sort keys %$files) {
        (my $t = $files->{$f}) =~ s{//[^\n]*}{}g;
        $t =~ s{/\*.*?\*/}{}gs;
        while ($t =~ /^(?:library\s+)?package\s+('[^']+'|\w+)\s*\{/mg) {
            (my $p = $1) =~ s/'//g;
            my $start = pos($t); my $d = 1; my $i = $start;
            while ($d && $i < length $t) { my $c = substr($t, $i++, 1); $d++ if $c eq '{'; $d--
            if $c eq '}' }
            $body{$p} .= substr($t, $start, $i - $start);
            $pkg{$p} = { file => $f, gov => (_gov_file($files->{$f}) ? 1 : 0) };
            pos($t) = $i;
        }
    }
    for my $p (sort keys %body) {
        while ($body{$p} =~ /\b$DEFKW\s+(\w+)/g) { my $n = $1; if (exists $defs{$n} && $defs{$n}
        ne $p) { $dup{$n} = 1 } else { $defs{$n} = $p } }
        while ($body{$p} =~ /\brequirement\s+(\w+)\s*\{/g) { my $n = $1; if (exists $defs{$n} &&
        $defs{$n} ne $p) { $dup{$n} = 1 } else { $defs{$n} = $p } }
    }
    delete $defs{$_} for keys %dup;           # ambiguous names resolve to nothing (both
    converters define TW_SYS_001)
    my (%edges, %rel, @wild);
    for my $p (sort keys %body) {
        my $b = $body{$p};
        while ($b =~ /\bimport\s+('[^']+'|\w+)((?:::[\w*']+)*)\s*;/g) {
            (my $q = $1) =~ s/'//g;
```

```
            my $rest = $2 // '';
            $edges{$p}{$q} = 1 if $pkg{$q} && $q ne $p;
            push @wild, [$p, $q, $pkg{$p}{file}] if $rest =~ /::\*$/;
        }
        while ($b =~ /(?::>?|~|\bby|\bverify|\bsatisfy|\bto|\bfrom|\ballocate|\bexhibit\s+state\s
        +\w+\s*:|\bperform\s+action\s+\w+\s*:)\s*(\w+)/g)
        {
            my $q = $defs{$1} // next;
            $edges{$p}{$q} = 1 if $q ne $p;
        }
        for my $l (split /\n/, $b) {          # relationship ends per part def
            next unless $l =~ /\b(satisfy|verify|allocate|dependency|connect|connection)\b/;
            $rel{$_}++ for grep { ($defs{$_} // '') ne '' } $l =~ /\b(\w+)\b/g;
        }
        while ($b =~ /:>?\s*(\w+)/g) { $rel{$1}++ if $defs{$1} }
    }
    my %parts; for my $p (sort keys %body) { $parts{$1} = 1 while $body{$p} =~
    /\bpart\s+def\s+(\w+)/g }
    %rel = map { ($_ => $rel{$_} // 0) } grep { $parts{$_} } keys %parts;
    { pkgs => \%pkg, edges => \%edges, rel => \%rel, defs => \%defs, wild => \@wild, dup => [
    sort keys %dup ] };
}
sub sccs {                                    # Tarjan: the strongly connected components with
more than one node, each sorted
    my ($edges, @nodes) = @_;
    my ($i, %idx, %low, %on, @st, @out) = (0);
    my $visit; $visit = sub {
        my $v = shift;
        $idx{$v} = $low{$v} = $i++; push @st, $v; $on{$v} = 1;
        for my $w (sort keys %{ $edges->{$v} // {} }) {
            if (!exists $idx{$w}) { $visit->($w); $low{$v} = $low{$w} if $low{$w} < $low{$v} }
            elsif ($on{$w}) { $low{$v} = $idx{$w} if $idx{$w} < $low{$v} }
        }
        if ($low{$v} == $idx{$v}) { my @c; while (1) { my $w = pop @st; $on{$w} = 0; push @c, $w;
        last if $w eq $v } push @out, [ sort @c ] if @c > 1 }
    };
    $visit->($_) for grep { !exists $idx{$_} } sort @nodes;
    [ sort { $a->[0] cmp $b->[0] } @out ];
}
sub complexity_of {                           # (\%files, \%conf) -> measures and violations
    my ($files, $conf) = @_;
    my $g = model_graph($files);
    my @p = sort keys %{ $g->{pkgs} };
    my (%layer, $n);
    for my $grp (split /;/, $conf->{layers} // '') { $n++; for (split /,/, $grp) {
    s/^\s+|\s+$//g; s/'//g; $layer{$_} = $n if length } }
    # fan-in/fan-out leave out the first layer of `layers` (the foundation: Conventions, Types):
    importing the shared
    # vocabulary is what every package should do, like using ScalarValues, not a tangle. Cycles
    and upward
    # dependencies still see every edge.
    my (%in, %out, %fout, %fin);
    for my $a (@p) { for my $b (keys %{ $g->{edges}{$a} // {} }) { $out{$a}{$b} = 1; $in{$b}{$a}
    = 1;
        next if ($layer{$b} // 0) == 1; $fout{$a}{$b} = 1; $fin{$b}{$a} = 1 } }
    my %fo = map { ($_ => scalar keys %{ $fout{$_} // {} }) } @p;
    my %fi = map { ($_ => scalar keys %{ $fin{$_} // {} }) } @p;
    my $cyc = sccs($g->{edges}, @p);
```

```
    my @up = map { my $a = $_; map { [$a, $_] } grep { defined $layer{$a} && defined $layer{$_}
    && $layer{$_} > $layer{$a} } sort keys %{ $out{$a} // {} } } @p;
    my @gw = grep { $g->{pkgs}{ $_->[0] }{gov} } @{ $g->{wild} };
    my $mean = sub { my @v = @_; @v ? sprintf('%.1f', (eval { my $s = 0; $s += $_ for @v; $s }) /
    @v) : '0.0' };
    my @rel = sort { $g->{rel}{$b} <=> $g->{rel}{$a} || $a cmp $b } keys %{ $g->{rel} };
    my %m = (packages => scalar @p, max_fan_out => (sort { $b <=> $a } values %fo)[0] // 0,
    mean_fan_out => $mean->(values %fo),
             max_fan_in => (sort { $b <=> $a } values %fi)[0] // 0, mean_fan_in => $mean->(values
             %fi), cycles => scalar @$cyc,
             upward_imports => scalar @up, gov_wildcard_imports => scalar @gw, duplicates =>
             scalar @{ $g->{dup} },
             max_relationships_per_part => @rel ? $g->{rel}{ $rel[0] } : 0,
             mean_relationships_per_part => $mean->(map { $g->{rel}{$_} } @rel));
    my @v;
    my $mfo = $conf->{max_package_fan_out} // 7;
    push @v, map { [ 'major', $g->{pkgs}{$_}{file}, "package $_ depends on $fo{$_} packages
    (max_package_fan_out $mfo)" ] } grep { $fo{$_} > $mfo } @p;
    my $mrp = $conf->{max_relationships_per_part} // 10;
    push @v, map { [ 'major', $g->{pkgs}{ $g->{defs}{$_} }{file}, "part def $_ has $g->{rel}{$_}
    relationship ends (max_relationships_per_part $mrp)" ] } grep { $g->{rel}{$_} > $mrp } @rel;
    push @v, map { [ 'critical', $g->{pkgs}{ $_->[0] }{file}, 'package dependency cycle: ' .
    join(' -> ', @$_, $_->[0]) ] } @$cyc if @$cyc > ($conf->{max_cycles} // 0);
    push @v, map { [ 'major', $g->{pkgs}{ $_->[0] }{file}, "upward-layer dependency: $_->[0]
    (layer $layer{$_->[0]}) -> $_->[1] (layer $layer{$_->[1]})" ] } @up;
    push @v, map { [ 'major', $_->[2], "wildcard import $_->[1]::* in government file (name what
    you use)" ] } @gw;
    { metrics => \%m, fan_in => \%fi, fan_out => \%fo, cycles => $cyc, upward => \@up, rel =>
    $g->{rel}, rel_order => \@rel, violations => \@v, packages => \@p };
}
sub complexity_worse {                       # (\%old, \%new metrics, \%conf) -> the measures
this change made worse
    # fan-out and relationships per part are sizes, not faults: a model grows, and its first part
    with a sub-part takes
    # the maximum from 0 to 1. They ratchet only above their limits (sysml.conf
    max_package_fan_out,
    # max_relationships_per_part): once past the limit, a change may not make them worse. Cycles,
    upward-layer imports
    # and duplicates are faults: any new one fails.
    my ($old, $new, $conf) = @_;
    my %limit = (max_fan_out => $conf->{max_package_fan_out} // 7, max_relationships_per_part =>
    $conf->{max_relationships_per_part} // 10);
    grep { ($new->{$_} // 0) > ($old->{$_} // 0) && (!exists $limit{$_} || $new->{$_} >
    $limit{$_}) }
        qw(max_fan_out max_relationships_per_part cycles upward_imports duplicates);
}
sub model_files { my $root = shift // 'model'; my %f; File::Find::find({ no_chdir => 1, wanted =>
sub { $f{$File::Find::name} = slurp($File::Find::name) if /\.sysml$/ && -f } }, $root) if -d
$root; \%f }
$run{complexity} = sub {
    my $conf = sysml_conf();
    my $c = complexity_of(model_files('model'), $conf);
    my $txt = sprintf "%-28s %7s %7s\n", 'package', 'fan-in', 'fan-out';
    $txt .= sprintf "%-28s %7d %7d\n", $_, $c->{fan_in}{$_}, $c->{fan_out}{$_} for @{
    $c->{packages} };
    $txt .= "\ncycles: " . (@{ $c->{cycles} } ? join('; ', map { join(' <-> ', @$_) } @{
    $c->{cycles} }) : 'none') . "\n";
    $txt .= "upward-layer dependencies: " . (@{ $c->{upward} } ? join('; ', map { "$_->[0] ->
    $_->[1]" } @{ $c->{upward} }) : 'none') . "\n";
```

```
    $txt .= "\nrelationship ends per part def (top 5):\n" . join('', map { sprintf "  %-26s
    %d\n", $_, $c->{rel}{$_} } grep { defined } @{ $c->{rel_order} }[0 .. 4]);
    $txt .= "\n" . join('', map { "$_ $c->{metrics}{$_}\n" } sort keys %{ $c->{metrics} });
    spew('complexity.txt', $txt);
    print $txt;
    spew('complexity.metrics', join('', map { "complexity_$_ $c->{metrics}{$_}\n" } sort keys %{
    $c->{metrics} }));
    spew('complexity-cq.json', "[\n" . join(",\n", map { my ($sev, $f, $msg) = @$_; (my $m =
    $msg) =~ s/(["\\])/\\$1/g;
        qq(  {"description":"$m","check_name":"complexity","fingerprint":") .
        Digest::SHA::sha1_hex("$f $msg") .
        qq(","severity":"$sev","location":{"path":"$f","lines":{"begin":1}}}) } @{
        $c->{violations} }) . "\n]\n");
    my @hard = grep { $_->[2] =~ /^wildcard import/ } @{ $c->{violations} };
    if (@hard) { print "$_->[1]: $_->[2]\n" for @hard; return 1 }
    if (($ENV{CI_PIPELINE_SOURCE} // '') ne 'merge_request_event') { print "not a merge request:
    reported only\n"; return 0 }
    # A prime drop (gl flow vendor merges vendor/prime into feature/drop-VERSION) brings the
    prime's structure as it
    # is: the government does not edit it to pass a ratchet; what got worse goes back to the
    prime as a Discrepancy
    # Report. The measures above are still reported in the MR; the ratchet applies to the
    government's own changes.
    if (($ENV{CI_MERGE_REQUEST_SOURCE_BRANCH_NAME} // '') =~ m{^feature/drop-}) {
        print "a prime drop: measured and reported, not ratcheted (raise a Discrepancy Report for
        what got worse)\n";
        return 0;
    }
    # a release branch carries develop to main: every change in it was ratcheted on its way into
    develop, and main lags
    # (before the first baseline it has no model at all), so comparing with main would count all
    of it as new
    if (($ENV{CI_MERGE_REQUEST_SOURCE_BRANCH_NAME} // '') =~ m{^release/}) { print "a release:
    its changes were checked on develop; reported only\n"; return 0 }
    my $base = $ENV{CI_MERGE_REQUEST_DIFF_BASE_SHA} // '';
    return 0 if $base eq '';
    require File::Temp;
    my $wt = File::Temp::tempdir('ci-cx-XXXXXX', TMPDIR => 1, CLEANUP => 1); rmdir $wt;
    run(undef, 'git', 'worktree', 'add', '-q', '--detach', $wt, $base) and return 0;
    my $old = complexity_of(model_files("$wt/model"), $conf);
    run(undef, 'git', 'worktree', 'remove', '--force', $wt);
    my @worse = complexity_worse($old->{metrics}, $c->{metrics}, $conf);
    if (@worse) { print "this change makes the model more tangled: " . join(', ', map { "$_
    $old->{metrics}{$_} -> $c->{metrics}{$_}" } @worse) . "\n"; return 1 }
    print "no measure got worse than on the target branch\n";
    0;
};
$run{recipe} = sub {                          # requirement statements graded against the recipe
(tools/sysml/req-recipe.pl); reports only
    return do { print "the kit has no tools/sysml/req-recipe.pl: skipped\n"; 0 } unless -f
    "$KIT/tools/sysml/req-recipe.pl";
    my @in = grep { !is_doors($_) } sysml_files();      # the model side (the coordinator's
    recipe: DOORS's own conversion excluded)
    if (!@in) { print "no model yet\n"; return 0 }
    perl_tool('recipe.txt', 'tools/sysml/req-recipe.pl', '--format', 'text', @in);
    perl_tool(undef, 'tools/sysml/req-recipe.pl', '--format', 'codequality', '-o',
    'recipe-cq.json', @in);
    my ($n, $f, $m) = (slurp('recipe.txt') // '') =~ /requirements=(\d+) findings=(\d+)
    missing=(\d+)\s*$/;
```

```
    spew('recipe.metrics', sprintf("recipe_requirements %d\nrecipe_findings
    %d\nrecipe_missing_parts %d\n", $n // 0, $f // 0, $m // 0));
    0;
};
$run{docs} = sub {                          # generated docs are committed with the model that
made them
    my $rc = perl_tool(undef, 'tools/sysml/model.pl', 'docs');
    return $rc if $rc;
    my $st = out('git', 'status', '--porcelain', '--', 'docs');
    if ($st =~ /\S/) { print $st, "docs/ is stale: run model.pl docs and commit\n"; return 1 }
    print "docs/ is current\n";
    0;
};
$run{delta} = sub {                         # what the prime changed in v1 between the two
newest drops
    my @d = imports();
    if (@d < 2) { spew('delta.txt', "first drop: nothing to compare\n"); print "first drop:
    nothing to compare\n"; return 0 }
    my ($old, $new) = @d[-2, -1];
    perl_tool('delta.txt', 'bin/xmi2sysml.pl', '--delta', "$old/tidewater.xmi",
    "$new/tidewater.xmi");
};
$run{'drop-package'} = sub {                # the newest drop, exactly as delivered, into the
Generic Package Registry
    if (!$ENV{CI_API_V4_URL} || !$ENV{CI_JOB_TOKEN}) { print "no package registry here (a local
    run): skipped\n"; return 0 }
    my ($d) = reverse imports() or do { print "no drop imported\n"; return 0 };
    (my $ver = $d) =~ s{^imports/}{};
    my @f = grep { -f "$d/$_" } ls($d);
    for my $f (@f) {
        print "prime-drop $ver: $f\n";
        my $rc = run(undef, 'curl', '--fail', '--silent', '--show-error', '--header', "JOB-TOKEN:
        $ENV{CI_JOB_TOKEN}", '--upload-file', "$d/$f",
                     "$ENV{CI_API_V4_URL}/projects/$ENV{CI_PROJECT_ID}/packages/generic/prime-dro
                     p/$ver/$f");
        return 1 if $rc;
    }
    0;
};
sub toolchain {                             # the tools that produced (and can reproduce) a
baseline -- an inventory for the audit
    my $kit = kit_version();
    my ($perl) = sprintf('perl %vd', $^V);
    my ($git) = out('git', '--version') =~ /(git version \S+)/;
    join '', map { "$_\n" } "baseline: " . ($ENV{CI_COMMIT_TAG} // ''), "commit: " . (out('git',
    'rev-parse', 'HEAD') =~ /([0-9a-f]{40})/)[0],
        "kit: $kit", $perl, $git // 'git: unknown', "os: " . ($ENV{CI_RUNNER_EXECUTABLE_ARCH} //
        $^O), "runner: " . ($ENV{CI_RUNNER_DESCRIPTION} // 'unknown'),
        "image: " . ($ENV{CI_JOB_IMAGE} ? "$ENV{CI_JOB_IMAGE}" . ($ENV{CI_JOB_IMAGE_DIGEST} ?
        "\@$ENV{CI_JOB_IMAGE_DIGEST}" : '') : 'none (shell runner)');
}
$run{baseline} = sub {                      # the tagged tree rebuilds its docs, and the hash
matches the tag message
    my $tag = $ENV{CI_COMMIT_TAG} // '';
    if ($tag eq '') { print "baseline runs on a tag (CI_COMMIT_TAG is empty)\n"; return 1 }
    my $rc = perl_tool(undef, 'tools/sysml/model.pl', 'docs');
    return $rc if $rc;
    if (out('git', 'status', '--porcelain', '--', 'docs') =~ /\S/) { print "the tagged tree does
    not rebuild its docs\n"; return 1 }
```

```
    my $hash = docs_hash('HEAD');
    my $msg = out('git', 'tag', '-l', '--format=%(contents)', $tag);
    if (index($msg, "sha256=$hash") < 0) { print "docs hash $hash is not the one recorded in
    $tag\n"; return 1 }
    spew('toolchain.txt', toolchain() . "docs sha256: $hash\n");
    $run{vcrm}->();                          # the baseline's VCRM, kept with the baseline
    (expire_in: never)
    my $ms = sprint_of((out('git', 'log', '-1', '--format=%cs', 'HEAD') =~
    /(\d{4}-\d\d-\d\d)/)[0]);
    spew('release.env', "RELEASE_MILESTONE=" . ($ms // '') . "\n");   # a dotenv report: the
    release names the sprint it closes
    print "$tag rebuilds identically (sha256=$hash)\n", slurp('toolchain.txt');
    0;
};
sub sprint_of {                              # the sprint milestone a date falls in, from
SPRINT_START (a Monday) and SPRINT_WEEKS
    my $d = shift // return undef;
    my $s = $ENV{SPRINT_START} // return undef;
    require Time::Local;
    my $e = sub { my ($y, $m, $dd) = split /-/, shift; Time::Local::timegm(0, 0, 12, $dd, $m - 1,
    $y) };
    my $days = ($e->($d) - $e->($s)) / 86400;
    return undef if $days < 0;
    ($ENV{SPRINT_PREFIX} // 'Sprint ') . (int($days / (7 * ($ENV{SPRINT_WEEKS} || 2))) + 1);
}
$run{'publish-package'} = sub {              # the baseline as a package for downstream users:
model + docs, VCRM, toolchain
    my $tag = $ENV{CI_COMMIT_TAG} // '';
    if ($tag eq '') { print "publish-package runs on a tag (CI_COMMIT_TAG is empty)\n"; return 1
    }
    my $dir = "package-upload/tidewater-model/$tag";
    make_path($dir);
    my $tgz = "tidewater-model-$tag.tar.gz";
    my $rc = run(undef, 'git', 'archive', '--format=tar.gz', "--prefix=tidewater-model-$tag/",
    '-o', "$dir/$tgz", $tag, '--', 'model', 'docs');
    return $rc if $rc;
    for my $f (grep { -f } 'vcrm.csv', 'toolchain.txt') { copy($f, "$dir/$f") }   # from the
    baseline job (needs: baseline)
    my @files = grep { -f "$dir/$_" } $tgz, 'vcrm.csv', 'toolchain.txt';
    spew("$dir/SHA256SUMS", join('', map { sha256_file("$dir/$_") . "  $_\n" } @files));
    push @files, 'SHA256SUMS';
    print "tidewater-model $tag: @files\n";
    if (!$ENV{CI_API_V4_URL} || !$ENV{CI_JOB_TOKEN}) { print "no package registry here: the files
    are in $dir (a local run)\n"; return 0 }
    for my $f (@files) {
        my $rc2 = run(undef, 'curl', '--fail', '--silent', '--show-error', '--header',
        "JOB-TOKEN: $ENV{CI_JOB_TOKEN}", '--upload-file', "$dir/$f",
                      "$ENV{CI_API_V4_URL}/projects/$ENV{CI_PROJECT_ID}/packages/generic/tidewate
                      r-model/$tag/$f");
        return 1 if $rc2;
    }
    0;
};
$run{'publish-release'} = sub {              # the release notes: the tag's section of
CHANGELOG.md, then the toolchain
    my $tag = $ENV{CI_COMMIT_TAG} // '';
    my ($on, $notes) = (0, '');
    (my $cl = slurp('CHANGELOG.md') // '') =~ s/\r\n/\n/g;
```

```
    $cl =~ s/<!--.*?-->//gs;                 # the scaffold's instructions are an HTML comment:
    never release notes
    for my $l (split /\n/, $cl) {              # exactly the `## [TAG]` section: up to the next
    `## ` heading or the link lines
        if ($l =~ /^## /) { $on = ($l =~ /^## \[\Q$tag\E\]/) ? 1 : 0; next }
        next if $l =~ /^\[[^\]]+\]:\s/;      # [baseline-1]: ../-/tags/baseline-1
        $notes .= "$l\n" if $on;
    }
    1 while $notes =~ s/^#{3,6} [^\n]*\n(?:[ \t]*\n)*(?=#{2,6} |\z)//m;   # headings with nothing
    under them (### Fixed, left empty)
    $notes =~ s/\n{3,}/\n\n/g;
    $notes =~ s/^\s+|\s+$//g;
    $notes = "No CHANGELOG section for $tag." unless length $notes;
    if (-f 'vcrm.csv') { my @r = grep { /\S/ } split /\n/, slurp('vcrm.csv'); shift @r; my $v =
    grep { /,verified,/ } @r;
        $notes .= "\n\n## Verification\n\n" . scalar(@r) . " requirements, $v verified (VCRM:
        vcrm.html and vcrm.csv, kept with this release's pipeline)." }
    my $tc = slurp('toolchain.txt');          # from the baseline job (needs: baseline)
    $notes .= "\n\n## Toolchain\n\n```text\n$tc```" if defined $tc && length $tc;
    spew('release-notes.md', "$notes\n");
    print "$notes\n";
    0;
};
$run{pages} = sub {                           # the model's web site: docs, views, trace,
burn-down
    make_path('public/views', 'public/docs');
    if (-d 'docs') { File::Find::find({ no_chdir => 1, wanted => sub { return unless -f; (my $r =
    $File::Find::name) =~ s{^docs/?}{}; my $to = "public/docs/$r";
                                                                  (my $dir = $to) =~
                                                                  s{/[^/]+$}{}; make_path($dir);
                                                                  copy($File::Find::name, $to) }
                                                                  }, 'docs') }
    for my $v (qw(tree trace ibd pkg)) {          # exit 1 = drawn, with findings (the drawing
    shows them); more than 1 = not drawn
        my $rc = perl_tool("view-$v.txt", 'tools/sysml/model.pl', 'draw', $v, '-o',
        "public/views/$v.svg");
        next if $rc <= 1 && -f "public/views/$v.svg";
        spew("public/views/$v.txt", "view $v not drawn (exit $rc): see the job log\n");
        print "view $v not drawn (exit $rc)\n";
        unlink "public/views/$v.svg";
    }
    unlink glob('view-*.txt');
    if (-d 'docs/idef0') {                         # the IDEF0 plates model.pl docs generated
        make_path('public/idef0');
        copy($_, 'public/idef0/') for grep { -f } glob('docs/idef0/svg/*.svg'),
        'docs/idef0/plates.html';
    }
    my ($d) = reverse imports();
    my ($reqif) = $d ? grep { -f } "$d/tidewater.reqif" : ();
    perl_tool('public/trace.txt', 'bin/reqif2sysml.pl', '--report', '--model',
    model_side('model'), $reqif) if $reqif;
    if (-d 'verification' || grep { /verification\s+def/ } map { slurp($_) } sysml_files()) {
    $run{vcrm}->(); copy($_, "public/$_") for grep { -f } 'vcrm.html', 'vcrm.csv' }
    if (grep { /complexity/ } slurp('.gitlab-ci.yml') // '') {   # the spider-web trend: the last
    12 days with a merge
        $run{complexity}->();
        copy('complexity.txt', 'public/complexity.txt');
    }
```

```
    my (%day, @order);                          # one point per day: the last first-parent commit
    of that day
    for my $l (grep { length } split /\n/, out('git', 'log', '--first-parent', '--reverse',
    '--format=%H %cs', 'HEAD')) {
        my ($sha, $date) = split ' ', $l;
        push @order, $date unless exists $day{$date};
        $day{$date} = $sha;
    }
    my $csv = "date,todo_lines\n";
    for my $date (@order) {
        my $n = 0;
        $n += (split /:/)[-1] for grep { length } split /\n/, out('git', 'grep', '-c', '// TODO',
        $day{$date}, '--', 'model');
        $csv .= "$date,$n\n";
    }
    spew('public/todo-burndown.csv', $csv);
    if (-f 'public/complexity.txt') {
        my $tr = "date,cycles,max_fan_out,max_relationships_per_part,upward_imports\n";
        my $conf = sysml_conf();
        for my $date (grep { defined } @order[ ($#order > 11 ? $#order - 11 : 0) .. $#order ]) {
            my %f;
            for my $path (grep { /\.sysml$/ } split /\n/, out('git', 'ls-tree', '-r',
            '--name-only', $day{$date}, '--', 'model')) {
                $f{$path} = out('git', 'show', "$day{$date}:$path");
            }
            my $m = complexity_of(\%f, $conf)->{metrics};
            $tr .= join(',', $date, @$m{qw(cycles max_fan_out max_relationships_per_part
            upward_imports)}) . "\n";
        }
        spew('public/complexity-trend.csv', $tr);
    }
    perl_tool(undef, 'tools/gitlab/pages-index.pl', '--title', 'Tidewater model', 'public');
};
$run{status} = sub {                            # weekly: what GitLab knows, in the status-metrics
formats
    if (!$ENV{STATUS_TOKEN}) { print "STATUS_TOKEN is not set: nothing to export (set it under
    Settings > CI/CD > Variables)\n"; return 0 }
    my $rc = perl_tool(undef, 'bin/gitlab.pl', '--url', $ENV{CI_SERVER_URL}, '--token-env',
    'STATUS_TOKEN', '--project', $ENV{CI_PROJECT_PATH}, 'export', 'exports');
    return $rc if $rc;
    return perl_tool(undef, 'bin/status-metrics.pl', 'collect', '--config', 'metrics.json',
    '--history', 'exports/history.jsonl') if -f 'metrics.json';
    0;
};

exit main(@ARGV) unless caller;   # `require` it to use the functions (tests/ci-job.t)
1;
```

20.15 Exercises

Exercise 20.1: Which jobs, which event

For each event in the "which jobs run" table, run `gitlab.pl ci --list` with the matching `--event`, `--branch`, `--tag` options in your practice repository and compare the result with the table. Which job's rule surprised you?

Exercise 20.2: Turn the ratchet

Fix one outline violation in your practice model (give a one-child part a second child, or split a package). Run the `outline` job. What does it tell you? Make the change it asks for and run it again.

Exercise 20.3: Find your ratchet

Name a rule in your own organization that everyone agrees with and nobody meets today. Write the ratchet for it: what is counted, where the allowed number lives, who may lower it, who may raise it.

Exercise 20.4: Stale documents

Change a part's documentation comment in your practice model, commit without running `model.pl docs`, and run the `docs` job. Then fix it. Why does the team commit generated files at all?

Exercise 20.5: Review a pipeline change

A merge request adds `allow_failure: true` to the `lint` job with the description "lint is noisy this week". Write the review comment Tom should leave, and the alternative you would propose.

20.16 Applying this to your project

- **Copy**: one-line job scripts that call a script in the repository (`perl ci/job.pl NAME`); `GIT_DEPTH: "0"` if any job reads history; a `workflow:` that gives one pipeline per change; a shared rule block; `artifacts: when: always` for reports; the zero-over-zero lint gate; the ratchet for inherited problems; a trace job that is allowed to fail; committed, checked generated documents; a baseline job that rebuilds and checks a checksum.
- **Adapt**: the checks to your modeling rules; which jobs block and which warn; the schedules to your reporting rhythm; the site to your audience.
- **Decide locally**: where the kit lives on the runner (the `KIT` variable); whether you have a Docker runner for GitLab's scanners; who owns the pipeline file; who owns the schedules.

CHAPTER

21

Runners: where the pipeline runs

From what you know →

You know

the lab machine that runs the analysis tools.

→ Here it is called

a runner: the machine GitLab sends pipeline jobs to.

→ What is new

it works unattended, and if it is offline, nothing merges.

Survival kit

What you have

a pipeline file (Chapter 20, The pipeline, job by job (p. 302)) and a GitLab project (Chapter 15, Setting up GitLab (p. 226)).

What you need

a computer that actually runs the jobs. This chapter explains what a runner is, the kinds GitLab offers, why the Tidewater team chose a project runner with the Windows shell executor on a dedicated virtual machine, how it was registered and secured, what goes wrong ("pending", "stuck", offline), and how the team kept the same pipeline file working on Windows, on Linux, in Docker and on a laptop.

21.1 GitLab runs nothing

The most common misunderstanding about GitLab, and the one that cost Tom the most time in the first weeks, is that the GitLab server runs the pipeline. It does not. The server keeps the repository, decides which jobs a pipeline needs, and waits. A separate program, the **GitLab Runner**, installed on some other computer, asks the server every few seconds "is there a job for me?", takes one, runs it, sends the log and the artifacts back, and asks again.

In systems engineering terms, the server is the configuration management office that schedules

the audit, and the runner is the auditor who shows up and does it. No auditor, no audit: a pipeline with no runner to take its jobs sits in the state GitLab calls **pending**, indefinitely. With "pipelines must succeed" switched on, that means no merge request can merge. An offline runner stops the whole team.

That is why the request to the enterprise administrators in Chapter 15, Setting up GitLab (p. 226) asked for a runner on day one, and why Tom's stand-up said "blocked" until it existed.

21.2 Who a runner serves: scopes

Table 21.1: Runner scopes

Kind	Who registers it	Who it serves	Typical use
Instance (shared) runner	the GitLab administrators	every project on the server	general-purpose jobs, usually Linux with Docker
Group runner	a group owner	every project in the group	a team's projects with shared needs
Project runner	a project maintainer	one project (or a few it is enabled for)	special tools, special security rules
GitLab.com hosted runner	GitLab	GitLab.com projects	no machine to maintain; billed in compute minutes

The Tidewater team used a **project runner**, registered to `tidewater/model`. Three reasons:

- **The tools.** The jobs need Perl and the agile kit. The organization's shared runners were Linux machines running Docker containers built for software teams, with no kit and no way for the team to add one in the first quarter.
- **The security conversation.** The ISSM wanted one machine, one service account, one purpose, a list of exactly what runs on it, and no other project's jobs on it. A project runner is that list.
- **The cost.** On the organization's self-managed GitLab, a runner on the team's own virtual machine costs nothing per minute; on GitLab.com, the hosted runners bill compute minutes after a free quota (Section 17.7, Gates, and what they cost (p. 263)).

21.3 Hosted or your own: the decision

Many readers will not have a choice: the organization will have decided where pipelines may run. If you do have one, these are the questions that decided it for Tidewater.

Table 21.2: Hosted runners against your own

Question	Hosted runners (GitLab.com)	Your own runner
May the model's data leave the organization's network?	only if the data is cleared for the vendor's cloud	stays where the runner is
Who patches the machine?	GitLab	your administrators
What does a minute cost?	compute minutes after a free quota	nothing per minute; the machine's cost
Can you install the kit and its tools?	in a container image you build and maintain	on the machine, once

Question	Hosted runners (GitLab.com)	Your own runner
What does the ISSM have to approve?	the vendor's service and the data flow to it	one machine, one account, one network rule
What happens if it is down?	GitLab's problem, GitLab's status page	your problem; plan a second runner

For a model that is controlled, contractually sensitive or simply not cleared for a public cloud, the answer is almost always your own runner on the organization's network. For an open, unclassified training project like this book's exercises, hosted runners with the kit's Docker image are the quickest start.

21.4 What to put in the request

The enterprise administrators will need the same facts every time. Tom's request, which got the runner in about two weeks, listed:

- **Purpose**: run the automated checks of the Tidewater model project's pipeline; no other project.
- **Scope**: project runner for `tidewater/model`, tags `windows`, `shell`, `mbse`; runs untagged jobs; not protected in the first year.
- **Machine**: a Windows Server virtual machine, 2 processors, 8 GB of memory, a 100 GB data disk for builds; patched on the organization's monthly schedule.
- **Software**: GitLab Runner (the version matching the server), Git for Windows for all users; nothing else. The agile kit is a folder copied from the team's approved release.
- **Account**: a dedicated service account with no interactive logon, rights only to the kit folder (read) and the builds folder (write).
- **Network**: outbound HTTPS to the GitLab server only.
- **Owner**: Tom, with Jim as backup; the administrators keep the machine.
- **Security documentation**: the runner section of the toolchain security plan (Chapter 17, The DevSecOps tools that earn their keep (p. 255)).

A request that answers every question the administrators will ask is a request that is not sent back. Ask for it on day one.

21.5 How a runner runs a job: executors

The **executor** is how the runner runs each job's script.

Table 21.3: Runner executors

Executor	What it does	Good for	Watch out for
shell	runs the script directly on the runner's machine, as the runner's account	simple jobs with tools installed on the machine	jobs share the machine; nothing is isolated
docker	starts a fresh container from an image for each job	clean, repeatable environments	needs Docker on the runner; images must be built and maintained

Executor	What it does	Good for	Watch out for
docker-windows	the same with Windows containers	Windows tools in containers	large images, slow starts
kubernetes	runs each job as a pod in a cluster	large organizations	needs a cluster and its administrators
custom	your own scripts start and stop the environment	special cases	you maintain it

The team chose the **shell executor on Windows**: the runner machine is a Windows virtual machine with Git for Windows installed (which brings Perl, Git and a POSIX shell), and the agile kit checked out at a fixed path. Every job runs as the runner's service account, in a work folder the runner creates per project.

Why not Docker, which is cleaner? Because in the first quarter the organization had no Docker runners for the team's network, and the ISSM's approval for one would have taken longer than the twelve weeks. The kit does include an optional Docker image for later (Section 21.12, The Docker alternative (p. 355)), and the pipeline was written so that switching costs one line.

21.6 The lesson of the first week: PowerShell

When the runner came online at the start of sprint 2, Tom's first pipeline on the real server failed in a way it never had on the laptop:

```
$ test -n "$KIT" || { echo "KIT is not set: add it under Settings > CI/CD > Variables"; exit 1; }
```

followed by a PowerShell error about an unexpected token. The **Windows shell executor runs PowerShell** by default, not the Bash-style shell Git for Windows provides. In PowerShell, a variable is `$env:KIT`, not `$KIT`; `test`, `[ … ]`, `$(find …)`, `| xargs` and `||` either do not exist or mean something else. A pipeline written for one shell is a program in that shell's language.

There were three ways out:

1. Configure the runner to use Bash (possible, but fragile on Windows, and different from every other runner the team might use later).
2. Write the pipeline in PowerShell (and then it would not run on Linux, in Docker, or with the kit's local runner).
3. Make every script line **shell-neutral**: one command that means the same thing in every shell.

The team chose the third. Every job's script became one line, `perl ci/job.pl NAME`, and the commands moved into `ci/job.pl`, a Perl script in the repository. Perl is the same program under PowerShell, cmd, Git Bash, a Linux runner, the Docker image and a laptop. The header of the script says why:

tools/gitlab/templates/ci/job.pl

```
#!/usr/bin/perl
# ci/job.pl -- every job of the model pipeline, in core Perl, so .gitlab-ci.yml only ever says
`perl ci/job.pl NAME`.
```

```
#
# Why a script and not shell lines in the YAML: GitLab Runner's Windows shell executor runs
PowerShell, where $KIT is
# $env:KIT and `$(find ...)`, `[ -f ... ]`, `| xargs`, `&&` and quoting all differ from sh. One
Perl script behaves the
# same under PowerShell, cmd, Git Bash, a Linux runner, the Docker image and `gitlab.pl ci` on a
laptop. The YAML keeps
# what GitLab needs to know (when a job runs, what it keeps, which reports it attaches); this
file keeps what it does.
#
#   perl ci/job.pl grammar | lint | outline | todo | secrets | trace | vcrm | docs | delta |
drop-package | intake | baseline
#                  | publish-package | publish-release | pages | status | sil-regression |
hil-test
#
# Every job needs the CI/CD variable KIT (the agile kit's checkout on the runner) and writes its
log and report files
# in the project directory: NAME.txt (the checker's own output), NAME.xml (JUnit), NAME-cq.json
(Code Quality),
# NAME.metrics (metrics), via $KIT/tools/gitlab/ci-reports.pl. Exit status: 0 pass, 1 the check
failed, 2 the job
# cannot run (KIT missing, bad usage).
#
# Copied into the model repository as ci/job.pl (from tools/gitlab/templates/ci/job.pl). Core
Perl only.
```

You can see both halves of the idea in the sandbox of practice lesson 11. The same line runs the job on a laptop that the runner runs on the server, and a missing `KIT` stops the job with a sentence instead of a cryptic error:

```
. ../env.sh > /dev/null
perl ci/job.pl lint | tail -2
env -u KIT perl ci/job.pl lint || echo "exit status $?"
```

```
ci-reports lint: 1 check(s), 0 failed, 0 finding(s); model_elements=148 lint_errors=0
lint_warnings=0
KIT is not set: add it under Settings > CI/CD > Variables (the kit's path on the runner)
exit status 2
```

Exit status 2, not 1, means “the job could not run”, as opposed to “the check failed”: a runner problem, not a model problem.

Tip: The rule that came out of it

Keep the YAML for what GitLab needs to know (when a job runs, which runner, what it keeps); put what the job does in a script in the repository. You can then run any job by hand, on any machine, with the same line the runner uses.

21.7 Registering the runner

Registration connects a runner program to a GitLab project. Since GitLab 16 it goes in two steps: you create the runner in GitLab's web page, which gives you a **runner authentication token**, and you give that token to the runner program on the machine.

Step 1, in GitLab (Tom, as maintainer): Settings > CI/CD > Runners > New project runner. Fill in:

- **Tags**: `windows`, `shell`, `mbse`. Tags are how jobs choose runners: a job with `tags: [mbse]` runs only on a runner that has the `mbse` tag.
- **Run untagged jobs**: off. Every job in the team's pipeline asks for the tags `windows` and `mbse` (the pipeline's `default:` section sets them, Chapter 20, The pipeline, job by job (p. 302)), so an untagged job from a careless edit waits visibly instead of running somewhere unexpected.
- **Protected**: off for this runner (it must take merge request pipelines from feature branches). See Section 21.10, Protected runners (p. 354) for when to turn it on.
- **Description**: `tidewater-win-01`.

GitLab shows the token once, starting `glrt-`. Treat it as a password.

Step 2, on the runner machine (the enterprise administrator with Tom, in an elevated PowerShell window). These are the commands they typed on the Windows virtual machine; the book's material has no server, so they are shown, not run:

```
PS> cd C:\GitLab-Runner
PS> .\gitlab-runner.exe register --non-interactive `
      --url https://gitlab.example.gov `
      --token glrt-xxxxxxxxxxxxxxxxxxxx `
      --executor shell --shell pwsh `
      --description tidewater-win-01
PS> .\gitlab-runner.exe install --user ".\svc-tw-runner" --password "********"
PS> .\gitlab-runner.exe start
```

The runner then appears in the project's runner list with a green dot. The registration writes `config.toml` next to the program; the parts that matter:

```
concurrent = 2

[[runners]]
  name = "tidewater-win-01"
  url = "https://gitlab.example.gov"
  token = "glrt-xxxxxxxxxxxxxxxxxxxx"
  executor = "shell"
  shell = "pwsh"
  builds_dir = "D:/gitlab-runner/builds"
  cache_dir = "D:/gitlab-runner/cache"
```

- `concurrent` is how many jobs the runner runs at once. Two let a merge request pipeline and a `develop` pipeline run side by side; more did not help on a small virtual machine.

- `builds_dir` is where each job's fresh copy of the repository goes. Put it on a data disk with room to spare.
- `shell = "pwsh"` is the PowerShell default that taught the team the lesson above.

Step 3, the machine itself:

- Install Git for Windows for all users, so `perl` and `git` are on the service account's path (`C:\Program Files\Git\usr\bin` and `C:\Program Files\Git\cmd`).
- Check out the agile kit at a fixed path (`D:\agile-kit`) and set the project's `KIT` CI/CD variable to it.
- `git config --system core.longpaths true`, because Windows' 260-character path limit bites deep model folders under a deep builds folder.
- Ask for antivirus exclusions for `builds_dir`: scanning every file of every fresh checkout made jobs several times slower in the first week.

21.8 The service account: least privilege

The runner runs as a dedicated Windows account, `svc-tw-runner`, with:

- no interactive logon and no administrator rights;
- read access to the kit's folder, write access to `builds_dir` and `cache_dir`, nothing else;
- network access to the GitLab server and nothing else (the kit needs no network);
- no credentials of its own: jobs receive the short-lived job token from GitLab, and the weekly export's token comes from a masked, protected variable.

These five lines were most of the ISSM's review of the runner (Chapter 17, The DevSecOps tools that earn their keep (p. 255) has the full list of questions). The shell executor's weakness is that jobs are not isolated from each other on the machine; the answers are that only one project's jobs run there and that every job starts from a fresh checkout.

21.9 Tags, pending and stuck

When a job will not start, GitLab shows one of two states:

- **pending**: the job is waiting for a runner. Either every suitable runner is busy (it will start soon), or no online runner matches the job's tags and protection (it will wait forever). The job page says "This job is stuck because you don't have any active runners online or available with any of these tags assigned to them", with the tags.
- **stuck** is GitLab's word for the second case.

The checklist Tom kept on the wiki:

1. Is the runner online? Settings > CI/CD > Runners: a green dot and "last contact" a few seconds ago. If not, the service is stopped or the machine cannot reach the server.
2. Do the job's `tags:` match the runner's tags? A job tagged `docker` will never run on `tidewater-win-01`.
3. Is the job on a protected branch while the runner is protected, or the reverse?

4. Is the runner paused?

The team's one "stuck" episode was self-inflicted, on a scratch project: Tom added GitLab's Secret Detection template unconditionally. Its job needs a Docker runner, so it waited forever, and with "pipelines must succeed" it blocked every merge request on that project until the include was removed. That is why the include is now conditional on `DOCKER_RUNNER` (Chapter 20, The pipeline, job by job (p. 302)).

21.10 Protected runners

A runner marked **protected** takes jobs only from pipelines on protected branches and tags. Organizations use this to keep jobs that hold secrets or publish things (releases, the model site, package uploads) on a machine that untrusted branches can never reach. A merge request from a feature branch cannot run there, so it cannot be tricked into printing a protected variable.

The team ran one unprotected runner in the twelve weeks and wrote down the plan for a second, protected one for the `release`, `publish` and `drop-package` jobs, to be added when the second year's license and machine arrived (Chapter 42, Administration (p. 586)). With only one runner, the protection comes from the variables instead: `STATUS_TOKEN` is protected, so only pipelines on protected branches receive it.

21.11 Keeping the runner healthy

Table 21.4: Runner care

What	How often	Who
Runner online, last contact recent	glance daily (the stand-up board shows it)	Tom
Disk space on `builds_dir`	weekly; alarm at 80%	Tom and the admins
Runner version matches GitLab's major.minor	after each GitLab upgrade	admins
Windows patches and reboot	monthly window, announced in the stand-up	admins
Kit version on the runner	at each kit release, by merge request to the `KIT` path	Tom
Runner token rotation	yearly, or when someone with access leaves	Tom

The sprint 6 incident in Chapter 31, Sprint 6: Behavior by hand, a table for the program, and baseline-0 (p. 495) is the disk row: `builds_dir` filled up, every job failed with a disk error, and `develop` was red for most of a day. Two changes came out of it: old builds are cleaned nightly, and a second runner went onto the plan, because one runner is a single point of failure for the whole team's ability to merge.

The runner's own log (the Windows event log, source `gitlab-runner`) is where to look when a job never reaches the runner at all; the job log in GitLab is where to look once it has.

21.12 The Docker alternative

If your organization offers Docker runners, the kit includes an image with Perl, Git and the kit, and a three-key file to switch the pipeline to it:

tools/gitlab/docker/gitlab-ci.docker.yml

```
# The Docker-runner variant of the model pipeline: paste these keys at the top of .gitlab-ci.yml
(or `include:` this
# file) ONLY if your GitLab runs jobs on Docker runners. Everything else in the pipeline stays
the same.
#
# The image comes from this project's Container Registry (built and pushed as README.md shows).
KIT points at the
# kit inside the image, so the project's KIT CI/CD variable is not needed on Docker runners.
default:
  image: $CI_REGISTRY/tidewater/model/mbse-ci:1.0     # pin a version; never :latest
  tags: [docker]                                      # the runner tag your admins gave the
  Docker runners

variables:
  KIT: /opt/agile-kit
  DOCKER_RUNNER: "true"          # lets the GitLab-managed Secret Detection template run (it
  needs a Docker runner)
```

Because every script line is `perl ci/job.pl NAME`, nothing else in the pipeline changes. The image is built from a committed kit version and tagged with it, so the toolchain is baselined along with the model; `tools/gitlab/docker/README.md` has the build, test and care instructions. With a Docker runner, GitLab's managed scanners (Secret Detection and others) can run too.

21.13 The laptop is a runner too

`gitlab.pl ci` (Chapter 19, CI/CD from zero (p. 288)) is a runner that takes no orders from a server: it reads `.gitlab-ci.yml`, picks the jobs GitLab would pick for the event you name, and runs them in your working copy with Perl. It is not a substitute for the real runner (your laptop is not a fresh environment), but it answers "will my change pass?" in seconds before you push, and it is how every pipeline output in this book was produced.

21.14 Exercises

Exercise 21.1: Make a job pend

In a test project on your organization's GitLab (or with the emulator's `gitlab.pl ci`, by reading the `--list` output), add `tags: [does-not-exist]` to one job and push. What does GitLab say about the job? Fix it two ways: remove the tag, and register a runner with that tag.

Exercise 21.2: Translate a line

Take the line `test -n "$KIT" || { echo "KIT is not set"; exit 1; }` and write what it would have to be in PowerShell. Then explain in two sentences why the team chose `perl ci/job.pl NAME` instead of maintaining both.

Exercise 21.3: Write the runner's security description

In one page, describe a runner for your project the way the ISSM needs it: machine, account, rights, network, credentials, what runs on it, who maintains it. Use Section 21.8, The service account: least privilege (p. 353) as a model.

21.15 Applying this to your project

- **Copy**: ask for the runner on day one; one purpose per runner; a least-privilege service account; shell-neutral job scripts (`perl ci/job.pl NAME` or the equivalent in your tool's language); a written "pending" checklist; disk monitoring; a plan for a second runner.
- **Adapt**: the executor (Docker if your organization has it, shell if not); the scope (group runner if several model projects share tools); protected runners for publishing jobs.
- **Decide locally**: who owns the runner machine and its patches; which network it may reach; whether hosted runners and their compute minutes are allowed for your data at all.

CHAPTER

22

Reviewing a model change

From what you know →

You know

red-lining a document and filling in the comment matrix.

→ Here it is called

reviewing a merge request: line comments, threads, approve or request changes.

→ What is new

you review only what changed, with the generated views and the pipeline results beside it.

Survival kit

What you have

decades of document reviews – comment matrices, redlines, the review board where forty people looked at slide 37 – and three years of Cameo reviews by screenshot, where nobody could tell exactly what had changed.

What you need

to review a merge request in under half an hour, knowing exactly what changed; to read a `.sysml` diff and the generated views beside it; to write findings that teach rather than blame; and to know when to approve, when to ask for a change, and when to say "this is too big to review."

Note: Core path

This chapter is CORE. Every model change on Tidewater is reviewed by a second person from the first week; the team budgeted half a point each in sprint 4 to learn to do it well.

22.1 Slide 37

Jim's last review before the directive was a Cameo model review for the prime's preliminary design. The model was presented as sixty slides of diagram screenshots. On slide 37, an internal block diagram of the power system, Jim noticed that the charge regulator now connected straight to the battery. Had that changed since the last review? Nobody in the room could say. The prime's modeler thought so; the screenshot from three months earlier was in another deck; the action item to find out took two weeks.

A merge request answers that question before anyone asks it. It shows exactly which lines changed, who changed them, why, what the automated checks found, and what the change does to every view of the model. The reviewer's job is no longer to discover what changed. It is to judge whether the change is right.

Objection: "We don't have time to review every change. We reviewed at milestones."

Answer

A milestone review of three months of changes takes the program days and finds problems weeks after they were made. A review of one small change takes fifteen to thirty minutes and finds the problem the day it was made. The total time is smaller, and the problems are cheaper. Tidewater's median review turnaround was measured every sprint (Chapter 32, Measuring with GitLab analytics (p. 513)); it fell below a day by sprint 4.

22.2 What the pipeline already checked

Before a reviewer opens a merge request, the pipeline has run on it (Chapter 20, The pipeline, job by job (p. 302)). By the time it is green, these are already true, and the reviewer does not spend a minute on them:

- Every `.sysml` file parses against the official SysML v2 grammar.
- Lint reports 0 errors and 0 warnings; the outline and level rules hold.
- The trace has no *new* orphan, unverified or drifted requirement (from sprint 5).
- The generated documents in `docs/` match the model.
- No token, password or key is in the change.

That list is the reason a review can be short. **If the pipeline is red, do not review.** Say so in one line and wait; reviewing a change that does not yet pass is reviewing the wrong thing.

22.3 The reviewer's question

What is left is the part no machine can judge: *is the change right?* Tidewater's review checklist (`docs/conventions/review-checklist.md` in every scaffolded repository) breaks that question into a handful of things to look at. Here it is in full:

GitLab Markdown: tools/gitlab/templates/docs/conventions/review-checklist.md

```
# Reviewing a model merge request
```

```
[[_TOC_]]

A model review in GitLab is a peer review with the redlines in front of you. The pipeline has
already checked what a
machine can check: the grammar, lint, the outline, the trace and whether the generated documents
are current.
**Your job is what a machine cannot judge: is the change right?**

Read the merge request's description first, then the **Changes** tab, then the generated
documents (the pipeline's
artifacts, or the Pages preview). Leave a comment on the exact line for each finding. Use
**Request changes** when
something must change, and **Approve** when you would sign for it.

## Before you start

- [ ] The pipeline is green. If it is red, do not review yet: say so and wait.
- [ ] The description says what and why, and links its issue (`Closes #N`).
- [ ] The change is small enough to review in 30 minutes. If not, ask for it to be split.

## The engineering

- [ ] **It does what the issue asked**, and no more. Unrelated edits go in their own merge
request.
- [ ] **Requirements**: every new or changed requirement is one testable statement, has a DOORS
id (or a DR asking
      the prime for one), a verification method, and a `satisfy` from the element that really
      satisfies it.
- [ ] **Trace at the right level**: `satisfy` and `allocate` point at the element that does the
work, not at the
      whole system and not at every component.
- [ ] **Interfaces**: a change to a port, an interface or an item that another package uses has
that package's
      owner as a reviewer (CODEOWNERS asks for it).
- [ ] **Behavior**: states have a way in and a way out; every action has its inputs; the flows
connect.
- [ ] **Values**: units are stated (`Power`, `Energy`), assumptions are written in a `doc`
comment, numbers are
      notional or come from a cited source.
- [ ] **Names** follow [naming.md](naming.md). No `copy`, `old`, `tbd` or numbered names.
- [ ] **No new spider web**: no new package dependency cycle, no import from a higher layer, no
duplicated
      definition (reuse the existing one).

## Configuration and security

- [ ] **Markings**: new packages carry a marking; nothing raises a marking without the security
officer's comment
      ([markings.md](markings.md)).
- [ ] **Converter files**: lines that came from the prime are changed only to close a `// TODO`
line or by an
      accepted DR. Other changes to the prime's content go back to the prime.
- [ ] **No secrets**: no tokens, passwords or keys, even in an example.
- [ ] **Generated documents** in `docs/` were regenerated, not edited by hand.

## How to write a finding

| Instead of | Write |
|---|---|
```

```
| "This is wrong." | "The battery `capacity` has no unit; `Energy` would let the energy balance
check it." |
| "Why?" | "Why does `QualityControl` depend on `SensorSuite`? It creates a cycle with Stations."
|
| a long thread | a suggestion block with the exact replacement text |

The author resolves each thread by changing the model or by answering it. The merge request
cannot merge with an
open thread.
```

Two items on that list do most of the work in practice.

"It does what the issue asked, and no more." The most common problem in Tidewater's first sprints was not a wrong change; it was a change that did two things. Bob's first Stations merge request tagged levels *and* reformatted the indentation of the whole file, so the diff showed 180 changed lines for twelve real changes. Jim asked for it to be split, and the second version was reviewed in ten minutes.

"Converter files: lines that came from the prime are changed only to close a TODO line or by an accepted DR." This is the rule that keeps prime drops merging (Section 16.7, The rules that make the layout work (p. 251)). A reviewer who sees a change to a prime-delivered line that is not a TODO closure asks: is this the prime's to fix? If yes, it becomes a discrepancy report, not a merge request.

22.4 A review, start to finish

Practice lesson 11's sandbox has the Tidewater project in an offline GitLab, with a clone for Bob and the pipeline the team uses. Play both sides: Bob makes a small change and opens a merge request; Jim reviews it.

22.4.1 Bob: the change

Bob tags the station segment with its level – one TODO line closed, one import added. (The `perl -0pi` line is a scripted stand-in for the edit you would make in Vim.)

```
. ./env.sh > /dev/null
cd work
gl flow start feature 1-stationsegment-level
perl -0pi -e 's{// TODO stereotype Tidewater_Profile:L2Segment[^\n]*}{\@L2Segment;}; s{(package
Stations \{\n)}{$1    private import Conventions::*;\n}' model/physical/stations/Stations.sysml
git commit -qam "Tag StationSegment with its level (L2Segment)"
gl flow finish --title "Tag StationSegment with its level" --description "Closes #1"
```

`flow finish` pushes the branch, opens the merge request against `develop`, and runs its pipeline. The last line says which merge request it is and that its pipeline succeeded. Each block after this one starts with `. ../env.sh`: the book's own test runs every block in a fresh shell, so it loads the lesson's environment again. In your terminal you load it once, and it stays loaded until you close the window.

22.4.2 Jim: what changed, in text

The reviewer starts with the merge request itself – its description, its pipeline, its test widget – and then the diff. On the web these are the Overview and Changes tabs. From the clone:

```
. ../env.sh > /dev/null
gl --as jim mr show 2
git diff develop...feature/1-stationsegment-level
```

The three dots in `develop…feature/…` mean "what the branch changed since it left `develop`," which is exactly what the merge request will bring in. The diff shows one line removed (the TODO), one added (`@L2Segment;`) and the import. That is a change you can review completely in two minutes.

How to read a `.sysml` diff:

- **Lines starting with - are gone, + are new.** A changed line shows as one of each.
- **The `@@ … @@` lines say where.** The text after the second `@@` is often the enclosing package or definition, which tells you the context without opening the file.
- **Look at what each + line *means*,** not only what it says. `@L2Segment;` means "this part is a segment." Is StationSegment a segment? (Yes: it is one of the network's three segments.)
- **Look for what is missing.** A new `satisfy` without a `verify`; a new port with no connection; a requirement with no DOORS id. The pipeline catches some of these; the reviewer catches the rest.

22.4.3 Jim: what changed, in pictures

Text diffs show lines. Model changes are often easier to judge as pictures: what the change did to the part tree, the trace, the package dependencies. The kit draws the views of both versions with the differences colored (Section 11.8, Comparing two versions (p. 166)). Write them outside the clone, so they do not end up in a commit:

```
. ../env.sh > /dev/null
mkdir -p ../review
perl "$KIT/tools/sysml/model.pl" diff --git develop..feature/1-stationsegment-level -o
../review/stations-level
cat ../review/stations-level-changes.txt
```

The summary line is the most useful single line a reviewer reads: how many changes, how many *new trace gaps*, how many new error findings. "0 new gaps" tells Jim the change cannot have broken traceability. The colored SVGs in `../review/` show where the change sits in the part tree.

In the real GitLab the reviewer often does not need this command at all: from sprint 4 the pipeline publishes the model site, and the merge request's pipeline has the drawn views as artifacts. Open the artifact, not the text.

22.4.4 Jim: the finding

The change is right. But Jim wants one more thing: a line of documentation saying why Station-Segment is the L2 level, so a reader of the part tree does not have to guess. That is a finding, and it must be resolved before the merge:

```
. ../env.sh > /dev/null
gl --as jim mr note 2 "Line 12: the tag is right. Please add a doc line saying the station
segment is the L2 level of the network, so a reader of the part tree knows why." --resolvable
gl --as jim mr approve 2
gl mr merge 2 || true
```

The merge is refused: "discussions not resolved." The project setting *all threads must be resolved* turns every resolvable comment into a small gate. Bob answers the finding – with a change, or with a reply that explains – and resolves the thread; then the merge goes through:

```
. ../env.sh > /dev/null
gl mr resolve 2
gl mr merge 2
```

Notice what Jim did *not* do: Jim did not withhold the approval while waiting for a one-line doc comment. Jim approved the engineering and left a resolvable thread for the polish. The order matters to the author's day.

22.5 Suggestions: the redline you can apply

On the web, a reviewer who knows the exact replacement text can write it as a **suggestion**: a comment on a line with a fenced block marked `suggestion`. The author applies it with one click, and the merge request gets a commit with the reviewer's text. It is the redline, applied.

GitLab Markdown: tools/gitlab/samples/08-review-thread.md

````
<!-- An MR discussion as it reads on !19, top to bottom. Each block is one comment; "resolvable"
threads must be
     resolved before the MR can merge (project setting). -->

**@jim** commented on `model/stations/Stations.sysml` line 47 (resolvable):

`ShoreStation` is tagged `@L3Station`, but in v1 it is a specialization of `ObservingStation`,
which is also L3.
That is fine. Please also keep the v1 `xmi:id` next to the tag, as #20 does for BuoyModes, so the
delta can
point at it.

```suggestion:-0+1
    part def ShoreStation :> ObservingStation {
        @L3Station;   // v1 stereotype L3Station (xmi:id _22_1_7f3a0c1_…)
```

**@bob** replied:
````

```
Applied the suggestion (commit `3c9e1a2`). I did the same for the other 24 tags in a second
commit so the file is
consistent.

**@jim** resolved the thread.

---

**@tom** commented (resolvable):

The `trace` job is yellow because of TW-SYS-012 and TW-SYS-016. Those are DR-1 (#8) and DR-4
(#11), both with the
prime, so not this MR's problem. Linking them so the reviewer sees why.

/relate #8 #11

**@tom** resolved the thread.

---

**@jim** commented:

Lint 0/0, outline passes, docs current. Approving.

/approve
/label ~"workflow::done"

**@bob** commented:

Thanks. Merging when the pipeline on the merged result finishes.

/merge
```

The first comment in that thread shows the form: three backticks, the word `suggestion`, an optional range (`:-0+1` means "this line and the one after"), and the replacement lines. Use suggestions for exact wording, a missing unit, a name that breaks the convention – anything where you already know the answer. Do not use them for design questions; ask those as questions.

22.6 Different changes, different questions

Not every merge request is reviewed the same way. The kinds of task in Section 14.6, Writing the task issues (p. 217) produce different kinds of change, and each has its own first question:

Table 22.1: What to ask first, by kind of change

Change	Typical size	The reviewer's first question	Look at
TODO closure (level tags, procured parts)	5 to 40 lines	is each tag the right level for that part?	the diff; the part tree view
Hand-written construct (state machine, flows, constraint)	20 to 80 lines	does it say what the v1 model meant?	the v1 element in the drop (its xmi:id is in the TODO line); the IDEF0 or state view

Change	Typical size	The reviewer's first question	Look at
Requirement change	a few lines	is it one testable statement, by the recipe?	the recipe report; the trace view
Interface change	a few lines, wide effect	who uses this port or item, and do they agree?	the interconnection view; the second approver
Prime drop merge	hundreds of lines	what did the prime change, and what does it break of ours?	the delta and impact report, not the diff
Pipeline change	`.gitlab-ci.yml`, `ci/job.pl`	does every job still run, and does a red still block?	the pipeline of the MR itself, all jobs listed
Decision record or convention	one file	is the decision clear, and are its consequences named?	the text

22.7 Reviewing a prime drop merge

A prime drop merge request is the one review where reading the diff line by line is the wrong approach. A drop changes hundreds of lines, most of them exactly as the prime delivered them; the government does not review the prime's work line by line, it reviews *what the drop does to the government's model*.

So the drop's merge request carries the two reports from the converter, and the reviewer reads those first (Chapter 45, The prime's model changes: receive, assess, disposition (p. 619)):

- **The delta**, `xmi2sysml.pl --delta OLD.xmi NEW.xmi`: every element the prime added, removed or changed, by its v1 id, each marked *carried* (the converter brings it over) or *manual* (the converter cannot, and someone must rework it by hand). The manual list is the reviewer's checklist: each item is a task, or already done in this MR.
- **The impact**, `--impact model`: for each change, which of the government's requirements, verification cases, interfaces and allocations it touches, and which of the government's hand edits must be redone. A hand edit listed here and not redone in the merge request is a finding.

Then the conflicts: every file git could not merge on its own is listed in the merge request description, with how it was resolved. The reviewer checks that each resolution took the prime's side for the prime's content and re-applied the government's edits on top – never the other way round (Section 51.3.4, Silent conflict resolution (p. 688)).

22.8 Writing findings that teach

The team's first reviews were stiff. Bob wrote "This is wrong." Tom wrote nothing and approved. Jim wrote three paragraphs. By sprint 4 they had settled on a few habits, which the review checklist's last table records:

- **Say what and why in one or two sentences, on the exact line.** "The battery `capacity` has no unit; `Energy` would let the energy balance check it" teaches; "units?" does not.
- **Ask, when you are not sure.** "Why does QualityControl depend on SensorSuite? It closes a cycle with Stations." The author may know something you do not.

- **Separate must from could.** Prefix optional ideas with "Optional:" or "Nit:", and do not hold the merge for them.
- **Praise specifically, once.** "Good split – this is reviewable in ten minutes" tells the author what to repeat.
- **Never review the person.** The comment is about the line, not about who wrote it. On a three-person team, every reviewer is reviewed by the people they review.

> **Case study: Tom approved everything**
>
> In sprints 2 and 3, Tom approved every merge request Tom was asked to review, usually within minutes, without a comment. At the sprint 3 retrospective Tom said why: "Who am I to tell Bob the model is wrong? Bob knows it better than I do." Jim's answer became a team rule: the reviewer's job is not to know more than the author; it is to be the second pair of eyes the change has never had. Tom's first real finding, in sprint 4, was a requirement `satisfy` pointing at the whole Network instead of the Telemetry Unit – exactly the "trace at the right level" item on the checklist, and something Bob had not seen because Bob had written it.

22.9 Approve, ask for changes, or ask to split

A reviewer ends with one of three outcomes:

Table 22.2: The three outcomes of a review

Outcome	When	How
Approve	You would sign for the change as it is, or with only optional comments	the Approve button, or the `/approve` quick action
Ask for changes	Something must change before merging	a resolvable thread on each finding (some GitLab versions also offer "Request changes" when you submit a review); do not approve yet
Ask to split	The change is too big to review in about half an hour, or does two unrelated things	one comment saying so and suggesting the split; do not review the rest

Approval is enforced, not ceremonial: `CODEOWNERS` names who must approve a change to each folder, and an interface change needs two approvals, Jim's as systems lead and the other modeler's (Section 15.8, Code owners: the signature authority matrix (p. 239)). GitLab will not let an author approve their own change.

22.10 Turnaround

A review that waits three days costs more than a review that finds a problem. The team agreed on three habits:

- **Review before you start new work in the morning.** The GitLab To-Do list shows every merge request waiting for you.
- **A first response within one working day**, even if it is "I'll look at this after the stand-up tomorrow."

- **Small changes get reviewed first.** They are quick, and they unblock someone.

Review turnaround is one of the analytics the team reported to management from sprint 4 (Chapter 32, Measuring with GitLab analytics (p. 513)): the time from a merge request being opened to its first review comment or approval.

22.11 Applying this to your project

- **Copy:** the review checklist, the rule that a red pipeline is not reviewed, resolvable threads as gates, and the habit of reviewing the generated views, not only the text.
- **Adapt:** the checklist's engineering items to your model's conventions and your program's review criteria; the CODEOWNERS sections to your team's ownership.
- **Decide locally:** who reviews whom (on three people, everyone reviews everyone), whether a second approval is needed for interfaces, and the turnaround you will measure.

Exercise 22.1: Review a change

In the practice-11 sandbox, start a branch, change the `doc` comment of `StationSegment` and, in the same commit, remove the `private import Interfaces::*;` line from `Stations.sysml`. Open the merge request. Which of the two changes does the pipeline catch, and which would only a reviewer catch?

Exercise 22.2: Rewrite a finding

Rewrite these three review comments so they teach: (a) "wrong"; (b) "trace?"; (c) "This whole package is a mess, please clean it up."

You can now review a model merge request: check that the pipeline is green, read the description and the diff, compare the generated views of both versions, write a finding on the exact line, use suggestions, and choose between approving, asking for changes and asking for a split.

CHAPTER

23

Stand-ups, the journal and the board

From what you know →

You know

the status spreadsheet and the action-item list.

→ Here it is called

the journal of stories, and the GitLab board for tasks.

→ What is new

status is recorded once, as it happens, and every report is derived from it.

Survival kit

What you have

a GitLab project where tasks live as issues (Chapter 15, Setting up GitLab (p. 226), Chapter 17, The DevSecOps tools that earn their keep (p. 255)), and an operating model that says work comes in tomes, epics, stories and tasks (Chapter 13, The operating model (p. 196)).

What you need

a record of the plan that management can trust: what each sprint committed, what it finished, what slipped, how fast the team goes, and where each epic and tome stands. This chapter sets up the agile kit's **journal**, a plain-text ledger of stories kept in Vim, explains how it divides the work with GitLab, and shows the reports it produces. Chapter 24, The rhythm: stand-ups, poker, refinement, kickoff, demo, retro (p. 377) then shows the daily and fortnightly rhythm that feeds it.

23.1 Two records, one plan

By the end of sprint 1 the team had two places that knew about the work, and Bob asked the obvious question: why two?

If every task is an issue in GitLab, why do we also keep a journal? That's the double book-

keeping we were promised would go away.

— Bob, sprint 1 review

The answer is that the two records hold different things, for different readers, at different levels of the hierarchy.

Table 23.1: The hierarchy, and where each level is recorded

Level	Size rule	Where it lives	Who reads it
Project	years; this one is 39 sprints	the journal: one journal per project	the program
Tome	under a year (two semesters)	the journal, `t:` on each story; a `tome::` label in GitLab	the program manager
Epic	under a semester (13 sprints)	the journal, `e:` on each story; an `epic::` label in GitLab	Jim, management
Story	under one sprint	**the journal** (id `TW-N`) and a GitLab issue labeled `type::story` (number N)	Jim, the team
Task	under a week (5 points or fewer)	**GitLab only**: an issue labeled `type::task`, closed by a merge request	the person doing it

The journal tracks **stories and above**: the commitments the team makes to management, sprint by sprint, and the roll-up into epics and tomes. GitLab tracks **tasks and the stories they belong to**: the day-to-day work, the merge requests that do it, the reviews and the pipelines.

The link between them is the number. A story is GitLab issue 12 and journal item `TW-12`. Anyone reading a stand-up note that mentions `TW-12` opens issue 12 in GitLab to see its tasks, merge requests and discussion; anyone looking at issue 12 knows the journal tracks it as `TW-12`.

This is a deliberate application of a rule the agile kit has always had: the architect's journal holds commitments and their state, and task-level detail stays in the team's own tool. Tasks change daily; they are cut, merged, re-cut. A journal that tracked them would be noisy and always out of date. Stories change state a few times each, and every change is a fact management cares about: committed, done, carried, blocked.

Note: Why not GitLab for everything?

GitLab's Premium tier has epics and iterations with burndown charts, and with it a team could keep the whole hierarchy in GitLab. The team had the Free tier, and more importantly a need GitLab does not meet: a ledger in which points are never edited after the fact, only moved, so last month's numbers are exactly what they were last month. That property is what lets Chapter 33, The twelve-week scorecard (p. 520) compare sprint 1 with sprint 6 without anyone asking "has this been restated?"

23.2 Everything is a ledger

The journal is a text file, `scrum.txt`, in the format of **ledger-cli**, a decades-old plain-text accounting tool. That sounds odd for a scrum record until you see what it buys.

Every story has **points** (the team's estimate of its size: one point is one ideal person-day). The points sit in an **account**, and the account *is* the story's state:

```
Backlog:Model  ->  Sprint:N:Model:Committed  ->  Done
                                             ->  Carryover  ->  next sprint's Committed
                                             ->  Removed
```

When the team commits a story to sprint 1, its points move from `Backlog:Model` to `Sprint:1:Model:Committed`. When it is done, they move to `Sprint:1:Model:Done`. Nothing is ever edited; each event is a new entry that moves points. Re-estimating a story posts the difference, not a new number. Like money in double-entry bookkeeping, points are conserved: every entry balances, and the totals cannot drift.

Here is what the journal recorded on the first day of sprint 1, written by the kit from the planning notes (the compile step is below):

```
; ---- standup 2026-10-05 (standups/2026-10-05.txt)
~ Sprint 1
    Model   19 SP

2026-10-05 Intake TW-1 Create the GitLab group and model project
    Backlog:Model                    4 SP   ; id: TW-1, epic: E1 GitLab project and gitflow,
    owner: Tom, prio: 1, tome: T1 Stand up MBSE DevSecOps
    Equity:Intake

2026-10-05 Standup Model
    Backlog:Model                    -4 SP   ; id: TW-1
    Sprint:1:Model:Committed         4 SP   ; id: TW-1, owner: Tom
```

(an excerpt). The `~ Sprint 1` entry records the team's capacity for the sprint, 19 points. The `Intake` entry brings a new story into the backlog with its metadata after the semicolon: its epic, owner, priority and tome. The `Standup` entry moves its 4 points into the sprint's commitment.

You never write these entries by hand. You write terse notes, and the kit writes the ledger.

23.3 Setting up the journal

The journal is its own git repository, separate from the model, because it has a different audience and different rules (it is the team's management record; the model is a controlled item). The kit creates one:

```
mkdir -p ~/tidewater-journal && cd ~/tidewater-journal
perl $KIT/bin/daily.pl init .
```

```
created .gitattributes
created .gitignore
created scrum.conf
created scrum.txt
git init
```

```
Next:
  1. edit scrum.conf (recipients, markings, townhall time, facilitator = your Teams display name)
  2. the people:  daily.pl roster add "Lee, Ann" ann.lee@example.com Alpha Lead ACME   (name
  exactly as Teams shows it)
  3. the backlog: daily.pl new, then 'new ID PTS title e:Epic t:Tome o:"Owner Name"' lines under
  '== Team', then compile
  4. add to ~/.vimrc:   source $KIT/vim/scrum.vim
     then in Vim:  :SNew  :SCompile  :SReport  :SDraft  :SAll  :SStatus
```

`scrum.conf` holds the settings, one `key = value` per line. For a single team the ones that matter on day one are `default_team = Model` (the team's name in the journal; the new file leaves it empty) and `unit = SP` (story points, already set). In Vim it is one edit; from the shell, one command. The people go in `roster.txt`, with their names exactly as the meeting chat shows them, because the stand-up tools match chat names to the roster:

```
perl -pi -e 's/^default_team\s*=\s*/default_team  = Model   /' scrum.conf
perl $KIT/bin/daily.pl roster add "Jim" jim@example.invalid Model "MBSE lead" Gov
perl $KIT/bin/daily.pl roster add "Bob" bob@example.invalid Model "MBSE engineer" Gov
perl $KIT/bin/daily.pl roster add "Tom" tom@example.invalid Model "CM and DevSecOps engineer" Gov
cat roster.txt
```

```
# name | email               | team  | role                       | org
  Bob  | bob@example.invalid | Model | MBSE engineer              | Gov
  Jim  | jim@example.invalid | Model | MBSE lead                  | Gov
  Tom  | tom@example.invalid | Model | CM and DevSecOps engineer | Gov
```

Jim keeps the journal repository on the team's share as well as in GitLab (a second project in the `tidewater` group, private to the three of them), and commits it at the end of each day with `daily.pl commit`.

23.4 The stand-up file

Each day's notes go in one small text file, `standups/YYYY-MM-DD.txt`, written in Vim (`:SNew` creates today's file with the open stories listed as comments). The notes use a handful of four-letter verbs, each one line:

Table 23.2: The stand-up verbs used most

Verb	Meaning	Example
`sprint N`	which sprint this file belongs to	`sprint 1`
`== Team`	the team the following lines are about	`== Model`
`cap N`	the team's capacity for the sprint, in points	`cap 19`
`new ID PTS title …`	a new story into the backlog, with `p:` priority, `e:` epic, `t:` tome, `o:` owner	see below
`commit ID Owner`	commit a story to the sprint	`commit TW-1 Tom`
`done ID …`	stories finished today	`done TW-3`

Verb	Meaning	Example
`block ID why` / `unblock ID`	a story is blocked, or no longer	`block TW-1 waiting on the enterprise GitLab admins`
`carry ID …`	at the sprint's end, stories that go to the next sprint	`carry TW-11`
`est ID N`	a new estimate (posts the difference)	`est TW-7 5`
`note text`	a line for the record and the status mail	`note Demo went well`

Sprint 1's planning file, written by Jim on Monday 5 October after the kickoff. In Vim, Jim typed the lines into the file `:SNew` opened; to type along from the shell, write the same lines with a here-document (everything between the `<<'PLAN'` line and the `PLAN` line goes into the file):

```
mkdir -p standups && cat > standups/2026-10-05.txt <<'PLAN'
2026-10-05
sprint 1
== Model
cap 19
new TW-1 4 Create the GitLab group and model project p:1 e:"E1 GitLab project and gitflow" t:"T1
Stand up MBSE DevSecOps" o:Tom
new TW-2 4 First pipeline gates every MR p:1 e:"E2 Pipeline and quality gates" t:"T1 Stand up
MBSE DevSecOps" o:Tom
new TW-3 3 Receive drop 1 and record it p:1 e:"E5 Drop 1 intake and first import" t:"T2 Port
Tidewater v1 to SysML v2" o:Jim
new TW-4 4 Convert drop 1 and assess it p:1 e:"E5 Drop 1 intake and first import" t:"T2 Port
Tidewater v1 to SysML v2" o:Bob
new TW-5 2 Working agreements p:2 e:"E3 Team practices and onboarding" t:"T1 Stand up MBSE
DevSecOps" o:Jim
commit TW-1 Tom
commit TW-2 Tom
commit TW-3 Jim
commit TW-4 Bob
commit TW-5 Jim
note Sprint 1 kickoff: goal "A GitLab project the team can work in; drop 1 received and assessed"
PLAN
tail -1 standups/2026-10-05.txt
```

(The story numbers in this chapter's examples come from a practice journal; Part III uses the numbers from the real run.) Quoted values (`e:"E1 GitLab project and gitflow"`) may contain spaces; unquoted ones may not.

Then the **compile** step checks the file and, if every line is valid, appends the ledger entries to `scrum.txt`:

```
perl $KIT/bin/daily.pl compile
```

```
applied standups/2026-10-05.txt (6 transactions)
```

A stand-up file with a single mistake (an unknown story id, a `done` for a story that was never committed, a verb misspelled) is rejected whole, with every problem listed, and nothing is written. That is the journal's own gate: it never contains half a day.

In Vim the same steps are `:SNew` (today's file), `:SCompile`, and `:SStatus`. Chapter 10, Vim for models (p. 143) lists the rest.

23.5 What the journal tells you

Everything the journal reports is computed from the ledger when you ask; nothing is stored separately, so nothing can disagree.

The sprint view, the morning after planning (`--today` sets the date the journal reads as "today", so this works on any day you type it):

```
perl $KIT/bin/daily.pl --today 2026-10-06 sprint
```

```
Teams: sprint 1
Team   Cap  Commit  Done  Open  Carry  Removed  Done%  Load%
-----  ---  ------  ----  ----  -----  -------  -----  -----
Model   19      17     0    17      0        0     0%    89%
Total   19      17     0    17      0        0     0%    89%
```

`Load%` is commitment divided by capacity: 17 of 19 points, 89%. The kit's metrics treat a load above 110% as a warning (the team is promising more than it can do) and below 70% as another (it is not using its capacity, or the capacity is wrong). Jim aimed for 85 to 95% every sprint.

The epics view, at the end of sprint 1:

```
Epics by tome
Tome                            Epic                                    Tasks  Total  Done  In
sprint  Backlog  Done%
--------------------------------  --------------------------------------  -----  -----  ----
---------  -------  -----
T1 Stand up MBSE DevSecOps        E1 GitLab project and gitflow               1      4     4
     0        0   100%
T1 Stand up MBSE DevSecOps        E2 Pipeline and quality gates               2      9     4
     0        5    44%
T1 Stand up MBSE DevSecOps        E3 Team practices and onboarding            1      2     2
     0        0   100%
T2 Port Tidewater v1 to SysML v2  E5 Drop 1 intake and first import           3     10     7
     0        3    70%
T2 Port Tidewater v1 to SysML v2  E6 Structure: close the structure TODOs     1      3     0
     0        3     0%
```

(The column is headed "Tasks" because the kit calls every journal item a task; in this project every journal item is a story.)

Velocity, the points done per sprint, which becomes the basis for forecasts once there are three sprints of history:

```
Model  (avg velocity 17.0 over the last 1 completed sprint)
Sprint  Cap  Commit  Done  Done%
```

```
------  ---  ------  ----  -----
     1   19      17    17   100%
```

And the ledger itself, through the kit's ledger tool, balances to zero, which is the proof that no points were created or lost. Right after planning, in your journal:

```
perl $KIT/bin/ledger.pl -f scrum.txt bal
```

```
             -17 SP  Equity:Intake
              17 SP  Sprint:1:Model:Committed
--------------------
                   0
```

and in the practice journal at the end of sprint 1, after the stories were done and the next ones were filed:

```
              11 SP  Backlog:Model
             -28 SP  Equity:Intake
              17 SP  Sprint:1:Model
              17 SP    Done
--------------------
                   0
```

Twenty-eight points came in (`Equity:Intake` is the counterweight of every intake); seventeen are done in sprint 1; eleven wait in the backlog for sprint 2.

23.6 Estimates that change

Estimates change, especially in the first weeks, when the team does not yet know how long anything takes. In the journal a new estimate is a posting of the difference. When the team re-estimated story `TW-7` from 4 to 5 points in refinement (Chapter 24, The rhythm: stand-ups, poker, refinement, kickoff, demo, retro (p. 377) shows the poker round), the journal recorded:

```
2026-10-14 Intake TW-7 Docs, outline and TODO jobs
    Backlog:Model                    4 SP   ; id: TW-7, epic: E2 Pipeline and quality gates,
    owner: Tom, prio: 1, tome: T1 Stand up MBSE DevSecOps
...
    Backlog:Model                    1 SP   ; id: TW-7
```

The original 4 points and the added 1 are both visible forever. A forecast made on the 13th and one made on the 15th can each be reproduced from the journal as it stood that day; the kit can load the journal "as of" any date. In the document era, the action item spreadsheet had one estimate column, overwritten.

23.7 A story from end to end

It helps to follow one story through both records. Take the story Bob owned in sprint 1, "Convert drop 1 and assess it", four points.

Monday, planning. Jim creates the story as a GitLab issue with the Story template (its number becomes the id: in the practice project it was issue 1) and the journal line `new TW-4 4 Convert drop 1 and assess it …` with `commit TW-4 Bob`. In GitLab the issue gets the labels `type::story`, `tome::port`, `epic::drop-intake` and the milestone "Sprint 1". Bob cuts it into tasks during the kickoff, each its own issue with the Task template and "Part of" the story:

Table 23.3: The tasks of one story

Task	Points	What it produces
Run both converters with –report	1	the two converter reports, attached to the story
TODO inventory by package	2	a table of the 52 TODO lines by package and kind, in the story
Estimate the port backlog with the team	1	candidate stories for the structure epic, poker-estimated

Tuesday to Thursday, the work. Each task moves across the board: `workflow::in-progress` when Bob starts it, a comment with the result and `/spend` when it is finished, `/close`. Chapter 17, The DevSecOps tools that earn their keep (p. 255) shows the first task's history, driven entirely by quick actions. Bob's stand-up lines name the story, not the tasks (`Y TW-4 read the converter report`), because the journal knows stories.

Friday of week 2, done. When the last task closes, Bob says `done` in the stand-up; the answers step suggests the `done TW-4` line; Jim confirms and compiles. In GitLab, Jim checks the story's acceptance criteria and closes it with a comment. The journal moved 4 points from `Sprint:1:Model:Committed` to `Sprint:1:Model:Done`, dated that day; GitLab recorded three closed tasks with their time and their outputs.

Neither record holds the other's detail, and neither needs to. Management sees the story in the quad; anyone who wants the details follows the number.

Tip: When a story will not fit

If, during the sprint, a story turns out bigger than the sprint, do not stretch it. Split it: the finished part is a story that is done; the rest is a new story in the backlog, estimated at the next refinement. The journal's `carry` exists for the story that is nearly done on the last day, not for the one that was never going to fit.

23.8 The board and the journal, day to day

The team kept the two in step with three habits:

- **The board is for tasks; the journal is for stories.** The GitLab board, scoped to the current sprint's milestone, is what the team looks at during the day. The journal is updated once a day, after the stand-up.

- **A story is done when its last task closes.** GitLab closes a task when its merge request merges; when a story's last task is closed, the owner says so in the stand-up (`done TW-12`), and Jim's journal records it.
- **Blocked is said out loud.** A blocked task gets the `workflow::blocked` label in GitLab and a comment saying why; the story gets a `block` line in the journal the same day, so the weekly report to management shows it (Chapter 24, The rhythm: stand-ups, poker, refinement, kickoff, demo, retro (p. 377), Chapter 36, Telling management (p. 538)).

23.9 The reports

The journal feeds every report the team sends upward. Each one is a command, and each was produced every sprint without anyone assembling a slide:

- `daily.pl review N`: the sprint report at the review: committed, done, carried, velocity, predictability, and each epic's remaining points and forecast completion sprint. Sprint 1's began:

```
Sprint 1 report -- as of 2026-10-16: 17/17 SP done (100%), 0 carried over, 0 removed.

Team           Commit  Done  Done%  Carry Carry%   Load Velocity  Predict%  Punted Interrupt
Cap
Model              17    17   100%      0     0%    89%     17.0      100%       0         0
19
```

- `daily.pl quad`: the weekly one-page status in four quadrants (technical priorities, watch items, schedule milestones, accomplishments), shown in Chapter 24, The rhythm: stand-ups, poker, refinement, kickoff, demo, retro (p. 377) and Chapter 36, Telling management (p. 538).
- `daily.pl health`: every metric against its threshold, red, amber or fine.
- `daily.pl report`: all of the above as HTML pages, plus the cockpit dashboard.

Warning: One hundred percent is not the goal

Sprint 1 in the practice journal finished 17 of 17, and so did the real sprint 1 (16.5 of 16.5). Sprint 4 did not: it finished 12 of 17 story points (Section 13.8, Two ways to count "done" (p. 205)) and carried a walk-through into sprint 5, and that is fine (Chapter 29, Sprint 4: the model site, a cycle broken, and the demo that landed (p. 459)). A team that finishes everything every sprint is committing too little. The kit's metrics look for predictability (done over committed) of 80% or better, not 100%.

23.10 Exercises

Exercise 23.1: Make a journal

Create a journal with `daily.pl init`, add yourself and two colleagues to the roster, and write a planning file for a sprint of your own work with three stories. Compile it, then run `daily.pl sprint` and `daily.pl epics`.

Exercise 23.2: Break the compile

Add a line `done XX-99` to a stand-up file and compile. What does the kit say, and what did it write to `scrum.txt`?

Exercise 23.3: Where does it go?

For each item, decide whether it belongs in the journal, in GitLab, or both, and why: (a) "write the state definition for BuoyModes, 3 points"; (b) "close the Stations TODO lines, 8 points"; (c) "the structure port"; (d) "Bob spent 4 hours on the review"; (e) "blocked: waiting for the ISSM".

Exercise 23.4: Re-estimate

Re-estimate one of your stories with an `est` line. Look at the ledger entries it produced. Why does the kit post the difference instead of replacing the number?

23.11 Applying this to your project

- **Copy**: two records with a clear split (commitments and roll-ups in the journal, tasks in the team's tool); one number for a story in both; points that move and are never edited; a compile step that rejects a bad day whole; reports computed from the record, never assembled by hand.
- **Adapt**: the hierarchy's names and size rules to your program; the capacity per person; the load band.
- **Decide locally**: who keeps the journal (one person, usually the lead); where it lives; who may read it; which reports go to whom (Chapter 36, Telling management (p. 538)).

CHAPTER

24

The rhythm: stand-ups, poker, refinement, kickoff, demo, retro

From what you know →

You know

the weekly status meeting, the technical interchange meeting and the program review.

→ Here it is called

stand-up, planning poker, refinement, kickoff, demo and retrospective.

→ What is new

short meetings of fixed length at a fixed rhythm, each with one job.

Survival kit

What you have

a journal of stories (Chapter 23, Stand-ups, the journal and the board (p. 367)), tasks in GitLab (Chapter 17, The DevSecOps tools that earn their keep (p. 255)), and two-week sprints.

What you need

the meetings that keep the plan honest, run in a way three systems engineers can sustain: a daily stand-up of terse written statuses, planning poker for estimates, a mid-sprint refinement, a kickoff on day one and a demo and retrospective on the last day. Each meeting produces a record the kit reads, so nobody writes minutes. This chapter shows each one with real chat pastes and the kit's real output.

24.1 Why meetings at all

The team's first reaction to the word "ceremonies" was predictable.

> *We already have too many meetings. Now we get a daily one and four more every two weeks?*
>
> — Tom, week zero

The answer the team arrived at, after a sprint of trying it, was that these meetings *replace* others rather than add to them. The weekly status meeting, the action item review, the monthly schedule review and most of the e-mail chains went away, because their questions were answered in fifteen minutes a day and two hours a fortnight, from records the meetings themselves produced.

Table 24.1: The two-week rhythm

When	Meeting	Length	Produces	Replaces
Daily, 09:00	stand-up	15 minutes	Y/T/B statuses; blocks; done stories	the weekly status meeting, the action item review
Day 1 (Monday, week 1)	sprint kickoff and planning	90 minutes	the sprint goal; capacity; committed stories; task issues	the monthly schedule review
Wednesday, week 2	refinement	1 hour	the next sprint's candidate stories, estimated and ready	ad hoc tasking e-mails
Day 10 (Friday, week 2)	demo (sprint review)	60 minutes	what was shown, feedback, accepted stories	review-by-screenshot briefings
Day 10, after the demo	retrospective	60 minutes	one or two changes; the pulse survey	the annual lessons-learned exercise
Every Friday	weekly status	0 minutes of meeting	the quad, sent to management	the weekly status slides

24.2 The daily stand-up: terse, written, linted

The team's stand-up is a Teams meeting at 09:00 whose main event is the chat. Each person posts one line with three parts, **Y** (what I did yesterday), **T** (what I will do today) and **B** (what blocks me), each naming the story it is about:

```
Y TW-4 read the converter report T TW-4 TODO inventory by package B none
```

The kit reads two shapes. In sprints 1 and 2 the team posted the three parts on three lines, as most people do at first; this is Tidewater's first stand-up, from the meeting chat:

docs/tidewater/sprints/01/standups/2026-10-05-chat.txt

```
Meeting started
[9:00 AM] Jim
Morning. Y/T/B in chat, please.
[9:01 AM] Jim
Y: meetings, reviews
T: TW-1 practice lessons 10-11; the operating model
B: none
[9:02 AM] Bob
Y: meetings, reviews
T: TW-1 practice lessons 10-11; the survival kit, git
B: none
[9:03 AM] Tom
Y: meetings, reviews
T: TW-1 practice lessons 10-11; the survival kit, git
B: none
```

From sprint 3 they posted one line per person, drafted by `talk.pl` (Chapter 25, Talking about progress: helps for engineers who would rather not (p. 390)) and edited before posting, with the outcome and a number first. The examples in this section use the one-line shape because it is shorter to type; the lint treats both the same way.

Then they talk only about the blockers and anything that needs two people. The statuses are the record; the talk is for help.

Why written, and why so terse? Because a written status can be checked by a program, compiled into the journal and compared with yesterday's, and a spoken one cannot. And because three engineers who had spent years writing status paragraphs for slides found that one line, with an id, said more.

The rules, from the team's wiki:

- **Every part is present.** `B none` when nothing blocks you; an empty B is not "none", it is a missing answer.
- **Every Y and T names a story id** (`TW-4`). Work with no id is invisible in the plan; if it matters, it should be a story.
- **A blocker goes in B, not in T.** "Waiting on the admins" is not something you will *do* today.
- **Capital Y, T, B as delimiters**; lower-case letters inside the text are just letters.

24.2.1 The lint

To type along, you need a journal with sprint 1 planned. If you made one in Chapter 23, Stand-ups, the journal and the board (p. 367), use it; otherwise these commands make the same one in a few seconds:

```
mkdir -p ~/tidewater-journal && cd ~/tidewater-journal
if [ ! -f scrum.txt ]; then                # only if you do not have the journal from the last
chapter
perl $KIT/bin/daily.pl init . > /dev/null
perl -pi -e 's/^default_team\s*=\s*/default_team  = Model   /' scrum.conf
for who in Jim Bob Tom; do perl $KIT/bin/daily.pl roster add "$who" $(echo $who | tr A-Z
a-z)@example.invalid Model Engineer Gov; done
mkdir -p standups && cat > standups/2026-10-05.txt <<'PLAN'
2026-10-05
sprint 1
== Model
cap 19
new TW-1 4 Create the GitLab group and model project p:1 e:"E1 GitLab project and gitflow" t:"T1
Stand up MBSE DevSecOps" o:Tom
new TW-2 4 First pipeline gates every MR p:1 e:"E2 Pipeline and quality gates" t:"T1 Stand up
MBSE DevSecOps" o:Tom
new TW-3 3 Receive drop 1 and record it p:1 e:"E5 Drop 1 intake and first import" t:"T2 Port
Tidewater v1 to SysML v2" o:Jim
new TW-4 4 Convert drop 1 and assess it p:1 e:"E5 Drop 1 intake and first import" t:"T2 Port
Tidewater v1 to SysML v2" o:Bob
new TW-5 2 Working agreements p:2 e:"E3 Team practices and onboarding" t:"T1 Stand up MBSE
DevSecOps" o:Jim
commit TW-1 Tom
commit TW-2 Tom
commit TW-3 Jim
commit TW-4 Bob
commit TW-5 Jim
```

```
PLAN
perl $KIT/bin/daily.pl compile
fi
perl $KIT/bin/daily.pl --today 2026-10-06 sprint | head -4
```

While people are still on the call, Jim pastes the chat into a file and runs the kit's **lint** on it: a check of every status against the rules. Here is the chat of Tuesday 6 October, sprint 1, day 2, the team's first real stand-up. Jim pasted it into `standups/2026-10-06-chat.txt` (in Vim, `\sp` does the paste; from the shell, a here-document does the same):

```
cat > standups/2026-10-06-chat.txt <<'CHAT'
[9:01 AM] Jim
Y TW-3 logged the CDRL delivery, checksums match T TW-3 move drop 1 to approved storage, TW-5
draft DoR B none
[9:02 AM] Tom
Y requested the group and a runner T TW-1 waiting on the admins B
[9:03 AM] Bob
yesterday: read the converter report. today: TODO inventory. blockers: none
CHAT
perl $KIT/bin/daily.pl --today 2026-10-06 lint standups/2026-10-06-chat.txt || echo "exit status
$?"
```

```
standups/2026-10-06-chat.txt:2: ok    Jim: Y/T/B parsed
standups/2026-10-06-chat.txt:4: ERROR Tom: B is empty (write 'none')
standups/2026-10-06-chat.txt:6: warn  Bob: no task id in Y (e.g. TW-1); no task id in T (e.g.
TW-1)
# 3 answered, 1 error, 1 warning
```

The lint reads Bob's longhand correctly but warns that there are no story ids. It refuses Tom's status: the B is empty, and the blocker is sitting in the T. The `--reply` option writes the lines to paste back into the chat, so people fix their own statuses during the call:

```
perl $KIT/bin/daily.pl --today 2026-10-06 lint standups/2026-10-06-chat.txt --reply || echo "exit
status $?"
```

```
Hey Tom! I couldn't read your status (B is empty (write 'none')). Repost as: Y did TW-1 T doing
TW-2 B none
Hey Bob! Got it. Add the task id next time so it counts (e.g. Y did TW-4 T doing TW-4 B none).
```

Tom and Bob reposted, and Jim pasted the new lines under the old ones:

```
cat >> standups/2026-10-06-chat.txt <<'CHAT'
[9:06 AM] Tom
Y TW-1 requested the group and a runner T TW-2 draft the first .gitlab-ci.yml B TW-1 waiting on
the enterprise GitLab admins
[9:07 AM] Bob
```

```
Y TW-4 read the converter report T TW-4 TODO inventory by package B none
CHAT
perl $KIT/bin/daily.pl --today 2026-10-06 lint standups/2026-10-06-chat.txt
```

and the second lint was clean (the parser keeps each person's last status):

```
standups/2026-10-06-chat.txt:2: ok    Jim: Y/T/B parsed
standups/2026-10-06-chat.txt:8: ok    Tom: Y/T/B parsed
standups/2026-10-06-chat.txt:10: ok    Bob: Y/T/B parsed
# 3 answered, 0 errors, 0 warnings
```

`--table` shows what the parser understood, one row per person, which is how you check that it read what you meant:

```
perl $KIT/bin/daily.pl --today 2026-10-06 lint standups/2026-10-06-chat.txt --table
```

```
state   who     Y       T       B       note
ok      Jim     TW-3 logged the CDRL delivery, checksums match  TW-3 move drop 1 to approved
storage, TW-5 draft DoR    none
ok      Tom     TW-1 requested the group and a runner   TW-2 draft the first .gitlab-ci.yml
TW-1 waiting on the enterprise GitLab admins
ok      Bob     TW-4 read the converter report  TW-4 TODO inventory by package  none
```

In the first two weeks the lint flagged something almost every day. By sprint 3 the statuses were clean, and the kit's read-backs (a confirmation line for each clean status) stopped by themselves after a run of clean days, as `scrum.conf`'s `readback_clean_days` sets.

In Vim, the whole loop is one layout: `:STown` opens the chat buffer, the lint table and the replies side by side; `\sp` pastes and lints, `\sy` copies the replies to the clipboard. Chapter 10, Vim for models (p. 143) has the details.

24.2.2 After the call: answers and flags

After the call, `daily.pl answers` reads the day's chat, writes the statuses into the record, raises **flags**, and appends suggested journal lines to today's stand-up file for Jim to confirm:

```
perl $KIT/bin/daily.pl --today 2026-10-06 answers
```

```
Model stand-up answers: 3 people, 1 blocked
  Jim            Y: TW-3 logged the CDRL delivery, checksums match
                 T: TW-3 move drop 1 to approved storage, TW-5 draft DoR
                 B: none
  Tom            Y: TW-1 requested the group and a runner
                 T: TW-2 draft the first .gitlab-ci.yml
                 B: TW-1 waiting on the enterprise GitLab admins
```

```
  Bob            Y: TW-4 read the converter report
                 T: TW-4 TODO inventory by package
                 B: none
flags:
  AMBER Tom            blocked: TW-1 waiting on the enterprise GitLab admins
wrote standups/2026-10-06-Model-answers.txt
  attendance: 3 rows (3 answered) -> attendance.csv
  appended suggested lines to standups/2026-10-06.txt
```

The suggested line in the stand-up file was `block TW-1 waiting on the enterprise GitLab admins`. Jim compiled it, and the block was on the record. The flags grow with the days: a blocker reported two days running is amber, three days red; the same "today" three days running is amber ("is this stuck?"); a story id that is not in the sprint, or not in the journal at all, is flagged; someone who says "done" in Y gets a suggested `done` line to confirm. None of these is applied automatically. The kit suggests, the lead confirms.

> **Case study: What the terse format did for the team**
>
> By the end of sprint 2 Bob, who had complained loudest about "status theater", said the one-line status was the first status format that had ever told Bob anything about the other two's work. It also made the blockers impossible to bury: the enterprise administrators' slow runner request was red on the weekly report by its third day, and management called the administrators before the team had to.

24.3 Planning poker

Planning poker is a way to estimate that avoids the loudest voice setting the number. Everyone votes at the same time, in points, without seeing the others' votes; if the votes differ, the highest and lowest explain their reasons, and the team votes again. The discussion is the point: a wide spread means people understand the story differently.

The team plays it in the meeting chat with the kit's `#est` rounds. Jim posts `#est` and the story; each person posts a number; the kit tallies the round from the pasted chat. Here is the refinement session of Wednesday 14 October, sprint 1, week 2, with three candidate stories for sprint 2:

```
cat > standups/2026-10-14-chat.txt <<'CHAT'
[10:02 AM] Jim
Refinement. Three candidates for sprint 2. Vote in points: 1 point = one ideal day.
#est TW-6 import drop 1 on vendor/prime and merge to develop
[10:03 AM] Bob
3
[10:03 AM] Tom
3
[10:03 AM] Jim
2
[10:05 AM] Jim
#est TW-7 docs, outline and TODO jobs
[10:06 AM] Tom
3
[10:06 AM] Bob
8
```

```
[10:06 AM] Jim
3
[10:09 AM] Bob
the docs job means generating docs for the first time, model.pl docs warned on our model. that is
not 3 days
[10:12 AM] Jim
#est TW-7 docs, outline and TODO jobs (revote)
[10:13 AM] Tom
5
[10:13 AM] Bob
5
[10:13 AM] Jim
5
[10:15 AM] Jim
#vote Split TW-7 so the docs job is its own story? y/n
[10:15 AM] Bob
y
[10:15 AM] Tom
n
[10:16 AM] Jim
n
CHAT
grep -c "#est" standups/2026-10-14-chat.txt      # three rounds
```

```
perl $KIT/bin/daily.pl --today 2026-10-14 chat standups/2026-10-14-chat.txt
```

```
ESTIMATE TW-6 import drop 1 on vendor/prime and merge to develop  (3 answers at 10:02)
  Bob            3
  Jim            2
  Tom            3
  no consensus: median 3, mode 3, range 2-3

ESTIMATE TW-7 docs, outline and TODO jobs  (3 answers at 10:05)
  Bob            8
  Jim            3
  Tom            3
  no consensus: median 3, mode 3, range 3-8 — re-discuss with Bob

ESTIMATE TW-7 docs, outline and TODO jobs (revote)  (3 answers at 10:12)
  Bob            5
  Jim            5
  Tom            5
  consensus: 5

VOTE Split TW-7 so the docs job is its own story?  (3 votes at 10:15)
  n           2  Jim, Tom
  y           1  Bob
  result: n (majority)
appended to standups/2026-10-14.txt (uncomment the est lines you accept; note lines apply as-is)
```

Three things in that output are worth slowing down for.

The real disagreement was on TW-7. Bob voted 8 against two 3s, and explained: generating the model documents for the first time had produced warnings, so the docs job was not a routine

job. After Bob's explanation the team voted 5, unanimously. The estimate moved because of information, not pressure. That is what poker is for.

Bob's explanation did not count as a vote. It contained a number ("that is not 3 days"), but the kit reads only answers as votes: a number with an optional unit, optionally after "actually" or "make it" and the like. Anything else in a round is discussion, so Bob's 8 stood, and the tally flagged it for re-discussion ("re-discuss with Bob"). People can still change their vote by posting a new answer ("actually 5"); the latest answer wins.

The kit suggests; Jim decides. The tally appends lines to the day's stand-up file, commented out, for Jim to accept by removing the semicolon:

```
; ---- chat 2026-10-14
== Model
; est TW-6 3   ; median, no consensus: Bob 3, Jim 2, Tom 3
; est TW-7 3   ; median, no consensus: Bob 8, Jim 3, Tom 3
; est TW-7 5   ; consensus: Bob 5, Jim 5, Tom 5   ; task not in journal yet
note vote Split TW-7 so the docs job is its own story? -> n (n 2, y 1)
```

Jim accepted the revote (`est TW-7 5`) and left the others commented. TW-6 stayed at the 3 it was proposed at.

Note: Which team section the lines go under

Est lines for tasks the journal already knows go under that task's team. The rest – stories not filed yet, like TW-7 here, and the vote notes – go under `default_team` from `scrum.conf` (the team set it to `Model` in week zero), or under the team in the chat file's name (`standups/DATE-Model-chat.txt`). Look at the end of the file the tally wrote:

```
tail -6 standups/2026-10-14.txt
```

With neither set, the tally says so (`chat: no team for est lines of tasks not in the journal yet`) and leaves those lines under a bare ==, where an uncommented `est` line would fail the compile. Set `default_team` once and the question never comes up.

24.3.1 How big is a point?

The team's scale was 1 point = one ideal person-day: a day with no meetings and no interruptions. A task may be at most 5 points (under a week); a story is a few tasks and must fit in one sprint; Jim's capacity was about 5 points a sprint (the rest of Jim's time went to management), Bob's and Tom's about 8. Using ideal days rather than abstract "story points" was a concession to the team's background: systems engineers estimate in days, and pretending otherwise would have cost more trust than it bought.

24.4 Refinement and the Definition of Ready

Refinement (also called backlog grooming) prepares the next sprint's stories so planning is quick. It happens on the Wednesday of the second week, so there is time to fix what it finds

before Monday. Jim brings candidate stories; the team splits them, estimates them, and checks each against the **Definition of Ready**, kept in the wiki and in the Story template (Chapter 15, Setting up GitLab (p. 226)):

Checklist: Definition of Ready (a story may enter a sprint when)

- ☐ it fits in one sprint and names its epic and tome
- ☐ its acceptance criteria are written so someone who did not do the work can check them
- ☐ its tasks are cut, each at most 5 points (under a week)
- ☐ its dependencies on the prime or on another story are linked
- ☐ it has been estimated in poker, and the spread was explained

A ready story gets the `workflow::ready` label in GitLab. The refinement's journal file brings the new stories in with `new` lines (into the backlog, not yet committed), plus the accepted `est` lines:

```
cat > standups/2026-10-14.txt <<'REF'
2026-10-14
sprint 1
== Model
note Refinement: sprint 2 candidates, Definition of Ready applied
new TW-6 3 Import drop 1 on vendor/prime and merge it to develop p:1 e:"E5 Drop 1 intake and
first import" t:"T2 Port Tidewater v1 to SysML v2" o:Bob
new TW-7 4 Docs, outline and TODO jobs p:1 e:"E2 Pipeline and quality gates" t:"T1 Stand up MBSE
DevSecOps" o:Tom
new TW-8 3 Level conventions p:2 e:"E6 Structure: close the structure TODOs" t:"T2 Port Tidewater
v1 to SysML v2" o:Bob
est TW-7 5
note vote Split TW-7 so the docs job is its own story? -> n (n 2, y 1)
REF
perl $KIT/bin/daily.pl compile
perl $KIT/bin/daily.pl --today 2026-10-14 backlog Model
```

(Jim rewrote the day's file with the refinement lines and the accepted estimate, replacing the tally's commented suggestions.) After compiling, the backlog showed the three stories with TW-7 at its new 5 points:

```
Model backlog: 3 tasks, 11 SP
ID    SP  Prio  Tome                              Epic                                  Owner
Age  Title                                              Blocked
----  --  ----  --------------------------------  ------------------------------------  -----
---  --------------------------------------------------  -------
TW-6   3     1  T2 Port Tidewater v1 to SysML v2  E5 Drop 1 intake and first import     Bob
  0  Import drop 1 on vendor/prime and merge it to develop
TW-7   5     1  T1 Stand up MBSE DevSecOps        E2 Pipeline and quality gates         Tom
  0  Docs, outline and TODO jobs
TW-8   3     2  T2 Port Tidewater v1 to SysML v2  E6 Structure: close the structure TODOs  Bob
  0  Level conventions
```

24.5 Kickoff and planning

The sprint starts on Monday morning with ninety minutes that do three things:

1. **The goal.** One sentence that says what the sprint is *for*, in terms management would recognize. Sprint 1's: "A GitLab project the team can work in; drop 1 received and assessed." A goal lets the team make trade-offs during the sprint without asking: anything that serves the goal wins.
2. **Capacity.** Each person's available days, minus holidays, leave and known meetings, summed into the `cap` line. Sprint 1 lost a day to the Columbus Day holiday; sprint 4 lost Thanksgiving and a day of Jim's leave.
3. **Commitment.** The team pulls ready stories, highest priority first, until the total is about 85 to 95% of capacity. Each committed story gets the sprint's milestone in GitLab and its tasks are created (or confirmed) as task issues; the journal gets the `cap` and `commit` lines (Chapter 23, Stand-ups, the journal and the board (p. 367) shows sprint 1's file).

The kickoff ends with the board reset: last sprint's closed cards gone, this sprint's stories in `workflow::ready`.

24.6 The demo

The **sprint review**, or demo, is where the team shows working results to the people who care about them: management, the deputy program manager who chairs the configuration control board, sometimes the prime's modeler. The rule that made the team's demos work was borrowed from the document era's worst habit and turned inside out:

> *Show the thing, not a slide about the thing.*
>
> — Jim's note on the demo agenda, sprint 1

So the demo shows real artifacts, in the tools: the merge request with its pipeline widgets, the board, the TODO count falling, the trace report, from sprint 4 the model website. Sprint 1's demo showed the project and its protections, a merge request blocked by a lint failure, and the drop 1 TODO inventory of 52 lines. It took twenty minutes.

At the end of the demo, Jim marks the stories the audience accepted as done in the journal, and runs the sprint report:

```
printf '2026-10-16\nsprint 1\n== Model\ndone TW-1 TW-2 TW-3 TW-4 TW-5\nnote Sprint 1 review: all
five accepted\n' > standups/2026-10-16.txt
perl $KIT/bin/daily.pl compile
perl $KIT/bin/daily.pl --today 2026-10-16 review 1
```

```
Sprint 1 report -- as of 2026-10-16: 17/17 SP done (100%), 0 carried over, 0 removed.

Team           Commit  Done  Done%  Carry Carry%   Load Velocity  Predict%  Punted Interrupt
Cap
Model              17    17   100%      0     0%    89%     17.0      100%       0         0
19
```

```
Epics (remaining / team velocity = sprints to done):
  T1 Stand up MBSE DevSecOps > E1 GitLab project and gitflow  100%     0 SP left  ETA sprint 1
  T1 Stand up MBSE DevSecOps > E2 Pipeline and quality gates   44%     5 SP left  ETA sprint 2
  T1 Stand up MBSE DevSecOps > E3 Team practices and onboarding  100%     0 SP left  ETA sprint 1
  T2 Port Tidewater v1 to SysML v2 > E5 Drop 1 intake and first import   70%     3 SP left  ETA
  sprint 2
  T2 Port Tidewater v1 to SysML v2 > E6 Structure: close the structure TODOs    0%     3 SP left
  ETA sprint 2
wrote reports/sprint-1-report.txt
wrote reports/sprint-1-report.html
```

(from the practice journal; the real sprint 1 is in Chapter 26, Sprint 1: A GitLab project the team can work in (p. 404)). Unfinished stories are either carried into the next sprint (`carry`) or returned to the backlog, never quietly left open.

24.7 The retrospective

The **retrospective** is the team talking to itself about how it works, not what it built. The format the team used:

1. **The pulse survey**, five questions answered anonymously on a 1 to 5 scale (Chapter 3, The directive (p. 22) defines them): I understand why we are working this way; I can do my work in the new tools without help; the checks catch problems I would have missed; I can see our progress; I would choose to keep working this way. Five minutes, on paper or a form. The averages go on the scorecard (Chapter 33, The twelve-week scorecard (p. 520)).
2. **Start, stop, continue.** Each person writes what the team should start doing, stop doing and keep doing.
3. **One or two changes.** Not ten. Each change becomes a story or a working-agreement edit, with an owner.
4. **Last sprint's changes.** Did they help? Keep or drop.

The retrospective is the one meeting where nothing is recorded in the journal except the survey average and the changes. What people say stays in the room. That rule mattered: the team was discouraged at the start (Chapter 3, The directive (p. 22)), and the retrospective had to be a place to say so.

The changes that came out of the first six retrospectives (the report widgets, the ratchet, the "keep numbers out of chat comments" rule, the second runner) are told in Part III, and the pulse survey's arc in Chapter 38, How the team changed its mind (p. 557).

24.8 Telling management every week

Every Friday, Jim sends management the **quad**: one page in four quadrants, generated from the journal. The practice journal's at the end of sprint 1:

```
perl $KIT/bin/daily.pl --today 2026-10-16 quad Model
```

```
Weekly status Model -- as of 2026-10-16 (sprint 1)
Sprint progress 100% (17/17 SP) improving (was 18%)   On-time delivery: 4/4 this week, 5/5 this
sprint

TECHNICAL PRIORITIES
  DONE  TW-1         4  Create the GitLab group and mo Tom
  DONE  TW-2         4  First pipeline gates every MR  Tom
  DONE  TW-3         3  Receive drop 1 and record it   Jim
  DONE  TW-4         4  Convert drop 1 and assess it   Bob
  DONE  TW-5         2  Working agreements             Jim
  TODO  TW-6         3  Import drop 1 on vendor/prime  Bob
  TODO  TW-7         5  Docs, outline and TODO jobs    Tom
  TODO  TW-8         3  Level conventions              Bob

WATCH ITEMS / PM HELP NEEDED
  none

SCHEDULE MILESTONES
  30 days out
    1. T1 Stand up MBSE DevSecOps > E2 Pipeline and quality gates <-  44%  ETA 2026-11-01
    2. T2 Port Tidewater v1 to SysML v2 > E5 Drop 1 intake and first import <-  70%  ETA
    2026-11-01
    3. T2 Port Tidewater v1 to SysML v2 > E6 Structure: close the structure TODOs +   0%  ETA
    2026-11-01

ACCOMPLISHMENTS (since 2026-10-09)
  v TW-1         4  Create the GitLab group and mo Tom
  v TW-2         4  First pipeline gates every MR  Tom
  v TW-4         4  Convert drop 1 and assess it   Bob
  v TW-5         2  Working agreements             Jim
```

(an excerpt; in your journal every story was done on the Friday, so the on-time line reads 5/5 this week, and the trend arrows differ). Watch items are where a blocker more than three days old appears with "PM help needed", which is how the runner request reached management in the real sprint 1. Chapter 36, Telling management (p. 538) covers the quad, the sprint mail and the twelve-week briefing in full.

24.9 Exercises

Exercise 24.1: Lint your own status

Write yesterday's work as a Y/T/B line in a chat file, in the Teams copy format above, with one deliberate mistake. Run `daily.pl lint` on it, then `--reply`. Fix it until the lint is clean.

Exercise 24.2: Play a round

With two colleagues, estimate three of your real tasks in an `#est` chat and tally it with `daily.pl chat`. Where did the spread come from? Did discussion change the number?

Exercise 24.3: Apply the Definition of Ready

Take a work item from your current plan and check it against the Definition of Ready. What would you have to do to make it ready? Which item failed first?

Exercise 24.4: Plan a demo without slides

Write the agenda for a twenty-minute demo of your team's last two weeks that shows only real artifacts. What would you show, in what order, and what question do you want the audience to answer?

24.10 Applying this to your project

- **Copy**: terse written statuses with an id in every part, linted during the call; poker in the chat with a revote after the outliers explain; refinement mid-sprint against a written Definition of Ready; a one-sentence sprint goal; demos that show the thing; a short retrospective with a pulse survey and one or two changes; a weekly one-page status generated, not written.
- **Adapt**: the times (a team across time zones may need asynchronous stand-ups in the chat alone); the scale (ideal days or abstract points, but pick one and keep it); who attends the demo.
- **Decide locally**: what goes upward and what stays in the team; who confirms the kit's suggested lines; which meetings these replace, agreed with management in writing so the calendar actually gets lighter.

CHAPTER

25

Talking about progress: helps for engineers who would rather not

From what you know →

You know

briefing a chief from a slide deck you rehearsed the night before.

→ Here it is called

talking about progress: the stand-up line, the demo and the weekly note.

→ What is new

one short sentence a day, outcome first, with one number, and tools that draft it for you.

Survival kit

What you have

a stand-up every morning (Chapter 24, The rhythm: stand-ups, poker, refinement, kickoff, demo, retro (p. 377)), a demo every second Friday, a weekly note to management, and a GitLab project that already knows almost everything you did.

What you need

words. Short ones, in an order management can follow, said out loud by people who have spent a career letting the documents speak for them. This chapter gives you a pattern, a ladder, a stakeholder map, a few scripts for bad news, and a tool that drafts the first version of every sentence from GitLab so you only have to correct it.

25.1 Why engineers go quiet

The first two weeks of Tidewater stand-ups produced statuses like these, copied from the meeting chat:

```
Y fixed lint warnings on the connector ends in PowerSystem T more of the same B none
Y rebased my branch after the vendor merge T trace job B waiting on the runner
Y stuff on TW-4 T TW-4 B none
```

Each one is true. None of them tells the program manager, who started coming to the sprint reviews in sprint 2, whether the port is on schedule. And in the sprint 2 demo, Tom spent eleven minutes showing the pipeline file line by line, while the program manager checked e-mail and the deputy program manager said, at the end, "I did not follow the demo. What did we get for the two weeks?" Nobody answered for a long moment.

It is worth understanding why this happens, because the fix is different for each cause.

- **The precision instinct.** Systems engineers are trained not to claim more than they can show. "On track" feels like a claim; "fixed lint warnings on the connector ends" feels like a fact. So the fact is said and the claim is left for the listener to infer, which the listener cannot do.
- **Fear of being wrong in front of managers.** For thirty years the team's statements to management were reviewed, edited and signed before anyone said them. A live sentence with a number in it, said without a document behind it, feels exposed.
- **Not knowing what the audience needs.** The program manager needs a different sentence from the ISSM, and both need a different one from Bob. Engineers default to the sentence *they* would want to hear: the mechanism.
- **Talking in mechanism.** Mechanism is what the engineer spent the day on, so it is what is in their head. "Rebased my branch" is the most recent memory, not the most important news.
- **No practice.** Nobody on the team had ever rehearsed a status. Documents were drafted, reviewed and redlined; spoken updates were improvised. Improvisation under mild stress produces mechanism or silence.

None of these is a character flaw, and none is fixed by telling people to "communicate better". Each is fixed by a structure: a pattern that puts the claim first, a number that backs it, a tool that drafts it, and a rehearsal.

> *I wasn't hiding anything. I just didn't know which of the forty things I did was the one they wanted.*
>
> — Bob, sprint 2 retrospective

25.2 The pattern: outcome, evidence, next or ask

Every status the team says to anyone outside the team follows one shape:

1. **Outcome**: what is now true that was not true before, in the listener's words.
2. **Evidence**: one number that proves it.
3. **Next step or ask**: what happens next, by when; or what you need, from whom, by when.

The mechanism, the *how*, is kept in reserve. If someone asks, you have it; if nobody asks, nobody needed it.

Table 25.1: Stand-up statuses, before and after

Instead of	Say
"Y fixed lint warnings on the connector ends in PowerSystem"	"Y TW-12 Stations structure done: nine TODO lines closed, forty-three left. T TW-15 decide who owns the observation record, by Thursday. B none"
"B waiting on the runner"	"B TW-3 checks can't run on every change until IT approves the runner machine. Need: Jim to escalate today"
"T rebasing my branch after the vendor merge"	"T TW-31 the prime's January update: a list of what it changes for us by Wednesday"
"Y trace job is red"	"Y TW-22 the check found two requirements nothing satisfies. They're the prime's to fix; the report goes to them Friday"
"Refactoring the library package"	"Cleaning up shared definitions so the next change takes one edit instead of ten"

Notice three things about the right-hand column. Every line still fits the kit's terse format (Y, T and B, each naming a story), so the lint in Section 24.2, The daily stand-up: terse, written, linted (p. 378) still passes. Every line has exactly one number; two numbers make the listener choose, and they choose wrong. And every blocker is an *ask with a name and a day*, because management can act on "Jim to escalate today" and cannot act on "waiting".

Three sentence shapes cover most days. Copy them onto a card (Appendix H, Quick reference: saying it so they hear it (p. 745)) and keep it beside the keyboard:

- "**[X] of [Y] [things] are [done / covered / passing], up from [Z].**" Forty-three of fifty-two TODO lines left, down from fifty-two.
- "**We caught [problem] at [early stage] instead of [late stage].**" The check caught a broken interface at merge, the same day, instead of at integration.
- "**Blocked by [thing]. Need [specific help] from [person] by [day].**"

Tip: The "so what?" test

Read your sentence and ask "so what?" as the program manager would. If the honest answer is "so nothing, I just did it", the sentence is mechanism: replace it with the outcome it served, or leave it out. If the answer is "so we're one step closer", say how close, with the number.

25.3 The ladder: one sentence, thirty seconds, five minutes

Management will give you one of three slots, and you rarely know in advance which. Prepare all three for anything you will talk about this week.

Table 25.2: The ladder

Rung	Length	What it holds	When you use it
One sentence	about 10 seconds	the outcome and the number	the hallway, the elevator, the start of every longer version
Thirty seconds	three or four sentences	outcome, number, what it means for the next milestone, the ask	the stand-up, a question in a meeting, the opening of the demo
Five minutes	the thirty seconds plus one piece of evidence shown and one before-and-after	the demo, a briefing slot	the sprint review

Write the one-sentence version first. If you cannot, you do not yet know what the outcome is, and no amount of detail in the five-minute version will hide that. Then say each rung out loud, once, to a colleague or to the wall. Saying it is different from reading it: you will hear the word you trip over, and that word is usually jargon.

Here is Tidewater's sprint 4 review, on all three rungs, as Tom (the narrator that sprint) rehearsed it on Day 9:

> *One sentence: The model site management asked for is live, and the check now shows only 22 of the 52 conversion items left.*
>
> *Thirty seconds: The model site is live and rebuilt every night, so anyone on the program can see the current model without asking us. The conversion is more than half done, and the checks found four gaps in the prime's requirements that we have reported to them. For the January drop, that means we can tell you what it changes for us within a day. We need the second runner approved by the 15th.*
>
> — Tom, sprint 4 review, rehearsed version

The five-minute version was the thirty seconds, then the live site on the screen, then one before-and-after, then the ask again. Nothing else.

25.4 Tools that draft the words

Writing the outcome-first sentence is the hard part, and it is hardest at 8:55 in the morning. Most of what it needs is already in GitLab: which tasks you closed, which merge requests merged, which issue is blocked and why. The kit's `talk.pl` reads that and drafts the sentence. It never posts anything; it prints a draft for you to correct.

`talk.pl` has four commands:

Table 25.3: talk.pl

Command	Drafts	From
`standup --as USER`	one Y/T/B line, outcome first, one number, a blocker as an ask	your closed tasks and merged merge requests since the last working day; the task in progress; issues labelled `workflow::blocked`

Command	Drafts	From
`review --sprint N`	a demo script: the question, the four headline numbers, what to show, a before-and-after, a safe live change, the ask, and the ladder	the sprint's milestone, issues, merge requests, pipelines and their metrics
`weekly`	a plain-language note for management: a headline that is a conclusion, three numbers, risks, asks	the same, for the week
`translate FILE`	a list of jargon in your text with plain alternatives, and the sentences that fail the "so what?" test	your draft

The kit also has `daily.pl propose`, which drafts each person's status from yesterday's answers in the journal (a "+1" in the chat accepts it). `propose` is good for days when nothing changed; `talk.pl` is good for days when something did, because it reads GitLab, where the change is recorded.

25.4.1 Try it on a practice project

The examples in this section run against the offline GitLab of practice lesson 11 (see Section 12.4, A practice repository (p. 173)). First, give it a small sprint to talk about: a story with three tasks for Bob, a blocked task for Tom with the reason in a comment, and two of Bob's tasks closed on Wednesday. Each `gl` command is an ordinary GitLab action; `--now` stamps it with a date, so the example reads the same whatever day you run it. Run everything in this section from the sandbox's `work` folder (`cd work` first).

```
. ../env.sh > /dev/null
at() { d=$1; shift; gl --now "$d" "$@"; }
at 2026-10-05T09:00 --as jim issue new "Close the Stations level tags" --type issue --label
type::story --milestone "Sprint 1" --assign bob
at 2026-10-05T09:10 --as jim issue new "Level tags in StationSegment and ObservingStation" --type
task --label type::task --milestone "Sprint 1" --assign bob --description "Part of #2"
at 2026-10-05T09:11 --as jim issue new "Level tags in the station parts" --type task --label
type::task --milestone "Sprint 1" --assign bob --description "Part of #2"
at 2026-10-05T09:12 --as jim issue new "Decide who owns ObservationRecord" --type task --label
type::task --label workflow::in-progress --milestone "Sprint 1" --assign bob --description "Part
of #2"
at 2026-10-05T09:20 --as jim issue new "Register the second runner" --type task --label
type::task --label workflow::blocked --milestone "Sprint 1" --assign tom
at 2026-10-06T15:00 --as tom issue note 6 "Blocked by: IT approval of the runner machine -- need:
Jim to escalate to the IT lead today"
at 2026-10-07T16:00 --as bob issue close 3
at 2026-10-07T16:30 --as bob issue close 4
```

You are now in the sandbox's `work` folder; the examples below stay there. (Each one sources `../env.sh` again so it also works in a new window.) Now draft Thursday's stand-up for Bob. `--prefix TW-` makes the story ids look like the journal's; `--why` adds the line that says where each part of the draft came from.

```
. ../env.sh > /dev/null
perl $KIT/tools/gitlab/talk.pl standup --as bob --prefix TW- --date 2026-10-08 --why
```

```
Y TW-2 Close the Stations level tags: 2 of 3 tasks done. Reviewed !1. T TW-2 decide who owns
ObservationRecord, by Oct 16. B none
# draft for bob on Thu Oct 8 (since Wed Oct 7): edit it, then post it yourself
# from: #3 closed, #4 closed, reviewed !1, today: #5 (story #2)
# lint: ok
```

The draft already has the shape: the outcome (the story's tasks), one number (two of three), the next step with a day (the end of the sprint, which is a default worth correcting to "by Thursday" if you know better), and a lint check, so the line will not bounce in the meeting. What Bob actually posted that morning was:

```
Y TW-2 Stations level tags: 2 of 3 done, the part tree now shows every station's level. T TW-2
decide who owns ObservationRecord, by Thu. B none
```

Bob kept the number, added what it means ("the part tree now shows every station's level"), dropped the review of !1 (true, but not what the team needed to hear that morning), and tightened the date. That is the intended use: the tool does the remembering, the engineer does the meaning.

Tom's draft turns the blocked task into an ask, using the reason Tom wrote in the issue's comment:

```
. ../env.sh > /dev/null
perl $KIT/tools/gitlab/talk.pl standup --as tom --prefix TW- --date 2026-10-08
```

```
Y none T none B TW-6 register the second runner blocked: IT approval of the runner machine --
need: Jim to escalate to the IT lead today
```

"Y none T none" is honest, and it is also the moment to say something out loud: what *did* Tom do on Wednesday? GitLab only knows what was recorded. If you spent the day on a meeting with the ISSM, the tool cannot know it, and you should write it in.

Warning: Read every number before you say it

A draft is a draft. In this practice project every change was made by a script in the same second, so `talk.pl`'s timing figures come out as zero minutes; on a real project they are real. On every project, check each number against what you know before you say it in a meeting. The tool counts what GitLab recorded; you know what actually happened.

25.4.2 The weekly note

On Friday, the weekly note drafts the headline and the asks. Add an open discrepancy report so there is something to count, and a small file of document-era numbers (the "baseline") so the review can make its before-and-after:

```
. ../env.sh > /dev/null
gl --now 2026-10-08T10:00 --as jim issue new "DR-1: TW-SYS-006 text differs between DOORS and the
model" --label type::discrepancy --label owner::prime --milestone "Sprint 1" > /dev/null
printf 'review_hours = 360 | a document review cycle: 3 weeks of comments on a Word
file\nlead_hours = 552 | a change through the CCB: 23 working days\n' > ../baseline.txt
perl $KIT/tools/gitlab/talk.pl weekly --date 2026-10-09 --baseline ../baseline.txt
```

```
Weekly note, week of Oct 5 -- On track

On track: 0 stories finished this week.

The numbers:
- 0 stories finished this week (2 changes accepted into the model)
- 1 open discrepancy reports and prime changes

Risks:
- #6 Register the second runner is blocked: IT approval of the runner machine.

Asks:
- Jim to escalate to the IT lead today (#6)

(The weekly quad -- daily.pl quad -- has the detail; this note is the headline.)
```

Two edits make this sendable. "On track: 0 stories finished" will alarm a reader in week one of a sprint, so Jim adds the context the tool does not have ("two of three tasks in the first story are done; stories finish at the end of the sprint"). And the risk deserves a date: "if the runner is not approved by the 15th, changes wait for one machine". The note pairs with the kit's weekly quad (Section 24.8, Telling management every week (p. 387)), which carries the detail.

25.4.3 The review script

Before each sprint review, the person who will narrate drafts the demo script:

```
. ../env.sh > /dev/null
perl $KIT/tools/gitlab/talk.pl review --sprint 1 --baseline ../baseline.txt
```

```
Sprint 1 review -- demo script (draft: edit before the meeting)

1. THE QUESTION THIS SPRINT ANSWERED
"What did Sprint 1 change for the model, and how do we know?"
Answer in one line: 0 of 1 stories done.

2. THE SAME FOUR NUMBERS AS EVERY REVIEW
- TODO lines left: n/a (no todo metrics yet)
```

```
- Verify coverage: n/a (no trace or VCRM metrics yet)
- Open discrepancy reports and prime changes: 1 (up 1 from 0)
- Blocked items: 1
...
7. THE ASK
- Need: Jim to escalate to the IT lead today (#6 Register the second runner is blocked: IT
approval of the runner machine)

30-SECOND VERSION
Sprint 1: 0 of 1 stories done. We need help with 1 blocked item.

1-SENTENCE VERSION
Sprint 1 moved the model forward by 0 stories.
```

The script has seven parts, which are the seven moves of a demo in Section 25.6, Demos that land (p. 398). On the practice project the numbers are thin: "n/a" for TODO lines and verify coverage, because practice lesson 11's pipeline has no TODO or trace job. On the Tidewater project those come from the pipeline's metrics, and Part III shows the real drafts for every sprint review next to what the team actually said.

25.4.4 Translating your own draft

The fourth command is for text you wrote yourself: a status, a demo opening, a paragraph for the weekly note. It does *not* rewrite your sentences. It lists the jargon it found with a plain alternative and a hint, and it flags the sentences that fail the "so what?" test, with a fill-in suggestion in the outcome, evidence, ask shape.

```
printf 'Rebased my feature branch and fixed the lint errors so the pipeline is green.\nWorking on
the VCRM.\n' > draft.txt
perl $KIT/tools/gitlab/talk.pl translate draft.txt
```

```
Jargon to replace:
  "the pipeline is green" -> all automated checks passed. Hint: say what that proves (e.g. "no
  grammar or lint errors").
  "lint errors" -> errors the automated checks found. Hint: say how many, and that they were
  caught before review.
  "Rebased" -> brought my change up to date. Hint: say what it achieved, or leave it out: nobody
  needs to hear it.
  "feature branch" -> private working copy of the model. Hint: usually leave it out: say what the
  change does.
  "VCRM" -> verification matrix. Hint: say how many requirements are verified.

So what? -- sentences to rework:
  1. "Rebased my feature branch and fixed the lint errors so the pipeline is green." -- no number
     SUGGESTION: Done: [what is now true] ([one number: how many, how much]). Next: [outcome, by
     a day]. Need: [an ask, or none].
  2. "Working on the VCRM." -- no number; no outcome (done, caught, ready, decided ... or an ask)
     SUGGESTION: Done: [what is now true -- topic: verification matrix] ([one number: how many,
     how much]). Next: [outcome, by a day]. Need: [an ask, or none].
```

The rework of those two sentences, by Tom, became: "Done: my change to the power system is checked and accepted, with no errors found. Next: the verification matrix shows which requirements are verified, by Friday. Need: none." Longer than the original, and something a manager can repeat to *their* manager, which is the real test.

You can give `translate` your own glossary with `--glossary FILE` (lines `term = plain alternative | hint`), so the team's words, the prime's acronyms and your program's jargon get the same treatment.

25.5 Know your audience: the Tidewater stakeholder map

The same week sounds different to different people. Before a meeting, ask who is in the room and what one number each of them wants. The Tidewater team kept this table on the wiki and updated it after every review:

Table 25.4: Who wants which number

Who	Cares about	The one number they want	Say it as
Program manager	Is the money producing measurable results? Will the December review go well?	TODO lines left, and the trend	"22 of 52 left after sprint 4; about two sprints to go at this pace."
Deputy program manager (chairs the CCB)	Can a change be traced from request to approval to implementation?	changes accepted with a green pipeline and a code-owner approval	"Every change this quarter is traceable from its issue to its merge."
ISSM	Is the toolchain approved, and is anything leaking?	open security findings, last secret-detection result	"Secret detection ran on every change; zero findings since the runner was hardened."
The prime's modeler	Which of our discrepancy reports are open, and what do we need from the next drop?	open DRs, oldest first	"7 open, none closed yet; the oldest is the TW-SYS-006 wording, open since sprint 4."
The test organization	Which requirements have a verification case and a result?	verify coverage	"9 requirements still have no verification case, and all of them are the prime's to close."
The team itself	Are we getting better at this?	the pulse survey, the share of stand-up lines with an outcome and a number	"Morale up from 1.9 in sprint 1; most statuses now have a number."

The numbers in this table are Tidewater's at the end of sprint 4 or 6, from the evidence; in Part III every sprint chapter shows them sprint by sprint.

25.6 Demos that land

The demo is where the quiet engineer is most exposed: a live screen, managers in the room, and no document to hide behind. Tidewater's sprint 2 demo went badly (Tom's eleven minutes on the pipeline file). The sprint 4 demo, after one retro change and a rehearsal, went well. The change was not "Tom should present better". It was a structure.

25.6.1 Three roles

Table 25.5: Demo roles

Role	Does	Says
Driver	runs the keyboard: opens the merge request, the site, the board	nothing, or "here"
Narrator	tells the story in the listener's words	the question, the outcome, the before-and-after, the meaning
Note-taker	writes down every question, comment and request	nothing during; reads them back at the end

Splitting driving from narrating is the single most useful change. An engineer who is typing cannot also be choosing words for a manager; an engineer who is only talking can look at the audience. The note-taker turns the room's reactions into issues afterwards, so nothing said in the demo is lost.

The team **rotates** the roles every sprint. In sprint 4 Bob drove and Tom narrated and closed with the ask, because Jim was on leave that Friday (the meeting record, `docs/tidewater/sprints/04/meetings.csv`, lists Bob, Tom, the deputy program manager and the program manager); in sprint 5 Tom drove and Bob narrated. Rotation means everyone speaks to management at least once a month, and nobody becomes "the presenter" by default while the others stay quiet.

25.6.2 The Day 9 dry run

On Day 9 (Thursday of week 2), after refinement, the team spends thirty minutes running the demo once, out loud, with the roles they will have on Friday. Nothing is polished; the point is to hear it. The dry run catches:

- The demo step that needs a password nobody has typed this sprint.
- The jargon the narrator trips over ("so the MR's pipeline, I mean the merged-results pipeline...").
- The number nobody can explain ("why did the TODO count go *up* on Tuesday?" Because the drop-2 preview added three).
- The ask nobody has written down.

25.6.3 The six moves

A demo script, whoever drafts it, has six moves. `talk.pl review` drafts them; you edit them.

1. **Open with the question this sprint answered.** "Can we show every station requirement has a verification? Yes, twenty-three of twenty-six. Here are the other three."
2. **Show the output, not the tool.** The model site, the verification matrix, a merge request going from red to green, the data exchange table. Keep the `.sysml` text and the pipeline file ready in case someone asks.
3. **One before-and-after, with a number.** "Finding where DOORS and the model disagree used to be done never, or once before a review. Now it takes two minutes, on every change."
4. **One safe live change.** On a scratch branch, never on `develop`: remove one satisfy link and open a draft merge request; in about four minutes the trace check names the requirement

that lost its design. Cause and effect, live, lands better than any slide. Close the merge request afterwards.
5. **What it means for the next milestone.** "For the January drop, we'll know what it changes for us within a day."
6. **The ask, with a date.** "We need the deputy program manager's decision on the delivery agreement by Friday."

And close the loop: the note-taker reads back the questions, and each becomes an issue before the end of the day.

25.6.4 The same four numbers, every time

Management learns to read a trend only if it sees the same numbers, in the same order, every review. Tidewater's four:

1. **TODO lines left**: how much of the conversion is still to do.
2. **Verify coverage**: how many requirements have a verification case.
3. **Open discrepancy reports and prime changes**: what is waiting on the prime, or on us.
4. **Blocked items**: what needs management's help.

They open every review, every weekly note and every memo. A headline states the conclusion of the numbers ("Trace: 29 of 30 linked, green"), not the topic ("Trace analysis").

25.7 Bad news, said so it builds trust

Engineers dread bad news more than managers do. Managers expect it; what they cannot forgive is hearing it late. These are the phrases the Tidewater team used, each tied to a real moment.

Table 25.6: Bad-news scripts

Situation	Say
A check found a problem	"Found early. That's the system working. Here is the impact and the fix."
A number got worse because you can now see it	"It's not new, it's newly visible. The checks show twelve requirements without a test; here is the date each closes."
You are behind	"This is behind by a week because the ISSM review moved. To recover we need the review slot on the 6th, or we can drop the second runner from this sprint."
You do not know	"I don't know yet. I'll know by Thursday, after the impact report."
Something broke on your watch	"The develop pipeline was red for a day after my merge. It's green now; the cause was X; the check that would have caught it is in this sprint."

Every row has the same skeleton: the fact, the cause in one clause, and what happens next with a date. None of them apologizes at length, and none of them blames.

25.7.1 Jim models "I don't know yet"

The hardest of these for the team was "I don't know yet", because for thirty years not knowing was something you fixed before the meeting. In sprint 3 the deputy program manager asked, in the review, whether the prime's January drop would be "a big one". Jim did not know, and said so:

> *I don't know yet. The prime's modeler said "a few sensor changes". I'll know by the Tuesday after it lands, because the delta report lists every change and what it touches in our model. I'll send you the count that day.*
>
> — Jim, sprint 3 review

Two things happened. The program manager wrote "Tue after drop: count" on a pad and moved on, which surprised the team. And in the next stand-up, Bob said "I don't know yet, I'll know by tomorrow" about the ObservationRecord ownership question, for the first time. The lead going first is what makes it safe for everyone else.

25.8 Measuring whether it is working

Like everything else in this book, the team measured its own talking. Each sprint, Jim counted the share of stand-up lines that had both an outcome and a number. In sprint 1 it was low: most lines were mechanism. By sprint 6 most lines had both. Part III gives the real counts per sprint (Chapter 31, Sprint 6: Behavior by hand, a table for the program, and baseline-0 (p. 495)), and the scorecard in Part IV includes it.

The pulse survey's question "I can explain our progress to management in one sentence" moved too. That question was the one Jim watched most closely, because it is the one that tells you whether the engineers will speak up when the lead is on leave.

25.9 Exercises

Exercise 25.1: Rewrite five statuses

Take your last five spoken or written statuses (or the three at the start of this chapter). Rewrite each in the outcome, evidence, next-or-ask shape, keeping the Y/T/B format and a story id. Run them through `talk.pl translate` and fix what it flags. Which one was hardest, and why?

Exercise 25.2: Climb the ladder

Pick one thing your team finished this week. Write the one-sentence, thirty-second and five-minute versions. Say each out loud to a colleague and ask them to repeat the one-sentence version back to you. If they can't, rewrite it.

Exercise 25.3: Your stakeholder map

Make the table in Section 25.5, Know your audience: the Tidewater stakeholder map (p. 398) for your own program: every person who hears your status, what they care about, the one number they want, and the sentence. Which of those numbers can your pipeline produce today?

Exercise 25.4: Draft from GitLab

In the practice-lesson-11 sandbox, close one more of Bob's tasks with a `--now` date, then draft Friday's stand-up with `talk.pl standup`. Edit the draft so it says what the change *means*, not only what was closed.

Exercise 25.5: Run a dry run

Plan next sprint's demo with three roles and the six moves. Hold a thirty-minute dry run on Day 9. Write down every place the narrator stumbled, and change the script, not the narrator.

25.10 Applying this to your project

- **Copy**: the outcome, evidence, ask pattern; the ladder; the demo roles and their rotation; the Day 9 dry run; the four headline numbers; the bad-news skeleton (fact, cause, next with a date).
- **Adapt**: the stakeholder map (your program's people and their numbers); the four headline numbers (pick the ones that say whether *your* project is on track: for a new model rather than a port, "requirements with a design" might replace "TODO lines left"); the glossary for `talk.pl translate`.
- **Decide locally**: whether drafts from `talk.pl` or `daily.pl propose` are the default; who keeps the stakeholder map current; how often the lead reviews the share of statuses with a number (Tidewater: once a sprint, in the retro).

Note: You can now…

- say a status as outcome, one number, and next step or ask, in the stand-up format the kit lints;
- prepare one-sentence, thirty-second and five-minute versions of anything you report;
- draft stand-ups, weekly notes and review scripts from GitLab with `talk.pl`, and correct them;
- find the jargon and the "so what?" sentences in your own text;
- run a demo with three roles, six moves and a dry run;
- say "I don't know yet" with a date, and deliver bad news in a way that builds trust.

PART III

Twelve weeks

Part II set up the shop: the operating model, the laptops, git, Vim, GitLab, the pipeline, the journal and the meetings. Part III is the work itself. Each of the next six chapters is one two-week sprint of Jim, Bob and Tom's first twelve weeks, from Monday 5 October to Friday 18 December 2026, told from the record the sprint left behind.

That record is real in the only sense that matters for a study book: every stand-up status, poker round, issue, merge request, pipeline log, quad and survey answer quoted in these chapters was produced by running the kit's own tools on the Tidewater scenario, and the numbers are cited from those files rather than typed. The files are in `docs/tidewater/` beside the book; its `README.md` lists every one with a line on what it shows. When a chapter says a pipeline failed, you can open the log. When it says the team committed so many points, the number on the page was read from the sprint's scorecard when the book was built.

Each sprint chapter has the same shape, so you can read them in order or jump to the one that matches where your own team is:

- **The question and the mood.** Every sprint answered one systems engineering question, and the pulse survey at the retrospective measured how the team felt about the answer.
- **The kickoff.** The goal, the capacity, the plan, the planning-poker rounds, and the learning budget.
- **Week one and week two, day by day.** The stand-ups as they were posted, the GitLab work as it happened, and the thought process behind each decision: what the team chose, what else it could have done, and what it would do differently now.
- **Refinement, review and retrospective.** What was estimated for the next sprint, what was shown to management and how it landed, and what the team changed.
- **The numbers.** The scorecard at the end of the sprint, and what Jim told management.
- **You can now..., exercises, and applying this to your project.**

Two warnings before you start. First, the team is not good at this yet. In sprint 1 their stand-up statuses fail the lint, their first merge request has no pipeline, and their first local pipeline is red. In sprint 2 their first real pipeline dies on its first line and their demo loses the room. That is the point: the book shows what a first attempt actually looks like, so yours does not have to be perfect to be on track. Second, the early sprints are mostly about the shop rather than the model. The bottom line up front (Bottom line up front (p. iii)) shows how quickly that changes; by sprint 4 most of the board is systems engineering.

CHAPTER

26

Sprint 1: A GitLab project the team can work in

From what you know →

You know

the first weeks of a new program: kickoffs, accounts, and a plan nobody believes yet.

→ Here it is called

sprint 1: planning, stand-ups, the first drop and the first pipeline.

→ What is new

by day ten, something real is finished and measured.

Survival kit

What you have

three laptops that have never run git, a prime drop that arrived the Friday before, a directive, and a team that would rather be writing documents.

What you need from this sprint: a GitLab project with its rules switched on, laptops that pass the kit's doctor, the drop converted and measured, and an honest estimate of what the port will cost. Nothing in the model changes yet.

26.1 The question, and the mood

Every sprint in this book answered one systems engineering question. Sprint 1's was the one a systems engineer asks first about any inherited work product: **what do we actually have, and what will it cost to make it ours?**

The prime's model had arrived the Friday before, by CDRL file transfer, as an XMI export, a DOORS ReqIF export, a CSV and a manifest. Nobody on the team had opened the XMI, and nobody wanted to. The team's mood at the kickoff is best captured by the first thing Bob said that morning, which the stand-up chat preserved:

docs/tidewater/sprints/01/quotes.txt

```
2026-10-05  Bob: Why can't we just send the Word doc? It worked for twenty years.
```

The pulse survey at the end of the sprint put a number on it: a mean of 1.9 on a scale of one to five, the lowest of the twelve weeks. The five questions, and how each person answered them on Friday 16 October, were:

Table 26.1: The pulse survey at the sprint 1 retrospective (1 = strongly disagree, 5 = strongly agree)

Question	Jim	Bob	Tom
I understand why we are working this way.	2	1	2
I can do my work in the new tools without help.	2	2	2
The checks catch problems I would have missed.	2	2	2
I can see our progress.	2	2	2
I would choose to keep working this way.	2	1	2

Read that table as a baseline, not a verdict. Nobody understood yet why they were working this way, and nobody could work without help. The rest of Part III is, among other things, the story of those cells changing.

26.2 Week zero, briefly

Sprint 1 did not start from nothing. The week before it, the team spent about two person-days on things that had to happen before a sprint could: the directive meeting, the scorecard agreed with the program manager, the software request for Git for Windows and Vim, the first meeting with the ISSM, the decisions drafted, and the delivery call with the prime's modeler. The record is short enough to read whole:

docs/tidewater/sprints/00/week-zero.txt

```
Week zero, Tuesday 2026-09-29 to Friday 2026-10-02 -- before sprint 1 (hours per person; outside
the sprint backlog)

Date        Who   Hours  Activity
2026-09-24  all    1.0   The directive meeting: the program manager reads the DevSecOps memo to
the team (docs/tidewater/memos/2026-09-24-directive)
2026-09-29  jim    2.0   Kickoff with the program manager and deputy PM: the twelve-week
scorecard agreed (chapter 4)
2026-09-29  tom    2.5   Software request for Git for Windows and Vim on three laptops;
kickoff.pl doctor on Tom's own laptop (2 FAIL, 4 WARN)
2026-09-30  tom    1.5   kickoff.pl scaffold: the model repository skeleton; kickoff.pl start
--dry-run read with Jim
2026-09-30  jim    1.5   First meeting with the ISSM: what the runner and the repository will
hold (notional, unclassified)
2026-10-01  jim    2.0   Decisions D-1 .. D-10 drafted (recorded as ADRs in sprint 2)
2026-10-01  bob    3.0   Practice repository: clone, branch, commit, merge (the primer's git
chapter); "configuration management with fewer forms"
2026-10-02  jim    1.0   Delivery agreement call with the prime's modeler: stable xmi:ids, DRs
that cite v1 ids, a MANIFEST with every drop
```

```
2026-10-02  jim    0.5   Drop 1 (v1.0) arrives by CDRL file transfer
Total              15.0  hours across the three, about 2 person-days: not in sprint 1's capacity
```

Two things in it matter for what follows. First, those hours are *outside* sprint 1's capacity, on purpose: they were the cost of deciding to start, and the team did not want them hidden inside a sprint where they would make the first velocity look worse than it was. Second, the delivery call on 2 October is where Jim asked the prime for stable element ids and a manifest with every drop. Everything the team later automated about prime drops depends on that one conversation. If you take one thing from week zero into your own project, take that call (Chapter 45, The prime's model changes: receive, assess, disposition (p. 619)).

The record is in the simulation's state too; this is the first command of the chapter, and like every runnable block in this book it is tested:

```
cat sprints/00/week-zero.txt | head -3
```

26.3 The kickoff: Monday 5 October, 09:00

The kickoff ran ninety minutes and replaced the day's stand-up. Jim opened with the goal Jim had written on the whiteboard the night before:

> *A GitLab project the team can work in; drop 1 received and assessed.*

Then capacity. The standing numbers (Chapter 13, The operating model (p. 196)) are five points a sprint for Jim, whose other duties do not stop for a port, and eight each for Bob and Tom. Monday 12 October was a federal holiday, so each person's capacity was prorated by working days. Then the commitment, kept deliberately below capacity. The kit printed the result as the kickoff record, and this is the plan the team worked to for the next two weeks, exactly as it was recorded:

docs/tidewater/sprints/01/kickoff.txt

```
Sprint 1 kickoff, 2026-10-05 09:00 (90 minutes)

Goal: A GitLab project the team can work in; drop 1 received and assessed
Capacity: 18.9 points (Jim 5, Bob 8, Tom 8 per full sprint, less holidays and leave)
Committed: 16.5 points in 7 stories (87% of capacity): engineering 4.5, platform 8.0 (48%),
learning 4.0 (24%)

TW-1     E3    4.0 pts  learning     Learn the tools: survival kit, git, Vim, the operating model
           #8    1.0 pt  jim  2026-10-05 .. 2026-10-06  Practice lessons 10-11; the operating
           model chapter
           #9    1.5 pt  bob  2026-10-05 .. 2026-10-06  Practice lessons 10-11; the survival kit,
           git and Vim chapters
           #10   1.5 pt  tom  2026-10-05 .. 2026-10-06  Practice lessons 10-11; the survival kit,
           git and Vim chapters
TW-2     E3    3.0 pts  platform     Laptops ready: Git for Windows, Vim, the kit
           #11   1.0 pt  jim  2026-10-07 .. 2026-10-08  Set up Jim's laptop; kickoff.pl doctor
           #12   1.0 pt  bob  2026-10-06 .. 2026-10-08  Set up Bob's laptop; kickoff.pl doctor
```

```
          #13   1.0 pt  tom  2026-10-06 .. 2026-10-08  Set up Tom's laptop; kickoff.pl doctor
TW-3     E1    3.0 pts  platform     Create the GitLab group and model project
          #14   1.0 pt  tom  2026-10-08 .. 2026-10-09  Apply project.conf with gitlab.pl setup;
          request the runner
          #15   1.0 pt  tom  2026-10-09 .. 2026-10-13  Prove a direct push to develop is refused
          #16   1.0 pt  tom  2026-10-13 .. 2026-10-14  Commit CODEOWNERS, the templates and
          .gitignore
TW-4     E2    1.5 pts  platform     Draft the first pipeline and run it locally
          #17   0.5 pt  tom  2026-10-14 .. 2026-10-15  Draft .gitlab-ci.yml: grammar, lint,
          outline
          #18   1.0 pt  tom  2026-10-15 .. 2026-10-16  Run the draft locally on drop 1 with
          gitlab.pl ci
TW-5     E5    1.5 pts  engineering  Receive drop 1 and record it
          #19   1.0 pt  jim  2026-10-09 .. 2026-10-13  Log the CDRL delivery and check the
          sha256 of every file
          #20   0.5 pt  jim  2026-10-14 .. 2026-10-14  Open the Prime Drop issue with its
          checklist
TW-6     E5    3.0 pts  engineering  Convert drop 1 and assess it
          #21   1.0 pt  bob  2026-10-08 .. 2026-10-09  Run both converters with --report on drop
          1
          #22   1.0 pt  bob  2026-10-09 .. 2026-10-13  Write the TODO inventory by package
          #23   1.0 pt  bob  2026-10-13 .. 2026-10-14  Estimate the port backlog with the team
TW-7     E3    0.5 pts  platform     Working agreements: Definition of Ready and Done
          #24   0.5 pt  jim  2026-10-15 .. 2026-10-15  Write the Definition of Ready and Done in
          the wiki
```

Read the plan from the top down and three decisions stand out.

The learning story comes first. TW-1, "Learn the tools," is a story with points, an owner per task and a done date, like any other. It is 24% of the sprint's task points done (Section 13.8, Two ways to count "done" (p. 205)). Jim's argument, made to the program manager in week zero and repeated to the team at the kickoff, was that the learning would happen either way: on the board, where it is visible and can be protected, or on nights and weekends, where it becomes resentment. Putting it on the board also made it measurable, which mattered later when management asked what the overhead had cost (Bottom line up front (p. iii)).

Most of the sprint is platform work. Laptops, the GitLab project, the first pipeline, the working agreements: none of it changes the model. That is the honest shape of a first sprint, and the team said so in the plan rather than dressing it up. Only TW-5 and TW-6, receiving and assessing the drop, are engineering.

The commitment is below capacity. The kickoff line says how far below. Jim's first draft had been much larger (Chapter 13, The operating model (p. 196)); cutting it was, by Jim's own account later, the most useful thing Jim did in week zero, because it meant sprint 1 finished.

The journal holds the same numbers. This is the sprint report the kit prints from the ledger, run against the simulation's journal:

```
perl "$KIT/bin/daily.pl" --conf journal/scrum.conf sprint 1
```

26.3.1 Planning poker, the first time

Each story was estimated in the meeting chat with a `#est` round: Jim posted the story, each person posted a number, and the kit's tally (Chapter 24, The rhythm: stand-ups, poker, refinement, kickoff, demo, retro (p. 377)) read the chat afterwards. When the votes spread, Jim did what the method asks: the highest and lowest voters said what they knew, and the team voted again. The round that taught the team the most was the one for converting and assessing the drop:

docs/tidewater/sprints/01/poker-2026-10-05.txt

```
ESTIMATE TW-6 Convert drop 1 and assess it  (3 answers at 10:19)
  Bob            8
  Jim            3
  Tom            5
  no consensus: median 5, mode 3, range 3-8

ESTIMATE TW-6 Convert drop 1 and assess it  (3 answers at 10:21)
  Bob            3
  Jim            3
  Tom            3
  consensus: 3
```

The spread was wide. The high vote came from Bob, who had spent two years of evenings fighting Cameo's exports and expected the conversion to be a week of hand work. The low vote came from Jim, who had run the converter once in week zero and knew it does the bulk mechanically. The conversation in between was the useful part: the story was not "convert the model" but "convert it and *measure* what is left," and the measuring is a report, not hand work. The second round converged. Bob's worry did not disappear; it moved, correctly, into the later stories that close the TODO lines the converter leaves behind.

Tip: What poker is for

A spread is information, not a failure. If everyone votes the same number at once, either the story is well understood or nobody has thought about it. The second round matters less than the sentence each outlier says between the rounds. In the chat record the sentences are not there, only the numbers; write them into the story's notes when they change the plan.

26.4 Week one

26.4.1 Monday 5: the first stand-up, and the first lint

The stand-up is written (Chapter 24, The rhythm: stand-ups, poker, refinement, kickoff, demo, retro (p. 377)). Each person posts Yesterday, Today and Blockers in the meeting chat, and after the meeting Jim pastes the chat into the kit, which checks it. The very first stand-up looked like this:

docs/tidewater/sprints/01/standups/2026-10-05-chat.txt

```
Meeting started
[9:00 AM] Jim
Morning. Y/T/B in chat, please.
```

```
[9:01 AM] Jim
Y: meetings, reviews
T: TW-1 practice lessons 10-11; the operating model
B: none
[9:02 AM] Bob
Y: meetings, reviews
T: TW-1 practice lessons 10-11; the survival kit, git
B: none
[9:03 AM] Tom
Y: meetings, reviews
T: TW-1 practice lessons 10-11; the survival kit, git
B: none
```

and the kit's lint said:

docs/tidewater/sprints/01/standups/2026-10-05-lint.txt

```
standups/2026-10-05-Model-chat.txt:5: warn  Jim: no task id in Y (e.g. TW-1)
standups/2026-10-05-Model-chat.txt:9: warn  Bob: no task id in Y (e.g. TW-1)
standups/2026-10-05-Model-chat.txt:13: warn  Tom: no task id in Y (e.g. TW-1)
# 3 answered, 0 errors, 3 warnings
```

The warnings are fair. "Meetings, reviews" is what everyone wrote on a Monday in the document era, and it tells the reader nothing about the plan. The fix is not to invent work for yesterday; it is to name the story the work served, or to write that the day was the kickoff. Nobody minded the warnings on day one. The point of the lint is that it is the same every day, so the habit forms without anyone having to nag.

26.4.2 Tuesday 6: the lint finds an error, and Bob finds a phrase

On Tuesday Bob wrote the status the way one would write an e-mail:

docs/tidewater/sprints/01/standups/2026-10-06-chat.txt

```
[9:02 AM] Bob
yesterday the kickoff, then the practice repo
today practice lesson 10
no blockers
```

docs/tidewater/sprints/01/standups/2026-10-06-lint.txt

```
standups/2026-10-06-Model-chat.txt:5: ok    Jim: Y/T/B parsed
standups/2026-10-06-Model-chat.txt:9: ERROR Bob: no Y/T/B found
standups/2026-10-06-Model-chat.txt:13: ok    Tom: Y/T/B parsed
# 3 answered, 1 error, 0 warnings
```

An error, not a warning: the kit could not find the three parts at all, so Bob's day did not reach the journal. Jim sent Bob the reply line the lint suggests, Bob reposted, and that was the end of

it. It is worth noticing what did *not* happen: nobody had to read Bob's paragraph and decide whether it was a status. The format did that.

The same afternoon Bob finished the practice lessons and said the sentence the team quoted to each other for the rest of the twelve weeks:

docs/tidewater/sprints/01/quotes.txt

```
2026-10-06  Bob: Spent the afternoon in the practice repo. Branch, commit, merge. It is
configuration management with fewer forms.
```

Bob meant it as a complaint. It is also the best one-line description of git for a systems engineer that this book knows of.

26.4.3 Wednesday 7 and Thursday 8: three laptops pass the doctor

Each laptop was sized at one point in the laptop story, TW-2, after week zero had shown that a software request on a government laptop takes days, not minutes. Each person ran `kickoff.pl doctor` (Chapter 9, The survival kit (p. 126)) until it passed. Tom's final run:

docs/tidewater/sprints/01/doctor-tom.txt

```
$ perl $KIT/tools/gitlab/kickoff.pl doctor --quick
PASS  perl (version as installed)
PASS  perl core modules the kit uses (18)
PASS  git version (version as installed)
PASS  git user.name = Tom
PASS  git user.email = tom@example.invalid
PASS  git init.defaultBranch = main
PASS  git core.autocrlf = false
PASS  git pull.ff = only
PASS  git merge.conflictstyle = diff3
PASS  VIM - Vi IMproved (version as installed)
PASS  the kit's Vim setup is sourced (~/.vimrc)
PASS  curl (version as installed)
SKIP  the kit's self-test (--quick)
PASS  GitLab: the local emulator (data/tidewater/gitlab is created on first use)
WARN  no SSH key (HTTPS with a token works too)
      fix: ssh-keygen -t ed25519 -C "you@agency.gov"   then paste ~/.ssh/id_ed25519.pub into
      GitLab > Preferences > SSH Keys
doctor: 13 pass, 1 warn, 0 fail, 1 skipped
```

The one warning is a missing SSH key. The team decided to use HTTPS with a personal access token instead, because the token was simpler to get approved, and wrote that down rather than leaving the warning unexplained. A doctor run with warnings is fine. A doctor run with warnings nobody can explain is how laptops drift apart.

26.4.4 Thursday 8: drift in two minutes

The best moment of sprint 1 happened on its fourth day, and it happened to the team's biggest skeptic. Bob ran both converters on drop 1 for TW-6, the first time anyone on the team had used them on the real drop. The XMI converter reported what it carried and what it could not:

docs/tidewater/sprints/01/drop1-xmi2sysml-report.txt

```
total                           560     444      65     51

ported vs remaining (metric A filter uml:Class|Port|Property|Connector|Activity): 206 of 206
ported, 0 remaining
```

Then the requirements converter compared the prime's DOORS text with the requirement text inside the prime's own model, and printed this at the bottom:

docs/tidewater/sprints/01/drop1-trace.txt

```
C requirements linked (doorsId in the model)     29 of 30
F orphan requirements (nothing satisfies)               2
G unverified requirements (no verify)                   9
I trace completeness (satisfied and verified)        66.7%
doorsIds in the model but not in this export            1
text differs from DOORS                                 1
not in the model: TW-SYS-016
orphans: TW-SYS-012 TW-SYS-016
unverified: TW-STK-001 TW-STK-002 TW-STK-003 TW-STK-004 TW-STK-005 TW-STK-006 TW-SYS-011
TW-SYS-016 TW-SEG-002
drift: TW-SYS-006
  TW-SYS-006 DOORS: The data center shall publish a marine bulletin twice a day, on a fixed
  schedule.
  TW-SYS-006 model: The data center shall publish a marine bulletin on a fixed schedule.
```

The last three lines are the drift: the same requirement, TW-SYS-006, says "twice a day" in DOORS and does not in the model. In the document era nobody would have found it until a review, if then, because finding it meant reading two documents side by side. Bob posted in the chat:

docs/tidewater/sprints/01/quotes.txt

```
2026-10-08  Bob: I found the drift in two minutes. It used to take a week, and we only looked
before a review.
```

You can do what Bob did. These two commands convert the drop into a scratch folder with the team's layout map and run the trace against it:

```
perl "$KIT/bin/xmi2sysml.pl" --out s1/model --layout-map "$KIT/examples/tidewater-v1/layout.map"
drops/v1.0/tidewater.xmi
perl "$KIT/bin/reqif2sysml.pl" --report --model s1/model drops/v1.0/tidewater.reqif
```

and this counts the TODO lines the converter left, which is the port's starting debt:

```
grep -r "// TODO" s1/model | wc -l
```

> **Case study: Why the drift mattered more than the number**
>
> The converter's numbers were reassuring: everything in scope ported, and a manageable set of TODO lines. But numbers like that do not change minds; they confirm what the person who ran the tool already believed. The drift changed a mind, because it was a real defect in a real deliverable, found in two minutes by the person who least wanted to be there. If you are leading a team like this one, look for that moment early and let the skeptic have it.

The drift became the team's first discrepancy report to the prime later, in sprint 4 (Chapter 29, Sprint 4: the model site, a cycle broken, and the demo that landed (p. 459)). In sprint 1 it stayed a note on TW-6, because the team had not yet agreed how to report discrepancies, and Jim did not want the first thing the prime received from the new process to be improvised.

26.4.5 Friday 9: the project exists

Tom applied the project file to the GitLab group the enterprise administrators had created. The command is one line; its output is long, because it is the whole configuration of the project, read back:

docs/tidewater/sprints/01/setup.txt

```
perl $KIT/bin/gitlab.pl --as tom setup tools/gitlab/templates/project.conf
  created project tidewater/model
  settings: default_branch=develop, merge_method=merge,
  only_allow_merge_if_all_discussions_are_resolved=true,
  only_allow_merge_if_pipeline_succeeds=false, remove_source_branch_after_merge=true,
  squash_option=never
  member jim maintainer
  member bob developer
  member tom maintainer
  variable KIT
  variable DOCKER_RUNNER
```

Look at one setting in particular: `only_allow_merge_if_pipeline_succeeds=false`. In sprint 1 there was no runner, so there could be no pipeline, so the rule that a merge needs a green pipeline had to wait. The team turned it on in sprint 2, the day the runner came online (Chapter 27, Sprint 2: Drop 1 on develop, every change gated (p. 424)). Switching on a gate before the thing it checks exists is a reliable way to make a team hate the gate.

The first weekly quad went to management that afternoon. Its first two lines told the story of the first week:

docs/tidewater/sprints/01/quad-2026-10-09.txt

```
Weekly status Model -- as of 2026-10-09 (sprint 1)
Sprint progress 42% (7/16 SP) new   On-time delivery: 2/2 this week, 2/2 this sprint
```

Less than half the sprint was done after half the sprint, which is normal when the first week is learning and laptops. The quad says so without anyone having to explain it.

26.5 Week two

Monday 12 October was a holiday. The second week was four days.

26.5.1 Tuesday 13: proving a refusal

Tom's task #15 sounds odd for a systems engineer: prove that a push is *refused*. But a rule you have never seen fire is a rule you do not know works. Tom pushed a commit straight to the protected integration branch, and GitLab answered:

docs/tidewater/sprints/01/protect-proof.txt

```
$ git push origin HEAD:develop
remote: GitLab: You are not allowed to push code to protected branches on this project.
To data/tidewater/gitlab/repos/tidewater/model.git
 ! [remote rejected] HEAD -> develop (pre-receive hook declined)
error: failed to push some refs to 'data/tidewater/gitlab/repos/tidewater/model.git'
(exit 1)
```

That rejection is the configuration management board's rule, enforced by software: nothing reaches the integration branch except through a reviewed merge request. Tom saved the output to the issue as evidence and closed it.

The same day Jim checked the drop against its manifest. Every file the prime delivered is listed in the manifest with a SHA-256 checksum, and checking them takes one line. The `tr` at the front removes Windows line endings from the manifest, which the prime's build writes:

```
cd drops/v1.0 && tr -d '\r' < MANIFEST.txt | sed -n 's/^  \([0-9a-f]\{64\}\)  /\1  /p' |
sha256sum -c
```

The first time Jim ran it without the `tr`, every file "failed," which is the kind of thing that ruins a morning if you do not know about line endings. It is in the rescue appendix now (Appendix E, Rescue: when something goes wrong (p. 724)).

26.5.2 Wednesday 14: the first merge request, without a pipeline

Tom's first merge request committed the CODEOWNERS file, the issue and merge request templates, and the `.gitignore`. It is the first merge request the project ever had, and its description says something unusual:

GitLab Markdown: docs/tidewater/sprints/01/samples/mr-1.md

```
# !1 CODEOWNERS, issue and MR templates, .gitignore

## What and why

Who approves what, and the shapes of our issues and merge requests. The project has no pipeline
yet: .gitlab-ci.yml arrives in sprint 2, and until "pipelines must succeed" is turned on, merges
like this one are checked by review only (Jim approves). This MR therefore has no pipeline, on
purpose.
```

```
Closes #16
Part of #3

## Checks

- [ ] `gitlab.pl ci` passed locally (no pipeline in the project yet)
- [ ] `docs/` regenerated and committed (no docs job yet)
- [x] No new `// TODO` lines
- [ ] Trace job: no new orphan, unverified or drifted requirement (no trace job yet)

---- thread

**@jim** (2026-10-14T17:00:00Z) [system]:
approved this merge request

**@tom** (2026-10-14T17:10:00Z) [system]:
merged
```

It has no pipeline on purpose: the pipeline file arrives in sprint 2. Jim approved it, Tom merged it, and the project had its first reviewed change. Look at the checklist: only one box is ticked. The other three could not apply yet, because there was no pipeline to run, no docs job and no trace job, and Tom left them empty with the reason beside each. It is a small thing, and it is exactly the kind of small thing that decides whether a checklist means anything: tick only what you actually did.

The kit lets you read any merge request the team made, from the simulation's GitLab:

```
perl "$KIT/bin/gitlab.pl" --local gitlab --project tidewater/model mr show 1
```

Bob closed the assessment the same day. The story issue shows what "done" meant for an engineering story in sprint 1:

GitLab Markdown: docs/tidewater/sprints/01/samples/story-TW-6.md

```
# Convert drop 1 and assess it

## Why

Before estimating the port, the team needs to know what the converter carries and what it leaves
as TODO lines.

## What "done" looks like

- [ ] Both converter reports are attached
- [ ] The TODO inventory by package is on the wiki
- [ ] The port backlog has estimates
- [ ] The pipeline is green on every MR of this story

## Tasks

| Task | Points | Who |
|---|---|---|
```

```
| #21 Run both converters with --report on drop 1 | 1 | @bob |
| #22 Write the TODO inventory by package | 1 | @bob |
| #23 Estimate the port backlog with the team | 1 | @bob |

## Notes

Epic E5 (Drop 1 intake and first import), tome T2 (Port Tidewater v1 to SysML v2). Journal id
**TW-6**.
```

and the task for running the converters, closed on Friday 9, shows the habit the team adopted for every task: the result goes in a comment on the issue, where the next person will look for it, not in an e-mail:

GitLab Markdown: docs/tidewater/sprints/01/samples/task-21.md

```
# Run both converters with --report on drop 1

Part of #6

## Do

- [ ] Run both converters with --report on drop 1
- [ ] Run `gitlab.pl ci` locally before pushing

## Done when

The result is recorded in a comment on this issue.

---- comments

**@bob** (2026-10-08T14:30:00Z) [system]:
removed ~"workflow::ready" label

**@bob** (2026-10-08T14:30:00Z) [system]:
added ~"workflow::in-progress" label

**@bob** (2026-10-09T15:00:00Z) [system]:
added 1d of time spent

**@bob** (2026-10-09T15:00:00Z) [system]:
removed ~"workflow::in-progress" label

**@bob** (2026-10-09T15:00:00Z) [system]:
added ~"workflow::done" label

**@bob** (2026-10-09T15:00:00Z) [system]:
closed

**@bob** (2026-10-09T15:00:00Z):
Both converters run on drop 1.

- xmi2sysml: ported vs remaining (metric A filter uml:Class|Port|Property|Connector|Activity):
206 of 206 ported, 0 remaining
- trace: F orphan requirements (nothing satisfies), 2, G unverified requirements (no verify), 9,
I trace completeness (satisfied and verified), 66.7%
- drift: TW-SYS-006
```

```
Reports attached to the sprint evidence.
```

26.5.3 Thursday 15: the Definition of Done, and refinement

Jim wrote the working agreements into the wiki. The Definition of Done is the part the team argued about longest, because it is where "finished" stops being a matter of opinion:

GitLab Markdown: docs/tidewater/sprints/01/samples/wiki-definition-of-done.md

```
# Definition of Done

"Done" means the same thing for everyone, and someone other than the author can check it. Think
of it as the closure
criteria of an action item, written once for every item of that kind.

## A task is done when

- [ ] **Its merge request is merged into `develop`**, the master copy. (The MR description says
`Closes #task`, so the
      task closes by itself.)
- [ ] **The pipeline passed.** These are the automated audit checks, each in plain words:

  | Check | What it means |
  |---|---|
  | `grammar` | every model file is valid SysML v2 text |
  | `lint` **0/0** | every name points at something that exists; zero errors, zero warnings |
  | `outline` | every level of the part tree has 2 to 9 children, and each part carries its level
  tag |
  | `secrets` | no password, token or key was committed |
  | `docs` | the generated documents match the model |

- [ ] **It adds no new `// TODO` line**, unless a task exists for it. (A TODO line is something
the converter could not
      translate; a person must close it.)
- [ ] **The owner approved it.** For a requirement change that is Jim; for structure, Bob or Jim;
for the pipeline, Tom.

## A story is done when

- [ ] Every task under it is closed.
- [ ] Its acceptance criteria are checked, by someone other than the author.
- [ ] The **trace** check shows no *new* gap: no new requirement without something that satisfies
it (an *orphan*), no
      new requirement without a verification (*unverified*), and no new difference between DOORS
      and the model
      (*drift*). A gap that is the prime's to fix has a discrepancy report.
- [ ] It was shown at the sprint review, in the model or on the model's web site.

## A drop intake is done when

- [ ] The drop is on the vendor branch `vendor/prime`, filed in the package registry, and merged
into `develop`.
- [ ] Every merge conflict and every *manual* line of the delta report is resolved, with a task
for each.
- [ ] Discrepancy reports this drop fixes are closed; new ones are open and sent to the prime.
```

```
## Writing the change's reason (the commit message)

A commit message is the "reason for change" block. Write a short subject saying what the change
does, a blank line,
why, and then trailers that GitLab and the changelog read:

```text
Close the BuoyModes TODO with a state definition

The converter does not translate state machines. Written from the v1
diagram "stm BuoyModes" (drop v1.0, xmi:id _22_1_7f3a0c1_9c2e...).

Changelog: added
Issue: #23
Reviewed-by: Jim <jim@example.invalid>
```

> [!tip]
> If you cannot meet one of these, say so in the MR and ask. A known, written exception is fine;
a silent one is not.
```

At 10:00 the team held the sprint's refinement (Chapter 24, The rhythm: stand-ups, poker, refinement, kickoff, demo, retro (p. 377)): the stories for sprint 2, checked against the Definition of Ready and estimated with poker. The record:

docs/tidewater/sprints/01/refinement.txt

```
Refinement, 2026-10-15 10:00: the Definition of Ready applied to sprint 2's stories

TW-26  3 pts  Learn: practice lessons 12-13, Vim for models, the SysML v2 workbook (first half)
TW-27  3 pts  Runner online; the first pipeline gates every MR
TW-28  4 pts  Import drop 1 through the vendor branch, in layers
TW-29  2 pts  TODO burn-down and generated docs in the pipeline
TW-30  2 pts  Level conventions for the part tree
TW-31  4.5 pts  Enablers, decisions and conventions on record
TW-32  0.5 pts  Sprint 2 demo: the pipeline and the drop on develop
```

Notice what the list says about the team's thinking at the end of its first week and a half. The drop is not merged yet; the plan is to bring it in through the vendor branch, *in layers*, which means Bob had already decided the model's folder layout would follow the layered structure rather than the converter's default (Chapter 16, The model repository: what goes where (p. 245)). That decision is cheap in sprint 2 and very expensive later, because every future drop must land in the same folders.

26.5.4 Friday 16: a red pipeline on a laptop

Tom's last task was to run the draft pipeline on Tom's laptop against drop 1, with the kit's local runner (Chapter 19, CI/CD from zero (p. 288)). There was still no GitLab runner, so this was the only way to see the pipeline work. It did not pass:

docs/tidewater/sprints/01/local-ci-draft.txt

```
$ perl $KIT/bin/gitlab.pl ci --event push --branch feature/pipeline-draft --var KIT=$KIT
grammar                  check      success             2.0s
lint                     check      success             2.4s
outline                  check      failed              0.7s
    |     1  directory entries  (model/logical/)
    |     1  directory entries  (model/physical/allocation/)
    |     1  directory entries  (model/physical/datacenter/)
    |     1  directory entries  (model/physical/network/)
    |     1  directory entries  (model/physical/operations/)
    |     1  part def ServiceVessel: parts  (model/physical/operations/Operations.sysml)
    |     1  directory entries  (model/physical/stations/)
    |    25  package Stations: members  (model/physical/stations/Stations.sysml)
    |     1  part def Battery: parts  (model/physical/stations/Stations.sysml)
    |     1  directory entries  (model/requirements/model/)
    |    32  package Requirements: members  (model/requirements/model/Requirements.sysml)
    |     1  directory entries  (model/requirements/segment/)
    |     1  directory entries  (model/requirements/stakeholder/)
    |     1  directory entries  (model/requirements/system/)
    |     1  directory entries  (model/verification/)
pipeline: failed (3 jobs)
logs: /tmp/gitlab-ci
(exit 1)
```

Grammar and lint passed: the converted model is valid SysML v2 and structurally sound. The outline check failed, because the converted packages and folders do not meet the rule that every parent has between two and nine children. That is not a defect Tom introduced; it is what the prime's model looks like.

This is the first real decision of the port, and it is worth slowing down for, because every team porting an inherited model will face it.

Objection: "Just turn the outline check off until the port is done"

That was Tom's first suggestion, and it is the most common one. The trouble is that a check you have turned off does not come back on by itself; the team would have shipped baseline-0 with the check still off.

Answer

The team kept the check and gave it an *allowance*: the number of violations the drop arrived with is recorded in a file in the repository, the pipeline fails only if a change makes the number bigger, and every change that makes it smaller lowers the allowance in the same merge request. The inherited debt is visible, counted and paid down; new debt is refused. This is a *ratchet*, and the team used the same idea later for the trace and complexity checks (Chapter 20, The pipeline, job by job (p. 302)).

The review on Friday morning was short. The deputy program manager attended, as the meeting record shows:

docs/tidewater/sprints/01/meetings.csv

```
2026-10-16,sprint review (demo),10:00,11:00,Jim; Bob; Tom; Deputy PM
```

```
2026-10-16,retrospective,11:00,12:00,Jim; Bob; Tom
```

and the review record is plain:

docs/tidewater/sprints/01/review.txt

```
Sprint 1 review, 2026-10-16 10:00

Goal: A GitLab project the team can work in; drop 1 received and assessed

Demonstrated:
  TW-1 Learn the tools: survival kit, git, Vim, the operating model
  TW-2 Laptops ready: Git for Windows, Vim, the kit
  TW-3 Create the GitLab group and model project
  TW-4 Draft the first pipeline and run it locally
  TW-5 Receive drop 1 and record it
  TW-6 Convert drop 1 and assess it
  TW-7 Working agreements: Definition of Ready and Done

Artifacts shown: the merge requests and pipelines of the sprint.

Feedback (Deputy PM): "You received the drop and checked every file. When does the model itself
move?" Jim: drop 1 goes onto develop through the vendor branch in sprint 2, and every change
after that is checked by the pipeline.

The four headline numbers: TODO lines left 0; verify coverage -; open DRs 0; blocked items 0.
```

The four headline numbers at the bottom show how little there was to show: no TODO lines because the drop was not in the model yet, no verification coverage because the trace did not run in a pipeline yet. The demonstration was a walk through the board and the GitLab project. The deputy's one question, recorded above, is the one every manager asks of a port: when does the model itself move? Jim's answer put a date on it, sprint 2, and named the mechanism that would check every change after that. The team took the question as the measure of sprint 2.

26.5.5 The retrospective

docs/tidewater/sprints/01/retro.txt

```
Sprint 1 retrospective, 2026-10-16 11:00

Start:
  - Writing the task id in every stand-up status
  - Running the converter report before estimating
Stop:
  - Estimating a story before anyone has read the drop
Continue:
  - Pairing at the terminal for the first git commands
  - One issue per action item

Pulse survey (1-5, anonymous): mean 1.9
```

The *start* items are both about evidence. Writing the story id in every status is the lint lesson from Monday. Running the converter report before estimating is the poker lesson: the TW-6 round would have taken one vote instead of two if Bob had seen the report first. The *stop* item says the same thing from the other side. The *continue* items are about how the team learned: at the terminal, in pairs, with one issue per action so nothing lived only in someone's memory.

26.6 The numbers at the end of sprint 1

The scorecard (Chapter 3, The directive (p. 22)) was computed by the kit from GitLab and the journal at the end of the sprint:

Table 26.2: The scorecard at the end of sprint 1

Measure	End of sprint 1
Story points committed / done	16.5 / 16.5 (100%)
Work mix: engineering / platform / learning points	4.5 / 8 / 4
Merges to the integration branch	1
Gated changes (pipeline and code-owner approval)	0 of 1
Change lead time, median hours	2.2
Review turnaround, median hours	2
Share of stand-up lines with an outcome and a number	0.48
Pulse survey mean	1.9

Everything committed was done. The one gated-changes figure that reads badly, no change gated, is the merge request with no pipeline, and it is honest: there was nothing to gate it with yet. The stand-up outcome share needs a word of explanation: nearly half the stand-up lines had an outcome and a number in them, which looks good until you read the lines and see that the numbers are mostly lesson numbers. The share fell in sprint 2 and only became meaningful when the team learned to say what a day changed (Chapter 27, Sprint 2: Drop 1 on develop, every change gated (p. 424), Chapter 25, Talking about progress: helps for engineers who would rather not (p. 390)).

26.7 What Jim told management

The weekly quad went to the program manager on Friday afternoon (Chapter 36, Telling management (p. 538)). Its top lines:

docs/tidewater/sprints/01/quad-2026-10-16.txt

```
Weekly status Model -- as of 2026-10-16 (sprint 1)
Sprint progress 100% (16/16 SP) improving (was 42%)   On-time delivery: 5/5 this week, 7/7 this
sprint

TECHNICAL PRIORITIES
  DONE  TW-3         3  "Create the GitLab group and m Tom
  DONE  TW-5       1.5  "Receive drop 1 and record it" Jim
  DONE  TW-6         3  "Convert drop 1 and assess it" Bob
  DONE  TW-1         4  "Learn the tools: survival kit Jim
  DONE  TW-2         3  "Laptops ready: Git for Window Tom
  DONE  TW-4       1.5  "Draft the first pipeline and  Tom
```

```
  DONE  TW-7      0.5  "Working agreements: Definitio Jim

WATCH ITEMS / PM HELP NEEDED
```

Jim added two sentences of Jim's own in the covering e-mail, and they are a fair model for a first-sprint report:

> *We have a project, protected branches, three working laptops and drop 1 measured: everything in scope converts, and the conversion debt is counted. We also found a requirement whose text differs between DOORS and the prime's model, which we will report to the prime formally once the reporting route is agreed.*

No adjectives, no promises about December, one finding, and the next thing management will see: the model itself, on the integration branch, at the end of sprint 2.

26.8 Tools adopted this sprint, and the pain each removed

Table 26.3: What sprint 1 brought in

Tool	The pain it removed
`kickoff.pl doctor`	"It works on my machine": three laptops checked the same way, with the fix printed beside each problem
GitLab issues, labels and the board	action items in e-mail threads that nobody could find a week later
Protected branches and code owners	changes reaching the baseline without a reviewer; enforced, and proved on day six
Merge requests	review by meeting; every change now has a description, a reviewer and a record
The stand-up lint	statuses that read like news rather than plans
Both converters with `--report`	guessing what the prime's model contains
`gitlab.pl ci`, the local runner	waiting for a server to find out whether the pipeline works

26.9 What went wrong, and why it was fine

- **The first stand-ups failed the lint.** Expected, cheap to fix, and fixed by the habit rather than by a rule.
- **The first merge request had no pipeline.** Unavoidable without a runner; the description said so, which is what made it acceptable.
- **Checklist boxes ticked by habit.** Not caught in review. A template checklist is only worth what the person ticking it actually did.
- **The first pipeline was red.** On a laptop, before anything depended on it, which is the cheapest place for a pipeline to be red. The red outline check produced the ratchet, one of the most useful ideas of the twelve weeks.
- **The drift was found and then parked.** Deliberately: the team wanted an agreed discrepancy-report route before sending the prime anything.

26.10 You can now…

- run `kickoff.pl doctor` and act on its output;
- read a stand-up lint and fix a status that fails it;
- run planning poker and use the spread, not just the result;
- convert a prime drop, read the converter's report, and count the conversion debt;
- run the trace and find text drift between DOORS and the model;
- verify a delivery against its manifest;
- explain why a gate is switched on only when the thing it checks exists;
- explain a ratchet, and why it beats turning a check off.

26.11 Exercises

Exercise 26.1: The first status

Rewrite Bob's Tuesday status from the chat above so that the lint parses it and it names the story it served. Then write Monday's "meetings, reviews" line the way the lint would accept on a kickoff day.

Exercise 26.2: Read the drop yourself

In the `tidewater` sandbox, convert drop 1 as Bob did, then open `s1/model/physical/stations/Stations.sysml` in Vim and find the TODO line for the `BuoyModes` state machine. What v1 element does its id point at? (Practice lesson 10 walks through the same edit on a smaller scale; Chapter 10, Vim for models (p. 143).)

Exercise 26.3: A spread worth having

Look at the TW-3 rounds in `docs/tidewater/sprints/01/poker-2026-10-05.txt`. Bob voted highest. Write the one sentence Bob might have said between the rounds, and the sentence you would add to the story's notes.

Exercise 26.4: Your week zero

List the hours your own team would spend before its first sprint, in the format of the week-zero record. Which of them would you be tempted to hide inside sprint 1, and why should you not?

26.12 Applying this to your project

- **Make the first sprint about the shop and say so.** Laptops, the project, the rules, the first pipeline on a laptop. Do not promise model progress in sprint 1; promise a measured starting point.
- **Put learning on the board** with points and owners. It is the first thing management will ask about later.

- **Measure the inherited work before you estimate it.** Run the converters, count the debt, run the trace. The first real finding is your best argument; let the person who doubts the most find it.
- **Switch gates on in the order the things they check come into being,** and prove each one refuses what it should.
- **Ratchet inherited debt** instead of switching checks off.
- **Ask the prime for stable ids and a manifest** before the first drop, not after.

CHAPTER

27

Sprint 2: Drop 1 on develop, every change gated

From what you know →

You know

the first delivery from a contractor, and the argument about where it lives.

→ Here it is called

the vendor branch and the first merge request into develop.

→ What is new

the prime's model arrives through a gate, and every change after it is checked.

Survival kit

What you have

a GitLab project with its rules, three working laptops, drop 1 converted on a laptop and measured, a pipeline that runs only on Tom's machine, and a team still unconvinced.

What you need from this sprint: a runner, so the pipeline runs on the server for every change; the drop on the integration branch through the vendor branch, in a folder layout that every future drop will land in; and the team's decisions written down where the next person will find them.

27.1 The question, and the mood

Sprint 2's systems engineering question was about configuration control: **can the prime's model live on our integration branch, with every change to it checked and reviewed, without anyone editing the prime's model?** If the answer was yes, the team had a baseline it controlled. If it was no, the whole port was a copy that would go stale with the next drop.

The answer was yes, and the pulse survey moved for the first time, to a mean of 2.4:

Table 27.1: The pulse survey at the sprint 2 retrospective

Question	Jim	Bob	Tom
I understand why we are working this way.	3	2	2
I can do my work in the new tools without help.	2	2	2
The checks catch problems I would have missed.	2	3	2
I can see our progress.	3	2	3
I would choose to keep working this way.	3	2	3

Bob's answer to the third question moved first. The checks had caught something, and Bob had seen it. The sprint also had the twelve weeks' worst moment in front of management, which is why the last row is still low. Both stories are below.

27.2 The kickoff: Monday 19 October

The stories had been estimated at the sprint 1 refinement (Chapter 26, Sprint 1: A GitLab project the team can work in (p. 404)), so the kickoff was about capacity, the commitment and who did what. A full ten-day sprint gave more capacity than sprint 1, and the team committed a larger share of it, because the first sprint had finished and they trusted their numbers a little more:

docs/tidewater/sprints/02/kickoff.txt

```
Sprint 2 kickoff, 2026-10-19 09:00 (90 minutes)

Goal: Drop 1 on develop through the vendor branch; every change gated
Capacity: 21.0 points (Jim 5, Bob 8, Tom 8 per full sprint, less holidays and leave)
Committed: 19.0 points in 7 stories (90% of capacity): engineering 6.0, platform 10.0 (52%),
learning 3.0 (15%)

TW-26    E3    3.0 pts  learning     Learn: practice lessons 12-13, Vim for models, the SysML v2
workbook (first half)
           #33   0.5 pt  jim  2026-10-19 .. 2026-10-19  Practice lessons 12-13; the diagrams
           chapter
           #34   1.5 pt  bob  2026-10-19 .. 2026-10-20  Lessons 12-13; Vim for models; workbook
           first half
           #35   1.0 pt  tom  2026-10-19 .. 2026-10-20  Lessons 12-13; Vim for models; workbook
           first half
TW-27    E2    3.0 pts  platform     Runner online; the first pipeline gates every MR
           #36   0.5 pt  tom  2026-10-20 .. 2026-10-20  Check the runner registration and tags
           #37   2.0 pt  tom  2026-10-20 .. 2026-10-23  Commit the pipeline; fix it for the
           PowerShell runner with ci/job.pl
           #38   0.5 pt  tom  2026-10-23 .. 2026-10-23  Turn on "pipelines must succeed" and the
           MR report widgets
TW-28    E5    4.0 pts  engineering  Import drop 1 through the vendor branch, in layers
           #39   1.0 pt  bob  2026-10-20 .. 2026-10-22  Decide the layered layout; the layout map
           #40   1.0 pt  tom  2026-10-26 .. 2026-10-27  Import drop 1 onto vendor/prime
           #41   2.0 pt  bob  2026-10-27 .. 2026-10-29  Review and merge the drop-1 MR; record
           the outline count
TW-29    E2    2.0 pts  platform     TODO burn-down and generated docs in the pipeline
           #42   2.0 pt  tom  2026-10-27 .. 2026-10-29  Add the todo and docs jobs; commit docs/
TW-30    E6    2.0 pts  engineering  Level conventions for the part tree
           #43   1.5 pt  bob  2026-10-22 .. 2026-10-23  Write the conventions package
           #44   0.5 pt  bob  2026-10-26 .. 2026-10-26  Name the level tags in sysml.conf
TW-31    E1    4.5 pts  platform     Enablers, decisions and conventions on record
           #45   1.0 pt  jim  2026-10-20 .. 2026-10-21  Memo: request for enablers
```

```
          #46   1.5 pt  jim  2026-10-22 .. 2026-10-26  Write docs/adr/0001..0010
          #47   1.0 pt  jim  2026-10-27 .. 2026-10-28  Conventions and onboarding docs into
          docs/
          #48   1.0 pt  jim  2026-10-29 .. 2026-10-30  Toolchain security kickoff with the ISSM
TW-32    E3    0.5 pts  platform     Sprint 2 demo: the pipeline and the drop on develop
          #49   0.5 pt  tom  2026-10-29 .. 2026-10-30  Prepare and give the sprint 2 demo
```

The learning story shrank from sprint 1's, as planned. The platform share was still more than half, because the runner and the real pipeline were the sprint's main platform work and could not wait.

Two estimates had needed a second round at the refinement. The import of drop 1 through the vendor branch:

docs/tidewater/sprints/01/poker-2026-10-15.txt

```
ESTIMATE TW-28 Import drop 1 through the vendor branch, in layers  (3 answers at 10:05)
  Bob            5
  Jim            3
  Tom            3
  no consensus: median 3, mode 3, range 3-5

ESTIMATE TW-28 Import drop 1 through the vendor branch, in layers  (3 answers at 10:07)
  Bob            4
  Jim            4
  Tom            4
  consensus: 4
```

The high vote was Tom's. Tom had read the gitflow chapter (Chapter 12, Git from zero, for systems engineers (p. 171)) and knew the vendor branch meant two merges, one onto the vendor branch and one from it into the integration branch, plus a layout decision before either. The team settled between the two first votes. The other split was over Jim's own story, getting the team's enablers, decisions and conventions on record:

docs/tidewater/sprints/01/poker-2026-10-15.txt

```
ESTIMATE TW-31 Enablers, decisions and conventions on record  (3 answers at 10:15)
  Bob            3
  Jim            3
  Tom            5
  no consensus: median 3, mode 3, range 3-5

ESTIMATE TW-31 Enablers, decisions and conventions on record  (3 answers at 10:17)
  Bob            4.5
  Jim            4.5
  Tom            4.5
  consensus: 4.5
```

Jim gave the lowest vote on work Jim would do, which is the usual mistake: the person who will do the writing remembers the writing and forgets the reviews. The second round added the reviews back.

27.3 Week one

27.3.1 Monday 19: statuses with no outcome

The first stand-up of sprint 2 shows a habit that sprint 1 had not fixed:

docs/tidewater/sprints/02/standups/2026-10-19-chat.txt

```
Morning. Y/T/B in chat, please.
[9:01 AM] Jim
Y: meetings, reviews
T: TW-26 practice lessons 12-13; the diagrams chapter
B: none
[9:02 AM] Bob
Y: meetings, reviews
T: TW-26 lessons 12-13; Vim for models; workbook first
B: none
[9:03 AM] Tom
Y: TW-4 draft locally on drop 1 with gitlab.pl ci
T: TW-26 lessons 12-13; Vim for models; workbook first
B: none
```

docs/tidewater/sprints/02/standups/2026-10-19-lint.txt

```
standups/2026-10-19-Model-chat.txt:5: warn  Jim: no task id in Y (e.g. TW-1)
standups/2026-10-19-Model-chat.txt:9: warn  Bob: no task id in Y (e.g. TW-1)
standups/2026-10-19-Model-chat.txt:13: ok    Tom: Y/T/B parsed
# 3 answered, 0 errors, 2 warnings
```

The lint caught the missing story ids again. What the lint cannot catch is that not one of these lines says what is now true that was not true yesterday. That is what the scorecard's stand-up outcome share measures, and for sprint 2 it was 0: every line was a list of activities. Nobody flagged it at the time. It surfaced at the review, in the worst possible way.

27.3.2 Tuesday 20: the runner is online

The enterprise administrators registered the runner Tom had requested in sprint 1: a Windows virtual machine with Git for Windows, a shell executor, and the tags the pipeline asks for (Chapter 21, Runners: where the pipeline runs (p. 347)). Tom's half-point task was to check the registration and the tags. It was the shortest task of the sprint and the one everything else waited on.

27.3.3 Wednesday 21: asking for what the team needs, in writing

Jim wrote the request for enablers that the operating model calls for (Chapter 37, Writing it down: DoD-style memos from Vim (p. 545)), using the memo template. The heart of it is a table of dated asks, each with what it buys and what happens if the answer is no:

docs/tidewater/memos/2026-10-21-request-for-enablers.md

```
| # | Ask | Why (the outcome it buys) | Needed by | If not |
|:-:|:----|:--------------------------|:---------:|:-------|
| 1 | Four protected hours a week per person for training, through 18 December | the team learns
while it delivers | 26 Oct | the conversion burn-down slows by about a third |
| 2 | A two-hour review slot with the ISSM for the runner and repository security description |
the toolchain is approved for program data | 6 Nov | secret detection and runner hardening stay
unapproved |
| 3 | Keep one Cameo seat, for reading the prime's drops only | each drop can be opened when its
XMI export is in question | 30 Nov | a drop with a bad export cannot be checked |
| 4 | A second runner machine on the program network | no merge waits when the first runner is
down | 8 Jan | one runner outage blocks every merge |
| 5 | A decision on the GitLab tier that enforces code-owner approval rules | approvals enforced
by the platform, not by habit | 15 Jan | approvals rely on the team's discipline alone |
```

The memo did two things a spoken request in a meeting would not. It made each ask specific enough to say yes or no to, and it put a date on the cost of no. The training hours in the first row are the learning budget, which until then had been protected only by Jim's say-so.

27.3.4 Thursday 22: the layout decision

Bob's task #39 was the most consequential half-day of the sprint: deciding where the converted model would live in the repository. The converter's default is one folder per v1 package, named after the package. The alternative is a layered layout, with libraries, requirements, logical behavior, physical structure, analysis and verification in their own folders and the packages placed in them by a *layout map* (Chapter 16, The model repository: what goes where (p. 245)).

The thought process went like this.

- **Every future drop must land in the same folders.** The vendor branch works because git can compare the prime's new drop with the old one file by file. If a package moves between drops, git sees a deletion and an addition, and the merge with the government's edits becomes manual. So the layout chosen now is the layout for the life of the program.
- **The layers are how the team wanted to review.** A reviewer looking at a change to a requirement package wants to know immediately that it is a requirement change, and the code owners for requirements are not the code owners for physical structure (Chapter 22, Reviewing a model change (p. 357)).
- **The layout map costs one file.** A text file that says which package goes in which folder, applied by the converter on every drop. Moving a package later means editing that file and re-importing on the vendor branch, which is possible but not something to do casually.

Bob chose the layered layout, and the team recorded the choice in the merge request that imported the drop.

27.3.5 Friday 23: the pipeline dies on its first line

Tom committed the pipeline that had run on Tom's laptop the Friday before, opened a first merge request with a real pipeline, and watched it fail on every job:

docs/tidewater/sprints/02/pipelines/mr2-first-on-powershell-1.log

```
pipeline #1 (push, feature/37-first-pipeline) failed, 3 jobs, 57.8 s

==== grammar          check    failed    35.6 s
$ FILES=$(find model -name '*.sysml' | sort)
FILES=$(find : The term 'FILES=$(find' is not recognized as the name of a cmdlet, function,
script file, or operable program. Check the
spelling of the name, or if a path was included, verify that the path is correct and try again.
At line:1 char:1
+ FILES=$(find model -name '*.sysml' | sort)
+ ~~~~~~
    + CategoryInfo          : ObjectNotFound: (FILES=$(find:String) [], CommandNotFoundException
    + FullyQualifiedErrorId : CommandNotFoundException

==== lint             check    failed    42.8 s
$ perl "$KIT/tools/sysml/model.pl" lint > lint.txt || true
At line:1 char:50
+ perl "$KIT/tools/sysml/model.pl" lint > lint.txt || true
+                                                  ~~
The token '||' is not a valid statement separator in this version.
    + CategoryInfo          : ParserError: (:) [], ParentContainsErrorRecordException
    + FullyQualifiedErrorId : InvalidEndOfLine

==== outline          check    failed    15.4 s
$ perl "$KIT/tools/sysml/model.pl" check
Can't open perl script "/tools/sysml/model.pl": No such file or directory
```

Read the errors. `The term 'FILES=$(find' is not recognized as the name of a cmdlet` is PowerShell talking. The Windows runner's shell executor runs PowerShell by default, and the pipeline had been written in the Bash dialect that Git Bash uses on Tom's laptop. `$(…)`, `||` and `$KIT` mean nothing to PowerShell. Tom's reaction is in the record:

> *The first pipeline on the real runner died on line one. PowerShell. Nobody told me the runner speaks PowerShell.*
>
> — Tom, sprint 2

This is the lesson of the runners chapter (Section 21.6, The lesson of the first week: PowerShell (p. 350)), and the team took the fix described there: every job's script became one line, `perl ci/job.pl NAME`, with the logic in a Perl script in the repository. Perl reads the same under PowerShell, Git Bash, Linux and Docker. The second pipeline on the same merge request:

docs/tidewater/sprints/02/pipelines/mr2-job-pl-2.log

```
pipeline #2 (merge_request_event, refs/merge-requests/2/merge) success, 3 jobs, 59.8 s

==== grammar          check    success   44.8 s
$ perl ci/job.pl grammar
no .sysml files under model/ yet
```

Look at line five: `no .sysml files under model/ yet`. The pipeline is green because there is nothing in the repository for it to check. That is worth saying out loud, because a green pipeline

on an empty model proves the plumbing, not the model. Jim approved the merge request on that understanding:

GitLab Markdown: docs/tidewater/sprints/02/samples/mr-2.md

```
# !2 The first pipeline: grammar, lint, outline

## What and why

The configuration audit, run by the server on every change.

Closes #37
Part of #27

## Checks

- [ ] `gitlab.pl ci` passed locally (no pipeline in the project yet)
- [ ] `docs/` regenerated and committed (no docs job yet)
- [x] No new `// TODO` lines
- [ ] Trace job: no new orphan, unverified or drifted requirement (no trace job yet)

## Review notes

Ran green locally with `gitlab.pl ci`; first run on the real runner.

---- thread

**@jim** (2026-10-23T20:00:00Z) [system]:
approved this merge request

**@tom** (2026-10-23T20:10:00Z) [system]:
merged
```

The simulation's GitLab still holds both pipelines. This is the failed one as GitLab lists it:

```
perl "$KIT/bin/gitlab.pl" --local gitlab --project tidewater/model pipeline show 1
```

and the first lines of its grammar job's log:

```
head -4 gitlab/pipelines/tidewater-model/1/grammar.log
```

The same afternoon Tom switched on "pipelines must succeed" and the merge request report widgets (Chapter 17, The DevSecOps tools that earn their keep (p. 255)). From that moment no change could reach the integration branch without a green pipeline. Bob's conventions package, the government's own library of level tags (Chapter 16, The model repository: what goes where (p. 245)), was the first model change to go through the full gate, and it passed on its first pipeline.

27.4 Week two

27.4.1 Monday 26: level tags and decision records

Bob named the level tags in `sysml.conf`, and Jim merged the team's decision records: the ten decisions drafted in week zero, plus the scaffold's own, as numbered files under `docs/adr/` in the repository rather than slides in a briefing (Chapter 13, The operating model (p. 196)). The list is worth reading as a summary of how the team decided to work:

```
ls work/jim/docs/adr
```

Each file is a page: the context, the decision, and its consequences. The point of keeping them in the repository is that a decision is changed the same way a model element is, by a reviewed merge request, and the history shows when and why.

27.4.2 Tuesday 27: the drop on the vendor branch

Tom imported drop 1 onto the vendor branch with `gitlab.pl flow vendor` (Chapter 45, The prime's model changes: receive, assess, disposition (p. 619)). The vendor branch holds nothing but the prime's deliveries, converted with the layout map, one commit per drop. Nobody edits it. The government's work happens on the integration branch, and each drop reaches it through a merge request from a short feature branch that merges the vendor branch in. That merge request was opened the same morning, and Jim said the sentence the team remembers from the sprint:

> *The model is on develop and nobody touched it with a mouse.*
>
> — Jim, sprint 2

It was not quite on develop yet. It was on its way through review, which is the point.

27.4.3 Wednesday 28: refinement for sprint 3

docs/tidewater/sprints/02/refinement.txt

```
Refinement, 2026-10-28 10:00: the Definition of Ready applied to sprint 3's stories

TW-50  2.5 pts  Learn: the workbook second half; talking about progress
TW-51  3 pts  Close the Network TODO lines
TW-52  3 pts  DOORS modules into the model
TW-53  2.5 pts  Trace report and secret detection in the pipeline
TW-54  2 pts  Runner hardening with the ISSM
TW-55  2 pts  First drift and gap analysis from the trace
```

The list turned toward engineering for the first time: closing the conversion debt in the Network package, bringing the DOORS modules into the model, and the first drift and gap analysis. The platform stories that remained were about making the checks report something useful: the trace report and secret detection.

27.4.4 Thursday 29: the drop is merged

Bob reviewed the drop merge request. The review note is short, and it records the outline decision from sprint 1:

GitLab Markdown: docs/tidewater/sprints/02/samples/mr-6.md

```
# !6 Prime drop v1.0

## What and why

The prime's drop v1.0, converted on vendor/prime and merged here.

Closes #40
Closes #25
Part of #28

## Checks

- [x] `gitlab.pl ci` passed locally
- [x] `docs/` regenerated and committed
- [ ] No new `// TODO` lines (a prime drop: its TODO lines are the port backlog)
- [ ] Trace job: no new orphan, unverified or drifted requirement (no trace job yet)

## Review notes

Prime changes in this drop: NWM-CR-0001.

No conflicts.

---- thread

**@bob** (2026-10-29T15:00:00Z) [system]:
added 2h of time spent

**@bob** (2026-10-29T15:00:00Z):
Read the delta and the conflict table. Outline allowance recorded in `.ci/outline-max`.

**@bob** (2026-10-29T15:00:00Z) [system]:
approved this merge request

**@jim** (2026-10-29T15:00:00Z) [system]:
approved this merge request

**@bob** (2026-10-29T15:00:00Z) [system]:
merged
```

The pipeline on the drop merge request is the first one with the whole model in it. Its outline job shows the allowance at work:

docs/tidewater/sprints/02/pipelines/mr6-drop1-14.log

```
== Outline (2..9 children per parent) ==
violations               17
    1  directory entries  (model/analysis/)
    1  package Analysis: members  (model/analysis/Analysis.sysml)
```

```
     1  directory entries  (model/legacy/)
     1  part def Battery_old: parts  (model/legacy/Legacy_Import.sysml)
     1  directory entries  (model/logical/)
     1  directory entries  (model/physical/allocation/)
     1  directory entries  (model/physical/datacenter/)
     1  directory entries  (model/physical/network/)
     1  directory entries  (model/physical/operations/)
     1  part def ServiceVessel: parts  (model/physical/operations/Operations.sysml)
     1  directory entries  (model/physical/stations/)
    25  package Stations: members  (model/physical/stations/Stations.sysml)
     1  part def Battery: parts  (model/physical/stations/Stations.sysml)
     1  directory entries  (model/requirements/)
     1  directory entries  (model/requirements/model/)
    32  package Requirements: members  (model/requirements/model/Requirements.sysml)
     1  directory entries  (model/verification/)
 $ perl $KIT/tools/gitlab/ci-reports.pl outline --junit outline.xml --codequality outline-cq.json
--metrics outline.metrics outline.txt
 ci-reports outline: 2 check(s), 1 failed, 17 finding(s); level_tag_mismatches=0
 outline_violations=17
 outline violations: 17 (allowed: 17)
 a prime drop: reported, not ratcheted
```

The last two lines are the ratchet from sprint 1 in action: the drop arrives with its outline violations, they are *reported*, the allowance is set to that count, and from now on any change that adds a violation fails. The same merge request shows the code-quality widget in GitLab listing every one of those findings against the change that brought them in:

```
perl "$KIT/bin/gitlab.pl" --local gitlab --project tidewater/model mr show 6
```

Bob and Jim both approved, because the drop touched folders owned by each of them, and the merge request was merged. The history of the integration branch now reads like a change log, one reviewed merge per line:

```
git -C work/bob log --oneline --first-parent --before=2026-10-31 origin/develop
```

The same day Tom added two more jobs to the pipeline: `todo`, which counts the conversion debt on every change, and `docs`, which regenerates the model's generated documents and fails if the committed copies are out of date. The todo job's first count on the integration branch was 52, the number the port would burn down for the next four sprints.

27.4.5 Friday 30: the demo that lost the room

The review was the low point of the twelve weeks, and it is in the record:

docs/tidewater/sprints/02/review.txt

```
Sprint 2 review, 2026-10-30 10:00
```

```
Goal: Drop 1 on develop through the vendor branch; every change gated

Demonstrated:
  TW-26 Learn: practice lessons 12-13, Vim for models, the SysML v2 workbook (first half)
  TW-27 Runner online; the first pipeline gates every MR
  TW-28 Import drop 1 through the vendor branch, in layers
  TW-29 TODO burn-down and generated docs in the pipeline
  TW-30 Level conventions for the part tree
  TW-31 Enablers, decisions and conventions on record
  TW-32 Sprint 2 demo: the pipeline and the drop on develop

Artifacts shown: the merge requests and pipelines of the sprint.

Feedback (Deputy PM): "I did not follow the demo. What did we get for the two weeks?" Tom showed
the terminal, the YAML and a green pipeline, in that order; the room heard jargon and no number.
Retro action: a Day-9 rehearsal from now on, and the driver rotates.

The four headline numbers: TODO lines left 52; verify coverage 70%; open DRs 0; blocked items 0.
```

Tom drove the demo, opened a terminal, scrolled through the pipeline file, explained the PowerShell problem, and finished on a green pipeline, which on that day was the most impressive thing Tom had seen in a month. The deputy program manager saw a terminal, a file full of words like `rules` and `artifacts`, and a green dot. The question at the end was fair, and nobody had an answer ready:

> *I did not follow the demo. What did we get for the two weeks?*
>
> — Deputy PM, sprint 2 review

What they had got was considerable: the prime's model under the government's configuration control, every change checked and reviewed, the conversion debt counted, the decisions on record. The team knew that. They had not said it in words the deputy could repeat to anyone else. The four headline numbers were at the bottom of the review record and nobody read them out.

Case study: Why the demo failed, in one sentence

The team demonstrated the *machinery* to someone who had come to see the *result*. The fix (Chapter 25, Talking about progress: helps for engineers who would rather not (p. 390)) is a structure, not a talent: open with the question the sprint answered, show the output rather than the tool, give one before-and-after number, make one safe change live, and close with what it means and what you need. The team used exactly that structure in sprint 4, and it worked (Chapter 29, Sprint 4: the model site, a cycle broken, and the demo that landed (p. 459)).

27.4.6 The retrospective

docs/tidewater/sprints/02/retro.txt

```
Sprint 2 retrospective, 2026-10-30 11:00
```

```
Start:
  - A Day-9 demo rehearsal: question, output, before and after, one live change, the ask
  - Rotating who drives the demo
Stop:
  - Writing shell in the YAML
  - Demoing the terminal instead of the answer
Continue:
  - Running gitlab.pl ci before every push
  - Small MRs: the drop-1 merge was the only big one, and it was reviewable

Pulse survey (1-5, anonymous): mean 2.4
```

The *start* items are the two lessons from the demo: rehearse it on Day 9, and rotate the driver so that nobody's first demo is in front of the deputy. The *stop* items are the two lessons from the week: no shell in the pipeline file, and no terminals in demos. The *continue* items are what was already working: running the pipeline locally before pushing, and small merge requests.

One more thing turned up at the review, in the GitLab To-Do lists the team looked at before the meeting:

docs/tidewater/sprints/02/todo-at-review.txt

```
GitLab To-Do lists at the review (2026-10-30 10:00)

  Jim: 0 review(s) requested, 2 open issue(s) assigned
    #52 DOORS modules into the model
    #55 First drift and gap analysis from the trace

  Bob: 0 review(s) requested, 1 open issue(s) assigned
    #51 Close the Network TODO lines

  Tom: 0 review(s) requested, 3 open issue(s) assigned
    #50 Learn: the workbook second half; talking about progress
    #53 Trace report and secret detection in the pipeline
    #54 Runner hardening with the ISSM
```

The first issue in Tom's list is the intake issue for drop 1, opened when the drop arrived and never closed, although the drop had been received, checked and merged. It is a small housekeeping slip, and a useful one: the To-Do list is where it shows, and nobody had looked at the To-Do list until the review.

27.5 The numbers at the end of sprint 2

Table 27.2: The scorecard at the end of sprint 2

Measure	End of sprint 2
Story points committed / done	19 / 19 (100%)
Work mix: engineering / platform / learning points	6 / 10 / 3
Conversion debt (TODO lines on the integration branch)	52
Merges / gated changes	6 / 6 of 6
Merge requests whose first pipeline passed	5 of 6

Measure	End of sprint 2
Integration-branch pipelines passing	6 of 6
Trace: requirements linked / unverified / drift	29 of 30 / 9 / 1
Spider web: package cycles / max fan-out / max relationships on one part	1 / 4 / 19
Change lead time / review turnaround, median hours	3.6 / 3.3
Share of stand-up lines with an outcome and a number	0
Pulse survey mean	2.4

Two lines are worth reading against the document era. Every change to the model was gated, by a pipeline and by a code owner other than the author. And the median change took hours, not the weeks a change took through the document-era board (Chapter 3, The directive (p. 22)). The comparison is not entirely fair yet: these were small, uncontroversial changes. It became fair in sprint 4, when the changes were engineering ones.

GitLab's own view of the sprint agrees. The kit's analytics command reads it from the GitLab data, and its pipeline section for sprint 2 is:

docs/tidewater/sprints/02/analytics.txt

```
3. CI/CD: 18 pipelines, success rate 94.4%, duration median 1 min, p85 1.7 min
   period                          runs   failed success%   median min
   Sprint 2                          18        1     94.4            1
   job failures: grammar 1/18, lint 1/18, outline 1/18
```

The one failed pipeline is the PowerShell one. You can produce the full report from the simulation's GitLab (Chapter 32, Measuring with GitLab analytics (p. 513)):

```
perl "$KIT/bin/gitlab.pl" --local gitlab --project tidewater/model analytics --since 2026-10-19
--until 2026-10-30 --by sprint
```

27.6 What Jim told management

The weekly quad on Friday said, at the top:

docs/tidewater/sprints/02/quad-2026-10-30.txt

```
Weekly status Model -- as of 2026-10-30 (sprint 2)
Sprint progress 100% (19/19 SP) improving (was 32%)   On-time delivery: 5/5 this week, 7/7 this
sprint
```

and the covering note, written after the review and with the deputy's question in mind, was the first one Jim wrote in the outcome-first pattern (Chapter 25, Talking about progress: helps for engineers who would rather not (p. 390)):

> *The prime's drop 1 is now under our configuration control: on our integration branch, every change checked automatically and approved by a code owner before it is accepted, none made*

> *by hand to the prime's model. The conversion left a counted debt of TODO lines, which is our port measure from now on and goes down every sprint. We need decisions on the five enablers in the 21 October memo; two of them by 26 October.*

It does not mention PowerShell, YAML or runners. It says what is now true, gives one measure, and asks for something.

27.7 Tools adopted this sprint, and the pain each removed

Table 27.3: What sprint 2 brought in

Tool	The pain it removed
The runner and the server pipeline	a pipeline that only ran on Tom's laptop, when Tom remembered
`perl ci/job.pl NAME`	a pipeline that worked in Git Bash and died in PowerShell
"Pipelines must succeed"	merges that relied on the reviewer noticing a red pipeline
Merge request widgets (tests, code quality)	reading job logs to find out what a change broke
The vendor branch and `flow vendor`	the prime's deliveries mixed into the government's edits
The layout map	every future drop landing in different folders
The outline allowance	a check turned off because the inherited model fails it
The `todo` and `docs` jobs	progress by opinion; generated documents out of date
Decision records under `docs/adr/`	decisions that lived in slides and memories
The request-for-enablers memo	asks made in meetings and lost

27.8 What went wrong, and what the team did about it

- **The pipeline was written for the wrong shell.** Found on the first server run, fixed within the day, and the fix made the pipeline more portable than it would otherwise have been.
- **A green pipeline on an empty model.** Not wrong, but easy to misread. The team learned to ask what a green pipeline actually checked.
- **The demo.** The fix was a structure and a rehearsal, adopted at the retrospective and used from sprint 3.
- **Stand-up lines with no outcome.** Not caught by the lint, caught by the deputy's question. The team started writing outcome-first statuses in sprint 3, with drafts from `talk.pl` (Chapter 25, Talking about progress: helps for engineers who would rather not (p. 390)).
- **The intake issue for the first drop left open.** Caught in the To-Do list at the review. Closing the intake issue belongs on the drop intake checklist (Chapter 45, The prime's model changes: receive, assess, disposition (p. 619)).

27.9 You can now…

- read a pipeline log and tell a shell error from a model error;

- explain why every job runs `perl ci/job.pl NAME`, and what goes wrong without it;
- explain what a green pipeline proves and what it does not;
- import a prime drop onto the vendor branch and merge it through a feature branch;
- choose a repository layout with the next drop in mind, and record it in a layout map;
- write a request for enablers as dated asks with the cost of no;
- recognise a demo that shows machinery instead of results, and restructure it.

27.10 Exercises

Exercise 27.1: Translate the failure

Take the first job in the failed pipeline log above. Write, in one sentence each, what a systems engineer who has never seen PowerShell needs to understand to fix it, and what Tom changed. (The runners chapter has the answer; Section 21.6, The lesson of the first week: PowerShell (p. 350).)

Exercise 27.2: Re-run the demo

Write the sprint 2 demo script that should have been given, using the structure in Chapter 25, Talking about progress: helps for engineers who would rather not (p. 390): the question, the output to show, one before-and-after with a number from the scorecard table above, one safe live change, and the ask. Keep it to five minutes.

Exercise 27.3: Read the integration branch

In the `tidewater` sandbox, list the integration branch's merges up to the end of sprint 2 with the `git log` command above. For each line, name the story it closed. Which merge brought the model in, and which merges changed only the machinery?

Exercise 27.4: A layout for your model

Sketch the layout map for a model you know: which packages go in the library, requirements, logical, physical, analysis and verification folders? Which package would you be least sure about, and what would it cost to move it after the third drop?

27.11 Applying this to your project

- **Run the pipeline on the real runner as early as you can,** and expect the first run to fail for a reason that has nothing to do with the model.
- **Keep the pipeline's scripts shell-neutral** from the start: one command per job, logic in a script in the repository.
- **Decide the repository layout before the first drop is merged,** and write it down as a map the converter applies.
- **Bring the inherited model in through a vendor branch** and never edit it there. Record the inherited debt as the starting point of a ratchet.

- **Write your asks down** with dates and the cost of no. A memo gets decided; a remark in a meeting gets remembered differently by everyone who heard it.
- **Rehearse the demo** and say the result before the mechanism. The first time you get it wrong in front of management is the last time you will need to be told.

CHAPTER

28

Sprint 3: the first red X, DOORS in the model, and a wait nobody named

From what you know →

You know

the day a reviewer finds the mistake you missed.

→ Here it is called

a red pipeline.

→ What is new

the mistake is found minutes after you make it, by a machine, before anyone else sees it.

Survival kit

What you have

a pipeline that gates every change (Chapter 27, Sprint 2: Drop 1 on develop, every change gated (p. 424)), drop 1 on `develop` through the vendor branch, the level conventions, and a team that has merged its first model changes but is still unsure it likes any of this.

What you need

the DOORS requirements inside the model next to the prime's copy, a report on every change that says which requirements have no design and no test, and a way of working with the pipeline when it says no. You also need to learn that a wait you do not say out loud is a blocker nobody can help with.

Sprint 3 ran from Monday 2 November to Friday 13 November 2026. It is the sprint the team remembers as the low point, and the survey agrees: the pulse fell from 2.4 at the end of sprint 2 to 2.2. Nothing went badly wrong. The pipeline stopped one of Bob's changes, the security office took three days to sign a form, and the burn-down chart barely moved. Those three things, each small, landed on people who had not yet seen the new way of working pay them back.

This chapter is worth reading slowly for that reason. Every team that makes this change has a

sprint 3. The work gets done; the feeling lags behind it. What the team did about the feeling matters as much as what it did about the model.

28.1 The question this sprint answered

Sprint 2 put the prime's model into GitLab. Sprint 3 asked the first question a systems engineer actually cares about:

> *Do the requirements in our model match the requirements in DOORS, and which of them have nothing designed to meet them or nothing planned to verify them?*

For thirty years the team answered that question with a spreadsheet, two days before a milestone review. Jim's scorecard records the document-era answer time as 2 working days of spreadsheet work (a notional figure, from the program's last document-era review cycle). At the end of this sprint the pipeline answers it on every change, in about 26 seconds of job time. That is the sprint's real result. The rest of this chapter is how they got there and what it felt like.

28.2 Kickoff: Monday 2 November

The kickoff ran from 09:00 to 10:30, as every kickoff does, and stood in for that day's stand-up. Jim brought the plan the team had refined and estimated on the Wednesday of the previous sprint. The kit wrote the record:

docs/tidewater/sprints/03/kickoff.txt

```
Sprint 3 kickoff, 2026-11-02 09:00 (90 minutes)

Goal: Network closed; DOORS in the model; the trace report on every change
Capacity: 18.9 points (Jim 5, Bob 8, Tom 8 per full sprint, less holidays and leave)
Committed: 15.0 points in 6 stories (79% of capacity): engineering 8.0, platform 4.5 (30%),
learning 2.5 (16%)

TW-50    E3    2.5 pts  learning     Learn: the workbook second half; talking about progress
           #56   0.5 pt  jim  2026-11-02 .. 2026-11-02  talk.pl: draft, then edit, the stand-up
           #57   1.0 pt  bob  2026-11-02 .. 2026-11-03  Workbook second half; talk.pl
           #58   1.0 pt  tom  2026-11-10 .. 2026-11-12  talk.pl; the Day-9 demo rehearsal
TW-51    E6    3.0 pts  engineering  Close the Network TODO lines
           #59   1.0 pt  bob  2026-11-03 .. 2026-11-04  Level tags in Network
           #60   2.0 pt  bob  2026-11-04 .. 2026-11-06  The include use case in
           ReviewObservations
TW-52    E7    3.0 pts  engineering  DOORS modules into the model
           #61   2.0 pt  jim  2026-11-03 .. 2026-11-06  Re-import drop 1 with the DOORS modules
           on vendor/prime
           #62   1.0 pt  bob  2026-11-09 .. 2026-11-10  Merge the DOORS modules to develop
TW-53    E2    2.5 pts  platform     Trace report and secret detection in the pipeline
           #63   2.5 pt  tom  2026-11-03 .. 2026-11-06  Add the trace job (report only) and the
           secrets job
TW-54    E4    2.0 pts  platform     Runner hardening with the ISSM
           #64   2.0 pt  tom  2026-11-06 .. 2026-11-12  Runner service account and hardening
           checklist
TW-55    E7    2.0 pts  engineering  First drift and gap analysis from the trace
           #65   2.0 pt  jim  2026-11-09 .. 2026-11-13  Write up the drift and the trace gaps
```

Three things in that record are worth pausing on.

The commitment is lower than last sprint's. The team committed 15 points against the capacity line above, where in sprint 2 it had committed 19. Sprint 2 went well: every committed point was done. The natural instinct after a good sprint is to commit more. Jim did the opposite, for two reasons Jim gave out loud at the kickoff. Wednesday 11 November is Veterans Day, a federal holiday, so the sprint has nine working days, not ten; the capacity line already accounts for that. And this sprint brings two things the team has never done: putting the DOORS export into the model, and a trace report that will show, for the first time, numbers about requirements that nobody has looked at systematically in years. Unknowns deserve slack.

Note: The thought process: commit to what you can finish, not to what you hope

A team that commits to everything it can imagine and finishes three quarters of it has told management nothing. A team that commits to less and finishes all of it has told management exactly what it can do. The second team gets trusted with more. Jim's rule, borrowed from the operating model (Chapter 13, The operating model (p. 196)), is to commit no more than about nine tenths of capacity and less when the sprint has a new kind of work in it.

Learning is still on the board. The story TW-50, "Learn: the workbook second half; talking about progress", carries 2.5 points: the second half of the SysML v2 workbook (Chapter 8, SysML v2 by example: a workbook (p. 92)) for Bob, and for everyone the stand-up drafting tool `talk.pl` (Chapter 25, Talking about progress: helps for engineers who would rather not (p. 390)). In sprint 1 learning was a quarter of the board; here it is 17% of the task points done (Section 13.8, Two ways to count "done" (p. 205)), and it keeps falling. That is the curve the BLUF promised management (Bottom line up front (p. iii)), and it only happens if the learning is on the board where everyone can see it being spent.

Platform work has shrunk. In sprint 2, building the shop took more than half the board. This sprint the platform share is 30% and engineering is 53% for the first time. Tom has two platform stories, the trace and secrets jobs and the runner hardening, and both exist because a real need showed up, not because a list of DevSecOps features said so.

Each story on that list was estimated with planning poker at the sprint 2 refinement and checked against the Definition of Ready. The kickoff did not re-estimate. It cut each story into tasks of at most a few days, assigned them, and committed. Ninety minutes, including the stand-up.

28.3 Week 1

28.3.1 Monday 2 November: drafting the stand-up

The sprint 2 retrospective had produced one change to how the team talks: statuses should lead with an outcome and carry one number. In sprint 2, not one stand-up line did; the scorecard's measure for it is 0. People reported motion ("worked on the pipeline") because that is what thirty years of status meetings had taught them to report.

So from this sprint the team drafts its statuses with `talk.pl`, which reads yesterday's GitLab activity and writes a line in the kit's terse format, outcome first. Each person then edits the draft before posting it. The kit keeps both:

docs/tidewater/sprints/03/standups/2026-11-05-talk-drafts.txt

```
jim draft:  Y TW-51 Reviewed !8. T TW-52 re-import drop 1 with the DOORS modules on..., by Nov 13.
B none
jim posted: Y TW-51 Reviewed !8. T TW-52 re-import drop 1 with the DOORS modules on, by Nov 13. B
none
bob draft:  Y TW-51 Close the Network TODO lines: 1 TODOs closed, 51 left. T TW-51 the include
use case in ReviewObservations, by Nov 13. B none
bob posted: Y TW-51 Close the Network TODO lines: 1 TODOs closed, 51 left. T TW-51 the include
use case in ReviewObservations, by Nov 13. B none
tom draft:  Y none T TW-53 add the trace job (report only) and the secrets..., by Nov 13. B none
tom posted: Y none T TW-53 (first thing: read the review comments) add the trace job (report
only) and the secrets, by Nov 13. B none
```

Look at what changed between draft and posted line. Jim and Bob posted the draft as written, apart from the ellipsis where a long task title was cut. Tom changed the plan: "(first thing: read the review comments)", because a review waiting on Tom's merge request mattered more that morning than the task the draft picked. That is the honest finding from the evidence: most mornings the draft is right, and the edit, when there is one, is the engineer's judgement. Once the board is kept up to date, the draft is usually right, and editing it takes seconds. The value is not in the editing. It is that the draft starts from what actually happened, so nobody has to remember yesterday at 09:00 or invent a number.

For the sprint the share was 1, as a fraction: every stand-up line that reported work carried an outcome and a number. Chapter Chapter 25, Talking about progress: helps for engineers who would rather not (p. 390) explains why the draft helps engineers who would otherwise say nothing, and its limits.

28.3.2 Tuesday 3 November: the DOORS modules go onto the vendor branch

Jim's first task of the sprint was to put the DOORS export into the model. The prime's drop 1 included a ReqIF file alongside the Cameo XMI. In sprint 2 the team converted only the XMI. Now Jim re-ran the import with both converters, so the DOORS modules become SysML v2 packages under `model/requirements/` next to the Cameo copy of the requirements.

Jim did it on the vendor branch, not on `develop`, and that choice deserves a sentence because it was argued. Bob's first instinct was to convert the ReqIF on a feature branch, since "it's just more files." Jim's answer was the rule the team wrote down in sprint 2 (ADR 0002, the prime boundary): anything that comes from the prime enters through `vendor/prime` and nowhere else. The DOORS export is the prime's. If it enters through a feature branch, the next drop cannot merge cleanly against it, because git has no record of the prime's version to merge from. Chapter Chapter 45, The prime's model changes: receive, assess, disposition (p. 619) works through why the vendor branch is the only way the drops stay mergeable.

28.3.3 Wednesday 4 November: a green first pipeline

Bob opened merge request !8, "Level tags in Network", at 10:00: the TODO lines in the Network package that stood for the old Cameo level stereotypes, replaced by the `@L` metadata the team defined in sprint 2. The pipeline went green on the first try, Jim approved it, and Bob merged it the same afternoon. You can look at it in the team's GitLab:

```
perl "$KIT/bin/gitlab.pl" --local gitlab --project tidewater/model mr show 8 | head -4
```

The first line names the merge request and its branch, the second its author, labels, approver and pipeline. It is a small change and it is exactly what the team wanted every change to look like: one task, one branch, one merge request, green, reviewed, merged, closed.

28.3.4 Thursday 5 November: what a good stand-up line looks like

Here is Bob's line from Thursday's stand-up, as posted:

docs/tidewater/sprints/03/standups/2026-11-05-chat.txt

```
[9:02 AM] Bob
Y TW-51 Close the Network TODO lines: 1 TODOs closed, 51 left. T TW-51 the include use case in
ReviewObservations, by Nov 13. B none
```

It names the story (TW-51), says what is now true ("Close the Network TODO lines: 1 TODOs closed, 51 left"), says what Bob will do today, and says nothing blocks the work. A manager reading it learns that one TODO line was closed and how many remain, without a meeting. The kit's lint checked it:

docs/tidewater/sprints/03/standups/2026-11-05-lint.txt

```
standups/2026-11-05-Model-chat.txt:5: ok    Jim: Y/T/B parsed
standups/2026-11-05-Model-chat.txt:7: ok    Bob: Y/T/B parsed
standups/2026-11-05-Model-chat.txt:9: ok    Tom: Y/T/B parsed
# 3 answered, 0 errors, 0 warnings
```

28.3.5 Friday 6 November: the red X

Bob opened merge request !9 at ten in the morning. The story was the last Network TODO line, a Cameo "include" between two use cases. The converter had left it as a comment, because it does not map UML includes:

```
// TODO uml:Include (xmi:id _22_1_7f3a0c1_08793d6918c3): no SysML v2 mapping in this converter
```

Bob replaced the TODO with what Bob believed was the SysML v2 form. You can see the edit in Bob's clone. The commit is named by its subject, `:/^Replace the Include TODO` (the newest commit whose message starts with those words), because commit ids differ from one copy of the repository to the next:

```
git show --format='%h %an %s' ':/^Replace the Include TODO' --
model/physical/network/Network.sysml | grep '^[-+ ]  *include\|^[0-9a-f]'
```

The pipeline for !9 failed. Two jobs went red, grammar and lint, and the merge request showed a red X. This is the first time the team saw the gate stop a change, and here is the part of the log that matters:

docs/tidewater/sprints/03/pipelines/mr9-first-23.log

```
pipeline #23 (merge_request_event, refs/merge-requests/9/merge) failed, 5 jobs, 57.2 s

==== grammar          check    failed      42 s
$ perl ci/job.pl grammar
$ perl $KIT/tools/sysml/sysml.pl check model/analysis/Analysis.sysml
model/legacy/Legacy_Import.sysml model/library/Interfaces.sysml model/library/Tidewater.sysml
model/library/Types.sysml model/library/conventions/Conventions.sysml
model/logical/Behavior.sysml model/physical/allocation/Allocations.sysml
model/physical/datacenter/DataCenter.sysml model/physical/network/Network.sysml
model/physical/operations/Operations.sysml model/physical/stations/Stations.sysml
model/requirements/model/Requirements.sysml model/verification/Verification.sysml
model/physical/network/Network.sysml:39:41: error: unexpected 'queryArchive'; expected ',', '.',
':', '::', '::>', ':=', ':>', ':>>', ...
sysml: 14 file(s), 1 error(s)
$ perl $KIT/tools/gitlab/ci-reports.pl grammar --junit grammar.xml --codequality grammar-cq.json
grammar.txt model/analysis/Analysis.sysml model/legacy/Legacy_Import.sysml
model/library/Interfaces.sysml model/library/Tidewater.sysml model/library/Types.sysml
model/library/conventions/Conventions.sysml model/logical/Behavior.sysml
model/physical/allocation/Allocations.sysml model/physical/datacenter/DataCenter.sysml
model/physical/network/Network.sysml model/physical/operations/Operations.sysml
model/physical/stations/Stations.sysml model/requirements/model/Requirements.sysml
model/verification/Verification.sysml
ci-reports grammar: 14 check(s), 1 failed, 1 finding(s); grammar_files=14 grammar_errors=1
```

The grammar job ran the kit's checker against the official SysML v2 grammar and found the problem on line 39, column 41: after `include use case :` it expected a name, a `:>` or another of the listed tokens, and it found `QueryArchive queryArchive` instead. Bob had written the type before the name, the way Cameo shows a property in a diagram compartment and the way many programming languages declare a variable. SysML v2 text puts the name first and the type after a colon: `queryArchive : QueryArchive`.

You can reproduce the error yourself from the commit, which is still in the repository:

```
before=$(git -C work/bob rev-parse --short ':/^Replace the Include TODO')
git -C work/bob show "$before:model/physical/network/Network.sysml" > network-before.sysml
perl "$KIT/tools/sysml/sysml.pl" check network-before.sysml || echo "exit status $?"
```

And the fixed file, from the commit Bob pushed to the same branch:

```
after=$(git -C work/bob log -1 --format=%h --grep='^Fix what the pipeline found' origin/develop
-- model/physical/network/Network.sysml)
git -C work/bob show "$after:model/physical/network/Network.sysml" > network-after.sysml
perl "$KIT/tools/sysml/sysml.pl" check network-after.sysml
```

28.3.6 Reading the log, line by line

Nobody on the team had read a job log before October. It is worth taking one apart once, slowly, because every red X from now on is read the same way.

Table 28.1: Reading a failing job log

Line in the log	What it tells you
`pipeline #23 (merge_request_event, refs/merge-requests/9/merge) failed, 5 jobs`	which pipeline, why it ran (a merge request), what it tested (the merge of !9 into develop, not just Bob's branch), and the result
`==== grammar check failed`	one job: its name, its stage, its result, how long it took
`$ perl ci/job.pl grammar`	the command the job ran, exactly as you can run it yourself
`$ /usr/bin/perl $KIT/tools/sysml/sysml.pl check model/…`	what that command ran in turn: the grammar check on every model file
`model/physical/network/Network.sysml:39: 41: error: unexpected 'queryArchive'; expected …`	the finding: file, line, column, what was found and what the grammar allowed there
`sysml: 14 file(s), 1 error(s)`	the checker's summary
`ci-reports grammar: 14 check(s), 1 failed, 1 finding(s)`	the same result turned into the test and code-quality reports GitLab shows on the merge request

Three habits follow from that table. Read the first line, so you know whether you are looking at your branch alone or at the merge result. Scroll to the first line with `error:` in it, not the last line of the log, because later jobs often fail *because of* the first error (here the lint job failed on the same syntax error). And copy the `$` command: it runs the same way on your laptop, which is the fastest way to try a fix. In Vim, `:make` on the file puts the same error in the quickfix list and jumps to line 39, column 41 (Chapter 10, Vim for models (p. 143)).

> **Case study: What "include use case" means**
>
> In the Cameo model, the use case ReviewObservations had a UML *include* relationship to QueryArchive: reviewing observations always involves querying the archive. The converter has no mapping for UML includes, so it left a TODO line naming the include and its xmi:id.
>
> In SysML v2 text an include is a usage inside the including use case: `include use case queryArchive : QueryArchive;` says that every performance of ReviewObservations includes a performance of QueryArchive, and gives that included performance a name, `queryArchive`, that other parts of the model can refer to. The grammar check confirms the line is well formed; the lint step confirms that `QueryArchive` resolves to the use case defined a few lines below. Both are needed. The grammar check alone would accept a misspelled type name; Chapter 8, SysML v2 by example: a workbook (p. 92) shows the difference with a worked example.

The second pipeline, #24, was green, Jim approved, and Bob merged !9 at 15:00 the same day. The whole episode, from red X to merged, took one working morning.

Bob's reaction, recorded in the sprint's quotes, is the honest one:

A red X on my own MR. One missing word, and the machine found it before Jim did.

— Bob, sprint 3

There are two ways to hear that sentence, and the team heard both. One is embarrassment: a machine graded my work and I failed in front of my colleagues. The other is relief: the mistake never reached the model, never reached Jim's review, and never reached the prime or the program office. In the Cameo years, an error like this sat in the model until somebody tried to use the element, sometimes months later.

Objection: The machine is grading me

The objection, as Bob put it to Jim: "I have been doing this for twenty-six years. I do not need a script telling me my syntax is wrong in front of everybody."

Answer

the pipeline is not a judge, and nobody sees it but the team. A red X on a merge request is the cheapest possible place for a mistake to be found: before review, before merge, before anyone downstream relies on it. The retrospective turned the complaint into a practice the team kept: fix a red pipeline on the branch, not around it, and do not take it personally. The numbers back the habit: in this sprint 3 of 4 merge requests passed on their first pipeline, and the one that did not was green by lunch.

28.3.7 Friday 6 November, continued: DOORS and the trace report

Two more merge requests opened on the same Friday.

Jim's !10, "Prime drop v1.0 with the DOORS modules", merged the vendor branch, now carrying the DOORS packages, into `develop`. Its pipeline was green. Bob reviewed it the following Tuesday, and the review note is a good example of what reviewing a drop looks like: not reading every converted line, but reading the delta report and the conflict table.

GitLab Markdown: docs/tidewater/sprints/03/samples/mr-10.md

```
# !10 Prime drop v1.0 with the DOORS modules

## What and why

The prime's drop v1.0, converted on vendor/prime and merged here.

Closes #61
Part of #52

## Checks

- [x] `gitlab.pl ci` passed locally
- [x] `docs/` regenerated and committed
- [ ] No new `// TODO` lines (a prime drop: its TODO lines are the port backlog)
- [ ] Trace job: no new orphan, unverified or drifted requirement (no trace job yet)

## Review notes

Prime changes in this drop: NWM-CR-0001.
```

```
No conflicts.

---- thread

**@bob** (2026-11-10T15:00:00Z) [system]:
added 2h of time spent

**@bob** (2026-11-10T15:00:00Z):
Read the delta and the conflict table. Outline allowance recorded in `.ci/outline-max`.

**@bob** (2026-11-10T15:00:00Z) [system]:
approved this merge request

**@tom** (2026-11-10T15:00:00Z) [system]:
approved this merge request

**@bob** (2026-11-10T15:00:00Z) [system]:
merged
```

Tom's !11 added the trace report to the pipeline, and a secrets check. The trace job runs the DOORS-to-model trace on every pipeline. Here is what it printed on the first run, from the log of pipeline #29:

docs/tidewater/sprints/03/pipelines/mr11-first-29.log

```
trace against model (without the DOORS modules) (14 .sysml files)
C requirements linked (doorsId in the model)     29 of 30
F orphan requirements (nothing satisfies)               2
G unverified requirements (no verify)                   9
I trace completeness (satisfied and verified)        66.7%
doorsIds in the model but not in this export            1
text differs from DOORS                                 1
not in the model: TW-SYS-016
orphans: TW-SYS-012 TW-SYS-016
unverified: TW-STK-001 TW-STK-002 TW-STK-003 TW-STK-004 TW-STK-005 TW-STK-006 TW-SYS-011
TW-SYS-016 TW-SEG-002
drift: TW-SYS-006
  TW-SYS-006 DOORS: The data center shall publish a marine bulletin twice a day, on a fixed
  schedule.
  TW-SYS-006 model: The data center shall publish a marine bulletin on a fixed schedule.
$ perl $KIT/tools/gitlab/ci-reports.pl trace --junit trace.xml --metrics trace.metrics trace.txt
ci-reports trace: 14 check(s), 14 failed, 0 finding(s); trace_linked=29 trace_total=30
trace_orphans=2 trace_unverified=9 trace_completeness_pct=66.7 trace_drift=1 trace_not_in_model=1
trace gaps: 2 orphan(s), 9 unverified, 1 drifted
```

Read it slowly; this is the most important output of the sprint.

- **C, linked:** 29 of 30 DOORS requirements have a counterpart in the model with the same DOORS id. One does not: TW-SYS-016 is in DOORS and not in the prime's model at all.
- **F, orphans:** 2 requirements have nothing in the model that satisfies them.
- **G, unverified:** 9 requirements have no verification case.
- **I, completeness:** 66.7% of requirements are both satisfied and verified.

- **Drift:** TW-SYS-006 says something different in DOORS than in the model's copy.

The drift line is the easiest to see with your own eyes. Both copies of TW-SYS-006 are in the model now, one from the prime's Cameo export and one from the DOORS export:

```
git grep -h 'doorsId = "TW-SYS-006"' origin/develop -- model/requirements | sed 's/ attribute
.*//'
```

The Cameo copy says the data center shall publish a marine bulletin on a fixed schedule. DOORS says twice a day, on a fixed schedule. Which wording is right is not the government team's call to make by editing a file: DOORS is the requirements baseline the prime is contracted to, and the model is supposed to match it. The gap goes back to the prime as a discrepancy report, DR-1, in Chapter 29, Sprint 4: the model site, a cycle broken, and the demo that landed (p. 459).

How does the trace job know which copy is which? Every file the DOORS converter writes starts with a line that says so:

```
git show origin/develop:model/requirements/system/System_Requirements.sysml | head -2
```

The trace job compares DOORS, as exported, against the model *without* those generated packages; otherwise it would compare DOORS with its own copy and find nothing. Chapter Chapter 20, The pipeline, job by job (p. 302) shows the rule in the job.

None of these gaps were made by the government team. They are in the prime's model and the prime's DOORS module, and they had been there, unseen, since drop 1 arrived. The trace job made them visible. That is why Tom added it as **report only**: the job prints the gaps and never fails a pipeline because of them. If it failed, every merge request on the team would go red for problems none of them could fix, and the team would learn to ignore red. Chapter Chapter 20, The pipeline, job by job (p. 302) shows the job; Chapter 30, Sprint 5: Grading the requirements, and a trace that defends itself (p. 475) shows the day it becomes a gate, and why only then.

Note: The thought process: a check you cannot pass teaches people to ignore checks

Tom's rule for adding a job: if the team cannot make it green by its own work, the job reports; it does not fail. The gaps here belong to the prime. The right response is a discrepancy report to the prime (Chapter 29, Sprint 4: the model site, a cycle broken, and the demo that landed (p. 459)), not a red pipeline for Bob. A gate is for problems the person merging can fix.

The secrets job is the "Sec" in DevSecOps, at its smallest useful size. It scans every tracked file for things that look like keys, tokens or passwords and fails if it finds one. On the first run it found none in the repository's tracked files. It took Tom less than an hour, it runs in seconds, and it closes the most likely security incident for a modeling team: someone pastes a credential into a file and pushes it. Chapter Chapter 48, Security: protect the environment, engineer it into the system (p. 651) puts it in context.

28.4 Week 2

28.4.1 Monday 9 to Thursday 12 November: a wait said a day late, with no one to ask

Tom's second story, TW-54, was runner hardening: the GitLab runner that executes every pipeline job is a Windows machine running jobs under an account, and the security office wanted that account to be a service account with the least privilege it needs, approved by the ISSM, with a hardening checklist in the wiki. Tom started the task on Monday 9 November; it closed on Thursday 12 November.

Most of those days were waiting: the checklist was quick, the ISSM's signature on the service account was not. Here are Tom's stand-up lines from the three working days of that wait (Wednesday 11 November was a holiday), as posted:

docs/tidewater/sprints/03/standups/2026-11-09-chat.txt

```
[9:03 AM] Tom
Y TW-53 Trace report and secret detection in: 1 TODOs closed, 50 left. T TW-54 (first thing: read
the review comments) runner service account and hardening checklist, by Fri. B none
```

docs/tidewater/sprints/03/standups/2026-11-10-chat.txt

```
[9:03 AM] Tom
Y none T TW-54 runner service account and hardening checklist, by Fri. B TW-54 waiting on the
ISSM to approve the runner service account -- need: a date from the ISSM
```

docs/tidewater/sprints/03/standups/2026-11-12-chat.txt

```
[9:03 AM] Tom
Y none T TW-50 talk.pl; the Day-9 demo rehearsal, by Fri. B TW-54 runner service account and
hardening... blocked: waiting on the ISSM to approve the runner service account -- need: help to
unblock it
```

Read the last field, day by day. On Monday it says **B none**, although the request was already sitting with another office. On Tuesday the block appears, a day late, and the kit records it in the journal. But look at the ask: "a date from the ISSM". It names no one who can get that date. On Thursday it is still there, and the ask has become "help to unblock it", still with no name and no date. The kit's lint could not catch either problem, because a status that says "none", or a blocker with a vague ask, is a valid status.

At the retrospective, Tom said what had happened:

> *Three days waiting on a signature. The pipeline was the fast part.*
>
> — Tom, 10 November

and the team wrote it down as the sprint's one thing to stop:

docs/tidewater/sprints/03/retro.txt

```
Stop:
  - Waiting for the ISSM without a date
```

This is the most useful failure in the twelve weeks, because it is the one every engineer recognizes. Engineers do not like to say they are blocked. It sounds like an excuse, or like asking for help, or like criticizing another office. So they say “none” and wait. The stand-up exists to catch exactly this, and it only works if waiting is said out loud, with a date and an ask:

```
Y TW-54 checklist in the wiki. T TW-54 runner service account, by Tue. B TW-54 waiting on the
ISSM to approve the runner service account -- need: Jim to ask the ISSM for a date today
```

Said on Monday, that line would have started the block a day earlier, named the person who could act, and put the ask in Jim’s hands that morning. Chapter Chapter 25, Talking about progress: helps for engineers who would rather not (p. 390) has the templates; the change the team made was simply to agree that a wait on anyone outside the team for more than one day is a blocker and gets said.

28.4.2 Tuesday 10 November: the DOORS modules merge

Bob merged !10 on Tuesday, after reading the delta. The review note mentions the outline allowance. That is the file `.ci/outline-max`, which records how many outline violations (packages with too few or too many members, folders with a single file) the model is allowed to have. The prime’s drop brings its own: the converted Requirements package has more members than the 2-to-9 rule allows, because Cameo put every requirement in one package. The outline job does not fail the drop for that. It records the new count as the allowance, and from then on any change that *adds* a violation fails, while any change that removes one must lower the number. You can see the allowance move in the history, one line per commit that changed the file, with the number it recorded:

```
git log --format='%h %ad' --date=short origin/develop -- .ci/outline-max |
  while read h d; do echo "$d $(git show $h:.ci/outline-max)"; done
```

Read it from the bottom up: 1 at the start, 17 after the first import of drop 1, 19 once the DOORS modules came in, and back to 17 in December when Tom removed the Legacy Import clutter and lowered the number, as the ratchet requires.

That pattern, a measure that may only get better, is called a ratchet. The team uses it for every measure it cannot make perfect on day one: outline violations here, trace gaps from Chapter 30, Sprint 5: Grading the requirements, and a trace that defends itself (p. 475), package complexity later (Chapter 20, The pipeline, job by job (p. 302)).

From this point the DOORS requirements live in the model, under `model/requirements/stakeholder/`, `system/` and `segment/`, and the Cameo copy of the requirements lives beside them under `model/requirements/model/`. Two copies of the same requirements may look wasteful. It is the opposite: DOORS is the authoritative wording, the Cameo copy is what the prime’s model actually links to, and keeping both is what lets the trace job see where they disagree.

28.4.3 Wednesday 11 November: Veterans Day

No stand-up, no work. The capacity line accounted for it. It is worth saying because new teams often forget holidays when they plan, then discover at the review that they were short a day.

28.4.4 Thursday 12 November: refinement and planning poker

At 10:00 the team ran refinement for sprint 4: the stories Jim had drafted, checked against the Definition of Ready, and estimated with planning poker in the meeting chat. The kit tallied the votes. Most rounds went straight to consensus. One did not:

docs/tidewater/sprints/03/poker-2026-11-12.txt

```
ESTIMATE TW-67 Close the Stations level tags  (3 answers at 10:03)
  Bob            5
  Jim            3
  Tom            5
  no consensus: median 5, mode 5, range 3-5

ESTIMATE TW-67 Close the Stations level tags  (3 answers at 10:05)
  Bob            5
  Jim            5
  Tom            5
  consensus: 5
```

Jim voted 3 for "Close the Stations level tags"; Bob and Tom voted 5. The tally shows no consensus, so Jim asked the question the operating model says to ask: what does the high vote know? Bob knew something Jim did not. The Stations package is the largest in the model: it holds the buoys, the shore stations and everything inside them, and a good part of those parts are procured items from suppliers, which in the prime's model carry an extra stereotype with supplier and part-number tags. Bob expected those to take longer than the plain level tags. On the second round all three voted 5.

That conversation is the reason planning poker exists. The numbers matter less than the moment the outlier explains themselves. In Chapter 29, Sprint 4: the model site, a cycle broken, and the demo that landed (p. 459) you will see that even 5 was not enough, and why.

Bob's stand-up on Thursday and Friday said "T none": all of Bob's committed work was finished and the next sprint's stories were not yet started. The operating model's answer to an empty "today" is to pull the top story of the next sprint forward rather than to look busy. An empty "today" is a planning signal, not a problem.

Tom's lines on those two days show a different and smaller lesson:

docs/tidewater/sprints/03/standups/2026-11-12-chat.txt

```
[9:03 AM] Tom
Y none T TW-50 talk.pl; the Day-9 demo rehearsal, by Fri. B TW-54 runner service account and
hardening... blocked: waiting on the ISSM to approve the runner service account -- need: help to
unblock it
```

Tom's drafted status names the drop-intake issue for prime drop 1 (TW-25), due "by Oct 16". That issue was still open: the sprint 1 work it tracked was done, but the issue itself was never

closed. `talk.pl` drafts from the board, so a forgotten open issue shows up as somebody's "today". The analytics for the sprint list it as the oldest open issue, and it was still showing up in Tom's drafts at the end of sprint 4. The fix takes ten seconds: close it. The lesson is the one every tool on the team teaches sooner or later: the tools are only as good as the board you keep.

28.4.5 Friday 13 November: review and retrospective

Jim closed the last story of the sprint at 10:00, the write-up of the drift and the trace gaps. Then the review:

docs/tidewater/sprints/03/review.txt

```
Sprint 3 review, 2026-11-13 10:00

Goal: Network closed; DOORS in the model; the trace report on every change

Demonstrated:
  TW-50 Learn: the workbook second half; talking about progress
  TW-51 Close the Network TODO lines
  TW-52 DOORS modules into the model
  TW-53 Trace report and secret detection in the pipeline
  TW-54 Runner hardening with the ISSM
  TW-55 First drift and gap analysis from the trace

Artifacts shown: the merge requests and pipelines of the sprint.

Feedback (Deputy PM): "Which requirements have no verification? You said you would know." Bob
opened the trace report: 9 of 30, each with its ID; they are the prime's to fix and go out as
discrepancy reports in sprint 4.

The four headline numbers: TODO lines left 50; verify coverage 70%; open DRs 0; blocked items 0.
```

Every committed story was done: 15 of 15 story points. The four headline numbers are the ones the team reports at every review (Chapter 25, Talking about progress: helps for engineers who would rather not (p. 390)): TODO lines left, verify coverage, open discrepancy reports, and blocked items.

The demo script, as `talk.pl review` drafted it, is in the evidence. Its fourth and sixth sections are worth reading as a lesson in reading your own numbers:

docs/tidewater/sprints/03/review-script.md

```
## 4. One before-and-after
- Now: from proposing a change to the first review takes 4.1 h (median). Before: [the
document-era number -- put it in a --baseline file].
- Now: from first edit to accepted change takes 4.2 h (median). Before: [the document-era number
-- put it in a --baseline file].
- Before: [how long "which requirements are unverified?" used to take]. Now: the trace check
answers it on every change, in about 2 min.

## 5. One safe live change (on a scratch branch, never on develop)
- Delete one satisfy line on a scratch branch and open a draft merge request: in about 2 min the
trace check names the requirement that lost its design. Close the merge request afterwards.
```

```
## 6. What it means for the next milestone
- Sprint 4 starts Mon Nov 16: 50 TODO lines to go; at this sprint's pace that is about 25 more
sprints.
```

The first half is useful: the median time from opening a merge request to its first review, and the time from first edit to accepted change, measured in hours, against a document-era change cycle that took weeks. The second half is a trap. The draft divides the TODO lines left by this sprint's rate of closing them and announces that the port will take about twenty-five more sprints. That is arithmetically correct and practically wrong: this sprint's engineering was DOORS and the trace report, not TODO lines, so this sprint's TODO pace says nothing about the next one's. Delete that line before anyone reads it aloud. Every forecast a tool drafts for you is a hypothesis; you are the one who knows whether the assumption behind it holds.

The review showed the sprint's merge requests and their pipelines, including the trace report on a real merge request. Anyone watching asks the obvious question: who fixes the gaps? The plan for sprint 4 answers it: a discrepancy report to the prime for each gap, through the contract.

The retrospective followed at 11:00:

docs/tidewater/sprints/03/retro.txt

```
Sprint 3 retrospective, 2026-11-13 11:00

Start:
  - Splitting MRs before they reach 300 lines
  - Drafting the stand-up with talk.pl, then editing it
Stop:
  - Waiting for the ISSM without a date
Continue:
  - Fixing a red pipeline on the branch, not around it
  - The Day-9 rehearsal

Pulse survey (1-5, anonymous): mean 2.2
```

And the pulse survey, five questions answered anonymously on a 1-to-5 scale:

Table 28.2: Sprint 3 pulse survey (mean 2.2)

Question	Jim	Bob	Tom
I understand why we are working this way.	2	2	2
I can do my work in the new tools without help.	2	2	2
The checks catch problems I would have missed.	2	2	2
I can see our progress.	3	2	2
I would choose to keep working this way.	3	2	3

(The survey is anonymous to management. The team agreed to show its own answers in this book.)

Read Bob's column. Even "the checks catch problems I would have missed" is low, in the sprint where a check caught Bob's mistake. That is not a contradiction. Being caught does not feel like being helped the first time. Jim did not argue with the numbers. Jim said, at the end of

the retrospective, that the survey would be run every sprint and that Jim expected it to stay low until the work started paying people back personally. It did, in Chapter 29, Sprint 4: the model site, a cycle broken, and the demo that landed (p. 459).

28.4.6 The weekly status

Every Friday the kit writes the weekly quad from the journal and Jim sends it to management. Here is the one for 13 November:

docs/tidewater/sprints/03/quad-2026-11-13.txt

```
Weekly status Model -- as of 2026-11-13 (sprint 3)
Sprint progress 100% (15/15 SP) improving (was 37%)   On-time delivery: 4/4 this week, 6/6 this
sprint

TECHNICAL PRIORITIES
  DONE  TW-51        3  "Close the Network TODO lines" Bob
  DONE  TW-52        3  "DOORS modules into the model" Jim
  DONE  TW-53      2.5  "Trace report and secret detec Tom
  DONE  TW-54        2  "Runner hardening with the ISS Tom
  DONE  TW-55        2  "First drift and gap analysis  Jim
  DONE  TW-50      2.5  "Learn: the workbook second ha Tom

WATCH ITEMS / PM HELP NEEDED
```

Sprint progress 100%, every story on time, no watch items. Look again at the watch items: "none". The quad was as honest as its inputs. The wait in week 2 did reach the journal as a block, from Tuesday to Thursday, but it was cleared before Friday and stayed under the quad's three-day threshold for asking the program manager for help. What cost the team was not the tooling: it was the Monday nobody heard about, and an ask that named nobody.

28.5 What the numbers say

The GitLab analytics for the sprint, computed by `gitlab.pl analytics` from what GitLab recorded:

```
perl "$KIT/bin/gitlab.pl" --local gitlab --project tidewater/model analytics --since 2026-11-02
--until 2026-11-13 | sed -n '/3. CI/,/job failures/p'
```

The sprint's failed pipelines were both Bob's change: the push pipeline on Bob's branch and the merge request pipeline on !9, each failing grammar and lint (the lint job reports a syntax error as a lint error too). Every other pipeline passed. The value-stream section of the same report, in the evidence (Chapter 32, Measuring with GitLab analytics (p. 513)), shows the time from opening a merge request to its first review at a median of 4.1 hours.

Table 28.3: Sprint 3 against sprint 2

Measure	End of sprint 2	End of sprint 3
TODO lines left	52	50
DOORS requirements linked in the model	29 of 30	29 of 30
Changes gated by the pipeline	6 of 6	4 of 4
Median hours, first edit to accepted change	3.6	4.2
Package dependency cycles	1	2
Stand-up lines with an outcome and a number	0	1
Pulse (1 to 5)	2.4	2.2

The TODO count barely moved, because this sprint's engineering was DOORS and trace, not TODO lines. The trace and the stand-ups moved a lot. The pulse went down. All three are true at once, and the team learned to report all three.

28.6 Tools adopted this sprint, and the pain each one removed

Table 28.4: Tools adopted in sprint 3

Tool	The pain it removed	Where to read more
The trace job, report only	"Which requirements are unverified?" took two days of spreadsheet work before every review	Chapter 20, The pipeline, job by job (p. 302)
The secrets job	A credential pasted into a file and pushed is the most likely security incident for a modeling team	Chapter 48, Security: protect the environment, engineer it into the system (p. 651)
`talk.pl standup` drafts	Nobody remembered yesterday at 09:00, and status lines reported motion instead of outcomes	Chapter 25, Talking about progress: helps for engineers who would rather not (p. 390)
The runner service account	The runner ran jobs under a person's account with more rights than it needed	Chapter 21, Runners: where the pipeline runs (p. 347)

28.7 What Jim told management

Jim's Friday note to management is built from two things the kit already wrote: the review script's thirty-second version and the weekly quad. In words, it says this:

Note: Sprint 3, the Friday note

Every committed story is done (15 of 15 story points). The DOORS requirements are now in the model, and every change runs a trace report: 29 of 30 DOORS requirements are linked, 2 have nothing satisfying them, 9 have no verification, and 1 says something different in DOORS than in the model. These gaps are in the prime's model and are not new: we can now see them. Next sprint we send each one to the prime as a discrepancy report. Nothing

blocks us.

The note leaves out the morale dip. Whether, how and to whom a lead reports the team's mood is a judgment call, and Chapter 41, For the lead: leading a reluctant team (p. 574) is about exactly that kind of judgment.

28.8 You can now

- Read a failing pipeline log, find the file, line and column, and reproduce the error locally with the grammar check.
- Explain the SysML v2 naming order (`name : Type`) and spot the v1 habit that writes it backwards.
- Read the trace report's C, F, G, I and drift lines and say which gaps are yours and which are the prime's.
- Explain why a new check starts as report only and becomes a gate only when the team can pass it.
- Say a wait out loud as a blocker, with a date and an ask.
- Use planning poker's outlier question to surface what one person knows.

Exercise 28.1: Make the grammar check say no

Do practice lesson 11 (Chapter 27, Sprint 2: Drop 1 on develop, every change gated (p. 424) shows the setup). It plants exactly the kind of mistake Bob made: a model change with a syntax error. Watch the pipeline fail, read the job log, find the line, fix it on the branch, and push again. Then write the stand-up line you would post the next morning, outcome first, with one number.

Exercise 28.2: Read the trace report

In the Tidewater sandbox, show pipeline #29's trace job log with `perl "$KIT/bin/gitlab.pl" --local gitlab --project tidewater/model pipeline show 29`, open the trace log it names, and list the two orphans and the drifted requirement. For each, decide: is this ours to fix, or the prime's? Write one sentence per gap saying why.

Exercise 28.3: Rewrite Tom's stand-up

Rewrite Tom's posted lines from 9 and 10 November so the three-day wait would have reached Jim on the first morning. Run your line through the kit's lint (`daily.pl lint`, Chapter 24, The rhythm: stand-ups, poker, refinement, kickoff, demo, retro (p. 377)) to check it parses.

28.9 Applying this to your project

- **Expect a sprint 3.** Somewhere around week five or six, the novelty is gone and the payoff has not arrived. Measure it (a five-question pulse costs a minute) so you can see it, and do not argue with it.

- **Put your requirements source into the model next to the model's copy,** and run a trace on every change. The first report will show gaps that were always there. Decide in advance whose they are and how they will be reported.
- **Add every new check as report-only first.** Make it a gate only when the people merging can make it green.
- **Agree that waiting on anyone outside the team for more than a day is a blocker,** and that it is said at the stand-up with a date and an ask.
- **Close the issues you finish.** Every tool that reads the board, from the stand-up draft to the analytics, repeats your board's mistakes back to you.

CHAPTER

29

Sprint 4: the model site, a cycle broken, and the demo that landed

From what you know →

You know

the program review where the slides are the only evidence.

→ Here it is called

the model site, and a demo.

→ What is new

management looks at the live model, not at screenshots of it.

Survival kit

What you have

the DOORS requirements in the model, a trace report on every change that lists the prime's gaps (Chapter 28, Sprint 3: the first red X, DOORS in the model, and a wait nobody named (p. 440)), a team that has seen the pipeline say no and survived it, and a pulse survey that says morale is low.

What you need

the structure of the model's biggest package tagged and checked, one of the inherited dependency cycles gone, every gap the trace found sent back to the prime through the contract, and something management can open without a briefing. You also need a demo that tells management what they got for two weeks, because the last one did not.

Sprint 4 ran from Monday 16 November to Friday 27 November 2026. Thanksgiving fell on Thursday 26 November, so the sprint had fewer working days than a full one; the capacity line in the kickoff record accounts for the holiday and the leave around it.

It is the sprint in which the work started to pay the team back. The pulse went from 2.2 to 3. The TODO count fell from 50 to 22. And at the review, the program manager opened the model on

a phone.

29.1 The question this sprint answered

Sprint 3 asked whether the requirements match. Sprint 4 asked three questions about structure and one about communication:

> *Is every part in the Stations package at the right level of the part tree? Which of the dependency cycles the prime handed us can we break? What exactly do we ask the prime to fix? And can management see the model without us?*

The first three are systems engineering questions the team had always asked, in reviews, with screenshots. The fourth is new: in the Cameo years nobody outside the team ever looked at the model directly. They looked at slides made from it.

29.2 Kickoff: Monday 16 November

docs/tidewater/sprints/04/kickoff.txt

```
Sprint 4 kickoff, 2026-11-16 09:00 (90 minutes)

Goal: Stations levels and Types closed, one cycle broken, gaps reported; the model site for
management
Capacity: 18.4 points (Jim 5, Bob 8, Tom 8 per full sprint, less holidays and leave)
Committed: 17.0 points in 7 stories (92% of capacity): engineering 12.5, platform 3.0 (17%),
learning 1.5 (8%)

TW-66    E3    1.5 pts  learning     Learn: reviewing a model change; the rehearsed demo
           #73   0.5 pt  jim  2026-11-16 .. 2026-11-16  Reviewing a model change: the checklist
           #74   0.5 pt  bob  2026-11-16 .. 2026-11-16  Reviewing a model change: line comments
           #75   0.5 pt  tom  2026-11-25 .. 2026-11-25  The Day-9 demo rehearsal
TW-67    E6    5.0 pts  engineering  Close the Stations level tags
           #76   3.0 pt  bob  2026-11-16 .. 2026-11-20  Level tags in Stations
           #77   2.0 pt  bob  2026-11-27 .. 2026-12-01  Walk the Stations tree with the prime's
           modeler; record what they confirmed
TW-68    E6    2.0 pts  engineering  Units and value types
           #78   2.0 pt  bob  2026-11-24 .. 2026-11-27  Value types specialize ISQ quantities
TW-69    E6    2.0 pts  engineering  Break the Network <-> Operations cycle
           #79   2.0 pt  tom  2026-11-16 .. 2026-11-19  Remove the ServiceVessel -> Technician
           dependency
TW-70    E7    2.0 pts  engineering  Discrepancy reports DR-1 to DR-4 to the prime
           #80   2.0 pt  jim  2026-11-17 .. 2026-11-20  Open DR-1 to DR-4
TW-71    E2    3.0 pts  platform     The model site on GitLab Pages, rebuilt nightly
           #81   3.0 pt  tom  2026-11-19 .. 2026-11-24  Add the pages job and the nightly
           schedule
TW-72    E3    1.5 pts  engineering  First sprint review with the program office; the delivery
interface with the prime
           #82   1.0 pt  jim  2026-11-23 .. 2026-11-24  Sprint review with the program office
           #83   0.5 pt  jim  2026-11-25 .. 2026-11-25  Agree the delivery interface with the
           prime (meeting)
```

The commitment is higher than sprint 3's: 17 points, against 15. Jim committed more because sprint 3 had finished everything and because the kinds of work in sprint 4 were familiar: level tags (Bob had done Network), a dependency edit (small), a pipeline job (Tom's third), and discrepancy reports (writing, which the team had done all its career). The one story with real uncertainty is TW-67, "Close the Stations level tags", which the team had argued up to a higher estimate at the previous refinement (Section 28.4.4, Thursday 12 November: refinement and planning poker (p. 452)).

Look at the work mix. Engineering is 74% of the task points done (Section 13.8, Two ways to count "done" (p. 205)); platform has fallen to 18% and learning to 9%. Three sprints earlier, platform and learning together were most of the board. This is the moment the team stops building the shop and starts working in it.

The learning story is small and practical: how to review a model change (Chapter 22, Reviewing a model change (p. 357)), and a rehearsal of the sprint review. The rehearsal is there because of what happened at the sprint 2 review, and it deserves its own section.

29.2.1 Why the demo gets rehearsed

At the end of sprint 2 Tom demonstrated the new pipeline: the terminal, then the YAML, then a green pipeline, in that order, with the vocabulary Tom had spent two weeks learning. The deputy program manager's feedback, recorded in the review notes:

docs/tidewater/sprints/02/review.txt

```
Feedback (Deputy PM): "I did not follow the demo. What did we get for the two weeks?" Tom showed
the terminal, the YAML and a green pipeline, in that order; the room heard jargon and no number.
```

Nothing Tom said was wrong. It was the wrong order and the wrong words for the audience. The team's sprint 2 retrospective decided two things: the demo is rehearsed on the Wednesday of week 2, after refinement, and the person who drives the screen rotates. Chapter Chapter 25, Talking about progress: helps for engineers who would rather not (p. 390) explains the six moves of a demo that lands; this sprint is the first time the team used them.

29.3 Week 1

29.3.1 Monday 16 November: learning to review

All three closed their learning tasks on the first morning: Jim worked through the review checklist the team keeps in `docs/conventions/review-checklist.md`, Bob practiced line comments and suggestions on a scratch merge request, and Tom wrote the first draft of the demo plan for 27 November. Reviewing is a skill the team had in the document era, where it meant red pen on a printed specification. It transfers better than they expected. What is new is that the change under review is small, the reviewer sees exactly the lines that changed, and the pipeline has already checked the mechanical things, so the reviewer's attention goes to whether the change is *right*.

29.3.2 Thursday 19 November: breaking a cycle in two lines

The prime's model came with two package dependency cycles, measured in Chapter 2, The Cameo years (p. 10): Stations depends on DataCenter and DataCenter on Stations, and Network depends on Operations and Operations on Network. A cycle means neither package can be understood, reviewed or changed without the other. In a diagram-based tool they are hard to see; in text they are a line each.

Tom took the Network and Operations cycle. The Operations package depended on Network for one reason only: a `dependency` from ServiceVessel to Technician, a left-over trace from the document port that said nothing a systems engineer would miss. The fix removed it, and the import of Network that existed only to serve it:

```
git show --format='%h %an %ad %s' --date=short ':/^Break the Network <-> Operations cycle' | grep
'^[0-9a-f]\|^[-+] '
```

Two removed lines. Jim approved !12 the same day, and Tom merged it. The cycle count the scorecard tracks went from 2 to 1.

> *The cycle was two lines. In Cameo it was a week of looking at a diagram.*
>
> — Tom, sprint 4

Note: The thought process: is it ours to break?

Operations.sysml is a file the converter writes from the prime's drop. Editing it means the next drop will try to put the dependency back, unless the prime removes it too. Tom asked that question before writing the change, and the answer has two parts. First, the government's v2 model is the government's: if a dependency is meaningless, the v2 model should not carry it, and the vendor-branch merge will show the conflict when the prime's next drop arrives (Chapter 45, The prime's model changes: receive, assess, disposition (p. 619) works through how such edits survive). Second, the prime should know, so the team keeps a list of the clutter it removes for the discrepancy reports. The other cycle, Stations and DataCenter, is harder: it involves the observation record that both packages use, and deciding who owns it is a design decision, not a cleanup. It went to Chapter 30, Sprint 5: Grading the requirements, and a trace that defends itself (p. 475).

29.3.3 Friday 20 November: the Stations tags, a red X, and a question for the prime

Bob opened merge request !13, "Level tags in Stations", at ten in the morning. It replaced the package's level-stereotype TODO lines with `@L` metadata, the same mechanical change Bob had made in Network, on a much larger package. The grammar and lint jobs passed.

The pipeline failed anyway. Not grammar, not lint: the **outline** job. Here is the part of its log that matters:

docs/tidewater/sprints/04/pipelines/mr13-first-36.log

```
==== outline          check    failed      13 s
$ perl ci/job.pl outline
```

```
$ perl $KIT/tools/sysml/model.pl check

== Level tags ==
@L tag mismatches         1
  WaveSensor: tagged L5, first reached at L4
```

and at the end of the same job:

docs/tidewater/sprints/04/pipelines/mr13-first-36.log

```
ci-reports outline: 2 check(s), 2 failed, 20 finding(s); level_tag_mismatches=1
outline_violations=19
outline violations: 20 (allowed: 19)
this change adds outline violations (20 > 19): fix them, do not raise .ci/outline-max
```

Two findings. The outline violation count went up by one, which the ratchet does not allow. And the level-tag check found a part that is tagged at the wrong level: WaveSensor is tagged L5, an assembly, but the first place it appears in the part tree is directly inside MooredBuoy, an L3 station, which makes it an L4 subsystem. The extra outline violation comes from the same root.

You can reproduce the finding from Bob's commit:

```
git worktree add -q --detach ../../s4-stations ':/^Level tags in Stations'
cd ../../s4-stations && perl "$KIT/tools/sysml/model.pl" check | grep -A1 "tag mismatches"
```

Bob's tag was not a typo. Bob had copied the level from the prime's Cameo stereotype, exactly as the TODO line said. The prime's model is inconsistent with itself: the stereotype says L5, the structure says L4. In the document era this kind of inconsistency was found, if it was found at all, by someone reading two documents side by side.

This is the decision point the sprint is remembered for. Bob and Jim had three options:

1. Tag WaveSensor L5 as the prime says, and raise the outline allowance so the pipeline passes. The pipeline's own message forbids this: "fix them, do not raise .ci/outline-max".
2. Wait for the prime to fix its stereotype, leaving the Stations tags unmerged until a later drop.
3. Tag it L4 in the government's copy, where the structure says it belongs, and send the inconsistency to the prime as a discrepancy report so their model and the government's converge.

They chose the third, and the commit says so in the model itself:

```
git show --format='%h %s' ':/^Re-level WaveSensor' | grep '^[0-9a-f]\|^[-+] '
```

Jim opened DR-7 the same day. The second pipeline was green, Jim approved, and Bob merged !13 that afternoon.

29.3.4 Reviewing !13: what Jim actually checked

Jim's approval came after the second pipeline, not the first. That is the first line of the team's review checklist, learned on the Monday of this sprint:

tools/gitlab/templates/docs/conventions/review-checklist.md

```
## Before you start

- [ ] The pipeline is green. If it is red, do not review yet: say so and wait.
- [ ] The description says what and why, and links its issue (`Closes #N`).
- [ ] The change is small enough to review in 30 minutes. If not, ask for it to be split.
```

A red pipeline means the change is not finished; reviewing it wastes the reviewer's time and invites the author to argue with the machine through the reviewer. Once the pipeline was green, Jim's review was about the engineering. The size was fine: the change touched one file, mostly one kind of line, about fifty lines in all. The levels matched the part tree, which the outline job had now confirmed mechanically, so Jim did not need to count levels by eye. And one rule of the checklist applied directly to this change:

tools/gitlab/templates/docs/conventions/review-checklist.md

```
- [ ] **Converter files**: lines that came from the prime are changed only to close a `// TODO`
line or by an
      accepted DR. Other changes to the prime's content go back to the prime.
```

The WaveSensor line changed something the prime wrote, beyond closing a TODO. The rule allows such a change only with a discrepancy report behind it, and the line names one in its comment. Jim's review took minutes, and most of those minutes went to the one line that mattered. That is what a review checklist is for: it moves the reviewer's attention from the mechanical to the consequential.

Objection: We should not change the prime's model

The objection, in Bob's words: "If we change their levels, we own a model that disagrees with the contract deliverable."

Answer

the government's v2 model already disagrees with the prime's v1 model in every place the team has closed a TODO; that is the point of the port. What must not happen is a *silent* disagreement. The change carries the DR number in a comment on the line itself, the DR goes to the prime through the contract, and when the prime's drop fixes the stereotype the vendor-branch merge brings the two back together. A disagreement that is written down, reported and tracked is configuration management. One that is not is the spider web again.

29.3.5 Friday 20 November, continued: discrepancy reports to the prime

While Bob dealt with WaveSensor, Jim did the sprint's most important piece of systems engineering, and it involved no model edits at all. Jim turned each gap the trace report had found the

sprint before into a discrepancy report: an issue in GitLab, from the team's Discrepancy Report template, labeled for the prime, and transmitted through the contracting channel.

```
perl "$KIT/bin/gitlab.pl" --local gitlab --project tidewater/model issue list --state all --label
type::discrepancy | sed 's/~[^ ]* //g'
```

(The list is the team's GitLab today, at the end of sprint 6; the reports about the Legacy Import clutter and the DOORS wording were opened in Chapter 30, Sprint 5: Grading the requirements, and a trace that defends itself (p. 475).) Here is the first of them, as Jim wrote it:

GitLab Markdown: docs/tidewater/sprints/04/samples/discrepancy-report-DR-1.md

```
## Discrepancy

| | |
|---|---|
| DR number | DR-1 |
| Prime tracking | (to be assigned by the prime) |
| Found in drop | v1.0, delivered 2026-10-02 |
| DOORS id | TW-SYS-006 |
| Severity | 2 |

## What we found

DOORS says "twice a day"; the model's copy does not. The trace job reports it as drift.

## What we ask the prime to do

Correct it in the v1 model and say so in the next drop's MANIFEST.

## Closure

- [x] Sent to the prime on 2026-11-20 by @jim
- [ ] Fixed in drop ...
```

Table 29.1: Anatomy of a discrepancy report

Part of the report	What goes in it	Why
DR number	the team's own sequence, assigned when the issue is opened	so every later reference, in a model comment or a drop's MANIFEST, can cite it
Prime tracking	left blank until the prime assigns its own number	the two organizations will each track it; both numbers end up on the issue
Found in drop	the drop version and its delivery date	the prime fixes what it delivered, not what it has now
DOORS id	the requirement, if there is one	the trace job reports by DOORS id, so closure can be checked mechanically
Severity	the team's scale, from the template	so the prime and the program office can order their work

Part of the report	What goes in it	Why
What we found	one or two sentences, in engineering words, and how it was found	the prime must be able to reproduce it without the government's tools
What we ask	exactly what the government wants changed, not how	the model is the prime's to change
Closure	checkboxes: sent, by whom, when; fixed in which drop	stays open until a drop actually fixes it

The report says what was found, where, how it was found, and exactly what the government asks the prime to do. It does not say how to fix it inside Cameo; that is the prime's business. And it has a closure section that stays open until a drop actually fixes the problem, which is how the team will know, months later, whether the prime did what it was asked.

Note: The thought process: why not just fix their model?

Every gap on that list would take Bob a minute to "fix" in the government's v2 model: add a satisfy for TW-SYS-012, copy TW-SYS-016 in from DOORS, write the missing verification case for TW-SYS-011. Jim said no to all of them, and the retrospective agreed (see below). The requirements and their satisfaction are the prime's contractual responsibility. If the government quietly patches its copy, the prime's model stays wrong, the next drop reintroduces every gap, and the government has taken on design responsibility nobody funded. The discrepancy report puts the fix where it belongs. The trace job will show, drop by drop, which reports the prime has closed.

The scorecard shows 5 discrepancy reports opened in the sprint and none closed yet; the first closure comes with drop 2 in January.

29.3.6 The weekly status, 20 November

docs/tidewater/sprints/04/quad-2026-11-20.txt

```
Weekly status Model -- as of 2026-11-20 (sprint 4)
Sprint progress 24% (4/17 SP) new   On-time delivery: 2/2 this week, 2/2 this sprint

TECHNICAL PRIORITIES
  OPEN  TW-67        5  "Close the Stations level tags Bob
  DONE  TW-70        2  "Discrepancy reports DR-1 to D Jim
  OPEN  TW-68        2  "Units and value types"        Bob
  DONE  TW-69        2  "Break the Network <-> Operati Tom
  OPEN  TW-71        3  "The model site on GitLab Page Tom
  OPEN  TW-66      1.5  "Learn: reviewing a model chan Bob
  OPEN  TW-72      1.5  "First sprint review with the  Jim

WATCH ITEMS / PM HELP NEEDED
  none
```

The week-1 sprint progress in that quad looks alarming next to sprint 3's finish, and it is not. Points count when a *story* is done, and the large Stations story was still open on Friday even

though its main merge request had merged. Read the quad's "improving" and "degrading" arrows across weeks, not one week's percentage.

29.4 Week 2

29.4.1 Monday 23 November: the Stations package in numbers

Bob's stand-up on Monday:

docs/tidewater/sprints/04/standups/2026-11-23-chat.txt

```
[9:02 AM] Bob
Y TW-67 Close the Stations level tags: 25 TODOs closed, 25 left. T TW-67 walk the Stations tree
with the prime's modeler;, by Fri. B none
```

One merge request closed a large share of the remaining TODO lines. That is the shape of a port: long stretches of small, careful changes, and a few large mechanical ones once the pattern is proven on a small package. Network in sprint 3 was the proof; Stations in sprint 4 was the payoff.

The story was not finished, though. Its second task was to walk the Stations part tree with the prime's modeler and record what they confirmed: which parts are procured, which supplier and part-number tags the prime's stereotypes carry, and whether the tree matches what the prime is actually building. That needed a meeting with someone outside the team, and Bob's stand-up already names it as today's work.

29.4.2 Tuesday 24 November: the model site

Tom's !14 added the `pages` job to the pipeline. On every merge to `develop`, and every night on a schedule, the job builds a small web site from the model and publishes it with GitLab Pages: the generated documentation (the part tree, the function tree, the IDEF0 plates, the trace report), the diagrams the kit draws, and the TODO burn-down. Nothing on the site is written by hand. If the model changes, the site changes with it, the same day.

```
perl "$KIT/bin/gitlab.pl" --local gitlab --project tidewater/model pages | head -1
```

The site is small on purpose. Each page answers a question someone outside the team used to ask the team, usually the week before a review:

Table 29.2: The model site in sprint 4, page by page

Page	The question it answers	Who asks it
The part tree (`docs/`, the generated Nouns page and the tree view)	What is the system made of, and at what level?	the program manager, new engineers, the prime
The function tree and the IDEF0 plates	What does the system do, and what does each function take and produce?	the test organization, the operations community

Page	The question it answers	Who asks it
The trace report	Which requirements have a design and a verification, and which do not?	the program office, the independent reviewers
The diagram views	What connects to what?	everyone, in a review
The TODO burn-down	How much of the port is left?	the program manager, every Friday

None of the pages is a new kind of information. Every one of them used to be a slide, a table in a document, or an answer to an e-mail, prepared by hand from the model. What is new is that they are never out of date and nobody has to prepare them.

The nightly rebuild is a pipeline schedule, the first the team created:

```
perl "$KIT/bin/gitlab.pl" --local gitlab --project tidewater/model schedule list | head -1
```

It runs at 02:00 Eastern on weekdays, against `develop`, with every check switched on. If anything broke during the day, it is red before the stand-up. Chapter Chapter 17, The DevSecOps tools that earn their keep (p. 255) explains why the site was the single most valuable tool the team adopted, for an audience the team had never served directly: everyone else.

The same Tuesday, Jim prepared the sprint review with the program office (task #82), and on Wednesday Jim met the prime's modeler about how future drops should be delivered (task #83). That conversation, about stable element ids, change notes, and eventually letting the prime's Jenkins deliver straight into GitLab, is the subject of Chapter 45, The prime's model changes: receive, assess, disposition (p. 619).

29.4.3 Wednesday 25 November: refinement, and an outlier again

Refinement for sprint 5, with planning poker. One round split:

docs/tidewater/sprints/04/poker-2026-11-25.txt

```
ESTIMATE TW-90 Procured parts as metadata; DataCenter level tags  (3 answers at 10:03)
  Bob            3
  Jim            3
  Tom            5
  no consensus: median 3, mode 3, range 3-5

ESTIMATE TW-90 Procured parts as metadata; DataCenter level tags  (3 answers at 10:05)
  Bob            3.5
  Jim            3.5
  Tom            3.5
  consensus: 3.5
```

Tom voted 5 for "Procured parts as metadata; DataCenter level tags"; Jim and Bob voted 3. This time the high vote knew what the team had just learned the hard way: procured parts carry supplier and part-number tags that must become metadata with values, not just a level tag, and

the Stations walk-through had not even happened yet. The second round settled at 3.5, which is the sprint 4 lesson written as an estimate.

The demo script for the review was drafted with `talk.pl review` (Chapter 25, Talking about progress: helps for engineers who would rather not (p. 390)) and rehearsed out loud before the meeting; the rehearsal was Tom's task on Wednesday 25 November, Day 9, and the script itself is in the evidence (`docs/tidewater/sprints/04/review-script.md`).

29.4.4 Thursday 26 November: Thanksgiving

29.4.5 Friday 27 November: units, a carryover, and the review

Bob merged !15 in the morning: the value types Power and Energy now specialize the ISQ quantity kinds from the SysML v2 standard library, so a power is a power and not just a number:

```
git show ':/^Units for Power and Energy' | grep '^+.*attribute def'
```

The three TODO lines for the Cameo unit instances went with it.

Then the carryover. The Stations walk-through needed the prime's modeler, and it could not start until the last day of the sprint, so it did not fit. Bob closed the task and continued it in sprint 5, with its estimate, and the journal moved the story's remaining points into the next sprint:

```
perl "$KIT/bin/gitlab.pl" --local gitlab --project tidewater/model issue show 77 | tail -1
```

Delivery for the sprint: 12 of 17 story points, or 71%. That is the first sprint the team did not finish everything it committed, and it is handled the way the operating model says: the story carries, visibly, with its remaining points, and the quad shows it as a watch item (Chapter 13, The operating model (p. 196)).

At 10:00, the review:

docs/tidewater/sprints/04/review.txt

```
Sprint 4 review, 2026-11-27 10:00

Goal: Stations levels and Types closed, one cycle broken, gaps reported; the model site for
management

Demonstrated:
  TW-66 Learn: reviewing a model change; the rehearsed demo
  TW-68 Units and value types
  TW-69 Break the Network <-> Operations cycle
  TW-70 Discrepancy reports DR-1 to DR-4 to the prime
  TW-71 The model site on GitLab Pages, rebuilt nightly
  TW-72 First sprint review with the program office; the delivery interface with the prime

Carried to sprint 5:
  TW-67 Close the Stations level tags

Artifacts shown: the merge requests and pipelines of the sprint, the model site (GitLab Pages).
```

```
Feedback (Program manager): "I can open this on my phone?" The rehearsed demo led with the
question, showed the model site, one before-and-after (the cycle: two lines removed, a week of
diagram reading before), one live change, and the ask.

The four headline numbers: TODO lines left 22; verify coverage 70%; open DRs 5; blocked items 0.
```

The review notes record how the rehearsed demo ran: it led with the question, showed the model site, gave one before-and-after, made one live change, and closed with the ask. Those are the six moves of Chapter 25, Talking about progress: helps for engineers who would rather not (p. 390), and here is what each one looks like for a sprint like this one:

1. **The question first.** What did this sprint settle about the model, in the audience's words: is the structure right, and can they see it without the team? Not "here is what we built".
2. **The output, not the tool.** The model site, not the terminal. The part tree, the trace report and the burn-down, as pages anyone can read.
3. **One before-and-after.** The notes name the one used: the cycle, two lines removed, against a week of reading a diagram in the Cameo years.
4. **One safe live change.** The script's suggestion: on a scratch branch, delete one satisfy line and open a draft merge request; a couple of minutes later the trace report names the requirement that lost its design. Then close the merge request.
5. **What it means for the next milestone.** How far the structure TODOs have to go, and when the interim baseline for the program review is due.
6. **The ask.** One decision or one kind of help, with a date.

The program manager's reaction is in the review notes, and it is the moment the team remembers:

> *I can open this on my phone? I have never seen the model without a briefing.*
>
> — Program manager, sprint 4 review

The same four headline numbers as every review: 22 TODO lines left; verify coverage unchanged at the level the trace found in sprint 3; 5 discrepancy reports open with the prime; nothing blocked.

Warning: Verify coverage did not move, and that is correct

If management asks why the share of requirements with a verification stayed flat while everything else improved, the answer is the discrepancy reports. Every remaining verification gap is in the prime's model: TW-SYS-011, the stakeholder needs, and the requirement that is not in the model at all. The government team will not paper over them with its own verification cases. Coverage moves when the prime's drops close the DRs, and the trace job will show exactly when. A flat line here is a measure working honestly, not a team failing.

29.4.6 The retrospective and the pulse

docs/tidewater/sprints/04/retro.txt

```
Sprint 4 retrospective, 2026-11-27 11:00

Start:
  - Showing the model site at every review
Stop:
  - Estimating procured parts as if they were simple
Continue:
  - Discrepancy reports instead of fixing the prime's model
  - The rehearsed demo: it landed

Pulse survey (1-5, anonymous): mean 3
```

Table 29.3: Sprint 4 pulse survey (mean 3)

Question	Jim	Bob	Tom
I understand why we are working this way.	4	3	3
I can do my work in the new tools without help.	3	3	3
The checks catch problems I would have missed.	3	3	3
I can see our progress.	3	3	3
I would choose to keep working this way.	3	2	3

Compare Bob's column with Section 28.4.5, Friday 13 November: review and retrospective (p. 453)'s. "The checks catch problems I would have missed" moved after the WaveSensor finding, which was not Bob's mistake and which Bob would not have found by reading. "I can see our progress" moved after the model site went up. The last question, whether Bob would choose to keep working this way, did not move yet. The team did not push. That answer changes later (Chapter 38, How the team changed its mind (p. 557)).

29.4.7 The weekly status, 27 November

docs/tidewater/sprints/04/quad-2026-11-27.txt

```
Weekly status Model -- as of 2026-11-27 (sprint 4)
Sprint progress 71% (12/17 SP) improving (was 24%)   On-time delivery: 4/4 this week, 6/6 this
sprint

TECHNICAL PRIORITIES
  OPEN  TW-67        2  "Close the Stations level tags Bob  [CARRY]
  DONE  TW-70        2  "Discrepancy reports DR-1 to D Jim
  DONE  TW-68        2  "Units and value types"        Bob
  DONE  TW-69        2  "Break the Network <-> Operati Tom
  DONE  TW-71        3  "The model site on GitLab Page Tom
  DONE  TW-66      1.5  "Learn: reviewing a model chan Bob
  DONE  TW-72      1.5  "First sprint review with the  Jim

WATCH ITEMS / PM HELP NEEDED
  (WI) TW-67      carried to next sprint

SCHEDULE MILESTONES
  30 days out
    1. T2 Port Tidewater v1 to SysML v2 > E6 Structure: close the structure TODOs =  82%  ETA
    2026-12-13
```

```
  60 days out
    -
  90 days out
    -
```

The carried story is the only watch item. The schedule milestone line says the structure epic is most of the way done, with a projected end in mid-December; Chapter 30, Sprint 5: Grading the requirements, and a trace that defends itself (p. 475) shows how close that came.

29.5 What the numbers say

```
perl "$KIT/bin/gitlab.pl" --local gitlab --project tidewater/model analytics --since 2026-11-16
--until 2026-11-27 | sed -n '/2. DORA/,/job failures/p'
```

Two numbers in that output are new this sprint. **Deployment frequency** is no longer zero: every publish of the model site counts as a deployment, so management now receives a fresh model several times a week without anyone preparing it. And the only job that failed all sprint was **outline**: the WaveSensor finding, on the branch and on the merge request. Grammar and lint never failed. The team had stopped making mechanical mistakes; the checks were now finding the prime's.

Table 29.4: Sprint 4 against sprint 3

Measure	End of sprint 3	End of sprint 4
TODO lines left	50	22
Package dependency cycles	2	1
Discrepancy reports opened (sprint)	0	5
Median hours, opening a change to first review	4.1	3.3
Median hours, first edit to accepted change	4.2	3.5
Changes gated by the pipeline	4 of 4	4 of 4
Stand-up lines with an outcome and a number	1	1
Pulse (1 to 5)	2.2	3

The document-era comparison the team kept beside these numbers: a requirement change through the configuration control board took 23 working days, and a model review was screenshots in a briefing that took 3 weeks to prepare (both notional, from the program's last document-era review cycle). Chapter Chapter 33, The twelve-week scorecard (p. 520) puts the whole twelve weeks side by side.

29.6 Tools adopted this sprint, and the pain each one removed

Table 29.5: Tools adopted in sprint 4

Tool	The pain it removed	Where to read more
The model site (GitLab Pages), rebuilt on every merge	Nobody outside the team could see the model without a briefing built by hand	Chapter 17, The DevSecOps tools that earn their keep (p. 255)
The nightly pipeline schedule	A problem merged late in the day was found the next week, by whoever tripped over it	Chapter 19, CI/CD from zero (p. 288)
Discrepancy reports as GitLab issues from a template	Gaps in the prime's model were found in reviews and forgotten, with no record of what was asked or answered	Chapter 45, The prime's model changes: receive, assess, disposition (p. 619)
The review checklist	Reviews looked for typos the pipeline already catches and missed whether the change was right	Chapter 22, Reviewing a model change (p. 357)
The rehearsed demo	The sprint 2 demo lost the room	Chapter 25, Talking about progress: helps for engineers who would rather not (p. 390)

29.7 What Jim told management

Note: Sprint 4, the Friday note

The model's biggest package, Stations, is now tagged and checked against the part-tree rules; the check found one part the prime's model puts at the wrong level, and we have asked the prime to correct it (DR-7). We removed one of the two dependency cycles the prime's model came with. We sent the prime 5 discrepancy reports for the gaps the trace found. 22 TODO lines remain. The model is now published as a web site, rebuilt every night: [the link]. One story carried into sprint 5 (the walk-through with the prime's modeler). We need the program office's decision on the delivery interface with the prime by the end of sprint 5.

29.8 You can now

- Break a package dependency cycle by finding the one dependency that creates it, and decide whether it is yours to break.
- Read an outline and level-tag finding, reproduce it on your own checkout, and decide between fixing your copy, waiting for the prime, and doing both.
- Write a discrepancy report that says what was found, how, and what you ask the prime to do, and keep it open until a drop closes it.
- Explain why verify coverage can stay flat while the model improves.
- Set up a model site and a nightly schedule, and say what each one is for.
- Run a demo with the six moves, and carry a story honestly when it does not fit.

Exercise 29.1: Find a cycle

In the Tidewater sandbox, draw the package view with `perl "$KIT/tools/sysml/model.pl" draw pkg -o pkg.svg` in Bob's clone and find the cycle that remains. Name the dependency you would remove to break it, and say why that is a design decision, not a cleanup (Chapter 30, Sprint 5: Grading the requirements, and a trace that defends itself (p. 475) shows what the team decided).

Exercise 29.2: Write a discrepancy report

Pick a gap from the trace report in Chapter 28, Sprint 3: the first red X, DOORS in the model, and a wait nobody named (p. 440) that does not already have a discrepancy report (hint: look at the unverified stakeholder needs). Write the discrepancy report from the team's template, in Vim, and open it as an issue on a practice sandbox with `gitlab.pl issue new --template "Discrepancy Report"`.

Exercise 29.3: Rehearse a demo

Take the sprint 4 review script in `docs/tidewater/sprints/04/review-script.md`. Rewrite it so it opens with the question, shows the model site first, has one before-and-after with a number, one safe live change, and one ask with a date. Read it aloud against a five-minute timer.

29.9 Applying this to your project

- **Publish the model where management can open it,** generated from the model on every merge, as early as you can. It changes the conversation more than any metric.
- **Run a nightly full check** so problems are red before the stand-up, not found next week.
- **Send the prime's gaps back as discrepancy reports,** one per gap, with a closure section. Do not patch the prime's model quietly in your copy; when you must diverge, write the DR number on the line.
- **Rehearse the demo,** with the question first and the ask last. Rotate the driver.
- **Expect your first carryover** when a task depends on someone outside the team. Carry it visibly with its remaining points; the quad and the journal are built for it.

CHAPTER

30

Sprint 5: Grading the requirements, and a trace that defends itself

From what you know →

You know

requirement review comments, and the trace matrix you rebuild before each review.

→ Here it is called

the recipe check and the trace gate.

→ What is new

the trace defends itself: a change that breaks it cannot merge.

Survival kit

What you have

a model whose structure is nearly closed, a pipeline that checks every change, a model site the program office has seen once, and a team that has started to trust the checks (Chapter 29, Sprint 4: the model site, a cycle broken, and the demo that landed (p. 459)).

What you need from this chapter: how the team turned two questions into machinery – *are the requirements we inherited good enough to verify?* and *can we stop a change that quietly breaks the trace?* – and how a mistake that used to survive until the next milestone review was caught in four minutes on a Friday morning.

30.1 The question this sprint answered

Every sprint chapter in this part starts with the systems-engineering question the team answered, because that is how the team learned to start every sprint. Sprint 5's question was one Jim kept coming back to after the sprint 4 review, where the program manager had opened the model site on a phone and said they had never seen the model without a briefing: *if the model is checked, what is it checked against? If the requirements are weak, a checked model is a well-organized mistake.*

The question was fair, and it split into two questions the team could work on:

1. **Are the requirements good enough to be verified?** Not "are they right" – that is the stakeholders' call – but are they written so that someone could design to them and test them? That is what the requirement recipe (Chapter 7, Requirements a pipeline can check: the recipe (p. 73)) measures.
2. **Can the model defend its own trace?** By now the pipeline *reported* trace gaps on every merge request: requirements nothing satisfies, requirements nothing verifies, text that differs from DOORS. Reporting is not the same as preventing. Nothing stopped an engineer from deleting a `satisfy` line by accident.

The mood had turned. The pulse survey at the sprint 4 retrospective came in at 3 out of 5, up from the sprint 3 dip; Bob, who had asked in week one why they could not just send the Word document, now opened merge requests without being asked. The team was not yet enthusiastic. It was, in Jim's words at the kickoff, "past the point of wanting to go back."

30.2 Kickoff: a carried story, a small learning budget, mostly engineering

The kickoff ran on Monday 30 November, 09:00 to 10:30, and stood in for that day's stand-up. The record the kit wrote:

docs/tidewater/sprints/05/kickoff.txt

```
Sprint 5 kickoff, 2026-11-30 09:00 (90 minutes)

Goal: Procured parts and DataCenter closed; Legacy Import untangled; trace becomes a gate
Capacity: 21.0 points (Jim 5, Bob 8, Tom 8 per full sprint, less holidays and leave)
Committed: 18.5 points in 8 stories (88% of capacity): engineering 12.5, platform 5.0 (27%),
learning 1.0 (5%)

TW-67    E6    2.0 pts  engineering  Close the Stations level tags
           #76   3.0 pt  bob  2026-11-16 .. 2026-11-20  Level tags in Stations
           #96   2.0 pt  bob  2026-11-30 .. 2026-12-02  Walk the Stations tree with the prime's
           modeler; record what they confirmed
TW-89    E3    1.0 pts  learning     Learn: the requirement recipe; writing memos from the model
           #97   0.5 pt  jim  2026-11-30 .. 2026-11-30  The recipe chapter; a memo from pipeline
           numbers
           #98   0.5 pt  bob  2026-12-02 .. 2026-12-03  The recipe chapter
TW-90    E6    3.5 pts  engineering  Procured parts as metadata; DataCenter level tags
           #99   2.0 pt  bob  2026-12-03 .. 2026-12-07  Procured parts as metadata
           #100  1.5 pt  bob  2026-12-07 .. 2026-12-09  Level tags in DataCenter
TW-91    E7    2.0 pts  engineering  Prune ObservingStation's trace spaghetti
           #101  2.0 pt  bob  2026-12-09 .. 2026-12-11  Remove the duplicate trace links
TW-92    E6    3.0 pts  engineering  Untangle the Legacy Import package
           #102  2.0 pt  tom  2026-11-30 .. 2026-12-02  Remove the Legacy Import package from the
           v2 model
           #103  1.0 pt  jim  2026-12-01 .. 2026-12-02  DR-5: ask the prime to delete Legacy
           Import
TW-93    E2    2.0 pts  platform     The trace job becomes a gate
           #104  2.0 pt  tom  2026-12-02 .. 2026-12-04  Trace ratchet: fail only on new gaps
TW-94    E7    2.0 pts  engineering  Grade our requirements with the recipe
           #105  1.0 pt  jim  2026-12-03 .. 2026-12-04  Read the recipe findings for the DOORS
           modules
```

```
           #106  1.0 pt  jim  2026-12-07 .. 2026-12-08  DR-6: weak verbs and missing parts in the
           DOORS text
TW-95    E1    3.0 pts  platform     Release rehearsal and a spillage tabletop
           #107  2.0 pt  tom  2026-12-07 .. 2026-12-09  Rehearse release and hotfix in
           tidewater/sandbox
           #108  1.0 pt  tom  2026-12-09 .. 2026-12-10  Spillage tabletop: a marked file pushed
           by mistake
```

Three things in that record are worth reading slowly.

The first story is a carryover. TW-67, *Close the Stations level tags*, was committed in sprint 4. Its first task (#76, the level tags themselves) finished in sprint 4; its second task – walking the Stations part tree with the prime's modeler and recording what they confirmed – did not happen in sprint 4. The story carried into sprint 5 with its remaining points. Nobody hid it: the sprint 4 review said so, the journal recorded a `carry`, and the weekly quad marked the story late. A carried story is not a failure. A carried story nobody mentions is.

The learning budget is small now. Learning was 24% of sprint 1's task points done (Section 13.8, Two ways to count "done" (p. 205)). In sprint 5 it is 6%: one story, TW-89, in which Jim and Bob read the recipe chapter of this book and Jim drafted a memo from pipeline numbers. The budget fell because the team needed it less, not because the plan squeezed it out. That distinction matters when you explain the curve to management (Bottom line up front (p. iii)).

Engineering dominates. 64% of the task points done are engineering work – requirements, structure, trace – and 30% is platform work (the trace gate and the release rehearsal). In sprint 2 those shares were almost reversed.

The goal Jim wrote on the board was three clauses long: *Procured parts and DataCenter closed; Legacy Import untangled; trace becomes a gate.* Two of the three are engineering. The third is the one this chapter is mostly about.

30.3 Week 1: clutter out, a ratchet in

30.3.1 Monday 30 November: the recipe, read before it is run

Jim spent the morning on task #97, half a point from the learning story: read Chapter 7, Requirements a pipeline can check: the recipe (p. 73) and draft a memo from the pipeline's numbers. The order was deliberate. The team had learned in sprint 2 that running a new checker before anyone understands what it checks produces a list of findings nobody trusts. Jim wanted to be able to explain every finding to the prime before the prime saw a single one.

30.3.2 Tuesday and Wednesday: the Legacy Import package

The drop-1 model carries a package called *Legacy Import*: the residue of the document port years ago. It holds a duplicated block (`Battery_old`), a copy of a subsystem (`Copy of SensorSuite`), an unused placeholder (`TBD Component`) and a "requirement" that is really a slide title. Chapter 2, The Cameo years (p. 10) measured it as part of the spider web.

Tom's task (#102) was to remove it from the government's v2 model. Jim's task (#103) was to ask the prime to remove it from the v1 model, as discrepancy report DR-5.

The thought process behind doing both is worth spelling out, because it is the pattern for every piece of clutter the team inherits:

- **Deleting it in v2 alone is not enough.** The Legacy Import file is converter output. It lives on the vendor branch (Chapter 20, The pipeline, job by job (p. 302)), and every future prime drop will bring it back. Git will see the government's deletion on `develop` and the prime's unchanged copy on `vendor/prime`, and every drop merge will raise a delete/modify question until the prime deletes it upstream.
- **Waiting for the prime alone is not enough either.** The clutter inflates the spider-web measures, confuses every new reader, and the prime's next drop is weeks away.
- **So: delete it now, and file the DR.** The deletion makes the government's model clean today. The DR makes the problem the prime's to fix, with an id both sides can cite. When the prime's drop finally removes the package, the conflict disappears on its own.

Tom's merge request, !16, removed the package. Its pipeline went green on the first run. Jim sent DR-5 the same morning:

GitLab Markdown: docs/tidewater/sprints/05/samples/discrepancy-report-DR-5.md

```
## Discrepancy

| | |
|---|---|
| DR number | DR-5 |
| Prime tracking | (to be assigned by the prime) |
| Found in drop | v1.0, delivered 2026-10-02 |
| DOORS id | - |
| Severity | 2 |

## What we found

Battery_old, Copy of SensorSuite, TBD Component and "Requirement from slide 14" are unused or
duplicates.

## What we ask the prime to do

Correct it in the v1 model and say so in the next drop's MANIFEST.

## Closure

- [x] Sent to the prime on 2026-12-02 by @jim
- [ ] Fixed in drop ...
```

Notice what the DR does not say. It does not say *you made a mess*. It names the elements, says why they are a problem, and asks for one concrete action. A discrepancy report is an engineering request between two teams that will work together for years; write it the way you would want to receive it.

You can see the deletion in the team's history. In the Tidewater sandbox, from Tom's clone:

```
git checkout -q develop
git log --oneline -3 -- model/legacy
```

The second line of the output is the drop-1 import that created the file; the first is Tom's removal. That is the whole history of the package in the government's model: arrived with the prime's drop, left by a reviewed merge request. Compare that with the Cameo years, when a package like this could be deleted from a shared model by anyone with a license and nobody would know who or why.

30.3.3 Wednesday: Bob and the prime's modeler

Bob closed the carried task the same day: a walk through the Stations part tree with the prime's modeler, on a shared screen, with the generated part tree from the model site (Chapter 11, Seeing the model: diagrams from text (p. 158)) instead of a Cameo diagram. It also gave both sides a second look at DR-7, opened in sprint 4: `WaveSensor` is tagged as a level-5 assembly in the v1 model while being a direct part of the level-4 `MooredBuoy`. DR-7 is still open at the end of the sprint – correctly, because only the prime can say which of the two is wrong.

The walk-through is the first time the team used the model site *with the prime*. It will not be the last. A generated, read-only view that both sides can look at removes the oldest argument in a model review: "that's not what my diagram shows."

30.3.4 Thursday: stand-ups that say something

By sprint 5 the stand-up statuses had changed character. Here is Thursday's, pasted from the meeting chat as the kit stores it:

docs/tidewater/sprints/05/standups/2026-12-03-chat.txt

```
Meeting started
[9:00 AM] Jim
Morning. Y/T/B in chat, please.
[9:01 AM] Jim
Y TW-92 Untangle the Legacy Import package: done (2 tasks). Reviewed !16. T TW-94 read the recipe
findings for the DOORS modules, by Dec 11; pairing with Bob after lunch. B none
[9:02 AM] Bob
Y TW-67 Close the Stations level tags: done (3 tasks). T TW-89 (first thing: read the review
comments) the recipe chapter, by Dec 11. B none
[9:03 AM] Tom
Y TW-92 Untangle the Legacy Import package: done (2 tasks). T TW-93 trace ratchet: fail only on
new gaps, by Dec 11. B none
```

and the kit's lint of it:

docs/tidewater/sprints/05/standups/2026-12-03-lint.txt

```
standups/2026-12-03-Model-chat.txt:5: ok    Jim: Y/T/B parsed
standups/2026-12-03-Model-chat.txt:7: ok    Bob: Y/T/B parsed
standups/2026-12-03-Model-chat.txt:9: ok    Tom: Y/T/B parsed
# 3 answered, 0 errors, 0 warnings
```

Compare that with sprint 1 (Chapter 26, Sprint 1: A GitLab project the team can work in (p. 404)), when the lint flagged missing story ids and statuses that described keystrokes ("rebased my

branch"). Every line now names a story, says what is *true* (a story done, with its tasks counted), and names a date for what is next. The share of stand-up lines with an outcome and a number – one of the twelve-week measures from Chapter 3, The directive (p. 22) – was 1 for the sprint, as a fraction of lines.

Each line was drafted by `talk.pl standup` (Chapter 25, Talking about progress: helps for engineers who would rather not (p. 390)) from the previous day's GitLab activity, and each person edited or posted the draft. The evidence keeps both:

docs/tidewater/sprints/05/standups/2026-12-03-talk-drafts.txt

```
jim draft:  Y TW-92 Untangle the Legacy Import package: done (2 tasks). Reviewed !16. T TW-94
read the recipe findings for the DOORS modules, by Dec 11. B none
jim posted: Y TW-92 Untangle the Legacy Import package: done (2 tasks). Reviewed !16. T TW-94
read the recipe findings for the DOORS modules, by Dec 11; pairing with Bob after lunch. B none
bob draft:  Y TW-67 Close the Stations level tags: done (3 tasks). T TW-89 the recipe chapter, by
Dec 11. B none
bob posted: Y TW-67 Close the Stations level tags: done (3 tasks). T TW-89 (first thing: read the
review comments) the recipe chapter, by Dec 11. B none
tom draft:  Y TW-92 Untangle the Legacy Import package: done (2 tasks). T TW-93 trace ratchet:
fail only on new gaps, by Dec 11. B none
tom posted: Y TW-92 Untangle the Legacy Import package: done (2 tasks). T TW-93 trace ratchet:
fail only on new gaps, by Dec 11. B none
```

The drafts and the posted lines are identical, every day of the sprint. That can mean two things. The good reading: the drafts were already in the form the team wanted, so there was nothing to fix. The worrying reading: people had stopped reading their own statuses. You cannot tell which from the record, and that is the point. The rule worth adopting is simple – read the draft before you post it; if it would not make sense to the deputy PM, change it. A drafting tool is a starting point, not a substitute for knowing what you did yesterday.

30.3.5 Friday 4 December: the trace becomes a ratchet

One job in the pipeline changed this week, in Tom's story TW-93. Until then, the `trace` job reported gaps and was allowed to fail: red in the job list, green for the merge. The reason was sound. The model inherited gaps that only the prime can close – requirements the v1 model never satisfied, never verified, or worded differently from DOORS – and a gate that failed on every one of them would have blocked every merge request the team ever opened.

The ratchet keeps that reason and adds a rule: **a merge request may not make the trace worse.** The job runs the trace twice – once on the merge result, once on the target branch – and compares the *sets* of orphan, unverified and drifted requirement ids. A gap that is already on `develop` is old news and is reported. A gap that this change adds fails the job, names the requirement, and blocks the merge.

!17 put the ratchet in. Its own pipeline was green. The interesting part came a week later.

> **Case study: The decision: gate, ratchet, or report**
>
> The team considered three designs before Tom wrote a line, and the reasoning is the part to copy.

> **Make the trace a hard gate.** Every orphan, every unverified requirement, every drifted text fails the merge request. Clean, simple, and unusable: the inherited gaps would fail every merge request until the prime fixed its model, which could take months. Within a week someone would set the job to `allow_failure`, and the gate would be a decoration.
>
> **Keep reporting.** Leave the job allowed to fail and trust people to read it. That is what the team had been doing since sprint 3 (Chapter 28, Sprint 3: the first red X, DOORS in the model, and a wait nobody named (p. 440)). It works until the day someone is in a hurry, and a reporting job that is always red trains everyone to ignore red.
>
> **Ratchet.** Fail only on what this change makes worse. The inherited gaps stay visible in every report and on the model site; a new gap blocks the merge and names the requirement. The cost is complexity in the job – it has to check out the target branch and run the trace twice – and one subtle requirement: the comparison must be between *sets of requirement ids*, not counts. A change that fixes one orphan and creates another leaves the count the same and the model worse; a ratchet that compared counts would let it through. Test your ratchet with exactly that case.
>
> The same pattern – report everything, block only what gets worse – is how the team handles every inherited problem: outline violations, complexity, recipe findings. It is the single most useful idea in this sprint.

Jim closed #105 the same Friday, having read the recipe findings for the DOORS modules and decided which ones to send to the prime. That is the next section.

The week ended with the first of two weekly quads for management (Chapter 36, Telling management (p. 538)):

docs/tidewater/sprints/05/quad-2026-12-04.txt

```
Weekly status Model -- as of 2026-12-04 (sprint 5)
Sprint progress 43% (8/18 SP) new   On-time delivery: 3/4 this week, 3/4 this sprint  ! under 80%

TECHNICAL PRIORITIES
  DONE  TW-67        2  "Close the Stations level tags Bob
  OPEN  TW-91        2  "Prune ObservingStation's trac Bob
  DONE  TW-92        3  "Untangle the Legacy Import pa Tom
  DONE  TW-93        2  "The trace job becomes a gate" Tom
  OPEN  TW-94        2  "Grade our requirements with t Jim
  OPEN  TW-95        3  "Release rehearsal and a spill Tom
  DONE  TW-89        1  "Learn: the requirement recipe Jim
  OPEN  TW-90      3.5  "Procured parts as metadata    Bob

WATCH ITEMS / PM HELP NEEDED
  none

SCHEDULE MILESTONES
  30 days out
    1. T2 Port Tidewater v1 to SysML v2 > E7 Requirements and trace ->  64%  ETA 2026-12-27
    2. (none) > (none)                   ->  73%  ETA 2026-12-27
    3. T1 Stand up MBSE DevSecOps > E1 GitLab project and gitflow ->  71%  ETA 2026-12-27
  60 days out
    -
  90 days out
    -
```

```
ACCOMPLISHMENTS (since 2026-11-27)
  x TW-67        2  "Close the Stations level tags Bob  (carried over)
  v TW-92        3  "Untangle the Legacy Import pa Tom
  v TW-93        2  "The trace job becomes a gate" Tom
  v TW-89        1  "Learn: the requirement recipe Jim

PUNT RATE (tasks punted back to TODO / tasks committed, last 4 sprints)
  sprint 2    Model          0 / 7     0%
  sprint 3    Model          0 / 6     0%
  sprint 4    Model          0 / 7     0%
  sprint 5    Model          0 / 8     0%

Legend: v on time  x late (carried over, or rework after the demo)  ! slipped   -> pushed right
<- pulled left  = no change  + new
```

30.4 The recipe: grading the requirements

The story *Grade our requirements with the recipe* (TW-94) is the engineering heart of the sprint. The recipe (Chapter 7, Requirements a pipeline can check: the recipe (p. 73)) describes a requirement as a single sentence with up to eighteen parts, six of them required, and lists the target verbs that are rarely verifiable ("provide", "have", "be", "be able to"). `req-recipe.pl` reads each requirement, decomposes it into those parts, and reports what is missing or weak.

Jim ran it on the three DOORS modules – the stakeholder needs, the system requirements and the segment requirements – in the government's model, where the DOORS text lands unchanged. You can run the same command in the Tidewater sandbox:

```
perl "$KIT/tools/sysml/req-recipe.pl" model/requirements/stakeholder model/requirements/system
model/requirements/segment | tail -1
```

The summary line says the DOORS text holds 30 graded requirements and that none of them is missing a required part. The findings are about *quality*, not completeness. Here is the first one in the full report the pipeline kept:

docs/tidewater/sprints/05/recipe-doors.txt

```
TW-STK-001  score 85  model/requirements/stakeholder/Stakeholder_Needs.sysml:3  (Coastal
coverage)
  The network shall provide marine weather observations for the coastal region it covers.
    Specification Target        network
    Operative Verb              shall
    Target Verb                 provide
    Precise Object(s)           marine weather observations
    Modifier(s)                 for
    Modification(s)             the coastal region it covers
    Requirement ID              TW-STK-001
    major  weak target verb 'provide' (the recipe): rarely verifiable -- name the concrete action
    (send, store, measure, report ...)
```

The network "shall provide marine weather observations." *Provide* how? To whom, how often, within what? A test engineer cannot write a pass criterion for *provide*. That is not pedantry: it is the difference between a requirement that can be verified at acceptance and one that will be argued about.

30.4.1 Why a discrepancy report, and not an edit

The obvious next step would have been to fix the wording in the model. Jim did not, and the reason is ADR 0008 (*DOORS stays authoritative for requirement text*), recorded in sprint 2 (Chapter 27, Sprint 2: Drop 1 on develop, every change gated (p. 424)). The DOORS text is the prime's, under the contract. If the government rewrites it in the v2 model, the model and DOORS disagree – which is exactly the *drift* the trace job exists to catch – and the next drop will undo the edit anyway.

So the findings went to the prime as DR-6, sent on Tuesday 8 December:

GitLab Markdown: docs/tidewater/sprints/05/samples/discrepancy-report-DR-6.md

```
## Discrepancy

| | |
|---|---|
| DR number | DR-6 |
| Prime tracking | (to be assigned by the prime) |
| Found in drop | v1.0, delivered 2026-10-02 |
| DOORS id | - |
| Severity | 2 |

## What we found

The requirement recipe (tools/sysml/req-recipe.pl) on the DOORS modules: requirements=30
findings=15 missing=0. Examples:

- weak target verb 'provide' (the recipe): rarely verifiable -- name the concrete action (send,
store, measure, report ...)
- weak target verb 'be able to' read: state the action itself ('shall read'), with its threshold
- weak target verb 'be able to' service: state the action itself ('shall service'), with its
threshold
- weak target verb 'be' (the recipe): rarely verifiable -- name the concrete action (send, store,
measure, report ...)

## What we ask the prime to do

Correct it in the v1 model and say so in the next drop's MANIFEST.

## Closure

- [x] Sent to the prime on 2026-12-08 by @jim
- [ ] Fixed in drop ...
```

The DR quotes the tool's summary and four representative findings, and asks for one action. It does not list every finding: the prime can run the same tool on the same text (it is in the kit) and get the same answer. Making the check reproducible on the other side is worth more than a long attachment.

You can open the DR in the sandbox's GitLab, as Jim did:

```
perl "$KIT/bin/gitlab.pl" --local gitlab --project tidewater/model issue show 110 | head -3
```

Its labels say a lot in a small space: `type::discrepancy`, `owner::prime`, `severity::2`, `area::requirements`, `phase::port`. Anyone in the program can filter the board for *everything the prime owes us* without a spreadsheet.

30.4.2 Why the recipe is not a gate yet

The obvious question once Jim's findings came in was: *should the recipe fail the pipeline?* Not yet, and the reason is ownership. Every finding in the DOORS text is the prime's to fix; a gate on it would block the government's merge requests for the prime's wording. What the team did instead was add the recipe to the pipeline as a *report* – its findings show up in the merge request's code quality widget and as metrics on the model site – so that a merge request which *adds* a requirement in the government's own packages can be held to the recipe in review, and so that the count is visible when the prime's next drop arrives. When the government writes its own requirements, later in the program, a recipe ratchet on its own packages is the natural next step.

30.4.3 The stand-ups that week

Monday 7 December shows the lint doing a small, useful thing:

docs/tidewater/sprints/05/standups/2026-12-07-chat.txt

```
Meeting started
[9:00 AM] Jim
Morning. Y/T/B in chat, please.
[9:01 AM] Jim
Y TW-94 Grade our requirements with the recipe: 1 of 2 tasks done. Reviewed !17. T TW-94 DR-6:
weak verbs and missing parts in the DOORS, by Fri. B none
[9:02 AM] Bob
Y none T TW-90 procured parts as metadata, by Fri. B none
[9:03 AM] Tom
Y TW-93 The trace job becomes a gate: done (1 task). T TW-95 rehearse release and hotfix in
tidewater/sandbox, by Fri; pairing with Jim after lunch. B none
```

docs/tidewater/sprints/05/standups/2026-12-07-lint.txt

```
standups/2026-12-07-Model-chat.txt:5: warn  Jim: DR-6 is not in the journal
standups/2026-12-07-Model-chat.txt:7: ok    Bob: Y/T/B parsed
standups/2026-12-07-Model-chat.txt:9: ok    Tom: Y/T/B parsed
# 3 answered, 0 errors, 1 warning
```

The lint warns that *DR-6 is not in the journal*. It is right: DR-6 is a GitLab issue, not a story in the journal, and the lint matches anything that looks like an id. The warning costs Jim nothing –

warnings do not stop the compile – and it is a reminder of the boundary the kit keeps on purpose (Chapter 23, Stand-ups, the journal and the board (p. 367)): the journal holds the team's stories; discrepancy reports, tasks and everything else live in GitLab. Jim could have written the status around the story id (TW-94) alone; naming the DR is still worth the warning, because *DR-6* is what anyone outside the team would ask about.

Two other things in the week's statuses are worth a second look. Bob's line on Tuesday 1 December says *Y none*: the drafting tool found nothing of Bob's in GitLab from Monday, because Monday was the kickoff and a learning task Bob had not started. *None* is honest. It is also the line most likely to make a manager wonder what Bob did all day. When the truthful answer is "meetings and reading", say so – *Y kickoff; read the recipe chapter* – rather than letting the tool's *none* stand. And several lines end in an ellipsis, where the tool shortened a long story title. Nobody minded. A stand-up line is for the people in the meeting, who know the stories; it does not need to be a complete sentence.

Objection: "This is just more paperwork for the prime."

It is less. Before, requirement quality came up at a milestone review, as red ink on a PDF, months after the text was written, with no way for the prime to check the reviewer's opinion. Now it arrives as a reproducible check with named findings, early enough to fix before anyone designs to the weak text.

30.5 Week 2: metadata, a dry run, a tabletop and a catch

30.5.1 Monday 7 December: procured parts as metadata

Bob's !18 replaced the converter's TODO lines for the v1 `Procured` stereotype with SysML v2 metadata: each procured part now carries `@Procured { supplier = "…"; partNumber = "…"; }` from the government's Conventions package. It is a small change with a long tail. Supply-chain questions – *which parts come from which supplier?* – become a search over the model instead of a call to the prime (Chapter 48, Security: protect the environment, engineer it into the system (p. 651) returns to this).

The design choice behind it is worth a paragraph, because every v1 profile a team inherits raises the same question. The v1 model expressed "procured" as a stereotype from a project profile, with the supplier and part number as tagged values. SysML v2 offers three ways to carry the same information: plain attributes on each part definition, a specialization of a `ProcuredPart` part definition, or metadata. Bob chose metadata, for two reasons. Procurement is a fact *about* the part, not a property *of* the system – the buoy does not behave differently because its modem came from a particular vendor – and metadata is the v2 construct for facts about elements. And metadata is what the kit's checks and views read: the same pattern the level tags use (Chapter 8, SysML v2 by example: a workbook (p. 92)). One rule for all the metadata keeps the model's conventions few enough to remember.

Sprint 4's retrospective had already said *stop estimating procured parts as if they were simple*. Procured parts carry more than they show: every one needs a look before its metadata can be written, because a v1 profile enforces that the stereotype is applied, not that its tagged values are consistent or complete.

30.5.2 Wednesday 9 December: refinement and planning poker

Refinement for sprint 6 took the second half hour of Wednesday morning. The stories had been written during the week; refinement applied the Definition of Ready to each and estimated it with planning poker in the meeting chat (Chapter 24, The rhythm: stand-ups, poker, refinement, kickoff, demo, retro (p. 377)). The record:

docs/tidewater/sprints/05/refinement.txt

```
Refinement, 2026-12-09 10:00: the Definition of Ready applied to sprint 6's stories

TW-111  0.5 pts  Learn: memos and briefings from the model
TW-112  3 pts  BuoyModes state machine by hand
TW-113  1 pts  Fork, pin and object flow in ObserveCoast
TW-114  1.5 pts  Level tags in Operations
TW-115  2 pts  Data exchange table for the program review
TW-116  2 pts  VCRM: which requirements are verified, and how
TW-117  2 pts  Interim baseline-0 for the program review
TW-118  2 pts  Retrospective, the management briefing and the plan for next semester
```

Three of the eight stories took two rounds. The tally the kit printed for one of them shows how the conversation went:

docs/tidewater/sprints/05/poker-2026-12-09.txt

```
ESTIMATE TW-116 VCRM: which requirements are verified, and how  (3 answers at 10:15)
  Bob            3
  Jim            2
  Tom            3
  no consensus: median 3, mode 3, range 2-3

ESTIMATE TW-116 VCRM: which requirements are verified, and how  (3 answers at 10:17)
  Bob            2
  Jim            2
  Tom            2
  consensus: 2
```

In the first round for the VCRM story, Jim voted low and Bob and Tom voted high. Jim asked the question the facilitation prompt in the chat suggests – *what does the high vote know?* The answer was the inspection verdicts: someone has to enter and sign a verdict for every requirement verified by inspection, and both high votes had counted that work. The plan already gave it to Jim, as Jim's own task (#126), because a signed verdict needs the lead's authority. With that settled, the second round agreed. That is what planning poker is for: not the number, but the minute in which the team finds out it was imagining two different stories.

30.5.3 Wednesday 9 December: DataCenter closed; the release rehearsal

Bob's !19 closed the level-tag TODO lines in DataCenter. Tom spent two days on #107: rehearsing the release and hotfix flows in a sandbox project, `tidewater/sandbox`, before doing either for real. The release path is the gitflow path (Chapter 12, Git from zero, for systems engineers (p. 171), Chapter 20, The pipeline, job by job (p. 302)): a `release/` branch from `develop`, a merge request

into `main` with the CCB's concurrence, a tag, and a back-merge into `develop`. The hotfix path starts from `main` and ends in both branches.

Rehearsing in a sandbox is cheap insurance. The first real baseline – baseline-0, three weeks later – went to the program office with the CCB watching. Tom wanted to have made the mistakes already, in a project nobody else could see.

30.5.4 Thursday 10 December: the spillage tabletop

Tom, as the team's part-time security owner, ran a tabletop with the ISSM (#108): *a marked file has been pushed to the model repository by mistake. What do you do?* The steps the team walked through are the ones in Chapter 48, Security: protect the environment, engineer it into the system (p. 651): stop pushing, tell the ISSM before doing anything else, do not "fix" history yourself (a rewritten branch on a server does not remove the data from clones, caches and backups), and let the security office decide the containment.

The issue's record of the tabletop is one line: *Done: Spillage tabletop: a marked file pushed by mistake.* That is not good enough, and it is worth saying plainly because it is the most common way a tabletop is wasted. Its value is in what you learned – who to call, how long the first hour took, what you did not know – and a closed issue with no notes preserves none of it. A tabletop's record should hold three bullets: what we did, what surprised us, what we change.

30.5.5 Friday 11 December: the ratchet catches a real mistake

Bob's last story of the sprint was TW-91, *Prune ObservingStation's trace spaghetti*: the v1 model gives `ObservingStation` more relationship ends than any other block, many of them duplicate `trace` links made during the document port. Bob's task was to remove the duplicates and keep the real ones.

Bob opened !20 at 10:00. Its first pipeline failed. Here is the end of the `trace` job's log, from the evidence:

docs/tidewater/sprints/05/pipelines/mr20-first-71.log

```
trace gaps: 3 orphan(s), 9 unverified, 1 drifted
$ git worktree add -q --detach /tmp/ci-XXXXXX e9a045e31092a6168e07caa1da3048f44f5d2bb3
$ git worktree remove --force /tmp/ci-XXXXXX
target branch: 29 of 30 linked; 2 orphan(s), 9 unverified, 1 drifted (full report:
trace-base.txt)
this change adds trace gaps: orphans 2 -> 3 (new: TW-SYS-002) (see trace.txt)
```

The second-to-last line is the target branch – `develop` – with the gaps it already had. The last line is the verdict: *this change adds trace gaps: orphans 2 -> 3* (*new: TW-SYS-002*). Among the links Bob had judged duplicates was a `satisfy` that was not a duplicate at all: it was the only link from the design to TW-SYS-002, *Scheduled transmission*. Without it, nothing in the model satisfied the requirement.

> *The ratchet stopped me. I deleted a satisfy I thought was a duplicate. It was not.*
>
> — Bob

In the document era this mistake would have survived until someone built a traceability matrix by hand for a milestone review – weeks or months later, by which time nobody would remember which edit removed the link. Here it was caught on the merge request that made it, before review, with the requirement named. Bob restored the line, pushed, and the second pipeline passed:

docs/tidewater/sprints/05/pipelines/mr20-fixed-72.log

```
trace gaps: 2 orphan(s), 9 unverified, 1 drifted
$ git worktree add -q --detach /tmp/ci-XXXXXX e9a045e31092a6168e07caa1da3048f44f5d2bb3
$ git worktree remove --force /tmp/ci-XXXXXX
target branch: 29 of 30 linked; 2 orphan(s), 9 unverified, 1 drifted (full report:
trace-base.txt)
no new trace gaps (target branch: orphans 2, unverified 9, drift 1)
```

You can find both verdicts in the sandbox's pipeline records:

```
grep -h "this change adds\|no new trace" gitlab/pipelines/tidewater-model/71/trace.log
gitlab/pipelines/tidewater-model/72/trace.log
```

Open !20 in the sandbox's GitLab and look at its test report:

```
perl "$KIT/bin/gitlab.pl" --local gitlab --project tidewater/model mr show 20 | head -6
```

Warning: A green merge request with "failed" tests

The pipeline is green, yet the test widget lists the inherited trace gaps as failed tests. They are the old gaps – TW-SYS-012, TW-SYS-016 and the rest – that belong to the prime and that the ratchet deliberately lets through. Teach your reviewers to read the job's last line (*no new trace gaps*), not the widget's count. (The outline job already reports its inherited findings as *skipped*; the trace job's JUnit report should do the same, and this is on the team's list for the tooling.)

30.6 Reading the sprint's own numbers

At every sprint end the pipeline's records go through `gitlab.pl analytics` (Chapter 32, Measuring with GitLab analytics (p. 513)). Sprint 5's report starts like this:

docs/tidewater/sprints/05/analytics.txt

```
GitLab analytics: tidewater/model, 2026-11-30T00:00:00Z .. 2026-12-11T23:59:59Z, by sprint

1. Value stream (hours; 5 issues closed by merge requests)
   stage                                n   median      p85
   issue created -> first commit        5    168.5    264.5
   first commit -> MR opened            5        0        0
   MR opened -> first review            5      3.3      4.8
   first review -> merged               5      0.2      0.2
```

```
   merged -> issue closed                 5        0        0
   issue created -> issue closed          5      172    269.5

2. DORA, for a model repository
   deployment frequency               15 (8.75 a week)
   lead time (first commit -> merged on develop) median 3.5 h, p85 5 h
   change failure rate                0% (0 fix-forward or revert MRs / 5 merges)
   time to restore (develop red -> green) median - h, p85 - h (0 restores)

3. CI/CD: 26 pipelines, success rate 96.2%, duration median 3.4 min, p85 3.7 min
   period                          runs   failed success%   median min
   Sprint 5                          26        1     96.2          3.4
```

Three readings matter, and one of them is a trap.

The trap: "issue created -> first commit". The median is the longest stage in the value stream by far. Read naively, it says the team let issues sit for a week before touching them. Read correctly, it says the team filed sprint 5's stories and tasks at the sprint 4 refinement and the sprint 5 kickoff, and started each task on its planned day. That is a queue by design, not a delay. A value stream measures *waiting*; whether the waiting is a problem depends on whether anybody needed the work sooner.

The real signal: "MR opened -> first review" and "first review -> merged". A merge request waited a few hours for its first review and was merged within minutes of approval. In the document era, a model change waited for a review board measured in weeks (Chapter 3, The directive (p. 22) has the notional baseline). The scorecard's review turnaround, 3.3 hours, is the same fact.

Change failure rate: zero. No merge to `develop` needed a revert or a fix-forward. The one failed pipeline of the sprint was !20's first run – a merge request pipeline, which is where failures *should* happen. A failure on a merge request is the system working; a failure on `develop` is the system missing something. Keep the two apart when you talk about pipeline health, or a good sprint will sound like a bad one.

30.7 What went wrong, and what the team did about it

A case study that only reports successes teaches nothing. Sprint 5's problems were small, and each one taught something.

- **A real `satisfy` was deleted as a duplicate.** Caught by the ratchet on the merge request, fixed in the same merge request (Section 30.5.5, Friday 11 December: the ratchet catches a real mistake (p. 487)). The retrospective turned it into practice: prune trace links from the generated trace report, never by eye.
- **The spillage tabletop left no lesson behind.** One closed issue and a one-line comment (Section 30.5.4, Thursday 10 December: the spillage tabletop (p. 487)). The fix is a three-bullet record, and it costs five minutes.
- **The intake issue for the first drop is still open.** Tom's stand-up on Friday 11 December still lists *TW-25 intake prime drop v1.0 (2026-10-02), by Oct 16* as today's work – a task due two months earlier. The drop was imported and merged in sprint 2; nobody closed the intake issue, so the drafting tool keeps offering it. The analytics for sprint 6 will list it as the oldest open issue. An issue that is really done but not closed pollutes every report built

on issues; close what you finish, or the tools will keep reminding you of things that are not true.

- **On-time delivery was seven of eight, not eight of eight.** The quad counts TW-67 as late because it carried over from sprint 4 (Section 30.10, What Jim told management (p. 492)). That is correct. A story that crosses a sprint boundary is late, even when nobody did anything wrong. Do not argue with the measure; explain it once.

30.8 The review: numbers someone asked for

Two days before the review, on Wednesday 9 December, the deputy program manager asked Jim for the trace numbers. Jim did not build a slide.

> *The deputy PM asked for the trace numbers. I sent a link.*
>
> — Jim

The link was the model site's trace page (Chapter 17, The DevSecOps tools that earn their keep (p. 255)), regenerated by the pipeline on every merge to `develop`. In the document era the same question meant two days of spreadsheet work, notionally (Chapter 3, The directive (p. 22)). That small exchange is the whole argument of this book in two sentences.

The review itself ran on Friday 11 December, 10:00 to 11:00, in the rehearsed format the team adopted in sprint 4 (Chapter 29, Sprint 4: the model site, a cycle broken, and the demo that landed (p. 459), Chapter 25, Talking about progress: helps for engineers who would rather not (p. 390)): lead with the question, show the output, one before-and-after, one safe live change, the ask. The demo script started as a `talk.pl review` draft:

docs/tidewater/sprints/05/review-script.md

```
# Sprint 5 review -- demo script (draft: edit before the meeting)

## 1. The question this sprint answered
"Did we do what we set out to do: Procured parts and DataCenter closed; Legacy Import untangled;
trace becomes a gate?"
Answer in one line: 8 of 8 stories done, 21 of 30 requirements with a verification case.

## 2. The same four numbers as every review
- TODO lines left: 14 (down 8 from 22)
- Requirements with a verification case: 21 of 30 (70%) (no change)
- Open discrepancy reports and prime changes: 7 (up 2 from 5)
- Blocked items: 0

## 3. Show the output, not the tool
- The model site: https://tidewater.pages.gitlab.gov.example.invalid/model/
- The generated documents (docs/ on the site: part tree, trace, data exchange tables)
- 5 changes merged this sprint, 5 with green checks; show one going red to green:
```

The draft's title says *edit before the meeting*, and the team did: the live change – delete a `satisfy` on a scratch branch, watch the trace job name the requirement – had just happened for real to Bob that morning, so the demo could point at !20 instead of staging it.

The review record shows what was accepted and the four numbers:

docs/tidewater/sprints/05/review.txt

```
Sprint 5 review, 2026-12-11 10:00

Goal: Procured parts and DataCenter closed; Legacy Import untangled; trace becomes a gate

Demonstrated:
  TW-67 Close the Stations level tags
  TW-89 Learn: the requirement recipe; writing memos from the model
  TW-90 Procured parts as metadata; DataCenter level tags
  TW-91 Prune ObservingStation's trace spaghetti
  TW-92 Untangle the Legacy Import package
  TW-93 The trace job becomes a gate
  TW-94 Grade our requirements with the recipe
  TW-95 Release rehearsal and a spillage tabletop

Artifacts shown: the merge requests and pipelines of the sprint, the model site (GitLab Pages).

Feedback (Deputy PM): "Can I get the trace numbers without asking you?" Jim showed the trace page
on the model site, rebuilt on every merge; the link goes in the weekly note.

The four headline numbers: TODO lines left 14; verify coverage 70%; open DRs 7; blocked items 0.
```

Every committed story was demonstrated: delivery was 18.5 of 18.5 story points (100%), the first full sprint since sprint 3. The TODO burn-down went from 22 to 14 lines of conversion debt. Verify coverage did not move – and should not have: the unverified requirements are gaps in the prime's model, and the way to close them is through the discrepancy reports, not by the government inventing verification cases.

At the review, GitLab's own To-Do lists (Chapter 17, The DevSecOps tools that earn their keep (p. 255)) showed who owned what next:

docs/tidewater/sprints/05/todo-at-review.txt

```
GitLab To-Do lists at the review (2026-12-11 10:00)

  Jim: 0 review(s) requested, 10 open issue(s) assigned
    #84 DR-1: TW-SYS-006 text differs between DOORS and the model
    #85 DR-2: TW-SYS-012 has no satisfy
    #86 DR-3: TW-SYS-016 is in DOORS but not in the model
    #87 DR-4: TW-SYS-011 has no verification case
    #88 DR-7: WaveSensor is tagged L5 but is a direct part of MooredBuoy (L4)
    #109 DR-5: The Legacy Import package is clutter from the document port
    #110 DR-6: Weak verbs and missing parts in the DOORS requirement text
    #111 Learn: memos and briefings from the model
    #116 VCRM: which requirements are verified, and how
```

Jim's list is long, and it is mostly discrepancy reports waiting on the prime. That is the shape of the job for a government lead on a program like this one: the government's model work is done in the team; the prime's is tracked in the team.

30.9 The retrospective

docs/tidewater/sprints/05/retro.txt

```
Sprint 5 retrospective, 2026-12-11 11:00

Start:
  - Lowering .ci/outline-max in the same MR that fixes a violation
Stop:
  - Cleaning up trace links by eye
Continue:
  - The ratchet: it caught a real mistake

Pulse survey (1-5, anonymous): mean 3.8
```

Two of the three actions came straight from the week. *Stop cleaning up trace links by eye* is Bob's lesson from !20; the ratchet will catch a bad deletion, but a pruning task should start from the generated trace report, not from memory. *Start lowering `.ci/outline-max` in the same merge request that fixes a violation* closes a gap the outline ratchet left open: an engineer who fixes an outline violation and does not lower the allowance leaves room for someone else to add one back.

The pulse rose to 3.8. The survey is anonymous in the meeting, but the evidence keeps each person's answers (Chapter 3, The directive (p. 22) lists the five questions), and the lowest are Bob's: on *I understand why we are working this way*, on *I can see our progress*, and on *I would choose to keep working this way*. On *the checks catch problems I would have missed* Bob answered higher – in the week the ratchet caught Bob's mistake. A lead reading those answers should not try to argue the low ones up. They are a prompt for a conversation, one to one, about what would make the *why* and the progress visible to the person doing the work.

30.10 What Jim told management

The weekly status on Friday 11 December was the quad, as the kit printed it:

docs/tidewater/sprints/05/quad-2026-12-11.txt

```
Weekly status Model -- as of 2026-12-11 (sprint 5)
Sprint progress 100% (18/18 SP) improving (was 43%)   On-time delivery: 4/4 this week, 7/8 this
sprint

TECHNICAL PRIORITIES
  DONE  TW-67        2  "Close the Stations level tags Bob
  DONE  TW-91        2  "Prune ObservingStation's trac Bob
  DONE  TW-92        3  "Untangle the Legacy Import pa Tom
  DONE  TW-93        2  "The trace job becomes a gate" Tom
  DONE  TW-94        2  "Grade our requirements with t Jim
  DONE  TW-95        3  "Release rehearsal and a spill Tom
  DONE  TW-89        1  "Learn: the requirement recipe Jim
  DONE  TW-90      3.5  "Procured parts as metadata    Bob

WATCH ITEMS / PM HELP NEEDED
  none
```

```
SCHEDULE MILESTONES
  30 days out
    -
  60 days out
    -
  90 days out
    -
```

The covering note on top of a quad like this one needs only two sentences, for example: *Sprint 5 delivered everything it committed. The pipeline now refuses a change that breaks the requirement trace; it caught one this morning, before review.* It does not need to explain how the ratchet works. If management wants to know, they will ask, and the answer is a link (Chapter 36, Telling management (p. 538)).

30.11 Tools adopted this sprint, and the pain each removed

Table 30.1: Sprint 5's tools

Tool	The pain it removed	Where it is explained
The trace ratchet	a `satisfy` deleted by accident survived until the next manual matrix	Chapter 20, The pipeline, job by job (p. 302)
The requirements recipe check	weak requirement text found at review, as red ink, months late	Chapter 7, Requirements a pipeline can check: the recipe (p. 73)
`@Procured` metadata	"which parts come from which supplier?" needed a call to the prime	Chapter 8, SysML v2 by example: a workbook (p. 92)
A sandbox project for rehearsals	the first real release would also have been the first try	Chapter 20, The pipeline, job by job (p. 302)

30.12 You can now…

Note: You can now…

- Run the requirements recipe on DOORS text and decide which findings to send to the prime, and how.
- Explain why the government does not edit the prime's requirement text, and what it does instead.
- Read a trace ratchet verdict: tell the target branch's old gaps from the gaps a change adds.
- Remove inherited clutter from the v2 model and file the discrepancy report that removes it upstream.
- Run a planning-poker round where the second vote matters more than the first.

30.13 Exercises

Exercise 30.1: Grade a module

In the Tidewater sandbox, run `req-recipe.pl` on `model/requirements/segment` alone. Pick the two lowest-scoring requirements and rewrite them to the recipe, as a reply to DR-6 would (Chapter 7, Requirements a pipeline can check: the recipe (p. 73) has the pattern). Check your rewrites with `req-recipe.pl --strict` on a text file.

Exercise 30.2: Break the trace on purpose

Do practice lesson 11 (`perl agile.pl practice 11`) if you have not. Then, on a branch in the Tidewater sandbox's `work/bob` clone, delete one `satisfy` line, commit, and run `perl ci/job.pl trace` with both `CI_PIPELINE_SOURCE=merge_request_event` and `CI_MERGE_REQUEST_TARGET_BRANCH_NAME=develop` in the environment (with only the target branch set, the job treats the run as a branch pipeline and says "not a merge request: reported only"). Find the line that names the requirement you orphaned.

Exercise 30.3: Write a discrepancy report

Choose one finding from `recipe-doors.txt` that DR-6 does not quote. Write a DR for it from the Discrepancy Report template (Appendix I, Templates and GitLab Markdown, field by field (p. 747)): what we found, why it matters to verification, what we ask the prime to do. Keep it under a hundred words.

Exercise 30.4: Read the record of a tabletop

Read issue #108 in the sandbox (`gitlab.pl issue show 108`). Write the three bullets the team's new template asks for, as if you had run the tabletop: what you did, what surprised you, what you would change.

30.14 Applying this to your project

- **Grade the requirements before you design to them.** Run a recipe-style check on the inherited text in your first month, not before your first review. Send the findings to whoever owns the text.
- **Make the trace a ratchet, not a gate.** If you inherit gaps you cannot close, a gate that fails on every gap will be turned off within a week. A gate that fails only on *new* gaps will stay on for the life of the program.
- **Delete inherited clutter in your model and upstream at the same time.** One without the other leaves you fighting the same file at every delivery.
- **Rehearse your first release where nobody can see it.** A sandbox project costs nothing and turns the first real baseline into the second attempt.
- **Record what a tabletop taught you.** A closed issue with no notes is a meeting nobody can learn from.

CHAPTER

31

Sprint 6: Behavior by hand, a table for the program, and baseline-0

From what you know →

You know

the interim design review, and the baseline you freeze for it.

→ Here it is called

baseline-0: a release branch, a protected tag and a briefing.

→ What is new

the baseline rebuilds identically, on demand, with its evidence attached.

Survival kit

What you have

a model whose structure is almost closed, a trace that defends itself (Chapter 30, Sprint 5: Grading the requirements, and a trace that defends itself (p. 475)), a recipe-graded set of requirements, and a team that has rehearsed a release in a sandbox.

What you need from this chapter: how a three-person team finished its first twelve weeks with something management could hold – a tagged, reproducible baseline, a generated table of what crosses each interface, a verification matrix – and how it handled the sprint's real events: a failed nightly pipeline, a token committed by mistake, a review comment that blocked a merge, a story that could not finish, and a number that almost told management the wrong thing.

31.1 The question this sprint answered

Sprint 6 ended with the twelve-week briefing to program management. The question behind every story in it was the one a program manager asks at any milestone: *can you hand me something I can rely on?* In the document era the answer was a set of documents with signatures. For this team it had to be three things:

1. **A baseline**: a tagged, reproducible version of the model, approved by the CCB, that can be rebuilt byte for byte and named in a contract letter.
2. **An answer to "what crosses each interface?"** – the question every interface control working group asks – generated from the model, not typed into a spreadsheet.
3. **An honest answer to "which requirements are verified, and how?"**, in the form a test organization uses: a verification cross-reference matrix (VCRM).

The pulse at the sprint 5 retrospective was 3.8. Nobody on the team would have predicted that number in week one. The risk now was the opposite of discouragement: a team pleased with itself, with a short sprint, holidays, and an audience waiting.

31.2 Kickoff: a short sprint with a big ending

docs/tidewater/sprints/06/kickoff.txt

```
Sprint 6 kickoff, 2026-12-14 09:00 (90 minutes)

Goal: Behavior started; Operations closed; the data exchange table; interim baseline-0 for the
program review
Capacity: 15.7 points (Jim 5, Bob 8, Tom 8 per full sprint, less holidays and leave)
Committed: 14.0 points in 8 stories (89% of capacity): engineering 11.5, platform 2.0 (14%),
learning 0.5 (3%)

TW-111   E3    0.5 pts  learning     Learn: memos and briefings from the model
           #119  0.5 pt  jim  2026-12-14 .. 2026-12-14  The memo and briefing chapters
TW-112   E8    3.0 pts  engineering  BuoyModes state machine by hand
           #120  3.0 pt  bob  2026-12-18 .. 2026-12-29  Write the BuoyModes state def; exhibit it
           on MooredBuoy
TW-113   E8    1.0 pts  engineering  Fork, pin and object flow in ObserveCoast
           #121  1.0 pt  bob  2026-12-17 .. 2026-12-17  Fork, pin, object and control flow by
           hand
TW-114   E6    1.5 pts  engineering  Level tags in Operations
           #122  1.5 pt  tom  2026-12-14 .. 2026-12-15  Level tags in Operations
TW-115   E8    2.0 pts  engineering  Data exchange table for the program review
           #123  1.0 pt  tom  2026-12-16 .. 2026-12-17  Document the flow properties in
           Interfaces
           #124  1.0 pt  tom  2026-12-17 .. 2026-12-18  Generate the data exchange table
TW-116   E7    2.0 pts  engineering  VCRM: which requirements are verified, and how
           #125  1.0 pt  tom  2026-12-18 .. 2026-12-21  Add the vcrm job
           #126  1.0 pt  jim  2026-12-15 .. 2026-12-16  Enter and sign the inspection verdicts
TW-117   E1    2.0 pts  platform     Interim baseline-0 for the program review
           #127  2.0 pt  tom  2026-12-21 .. 2026-12-24  Release branch, CCB concurrence, tag
           baseline-0, back-merge
TW-118   E3    2.0 pts  engineering  Retrospective, the management briefing and the plan for next
semester
           #128  2.0 pt  jim  2026-12-17 .. 2026-12-22  The twelve-week briefing and memo
```

Capacity is the lowest of the twelve weeks: the sprint ends on Christmas Eve, and Bob is on leave for its whole second week. Read the plan with that in mind and one line stands out. TW-112, *BuoyModes state machine by hand*, is a three-point task scheduled to start on Friday 18 December – the day before Bob's leave – and to finish on 29 December, in the next sprint. Nobody planned to fail it. But a task that starts the day before a week of leave and is due after the sprint

ends is not a commitment; it is a hope. The team committed it because BuoyModes had to start before the January drop, and it carried, exactly as the dates said it would (Section 31.6, The carryover (p. 507)).

The work mix shows how far the team had come. Platform work is 14% of the task points done (Section 13.8, Two ways to count "done" (p. 205)) and learning 4%; engineering is 82%. In sprint 1 the engineering share was 27% (Bottom line up front (p. iii)).

31.3 Week 1: the quiet days and the busy one

31.3.1 Monday to Wednesday: Operations closed, verdicts signed

Jim spent Monday morning on the last learning task of the twelve weeks: the memo and briefing chapters of this book (Chapter 37, Writing it down: DoD-style memos from Vim (p. 545), Chapter 35, The briefing (p. 532)), because the briefing was Jim's to give in ten days.

Tom's !21 closed the level-tag TODO lines in the Operations package. Its first pipeline was green. By sprint 6 a green first pipeline on a structure change is unremarkable, which is the point: the team had stopped noticing the checks on routine work, the way an engineer stops noticing a calculator.

On Wednesday Jim opened !22, *Inspection verdicts, signed*. Some Tidewater requirements are verified by inspection – someone looks at the thing and signs that it complies – and the results schema the team adopted in week zero (ADR 0003) puts those verdicts in a file in the repository, with the name of the person who signed. A signed verdict is a configuration item like any other: it goes through a merge request, it has an author and an approver, and it is in the baseline. Jim's merge request entered the verdicts Jim could sign from the drop-1 evidence; the VCRM job, later in the sprint, reads them. Here is the file, in Tom's clone of `develop`:

```
git checkout -q develop
cat verification/results-gov.csv
```

Each row is one verdict: the requirement, the verification case, the verdict, the commit of the model it was judged against, how it was judged, when, and who signed. Read the `commit` column twice. It is what turns a signature into evidence: years from now, anyone can check out exactly the model Jim inspected. In the document era an inspection record named a document revision, and the revision named nothing that could be rebuilt.

The file is named `results-gov.csv` because it holds the government's own verdicts. When the prime's test results arrive with later drops (Chapter 44, Verification as a closed loop (p. 613)), they land in a sibling file in the same schema, and the VCRM reads both. One schema for every source of verdicts is ADR 0003's whole point.

31.3.2 Thursday 17 December: Bob's day

Thursday was the busiest day of the twelve weeks, and it is told in full, hour by hour, in Chapter 39, A day in the life, week twelve (p. 563). The short version, from GitLab's own record:

docs/tidewater/sprints/06/day-in-life/timeline.txt

```
Thursday 2026-12-17, as GitLab recorded it (Eastern time)

  09:15 bob  opened #132 for the failed nightly pipeline #83
  10:00 bob  opened !23 Draft: Fork, pin and flows in ObserveCoast (pipeline #86 failed)
  10:30 tom  cleared the runner's disk; re-ran the full check: #84 success; closed #132
  10:45 tom  opened !24 Document what crosses each interface (pipeline #90 success)
  12:00 bob  marked !23 ready
  14:05 bob  line comment on !24: request changes (the 'late' assumption)
  14:20 tom  merge refused: discussions not resolved (1 open) (405 Method Not Allowed)
  14:50 bob  resolved the thread on !24
  15:10 tom  closed #123 Document the flow properties in Interfaces
  15:10 tom  merged !24
  15:30 bob  closed #121 Fork, pin, object and control flow by hand
  15:30 bob  merged !23
```

Four things happened before lunch, and each one is a pattern worth knowing.

A failed nightly pipeline, triaged at the stand-up. The scheduled 02:00 full check failed in every job before running any check. Bob saw it first, opened an issue at 09:15 and linked the failing pipeline; Tom took it at the stand-up and closed it by 10:30:

GitLab Markdown: docs/tidewater/sprints/06/day-in-life/incident.md

````
# #132 Nightly full check failed: no space left on the runner

The nightly full check (pipeline #83, scheduled 02:00) failed in every job before running any
check:

```text
fatal: could not create work tree dir 'builds/tidewater/model': No space left on device
```

Job: https://gitlab.gov.example.invalid/tidewater/model/-/pipelines/83/../jobs/543

Not a model problem: develop's last MR pipeline was green. Picked up at the stand-up.
---- comments

**@tom** (2026-12-17T15:30:00Z) [system]:
added 1h of time spent

**@tom** (2026-12-17T15:30:00Z) [system]:
removed ~"workflow::in-progress" label

**@tom** (2026-12-17T15:30:00Z) [system]:
added ~"workflow::done" label

**@tom** (2026-12-17T15:30:00Z) [system]:
closed

**@tom** (2026-12-17T15:30:00Z):
Cleared the runner's old build directories (14 GB) and set the runner to clean up after each job.
Re-ran the full check: pipeline #84 success.
````

The cause had nothing to do with the model: the runner's disk was full. The fix had two parts,

and the second is the one that matters – clearing the disk fixes today; making the runner clean up after each job fixes the next month (Chapter 21, Runners: where the pipeline runs (p. 347)). The scorecard records how long `develop` was red: the worst time to restore in the sprint was 8.5 hours, from the failed 02:00 run to the green rerun.

A draft merge request that failed on purpose. Bob opened !23, the fork, pin and object flow in `ObserveCoast`, as a *draft* at 10:00 – a draft says "not ready for review, but run the checks". Its first pipeline failed the trace job:

```
grep -h "this change adds\|no new trace" gitlab/pipelines/tidewater-model/86/trace.log
gitlab/pipelines/tidewater-model/87/trace.log
```

While editing the behavior, Bob had also touched the text of TW-SYS-007 in the model's copy of the requirements. The model's text no longer matched DOORS: a new *drift*. The ratchet named it, Bob restored the DOORS wording, and the second pipeline passed. Bob marked the merge request ready at noon. Notice the order: the check ran *before* anyone was asked to review. Reviewers never saw the mistake.

A review comment that blocked a merge. Tom's !24 documented the flow properties in `Interfaces` – what each interface carries – so the data exchange table could be generated from them. Bob reviewed it at 14:05 and asked for a change, with a suggestion block:

GitLab Markdown: docs/tidewater/sprints/06/samples/mr-24.md

```
# !24 Document what crosses each interface

## What and why

Document the flow properties in Interfaces (TW-115).

Closes #123
Part of #115

## Model changes

| Package | Change |
|---|---|
| `model/library/Interfaces.sysml` | changed |

## Checks

- [x] `gitlab.pl ci` passed locally
- [x] `docs/` regenerated and committed
- [x] No new `// TODO` lines
- [x] Trace job: no new orphan, unverified or drifted requirement

---- thread

**@bob** (2026-12-17T19:05:00Z):
**Request changes** -- `model/library/Interfaces.sysml`, the `health` flow:

> Sent with every transmission; the data center flags a station whose report is late.
```

```
"Late" assumes a schedule we have not written down anywhere. TW-SYS-009 does not say how late is
late. Either cite where the period comes from, or say it is an assumption:

```suggestion:-0+0
        out item health : HealthReport { doc /* Sent with every transmission. Assumption (to
        confirm with the prime): late = two missed transmissions. */ }
```

@jim (2026-12-17T19:20:00Z) [system]:
approved this merge request

@tom (2026-12-17T19:50:00Z):
Applied the suggestion; the assumption goes in the next prime-tbd list.

@bob (2026-12-17T20:00:00Z) [system]:
approved this merge request

@tom (2026-12-17T20:10:00Z) [system]:
merged
```

"Late" assumes a schedule nobody had written down. At 14:20 Tom tried to merge anyway and GitLab refused: *discussions not resolved* (*1 open*). That refusal is the project setting the team chose in week zero (Chapter 15, Setting up GitLab (p. 226)): a merge request cannot merge while a reviewer's thread is open. Tom applied the suggestion, Bob resolved the thread, Jim and Bob approved, and it merged at 15:10. You can see the approvals in the sandbox:

```
perl "$KIT/bin/gitlab.pl" --local gitlab --project tidewater/model mr show 24 | head -3
```

Two approvals, because `Interfaces.sysml` sits in a CODEOWNERS section that needs both the package owner and Jim as systems lead (Chapter 22, Reviewing a model change (p. 357)).

An assumption made visible. Tom's reply on !24 says the assumption "goes in the next prime-tbd list". This is the team's practice for work that cannot wait for the prime (Chapter 45, The prime's model changes: receive, assess, disposition (p. 619)): proceed on a written assumption, label it `prime-tbd`, and close it when a drop confirms or corrects it. The suggestion put the assumption into the model's own doc text, so it shows up in everything generated from the model – including, the next day, the data exchange table.

> *Six things happened before lunch and I knew about every one of them from GitLab, not from email.*
>
> — Bob, 17 December

Bob's GitLab To-Do list at 14:05 held exactly one item – Tom's review request:

docs/tidewater/sprints/06/day-in-life/todo-list.txt

```
Bob's To-Do list, 2026-12-17 14:05

  !24 Document what crosses each interface (review requested by @tom)
  (the failed nightly, #132, was picked up at the stand-up and closed by Tom at 10:30)
```

31.3.3 Friday 18 December: the table, and a token

Tom's !25 generated the data exchange table: for each interface, the flows, their direction, their type, their fields, and the notes from the model's doc text. Its first pipeline failed – not on the model, on the `secrets` job:

```
tail -3 gitlab/pipelines/tidewater-model/95/secrets.log
```

Tom had pasted a personal access token into a helper script to test it, and committed the script. The pipeline stopped it before it reached `develop`. What Tom did next is the whole procedure for a leaked credential, and its order matters:

GitLab Markdown: docs/tidewater/sprints/06/samples/mr-25.md

```
# !25 Generate the data exchange table

## What and why

What crosses each interface, generated from the interface items and flow properties.

Closes #124
Part of #115

## Checks

- [ ] `gitlab.pl ci` passed locally (not run before pushing)
- [x] `docs/` regenerated and committed
- [x] No new `// TODO` lines
- [x] Trace job: no new orphan, unverified or drifted requirement

---- thread

**@tom** (2026-12-18T16:30:00Z):
The secrets job caught a personal access token I pasted into `ci/exchange-check.pl` to test it.
Revoked it in GitLab, created a read_api project token instead, stored it as the masked,
protected variable `READ_TOKEN`, and rewrote this branch so the token is in no commit.

**@jim** (2026-12-18T19:50:00Z) [system]:
approved this merge request

**@tom** (2026-12-18T20:00:00Z) [system]:
merged
```

Revoke first, because the token may already have been copied from the branch. Replace it with a narrower one stored as a masked, protected CI/CD variable (Chapter 17, The DevSecOps tools that earn their keep (p. 255)). Then rewrite the branch so the token is in no commit. Deleting the line in a new commit is not enough: the old commit still holds it (Appendix E, Rescue: when something goes wrong (p. 724)).

The table itself is a generated document in `docs/`, committed with the model and rebuilt by the pipeline. Here it is in the sandbox, as of the end of the sprint:

```
git checkout -q develop
head -12 docs/data-exchange.md
```

Look at the `HealthIF` row: the assumption from Thursday's review is in the table, labeled as an assumption to confirm with the prime. In the document era, that assumption would have lived in someone's head or in a comment on a PDF. Here it is part of the deliverable, and it will stay visible until the prime answers.

The week's quad went to management on Friday:

docs/tidewater/sprints/06/quad-2026-12-18.txt

```
Weekly status Model -- as of 2026-12-18 (sprint 6)
Sprint progress 36% (5/14 SP) new   On-time delivery: 4/4 this week, 4/4 this sprint

TECHNICAL PRIORITIES
  OPEN  TW-112       3  "BuoyModes state machine by ha Bob
  DONE  TW-111     0.5  "Learn: memos and briefings fr Jim
  DONE  TW-113       1  "Fork, pin and object flow in  Bob
  DONE  TW-114     1.5  "Level tags in Operations"     Tom
  DONE  TW-115       2  "Data exchange table for the p Tom
  OPEN  TW-116       2  "VCRM: which requirements are  Jim
  OPEN  TW-117       2  "Interim baseline-0 for the pr Tom
  OPEN  TW-118       2  "Retrospective, the management Jim
```

31.4 Week 2: two of three, and the ending

Bob was on leave. The attendance record shows it – stand-ups with two people from Monday 21 December on – and the plan had accounted for it in the capacity. Jim and Tom finished the sprint.

31.4.1 Monday 21 December: the VCRM job

Tom's !26 added the `vcrm` job and the baseline jobs to the pipeline. The VCRM reads every requirement, its verification method, the verification cases that claim to verify it, and the verdicts in the results files, and writes `vcrm.html` and `vcrm.csv`. In the sandbox:

```
perl ci/job.pl vcrm | tail -1
```

Read that line carefully, because the team almost did not. Hardly any requirement is *verified*, most are *not run*, and some have no case at all. That sounds like a disaster. It is not: it says something different from the trace.

31.4.2 The number that almost misled management

The trace measure the team had reported for twelve weeks – *verify coverage* – asks: **does every requirement have a verification case that claims to verify it?** For Tidewater that was 29 of 30

linked requirements with 9 unverified, the same as at the drop-1 import, because the gaps are the prime's.

The VCRM asks a different question: **has the verification actually been done, with a recorded verdict?** In a program that has not reached test, almost nothing has been. The verified ones are the inspection verdicts Jim signed in !22.

When Jim drafted the review script with `talk.pl review`, the tool did what tools do: it found a newer verification measure and used it.

docs/tidewater/sprints/06/review-script.md

```
# Sprint 6 review -- demo script (draft: edit before the meeting)

## 1. The question this sprint answered
"Did we do what we set out to do: Behavior started; Operations closed; the data exchange table;
interim baseline-0 for the program review?"
Answer in one line: 7 of 8 stories done, 21 of 30 requirements with a verification case.

## 2. The same four numbers as every review
- TODO lines left: 5 (down 9 from 14)
- Requirements with a verification case: 21 of 30 (70%) (no change)
    (a different measure: requirements with a passing recorded result, from the VCRM: 2 of 30)
- Open discrepancy reports and prime changes: 7 (no change)
- Blocked items: 0
```

Verify coverage: 6.7% verified (*VCRM*) (*down 63.3 from 70*). Read naively, that line tells the program manager that verification collapsed in the last sprint. It did not collapse; a second, stricter measure appeared. The draft was corrected before the meeting – the review record uses the measure the program had been seeing all along:

docs/tidewater/sprints/06/review.txt

```
Sprint 6 review, 2026-12-24 10:00

Goal: Behavior started; Operations closed; the data exchange table; interim baseline-0 for the
program review

Demonstrated:
  TW-111 Learn: memos and briefings from the model
  TW-113 Fork, pin and object flow in ObserveCoast
  TW-114 Level tags in Operations
  TW-115 Data exchange table for the program review
  TW-116 VCRM: which requirements are verified, and how
  TW-117 Interim baseline-0 for the program review
  TW-118 Retrospective, the management briefing and the plan for next semester

Carried to sprint 7:
  TW-112 BuoyModes state machine by hand

Artifacts shown: the merge requests and pipelines of the sprint, the model site (GitLab Pages).

The four headline numbers: TODO lines left 5; verify coverage 70%; open DRs 7; blocked items 0.
```

This is the most important lesson about drafting tools in this book (Chapter 25, Talking about progress: helps for engineers who would rather not (p. 390)): **read every number before you say it, and never compare two measures as if they were one.** The fix is not a better tool; it is a person who knows what each number means.

The kit's `talk.pl` was changed after this sprint, because of exactly this near miss: it now keeps the two measures apart and labels the second one as a different measure. Run it on the same sprint in the sandbox and compare its lines with the draft above:

```
perl "$KIT/tools/gitlab/talk.pl" --local gitlab --project tidewater/model review --sprint 6 | sed
-n 7,11p
```

That is how a tool should improve: a person caught a misleading number, and the tool was changed so the next person does not have to.

31.4.3 Tuesday and Wednesday: the briefing, and refinement for January

Jim wrote the twelve-week briefing and its memo on Tuesday (#128). The briefing's content is in Chapter 35, The briefing (p. 532); here is its outline as the evidence keeps it:

docs/tidewater/sprints/06/briefing.txt

```
Tidewater: twelve weeks of MBSE in GitLab -- briefing to program management, 2026-12-24

1. Where we started: documents, then a Cameo model nobody could merge or review (47 diagrams, 2
package cycles).
2. What we built: a GitLab project, a pipeline that checks every change, the model in SysML v2
text, a model site.
3. The scorecard, sprint by sprint:

   sprint  TODO lines  lead time (h)  gated        morale
   1       0           2.2            0 of 1       1.9
   2       52          3.6            6 of 6       2.4
   3       50          4.2            4 of 4       2.2
   4       22          3.5            4 of 4       3
   5       14          3.5            5 of 5       3.8
   6       5           3.5            7 of 7       4.1

4. Before and after (document-era numbers are notional): a requirement change took 23 working
days through the CCB;
   a model change now takes hours from MR to merge, reviewed and checked.
5. The demo: open an MR, watch the pipeline, see the trace and the VCRM on the site.
6. Next semester: drops 2 and 3, behavior, baseline 1 in sprint 20.

The written twin of this briefing is the twelve-week progress memo
(docs/tidewater/memos/2026-12-24-twelve-week-progress.md).
```

Every number in it came from the pipeline's records: the TODO lines, the lead time, the gated changes, the pulse. None was typed into a slide from memory. The written twin of the briefing is the progress memo (Chapter 37, Writing it down: DoD-style memos from Vim (p. 545)).

Wednesday's refinement prepared sprint 7: breaking the last package cycle, the `DailyUpload` interaction by hand, the `EnergyBalance` constraint expression, retiring the profile TODO, and pinning the kit version.

docs/tidewater/sprints/06/refinement.txt

```
Refinement, 2026-12-23 10:00: the Definition of Ready applied to sprint 7's stories

TW-133  2 pts  Break the Stations <-> DataCenter cycle
TW-134  2 pts  The DailyUpload interaction by hand
TW-135  2 pts  The EnergyBalance constraint expression
TW-136  1 pts  Retire the Tidewater Profile TODO
TW-137  2 pts  Pin the kit version
```

Case study: The decision: an interim baseline, now

The plan called for the first real baseline – baseline-1, the port's acceptance – in sprint 20. Jim proposed an interim *baseline-0* for the twelve-week review, and the reasoning is worth copying.

Against: the model was not finished. BuoyModes was still a TODO line, the trace still had the prime's gaps, the package view still showed a cycle. A baseline of an unfinished model, the argument went, would look like a claim that it was finished.

For: a baseline is not a claim of completeness; it is a claim of *identity*. Baseline-0 says: *this* is the model on 24 December, these are its known gaps (the release notes and the VCRM list them), and anyone can rebuild it exactly. That is precisely what a program needs before a review – a fixed point to discuss – and it is what the document era could never provide: the "baseline" was a set of PDFs whose source nobody could reproduce. The other reason was practice. The release path has steps that only run on a tag: the baseline audit, the toolchain inventory, the release. Running them first in December, with nothing riding on them, was cheaper than running them first for baseline-1.

The CCB agreed on the second argument. The deputy program manager's concurrence comment on !27 says what the board reviewed: the change list and the pipeline.

31.5 Thursday 24 December: baseline-0

The interim baseline is the first time the team's work went out under the CCB's name. Tom had rehearsed the path in sprint 5 in a sandbox project. The real path, on the real project:

1. A `release/baseline-0` branch from `develop`, with the CHANGELOG section and regenerated docs.
2. A merge request into `main`, !27, with the release template's checklist and one line the CCB writes itself.
3. The deputy program manager's concurrence, recorded as a comment on the merge request – the CCB's approval, in the same place as the engineering approvals:

GitLab Markdown: docs/tidewater/sprints/06/samples/mr-27.md

```
# !27 Release baseline-0

## Release baseline-0

Closes #127

- [x] Pipeline green
- [ ] CCB concurrence recorded below by the deputy PM

---- thread

**@dpm** (2026-12-24T16:00:00Z):
CCB concurrence for baseline-0: the board reviewed the change list and the pipeline and concurs.
-- Deputy PM, for the CCB

**@jim** (2026-12-24T16:00:00Z) [system]:
approved this merge request

**@tom** (2026-12-24T16:00:00Z) [system]:
merged
```

1. The merge into `main`, the annotated tag `baseline-0` whose message carries the sha256 of the generated docs, and the back-merge into `develop`.
2. The tag pipeline: it regenerates the docs from the tagged model, checks the hash against the tag message, writes the toolchain inventory, rebuilds the VCRM and publishes the release.

The tag in the sandbox:

```
git tag -n4 -l 'baseline-*'
```

And the end of the tag pipeline's `baseline` job:

docs/tidewater/sprints/06/pipelines/tag-baseline-0-108.log

```
baseline-0 rebuilds identically
(sha256=9b7c27d309be87b5fa96d49eb0aee8fcfcb04cc690e3e80027cdf0fbbec802ef)
baseline: baseline-0
commit: 7884d6291735702e4f0d926126da29603898453c
kit: agile-kit 2026.10
perl 5.34.0
git version 2.34.1
os: windows/amd64
runner: mbse-win-01
image: none (shell runner)
docs sha256: 9b7c27d309be87b5fa96d49eb0aee8fcfcb04cc690e3e80027cdf0fbbec802ef
```

baseline-0 rebuilds identically. That one line is a physical configuration audit, done by a machine in a minute, on every baseline, forever. The toolchain lines below it say which kit, Perl and git produced the baseline, so the audit can be repeated years from now with the same tools.

The scorecard counts it: 1 baseline tagged, 1 reproducible.

31.6 The carryover

At 16:30 on Christmas Eve the journal recorded the sprint's one carryover: Bob's BuoyModes task, #120, continued as #138 in sprint 7 with its remaining points.

```
perl "$KIT/bin/gitlab.pl" --local gitlab --project tidewater/model issue show 138 | sed -n 1,6p
```

Delivery for the sprint was 11 of 14 story points (79%). The carryover was predictable from the kickoff – the task's dates ran past the sprint – and that is the lesson: when the plan already says a task cannot finish, either split it so the first part can, or leave it out and say why. Committing it anyway made the sprint look worse than the team's work was. The converter's TODO line for the state machine is still in the model, and the TODO burn-down counts it.

31.7 Reading the sprint's own numbers

The sprint-end analytics (Chapter 32, Measuring with GitLab analytics (p. 513)) tell the story of the busy Thursday in four lines:

docs/tidewater/sprints/06/analytics.txt

```
2. DORA, for a model repository
   deployment frequency               17 (10.82 a week)
   lead time (first commit -> merged on develop) median 3.5 h, p85 4.4 h
   change failure rate                0% (0 fix-forward or revert MRs / 7 merges)
   time to restore (develop red -> green) median 8.6 h, p85 8.6 h (1 restores)

3. CI/CD: 37 pipelines, success rate 89.2%, duration median 2.2 min, p85 3.7 min
   period                          runs   failed success%   median min
   Sprint 6                          37        4     89.2          2.2
   job failures: secrets 3/37, grammar 1/37, lint 1/37, outline 1/37, todo 1/37, trace 1/34
```

Time to restore is no longer empty, as it was in sprint 5: `develop` was red once, from the 02:00 nightly failure to the green rerun. That is the disk-full incident, and the number is honest about it – the nightly failed at 02:00 and nobody looked until the stand-up. A team that wants that number smaller has two choices: a runner that cannot fill its disk (the fix Tom made), or a notification that wakes someone up. For a model repository, the first is the right answer; nobody's model needs to be green at 03:00.

Pipeline success rate dropped from sprint 5. Look at the job failures line before worrying: the `secrets` failures are Tom's token (the merge request pipeline, a rerun, and the push to the branch), and the other single failures are the nightly run, where every job failed for the same disk-full reason. Not one failure was a model error that reached `develop`. A falling success rate with that explanation is a sign the checks are working; the same number without the explanation would worry a manager for nothing. Always bring the explanation.

Change failure rate is still zero. Seven merges to `develop`, none reverted or fixed forward.

31.8 What went wrong, and what the team did about it

- **The runner's disk filled up.** Fixed the same morning; the runner now cleans up after each job. The lesson for administration is in Chapter 42, Administration (p. 586): a runner is a server, and servers need the same care as anything else.
- **A personal access token was committed.** Caught by the pipeline before it reached `develop`; revoked, replaced with a masked project token, history rewritten. The retrospective made it a rule: no tokens in scripts, even test ones (Chapter 48, Security: protect the environment, engineer it into the system (p. 651)).
- **A story was committed that could not finish.** BuoyModes, planned across Bob's leave (Section 31.6, The carryover (p. 507)).
- **A drafting tool nearly misreported a measure to management.** Corrected before the review (Section 31.4.2, The number that almost misled management (p. 502)).
- **A requirement's text was changed by accident.** Bob's edit to TW-SYS-007 in !23, caught by the trace ratchet in a draft merge request, before any reviewer saw it.

None of these reached the model on `develop`. In the document era, the same five mistakes – a lost day of work, a leaked credential, an over-committed plan, a misleading slide, an altered requirement – would have surfaced, if at all, at the next milestone review.

31.9 The review and the retrospective

The review on 24 December had the largest audience of the twelve weeks: the deputy program manager and the program manager joined Jim and Tom. The demo used the model site as its home page and ended on the baseline. The program manager's feedback was short:

> *Keep doing this. Show me the same numbers in March.*
>
> — Program manager, 24 December

The retrospective:

docs/tidewater/sprints/06/retro.txt

```
Sprint 6 retrospective, 2026-12-24 11:00

Start:
  - Writing acceptance criteria as pipeline checks
Stop:
  - Pasting tokens into scripts, even test ones
Continue:
  - The nightly pipeline
  - Reviews with line comments instead of red-pen PDFs

Pulse survey (1-5, anonymous): mean 4.1
```

Stop pasting tokens into scripts, even test ones is Tom's lesson from Friday. *Start writing acceptance criteria as pipeline checks* is the sprint's most important process change: a story is done when a named check says so, not when someone feels finished (Chapter 14, Writing stories and tasks

for model work (p. 214)). *Continue reviews with line comments instead of red-pen PDFs* is the team recognizing what Thursday's review of !24 was: the same scrutiny as a document review, two hours after the change instead of three weeks.

The pulse: 4.1, the highest of the twelve weeks, from 1.9 at the end of sprint 1. Jim's answers were the highest on the first and last questions – *I understand why we are working this way* and *I would choose to keep working this way*. For the person who carried the directive into the room in week zero, those are the two that count.

Objection: "The morale numbers are the team grading its own homework."

They are, and the book does not pretend otherwise. Three people answering five questions is not a survey; it is a conversation starter. What makes the numbers worth reporting is that they were defined in week zero, asked the same way every sprint, and published even when they went down – sprint 3's dip is in every briefing. A measure you only report when it is good is advertising. The other eleven measures on the scorecard do not depend on anyone's mood.

31.10 Twelve weeks, in the measures agreed on day one

The scorecard was defined in Chapter 3, The directive (p. 22) before the first sprint. These are its values at the end of each sprint, from the pipeline's own records:

Table 31.1: The scorecard at the end of each sprint (sprint 1's TODO count is before the drop-1 import)

Sprint	TODO lines left	Change lead time (h)	Gated merges	Pulse
1	0	2.2	0 of 1	1.9
2	52	3.6	6 of 6	2.4
3	50	4.2	4 of 4	2.2
4	22	3.5	4 of 4	3
5	14	3.5	5 of 5	3.8
6	5	3.5	7 of 7	4.1

The first sprint shows no TODO lines because the drop had not been imported yet; the burn-down starts with the import in sprint 2. The one ungated merge in sprint 1 is the scaffold commit, made before the pipeline file existed. Everything after it was checked. The full scorecard, with the document-era baselines beside it, is Chapter 33, The twelve-week scorecard (p. 520).

31.11 What Jim told management

The briefing on 24 December (Chapter 35, The briefing (p. 532)) and the progress memo (Chapter 37, Writing it down: DoD-style memos from Vim (p. 545)) carried the twelve-week story. A covering note for the last quad of the year needs one sentence: *Baseline-0 is tagged, reproducible and CCB-approved; the model, the trace, the verification matrix and the data exchange table are on the model site, and they will stay current without anyone rebuilding them.*

31.12 Tools adopted this sprint, and the pain each removed

Table 31.2: Sprint 6's tools

Tool	The pain it removed	Where it is explained
The VCRM job	"which requirements are verified, and how?" answered by hand, once, before a review	Chapter 44, Verification as a closed loop (p. 613)
The data exchange table	the interface spreadsheet, typed from diagrams, out of date the day it was sent	Chapter 11, Seeing the model: diagrams from text (p. 158)
Release branch, CCB comment, tag, baseline job	baselines as copies of documents nobody could rebuild	Chapter 20, The pipeline, job by job (p. 302)
The nightly schedule and its restore	breakage found days late, by whoever ran the full check next	Chapter 19, CI/CD from zero (p. 288)
Secret detection, for real	a credential in a script, found by an audit months later	Chapter 48, Security: protect the environment, engineer it into the system (p. 651)

31.13 You can now…

Note: You can now…

- Cut a baseline through a release branch, record the CCB's concurrence on the merge request, tag it, and read the pipeline's proof that it rebuilds identically.
- Tell verify coverage (a case exists) from verification status (a verdict exists), and keep them apart in a briefing.
- Respond to a committed credential in the right order: revoke, replace, rewrite.
- Generate a document – the data exchange table – from the model, with its assumptions visible.
- Read a sprint plan and spot the task that cannot finish before the sprint starts.

31.14 Exercises

Exercise 31.1: Audit baseline-0

In the Tidewater sandbox, check out the `baseline-0` tag in Tom's clone, run `perl ci/job.pl baseline` with `CI_COMMIT_TAG=baseline-0` in the environment, and confirm the rebuild line. Then rename one part definition in a model file (for example `Thermometer` in `model/physical/stations/Stations.sysml`), run it again, and read what the job says. Why would changing a file under `docs/` instead not trip the audit? (Restore the tree afterwards with `git checkout -- .`.)

Exercise 31.2: Two measures, one slide

Write the two-line statement Jim could have put on a slide about verification at the end of sprint 6: one line for verify coverage, one for the VCRM, each saying what it measures, with

its number. Have someone outside the team read it and tell you what they think it means.

Exercise 31.3: Fix the plan, not the sprint

Rewrite TW-112 from the sprint 6 kickoff so that its first task can finish inside the sprint despite the leave. What would "done" mean for that first task, as a pipeline check?

Exercise 31.4: An assumption to the prime

Using the Discrepancy Report or Prime Drop template (Appendix I, Templates and GitLab Markdown, field by field (p. 747)), write the request that asks the prime to confirm or correct the "late = two missed transmissions" assumption in the data exchange table. Say what changes in the model if the prime disagrees.

31.15 Applying this to your project

- **End your first twelve weeks with a baseline.** A tagged, reproducible, CCB-approved version is the thing management can hold; everything else is a promise.
- **Put the CCB's approval where the engineering approvals are.** A comment on the release merge request is a record nobody can lose and everybody can find.
- **Introduce a new measure next to the old one, never in its place.** Name both; explain the difference once.
- **Make assumptions part of the generated documents.** An assumption in the model's doc text shows up in every table the pipeline builds, until somebody confirms it.
- **Plan around leave and holidays before the sprint, not at the review.** A carryover you could see on day one is a planning error, not bad luck.

PART IV

Making the case

Part III followed the team through twelve weeks, one sprint at a time. Part IV steps back and asks the question management asked on 24 December: was it worth it? The answer has to be built from measurements the team agreed before the work started, computed by the system rather than typed into slides, and honest about what did not move.

The chapters run in the order Jim prepared them. Chapter 32, Measuring with GitLab analytics (p. 513) shows the views GitLab offers for free and what each one says about a model team. Chapter 33, The twelve-week scorecard (p. 520) puts the twelve measures from Chapter 3, The directive (p. 22) side by side, before and after. Chapter 34, Proving the port is complete and correct (p. 527) answers the question every port must answer: is the v2 model complete and correct? Chapter 35, The briefing (p. 532) is the briefing itself, slide by slide, with the demo script. Chapter 36, Telling management (p. 538) is the weekly rhythm of reports that made the briefing a summary rather than a surprise. The last chapters of the part turn to the people: how the team changed its mind, a day in the life at week twelve, and how documents still matter.

Everything quoted here comes from `docs/tidewater/`, the evidence of the simulated run. Where this part says a number, it cites the file it came from.

CHAPTER

32

Measuring with GitLab analytics

From what you know →

You know

the metrics slide assembled the night before a review.

→ Here it is called

GitLab analytics, and gitlab.pl analytics.

→ What is new

the numbers come from the work itself, every day, and nobody counts by hand.

Survival kit: Before you start

What you have

twelve weeks of issues, merge requests and pipelines in one GitLab project, and the kit's journal of stories.

What you need

to know which of GitLab's analytics views answer a question a systems engineer or a program manager actually asks, which of them your license includes, and how to get the same numbers when it does not.

Jim's first instinct, when the program manager asked for "the metrics," was to open GitLab's Analytics menu and screenshot whatever was there. Tom talked Jim out of it. Half the menu was greyed out on their license tier, and the other half answered questions nobody on the program had asked. "Deployment frequency" sounds impressive until someone asks what Tidewater deploys. This chapter is the sorting Jim and Tom did in sprint 6: what each view measures, what it means for a team whose product is a model, and how to get the numbers without buying a tier you do not need.

32.1 Two kinds of numbers

The team keeps two records, and each one is good at different questions.

- **The journal** (the kit's `scrum.txt`, Chapter 23, Stand-ups, the journal and the board (p. 367)) holds stories: what was committed, done, carried, blocked, and how many points each was worth. It answers planning questions: are we delivering what we commit to, and how fast does the backlog shrink?
- **GitLab** holds tasks, merge requests, approvals, pipelines and tags. It answers flow and quality questions: how long does a change take from proposal to acceptance, how often does a check catch something, how long are we red?

Neither replaces the other. The scorecard in Chapter 33, The twelve-week scorecard (p. 520) takes its delivery and morale measures from the journal and the retrospectives, and its flow and quality measures from GitLab. This chapter is about the second kind.

32.2 The views GitLab offers

GitLab's analytics are spread over several pages, and which ones you see depends on the license tier. Tiers and the features in them change from release to release, so treat the tier column below as a guide and check GitLab's current pricing page before you plan around a view.

Table 32.1: GitLab's analytics views, as a systems engineer reads them.

View	Tier (check current pricing)	What it measures	The question it answers for a model team
Value stream analytics	Free (basic), Premium (custom stages)	time an issue spends in each stage, from created to closed	where do our changes wait: before work starts, in review, or after merge?
CI/CD analytics	Free	pipeline success rate and duration over time	do our checks run reliably, and how long does a change wait for them?
Repository analytics	Free	commits, languages, contributors	who is touching the model, and how often?
Merge request analytics	Premium	throughput, time to merge	how many changes did we accept, and how long did each take?
Code review analytics	Premium	open merge requests and their review activity	which proposed changes are stuck waiting for a reviewer?
Issue analytics	Premium	issues opened and closed per month	is the backlog of discrepancies and tasks growing or shrinking?
Contribution analytics	Premium (group level)	events per member	is the work spread across the team or resting on one person?
Milestone burndown and burnup	Premium	issues and weight closed against a milestone	are we on track for this sprint?
DORA metrics	Ultimate	deployment frequency, lead time, change failure rate, time to restore	how fast and how safely do changes flow?
Insights and analytics dashboards	Ultimate	charts you configure in YAML	anything above, on one page

The Tidewater project ran on the Free tier for the first twelve weeks; the tier decision was one of Jim's asks in the request for enablers (Chapter 37, Writing it down: DoD-style memos from Vim (p. 545)). That meant value stream, CI/CD and repository analytics in the web pages, and everything else from the API.

32.3 The Free-tier route: gitlab.pl analytics

Every number in the paid views is computed from data that every tier exposes through GitLab's REST API: merge requests with their commits, notes and approvals; issues with their labels and dates; pipelines and their jobs; deployments and releases. The kit's `gitlab.pl analytics` reads that data and prints the same five sections a manager would want from the paid pages. It works the same way against a real GitLab server and against the offline stand-in the book uses.

Here it is on the Tidewater project at the end of sprint 6. The sandbox is a copy of the simulated project, with its GitLab state in `gitlab/`:

```
perl "$KIT/bin/gitlab.pl" --local gitlab --project tidewater/model analytics --since 2026-12-14
--until 2026-12-24
```

The same report, as the team filed it with the sprint 6 evidence:

docs/tidewater/sprints/06/analytics.txt

```
GitLab analytics: tidewater/model, 2026-12-14T00:00:00Z .. 2026-12-24T23:59:59Z, by sprint

1. Value stream (hours; 7 issues closed by merge requests)
   stage                                n   median      p85
   issue created -> first commit        7     72.5       98
   first commit -> MR opened            7        0        0
   MR opened -> first review            7      3.3      4.8
   first review -> merged               7      0.2      0.2
   merged -> issue closed               7        0        0
   issue created -> issue closed        7       78      172

2. DORA, for a model repository
   deployment frequency              17 (10.82 a week)
   lead time (first commit -> merged on develop) median 3.5 h, p85 4.4 h
   change failure rate               0% (0 fix-forward or revert MRs / 7 merges)
   time to restore (develop red -> green) median 8.6 h, p85 8.6 h (1 restores)

3. CI/CD: 37 pipelines, success rate 89.2%, duration median 2.2 min, p85 3.7 min
   period                         runs   failed success%   median min
   Sprint 6                         37        4     89.2          2.2
   job failures: secrets 3/37, grammar 1/37, lint 1/37, outline 1/37, todo 1/37, trace 1/34

4. Merge requests: 8 opened, 8 merged; time to merge median 3.5 h (p85 5 h); review turnaround
median 3.3 h (p85 4.8 h)
   period                        merged ttm median h review median h
   Sprint 6                             8          3.5            3.3
   per author: bob 1/1, jim 1/1, tom 6/6 (merged/opened)

5. Issues: 15 open, age median 15.4 days, p85 34.3 days
   period                       type            opened  closed
```

```
Sprint 6                    defect              1       1
Sprint 6                    issue               3       2
Sprint 6                    story               5       7
Sprint 6                    task               10       9
oldest open: #84 (discrepancy, 34.3 d), #85 (discrepancy, 34.3 d), #86 (discrepancy, 34.3 d),
#87 (discrepancy, 34.3 d), #88 (discrepancy, 34.3 d)
```

Against a real server, the only change is the first two options: `--url https://gitlab.example.gov --token-env GITLAB_TOKEN` instead of `--local gitlab`. The token needs only the `read_api` scope.

32.4 Reading the five sections

Value stream. Each row is one stage of a change's life, with the median and the 85th percentile in hours. The stage that matters most for a model team is *MR opened -> first review*: that is the wait for a colleague's eyes, and it is where document-era reviews used to take weeks. In sprint 6 the median was 3.3 hours. The long stage at the top, *issue created -> first commit*, is not a problem: stories are filed at refinement, a week before work starts, so that stage measures planning lead, not delay.

DORA, for a model repository. The four DORA measures were defined for software teams that deploy services. A model team deploys too, just not to production servers. The table below is the translation the kit uses.

Table 32.2: The four DORA measures, translated for a model repository.

DORA measure	Software meaning	Model meaning in `gitlab.pl analytics`
Deployment frequency	releases to production	publishes of the model site (the `pages` environment) and baseline releases
Lead time for changes	commit to running in production	first commit on a branch to merged on `develop`
Change failure rate	deployments that caused an incident	merges that needed a fix-forward or a revert merge request
Time to restore	incident to recovery	`develop` pipeline red to the next green

Two of these belong in front of management and two do not. Lead time (3.5 hours median in sprint 6) is the direct answer to "how long does a change take now?", and time to restore (the worst case in sprint 6 was 8.5 hours, the night the runner's disk filled up) shows the safety net works. Deployment frequency is mostly the nightly schedule rebuilding the site, so it measures a cron line, not the team. Change failure rate was zero in the first twelve weeks, which says more about the small number of merges than about the team's skill; quote it only with its denominator.

CI/CD. Pipeline runs, failures and duration. The job failure line is the one to read: it says which checks caught something. In sprint 6 most failures were the `secrets` job, which is the planted test token in Bob's day in the life (Chapter 39, A day in the life, week twelve (p. 563)) doing exactly what it should.

Merge requests. Throughput, time to merge and review turnaround per sprint, and merged-over-opened per author. The per-author line is useful to the team and dangerous in a briefing: it reads as a productivity ranking, and in sprint 6 it mostly shows that Tom prepared the baseline and the release.

Issues. Opened and closed by type, and the oldest open items. The discrepancy reports to the prime sit at the top of the oldest list. That is correct: they are the prime's to close, with the next drop.

32.5 The trend across the twelve weeks

One sprint is a snapshot. The case for management is the trend, so run the report over the whole period, by sprint, and write the tables as CSV files for a spreadsheet:

```
perl "$KIT/bin/gitlab.pl" --local gitlab --project tidewater/model analytics --since 2026-10-05
--until 2026-12-24 --by sprint --csv analytics-csv
ls analytics-csv
```

The two flow measures the team agreed to report, taken from each sprint's scorecard:

Table 32.3: Flow measures by sprint, from the scorecard files.

Sprint	Lead time, median hours	Review turnaround, median hours	Changes gated
1	2.2	2	0 of 1
2	3.6	3.3	6 of 6
3	4.2	4.1	4 of 4
4	3.5	3.3	4 of 4
5	3.5	3.3	5 of 5
6	3.5	3.3	7 of 7

Read it the way the program manager will. Lead time and review turnaround are measured in hours from the first sprint, against a document-era baseline measured in working days and weeks. They do not improve much after sprint 2, and that is the honest finding: the gain came from the way of working, not from the team getting faster at it. The one ungated change is merge request !1 in sprint 1, made before the pipeline file existed; its description says so.

32.6 Small numbers and other traps

A three-person team produces a few merge requests a week. Statistics on numbers that small are fragile, and the team learned to say so before anyone else did.

- **Quote the count with the median.** "Median 3.5 hours over 7 merges" is honest; the median alone invites a comparison with a team of fifty.
- **Do not chase a measure.** If review turnaround becomes a target, reviewers approve faster and catch less. The team watched it as a symptom, never as a goal.
- **Watch the definitions.** The `talk.pl review` draft for sprint 6 reported "verify coverage" from the verification matrix, which counts requirements with a *passing recorded result*. The scorecard reports requirements with a *verification case*. The draft therefore showed coverage collapsing from seventy percent to almost nothing, the week the matrix job arrived. Tom caught it in the Day-9 rehearsal; Chapter 33, The twelve-week scorecard (p. 520) explains the two measures.

- **Separate the prime's numbers from the team's.** Open discrepancy reports and unverified requirements belong to the prime until a drop closes them. A government team that reports them as its own backlog looks stuck.

32.7 What goes in the briefing

Jim put three numbers from GitLab into the December briefing, and left the rest in the appendix:

1. **Lead time for a change**, with the document-era baseline beside it.
2. **Changes gated**: every change since sprint 2 passed the checks and a code owner's review.
3. **Pipeline health**: share of `develop` pipelines passing, and the worst time to restore.

Value stream stages, DORA deployment frequency, per-author counts and issue ages stayed in the team's own review. The reasoning is in Chapter 35, The briefing (p. 532): management can act on three numbers with a trend; it cannot act on twenty.

32.8 How this relates to the kit's other measures

The kit already had measures before GitLab arrived. `daily.pl health` checks the journal against the thresholds of the operating model (blocked items, load, velocity, carryover). The weekly quad (Chapter 36, Telling management (p. 538)) reports sprint progress and on-time delivery. `bin/status-metrics.pl` computes the A to Z status metrics of a port from a model repository's git history. GitLab analytics adds the flow view on top: the same project seen through its merge requests and pipelines. When two sources disagree, believe the one that is closest to the work, and find out why.

Exercise 32.1: Run the report yourself

In the Tidewater sandbox, run `gitlab.pl analytics` for sprint 3 alone (2 to 13 November). Which stage of the value stream is longest, and why is that not a problem? Then run it for sprint 6 with `--by week`.

Exercise 32.2: Translate a DORA measure

Write one sentence a program manager would understand for each of the four DORA measures, using the model meanings in the table above. Which two would you leave out of a briefing, and why?

Exercise 32.3: Find the trap

Open `docs/tidewater/sprints/06/review-script.md` and `docs/tidewater/sprints/06/review.txt`. The draft and the final version disagree about verify coverage. Which one is right, and what would have happened if the draft had been read out?

32.9 You can now

- name the GitLab analytics views, what each measures and roughly which tier it needs;

- get the same numbers on the Free tier with `gitlab.pl analytics`, by sprint, with CSV output;
- translate the DORA measures into model terms, and say which belong in front of management;
- spot the traps of small numbers and mismatched definitions before management does.

32.10 Applying this to your project

Decide the three flow numbers you will report before the first sprint, write them into your scorecard, and run `gitlab.pl analytics --by sprint` at every sprint end into the sprint's evidence folder, as Tidewater did. Do not buy a tier for its dashboards until you have used the Free-tier report for a few sprints and know which questions you keep asking. If your license includes the paid views, use them for the team's own retrospectives and keep the briefing to the few numbers you defined in advance.

CHAPTER

33

The twelve-week scorecard

From what you know →

You know

the stoplight chart.

→ Here it is called

the twelve-week scorecard.

→ What is new

every measure was agreed in week zero and is computed from the evidence, including the one that did not move.

Survival kit: Before you start

What you have

the twelve measures the program manager signed on 1 October (Section 3.6, Defining measurable progress (p. 28)), and a scorecard file the pipeline and the journal filled in at the end of every sprint.

What you need

to read the scorecard the way a skeptical manager will: measure by measure, before against after, including the measures that did not move.

The scorecard was the team's insurance. Bob had said it in the first week: "If we don't write down now what counts as progress, in December they'll judge us on whatever they happen to look at." So the measures were agreed before sprint 1, each with a definition, a source and a target, and each sprint's numbers were written by the system into `docs/tidewater/sprints/NN/scorecard.json`. Nobody typed them. This chapter reads the last of those files against the first, measure by measure.

33.1 The whole scorecard on one page

Table 33.1: The twelve-week scorecard, sprint 1 against sprint 6. “Before” values are notional, from the program’s last document-era review cycle.

#	Measure	Target	Before (document era, notional)	Sprint 1	Sprint 6	Met?
1	Conversion debt: TODO lines left	structure closed, behavior started	not applicable	0 (nothing imported yet)	5	yes
2	Change lead time, median hours	under 3 working days	23 working days	2.2	3.5	yes
3	Gated changes	100%	not measured	0 of 1	7 of 7	yes, from sprint 2
4	Trace answer time, seconds	under one minute	2 working days	not yet	24	yes
5	Trace measures	computed on every change, every gap owned	computed at reviews only	not yet	linked 29 of 30, completeness 66.7%	yes (computed); the gaps are the prime’s
6	Discrepancy reports to the prime	every gap reported	ad hoc emails	0	7 opened, 0 closed	yes
7	Spider web: package cycles	none	see Chapter 2, The Cameo years (p. 10)	not yet	1	no
8	Pipeline health: passing on develop	above 90%, restore within a day	not applicable	not yet	16 of 17, worst restore 8.5 h	yes
9	Reproducible baselines	interim baseline tagged and reproducible	revision letters	0	1 of 1	yes
10	Review turnaround, median hours	under 1 working day	3 weeks per review package	2	3.3	yes
11	Delivery against plan	80% or better	not measured	100%	79%	partly: two sprints below
12	Team morale (pulse, 1 to 5)	3.5 or better, rising	not measured	1.9	4.1	yes, from sprint 5

Ten measures met their target, one partly, one not. The rest of this chapter takes them in turn, starting with the ones a skeptic will ask about first.

To see the source for yourself, open the last scorecard file in the Tidewater sandbox:

```
cat sprints/06/scorecard.json
```

33.2 Conversion debt: the TODO burn-down

The converter leaves a `// TODO` line for every v1 construct it cannot carry. Counting those lines on every change was the team’s port burn-down (Chapter 20, The pipeline, job by job (p. 302)). Nothing was imported in sprint 1, so the first count arrived with drop 1 in sprint 2:

Table 33.2: TODO lines left on develop, from the pipeline's todo job.

After sprint	TODO lines left
2	52
3	50
4	22
5	14
6	5

The steep fall in sprint 4 is the Stations package: most of its level tags closed in one story. The lines left at the end are behavior and analysis, the constructs the converter never carries: the BuoyModes state machine (carried into sprint 7), the energy-balance expression, and the profile note. You can count them yourself on the integration branch:

```
git clone -q -b develop gitlab/repos/tidewater/model.git port
grep -rh "// TODO" port/model --include=*.sysml | grep -vc "generated by"
```

33.3 Flow: lead time, review and gating

These three are the measures that changed the program manager's mind, because the document-era numbers beside them are measured in working days and weeks. A requirement change used to take 23 working days through the CCB; a model change now goes from the first commit to merged in a median of 3.5 hours, and every one of them was checked by the pipeline and approved by a code owner who did not write it. Chapter 32, Measuring with GitLab analytics (p. 513) shows the sprint-by-sprint trend.

Two cautions keep this honest. First, the comparison is not like for like: a CCB change carried a requirement through several organizations, while a merge request carries one model change through one team. The fair claim is that the team's own part of the work is now measured in hours. Second, the baseline numbers are notional, taken from the program's last document-era cycle as the team remembered it; they were never measured the way the new numbers are. The scorecard says so in its own file, under `document_era_baseline_notional`.

33.4 Trace: computed every time, and flat

The trace job compares the prime's DOORS export with the model on every change. Its measures did not move in twelve weeks, and they were not supposed to.

Table 33.3: Trace measures at the end of sprint 6, identical to sprint 2.

Measure	Value at sprint 6
C: requirements linked	29 of 30
F: orphans (nothing satisfies them)	2
G: unverified (no verification case)	9
I: completeness	66.7%
Text drift between DOORS and the model	1

The gaps were planted in the prime's model before the government touched it: two requirements nothing satisfies, a set without verification cases, one requirement whose DOORS wording differs from the model. They are the prime's to fix, through discrepancy reports, and the drops that close them arrive from January. A government team that edited the v2 model to close them would have made its model disagree with the contractual baseline. So the right twelve-week result is not "coverage went up"; it is "every gap is found by a program on every change, and every gap has a discrepancy report with an owner and a due drop." That is what measure 5 asked for.

Warning: Two different "verify coverage" numbers

The scorecard counts requirements that have a *verification case* (G above: the rest are covered, which is the seventy percent the reviews quote). The verification matrix, new in the last sprint, also counts requirements with a *passing recorded result*, and on that definition almost none qualify yet (the "Verification" line of the baseline-0 release notes gives the count): the prime's test results only start arriving with drop 4. The `talk.pl review` draft for sprint 6 quoted the matrix number as if it were the scorecard's and announced a collapse. Name the definition every time you say "coverage."

33.5 Discrepancy reports

By the end of sprint 6 the team had opened 7 discrepancy reports to the prime and closed 0. Closed zero sounds bad and is correct: a discrepancy report closes when a prime drop fixes the problem, and the first drop after the port started arrives in January (Chapter 45, The prime's model changes: receive, assess, disposition (p. 619)). What management should see is that every gap the trace found became a written, numbered, dated request to the prime within a sprint, where in the document era such gaps went into an email, if anywhere.

33.6 The spider web: better, not gone

Measure 7 is the one the team did not meet, and the briefing said so.

Table 33.4: Spider-web measures of the v2 model, from the pipeline's complexity job.

After sprint	Package cycles	Duplicates	Max fan-out	Max relationships per part
2	1	0	4	19
3	2	29	3	19
4	1	29	3	19
5	1	29	3	8
6	1	29	3	8

Three things happened. The worst block, ObservingStation, lost its trace spaghetti in sprint 5: the maximum relationships per part fell to less than half. Sprint 4 attacked both package cycles from Chapter 2, The Cameo years (p. 10). The Network and Operations cycle is gone. The DataCenter and Stations cycle is not: the story removed the dependency it was written against (QualityControl on SensorSuite), and the complexity job still reports `cycles: DataCenter <-> Stations`, so another reference between the two packages remains. The team reported that at the review rather than declaring the story's goal met, and finding the remaining edge went into the

sprint 7 backlog. And the "duplicates" column jumped in sprint 3, when the DOORS modules were imported: the model now holds each requirement twice, once as the Cameo copy and once as the DOORS module's conversion. That doubling is deliberate (DOORS is authoritative, and the trace compares the two), but the measure cannot tell a deliberate copy from clutter. The team reported the number as it was and explained it, rather than redefining the measure to make the column look better. Chapter 47, Keeping SysML v2 from becoming a spider web (p. 645) covers what it would take to finish.

33.7 Pipeline health and baselines

In sprint 6, 16 of 17 pipelines on `develop` passed. The one failure was the night the runner's disk filled up; the worst time from red to green was 8.5 hours, inside the one working day the scorecard allowed. The interim baseline-0 was tagged on 24 December, and the pipeline's baseline job rebuilt it and matched the checksum recorded in the tag. In the document era a baseline was a revision letter on a cover page, and nobody could rebuild one.

33.8 Delivery and morale

Delivery against plan was the measure that kept the team honest about its own estimates:

Table 33.5: Delivery and morale by sprint, from the journal and the retrospectives.

Sprint	Committed points	Done points	Done %	Pulse (1 to 5)
1	16.5	16.5	100	1.9
2	19	19	100	2.4
3	15	15	100	2.2
4	17	12	71	3
5	18.5	18.5	100	3.8
6	14	11	79	4.1

Two sprints fell below the eighty percent target, and both for the same reason: a story carried over. In sprint 4 the Stations walk-through moved to sprint 5; in sprint 6 the hand-written BuoyModes state machine moved to sprint 7. Both were the hardest engineering of their sprint, and in both cases the team chose to finish the work properly rather than to call it done. The journal records the carry; the quad marks the story as late.

The pulse survey asked the same five questions at every retrospective. It started at 1.9, dipped to 2.2 in sprint 3 (a grammar error on a hand edit and a three-day wait for the ISSM, the same week), and ended at 4.1. Chapter 38, How the team changed its mind (p. 557) tells that story; the scorecard only says it was measured every sprint, by the same questions, and reported even when it was low.

33.9 Measures beyond the twelve

The sprint files carry three more numbers that the team found useful, though they were not on the signed scorecard:

- **Work mix.** The share of done points that was engineering, platform or learning, by sprint. It opens this book (Bottom line up front (p. iii)): engineering grew from a quarter of sprint 1 to most of sprint 6.
- **Stand-up outcome share.** The share of stand-up lines that stated an outcome and a number. It went from 0.48 in sprint 1 to 1 from sprint 3 on (Chapter 25, Talking about progress: helps for engineers who would rather not (p. 390)). Sprint 2 scored lower than sprint 1 because sprint 1's statuses happened to quote counts while sprint 2's described mechanism.
- **First-pipeline pass rate.** How many merge requests passed their checks on the first try: 4 of 6 in sprint 6. A failure on the first pipeline is the system working; the measure shows how much the checks are doing.

33.10 What the scorecard does not say

A scorecard is a set of chosen numbers. This one does not measure the quality of the engineering decisions (whether the cycle break put ObservationRecord in the right package), it does not compare cost with the document era, and its "before" column is memory, not measurement. The team said so in the briefing. A scorecard that admits its limits is easier to believe than one that does not.

Exercise 33.1: Read it like a skeptic

Pick the three measures you would challenge first if you were the program manager. For each, write the question you would ask and find the evidence file that answers it.

Exercise 33.2: The measure that missed

Measure 7 was not met. Draft the two sentences Jim should say about it in the briefing: what happened, and what the plan is. Compare with Chapter 35, The briefing (p. 532).

Exercise 33.3: Your own baseline

For your program, write down the document-era equivalent of measures 2, 4 and 10, with where the number comes from. Mark each one "measured" or "remembered."

33.11 You can now

- read a sprint's scorecard file and trace each value to the job or record that produced it;
- explain why trace coverage stays flat during a port, and why that is the right result;
- separate a measure that missed for a good reason from one that missed for a bad one, and say which is which;
- present a scorecard that includes its misses and its limits.

33.12 Applying this to your project

Write your scorecard before sprint 1, with a definition, a source and a target for each measure, and have the program manager sign it. Make the pipeline and the journal write it at every sprint end, so no number is typed. Include at least one measure you might miss, and one that measures

the team rather than the model. Keep the “before” column honest: label remembered numbers as notional, and replace them with measured ones the first time you can.

CHAPTER

34

Proving the port is complete and correct

From what you know →

You know

the configuration audit, and the sign-off on a data conversion.

→ Here it is called

proving the port: counts, the TODO burn-down, the trace against DOORS and a sampled review.

→ What is new

most of the evidence is produced by scripts anyone can rerun.

Survival kit: Before you start

What you have

the prime's drop 1 (the v1 model and the DOORS export), the converted v2 model on `develop`, and the pipeline's reports.

What you need

an answer to the first question management asks about any port, in a form an independent reviewer can check: *is the new model complete, and does it say the same thing as the old one?*

The deputy program manager put the question plainly at the sprint 4 review: "How do I know nothing fell on the floor between the Cameo model and the new one?" In the document era the answer would have been a person's signature on a comparison spreadsheet. The team's answer was a set of evidence a reviewer can regenerate: counts the converter produces, a burn-down the pipeline keeps, a trace a program computes, a sample a person checks, and a baseline anyone can rebuild. This chapter lays out that evidence, shows where Tidewater stood at the end of the twelve weeks, and says what is still missing before the port can be accepted at baseline-1.

34.1 Complete, and correct

The question has two halves, and they need different evidence.

- **Complete**: every element of the v1 model that matters is either in the v2 text or accounted for. Nothing was silently dropped. This is mostly counting.
- **Correct**: what the v2 model says matches what the v1 model and DOORS say. Names, structure, relationships and requirement text agree. This is counting plus comparison plus a human look.

A port is acceptable when both halves are evidenced and someone with the authority to accept it has signed. For Tidewater that someone is the deputy program manager, through a concurrence comment on the `baseline-1` release merge request (Chapter 43, Maintenance through the life cycle (p. 602)).

34.2 Evidence 1: the converter accounts for every element

The converter's report lists every v1 element type it read, how many it mapped to v2 text, how many it folded into a parent, and how many it left as a TODO line. Nothing is dropped without a line in that table. Run it on drop 1 in the Tidewater sandbox:

```
perl "$KIT/bin/xmi2sysml.pl" --report --layout-map "$KIT/examples/tidewater-v1/layout.map" --map
map.csv --out v1check drops/v1.0/tidewater.xmi
```

Read the totals row and the line below it: every in-scope element (classes, ports, properties, connectors and activities, the filter the status metrics use) was ported. The `--map` option also wrote `map.csv`, one row per v1 element with the v2 name, kind, file and line it became. That map is the backbone of the rest of the evidence:

```
awk -F, 'NR > 1 { print $5 }' map.csv | sort | uniq -c | sort -rn
```

Each v2 kind is a count a reviewer can compare with the v1 side of the report: requirements in, requirements out; part definitions in, part definitions out. A difference must be explained by a fold (a v1 element that became part of another v2 element) or a TODO line.

34.3 Evidence 2: the TODO burn-down reaches zero

Every TODO line is a v1 construct the converter could not carry, waiting for a person. A complete port has none. Tidewater's count, from the pipeline's `todo` job at the end of each sprint, stood at 5 at the end of sprint 6. Count it on the integration branch:

```
git clone -q -b develop gitlab/repos/tidewater/model.git port
cd port && perl ci/job.pl todo
```

The remaining lines are behavior and analysis: the BuoyModes state machine (carried into sprint 7) and the energy-balance expression, both written by hand. Acceptance waits for zero.

34.4 Evidence 3: the trace agrees with DOORS

DOORS is authoritative for requirement text. The trace job compares the prime's DOORS export with the requirements in the model on every change, leaving out the DOORS-generated packages so the model is not compared with itself:

```
perl ci/job.pl trace
```

Four numbers and two lists come out. *Linked* says how many DOORS requirements the model carries (29 of 30 at the end of sprint 6: the missing one exists only in DOORS and is a discrepancy report). *Orphans* and *unverified* are gaps in the prime's model, also reported. The *drift* list names requirements whose text differs between DOORS and the model, with both versions printed. For the port, the rule is simple: every number must be either identical to the v1 model's own trace, or explained by a discrepancy report. A port that changed the trace has changed the meaning.

34.5 Evidence 4: a sampled side-by-side review

Counts prove nothing was lost; they do not prove nothing was garbled. For that, a person compares a sample of elements in both forms. The team picked every twenty-fifth row of the map, which is reproducible (anyone running the same command gets the same sample) and spreads across packages:

```
perl -ne 'next if $. == 1; my @f = split /,/; print if $f[4] =~ /^(part
def|requirement|port|connection|action def)/ && $. % 25 == 0' map.csv
```

For each sampled row the reviewer opens the v1 element in the XMI and the v2 text at the file and line the map gives, and records "same", "same, reworded", or "different" in the review issue. One of the sample, side by side:

```
grep -n 'name="WaveSensor"' drops/v1.0/tidewater.xmi | head -3
grep -n -A2 "part def WaveSensor" port/model/physical/stations/Stations.sysml
```

At the end of sprint 6 the sample had been drawn but not reviewed: it is a story for the semester that ends with baseline-1, done by Jim and the prime's modeler together, so the prime agrees with the result.

34.6 Evidence 5: the views say the same thing

The v1 model was mostly diagrams. The v2 model regenerates its views on every change (Chapter 11, Seeing the model: diagrams from text (p. 158)): the part tree, the requirements trace, the

interconnection view and the package view. A reviewer who knows the old diagrams compares them with the generated ones for the same packages. This is a quick check that catches a wrong parent or a missing connection that the counts would not. It is also the check that convinces people who learned the system from diagrams.

34.7 Evidence 6: the baseline rebuilds

The accepted port is a baseline, and a baseline is only evidence if it can be rebuilt. The interim baseline-0 of sprint 6 records the checksum of its generated documents in the tag:

```
git -C port tag -n3 baseline-0
```

The pipeline's `baseline` job regenerates the documents from the tagged model and compares the checksum, and writes the toolchain inventory beside it (Chapter 20, The pipeline, job by job (p. 302)). The baseline-0 audit passed: the scorecard shows 1 reproducible of 1 tagged.

34.8 Where Tidewater stood at twelve weeks

Table 34.1: Port acceptance evidence, interim status after twelve weeks.

Evidence	Status at the end of sprint 6	Needed for acceptance
Converter accounts for every element	in-scope elements all ported; every fold and TODO listed	done
TODO burn-down	5 lines left (behavior, analysis)	zero
Trace against DOORS	computed on every change; gaps are discrepancy reports to the prime	unchanged, or explained
Drift list	one requirement, reported	reported and dispositioned
Sampled review	sample drawn, review not yet done	recorded in an issue, with the prime's modeler
Views compared	views generated on every change	reviewer's note in the same issue
Baseline rebuilds	baseline-0 reproducible	baseline-1 reproducible
Sign-off	not yet	deputy PM's concurrence on the `baseline-1` release merge request

The completeness report that ties these together, with every count and the sign-off in one document, is produced at `baseline-1` in sprint 20, when the TODO count reaches zero and the sampled review is done. That is beyond the twelve weeks this part covers; Chapter 43, Maintenance through the life cycle (p. 602) describes it.

Checklist: Port acceptance

- ☐ The converter report shows every in-scope element ported, and every fold and TODO accounted for
- ☐ The TODO count on the integration branch is zero
- ☐ The trace numbers equal the v1 model's own trace, or each difference has a discrepancy

report
- ☐ The drift list is empty, or each drifted requirement has a disposition
- ☐ A reproducible sample of elements has been compared side by side, and the result recorded
- ☐ The generated views have been compared with the v1 diagrams of the same packages
- ☐ The baseline tag rebuilds to its recorded checksum, with the toolchain inventory kept
- ☐ The accepting authority has concurred, in writing, on the release merge request

Exercise 34.1: Draw your own sample

In the Tidewater sandbox, change the sampling rule to every fifteenth row and include `satisfy` rows. Pick two sampled elements and compare them side by side as the chapter does. Which are "same", and which "same, reworded"?

Exercise 34.2: Explain a difference

The converter report shows elements that were *folded*. Find two in `map.csv` (kind `folded`) and write one sentence each on why folding them is not a loss.

34.9 You can now

- produce the counts that show a port lost nothing, from the converter's report and its map;
- show that the port did not change the meaning of the requirements, with the trace and the drift list;
- draw a reproducible sample for a side-by-side review and record the result;
- list what is still missing before a port can be accepted, and who signs.

34.10 Applying this to your project

Agree the acceptance evidence with whoever will accept the port before you start converting, and write it into the scorecard or the definition of done for the port epic. Keep the converter's report and map for every drop in the repository, so the counts can be regenerated. Make the sample reproducible, and do the review with someone from the organization that owns the old model. Do not call the port done until the TODO count is zero and someone with authority has said so in writing.

CHAPTER

35

The briefing

From what you know →

You know

the program review briefing.

→ Here it is called

the twelve-week briefing.

→ What is new

every number on it traces to a file in the repository, and the demo is live.

Survival kit: Before you start

What you have

six sprints of evidence, the scorecard (Chapter 33, The twelve-week scorecard (p. 520)), and twenty minutes on the program management review agenda on 24 December.

What you need

a briefing that a program manager who has never opened GitLab can follow, that leads with conclusions, shows the real thing working, admits what did not work, and ends with an ask.

The directive of September promised management a briefing "at the December PMR." For twelve weeks that line hung over the team. Bob's fear, said out loud in week one, was that the briefing would be judged on how polished the slides were, the way briefings always had been. Jim's answer was to build the briefing the same way the team now built everything else: from the evidence, rehearsed, reviewed, with as few slides as possible and one live demonstration. This chapter is that briefing, slide by slide, with the reasoning behind each choice.

35.1 Preparing it

The preparation took one story in sprint 6 ("Retrospective, the management briefing and the plan for next semester," two points, Jim) and used the same rhythm as every sprint review since

sprint 4 (Chapter 25, Talking about progress: helps for engineers who would rather not (p. 390)):

1. **Draft from the system.** `talk.pl review` drafts a demo script from GitLab and the journal: the question the sprint answered, the four headline numbers with their change, what to show, a before-and-after, a safe live change and a place for the ask.
2. **Edit for the audience.** The draft is a starting point. Jim cut it to what the program manager asks about and added the document-era comparison.
3. **Rehearse on Day 9.** Bob and Tom played the program manager and the deputy and interrupted with the questions they expected. The rehearsal caught the one mistake that would have hurt most (below).
4. **Write the twin.** The briefing has a written twin, the twelve-week progress memo (Chapter 37, Writing it down: DoD-style memos from Vim (p. 545)), so the numbers survive after the meeting.

Here is the draft the system produced for sprint 6, run in the Tidewater sandbox:

```
perl "$KIT/tools/gitlab/talk.pl" --local gitlab --project tidewater/model review --sprint 6
```

Warning: The number that nearly went to management

The draft's second section says verify coverage fell by more than sixty points. It did not. The draft took "coverage" from the new verification matrix, which counts requirements with a *passing recorded result*, while every earlier review had used the trace's count of requirements with a *verification case*. On the trace's definition coverage was unchanged at seventy percent. Tom spotted it in the rehearsal because the number looked too dramatic to be true. The final script (Chapter 33, The twelve-week scorecard (p. 520) explains both measures) used the trace's number and said which definition it was. Read every drafted number before you say it.

35.2 The briefing, as given

Jim's outline, as filed with the sprint 6 evidence:

docs/tidewater/sprints/06/briefing.txt

```
Tidewater: twelve weeks of MBSE in GitLab -- briefing to program management, 2026-12-24

1. Where we started: documents, then a Cameo model nobody could merge or review (47 diagrams, 2
package cycles).
2. What we built: a GitLab project, a pipeline that checks every change, the model in SysML v2
text, a model site.
3. The scorecard, sprint by sprint:

   sprint  TODO lines  lead time (h)  gated        morale
   1       0           2.2           0 of 1       1.9
   2       52          3.6           6 of 6       2.4
   3       50          4.2           4 of 4       2.2
   4       22          3.5           4 of 4       3
   5       14          3.5           5 of 5       3.8
```

```
   6      5          3.5           7 of 7      4.1

4. Before and after (document-era numbers are notional): a requirement change took 23 working
days through the CCB;
   a model change now takes hours from MR to merge, reviewed and checked.
5. The demo: open an MR, watch the pipeline, see the trace and the VCRM on the site.
6. Next semester: drops 2 and 3, behavior, baseline 1 in sprint 20.

The written twin of this briefing is the twelve-week progress memo
(docs/tidewater/memos/2026-12-24-twelve-week-progress.md).
```

Six points, one demonstration, one ask. The sections below take them in order.

35.3 Slide 1: where we started

Headline: "We started with documents, then a Cameo model nobody could merge or review."

One slide, two numbers from the drop-1 assessment of Chapter 2, The Cameo years (p. 10): the diagram count and the package cycles. Jim did not criticize the Cameo work; the people in the room had paid for it, and Bob and Tom had built it. Jim said what it could not do: two people could not change it at the same time and merge the result, a review meant screenshots in a briefing, and nobody could say which version of a diagram was current. The point of the slide is to name the problem the next five slides solve.

35.4 Slide 2: what we built

Headline: "Every change to the model is now checked and reviewed before it is accepted."

A picture, not a list: the flow from a proposed change through the pipeline's checks and a code owner's review to the model site. Jim named four things and no tool names: a project where the model lives, a set of automatic checks, the model itself in SysML v2 text, and a web site that management can open on a phone. The vocabulary on the slide was systems engineering vocabulary (Chapter 5, DevSecOps in systems-engineering words (p. 46)): "configuration control," "peer review," "audit," "baseline."

35.5 Slide 3: the scorecard, sprint by sprint

Headline: "The measures you signed in October, computed by the system."

The table on the slide is the one in the outline above: TODO lines, lead time, gated changes and morale for each of the six sprints. Jim chose four columns because each one answers a question the program manager had asked during the twelve weeks:

- *TODO lines*: "How much of the port is left?" From 52 after drop 1 to 5.
- *Lead time*: "How long does a change take now?" A median of 3.5 hours in sprint 6.
- *Gated*: "Is anything getting in unchecked?" Every change since sprint 2: 7 of 7 in sprint 6.
- *Morale*: "Is the team going to quit?" The program manager had not asked this out loud. Jim put it on the slide anyway, dip and all, because the dip in sprint 3 was the most believable number in the table.

The full twelve-measure scorecard went in the handout, with the one measure that missed (Section 33.6, The spider web: better, not gone (p. 523)) explained in a sentence: "We broke one of the two package cycles; the other is still there, and finding its last edge is in the January backlog."

35.6 Slide 4: before and after

Headline: "A requirement change took weeks. A model change takes hours, reviewed and checked."

The comparison management had asked for, with its honesty built in. The document-era number, 23 working days through the CCB, is labelled notional on the slide, and Jim said why: it is the program's memory of its last document-era cycle, not a measurement. Jim also said what the comparison does not prove: a CCB change crossed several organizations, a merge request crosses one team. The claim Jim made was narrower and true: the government team's part of a change is now measured in hours, and every one is reviewed and checked.

35.7 Slide 5: the demonstration

Headline: "Watch the model catch a mistake."

The demonstration followed the review moves the team had practised since sprint 4: start from the question, show the output and not the tool, make one safe live change, and say what it means. The live change was the same one in every rehearsal: on a scratch branch, never on `develop`, delete one line that says a part satisfies a requirement, and let the trace check name the requirement that lost its design.

Rehearse it yourself in the sandbox. First, a scratch copy of the integration branch and the deletion:

```
git clone -q -b develop gitlab/repos/tidewater/model.git demo
cd demo && git checkout -q -b scratch/demo-satisfy
grep -n "satisfy TW_SYS_003" model/requirements/model/Requirements.sysml
```

Then remove the line, commit, and run the trace check the way a merge request pipeline runs it:

```
sed -i '/satisfy TW_SYS_003/d' model/requirements/model/Requirements.sysml
git commit -qam "demo: drop one satisfy"
CI_PIPELINE_SOURCE=merge_request_event CI_MERGE_REQUEST_TARGET_BRANCH_NAME=develop perl ci/job.pl
trace || echo "exit status $?"
```

The last line of the output is the whole demonstration: "this change adds trace gaps," naming the requirement. In the meeting Jim opened the draft merge request on the projector, the pipeline turned red in about two minutes, and clicked through to the line, then closed the merge request. The program manager's question was the one Jim had hoped for: "And that runs on every change?" It does.

The model site came second: the part tree, the trace report and the data exchange table Tom had generated in sprint 6, all from the same model, all current as of that morning's nightly pipeline.

35.8 Slide 6: next semester, and the ask

Headline: "January: the prime's first new drop. We will show what it changes within a day."

The next milestones in one line each: the prime's drops of January and April, the behavior modeling the converter cannot carry, and the first full baseline in sprint 20. Then the ask, with dates, carried over from the request for enablers of October: approval of a second runner machine, and the GitLab tier decision, both by 15 January. An ask without a date is a hope; an ask with a date is a decision management can make in the room.

35.9 What management asked, and how the numbers answered

The questions came in the order the team had predicted in the rehearsal, with one surprise.

Table 35.1: Questions at the December briefing.

Question	Who asked	The answer, and its source
"How do I know nothing fell on the floor in the port?"	deputy PM	the converter's counts and the trace, every change; the sampled review is next semester (Chapter 34, Proving the port is complete and correct (p. 527))
"What happens when the prime sends the next version?"	program manager	a vendor branch, a change report, an impact report, and one issue per change for the CCB (Chapter 45, The prime's model changes: receive, assess, disposition (p. 619))
"Is this approved to hold program data?"	ISSM	the runner and repository were reviewed in sprints 2 to 5; markings are decided before real content (Chapter 48, Security: protect the environment, engineer it into the system (p. 651))
"Can I see this without asking you?"	program manager	the model site, rebuilt nightly, and the weekly note (Chapter 36, Telling management (p. 538))

The surprise was the last comment, recorded in the sprint 6 quotes: the program manager's "Keep doing this. Show me the same numbers in March." Jim's reading of it, at the retrospective an hour later, was that the scorecard had become the program's, not the team's.

35.10 What the briefing left out

It left out the tools. Nobody on the slides said "GitLab," "pipeline," "YAML" or "merge request" without a plain translation beside it. It left out the per-person numbers, which read as a ranking. It left out DORA measures and value stream stages, which are for the team's own review (Chapter 32, Measuring with GitLab analytics (p. 513)). And it left out the parts of the story that belonged to the team: the night the runner's disk filled up was one line on the scorecard, not a slide.

Exercise 35.1: Rewrite a slide headline

Slide 2's first draft headline was "Our DevSecOps toolchain." Write three alternatives that state a conclusion, and pick the one a program manager would remember.

Exercise 35.2: Rehearse the demo

Run the two demo blocks above in the sandbox, then change the deletion to a `verify` line instead of a `satisfy` line. What does the trace check say now, and how would you say it in one sentence to the program manager?

Exercise 35.3: Find your ask

Write the ask for your own first management briefing: one decision, one date, and what happens if it is not made.

35.11 You can now

- build a management briefing from evidence, with a conclusion as every headline;
- draft a review script with `talk.pl review`, then check every number it drafted before you say it;
- run a safe live demonstration on a scratch branch that shows the checks catching a real mistake;
- answer the predictable questions with a source for each answer, and close with a dated ask.

35.12 Applying this to your project

Agree the date of the first management briefing before sprint 1, and work backwards: the scorecard it will show, the evidence each number comes from, the demonstration you will rehearse. Keep it to a handful of slides with conclusions as headlines, put the full scorecard in a handout, and rehearse with someone playing the skeptic. Show one thing working live. Say what did not work. End with an ask that has a date.

CHAPTER

36

Telling management

From what you know →

You know

the weekly status report and the quad chart.

→ Here it is called

the weekly quad, the sprint mail and the weekly note.

→ What is new

they are drafted from the journal and GitLab, so writing them takes minutes.

Survival kit: Before you start

What you have

a journal of stories, a GitLab project, and a program manager who will not open either.

What you need

a rhythm of short reports, each built by the system from the same records, so that management sees progress every week and the December briefing is a summary rather than a surprise.

The briefing in Chapter 35, The briefing (p. 532) worked because nothing in it was new to the program manager. Every number on its slides had appeared, a few at a time, in reports that arrived every Friday and every second Friday for twelve weeks. This chapter is about that rhythm: what each report is, what is in it, how it is produced, and how the team learned to write the parts a program cannot produce.

36.1 Three rhythms

Jim agreed three kinds of report with the program manager in week zero (Section 3.10, What management needed to hear, and when (p. 32)):

Table 36.1: The reporting rhythm agreed in week zero.

Report	When	What is in it	Built from
Weekly note and quad	every Friday	one paragraph of headline, three numbers, risks, asks; the one-page quad behind it	the journal (`daily.pl quad`), GitLab (`talk.pl weekly`)
Sprint review	every second Friday	what was finished, shown live; the four headline numbers; the ask	GitLab, the journal, the pipeline; the demo script
Program review	end of sprint 6, then quarterly	the scorecard, before and after, a demonstration, the ask	everything above (Chapter 35, The briefing (p. 532))

The rule behind all three: the system produces the numbers, a person writes the sentences. Nobody types a count into a report.

36.2 The weekly quad

The quad is a one-page weekly status the program already used before Tidewater, in a slide template the team's lab had used for years. The template, *Quad Explanation*, describes four quadrants under a "Weekly Status" title with the lab's name and an "As of" date. The kit's `daily.pl quad` produces the same page from the journal. Here is the template's meaning, element by element, and where each element comes from in the kit.

36.2.1 The four quadrants

Table 36.2: The quad's quadrants, the template and the kit.

Quadrant	What the template says it holds	What the kit puts there
Technical Priorities	the current sprint backlog: every task, tagged with its state	this sprint's stories from the journal, each with a four-letter word, points and owner
Watch Items / PM Help Needed	tasks with a blocker or that could not be completed on time, marked for the program manager	blockers older than three days as (PM), fresh blockers, holds, carryovers and unassigned work as (WI)
Schedule Milestones	the 30-60-90 day backlog: what is due 30, 60 and 90 days out, priority 1 to 3	epics whose forecast finish falls in each window, with progress and the forecast date
Accomplishments	items from the DONE list, marked on time or late	stories done this week, marked on time unless they were ever carried over or reworked

36.2.2 The four-letter words

The Technical Priorities quadrant tags every item with one word, so the program manager can read the state of the sprint at a glance:

Table 36.3: The quad's four-letter words.

Word	Meaning in the template
TODO	a task that needs to be completed: on the backlog, not started

Word	Meaning in the template
OPEN	a task that has begun
DONE	a completed task
WAIT	a task that cannot proceed because of a blocker outside the team
HOLD	an open task interrupted by a higher priority
PASS [team]	handed to another team, to overcome a blocker or because they can do it better
DROP	a task that should not be done after all
REDO	done, but the demonstration showed it was wrong, so it is back in the sprint
PUNT	too hard as written: back to the backlog to be re-cut
SYNC	coordinated across teams, with a shared DONE

The stand-up verbs map onto them: `hold` and `resume`, `punt`, `redo`, `pass`, `sync` in the stand-up file set the word the quad prints (Chapter 23, Stand-ups, the journal and the board (p. 367)).

36.2.3 The two SWAT marks

Watch items carry one of two marks, and the difference matters to a program manager:

- **(PM) SWAT**: the program manager needs to *help remove* a blocker. This is an ask.
- **(WI) SWAT**: the program manager needs to *be aware* of an item. This is information.

The kit raises an item to (PM) when a blocker is older than three days or a team is loaded over a hundred and ten percent of its capacity; everything else on the watch list is (WI).

36.2.4 The legend

The template's legend has three groups. Schedule movement: *pushed right* (a milestone slipped later), *pulled left* (it moved earlier). Issue level: *minor or no issues*, *significant issues requiring PM awareness*, *show stopper issues*. Trend: *improving*, *degrading*, *no change*. The kit prints the first and third group as marks (`->`, `<-`, `=`, and the sprint progress trend in the header). It does not draw the three issue levels; the (PM) and (WI) marks carry most of that meaning, and a show stopper on a three-person team is a phone call, not a quad entry.

36.2.5 The two metrics

The template's second slide makes the quad do double duty as a metrics report: "Weekly Status satisfies both Metrics."

- **Sprint Progress**: the share of the current sprint backlog (the Technical Priorities quadrant) that is done. The kit prints it in the header, with its trend against the previous week.
- **On-Time Delivery of Milestone Products**: the share of accomplishments delivered on time, marked with a tick or a cross in the Accomplishments quadrant. The kit prints this week's and this sprint's counts beside it.

36.2.6 Tidewater's quads

The first quad of the project, Friday of sprint 1's first week:

docs/tidewater/sprints/01/quad-2026-10-09.txt

```
Weekly status Model -- as of 2026-10-09 (sprint 1)
Sprint progress 42% (7/16 SP) new   On-time delivery: 2/2 this week, 2/2 this sprint

TECHNICAL PRIORITIES
  OPEN  TW-3         3  "Create the GitLab group and m Tom
  OPEN  TW-5       1.5  "Receive drop 1 and record it" Jim
  OPEN  TW-6         3  "Convert drop 1 and assess it" Bob
  DONE  TW-1         4  "Learn the tools: survival kit Jim
  DONE  TW-2         3  "Laptops ready: Git for Window Tom
  OPEN  TW-4       1.5  "Draft the first pipeline and  Tom
  OPEN  TW-7       0.5  "Working agreements: Definitio Jim

WATCH ITEMS / PM HELP NEEDED
  none

SCHEDULE MILESTONES
  30 days out
    1. T2 Port Tidewater v1 to SysML v2 > E5 Drop 1 intake and first import +   0%  ETA
    2026-11-01
    2. T1 Stand up MBSE DevSecOps > E1 GitLab project and gitflow +   0%  ETA 2026-11-01
    3. T1 Stand up MBSE DevSecOps > E2 Pipeline and quality gates +   0%  ETA 2026-11-01
    (+1 more)
  60 days out
    -
  90 days out
    -

ACCOMPLISHMENTS (since 2026-10-02)
  v TW-1         4  "Learn the tools: survival kit Jim
  v TW-2         3  "Laptops ready: Git for Window Tom

PUNT RATE (tasks punted back to TODO / tasks committed, last 4 sprints)
  sprint 1    Model          0 / 7     0%

Legend: v on time  x late (carried over, or rework after the demo)  ! slipped   -> pushed right
<- pulled left  = no change  + new
```

And the last of the twelve weeks:

docs/tidewater/sprints/06/quad-2026-12-18.txt

```
Weekly status Model -- as of 2026-12-18 (sprint 6)
Sprint progress 36% (5/14 SP) new   On-time delivery: 4/4 this week, 4/4 this sprint

TECHNICAL PRIORITIES
  OPEN  TW-112       3  "BuoyModes state machine by ha Bob
  DONE  TW-111     0.5  "Learn: memos and briefings fr Jim
  DONE  TW-113       1  "Fork, pin and object flow in  Bob
  DONE  TW-114     1.5  "Level tags in Operations"     Tom
  DONE  TW-115       2  "Data exchange table for the p Tom
  OPEN  TW-116       2  "VCRM: which requirements are  Jim
  OPEN  TW-117       2  "Interim baseline-0 for the pr Tom
  OPEN  TW-118       2  "Retrospective, the management Jim

WATCH ITEMS / PM HELP NEEDED
```

```
  none

SCHEDULE MILESTONES
  30 days out
    1. T2 Port Tidewater v1 to SysML v2 > E8 Behavior and analysis +  50%  ETA 2027-01-10
    2. T1 Stand up MBSE DevSecOps > E1 GitLab project and gitflow ->  84%  ETA 2027-01-10
    3. T1 Stand up MBSE DevSecOps > E3 Team practices and onboarding ->  85%  ETA 2027-01-10
    (+1 more)
  60 days out
    -
  90 days out
    -

ACCOMPLISHMENTS (since 2026-12-11)
  v TW-111     0.5  "Learn: memos and briefings fr Jim
  v TW-113       1  "Fork, pin and object flow in  Bob
  v TW-114     1.5  "Level tags in Operations"     Tom
  v TW-115       2  "Data exchange table for the p Tom

PUNT RATE (tasks punted back to TODO / tasks committed, last 4 sprints)
  sprint 3    Model          0 / 6     0%
  sprint 4    Model          0 / 7     0%
  sprint 5    Model          0 / 8     0%
  sprint 6    Model          0 / 8     0%

Legend: v on time  x late (carried over, or rework after the demo)  ! slipped   -> pushed right
<- pulled left  = no change  + new
```

Read them the way the program manager did. The header gives the two metrics. Technical Priorities shows the sprint; in the first quad, two of its first stories are learning and laptops, which is the learning budget of Bottom line up front (p. iii) showing up where management can see it. The milestones quadrant shows the epics converging. The accomplishments are all ticked. The watch list is empty in both; across the twelve weeks only one item ever reached it, the Stations story carried out of sprint 4, as a (WI).

That empty watch list is worth a second look. In sprint 3 the toolchain review waited three days for the ISSM, and the quad of that Friday does not show it, because the wait began and ended between two Fridays. A weekly snapshot can miss a short block; the daily stand-up record (Chapter 23, Stand-ups, the journal and the board (p. 367)) has it.

You can produce a quad yourself from the Tidewater journal, as of any date:

```
perl "$KIT/bin/daily.pl" --conf journal/scrum.conf --today 2026-12-18 quad
```

The numbers will differ from the evidence copy above: the journal now holds the whole sprint, so “as of 18 December” is computed with the later stand-ups in it. The evidence copy is the page Jim actually sent that Friday afternoon. That is a lesson in itself: file the report you sent, not one you can regenerate.

36.3 The weekly note

The quad is the detail. The note is the headline: one paragraph a program manager reads on a phone between meetings. `talk.pl weekly` drafts it from GitLab, with a conclusion as its first line (“On track,” “At risk” or “Needs help”), three numbers, the risks and the asks:

```
perl "$KIT/tools/gitlab/talk.pl" --local gitlab --project tidewater/model weekly --date
2026-12-18
```

Jim edited every draft before sending it. The risks section of this one lists the discrepancy reports to the prime by age, each on its own line; in the note Jim sent, Jim folded them into one sentence (“our discrepancy reports to the prime, the oldest four weeks old, all due with the January drop”) because the program manager reads a list of identical lines as that many separate problems. The draft is a starting point; the sentence is the lead’s job (Chapter 25, Talking about progress: helps for engineers who would rather not (p. 390)).

36.4 The sprint review report

Every second Friday the review notes go to the program manager and the deputy: the sprint goal, what was demonstrated, what carried over, the artifacts shown, any feedback, and the four headline numbers. Sprint 6’s review, as filed:

docs/tidewater/sprints/06/review.txt

```
Sprint 6 review, 2026-12-24 10:00

Goal: Behavior started; Operations closed; the data exchange table; interim baseline-0 for the
program review

Demonstrated:
  TW-111 Learn: memos and briefings from the model
  TW-113 Fork, pin and object flow in ObserveCoast
  TW-114 Level tags in Operations
  TW-115 Data exchange table for the program review
  TW-116 VCRM: which requirements are verified, and how
  TW-117 Interim baseline-0 for the program review
  TW-118 Retrospective, the management briefing and the plan for next semester

Carried to sprint 7:
  TW-112 BuoyModes state machine by hand

Artifacts shown: the merge requests and pipelines of the sprint, the model site (GitLab Pages).

The four headline numbers: TODO lines left 5; verify coverage 70%; open DRs 7; blocked items 0.
```

The four headline numbers were the same at every review from sprint 2 on: TODO lines left, verify coverage, open discrepancy reports, blocked items. Keeping them identical was deliberate. By sprint 4 the deputy program manager stopped asking what they meant and started asking why one had changed.

The reviews also recorded the feedback, including the feedback that hurt. Sprint 2's says the deputy did not follow the demo; sprint 4's records the program manager asking whether the model site works on a phone. Both are quoted in Chapter 25, Talking about progress: helps for engineers who would rather not (p. 390), and the change between them is the reason the team rehearses.

36.5 How the reports changed management's questions

In September the program manager's question about the model was "Is it done?" By November it was "Which of these watch items needs me?" By December it was "Show me the same numbers in March." The reports did not just inform management; they changed what management asked for, from a yes or no to a trend. That is the point of a rhythm of small, regular, system-built reports: the conversation moves from whether the team is working to what the work shows.

Exercise 36.1: Read a quad

Open `docs/tidewater/sprints/04/quad-2026-11-27.txt`. What is the one watch item, what does its mark mean, and what would you expect the program manager to do about it?

Exercise 36.2: Write the note

Run `talk.pl weekly` for 13 November in the sandbox. Rewrite the draft as the note you would send: one conclusion, three numbers, at most one risk, and an ask if there is one.

Exercise 36.3: Map your own template

If your program already has a weekly status template, map each of its elements to a source the system can fill, as the quadrant table in this chapter does. Which elements have no source, and who writes them?

36.6 You can now

- read and explain every element of the weekly quad: quadrants, four-letter words, SWAT marks, legend and metrics;
- produce a quad from the journal and a weekly note from GitLab, and edit the draft into what you send;
- keep a sprint review report with the same headline numbers every time;
- explain why a weekly snapshot can miss a short block, and where to look instead.

36.7 Applying this to your project

Use the status template your management already reads, and have the system fill it, rather than inventing a new format they must learn. Agree the rhythm in week zero, and keep the headline numbers identical from report to report. Let the tools draft and let a person write the sentences. File every report you send with the sprint's evidence, so the briefing at the end of the quarter is assembled, not written.

CHAPTER

37

Writing it down: DoD-style memos from Vim

From what you know →

You know

the DoD memo: office symbol, MEMORANDUM FOR, numbered paragraphs, signature block.

→ Here it is called

Markdown memos rendered with md2memo.

→ What is new

you write the memo in Vim, render it with one command and review it like a model change.

Survival kit

What you have

Vim, a model repository, and a management chain that still runs on signed memoranda.

What you need

a way to write a proper memorandum (office symbol, date, MEMORANDUM FOR, FROM, SUBJECT, numbered paragraphs, signature block, portion markings when required) without leaving Vim, without a word processor template that fights you, and with the memo under review in the repository like any other change. The kit has it: `md2memo.pl` turns Markdown into a memorandum PDF in pure Perl, and `memo-build.pl` wraps it for templates.

37.1 Why memos still matter

Everything in the earlier chapters is live: the board, the pipeline, the model site, the stand-up. Management can look at any of it at any time. So why write a memorandum?

Because some things have to be *on the record*, in the form the organization files and forwards:

- **Requests that need a signature.** Training time, hardware, licenses and tool approvals are decided by someone who will forward the request up a chain. A chat message does not

travel; a memo does.

- **Decisions that must outlive the people who made them.** A memorandum for record (MFR) of why the team chose gitflow, or how the change board dispositioned a prime drop, is what an auditor or a successor reads in three years.
- **Results reported upward.** The twelve-week progress memo is what the program manager attached to their own report. The live site is the evidence; the memo is the claim, signed.
- **Correspondence with the prime.** Discrepancy reports go to the contractor through the contracting officer, as formal correspondence, not as a GitLab issue the contractor cannot see.

The team's objection, in week two, was that this was the old way coming back in by the side door. Jim's answer was that the memo was now the *thin* layer: one page that points at evidence the system produces, instead of a forty-page document that *was* the evidence. Writing it took twenty minutes instead of two days, because every number in it came from the pipeline or `talk.pl` (Chapter 25, Talking about progress: helps for engineers who would rather not (p. 390)).

37.2 The format, and how Markdown maps to it

A DoD-style memorandum has a fixed shape. `md2memo.pl` maps plain Markdown onto it:

Table 37.1: Markdown to memorandum

Memo part	Markdown	Rendered
Organization (letterhead)	`# Tidewater Coastal Observation Program`	centered, upper case
Office symbol	`## TW-SE`	flush left, upper case
Date	`## 24 December 2026`	flush left
MEMORANDUM FOR	`## Memorandum for Tidewater Program Manager`	upper case
FROM, SUBJECT, REFERENCES	`## From: …`, `## Subject: …`, `## References: (a) …; (b) …`	upper case
Numbered paragraphs	each top-level paragraph	1., 2., 3. …
Subparagraphs	a list right after a paragraph	a., b., c. (then (1), (a) …)
Emphasis	`*word*` or `**word**`	underlined
Tables	pipe tables	a ruled grid in the PDF, box characters in text
Signature block	`## JIM` then `## Tidewater MBSE Lead`	upper case, after the last paragraph
Portion marking	`{CUI}text{/}` (and U, C, S, TS with controls)	a portion mark in the margin; banners top and bottom

The output is US Letter with one-inch margins, Times 12 by default (`--font courier` for the typewriter look), as a PDF, PostScript or plain text. Nothing needs installing: it is the same core Perl as the rest of the kit.

37.3 The five templates

Writing a memo from a blank page is how memos get long. The kit ships five templates in `tools/gitlab/templates/memos/`, and `kickoff.pl scaffold` copies them into every new model

repository as `docs/memos/templates/`. Each one carries its guidance in comments that never print, and marks every blank as `<<placeholder>>`.

Table 37.2: The memo templates

Template	Use it when	The rule it teaches
`memorandum-for-record.md`	a decision must be on the record: gitflow, a baseline, a waiver	state the decision, the options rejected, and who can change it
`request-for-enablers.md`	the team needs time, hardware, licenses or approvals	every ask has what, why (in management's terms), by when, and what happens if not
`progress-memo.md`	management needs measured progress: the twelve-week memo, a quarterly memo	the outcome in the subject line; the agreed measures in a table; before and after; the ask
`drop-disposition.md`	the change board has dispositioned the changes in a prime drop	one row per prime change: accept, accept with impact, reject or clarify, no impact
`discrepancy-report-transmittal.md`	discrepancy reports go to the prime through the contracting channel	cite the prime's own element ids so the contractor can find each one

Here is the start of the request-for-enablers template; the comment block is its instructions, and it does not print:

tools/gitlab/templates/memos/request-for-enablers.md

```
<!--
TEMPLATE: Request for enablers.

Use it when the team needs something it cannot give itself: protected training time, runner
hardware, a license,
a tool approval, a person. A transition that is mandated without its enablers is a schedule risk.
This memo
states that plainly, and it gives management something specific to approve.

The rule for every ask: WHAT, WHY (the outcome it buys, in management's terms), BY WHEN, and WHAT
HAPPENS IF NOT.
"We're not ready" sounds like foot-dragging. "We need X by date Y, or the trace report slips two
weeks" is an
engineering position management can act on.

How to fill it in:
  - Replace every <<placeholder>>; `memo-build.pl --check FILE` lists what is left.
  - Put the asks in a table: management reads the table first.
  - Lead with the outcome the asks protect, not with the asks.
  - Offer a fallback for each ask. It shows you have thought about the trade.
Render: perl $KIT/tools/memo/memo-build.pl FILE.md
-->
# <<Organization name>>

## <<Office symbol>>
```

The rule in that comment is the one that matters: "We're not ready" sounds like foot-dragging; "we need X by date Y, or the trace report slips two weeks" is an engineering position management

can act on. It is the same outcome, evidence, ask pattern as the stand-up in Section 25.2, The pattern: outcome, evidence, next or ask (p. 391), in a more formal register.

37.4 Writing one

37.4.1 From the shell

Start from the template, in the repository's `docs/memos/` folder, and ask the build script what is still blank:

```
mkdir -p docs/memos
cp $KIT/tools/gitlab/templates/memos/request-for-enablers.md docs/memos/2026-10-21-enablers.md
perl $KIT/tools/memo/memo-build.pl --check docs/memos/2026-10-21-enablers.md
```

```
memo-build: docs/memos/2026-10-21-enablers.md: warning: 24 placeholder(s) to fill in:
<<Organization name>>, <<Office symbol>>, <<DD Month YYYY>>, <<Program Manager, office>>, <<Title,
e.g. Tidewater MBSE Lead>>, ...
```

`--check` exits with status 1 while anything is still blank, so it can also run in the pipeline: a memo with a placeholder never reaches `develop`. Fill in the blanks in Vim (next section), then build it. Here is Jim's finished memo from sprint 2, built from the same template:

```
cp $KIT/docs/tidewater/memos/2026-10-21-request-for-enablers.md docs/memos/
perl $KIT/tools/memo/memo-build.pl docs/memos/2026-10-21-request-for-enablers.md
```

```
wrote docs/memos/2026-10-21-request-for-enablers.pdf
wrote docs/memos/2026-10-21-request-for-enablers.txt
```

The `.txt` beside the PDF is the same memo as plain text, which is what you paste into an e-mail when the PDF is an attachment, and what `git diff` can show in review:

```
sed -n 1,24p docs/memos/2026-10-21-request-for-enablers.txt
```

docs/tidewater/memos/2026-10-21-request-for-enablers.txt

```
              TIDEWATER COASTAL OBSERVATION PROGRAM

TW-SE

21 OCTOBER 2026

MEMORANDUM FOR TIDEWATER PROGRAM MANAGER

FROM: TIDEWATER MBSE LEAD
```

```
SUBJECT: ENABLERS NEEDED TO BRIEF MEASURED RESULTS AT THE
DECEMBER PROGRAM MANAGEMENT REVIEW

REFERENCES: (A) PROGRAM MEMORANDUM, ADOPTION OF DEVSECOPS
PRACTICES FOR SYSTEMS ENGINEERING ARTIFACTS, 24 SEPTEMBER 2026;
(B) TWELVE-WEEK SCORECARD AND PLAN, 2 OCTOBER 2026

1.  The team is on track for the December briefing. Drop 1 of the
prime's model is received, checked against its manifest and
converted: the converter left 52 items that need a person, and
the drop goes onto the integration branch this sprint. The first
pipeline is written and runs locally; it checks every change once
the runner below is in place.

2.  To keep that date, the team needs the following decisions:

+---+--------------+--------------+-----------+--------------+
| # | Ask          | Why (the     | Needed by | If not       |
|   |              | outcome it   |           |              |
|   |              | buys)        |           |              |
+===+==============+==============+===========+==============+
| 1 | Four         | the team     |  26 Oct   | the          |
|   | protected    | learns while |           | conversion   |
|   | hours a week | it delivers  |           | burn-down    |
|   | per person   |              |           | slows by     |
|   | for training, |             |           | about a third |
|   | through 18   |              |           |              |
```

Look at what Jim did. The subject line names the outcome (“brief measured results at the December program management review”), not “resources”. Paragraph 1 says where the team stands with one number. The asks are a table, because management reads the table first, and each row says what happens if the answer is no. The full memo is in `docs/tidewater/memos/`, with the four others the team wrote in the twelve weeks and the first prime drop after them.

37.4.2 From Vim

The whole loop stays in Vim. Open the memo, write, and build with one Ex command; `%` is the current file:

```
edit docs/memos/2026-10-21-request-for-enablers.md
w | !perl $KIT/tools/memo/memo-build.pl %
```

To read the result beside the source, open the text version in a split:

```
:w | !perl $KIT/tools/memo/memo-build.pl %
:vsplit %:r.txt
```

and to find the blanks left in a template, search for the placeholder marker:

```
/<<
```

`n` jumps to the next one. When `memo-build.pl --check %` stops complaining, the memo is ready for review.

If you want `md2memo.pl` itself (for options such as `--font courier` or a default marking), call it directly; but it prints the template's instruction comments, so use `memo-build.pl` for anything that started from a template. Options for `md2memo.pl` go after `--`:

```
mkdir -p docs/memos
cp $KIT/docs/tidewater/memos/2026-10-21-request-for-enablers.md docs/memos/
perl $KIT/tools/memo/memo-build.pl docs/memos/2026-10-21-request-for-enablers.md -- --font
courier
```

37.4.3 Portion marking

Tidewater is notional and unmarked, so the team's memos carry no markings. Your program's memos may need them, and `md2memo.pl` handles them: tag a portion with `{CUI}…{/}` (or U, C, S, TS with dissemination controls), and every paragraph gets its portion mark and every page its banner, rolled up from the portions on it. Here is a discrepancy report transmittal with one paragraph tagged, as a demonstration (the CUI label is notional, like everything in Tidewater):

```
cat > docs/memos/2026-11-20-dr-transmittal.md <<'EOF'
# Tidewater Coastal Observation Program

## TW-SE

## 20 November 2026

## Memorandum for Northwind Marine Systems, Tidewater Program Office (through the contracting
officer)

## From: Tidewater MBSE Lead

## Subject: Discrepancy reports DR-1 to DR-4 against prime drop v1.0

## References: (a) CDRL A003, System Model; (b) Model exchange agreement, 2 October 2026

The government's review of drop v1.0 found four discrepancies between the delivered model and the
DOORS requirements. Each cites the v1 element identifier so it can be found in the model.

{CUI}The details of each discrepancy, with element identifiers, are in the attached report.{/}

Request a disposition of each discrepancy with the next drop, or by 15 January 2027.

## JIM

## Tidewater MBSE Lead
EOF
perl $KIT/tools/memo/memo-build.pl docs/memos/2026-11-20-dr-transmittal.md
cat docs/memos/2026-11-20-dr-transmittal.txt
```

```
                                  CUI

(U)                 TIDEWATER COASTAL OBSERVATION PROGRAM

(U)   TW-SE

(U)   20 NOVEMBER 2026
...
(U)   1.  The government's review of drop v1.0 found four discrepancies
      between the delivered model and the DOORS requirements. Each cites
      the v1 element identifier so it can be found in the model.

(CUI) 2.  The details of each discrepancy, with element identifiers,
      are in the attached report.

(U)   3.  Request a disposition of each discrepancy with the next drop,
      or by 15 January 2027.
...
                                  CUI
```

Warning: The tool marks what you tag; it does not decide what is controlled

`md2memo.pl` applies the markings you put in the text and rolls them up correctly. Deciding *what* is CUI, export controlled or classified is the security office's job, under your program's classification guide. Agree the rules before the first marked memo, and record them in `docs/conventions/markings.md`.

37.5 Memos under review, like model changes

The team keeps its memos in the model repository, under `docs/memos/`, with the date first in the file name so the folder lists them in order. A memo is a change like any other:

1. Open an issue ("Twelve-week progress memo for the December review"), with the `doc::grey` label from the document review stages (Chapter 40, Documents still matter: review stages for generated documents (p. 570)).
2. Write the memo on a feature branch, from the template.
3. Open a merge request. The reviewers are the people whose numbers are in it: Bob checks the model figures, Tom the pipeline figures.
4. The pipeline runs `memo-build.pl --check` on every changed memo, so a placeholder or a missing subject line fails the merge request.
5. The issue's `doc::` label moves through the stages as reviews finish; at `doc::white` Jim signs and sends the PDF.
6. Merge. The memo, its PDF and its text are now in the history, next to the model they describe, at the same commit.

That last point is the quiet benefit. A year later, "what did we tell management in December, and what did the model look like when we said it?" is one `git log` away: the memo and the model share a commit.

37.6 Tidewater's memos

The team wrote five memos in its first four months. All five are in `docs/tidewater/memos/` as Markdown, PDF and text.

Table 37.3: The Tidewater memos

Date	Memo	From	Template
24 Sep 2026	the directive: adopt DevSecOps practices for systems engineering artifacts (Section 3.1, The memo (p. 22))	the program manager	none: it came first
21 Oct 2026	request for enablers: training hours, the ISSM slot, a Cameo seat, a second runner, the GitLab tier decision	Jim, sprint 2	`request-for-enablers.md`
23 Oct 2026	memorandum for record of decisions D-1 to D-10 (Section 13.11, The decisions the team wrote down (p. 210))	Jim, sprint 2	`memorandum-for-record.md`
24 Dec 2026	twelve-week results for the December review	Jim, sprint 6	`progress-memo.md`
13 Jan 2027	disposition of prime drop v1.1	Jim, as chair of the model change board	`drop-disposition.md`

37.6.1 The twelve-week progress memo

The most important of the five is the one that told management the twelve weeks worked. Its subject line is a conclusion: *Twelve-week results: the Tidewater model is under configuration control and checked on every change*. Its second paragraph is the scorecard agreed in week zero (Section 3.6, Defining measurable progress (p. 28)), with the value at the start, the value now and the target, and the source of each number:

docs/tidewater/memos/2026-12-24-twelve-week-progress.md

```
In twelve weeks the team moved the Tidewater system model from SysML v1 into SysML v2 text under
configuration control in GitLab, and every change to it is now checked automatically and reviewed
before it is accepted. The most important result: 47 of the 52 conversion items are closed. The 5
left are behavior and analysis items the team writes by hand.

The measures agreed before the work began, computed by the system:

| Measure | At start | Now | Target | Source |
|:--------|:--------:|:---:|:------:|:-------|
| Conversion items (TODO lines) left | 52 | 5 | structure closed | pipeline todo job |
| Requirements with a verification | 21 of 30 | 21 of 30 | every gap owned | trace job |
| Open discrepancy reports to the prime | 0 | 7 | every gap reported | GitLab issues |
| Blocked items | 0 | 0 | none older than 3 days | journal |
| Changes gated (pipeline and code-owner review) | not measured | 26 of 27 | 100% | GitLab |
| Change lead time, median | 23 working days (notional) | 3.5 hours | under 3 working days |
gitlab.pl analytics |
| Review turnaround, median | 3 weeks per package (notional) | 3.3 hours | under 1 working day |
gitlab.pl analytics |
| Package dependency cycles | 2 | 1 | none | pipeline |
| Team morale (pulse, 1 to 5) | 1.9 | 4.1 | 3.5 and rising | retrospectives |
```

```
Before: finding where the prime's DOORS text and the model disagreed was never done
systematically, and a model review was a sixty-slide briefing of screenshots that took three
weeks. Now: the pipeline compares every requirement's text on every change in under a minute, and
the model site is current every night.
```

The "Now" column is filled from the simulation's sprint 6 evidence (the scorecard and the analytics report), the same numbers the briefing in Chapter 35, The briefing (p. 532) shows. The rest of the memo follows the progress template: one before-and-after paragraph, one honest paragraph about what did not go well and what the team did about it, what the results mean for the next milestone (the January drop), and the ask with a date. It is the written twin of the sprint 6 briefing, and it went to the program manager the day after.

37.6.2 The disposition memo

The drop-2 disposition memo shows the change board's work in the form a contracting officer files: one row per change the prime delivered, the board's disposition, and the issue that tracks each. It also closes DR-1, because drop 2 fixed the TW-SYS-006 wording. Chapter 45, The prime's model changes: receive, assess, disposition (p. 619) explains the disposition process; the memo is its record.

37.7 The progress memo, step by step

Jim wrote the twelve-week memo on the Monday of sprint 6's second week, in about forty minutes, and it went through one review round. The order Jim worked in is the order to copy, because each step uses something the system already produced.

1. **Start from the measures, not the prose.** Copy `progress-memo.md` and fill the measures table first: the "At start" column comes from week zero's scorecard (Section 3.6, Defining measurable progress (p. 28)), the "Now" column from the latest pipeline metrics and `talk.pl review` (Section 25.4, Tools that draft the words (p. 393)), the "Source" column says where each number lives so a reader can check it. If a number cannot be sourced, it does not go in the table.
2. **Write the subject line as the conclusion the table supports.** Not "Twelve-week status", but what the numbers say: "the model is under configuration control and checked on every change". If the table does not support a conclusion you would sign, the memo is not ready, and the conversation with management should happen before the memo, not in it.
3. **Paragraph 1: the most important result, in one sentence with one number.** The same rule as the one-sentence rung of the ladder (Section 25.3, The ladder: one sentence, thirty seconds, five minutes (p. 392)).
4. **One before-and-after paragraph.** Pick the comparison management will repeat. For Tidewater it was finding the difference between the prime's DOORS text and the model: never done systematically before, now done on every change in under a minute.
5. **One honest paragraph about what did not go well**, and what the team did about it. Sprint 3's dip went in, with its two causes and the two fixes. A progress memo with no bad news reads as a sales document, and management discounts it.
6. **What it means for the next milestone.** The next thing management cares about (the January drop) and what the team can now do for it.

7. **The ask, with a date, and a reference to where it was first asked.** The twelve-week memo repeated two asks from the October request-for-enablers memo that were still open, and cited it.
8. **Attachments that are evidence, not decoration.** The briefing and a link to the model site. Not a printout of the pipeline.

Then `memo-build.pl --check`, a merge request with Bob and Tom as reviewers (Bob checks every model number against the site, Tom every pipeline number against the job logs), and a review in the document stages before Jim signs.

37.8 Which record for what: memo, ADR, issue, wiki

The team found itself with four places to write things down and, for a sprint, wrote some things in all four. The rule they settled on:

Table 37.4: Where to write it down

Record	For	Audience	Lives in	Changes by
Issue	work to be done, and the conversation about it	the team, the reviewers	GitLab	comments, labels, closing
ADR (architecture decision record)	a decision about the model or the toolchain, with context and consequences	engineers now and later	`docs/adr/` in the repository	a new ADR that supersedes it
Memorandum for record	a decision the organization must be able to cite, in its own format	auditors, successors, the program office	`docs/memos/` (and the official file)	a new memo
Wiki page	how the team works: the definitions of ready and done, the runbooks, the glossary	the team, newcomers	the GitLab wiki	editing, with history
Memo to someone	a request, a report, correspondence	a named person outside the team	`docs/memos/`, then sent	not changed once sent

Some decisions belong in two places. The ten decisions of week zero were written as ADRs, because engineers need the reasoning next to the model, and summarized in one memorandum for record, because the program office needed a document it could cite. The memo points at the ADRs; it does not repeat them.

Tip: One fact, one home

If the same fact appears in an ADR, a memo and the wiki, it will drift. Give each fact one home and point at it from the others. The memo is the thinnest layer: it cites, it does not restate.

37.9 What md2memo cannot express

The tool covers what a working memorandum needs, and the team found its edges quickly. They are worth knowing so you do not fight them:

- **Header lines are upper case.** The ## lines become upper case, so a long reference list prints in capitals.
- **The date cannot sit flush right on the office-symbol line.** Runs of spaces collapse; the office symbol and the date go on separate lines.
- **The signature block is flush left,** not started at the center of the page.
- **Attachments are a small table.** A list after the signature would continue the paragraph numbering, so the templates use a two-column table instead.
- **References are one line,** not (a) and (b) subparagraphs under the header.
- **Wide tables wrap mid-word in the text output.** The PDF measures widths and is fine; the 65-column text version breaks long cell words.
- **Instruction comments print** if you call `md2memo.pl` directly on a template; `memo-build.pl` strips them first.

If your organization's correspondence manual requires any of the first four exactly, render the PDF and fix the last mile in the office's word processor; keep the Markdown as the reviewed source. Most program offices accepted the output as is.

37.10 Exercises

Exercise 37.1: Ask for what you need

Copy `request-for-enablers.md` and write the request your team would need to adopt this book's practices: training time, a runner machine, a tool approval, one more. For each ask write what, why in management's terms, by when, and what happens if not. Run `memo-build.pl --check` until it is silent, then build it.

Exercise 37.2: Record a decision

Pick one decision from Section 13.11, The decisions the team wrote down (p. 210) (or one your team made this month) and write a memorandum for record with the `memorandum-for-record.md` template: the decision, the options rejected and why, and who can change it. Compare it with the same decision as an ADR in `docs/adr/`: what does each one do that the other does not?

Exercise 37.3: Build from Vim

Open a memo in Vim, change one paragraph, and rebuild it with `:w | !perl $KIT/tools/memo/memo-build.pl %`. Open the text version in a vertical split. Then commit the memo on a branch and look at `git diff` of the `.txt`: is the change easy to review?

Exercise 37.4: Draft the progress memo from evidence

Using `talk.pl weekly` and `review` output for a sprint (Section 25.4, Tools that draft the words (p. 393)), fill in the measures table of `progress-memo.md` for that sprint. Which numbers came from the pipeline, which from GitLab, and which did you have to supply yourself?

37.11 Applying this to your project

- **Copy**: the five templates; memos in the repository under `docs/memos/`, dated file names, reviewed by merge request; `memo-build.pl --check` in the pipeline; the rule that every number in a memo comes from the system.
- **Adapt**: the letterhead, office symbols and signature blocks to your organization's correspondence manual; the marking rules to your security classification guide (write them in `docs/conventions/markings.md` first).
- **Decide locally**: which decisions need an MFR as well as an ADR; who signs which memo; whether the PDF from `md2memo.pl` goes out as is or through the office's word processor for the last mile.

Note: You can now…

- write a DoD-style memorandum in Markdown, in Vim, from a template, and render it to PDF and text;
- find what is still blank with `memo-build.pl --check`, and keep placeholders out of the repository;
- tag portions for marking and get the banners rolled up correctly;
- keep memos under review in the model repository, at the same commit as the model they describe;
- know where the tool's format stops and the correspondence manual takes over.

CHAPTER

38

How the team changed its mind

From what you know →

You know

the objections people raise to any new process: some fair, some fear.

→ Here it is called

the objections, revisited.

→ What is new

each one is answered with evidence from the twelve weeks, not with a promise.

Survival kit: Where this chapter fits

What you have

twelve weeks of evidence – stand-up chat, retrospectives, a pulse survey every second Thursday, quotes people did not expect to have written down.

What you need

an honest account of how three skeptical engineers moved from "why can't we just send the Word doc?" to "keep doing this", without pretending the middle was smooth. You also need a way to measure that change on your own team, because management will ask, and because the team deserves to see it.

In Chapter 3, The directive (p. 22) the team raised nine objections in one meeting. They were not excuses. Each one was a real fear held by a person who had been burned before: by a document port that took two years, by a tool nobody else could read, by an approval that never came, by management goals that moved. This chapter takes the objections one at a time, in the order they were raised, and sets beside each one the evidence the twelve weeks produced. Where the evidence does not answer the objection, the chapter says so.

38.1 The pulse, sprint by sprint

At every retrospective each person answered the same five questions, anonymously, on a scale of one (strongly disagree) to five (strongly agree). The questions were chosen in Chapter 3, The directive (p. 22) before any work started, so nobody could tune them to the result.

Table 38.1: The pulse survey's mean, from each sprint's morale.json.

Sprint	Mean	What happened that sprint
1	1.9	Learning the tools, the GitLab project, drop 1 received
2	2.4	The first pipeline died on the PowerShell runner; drop 1 reached `develop`
3	2.2	Bob's first red merge request; three days waiting on the ISSM
4	3	The model site on a phone; the first cycle broken
5	3.8	The trace gate stopped a mistake before it merged
6	4.1	Interim baseline; the management briefing

The shape matters more than the end point. The line rose in sprint 2, *fell* in sprint 3, and only then climbed. The dip is the most useful number in this chapter. It came in the sprint where the team stopped learning in a sandbox and started being judged by the pipeline on real work – and in the same sprint, a security approval sat unsigned for three days. If you run this survey on your own team and see a dip in the third or fourth sprint, it does not mean the change is failing. It means the change has become real.

The five questions moved differently. Here are sprint 1 and sprint 6, person by person:

Table 38.2: The five pulse questions, sprint 1 and sprint 6.

Question	Jim, S1	Bob, S1	Tom, S1	Jim, S6	Bob, S6	Tom, S6
I understand why we are working this way.	2	1	2	5	4	4
I can do my work in the new tools without help.	2	2	2	4	4	4
The checks catch problems I would have missed.	2	2	2	4	4	4
I can see our progress.	2	2	2	4	4	4
I would choose to keep working this way.	2	1	2	5	4	4

Bob started lowest on the first and last questions – Bob did not see why, and would not have chosen this. Those are the two that changed most. Nobody's answer reached the top of the scale on "I can do my work in the new tools without help" by sprint 6. That is honest: twelve weeks makes a team competent, not expert, and the book does not claim more.

You can read every answer yourself. The survey files are plain JSON in the evidence:

```
for f in docs/tidewater/sprints/0*/morale.json; do
  perl -MJSON::PP -0ne '$d = JSON::PP->new->decode($_); print "sprint $d->{sprint}: mean
  $d->{mean}\n"' "$f"
done
```

38.2 The objections, revisited

38.2.1 "We already did this once. Modeling took two years and gave us a spider web."

Bob's objection was the deepest, because it was about the last time. The Cameo port had been sold as the end of document drift; it delivered a model that only three people could open and that nobody trusted.

The evidence is partial, and that is worth saying. The v1 drop arrived with 1 package dependency cycle remaining in the v2 model by sprint 6, against two in drop 1 – the Network and Operations cycle was broken in sprint 4; the DataCenter and Stations cycle is still there at the end of sprint 6. The worst part definition dropped from fourteen relationship ends in v1 to 8 in v2. Tom's reaction to the first cycle break, in sprint 4:

> *The cycle was two lines. In Cameo it was a week of looking at a diagram.*
>
> — Tom, 24 November 2026

The measurement is in Chapter 47, Keeping SysML v2 from becoming a spider web (p. 645). What changed Bob's mind was not the numbers, though. It was that the complexity was *visible on every change* instead of discovered at a review. A spider web grows one innocent link at a time; a check that runs on every merge request stops the innocent link.

38.2.2 "Why can't we just do systems engineering the old way?"

Tom asked it; Bob said it out loud on the first morning:

> *Why can't we just send the Word doc? It worked for twenty years.*
>
> — Bob, 5 October 2026

The answer in Chapter 3, The directive (p. 22) was that nothing changes what a requirement or a baseline *is*; what changes is how the team keeps them. The evidence for that answer came three days later, from Bob:

> *I found the drift in two minutes. It used to take a week, and we only looked before a review.*
>
> — Bob, 8 October 2026

That sentence was the first time anyone on the team compared the new way to the old way in a unit they cared about. It was also the first number in the scorecard (Chapter 33, The twelve-week scorecard (p. 520)): the trace answers in 24 seconds what the document era answered in 2 days (notional, from the program's last document-era review cycle).

38.2.3 "We're systems engineers, not programmers."

Nobody wrote a program in twelve weeks. They edited text files in Vim, which all three already used, and typed git commands they learned in an afternoon (Chapter 12, Git from zero, for systems engineers (p. 171)). Bob's verdict after that afternoon:

> *Spent the afternoon in the practice repo. Branch, commit, merge. It is configuration management with fewer forms.*
>
> — Bob, 6 October 2026

The harder moment came in the third sprint, when the pipeline said no to Bob for the first time:

> *A red X on my own MR. One missing word, and the machine found it before Jim did.*
>
> — Bob, 4 November 2026

That quote is from the dip sprint, and it is not a happy one. It became one of the reasons the pulse rose afterwards: the red X was private, immediate and fixable, which a red-pen review two weeks later never was.

38.2.4 "The contractor owns the model. Anything we do is a copy that will go stale."

This objection is only partly answered by the first twelve weeks, because only one prime drop arrived in them. What the twelve weeks proved is the *mechanism*: drop 1 came in on the vendor branch (Chapter 27, Sprint 2: Drop 1 on develop, every change gated (p. 424)), the government's own edits sat beside it without being overwritten, and every gap the government found went back to the prime as a discrepancy report – 7 of them by the end of sprint 6, none closed yet, because the prime closes them in later drops. Whether the copy stays current is answered by the drops in Chapter 45, The prime's model changes: receive, assess, disposition (p. 619), not by this chapter.

38.2.5 "Nobody outside our team will be able to read SysML v2 text."

Jim raised it as devil's advocate. The answer turned out to be that nobody outside the team *needed* to read the text: they read the model site the pipeline publishes. The program manager's reaction in the sprint 4 review:

> *I can open this on my phone? I have never seen the model without a briefing.*
>
> — Program manager, 20 November 2026

Fewer people could read the Cameo model, which needed a licence. More people read the v2 model than ever read the v1 one, because it was a web page.

38.2.6 "The ISSM will never approve this in time."

Tom was half right, and the evidence shows it. The toolchain review did slow the team down:

> *Three days waiting on a signature. The pipeline was the fast part.*
>
> — Tom, 10 November 2026

That wait is part of the sprint 3 dip. The lesson the team drew, recorded in the sprint 3 retrospective, was to stop "waiting for the ISSM without a date" – to ask for a date, and to keep working on what did not need the approval. The approval came. The objection was right about the delay and wrong about the outcome.

38.2.7 "We'll spend twelve weeks learning tools and have nothing to show."

The BLUF (Bottom line up front (p. iii)) answers this with the time split: learning and platform work fell from most of sprint 1 to a small share of sprint 6. The scorecard answers it with results: TODO lines from the conversion fell to 5, every change in sprint 6 was gated (7 of 7), and an interim baseline was tagged and reproduced (1). There was something to show from sprint 2 onward; the problem in sprint 2 was that the team showed it badly (Chapter 25, Talking about progress: helps for engineers who would rather not (p. 390)).

38.2.8 "Management will just move the goalposts."

Tom said this quietly, at the end of the meeting. The defense was the scorecard, agreed in writing in week one. At the sprint 6 review the program manager did not move the goalposts; they asked for the same ones again:

> *Keep doing this. Show me the same numbers in March.*
>
> — Program manager, 24 December 2026

38.2.9 "In three years there'll be another directive."

Probably true, and the twelve weeks cannot disprove it. What the team kept from them is not a tool but a habit: the system's description as reviewed text under configuration control, with checks that run on every change. The text is in an open standard, the checks are scripts the team owns, and the history is in git. Whatever the next directive brings, it starts from a model that can be read, diffed and converted – which the v1 model could not.

38.3 What actually changed minds

Reading the quotes in order, three things recur:

- **A number in a unit the engineer cared about.** "Two minutes, it used to take a week." Not a dashboard, not a velocity chart – a comparison to their own experience.
- **A mistake caught privately and early.** The red X on Bob's merge request, the trace gate that stopped Bob deleting a satisfy Bob thought was a duplicate (sprint 5). Being stopped by a machine before a colleague saw the mistake built trust in the checks faster than any explanation.
- **Being seen by management without a briefing.** The program manager on the phone; Jim answering the deputy PM's trace question with a link instead of a week of slides:

> *The deputy PM asked for the trace numbers. I sent a link.*
>
> — Jim, 9 December 2026

What did *not* change minds: being told the new way was better, the GitLab feature list, and the sprint 2 demo, which showed the terminal instead of the answer. Management's feedback on that demo was blunt – "I did not follow the demo. What did we get for the two weeks?" – and it is in the evidence too.

Exercise 38.1: Run the pulse on your own team

Take the five questions above. Ask them at your next three retrospectives, anonymously, on a one-to-five scale. Plot the mean. Before you look at the result, write down which sprint you expect the dip in, and why.

Exercise 38.2: Write your team's objections down

In your kickoff meeting, write every objection on the whiteboard in the speaker's words, with the speaker's name if they agree. Put the list in the wiki. At the end of each sprint, add the evidence for or against each one. Which objections are still unanswered after six sprints?

38.4 You can now

- Measure a team's change of mind with a short, fixed pulse survey and read its shape, not just its end point.
- Answer an old-way objection with evidence from your own project instead of argument.
- Recognize the third-sprint dip as a sign the change has become real.

38.5 Applying this to your project

Copy the five questions and the habit of writing objections down in the speaker's words. Adapt the scale and the timing to your retrospectives. Decide locally whether the survey is anonymous (Tidewater's was) and who sees the raw answers. Do not promise management a rising line; promise them the measurement, and explain the dip in advance.

CHAPTER

39

A day in the life, week twelve

From what you know →

You know

an ordinary day in the Cameo years: e-mail, meetings, screenshots, a briefing to update.

→ Here it is called

Bob's day in week twelve.

→ What is new

the day goes to engineering, and the machinery does the bookkeeping.

Survival kit: Where this chapter fits

What you have

Bob's Thursday, 17 December 2026 – the second-to-last week of the twelve – exactly as GitLab and the stand-up chat recorded it.

What you need

a concrete picture of what the work feels like once the tools are learned, beside the same kind of day three years earlier in the Cameo era. Managers ask "what does a normal day look like now?" This is the answer.

Nothing on this day was staged for the book. It is an ordinary working day in sprint 6, chosen because it has most of what a small model team meets in practice: a failed overnight check, a change that the pipeline sends back, a review that requests changes, a merge that is refused until a discussion is resolved, and the next morning a secret that nearly got into the repository. Every time below is Eastern time, as GitLab recorded it.

docs/tidewater/sprints/06/day-in-life/timeline.txt

```
Thursday 2026-12-17, as GitLab recorded it (Eastern time)

  09:15 bob  opened #132 for the failed nightly pipeline #83
  10:00 bob  opened !23 Draft: Fork, pin and flows in ObserveCoast (pipeline #86 failed)
```

```
  10:30 tom  cleared the runner's disk; re-ran the full check: #84 success; closed #132
  10:45 tom  opened !24 Document what crosses each interface (pipeline #90 success)
  12:00 bob  marked !23 ready
  14:05 bob  line comment on !24: request changes (the 'late' assumption)
  14:20 tom  merge refused: discussions not resolved (1 open) (405 Method Not Allowed)
  14:50 bob  resolved the thread on !24
  15:10 tom  closed #123 Document the flow properties in Interfaces
  15:10 tom  merged !24
  15:30 bob  closed #121 Fork, pin, object and control flow by hand
  15:30 bob  merged !23

Friday 2026-12-18

  10:00 tom  opened !25 Generate the data exchange table (pipeline #95 failed)
  11:30 tom  secrets job failed on !25: token revoked, rotated to masked READ_TOKEN, branch
  rewritten
  15:00 tom  closed #124 Generate the data exchange table
  15:00 tom  merged !25
```

39.1 09:00 – the stand-up, and the night's failure

The first thing Bob saw was not an email. It was the GitLab To-Do list and the board: the scheduled "Nightly full check" pipeline from 02:00 had failed, in every job, before any check ran. Bob opened an issue for it at 09:15, labelled it, and linked the failing job:

GitLab Markdown: docs/tidewater/sprints/06/day-in-life/incident.md

````
# #132 Nightly full check failed: no space left on the runner

The nightly full check (pipeline #83, scheduled 02:00) failed in every job before running any
check:

```text
fatal: could not create work tree dir 'builds/tidewater/model': No space left on device
```

Job: https://gitlab.gov.example.invalid/tidewater/model/-/pipelines/83/../jobs/543

Not a model problem: develop's last MR pipeline was green. Picked up at the stand-up.
---- comments

**@tom** (2026-12-17T15:30:00Z) [system]:
added 1h of time spent

**@tom** (2026-12-17T15:30:00Z) [system]:
removed ~"workflow::in-progress" label

**@tom** (2026-12-17T15:30:00Z) [system]:
added ~"workflow::done" label

**@tom** (2026-12-17T15:30:00Z) [system]:
closed

**@tom** (2026-12-17T15:30:00Z):
````

```
Cleared the runner's old build directories (14 GB) and set the runner to clean up after each job.
Re-ran the full check: pipeline #84 success.
```

The failure was not in the model. The runner – the Windows machine that runs the team's pipeline – had filled its disk with old build directories. At the stand-up it took one sentence to hand over ("Tom, the nightly died on disk space, #132"), and Tom had it cleared and re-run by 10:30. In the Cameo era the equivalent – a tool failing overnight – would have been discovered when someone opened the model and it would not load.

The stand-up itself was three lines in the meeting chat, drafted with `talk.pl` and edited before posting (Chapter 25, Talking about progress: helps for engineers who would rather not (p. 390)):

docs/tidewater/sprints/06/standups/2026-12-17-chat.txt

```
Meeting started
[9:00 AM] Jim
Morning. Y/T/B in chat, please.
[9:01 AM] Jim
Y TW-116 VCRM: which requirements are: 1 of 2 tasks done. T TW-118 the twelve-week briefing and
memo, by Dec 24; pairing with Bob after lunch. B none
[9:02 AM] Bob
Y TW-116 Reviewed !22. T TW-112 write the BuoyModes state def; exhibit it on, by Dec 24. B none
[9:03 AM] Tom
Y none T TW-115 document the flow properties in Interfaces, by Dec 24; pairing with Jim after
lunch. B none
```

You can open the issue Bob filed in the simulation's GitLab:

```
perl "$KIT/bin/gitlab.pl" --local gitlab --project tidewater/model issue show 132
```

39.2 10:00 – a draft that the pipeline sends back

Bob's own work that week was the hand-written flows in the `ObserveCoast` activity: the fork, the pin, the object and control flows that the converter could not carry from v1 (Chapter 8, SysML v2 by example: a workbook (p. 92)). Bob opened merge request !23 as a *draft* at 10:00 – a draft is a merge request that says "look, but do not merge yet" – and the pipeline ran on it straight away. It failed. The grammar, lint and outline checks were green; the trace check was red:

docs/tidewater/sprints/06/pipelines/mr23-first-86.log

```
trace gaps: 2 orphan(s), 9 unverified, 2 drifted
$ git worktree add -q --detach /tmp/ci-XXXXXX 418ad2ad3674c27d9c34b2efcac01ce540f5468c
$ git worktree remove --force /tmp/ci-XXXXXX
target branch: 29 of 30 linked; 2 orphan(s), 9 unverified, 1 drifted (full report:
trace-base.txt)
this change adds trace gaps: drift 1 -> 2 (new: TW-SYS-007) (see trace.txt)
```

Bob had tidied a doc comment on requirement TW-SYS-007 while in the file, and the new wording no longer matched DOORS. The trace job compares the model's requirement text with the DOORS export on every change; a *new* difference fails the merge request (Chapter 20, The pipeline, job by job (p. 302)). The old difference, TW-SYS-006, is the prime's and is tracked as a discrepancy report, so it does not fail anything. Bob put the wording back, pushed, and the next pipeline (#87) was green. Bob marked the draft ready at noon. In the Cameo era a reworded requirement in the model would have been found – if at all – at the next DOORS synchronisation, weeks later, by someone else.

39.3 14:05 – reviewing Tom's change

At 14:05 Bob's To-Do list showed one item: Tom had asked Bob to review !24, "Document what crosses each interface". Bob did not read the raw text diff alone. Bob read the generated documentation the pipeline attached, then went to the line that looked wrong and left a line comment with a *suggestion* – a proposed replacement the author can apply with one click – and requested changes:

GitLab Markdown: docs/tidewater/samples/mr-review-thread.md

````
# !24 Document what crosses each interface

## What and why

Document the flow properties in Interfaces (TW-115).

Closes #123
Part of #115

## Model changes

| Package | Change |
|---|---|
| `model/library/Interfaces.sysml` | changed |

## Checks

- [x] `gitlab.pl ci` passed locally
- [x] `docs/` regenerated and committed
- [x] No new `// TODO` lines
- [x] Trace job: no new orphan, unverified or drifted requirement

---- thread

**@bob** (2026-12-17T19:05:00Z):
**Request changes** -- `model/library/Interfaces.sysml`, the `health` flow:

> Sent with every transmission; the data center flags a station whose report is late.

"Late" assumes a schedule we have not written down anywhere. TW-SYS-009 does not say how late is
late. Either cite where the period comes from, or say it is an assumption:

```suggestion:-0+0
        out item health : HealthReport { doc /* Sent with every transmission. Assumption (to
        confirm with the prime): late = two missed transmissions. */ }
````
````

```
```

@jim (2026-12-17T19:20:00Z) [system]:
approved this merge request

@tom (2026-12-17T19:50:00Z):
Applied the suggestion; the assumption goes in the next prime-tbd list.

@bob (2026-12-17T20:00:00Z) [system]:
approved this merge request

@tom (2026-12-17T20:10:00Z) [system]:
merged
```

The comment is the review culture the team built in Chapter 22, Reviewing a model change (p. 357): it names the problem ("late" assumes a schedule nobody wrote down), cites the requirement, and offers a concrete fix that keeps the uncertainty visible as an assumption to confirm with the prime.

At 14:20 Tom tried to merge. GitLab refused: the project requires every discussion on a merge request to be resolved first (Chapter 15, Setting up GitLab (p. 226)), and Bob's thread was open. Tom applied the suggestion; Bob resolved the thread at 14:50 and approved; Jim, as the code owner for interfaces, had already approved; Tom merged at 15:10. The same click closed task #123, because the merge request description said "Closes #123".

The merge request's page shows its checks as GitLab's widgets would:

```
perl "$KIT/bin/gitlab.pl" --local gitlab --project tidewater/model mr show 24 | head -8
```

One rough edge is visible there, and the book does not hide it: the test widget lists the requirement gaps the government has already reported to the prime as *failed tests*, even though the pipeline is green, because the trace report marks every gap, old or new, as a failure. The pipeline itself only fails on new gaps. Until the report is changed to mark inherited gaps as skipped, read the widget's "failed" count together with the trace job's verdict line.

39.4 15:30 – the day closes

Bob merged !23 at 15:30, which closed Bob's task #121. By the end of the day both changes were on `develop`, both had passed their checks on the merge request, both had been reviewed and approved by someone other than the author, and the board, the burn-down and the model site all showed it without anyone writing a status report. The nightly schedule would run the full check again at 02:00.

> *Six things happened before lunch and I knew about every one of them from GitLab, not from email.*
>
> — Bob, 17 December 2026
```

The next morning, Friday, Tom's merge request !25 failed a different check: the secrets scan found a test access token pasted into a helper script. By 11:30 it was fixed: revoke the token, issue a new one as a masked CI/CD variable, and rewrite the branch so the token never reached `develop` (Appendix E, Rescue: when something goes wrong (p. 724)). The sprint 6 retrospective's "stop" list has one line that came from that morning: "Pasting tokens into scripts, even test ones."

39.5 The same Thursday, 2023, the Cameo way (notional)

Note: This half is notional

Nothing below comes from the simulation. It is a composite of how the team described their Cameo-era days in Chapter 2, The Cameo years (p. 10), written as one day so it can be compared. Treat the times as typical, not measured.

Table 39.1: One Thursday, three years apart. The left column is notional; the right column is the evidence above.

Time	December 2023, Cameo and documents	December 2026, GitLab and SysML v2
08:30	Email from the prime: the model export for the review will be late.	GitLab To-Do: the nightly check failed; an issue is open by 09:15.
09:00	Status meeting, forty minutes, everyone reads their slide.	Stand-up, fifteen minutes, three lines in chat, drafted from GitLab.
10:00	Bob checks out a Cameo project lock; Tom has it, Bob waits.	Bob opens a draft merge request; the pipeline runs on it in minutes.
11:00	Bob edits the activity diagram; layout takes most of the hour.	The pipeline finds a reworded requirement before anyone else sees it.
13:00	Screenshots pasted into a review briefing, slide by slide.	Bob reads Tom's change in the generated docs and leaves a line comment.
14:30	The review comments come back as a marked-up PDF.	The comment is a suggestion Tom applies with one click.
15:30	Someone asks which requirements are unverified; "I'll get back to you".	The trace answer is on the model site, current as of the last merge.
16:30	A requirement changed in DOORS last month; nobody noticed.	Every DOORS difference is checked on every change.

The point of the comparison is not that the old day was lazy. The engineers worked as hard in 2023. The difference is where the effort went: in 2023 most of it went into moving information between tools and people by hand, and the checks happened at reviews, weeks later. In 2026 the moving and checking are done by the pipeline on every change, and the people spend their day on the model and on each other's work.

Exercise 39.1: Write your own two-column day

Pick one ordinary day from last month. Write what you did, hour by hour, in the left column. In the right column write what the same work would look like with a merge request, a pipeline and a model site. Which rows would change most? Which would not change at all?

39.6 You can now

- Describe a normal working day in a GitLab model team, hour by hour, with the evidence a manager can check.
- Explain why a draft merge request that fails its pipeline is good news, not bad.
- Read a merge request's widgets and know which numbers are new problems and which are inherited ones.

39.7 Applying this to your project

Copy the shape: a To-Do list instead of email, a nightly full check, draft merge requests, required discussion resolution, suggestions in reviews. Adapt the times to your team's hours and your stand-up. Decide locally whether a nightly failure is picked up at the stand-up (Tidewater) or by whoever is on call that week.

CHAPTER

40

Documents still matter: review stages for generated documents

From what you know →

You know

color-team reviews and document maturity grades.

→ Here it is called

review stages for generated documents, as doc:: labels.

→ What is new

some of the criteria are checked by the pipeline, so a grade means something before the reviewers open the document.

Survival kit: Where this chapter fits

What you have

a model that generates most of what used to be hand-written – the data exchange table, the trace report, the verification matrix – and a handful of documents that are still written by people: the briefing memo, the request for enablers, the drop disposition.

What you need

a shared, simple way to say how finished a document is, that management already understands, and a clear line between what the pipeline can check and what still needs a human reader.

A model does not abolish documents. Management still reads memos; the customer still receives deliverables; the prime still gets formal letters. What changes is that most of a document's *content* now comes from the model and is regenerated on every change, so the question "is this document ready?" splits in two. Part of the answer is mechanical and the pipeline can give it. Part of it is judgment, and only a reviewer can.

The team kept the document maturity scale the program already used: the *Document Review*

Stages chart, a colour and a grade for each stage of a document's life. Everyone on the program knew it, and the point of this chapter is that the team did not invent a new vocabulary for documents just because the model moved to GitLab.

40.1 The scale

Table 40.1: The Document Review Stages, from the program's grading chart.

Colour	Stage	Grades	Percent complete	What it means
(none)	TODO	–	0	The document is in the queue.
Grey	Collection	G-, G, G+	1 to 24	A loose collection of content has been started.
Blue	Compliant	F-, F, F+	25 to 45	The outline is complete.
Pink	Complete	D-, D, D+	46 to 63	The content is stable and consistent.
Red	Compelling	C-, C, C+	64 to 78	Graphics, tables, data, spelling, acronyms and grammar are finalized.
Gold	Comprehensive	B-, B, B+	79 to 90	The document is sound, coherent and actionable.
White	Copy Ready	A-, A, A+	91 to 99	It is ready for hard-copy review.
Green	Done	–	100	It is releasable.

Note: Two versions of the chart

The deck the program circulates has two slides. The first (above) gives the full grading scale. The second gives shorter descriptions with different percentage bands – for example, it calls Pink 45 to 75 percent and uses Green for "analysis underway" and White for "done". This book follows the first slide, which is the more detailed one. If your program uses the chart, agree which slide governs before the first document is graded.

40.2 The stages as GitLab labels

A document that people write is tracked by an issue, the same as any other piece of work. Its stage is a *scoped label* – `doc::grey`, `doc::blue` and so on – and because the labels are scoped, an issue can carry only one of them at a time: moving a document to Pink removes Blue automatically. The labels are defined in the project file (Chapter 15, Setting up GitLab (p. 226)); the document's source lives in the repository, and each change to it goes through a merge request like a model change.

Warning: The label descriptions in the template do not match the chart

The project file template (`tools/gitlab/templates/project.conf`) describes the `doc::` labels in proposal colour-team terms – for example, Red as "the reader's view, scored as the customer would score it". The program's chart above describes Red as "graphics, tables, data, spelling, acronyms and grammar are finalized". When you adopt the labels, copy the descriptions from the chart your program uses, so the label tooltip in GitLab and the chart on the wall say the same thing.

40.3 Sprint 6: two documents through the stages

In sprint 6 the team produced two documents for the program review: the twelve-week progress memo (Chapter 37, Writing it down: DoD-style memos from Vim (p. 545)) and the data exchange table, which is generated from the model's interface definitions. Each had an issue, and Jim moved each issue through the stages as reviews finished. You can read the history in the simulation's GitLab:

```
perl "$KIT/bin/gitlab.pl" --local gitlab --project tidewater/model issue show 129 | grep -E
"^#|doc::"
```

```
perl "$KIT/bin/gitlab.pl" --local gitlab --project tidewater/model issue show 130 | head -3
```

Both documents went from Grey on 16 December to Green on 24 December, the day of the review. The simulation moves them one stage per working day; real documents usually spend longer in Pink and Gold, where the arguments happen. What matters is not the speed but that the stage is visible on the board, dated, and attributed: anyone could see on Monday that the memo was still Red and the table was already Gold.

40.4 What the pipeline can check, and what it cannot

The stages were written for hand-made documents, where every one of them was a human judgment. For a document built from a model, several of them become checks that run on every change. The line between the two is the useful thing to agree on.

Table 40.2: Mechanical checks and human judgment, stage by stage.

Stage	What the pipeline can check	What still needs a person
Grey	The file exists in the repository; it renders.	Whether this is the right document to write.
Blue	Every required heading is present; required header fields (for a memo: Memorandum for, From, Subject).	Whether the outline answers the reader's question.
Pink	No placeholders left; every number cited from evidence, not typed; no TODO markers.	Whether the content is consistent in meaning, not just in form.
Red	Generated tables are current (the docs job); figures render; banner and portion markings present.	Spelling and grammar in the prose; whether the graphics say what the text says.
Gold	Nothing new.	Whether it is sound, coherent and actionable – the management review.
White	The PDF builds cleanly; the marking rolls up correctly.	A last read on paper.
Green	The release or merge is recorded; the issue is closed.	The decision to release.

The memo tool shows the Pink line directly. Run its check on a fresh copy of the progress memo template, before anyone fills it in:

```
cp "$KIT/tools/gitlab/templates/memos/progress-memo.md" .
perl "$KIT/tools/memo/memo-build.pl" --check progress-memo.md || echo "exit status $?"
```

A memo that still has placeholders cannot be Pink, whatever its author feels. That is the kind of rule the pipeline is good at. Whether the memo's ask is the right ask is not.

For generated documents the mechanical part is even larger. The data exchange table is rebuilt from the model on every merge request, and the docs job fails if the committed copy is stale (Chapter 20, The pipeline, job by job (p. 302)). So the table can never be "Red" in the sense of out-of-date data: if the data were stale, the merge request would not merge. What review adds for a generated table is the reader's question – does this table say what the interface owners need to know? – and that question belongs in Gold.

Exercise 40.1: Grade a document both ways

Take a document your team is writing now. Grade it on the chart by hand. Then list which of the "pipeline can check" items in the table above it would pass today. Where the two disagree, which one is right?

40.5 You can now

- Track a document's maturity on the board with scoped labels, using the stages the program already knows.
- Separate the mechanical part of a document review from the judgment part, and automate the mechanical part.
- Explain to management why a generated table cannot be "out of date" once it is checked on every merge request.

40.6 Applying this to your project

Copy the idea of one scoped label per stage and of moving a document's issue as reviews finish. Use your own program's stage names, colours and descriptions – if your program has no chart, the proposal colour-team names in the kit's template are a reasonable start. Decide locally which slide or version of a chart governs, and who is allowed to move a document to Gold and to Green.

CHAPTER

41

For the lead: leading a reluctant team

From what you know →

You know

leading engineers through a schedule crunch.

→ Here it is called

leading a reluctant team through a change of method.

→ What is new

learning time is protected explicitly, and small wins are made visible.

Survival kit

What you have

a directive you did not ask for, a deadline, two or more good engineers who did not ask for it either, and the same lack of experience they have.

What you need

a way to lead the change without pretending to know more than you do: a team that feels safe to be slow and wrong for a few weeks, protected time to learn, early wins that are real, a workload that does not quietly become nights and weekends, and a management chain that hears dated plans instead of excuses. This chapter is Jim's view of the twelve weeks, written for whoever is in Jim's chair on the next project.

41.1 What Jim was handed

The directive arrived on a Thursday (Section 3.1, The memo (p. 22)). By the following Tuesday Jim had read it four times, sat through the program manager's explanation, and come out with three facts. The program would adopt DevSecOps practices for its systems engineering artifacts. The team would show measurable progress in twelve weeks. And nobody on the team, including Jim, had ever done any of it.

The team's reaction was not subtle.

> *We did documents for twenty years and the system flew. Then they made us do Cameo, and we did it, and now the model is a hairball nobody can review. Now this. What happens in three years when the next thing comes along?*
>
> — Bob, the first team meeting about the directive

Jim's private reaction was close to Bob's. What made the difference over the next twelve weeks was not that Jim became an expert first. It was a handful of decisions about how to lead three beginners, one of them Jim, through a change none of them had chosen. Each is a section of this chapter.

41.2 Make it safe to be slow and wrong

For the first month, everyone on the team would be worse at their job than they had been for years. Bob, who could find any block in the Cameo model blindfolded, would spend twenty minutes on a merge conflict. Tom, who had run the program's CM database for a decade, would push to the wrong branch. Jim would forget the command to create a branch in front of the program manager. That is not a risk to manage; it is a certainty to plan for.

Psychological safety, for a team of three systems engineers, comes down to a few concrete habits:

- **A red pipeline is information, not an accusation.** The pipeline will fail on everyone's changes, often, and most of all in the first weeks. The team agreed, in the first retro, that nobody says "who broke it" in the stand-up; they say "what did it catch". The pipeline's job is to be wrong-detector; a failure means it worked.
- **The lead goes first.** At the sprint 1 kickoff Jim did practice lesson 11 live, on the shared screen, with the whole team watching, including the lesson's planted mistake: the merge request goes red on a missing semicolon. Jim read the log out loud, found the line with the quickfix list in Vim (Section 10.6, Checking from Vim (p. 148)), fixed it and pushed again. It took six minutes. Nobody remembers the semicolon; everybody remembers that the lead's change went red first and that it was fine.
- **"I don't know yet" is an acceptable answer** with a date attached (Section 25.7, Bad news, said so it builds trust (p. 400)). The lead says it out loud in front of management, early, so the team learns that it is safe.
- **No question is a nuisance during pairing.** The rule the team wrote on the wiki: "If you have been stuck for fifteen minutes, ask. If someone asks you, stop and help." Fifteen minutes is short enough that nobody suffers in silence and long enough that people try first.
- **Complaints are heard, then turned into changes.** The retrospective is where "this is pointless" gets said, and Jim's rule was to answer a complaint with one of three things: a change for next sprint, an explanation of why not, or "I'll ask management", with a date.
- **Measure how people feel, not only what they ship.** The pulse survey at every retro (Section 24.7, The retrospective (p. 387)) is five questions, one to five, anonymous. It costs two minutes and it is the only early warning the lead gets.

> **Case study: The semicolon**
>
> Jim nearly did not share the screen for that lesson. The plan had been to rehearse it the night before, so the demonstration would go smoothly. Jim decided against it: the team needed to see a failure handled, not a performance. In the sprint 1 retrospective, Tom wrote on the "continue" list: "Jim breaking things first."

41.3 Protect the time to learn

The most important decision Jim made was to put learning into the plan as work, with points, where everyone could see it.

The temptation is the opposite. Management wants measurable progress in twelve weeks; every hour spent on a practice lesson is an hour not spent closing TODO lines. So the learning happens "on the side": at lunch, after hours, at weekends. It always does happen, because the work cannot be done without it. But then the plan is a lie (the team's capacity looks bigger than it is), the people doing the learning resent it, and the first time the schedule slips, the learning is what gets cut.

Tidewater's plan carries learning as stories, labelled `work::learning`, with a reserve that falls as the team gets better: 24% of the task points done in sprint 1 (practice lessons 10 and 11, the survival kit, git from zero, Vim for models), less in sprint 2 (lessons 12 and 13, the diagrams chapter, the first half of the SysML v2 workbook), and a small remainder by sprint 6. Part III gives the real numbers for each sprint.

Making learning visible takes one label. In GitLab, a learning story looks like any other (here in the `work` folder of practice lesson 11's sandbox):

```
. ../env.sh > /dev/null
for who in jim bob tom; do
  gl --now 2026-10-05T09:30 --as jim issue new "Learning: practice lessons 10 and 11 ($who)"
  --label type::story --label work::learning --milestone "Sprint 1" --assign $who --weight 2 >
  /dev/null
done
gl issue list --label work::learning
```

```
#2    opened  ~type::story ~work::learning            Learning: practice lessons 10 and 11 (jim)
#3    opened  ~type::story ~work::learning            Learning: practice lessons 10 and 11 (bob)
#4    opened  ~type::story ~work::learning            Learning: practice lessons 10 and 11 (tom)
```

Three things follow from making it visible:

1. **The plan is honest.** Velocity in sprint 1 is low, and everybody knows why. When the program manager asked in the first review why only part of the conversion was done, Jim pointed at the learning stories: "That is the training you approved, in the plan where you can see it."
2. **The learning is protected.** A story on the board is harder to cut than an evening nobody sees. Jim asked for it formally in the request-for-enablers memo (Section 37.6, Tidewater's

memos (p. 552)): four protected hours a week per person, with what happens if not ("the conversion burn-down slows by about a third").

3. **The shrinking reserve is itself a result.** The scorecard shows learning time falling as competence rises. Management reads that as the investment paying off, which it is.

Tip: The lead learns first

Jim did practice lessons 10 and 11 in week zero, before the team, alone, and got stuck twice. That made the first pairing sessions easier: Jim could say "I got stuck there too; here is what helped." A lead who has not done the lessons can only say "it's in the book".

41.4 Pair at the terminal

Pairing is two people at one keyboard: a **driver** who types and a **navigator** who reads, thinks ahead and catches mistakes. They swap every twenty minutes or so. For a team learning a new toolset together it is the fastest way to spread skill, because the person who knows a thing teaches it while doing real work.

Tidewater's pairing in the twelve weeks:

Table 41.1: Who paired with whom

When	Pair	On
Sprint 1	Tom drives, Bob navigates	the first git commands in the practice repository: branch, commit, push
Sprint 1	Bob drives, Jim navigates	the first conversion report and the TODO inventory
Sprint 2	Tom drives, Jim navigates	the first pipeline, and the PowerShell failure on the real runner
Sprint 2	Tom drives, Bob navigates	the first vendor-branch import; Bob then reviews and merges its merge request
Sprint 3	Jim drives, Bob navigates	the DOORS import and the first trace report
Sprint 4 onward	as needed	conflicts, new job types, the first release

Two lessons from that table. First, the person who knows *less* drives. When Tom drove the first git commands with Bob navigating, Tom learned git and Bob learned how to explain it, which is the step where Bob's own understanding became solid. Second, the lead pairs too, as navigator and as driver. Jim at the keyboard with Bob navigating was the moment Bob stopped treating the new tools as Jim's project and started treating them as the team's.

Pairing feels slow. Two people on one task looks like half the throughput. In the first weeks it is the opposite: two beginners together get unstuck in minutes, where one alone loses an afternoon. The team stopped pairing by default in sprint 4, when it was no longer needed for routine work, and kept it for anything new.

41.5 Pick the first wins on purpose

A discouraged team needs evidence, early, that the new way does something the old way could not. The lead's job is to make sure the first wins are real, are noticed and are the team's.

Jim chose the first win before sprint 1 started: run the trace report on drop 1 and look for places where the prime's DOORS text and the model disagreed. In the document era that comparison was done once before a major review, by hand, over a week, if at all. Bob ran it on the fourth day of sprint 1, from the command line, and found the TW-SYS-006 drift in about two minutes.

> *I found the drift in two minutes. It used to take a week, and we only looked before a review.*
>
> — Bob, sprint 1, day 4 (from the team chat)

That sentence did more for the transition than anything Jim said in twelve weeks, because Bob said it, about Bob's own work.

The pattern for choosing first wins:

- **It answers a question the old way could not answer cheaply.** Drift between DOORS and the model; requirements with no verification; which packages depend on each other in a cycle.
- **The skeptic does it.** A win the lead announces is a claim. A win the skeptic discovers is evidence.
- **It takes days, not weeks.** The first win had to land in sprint 1. The model site for management (sprint 4) was a bigger win, but it would have been too late to be the first one.
- **It is shown, not told.** Jim asked Bob to show the drift in the sprint 1 demo, live, on the command line.

And then the lead makes sure the win is noticed: in the stand-up ("Bob found the drift in two minutes"), in the demo (Bob shows it), in the weekly note to management, and in the retro ("what worked this sprint?").

41.6 Watch the workload

A team learning a new way of working while delivering under a deadline is a team at risk of quiet overwork. Nobody announces it. The signs are small: statuses posted at 6 a.m., merge requests opened at 9 p.m., a stand-up line that says "Y caught up on the vendor merge" on a Monday after a weekend.

Jim's routine for watching it took ten minutes a day:

1. **Read the stand-up statuses for time stamps**, not just content. Work after hours shows in the chat times and in the merge request times.
2. **Check blocked items daily**, because a blocked person often works around the block alone, late. A blocked item that is three days old is the lead's problem, not the engineer's (Section 24.2, The daily stand-up: terse, written, linted (p. 378)).
3. **Commit below capacity.** The team committed about nine-tenths of capacity in each sprint, never all of it; the remainder absorbs the surprises (a prime question, an ISSM review that moves) that always come.
4. **Look at the pulse survey's workload question** at every retro. A drop of one point is a conversation, not a statistic.

The blocked-items check is one command. Here a blocked task is added to the practice project, then listed:

```
. ../env.sh > /dev/null
gl --now 2026-10-05T09:40 --as jim issue new "Register the second runner" --type task --label
type::task --label workflow::blocked --milestone "Sprint 1" --assign tom > /dev/null
gl issue list --label workflow::blocked
```

```
#5    opened  ~type::task ~workflow::blocked          Register the second runner
```

When Jim saw the signs, in sprint 3, the response was not "work smarter". It was to drop scope: one story moved to the next sprint, explicitly, with a note in the weekly report to management saying why.

41.7 Manage up with dated plans

The lead stands between the team and the management chain that issued the directive. Done badly, the lead passes pressure down and excuses up. Done well, the lead passes *plans* up and *protection* down.

What worked for Jim:

- **Agree the measures before the work starts.** In week zero, Jim and the program manager agreed the twelve-week scorecard (Section 3.6, Defining measurable progress (p. 28)): which numbers would show progress, where they started, what the targets were. Every report after that was against an agreed yardstick, so nobody could move the goalposts and nobody could claim success by picking flattering numbers.
- **Report the same numbers every week**, in the same order (Section 25.6, Demos that land (p. 398)), in a one-page note that pairs with the kit's weekly quad. Management learns to read the trend and stops asking for special reports.
- **Ask for enablers formally, with dates and consequences.** The request-for-enablers memo of sprint 2 (Chapter 37, Writing it down: DoD-style memos from Vim (p. 545)) turned "we need more time" into five specific asks, each with what it buys, by when, and what happens if not. What management approved, and when, is told in Chapter 27, Sprint 2: Drop 1 on develop, every change gated (p. 424) and Chapter 28, Sprint 3: the first red X, DOORS in the model, and a wait nobody named (p. 440); an ask that is not approved becomes a known risk, on the record, rather than a surprise.
- **Bring management to the demo, not the demo to management.** The program manager attended sprint reviews from sprint 2 onward. Seeing the merge request go green, live, did more than any slide.
- **Never surprise them.** Bad news went into the next weekly note or the next stand-up summary, the week it was known, with a date for the fix (Section 25.7, Bad news, said so it builds trust (p. 400)).

Objection: "Management just wants the slide with green on it."

Some managers do. But a directive that says "show measurable progress" is a manager asking to be given something they can defend upward. The weekly note with four numbers,

a trend and a dated plan is that thing.

Answer

Give them the numbers in the form they forward. The Tidewater program manager attached Jim's weekly notes to the program's own monthly report unchanged by sprint 4. That is when the slide requests stopped.

41.8 Say no with a date

The lead's most frequent hard conversation is not with the team. It is with a manager or stakeholder who wants something now that is not in the sprint: a special report, a slide, "just a quick change to the model" before a meeting.

Saying yes to every request breaks the sprint and teaches everyone that the plan does not matter. Saying a flat no sounds like obstruction. The middle is **no, with a date**:

Table 41.2: No, with a date

Request	Say
"Can you add the thermal interfaces to the model before Thursday's review?"	"Not this sprint: it's not in the plan and it would push the Stations work. It goes to refinement on Wednesday; the earliest it lands is sprint 5. For Thursday I can show you where it will go."
"I need a one-off report of every requirement the prime changed."	"The delta report does that on every drop. The next one is in January; I'll send you its summary the day after it lands."
"Just update the briefing slide with the new numbers."	"The model site is updated every night with the numbers; here's the link. If you need a slide, the weekly note has them in the order you used last time."
"Can Bob help the other program for a week?"	"Not without moving the conversion: Bob owns the Stations package this sprint. If it's urgent, I'll put it to you as a trade: Bob for a week, Stations moves to sprint 5. Your call."

Each answer has the same parts: what you will not do now, why (in the requester's terms), when it could happen, and often what you *can* offer today. The last row is the most useful form of all: turn the request into a trade and give the decision back to the person who has the authority to make it.

The deputy program manager, who chairs the change board, made exactly this kind of request in sprint 4 (a change to a requirement before the board had seen it). Jim's answer was to open a merge request for it, so the board could approve it in the normal flow the same week. That became the team's standard answer to "just a quick change": a merge request is quick, and it is on the record.

41.9 Know when to slow down

There will be a sprint where it goes wrong. For Tidewater it was sprint 3: Bob's hand edit broke the grammar check on a merge request, the ISSM was unreachable for three days and the

toolchain review stalled, and the pulse survey dropped for the first time. Bob wrote "this is the Cameo year again" in the retro.

The signs that the team needs to slow down, not push harder:

- The pulse survey drops, especially on "I could explain our progress" or "my workload is sustainable".
- Stand-up statuses get shorter and vaguer ("Y stuff T stuff B none"), or stop naming outcomes.
- A red pipeline stays red for a day and nobody mentions it in the stand-up.
- Merge requests get bigger, because people stopped believing in small ones.
- Someone asks, for the first time, whether they could "just do it the old way this once".

What Jim did in sprint 3, and what the book recommends:

1. **Name it, out loud, in the retro.** "This sprint was hard. The numbers show it, and so do you." Pretending it was fine would have cost the trust earned in sprints 1 and 2.
2. **Cut scope, not learning.** One engineering story moved to sprint 4. The learning reserve stayed.
3. **Remove the cause, not the symptom.** The ISSM block was the real problem; Jim booked the ISSM's next two review slots two weeks ahead, so it could not happen again. The grammar failure was fine: the pipeline caught it before it reached anyone else, which is its job, and Jim said so.
4. **Pause new tools.** No new pipeline job or GitLab feature was introduced in sprint 4 beyond what was already planned. A team that is struggling does not need one more thing to learn.
5. **Tell management the same week**, as a dated plan: "Sprint 3 delivered less than planned for two reasons; here is what changes in sprint 4; the twelve-week targets still hold."

Sprint 4's pulse recovered. Slowing down is not losing; it is how the twelve weeks stay twelve weeks and do not become a year of burnout.

41.10 Hand over ownership, on purpose

In week zero, the transition was Jim's project: Jim got the directive, Jim set up the meetings, Jim talked to management. By the end of the twelve weeks it had to be the team's way of working, or it would stop the day Jim went on leave. That does not happen by itself; the lead has to give things away, one at a time, and let them be done differently.

What Jim handed over, and when:

Table 41.3: What the lead gave away

When	What	To	How Jim let go
Sprint 1	the pipeline	Tom	Tom chose the jobs and their order; Jim reviewed but did not redesign
Sprint 2	the vendor-branch import of each prime drop	Tom, with Bob reviewing	the drop runbook became Tom's wiki page
Sprint 3	security owner, with the ISSM	Tom	Tom ran the ISSM meetings; Jim attended the first only

When	What	To	How Jim let go
Sprint 4	the Stations package and its code-owner section	Bob	Bob approves Stations changes, including Jim's
Sprint 4	narrating the demo	rotating	Jim closes with the ask, and only that
Sprint 5	preparing the change board's agenda from the open prime changes	Bob	Bob briefs the deputy program manager; Jim chairs
Sprint 6	the weekly note to management	rotating, drafted with `talk.pl weekly`	Jim reads it before it goes, and changes only facts

Two things made the hand-overs stick. First, each one was recorded where the system enforces it: `CODEOWNERS` names Bob as an owner of the Stations folder, so Jim's own Stations changes wait for Bob's approval. A hand-over that is only spoken is reclaimed the first time the lead is in a hurry. Second, Jim accepted that the new owner would do it differently. Tom's pipeline ran jobs in an order Jim would not have chosen; it worked, and changing it would have taken the pipeline back.

> *The first time my approval was the one holding up Jim's merge request, I thought I'd done something wrong. Then I realized that was the point.*
>
> — Bob, sprint 4 retrospective

41.11 The lead is a beginner too

Jim had twenty-six years of systems engineering and zero weeks of git. The temptation for an experienced lead is to hide the second fact behind the first: to stay in the management role, delegate the tools and never be seen not knowing.

Jim did the opposite, deliberately:

- **Did the practice lessons first**, and told the team where they got stuck.
- **Took real tasks**, with points, every sprint: the DOORS import in sprint 3 was Jim's, from the command line, with Bob navigating.
- **Asked questions in the stand-up**, in front of everyone: "How do I see which pipeline job failed?" Tom answered in one sentence, and two other people learned it.
- **Used the tools the team used.** Jim's weekly note was drafted with `talk.pl`, edited in Vim and built with the same memo tool everyone else used. A lead who keeps a private Word template is telling the team the new tools are for them, not for leads.

The effect was that the team never had to pretend in front of Jim, because Jim was visibly learning alongside them. It also meant Jim's judgment about what was hard was accurate: a lead who has not done the work under-estimates it, and that is how teams end up with plans they cannot meet.

41.12 The lead's fortnight

Most of the lead's work in the new way is a rhythm, not heroics. Jim's two weeks, once it settled down in sprint 4:

Table 41.4: Jim's two weeks

When	The lead	Time
Every day	reads the stand-up chat (times as well as content); checks blocked items; answers one question in the chat or pairs	about 20 minutes
Day 1	runs the kickoff: the goal, capacity including the learning reserve, the commitment	90 minutes
Days 2 to 8	takes one or two real tasks; reviews merge requests in their code-owner areas	about half the time
Day 9	refinement and planning poker; the demo dry run; the ask for Friday written down	2 hours
Day 10	closes the demo with the ask; runs the retrospective and its pulse survey	90 minutes
Every Friday	reads the weekly note before it goes; sends it with the quad	15 minutes
Every second week	thirty minutes with the program manager: the four numbers, the trend, the open asks	30 minutes

What is *not* on the list is as important: status meetings, slide preparation and chasing people for updates. The records the meetings produce replaced them (Section 24.1, Why meetings at all (p. 377)). That is the time the lead gets back, and Jim spent it on two things: real modeling work, and the conversations with management that protect the team.

41.13 Exercises

Exercise 41.1: Budget the learning

Plan the first three sprints of your team's transition with an explicit learning reserve: what each person learns, which practice lessons and chapters, how many points per sprint. Write it as stories with a learning label. What engineering scope did you have to move to make it fit?

Exercise 41.2: Choose the first win

List three questions about your current model that the old way could only answer slowly. For each, name the tool that answers it in minutes, and the skeptic on your team who should be the one to run it. Which one can land in sprint 1?

Exercise 41.3: Say no with a date

Write your answer to three requests you expect from management in the first month (a special report, an urgent model change, borrowing a person). Each answer: what not now, why, when, and what you can offer today, or the trade.

Exercise 41.4: Read the signs

Look back at your team's last two weeks of statuses, merge requests and chat. Which of the slow-down signs do you see? What would you change in the next sprint, and what would you tell management?

41.14 Applying this to your project

- **Copy**: the lead goes first (shares their own failing change); a visible learning reserve as labelled stories; pairing with the less experienced person driving; first wins chosen on purpose and shown by the skeptic; the daily ten-minute workload check; the measures agreed with management before the work starts; no with a date.
- **Adapt**: the size of the learning reserve to your team's starting point (a team that already uses git needs less); the pulse survey questions to what your team worries about; the format of your weekly note to what your management already reads.
- **Decide locally**: who in management gets the weekly note and attends the demo; how much scope the lead may move without asking; what the team does when the lead is away (someone has to say "I don't know yet" first).

Note: You can now…

- make it safe for a team to be slow and wrong while it learns, starting with yourself;
- put learning in the plan as visible, protected, shrinking work;
- pair to spread skill quickly, and stop pairing when it is no longer needed;
- choose first wins that convince skeptics because the skeptics find them;
- watch for quiet overwork and respond by cutting scope;
- manage up with agreed measures, dated plans and formal requests for enablers;
- say no with a date, and turn requests into trades;
- recognize a sprint that has gone wrong and slow down on purpose.

PART V

Living with it

The first twelve weeks are a sprint in both senses: a burst of effort, watched closely, with a briefing at the end. What follows is longer and quieter. Tidewater's model has to stay correct for years after the briefing, through staff changes, server upgrades, expiring tokens, a contractor who keeps delivering Cameo drops, and the slow temptation to stop doing the dull parts because nothing has gone wrong lately.

This part is about that long tail. Chapter 42, Administration (p. 586) covers the recurring administration that keeps the environment trustworthy: who has access, which tokens exist, whether the runner is patched, whether a backup can actually be restored. Chapter 43, Maintenance through the life cycle (p. 602) covers the engineering rhythm after the port: taking in each prime drop, cutting baselines, fixing a released baseline in a hurry, and retiring the tools that only the port needed. Chapter 51, Pitfalls and anti-patterns (p. 684) collects the ways teams like Jim's go wrong when they move from documents and Cameo to a model in GitLab, each with the symptom you will notice first, why it happens, and what to do.

None of it is glamorous. All of it is what separates a demonstration from a capability. The program manager put it to Jim after the sprint 6 briefing, in a sentence Jim wrote on the whiteboard and left there: "Now show me it still works in a year."

CHAPTER

42

Administration

From what you know →

You know

the CM manager's routine: accounts, access lists, audits and backups.

→ Here it is called

administering GitLab, runners and tokens.

→ What is new

most of the routine becomes a scheduled check that reports drift.

Survival kit: The upkeep

What you have

a GitLab project that works, a runner that runs, three people who know their way around both, and a model that passed its first baseline.

What you need

a short list of recurring chores, each with a cadence, an owner and a checklist, so the environment is still trustworthy a year from now when nobody remembers how it was set up.

42.1 The Friday the nightly pipeline went red

Eleven weeks after the briefing, on a Friday morning, Tom opened the project page and saw the nightly status pipeline in red. Nothing in the model had changed since Wednesday. The job log ended with a single line from the export step: the server had answered `401 Unauthorized`.

It took Tom twenty minutes to find the cause, and most of that time was spent remembering where things were. The scheduled pipeline exported GitLab's records for the status metrics, and to do that it used a project access token stored in a masked CI/CD variable. The token had been created during week zero with GitLab's default expiry. It had expired at midnight.

The fix took two minutes: create a new token, paste it into the variable, re-run the job. The lesson took longer to absorb. In the document era, the team's tools had not expired. A Word template did not stop working on a date, and a shared drive did not need its permissions reviewed for the system to keep running. The new environment is made of parts that need upkeep: credentials that age, a runner whose operating system needs patches, protections that can drift when someone with the right role clicks the wrong box, backups that are worthless until someone restores one.

That afternoon Jim and Tom wrote down every recurring chore they could think of, with how often it had to happen and who would do it. The list became a page in the wiki and a standing story on the board. This chapter is that list, explained.

Note: Administration is configuration management

None of this is new in kind. A CM office has always kept access lists, verified backups, audited baselines and tracked tool versions. What is new is that GitLab records most of it automatically, so the chore is usually a review of a record, not the creation of one.

42.2 The administration calendar

Every item in this chapter appears in one table. Tom printed it and taped it beside the monitor; it also lives in the wiki as the Administration page (the version the team started from, `wiki/Administration.md`, is shown in Appendix I, Templates and GitLab Markdown, field by field (p. 747)).

Table 42.1: The administration calendar

Chore	Cadence	Owner	Typical time	Evidence it was done
Access review: members, roles, the prime's accounts	quarterly, and the same day someone leaves	Jim	30 min	a comment on the Admin story with the member list
Token inventory and rotation	monthly check; rotate before expiry	token owner, checked by Tom	15 min per token	the inventory table in the wiki, updated
Runner patching: operating system, gitlab-runner, Git for Windows, the kit	monthly	Tom	1 hour	a green pipeline after the update; versions in the runner page
Runner health: last contact, disk, job queue times	weekly	Tom	10 min	a note in the Friday quad if anything is wrong
Pipeline schedules: owner current, last run passed	monthly, and on every departure	Tom	5 min	the schedules page; owner noted in the wiki
Container image rebuild and scan (only if the Docker image is used)	monthly	Tom	30 min	a new image tag in the registry
Protection audit: settings versus `project.conf`	monthly, and after any admin change	Tom	20 min	the settings pages match `project.conf` (`setup` re-applied if not)
Project export (backup of issues, MRs, wiki)	each baseline, and monthly	Tom	20 min	the export file in the package registry, with its checksum
Restore test	quarterly	Tom, watched by Jim	2 hours	a scratch project restored and its pipeline green

Chore	Cadence	Owner	Typical time	Evidence it was done
Audit event review	monthly (Premium)	Jim	20 min	a comment on the Admin story
Milestones two sprints ahead	each sprint planning	Jim	5 min	the next two milestones exist
Label and board tidy-up	each refinement	Jim	10 min	no unlabelled open issues
Upgrade watch: GitLab release notes for CI and API changes	monthly	Tom	30 min	a note in the wiki's Administration page
CODEOWNERS matches the team	on every membership change	Jim	5 min	an MR changing `CODEOWNERS`

The total is about four hours a month in a normal month and a day in a quarter-end month. Tom tracked it with `/spend` on the standing Admin story so the cost was visible in the sprint report rather than hidden in overtime.

Tip: Make the chores into issues

A chore that lives only in someone's head stops happening when that person is on leave. The team created one story per sprint titled "Administration, sprint N" from a template with the sprint's chores as a checklist. It was deliberately small (two or three points) and it never carried over: an unchecked chore was moved to the next sprint's story with a comment saying why.

42.3 Members and access reviews

42.3.1 The five roles

GitLab gives every member of a project one of five roles. The names sound like a hierarchy, and they are, but what matters for a model project is what each role can do to the controlled configuration.

Table 42.2: GitLab roles as Tidewater uses them

Role	Who on Tidewater	Can do	Cannot do
Guest	nobody (sometimes a reviewer from another program)	see issues, comment	see the repository on a private project
Reporter	the prime's modeler, the ISSM, the program office	read the model, issues, MRs, pipelines; comment; download packages	push, merge, change settings
Developer	Bob	push feature branches, open and approve MRs, run pipelines, merge into `develop` where the branch rule allows	push to protected branches, change settings or protections, create protected tags
Maintainer	Jim, Tom	everything a developer can, plus merge into `main`, create `baseline-*` tags, manage CI/CD variables, protections and members	delete the project, transfer it (owner only)
Owner	the group owner (the enterprise admins)	everything	n/a

Two choices in that table were argued over.

Bob is a developer, not a maintainer. Bob objected at first: Bob does most of the model work and felt a lower role was a statement of trust. Jim's answer was about separation of duties, the same principle that kept engineers from signing their own CCB approvals in the document era. A developer can do everything the daily work needs. The maintainer-only powers are the ones that change the rules (protections, tokens, baselines), and the team agreed those should sit with two people, neither of whom does most of the model edits.

The prime's modeler is a reporter. The prime's modeler can read the government model and comment on issues, which makes discrepancy reports a conversation instead of an email chain. The prime's modeler cannot push, because the government v2 model is the government's record; the prime's changes arrive as drops on the vendor branch (see Chapter 43, Maintenance through the life cycle (p. 602)).

42.3.2 Adding a person

Checklist: Onboarding one person

- ☐ Account exists on the enterprise GitLab (the enterprise admins create it; the person signs in once)
- ☐ Added to the project with the lowest role that lets them do their job (`[members]` in `project.conf`, then `gitlab.pl setup`)
- ☐ SSH key or personal access token set up on their laptop (Chapter 9, The survival kit (p. 126))
- ☐ Added to `CODEOWNERS` if they will own a package, by MR
- ☐ Added to `roster.txt` in the journal (`daily.pl roster add …`) and to the stand-up invitation
- ☐ Walked through the Onboarding wiki page; first task is a small one with a pairing partner
- ☐ Their first MR merged within their first week

Membership is in `project.conf` so that the list is reviewed like any other change. When Jim added the deputy program manager as a reporter in sprint 8, it was a two-line MR: one line in `[members]`, one line in the wiki's contact page, approved by Tom.

```
# in a clone of the project configuration (or the model repo, where project.conf lives under
tools/)
vim project.conf                          # add:  dpm = reporter
perl bin/gitlab.pl --url https://gitlab.example.com --dry-run setup project.conf    # every
request the file implies
perl bin/gitlab.pl --url https://gitlab.example.com setup project.conf              # apply it
```

42.3.3 Removing a person

Offboarding is the one chore with a same-day deadline. Access that outlives its need is the most common finding in any security review, and it is the easiest to prevent.

Checklist: Offboarding one person (the same day)

- ☐ Removed from the project and from the group (a group membership grants the project too)
- ☐ Their personal access tokens revoked by an admin, or confirmed expired
- ☐ Any CI/CD variable they could read (maintainers can read all of them) rotated
- ☐ Their pipeline schedules taken over by a current member (a schedule runs as its owner)
- ☐ Their open issues and MRs reassigned, with a comment saying to whom and why
- ☐ `CODEOWNERS` updated if they owned a package (the approval rule would otherwise wait for someone who is gone)
- ☐ Removed from `roster.txt` and the stand-up invitation
- ☐ The runner machine's local accounts checked if they had one

42.3.4 The quarterly access review

Every quarter Jim opens the project's member list and compares it with reality: who is on the team, who still needs read access, which accounts have not signed in for ninety days. The review takes half an hour and produces one comment on the Admin story, listing the members and any change made. Over eighteen months the review removed four accounts: a program-office analyst who had moved on, two of the prime's staff who no longer worked on the model, and a test account Tom had created during week zero and forgotten.

Warning: Inherited membership is invisible from the project page

A person who is a member of the `tidewater` group is a member of every project in it, with the group role. When you review a project's members, include the inherited ones (the member list shows them with the group's name beside them). Most forgotten access is inherited.

42.4 Tokens and rotation

42.4.1 What a token is, in plain terms

A token is a password issued to a program rather than to a person. When the pipeline needs to call GitLab's API, or a script on Tom's laptop needs to create issues, it cannot type a password into a login page. It sends a long random string instead, and GitLab checks the string against its list of tokens, each with an owner, a set of permissions (called *scopes*) and an expiry date.

GitLab has several kinds of token. A team new to all this needs to know which kind to use where, because using the wrong kind is the commonest security mistake.

Table 42.3: Kinds of token and where Tidewater uses each

Token	Belongs to	Use it for	Tidewater example
Personal access token	one person	scripts that person runs from a laptop	Jim's `gitlab.pl` calls from Vim
Project access token	a bot user in one project	automation that must outlive any one person	the nightly status export
Group access token	a bot user in a group	automation across several projects (Premium on gitlab.com)	not used

Token	Belongs to	Use it for	Tidewater example
Deploy token	the project	read-only access to the repository or packages from outside	the program office's read-only mirror
`CI_JOB_TOKEN`	one running job	calls from inside a pipeline job; expires when the job ends	uploading a prime drop to the package registry
Runner authentication token	one runner	the runner talking to GitLab	`glrt-…` in the runner's `config.toml`
Pipeline trigger token	the project	starting a pipeline from outside	not used

The rule the team adopted fits on one line: *use `CI_JOB_TOKEN` inside the pipeline whenever it can do the job; use a project access token for automation; use personal tokens only for a person's own interactive work.* `CI_JOB_TOKEN` is the best kind because nobody has to store, rotate or protect it; it exists only while a job runs.

42.4.2 Scopes: the least that works

Each token carries scopes that limit what it can do. The scope names are GitLab's: `read_api`, `api`, `read_repository`, `write_repository`, `read_registry`, and a few others. A token for the nightly export only reads, so it gets `read_api` and nothing else. A token that creates issues needs `api`, which is broad: it can do anything the token's user can do through the API. Give `api` only to tokens whose bot user has a role no higher than the work needs (a project access token can be created with the *reporter* or *developer* role, not just maintainer).

42.4.3 Where tokens live

A token used by the pipeline is stored as a CI/CD variable in the project's settings (Settings, CI/CD, Variables), with two flags set:

- **Masked**: GitLab replaces the value with `[MASKED]` if a job prints it to the log. This protects against the most common leak, a debugging `echo` left in a script.
- **Protected**: the variable is only given to pipelines on protected branches and tags (`develop`, `main`, `baseline-*`). A feature branch pipeline does not see it, so a change on an unreviewed branch cannot read it.

The nightly status job runs on `develop`, so a protected variable is fine for it. A job that must run on feature branches cannot use a protected variable; if such a job needs a secret, that is a design smell worth a conversation with the ISSM.

Tokens never go in files in the repository, not even in a branch that will be deleted: git keeps history, and the secret detection job (Chapter 20, The pipeline, job by job (p. 302)) exists to catch exactly that mistake. On laptops, personal tokens go in an environment variable set by the user's shell profile (`GITLAB_TOKEN` for `gitlab.pl`, as `gitlab.pl help` says), never in a script.

42.4.4 The token inventory and rotation

Tom keeps a table on the wiki's Administration page, one row per token: what it is for, its kind, its scopes, the variable or place where it is stored, who owns it, and when it expires. GitLab shows the expiry dates too, and sends email a week before a personal or project token expires, but the

email goes to the token's owner, and a bot user has no inbox anybody reads. The inventory is how the Friday incident is prevented.

Checklist: Rotating a project access token

- ☐ Create the new token (Settings, Access tokens) with the same scopes and role, expiry at most ninety days out
- ☐ Paste the value into the CI/CD variable (Settings, CI/CD, Variables), keeping Masked and Protected set
- ☐ Run the job that uses it (`gitlab.pl pipeline run develop --source web`, or the Run pipeline button) and check it passes
- ☐ Revoke the old token
- ☐ Update the expiry date in the inventory table

Ninety days was a compromise. The ISSM's first position was thirty days, which would have meant a rotation every other week for the team's three tokens. Tom showed the inventory and the checklist, and the ISSM agreed to ninety days with a monthly check, on the condition that the rotation itself was recorded. It is: each rotation is a comment on the Admin story.

42.4.5 When a token leaks

If a token is ever seen where it should not be (pasted into an issue, committed to a branch, printed in a log before masking was set), the order of actions matters:

1. **Revoke it first.** Settings, Access tokens, Revoke. Everything else can wait five minutes; this cannot.
2. Create a replacement and update the variable.
3. Find where it appeared and remove it (edit the comment; for a commit, the secret is in history even if the file is fixed, which is why revocation comes first).
4. Check the audit events for use of the old token since it leaked (Premium), or ask the instance admins.
5. Tell the ISSM, with the times. A leak reported in an hour is an incident handled; a leak found later is a finding.

The team never had a real leak. They rehearsed the procedure once, in sprint 9, with a deliberately created and deliberately "leaked" token on a scratch project, and timed it: seven minutes from "found" to "reported".

42.5 The runner

42.5.1 What the runner is, and what this section covers

The *runner* is the machine that executes pipeline jobs. GitLab the server runs nothing itself: when a pipeline starts, the server queues its jobs, and a runner program on some machine asks for work, runs the job's script and sends back the log and the result. With no runner available, jobs sit in *pending* forever, which is exactly what happened to Tidewater's first pipeline for the two days before Tom's runner was registered (Chapter 26, Sprint 1: A GitLab project the team can work in (p. 404)).

Chapter 21, Runners: where the pipeline runs (p. 347) explains runners from the beginning: scopes, executors, registration, tags and why Tidewater chose a project runner with the Windows shell executor on a dedicated virtual machine, running the same Git for Windows Perl as the laptops. This section is about keeping that runner healthy for years: the chores, their cadence, and what breaks when they are skipped.

42.5.2 The runner's set-up, as a record

Tom kept the runner's installation as a short runbook in the wiki, because the second runner (below) had to be built the same way and so will its replacement after the virtual machine is retired. The essentials:

1. A dedicated Windows virtual machine from the enterprise, never a person's laptop: a laptop sleeps, travels and is reimaged, and every MR would wait for it.
2. Git for Windows at the laptops' version; `perl -v` matches; the kit cloned to `C:\agile` at the agreed tag and its tests passing there (`perl tests/run.pl`).
3. A local service account, `svc-runner`, with no administrator rights and no interactive logon; gitlab-runner installed as a Windows service running as that account.
4. The runner created in GitLab (Settings, CI/CD, Runners, New project runner) with tags `windows`, `shell` and `mbse`, then registered on the machine with `gitlab-runner register --url https://gitlab.example.com --token glrt-…` and the `shell` executor.
5. `config.toml` adjusted (below), the service restarted, a trivial branch pushed and its first job watched from pending to passed.

```
concurrent = 2
[[runners]]
  name = "tidewater-win-1"
  url = "https://gitlab.example.com"
  executor = "shell"
  shell = "pwsh"
  environment = ["KIT=C:/agile"]
  builds_dir = "D:/runner/builds"
```

The shell executor on Windows runs each job's script in PowerShell (`pwsh`) unless told otherwise, and PowerShell reads `$KIT` differently from Git Bash and has no `$(find …)`. Rather than depend on a particular shell, every line of Tidewater's pipeline is one shell-neutral command, `perl ci/job.pl NAME`, and the Perl script does the work. The same `.gitlab-ci.yml` therefore runs under `pwsh` on this runner, under Git Bash on a laptop with `gitlab.pl ci`, and inside the optional Docker image. The `KIT` variable reaches the script through the environment in every case.

42.5.3 Least privilege on the runner

A runner executes whatever is in `.gitlab-ci.yml` on the branch being tested, so anyone who can push a branch can run commands on the runner machine as the runner's account. That is the main security fact about runners, and the ISSM asked about it in the first meeting. The team's answers, each a setting:

- **The service account has no admin rights** and cannot log on interactively, so a job cannot install software or read other accounts' files.

- **The runner is a project runner**, registered to `tidewater/model` only, not shared with projects whose members the team does not know.
- **The runner machine holds no secrets** beyond its own `glrt-` token. Tokens jobs need come from protected, masked CI/CD variables, which feature-branch pipelines never see.
- **Only members can push.** The prime's modeler is a reporter and cannot push, so the prime cannot run code on the government runner.
- **The machine follows the program's workstation baseline**: patching, antivirus, logging.

GitLab's *protected* runner flag makes a runner take jobs only from protected branches and tags. The checking runner cannot be protected, because merge request pipelines run on feature branches and checking those is the point. From sprint 8 the team ran a pattern the ISSM liked: the general runner, unprotected, for checks; and a second, protected runner with an extra tag, `release`, for the jobs that publish or hold credentials (`pages`, the package upload, the release job), which carry `tags: [release]`. A change on an unreviewed branch can never reach the protected runner.

42.5.4 Antivirus, long paths and disk

Three Windows details caused most of the runner's trouble in the first months.

- **Antivirus.** Real-time scanning of the builds folder made each job several times slower, because a checkout writes thousands of small files. The enterprise security team agreed to exclude `D:\runner\builds` and the kit folder from on-access scanning, with a scheduled full scan nightly instead. Ask for this in writing; do not just turn scanning off.
- **Long paths.** Windows limits paths to 260 characters unless told otherwise, and a builds folder plus a deep model path plus a generated file name can exceed it. The fix is `git config --system core.longpaths true` on the runner (and the laptops) and a short `builds_dir`, as above.
- **Disk.** Each job leaves a checkout and its artifacts behind. The sprint 6 incident, when the builds drive filled and every pipeline failed for most of a day (Chapter 31, Sprint 6: Behavior by hand, a table for the program, and baseline-0 (p. 495)), led to a scheduled task that deletes build folders older than a week, and a free-space check as the first line of the weekly health look.

42.5.5 Updates, health and a second runner

Once a month Tom updates the runner, in this order, running a pipeline after each step so a failure points at its cause:

1. Windows updates, by the enterprise's normal process.
2. gitlab-runner. Its version tracks GitLab's: the runner's major and minor version should match the server's, so a server upgrade is followed by a runner upgrade within the month.
3. Git for Windows, only to the version the laptops move to, after the kit's tests pass under it on one laptop.
4. The kit, to the agreed tag (Section 42.9, Upgrades (p. 599)).

Every Friday Tom spends ten minutes on health: the runner page in GitLab (Settings, CI/CD, Runners) shows each runner's *last contact*; a runner that has not contacted GitLab for more than

a few minutes is offline, and an offline runner means every MR is blocked. Tom also checks free disk space and the *queued duration* of the week's jobs; a job that waited more than a minute or two for a runner means the runner is busy or sick. When the runner stops, the gitlab-runner service's own log (Windows Event Viewer, Application log) usually says why.

From sprint 7 a second Windows virtual machine runs an identical runner with the same tags. Either can take any checking job, so one runner being patched or broken no longer stops the team. Two runners cost the enterprise one more small virtual machine; the day the first runner's disk filled cost the team three person-days.

42.5.6 The Docker image, if you use one

The kit offers an optional container image, `mbse-ci` (`tools/gitlab/docker/`), for organizations whose GitLab offers only Docker runners. Tidewater did not use it, but a reader's project may. If you do, the image is part of the toolchain baseline and needs the same upkeep as the runner:

- **Pin the base image by digest**, not by a moving tag, so a rebuild is deliberate.
- **Rebuild monthly**, or when the base distribution publishes a security update, and give each rebuild a new tag.
- **Tag the image with the kit's version.** A pipeline that names `mbse-ci:1.0` always runs the same checks; never use `:latest` in a pipeline.
- **Scan it** with whatever your security office uses before pushing it to the registry, and keep no credentials in it.

42.5.7 Pipeline schedules and who owns them

The project has two scheduled pipelines: a nightly full check of `develop` on working days (cron `0 2 * * 1-5`) and the weekly status export on Friday mornings (`0 6 * * 5`). Both are listed in `project.conf`'s `[schedules]` section, so they are recreated by `gitlab.pl setup`, and `gitlab.pl schedule` lists and runs them.

A schedule runs *as the user who owns it*. When that user leaves, or loses the role the jobs need, the schedule stops or fails, and nobody notices until a Friday report is missing. Two rules followed: schedules are owned by a team member who is not about to leave (Tom, with Jim as the named backup), and the offboarding checklist has a line to take over the leaver's schedules (Build, Pipeline schedules, Take ownership).

42.5.8 What the pipeline costs

The kit and its checks cost nothing. On a self-managed GitLab with the team's own runners, there are no per-minute charges: the cost is the virtual machines and the time to look after them, about an hour a month each. On GitLab.com with GitLab's hosted runners, jobs consume *compute minutes*: each plan includes a quota, and more must be bought. Tidewater's pipeline is small (a few minutes per MR), but a reader on hosted runners should estimate MRs per month times minutes per pipeline before choosing.

Some gate settings need a paid tier: required approval rules, enforced code-owner approval, push rules and merge trains are Premium; security policies and compliance frameworks are Ultimate (Section 42.10, License tiers (p. 599)). Prices and the features in each tier change; check GitLab's current pricing page before planning a budget.

42.6 Protection audits

42.6.1 What drifts and why

Protections are settings, and settings drift. Someone with the maintainer role unprotects a branch "for five minutes" to fix something and forgets to protect it again. A new GitLab version changes a default. A group setting overrides a project setting. In the document era the equivalent was a controlled document quietly edited outside the CCB; the defence then was a configuration audit, and it is now.

The difference is that GitLab's protections are in a file. `project.conf` says what the project's settings should be, and `gitlab.pl setup` applies it; running it again re-applies the file, so a setting someone changed by hand in the web pages goes back to what the file says. Running setup with `--dry-run` prints every API request the file asks for, without contacting the server at all:

```
perl bin/gitlab.pl --url https://gitlab.example.com --dry-run setup
tools/gitlab/templates/project.conf
```

That is a preview of what the file says, not a comparison with the server: because a dry run reads nothing, it prints the full list every time. To audit drift, Tom runs the read-only comparison once a month and after any administrative change by anyone. `gitlab.pl setup --diff FILE` reads the live project, prints one line per difference, and exits 1 when anything has drifted. Here someone has turned force push on for `develop` in the web pages:

```
cp "$KIT/tools/gitlab/templates/project.conf" audit.conf
ga() { perl "$KIT/bin/gitlab.pl" --local audit --project tidewater/model "$@"; }
for u in jim bob tom; do ga user $u > /dev/null; done
sed -i 's/^\(develop = .*\)force:no/\1force:yes/' audit.conf
ga --as tom setup audit.conf > /dev/null 2>&1
cp "$KIT/tools/gitlab/templates/project.conf" project.conf
ga setup --diff project.conf 2>&1 | grep "^\[branches\]" || true
```

The file says no force push; the live project says yes. A difference is investigated before it is fixed: the question is not only "what changed" but "who changed it and why", which the audit events answer on Premium (and the instance admins can answer on Free). Then `setup` puts the project back. On a new project the two schedules also show as missing until `develop` exists; that is expected.

42.6.2 The monthly audit

Checklist: Monthly protection audit

- ☐ the project's settings pages match `project.conf`; `gitlab.pl setup project.conf` re-applied if they did not
- ☐ `main` and `develop`: push allowed to no one, merge by the intended role, force push off, code owner approval on (Premium)
- ☐ `vendor/prime`: push by maintainers only, merge by no one

- ☐ `baseline-*` tags protected, create by maintainers only
- ☐ Merge settings: merge commit, no squash, pipelines must succeed, discussions must be resolved
- ☐ `CODEOWNERS` on `develop` names only current team members
- ☐ Every merge into `develop` since the last audit had a passing pipeline and an approval by someone other than the author (scorecard measure 3; `gitlab.pl export` gives the MR list)
- ☐ No CI/CD variable is unprotected or unmasked unless the inventory says why

The seventh line is the important one. It checks the outcome, not the setting: even with every setting right, the question a reviewer will ask is "can you show me that every change was reviewed?", and the export answers it with the evidence.

42.7 Backups and restore tests

42.7.1 What is already safe and what is not

Git is distributed: every clone holds the full history of every branch it has fetched. With three engineers and a runner, the model's history exists in at least five places, and losing the server would not lose a single commit. New users often stop there. They should not, because the repository is only part of what the team now depends on.

Table 42.4: What a git clone does and does not protect

What	Where it lives	Safe if the server is lost?
The model, its history, tags	every clone	yes
Issues, comments, the board, labels, milestones	the server only	no
Merge requests, reviews, approvals	the server only	no
The wiki	the server (it is a git repository you can clone)	only if someone clones it
Pipeline history, job logs, artifacts	the server only	no
CI/CD variables, protections, members	the server; mostly in `project.conf`	partly
Packages (the prime drops, baseline bundles)	the server's package registry	no
The journal (`data/tidewater/journal`)	Jim's laptop and its own GitLab project	yes, if pushed

Merge requests and their approvals are the audit trail the whole process rests on. Losing them would be like losing every signed CCB form.

42.7.2 Three layers of backup

1. **The instance backup** belongs to the enterprise GitLab administrators, who back up the whole server daily. Tom asked them, in writing, what is backed up, how often, how long it is kept and when a restore was last tested. The answers went in the wiki. The team does not run this layer, but it should know what it covers.

2. **The project export.** GitLab can export a project (Settings, General, Advanced, Export project) into one archive holding the repository, issues, MRs with their discussions, labels, milestones, the wiki and more. Tom exports at every baseline and once a month, and uploads the archive to the package registry under a `project-export` package, versioned by date, so its checksum is recorded.
3. **The clones.** Everyone pushes their work daily, and the wiki is cloned onto Tom's laptop and pulled weekly.

42.7.3 The restore test

A backup that has never been restored is a hope, not a backup. Once a quarter Tom restores the latest project export into a scratch project (a project in Tom's personal namespace, deleted afterwards) and checks it:

Checklist: Quarterly restore test

- ☐ Import the latest export archive as a new project
- ☐ The default branch, `main`, `develop` and every `baseline-*` tag are present
- ☐ A merge request from the last sprint shows its discussion and approvals
- ☐ The wiki's Home page and Administration page are present
- ☐ Register nothing new: point the existing runner at the scratch project temporarily, run the pipeline on `develop`, and check it passes
- ☐ Rebuild the latest baseline's docs and compare the checksum with the tag message
- ☐ Record the date, the archive's checksum and the result on the Admin story; delete the scratch project

The first restore test, in sprint 10, failed on the fifth item: the scratch project had no CI/CD variables (an export does not carry them, by design, because they hold secrets). That was the useful result. The team added the variable list, without values, to `project.conf`'s `[variables]` section, so recreating them is part of the setup file.

The offline emulator (Chapter 17, The DevSecOps tools that earn their keep (p. 255)) let Tom rehearse the whole procedure on a laptop before touching the server: the emulator's state is one file, and restoring it is copying that file and the bare repository back.

42.8 Audit events

GitLab Premium records *audit events*: who added a member, changed a protection, created or revoked a token, changed a CI/CD variable, and similar administrative actions, with the time and the source address. (Who pushed what and who merged what is in the repository and the MRs on every tier.) Jim reviews the project's audit events once a month, looking for three things:

- **Any change to protections or merge settings.** Each should match an MR to `project.conf` or a comment on the Admin story. One that matches nothing is asked about the same day.
- **Membership changes.** They should match the onboarding and offboarding records.
- **Token creation and revocation.** They should match the token inventory.

On the Free tier there is no audit event page for projects. The team's fallback was the monthly protection audit above, plus the instance administrators' logs on request. That is weaker, and Jim said so in writing when the Premium question came up (see Section 42.10, License tiers (p. 599)).

42.9 Upgrades

Four things in the environment change version, and each needs a different kind of care.

GitLab itself. The enterprise administrators upgrade the server, usually monthly. GitLab announces deprecations in its release notes months ahead, and occasionally removes a CI keyword or an API field. Tom reads the release notes each month for two sections only: CI/CD (anything about keywords the pipeline uses) and the REST API (anything `gitlab.pl` calls). After each server upgrade, Tom opens the pipeline editor (Build, Pipeline editor), which validates `.gitlab-ci.yml` against the server's current rules, and runs a pipeline on `develop`.

gitlab-runner. Upgraded monthly with the runner patching, within the version range the server supports.

Git for Windows and its Perl. Upgraded on the laptops and runner together, a few times a year. The kit's tests run first on one laptop under the new version (Appendix A, Command reference (p. 696) has the command); only when they pass does everyone upgrade.

The kit. The kit is a dependency like any other. The runner and the laptops use the same *tag* of the kit, never whatever is newest. Upgrading the kit is a story: read the kit's changes, run its tests, run the model's pipeline locally with the new kit (`KIT=/c/agile-new perl bin/gitlab.pl ci --event merge_request_event --target develop`), then move the runner and laptops together. A new kit that changes a converter's output changes what the next vendor import produces; that is planned, not discovered.

Warning: Never upgrade on a drop week

The week a prime drop arrives is the week the team most needs the environment to behave exactly as it did last time, so the delta shows only the prime's changes. Upgrades wait for the sprint after.

42.10 License tiers

GitLab comes in three tiers: Free, Premium and Ultimate. Several features this book uses are Premium or above, and the team spent part of sprint 2 working out which they actually needed. The honest answer was that the process works on Free with a few manual substitutes, and that Premium buys enforcement of things Free can only ask people to do.

Table 42.5: Features by tier, and what Tidewater did on each

Feature	Tier	How Tidewater uses it	On Free instead
Protected branches and tags	Free	`develop`, `main`, `vendor/prime`, `baseline-*`	n/a
Merge request approvals (optional)	Free	one approval per MR	n/a

Feature	Tier	How Tidewater uses it	On Free instead
Required approval rules, code owner approval enforced	Premium	a code owner must approve; merge is blocked otherwise	`CODEOWNERS` suggests reviewers; the team's rule and the monthly audit check it was followed
Pipelines, artifacts, schedules, Pages, releases, package registry	Free	all of them	n/a
JUnit and Code Quality reports on the MR	Free	the MR widgets	n/a
Metrics report on the MR	Premium	scorecard numbers shown with their change	the numbers are in the job log and artifact
Scoped labels (`key::value`, one per key)	Premium	`workflow::`, `type::`, `priority::`	plain labels; the board still works; two `workflow` labels can coexist by mistake
Milestone burndown charts	Premium	the sprint burndown	the journal's sprint report
Epics, roadmaps	Premium (multi-level: Ultimate)	not needed	`epic::` and `tome::` labels, the journal's roadmap
Iterations	Premium	not used	milestones named `Sprint N`
Push rules	Premium	reject commits with secrets or large files at push time	the secrets job in the pipeline
Audit events (project)	Premium	the monthly review	the protection audit and the instance logs
DORA metrics, value stream analytics dashboards	Ultimate (VSA basics on Free)	not essential	`gitlab.pl export` and the status metrics

The program bought Premium for the team in the second quarter, on the strength of one argument Jim made in the sprint 6 briefing: on Free, the rule "every change approved by a code owner other than its author" is a team habit checked monthly; on Premium, it is a setting the server enforces on every merge. For a program that has to show an auditor its change control, the difference was worth the licence cost of three seats.

42.11 Applying this to your project

- **Copy** the administration calendar and turn it into a recurring story. Adjust the cadences to your organization's policies, but keep every row; each one exists because a team like Tidewater's got hurt without it.
- **Copy** the token rule (job token inside pipelines, project tokens for automation, personal tokens only for a person's own work) and the token inventory. Agree the rotation period with your security officer before you create the first token, not after the first expiry.
- **Adapt** the runner to your environment. If containers are allowed and your tools run in them, a Linux container runner is simpler to keep clean. Keep the principle that the runner runs the same tool versions as the engineers.
- **Adapt** the role assignments to your separation-of-duties rules. Write down why each person has the role they have; you will be asked.
- **Decide locally** which tier you need. List the rules your program must be able to prove, and check which of them your tier can enforce rather than merely record.

- **Do not skip** the restore test. The first one will find something, as Tidewater's did.

42.12 Exercises

Exercise 42.1: Assign the roles

Your project has five people: a model lead, two modelers, a CM engineer who runs the pipeline, and a contractor liaison who must read the model and comment on discrepancy reports. Assign each a GitLab role and write one sentence per person justifying it in terms of separation of duties.

Exercise 42.2: Choose the token

For each automation, name the kind of token and its scopes: (a) a pipeline job that uploads the prime's drop to the package registry; (b) a nightly job that exports MR and pipeline records for the status metrics; (c) Jim's script that creates the sprint's milestones from a laptop; (d) a read-only mirror of the repository for the program office.

Exercise 42.3: Find the drift

Using the offline emulator, apply `tools/gitlab/templates/project.conf` to a fresh emulator directory with `gitlab.pl --local DIR setup`. Then edit the file to allow force push on `develop` and apply it again: what does setup print for the protected branch? Now preview the same file with `--url https://gitlab.example.com --dry-run` (a dry run contacts nothing). Why does the preview list every request, not only the changed one? How would you have noticed the change if someone had made it in the web pages instead of the file? (Try `gitlab.pl setup --diff` on the original file. `--dry-run` also works on the offline emulator: it prints the requests and applies nothing.)

Exercise 42.4: Plan a restore test

Write the checklist you would use to restore your own project from an export. Which items on Tidewater's checklist would fail on your server, and why? (Hint: think about what an export deliberately leaves out.)

Exercise 42.5: Make the tier case

Your program is on GitLab Free. Write the three-sentence argument for or against Premium for your team, naming the one rule you most need the server to enforce rather than record.

CHAPTER

43

Maintenance through the life cycle

From what you know →

You know

the long sustainment phase: ECPs, contractor deliveries and re-baselines.

→ Here it is called

maintaining the model against prime drops, with baselines and hotfixes.

→ What is new

each delivery is compared automatically, and its impact on your model is listed before anyone decides.

Survival kit: Staying current for eighteen months

What you have

a SysML v2 model under configuration control, a pipeline that checks every change, an interim baseline, and a prime contractor who will keep delivering changes to the old SysML v1 model for as long as the contract runs.

What you need

a routine that takes each new drop in without drama, cuts baselines on a schedule, fixes an urgent error in a released baseline without waiting for the next one, and keeps the change board, the discrepancy reports and the archive in order – until the day the model is retired.

The first twelve weeks were a port. Everything after them is maintenance: the steady work of keeping the government's SysML v2 model equal to the system the prime is building, while the engineering on top of it grows. On Tidewater that is thirty-three more sprints, six more prime drops, three baselines and one hotfix. This chapter is the routine for that period, shown end to end on real drops in a sandbox you build yourself.

Note: What is evidence and what is plan in this chapter

Sprints 1 to 6 of Tidewater have been simulated and their evidence is in `docs/tidewater/`. Sprints 7 to 39 have not run yet. In this chapter the prime's drops are real (each one is generated by `sim/tidewater-v1.pl`, with its manifest and change requests), and every command runs on them. What the team does in sprints 7 to 39, and when, is taken from the team's plan, `sim/tidewater-plan.pl`, and is marked **From the plan**. A later edition replaces those passages with the simulated sprints' evidence.

43.1 The eighteen months at a glance

The prime delivers a drop roughly every quarter under CDRL A003. Each drop comes with a manifest that lists its change requests, which is what the government's change board dispositions. These are the seven drops as the prime delivered them:

Table 43.1: The prime's drops, from their manifests

Drop	Delivered	Change requests	What changes
v1.0	2 Oct 2026	NWM-CR-0001	The baseline at contract start: the spider web the team ported in the first twelve weeks
v1.1	8 Jan 2027	NWM-CR-0012 to 0014	A hygrometer in the sensor suite, its requirement TW-SYS-017, and TW-SYS-006's text aligned with DOORS
v1.2	2 Apr 2027	NWM-CR-0015 to 0019	A storm mode in the buoy's state machine, a fuse in the power system, TW-SEG-009 deleted, TW-SYS-012 satisfied, a duplicate block removed
v1.3	2 Jul 2027	NWM-CR-0020 to 0023	Salinity in the observation record, drifting buoys, TW-SYS-016 brought into the model, TW-SYS-002's interval revised
v1.4	1 Oct 2027	NWM-CR-0024 and 0025	A reserve term in the energy balance; an activity renamed with no reason given
v1.5	7 Jan 2028	NWM-CR-0026 to 0028	Partner data sharing (TW-SYS-019), a resend message, the Legacy Import package deleted
v1.6	3 Mar 2028	NWM-CR-0029 to 0031	Verification for TW-SYS-011 and TW-SEG-002, documentation edits

Make the first three drops now. The rest of this chapter works on them:

```
for n in 1 2 3; do perl "$KIT/sim/tidewater-v1.pl" --drop $n drops/v$n > /dev/null; done
sed -n '/^Change requests/,/^Files/p' drops/v3/MANIFEST.txt
```

Three of the drop 3 change requests are engineering decisions the prime made for its own reasons (a storm mode, a fuse, a deleted requirement). Two are answers to the government's own discrepancy reports: the requirement that nothing satisfied, and the duplicate block left by the 2023 document import. That mix is normal. A healthy maintenance relationship sends questions upstream and gets answers back in the next drop.

From the plan: the team's backlog for sprints 7 to 39 has these milestones, one or two engineering threads running between them:

Table 43.2: From the plan: sprints 7 to 39

Sprint	Planned	What it is for
7	Behavior closed	The interaction and the constraint written by hand; the last Stations-DataCenter cycle broken; the kit version pinned
8	Drop 2 merged	The hygrometer in the model; the pipeline split into included files
9 to 13	Trace complete, package by package	Complexity guardrails, the local assistant approved, a token-leak rehearsal, the first backup and restore test, a second runner
14	Drop 3 merged	The storm mode reworked by hand (the converter cannot carry it)
15 to 19	Acceptance	The behavior epic closed, an independent review of the v2 model, intake automated from the prime's Jenkins, a release candidate
20	Drop 4 merged; **baseline-1**	The port accepted; baselines published as packages
21 to 26	Engineering with the model	Energy balance per mode, the latency chain, the telemetry trade, the parallel verification run, the hotfix (sprint 24), markings and trust zones
27	Drop 5 merged	The reserve term and the rename (dispositioned "reject, clarify")
28 to 33	Engineering and security	The legacy test matrix retired, a threat register, **baseline-2** (sprint 30), security requirements through the prime, signed tags
34	Drop 6 merged	Partner sharing; the Legacy Import package gone upstream
35 to 39	Sustainment	Drop 7 (sprint 38), **baseline-3** (sprint 39), the runbook and lessons learned

Over the whole plan, engineering is 72% of the work (see Bottom line up front (p. iii)); the maintenance routine itself is small, which is the point of making it routine.

43.2 The drop intake runbook

Chapter 45, The prime's model changes: receive, assess, disposition (p. 619) explains each step of the drop loop and why it exists. Here is the routine as a team runs it every quarter, start to finish, on a project you build in the sandbox. It takes the first drop through the project's week zero, then takes the second drop through the full loop.

First, the project. `kickoff.pl start` is the week-zero sequence from Chapter 50, Your project in twelve weeks: a playbook (p. 672); with the layout map it puts every prime package into the layered folders from the first drop:

```
perl "$KIT/tools/gitlab/kickoff.pl" start --yes --local gitlab --drop drops/v1 --version v1.0 \
    --layout-map "$KIT/examples/tidewater-v1/layout.map" --no-wiki --no-issues > start.log 2>&1
grep "week zero is set up" start.log
cat > env.sh <<EOF
export GITLAB_LOCAL="$PWD/gitlab" GITLAB_PROJECT=tidewater/model
gl() { perl "\$KIT/bin/gitlab.pl" "\$@"; }
EOF
```

`env.sh` gives you the `gl` shortcut for the project's GitLab (here the kit's offline stand-in). In a real terminal you source it once; the blocks below each start with `. ../env.sh` because each one is tested on its own.

Week zero left drop 1 on its own branch, `feature/drop-v1.0`, ready for review. The prime's structure arrives all at once, and with it the prime's outline violations (packages with one member, folders with one file). The government did not make them and does not fix them in the drop's merge request. It records them, so that from now on the outline gate fails only on violations the government adds. `flow vendor`, which `kickoff.pl start` ran for drop 1, records the new allowance in a commit of its own and regenerates the documents in another. Look at the branch, then open the merge request:

```
. ../env.sh
git log --format='%s' -3
echo "outline allowance: $(cat .ci/outline-max)"
gl --as tom flow finish --title "Prime drop v1.0 (NWM-CR-0001)"
gl --as jim mr approve 1
gl --as bob mr approve 1
gl --as tom mr merge 1
```

Tom opened the merge request; Jim and Bob approved it. The project's approval rule wants one reviewer, the interface files want a code owner other than the author (Bob), and no one approves their own change. That is the drop loop's first pass. The second drop shows every step.

1. Receive and verify. The drop arrives (by file transfer in the first months, later from the prime's Jenkins, see Section 45.9, The prime's Jenkins: a delivery interface (p. 630)). Before anything else, check that every file is the file the prime sent:

```
tr -d '\r' < MANIFEST.txt | sed -n 's/^  \([0-9a-f]\{64\}\)  \(.*\)$/\1  \2/p' | sha256sum -c
```

The manifest is written with Windows line endings, so `tr` strips them before `sha256sum` reads the list. A failed check stops the intake: the drop goes back to the prime with a request to resend, and nothing is imported.

2. Assess the impact against the government's model, before importing. Run the impact report on `develop`, the model as the government has it today. Run it before the import: on the drop's own branch, the drop's changes are already in the files and the report mistakes them for government edits.

```
. ../env.sh
gl flow sync
git checkout -q develop
perl "$KIT/bin/xmi2sysml.pl" --delta ../drops/v1/tidewater.xmi ../drops/v2/tidewater.xmi \
    --impact model --layout-map .ci/layout.map > ../impact-v1.1.txt 2>&1
grep -E "^== NWM|touched|change requests|hand edits" ../impact-v1.1.txt
```

The report groups the changes by the prime's change requests and lists, for each, the requirements, verification cases, interfaces and allocations it touches. "Hand edits to redo" is the line

that matters most in maintenance: every TODO the government closed by hand, and every definition it edited, that the drop changes underneath. Here it is zero, because this sandbox's model has no hand edits yet; on Tidewater's real `develop` the drop 3 report flags the hand-written state machine, because the prime added a storm mode the converter cannot carry (Chapter 45, The prime's model changes: receive, assess, disposition (p. 619) shows that run).

3. Disposition. Each significant change request becomes an issue with a `disposition::` label, decided at the change board (Section 43.5, The change board in GitLab (p. 609)). Nothing is merged before its disposition.

4. Import on the vendor branch and merge the drop into a feature branch. `flow vendor` converts the drop, commits the raw conversion on `vendor/prime` (never edited by hand), and merges it into `feature/drop-v1.1`, listing any conflicts:

```
. ../env.sh
gl --as tom flow vendor ../drops/v2 v1.1
perl "$KIT/tools/sysml/model.pl" docs > /dev/null
git add -A docs
git commit -qm "Regenerate docs for prime drop v1.1"
gl --as tom flow finish --title "Prime drop v1.1 (NWM-CR-0012 to 0014)"
gl --as jim mr approve 2
gl --as bob mr approve 2
gl --as tom mr merge 2
```

Two lines in that block are the lessons of past drops. The docs are regenerated on the drop's branch, because the documents are generated from the model and kept in the repository: a drop changes the model, so it changes the documents, and the pipeline's docs job refuses a merge request whose documents are stale. And the outline allowance is checked again whenever a drop adds structure; this drop added none.

5. Rework. Conflicts and hand edits to redo become tasks on the drop's story, and the government's edits are re-applied on top of the prime's new text (Chapter 45, The prime's model changes: receive, assess, disposition (p. 619) and practice lesson 12 show a conflict resolved in Vim).

6. Close the loop. Close the Prime Drop issue, close the discrepancy reports the drop answers, open new ones for what it got wrong, and record the drop latency (delivered to merged on `develop`) for the monthly status.

Checklist: The drop intake runbook

- ☐ Checksums verified against the manifest; markings checked; the drop archived as received
- ☐ Prime Drop issue opened with the transmittal record and the change requests listed
- ☐ Impact report run on develop, before the import; attached to the issue
- ☐ Every significant change request dispositioned at the CCB (disposition:: label, deputy PM concurrence)
- ☐ flow vendor: raw conversion on vendor/prime; merged into feature/drop-VERSION
- ☐ Conflicts resolved by taking the prime's text and re-applying the government's edits

- ☐ Hand edits to redo done as tasks; docs regenerated; outline allowance recorded if the drop added structure
- ☐ Merge request green, approved by two people other than the author, merged to develop
- ☐ DRs answered by the drop closed; new DRs opened; Prime Drop issue closed; latency recorded

From the plan: the drops' latency is a measured result of sprints 7 to 39 and is reported in a later edition. The plan's target is that a drop is merged on `develop` within its sprint, and from drop 4 on (sprint 17's intake automation) within days of delivery.

43.3 Baselines and releases

A baseline is the configuration the program agrees to stand behind: the model, its generated documents, its verification matrix and the tools that produced them, all at one commit, tagged and published. Tidewater plans four: baseline-0 (the interim baseline at the end of the first twelve weeks, sprint 6), baseline-1 (port acceptance, sprint 20), baseline-2 (sprint 30) and baseline-3 (the end of the eighteen months, sprint 39).

Every baseline follows the same path through gitflow: a release branch from `develop`, a changelog section, a merge request to `main` with the change board's concurrence, a tag, a release, and a back-merge so `develop` has everything `main` has. Cut baseline-1 from the two drops now on `develop`:

```
. ../env.sh
gl flow sync
git checkout -q develop
gl --as tom flow start release baseline-1
perl -0pi -e 's/^\[Unreleased\]: /## [baseline-1] -- 2027-07-02\n\n### Prime drops merged\n\n-
v1.0 (NWM-CR-0001) and v1.1 (NWM-CR-0012 to 0014)\n\n[Unreleased]: /m' CHANGELOG.md
git commit -qam "Changelog for baseline-1"
gl --as tom flow finish --title "Baseline 1"
gl --as jim mr approve 3
gl --as bob mr approve 3
gl --as tom mr merge 3
gl --as tom flow released 3
```

Tom prepared the baseline; Jim and Bob approved it. On Tidewater the release merge request also carries the deputy program manager's concurrence as a comment: in GitLab terms that comment is the change board's signature (Section 43.5, The change board in GitLab (p. 609)). `flow released` then tags `main` as `baseline-1` (the tag's message records the checksum of the generated documents), publishes the release, and opens the back-merge request to `develop`:

```
. ../env.sh
gl --as jim mr approve 4
gl --as bob mr approve 4
gl --as tom mr merge 4
```

The tag's pipeline did the rest. The `baseline` job rebuilt the documents and checked their checksum against the tag, which is the physical configuration audit done by machine. The release jobs published the baseline as a package, so other teams can take a versioned copy without access to the repository:

```
. ../env.sh
gl package list
gl pipeline list | grep baseline-1
```

The package holds the model and documents as a tarball, the verification matrix, the toolchain inventory (which versions of the kit, Perl and git produced the baseline) and a checksum file. The release links to each of them and names the sprint milestones it closes; GitLab keeps the release's evidence (the milestones' issues and the test report at release time) with it.

Tip: The release branch is the last place to fix, not to change

A release branch takes only fixes that make the baseline correct: a changelog line, a missing document, an outline allowance a drop should have recorded. New work goes to `develop` and waits for the next baseline. If a fix is needed on the release branch, it goes back to `develop` in the back-merge, so nothing is lost.

From the plan: baseline-1 at sprint 20 also carries the port completeness report (Chapter 34, Proving the port is complete and correct (p. 527)) and the deputy program manager's sign-off of the port; baseline-2 at sprint 30 is the first with signed tags (Chapter 48, Security: protect the environment, engineer it into the system (p. 651)) and retires the prime's legacy test matrix for Stations (Chapter 44, Verification as a closed loop (p. 613)); baseline-3 closes the eighteen months.

43.4 Hotfixes

Sometimes a released baseline is wrong in a way that cannot wait for the next one. **From the plan:** in sprint 24 the program office finds a satisfy relationship in baseline-1 that points at the wrong element. The fix goes on a hotfix branch from `main`, is merged to `main` as a patch baseline, and is merged back to `develop`:

```
. ../env.sh
gl flow sync
git checkout -q main
gl --as tom flow start hotfix sys-007-satisfy
sed -i 's/satisfy TW_SYS_007 by BulletinPublisher;/satisfy TW_SYS_007 by
DataSegment::publicOut;/' \
    model/requirements/model/Requirements.sysml
perl "$KIT/tools/sysml/model.pl" docs > /dev/null
git add -A model docs
git commit -qm "Satisfy TW-SYS-007 by the public interface, not the publisher"
gl --as tom flow finish --title "Hotfix: TW-SYS-007 is satisfied by the public interface"
gl --as jim mr approve 5
gl --as bob mr approve 5
```

```
gl --as tom mr merge 5
gl --as tom flow released 5 --tag baseline-1.1
gl --as jim mr approve 6
gl --as bob mr approve 6
gl --as tom mr merge 6
```

Two things are easy to miss. The satisfy line lives in a file the converter writes from the prime's model, so the hotfix is also a discrepancy report to the prime: unless the prime corrects its v1 model, the next drop puts the wrong satisfy back, and the impact report will flag it as a hand edit to redo. And the patch baseline gets its own tag, `baseline-1.1`, so anyone holding baseline-1 knows a corrected version exists.

43.5 The change board in GitLab

Tidewater's change control board did not go away; it moved. The board's decisions are now records in GitLab:

Table 43.3: The change board's records, before and after

Board activity	Where it lives in GitLab
An engineering change proposal	A merge request, with the impact report attached
A prime change request to disposition	A `prime-change` issue with a `disposition::` label: accept, accept with impact, reject or clarify, no impact
The board's approval of a baseline	The deputy program manager's concurrence comment on the release merge request
The board's agenda	The open `prime-change` issues and release merge requests, listed by `gl issue list --label prime-change`
The minutes	The issues' and merge requests' comment threads, kept forever

The board meets once a month (the plan's "Monthly status and CCB items" story, every other sprint). Most decisions are quick, because the impact report has already done the analysis a board used to spend its meeting on. The ones that take time are the ones that should: a change with cost or schedule impact, or a change the prime made without a reason. **From the plan:** drop 5's rename comes with no rationale; the board dispositions it "reject, clarify", sends a discrepancy report asking why, and closes it when drop 6 answers. Drop 4's drifting buoys are "accept with impact": they add verification work, and the board records the cost.

43.6 Discrepancy reports, upstream and back

A discrepancy report is the government's formal question to the prime about its model: a requirement nothing satisfies, a wording difference between DOORS and the model, a duplicate block, a change with no reason. On Tidewater they are GitLab issues labelled `type::discrepancy` and `owner::prime`, sent through the contracting channel with a transmittal memo (Chapter 37, Writing it down: DoD-style memos from Vim (p. 545)), and kept open until a drop answers them.

The drops show the loop working. Drop 2's NWM-CR-0014 aligns TW-SYS-006 with DOORS, closing the first drift the team found in sprint 1. Drop 3's NWM-CR-0018 and 0019 answer the

reports about TW-SYS-012 and the duplicate SensorSuite. Drop 4's NWM-CR-0022 brings TW-SYS-016 into the model. Drop 6's NWM-CR-0028 deletes the Legacy Import package the team untangled in sprint 5. Each change request names the report it answers in its rationale, which is why the team asked for change notes in week zero.

43.7 The life cycle beyond the port

Tidewater's eighteen months sit inside a longer life. The same repository, pipeline and routine serve every phase; what changes is where the pipeline's emphasis sits:

Table 43.4: The life cycle and the pipeline's emphasis

Phase	Repository activity	Pipeline emphasis
Concept	Stakeholder needs, use cases, early trades	Validation, trade analysis
Requirements (to SRR)	Requirement models, satisfy links	Trace coverage, completeness, the recipe
Architecture (to PDR)	Structure, interfaces, behavior	Interface consistency, budgets, the data exchange table
Detailed design (to CDR)	Refined parts, parameters	Full analysis, simulation hooks
Integration and test	Verification cases, results linked back	The verification matrix, results ingestion (Chapter 44, Verification as a closed loop (p. 613))
Production and sustainment	One configuration per fielded block	Change impact on old baselines, fixes upstream first

Tidewater uses gitflow, with baselines as releases on `main`. When a program fields several configurations at once (block 1 in service while block 2 is designed), the alternative is a long-lived release branch per fielded configuration, cut from its baseline tag: fixes land on `develop` first and are cherry-picked onto each fielded branch that needs them. That is GitLab flow's release-branch variant, and the choice is worth a decision record the day the second configuration is fielded.

43.8 Retirement and archival

Every model ends. The prime's contract closes, the system leaves service, or a successor model takes over. Plan the end while the team still remembers how everything works:

- **Freeze.** Cut a final baseline, protect every branch so nothing more can merge, and archive the GitLab project (archived projects are read-only and stay searchable).
- **Keep what an auditor will ask for.** The baseline packages, the release evidence, the toolchain inventory and the drops as received are formal records; their retention is set by the program's records schedule, not by GitLab's default artifact expiry. Copy them to the program's records system with their checksums.
- **Keep the tools that read them.** A tagged copy of the kit (the slim `release.pl --profile mbse` zip) beside the archive means the model can be checked and its documents rebuilt years later, on whatever Perl is current then.

- **Hand over or shut down.** Transfer the project to its successor, or revoke every token, remove the runners and close the schedules (a schedule runs as its owner: an owner who leaves stops it, which is the one clean-up that happens by itself).
- **Write it down.** The runbook and lessons learned are the plan's last story (sprint 39). The next team that ports a model will read them before they read this book.

Exercise 43.1: Run a drop through the loop yourself

In the sandbox above, take drop 3 through the runbook: verify its checksums, run the impact report on `develop`, import it with `flow vendor`, regenerate the docs, open and merge the merge request. Which of drop 3's change requests appear in the impact report as "manual", and why can't the converter carry them?

Exercise 43.2: Cut the next baseline

After drop 3 is merged, cut `baseline-2` the way `baseline-1` was cut. What must the changelog section say, which change requests does it list, and what does the release link to?

Exercise 43.3: A hotfix that the prime must also fix

Explain to the prime's modeler, in three sentences for a discrepancy report, why the government's hotfix to TW-SYS-007's satisfy relationship will be undone by the next drop unless the prime changes its model.

43.9 Applying this to your project

- Make the drop loop a checklist on day one and run it the same way every time; the second drop is where teams either have a routine or have a crisis.
- Run the impact report on your integration branch before the import, never on the drop's branch.
- Record inherited counts (outline, complexity, trace gaps) in the drop's merge request so the gates measure only what your team adds.
- Baseline on a calendar the program understands (milestones, quarters), and publish each baseline as a package with its toolchain inventory.
- Put the change board's decisions in the issues and merge requests; the board's minutes then write themselves.
- Plan retirement before the people who built the model move on.

43.10 You can now…

- take a prime drop from delivery to merged: checksums, impact report on develop, disposition, vendor import, regenerated documents, two approvals;
- record what a drop brings in so the gates measure only your team's changes;
- cut a baseline, publish it as a package with its toolchain inventory, and merge it back to develop;
- ship a hotfix to a released baseline and tell the prime why it must change its model too;

- run the change board in GitLab, and plan the model's archival.

CHAPTER

44

Verification as a closed loop

From what you know →

You know

the VCRM spreadsheet and the test reports that feed it.

→ Here it is called

the verification matrix generated by the pipeline, and one results schema.

→ What is new

results flow in from every source, and the matrix is current every day.

Survival kit: Where this chapter fits

What you have

requirements in the model with `verify` relationships to verification cases; a pipeline; a prime contractor and a test organization who actually run the tests.

What you need

a way to turn "which requirements are verified, and by what evidence?" into a report the pipeline regenerates on every change – and a plan for moving testing onto that footing under management pressure, without putting formal verification evidence at risk.

Testing is where a program proves the system works and where the customer accepts it. It is the worst place for a big-bang tool change. This chapter shows what the Tidewater team built for verification in the first twelve weeks, which is deliberately small, and then the staged plan for the rest of the program. Everything after sprint 6 in this chapter is the plan, not the evidence; it says so where it applies.

44.1 The loop

Verification in a model repository is a loop with three parts.

1. **Plan the tests in the model.** Each requirement has a `verify` relationship to a verification case, and the case states its method (test, analysis, inspection or demonstration), its subject and its objective (Chapter 8, SysML v2 by example: a workbook (p. 92)).
2. **Execute** the tests wherever they run – the prime's lab, a simulation, an engineer's inspection.
3. **Feed the results back** in one results format, and let the pipeline regenerate the verification cross-reference matrix (VCRM) on every change.

The government team owns the first and third parts for Tidewater. The prime and its test organization own the second. That split is the most important design decision in this chapter: the government does not need to run the prime's tests to know, at any moment, which requirements have evidence behind them.

44.2 One results format

Every verdict – automated or not, the prime's or the government's – goes into a CSV file in the model repository's `verification/` folder, with the same columns:

```
requirement,case,verdict,commit,environment,timestamp,signed_by
```

The `verdict` is `passed`, `failed` or `not run`. `commit` names the version of the model the test was run against, so a result can never be mistaken for evidence about a later design. `signed_by` is empty for an automated result and carries a person's name for an inspection, analysis or demonstration – the results that will never come out of a test harness, and that matter just as much. Keeping them in the same file is what keeps the matrix complete: an inspection verdict signed by Jim counts exactly like a passed automated test.

44.3 Sprint 6: the VCRM in the pipeline

In sprint 6 Jim's story "VCRM: which requirements are verified, and how" added the `vcrm` job to the pipeline and entered the government's first signed inspection verdicts. From then on every merge request's pipeline printed the matrix's summary line. Here it is from the last merge request of the twelve weeks:

docs/tidewater/sprints/06/pipelines/mr26-first-100.log

```
==== vcrm             trace    success   13.1 s
$ perl ci/job.pl vcrm
VCRM: 30 requirements, 2 verified, 0 failed, 19 not run, 9 without a case (vcrm.html, vcrm.csv)
```

Read it the way a test lead would. Of the requirements in the model, a few have passing evidence, many have a verification case but no result yet – the prime has not run those tests – and some have no verification case at all. The last group is the one that matters: those are the requirements the trace reports as *unverified* (9 of them at the end of sprint 6), and they are the prime's to fix, raised as discrepancy reports. That is why verify coverage stayed flat through the first twelve weeks:

the team made the gap visible and reported it; it did not, and should not, write verification cases for the prime.

You can regenerate the matrix yourself from the simulation's model at the end of sprint 6. Bob's clone is in the sandbox; switch it to `develop` and run the job exactly as the pipeline does:

```
git checkout -q develop
perl ci/job.pl vcrm
```

The government's signed verdicts are the results file the job read:

```
cat verification/results-gov.csv
```

And the matrix itself is a CSV the model site publishes, one row per requirement:

```
head -4 vcrm.csv
```

44.4 The testing transition, under pressure

Management's pressure to "move testing to DevSecOps" arrived in the same directive as everything else. The team's answer was to move testing last, in stages, and never to put formal evidence at risk. These are the ground rules it adopted, and the five stages, each with what Tidewater did or plans to do.

Ground rules.

- **Don't break what is contractually committed.** Test plans, procedures and reports are often contract deliverables with an agreed format and approval chain. The new approach has to produce them, or the customer has to agree to a change, before it replaces the old one. Check with the program office and contracts first.
- **Protect the test schedule from the tool change.** If a test event is near, freeze the method for it, and change between campaigns, never during one. For Tidewater the prime's Station acceptance test campaign (notional, spring 2027) is a frozen window: no verification-process change merges during it.
- **Separate the three changes.** SysML v2, a DevSecOps pipeline and agile ways of working are three transitions that management tends to bundle. Tidewater sequenced them: ways of working and git first, the port next, testing last.

Table 44.1: The five stages and Tidewater's mapping. Only stage 1 is evidence; stages 2 to 5 are the plan.

Stage	What it is	Tidewater
1. Traceability first	Verification cases in the model and a generated VCRM; tests executed exactly as today.	Done: the trace report from sprint 3, the VCRM from sprint 6.
2. Automate result capture	Existing test harnesses emit a standard format (JUnit XML or the results CSV); the pipeline ingests it.	Planned: the prime's results arrive with each drop from drop 4.
3. Pilot one subsystem in parallel	The full approach for one contained package, run beside the legacy process for one cycle; differences reconciled.	Planned: the Stations VCRM beside the prime's legacy test matrix for drops 4 and 5.
4. Shift testing left	Model-level checks on every merge request; long regressions nightly; hardware time for what only hardware can prove.	Planned: the energy-balance and latency checks of the engineering epic.
5. Scale and retire	Package by package; retire legacy artifacts only once the customer accepts the new ones.	Planned: the legacy matrix retired at baseline-2, with the program office's agreement.

Managing the pressure. Report what management cares about: verification coverage, time from a design change to test feedback, problems found before the lab versus in it, hours spent assembling review packages. Push back with specifics and a date ("we switch at the campaign boundary, after the pilot, with the customer's agreement"), not with "we're not ready". Ask for the enablers in writing: training time, a test-automation engineer, runner hardware for the lab, the tool approvals (Chapter 37, Writing it down: DoD-style memos from Vim (p. 545)). Bring the test team in from the start to co-own the verification case format and the results schema; a process designed without them fails at the bench. Accept that some verification stays manual, and make sure its results flow into the same matrix. Get the customer and any independent test organization to accept model-generated evidence early, by showing them the pilot's output.

Risks to watch.

- *Tool maturity.* Keep the model and the results in plain, tool-neutral text so a tool that falls short does not strand the evidence.
- *Model-test drift.* If test procedures change outside the model, the matrix lies. Procedure changes go through merge requests like model changes.
- *Over-automation theater.* A green pipeline that checks trivial things breeds false confidence. Every automated check should map to a real verification objective.
- *Hardware contention.* Automated hardware tests can starve manual testing of bench time; schedule the bench.

44.5 Higher fidelity: model, software and hardware in the loop

Tidewater's government team owns no lab, so it runs no software-in-the-loop (SIL) or hardware-in-the-loop (HIL) tests. The kit's pipeline template still carries both, switched off, so a program that does have a simulation or a bench starts from a tested template. The levels are:

Table 44.2: Increasing fidelity, and where each level runs.

Level	What runs	Where	Trigger
Model in the loop	Constraint and calculation checks, executable behavior	The normal runner	Every merge request
Software in the loop	Real software against a simulated environment	A normal or high-performance runner	Nightly schedule
Hardware in the loop	Real hardware on a lab bench	A runner attached to the bench	By hand, gated
Qualification	The full system, formal procedures	The test facility	At the test readiness review baseline

The HIL job in the template shows the GitLab features a lab needs: a runner *tag* that only the bench's runner has, a *resource group* so two pipelines never fight over one bench, `when: manual` so a person confirms the bench is ready, and an *environment* so the bench's history of what ran on it is recorded:

tools/gitlab/templates/gitlab-ci.yml

```
      - verification/results-sil.csv

hil-test:
  stage: trace
  tags:
    - hil-bench
  resource_group: hil-bench
  rules:
    - if: '$HIL_ENABLED == "true" && $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH'
      when: manual
  environment:
    name: hil-lab
  script:
    - perl ci/job.pl hil-test
  artifacts:
    expire_in: never
    paths:
```

Both jobs are off unless a project variable switches them on. You can confirm which jobs a pipeline would run, on your laptop, without a lab:

```
git init -q -b develop lab && cd lab
cp "$KIT/tools/gitlab/templates/gitlab-ci.yml" .gitlab-ci.yml
git add . && git commit -qm "pipeline template"
perl "$KIT/bin/gitlab.pl" ci --list --event schedule --branch develop --var SIL_ENABLED=true |
grep -E "sil|hil"
```

```
cd lab
perl "$KIT/bin/gitlab.pl" ci --list --event push --branch develop --var HIL_ENABLED=true --var
CI_DEFAULT_BRANCH=develop | grep hil
```

The local runner accepts the resource group and the manual rule, but only a real GitLab enforces them – one job at a time on the bench, and a protected environment that only authorized test engineers can start. Test those on the server before the first bench run.

Formal evidence has to outlive GitLab's default artifact expiry. The template marks the verification jobs' artifacts `expire_in: never`; for qualification evidence, also archive the results to the package registry with the baseline (Chapter 20, The pipeline, job by job (p. 302)).

44.6 Closing the loop on failures

When a test fails, the result goes into the results file like any other, the matrix shows it red on the next pipeline, and an issue is opened as the discrepancy report – linked to the requirement, the verification case, the failing job and its artifacts. The fix goes through the normal merge request flow, and the test runs again as regression. In Tidewater's plan the first failed result arrives with drop 5 and opens a discrepancy report automatically (Chapter 45, The prime's model changes: receive, assess, disposition (p. 619)).

Tag the configuration under test – a test readiness baseline, a qualification baseline – so every result names exactly which design it tested. A result without a commit is an anecdote.

Exercise 44.1: Read the matrix

In the tidewater sandbox, regenerate the VCRM as above and open `vcrm.csv`. Pick one requirement with status "no case". Find it in the trace report. Who should write its verification case, and how would you ask them?

Exercise 44.2: Map your program to the stages

Write the five stages down for your own program. For each, name the owner (government, prime, test organization), the date it could start, and the test campaign it must not interrupt.

44.7 You can now

- Keep every verdict – automated, inspection, analysis – in one results format with the commit it applies to.
- Regenerate a VCRM on every change and read what its numbers mean for who has to act.
- Plan a testing transition in stages that management can track and that never gambles a qualification.

44.8 Applying this to your project

Copy the results schema and the rule that every verdict names a commit. Adapt the VCRM's columns to your contract's deliverable format. Decide locally who owns each stage, which test campaigns are frozen windows, and when the customer sees the pilot's output. Start with stage 1 even if nothing else is possible: a generated matrix that shows the unverified requirements is the quickest visible win in testing.

CHAPTER

45

The prime's model changes: receive, assess, disposition

From what you know →

You know

contractor deliverables under a CDRL, and the review cycle that follows each one.

→ Here it is called

prime drops: receive, assess, disposition.

→ What is new

each delivery is compared element by element, its impact on your model is computed, and every decision is an issue.

Survival kit: The prime keeps changing the model you just ported

What you have

a SysML v2 model the team built from the prime's Cameo export, a contract that says the prime maintains the authoritative v1 model, and a calendar with a new drop on it every quarter.

What you need

a way to take each drop in without losing a week, without losing your own work, and without accidentally accepting a change nobody authorized. This chapter is that way, end to end, with every command run against real drops.

The port was never a one-time conversion. Northwind Marine Systems, the prime, kept its Cameo model and kept changing it: a new sensor, a new operating mode, a deleted requirement, a renamed activity, test results from its lab. Seven drops arrive over the eighteen months of the Tidewater project, and every one of them lands on a model that Jim, Bob and Tom have been editing in the meantime.

In the document era this was the problem everybody dreaded and nobody owned. A new revi-

sion of the SRS arrived as a PDF with change bars, somebody compared it by eye with the old one, and the differences that mattered to the government's own documents were found weeks later, often at a review, often by the customer. Bob remembered it well:

> *Rev D of the ICD came in on a Friday. By the next Friday I had found eleven changes. The twelfth one we found at the PDR, in front of the program manager.*
>
> — Bob, at the sprint 6 retrospective

This chapter shows the loop the team built instead. It has six steps, each with a tool, and each step produces a record someone can audit later:

1. **Receive** the drop and check it is exactly what the prime sent.
2. **Archive** it unchanged, where nobody can edit it.
3. **Compare** it with the previous drop, element by element, by the prime's own element IDs.
4. **Assess** what each change touches in the government's model.
5. **Disposition** each significant change at the change board, as an issue with a decision.
6. **Implement** the accepted changes through ordinary merge requests, and report problems back to the prime.

Before any of that, there is one question that is not about tools at all.

45.1 Contract first

A change in the prime's model is not direction. This is the single most important sentence in this chapter. The prime maintains its Cameo model under a contract data requirement (CDRL A003 on Tidewater). When the prime adds a hygrometer to the sensor suite in its model, that tells you what the prime has done. It does not tell the government team to do anything. If Bob updates the government model because "the prime's model changed," and the change was never approved through the contract, Bob has just spent government effort on work nobody authorized, and possibly accepted a change the program office would have rejected.

So before the first drop, three questions go to the program office and the contracting officer's representative, in writing:

- **What is the prime's model, contractually?** Is it the authoritative source for requirements and interfaces, or supporting data? On many programs the specifications and interface control documents still govern, and the model is a deliverable that illustrates them. On Tidewater the System Requirements Specification in DOORS governs; the Cameo model is "supporting data, maintained by the contractor."
- **Which version is on contract?** The drop that was delivered last is not necessarily the one the program accepted. Tidewater's rule: a drop becomes the reference only after the change board has dispositioned it.
- **Through which channel do changes become direction?** A contract modification, technical direction letter, an approved engineering change proposal, or an interface change approved by the interface control working group. The drop itself is never the channel.

Jim's answer to the third question became the rule the team works by: **every drop goes through the change board, with the program office and contracts represented, before anything in it**

changes the government model. The board is small on Tidewater (Jim and the deputy program manager), and it meets in GitLab, but it exists, and its decisions are recorded.

Objection: "We are the government. We can just change our model."

Bob said it in sprint 2: the government team owns the v2 model, so why ask anybody's permission to keep it in step with the prime?

Answer

you can change your model whenever you like. What you cannot do is let the prime's model decide what your model says without a decision on record. If the prime renames an activity, follows it with three more changes, and one of them conflicts with a requirement the program office cares about, the record has to show who looked at it and what they decided. The change board costs Jim and the deputy program manager twenty minutes per drop. The alternative is an audit finding, or an unfunded change.

45.1.1 The model exchange agreement

The second thing to settle before the first drop is how drops arrive. Tidewater wrote it down as a short model exchange agreement, a page in the repository's `docs/` folder that both sides signed in week zero. It is the interface control document for the interface between two organizations' engineering processes, and like any ICD it says what crosses, in what format, how often, and what happens when something is wrong:

tools/gitlab/templates/docs/model-exchange-agreement.md

```
# Model exchange agreement with the prime

[[_TOC_]]

> [!note]
> This is the working agreement between the government model team and the prime contractor's
modeling team: how the
> prime's SysML v1 model and DOORS requirements reach the government, and how problems go back.
It does **not**
> change the contract. Where it touches deliverables, it implements the CDRL and its DID; propose
changes to those
> through the contracting officer. Fill in every `<!-- -->` with the prime's modeler and record
the date below.

| | |
|---|---|
| Government lead | <!-- name, role --> |
| Prime lead | <!-- name, role --> |
| Contract data item | <!-- e.g. CDRL A003 System Model --> |
| Agreed on / last revised | <!-- date; each revision through a merge request --> |

## 1. What is delivered: one drop

A **drop** is one delivery of the model, complete and self-contained:

| File | Format | Notes |
|---|---|---|
```

```
| the model | XMI 2.5.1 from Cameo (Eclipse UML2 5.x export) | the whole model, every package;
not a partial export |
| the requirements | ReqIF 1.2 (`.reqif` or `.reqifz`) from DOORS | every formal module the model
traces to |
| the requirements, readable | CSV from the same DOORS view | for people without a ReqIF reader |
| test results, when there are any | JUnit XML, or CSV in the agreed results format ([ADR
0003](adr/0003-results-schema.md)) | one row per requirement and verification case |
| `MANIFEST.txt` | text, the fields in section 4 | written by the prime's build, not by hand |
| change notes | in `MANIFEST.txt` or a `CHANGES.md` | what changed since the last drop, in the
prime's own words |

## 2. When

| | Agreed |
|---|---|
| Cadence | <!-- e.g. every 3 months, plus before each review --> |
| Delivery day | <!-- e.g. a Friday, so the government can take it in the next sprint --> |
| Government acknowledgment | within <!-- 2 --> working days: received, checksums verified |
| Government disposition | within <!-- one sprint -->: accepted into the government model, with
any discrepancy reports |

## 3. Stable identifiers
```

Two clauses in that agreement made every later step possible, and they are worth asking for on any program:

- **Stable element IDs.** Cameo gives every element an ID (`xmi:id`). If the prime's export keeps those IDs from one drop to the next, a program can compare drops element by element and know that a renamed activity is the same activity. If the export generates new IDs each time, every comparison degrades to guessing by name. The prime's modeler agreed in week zero; it cost them a checkbox in the export settings.
- **Change notes with every drop.** A list of the prime's own change requests, each with an ID, a one-line rationale and the elements it touches. The prime already had this list for its own change control; the agreement just asks for it to be delivered.

45.2 Receive: what arrived, and is it what was sent?

To follow along you need drops. The kit's generator writes the same seven drops the Tidewater team received, each with the prime's manifest and change list. Make the first five:

```
for n in 1 2 3 4 5; do perl "$KIT/sim/tidewater-v1.pl" --drop $n drops/d$n > /dev/null; done
ls drops/d2
```

A drop is four or five files: the Cameo model as XMI (`tidewater.xmi`), the DOORS modules as ReqIF and as CSV, the prime's change list (`CHANGES.csv`), and a manifest. From drop 4 on there are also the prime's test results. The manifest is the transmittal record. Read it first, every time:

```
cat drops/d2/MANIFEST.txt
```

It names the delivery (version, date, CDRL), who built it and with what (the prime's Jenkins job and build number, its repository commit, the tool versions), the change requests in this drop, and a SHA-256 checksum for every file. The checksums are what make "is this what was sent?" a question with a yes-or-no answer instead of an opinion. Check them before anyone opens a file:

```
cd drops/d2
sed -n '/^Files/,$p' MANIFEST.txt | tr -d '\r' | awk 'NF == 2 { print $1 "  " $2 }' | sha256sum
-c
```

(The `tr -d '\r'` is there because the prime's Windows build writes the manifest with Windows line endings; without it `sha256sum` looks for a file whose name ends in an invisible carriage return.) Every line must say `OK`. A single `FAILED` means the file is not what the prime's build produced: it was damaged in transit, edited on the way, or the manifest itself is wrong. Either way the drop is not imported. The team does not fix it; the prime sends it again.

Tip: Why a checksum and not a careful look

A checksum is a fingerprint of the file's exact bytes. Change one character anywhere in a 120 KB XMI file and the fingerprint changes completely. It is the same idea as the configuration audit's check that the delivered configuration matches the documented one, done by a program in a second instead of by a person in an afternoon.

45.3 Archive: keep it exactly as delivered

The received drop is evidence. Months later someone will ask "what exactly did the prime deliver on 2 April?", and the answer has to be the files themselves, not a converted copy that has since been edited. Tidewater keeps every drop in two places, neither of which anybody edits:

- **The vendor branch** (`vendor/prime`) of the model repository. Each drop is converted there, as delivered, and committed with the raw files under `imports/<version>/`. Nobody commits anything else to that branch; its protection rule allows only the configuration manager to push, and nobody to merge into it. The vendor branch is the reason a new drop can be merged into a model the team has been editing: git compares the new drop with the old drop *as converted*, and carries only the prime's changes across. Chapter 12, Git from zero, for systems engineers (p. 171) explains the idea, and practice lesson 12 lets you do it.
- **The Generic Package Registry** of the GitLab project, as `prime-drop/<version>`. The pipeline's `drop-package` job files every drop there automatically, with its checksum, so the CDRL deliverable sits next to the model it became and can be downloaded by version.

If your program's access rules for the prime's data differ from those for the model, the archive moves to its own restricted project and the model project references it by version. The loop stays the same.

45.4 Compare: what changed, element by element

Converting a drop is the easy part. The hard question is what changed since the last drop, because that is what the team has to assess. The converter answers it directly from the two XMI files, matching elements by the prime's IDs:

```
perl "$KIT/bin/xmi2sysml.pl" --delta drops/d1/tidewater.xmi drops/d2/tidewater.xmi | head -25
```

Read the first four lines before anything else. They say how big the change is, and they split it in two:

- **Carried by the converter.** Re-converting the drop reproduces the change in SysML v2 text. The vendor-branch merge brings it across without anyone retyping it.
- **Manual.** The converter does not carry this kind of element (state machines, interactions, constraint expressions, which it leaves as TODO lines). A person has to rework the government's v2 text by hand. These are the expensive changes, and the report lists them at the end as a checklist.

Drop 2 is all carried. Drop 3 is not. Its change to the buoy's operating modes (a new Storm state) sits inside a state machine, which the converter leaves as a TODO; the government team wrote the state machine by hand in sprint 6. Look at the manual list:

```
perl "$KIT/bin/xmi2sysml.pl" --delta drops/d2/tidewater.xmi drops/d3/tidewater.xmi | sed -n
'1,4p;/manual rework list/,$p'
```

Five manual items, all inside `BuoyModes`: the Storm state, two transitions, and their two triggers. Without the element-level comparison this change would have been invisible: the converted v2 text of drop 3 is the same as drop 2's around the state machine, because both just say TODO there. A diff of the converted files would have shown nothing. This is the strongest argument for stable IDs and an element-level delta.

Note: Why compare v1 with v1, not v2 with v2

It is tempting to convert both drops and diff the SysML v2 text. That catches everything the converter carries and misses everything it does not, which is exactly the part that needs a person. The comparison has to happen on the prime's model, in the prime's terms, with the converter's coverage reported per change.

45.5 Assess: what does it touch in our model?

The delta says what changed in the prime's model. The change board needs to know what that touches in the government's model: which requirements, which verification cases, which interfaces, and which hand edits the team made that now need redoing. That is impact analysis, and it needs one more piece of information: where each of the prime's elements ended up in the government's v2 text.

The converter writes that as a map when it converts a drop. Start the government's model from drop 1, laid out in layers with the same layout map the team uses (Chapter 16, The model repository: what goes where (p. 245)), and keep the map:

```
mkdir -p maps
perl "$KIT/bin/xmi2sysml.pl" --out gov/model --layout-map "$KIT/examples/tidewater-v1/layout.map"
\
    --map maps/v1.0-map.csv drops/d1/tidewater.xmi
head -4 maps/v1.0-map.csv
```

Each row ties one of the prime's elements (its `xmi_id`) to the v2 name, kind, file and line it became. Now ask the question the board needs answered, for drop 2:

```
perl "$KIT/bin/xmi2sysml.pl" --delta drops/d1/tidewater.xmi drops/d2/tidewater.xmi \
    --impact gov/model --map maps/v1.0-map.csv --impact-csv impact-v1.1.csv | sed -n '/impact
    summary/,$p'
```

The summary is the change board's agenda in eight lines: three requirements touched (the new humidity requirement TW-SYS-017, the stakeholder need it derives from, and TW-SYS-006, whose wording the prime aligned with DOORS), two verification cases, one interface and one allocation, grouped under three of the prime's change requests. Above the summary, the full report walks through each change request with its rationale and every v2 element it touches, with file and line, so a reviewer can open exactly those lines.

45.5.1 Change requests: the prime's own reasons

Because the drop carries the prime's change list, the impact report is grouped by the prime's change requests rather than by element. That turns a list of forty element changes into three or four decisions:

```
cat drops/d5/CHANGES.csv
```

Drop 5 has two change requests. NWM-CR-0024 adds a reserve term to the energy balance constraint, with a reason. NWM-CR-0025 renames the activity `PublishBulletin` to `IssueBulletin`, and the rationale column is empty. The impact report says so plainly:

```
perl "$KIT/bin/xmi2sysml.pl" --delta drops/d4/tidewater.xmi drops/d5/tidewater.xmi --impact
gov/model \
    | sed -n '/^== NWM-CR-0025/,/^$/p;/impact summary/,$p'
```

A rename with no reason is not something to accept by reflex. Names in the model are referenced from requirement text, from test procedures, from the government's own hand-written behavior and from documents outside the model. The board needs to know why before it agrees. That case, a change request with no rationale, is the textbook reason for the "reject or clarify" disposition below.

45.5.2 Hand edits that need redoing

The most valuable line in the impact summary is the last one: **hand edits to redo**. The government team closes the converter's TODO lines by hand, writing the state machines, interactions and constraint expressions the converter could not carry. When the prime then changes one of those constructs, the vendor-branch merge brings nothing across (the converter carried nothing), and the team's hand-written version silently goes stale. The impact analysis looks for exactly that: a TODO line the converter would write, which is no longer in the government's model because someone replaced it.

Play the part of Bob in sprint 6, closing the `BuoyModes` TODO by hand:

```
perl -i -pe 's{^(\s*)// TODO uml:StateMachine .BuoyModes.*$}{$1exhibit state buoyModes :
BuoyModes;   // written by hand}' \
    gov/model/physical/stations/Stations.sysml
grep -n "buoyModes" gov/model/physical/stations/Stations.sysml
```

Then drop 3 arrives, with the prime's new Storm state:

```
perl "$KIT/bin/xmi2sysml.pl" --delta drops/d2/tidewater.xmi drops/d3/tidewater.xmi --impact
gov/model \
    | sed -n '/impact summary/,$p'
```

The report names the hand edit, the file and the line: Bob's state machine needs a Storm state, and nothing else in the loop would have said so. On Tidewater this became a task in the drop-3 intake story, assigned to the person who wrote the original edit.

45.6 Disposition: one issue per significant change

The impact report is the input to the change board. The board's output is a decision per change request, recorded where the work happens: in GitLab issues. Tidewater opens one issue per drop (the intake, from the Prime Drop template) and one issue per significant change request, labeled `prime-change`, with exactly one of four disposition labels:

Table 45.1: The four dispositions a change request can get

Disposition	Meaning	What happens next
`disposition::accept`	Accepted as delivered	The vendor-branch merge brings it in; nothing more
`disposition::accept-with-impact`	Accepted, and it adds government work	Linked tasks for the extra work; the cost and schedule effect noted on the issue
`disposition::reject-clarify`	Not accepted yet	A discrepancy report asks the prime for the rationale or a correction; the issue stays open until the prime answers
`disposition::no-impact`	Cosmetic	Nothing for the government model to do

On Tidewater, drop 4's drifting buoys were accepted with impact (new verification cases, a new part of the station segment to review); drop 5's unexplained rename was sent back for clarification; most change requests were simply accepted; and the documentation edits in drop 7 had no impact. Which drops and change requests got which disposition is part of the simulation's later sprints, and the counts belong to its evidence rather than to this page.

The issues themselves take a minute to open. Create a scratch GitLab and the project from the team's project file:

```
gl() { perl "$KIT/bin/gitlab.pl" --local gitlab --project tidewater/model "$@"; }
for u in jim bob tom; do gl user $u; done
gl --as tom setup "$KIT/tools/gitlab/templates/project.conf" | grep -E "^(created|protected
branch main)"
```

Then open the intake issue and one issue per change request, straight from the impact CSV the converter wrote:

```
gl() { perl "$KIT/bin/gitlab.pl" --local gitlab --project tidewater/model "$@"; }
gl --as jim issue new "Prime drop v1.1: intake and disposition" --label type::drop --label
prime-change
perl -ne 'chomp; my @c = split /,/; print "$c[-1]\n" if $. > 1 && $c[-1]' impact-v1.1.csv | sort
-u |
while read cr; do
  title=$(grep "^$cr," drops/d2/CHANGES.csv | cut -d, -f2)
  gl --as jim issue new "$cr $title" --label prime-change --description "Drop v1.1. See the
  impact report on the intake issue."
done
gl issue list --label prime-change
```

Each disposition is a quick action in a comment, so the decision, who made it and when are all in the issue's history:

```
gl() { perl "$KIT/bin/gitlab.pl" --local gitlab --project tidewater/model "$@"; }
gl --as jim issue note 2 '/label ~"disposition::accept"'
gl issue show 2 | head -3
```

The deputy program manager concurs in a comment on the intake issue. That comment is the change board's minutes.

Objection: "This is a lot of ceremony for a sensor."

Tom raised it at the drop-2 intake: three issues and a board decision for one hygrometer and a requirement.

Answer

the ceremony is proportional. A drop with three change requests takes three issues and one twenty-minute conversation. The issues are generated from the impact report, so nobody

types the list. And the record is what lets the team answer, six months later, "when did we accept the humidity requirement, and who agreed?" in one search instead of an archaeology project. When a drop is truly trivial, the board dispositions everything as accept in one comment, and the intake closes the same day.

45.6.1 prime-tbd: when you cannot wait for the drop

Sometimes the government team needs a change before the prime delivers it. In sprint 6 the team knew from the prime's modeler that a humidity sensor was coming, and Bob needed something in the model for the data exchange table. Waiting for drop 2 would have stalled the table for a month. So Bob modeled an interim `HumiditySensor` on develop, and the team opened an issue labeled `prime-tbd`: work that rests on an assumption about the prime's next drop, closed when the drop lands.

When drop 2 arrived, the vendor-branch merge conflicted exactly where Bob's interim sensor sat, which is the point: the conflict forces the decision. The team kept the prime's hygrometer, deleted the interim, and closed the `prime-tbd` issue in the same merge request. Practice lesson 12 replays that merge, and you can run its first steps here:

```
. ../env.sh
perl "$KIT/bin/xmi2sysml.pl" --delta ../drops/v1.0/tidewater.xmi ../drops/v1.1/tidewater.xmi |
head -4
gl flow vendor ../drops/v1.1 v1.1 || echo "exit status $?"
```

The conflict is the system working: two teams changed the same thing, and git stopped to ask a person which one wins. Finish the lesson to resolve it in Vim (`agile.pl practice 12`).

45.7 Implement, and report back

Accepted changes reach the government model through the normal merge request flow, nothing special:

- The vendor-branch import and its merge into a `feature/drop-<version>` branch are done by the configuration manager (`gitlab.pl flow vendor`), and the merge request carries the delta and impact reports in its pipeline (the `delta` job runs on drop branches automatically).
- Each hand edit to redo becomes a task, linked to its change request issue, and goes in on its own branch with its own review. The merge request description cites the prime's change request ID, so the history reads "Storm state added to BuoyModes (NWM-CR-0015)."
- Conflicts are resolved upstream-first: the prime's version wins, then the government's edit is reapplied on top of it. Never the other way around, or the next drop will conflict again.

Problems the team finds go back to the prime as **discrepancy reports**: GitLab issues labeled `type::discrepancy` and `owner::prime`, transmitted through the contracting channel with the memo template in `docs/memos/templates/` (Chapter 37, Writing it down: DoD-style memos from Vim (p. 545)), and kept open until a later drop fixes them. The prime's change requests cite them when they do, which closes the loop visibly. Drop 3's NWM-CR-0018 and NWM-CR-0019

say "Answers government DR" in their rationale; drop 4's NWM-CR-0022 brings TW-SYS-016 into the model in answer to another. The discrepancy report numbers in this paragraph are the ones the prime's change notes cite; the simulation's issue numbers for them are in its evidence.

45.7.1 Returning the government's model

The exchange runs both ways. When the prime needs the government's design data (at a baseline, or for an interface the government owns), the team returns it as a controlled deliverable: built from a baseline tag, never from a working branch, with a transmittal record just like an incoming drop. Here is the mechanics, on a scaffolded model repository:

```
perl "$KIT/tools/gitlab/kickoff.pl" scaffold model --project tidewater/model --title Tidewater >
/dev/null
cd model
git tag -a baseline-0 -m "baseline-0 (practice)"
mkdir -p ../deliverable
git archive --format=tar.gz --prefix=tidewater-model-baseline-0/ -o
../deliverable/tidewater-model-baseline-0.tar.gz baseline-0 model docs
cd ../deliverable && echo "sha256: $(sha256sum tidewater-model-baseline-0.tar.gz)"
```

On Tidewater the pipeline does this on every baseline tag (the `publish-package` job puts the archive, the verification matrix and the toolchain inventory in the package registry), and the transmittal memo goes out with it. Be honest with the prime about one limit: **the kit has no SysML v2 to SysML v1 converter.** The government returns SysML v2 text, the generated tables and the verification matrix. Whether the prime's Cameo can import SysML v2 text depends on the prime's tool version; settle it in the model exchange agreement, not on delivery day.

45.8 Two strategies at the v1-to-v2 boundary

Tidewater mirrors the prime's whole model: every drop is converted in full onto the vendor branch, and the government's model is that conversion plus the government's edits. There is a second strategy, and on many programs it is the better one.

Table 45.2: Mirroring the whole model versus translating only the interface surface

	Full mirror (Tidewater)	Interface surface
What is converted	the whole v1 model, every drop	only what you consume: your allocated requirements, your external interfaces, the parameters you depend on
Where it lives	the government model itself, layered folders	one generated package (for example `model/external/prime/`), never hand-edited
How your design uses it	you edit the converted text in place, closing TODO lines	your design satisfies and connects to elements of the generated package
What a new drop does	a 3-way merge on the vendor branch; conflicts where both sides edited	the package is regenerated; validate and trace checks show what broke

	Full mirror (Tidewater)	Interface surface
Best when	the government needs the whole system model in v2 (the port is the goal)	you are a subsystem or segment team consuming a prime's model

Tidewater chose the full mirror because the government's goal was the system model itself. It already uses the interface-surface pattern in one place: the DOORS modules. They are converted by script into `model/requirements/` on every drop and never hand-edited; the government's design satisfies them. If your team owns a subsystem and the prime owns the system, start with the interface surface: it is smaller, the generated package never conflicts, and the boundary is explicit.

45.9 The prime's Jenkins: a delivery interface

Northwind builds its drops in Jenkins, a continuous integration server older than GitLab's and still common in industry. The government team uses GitLab. Nobody on Jim's team needs to run Jenkins, and the government never asks the prime to migrate. What the team does need is to understand what Jenkins hands them, and to agree how.

45.9.1 Jenkins in GitLab terms

Most Jenkins words have a GitLab counterpart. You need them to read the prime's build records and to talk to the prime's build engineer:

Table 45.3: Jenkins and GitLab, side by side

Jenkins	GitLab	In systems-engineering words
Jenkinsfile	`.gitlab-ci.yml`	the procedure, written down and versioned
agent (a build machine, by label)	runner (by tag)	the bench the procedure runs on
stage / steps	stage / `script`	the procedure's sections and lines
`environment { }`	`variables:`	the procedure's settings
`withCredentials`	a masked CI/CD variable	a credential handed over for one step, never written down
JUnit plugin (`junit 'results.xml'`)	`artifacts: reports: junit:`	test results read into a trend
`archiveArtifacts …` `fingerprint: true`	job artifacts, package registry	the delivered items, kept with their checksums
build number, job URL	pipeline ID, pipeline URL	the record's serial number
shared workspace between stages	a fresh workspace per job	GitLab starts each job clean; pass files on as artifacts

The last row is the one that surprises people moving between the two: a Jenkins job usually keeps one working folder for all its stages, while GitLab gives each job a fresh checkout. Chapter 19, CI/CD from zero (p. 288) explains why.

45.9.2 Three ways a drop can arrive

From least to most automated, these are the delivery patterns a model exchange agreement can choose between:

1. **File delivery.** The prime's Jenkins builds the drop and archives it; someone transfers the files under the CDRL to approved government storage; Tom runs the intake by hand. This is how Tidewater's first three drops arrived, and it is the right place to start: nobody has to grant anybody network access.
2. **Push into GitLab.** The prime's Jenkins uploads the drop to the government project's Generic Package Registry, using a deploy token that can do nothing but write packages, and then fires a pipeline trigger with the drop's version. The government's `intake` job runs on its own. Tidewater switched to this from drop 4.
3. **GitLab pulls from Jenkins.** A scheduled government job fetches the prime's last successful build over the prime's API. Possible, and usually blocked: it needs a network path from the government enclave into the contractor's, and credentials for the prime's server held by the government. Few security officers approve it.

The illustrative Jenkinsfile in the kit shows what pattern 2 looks like on the prime's side. The kit does not run it (there is no Jenkins here); read it for the shape:

tools/gitlab/templates/prime/Jenkinsfile

```
// Jenkinsfile -- the PRIME's delivery job for the Tidewater model
(tools/gitlab/templates/prime/Jenkinsfile).
//
// ILLUSTRATIVE: this file is not run by the kit (there is no Jenkins here). It shows what the
prime's legacy Jenkins
// workflow does for each drop, and which part of it is the interface with the government's
GitLab -- the "ICD between
// pipelines" agreed in sprint 4. The government never asks the prime to migrate; it asks for
these outputs:
//   1. the drop files (XMI from Cameo, ReqIF from DOORS, the test results as CSV in the
government's results schema and
//      as JUnit XML), with a MANIFEST.txt that records the Jenkins job URL and build number, the
prime's repository
//      commit, the tool versions, and the sha256 of every file;
//   2. an upload of those files to the government project's Generic Package Registry as
prime-drop/<version>, with a
//      DEPLOY TOKEN that can only write packages (GOV_DEPLOY_TOKEN; expires; rotated with each
contract option);
//   3. a pipeline TRIGGER with DROP_VERSION=<version> (GOV_TRIGGER_TOKEN), which starts the
government's `intake` job:
//      it re-downloads every file, checks every sha256 against the MANIFEST, and opens the Prime
Drop issue. The
//      prime's green build is evidence, not acceptance; the government's own gates run on the
import MR.
// Both tokens live in Jenkins credentials (withCredentials), never in this file or the job log.
//
// The government-side YAML is the `intake` job in tools/gitlab/templates/gitlab-ci.yml; test it
locally with
//   perl $KIT/bin/gitlab.pl ci --event trigger --branch develop --var DROP_VERSION=v1.3 --var
INTAKE_DROP_DIR=/path/to/drop
```

45.9.3 Two tokens, each able to do one thing

Pattern 2 needs two credentials, created by a maintainer of the government project and handed to the prime through the contracting channel. Each can do exactly one thing, and each expires:

```
gl() { perl "$KIT/bin/gitlab.pl" --local gitlab --project tidewater/model "$@"; }
gl --as tom token deploy new prime-jenkins --scope write_package_registry --expires 2027-12-31
```

The deploy token can upload packages. It cannot read the model, push code, open issues or see anything else. If it leaks, the worst an attacker can do is upload a fake drop, which the intake job will reject because its checksums will not match the manifest the prime's build wrote.

```
gl() { perl "$KIT/bin/gitlab.pl" --local gitlab --project tidewater/model "$@"; }
gl --as tom trigger new "prime Jenkins delivery"
```

The trigger token can start a pipeline and pass it variables. It cannot change anything. The triggered pipeline runs the government's own `intake` job, as the token's owner, with the government's rules. Both tokens live in Jenkins' credential store on the prime's side, and in the government's token inventory (Section 42.4, Tokens and rotation (p. 590)) with an owner and an expiry date.

With the deploy token, the prime's build uploads the drop; in the scratch GitLab you can play that part:

```
gl() { perl "$KIT/bin/gitlab.pl" --local gitlab --project tidewater/model "$@"; }
for f in drops/d4/*; do gl --as tom package put prime-drop v1.3 "$f" > /dev/null; done
gl package list
```

45.9.4 The intake job: the government's gate

The triggered pipeline runs one job: `intake`. It is in the team's pipeline file, and short:

tools/gitlab/templates/gitlab-ci.yml

```
    - perl ci/job.pl drop-package

# INTAKE -- a delivery from the prime's Jenkins: its build uploads the drop to the Generic
Package Registry (prime-drop/
# <version>, with a deploy token that can do nothing else) and fires a pipeline trigger with
DROP_VERSION. This job then
# fetches every file, checks each sha256 against the MANIFEST the prime's build wrote, keeps the
drop as received
# (formal evidence: never expires) and opens the Prime Drop issue that starts the vendor import.
A green Jenkins build
# on the prime's side is evidence, not acceptance: the government's own gates run on the import
MR.
intake:
  stage: delta
  rules:
```

```
    - if: '$CI_PIPELINE_SOURCE == "trigger" && $DROP_VERSION'
  script:
    - perl ci/job.pl intake
  artifacts:
    expire_in: never
```

It fetches every file of the drop, checks each one against the checksum in the manifest the prime's build wrote, keeps the drop as received (an artifact that never expires: it is formal evidence), and opens the Prime Drop issue. A green build on the prime's side is evidence that the prime's checks passed. It is not acceptance. Acceptance is the government's own gates on the import merge request, and the board's disposition.

You can run the intake exactly as the trigger would, locally, against a folder instead of the registry. It needs a model repository with the team's pipeline, which the scaffold you made above has:

```
cd model
perl "$KIT/bin/gitlab.pl" ci --event trigger --branch develop --job intake \
    --var DROP_VERSION=v1.3 --var INTAKE_DROP_DIR="$PWD/../drops/d4" --logs ../intake-logs
grep -E "^(ok|ACCEPTED)" ../intake-logs/intake.log
```

Now the case the intake exists for: a file changed somewhere between the prime's build and the government. One line added to the XMI, the kind of edit nobody would notice by eye:

```
cp -r drops/d4 drops/d4-altered && echo "<!-- edited in transit -->" >>
drops/d4-altered/tidewater.xmi
cd model
perl "$KIT/bin/gitlab.pl" ci --event trigger --branch develop --job intake \
    --var DROP_VERSION=v1.3 --var INTAKE_DROP_DIR="$PWD/../drops/d4-altered" --logs ../intake-bad
    \
    || echo "exit status $?"
grep -E "^(MISMATCH|REJECTED)" ../intake-bad/intake.log
```

The pipeline fails, the log names the file and both checksums, and the drop is not imported. The team tells the prime, and the prime's build sends it again. Nobody spends an afternoon working out what changed.

45.9.5 The ICD between the two pipelines

Everything above is an interface between two organizations' processes, and it deserves the same treatment as any other interface: written down, agreed, and changed under control. Tidewater's version is a section of the model exchange agreement. Its checklist, which you can lift for your own program:

Checklist: The pipeline interface, agreed with the prime

- ☐ What crosses: the file list per drop (XMI, ReqIF, CSV, CHANGES.csv, results, MANIFEST.txt)

- ☐ Formats and versions: Cameo version and export settings (stable IDs on), DOORS export options
- ☐ The manifest fields: version, date, CDRL, Jenkins job URL and build number, prime repository commit, tool versions, a checksum per file, the change requests
- ☐ How it crosses: file delivery, or registry upload plus trigger (and who may change the pattern)
- ☐ Who holds which credential: the deploy token and the trigger token, their scopes, expiry and rotation
- ☐ What the government does on receipt: the intake checks, the board, the disposition time
- ☐ What happens when something is wrong: rejected drop, resend, discrepancy reports and their return path
- ☐ Markings: the prime's proprietary and export markings, preserved through every conversion

45.10 Cautions

A few things went wrong, or nearly did, on Tidewater and on programs like it:

- **You will probably still need a Cameo seat.** If the prime delivers `.mdzip` project files instead of XMI, or if an export goes wrong and someone has to look at the original, the government needs at least one Cameo licence to open it. Budget for one during the transition rather than discovering the need on drop day. The model exchange agreement should ask for XMI and ReqIF so the seat is rarely used.
- **The prime's markings travel with the data.** Proprietary and export-control markings in the prime's model apply to every conversion of it: the v2 text, the generated documents, the packages. Preserve them, mark the vendor branch and the archive accordingly, and restrict access to the people the markings allow. If those people are fewer than the model team, the archive goes in its own project (Chapter 48, Security: protect the environment, engineer it into the system (p. 651)).
- **Don't freeze waiting for the prime.** If a drop is late, keep working against the last accepted one, and record every assumption about the next drop as a `prime-tbd` issue. The conflict at the next merge will make the team revisit each one.
- **Never edit the vendor branch by hand.** It is tempting to "just fix" a converter quirk there. The next drop's import overwrites it, or worse, merges around it. Fix converter problems in the converter, and government changes on develop.
- **Keep the drop loop's records in GitLab, not in email.** The intake issue, the change-request issues, the dispositions and the discrepancy reports are the configuration management record of the boundary. An email thread with the prime's modeler is fine for the conversation; the decision goes in the issue.

45.11 Applying this to your project

- **Before the first drop:** get the three contract answers in writing, write the model exchange agreement, and ask for stable IDs and change notes. Those two requests cost the supplier almost nothing and make everything else in this chapter possible.

- **Choose your boundary strategy:** full mirror if the model itself is your deliverable, interface surface if you consume someone else's model.
- **Start with file delivery,** move to registry-and-trigger once the intake job has proven itself on a few drops.
- **Copy the six steps** (receive, archive, compare, assess, disposition, implement) and the four disposition labels. The tools are in the kit; the discipline is the change board meeting on every drop.
- **Measure the loop:** the time from delivery to merged on the integration branch, the share of change requests with a disposition within a week, and the open discrepancy reports. These are the numbers that tell management the boundary is under control.

Exercise 45.1: Read a drop like a change board

Generate drops 3 and 4 (`perl "$KIT/sim/tidewater-v1.pl" --drop N DIR`) and run the delta between them with `--impact` against a model converted from drop 3. For each change request, propose a disposition and write one sentence of justification. Which change adds government work, and what work?

Exercise 45.2: Write the clarification

Draft the discrepancy report for NWM-CR-0025 (the rename with no rationale) from the Discrepancy Report issue template: what you need from the prime, why the rename matters to the government model, and by which drop you need the answer.

Exercise 45.3: Break the intake on purpose

Copy a drop, change one character in `tidewater.reqif`, and run the intake job locally against the copy. Then fix the copy by regenerating the file instead. What does the intake report each time, and why is "regenerate" the only acceptable fix?

Exercise 45.4: Choose a strategy

Your team owns the data center segment; the prime owns the system model. Sketch which elements of the Tidewater model would go in an `external/prime` package under the interface-surface strategy, and which the data center team would own.

CHAPTER

46

Systems engineering work, not pipeline work

From what you know →

You know

the engineering itself: requirements, architecture, interfaces, analyses and verification.

→ Here it is called

work::engineering, as distinct from work::platform and work::learning.

→ What is new

the board shows how much of the team's time goes to engineering, sprint by sprint.

Survival kit: The work is the engineering

What you have

a backlog full of stories, and a nagging feeling that too many of them are about GitLab, runners and YAML, and too few are about the system.

What you need

a clear line between systems engineering work (deciding and proving what the system is) and platform work (building and keeping the machinery that checks and renders it), a way to see the balance sprint by sprint, and a catalog of real engineering stories to fill the backlog with once the machinery runs.

46.1 The line

Bob asked the question in the second sprint, and the question was a fair one:

> *Two sprints in, and I have written a pipeline description, a runner request and a branching guide. When do I do systems engineering again?*
>
> — Bob, sprint 2

It is the right question, and the answer has to be a number, not a reassurance. Every story on the Tidewater board carries one of three labels:

Definitions: The three kinds of work

work::engineering: deciding or proving what the system is. Requirements, architecture, interfaces, behavior, analysis, verification, change decisions. The output is model content or an engineering judgment someone signs. work::platform: building or keeping the machinery that checks, renders and protects the model. Pipelines, runners, GitLab settings, templates, access, the kit version. The output is a working tool. work::learning: becoming able to do either. Practice lessons, chapters of this book, pairing at the terminal. The output is a person who can do something they could not do last sprint.

A quick test for a story you are not sure about: **would a systems engineer from the document era recognize the output as engineering?** A closed TODO line that adds `@L2Segment;` to a part: yes, it is a configuration decision about the system's structure. A new pipeline job that counts TODO lines: no, it is a tool, however useful. A data exchange table generated from the interface items: yes, it is the interface control document, produced differently. Deciding who owns `Observation Record` so a dependency cycle can be broken: very much yes, it is an architecture decision.

The label is not a value judgment. Platform work is how the engineering becomes cheap to check, and learning is how a team that had never seen a merge request learned to run one. The point of the label is to make the balance visible, so that platform and learning shrink as they should, and so that nobody has to argue from feelings.

46.2 The balance, sprint by sprint

The split comes straight out of the plan file the simulation runs (`sim/tidewater-plan.pl`), so it cannot be rounded in anyone's favour:

```
perl -e 'require "$ENV{KIT}/sim/tidewater-plan.pl"; for my $s (grep { $_->{n} <= 6 } @{ tw_plan()
}) { my %w; for my $st (@{ $s->{stories} }) { $w{ $st->{work} } += $_->{p} for @{ $st->{tasks} }
} my $t = 0; $t += $_ for values %w; printf "Sprint %d: %4.1f points  engineering %3.0f%%
platform %3.0f%%  learning %3.0f%%\n", $s->{n}, $t, map { 100 * ($w{$_} // 0) / $t }
qw(engineering platform learning) }'
```

The shape is the one to aim for. In the first two sprints platform and learning take most of the points: there is a project to create, a runner to get approved, three laptops to set up, git and Vim to learn. From sprint 3 engineering takes the larger share, and by sprint 6 most of the work is engineering again, which is the answer to Bob's question, and the one to give with these numbers on the screen. GitLab can show the same split live: filter the board or the issue list by `work::engineering`, and the milestone page sums the points.

Two warnings about the number. It is a planning measure, not a productivity score: a sprint that is half platform because the runner died is a sprint that kept the team working. And it does not go to zero: maintenance sprints still carry a story or two of platform work (a runner upgrade, a token rotation, a kit version bump), which is how the machinery stays trustworthy.

46.3 The Tidewater catalog

These are the engineering stories in Tidewater's plan, grouped by the kind of engineering, with what each one changes in the model, the check that proves it, and the artifact someone outside the team can look at. Branch names follow `feature/<issue number>-<a few words>` (Appendix G, Modeling style guide (p. 739)); the issue numbers are left out here because they differ from run to run. Sprints are the plan's (Chapter 13, The operating model (p. 196)).

46.3.1 Requirements

Table 46.1: Requirements stories

Story (sprint, epic)	Model change	Check	Artifact
DOORS modules into the model (S3, E7)	the prime's DOORS export converted into `requirements/` by the vendor import	trace job: C, F, G, I and drift	trace report
First drift and gap analysis from the trace (S3, E7)	none: an analysis of what the trace found	the trace job's numbers	a wiki page, the input to the DRs
Discrepancy reports DR-1 to DR-4 to the prime (S4, E7)	none in the model: issues to the prime	each DR closes when a drop fixes it	DR issues, transmittal memo
Grade our requirements with the recipe (S5, E7)	none: findings on the prime's wording	`recipe` job findings	DR-6 to the prime (Chapter 7, Requirements a pipeline can check: the recipe (p. 73))
Trace completeness, package by package (S8-S16, E7)	satisfy and verify links, one package per sprint	trace ratchet: no new gaps, falling counts	trace report per package
Security requirements through DRs (S30, E16)	none until a drop brings `TW-SEC-*`	trace and gate	DRs, then requirements
Retire TW-SEG-009 (S31, E15)	the requirement marked retired, not deleted	trace excludes retired requirements	the retirement note in the CCB item

The surprise for the team was how many requirements stories change *nothing* in the government model. The requirements belong to the prime's DOORS modules; the government's engineering is finding what is wrong with them and saying so in a form the prime must answer. That is still the most valuable requirements work on the program, and it is engineering by any definition.

46.3.2 Architecture

Table 46.2: Architecture stories

Story (sprint, epic)	Model change	Check	Artifact
Level conventions for the part tree (S2, E6)	`library/conventions/` with the six level metadata definitions	lint, outline check	the conventions package
Close the Network TODO lines (S3, E6)	level tags; the `include use case` written by hand	lint 0/0; TODO count falls	part tree view
Close the Stations level tags (S4, E6)	`@L2Segment;` through `@L6 Component;` on every part	level-tag check	part tree view
Units and value types (S4, E6)	value types specialize ISQ quantities	lint (unresolved types)	none: correctness

Story (sprint, epic)	Model change	Check	Artifact
Break the Network to Operations cycle (S4, E6)	one dependency moved; the decision recorded in the MR	package view: one cycle fewer	package view
Untangle the Legacy Import package (S5, E6)	the clutter removed from the government model	outline and duplicates	DR-5 to the prime
Procured parts as metadata (S5, E6)	the `Procured` stereotype as `@Procured` metadata	lint	supplier search
Data exchange table for the program review (S6, E8)	flow properties documented on the interfaces	the table regenerates	the data exchange table
Break the Stations to DataCenter cycle (S7, E6)	the decision on who owns `ObservationRecord`	package view: zero cycles	package view, the decision in the MR
Trust zones on the part tree (S26, E16)	`@TrustZone` on each part	`model.pl gate` zones check	interconnection view with zones

46.3.3 Behavior

Table 46.3: Behavior stories

Story (sprint, epic)	Model change	Check	Artifact
BuoyModes state machine by hand (S6, E8)	`state def BuoyModes`, exhibited on `MooredBuoy`	grammar and lint	the mode list in the docs
Fork, pin and object flow in ObserveCoast (S6, E8)	`fork`, `flow` and successions written by hand	grammar, lint, IDEF0 plates	the A0 plate (Chapter 11, Seeing the model: diagrams from text (p. 158))
The DailyUpload interaction by hand (S7, E8)	the sequence as messages between occurrences	grammar and lint	none yet: views later
The EnergyBalance constraint expression (S7, E8)	the constraint's expression in v2	grammar and lint	the expression in the docs

Every behavior story here starts from a converter TODO line, because the converter does not carry v1 state machines, interactions or opaque expressions. That makes behavior the part of the port where engineering judgment is unavoidable: the v1 diagram says what the prime drew; the v2 text has to say what the system does.

46.3.4 Analysis

Table 46.4: Analysis stories

Story (sprint, epic)	Model change	Check	Artifact
Energy balance per buoy mode (S21, E15)	notional generation, storage and load per mode in `analysis/`	a job that fails when a mode does not close	the energy table per mode
Observation-to-bulletin latency chain (S23, E15)	a latency on each step from sampling to publishing	the total against TW-STK-002	the latency chain table
Telemetry trade study (S25, E15)	three variants: cellular, satellite, hybrid	the scoring computes	the trade table
Station availability allocation (S27, E15)	reliability attributes per subsystem	the roll-up computes	availability per station type

Story (sprint, epic)	Model change	Check	Artifact
Energy balance with the drop-5 reserve term (S33, E15)	the prime's new constraint, carried into the analysis	the energy check	the updated energy table

All the values in Tidewater's analyses are notional; the stories exist to show the pattern: an analysis is model content plus a check that fails when the answer is wrong, so a later change that breaks the budget is caught on its merge request, not at the next review.

46.3.5 Verification

Table 46.5: Verification stories

Story (sprint, epic)	Model change	Check	Artifact
VCRM: which requirements are verified, and how (S6, E7)	inspection verdicts entered and signed in `verification/results.csv`	`vcrm` job	the verification matrix
Verification cases for Stations to 100% coverage (S29, E15)	one verification case per Stations requirement	VCRM coverage for Stations	the verification matrix
Security verification cases in the VCRM (S31, E16)	cases for the security requirements	VCRM and gate	the verification matrix

46.3.6 Change and sustainment

Table 46.6: Change and sustainment stories

Story (sprints, epic)	Model change	Check	Artifact
Take in prime drop v1.1 ... v1.6 (S8, S14, S20, S27, S34, S38; E9, E12, E13)	the vendor import merged; hand edits redone; manual rework from the delta	every gate, plus the delta and impact report	the drop's disposition memo
Hotfix: a wrong satisfy found by the program office (S24, E14)	one satisfy corrected on `main` and `develop`	trace	the hotfix release notes
Baseline 1, 2 and 3 (S20, S30, S39)	none: a tag of what is there	the baseline audit	the release and its package
Monthly status and CCB items (every other sprint)	none: decisions recorded on issues	none	the status memo, CCB minutes

A prime drop is the clearest example of engineering hiding inside a tooling-shaped story. The import itself is a command. The work is the disposition: which of the prime's changes the government accepts, which it questions, which cost money, and what each one does to requirements and verification. That is change-board work, and it is the reason the drop stories are labelled engineering (Chapter 45, The prime's model changes: receive, assess, disposition (p. 619)).

46.4 What platform work looks like, for contrast

Table 46.7: Some of Tidewater's platform stories

Story	Why it is platform
Create the GitLab group and model project	the place the work happens
Draft the first pipeline and run it locally	a tool that checks the model
Runner online; the first pipeline gates every MR	the tool, made enforcing
TODO burn-down and generated docs in the pipeline	tools that measure and render
The model site on GitLab Pages, rebuilt nightly	a tool that shows the model to others
The trace job becomes a gate	a tool, made stricter
Release and hotfix rehearsal	practice with the release machinery
Pin the kit version	keeping the tools stable

Notice how many of them exist to make an engineering story cheaper: the trace ratchet makes "Trace completeness" measurable; Pages makes the data exchange table visible to the program office; the burn-down makes the port's progress a number.

46.5 A generic catalog for your project

The catalog below is the pattern to build your own backlog from. It follows a list of thirty-eight planned-work items collected while this book was written, from a small team modeling a different system; the list mixed engineering and platform work in roughly the proportion a team reaches after its first semester, which is itself a useful benchmark. Each line is a story title, the model change, and the check that makes it done.

46.5.1 Concept and requirements

- **Capture stakeholder needs from the concept of operations.** A requirements package per stakeholder group, each need tagged with its source document. Check: the package resolves (lint).
- **Define the top-level use cases.** A use case per mission thread, with actors and objectives. Check: a generated use-case table for the customer walkthrough.
- **Derive system requirements from the needs.** Requirements with measurable constraints and derivation links. Check: the trace fails any system requirement without a parent.
- **Add a requirement metadata schema.** Verification method, priority, owner, status as metadata. Check: a job fails a requirement missing a required field.
- **Import the legacy requirements spreadsheet.** A script that turns the CSV into requirement text, run once, kept for the next import. Check: the merge request's diff, reviewed.
- **Establish the units and quantities library.** Shared value types on ISQ quantities; requirements refactored to use them. Check: lint (no unresolved types).

46.5.2 Architecture

- **Define the logical decomposition.** Part definitions for the function blocks, composed into the system context. Check: the part tree view.
- **Model a subsystem's interfaces.** Port and interface definitions, connections, a satisfy to the interface requirement, a verification case stub. Check: trace coverage for the subsystem.
- **Allocate functions to physical components.** Allocations at the agreed level. Check: the trace flags unallocated functions.

- **Model the power subsystem and its budget.** Power draw per part and a roll-up per mode. Check: a job fails when a mode exceeds supply.
- **Model the mass properties.** Mass with margin per part, rolled up. Check: the mass report.
- **Model the data bus and its messages.** Item definitions with fields and rates. Check: the generated ICD table.
- **Model the thermal interfaces.** Heat flows and dissipation. Check: the values handed to the thermal team as a file.
- **Define an external interface.** The external system as a part, with the agreed protocol and timing. Check: the interface lead is a code owner of the package.

46.5.3 Behavior

- **Model the operating modes.** A state definition with transitions and guards, linked to the budgets per mode. Check: grammar and lint; the budgets run per mode.
- **Model the startup sequence.** An action with ordered steps and timing. Check: total startup time against its requirement.
- **Model fault detection and recovery.** Fault states and recovery actions, each linked to the hazard it mitigates. Check: a fault-to-response table for the safety review.
- **Model the end-to-end data timeline.** Latency on each step. Check: the sum against the end-to-end requirement.

46.5.4 Analysis and trades

- **Run a trade study.** Candidate variants with their attributes and a weighted scoring. Check: the trade table.
- **Hook up an external simulation.** A wrapper that reads parameters from the model, runs the simulation, writes results back. Check: the job's artifact.
- **Allocate reliability.** Failure rates and a series-parallel roll-up. Check: a job fails below the requirement.
- **Run a sensitivity analysis.** Sweep the key parameters through a budget. Check: the chart for the review package.

46.5.5 Verification

- **Write the verification cases for a subsystem.** A case per requirement with method, objective and criteria. Check: verify coverage for the subsystem reaches 100%.
- **Ingest test results.** A script that maps results to verification cases. Check: the VCRM regenerates.
- **Record analysis and inspection verdicts.** A signed results format for what is not automated. Check: the VCRM merges them.

46.5.6 Baselines and sustainment

- **Prepare a review baseline.** Close the trace gaps, defer what remains, all checks green. Check: the baseline tag and its document set.
- **Process an engineering change after a review.** The change, its impact report and the change-board approval on the merge request. Check: a minor baseline tag.

- **Branch for a delivered configuration.** A protected release branch from the qualification tag; fixes land upstream first. Check: the branch rules.
- **Model a field discrepancy and its fix.** The discrepancy linked to parts and verification cases, a regression case added. Check: the regression passes on both branches.
- **Retire obsolete requirements.** Mark them retired rather than deleting them. Check: coverage metrics exclude them.

46.5.7 And the platform items in the same list

Model style checks; generating the review document package; pinning the tool image; enabling secret and dependency scans; code owners per subsystem; an impact report posted on each merge request. Six of thirty-eight: about one in six. That is roughly the steady state to expect once the machinery exists, and a useful target when you plan the second semester.

46.6 Splitting an engineering story into tasks

A story fits in a sprint; a task fits in a week, at most five points (Chapter 13, The operating model (p. 196)). Engineering stories split along the engineering, not along the tools. "Close the Stations level tags", split by what an engineer decides, could look like this:

1. Tag the segment and the station parts (L2, L3): 1 point. The levels everyone agrees on.
2. Tag the subsystems and assemblies (L4, L5): 1 point. Where the v1 profile is most often inconsistent, and a question to the prime's modeler may be needed.
3. Tag the components (L6) and run the level check: 1 point. The leaves, and the proof.

A split along the tools ("edit the file", "run lint", "open the MR") produces tasks nobody can demo and nobody can estimate. A split along the decisions produces tasks a reviewer can check one at a time.

46.7 Applying this to your project

- **Label every story** `work::engineering`, `work::platform` or `work::learning` from the first sprint, and show the split at every review. The trend answers "when do we do engineering again?" better than any promise.
- **Expect the first two sprints to be mostly platform and learning,** and say so to management before they start.
- **Fill the backlog from the catalog,** starting with the requirements and architecture stories your converted model already demands: every TODO line is an engineering decision waiting for a story.
- **Look for the engineering inside tooling-shaped stories** (prime drops, baselines, hotfixes) and label them by their decisions, not their commands.

Exercise 46.1: Label your backlog

Take the next ten stories in your backlog (or Tidewater's sprint 7 plan in `sim/tidewater-plan.pl`) and give each one of the three labels. For every story you labelled platform, write one sentence naming the engineering story it makes cheaper. If you cannot, ask whether the

story is needed now.

Exercise 46.2: Split along the decisions

Split “Break the Stations to DataCenter cycle” into tasks of at most five points each, along the decisions an engineer makes rather than the commands they type. Compare your split with the plan’s.

CHAPTER

47

Keeping SysML v2 from becoming a spider web

From what you know →

You know

the Cameo model that grew into a spider web.

→ Here it is called

layering, ownership, complexity metrics and gates.

→ What is new

the pipeline measures the tangle on every change and refuses changes that make it worse.

Survival kit: Where this chapter fits

What you have

a v2 model that started life as a converted v1 spider web (Chapter 2, The Cameo years (p. 10)), a pipeline, and a team that now changes the model every day.

What you need

the practices that stop a text model from growing the same web, which of them Tidewater already follows, and a way to measure the web so you notice it growing.

Bob's objection in Chapter 3, The directive (p. 22) – "we already did this once, and it gave us a spider web" – deserves a better answer than "SysML v2 is different". SysML v2 text does not prevent a spider web. A text model can be just as tangled as a diagram model; it is only easier to *see* the tangle and to stop it growing. This chapter is about the seeing and the stopping.

47.1 How tangled was v1, and how tangled is v2?

The v1 model the prime delivered measures itself when the drop is generated. Run it on drop 1:

```
perl "$KIT/sim/tidewater-v1.pl" --drop 1 drop1 | grep -A3 "^spider web"
```

The v2 model at the end of sprint 6 is measured by the pipeline's `complexity` job. Run it on `develop` in Bob's clone of the simulation:

```
git checkout -q develop
perl ci/job.pl complexity | sed -n '/^cycles:/,$p'
```

Table 47.1: The spider web measured, v1 against v2. v1 numbers from the drop's own measures; v2 from the complexity job. The cycle row compares two different counts of the same tangle (Chapter 11, Seeing the model: diagrams from text (p. 158) explains the three ways to count it).

Measure	v1, drop 1 (Cameo)	v2, end of sprint 6
Package cycles	2 pairs of packages that depend on each other (DataCenter and Stations; Network and Operations)	1 tangled group (the `complexity` job's count: DataCenter and Stations)
Most relationship ends on one part definition	14 (ObservingStation)	8
Largest package fan-out	not measured in v1	3
Hand-drawn diagrams to keep current	47, some stale	none: every view is generated
Unused and duplicated blocks (the Legacy Import clutter)	4 unused, 2 duplicated	removed from the v2 model in sprint 5

Two things in that table need a word. First, one cycle is still there: Network and Operations was broken in sprint 4 (story TW-69); DataCenter and Stations was not planned for the first twelve weeks and is still on the backlog. Second, the complexity job reports a large number of *duplicates* – 29 – which is not clutter. It counts requirement names defined twice: once in the Cameo copy of the requirements the XMI carries and once in the DOORS conversion. That is a known, deliberate duplication while DOORS stays the authoritative source; the job reports it so nobody mistakes one copy for the other.

The package view draws the same web for people (Chapter 11, Seeing the model: diagrams from text (p. 158)):

The view counts dashed arrows: six in this flat-layout figure, four in the layered layout. The job counts tangled groups: one on drop 1. Use the job's number for the trend and the picture for the conversation.

47.2 Twenty practices

These are the practices that keep a model untangled, grouped as the user's reference grouped them. Each is marked *applied* (Tidewater does it), *adapted* (Tidewater does a version of it, for a stated reason) or *later* (not in the first twelve weeks, with when it would make sense).

47.2.1 Structure

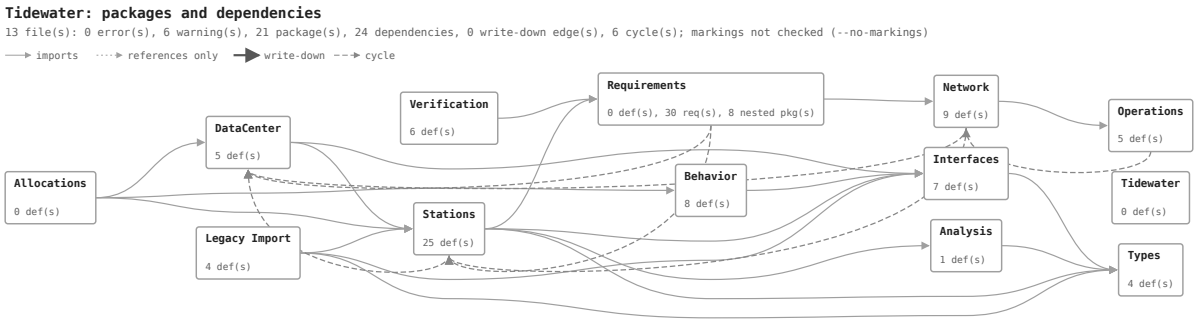


Figure 47.1: Package dependencies of drop 1 in the converter's flat layout: the dashed arrows close the cycles

Table 47.2: Structure practices.

Practice	Tidewater	How
1. Layer the model; dependencies point one way only	Applied	Folders by layer (library, requirements, logical, physical, analysis, verification) from the layout map at drop 1 (Chapter 16, The model repository: what goes where (p. 245)); the complexity job reports any upward dependency.
2. Modularize by ownership	Applied	One folder per subsystem under `physical/`; CODEOWNERS maps each folder to its owner.
3. Explicit, narrow imports	Adapted	Government-written files may not use wildcard imports (the job counts them). Files converted from the prime's drops keep the converter's wildcard imports: editing them would make every new drop conflict.
4. Federate into several projects when the model outgrows one team	Later	One three-person team, one project. Split along the layers when a second team or an outside consumer appears (Chapter 16, The model repository: what goes where (p. 245)).

47.2.2 Reuse and abstraction

Table 47.3: Reuse practices.

Practice	Tidewater	How
5. Separate definitions from usages	Applied	The converter writes part, port and interface definitions and uses them as typed usages.
6. A versioned project library	Adapted	`model/library/` holds the conventions, value types and interfaces, versioned with the model's baselines; publishing it as its own package comes with federation.

Practice	Tidewater	How
7. Interfaces as contracts at subsystem boundaries	Applied	Port and interface definitions in the library; subsystems connect through them.
8. Model variability explicitly	Later	No variants in the first twelve weeks; the drifting buoy in drop 4 arrives as a specialization of the station, and variant modeling is the planned way to handle more.

47.2.3 Relationships and traceability

Table 47.4: Traceability practices.

Practice	Tidewater	How
9. Trace at the right granularity	Adapted	ObservingStation's trace spaghetti was pruned in sprint 5; the remaining links are the prime's to rationalize, raised as a discrepancy report.
10. Declare each relationship once; generate reverse views	Applied	`satisfy` and `verify` are declared once; the trace report and the VCRM are generated (Chapter 44, Verification as a closed loop (p. 613)).
11. Allocate at agreed levels only	Applied	Functions are allocated to part definitions at one level, as the prime's model does.

47.2.4 Views, not diagrams

Table 47.5: View practices.

Practice	Tidewater	How
12. Diagrams and documents are generated views	Applied	`model.pl draw` and the model site; no diagram is drawn by hand (Chapter 11, Seeing the model: diagrams from text (p. 158)).
13. Standard viewpoints per stakeholder	Adapted	The model site for management, the VCRM for test, the data exchange table for interface owners; formal `view def` elements in the model come later.

47.2.5 Semantics and metadata

Table 47.6: Semantics practices.

Practice	Tidewater	How
14. Metadata instead of stereotype sprawl	Applied	The v1 profile's stereotypes became a handful of metadata definitions in the Conventions package.

Practice	Tidewater	How
15. Executable constraints over prose	Later	The energy-balance constraint is still a converter TODO at the end of sprint 6; writing it and evaluating it in the pipeline belong to the engineering epic after the port.

47.2.6 Process guardrails

Table 47.7: Process practices.

Practice	Tidewater	How
16. Automate complexity metrics with thresholds	Adapted	The measures are on the scorecard every sprint; the complexity job as a merge-request gate (a ratchet: fail only when a measure gets worse) is planned after sprint 6, so the first twelve weeks stayed lean (Chapter 20, The pipeline, job by job (p. 302)).
17. Small, reviewable changes	Applied	One issue, one branch, one merge request; the sprint 3 retrospective added "split MRs before they reach 300 lines".
18. Refactor continuously	Applied	Cycle breaking, the Legacy Import clean-up and the trace pruning were planned stories with points, not spare-time work.
19. Tool-neutral text as the source of truth	Applied	The `.sysml` text in git is the model; everything else is generated from it.
20. Architecture decision records	Applied	Decisions D-1 to D-10 as ADRs in `docs/adr/` from sprint 2.

47.3 Why the web grows, and how the guardrails stop it

A spider web is never designed. It grows one link at a time, each link reasonable when it is made: a trace "just to be safe", an import of a whole package to reach one definition, a copy of a block "for now". In the Cameo years the links were made in diagrams, nobody saw the whole, and nothing measured it until the model was too tangled to change.

The guardrails above work because they act at the moment a link is made. A wildcard import in a government file is counted on the merge request that adds it. An upward dependency is reported before it merges. A new cycle shows up in the complexity report the same day. None of these checks is clever; what matters is that they run on every change, so the web cannot grow unnoticed. The other half is the work: the cycle break and the clean-up stories were in the backlog with points (Chapter 46, Systems engineering work, not pipeline work (p. 636)), not left to good intentions.

Exercise 47.1: Measure a model you own

Run the complexity job (or the package view) on a model you work on. Which package has the largest fan-out? Is there a cycle? For each one, write down whether it is a real

engineering dependency or an accident of how the model was built.

Exercise 47.2: Mark the twenty practices for your project

Copy the tables above and mark each practice applied, adapted or later for your own project, with one line on why. Which three would make the biggest difference in the next sprint?

47.4 You can now

- Measure a model's tangle – cycles, fan-out, relationships per part, duplicates – and read what each number means.
- Explain why a text model can still become a spider web, and which guardrails stop it.
- Decide which of the twenty practices to apply now and which to defer, with a reason for each.

47.5 Applying this to your project

Copy the layering, the ownership folders, the generated views and the decision records from the first sprint; they cost almost nothing and are hard to add later. Adapt the import rule to how your model arrives: if most of it is converted from someone else's tool, keep their files as delivered and hold your own files to the stricter rule. Decide locally when to turn the complexity measures into a merge-request gate – after the inherited web is measured and the team has seen the numbers for a few sprints.

CHAPTER

48

Security: protect the environment, engineer it into the system

From what you know →

You know

the ISSM, the security classification guide, marked documents and the authorization package.

→ Here it is called

protecting the engineering environment, and engineering security into the system.

→ What is new

markings, secret scanning and security requirements are checked in the pipeline, not only at accreditation.

Survival kit: Two kinds of security

What you have

a GitLab project, a runner, a model, and an ISSM who has started asking questions. You also have a system to design, which will one day face people who want to misuse it.

What you need

to see that "security" means two different jobs. One is protecting the place where the engineering happens: the repository, the pipeline, the laptops, the data. The other is engineering security into the system itself: requirements, trust boundaries, threats, verification. Small teams usually do the first, because the ISSM insists, and forget the second, because nobody asks until the review where everyone asks.

48.1 Why the second half gets forgotten

On Tidewater, "security" arrived in sprint 2 as a meeting with the ISSM about the runner. For the next six weeks it meant hardening checklists, service accounts, secret scanning and a tabletop

exercise. All of that was necessary. None of it said anything about whether a spoofed buoy could feed false observations into the marine bulletin, which is the security question a mariner would ask.

That is the normal pattern. The environment half has an owner from day one (the ISSM, who must authorize the toolchain) and a deadline (no authorization, no GitLab). The system half has neither: it lives in the requirements, the architecture and the verification plan, which the program assumes "the engineers" will get to. In the document era it got done, if at all, in a security annex written near the end. In a model it can be done the way everything else is: as model content, checked by the pipeline on every change.

This chapter walks through twenty-six points, from a note Jim kept from a program security briefing, and marks each one for Tidewater as **applied** (done in the first year), **adapted** (done in a reduced or different form, with the reason), or **LATER** (planned, with the sprint or epic). Tidewater is a civilian, notional program and nothing in it is sensitive, so some points are taught here as general practice for programs that are. The table at the end (Section 48.8, The twenty-six points at a glance (p. 659)) lists all twenty-six.

48.2 Half one: protecting the engineering environment

48.2.1 Data classification and handling

1. Know what the model contains, before the first commit. Requirements, interface details and performance parameters can be controlled information, export-controlled, or classified. Decide the handling level with the security office *before* anything is committed, because a repository cannot be made "less sensitive" afterwards: its history keeps every version. *Tidewater: applied.* The decision is a decision record, "Data handling and markings", written by Jim with the ISSM in week zero and committed before the first drop was imported. It says the content is notional and unmarked, and it names the conditions that would change that (any real performance parameter, any real site).

2. Watch for aggregation. A model's purpose is to pull scattered facts into one connected, queryable whole. Individually harmless facts can become sensitive together: the list of every station, its location, its power budget and its link schedule says more than any one document did. *Tidewater: adapted.* There is no real content to aggregate, but the habit is practised: the ISSM reviews the model as a whole before each baseline is tagged, not one merge request at a time. Add that review to your release checklist; it is not in the kit's templates by default.

3. Mark at the package level, and let the pipeline enforce it. Use a metadata definition for markings, put one on every package, and let the pipeline fail a model with an unmarked package; generated documents then carry the right banner because the generator reads the marking. The kit ships this: the `SecMeta` vocabulary and `model.pl gate`, shown working in Section 48.5, A worked example: the security gate (p. 655). *Tidewater: LATER*, in the maintenance tome: the story "Markings on every package; model.pl gate in the pipeline". Until then every package is notional and unmarked by decision (the data-handling record), which is itself a marking decision on record.

4. Separate by sensitivity. When parts of a system sit at different handling levels, split them into separate repositories on networks accredited for each level. Access controls inside one repository are not a separation mechanism. *Tidewater: adapted.* One repository is enough because everything is at one level. The option is recorded in the data-handling decision: if the prime's drops

ever carry markings the team's repository is not approved for, the drops move to a separate, restricted project and are referenced by tag. Write the same clause into your model exchange agreement with the prime.

48.2.2 Platform and access

5. Self-host on an accredited network. GitLab and its runners run inside the program's enclave; shared cloud runners and software-as-a-service tools are out for controlled data unless specifically authorized. *Tidewater: applied.* The project lives on the enterprise GitLab, and the runner is a project runner on a government Windows VM (Chapter 21, Runners: where the pipeline runs (p. 347)).

6. Least privilege. Roles scoped to need: Jim and Tom are maintainers, Bob is a developer. `main` and `develop` are protected, merges need a code owner's approval, and nobody pushes to them directly. Membership is reviewed when someone joins or leaves. *Tidewater: applied* (Chapter 15, Setting up GitLab (p. 226), Chapter 42, Administration (p. 586)).

7. Strong authentication. Enterprise single sign-on with multi-factor or PKI for the web interface; SSH keys or short-lived tokens for git; no long-lived personal access tokens in scripts. *Tidewater: applied.* Project access tokens carry an expiry date and a single scope, and their rotation is a recurring task in Chapter 42, Administration (p. 586).

8. Sign commits and tags. A signed baseline tag proves who approved which configuration. That is configuration management evidence first and security second. Git can sign with an SSH key, which every laptop already has from Chapter 9, The survival kit (p. 126), so there is nothing extra to install. *Tidewater: LATER*, the story "Signed baseline tags" in the maintenance tome. The mechanics take one minute and are shown in Section 48.6, Signing a baseline tag (p. 658).

48.2.3 Pipeline and supply chain

9. Treat runners as privileged. A runner sees every file of every job it runs and every variable those jobs use. Harden it, run it as a service account with no other rights, and keep jobs that hold secrets (release, package upload, Pages) on a protected runner that only protected branches and tags can use. *Tidewater: applied* (runner hardening with the ISSM in sprint 3; the protected runner in Chapter 42, Administration (p. 586)).

10. Pin and vet the tools. Use specific versions, from internal mirrors where the network requires it, never "latest" pulled from the internet, and keep an inventory of the tools that produced each baseline. *Tidewater: adapted.* The kit is pinned by version in the model repository (`.kit-version`, checked by every job), each baseline's evidence includes `toolchain.txt` (kit version, Perl, git, operating system, runner), and the optional Docker image is pinned by digest (Chapter 21, Runners: where the pipeline runs (p. 347)). There is no internal package mirror because the kit needs no packages.

11. Manage secrets properly. Masked, protected CI/CD variables; a secret-detection job on every change; and when something leaks, rotate it at once, because deleting the commit is not enough. *Tidewater: applied.* The `secrets` job runs from sprint 3, GitLab's managed Secret Detection runs where a Docker runner exists, and the recipe is in Section E.13, "I pushed a secret" (p. 729).

12. Scan your own code. The team's tooling gets the same treatment as any software. *Tidewater: adapted.* The team writes almost no code of its own: `ci/job.pl` and the kit's tools are tested in the

kit's own pipeline, on both Perls, before a kit version is pinned. Static analysis and dependency scanning are not used, because there are no dependencies to scan (Chapter 17, The DevSecOps tools that earn their keep (p. 255) lists the features deliberately left off).

13. Retain artifacts deliberately. Keep formal evidence for the life of the program; let everything else expire, because a throwaway artifact can still contain something sensitive. *Tidewater: applied.* Baseline audits, the verification matrix at each baseline, and each prime drop as received are kept with `expire_in: never`; every other artifact uses the project's default expiry (Chapter 20, The pipeline, job by job (p. 302)).

48.2.4 AI tools

14. Keep model content out of external AI services. Only models approved for the data's handling level, which in practice means a local model on the laptop or an approved server inside the enclave, and only after the model's origin has been checked against policy. *Tidewater: applied* (no cloud AI; the local assistant goes through approval first, Chapter 18, A local AI assistant on the laptop (p. 269)).

15. Treat AI output as untrusted input. Generated text goes through the same gates and the same review as anything a person writes, and the merge request says so with the `ai::assisted` label. *Tidewater: applied.*

48.3 Half two: engineering security into the system

The second half is systems engineering, not administration. Every point below is model content: something you write in `.sysml` text, review in a merge request, and check in the pipeline. That makes it the same kind of work as a power budget or an interface, which is the argument for doing it in the model instead of an annex.

16. Security requirements are requirements. Confidentiality, integrity, availability and authentication requirements, derived from the threat analysis and the program's control baseline, with the same trace and verification coverage as any other requirement. *Tidewater: LATER.* The requirements belong to the prime's DOORS modules, so the government raises them as discrepancy reports, and they arrive as `TW-SEC-*` requirements in a later drop (the story "Security requirements TW-SEC through DRs to the prime").

17. Model threats and trust boundaries. Mark each part with the trust zone it sits in. Every interface that crosses from one zone to another is a boundary crossing, and each crossing must trace to a security requirement that covers it. Each threat in the register names the asset it attacks and the requirement that mitigates it. The kit's `model.pl gate` checks both rules (Section 48.5, A worked example: the security gate (p. 655)). *Tidewater: LATER* (stories "Trust zones on the part tree" and "A notional threat register"), and demonstrated now on a small model.

18. Model data flows with their sensitivity. Tag item definitions (the things that flow) with their handling level, and check that sensitive data never flows through an interface not rated for it. *Tidewater: LATER, not planned.* All Tidewater data is public weather data; the pattern is the package-marking pattern applied to item definitions.

19. Behavior under attack. Model what the system does when it detects an attack: detect, contain, degrade gracefully, beside the fault-recovery behavior. *Tidewater: LATER*, for the observation ingest path: what the data center does with records from a station that fails authentication.

20. Plan security verification. Verification cases for authentication, signing and access-control tests, with their results in the same verification matrix as everything else. *Tidewater: LATER* (the story "Security verification cases in the VCRM").

21. Generate certification evidence. An authorization package needs traceability from controls to design to test evidence. If the model already traces them, the pipeline produces most of that package: the gate's report, the trace report and the verification matrix at a baseline tag are evidence nobody assembles by hand. *Tidewater: adapted.* The baseline evidence includes the trace and the verification matrix; there is no authorization package for Tidewater's system because it is notional.

22. Model the supply chain. Record where components come from, so a counterfeit or provenance concern can be traced to the parts it affects. *Tidewater: applied.* The prime's model already carried a `Procured` stereotype with supplier and part number; the port turned it into `@Procured` metadata in sprint 5, so a question about a supplier is one search.

48.4 Process

23. Name a security owner. On a small team one person owns the security requirements, the threat model and the pipeline's security posture, part-time, so that "someone" means someone. *Tidewater: applied.* Tom, named in week zero, with the ISSM as the authority.

24. Bring the ISSM in before standing up the toolchain. Retrofitting an authorization onto a running pipeline is painful. *Tidewater: applied.* The ISSM was contacted in week zero, the toolchain review started in sprint 2, and the runner hardening in sprint 3 was done with the ISSM, which is also why sprint 3 had a three-day block waiting for the ISSM's calendar. That block was the cheap version of the lesson.

25. Use frameworks as checklists. The NIST SP 800-53 control baseline under the Risk Management Framework, NIST SP 800-171 and CMMC for controlled unclassified information, NIST SP 800-218 (the Secure Software Development Framework), and the DoD Enterprise DevSecOps reference design. Nobody implements all of them on day one; read them as checklists that stop you missing a whole category. *Tidewater: adapted.* They are used as checklists in the quarterly access and toolchain reviews (Chapter 42, Administration (p. 586)); this book does not restate their contents, and your ISSM will tell you which ones apply.

26. Rehearse incidents. The most likely serious incident for a modeling team is marked information pushed to the wrong place (spillage), and the response is time-critical. *Tidewater: applied.* A one-point tabletop exercise in sprint 5, Jim and Tom with the ISSM; the script is below (Section 48.7, The spillage tabletop (p. 659)).

48.5 A worked example: the security gate

This example builds a small Tidewater model in a scratch folder: a buoy in the untrusted zone, the data center in the trusted zone, a public site in between, one security requirement, two threats. It is notional and tiny, so that every line can be read. The security vocabulary comes from the kit (a copy ships with the Halberd example model); its comments are stripped on the way in.

```
mkdir -p model/library model/physical model/requirements model/verification
grep -v '^//' "$KIT/examples/halberd/model/engineering/SecMeta.sysml" >
model/library/SecMeta.sysml
printf 'title = TidewaterSecurity\nnoun_root = ObservationNetwork\n' > sysml.conf
cat > model/physical/Network.sysml <<'EOF'
package Network {
    doc /* The observation network, with its trust zones. */
    private import SecMeta::*;
    private import SecurityRequirements::*;
    @Marking { level = Level::U; }

    port def ObservationPort { doc /* Observation records out of a station. */ }

    part def ObservationNetwork {
        doc /* Stations in the field, the data center, and the public site. */
        part buoy : MooredBuoy;
        part dataCenter : DataCenter;
        part publicSite : PublicSite;
        interface ifcObservations { end ::> buoy.obsOut; end ::> dataCenter.obsIn; attribute
        fromZone = "untrusted"; attribute toZone = "trusted"; attribute secReq = "secR01"; }
        interface ifcPublication { end ::> dataCenter.publicOut; end ::> publicSite.feed;
        attribute fromZone = "trusted"; attribute toZone = "dmz"; attribute secReq = ""; }
        satisfy secR01 by dataCenter;
    }
    part def MooredBuoy { @TrustZone { zone = Zone::untrusted; } doc /* A buoy at sea: anyone in
    radio range can reach it. */ port obsOut : ObservationPort; }
    part def DataCenter { @TrustZone { zone = Zone::trusted; } doc /* Ingest, quality control,
    archive. */ port obsIn : ~ObservationPort; port publicOut : ObservationPort; }
    part def PublicSite { @TrustZone { zone = Zone::dmz; } doc /* The public bulletin site. */
    port feed : ~ObservationPort; }
}
EOF
cat > model/requirements/SecurityRequirements.sysml <<'EOF'
package SecurityRequirements {
    doc /* Notional security requirements for Tidewater. */
    private import SecMeta::*;
    @Marking { level = Level::U; }
    requirement secR01 { attribute doorsId = "TW-SEC-001"; @SecurityRequirement; @Mitigates {
    threat = "spoofedObservation"; } doc /* The data center shall accept observation records only
    from authenticated stations. */ }
}
EOF
cat > model/requirements/Threats.sysml <<'EOF'
package Threats {
    doc /* The notional threat register. */
    private import SecMeta::*;
    @Marking { level = Level::U; }
    metadata spoofedObservation : Threat { asset = "observation link"; mitigation = "secR01";
    stride = "Spoofing"; capec = "CAPEC-148"; }
    metadata tamperedBulletin : Threat { asset = "public bulletin"; mitigation = ""; stride =
    "Tampering"; capec = "CAPEC-272"; }
}
EOF
cat > model/verification/SecurityVerification.sysml <<'EOF'
package SecurityVerification {
    doc /* How the security requirements are verified. */
    private import SecMeta::*;
```

```
    private import SecurityRequirements::*;
    private import Network::*;
    @Marking { level = Level::U; }
    verification def TestStationAuthentication {
        doc /* Send a record from an unregistered station and confirm the data center rejects it.
        */
        subject network : ObservationNetwork;
        objective { verify secR01; }
    }
}
EOF
perl "$KIT/tools/sysml/model.pl" lint | tail -1
perl "$KIT/tools/sysml/views/sysml-pkg-svg.pl" model -o packages.svg 2>&1 | tail -1
```

Three things in that text are security engineering, not decoration:

- `@TrustZone { zone = Zone::untrusted; }` on `MooredBuoy` says where the part sits. A buoy at sea is untrusted: anyone with a radio in range can talk to it.
- The two `interface` lines are the **boundary crossings**. Each names the zone at both ends and the security requirement that covers it (`secReq`). The publication link to the public site has none yet.
- The threat register has two entries, each with its STRIDE category and CAPEC identifier; one is mitigated by `secR01`, the other by nothing yet.

Now the gate. It runs five checks: the text, the trace, the markings, the zones and the threats, and it fails if any of them finds an error.

```
perl "$KIT/tools/sysml/model.pl" gate || echo "exit status $?"
```

Two findings, both real security work, neither a typing mistake:

- the publication link crosses from the trusted data center into the semi-trusted public zone with no requirement covering it;
- the bulletin-tampering threat has no mitigation.

The fix is engineering: a requirement that the data center signs every bulletin it publishes, which covers the crossing and mitigates the threat at once, satisfied by the data center and verified by an inspection.

```
sed -i 's/attribute secReq = ""; }/attribute secReq = "secR02"; }/' model/physical/Network.sysml
sed -i 's/satisfy secR01 by dataCenter;/satisfy secR01 by dataCenter;\n        satisfy secR02 by
dataCenter;/' model/physical/Network.sysml
sed -i 's/mitigation = ""; stride = "Tampering"/mitigation = "secR02"; stride = "Tampering"/'
model/requirements/Threats.sysml
sed -i 's#^}$#    requirement secR02 { attribute doorsId = "TW-SEC-002"; @SecurityRequirement;
@Mitigates { threat = "tamperedBulletin"; } doc /* The data center shall sign every bulletin it
publishes. */ }\n}#' model/requirements/SecurityRequirements.sysml
```

```
sed -i 's#objective { verify secR01; }#objective { verify secR01; }\n    }\n    verification def
InspectBulletinSignature {\n        doc /* Inspect a published bulletin and check its signature.
*/\n        subject network : ObservationNetwork;\n        objective { verify secR02; }#'
model/verification/SecurityVerification.sysml
perl "$KIT/tools/sysml/model.pl" gate | tail -1
```

(In real work you would make these edits in Vim, not with `sed`; the commands are here so the book can be tested.)

Finally, the marking rule. Remove the marking from one package and the gate stops it, which is point 3 working:

```
sed -i '/@Marking/d' model/requirements/Threats.sysml
perl "$KIT/tools/sysml/model.pl" gate || echo "exit status $?"
```

`model.pl gate` also draws what it checked, if you ask: `--svg DIR` writes the package view with each package's marking and the interconnection view with the zones shaded, which is the picture the ISSM wants at a review (Chapter 11, Seeing the model: diagrams from text (p. 158)). The kit's Halberd example model (`examples/halberd`) carries the same vocabulary at full size, with planted gaps, if you want a larger model to read.

48.6 Signing a baseline tag

A signed tag records, cryptographically, who created it. Git can sign with an SSH key, so the key you already use for GitLab can sign baselines. This example uses a fresh key in a scratch repository; on the project, the signing key and the list of allowed signers are set up once by the security owner.

```
git init -q -b main signing-demo
cd signing-demo
echo "baseline content" > model.txt
git add model.txt
git commit -q -m "The configuration to baseline"
ssh-keygen -q -t ed25519 -N '' -C tom@example.invalid -f ../tom-signing
git config gpg.format ssh
git config user.signingkey ../tom-signing.pub
git tag -s baseline-0 -m "baseline-0: interim baseline for the program review"
echo "tom@example.invalid $(cat ../tom-signing.pub)" > ../allowed_signers
git config gpg.ssh.allowedSignersFile ../allowed_signers
git tag -v baseline-0
```

GitLab shows a **Verified** badge on a signed tag or commit when the signing key is registered on the signer's profile. The allowed-signers file is what lets anyone check a signature offline, years later, without GitLab.

48.7 The spillage tabletop

A tabletop exercise is a meeting in which you walk through an incident that has not happened, out loud, with the people who would handle it. Tidewater's took one point of Jim's and Tom's time in sprint 5, with the ISSM. The scenario:

> *Thursday, 14:10. Bob notices that a file pushed to Bob's feature branch an hour ago contains a paragraph pasted from a marked document. The merge request is open; the pipeline ran; GitLab sent notification e-mails to Jim and Tom.*

The questions the ISSM asked, and the answers the team wrote down:

1. **What does Bob do first?** Stops. Does not delete the branch, does not force-push, does not edit the file "to make it go away": all of those spread or destroy evidence. Calls Tom (security owner) and the ISSM, in that order, by phone.
2. **Who else has a copy?** Everyone who fetched since 13:10, the runner's build directory, the pipeline's artifacts and logs, the notification e-mails if they quoted the text. Tom lists them from GitLab's audit events and the job list.
3. **How is it contained?** The ISSM directs it. A maintainer can make the project private to the team and pause the runner; nobody rewrites history until the ISSM says how, because the program's procedure may require the systems to be handled as spilled equipment.
4. **What does the team learn?** The source of the paste, and whether a check could have caught it. On Tidewater the answer was the markings rule (point 3), moved earlier in the plan as a result.

The written answers became a page in the wiki, linked from the onboarding page, and the first step of the rescue recipe for a leaked secret became "call the security owner" (Section E.13, "I pushed a secret" (p. 729)).

48.8 The twenty-six points at a glance

Table 48.1: The twenty-six security points for Tidewater

#	Point	Tidewater	Where
1	Know what the model contains before the first commit	applied	data-handling decision record, week zero
2	Watch for aggregation	adapted	ISSM review at each baseline
3	Mark packages; the pipeline enforces	LATER	maintenance tome, `model.pl gate` in the pipeline
4	Separate repositories by sensitivity	adapted	one level; restricted drops project as the option
5	Self-host on an accredited network	applied	enterprise GitLab, government runner
6	Least privilege	applied	roles, protected branches, code owners
7	Strong authentication	applied	single sign-on, SSH keys, expiring tokens
8	Sign commits and tags	LATER	"Signed baseline tags" story
9	Treat runners as privileged	applied	hardening in sprint 3; protected runner
10	Pin and vet tools	adapted	`.kit-version`, `toolchain.txt`, image by digest

#	Point	Tidewater	Where
11	Manage secrets	applied	`secrets` job, masked variables, rotate first
12	Scan your own code	adapted	the kit's own tests; nothing else to scan
13	Retain artifacts deliberately	applied	`expire_in: never` for formal evidence only
14	No model content in external AI	applied	local model, after approval
15	AI output is untrusted input	applied	same gates, `ai::assisted` label
16	Security requirements are requirements	LATER	TW-SEC via discrepancy reports
17	Threats and trust boundaries	LATER	zones and threat register stories; shown above
18	Data flows with sensitivity	LATER	not planned: public data
19	Behavior under attack	LATER	the ingest path
20	Security verification	LATER	security cases in the VCRM
21	Certification evidence from the trace	adapted	baseline evidence, no package for a notional system
22	Supply chain of components	applied	`@Procured` metadata, sprint 5
23	A named security owner	applied	Tom
24	ISSM before the toolchain	applied	week zero, sprints 2 and 3
25	Frameworks as checklists	adapted	quarterly reviews
26	Rehearse incidents	applied	spillage tabletop, sprint 5

Most of the "applied" marks are in the first half and most of the "LATER" marks in the second: that is the pattern this chapter opened with, and Tidewater did not escape it either. The difference is that the second half is in the backlog with stories and sprints, not in a sentence that says "security will be addressed".

48.9 Applying this to your project

- **Copy:** the data-handling decision record before the first commit; the security owner by name; the ISSM in week zero; the `secrets` job and the rotate-first rule; the spillage tabletop; the twenty-six-point table, filled in for your program and reviewed each quarter.
- **Adapt:** the marking levels in `SecMeta` (Tidewater's are notional labels), the trust zones (three is a starting point, not a rule), and which framework your ISSM uses as the checklist.
- **Decide locally, early:** whether your data needs separate repositories (point 4) and whether the program's GitLab tier gives you approval rules and push rules, or whether those controls are procedural (Chapter 17, The DevSecOps tools that earn their keep (p. 255)).
- **Put the second half in the backlog** with real stories, owned by a systems engineer, measured by the gate. If the system half has no stories, it is not planned; it is hoped for.

Exercise 48.1: Close a boundary crossing

In the worked example's folder, add a fourth part, `MaintenanceLaptop`, in the untrusted zone, with an interface to the data center that has an empty `secReq`. Run `model.pl gate` and read the finding. Then write the security requirement that covers it, satisfy it, verify it, and run the gate until it says PASS. What did you have to decide that was engineering, not typing?

Exercise 48.2: Your twenty-six

Copy the table above into your project's wiki and mark each point for your program. For every LATER, name the story or epic; for every adapted, write the reason in one sentence. Which three points would your ISSM ask about first?

CHAPTER

49

Scaling up: segments, blocks and a ladder of fidelity

From what you know →

You know

big programs. Several major systems built by different organizations, each with its own schedule; interface control documents signed by both sides; hardware that takes a year to qualify; a test range booked months ahead; a program schedule that lines all of it up at the reviews.

→ Here it is called

a system of systems, run as segments with their own cadences, synchronized at integrated baselines. Hardware moves in blocks, each a full Vee; software moves in sprints underneath; verification climbs a ladder of fidelity from model checks to sea trials; the model in GitLab holds every interface.

→ What is new

one model and one pipeline tie the segments together, so an interface change on one side shows its effect on the other the same day, and the baselines that line the segments up are tags a machine can audit.

This chapter is LATER. Jim, Bob and Tom are a three-person government team porting one system model; in their first twelve weeks nothing here is their job. Read it when the program around your model is larger than your team: several segments, hardware with long lead times, a test campaign, a simulation that has to earn credit. It explains how the practices of this book stretch to that size, and where they need help from other disciplines.

Note: Where this chapter comes from

Adapted from a reference sketch the author uses for a weapon-system program, rewritten for this book's civilian example. The process lessons carry over unchanged; the weapon-specific content does not, and none of it appears here.

49.1 A system of systems

Tidewater is small enough to describe in a paragraph, and it is already a system of systems. Moored buoys and shore stations observe; telemetry units carry the observations ashore; the data center ingests, checks, archives and publishes the marine bulletin; the field crew keeps the stations running. Each of those is a major system in its own right, built and maintained by different people, and the places where they meet – the observation interface, the health interface, the public interface – are where integration problems show up.

The three kinds of segment move at different speeds, and pretending otherwise forces one of them to work at the wrong pace.

Table 49.1: Tidewater's segments and their cadences.

Segment	What it is mostly	Natural cadence	What sets the pace
Stations and buoys	hardware: hulls, moorings, sensors, power	months to a year per change	long-lead parts, environmental testing, a field season to install
Telemetry	mixed: radios plus firmware	a few months per radio change, weeks per firmware change	radio approvals, field updates of firmware
Data center	software: ingest, quality control, archive, publication	every sprint	the pipeline; nothing physical has to move

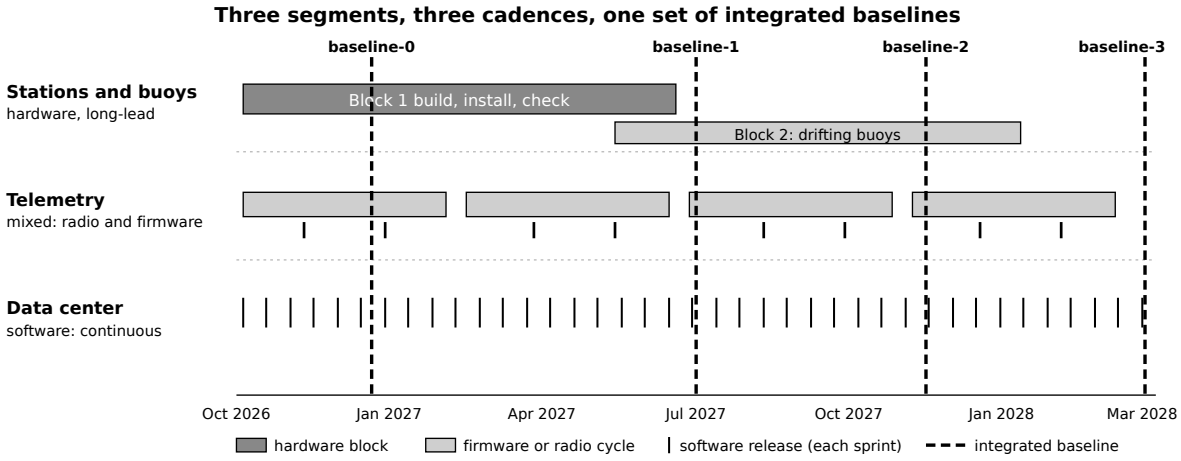


Figure 49.1: Three segments, three cadences. The stations move in hardware blocks, telemetry in radio and firmware cycles, the data center in sprints. The dashed lines are the integrated baselines where all of them are frozen and tested together.

49.1.1 Interfaces are owned at the system level

Interface management is the systems-engineering job at this size. Each segment owns its own part of the model, under its own code owners (Chapter 16, The model repository: what goes where (p. 245)); the interfaces between segments are owned at the system level, by the people who answer for the whole. In the model, an interface is a port definition and the items that flow through it. In Tidewater they live in the library layer, where every segment can use them and none can change them alone:

```
. ../env.sh > /dev/null
grep -n "port def\|item def" model/library/Interfaces.sysml
```

Each line is a row of what the document era called an interface control document. The observation record, the health report and the bulletin are the messages; the port definitions are the connectors; the data exchange table the team generated in sprint 6 (Chapter 31, Sprint 6: Behavior by hand, a table for the program, and baseline-0 (p. 495)) is the ICD's table, produced from these lines rather than typed. When a segment changes its side of an interface, the change is a merge request against the library, the system-level code owners must approve it, and every segment that uses the interface sees the effect in its next pipeline.

The rule that matters most: **one copy of each interface**. The most common failure at this size is two segments each keeping their own copy of an ICD and editing it separately. With the interface in the model, there is nothing to copy.

49.1.2 An integration lab with stand-ins

Segments should integrate long before real hardware meets in the field. A system integration lab does that with stand-ins: a buoy emulator that sends observation records over the real interface to the real ingest service, a telemetry emulator that drops and delays messages the way a bad link does, a test copy of the data center. The stand-ins are built from the interface definitions in the model, so they speak exactly what the real segments speak. Integration problems then appear on a bench in the first months, not on a pier in the second year.

49.2 Integrated baselines

The segments synchronize at **integrated baselines**: defined points where every segment's configuration is frozen, put together and tested end to end. Tidewater's baselines are exactly that, at the scale of one model; a larger program does the same across segments.

1. **Freeze.** In the last sprint of the period, each segment tags its configuration: the hardware build state, the software build, the firmware, and the model version. In GitLab each segment's repository gets its tag, and the system model gets a release branch (Chapter 43, Maintenance through the life cycle (p. 602)).
2. **Integrate.** The frozen configurations are loaded together in the integration lab, with stand-ins for anything not yet built.
3. **Test end to end.** Scripted scenarios run through the whole system: a buoy observes, the observation crosses the link, the data center ingests it, checks it, archives it and publishes the bulletin.
4. **Record.** Results go into the verification matrix (Chapter 44, Verification as a closed loop (p. 613)) and calibrate the simulation; every discrepancy becomes a story in the next period.
5. **Tag.** The combined configuration becomes one system baseline: a protected tag, audited by the pipeline, the reference for the period's review (Chapter 4, The Vee, kept: the systems-engineering V in MBSE DevSecOps (p. 34)).

49.3 The work plan, shared across segments

Every segment plans with the same four levels and the same timeboxes, so a station story and a data-center story mean the same size of commitment, and every segment's planning periods end on the same baseline.

The reference this chapter is adapted from names the levels slightly differently from this book. It calls the thirteen-sprint planning period an *epoch* and the multi-year capability block a *tome*. This book uses **epic** for work that fits in a semester (thirteen sprints) and **tome** for work that fits in a year (Chapter 13, The operating model (p. 196)). The mapping is simple:

Table 49.2: The levels of the work plan in this book and in the reference.

This book	The reference	Timebox	What "done" means at this size
Task	Task	under a week	merged, reviewed, the pipeline green
Story	Story	one sprint	acceptance criteria met and demonstrated
Epic	Epoch	up to thirteen sprints	every segment frozen, integrated and tested end to end: an integrated baseline
Tome	(part of a block)	up to a year	a capability verified
Project	Tome (a block)	years	a capability block fielded

Four habits make the shared plan work across hardware and software.

- **Reviews land on epic boundaries.** A hardware Vee spans several epics, and each of its reviews is scheduled as the closing event of an epic. Integration and the review become the same milestone.
- **Waiting is a dependency, not a big task.** Long-lead parts and environmental tests still break into tasks under a week: preparation, execution, data reduction, model calibration. The months of waiting for a hull or a test slot are tracked as a dependency between stories, not hidden inside an oversized task.
- **Field events close an epic.** Harbor tests and sea trials are scheduled as the closing stories of an epic, so their results calibrate the simulation at the start of the next one.
- **The last sprint hardens.** The thirteenth sprint of an epic is for integration fixes, the baseline freeze and the documents generated from the model, not for new features.

49.4 The spine: requirements, functions, logical, physical

At this size the model is organized as a chain often called **RFLP**: requirements, functions, logical architecture, physical implementation. Each layer traces to the one before it, so a requirement change shows its downstream impact without anyone searching for it. Tidewater's layered folders follow the same chain: requirements, logical (behavior), physical (structure), with the library and the verification layer around them (Chapter 16, The model repository: what goes where (p. 245)).

The physical layer becomes the product tree, one branch per segment. Decompose each branch until every leaf is a **configuration item**: something procured, built or written as a unit, with its own specification, its own verification and its own place in configuration management. Deeper than that belongs in the segment's own design; higher than that leaves requirements with nobody clearly answerable for them. The tree for Tidewater's stations:

```
perl "$KIT/tools/sysml/model.pl" nouns --plain | head -22
```

Each leaf – an anemometer, a modem, a charge regulator – is a candidate configuration item. Whether it is one is a program decision (is it bought as a unit, specified and verified as a unit?), recorded in the model with the procured metadata the team added in sprint 4 (Chapter 29, Sprint 4: the model site, a cycle broken, and the demo that landed (p. 459)).

49.4.1 Views worth generating

Programs that work under an architecture framework (DoDAF, or its successor the Unified Architecture Framework) review a standard set of views. Generate them from the model rather than drawing them; a drawn view is out of date at the next change (Chapter 11, Seeing the model: diagrams from text (p. 158)). For a civilian observation network the useful ones read like this:

Table 49.3: Architecture views generated from the model, with Tidewater's content.

View	What it shows for Tidewater
OV-1	the operational concept: buoys and shore stations along the coast, the link ashore, the data center, the forecasters and mariners who use the bulletin
OV-2, SV-1	who exchanges what with whom: station to data center (observations, health), data center to the public (bulletin, latest observations)
OV-5b	the observation cycle as activities: sample, transmit, ingest, check, archive, publish
SV-4	which part performs which function: the allocations in the model
SV-6	each message's content, rate and latency budget: the data exchange table
SV-7	performance measures: observation latency, data availability, station uptime

49.5 Hardware: blocks, not one big Vee

Each hardware-dominant segment follows a formal Vee with real review gates (Chapter 4, The Vee, kept: the systems-engineering V in MBSE DevSecOps (p. 34)), and the system runs an overarching Vee above them. A single Vee that takes years delivers nothing until its end. Instead the program fields a first capability as **Block 1**, then runs overlapping Vees for the next blocks. Each block is a complete, verified, fieldable baseline; in the work plan, each block is a tome. Sprints run continuously underneath.

Tidewater's blocks are not invented for this chapter: they are in the prime's drops. Drop 4 brings the drifting buoy as a change request, and the change report shows the new block arriving in the model:

```
perl "$KIT/sim/tidewater-v1.pl" --drop 3 d3 > /dev/null
perl "$KIT/sim/tidewater-v1.pl" --drop 4 d4 > /dev/null
grep -i "drift" d4/CHANGES.csv | cut -d, -f1-3
```

```
perl "$KIT/bin/xmi2sysml.pl" --delta d3/tidewater.xmi d4/tidewater.xmi | grep "DriftingBuoy" |
head -3
```

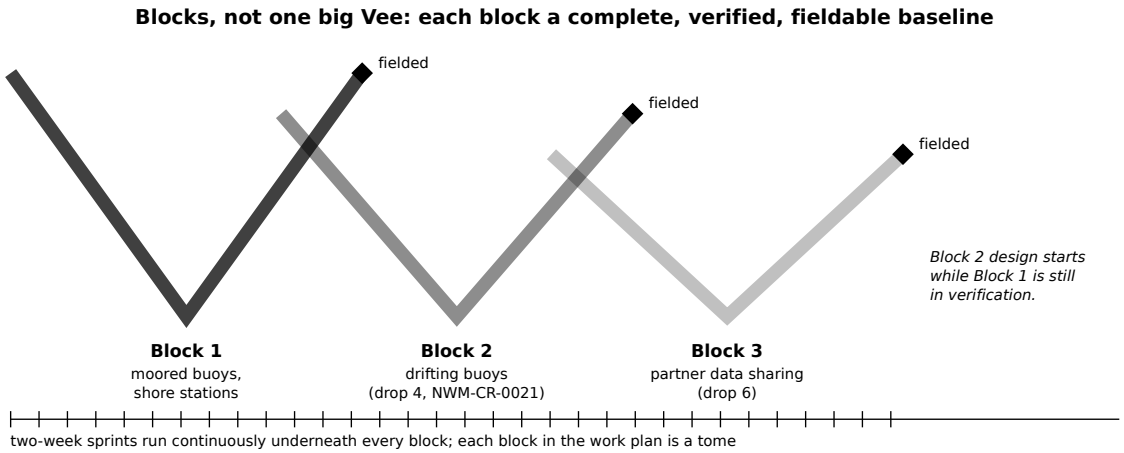


Figure 49.2: Overlapping blocks. Block 2 (drifting buoys) starts its design while Block 1 (moored buoys and shore stations) is still in verification; Block 3 (partner data sharing) follows. Sprints run underneath all of them.

49.5.1 What each review must show

Each step of a hardware Vee is gated by a review, and at this size the evidence for every review comes from the model and the pipeline, not from a document set assembled for the occasion.

Table 49.4: Reviews and the evidence each takes from the model.

Review	The question it answers	Evidence from the model
SRR	Do we understand the need?	requirements baselined, traced to needs, a verification method on each
SFR	Does the functional design meet the requirements?	functions allocated to segments; latency and availability budgets closed
PDR	Can the preliminary design meet its allocations?	the allocated baseline, interface definitions at version 1, simulation predictions against the measures
CDR	Is the design ready to build?	the product baseline, message-level interfaces, verification procedures
TRR	Are we ready to test?	test cases, the lab configuration, simulations accredited for the event
SVR, FCA	Does the system meet its requirements?	the verification matrix closed, deviations dispositioned

49.6 Software: sprints with a pipeline

The data center is software: ingest, quality control, archive, publication. So is the firmware in the loggers and telemetry units. All of it is developed in two-week sprints, rolled up into the same epics as the hardware.

The pipeline is what makes sprinting safe on software that runs in the field. The practices that carry over from safety-critical programs, scaled to an observation network:

- **A coding standard enforced by the pipeline,** with each deviation recorded and approved rather than silently allowed.

- **Structural test coverage scaled to criticality.** Firmware that can strand a buoy needs more than the archive's reporting scripts.
- **Reproducible builds.** The exact build that passed the tests, identified by its hash, is the one loaded in the field – the same idea as the baseline's checksum (Chapter 4, The Vee, kept: the systems-engineering V in MBSE DevSecOps (p. 34)).
- **Tests tagged with requirement ids,** so pipeline results update the verification matrix automatically.
- **Interface code generated from the model** where the toolchain allows, so message definitions cannot drift from the interface definitions.
- **Software loads under configuration control,** the same as hardware configuration items.

Security belongs in the pipeline from the first sprint (Chapter 48, Security: protect the environment, engineer it into the system (p. 651)). The data center is networked and public-facing, so it carries the largest attack surface, and it is the strongest candidate for a **continuous authorization**: the pipeline's evidence keeps the authorization current instead of re-certifying each release.

49.7 Verification: a ladder of fidelity

Sea trials are expensive, rare and at the mercy of the weather; a storm cannot be scheduled. Most verification evidence therefore comes from lower rungs, with each rung anchoring the one below it.

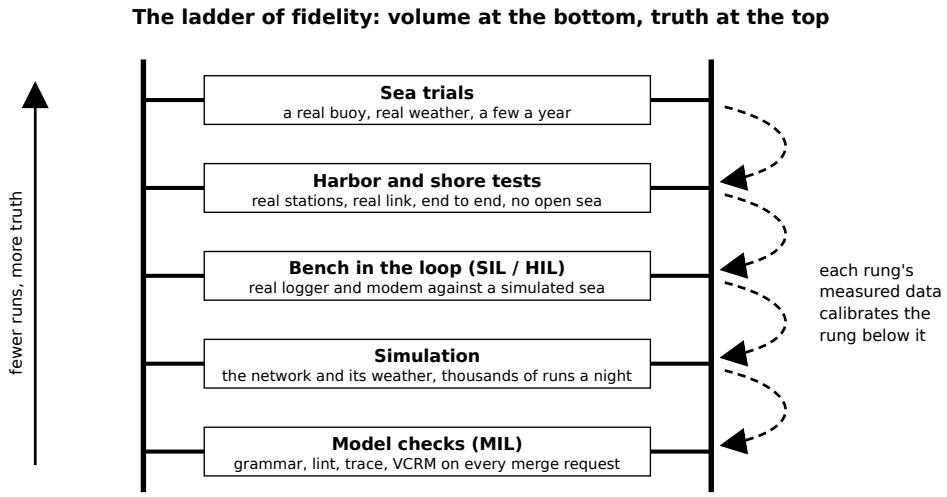


Figure 49.3: The ladder of fidelity. Model checks run on every change; sea trials happen a few times a year. Each rung's measured data calibrates the rung below.

- **Model checks** run on every merge request: grammar, lint, trace, the verification matrix (Chapter 20, The pipeline, job by job (p. 302)).
- **Simulation** models the network and its weather: station failures, link drop-outs, storms, the data center's load. This is where availability and latency statistics come from, thousands of runs a night.

- **The bench in the loop** runs real equipment against a simulated sea: a real data logger and modem fed by a scripted sensor stream, the real ingest service fed by a buoy emulator.
- **Harbor and shore tests** connect real stations, the real link and the real data center end to end, in sheltered water.
- **Sea trials** run a real buoy in real weather. They are the anchor points, and each is designed as much to validate the simulation as to demonstrate the system.

The pipeline template already carries the two middle rungs, switched off until a program has a simulation and a bench (Chapter 44, Verification as a closed loop (p. 613)). Switch them on in the question and the nightly schedule shows them:

```
. ../env.sh > /dev/null
perl "$KIT/bin/gitlab.pl" ci --list --event schedule --branch develop --var SIL_ENABLED=true
--var HIL_ENABLED=true | grep -v skipped
```

`sil-regression` runs every night on its own; `hil-test` waits, `manual`, for a person who has checked the bench.

49.8 The digital twin is a calibrated simulation

For a system like Tidewater the digital twin (Section 4.13, Digital thread and digital twin (p. 41)) is, in practice, the network simulation kept honest by field data. The station models are calibrated against recorded observations, the link model against measured drop-outs and delays, the data center model against measured processing times. Every test result flows back to calibrate the models, and the calibrated models flow back into the system model's budgets: the latency and availability numbers the requirements promise.

Warning: Accredit the simulation from the start

A simulation that has not been accredited for a specific use cannot earn verification credit, and accreditation late in a program is slow and painful. Plan the verification, validation and accreditation of the simulation itself from the first epic, as stories with owners. MIL-STD-3022 gives documentation templates for it.

49.9 Standards to plan around

These are the general standards a program of this size usually has to plan around. Which ones apply, and which revision, is set by your contract: check the current revisions and the contract before you plan.

Table 49.5: Standards to plan around. Check current revisions and your contract.

Standard	Covers
ISO/IEC/IEEE 15288	system life-cycle processes; the backbone of the Vee
DoDI 5000.97	digital engineering expectations on defense programs
DoDAF, UAF	architecture views generated from the model
MIL-STD-882E	system safety: hazard analysis and risk acceptance

Standard	Covers
MIL-STD-810	environmental testing: vibration, shock, thermal, salt fog
MIL-STD-461, MIL-STD-464	electromagnetic interference and compatibility, equipment and system level
MIL-STD-1472	human engineering for consoles and crew stations
MIL-STD-3022	documentation for model and simulation verification, validation and accreditation
RMF (DoDI 8510.01)	cybersecurity authorization

A civilian program would swap some of these for its own sector's standards; the planning habit is the same: list them in the first epic, give each an owner, and trace the requirements they impose into the model like any other requirement.

49.10 Where it breaks: the handoffs

Large programs rarely fail inside a segment. They fail at the handoffs between segments, and each failure point is a schedule risk worth tracking explicitly.

- **Interface copies drift.** Each segment edits its own copy of an ICD. Keep one copy, in the model, owned at the system level.
- **Software baselines out of step with hardware reviews.** The software that was reviewed is not the software that was integrated. Freeze both at the same integrated baseline, by tag.
- **Documents edited by hand.** Once someone edits a generated document, the model stops being authoritative. The baseline audit catches it (Chapter 4, The Vee, kept: the systems-engineering V in MBSE DevSecOps (p. 34)); the habit prevents it.
- **A simulation that never gets accredited.** All the verification credit it was supposed to earn evaporates at the end. Start accreditation in the first epic.

49.11 Exercises

Exercise 49.1: Name the cadences

List the segments of your own program (or of a program you know). For each, write its dominant kind (hardware, software, mixed), its natural change cadence, and what sets that pace. Then mark where the integrated baselines would have to fall for all of them to be frozen together at least twice a year.

Exercise 49.2: One copy of the interface

In the practice project from lesson 11, find the observation record and every place that uses it (`grep -rn ObservationRecord model/`). Which code owners would have to approve a change to it, and why should a segment not be able to change it alone?

Exercise 49.3: Find the next block in the drops

Using the drops (`perl "$KIT/sim/tidewater-v1.pl" --drop N DIR`), find which drop introduces partner data sharing, its change-request id, and the requirement it adds. Where in the block picture does it fall?

Exercise 49.4: Put a check on its rung

For each of these, name the rung of the ladder where it belongs: a missing `verify` relationship; a link model that drops one message in twenty; a buoy that loses power in a storm; a bulletin published late because the archive was slow; a modem that resets when its firmware is updated over the air.

49.12 You can now

- Explain why a system of systems needs several cadences and how integrated baselines line them up.
- Keep interfaces as one copy in the model, owned at the system level.
- Map the reference's epochs and blocks onto this book's epics and tomes.
- Decompose a product tree to configuration items and name the views worth generating.
- Plan hardware as overlapping blocks and verification as a ladder whose rungs calibrate each other.
- Name the handoffs where large programs break.

49.13 Applying this to your project

Start with the inventory, not the tools: list your segments, their cadences and their interfaces, and put every interface in the model as one copy with system-level owners before anything else. Then pick the integrated baselines from the slowest segment's natural rhythm, map your planning periods onto epics that end on them, and switch on the pipeline's simulation and bench jobs only when there is a simulation and a bench to run. Plan the simulation's accreditation in the same first epic as the standards list.

CHAPTER

50

Your project in twelve weeks: a playbook

From what you know →

You know

the program plan you write at the start of a new effort.

→ Here it is called

a twelve-week playbook for your own project.

→ What is new

a plan you can copy, already tested on Tidewater, with the decisions you must make locally marked.

Survival kit: Your turn

What you have

this book's account of what three engineers did with Tidewater, the kit, and a program of your own with a model in Cameo (or in documents), a supplier who keeps changing it, and a manager who wants to see progress.

What you need

a plan for your own first twelve weeks that keeps what worked on Tidewater, drops what was specific to it, and starts on Monday. This chapter is that plan, with the decisions you have to make locally and the commands that set it up.

Tidewater is a worked example, not a template to copy word for word. Your program has a different system, a different supplier, a different network, different people. What carries over is the shape: a week of setup, two sprints of building the shop while learning to use it, four sprints in which the engineering takes over, and a briefing at the end built from measurements. This chapter gives that shape in generic terms, marks every decision that is yours to make, and shows the kit's path through week zero on a project that is not Tidewater.

50.1 The shape of the twelve weeks

Table 50.1: A generic twelve-week plan, from week zero to the first baseline

When	Goal	Typical stories	Done when
Week zero	Agreements and access	Scorecard agreed with management; supplier exchange agreement; ISSM contact; software requests in; laptops checked	Every person has run `kickoff.pl doctor`; the scorecard page is signed
Sprint 1 (weeks 1-2)	A project the team can work in; the first drop received and assessed	GitLab project from the project file; first pipeline drafted and run locally; receive and check the first drop; TODO inventory; working agreements; practice lessons 10-11	Direct pushes to the integration branch are refused; the drop's checksums verified; the backlog estimated
Sprint 2 (weeks 3-4)	The drop on the integration branch through the vendor branch; every change gated	Runner online; pipeline gates every merge request; vendor import of drop 1; conventions; decision records; laptops and Vim; request for enablers	Every merge is green and approved by someone else; the TODO count is the baseline
Sprint 3 (weeks 5-6)	First packages closed; requirements in the model; the trace on every change	Close the first package's TODOs; units and value types; DOORS into the model; trace and secrets jobs	The trace report runs on every merge request
Sprint 4 (weeks 7-8)	The largest package closed; gaps reported to the supplier; something to show management	The biggest structure package; dependency cycles broken; discrepancy reports; model site on Pages; nightly pipeline	Management has seen the model site
Sprint 5 (weeks 9-10)	The remaining structure closed; clutter removed; trace becomes a gate	Remaining packages; duplicate and legacy clutter; the trace ratchet; requirements graded with the recipe; release rehearsal	No merge can add a trace gap
Sprint 6 (weeks 11-12)	Behavior started; an interim baseline; the briefing	Hand-written state machines and activity flows; a data exchange table; the verification matrix; baseline-0; the briefing and memo	The baseline tag rebuilds; the briefing is given from measured numbers

Two features of this plan are not negotiable, in the sense that every team that skipped them regretted it:

- **Something visible for management by sprint 4.** The team that shows nothing until week twelve spends weeks nine to twelve answering "is this working?" instead of working. On Tidewater the model site on GitLab Pages, shown at the sprint 4 review, did more for the program's confidence than any status slide.
- **Engineering takes over the board from sprint 3.** If platform work is still most of the points in sprint 4, something is wrong with the plan, not with the team: the pipeline is gold-plated, or the setup is blocked, or the learning budget was not taken. Look at the split (Section 50.8, What to measure (p. 679)) every sprint.

50.2 Week zero, with the kit

Week zero is the week before sprint 1. It is mostly conversations: the scorecard with management (see Section 3.6, Defining measurable progress (p. 28)), the exchange agreement with the supplier (Section 45.1.1, The model exchange agreement (p. 621)), the first meeting with the information system security manager (Chapter 48, Security: protect the environment, engineer it into the system (p. 651)), the software requests for every laptop. The kit's part is short. On each laptop, the doctor checks that everything the team needs is in place and says exactly what to fix:

```
perl "$KIT/tools/gitlab/kickoff.pl" doctor --quick || echo "exit status $?"
```

Expect warnings the first time (a missing Vim setup line, an SSH key not yet made). `kickoff.pl doctor --fix` sets the recommended git settings and prints each one; everything else is a line in the output with its fix.

Then one person, usually the configuration manager, runs the week-zero sequence. Read the plan before running it:

```
perl "$KIT/sim/tidewater-v1.pl" --drop 1 first-drop > /dev/null
perl "$KIT/tools/gitlab/kickoff.pl" start --local gitlab --project harbor/model --title Harbor \
    --drop first-drop --version v1.0 | grep -E "^== step"
```

Twelve steps: check the laptop, make the model repository from the kit's templates, create the GitLab project from the project file, push, start gitflow, add the schedules, create the sprints, import the first drop onto the vendor branch, start the journal, write the wiki pages, and open the first stories. Without `--yes` it only prints the plan. With `--yes` it does it, against the offline GitLab here or against your real server with `--url`:

```
perl "$KIT/tools/gitlab/kickoff.pl" start --yes --local gitlab --project harbor/model --title
Harbor \
    --drop first-drop --version v1.0 > start.log 2>&1 || echo "exit status $?"
grep -E "week zero is set up|^next:" start.log
ls
```

Against a real GitLab, the same command runs with `--url https://your.gitlab --token-env GITLAB_TOKEN`. If a step fails (a permission your account does not have yet, a runner not registered), the output says which step and how to continue from it with `--from N`. On Tidewater the runner request took until the start of sprint 2; plan for that.

Warning: Two things the kickoff cannot do for you

It cannot get your runner approved, and it cannot get your supplier to keep element IDs stable. Both take weeks of other people's time. Ask for them in week zero, in writing, with a date.

50.3 Sprint by sprint: what to watch

The table above says what each sprint is for. This section says what usually goes wrong in it, how you will see it coming, and what Tidewater did about it. None of it is specific to SysML or to GitLab; it is what happens when a team that knows its system well meets tools it does not know.

50.3.1 Sprint 1: the urge to plan everything

The first sprint's risk is a plan that is too big. A lead who has finally been given permission to change things tends to put the whole port on the board. Jim's first draft of sprint 1 held more points than the team had capacity for, before any learning time; the team cut it at the kickoff, after the planning poker made the gap obvious (Section 24.3, Planning poker (p. 382)).

Watch for: stories with no acceptance criterion a program can check; a sprint committed above about nine tenths of capacity; nobody on the board with a learning story. Fix it at the kickoff, not at the review.

The other risk in sprint 1 is silence. People who are unsure of the new tools say "working on it" at stand-up for three days in a row. The kit's stand-up lint flags a status with no story ID or no change since yesterday, and the "same plan three days" flag in Section 24.2, The daily stand-up: terse, written, linted (p. 378) exists for exactly this. Treat it as a request for help that nobody dared make.

50.3.2 Sprint 2: the runner, and the first red pipeline

The runner is almost always late, and the first pipeline on a real runner almost always fails on something the laptop run did not show (on Tidewater, the Windows runner's default shell was PowerShell, and every script line written for Git Bash failed: Chapter 21, Runners: where the pipeline runs (p. 347) tells the story). Neither is a reason to slip the sprint. Plan the runner as a blocked item with an owner and a date from the first day of the sprint, and keep the team working with the local pipeline (`gitlab.pl ci`) until it arrives.

Sprint 2 is also when the first real merge conflict happens, usually in the drop import. Treat the first one as a team event: resolve it together, at one screen, with the conflict markers explained (Chapter 10, Vim for models (p. 143), and practice lesson 12). The second conflict is then just work.

50.3.3 Sprints 3 and 4: the dip

Most teams have a dip around sprint 3. The novelty is gone, the setup work is still not quite finished, and the engineering has not yet sped up. On Tidewater the pulse survey fell in sprint 3, when an approval blocked the security work for three days and Bob's first hand-written edit failed the grammar check in front of everyone.

Watch for: the pulse survey's second question ("I can do my work in the new tools without help") falling; merge requests sitting unreviewed for days; the same person doing all the reviews. Fix: pair the struggling person with the most fluent one for a day; rotate reviews; make the sprint 4 review the moment something visible goes in front of management, so the team sees the work land.

50.3.4 Sprints 5 and 6: gold-plating, and the briefing

By sprint 5 the team is fluent, and the risk turns around: the pipeline grows jobs nobody asked for, and platform work creeps back up. Every new job should answer a problem the team actually had, named in the story ("we merged a trace gap twice; the ratchet stops it"). If you cannot name the problem, the job waits.

Sprint 6 ends with the briefing. Start it in sprint 5: decide the four or five numbers it will show, check they are computed by programs, and rehearse it at the sprint 5 review. Chapter 25, Talking about progress: helps for engineers who would rather not (p. 390) and the memo templates in Chapter 37, Writing it down: DoD-style memos from Vim (p. 545) are the tools; the numbers come from the measures agreed in week zero, and nothing new should appear in the briefing that management has not seen in a weekly note first.

50.4 Selling the plan in week zero

The plan has two audiences before it has a single story done: the team, who may not want it, and management, who asked for it but may expect too much. Each needs a different conversation.

For management, the plan has three points, in this order. First, the measures: what "measurable progress" will mean in twelve weeks, computed by programs, agreed now so nobody argues about them in December. Second, the cost: the split of the first two sprints is mostly platform and learning, on purpose, and it falls by sprint 4; the enablers (learning time, a runner, approvals, a tier decision) are asks with dates. Third, the checkpoint: something visible at the sprint 4 review, so management does not wait twelve weeks to know whether it is working.

For the team, the conversation is about what does not change. They still own the engineering. Their judgment still decides what the system is. What changes is where the work is recorded, how it is checked, and how often it is integrated. A line that says it plainly: "We are not becoming software developers. We are keeping our model the way software teams keep their code, because it is checked on every change." The objections are real; let each one be said, as Jim did in Chapter 3, The directive (p. 22), and point at the chapter that answers it.

50.5 The decisions that are yours

Every decision below was made one way on Tidewater. The column on the right says what should drive yours.

Table 50.2: Decisions to make locally, and what should drive them

Decision	Tidewater's choice	What should drive yours
Branching model	gitflow: develop for integration, main for baselines, a vendor branch for the supplier's drops	Baselines delivered under contract favor gitflow; a model nobody "releases" can use GitLab flow (Chapter 12, Git from zero, for systems engineers (p. 171))
Merge method	merge commits, never squash	Keep merge commits if you will measure from history; squash only if nobody will
Boundary with the supplier	full mirror of the supplier's model	Full mirror if the model is your deliverable; interface surface if you consume theirs (Section 45.8, Two strategies at the v1-to-v2 boundary (p. 629))

Decision	Tidewater's choice	What should drive yours
Repository layout	layered folders through a layout map	The layers suit any system; the folder names should match your subsystems (Chapter 16, The model repository: what goes where (p. 245))
Requirements source	DOORS authoritative, converted on every drop, never hand-edited	Whatever your contract says governs; never two editable copies
Runner	a Windows shell runner on a dedicated VM with Git for Windows' Perl	What your enterprise can approve; a Docker runner works with the optional image (Chapter 21, Runners: where the pipeline runs (p. 347))
GitLab tier	started on Free, Premium decided in sprint 2	Approval rules and code-owner enforcement are the Premium features worth paying for first (Chapter 17, The DevSecOps tools that earn their keep (p. 255))
Generated documents	committed, with a "docs are current" gate	Commit them if reviewers read diffs of documents; regenerate in CI only if nobody does
Primary progress measure	TODO lines left by the converter	Any count the strongest skeptic would accept, computed by a program
Story size	tasks at most five ideal days, stories within a sprint	Keep the limits; the numbers are what make a sprint plannable
Ceremony times	stand-up 09:00, refinement Wednesday week 2, review and retro Friday week 2	Whatever the team will actually attend; protect them from other meetings

Record each one as a decision record in `docs/adr/` in the model repository, with the context and the reason. The scaffold starts the folder with ADR 0001; Chapter 13, The operating model (p. 196) shows Tidewater's list. A new team member reading those files learns in an hour why the repository is the way it is.

50.6 The learning budget

No one on Jim's team had used git, written a pipeline or read SysML v2 text before sprint 1. The plan budgeted for that, in points, on the board, as stories labeled `work::learning`: practice lessons, chapters of this book, pairing at the terminal. On Tidewater the learning share started at about a quarter of sprint 1 and fell to a few percent by sprint 6 (the exact figures are in Bottom line up front (p. iii), cited from `docs/tidewater/work-split.json`).

Copy the budget, not the numbers. Two rules:

- **Budget the learning, or it happens on nights and weekends.** A team that is expected to learn git on top of a full sprint learns it badly, resents it, and tells management the new way does not work. Learning time on the board is visible, defensible, and shrinks by itself as competence grows.
- **Learning is on the plan, not in the margins.** Each person's learning stories have tasks and acceptance criteria like any others: "practice lesson 11 checks as done," "can open a merge request without help."

A reasonable starting reserve for a team new to all of it is a fifth of sprint 1, a sixth of sprint 2, a tenth of sprints 3 and 4, and a few percent after. Adjust at each retrospective from the pulse survey's second question ("I can do my work in the new tools without help").

50.7 Your backlog

The kit ships a generic backlog for a v1-to-v2 port: tomes, epics, stories and tasks with points, an owner role, a sprint hint and an acceptance criterion written as something a person or the pipeline can check. It is written in neutral terms (the supplier, the integration branch), so it fits a program that is not Tidewater:

tools/gitlab/templates/backlog.csv

```
tome,epic,story,task,points,owner_role,sprint_hint,acceptance
T1 Stand up MBSE DevSecOps,E1 GitLab project and gitflow,Create the group and the model
project,Apply the project file with gitlab.pl setup; request a runner,1,CM engineer,1,The project
exists with the settings in project.conf
T1 Stand up MBSE DevSecOps,E1 GitLab project and gitflow,Create the group and the model
project,Prove a direct push to the integration branch is refused,1,CM engineer,1,The push is
rejected by the server
T1 Stand up MBSE DevSecOps,E1 GitLab project and gitflow,Create the group and the model
project,"Commit CODEOWNERS, the templates and .gitignore",1,CM engineer,1,The files are on the
integration branch through a merge request
T1 Stand up MBSE DevSecOps,E3 Team practices and onboarding,Laptops ready,Install Git for Windows
and Vim; run kickoff.pl doctor (one per person),1,each engineer,1,kickoff.pl doctor shows no FAIL
T1 Stand up MBSE DevSecOps,E3 Team practices and onboarding,Learn the tools,"Practice lessons,
the survival kit, git and Vim chapters (one per person)",1.5,each engineer,1,Lessons done;
questions written in the wiki
T1 Stand up MBSE DevSecOps,E3 Team practices and onboarding,Working agreements,Write the
Definition of Ready and Done in the wiki,0.5,MBSE lead,1,Both pages on the wiki
T1 Stand up MBSE DevSecOps,E1 GitLab project and gitflow,Record the decisions,Write the week-zero
decisions as ADR files under docs/adr,1.5,MBSE lead,2,docs/adr holds one file per decision
T1 Stand up MBSE DevSecOps,E2 Pipeline and quality gates,Draft the first pipeline,"Draft
.gitlab-ci.yml: grammar, lint, outline",0.5,CM engineer,1,The draft runs locally with gitlab.pl
ci
T1 Stand up MBSE DevSecOps,E2 Pipeline and quality gates,The pipeline gates every merge
request,Commit the pipeline as perl ci/job.pl NAME lines; check it on the real runner,2,CM
engineer,2,A pipeline runs on every merge request
T1 Stand up MBSE DevSecOps,E2 Pipeline and quality gates,The pipeline gates every merge
request,Turn on pipelines-must-succeed and the report widgets,0.5,CM engineer,2,A red pipeline
blocks the merge
T1 Stand up MBSE DevSecOps,E2 Pipeline and quality gates,Burn-down and generated docs,Add the
todo and docs jobs; commit docs/,2,CM engineer,2,The todo count shows on every merge request
```

Sum it by sprint before you change anything, to see the shape:

```
perl -MText::ParseWords -ne 'next if $. == 1; s/\r//; my @c = parse_line(",", 0, $_); $p{$c[6]}
+= $c[4];
  END { printf "sprint %-10s %5.1f points\n", "$_:", $p{$_} for sort { ($a =~ /^\d+$/ ? $a : 99)
  <=> ($b =~ /^\d+$/ ? $b : 99) or $a cmp $b } keys %p }' \
  "$KIT/tools/gitlab/templates/backlog.csv"
```

Then change it: replace the system's names, cut the stories your program does not need (no supplier drops, no DOORS), add the engineering stories your program does need (Chapter 46, Systems engineering work, not pipeline work (p. 636) has a catalog), and re-estimate with your own team in planning poker (Section 24.3, Planning poker (p. 382)). Do not keep a story because Tidewater had it.

The kit has no command that loads the CSV into GitLab in one go; a short loop does it. Here the sprint 1 rows become task issues in the project the kickoff made:

```
perl -MText::ParseWords -ne 'next if $. == 1; chomp; s/\r//; my @c = parse_line(",", 0, $_); next
unless $c[6] eq "1";
  system($^X, "$ENV{KIT}/bin/gitlab.pl", "--local", "gitlab", "--project", "harbor/model",
  "--as", "jim",
         "issue", "new", $c[3], "--type", "task", "--label", "type::task", "--milestone", "Sprint
         1",
         "--description", "Story: $c[2]. Done when: $c[7]. Points: $c[4].") == 0 or die "issue
         new failed\n"' \
  "$KIT/tools/gitlab/templates/backlog.csv" > /dev/null
perl "$KIT/bin/gitlab.pl" --local gitlab --project harbor/model issue list --milestone "Sprint 1"
| head -5
echo "(milestone Sprint 1)"
```

In practice, load one sprint at a time, at each kickoff, after the team has refined it. A backlog loaded eighteen months ahead is a backlog nobody reads.

50.8 What to measure

Measure from day one, with programs, and agree the measures with management before sprint 1. Three layers of measures served Tidewater, and each answers a different audience.

50.8.1 The split: where the time goes

Every story carries `work::engineering`, `work::platform` or `work::learning`, and the split of points by label, sprint by sprint, goes on the first page of every status (Bottom line up front (p. iii)). Tidewater's is computed from its plan by one script; it shows how:

```
perl "$KIT/sim/tidewater-split.pl" --out "$HOME/split.json"
```

The shape to look for: platform and learning high in sprints 1 and 2, engineering the larger share from sprint 3, mostly engineering by sprint 6. A flat line means the setup never finished.

50.8.2 The four headline numbers

At every review, the same four numbers, so management learns to read the trend (Chapter 25, Talking about progress: helps for engineers who would rather not (p. 390)):

1. **Conversion debt** (TODO lines left by the converter), from the pipeline's `todo` job.

2. **Verify coverage** (requirements with a verification case), from the verification matrix or the trace.
3. **Open discrepancy reports and supplier changes** awaiting disposition, from GitLab labels.
4. **Blocked items**, from the board.

The first one is there from the moment the first drop is converted. In the repository the kickoff made, on the drop's branch:

```
git checkout -q feature/drop-v1.0
perl "$KIT/bin/gitlab.pl" ci --event merge_request_event --branch feature/drop-v1.0 --target
develop \
    --job todo --logs ../headline > /dev/null
grep "TODO lines remaining" ../headline/todo.log
```

50.8.3 The scorecard

The twelve measures Jim agreed with management before sprint 1 (Section 3.6, Defining measurable progress (p. 28)) are the case at week twelve. Copy the rule more than the list: *every measure is computed by a program from the repository or GitLab, and every measure is one a skeptic in the room would accept.* Then pick the measures your management cares about. The ones that carried the most weight on Tidewater:

- change lead time and review turnaround, against the document era's numbers (labeled notional if they are reconstructions);
- the share of changes gated (it must be 100%, and GitLab's protected branches make it a setting);
- the time to answer "which requirements are unverified?" (two days of spreadsheet work before, a pipeline job's seconds after);
- reproducible baselines;
- the team's own pulse survey, which the program manager insisted on and was right to.

GitLab's analytics give the flow measures on any tier through the kit (Chapter 32, Measuring with GitLab analytics (p. 513)):

```
perl "$KIT/bin/gitlab.pl" --local gitlab --project harbor/model analytics | head -6
```

On a project a day old the tables are empty. By the sprint 2 review they have a story to tell.

50.9 What to copy from Tidewater, and what not

Table 50.3: Copy the shape, adapt the details

Copy	Adapt	Leave behind
The scorecard rule: measured by programs, accepted by skeptics	The measures themselves	Tidewater's targets
Learning stories on the board	The size of the reserve	Tidewater's lesson order, if your team already knows git
Something visible for management by sprint 4	What it is (a model site, a data exchange table, a VCRM)	The Pages site, if your network forbids it
The vendor branch for a supplier's drops	How drops arrive (file, registry and trigger)	Seven drops; your supplier's cadence is in your contract
The change board for every drop, in GitLab	Who sits on it	Jim and the deputy PM
Merge requests, code owners, protected branches	The approval rules, by your organization's roles	Tidewater's CODEOWNERS sections
A small pipeline that grows one job at a time	Which jobs, in which order	Tidewater's exact job list
Terse stand-ups, poker, refinement, demo, retro	Times and lengths	Tidewater's 09:00 stand-up
The four headline numbers at every review	The fourth number, if blocked items are not your issue	—
Decision records in the repository	The decisions	Tidewater's D-1 to D-10

Three things from Tidewater are easy to copy and wrong to copy:

- **The spider-web cleanup stories.** Tidewater's v1 model had specific clutter (a Legacy Import package, two dependency cycles). Yours has its own. Measure it first (Section 11.4.4, Packages (p. 162) shows the package view), then plan.
- **The planted problems.** The simulation planted gaps, drift and failures so the book would have something to show. Your real model will have different ones, and probably more.
- **The cast.** Your team is not three people with these roles. A one-person team can run the same loop with approvals from a colleague in another group; a ten-person team needs more code-owner sections and a second runner. Keep the separation of author and approver, whatever the size.

50.10 When things go differently

50.10.1 No supplier drops

If your v1 model is your own and nobody else will change it, drop the vendor branch, the change board for drops and the impact analysis. Convert once, onto a branch, review it, merge it, and from then on the v2 text is the model. Tidewater's tome 3 (maintenance against drops) disappears, and the months it took go to engineering.

50.10.2 Documents, not Cameo

If the starting point is documents and spreadsheets rather than a v1 model, the port is writing, not converting. The plan's shape still holds, but sprint 1's "convert and assess" becomes

"inventory and decide the package structure," and the TODO count is replaced by a count of requirements and interfaces captured. The requirements recipe (Chapter 7, Requirements a pipeline can check: the recipe (p. 73)) and the workbook (Chapter 8, SysML v2 by example: a workbook (p. 92)) become the core of sprints 2 to 4.

50.10.3 A bigger team

Above about six people, split into teams along the model's layers or segments, give each team a CODEOWNERS section, and consider one repository per segment with the system repository consuming tagged releases of each (Chapter 16, The model repository: what goes where (p. 245) describes the federation). The twelve-week shape holds per team.

50.10.4 A slower organization

If the runner, the GitLab tier and the tool approvals take a quarter instead of a sprint, start anyway: the kit's offline GitLab and local pipeline (`gitlab.pl ci`) let the team learn and work on laptops while the approvals move. Tidewater's practice lessons are exactly that. Keep the request-for-enablers memo (Chapter 37, Writing it down: DoD-style memos from Vim (p. 545)) current and show it at every review: the blocked items are the fourth headline number for a reason.

50.11 The first Monday

Checklist: Before the first sprint 1 stand-up

- ☐ The scorecard page agreed with management, with the document-era baselines labeled
- ☐ The supplier's exchange agreement drafted, with stable IDs and change notes asked for
- ☐ The ISSM met; the marking decision recorded as a decision record
- ☐ Every laptop: `kickoff.pl doctor` shows no FAIL, or the software request is in with a date
- ☐ The runner request submitted, with a date asked for
- ☐ `kickoff.pl start` run against the real GitLab (or the offline one while access is pending)
- ☐ Sprint 1 refined and estimated in planning poker, learning stories included
- ☐ The ceremony times in everyone's calendar, the stand-up format agreed (Section 24.2, The daily stand-up: terse, written, linted (p. 378))
- ☐ The first drop received, its checksums verified
- ☐ The request-for-enablers memo drafted, to go out in sprint 2

50.12 Applying this to your project

This whole chapter is the application. If you keep three things from it:

- **Agree the measures before you start, and compute them with programs.** Everything else in the case follows from numbers nobody can argue with.

- **Put the learning on the board.** A team that is given time to learn will tell management the new way works, because for them it does.
- **Show something by sprint 4.** Confidence is built in the middle of the twelve weeks, not at the end.

Exercise 50.1: Your twelve-week table

Copy the twelve-week table into your program's terms: your system's packages, your supplier (or none), your reviews. For each sprint, write the goal as an outcome management would recognize, and the "done when" as something a program can check.

Exercise 50.2: Your decision records

Write ADRs 0002 to 0005 for your program from the decisions table: branching model, supplier boundary, merge method and primary progress measure. One page each: context, decision, consequences.

Exercise 50.3: Your sprint 1

Take the first two sprints of `backlog.csv`, rewrite them for your program, sum the points, and check them against your team's capacity with a learning reserve. What do you cut first?

Exercise 50.4: Run week zero offline

Run `kickoff.pl start --yes --local DIR` for an invented project of your own with drop 1 of Tidewater, then open `DIR` with `gitlab.pl --local DIR issue list` and read the stories it made. Which ones would you rewrite for your program?

CHAPTER

51

Pitfalls and anti-patterns

From what you know →

You know

lessons-learned reports that nobody reads.

→ Here it is called

pitfalls and anti-patterns.

→ What is new

each pitfall has a symptom you can spot early in your own project.

Survival kit: Knowing the ditches

What you have

a working process and, by now, some habits, good and bad.

What you need

a way to recognize the bad habits early, from the first symptom you would actually notice, before they cost a sprint. Every team moving from documents and Cameo to a text model in GitLab falls into some of these. Tidewater fell into more than half of them.

51.1 How to read this chapter

Each pitfall has the same four parts:

- **Symptom**: what you will see first. Usually it is small: a number that stops moving, a conversation that keeps recurring, a pipeline colour nobody comments on any more.
- **Why it happens**: the reason a sensible team ends up there. None of these pitfalls is stupidity. Most of them are an old habit, correct in the document era, applied in a place where it no longer fits.
- **Fix**: what to do, as concretely as possible.
- **At Tidewater**: where the team met it, and what happened.

The pitfalls are grouped by where they bite: the model, the change process, the pipeline, the measures, and the people. A checklist version, for a monthly look in the mirror, closes the chapter.

51.2 Pitfalls in the model

51.2.1 Porting the spider web

Symptom. The v2 model has the same duplicated blocks, stale packages and dependency cycles as the v1 model. Someone finds `Battery_old` in the part tree and asks whether it is real.

Why it happens. The converter is faithful: it carries what is in the XMI, clutter included, because it cannot know which blocks are clutter. A team under schedule pressure measures progress by TODO lines closed and treats everything the converter did not flag as done.

Fix. Treat untangling as its own work, with its own stories, measured by scorecard measure 7 (cycles, duplicates, the most-connected block). Do not delete clutter silently: raise a discrepancy report so the prime removes it upstream, and remove it in the government model in an MR that cites the report. Until the prime's drop removes it, the next vendor import will bring it back as a conflict, which is the reminder you want.

At Tidewater. The drop-1 conversion carried the `Legacy Import` package with its copied blocks, and the two package cycles from Chapter 2, The Cameo years (p. 10). Bob's first instinct was to delete the package in one commit. Jim stopped it: the deletion would conflict with every future drop until the prime removed it too. The team raised discrepancy reports, removed the clutter in the government model with the report numbers in the MR, and watched the prime's later drops remove it upstream (the `Copy of SensorSuite` block went in drop 3, the whole `Legacy Import` package in drop 6).

51.2.2 Reformatting the converter's output

Symptom. Every prime drop produces dozens of merge conflicts, in files where nobody changed anything that matters.

Why it happens. Converter output is not how a person would lay the text out, and a tidy engineer reformats it: reorders members alphabetically, re-indents, splits long lines, moves the `doc` comment above the tag. Each change is harmless alone. Together they mean that the next vendor import, which writes the converter's layout again, differs from the government file on nearly every line, and git's three-way merge can no longer tell the prime's changes from the government's.

Fix. Edit converter output *in place* and *minimally*: replace a `// TODO` line with the line that closes it, add a line where one is needed, and leave every other line exactly as the converter wrote it. Put government-only content (new packages, the conventions library, views) in separate files the converter never writes (on Tidewater, `model/library/conventions/` and its neighbours), which the vendor branch never touches. Write the rule into the Definition of Done.

At Tidewater. Tom reformatted `Interfaces.sysml` in sprint 2 to make it "readable" and the change sailed through review because the diff looked like pure layout. Drop 2's import then conflicted on forty lines of that file. Resolving them took Tom most of a day, after which the team wrote the in-place rule into the Definition of Done and reverted the reformatting.

51.2.3 Hand-editing the vendor branch

Symptom. A delta report shows changes the prime says it never made, or a vendor import does not reproduce when run again from the same drop.

Why it happens. The vendor branch looks like any other branch, and a quick fix there (a converter glitch, a typo in the prime's requirement text) seems faster than a discrepancy report.

Fix. The vendor branch holds exactly what the converters produce from the drop, nothing else. Protect it so only maintainers can push and nobody can merge into it. Fixes go on `develop` (government edits) or to the prime (a discrepancy report). If a converter is wrong, fix the converter, re-import, and record the converter version in the import commit.

At Tidewater. Never happened, because `vendor/prime` was protected from week zero with merge set to no one. Tom tested the protection in sprint 2 by trying to push a one-line change to it and kept the rejection message as evidence.

51.2.4 Changing requirement text in the model

Symptom. The trace job's drift count goes up after a government MR, not after a drop.

Why it happens. An engineer sees a badly worded requirement in the model, knows what it should say, and fixes the `doc` text. It feels like the obvious improvement.

Fix. DOORS is the authority for requirement text, and the prime owns DOORS. The model's requirement text is a copy. A wording problem is a discrepancy report to the prime; the fix arrives in a later drop, through both the ReqIF and the XMI. The government model may add notes or analysis around a requirement, never different words in it.

At Tidewater. TW-SYS-006 drifted from the start (Chapter 6, SysML v2 for SysML v1 modelers (p. 57)). Bob wanted to align the model text by hand in sprint

1. Jim raised a discrepancy report instead, and drop 2 fixed both sides. The drift count fell to zero through the

process rather than around it, which is what the scorecard needed to show.

51.2.5 Trusting the TODO count alone

Symptom. The TODO count is zero, the pipeline is green, and a reviewer finds that the government's hand-written state machine is missing a state the prime added months ago.

Why it happens. The converter writes a TODO line for each construct it does not carry. When the prime changes something *inside* such a construct (a new state inside a state machine, a new message in an interaction), the TODO line in the converted output is unchanged, so the vendor import shows no difference at all. The TODO count cannot see it.

Fix. Run the v1 delta on every drop: `xmi2sysml.pl --delta OLD.xmi NEW.xmi`. It compares the two drops element by element by `xmi:id` and marks every change the converter does not carry as *manual*. Each manual line becomes a task.

At Tidewater. Drop 3 added a `Storm` state to `BuoyModes`. The converted v2 output did not change at all. The delta listed five manual changes inside the state machine, and Bob's drop-3 story carried a task to add the state by hand (Chapter 43, Maintenance through the life cycle (p. 602)).

51.3 Pitfalls in the change process

51.3.1 The big-bang merge request

Symptom. An MR has thirty files and two thousand changed lines, it stays open for a week, and the review is a single comment saying "looks fine".

Why it happens. The document era trained people to batch: a document revision collected months of changes because each revision cost a CCB meeting. The instinct carries over: finish everything, then ask for review.

Fix. One task, one MR; one task, less than a week. If an MR touches more than one package or takes more than fifteen minutes to review, split it. Small MRs are reviewed properly, merge without conflicts, and keep lead time (scorecard measure 2) low.

At Tidewater. Bob's first structure MR in sprint 3 closed every TODO in two packages at once. Jim's review took two sessions and missed a wrong level tag, which a later MR had to fix. From sprint 4 on the team cut stories into package-sized tasks, and median review turnaround fell below a working day.

51.3.2 Long-lived branches

Symptom. A feature branch is two weeks old, and merging `develop` into it produces conflicts in files the branch's author never touched.

Why it happens. The engineer is "nearly done" for days, and keeping the branch private feels safer than showing an unfinished change.

Fix. Branches live days, not weeks. Merge `develop` into your branch daily if it must live longer, and push it so the MR pipeline checks the integration. If a task will take longer than a week, it is two tasks.

At Tidewater. The only branch older than a week was the behavior work in sprint 6, which straddled Bob's leave. Its merge needed a manual conflict resolution in `Behavior.sysml`. The retrospective added a rule: before leave, merge or hand over.

51.3.3 Rewriting shared history

Symptom. A colleague's `git pull` fails with "diverged" messages, or a merge request suddenly shows commits that were already merged.

Why it happens. Tutorials teach `git rebase` and `git push --force` as tidy ways to clean up a branch, and they are, on a branch only you use. On a shared branch they rewrite history other people have.

Fix. Protected branches forbid force push; keep it that way. On your own feature branch, tidy before you push for the first time, not after. The project uses merge commits and no squash, because the merge history is part of the record (who merged what, when, after which review) and feeds the status metrics.

At Tidewater. Tom followed a tutorial's advice to squash commits in sprint 2, on a test project. When the team saw that squashing would make the merge history unable to show which engineer made which change, they set squash to *never* in `project.conf` and wrote the reason beside it.

51.3.4 Silent conflict resolution

Symptom. After a vendor merge, a change the prime announced in its manifest is not in the government model.

Why it happens. Conflicts are uncomfortable, and git offers one-word ways out: take our side, or take theirs, for a whole file. Taking ours drops the prime's change; taking theirs drops the government's edits.

Fix. Resolve conflicts hunk by hunk in Vim (Chapter 10, Vim for models (p. 143)), or take the vendor side and re-apply the government's edits deliberately, which is the method the Drop Intake Runbook prescribes. Either way, the MR description lists each conflicted file and how it was resolved, and the reviewer checks the manifest's changes against the merged result.

At Tidewater. Drop 5 changed the `EnergyBalance` constraint expression in a TODO line the government had already replaced by hand. The conflict was real: both sides changed the same line. Tom took the vendor side and re-applied the hand-written constraint with the new expression, and listed the file in the MR. `gitlab.pl flow take` exists for the wholesale case, but the runbook uses it only on files the government never edits.

51.3.5 Old reviews in new clothes

Symptom. Every merge request is approved, and then a monthly meeting reviews the same changes again from slides.

Why it happens. The organization does not yet trust the MR approval as the review of record, so it keeps the old board as a safety net. Nobody says out loud that the MR review is now redundant or that the board is.

Fix. Decide which review is the review of record, write it into the configuration management plan, and make the other one a summary. For Tidewater, the MR approval by a code owner is the change approval; the CCB meets monthly to look at the *outcomes* (baselines, discrepancy reports, metrics), not at individual changes.

At Tidewater. The deputy program manager, who chairs the CCB, asked in sprint 4 for "the list of model changes for the board". Jim answered with the merged MR list from `gitlab.pl export` and offered to replace the change-by-change review with a fifteen-minute review of the export. The board agreed after seeing two months of it.

51.4 Pitfalls in the pipeline

51.4.1 Red is normal

Symptom. The pipeline on `develop` has been red for three days, and the stand-up no longer mentions it.

Why it happens. A failing check that blocks nothing becomes background. Usually it started with a job that failed for a reason outside the team's control (a runner problem, a server outage), and when the reason went away nobody re-ran it.

Fix. A red pipeline on `develop` or `main` is the team's first priority, above any story, until it is green. The stand-up asks one question first: "is develop green?" Scorecard measure 8 measures the time from a failure to the next pass so that the habit is visible.

At Tidewater. The runner's disk filled in sprint 6 and every job failed. The fix took Tom an hour once noticed, but it was noticed only the next morning. The team added "is develop green?" as the first line of the stand-up and a disk-space check to the weekly runner look (Chapter 42, Administration (p. 586)).

51.4.2 Allowed to fail, forever

Symptom. Half the jobs are yellow (allowed to fail) and nobody can remember why.

Why it happens. `allow_failure: true` is the easy way to add a check that does not pass yet. It is meant to be temporary and almost never is.

Fix. Every `allow_failure` has a comment in `.gitlab-ci.yml` saying why and until when, and an issue to remove it. Tidewater's `trace` job is allowed to fail by design (the planted requirement gaps are the prime's to fix, and the discrepancy reports track them); that reason is written beside the job.

At Tidewater. The `outline` check started as allowed to fail during the port because the converter's layout broke the outline rule. It had an issue, a waiver in the wiki, and a removal date tied to the end of the structure epic. When the date came the waiver was removed in an MR, and the job went back to blocking.

51.4.3 Bypassing the gate

Symptom. A protected branch was unprotected "for five minutes" to push a fix directly, and the audit shows it.

Why it happens. Urgency, and a maintainer's role that permits it. In the document era, the equivalent was walking a change past the CCB because the review was next month.

Fix. The gate has no bypass that leaves no trace. Urgent fixes go through a hotfix branch and a merge request like everything else; a small MR with a green pipeline merges in minutes. The monthly protection audit (Chapter 42, Administration (p. 586)) checks that nothing was unprotected, and the audit events (Premium) show who did it.

At Tidewater. Nobody unprotected a branch, which Jim attributes to the sprint 1 exercise: the team deliberately opened an MR with a lint error and watched the merge button refuse (Chapter 26, Sprint 1: A GitLab project the team can work in (p. 404)). Seeing the gate work made it feel like protection rather than obstruction.

51.4.4 Unpinned tools

Symptom. The pipeline fails on `develop` with no model change, and the failure is in a check's behavior.

Why it happens. The runner uses whatever version of the kit or the converters is newest, and someone updated it.

Fix. The runner and the laptops use the same pinned version of every tool. Upgrading a tool is a story with its own MR and its own pipeline run (Section 42.9, Upgrades (p. 599)).

At Tidewater. The runner and laptops ran the same tagged kit from week zero, after Tom saw what a moving dependency did in the vendor import rehearsal: re-running the drop-1 import

with a newer converter produced a different `Stations.sysml`, which would have shown up as a phantom prime change.

51.4.5 Tools before habits

Symptom. In week two the project has a board with nine columns, forty labels, epics, iterations, value stream analytics and a security dashboard, and the team still cannot say what the next task is.

Why it happens. GitLab has a feature for everything, and the enthusiastic member of the team turns them all on.

Fix. Adopt one tool when it pays for itself, and say what it pays for. Tidewater's adoption timeline (Chapter 17, The DevSecOps tools that earn their keep (p. 255)) added one or two features per sprint, each when a problem made it worth the cost of learning it.

At Tidewater. Tom proposed a full security scanning suite in week zero. Jim asked what it would scan: the model had no code, so SAST and dependency scanning had nothing to look at. Secret detection, which did have something to look at, went in at sprint 3.

51.5 Pitfalls in the measures

51.5.1 Measuring by hand

Symptom. The status chart is assembled on Thursday night from three spreadsheets, and the numbers on it cannot be reproduced the following week.

Why it happens. It is how status was always done, and the automated numbers are not yet trusted.

Fix. Every number in a status report comes from the repository, GitLab's records or the journal, produced by a script or the pipeline. If a number cannot be produced that way, it is not on the scorecard. The rule was set in Chapter 3, The directive (p. 22) and it is the one that made the sprint 6 briefing believable.

At Tidewater. In sprint 2 Jim was tempted to report "percent of model ported" from a judgment. The TODO count was not yet automated, so Jim reported nothing for that measure and said why. The `todo` job came in that sprint.

51.5.2 Gaming the TODO count

Symptom. The TODO count falls faster than the work could possibly have been done, and some MRs delete TODO lines without replacing them.

Why it happens. Any measure that becomes a target invites shortcuts. Deleting a TODO line is a one-keystroke "closure".

Fix. The Definition of Done says a TODO line is closed only by the construct that replaces it, and the reviewer checks the replacement. The v1 delta and the trace job catch missing content later, but review catches it first. Report the TODO count together with measures that would move if it were gamed (trace completeness, lint warnings).

At Tidewater. It did not happen, partly because Bob, the strongest skeptic, was also the one closing most TODO lines and saw the count as proof of work rather than a target set by others.

51.5.3 Reporting only the good numbers

Symptom. The sprint report shows four measures; last sprint it showed six.

Why it happens. Two measures looked bad, and leaving them out felt kinder than explaining them.

Fix. Report every agreed measure, every sprint. A bad number with a reason and a plan is credible; a missing number is noticed and remembered.

At Tidewater. Sprint 3 was bad: the pipeline caught Bob's grammar error, the ISSM blocked the security story for three days, and morale dipped (Chapter 28, Sprint 3: the first red X, DOORS in the model, and a wait nobody named (p. 440)). Jim reported all of it. At the sprint 6 briefing the program manager said that the sprint 3 report was the reason the sprint 6 numbers were believed.

51.6 Pitfalls with people

51.6.1 The hero

Symptom. Only one person can fix the pipeline, run a vendor import or cut a baseline, and that person's leave is planned around.

Why it happens. The person who set it up knows it best, and it is faster for them to do it than to teach it.

Fix. Rotate the chores. Every procedure has a wiki runbook, and every runbook has been followed at least once by someone other than its author. Pair on the first instance of anything.

At Tidewater. Tom built the pipeline and could have stayed its only mechanic. From sprint 4 on, Bob did one pipeline change per sprint with Tom reviewing, and Jim ran the drop-2 vendor import with Tom watching. When Tom was on leave in sprint 9 nothing stopped.

51.6.2 Forcing it on the skeptic

Symptom. The skeptic complies, silently, does the minimum, and the old way continues in a spreadsheet on the side.

Why it happens. Management directed the change, so arguing feels pointless; the skeptic's objections are treated as resistance instead of requirements.

Fix. Write the objections down and answer each one with evidence, as Chapter 3, The directive (p. 22) did and Chapter 38, How the team changed its mind (p. 557) revisits. Give the skeptic the measure they would believe and let them run it. Never ask anyone to pretend to be enthusiastic.

At Tidewater. Bob's objections were the most specific, and so the most useful. Bob's question "how will I know the model is right?" became the lint and trace jobs. Bob's quote from sprint 1 ("I found the drift in two minutes, it used to take a week") came from running one of them.

51.6.3 Stand-up as a status meeting

Symptom. The stand-up takes forty minutes, each person reports to Jim, and problems are solved in the meeting.

Why it happens. The team's last daily meeting format was a status meeting with a manager, and that is what a daily meeting feels like.

Fix. Terse statuses: yesterday, today, blockers, with story ids, posted in the meeting chat and checked by the kit's lint (Chapter 24, The rhythm: stand-ups, poker, refinement, kickoff, demo, retro (p. 377)). Problems get named in the stand-up and solved after it, by the people involved.

At Tidewater. The first stand-up ran twenty-five minutes. By sprint 3, statuses were one line each, the lint found nothing to complain about, and the meeting took ten minutes.

51.6.4 Tasks too big to finish

Symptom. Stories carry over sprint after sprint, and tasks sit in the "in progress" column for more than a week.

Why it happens. Estimates in the document era were in weeks or months, and a five-point ceiling feels artificial.

Fix. Enforce the sizes: a task is at most five points (five ideal days) and is cut further if it might not finish in a week; a story fits in a sprint. A carried-over story is discussed at the retrospective, not just moved.

At Tidewater. The Stations story in sprint 4 carried two points into sprint 5 (Chapter 29, Sprint 4: the model site, a cycle broken, and the demo that landed (p. 459)). The retrospective found that its level-tag task was really two tasks. Carryovers were rare after that.

51.7 A monthly look in the mirror

Checklist: Pitfall check, once a month at the retrospective

- ☐ The v2 model has no duplicate blocks, no package cycles, and no clutter without a discrepancy report
- ☐ Converter output is edited in place only; government-only content is in files the converter never writes (`model/library/conventions/` and the like)
- ☐ Nobody pushed to `vendor/prime` except imports
- ☐ Requirement text changes came only from drops
- ☐ Every drop had a v1 delta, and its manual items became tasks
- ☐ No MR this month was larger than a task; no branch lived longer than a week
- ☐ No force push, no squash, no unprotected branch
- ☐ Every conflicted file in a vendor merge is listed in its MR with how it was resolved
- ☐ `develop` was never red for more than a working day
- ☐ Every `allow_failure` has a reason and an issue
- ☐ Tools are pinned; upgrades were stories
- ☐ Every reported number came from a script; every agreed measure was reported
- ☐ Each runbook was followed by someone other than its author at least once
- ☐ Stand-ups took fifteen minutes or less
- ☐ No story carried over twice

51.8 Applying this to your project

- **Copy** the monthly checklist into your retrospective template. It costs five minutes.
- **Adapt** the pitfalls to your sources. If your v1 model has no prime contractor and no vendor branch, the vendor pitfalls become pitfalls about your own Cameo exports, but reformatting and in-place editing matter just as much.
- **Decide locally** which pitfalls your organization is most prone to. A team with a strong CCB culture will fall into old reviews in new clothes; a team with an enthusiastic tool-builder will fall into tools before habits. Watch for your own two.

51.9 Exercises

Exercise 51.1: Diagnose the symptom

For each symptom, name the pitfall and the first thing you would do: (a) a vendor merge conflicts on 60 lines of a file the government only added level tags to; (b) the drift count rose after a government MR; (c) the TODO count is zero but the prime's manifest mentions a new state; (d) `develop` has been red since Tuesday.

Exercise 51.2: Your two pitfalls

Pick the two pitfalls your own organization is most likely to fall into. For each, write the symptom you would expect to see first in your environment, and one rule that would prevent it.

Exercise 51.3: Prove the TODO count blind spot

Generate drops 2 and 3 with `perl sim/tidewater-v1.pl --drop 2 /tmp/d2` and `--drop 3 /tmp/d3`. Convert both with `xmi2sysml.pl --out` and compare the two `stations/Stations.sysml` files. Then run `xmi2sysml.pl --delta /tmp/d2/tidewater.xmi /tmp/d3/tidewater.xmi`. Which change appears in the delta and not in the converted files, and why?

Appendices

The appendices are for use at the keyboard. Appendix A, Command reference (p. 696) lists every command the book uses, with its options, checked against the tools' own help. The three quick reference guides, Appendix B, Quick reference: Vim for MBSE DevSecOps (p. 709), Appendix C, Quick reference: git, gitflow and the GitLab tools (p. 713) and Appendix D, Quick reference: SysML v1 to v2 (p. 719), are meant to be printed and kept beside the monitor. Appendix I, Templates and GitLab Markdown, field by field (p. 747) shows every GitLab template and filled-in sample, with the Markdown features each one uses. Appendix J, Checklists (p. 789) gathers every checklist in the book in one place, Appendix K, Glossary (p. 796) translates between systems engineering, DevSecOps and SysML v2, and Appendix O, Answers to the exercises (p. 815) answers the exercises.

CHAPTER

A

Command reference

From what you know →

You know

the cheat sheet taped to the monitor.

→ Here it is called

the command reference.

→ What is new

every command in it has been run.

This appendix lists every command the book uses, grouped by tool, with the options that matter for the process. The syntax lines are taken from each tool's own help, which is always the final word: run the help command shown at the top of each section when in doubt. Commands are run from Git Bash on Windows (or any shell on Linux) at the root of the kit unless the section says otherwise; `perl bin/…` paths are relative to the kit. In the model repository the pipeline reaches the same tools through the `KIT` variable, as `perl "$KIT/tools/sysml/model.pl"` and so on.

A.1 git

Help: `git help COMMAND` (or `git COMMAND -h` for the short form). The commands below are the ones a model engineer uses in a normal month; Chapter 12, Git from zero, for systems engineers (p. 171) teaches each one from zero.

Table A.1: The git commands a model engineer uses

Command	What it does	Configuration management analogy
`git clone URL`	copy the repository, with its whole history, to your laptop	draw a working copy from the library
`git status`	what you have changed, what is staged, which branch you are on	look at your desk
`git switch develop`	move to an existing branch	pick up the master copy

Command	What it does	Configuration management analogy
`git switch -c feature/19-stations-level-tags`	create a branch and move to it	start a working draft
`git pull`	bring the server's new commits into your current branch	receive the latest revision
`git add FILE`	stage a change for the next commit	put a page in the change package
`git restore --staged FILE`	unstage it again	take the page back out
`git restore FILE`	throw away your unsaved changes to a file	discard a draft markup
`git diff`	your changes not yet staged, line by line	compare draft to controlled copy
`git diff --staged`	what the next commit will contain	review the change package
`git commit`	record the staged changes, with a message (opens Vim)	sign a revision
`git push -u origin HEAD`	send your branch to the server the first time	submit the draft for review
`git push`	send further commits on the same branch	update the submission
`git log --oneline --graph --all`	the history of every branch, one line per commit	the revision history block
`git log -p FILE`	every change ever made to a file, with the lines	the file's full change record
`git blame FILE`	who last changed each line, and in which commit	who signed each paragraph
`git merge develop`	bring `develop`'s changes into your branch	incorporate approved changes into your draft
`git merge --abort`	give up a merge that has conflicts and go back	withdraw the incorporation
`git stash` / `git stash pop`	shelve unfinished changes, then bring them back	put a draft in the drawer
`git fetch --prune`	learn about the server's branches; forget deleted ones	update the library catalog
`git branch -d feature/19-stations-level-tags`	delete a local branch after its MR merged	file the finished draft away
`git tag -n baseline-*`	list baselines with their messages	list released baselines
`git show baseline-1`	the tag's message and the commit it marks	read a baseline's record

One-time set-up on each laptop:

```
git config --global user.name "Bob"
git config --global user.email "bob@example.invalid"
git config --global init.defaultBranch main
git config --global core.editor vim
git config --global core.autocrlf false        # the model and the kit are LF text everywhere
git config --global pull.ff only               # never create surprise merge commits on pull
ssh-keygen -t ed25519 -C "bob laptop"          # then paste ~/.ssh/id_ed25519.pub into GitLab:
Preferences, SSH keys
ssh -T git@gitlab.example.com                  # check the key works
```

A.2 gitlab.pl

Help: `perl bin/gitlab.pl help` and `perl bin/gitlab.pl help COMMAND`. One command-line tool for GitLab, with two back ends: a real server (`--url`, with a token) or the kit's offline emulator (`--local DIR`), which behaves like a server for practice, tests and the Tidewater simulation.

```
gitlab.pl [--url URL [--token-env VAR] [--sudo] | --local DIR] [--project PATH] [--as USER]
          [--now ISO] [--dry-run] [--no-cache] COMMAND ...
```

Table A.2: gitlab.pl global options

Option	Meaning
`--url URL`	talk to the GitLab server at URL, with the token in `$GITLAB_TOKEN`
`--token-env VAR`	read the token from another environment variable
`--sudo`	with an administrator token, act as the user named by `--as`
`--local DIR`	use the offline emulator whose state is in DIR (created on first use)
`--project PATH`	the project, such as `tidewater/model`
`--as USER`	act as this user (the emulator: anyone; a server: needs `--sudo`)
`--now ISO`	the emulator's clock, for replaying a dated scenario
`--dry-run`	print each API request instead of sending it
`--no-cache`	the emulator re-runs every pipeline job even when the tree is unchanged

The defaults come from the environment, so a laptop that sets them once can type short commands: `GITLAB_URL`, `GITLAB_LOCAL`, `GITLAB_PROJECT`, `GITLAB_USER`.

A.2.1 Project set-up and plan

```
gitlab.pl setup FILE
gitlab.pl sprints START COUNT [--weeks 2] [--prefix "Sprint "]
gitlab.pl user USERNAME [--name N] [--email E]
```

Table A.3: Set-up commands

Command	What it does
`setup FILE`	apply a project file: settings, members, labels, sprints, protections, approvals, variables; creates the project if needed; safe to re-run
`sprints START COUNT`	COUNT two-week milestones from START (a Monday), each due on the Friday of its last week
`user USERNAME`	create a user (the emulator: anyone; a server: needs an administrator token)

The project file's sections are `[project]`, `[members]`, `[branches]`, `[tags]`, `[labels]`, `[milestones]`, `[approvals]` and `[variables]`; `tools/gitlab/templates/project.conf` is the annotated example, shown in Appendix I, Templates and GitLab Markdown, field by field (p. 747).

A.2.2 Issues

```
gitlab.pl issue new TITLE [--template NAME|FILE] [--description TEXT] [--label L]... [--milestone
M]
                          [--assign USER]... [--type issue|task|incident] [--weight N] [--due
                          DATE]
gitlab.pl issue list [--state opened|closed|all] [--label L]... [--milestone M] [--type T]
gitlab.pl issue show IID | close IID | reopen IID
gitlab.pl issue note IID TEXT | --file FILE
gitlab.pl issue link IID TARGET [--type relates_to|blocks|is_blocked_by]
gitlab.pl issue label IID [--add L]... [--remove L]...
```

`--template NAME` reads `.gitlab/issue_templates/NAME.md` from the current directory, so run it inside a clone of the model repository. A note's quick actions (`/label`, `/milestone`, `/assign`, `/estimate`, `/spend`, `/close` and the others in Appendix C, Quick reference: git, gitflow and the GitLab tools (p. 713)) run as they do on the server.

A.2.3 Merge requests

```
gitlab.pl mr new --source BRANCH [--target BRANCH] --title TITLE [--template NAME|FILE]
[--description TEXT]
                 [--label L]... [--milestone M] [--reviewer USER]... [--keep-branch]
gitlab.pl mr list [--state opened|merged|closed|all] | show IID | approve IID | merge IID
gitlab.pl mr note IID TEXT [--resolvable] | resolve IID
```

The target defaults to the project's default branch (`develop`). `merge` refuses, with GitLab's own reason, until the pipeline has succeeded, the approvals (including code owners) are in and every resolvable discussion is resolved. `mr show` prints the pipeline's report summaries (tests, code quality, metrics), as the MR page's widgets do.

A.2.4 Pipelines, tags, releases, packages, Pages, environments

```
gitlab.pl pipeline run REF [--source web|push|schedule] [--var K=V]...
gitlab.pl pipeline list [--ref REF] [--status S] | show ID
gitlab.pl tag NAME REF [--message M]
gitlab.pl release TAG [--notes FILE] [--name N] [--milestone M]...
gitlab.pl package put NAME VERSION FILE | get NAME VERSION FILE [DEST] | list
gitlab.pl pages
gitlab.pl env list | env deployments [NAME]
gitlab.pl schedule new DESCRIPTION --cron "MIN HOUR DOM MON DOW" --ref BRANCH [--tz ZONE] [--var
K=V]... [--inactive]
gitlab.pl schedule list | run ID | delete ID | due --until ISO | next "CRON" [--tz ZONE] [--from
ISO] [-n N]
gitlab.pl wiki put TITLE FILE | wiki list | wiki show TITLE
gitlab.pl export DIR [--since YYYY-MM-DD] [--until YYYY-MM-DD]
gitlab.pl analytics [--since YYYY-MM-DD] [--until YYYY-MM-DD] [--by sprint|week|month] [--csv
DIR]
```

Table A.4: Pipeline, release and record commands

Command	What it does
`pipeline run REF`	start a pipeline on a branch or tag, as the Run pipeline button does
`pipeline show ID`	the pipeline's jobs and results (on the emulator, each job's log path too)
`tag NAME REF --message M`	an annotated tag; protected tags need the maintainer role
`release TAG --notes FILE`	a release for an existing tag, with Markdown notes
`package put NAME VERSION FILE`	upload a file to the Generic Package Registry; its sha256 is recorded
`package get NAME VERSION FILE`	download it; the emulator checks the sha256
`pages`	where the project's GitLab Pages site is (a `pages` job on the default branch publishes it)
`env list`	the environments jobs have deployed to, such as the model site
`schedule new`	a cron pipeline schedule (`project.conf [schedules]` makes the same at set-up); it runs as its owner
`schedule list, run ID`	the schedules with their next run; run one now without changing its next run
`schedule next "CRON"`	the next run times of a cron line, with daylight saving; needs no server: a way to check a line before using it
`analytics`	value stream, DORA for a model repository, pipelines, merge requests and issues, per sprint; from data every GitLab tier exposes (Chapter 32, Measuring with GitLab analytics (p. 513))
`wiki put TITLE FILE`	create or replace a wiki page from a Markdown file
`export DIR`	`ci_jobs.csv`, `mrs.csv`, `issues.csv`, `defects.csv` and `tracker/sprint-N.csv`: the status metrics' inputs

A.2.5 Gitflow: gitlab.pl flow

Run inside a clone of the model repository. Branch names: `main`, `develop`, `feature/`, `release/`, `hotfix/` and `vendor/prime`.

```
gitlab.pl flow init
gitlab.pl flow start feature|release|hotfix NAME
gitlab.pl flow publish
gitlab.pl flow finish [--title T] [--template NAME|FILE] [--description TEXT] [--label L]...
[--milestone M]
                    [--reviewer U]...
gitlab.pl flow released IID [--notes FILE] [--tag NAME]
gitlab.pl flow vendor DROPDIR VERSION
gitlab.pl flow take ours|theirs PATH...
gitlab.pl flow sync
```

Table A.5: The gitflow steps

Step	What it does
`flow init`	create `develop` from `main` (the first commit on an empty repository)

Step	What it does
`flow start feature NAME`	branch `feature/NAME` from `develop`; `release` also from `develop`; `hotfix` from `main`
`flow publish`	push the current branch; the emulator runs its pipelines
`flow finish`	push and open the MR: feature to `develop`, release and hotfix to `main`
`flow released IID`	after a release or hotfix MR merged: tag `main` (the sha256 of `docs/` in the message), create the release, open the back-merge MR to `develop`
`flow vendor DROPDIR VERSION`	commit the converted prime drop on `vendor/prime`, then merge it into `feature/drop-VERSION`; conflicts are listed, not resolved
`flow take ours|theirs PATH…`	resolve conflicted files wholesale (only for files the government never edits)
`flow sync`	fetch, and fast-forward local `main` and `develop`

A.2.6 Running the pipeline locally: gitlab.pl ci

```
gitlab.pl ci [--file .gitlab-ci.yml] [--event push|merge_request_event|tag|schedule|web|trigger]
[--branch B]
            [--target B] [--tag T] [--var K=V]... [--job NAME] [--manual] [--logs DIR] [--list]
```

Runs the current repository's pipeline on your own machine, choosing jobs the way GitLab would for the event you name. `--list` shows which jobs would run and why the others would not, without running anything, and which file each job comes from when the pipeline is split into included files. The two forms used every day, here in the clone of practice lesson 11:

```
perl "$KIT/bin/gitlab.pl" ci --list --event merge_request_event --target develop
perl "$KIT/bin/gitlab.pl" ci --event merge_request_event --target develop
```

A.3 The kit's journal: daily.pl and scrum.pl

Help: `perl bin/daily.pl help` (the list) and `perl bin/daily.pl help COMMAND`; `perl agile.pl help` for everything. Every command finds `scrum.conf` by searching upward from the current directory, so run them anywhere inside the journal project (`data/tidewater/journal` in the simulation).

Table A.6: daily.pl commands by time of day

When	Commands
Each day	`new compile report draft brief commit all status`
Before the call	`drill propose cards attend invite meetings`
During the call	`lint`
After the call	`answers chat joined ai post`
Views	`sprint velocity backlog members epics roadmap blocked quad health`
Reports	`review rollup csv`

When	Commands
Set-up and checks	`init check roster help`

The ones Tidewater uses every sprint:

Table A.7: The daily.pl commands Tidewater uses

Command	What it does	Vim
`daily.pl init [DIR]`	make a journal project: `scrum.conf`, `scrum.txt`, `standups/`, `reports/`, git	
`daily.pl roster add "Bob" bob@example.invalid Model "MBSE engineer" Gov`	add a person to `roster.txt`	
`daily.pl new`	today's stand-up file, `standups/DATE.txt`, with each team's open stories as comments	`:SNew`
`daily.pl lint [FILE] [--reply]`	check the pasted Y/T/B statuses while people are still on the call	`:SLint`
`daily.pl answers`	parse the statuses, raise flags, suggest done/unblock/absent lines for you to confirm	`:SAnswers`
`daily.pl chat`	tally `#est` (planning poker) and `#vote` rounds from the chat into `est` lines	`:STally`
`daily.pl propose`	draft each person's status from yesterday's, for a `+1` in the chat	`:SPropose`
`daily.pl --dry compile`	show what compile would append, write nothing	`:SDry`
`daily.pl compile`	apply the pending stand-up files to the journal (all or nothing per file)	`:SCompile`
`daily.pl sprint [N]`	capacity, committed, done, open, carryover, load	`:SSprint`
`daily.pl quad [TEAM]`	the weekly one-page quad for management	`:SQuad`
`daily.pl review [N]`	the end-of-sprint report	
`daily.pl epics` / `roadmap`	roll-up by tome and epic; epics across sprints with a forecast	`:SRoadmap`
`daily.pl health`	every metric against its threshold; exit 1 on a red	
`daily.pl report`	every report into `reports/`	`:SReport`
`daily.pl commit`	commit the journal project ("standup DATE")	`:SCommit`

`scrum.pl` is the lower-level view of the same journal, for ad hoc questions:

```
perl bin/scrum.pl [-f JOURNAL] [--today D] COMMAND [N] [ARGS]
    sprint [N] | velocity [--last N] | backlog [TEAM] | members [TEAM] | epics | roadmap [-o
    FILE]
    items [STATE] [TEAM] | dashboard [-o FILE] | tree [-o FILE] | email [N] [--text] | csv
    [TABLE] | check
```

A.4 The converters: xmi2sysml.pl and reqif2sysml.pl

Help: run either with `--help`. Both are in the kit's `bin/`.

```
xmi2sysml.pl [--report] [--out DIR [--layout '{seg}/{pkg}.sysml' | --layout-map FILE] | --stdout]
             [--req-style usage|def] [--name PKG] [--check] [--map MAP.csv] FILE.xmi
xmi2sysml.pl --delta OLD.xmi NEW.xmi [--impact MODEL_DIR [--map OLD-MAP.csv] [--impact-csv FILE]]
```

Table A.8: xmi2sysml.pl options

Option	Meaning
`--report`	the mapping report: element counts by type, what was mapped, what became a TODO line, stereotypes not converted
`--out DIR`	write one `.sysml` file per top-level package under DIR
`--layout`	the file path pattern under DIR
`--layout-map FILE`	which folder and file each top-level package goes to (`Package = folder/`), the same map for every drop so the vendor merges stay three-way; `examples/tidewater-v1/layout.map` puts Tidewater in layers
`--map MAP.csv`	also write, for every v1 element, where it landed in the v2 text (file, line, or the TODO line it sits under)
`--stdout`	print the converted text instead of writing files
`--req-style`	`usage` (default: the one-line requirement form) or `def` (`requirement def` blocks)
`--check`	also run the grammar check on the output
`--delta OLD NEW`	compare two drops of the same model by `xmi:id`: added, removed, changed; each marked *carried* by the converter or *manual*; grouped by the prime's change requests when the drop has a `CHANGES.csv`
`--impact MODEL_DIR`	with `--delta`: what each change touches in the government's model (requirements, verification cases, interfaces, allocations, references) and which hand edits must be redone

```
reqif2sysml.pl [--report] [--model DIR] [--out DIR [--layout '{seg}/{pkg}.sysml' | --layout-map
FILE] | --stdout]
               [--csv [--col id=NAME --col text=NAME --col heading=NAME --col level=NAME --col
               vm=NAME] [--sep C]]
               [--req-style usage|def] [--id-attr NAME] [--vm-attr NAME] [--id-prefix P] [--attr
               NAME]... FILE
```

Table A.9: reqif2sysml.pl options

Option	Meaning
`--report`	objects, requirements, headings, links and verification methods in the export
`--model DIR`	trace the export against an existing SysML v2 model: linked (C), orphans (F), unverified (G), completeness (I) and text drift
`--out DIR`	write the requirements as SysML v2 text, one file per DOORS module

Option	Meaning
`--csv`	read a DOORS CSV export instead of ReqIF; `--col` names the columns if they are not the usual ones
`--id-attr`, `--vm-attr`	which DOORS attributes hold the identifier and the verification method

The runs every drop gets, here on drops 1 and 2 made with the kit's generator (the first two lines stand in for the prime's deliveries):

```
perl "$KIT/sim/tidewater-v1.pl" --drop 1 drops/v1.0
perl "$KIT/sim/tidewater-v1.pl" --drop 2 drops/v1.1
perl "$KIT/bin/xmi2sysml.pl" --report --out v2 --check drops/v1.1/tidewater.xmi
perl "$KIT/bin/reqif2sysml.pl" --report --model v2 drops/v1.1/tidewater.reqif
perl "$KIT/bin/xmi2sysml.pl" --delta drops/v1.0/tidewater.xmi drops/v1.1/tidewater.xmi
```

On a project, the delta also runs with `--impact model` against the government's own model; Chapter 45, The prime's model changes: receive, assess, disposition (p. 619) shows its report.

A.5 The model tools: model.pl and sysml.pl

Help: `perl tools/sysml/model.pl help`. `model.pl` works on the project that holds a `model/` folder, found upward from the current directory (or `--root DIR`), configured by that project's `sysml.conf`.

Table A.10: model.pl commands

Command	What it does	Vim
`model.pl lint`	structural checks: grammar, unresolved types, imports, duplicates; ends with `elements=N errors=E warnings=W`	`\ml`
`model.pl check`	the 2-to-9 outline rule and the `@L` level tags; exit 1 on a violation	`\mk`
`model.pl stats`	size, nesting, tree depth, outline	
`model.pl nouns` / `verbs`	the part tree / the function tree (`--plain` for words)	`\mn` / `\mb`
`model.pl docs`	regenerate `docs/`: Nouns.md, Verbs.md, idef0/, Traceability.md	`\md`
`model.pl idef0`	regenerate only `docs/idef0/`: the IDEF0 plates drawn from the verb tree (`idef0_models` in `sysml.conf`)	
`model.pl trace`	requirement, satisfied by, verified by, to `docs/Traceability.md` (`--strict`: exit 1 on gaps)	`\mr`
`model.pl tags`	a Vim tags file: Ctrl-] jumps to any definition or requirement id	`\mt`
`model.pl rename OLD NEW`	rename a definition everywhere (dry run unless `--write`)	
`model.pl draw tree|trace|ibd|pkg`	an SVG view of the model	`\mw \mq \mi \mp`

Command	What it does	Vim
`model.pl diff --git OLD.. NEW`	changes between two versions as coloured views and a change list	
`model.pl gate`	the validate gate in one line: text, trace, markings, zones, threats; PASS or FAIL	`\mg`
`model.pl precommit` / `install-hooks`	what the git pre-commit hook runs / install that hook	

```
perl tools/sysml/sysml.pl check FILE... | corpus [DIR...] | tokens FILE | grammar | crosscheck
[N] [SEED]
```

`sysml.pl check` checks each file against the official SysML v2 grammar and prints `FILE:LINE: COL: error: …` for each problem, a form Vim's quickfix list reads directly (`\mx` checks the current file).

A.6 The pipeline's helpers

Every job in the model repository's `.gitlab-ci.yml` runs one shell-neutral line, `perl ci/job.pl NAME`, where NAME is the job (`grammar`, `lint`, `outline`, `todo`, `trace`, `vcrm`, `recipe`, `complexity`, `docs`, `delta`, `baseline`, ...; `perl ci/job.pl` alone lists them all). The script calls the tools below and the kit's model tools through `KIT`, so the pipeline runs unchanged under PowerShell, Git Bash or a container. To run one job's work by hand in a clone: `perl ci/job.pl lint`.

```
tools/gitlab/ci-reports.pl grammar|lint|outline|todo|trace [--junit F] [--codequality F]
[--metrics F] LOG|DIR [FILE...]
tools/gitlab/pages-index.pl [--title T] [--commit SHA] [--date D] DIR
```

`ci-reports.pl` turns a check's output into the report files GitLab shows on a merge request: JUnit XML for the test widget, Code Quality JSON for the code quality widget (one finding per error, warning or TODO line) and metrics text. It only reports; the job still passes or fails on the check's own exit status. `pages-index.pl` writes the front page of the model site that the `pages` job publishes.

A.7 Week zero, words, memos and requirement text

```
perl tools/gitlab/kickoff.pl doctor [--quick] [--fix] [--local DIR | --url URL [--token-env VAR]]
perl tools/gitlab/kickoff.pl scaffold DIR [--project GROUP/NAME] [--title T] [--noun-root N]
[--verb-root V] [--levels L1A,L2B,...] [--force]
perl tools/gitlab/kickoff.pl start [--yes] [--from N] ...
perl tools/gitlab/talk.pl [--local DIR | --url URL] [--project GROUP/NAME] standup --as USER
[--date D] [--journal DIR] [--why]
perl tools/gitlab/talk.pl ... review --sprint N [--md] | weekly [--date D] [--md] | translate
FILE|- [--glossary FILE]
perl tools/memo/memo-build.pl [--check] [--out DIR] [--pdf-only|--text-only] FILE.md... [--
md2memo options]
perl tools/sysml/req-recipe.pl [--format text|csv|junit|codequality] [--require classification]
[--strict] [-o FILE] INPUT...
```

Table A.11: Week zero, drafting and checking words

Command	What it does
`kickoff.pl doctor`	checks a laptop and prints PASS, WARN or FAIL with the fix for each problem; changes nothing unless `--fix` (git settings only)
`kickoff.pl scaffold DIR`	makes a new model repository from the kit's templates and commits it
`kickoff.pl start`	week zero, step by step: each step is printed with its reason and runs only with `--yes`
`talk.pl standup`, `review`, `weekly`	drafts a stand-up line, a sprint review script or a weekly note from GitLab; it never posts anything (Chapter 25, Talking about progress: helps for engineers who would rather not (p. 390))
`talk.pl translate`	lists the jargon in a draft with plain alternatives and flags sentences with no number or outcome
`memo-build.pl FILE.md`	a memo written from a template, as PDF and text (Chapter 37, Writing it down: DoD-style memos from Vim (p. 545))
`req-recipe.pl INPUT`	grades requirement text (SysML v2, DOORS ReqIF or CSV, plain text) against the requirement recipe (Chapter 7, Requirements a pipeline can check: the recipe (p. 73))

A.8 Status metrics

```
perl bin/status-metrics.pl init    --config metrics.json --xmi legacy.xmi --reqif export.reqif
perl bin/status-metrics.pl collect --config metrics.json --as-of YYYY-MM-DD [--history
history.jsonl]
perl bin/status-metrics.pl report  --config metrics.json --history history.jsonl --out DIR
```

`init` freezes the totals (v1 elements, DOORS requirements); `collect` takes one day's snapshot from the model repository and the exports `gitlab.pl export` writes; `report` writes the weekly table, the daily CSV and the dashboard data.

A.9 The kit itself

```
perl tests/run.pl                 # every test suite, in parallel; one line each
perl agile.pl help                # the whole offline help; agile.pl help TOPIC, help search
WORDS
perl agile.pl help error "MSG"    # what an error message means and how to fix it
perl sim/tidewater-v1.pl --drop N DIR      # generate the prime's drop N of Tidewater into DIR
perl sim/primer.pl --check        # check this book's source
```

Before upgrading Git for Windows on the team, run `perl tests/run.pl` from Git Bash on one laptop with the new version; every line should end "all N passed".

A.10 Vim

The Vim commands and keys are in Appendix B, Quick reference: Vim for MBSE DevSecOps (p. 709). The ones that run the tools above:

Table A.12: Vim keys that run the tools

Keys	Command	Runs
`\ml`	`:SysLint`	`model.pl lint`, errors to the quickfix list
`\mk`	`:SysCheck`	`model.pl check`
`\mx`	`:SysSyntax`	`sysml.pl check` on the current file
`\md`	`:SysDocs`	`model.pl docs`
`\mr`	`:SysTrace`	`model.pl trace`, in a split
`\sn`	`:SNew`	`daily.pl new`
`\sl`	`:SLint`	`daily.pl lint`
`\st`	`:STally`	`daily.pl chat`
`\sc`	`:SCompile`	`daily.pl compile`

CHAPTER

B

Quick reference: Vim for MBSE DevSecOps

Everything here runs in the Vim that ships with Git for Windows (type `vim` in Git Bash). `\` is the leader key: `\ml` means press `\`, then `m`, then `l`. One-time set-up: add `syntax on` and `source ~/projects/agile/vim/scrum.vim` (your clone's path) to `~/.vimrc`. Offline help: `:help agile`, `:help :SysLint`, `:help \ml`. The kit's full cheat sheet is `docs/VIM-CHEATSHEET.html`.

B.1 Survival

Table B.1: Survival Vim

Keys	Does
`Esc`	back to Normal mode; when in doubt, press it
`i a o O`	insert before cursor / after / new line below / above
`:w :q :wq :q!`	save / quit / both / quit without saving
`u Ctrl-r`	undo / redo
`h j k l, w b, 0 $`	left down up right; word forward/back; line start/end
`gg G :42`	top / bottom / line 42
`Ctrl-d Ctrl-u`	half a page down / up
`x dd dw D`	delete character / line / word / to end of line
`yy p P`	copy line / paste after / before
`cw cc .`	change word / change line / repeat last change
`v V Ctrl-v`	select characters / lines / a column block
`> <` (in Visual)	indent / outdent the selection
`/text n N`	search forward; next; previous
`*`	search for the word under the cursor
`:%s/old/new/gc`	replace everywhere, asking each time
`:e FILE :e #`	open a file / back to the previous one
`:sp :vsp Ctrl-w w`	split / vertical split / next window
`:ls :b N`	list open files / go to file N
`Ctrl-] Ctrl-t`	jump to a definition (needs `\mt` tags) / back
`Ctrl-o Ctrl-i`	back / forward through jump positions
`:set list`	show tabs and line ends (spot stray whitespace)

B.2 SysML v2 files (`*.sysml`)

Table B.2: The kit's SysML keys

Keys	Command	Does
`\ml`	`:SysLint`	lint the whole model; errors to the quickfix list
`\mk`	`:SysCheck`	the 2-to-9 outline rule and the `@L` level tags

Keys	Command	Does
`\mx`	`:SysSyntax`	this file against the official SysML v2 grammar
`\mv`	`:SysValidate`	full compile with the Pilot engine (only where Java is installed)
`\md`	`:SysDocs`	regenerate `docs/`
`\mr`	`:SysTrace`	requirement, satisfied by, verified by, in a split
`\mn` `\mb`	`:SysNouns` `:SysVerbs`	the part tree / the function tree
`\mt`	`:SysTags`	build the tags file; then `Ctrl-]` on a name or DOORS id
`\mw` `\mq` `\mi` `\mp`	`:SysDraw tree` (trace, ibd, pkg)	draw a view to an SVG
`\mg`	`:SysGate`	the validate gate; findings to the quickfix list
`\mW`	`:SysView`	the live diagram viewer beside Vim; redraws on save
`\mf` `\mo`		viewer: highlight the name under the cursor / open the file:line last clicked

B.3 The quickfix list: from error to line

Table B.3: Quickfix

Keys	Does
`\ml` or `:make`	run the check; each error becomes a quickfix entry
`:copen` `:cclose`	show / hide the list of errors
`:cn` `:cp`	next / previous error (jumps to the file and line)
`:cc 3`	go to error 3
`Enter` (in the list)	jump to that error
`:cnewer` `:colder`	newer / older error lists

The checkers print `FILE:LINE:COL: error: MESSAGE`, which Vim reads. To load a pipeline job's log into quickfix, save the log and run `:cfile job.log`.

B.4 Merge conflicts

A conflicted file has blocks like this; you keep the right text and delete the three marker lines.

```
<<<<<<< HEAD
       @L4Subsystem;   (your branch's version)
=======
       // TODO stereotype Tidewater_Profile:L4Subsystem on this element (...)   (the other side)
>>>>>>> vendor/prime
```

Table B.4: Conflicts in Vim

Keys	Does
`/^[<=>]\{7}` then `n`	find the next conflict marker
`dd`	delete a marker line or an unwanted line
`V` then `j`...`d`	delete a block of lines
`\mx`	after editing: check the file still parses
`git mergetool --tool=vimdiff`	three-way view; `:diffget LOCAL`, `:diffget REMOTE`, `:diffget BASE` take a side for the hunk under the cursor; `]c` `[c` next / previous difference

To list every marker line in the file at once:

```
:g/^\(<<<<<<<\|=======\|>>>>>>>\)/p
```

B.5 Git from Vim

Table B.5: Git from Vim

Keys	Does
`:!git status`	run any git command and see its output
`:!git diff %`	your changes to the current file
`:!git log --oneline -10 -- %`	the last ten commits that touched this file
`:r !git log -1 --format=\%H`	insert the last commit id below the cursor
`:terminal`	a shell in a Vim window (`Ctrl-w N` to scroll it, `i` to type again)
`:!git blame -L 40,60 %`	who changed lines 40 to 60
`:e .git/COMMIT_EDITMSG`	(inside `git commit`) the message buffer: write it, `:wq`

B.6 Issue and merge request Markdown

Write long descriptions in Vim, then paste or pass them with `--file` / `--description`.

Table B.6: Markdown for GitLab, in Vim

Keys	Does
`:e mr.md`	a scratch file for the description (Markdown highlighting by name)
`:r tools/gitlab/templates/.gitlab/merge_request_templates/Default.md`	read a template in below the cursor
`:setlocal spell ]s z=`	spell check; next misspelling; suggestions
`gqip`	re-wrap the paragraph
`:setlocal tw=0`	no hard wrapping (GitLab joins lines anyway)
`- [ ]`	a task list item (`- [x]` done)
`"+y "+p`	copy to / paste from the Windows clipboard
`:!perl "$KIT/bin/gitlab.pl" issue note 19 --file %`	post this file as a comment on issue 19 (see Appendix C, Quick reference: git, gitflow and the GitLab tools (p. 713))

B.7 Journal and stand-up keys

Table B.7: Journal keys

Keys	Command	Does
`\sn`	`:SNew`	today's stand-up file
`\ss`	`:SStatus`	the journal at a glance
`\sl`	`:SLint`	save and lint the pasted Y/T/B statuses
`\sp`		paste the chat from the clipboard and lint it
`\sy`		the replies to the clipboard, ready to paste into the chat
`\sw`	`:STown`	the meeting layout: chat, lint table, replies
`\st`	`:STally`	save and tally `#est` rounds (planning poker)
`\sd`	`:SDry`	save and show what compile would write
`\sc`	`:SCompile`	save and apply the stand-up file
`\sr`	`:SReport`	write the reports

Keys	Command	Does
`\sj`	`:SJournal`	open the journal
`\sm`	`:SDrill`	the memory drill sheet
	`:SAnswers` `:SPropose` `:SQuad` `:SSprint` `:SCommit`	answers, proposals, the weekly quad, the sprint, commit

CHAPTER

C

Quick reference: git, gitflow and the GitLab tools

Commands assume Git Bash and the kit at `$KIT` (for example `export KIT=/c/agile`). `gitlab.pl` needs `--url` and a token in `$GITLAB_TOKEN` for a real server, or `--local DIR` for the offline emulator; set `GITLAB_URL` and `GITLAB_PROJECT` once in `~/.bashrc` to shorten every line. Full syntax: Appendix A, Command reference (p. 696).

C.1 git, every day

Table C.1: git every day

Command	Does
`git status`	where am I, what changed
`git switch develop && git pull`	start from the latest integrated model
`git switch -c feature/19-stations-level-tags`	a branch for one task (issue 19)
`git diff` / `git diff --staged`	unstaged / staged changes
`git add -p FILE`	stage changes hunk by hunk, answering y/n
`git restore FILE` / `git restore --staged FILE`	discard a change / unstage it
`git commit`	record it; Vim opens for the message
`git push -u origin HEAD`	first push of a new branch
`git merge develop`	bring `develop` into your branch (do it daily on a long task)
`git merge --abort`	back out of a conflicted merge
`git stash` / `git stash pop`	shelve / restore unfinished work
`git log --oneline --graph --all -20`	the recent history of every branch
`git fetch --prune` / `git branch -d B`	tidy after a merge
`git show baseline-1`	a baseline's tag message and commit

Never: `git push --force` on a shared branch, `git rebase` of pushed commits, commits on `develop` or `main`.

C.2 Gitflow, by branch type

Table C.2: Gitflow branches

Branch	From	Merges into	Who	Steps
`feature/IID-slug`	`develop`	`develop`	anyone	`flow start feature IID-slug`; commit; `flow finish --template Default`; review; merge

Branch	From	Merges into	Who	Steps
`release/baseline-N`	`develop`	`main`	maintainer	`flow start release baseline-N`; only fixes; `flow finish --template Release`; merge; `flow released IID` (tag, release, back-merge)
`hotfix/slug`	`main`	`main`, then `develop`	maintainer	`flow start hotfix slug`; fix; `flow finish --template Hotfix`; merge; `flow released IID`
`vendor/prime`	first commit of `main`	`feature/drop-VERSION`	maintainer (import only)	`flow vendor DROPDIR VERSION`; resolve conflicts; `flow finish --template "Prime Drop"`
`develop`	`main`	(receives MRs)	nobody pushes	protected; the default branch
`main`		(receives release and hotfix MRs)	nobody pushes	protected; only baselines

All `flow` commands are `perl "$KIT/bin/gitlab.pl" flow …`, run inside a clone of the model repository.

C.3 gitlab.pl at a glance

Table C.3: gitlab.pl at a glance

Goal	Command
Apply the project settings	`gitlab.pl setup project.conf` (`--dry-run` first)
Make sprint milestones	`gitlab.pl sprints 2026-10-05 39`
New story from the template	`gitlab.pl issue new "Close the Stations TODO lines" --template Story --milestone "Sprint 4"`
New task	`gitlab.pl issue new "Level tags in Stations" --template Task --type task --label type::task`
Comment with quick actions	`gitlab.pl issue note 19 "/spend 3h"`
List my open issues	`gitlab.pl issue list --label workflow::in-progress`
Open an MR	`gitlab.pl mr new --source feature/19-x --title "…" --template Default --reviewer jim`
Approve / merge	`gitlab.pl mr approve 19` / `gitlab.pl mr merge 19`
Pipeline status	`gitlab.pl pipeline list --ref develop` / `pipeline show ID`
Run the pipeline locally	`gitlab.pl ci --event merge_request_event --target develop`
What would run	`gitlab.pl ci --list --event merge_request_event --target develop`
Upload a prime drop	`gitlab.pl package put prime-drop v1.1 tidewater.xmi`
Baseline release	`gitlab.pl flow released IID --notes release-notes.md`
Status metrics inputs	`gitlab.pl export exports/ --since 2026-10-05`
Pipeline schedules	`gitlab.pl schedule list` / `schedule run ID` (defined in `project.conf` `[schedules]`)
Check a cron line	`gitlab.pl schedule next "0 2 * * 1-5" --tz America/New_York -n 3` (needs no server)
Numbers for a review	`gitlab.pl analytics --by sprint` (value stream, DORA, pipelines, MRs, issues)
Practise offline	add `--local /tmp/gl` to any command

C.4 Quick actions

Type them on their own line in any issue or MR description or comment; they run when you save and disappear from the text.

Table C.4: Quick actions

Quick action	Does	Where
`/label ~"type::story" ~"workflow::ready"`	add labels (a scoped label replaces its sibling)	issue, MR
`/unlabel ~"workflow::blocked"`	remove a label	issue, MR
`/milestone %"Sprint 4"`	set the sprint	issue, MR
`/assign @bob` `/unassign @bob`	set / clear the assignee	issue, MR
`/assign_reviewer @jim`	ask for a review	MR
`/estimate 2d`	the task's size (1 point = 1 ideal day)	issue, MR
`/spend 3h`	time spent (`/spend -1h` corrects)	issue, MR
`/weight 3`	story points on an issue (Premium)	issue
`/due 2026-11-20`	due date	issue
`/relate #8 #11`	link related issues	issue
`/close` `/reopen`	close / reopen	issue, MR
`/approve`	approve the MR	MR
`/merge`	merge when the pipeline succeeds	MR
`/draft` `/ready`	mark the MR draft / ready	MR
`/duplicate #12`	close as a duplicate	issue
`/confidential`	make the issue confidential	issue

C.5 References and Markdown

Table C.5: References and GitLab Markdown

Type	Becomes
`#19`	a link to issue 19 (`tidewater/model#19` from another project)
`!19`	merge request 19
`%"Sprint 4"`	the milestone
`~"type::story"`	the label, as a coloured chip
`@bob`	a mention: Bob is notified
`9f3c1e2`	a commit (7 or more hex characters)
`Closes #19` (in an MR description)	closes issue 19 when the MR merges into the default branch
`- [ ] item` / `- [x] item`	a task list item, clickable
`[[_TOC_]]`	a table of contents of the headings
`<details><summary>Log</summary> … </details>`	a collapsed section
`> [!note]`	an alert box (also `[!warning]`, `[!tip]`)
a fenced block tagged `mermaid`	a diagram drawn from text
a fenced block tagged `suggestion:-0+1`	in a review comment: a change the author applies with one click
`[^1]` and a line `[^1]: text`	a footnote

C.6 Pipeline: triggers and rules

Table C.6: Pipeline sources

`$CI_PIPELINE_SOURCE`	When	Tidewater runs
`merge_request_event`	an MR is opened or its branch is pushed	the checks on the merged result
`push`	a push to a branch without an open MR (`develop`, `main`, `vendor/prime`)	the checks; `pages` on `develop`
(a tag, `$CI_COMMIT_TAG` set)	a tag is created	`baseline` and `publish-release` on `baseline-*`
`schedule`	a pipeline schedule fires	nightly full check (`FULL=1`): every check and `pages`; weekly: `status` only
`trigger`	the prime's Jenkins starts a pipeline with a trigger token and `DROP_VERSION`	`intake` only
`web`	someone presses Run pipeline	as for the branch

```
rules:
  - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'      # MR pipelines
  - if: '$CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH'            # develop
  - if: '$CI_COMMIT_TAG =~ /^baseline-/'                     # baselines
  - if: '$CI_PIPELINE_SOURCE == "schedule"'
    when: never                                              # not on the nightly
  - when: on_success                                         # otherwise
```

Table C.7: Pipeline keywords

Keyword	Means
`stage`	which column; stages run in order
`script`	the commands; any non-zero exit fails the job
`rules` / `when`	whether the job runs: `on_success`, `always`, `manual`, `never`
`allow_failure: true`	a failure shows yellow and does not block the merge
`artifacts: paths`	files kept after the job (download from the job page)
`artifacts: reports`	`junit`, `codequality`, `metrics`: shown on the MR
`artifacts: when: always`	keep them even when the job fails
`needs`	start as soon as these jobs finish
`extends` / `&anchor` `<<: *anchor`	reuse settings from a hidden `.job`
`environment`	record a deployment (the model site)
`include: template:`	GitLab's own job definitions (Secret Detection)

Tidewater's jobs keep every script line shell-neutral: `script: [perl ci/job.pl lint]`. The Perl script does the work, so the same file runs under PowerShell on the Windows shell runner, Git Bash on a laptop and the optional Docker image.

Every job starts in a fresh checkout: use `artifacts` to pass files to later stages and to keep evidence; `cache` only speeds things up and may be empty.

C.7 Variables and tokens

Table C.8: Variables and tokens

What	Use for	Setting
CI/CD variable, Protected	secrets for `develop`, `main`, tags only	Settings, CI/CD, Variables

What	Use for	Setting
CI/CD variable, Masked	hidden as `[MASKED]` in job logs	same
`KIT`	where the kit is on the runner	project variable or runner `environment`
Runner tags	which runner takes a job: `windows`, `shell`, `mbse`; `release` for the protected runner	job `tags:`; Settings, CI/CD, Runners
`CI_JOB_TOKEN`	API calls from inside a job (package upload)	automatic, expires with the job
Project access token	automation (the nightly export), scope `read_api`	Settings, Access tokens
Personal access token	your own `gitlab.pl` from a laptop, in `$GITLAB_TOKEN`	Preferences, Access tokens
Deploy token	read-only clone or package download from outside	Settings, Repository
SSH key	`git push` / `git pull` from your laptop	Preferences, SSH keys

Never put a token in a file in the repository. A leaked token: revoke first, then everything else (Chapter 42, Administration (p. 586)).

C.8 Where things are in GitLab

Menu names follow the left sidebar of GitLab 16 and 17; older versions differ slightly.

Table C.9: Where things are

You want	Go to
Issues, the board, sprints	Plan, Issues / Issue boards / Milestones
The wiki	Plan, Wiki
Merge requests	Code, Merge requests
Files, branches, tags, history	Code, Repository / Branches / Tags / Commits
Pipelines, job logs	Build, Pipelines / Jobs
Edit and validate `.gitlab-ci.yml`	Build, Pipeline editor (Validate, Visualize, Full configuration)
The nightly schedule	Build, Pipeline schedules
Releases, packages, the model site	Deploy, Releases / Package Registry / Pages
Environments	Operate, Environments
Members, labels	Manage, Members / Labels
Audit events (Premium)	Secure, Audit events
Merge method, squash, approvals	Settings, Merge requests
Protected branches and tags	Settings, Repository
Variables, runners	Settings, CI/CD
Tokens	Settings, Access tokens

CHAPTER

D

Quick reference: SysML v1 to v2

The converters are `bin/xmi2sysml.pl` (Cameo XMI to SysML v2 text) and `bin/reqif2sysml.pl` (DOORS ReqIF or CSV to requirements, and the trace). Nothing in the v1 model is dropped silently: what the converter cannot carry becomes a `// TODO` line naming the v1 type and its `xmi:id`, and the `--report` lists it. The kit's one-sheet port card is `docs/SYSML-PORT-QRG.pdf`; this appendix adds the hand fixes and the drop cycle.

D.1 What maps to what

Table D.1: The converter's mapping (from lib/Xmi.pm)

SysML v1 (XMI)	SysML v2 text written by xmi2sysml
Package, Model (top level)	`package` (one file per top-level package)
Block, Class, Component, Actor	`part def`; Generalization becomes `:>`
InterfaceBlock (or a Class used only as a port type)	`port def`; FlowProperty becomes an `in`/`out`/`inout` feature
ConstraintBlock	`constraint def` with its parameters; the expression becomes a TODO
Signal	`item def`
DataType, ValueType, PrimitiveType	`attribute def` (UML primitives map to `ScalarValues::*`)
Enumeration	`enum def`
Part property (composite, Block-typed)	`part n : T[m..n];`
Reference property (non-composite, Block-typed)	`ref part`
Value property (value-typed or untyped)	`attribute` (default value kept)
Signal-typed property	`item`
Port, ProxyPort, FullPort, FlowPort	`port n : PortDef;` (conjugated: `~PortDef`); atomic flow ports become `port { in attribute … }`
Connector	`connection NAME connect a.p to b.q;`
BindingConnector	`bind a = b;`
Activity	`action def` with parameters, call actions, control flows as `first/then`
UseCase	`use case def` (skeleton)
Requirement and Cameo's requirement kinds	`requirement` with `doc`, `doorsId`, `title`, `verificationMethod`
TestCase	`verification def`
Satisfy	`satisfy R by X;`
Verify	`verify R;` inside the test case's `verification def` objective
DeriveReqt	a `#derivation connection`

SysML v1 (XMI)	SysML v2 text written by xmi2sysml
Trace, Refine, Copy, Dependency, Usage, Realization, Abstraction	`dependency from A to B;`
Allocate	`allocate A to B;`
ownedComment	`doc /* … */`
Diagrams, `xmi:Extension`	counted and skipped (redraw views from the text)

Requirements are named from their DOORS id by both converters (`TW-SYS-001` becomes `TW_SYS_001`), so satisfy and verify written by one resolve against the other.

D.2 The TODO lines and their hand fixes

Table D.2: TODO lines and their fixes

The converter writes	Means	Hand fix (in place)
`// TODO stereotype Profile: L3Station on this element (xmi:id …) not converted`	a project-profile stereotype	`@L3Station;` with a `metadata def L3Station;` in `model/library/conventions/Conventions.sysml` and `private import Conventions::*;`
`// TODO stereotype Profile: Procured … not converted`	a stereotype with tagged values	`@Procured { supplier = "…"; partNumber = "…"; }` and a `metadata def` with those attributes
`// TODO uml:Instance Specification 'watt' …`	a unit	`attribute def Power :> ISQ::PowerValue;` (specialize the ISQ quantity)
`// TODO uml:StateMachine 'BuoyModes' …`	a state machine	a hand-written `state def` (below)
`// TODO uml:Interaction 'DailyUpload' …`	a sequence diagram	`message … of T from a to b;` in the context part def
`// TODO uml:Include …`	a use case include	`include use case x : X;`
`// TODO uml:ForkNode …: activity node not converted`	fork, join, decision, merge	`fork f;` with `first a then f; first f then b;`
`// TODO uml:InputPin …: pin (object flow) not converted`	a pin	an `in`/`out` parameter on the action def
`// TODO uml:ObjectFlow …: activity edge not converted …`	an object flow	`flow a.out to b.in;`
`// TODO uml:ControlFlow …: activity edge not converted …`	a control flow touching a fork or pin	`first a then b;`
`// TODO constraint expression (parametrics) not converted` and `// TODO uml:Constraint …: constraint: LANG: TEXT`	the constraint's expression, shown as text	write the expression in the `constraint def`
`// TODO uml:Profile …`	the project profile itself	replaced by `model/library/conventions/Conventions.sysml`; close with a comment

The converter writes	Means	Hand fix (in place)
`// TODO type … not resolved`	a type reference the export does not contain	find the type (often another export) or raise a DR

Find every TODO in Vim: `:vimgrep /\/\/ TODO/ model/**/*.sysml` then `:copen`. Close a TODO only with the construct that replaces it; never delete one without its replacement.
Hand-fix patterns, each checked against the SysML v2 grammar with `sysml.pl check`:

```
@L3Station;
@Procured { supplier = "Notional Logger Co."; partNumber = "LG-100"; }

include use case queryArchive : QueryArchive;

state def BuoyModes {
    entry; then Observing;
    state Observing;
    state Degraded;
    transition toDegraded first Observing accept lowBattery then Degraded;
    transition toObserving first Degraded accept charged then Observing;
}

fork afterCheck;
first check then afterCheck;
first afterCheck then store;
flow check.record to store.record;

constraint def EnergyBalance {
    in generation : Real; in sunHours : Real; in load : Real;
    generation * sunHours >= load * 24
}

message sendObservations of ObservationRecord from buoy to ingest;
```

D.3 Converter options

Table D.3: Converter options

Command	Use
`xmi2sysml.pl --report FILE.xmi`	read first: counts by type, ported versus remaining, the TODO list, stereotypes not converted
`xmi2sysml.pl --out DIR --check FILE.xmi`	write the files (one per top-level package) and grammar-check them
`xmi2sysml.pl --stdout FILE.xmi`	all files to the screen (`… | vim -`)
`xmi2sysml.pl --req-style def`	`requirement def` blocks instead of the default one-line usages
`xmi2sysml.pl --delta OLD.xmi NEW.xmi`	two drops compared by `xmi:id`: added, removed, changed; *carried* or *manual*
`xmi2sysml.pl --delta OLD NEW --impact model`	the same, plus what each change touches in your v2 model and which hand edits to redo
`xmi2sysml.pl --out DIR --layout-map FILE`	put each package in its layer folder (`Package = folder/`); the same map for every drop
`xmi2sysml.pl --out DIR --map MAP.csv`	also record where every v1 element landed in the v2 text
`reqif2sysml.pl --report FILE.reqif`	objects, requirements, headings, links, verification methods
`reqif2sysml.pl --out DIR FILE.reqif`	requirements as SysML v2 text, one file per DOORS module

Command	Use
`reqif2sysml.pl --report --model DIR FILE.reqif`	the trace: C, F, G, I and drift against the v2 model
`reqif2sysml.pl --csv FILE.csv`	a DOORS CSV export; `--col id=NAME --col text=NAME` if the columns are unusual

Exit codes for both: 0 ok; 1 the self-check or `--check` found a problem; 2 bad usage or an unreadable file.

D.4 Reading the v1 delta

```
xmi delta: tidewater.xmi (v1.1) -> tidewater.xmi (v1.2)
elements 567 -> 570: 12 added, 9 removed, 0 changed
carried by the converter (re-convert the drop and merge it): 16
manual (the converter does not carry these; rework the v2 text by hand): 5

added:
  + uml:State              Stations::MooredBuoy::BuoyModes::Storm
      manual: inside uml:StateMachine 'BuoyModes', which the converter leaves as a TODO in
      stations/Stations.sysml; rework the v2 text by hand
  + uml:Property           Stations::PowerSystem::fuse
      carried: stations/Stations.sysml
```

Table D.4: Delta marks

Mark	Means	Action
`carried: FILE`	the vendor import already contains this change	review it in the merge
`manual: a TODO line in FILE`	the element itself is not converted	a task to update the hand-written replacement
`manual: inside X, which the converter leaves as a TODO`	a change inside a construct the converter does not carry; **invisible in the converted text**	a task: the TODO count will not show it
`~ … -- renamed 'Old' -> 'New'` (same `xmi:id`)	a rename in v1, carried	check every government edit that names the old name (lint fails on it)
`~ … -- properties changed`	an attribute of the element changed	review it; manual if inside a TODO construct
`manual rework list: [ ] …`	the delta's own checklist of manual items, with `xmi:id`	paste into the drop story as tasks

D.5 The drop cycle

Table D.5: The drop cycle

Step	Command or place	Done when
1 Receive	files to a dated folder outside the repo; `sha256sum` against `MANIFEST.txt`	every hash matches
2 Record	issue from the *Prime Drop* template, current sprint	issue open, checklist pasted
3 Store	`gitlab.pl package put prime-drop VERSION FILE` (or the `drop-package` job)	package listed with its sha256
4 Import	`gitlab.pl flow vendor DROPDIR VERSION`	`vendor/prime` has the new commit; `feature/drop-VERSION` exists

Step	Command or place	Done when
5 Delta	`xmi2sysml.pl --delta OLD NEW` (the `delta` job)	each *manual* line is a task
6 Conflicts	per file: take the vendor side, re-apply the government edit; check with `\mx`	no conflict markers; lint 0/0
7 MR	`gitlab.pl flow finish --template "Prime Drop"`	delta summary and conflict table in the description
8 Trace	the `trace` job	fixed DRs closed, new gaps raised as DRs
9 Merge	code owner approval; merge	`develop` green; `CHANGELOG.md` updated; issue closed

D.6 Trace measures

Table D.6: Trace measures

Measure	Meaning	From
C linked	DOORS requirements whose `doors Id` is in the model	`reqif2sysml.pl --report --model`
F orphans	requirements nothing satisfies	same
G unverified	requirements no verification case verifies	same
I completeness	percent both satisfied and verified	same
drift	`doc` text in the model differs from DOORS (normalized)	same; line "text differs from DOORS" and a `drift:` list
in model, not in export	`doorsId` values the model has and DOORS does not	same

A drifted, orphaned or unverified requirement is never fixed by editing the model's requirement text: it is a discrepancy report to the prime, closed by the drop that fixes it (Section 51.2.4, Changing requirement text in the model (p. 686)).

CHAPTER

E

Rescue: when something goes wrong

From what you know →

You know

the restore-from-backup request to IT.

→ Here it is called

rescue recipes for git and Vim.

→ What is new

almost every mistake can be undone by you, in a minute, because nothing committed is ever really lost.

Survival kit: Before you panic

What you have

a mistake. A commit on the wrong branch, a merge full of conflict markers, a red pipeline, a Vim window that says a swap file exists. Perhaps a sinking feeling.

What you need

the one recipe for this mistake, and the reassurance that nearly everything here can be undone. Every recipe in this appendix was run, start to finish, in a sandbox when this book was built: first the mess is made, then it is cleaned up.

E.1 Three facts that make mistakes cheap

Your clone is yours. Until you push, nothing you do on your laptop reaches anyone else. A wrong commit, a bad merge, a deleted branch: all of it is local, and all of it can be put back.

Git almost never forgets. Every commit you have made in the last few months is still in your clone, even after a reset or a deleted branch, and `git reflog` lists where your branch has been. Losing work in git takes real effort.

The protected branches protect you too. Nobody can push straight to `develop` or `main`; changes

reach them only through a merge request that passed the pipeline and a review. So the worst a slip on your laptop can do is make a red merge request, which is what merge requests are for.

The one thing that is hard to undo is a secret (a token, a password) that reached GitLab. That has its own recipe, and the first step is a phone call, not a git command (Section E.13, "I pushed a secret" (p. 729)).

E.2 The practice set-up

The recipes below share one sandbox: a bare repository standing in for GitLab, and a clone of it called `work` with `main` and `develop`. To follow along, make an empty folder, open Git Bash in it, and type this first.

```
git init -q --bare gitlab.git
git clone -q gitlab.git work 2>/dev/null
cd work
git switch -q -c main
printf 'package Stations {\n    part def Hull;\n    part def Mooring;\n}\n' > Stations.sysml
git add Stations.sysml
git commit -q -m "Start the Stations package"
git push -q -u origin main
git switch -q -c develop
git push -q -u origin develop
git branch -a
```

Every recipe below starts with `cd work`, because each one is typed in a fresh Git Bash window.

E.3 "I committed to develop by mistake"

You meant to start a feature branch, forgot, and committed on `develop`. You have not pushed (you could not: the branch is protected). The fix is two moves: make a branch where you are, which keeps the commit, then put `develop` back where GitLab has it.

```
cd work
printf 'package Stations {\n    part def Hull;\n    part def Mooring;\n    part def Mast;\n}\n' >
Stations.sysml
git commit -q -am "Add the Mast"
git switch -q -c feature/7-add-mast
git switch -q develop
git reset -q --hard origin/develop
git log --oneline -1 --decorate feature/7-add-mast
git status --short --branch
```

`git reset --hard origin/develop` moves your local `develop` to match GitLab's and throws away uncommitted changes, so make the branch first. If you had uncommitted edits too, commit them on the new branch before you switch back.

Tip: The habit that prevents it

Start every piece of work with `gitlab.pl flow start feature NN-words` or `git switch -c feature/NN-words`. Your shell prompt in Git Bash shows the branch in brackets; glance at it before `git commit`.

E.4 “I want my last commit back as edits”

The commit was premature: the message is wrong, or you want to add one more file. `git reset --soft` takes the commit away and leaves its changes staged, ready to commit again.

```
cd work
git switch -q feature/7-add-mast
git reset --soft HEAD~1
git status
```

```
cd work
git commit -q -m "Add the Mast part def to Stations"
git log --oneline -1
```

Only do this to commits you have not pushed. Once a commit is on GitLab, add a new commit instead of rewriting the old one.

E.5 “I am editing on the wrong branch”

You started editing, then noticed the prompt says `develop`. If the right branch does not exist yet, `git switch -c` takes your uncommitted edits with it. If it does exist, park the edits with `git stash`, switch, and bring them back.

```
cd work
git switch -q develop
printf 'package Stations {\n    part def Hull { doc /* The floating body. */ }\n    part def
Mooring;\n}\n' > Stations.sysml
git stash -q
git switch -q -c feature/8-hull-doc
git stash pop -q
git status
```

E.6 “I lost a commit”

You ran a reset you did not mean, or deleted a branch, and a commit seems to be gone. It is not. `git reflog` lists every place your `HEAD` has been; `HEAD@{1}` is “where I was one move ago”.

```
cd work
git switch -q feature/8-hull-doc
```

```
git commit -q -am "Document the hull"
git reset -q --hard HEAD~1
git log --oneline -1
git reflog -3
git reset -q --hard HEAD@{1}
echo "Recovered: $(git log --oneline -1)"
```

If the lost commit is further back, read down the `git reflog` list until you see its message, and use its hash: `git branch rescue 1a2b3c4` makes a branch that points at it, without moving anything else.

E.7 “The merge is full of conflicts and I want out”

A merge stopped with `CONFLICT` and the file is full of `<<<<<<<` markers. If you are not ready to resolve them now, you do not have to: `git merge --abort` puts everything back exactly as it was before the merge.

```
cd work
git switch -q develop
git switch -q -c feature/10-hull-name
sed -i 's/part def Hull;/part def HullBody;/' Stations.sysml
git commit -q -am "Rename Hull to HullBody"
git switch -q develop
git switch -q -c feature/11-hull-tag
sed -i 's/part def Hull;/part def Hull { @L4Assembly; }/' Stations.sysml
git commit -q -am "Tag the hull"
git merge feature/10-hull-name || echo "the merge stopped"
git status --short
git merge --abort
git status
```

When you are ready, resolving the conflict is described in Chapter 10, Vim for models (p. 143): pick the right text between the markers, delete the markers, `git add` the file, `git commit`.

E.8 “I merged, and it was the wrong thing”

The merge finished, but it should not have happened: the wrong branch, or a resolution you regret. Before you push, `ORIG_HEAD` remembers where you were before the merge.

```
cd work
git switch -q feature/11-hull-tag
git merge -q --no-edit main
git reset -q --hard ORIG_HEAD
echo "before the merge: $(git log --oneline -1)"
```

E.9 “A change on develop is wrong”

This is the one case where the mistake has already been reviewed and merged. Nobody rewrites `develop`: you make a new change that undoes the old one, and it goes through a merge request like any other. `git revert -m 1` undoes a merge commit (the `-m 1` says “keep the side that was develop”).

```
cd work
git switch -q develop
git merge -q --no-ff --no-edit feature/10-hull-name
merged=$(git rev-parse HEAD)
git switch -q -c feature/12-revert-hull-rename
git revert -m 1 --no-edit "$merged" >/dev/null
grep "part def Hull" Stations.sysml
```

(The local `develop` was used here as a stand-in for what GitLab did when it merged the merge request. On the real project, start from `origin/develop` after `git pull`.) In the merge request, say what was wrong and link the original one. GitLab also has a **Revert** button on a merged merge request that does the same thing.

E.10 “The pipeline is red”

A red pipeline is not a mistake; it is the safety net working. Read the first failed job’s log from the top down to the first line that says `error`: it names the file and line. Reproduce it on your laptop, fix it, and push a new commit to the same branch. The merge request picks it up and the pipeline runs again.

```
cd work
printf 'package Broken {\n    part def Hull\n    part def Mast;\n}\n' > Broken.sysml
perl "$KIT/tools/sysml/sysml.pl" check Broken.sysml
```

The message points at line 3 because that is where the parser noticed; the missing `;` is at the end of line 2. That is common: when the error line looks fine, look at the line above.

```
cd work
sed -i 's/part def Hull$/part def Hull;/' Broken.sysml
perl "$KIT/tools/sysml/sysml.pl" check Broken.sysml
rm Broken.sysml
```

Never force-push a branch other people have, and never delete and re-open a merge request to “get a clean pipeline”: the history of the red run is part of the record. Appendix F, Model error catalog (p. 731) lists the messages you will meet and their fixes.

E.11 “I changed a file I should not have”

You opened a converter-written file “just to look” and reformatted it, or a search-and-replace went too far. `git restore` puts the file back as it was in your last commit; `git restore --staged` unstages it without touching your edits.

```
cd work
printf '\n// a stray edit\n' >> Stations.sysml
git restore Stations.sysml
git status --short --branch
```

E.12 “My clone is a mess”

Untracked files everywhere, branches you no longer understand. First look without deleting: `git clean -n` lists the untracked files it would remove. When you are sure, `git clean -fd` removes them. If the whole clone has become confusing, keep it aside and clone again; your pushed work is all on GitLab.

```
mv work work-confusing
git clone -q gitlab.git work
cd work
git switch -q develop
git status
```

E.13 “I pushed a secret”

A token, a password or a private key reached GitLab in a commit. The pipeline’s `secrets` job may have told you, or a reviewer did. The order matters:

1. **Tell the security owner now** (on Tidewater, Tom, with the ISSM). Do not wait until you have fixed it.
2. **Rotate the secret**: revoke the token in GitLab, change the password. This is the fix. Anything you do to the repository afterwards only tidies up, because a secret that was pushed must be treated as seen.
3. **Remove it from the branch** before the merge request merges: amend or rewrite your own feature branch, with the security owner watching, and put the new value in a masked CI/CD variable (Chapter 17, The DevSecOps tools that earn their keep (p. 255)).
4. If it already merged to a protected branch, a maintainer and the security owner decide together whether the history is rewritten; that is never a solo decision.

Marked information pushed to the wrong place (spillage) follows the same first step, and then the program’s spillage procedure: stop, notify, contain, and do not delete history yourself (Chapter 48, Security: protect the environment, engineer it into the system (p. 651)).

E.14 "Vim says a swap file exists"

Vim keeps a swap file (`.Stations.sysml.swp`) while a file is open, so that a crash or a closed laptop lid does not lose your edits. When you open the file again, Vim warns that the swap file exists and offers choices:

- **[R]ecover** if Vim crashed or the window was closed: your unsaved edits come back. Check them, `:w` to save, then quit and open the file again, answer **[D]elete** to remove the old swap file.
- **[E]dit anyway** or **[Q]uit** if the file is open in another window right now: go back to that window instead.

From the shell, `vim -r` lists the swap files Vim can recover. Inside Vim, `u` undoes, `Ctrl-R` redoes, and `:earlier 10m` takes the buffer back to how it was ten minutes ago, which rescues most "I deleted the wrong block" moments without git at all.

E.15 "I broke a practice lesson"

The practice lessons (Appendix A, Command reference (p. 696)) are sandboxes; breaking them is allowed. Reset the lesson and set it up again.

```
perl "$KIT/agile.pl" practice 10 --dir lessons >/dev/null
perl "$KIT/agile.pl" practice reset 10 --dir lessons
```

Without `--dir`, the lessons live in the kit's `data/practice/` folder and `perl agile.pl practice reset N` removes them there.

E.16 When to ask for help

Ask at once, without trying another command, when:

- the mistake involves a secret or marked information;
- you are about to use `--force` with `git push`, or to delete a branch other people use;
- a recipe here did not do what it says (that is a bug in this book or the kit, and the team wants to know).

On Tidewater, asking was a stand-up sentence: "B TW-21 my clone of the model is tangled after the drop merge; need: ten minutes with Tom after stand-up." It took nine minutes, and Tom wrote the fix down as a new recipe.

CHAPTER

F

Model error catalog

From what you know →

You know

Cameo's validation messages.

→ Here it is called

the model error catalog: grammar, lint, outline and recipe messages.

→ What is new

each message names a file and a line, and every one here was produced for real.

Survival kit: When a check says no

What you have

a message from the grammar check, lint, the outline check, a pipeline job or the recipe checker. It is terse, and it was written by a program, not a person.

What you need

what it means in plain words, the usual cause, and the fix. Every message in this appendix was produced by a runnable example when this book was built: the example breaks a real Tidewater file, shows the real message, and puts the file back.

F.1 How to read a message

Most messages share one shape:

```
model/physical/stations/Stations.sysml:38:21: error: unresolved type 'Hul'
```

From left to right: the **file**, the **line**, the **column** (characters from the left edge), the **severity** (`error` must be fixed before a merge; `warning` should be; `note` is information), and the **message**.

In Vim, `:make` or the kit's `\ml` (lint) and `\mx` (grammar) put these lines in the quickfix list, and `:cnext` jumps from one to the next (Chapter 10, Vim for models (p. 143)).

Two habits save most of the time spent on errors:

- **Fix the first message first.** One missing `;` can produce a cascade; the first message is the cause, the rest are often echoes.
- **When the line looks fine, look one line up.** Parsers notice a problem where the text stops making sense, which is usually just after the mistake.

The messages come from five places, in the order the pipeline runs them:

Table F.1: Where model errors come from

Tool	What it checks	Pipeline job
`tools/sysml/sysml.pl check`	the text against the official SysML v2 grammar	`grammar`
`tools/sysml/model.pl lint`	names resolve, no duplicates, brackets balance, imports exist	`lint`
`tools/sysml/model.pl check`	level tags and the 2-to-9 outline rule	`outline`
`ci/job.pl`	the pipeline's own rules: lint at 0/0, ratchets, secrets, docs current	every job
`tools/sysml/req-recipe.pl`	requirement wording against the recipe (Chapter 7, Requirements a pipeline can check: the recipe (p. 73))	`recipe`

F.2 The practice model

The examples share one sandbox: a model repository made with `kickoff.pl scaffold`, with Tidewater's drop 1 converted into it in layers, exactly as the team's first import, with the two roots of the part and function trees named in `sysml.conf` (the scaffold leaves them for you to set). To follow along, run this in an empty folder.

```
perl "$KIT/tools/gitlab/kickoff.pl" scaffold model --project tidewater/model --title Tidewater
--levels L1Network,L2Segment,L3Station,L4Subsystem,L5Assembly,L6Component >/dev/null
perl "$KIT/sim/tidewater-v1.pl" --drop 1 drop >/dev/null
cd model
perl "$KIT/bin/xmi2sysml.pl" --out model --layout-map "$KIT/examples/tidewater-v1/layout.map"
../drop/tidewater.xmi
printf 'noun_root = ObservationNetwork\nverb_root = ObserveCoast\n' >> sysml.conf
git add -A
git commit -q -m "Import drop 1"
perl "$KIT/tools/sysml/model.pl" lint | tail -1
```

In the examples below, a check that finds a problem is followed by `|| echo "exit status $?"`. A check that passes exits with status 0; one that finds errors exits with 1 (2 means it could not run at all). The pipeline reads exactly that number to colour a job red or green, and the `echo` shows it to you. Each example starts with `git checkout -q -- model`, which puts the model files back the way the last commit had them, so one broken example never spills into the next.

F.3 Grammar check messages

The grammar check reads each file against the official SysML v2 grammar. Its messages all say `unexpected` something, then list a few of the things it *would* have accepted there. The list is a hint about what kind of text belongs at that spot, not a menu: read the word it calls unexpected, and the line above it.

F.3.1 unexpected …; expected ‘,’, ‘.’, ‘:’ … – a missing semicolon

```
printf 'package A {\n    part def Buoy {\n        part hull : Hull\n        port obsOut :
ObservationPort;\n    }\n}\n' > a.sysml
perl "$KIT/tools/sysml/sysml.pl" check a.sysml || echo "exit status $?"
```

Means. Line 3 ends without `;`, so the parser read on into line 4 and stumbled on `port`. **Fix.** Add the `;` at the end of the line *above* the one named. Every member line ends with `;` or a `{ … }` body.

F.3.2 unexpected end of file – a brace never closed

```
printf 'package B {\n    part def Buoy {\n        doc /* A buoy. */\n    }\n' > b.sysml
perl "$KIT/tools/sysml/sysml.pl" check b.sysml || echo "exit status $?"
```

Means. The file ended while a `{` was still open. **Fix.** Count braces from the top; in Vim, put the cursor on a `{` and press `%` to jump to its partner. The kit’s lint names the line of the brace that is never closed (Section F.4.5, is never closed (p. 735)), which is often quicker.

F.3.3 unexpected ‘Name’ after a SysML v1 word

```
printf 'package C {\n    state machine BuoyModes {\n        region r { state Observing; }\n
}\n}\n' > c.sysml
perl "$KIT/tools/sysml/sysml.pl" check c.sysml || echo "exit status $?"
```

Means. `state machine` and `region` are SysML v1 vocabulary. The v2 grammar read `state` as the start of a state usage named `machine`, and did not expect a second name. **Fix.** Write the v2 form: `state def BuoyModes { entry; then Observing; state Observing; … }`, and exhibit it on the part with `exhibit state modes : BuoyModes;`. Chapter 8, SysML v2 by example: a workbook (p. 92) shows the full pattern; this is the mistake a v1 modeler makes most, and a local AI assistant makes it too (Chapter 18, A local AI assistant on the laptop (p. 269)).

F.3.4 unexpected ‘doc’ – a doc comment in quotes

```
printf 'package D {\n    part def Buoy {\n        attribute mass = 1.5\n        doc "a buoy"\n
}\n}\n' > d.sysml
perl "$KIT/tools/sysml/sysml.pl" check d.sysml || echo "exit status $?"
rm a.sysml b.sysml c.sysml d.sysml
```

Means. Two mistakes, and the parser reports the first one it can see: line 3 has no `;`, so `doc` on line 4 is unexpected. Fixing that reveals the second: a `doc` comment is written `doc /* text */`, not in quotes. **Fix.** `attribute mass = 1.5;` and `doc /* A buoy. */`.

F.4 Lint messages

Lint runs on the whole model at once, so it can see what the grammar check cannot: whether a name you use is declared somewhere. It ends with one line the pipeline reads, `elements=N errors=E warnings=W`; the `lint` job fails unless both counts are zero.

F.4.1 unresolved type

```
cd model
sed -i 's/part hull : Hull;/part hull : Hul;/' model/physical/stations/Stations.sysml
perl "$KIT/tools/sysml/model.pl" lint || echo "exit status $?"
```

Means. `Hul` is not declared in any file of the model, nor in the standard library. **Fix.** Correct the spelling, or add the `private import` of the package that declares the type. If the name comes from the prime's model, it may have been renamed in a newer drop: `xmi2sysml.pl --delta … --impact` lists renames (Appendix D, Quick reference: SysML v1 to v2 (p. 719)).

F.4.2 duplicate member

```
cd model
git checkout -q -- model
sed -i '0,/part def Hull {/s//part def Hull { doc \/* a second Hull *\/ }\n    part def Hull {/'
model/physical/stations/Stations.sysml
perl "$KIT/tools/sysml/model.pl" lint || echo "exit status $?"
```

Means. Two members of one package have the same name. Usually a copy-and-paste, or a hand-written element that the next prime drop also brought in. **Fix.** Keep one. If both are real, rename yours and raise a question with the prime.

F.4.3 a SysML keyword cannot be a name

```
cd model
git checkout -q -- model
sed -i 's/part hull : Hull;/part state : Hull;/' model/physical/stations/Stations.sysml
perl "$KIT/tools/sysml/model.pl" lint || echo "exit status $?"
```

Means. Words such as `state`, `action`, `port`, `in` and `out` are part of the language. **Fix.** Choose another name (`hullState`), or quote it (`'state'`) when it must match a name in the prime's model.

F.4.4 import of unknown package

```
cd model
git checkout -q -- model
sed -i '0,/private import /s/private import /private import Station::*;\n    private import /'
model/physical/stations/Stations.sysml
perl "$KIT/tools/sysml/model.pl" lint || echo "exit status $?"
```

Means. The package named in the import does not exist (here `Station` for `Stations`). **Fix.** Correct the name. Package names are the file's top-level package, not the file name.

F.4.5 is never closed

```
cd model
git checkout -q -- model
mkdir -p model/library/conventions
printf 'package Scratch {\n    part def Probe {\n        doc /* unfinished */\n}\n' >
model/library/conventions/Scratch.sysml
perl "$KIT/tools/sysml/model.pl" lint || echo "exit status $?"
rm model/library/conventions/Scratch.sysml
```

Means. A `{` has no matching `}`. Lint names the line where the unclosed brace *opens*, which the grammar check cannot. **Fix.** Add the `}` where the element ends.

F.5 Outline and level-tag messages

`model.pl check` prints two sections. **Level tags**: each part definition carries a level tag (`@L2 Segment;`), and the tag must match the level where the part tree first reaches it. **Outline**: every folder and package should hold between 2 and 9 entries, the span a person can take in at a glance.

F.5.1 tagged Ln, first reached at Lm

```
cd model
git checkout -q -- model
sed -i '0,/\/\/ TODO stereotype Tidewater_Profile:L2Segment.*/s//@L4Subsystem;/'
model/physical/stations/Stations.sysml
perl "$KIT/tools/sysml/model.pl" check || echo "exit status $?"
```

Means. `StationSegment` is two steps below the network (level 2), but someone tagged it as a level-4 subsystem. **Fix.** Use the tag the TODO line named (`@L2Segment;`). The converter's TODO line always says which stereotype the prime applied; copying it is the safe choice.

A misspelled tag (`@L2Segmnt;`) is *not* reported by lint or by the check: it is a different, undeclared name that nothing reads. The `outline` job's level count and practice lesson 10's checker catch it (Chapter 10, Vim for models (p. 143)); a review should too.

F.5.2 outline violations

```
cd model
git checkout -q -- model
perl ci/job.pl outline || echo "exit status $?"
```

Means. The model has folders or packages with fewer than 2 or more than 9 entries. On a fresh conversion this is normal: the prime's packages were never shaped to the rule. The pipeline therefore treats the outline as a **ratchet**: `.ci/outline-max` records the number the team accepted, and the job fails only when a change makes it worse. A new scaffold's allowance is its own few violations plus a little room for the first folders, so a whole converted drop goes far over it. **Fix.** On the first import, record the starting number in `.ci/outline-max` in the same merge request, with the reason in the description. After that, never raise it: split the folder or package that grew, and lower the number when it falls.

F.6 Pipeline job messages

These come from `ci/job.pl`, the small script every job runs (Chapter 20, The pipeline, job by job (p. 302)). They are written for the person reading the job log, so most of them already say what to do.

Table F.2: Messages from ci/job.pl

Message	Means	Fix
`lint is not 0/0 (elements= N errors=E warnings=W)`	lint found something	read `lint.txt` in the job's artifacts, or run `model.pl lint` locally
`this change adds outline violations (N > M): fix them, do not raise .ci/outline-max`	the outline ratchet	split what grew; see Section F.5.2, outline violations (p. 736)
`fewer violations than allowed: lower .ci/outline-max to N in this MR`	good news, not an error	lower the number in the same merge request
`this change adds trace gaps: orphans 1 -> 2 (see trace.txt)`	the merge request removed a satisfy or verify, or added a requirement nothing satisfies	restore the link, or link the new requirement, before merging
`this change makes the model more tangled: …`	the complexity ratchet: a cycle, an upward import, or more fan-out	see Chapter 47, Keeping SysML v2 from becoming a spider web (p. 645)
`docs/ is stale: run model.pl docs and commit`	the generated docs no longer match the model	run `perl $KIT/tools/sysml/model.pl docs` and commit `docs/`
`possible secret committed: remove it, rotate it, then rewrite the branch before it merges`	a token or key pattern in a tracked file	Section E.13, "I pushed a secret" (p. 729), at once
`KIT is not set: add it under Settings > CI/CD > Variables`	the runner does not know where the kit is	a maintainer sets the `KIT` variable

Message	Means	Fix
`the kit on this runner is X, but .kit-version pins Y`	the runner has another kit version than the model asks for	update the runner's kit, or bump `.kit-version` in a reviewed merge request

Four of them, produced:

```
cd model
git checkout -q -- model
sed -i 's/part hull : Hull;/part hull : Hul;/' model/physical/stations/Stations.sysml
perl ci/job.pl lint || echo "exit status $?"
```

```
cd model
git checkout -q -- model
perl ci/job.pl docs || echo "exit status $?"
```

```
cd model
git checkout -q -- model
printf 'GITLAB_TOKEN=glpat-%s\n' AbCdEfGhIjKlMnOpQrSt > notes.txt
git add notes.txt
perl ci/job.pl secrets || echo "exit status $?"
git rm -q --cached notes.txt
rm notes.txt
```

```
cd model
env -u KIT perl ci/job.pl lint || echo "exit status $?"
```

The token in the third example is assembled when the example runs, so the kit's own secret scan never sees a real- looking token in this book's source.

F.7 Requirement recipe findings

`req-recipe.pl` reads requirement text the way a careful reviewer does (Chapter 7, Requirements a pipeline can check: the recipe (p. 73)). Its findings are graded: `major` (the statement cannot be verified as written), `minor` (it can, but something is missing), `info`.

```
cat > reqs.txt <<'EOF'
The network shall provide marine weather observations for the coastal region it covers.
(TW-STK-001)

Each station shall measure wind, air pressure and air temperature, and each buoy shall also
measure waves. (TW-SYS-001)
EOF
perl "$KIT/tools/sysml/req-recipe.pl" --strict reqs.txt || echo "exit status $?"
```

Table F.3: Common recipe findings

Finding	Means	Fix
`weak target verb 'provide'` (also have, be, be able to, support, ensure ...)	no test can show "provide"	name the action: send, store, measure, report
`2 operative verbs (shall, shall)`	two requirements in one sentence	split it; each gets its own ID
`'measure' with no measurable threshold`	a verb that implies a quantity, without one	add the number and unit, or "at least", "within", "every"
`passive 'shall be kept'`	the subject that must act is missing	make the specification target the subject
`vague 'in time'` (also as appropriate, user-friendly, fast ...)	a word two readers read differently	replace it with the measure behind it
missing part `Requirement ID`	no project-unique ID	every requirement needs one

On Tidewater the requirements belong to the prime's DOORS modules, so the fix for a finding there is a discrepancy report to the prime, not an edit in the government model (Chapter 45, The prime's model changes: receive, assess, disposition (p. 619)).

F.8 When the message is not here

- Search the kit's help: `perl $KIT/agile.pl help error "the message"` finds the entry for most kit messages.
- Run the check on the one file alone (`sysml.pl check FILE`); a smaller input gives a clearer message.
- Undo your last edit (`git diff` shows it) and redo it in two halves; the half that brings the message back holds the mistake.
- Ask, with the exact message copied, the command you ran, and `git status` (Section E.16, When to ask for help (p. 730)).

CHAPTER

G

Modeling style guide

From what you know →

You know

the drafting standard and the document style guide.

→ Here it is called

the modeling style guide.

→ What is new

much of it is checked by the pipeline.

A style guide exists so that three people write one model, not three. Every rule here is short, has a reason, and is the same rule the model repository's `docs/conventions/naming.md` states, which `kickoff.pl scaffold` puts in every new project. When the two disagree, the repository's copy wins: it is the one the team reviews and changes by merge request.

There is one rule above all the others:

Warning: Government text and the prime's text follow different rules

Files the converter writes from the prime's drops keep the prime's names and layout, exactly as delivered, with only the `// TODO` lines replaced. Every rule below about names, order and formatting applies to what the **government** writes. Changing the prime's text for style makes every later drop conflict (Chapter 51, Pitfalls and anti-patterns (p. 684)); ask the prime through a discrepancy report instead.

G.1 Names

Table G.1: Names in the model

Thing	Rule	Example	Not
package	UpperCamelCase noun, the subject it holds	`Stations`, `Power System`	`stations_pkg`, `Pkg1`
definition (part, port, item, interface, state)	UpperCamelCase noun	`MooredBuoy`, `ObservationIF`	`moored_buoy`, `Block1`

Thing	Rule	Example	Not
action definition	UpperCamelCase verb phrase	`CheckQuality`, `PublishBulletin`	`QC`
usage (part, port, attribute, action)	lowerCamelCase, the role it plays	`buoys`, `obsOut`, `stateOfCharge`	`MooredBuoy1`, `p1`
requirement usage	the DOORS id, `-` and `.` as `_`	`TW_SYS_006`	`req6`
requirement id	as DOORS has it, never reused	`TW-SYS-006`	a new number for an edited requirement
verification definition	verb + what it shows	`TestDataFlow`, `InspectVesselAccess`	`Test1`
metadata definition	what it marks	`Procured`, `L3Station`	`Stereo7`
quoted names	only to match a source that used spaces	`'Stakeholder Needs'` (from DOORS)	in new government text

A name says what the thing **is**, not where it sits in a diagram or a document. `Copy of Sensor Suite`, `Battery_old` and `TBD Component` came from the v1 model's spider web; they do not come back. Acronyms are written as words in names (`ObservationIf` would be correct by the rule; `ObservationIF` is kept because the prime chose it), and abbreviations are avoided unless the program uses them in speech.

G.2 Files and folders

- **One top-level package per file**, named after the package: `Stations.sysml` holds `package Stations`.
- **Folders are layers**, in this order: `library/`, `requirements/`, `logical/`, `physical/`, `analysis/`, `verification/`. A package may depend on its own layer or a lower one, never a higher one; the `complexity` job checks it (Chapter 47, Keeping SysML v2 from becoming a spider web (p. 645)). For the prime's packages the layer is set by the converter's layout map, so every drop lands in the same folder (Chapter 16, The model repository: what goes where (p. 245)).
- **Government-only additions** go in the layer they belong to (`library/conventions/Conventions.sysml`, `logical/GovBehavior.sysml`), never inside a converter-written file, and begin with a comment that says why they exist.
- **Generated files are never edited by hand**: `docs/` comes from `model.pl docs`, the DOORS requirement packages from `reqif2sysml.pl`. Change the source and regenerate.

G.3 Inside a file

A government-written file, in the order its parts appear:

```
cat > GovBehavior.sysml <<'EOF'
// Government-owned: the observation cycle's hand-written behavior, which the converter
// cannot carry from v1 (opaque fork and object flows). Decision record: docs/adr/. Owner: Bob.
package GovBehavior {
    doc /* Hand-written behavior that completes the prime's activities. */
    private import ScalarValues::Real;
```

```
    action def ReportHealth {
        doc /* Send battery state, mode and faults to the data center. */
        in stateOfCharge : Real;
        out report : Real;
    }
}
EOF
perl "$KIT/tools/sysml/sysml.pl" check GovBehavior.sysml
```

- **The opening comment** answers three questions in two or three lines: why this file exists, which decision allowed it, who owns it.
- **The package's `doc` comment** says what the package holds, in one sentence a reviewer can check against its contents.
- **Imports come first and name exactly what is used** (`private import ScalarValues::Real;`). A wildcard (`::*`) is allowed only in files the converter writes, where it is the converter's choice.
- **Every definition has a `doc` comment** that says what the thing is or does, in the program's words, not the model's: "Send battery state, mode and faults to the data center", not "ReportHealth action def".
- **Members in a stable order**: parts, then ports, then attributes, then connections and satisfy relationships, then behavior. Not alphabetical: grouped by kind, so a diff shows what kind of thing changed.
- **One statement per line**, four spaces of indentation, no tabs. Lines under about 120 characters; a long requirement text is the exception, because its `doc` is the requirement.

G.4 Metadata, not stereotypes

- Use **metadata definitions** for what v1 did with stereotypes: level tags (`@L3Station;`), procurement (`@Procured { supplier = "…"; partNumber = "…"; }`), markings (`@Marking { level = Level::U; }`), trust zones.
- Keep the set **small and declared in one place**, the conventions library. A new metadata definition is a decision: it needs a merge request that says what will read it.
- A tag nobody reads is decoration. If no check, view or report uses a metadata definition, remove it.

G.5 Relationships

- **Declare each relationship once**, on the element that owns the responsibility: the part `satisfy`s the requirement, the verification case `verify`s it. Reverse views (which parts satisfy this requirement?) are generated, never maintained by hand.
- **Trace at the level that actually satisfies**, and no lower. A requirement satisfied by the data center is satisfied by `DataCenter`, not by each of its five services.
- **Allocate at the agreed level** (Tidewater: the segment and station level), not every step to every component.
- **No `dependency` used as a vague "related to"**: it is how the v1 model grew its cycles. If two elements are related, say how (satisfy, allocate, connect) or leave them alone.

G.6 Requirement text

Requirement statements follow the recipe (Chapter 7, Requirements a pipeline can check: the recipe (p. 73)): one requirement per statement, a measurable threshold where the verb implies one, no weak verbs (provide, support, be capable of), the specification target as the subject, and the project-unique ID. The prime's DOORS modules own the wording on Tidewater, so a recipe finding there becomes a discrepancy report, not an edit.

G.7 TODO lines

- A converter `// TODO` line is replaced by the line or block that closes it, **in place**, without moving or reformatting the lines around it. The next drop then merges cleanly.
- A TODO you cannot close yet stays as it is; do not delete it to make the count fall. The count is the port's burn-down, and it has to be true.
- Government text has no TODO lines of its own. Open an issue instead, and put the issue number in a comment if the reader needs to know.

G.8 Commits and merge requests

- **Commit message**: a first line under about 60 characters, in the imperative ("Add the Storm mode to BuoyModes"), then a blank line and *why*. The "what" is in the diff.
- **Merge request title**: what the model gains, as a verb phrase someone could demo ("Close the Stations TODO lines").
- **One story's worth per merge request.** A merge request that touches four packages for four reasons is four merge requests.
- **Decisions in ADRs**: one decision per file in `docs/adr/`, with the context, the decision and the consequences. Anything a new team member might "fix" without knowing why it is that way deserves one.

G.9 Branches, tags and issues

Table G.2: Names outside the model

Thing	Rule	Example
feature branch	`feature/<issue number>-<a few words>`	`feature/41-sys006-wording`
release branch	`release/baseline-N`	`release/baseline-1`
hotfix branch	`hotfix/<what>`	`hotfix/satisfy-sys-014`
drop branch	made by `gitlab.pl flow vendor`	`feature/drop-v1.1`
tag	`baseline-N`, annotated, made by `gitlab.pl flow released`	`baseline-0`
story title	something you could show at the review	`Break the Stations to Data Center cycle`
task title	one piece of work under a week	`Move the ObservationRecord dependency to Interfaces`
discrepancy report title	the v1 element and the problem	`DR: TW-SYS-012 has no satisfy in drop 1`

G.10 Changing this guide

The style guide is the team's, not this book's. Change `docs/conventions/naming.md` by merge request, with the reason in the description, and say in the review what existing text will be brought in line and when (never "all at once": a style sweep across converter-written files is exactly the reformatting Chapter 51, Pitfalls and anti-patterns (p. 684) warns about).

CHAPTER

H

Quick reference: saying it so they hear it

H.1 The pattern

Table H.1: Outcome, evidence, next or ask

Part	What	Example
Outcome	what is now true, in the listener's words	"Stations structure is done"
Evidence	one number	"nine TODO lines closed, forty-three left"
Next or ask	what happens next by a day, or what you need from whom by a day	"decide who owns the observation record by Thursday" / "need Jim to escalate today"

Keep the mechanism (how you did it) for when someone asks. The "so what?" test: if the honest answer is "so nothing", say the outcome it served, or leave it out.

H.2 Five stand-up shapes

```
Y TW-12 [outcome]: [X] of [Y] done, [Z] left. T TW-15 [next outcome], by [day]. B none
Y TW-22 the check caught [problem] at [early stage] instead of [late stage]. T ... B none
B TW-3 [what cannot happen] until [thing]. Need: [person] to [action] by [day]
T TW-31 [question] answered by [day]: [what you will hand over]
Y TW-9 I don't know yet whether [X]; I'll know by [day] after [action]
```

Every line keeps Y, T and B, a story id in each, and `B none` when nothing blocks. One number per line.

H.3 The ladder

Table H.2: Write the one-sentence version first; say each rung out loud once

Rung	Length	Holds
One sentence	10 seconds	the outcome and the number
Thirty seconds	3 to 4 sentences	outcome, number, what it means for the next milestone, the ask
Five minutes	the thirty seconds, one thing shown, one before-and-after	the demo

H.4 The demo in six lines

1. The question this sprint answered, and the one-line answer.

2. Show the output, not the tool (the site, the matrix, a red-to-green merge request).
3. One before-and-after, with a number.
4. One safe live change, on a scratch branch.
5. What it means for the next milestone.
6. The ask, with a name and a date.

Roles: **driver** (keyboard, silent), **narrator** (the words), **note-taker** (questions become issues). Rotate them every sprint. Dry run on Day 9.

H.5 The four numbers, every review

TODO lines left. Verify coverage. Open discrepancy reports and prime changes. Blocked items. Same order, every stand-up summary, weekly note, review and memo. Headlines state the conclusion: "Trace: 29 of 30 linked, green".

H.6 Jargon to plain

Table H.3: Jargon to plain

Say instead of	Plain
pipeline, CI	the automated checks that run on every change
merge request, MR	a proposed change, reviewed before it is accepted
the pipeline is green	all the automated checks passed (say what that proves)
lint errors	errors the checks found before review
rebased, merged develop in	(leave it out; say what the change does)
feature branch	my working copy of the change (usually leave it out)
trace coverage	how many requirements link to a design and a test
VCRM	the verification matrix: how and whether each requirement is verified
baseline, tag	a frozen, approved version we can always return to
prime drop	the prime's latest model update
refactoring, technical debt	cleanup that keeps future changes fast and cheap
TODO lines	conversion items still to finish by hand

H.7 Bad news

Table H.4: Fact, cause in one clause, next with a date

Situation	Say
a check found a problem	"Found early: that's the system working. Impact, fix."
a worse number you can now see	"Not new, newly visible. Here's the date it closes."
behind	"Behind by [time] because [cause]. To recover we need [X], or we can [trade-off]."
you don't know	"I don't know yet. I'll know by [day] after [action]."

H.8 Tools

```
perl $KIT/tools/gitlab/talk.pl standup --as bob --prefix TW-    # tomorrow's Y/T/B draft from GitLab
perl $KIT/tools/gitlab/talk.pl review --sprint 4 --md           # the demo script, four numbers, the ladder
perl $KIT/tools/gitlab/talk.pl weekly --baseline baseline.txt   # Friday's note to management
perl $KIT/tools/gitlab/talk.pl translate draft.txt              # jargon list and "so what?" flags
perl $KIT/bin/daily.pl propose                                  # today's statuses from yesterday's answers
perl $KIT/bin/daily.pl quad                                     # the weekly quad that goes with the note
```

Drafts only: nothing is posted. Read every number before you say it. Chapter: Chapter 25, Talking about progress: helps for engineers who would rather not (p. 390).

CHAPTER

Templates and GitLab Markdown, field by field

From what you know →

You know

document templates and CDRL formats.

→ Here it is called

GitLab templates for issues, merge requests and wiki pages, in Markdown.

→ What is new

a template opens pre-filled every time, with checklists a reviewer can tick.

This appendix prints every template the Tidewater project uses and every filled-in sample of a GitLab text field, with notes on the Markdown each one uses. The files are in the kit under `tools/gitlab/templates/` (copied into the model repository) and `tools/gitlab/samples/`. Read it with GitLab open in another window and try each feature in a scratch issue.

I.1 How GitLab uses templates

GitLab looks for description templates in two folders of the repository's default branch:

- `.gitlab/issue_templates/NAME.md` appears in the *Choose a template* list when someone creates an issue;
- `.gitlab/merge_request_templates/NAME.md` appears in the same list on a new merge request. A merge request template named `Default.md` is filled in automatically on every new MR.

Because templates are files in the repository, they are changed like the model: by merge request, reviewed by their code owner (Tom), and recorded in history. The Definition of Done can therefore say "the MR description follows the Default template" and mean a specific, versioned text.

Three things in templates do work beyond filling in text:

- **HTML comments** (`<!-- … -->`) are prompts. They are visible while you edit and invisible once the description is saved, so a template can explain each section without the explana-

tion cluttering the issue.

- **Quick actions** on their own lines at the end (`/label …`, `/milestone …`, `/assign …`) run when the issue or MR is created, so every story starts with the right labels without anyone remembering them. A quick action with nothing after it (`/assign @` with no name) does nothing and is removed, which lets a template remind the author to fill it in.
- **Task lists** (`- [ ] item`) become checkboxes that anyone with access can tick without editing the text, and GitLab counts them in the issue list ("2 of 5 checklist items").

I.2 GitLab Markdown in fifteen features

Table I.1: The GitLab Markdown this project uses

Feature	You write	Used in
Headings	`## What and why`	every template
Tables	rows of cells between `|` characters, a `|---|---|` line under the header	stories, MRs, DRs, releases
Task lists	`- [ ] item` and `- [x] item`	Story, Task, MR checks, DoR, DoD
HTML comments	`<!-- prompt -->`	every template
Quick actions	`/label ~"type::story"` on its own line	every issue and MR template
Issue and MR references	`#19`, `!19`, `tidewater/model#19`	stories, MRs, comments
Milestone references	`%"Sprint 4"`	Story, Task, release notes
Label references	`~"workflow::review"`	comments, the board legend
Mentions	`@jim`	MR reviews, DRs
Closing keywords	`Closes #19` in an MR description	Default, Hotfix
Table of contents	`[[_TOC_]]`	the wiki, the README
Collapsible sections	`<details><summary>Log</summary> … </details>`	Story (the DoR), MR pipeline logs
Alerts	a quote starting `> [!note]` (or `[!tip]`, `[!important]`, `[!warning]`, `[!caution]`)	wiki, README, release notes
Diagrams	a fenced block tagged `mermaid`	Gitflow, Drop Intake Runbook, the README
Suggestions	a fenced block tagged `suggestion:-0+1` in a review comment on a diff line	MR reviews

Footnotes (`[^1]` in the text and a line `[^1]: the note` at the end) are used in wiki decision records. Mathematics (`$…$`) and front matter are available but the project never needed them.

I.3 Issue templates

I.3.1 Story

GitLab Markdown: tools/gitlab/templates/.gitlab/issue_templates/Story.md

```
<!-- Story: something you can show at the sprint review, like a small deliverable.
     Story: a slice of value that fits in ONE sprint, made of tasks of less than a week each.
     Title: a verb and an object, e.g. "Close the Stations TODO lines". The journal id is
     TW-<this issue's number>. -->

## Why
```

```
<!-- The outcome for the program in one or two sentences: who gains what. -->

## What "done" looks like

<!-- Acceptance criteria as a checklist. Each line can be checked by someone who did not do the
work. -->
- [ ]
- [ ] The pipeline is green on the MR (grammar, lint 0/0, outline, docs current)
- [ ] The trace job shows no new orphan, unverified or drifted requirement

## Tasks

<!-- One task issue per line, each under 5 points (one point = one ideal day). Create them with
the Task template
     and reference them here; they close through their merge requests. -->
| Task | Points | Who |
|---|---|---|
| # | | @ |

## Notes

<!-- Links, decisions, the v1 elements involved (xmi:id), the DOORS ids involved. -->

<details><summary>Definition of Ready (check before the sprint takes it)</summary>

- [ ] Fits in one sprint and has an epic (~"epic::...") and a tome (~"tome::...")
- [ ] Acceptance criteria above are testable
- [ ] Tasks are cut, each at most 5 points
- [ ] Dependencies on the prime or another story are linked (`/relate #N`, or "blocked by")

</details>

/label ~"type::story" ~"workflow::ready" ~"priority::2"
/milestone %"Sprint "
/assign @
/weight
```

Line by line:

- The opening comment states the size rule (one sprint, tasks under a week) and the naming rule. It disappears when the story is saved, so it is a reminder, not clutter.
- **Why** asks for the outcome in one or two sentences. Jim insisted on it after sprint 1, when two stories turned out to be the same work described differently; the *why* makes duplicates obvious.
- **What "done" looks like** is the acceptance criteria as a task list. Two items are pre-filled because they apply to every story that changes the model: a green pipeline and no new trace gaps. The reviewer ticks them.
- **Tasks** is a table of task issues with points and owners. GitLab turns `#21` into a link showing the task's title and state, so the table is also a live status view.
- The Definition of Ready sits inside `<details>`, collapsed, so it is there for the refinement session and out of the way afterwards.
- The quick actions label the story (`type::story`, `workflow::ready`, a priority), and leave the milestone, assignee and weight for the author: `%"Sprint "` without a number reminds

without guessing.

I.3.2 Task

GitLab Markdown: tools/gitlab/templates/.gitlab/issue_templates/Task.md

```
<!-- Task: an action item. Task: under one week (at most 5 points, one point = one ideal day).
One person, one MR (usually).
     Title: what you will do, e.g. "Replace the BuoyModes TODO with a state definition". -->

Part of #<!-- story number -->

## Do

<!-- Steps, as a checklist the reviewer can follow. -->
- [ ]
- [ ] Run `gitlab.pl ci` locally before pushing

## Done when

<!-- The one observable result, e.g. "grep finds no TODO in
model/physical/stations/Stations.sysml". -->

/label ~"type::task" ~"workflow::ready"
/estimate 1d
/assign @
/milestone %"Sprint "
```

A task is the smallest unit and the template is correspondingly short. `Part of #` links it to its story. **Do** is a checklist the reviewer can follow; its pre-filled line, "Run `gitlab.pl ci` locally before pushing", is the habit that keeps pipeline failures off the shared branches. **Done when** asks for one observable result, such as "grep finds no TODO in `model/physical/stations/Stations.sysml`". `/estimate 1d` sets a default estimate the author changes; GitLab shows estimate against `/spend` time on the issue.

I.3.3 Defect, Discrepancy Report, Prime Drop, Epic, Spike

GitLab Markdown: tools/gitlab/templates/.gitlab/issue_templates/Defect.md

```
<!-- Defect: something in the GOVERNMENT v2 model or the pipeline is wrong.
     A problem in the prime's v1 model or DOORS is a Discrepancy Report instead. -->

## What is wrong

<!-- One sentence. Name the file and element, e.g. `model/physical/stations/Stations.sysml`,
`part def PowerSystem`. -->

## How it shows

<!-- Paste the pipeline output or the trace line. Link the job. -->

```text
```
```

```
```

Expected

<!-- What the model should say, and where that comes from (the drop version, the DOORS id, a
decision). -->

Found

Found in	<!-- MR !N, pipeline #N, review, analysis -->
Phase	<!-- port / integration / maintenance -->
Drop	<!-- v1.x the model was built from -->

/label ~"type::defect" ~"owner::gov" ~"severity::2" ~"workflow::ready"
/assign @
```

A **defect** is something wrong in the government's own model or pipeline. Its labels say where it was found (`phase::port` or `phase::maintenance`) and what kind it is (`area::interface` for problems between packages, the status metrics' measure H).

GitLab Markdown: tools/gitlab/templates/.gitlab/issue_templates/Discrepancy Report.md

```
<!-- Discrepancy Report (DR): a problem in the PRIME's v1 model or DOORS. The government does not
edit the prime's
     model; it reports, the prime fixes it in a later drop, and the intake of that drop closes
     this issue.
     Jim sends the DR to the prime (contract channel) and records the prime's tracking number
     below. -->

Discrepancy

DR number	DR-<!-- n -->
Prime tracking	<!-- the prime's number, once they assign one -->
Found in drop	<!-- v1.x, delivered YYYY-MM-DD -->
v1 element	<!-- qualified name and xmi:id -->
DOORS id	<!-- TW-... -->
Severity	<!-- 1 misleads an analysis, 2 wrong with a workaround, 3 cosmetic -->

What we found

<!-- Facts only: what the model says, what DOORS says, what the trace job printed. -->

What we ask the prime to do

<!-- One requested change, specific enough to verify in the next drop. -->

Closure

- [ ] Sent to the prime on <!-- date --> by @jim
- [ ] Fixed in drop <!-- v1.x -->, confirmed by the intake issue #<!-- n -->
```
```

```
/label ~"type::discrepancy" ~"owner::prime" ~"severity::2"
/assign @jim
```

A **discrepancy report** is a problem in the prime's v1 model or DOORS. The government does not fix it; it reports it. The two-column table without a header row (`| | |` then `|---|---|`) is a GitLab idiom for a form: labels on the left, values on the right. The **Closure** checklist has two steps, sent and fixed, and the second names the drop intake issue that confirmed the fix. That link is how a DR's life is traced from finding to closure.

GitLab Markdown: tools/gitlab/templates/.gitlab/issue_templates/Prime Drop.md

```
<!-- Prime Drop: receiving inspection of one CDRL delivery of the prime's v1 model.
     Prime Drop: the intake of one model drop (XMI + ReqIF + CSV + MANIFEST) delivered under CDRL
     A003.
     Title: "Intake prime drop v1.x (YYYY-MM-DD)". This is a story: it must finish within the
     sprint it lands in. -->

## The drop

| | |
|---|---|
| Version | v1. |
| Delivered | <!-- date, and how: contract portal, secure transfer --> |
| CDRL | A003 System Model |
| MANIFEST sha256 | <!-- from MANIFEST.txt --> |
| Previous drop | v1. (#<!-- its intake issue -->) |

## Intake checklist

- [ ] Files received and scanned; hashes match `MANIFEST.txt`
- [ ] `gitlab.pl flow vendor <dropdir> v1.x` committed the drop on `vendor/prime` (vendor
pipeline green)
- [ ] The `delta` job's report reviewed: every *manual* line has a task below
- [ ] Merge MR from `feature/drop-v1.x` opened; conflicts resolved (list them below)
- [ ] Trace re-run: discrepancy reports this drop fixes are closed, new ones opened
- [ ] Merged to `develop`; CHANGELOG updated

## Conflicts and manual rework

<!-- One line per conflicted file or manual delta line, with the task that resolves it. -->
| File / element | Why | Task |
|---|---|---|
| | | # |

## Discrepancy reports

<!-- Closed by this drop: #N. Still open: #N. New: #N. -->

/label ~"type::drop" ~"type::story" ~"workflow::ready" ~"priority::1"
/assign @tom
/milestone %"Sprint "
```

The **prime drop** template is the intake checklist for one delivery. Its task list follows the drop cycle in Appendix D, Quick reference: SysML v1 to v2 (p. 719) step by step, so the issue itself

records that every step was done, by whom and when.

GitLab Markdown: tools/gitlab/templates/.gitlab/issue_templates/Epic.md

```
<!-- Epic: a collection of stories finished within ONE semester (13 sprints).
     On GitLab Premium this text goes in a group epic; on Free it is an issue with ~"type::epic"
     and the epic label
     its stories carry. -->

## Goal

<!-- The capability or state the program has when this epic closes. -->

## Scope

| In | Out |
|---|---|
| | |

## Stories

<!-- The planned stories, in order. Each fits in one sprint. Update as they are created. -->
- [ ] #
- [ ] #

## Measures

<!-- How we will know: a metric and its target, e.g. "TODO lines in
model/physical/stations/Stations.sysml: 0". -->

## Timeline

| Start | Target | Semester |
|---|---|---|
| Sprint | Sprint | |

/label ~"type::epic" ~"tome::"
/assign @jim
```

On GitLab Free there are no group epics, so an epic is an ordinary issue labelled `type::epic` whose description lists its stories. On Premium the same text goes in a real epic, and the stories are attached to it.

GitLab Markdown: tools/gitlab/templates/.gitlab/issue_templates/Spike.md

```
<!-- Spike: a time-boxed investigation (at most 5 points) that ends in a DECISION, recorded below
and in the wiki.
     It changes no model content; follow-up work becomes stories. -->

## Question

<!-- The one question this spike answers. -->

## Time box

<!-- Points (at most 5) and the date the decision is due. -->
```

```
## Options considered

| Option | For | Against |
|---|---|---|
| | | |

## Decision

<!-- Filled in at the end: what we will do, and the story that does it. -->

/label ~"type::spike" ~"workflow::ready"
/estimate 2d
```

A **spike** is time-boxed investigation that ends in a decision, not in a model change. Its output is usually a wiki decision record (sample 12 below).

I.4 Merge request templates

GitLab Markdown: tools/gitlab/templates/.gitlab/merge_request_templates/Default.md

```
<!-- A merge request is your change request (an ECP in miniature): the exact lines you changed,
the reason, the
     automated audit (the pipeline) and the owner's approval, all in one place.
     Title: what the model gains, e.g. "Close the Stations TODO lines". Prefix "Draft:" until it
     is ready to review. -->

## What and why

<!-- Two or three sentences. Which story or task, and what changes in the model. -->

Closes #<!-- task number -->
Part of #<!-- story number -->

## Model changes

<!-- The packages and elements touched. Name added, changed and removed definitions. -->
| Package | Change |
|---|---|
| | |

## Checks

- [ ] `gitlab.pl ci` passed locally (grammar, lint 0/0, outline, docs)
- [ ] `docs/` regenerated and committed
- [ ] No new `// TODO` lines (or each one has a task)
- [ ] Trace job: no new orphan, unverified or drifted requirement (or a DR is linked)
- [ ] Commit messages carry a `Changelog:` trailer where the change is user-visible

## Review notes

<!-- Where to look first, what you are unsure about. -->

/label ~"workflow::review"
```

```
/assign_reviewer @bob
/milestone %"Sprint "
```

The Default template is filled in on every new MR. Its opening comment calls the MR what it is in systems engineering terms, a change request in miniature. `Closes #` and `Part of #` connect the MR to its task and story: when the MR merges into the default branch (`develop`), GitLab closes the task automatically. **Checks** is the author's own checklist, deliberately overlapping the pipeline: the author runs the same checks before asking anyone to review. `/assign_reviewer @bob` is a default the author changes; the code owner rules decide who must approve regardless.

GitLab Markdown: tools/gitlab/templates/.gitlab/merge_request_templates/Prime Drop.md

```
<!-- Title: "Merge prime drop v1.x into develop". Source: feature/drop-v1.x. Target: develop. -->

## The drop

Intake issue: #<!-- n -->  ·  Version: v1.  ·  Delivered: <!-- date -->  ·  Vendor commit: <!--
sha on vendor/prime -->

## What the prime changed

<!-- Paste the summary from the `delta` job (artifacts: delta.txt). Keep the "manual" lines: each
needs a task. -->

<details><summary>delta.txt</summary>

```text

```

</details>

Conflicts resolved

<!-- One line per conflicted file: whose side won and what was re-applied. The rule: take the
prime's text, then
     re-apply the government's edit (closed TODO, level tag) on top of it. -->
File	Resolution

Trace after the merge

	Before	After
Linked (C)		
Orphans (F)		
Unverified (G)		
Drift		

- [ ] Discrepancy reports fixed by this drop are closed (#)
- [ ] New discrepancy reports opened (#)
- [ ] CHANGELOG `Prime drops merged` updated
```

```
/label ~"type::drop" ~"workflow::review"
/assign_reviewer @jim
```

The Prime Drop MR template carries the delta summary and a conflict table: one row per conflicted file with how it was resolved. The reviewer checks the table against the drop's manifest.

GitLab Markdown: tools/gitlab/templates/.gitlab/merge_request_templates/Release.md

```
<!-- Title: "Release baseline-N". Source: release/baseline-N. Target: main. Only Jim or Tom
merges it. -->

## Baseline

| | |
|---|---|
| Baseline | baseline- |
| Built from | develop at <!-- sha --> |
| Prime drops included | v1.<!-- x --> (#<!-- intake issue -->) |
| Milestones | %"Sprint " |

## Contents

<!-- From CHANGELOG.md [Unreleased], moved under the new baseline heading in this MR. -->

## Release checklist

- [ ] Only release fixes on this branch (version strings, CHANGELOG, docs regenerated)
- [ ] Pipeline green; trace gaps listed below are accepted by @jim
- [ ] CCB concurrence recorded as a comment on this MR by the deputy PM
- [ ] After merge: `gitlab.pl flow released <this MR's number>` tags `baseline-N` on main (the
tag pipeline checks the docs hash)
- [ ] Release created with these notes; back-merge MR main → develop merged

## Accepted gaps

<!-- Orphans, unverified requirements and open DRs that ship in this baseline, each with its DR
or story. -->

/label ~"workflow::review"
/assign_reviewer @jim
```

GitLab Markdown: tools/gitlab/templates/.gitlab/merge_request_templates/Hotfix.md

```
<!-- Title: "Hotfix baseline-N: <what>". Source: hotfix/<name>. Target: main, then the back-merge
to develop. -->

## What is wrong in the released baseline

<!-- The defect (#n) and who is affected: which analysis or review used the baseline. -->

Closes #

## The fix
```

```
<!-- The smallest change that corrects it. Anything larger waits for the next baseline. -->

## After merge

- [ ] Tag `baseline-N.1` on main (the tag pipeline checks the docs hash)
- [ ] Back-merge to develop (the same fix must not be lost)
- [ ] Tell the users of baseline-N

/label ~"type::defect" ~"priority::1" ~"workflow::review"
/assign_reviewer @jim
```

Release and hotfix MRs target `main`. The Release template records the baseline in a two-column form (which commit of `develop`, which prime drops, which sprints), the release checklist and the gaps accepted with it. The Hotfix template asks for what is wrong in the released baseline, the smallest change that corrects it, and an after-merge checklist: tag `baseline-N.1`, back-merge to `develop`, tell the baseline's users. After either merges, `gitlab.pl flow released IID` tags `main`, creates the release and opens the back-merge MR.

I.5 Repository files

I.5.1 CODEOWNERS

tools/gitlab/templates/CODEOWNERS

```
# CODEOWNERS -- who must approve a change to which part of the Tidewater model repo.
# GitLab reads this file from the MR's TARGET branch (CODEOWNERS, docs/CODEOWNERS or
.gitlab/CODEOWNERS, first found).
# With "code owner approval" on a protected branch (project.conf [branches] codeowners:yes), an
MR that touches a path
# below needs an approval from one of that path's owners. The last matching pattern in a section
wins.
#
# Sections group the rules; each section is checked on its own, so one MR can need an approval
from several sections.
# [Name][2] would require two approvals from that section; ^[Name] makes a section optional
(advisory only).

# Every section names at least two owners: GitLab never lets an author approve their own MR, so a
section whose only
# owner wrote the change could never be satisfied. The model folders are the layers of the layout
map
# (examples/tidewater-v1/layout.map): library -> requirements -> logical -> physical -> analysis
-> verification.
[Pipeline and repository] @tom @jim
/.gitlab-ci.yml
/ci/
/.ci/
/.gitlab/
/.gitignore
/.kit-version
/CODEOWNERS             @tom @jim
/sysml.conf             @tom @bob

# Decisions: Jim records them, Bob (or Tom) reviews.
```

```
[Decisions] @jim @bob
/docs/adr/

[Library and conventions] @bob @jim
/model/library/

# Interfaces are where segments meet, so an interface change needs TWO approvals: the systems
lead's (Jim decides every
# interface; Jim does not author interface changes) and one of the people who build on both sides
(Bob or Tom,
# whoever did not write the change). Two sections, each must be satisfied.
[Systems lead] @jim
/model/library/Interfaces.sysml

[Interfaces] @bob @tom
/model/library/Interfaces.sysml

# Requirements belong to the government: Jim approves every change to them, from either converter
(Bob when Jim wrote it).
[Requirements and verification] @jim @bob
/model/requirements/
/model/verification/
/verification/

[Logical and physical] @bob @jim
/model/logical/
/model/physical/
/model/legacy/

[Analysis] @jim @bob
/model/analysis/

# Raw prime drops arrive through vendor/prime; Tom imports them, Jim accepts them.
[Prime drops] @tom @jim
/imports/

# Generated docs: whoever changed the model regenerates them; advisory only.
^[Generated docs] @bob @tom
/docs/
```

`CODEOWNERS` is not Markdown but it lives with the templates. Each `[Section]` lists default owners after its name; each path below it can name its own. GitLab reads the file from the MR's target branch. A `^` before a section makes it advisory: the generated `docs/` are listed so reviewers are suggested, but no approval is required for them. With code owner approval enforced on `develop` (Premium), an MR touching `model/doors/` cannot merge until Jim approves.

I.5.2 README, CHANGELOG, release notes

GitLab Markdown: tools/gitlab/templates/README.md

```
# Tidewater system model (SysML v2 text)

[[_TOC_]]

> [!note]
```

```
> Tidewater is a fictional coastal weather-observation network used to teach a model-based
systems engineering
> (MBSE) process. Every name and value in this repository is notional.

This repository holds the **government's SysML v2 text model** of Tidewater. The prime
contractor, Northwind Marine
Systems, keeps the authoritative SysML v1 model in Cameo and the requirements in DOORS, and
delivers them as model
drops under CDRL A003. The government team ports each drop to SysML v2 text here, closes what the
converter cannot
carry, and keeps the v2 model current as new drops arrive.

| Who | Role | GitLab |
|---|---|---|
| Jim | MBSE lead, responsible for the model; approves requirement changes and baselines | @jim |
| Bob | MBSE engineer | @bob |
| Tom | Configuration management engineer; runs the pipeline and the vendor branch | @tom |

## Layout

| Path | What | Who changes it |
|---|---|---|
| `model/library/` | types, interfaces and the government's conventions (`library/conventions/`)
| Bob, by MR; interfaces need Jim too |
| `model/requirements/` | the three DOORS modules (`stakeholder/`, `system/`, `segment/`, from
the ReqIF) and the Cameo copy (`model/`) | the drop intake; Jim approves |
| `model/logical/` | what the network does: the behavior, and the hand-written
`GovBehavior.sysml` | Bob, by MR |
| `model/physical/` | what it is made of: one folder per segment, and `allocation/` | Bob, by MR
|
| `model/analysis/`, `model/verification/` | constraints and analyses; verification cases | Jim,
Bob |
| `model/legacy/` | clutter from the document port (gone after drop 6) | nobody: a DR asked the
prime to delete it |
| `imports/<version>/` | each prime drop exactly as delivered, with its `MANIFEST.txt` | Tom,
through `vendor/prime` |
| `docs/` | generated by `model.pl docs`; never edit by hand | the pipeline checks it |
| `.gitlab/` | issue and merge request templates | Tom |

## Branches (gitflow)

```mermaid
gitGraph
  commit id: "skeleton"
  branch develop
  checkout develop
  branch vendor/prime
  checkout vendor/prime
  commit id: "drop v1.0"
  checkout develop
  merge vendor/prime id: "port starts"
  branch feature/12-stations
  commit id: "close TODOs"
  checkout develop
  merge feature/12-stations
  branch release/baseline-1
  commit id: "release notes"
  checkout main
  merge release/baseline-1 tag: "baseline-1"
```
```

```
  checkout develop
  merge main id: "back-merge"
```

- `develop` is the default branch. Every change arrives there by merge request from a
`feature/<issue>-<slug>` branch.
- `main` holds released baselines only, merged from `release/baseline-N` and tagged `baseline-N`.
- `vendor/prime` holds the raw converter output of each drop and is never edited by hand. A drop
reaches `develop`
  through a `feature/drop-<version>` branch that merges it.
- `hotfix/<name>` branches fix a released baseline and merge to `main` and back to `develop`.

The full rules are in the wiki page [Gitflow](../-/wikis/Gitflow).

Working on a story

```sh
perl $KIT/bin/gitlab.pl flow start feature 42-close-stations-todos   # from develop
# ... edit model/, then:
perl $KIT/tools/sysml/model.pl lint && perl $KIT/tools/sysml/model.pl docs
git add -A && git commit -m "Close the Stations TODO lines" -m "Closes #42"
perl $KIT/bin/gitlab.pl ci --event merge_request_event --branch feature/42-close-stations-todos
--target develop
perl $KIT/bin/gitlab.pl flow finish --title "Close the Stations TODO lines" --template
.gitlab/merge_request_templates/Default.md
```

The pipeline

`check` (grammar, lint 0/0, outline, secrets, TODO count) → `trace` (DOORS against the model) →
`docs` (generated
docs current) → `delta` (drop intake only) → `release` (baseline tags only) → `report` (weekly
schedule only). See
`.gitlab-ci.yml`; every job runs locally with `gitlab.pl ci`.

New to this?

Start with the wiki pages [Glossary](../-/wikis/Glossary) (systems engineering and CM terms, and
their GitLab names)
and [Onboarding](../-/wikis/Onboarding).

Getting help

- Process questions: the wiki ([Home](../-/wikis/Home)).
- A problem in the v2 model: open an issue with the **Defect** template.
- A problem in the prime's v1 model or DOORS: open an issue with the **Discrepancy Report**
template; Jim sends it to
  the prime.

The README is the first page anyone sees in the repository. It uses an alert to say up front that everything is notional, a table for who does what, a layout table and a mermaid diagram of the branches.

GitLab Markdown: tools/gitlab/templates/CHANGELOG.md

```
# Changelog
```

```
All notable changes to the Tidewater v2 model, one section per baseline. The newest is first.
Entries come from the
merge requests' `Changelog:` commit trailers (see the wiki page *Definition of Done*).

## [Unreleased]

### Added
- <!-- one line per MR: what changed, then the MR reference, e.g. "Hygrometer in the sensor suite
(!57)" -->

### Changed

### Fixed

### Prime drops merged
- <!-- version, delivery date, intake issue, e.g. "v1.1, 2027-01-08 (#88)" -->

## [baseline-1] -- 2027-03-19

### Added
- The full Tidewater structure ported from prime drop v1.0: 12 packages, 40 part definitions (!3
to !21).
- Requirements from DOORS with their `doorsId`, and the satisfy/verify trace (!22 to !30).
- Hand-written SysML v2 for what the converter could not carry: the `BuoyModes` states, the
`DailyUpload`
  sequence, the `EnergyBalance` constraint and the observation-cycle fork (!31 to !38).

### Prime drops merged
- v1.0, 2026-10-02 (#5)
- v1.1, 2027-01-08 (#61)

[Unreleased]: ../-/compare/baseline-1...develop
[baseline-1]: ../-/tags/baseline-1
```

The CHANGELOG follows the common *Keep a Changelog* layout: one section per baseline, newest first, with Added, Changed and Fixed. Its entries come from the `Changelog:` trailer the team puts in commit messages (sample 13). The link at the bottom compares the last baseline with `develop`; `../-/compare/` is GitLab's relative path for a comparison page.

GitLab Markdown: tools/gitlab/templates/release-notes.md

```
<!-- Release notes for baseline-N: the description of the GitLab release (gitlab.pl release
baseline-N --notes F).
     Start from the CHANGELOG section of the baseline. Keep it to what a reader of the baseline
     needs. -->

# Tidewater model baseline-<!-- N -->

**Released** <!-- date --> · **Tag** `baseline-<!-- N -->` · **From** prime drop v1.<!-- x --> ·
**Milestones** %"Sprint " to %"Sprint "

> [!important]
> Cite this baseline by its tag. The tag pipeline proved it rebuilds byte for byte (`sha256=` in
the tag message).

## What is in it
```

```
| | Count |
|---|---|
| Packages | |
| Part definitions | |
| Requirements (from DOORS) | |
| Satisfied and verified | % |
| TODO lines left | |

## Changes since the previous baseline

<!-- From CHANGELOG.md: Added / Changed / Fixed, each line with its MR (!n). -->

## Prime drops included

| Version | Delivered | Intake |
|---|---|---|
| v1. | | # |

## Known gaps (accepted)

<!-- Each open orphan, unverified requirement or DR shipped in this baseline, with its issue. -->
- [ ]

## Evidence

- Tag pipeline: <!-- link --> (`baseline` job)
- Trace report: the `trace` job artifact of the release MR !<!-- n -->
```

Release notes are the description of a GitLab release. `> [!important]` makes the "cite this baseline by its tag" rule impossible to miss. The **Known gaps** task list records what the baseline ships with on purpose, so nobody mistakes an accepted gap for an oversight.

I.5.3 The project file

tools/gitlab/templates/project.conf

```
# project.conf -- the Tidewater model project, as `gitlab.pl setup` applies it
(tools/gitlab/templates/project.conf).
#
#   perl bin/gitlab.pl --url https://gitlab.example.com --token-env GITLAB_TOKEN setup
tools/gitlab/templates/project.conf
#   perl bin/gitlab.pl --local data/tidewater/gitlab setup tools/gitlab/templates/project.conf
  (the offline emulator)
#
# Every setting here is something a maintainer would otherwise click through in Settings. Keeping
it in a file means the
# project can be rebuilt, reviewed in a merge request, and compared with what the server actually
has. Re-running setup
# is safe: it creates what is missing and updates what differs.
# Tidewater is fictional; the people are Jim (MBSE lead, responsible for the model), Bob (MBSE
engineer) and Tom
# (configuration management engineer, who also runs the pipeline). None of them had done
DevSecOps before this year.
```

```
[project]
path = tidewater/model
default_branch = develop                                    ; gitflow: develop is where work lands,
so issues close there
merge_method = merge                                        ; a merge commit for every MR: the
metrics read merge history
squash_option = never                                       ; keep each commit; squash would hide
who changed what
only_allow_merge_if_pipeline_succeeds = true
only_allow_merge_if_all_discussions_are_resolved = true
remove_source_branch_after_merge = true
visibility = private

[members]
jim = maintainer                                            ; MBSE lead: owns tomes, epics,
baselines and releases
bob = developer                                             ; MBSE engineer: ports and maintains
the model
tom = maintainer                                            ; CM: pipeline, runners, vendor branch,
tags

[branches]
main = push:none merge:maintainer force:no codeowners:yes          ; released baselines only,
through release/hotfix MRs
develop = push:none merge:developer force:no codeowners:yes        ; integration; every change
arrives by MR
release/* = push:maintainer merge:maintainer force:no
hotfix/* = push:maintainer merge:maintainer force:no
vendor/prime = push:maintainer merge:none force:no                 ; raw converter output of each
prime drop, never hand-edited

[tags]
baseline-* = create:maintainer                              ; a baseline is permanent: nobody can
move or delete it

[labels]
; type -- what kind of work item it is (scoped: an issue has exactly one type)
type::epic = #5843AD | A collection of stories finished within one semester (13 sprints)
type::story = #428BCA | A slice of value of several tasks that fits in one sprint
type::task = #69D100 | One piece of work under a story, less than a week (5 points or fewer)
type::defect = #D9534F | Something in the government v2 model or pipeline is wrong
type::discrepancy = #AD4363 | A problem in the prime's v1 model or DOORS, reported to the prime
(Discrepancy Report)
type::drop = #00B140 | Intake of one prime model drop (XMI + ReqIF + CSV under CDRL A003)
type::spike = #8E44AD | Time-boxed investigation that ends in a decision, not a model change
; tome -- the year-scale grouping (< 2 semesters)
tome::T1-stand-up = #36454F | T1 Stand up MBSE DevSecOps (sprints 1-8)
tome::T2-port = #36454F | T2 Port Tidewater v1 to SysML v2 (sprints 1-20)
tome::T3-maintain = #36454F | T3 Maintain against prime drops (sprints 17-39)

epic::E1-gitlab = #6699CC | E1 GitLab project and gitflow
epic::E2-pipeline = #6699CC | E2 Pipeline and quality gates
epic::E3-practices = #6699CC | E3 Team practices and onboarding
epic::E4-toolchain-security = #6699CC | E4 Toolchain security and authorization to use
epic::E5-drop-1 = #6699CC | E5 Drop 1 intake and first import
epic::E6-structure = #6699CC | E6 Structure: close the structure TODOs
epic::E7-requirements = #6699CC | E7 Requirements and trace
epic::E8-behavior = #6699CC | E8 Behavior and analysis
epic::E9-drops-2-3 = #6699CC | E9 Drops 2 and 3 during the port
```

```
epic::E10-baseline-1 = #6699CC | E10 Baseline 1 and acceptance
```

The full file continues with the area, owner, workflow, priority and severity labels, the sprint milestones, the approval rule and the variables. `gitlab.pl setup` applies it (Appendix A, Command reference (p. 696)); every comment beside a setting says why that setting was chosen, which is the record an auditor or a newcomer needs.

I.6 Wiki pages

The wiki is a git repository of Markdown pages beside the code. The team's pages started from these templates and were changed by the team over the project; `gitlab.pl wiki put TITLE FILE` publishes one.

GitLab Markdown: tools/gitlab/templates/wiki/Home.md

```
# Tidewater model wiki

[[_TOC_]]

> [!note]
> Tidewater is fictional. This wiki shows how a government model team moves from documents and a
Cameo model to a
> model kept the DevSecOps way, two weeks at a time.

## Why we are doing this

For decades we did systems engineering with documents, spreadsheets and briefings. A few years
ago we were told to move
into Cameo, and we ported our documents into a SysML v1 model. It became a giant spider web:
every element linked to
every other, nobody could see what had changed or why, a review meant clicking through diagrams,
and it did not scale.
The prime contractor now maintains that v1 model and sends us copies (drops).

This year we were directed to follow DevSecOps. None of us has done it before. That is fine:
**most of it is
configuration management, which we already know, done in smaller pieces and checked by machine.**
This wiki translates
every new word into the one you already use. Start with the [Glossary](Glossary).

## What changes, and what stays the same

| What stays | What changes |
|---|---|
| A master copy under configuration control | The master copy is text files (SysML v2), so every
change shows line by line, like a redline |
| Change proposals reviewed and approved before they go in | A change proposal (merge request) is
days of work, not months, and the approval is on line |
| Baselines that never change | A baseline is a protected tag; the server will not let anyone
move it, and a machine proves it rebuilds |
| Configuration audits | The audit (the pipeline) runs automatically on every change, in minutes
|
| Action items with owners and dates | Action items are issues on a board everyone sees |
| The prime delivers under a CDRL | Each delivery (drop) is converted and merged into our model
automatically, keeping our own edits |
```

```
| We report status every week | The status is computed from the work itself, not typed up on
Thursday night |

## Why it will not become another spider web

- **Small pieces.** The model is split into packages, one folder each, each with an owner. A
change touches one or two.
- **Every change is reviewed and checked** before it reaches the master copy. Nothing goes in by
accident.
- **Text, not diagrams, is the master.** Diagrams are drawn from the text on demand, so they can
never disagree with it.
- **History is complete.** For any line of the model you can see who changed it, when, and under
which change.
- **Rules are written once and enforced by the server**, not remembered by people.

## The team

| Who | Role | Owns |
|---|---|---|
| @jim | MBSE lead, responsible for the model | the plan (tomes, epics, the order of the
backlog), baselines, discrepancy reports to the prime, approving requirement changes |
| @bob | MBSE engineer | the model content: porting, closing what the converter could not
translate, reworking each drop |
| @tom | Configuration management engineer | the pipeline (our automated audit), the vendor
branch where drops arrive, tags and releases |

## How we size the work

We plan by time, from the biggest to the smallest. One point is one ideal person-day.

| Level | Size | Like | In GitLab |
|---|---|---|---|
| Project | years (this one: 18 months) | the program | the group `tidewater` |
| Tome | under 1 year | a major WBS element | label ~"tome::port" |
| Epic | under a semester (13 sprints) | a WBS work package | an issue ~"type::epic" and a label
~"epic::structure" |
| Story | under one sprint (2 weeks) | a deliverable you can show at a review | an issue
~"type::story" in milestone %"Sprint 4" |
| Task | under one week (5 points or fewer) | an action item | an issue ~"type::task" that says
"Part of #18" |

## The two weeks (a sprint)

| When | Event | Takes | Like |
|---|---|---|---|
| Day 1 | Planning: pick the stories that fit, cut them into tasks | 1 hour | a work-package
kickoff |
| Every day | Stand-up: yesterday, today, blockers | 15 minutes | a daily tag-up |
| Mid-sprint | Refinement: get the next stories ready | 1 hour | a pre-brief |
| Day 10 | Review: show what is done, in the model | 1 hour | a design review, small and frequent
|
| Day 10 | Retrospective: what to keep, what to change | 30 minutes | lessons learned, every two
weeks instead of at the end |

## Pages

- [Glossary](Glossary): systems engineering and CM words, and their GitLab names
- [Onboarding](Onboarding): your first day and first week
- [Definition of Ready](Definition-of-Ready) and [Definition of Done](Definition-of-Done)
```

```
- [Gitflow](Gitflow): how changes flow between the master copy, the baselines and the drops
- [Drop Intake Runbook](Drop-Intake-Runbook): a new prime drop, from receipt to merged
- [Baseline Procedure](Baseline-Procedure): cutting `baseline-N`
- [Administration](Administration): what keeps the project running, and who does it
```

GitLab Markdown: tools/gitlab/templates/wiki/Definition of Ready.md

```
# Definition of Ready

The entry criteria for a sprint: the checklist a story must meet before the team commits to it,
so nobody starts
work that is not understood. (Like the entry criteria of a review: if they are not met, the item
waits.)

A story can be pulled into a sprint (moved to %"Sprint N" and ~"workflow::ready") when:

- [ ] It fits in one sprint, and every task under it is at most 5 points (one point = one ideal
person-day).
- [ ] It carries exactly one ~"type::story", one ~"tome::..." and one ~"epic::..." label.
- [ ] The acceptance criteria are written as a checklist someone else can verify.
- [ ] The v1 elements (xmi:id) and DOORS ids it touches are named.
- [ ] Dependencies are linked: on the prime (a DR or a drop), or on another story (`/relate #N`,
*blocked by*).
- [ ] Jim has ordered it on the backlog board.

A **task** is ready when it names its story ("Part of #N"), has an estimate (`/estimate`) and an
assignee.

> [!tip]
> If a story does not fit in a sprint, split it by package, by requirement module, or by "convert
/ close TODOs /
> review". Never carry a story on purpose.
```

GitLab Markdown: tools/gitlab/templates/wiki/Definition of Done.md

```
# Definition of Done

"Done" means the same thing for everyone, and someone other than the author can check it. Think
of it as the closure
criteria of an action item, written once for every item of that kind.

## A task is done when

- [ ] **Its merge request is merged into `develop`**, the master copy. (The MR description says
`Closes #task`, so the
      task closes by itself.)
- [ ] **The pipeline passed.** These are the automated audit checks, each in plain words:

  | Check | What it means |
  |---|---|
  | `grammar` | every model file is valid SysML v2 text |
  | `lint` **0/0** | every name points at something that exists; zero errors, zero warnings |
  | `outline` | every level of the part tree has 2 to 9 children, and each part carries its level
  tag |
```

```
  | `secrets` | no password, token or key was committed |
  | `docs` | the generated documents match the model |

- [ ] **It adds no new `// TODO` line**, unless a task exists for it. (A TODO line is something
the converter could not
      translate; a person must close it.)
- [ ] **The owner approved it.** For a requirement change that is Jim; for structure, Bob or Jim;
for the pipeline, Tom.

## A story is done when

- [ ] Every task under it is closed.
- [ ] Its acceptance criteria are checked, by someone other than the author.
- [ ] The **trace** check shows no *new* gap: no new requirement without something that satisfies
it (an *orphan*), no
      new requirement without a verification (*unverified*), and no new difference between DOORS
      and the model
      (*drift*). A gap that is the prime's to fix has a discrepancy report.
- [ ] It was shown at the sprint review, in the model or on the model's web site.

## A drop intake is done when

- [ ] The drop is on the vendor branch `vendor/prime`, filed in the package registry, and merged
into `develop`.
- [ ] Every merge conflict and every *manual* line of the delta report is resolved, with a task
for each.
- [ ] Discrepancy reports this drop fixes are closed; new ones are open and sent to the prime.

## Writing the change's reason (the commit message)

A commit message is the "reason for change" block. Write a short subject saying what the change
does, a blank line,
why, and then trailers that GitLab and the changelog read:

```text
Close the BuoyModes TODO with a state definition

The converter does not translate state machines. Written from the v1
diagram "stm BuoyModes" (drop v1.0, xmi:id _22_1_7f3a0c1_9c2e...).

Changelog: added
Issue: #23
Reviewed-by: Jim <jim@example.invalid>
```

> [!tip]
> If you cannot meet one of these, say so in the MR and ask. A known, written exception is fine;
a silent one is not.
```

The Definition of Done is a task list grouped by level (task, story, sprint, baseline). Pasting it into a story's comment gives a checklist to tick; keeping it in the wiki gives one place to change it.

GitLab Markdown: tools/gitlab/templates/wiki/Gitflow.md

```
# Gitflow
```

```
[[_TOC_]]

Gitflow is the set of rules for how changes move between working copies, the master copy and the
released baselines.
If you have run configuration management, you know the ideas; only the names are new:

| You know | In gitflow |
|---|---|
| The controlled master copy | `develop` |
| Released baselines | `main`, where each baseline is a tag `baseline-N` |
| A working copy for one change | a feature branch, `feature/<issue>-<what>` |
| Preparing a baseline for release | a release branch, `release/baseline-N` |
| An urgent correction to a released baseline | a hotfix branch, `hotfix/<what>` |
| The contractor's deliveries, kept as received | `vendor/prime` |
| Incorporating an approved change | a merge, through a merge request |

## Branches

| Branch | From | Into | Who pushes | Who merges | Lives |
|---|---|---|---|---|---|
| `main` | | | nobody | maintainers (Jim, Tom) | forever; baselines only |
| `develop` | `main` | | nobody | developers, after approval | forever; the default branch |
| `feature/<iid>-<slug>` | `develop` | `develop` | its author | | until merged (deleted) |
| `release/baseline-N` | `develop` | `main` | maintainers | maintainers | days |
| `hotfix/<name>` | `main` | `main`, then `develop` | maintainers | maintainers | hours |
| `vendor/prime` | first commit | `develop` (through `feature/drop-*`) | Tom only | nobody |
forever; never hand-edited |

Tags `baseline-N` are protected: only maintainers create them, nobody moves or deletes them.

## The picture

```mermaid
gitGraph
  commit id: "skeleton"
  branch vendor/prime
  commit id: "drop v1.0"
  checkout main
  branch develop
  merge vendor/prime id: "feature/drop-v1.0"
  branch feature/12-stations
  commit id: "Stations TODOs"
  checkout develop
  merge feature/12-stations
  checkout vendor/prime
  commit id: "drop v1.1"
  checkout develop
  merge vendor/prime id: "feature/drop-v1.1"
  branch release/baseline-1
  commit id: "CHANGELOG"
  checkout main
  merge release/baseline-1 tag: "baseline-1"
  checkout develop
  merge main id: "back-merge"
```

Why gitflow here
```

````
- The model is **released** as baselines that analyses cite; `main` is exactly the list of those
baselines.
- Day-to-day work never waits on a release: it keeps merging into `develop`.
- The prime's drops come in on their own branch, so git's three-way merge carries the
government's edits (closed TODO
  lines, level tags) across every new drop. Conflicts appear only where both sides changed the
  same lines.

## Commands

```sh
perl $KIT/bin/gitlab.pl flow start feature 42-close-stations-todos
perl $KIT/bin/gitlab.pl flow publish
perl $KIT/bin/gitlab.pl flow finish --title "Close the Stations TODO lines"
perl $KIT/bin/gitlab.pl flow start release baseline-1
perl $KIT/bin/gitlab.pl flow finish --template .gitlab/merge_request_templates/Release.md
perl $KIT/bin/gitlab.pl flow vendor /secure/drops/2027-01-08 v1.1
perl $KIT/bin/gitlab.pl flow sync
```
````

GitLab Markdown: tools/gitlab/templates/wiki/Drop Intake Runbook.md

````
# Drop Intake Runbook

A prime drop is the prime contractor's CDRL delivery of its v1 model: an XMI export of the Cameo
model, a ReqIF and a CSV export of DOORS, and a `MANIFEST.txt`, delivered
under CDRL A003. Intake is one story (template **Prime Drop**), owned by @tom, finished in the
sprint it arrives.

```mermaid
flowchart LR
  A[Drop received] --> B[Hashes match MANIFEST]
  B --> C[flow vendor: commit on vendor/prime]
  C --> D[delta job: what changed in v1]
  D --> E[feature/drop-vX merges vendor/prime]
  E --> F{Conflicts?}
  F -- yes --> G[Take the prime's text, re-apply the government edit]
  F -- no --> H[MR to develop]
  G --> H
  H --> I[Trace: close fixed DRs, open new ones]
  I --> J[Merged; CHANGELOG]
```

## Steps

1. **Receive.** Put the files in a dated folder outside the repo. Check them:
   ```sh
   cd /secure/drops/2027-01-08 && sha256sum -c MANIFEST.sha256   # or compare with the hashes in
   MANIFEST.txt
   ```
2. **Open the intake issue** from the *Prime Drop* template in the current sprint.
3. **Import on the vendor branch.** From a clean clone:
   ```sh
   perl $KIT/bin/gitlab.pl flow vendor /secure/drops/2027-01-08 v1.1
   ```
   This converts the drop (`xmi2sysml.pl`, `reqif2sysml.pl`), replaces `model/` and adds
   `imports/v1.1/` on
````

```
   `vendor/prime`, pushes it, then starts `feature/drop-v1.1` from `develop` and merges
   `vendor/prime` into it. It lists
   any conflicted files.
4. **Read the delta.** The vendor pipeline's `delta` job (artifact `delta.txt`) lists what the
prime added, removed and
   changed in v1. A line marked **manual** is something the converter does not carry (a state
   machine, an interaction,
   a constraint expression): add a task for each.
5. **Resolve conflicts.** For each file: take the prime's version, then re-apply the government's
edit on top.
   ```sh
   git checkout --theirs model/analysis/Analysis.sysml   # the prime's text
   # re-apply the closed TODO / level tags, then:
   git add model/analysis/Analysis.sysml && git commit
   ```
6. **Open the MR** with the *Prime Drop* template; paste the delta summary and the conflict
table.
7. **Trace.** The `trace` job shows linked, orphan, unverified and drifted requirements. Close
the DRs this drop fixes;
   open new ones for what it breaks.
8. **File the drop.** The vendor pipeline's `drop-package` job puts the delivered files in the
project's package
   registry (Deploy > Package registry > `prime-drop`, version `v1.1`): the deliverable kept with
   what it became.
9. **Merge**, update `CHANGELOG.md`, close the intake issue.

> [!warning]
> Never edit `vendor/prime` by hand. If a drop is wrong, raise a DR and wait for the next drop;
the vendor branch must
> always be exactly "what the converter makes of what the prime delivered".
```

GitLab Markdown: tools/gitlab/templates/wiki/Baseline Procedure.md

```
# Baseline Procedure

A baseline is the model as released for analyses and reviews, exactly as you have always meant
it: a merge to `main`, an annotated tag `baseline-N`, and a
GitLab release with notes. It must rebuild: the tag pipeline regenerates `docs/` and checks its
hash against the one
recorded in the tag message. That is the physical configuration audit, done by machine in a
minute; the functional
audit is the trace report (every requirement satisfied and verified, or an accepted gap).

## Steps

1. Jim decides the content: which drops and stories are in. Move what is not ready to the next
milestone.
2. Cut the release branch and fill the *Release* MR template:
   ```sh
   perl $KIT/bin/gitlab.pl flow start release baseline-1
   # move CHANGELOG [Unreleased] under "## [baseline-1] -- date", regenerate docs, commit
   perl $KIT/bin/gitlab.pl flow finish --template .gitlab/merge_request_templates/Release.md
   ```
3. The deputy PM records the CCB's concurrence as a comment on the release MR; a code owner
approves; the MR merges
   into `main`. Then, with the MR's number:
```

```
   ```sh
   perl $KIT/bin/gitlab.pl flow released 42
   ```
   `flow released` tags `baseline-1` on the merge commit (message `sha256=<docs hash>`, the hash
   of
   `git ls-tree -r <commit> -- docs`) and opens the back-merge MR `main → develop`. The tag
   pipeline audits the
   baseline, writes its toolchain and VCRM, then its `publish-release` job creates the GitLab
   release with the
   CHANGELOG section, the verification summary and the toolchain as notes.
4. Check the tag pipeline: the `baseline` job prints `baseline-1 rebuilds identically`.
5. Close the milestones the baseline covers.

Baseline	When	Contents
baseline-1	end of the port	drop v1.0 ported, TODOs closed, trace complete but for accepted
gaps
baseline-N	after each drop merged in maintenance	the drop, its rework, closed DRs
```

GitLab Markdown: tools/gitlab/templates/wiki/Onboarding.md

```
# Onboarding

Your first day and your first week on the Tidewater model team. You do not need to have used Git,
GitLab or DevSecOps
before; nobody on this team had. Every new word is in the [Glossary](Glossary).

> [!tip]
> The fastest way to learn this is one small change, start to finish, with someone beside you. Do
the first task below
> with Tom or Bob on a call.

## What you already know

You have controlled configurations, reviewed changes, run audits and tracked action items. Here
they are in GitLab:

| You know | Here it is |
|---|---|
| The controlled master copy | the `develop` branch of the repository `tidewater/model` |
| A change request / ECP | a **merge request** (MR): your proposed change, shown line by line |
| CCB approval | an **approval** on the MR from the owner of what you changed |
| A configuration audit | the **pipeline**: checks the server runs on every MR, in minutes |
| An action item | an **issue**, assigned to you, on the sprint board |
| A released baseline | a **tag** `baseline-N` that nobody can move |

## Day 1: accounts and the laptop

1. **GitLab account.** Jim adds you to the project `tidewater/model` (as a *developer*;
*maintainer* if you cut
   baselines or manage drops).
2. **Git for Windows.** It is already on the standard laptop image. It brings Git (the
version-control program), a
   terminal called Git Bash, and Perl (which runs the kit's model tools). Nothing else to
   install.
3. **Get your own copy of the model** (a *clone*). Open Git Bash:
```

```
   ```sh
   git clone https://gitlab.example.com/tidewater/model.git     # your copy, with all its history
   cd model
   git config core.autocrlf false                               # keep files byte for byte as
   they are
   export KIT=/c/Users/you/agile                                 # where the agile kit is on your
   laptop
   ```
4. **Check the model on your laptop**, the same way the server will:
   ```sh
   perl $KIT/tools/sysml/model.pl lint      # the structure check; it should end: errors=0
   warnings=0
   perl $KIT/tools/sysml/model.pl install-hooks   # repeats that check every time you commit
   ```
5. **Look around GitLab** (the web page): *Plan > Issue boards* (the sprint board), *Code > Merge
requests* (changes in
   review), *Build > Pipelines* (the audits), *Plan > Wiki* (this page).

Days 2 to 5: your first task, start to finish

Pick a task on the board in the ~"workflow::ready" column with the label ~"type::task", and
assign it to yourself.
Then:

Step	What you do	What it is like
1	`perl $KIT/bin/gitlab.pl flow start feature 42-short-name`	check out a working copy of
the master
2	Edit the `.sysml` file in Vim (or any editor)	mark up your copy
3	`perl $KIT/tools/sysml/model.pl lint`	self-check before you submit
4	`git add -A` then `git commit -m "What you changed"`	record the revision, with a reason
5	`perl $KIT/bin/gitlab.pl ci`	run the audit yourself first
6	`perl $KIT/bin/gitlab.pl flow finish --title "What you changed"`	submit the change
request (the MR)
7	Answer the reviewer's comments, push fixes	respond to redlines
8	The owner approves; the MR merges	the CCB approves and the change is incorporated

When the MR merges, your task issue closes by itself (the MR says "Closes #42").

Read, in this order

1. Glossary
2. [Definition of Ready](Definition-of-Ready) and [Definition of Done](Definition-of-Done)
3. Gitflow
4. [Drop Intake Runbook](Drop-Intake-Runbook)
5. The kit's `docs/GITLAB.html` (the long version of this wiki)

Who to ask

About	Ask
What to work on, priorities, the prime	@jim
The model, SysML v2, the converter's TODO lines	@bob
Git, branches, the pipeline, a red check you do not understand	@tom

> [!note]
> A red check is not a failure on your part. It is the audit doing its job before the change
reaches the master
```

```
> copy, which is exactly where we want problems found. Read the job's log, or ask Tom.
```

GitLab Markdown: tools/gitlab/templates/wiki/Administration.md

```
# Administration

What Tom (configuration management) and Jim (the project owner) keep running, and how often. None
of this is
new in kind: it is the upkeep a CM office always did (access lists, backups, audits), with GitLab
doing the recording.

| What | How often | Who | How |
|---|---|---|---|
| Project settings match `project.conf` | each change; monthly check | Tom | `gitlab.pl setup
project.conf` (idempotent) |
| Sprint milestones exist two sprints ahead | each planning | Jim | `gitlab.pl sprints 2026-10-05
39` |
| Runner healthy, kit current on it | weekly | Tom | Admin > Runners; `git -C $KIT pull` on the
runner |
| Pipeline schedule (weekly status) | weekly | Tom | Build > Pipeline schedules |
| Access tokens rotated | before expiry (90 days) | token owner | Settings > Access tokens;
update the CI/CD variable |
| Members reviewed | quarterly; on every departure | Jim | Manage > Members: remove leavers the
same day |
| Audit events reviewed | monthly | Jim | Secure > Audit events (Premium) |
| Backups | daily (server); per baseline (export) | instance admin; Tom | `gitlab-backup create`;
project export per baseline |
| GitLab upgrades | per the instance policy | instance admin | read the release notes for CI and
API changes |

## Runner

A Windows shell runner uses the same Git for Windows Perl as the team's laptops. GitLab Runner's
Windows shell executor
runs PowerShell (`pwsh`); every job is a single `perl ci/job.pl NAME` line, so the same pipeline
works in PowerShell,
Git Bash and Docker alike:

```toml
[[runners]]
  name = "mbse-win-01"
  executor = "shell"
  shell = "pwsh"            # the Windows default; job lines are plain `perl ci/job.pl NAME`, so
  no sh syntax is needed
  environment = ["KIT=C:/agile"]   # where the kit is checked out on this machine
```

When someone leaves

- [ ] Remove them from the project and group the same day
- [ ] Revoke their tokens; rotate any shared CI/CD variable they could read
- [ ] Reassign their open issues and MRs
- [ ] Update `CODEOWNERS` if they were an owner
```

The Glossary wiki page (`wiki/Glossary.md`) carries the same translations as Appendix K, Glossary (p. 796), in the wiki's tables.

I.7 Filled-in samples

Each sample is what one GitLab field looks like after the team has filled it in, set in sprints 1 to 6. The index:

GitLab Markdown: tools/gitlab/samples/README.md

```
# GitLab Markdown, field by field -- filled-in samples for Tidewater (sprints 1 to 6)

Each file is what one GitLab text field looks like when the Tidewater team has filled it in. The
templates they came
from are in `../templates/`. Tidewater is fictional; Jim, Bob and Tom are the government team.

| File | GitLab field | Where you type it |
|---|---|---|
| `01-epic.md` | Epic description (Premium) or the description of a ~"type::epic" issue | Plan >
Epics, or a new issue |
| `02-story.md` | Issue description, story | Plan > Issues > New issue, template *Story* |
| `03-task.md` | Issue description, task | template *Task* |
| `04-defect.md` | Issue description, defect | template *Defect* |
| `05-discrepancy-report.md` | Issue description, discrepancy report to the prime | template
*Discrepancy Report* |
| `06-prime-drop.md` | Issue description, drop intake | template *Prime Drop* |
| `07-merge-request.md` | Merge request description | Code > Merge requests > New, template
*Default* |
| `08-review-thread.md` | MR discussion: review comments, a suggestion, replies, quick actions |
the MR's Overview tab |
| `09-issue-comments.md` | Issue comments: stand-up notes, time spent, blocked, quick actions |
the issue's Activity |
| `10-milestone.md` | Milestone description (one sprint) | Plan > Milestones |
| `11-release.md` | Release notes | Deploy > Releases |
| `12-wiki-decision.md` | Wiki page (a decision record) | Plan > Wiki |
| `13-commit-messages.md` | Commit messages with trailers | `git commit` |
| `14-board-and-labels.md` | Label descriptions and the board columns | Manage > Labels, Plan >
Issue boards |
| `15-snippet-and-mentions.md` | References and mentions anywhere | any Markdown field |
| `16-windchill-round-trip.md` and `windchill-export.csv` | Issues made from a Windchill export;
the status sheet sent back | `gitlab.pl windchill intake` / `status` |
| `17-insights.yml` | `.gitlab/insights.yml`: the Insights dashboard (Ultimate), generated from
project.conf | `gitlab.pl setup` writes it; commit it by MR |
```

I.7.1 Planning: epic, story, task

GitLab Markdown: tools/gitlab/samples/01-epic.md

```
## Goal

Every top-level package of the prime's v1 model (drop v1.0) exists in the government's SysML v2
model on `develop`,
with every converter TODO line closed or turned into a task, passing lint 0/0 and the outline
check.
```

```
## Scope

| In | Out |
|---|---|
| The 12 converted packages under `model/` | Requirements and trace (~"epic::requirements") |
| Level tags `@L1Network` … `@L6Component` | Behavior TODOs: state machine, sequence, fork
(~"epic::behavior") |
| Units, value types, the parametric binding | Any change to the prime's v1 model (raise a DR
instead) |

## Stories

- [x] #7 Import drop v1.0 onto `vendor/prime` and merge it into `develop`
- [x] #12 Define the level tags in `model/gov/Conventions.sysml`
- [x] #15 Close the Network and Interfaces TODO lines
- [ ] #18 Close the Stations TODO lines
- [ ] #21 Close the DataCenter and Operations TODO lines
- [ ] #24 Replace the unit TODOs with ISQ quantities
- [ ] #27 Review the structure with the prime's modeler

## Measures

| Measure | Target | Now (Sprint 4) |
|---|---|---|
| Structure TODO lines left (stereotypes, units, profile) | 0 | 24 of 43 |
| Packages accepted by @jim | 9 | 3 |
| Lint | 0 errors / 0 warnings | 0 / 0 |

## Timeline

| Start | Target | Semester |
|---|---|---|
| %"Sprint 2" | %"Sprint 9" | 1 (Oct 2026 – Mar 2027) |

Tome: ~"tome::port" · Owner: @jim · Engineers: @bob, @tom
```

GitLab Markdown: tools/gitlab/samples/02-story.md

```
## Why

The stations are half of Tidewater's structure, and every analysis of power and telemetry starts
from them. Until the
converter's TODO lines in `model/stations/Stations.sysml` are closed, the part tree has no level
tags and the outline
check cannot vouch for it.

## What "done" looks like

- [ ] No `// TODO stereotype` line left in `model/stations/Stations.sysml`; each became an `@L…`
level tag
- [ ] The `Procured` stereotype on DataLogger, SolarPanel and Modem is carried as `@Procured {
supplier; partNumber; }`
- [x] The `BuoyModes` state machine TODO has its own task (#20), not in this story
- [ ] The pipeline is green on the MR (grammar, lint 0/0, outline, docs current)
- [ ] The trace job shows no new orphan, unverified or drifted requirement
```

```
## Tasks

| Task | Points | Who |
|---|---|---|
| #19 Replace the level-tag TODOs in Stations with `@L…` tags | 2 | @bob |
| #20 Write `state def BuoyModes` from the v1 state machine | 3 | @bob |
| #22 Define `@Procured` in `model/gov/Conventions.sysml` and apply it | 2 | @tom |
| #23 Review Stations with the prime's modeler (30-minute call) | 1 | @jim |

## Notes

- v1 elements: `StationSegment` … `Cell`, 25 blocks (drop v1.0). The TODO lines carry each
element's `xmi:id`.
- Decision (wiki *Decision: level tags*): the v1 profile stereotypes become metadata definitions
in `model/gov`, applied
  in place, so the next drop's merge keeps them.

<details><summary>Definition of Ready</summary>

- [x] Fits in one sprint and has an epic (~"epic::structure") and a tome (~"tome::port")
- [x] Acceptance criteria above are testable
- [x] Tasks are cut, each at most 5 points
- [x] Dependencies linked: blocked by #12 (the level tags exist)

</details>
```

GitLab Markdown: tools/gitlab/samples/03-task.md

```
Part of #18

## Do

- [x] Read the v1 diagram *stm BuoyModes* in Cameo (drop v1.0): 4 states, 7 transitions, 6
triggers
- [x] Replace the line `// TODO uml:StateMachine 'BuoyModes' (xmi:id _22_1_7f3a0c1_…)` with a
`state def`
- [x] Keep the `xmi:id` in a comment above it, so the next drop's delta can find it
- [ ] Run `gitlab.pl ci` locally before pushing

```sysml
// v1: uml:StateMachine 'BuoyModes' (xmi:id _22_1_7f3a0c1_…), drop v1.0
state def BuoyModes {
    entry; then Observing;
    state Observing; state Degraded; state Maintenance; state Recovered;
    transition first Observing accept lowBattery then Degraded;
    transition first Degraded accept charged then Observing;
    transition first Observing accept crewOnSite then Maintenance;
    transition first Degraded accept crewOnSite then Maintenance;
    transition first Maintenance accept crewDeparts then Observing;
    transition first Maintenance accept liftedAboard then Recovered;
}
```

Done when
```

```
`grep -c TODO model/stations/Stations.sysml` drops by one, and the grammar job passes.

/spend 1d
```

The story's task table shows how references work as a status view: each `#N` renders with the task's title and an open or closed icon. The task is short because the story carries the context.

I.7.2 Problems: defect, discrepancy report, prime drop

GitLab Markdown: tools/gitlab/samples/04-defect.md

```
## What is wrong

`part def Cell` in `model/stations/Stations.sysml` has no level tag, so the outline check fails
on `develop`.

## How it shows

Pipeline #214 on `develop`, job **outline**:

```text
model/stations/Stations.sysml:212: part def Cell: no level tag (@L1Network..@L6Component)
check: 1 violation
```

Expected

`@L6Component;` on `Cell`, as in v1 (stereotype `L6Component`, drop v1.0). The tag was lost when
!31 resolved a merge
conflict by taking the prime's text and did not re-apply the government edit.

Found

Found in	pipeline #214 (post-merge on `develop`)
Phase	~"phase::port"
Drop	v1.0

> [!note]
> Root cause is a process slip, not the model: the Drop Intake Runbook step 5 ("re-apply the
government edit") was
> skipped. Fix-forward in !33; the runbook now has a checklist line for it.

/label ~"type::defect" ~"owner::gov" ~"severity::2" ~"area::stations"
/assign @bob
/milestone %"Sprint 5"
```

GitLab Markdown: tools/gitlab/samples/05-discrepancy-report.md

```
## Discrepancy

| | |
```

```
|---|---|
| DR number | DR-3 |
| Prime tracking | NMS-SM-0147 |
| Found in drop | v1.0, delivered 2026-10-02 |
| v1 element | `Requirements::System Requirements::Functions::Bulletin` (xmi:id
`_22_1_7f3a0c1_…`) |
| DOORS id | TW-SYS-006 |
| Severity | 2 — wrong with a workaround |

## What we found

The text of TW-SYS-006 differs between the two sources the prime delivered:

| Source | Text |
|---|---|
| DOORS (`tidewater.reqif`) | The data center shall publish a marine bulletin **twice a day**, on
a fixed schedule. |
| Cameo model (`tidewater.xmi`) | The data center shall publish a marine bulletin on a fixed
schedule. |

The trace job on !24 reported it:

```text
text differs from DOORS                          1
drift: TW-SYS-006
```

What we ask the prime to do

Synchronize the Cameo requirement with DOORS (DOORS is the requirements authority) and deliver it
in the next drop.

Closure

- [x] Sent to the prime on 2026-11-20 by @jim
- [ ] Fixed in drop v1.1, confirmed by the intake issue #<!-- next drop -->

Workaround until then: the government model cites the DOORS text (`model/doors/`), which is
authoritative.
```

GitLab Markdown: tools/gitlab/samples/06-prime-drop.md

```
## The drop

| | |
|---|---|
| Version | v1.0 |
| Delivered | 2026-10-02, contract portal |
| CDRL | A003 System Model |
| MANIFEST sha256 | `4f1c…9a7e` |
| Previous drop | none: this is the port's starting point |

## Intake checklist

- [x] Files received and scanned; hashes match `MANIFEST.txt`
- [x] `gitlab.pl flow vendor /secure/drops/2026-10-02 v1.0` committed the drop on `vendor/prime`
(pipeline #9 green)
```

```
- [x] The `delta` job: first drop, nothing to compare
- [x] Merge MR !4 from `feature/drop-v1.0` opened; no conflicts (first import)
- [x] Trace run: 29 of 30 DOORS requirements linked; DR-1 (TW-SYS-012 orphan), DR-2 (TW-SYS-011
unverified), DR-3
      (TW-SYS-006 drift) and DR-4 (TW-SYS-016 not in the model) opened
- [x] Merged to `develop`; CHANGELOG updated

## Conversion report

| | |
|---|---|
| v1 elements read | 521 |
| In scope ported | 194 of 194 |
| TODO lines | 52 (13 unmapped constructs, 39 stereotypes) |
| Grammar | 12 files, 0 errors |

## Conflicts and manual rework

| File / element | Why | Task |
|---|---|---|
| `BuoyModes` | state machine: not converted | #20 |
| `DailyUpload` | interaction: not converted | #29 |
| `EnergyBalance` | constraint expression: not converted | #30 |
| `ObserveCoast` fork, pin, object flow | activity nodes: not converted | #31 |

## Discrepancy reports

New: #8 (DR-1), #9 (DR-2), #10 (DR-3), #11 (DR-4).

/label ~"workflow::done"
```

I.7.3 Change: merge request, review, comments

GitLab Markdown: tools/gitlab/samples/07-merge-request.md

```
## What and why

Closes the 25 level-tag TODO lines in Stations (task #19, story #18). Each `// TODO stereotype
Tidewater_Profile:L…` line becomes the matching `@L…;` tag, in place, so the next prime drop
merges around it.

Closes #19
Part of #18

## Model changes

| Package | Change |
|---|---|
| `model/stations` | 25 `@L…` tags replace the stereotype TODOs; `private import Conventions::*;`
added |
| `model/gov` | none (the tags were defined in !12) |
| `docs/` | regenerated: `Nouns.md` now shows levels L2 to L6 under StationSegment |

## Checks

- [x] `gitlab.pl ci` passed locally (grammar, lint 0/0, outline, docs)
```

```
- [x] `docs/` regenerated and committed
- [x] No new `// TODO` lines (29 → 4 in this file; the 4 left are #20 and #22)
- [x] Trace job: no new orphan, unverified or drifted requirement
- [x] Commit messages carry a `Changelog:` trailer

## Review notes

Look at `PowerSystem` first: its `bind` lines sit right under the tag and are the likeliest place
for a merge conflict
with the next drop. @jim, the outline check now passes for Stations; please accept the package on
#27.

<details><summary>Pipeline (local run)</summary>

```text
check   grammar   success   0.8s
check   lint      success   0.7s   elements=138 errors=0 warnings=0
check   outline   success   0.1s
check   todo      success   0.0s   TODO lines remaining: 27
check   secrets   success   0.1s
trace   trace     allowed_failure  1.2s   orphans: TW-SYS-012 TW-SYS-016
docs    docs      success   0.9s
```

</details>
```

The MR description's pipeline table sits inside `<details>`, so the review starts with what changed and why, and the log is one click away.

GitLab Markdown: tools/gitlab/samples/08-review-thread.md

```
<!-- An MR discussion as it reads on !19, top to bottom. Each block is one comment; "resolvable"
threads must be
     resolved before the MR can merge (project setting). -->

**@jim** commented on `model/stations/Stations.sysml` line 47 (resolvable):

`ShoreStation` is tagged `@L3Station`, but in v1 it is a specialization of `ObservingStation`,
which is also L3.
That is fine. Please also keep the v1 `xmi:id` next to the tag, as #20 does for BuoyModes, so the
delta can
point at it.

```suggestion:-0+1
    part def ShoreStation :> ObservingStation {
        @L3Station;   // v1 stereotype L3Station (xmi:id _22_1_7f3a0c1_…)
```

@bob replied:

Applied the suggestion (commit `3c9e1a2`). I did the same for the other 24 tags in a second
commit so the file is
consistent.

@jim resolved the thread.

```

```
**@tom** commented (resolvable):

The `trace` job is yellow because of TW-SYS-012 and TW-SYS-016. Those are DR-1 (#8) and DR-4
(#11), both with the
prime, so not this MR's problem. Linking them so the reviewer sees why.

/relate #8 #11

**@tom** resolved the thread.

---

**@jim** commented:

Lint 0/0, outline passes, docs current. Approving.

/approve
/label ~"workflow::done"

**@bob** commented:

Thanks. Merging when the pipeline on the merged result finishes.

/merge
```

The review thread shows the three moves of a GitLab review: a comment on a line (resolvable, so the MR cannot merge until it is resolved), a *suggestion* the author applies with one click, and quick actions (`/relate`, `/approve`, `/merge`) that do the bookkeeping in the same comment.

GitLab Markdown: tools/gitlab/samples/09-issue-comments.md

```
<!-- Comments on story #18 over Sprint 4, newest last. Quick actions on their own line run when
the comment is saved
     and disappear from the text; GitLab shows a system note instead (e.g. "Bob added
     ~workflow::blocked"). -->

**@bob** · Mon 2026-11-16

Started. Doing #19 first, then #20.

/label ~"workflow::in-progress"
/spend 4h

---

**@bob** · Wed 2026-11-18

Blocked on #20: the v1 diagram *stm BuoyModes* shows a trigger `liftedAboard` that is not on any
transition in the
XMI. Asked the prime's modeler which one is right.

/label ~"workflow::blocked"
/relate #20

---
```

```
**@jim** · Wed 2026-11-18

The prime's modeler confirms the XMI is right: the diagram is stale. Not a DR (diagrams are not
delivered content),
but noted for the next model review. Unblocking.

/unlabel ~"workflow::blocked"
/label ~"workflow::in-progress"

---

**@bob** · Fri 2026-11-20

#19 merged (!19), #20 in review (!20). #22 is @tom's. On track for the sprint.

/spend 1d 2h

---

**@jim** · Fri 2026-11-27

All four tasks closed; acceptance criteria checked. Demoed at the review.

/label ~"workflow::done"
/close
```

Comments on a story double as its log: each stand-up-sized update carries a `/spend` and a workflow label change, so the board and the time report update themselves.

I.7.4 Time and record: milestone, release, wiki, commits

GitLab Markdown: tools/gitlab/samples/10-milestone.md

```
<!-- The description of milestone "Sprint 4" (Plan > Milestones). Written at planning, updated at
the review. -->

**Sprint 4** · Mon 2026-11-16 to Fri 2026-11-27 · Capacity 24 points (Jim 6, Bob 9, Tom 9)

## Sprint goal

Stations and DataCenter packages have no stereotype TODOs left, and the pipeline runs on the
team's Windows runner.

## Committed stories

| Story | Epic | Points | Owner |
|---|---|---|---|
| #18 Close the Stations TODO lines | ~"epic::structure" | 8 | @bob |
| #21 Close the DataCenter and Operations TODO lines | ~"epic::structure" | 5 | @bob |
| #25 Move CI to the Windows shell runner | ~"epic::pipeline" | 6 | @tom |
| #26 Onboarding page and laptop checklist | ~"epic::onboarding" | 3 | @jim |

## Review (Fri 2026-11-27)
```

```
- Done: #18, #25, #26 (17 points). Carried: #21 (5 points: the DataCenter ports need a decision
on proxy vs full
  ports, spike #32 next sprint).
- Structure TODO lines: 43 → 19.
- Pipelines on `develop`: 14 of 14 green.

> [!tip]
> The burn-down chart on this page counts issues and weight (Premium). On Free, use the issue
list filtered by this
> milestone and ~"type::story".
```

GitLab Markdown: tools/gitlab/samples/11-release.md

```
# Tidewater model baseline-1

**Released** 2027-03-19 · **Tag** `baseline-1` · **From** prime drop v1.1 · **Milestones**
%"Sprint 1" to %"Sprint 12"

> [!important]
> Cite this baseline by its tag. The tag pipeline proved it rebuilds byte for byte
> (`sha256=9d41…c07b` in the tag message).

## What is in it

| | Count |
|---|---|
| Packages | 12 + `model/doors` + `model/gov` |
| Part definitions | 41 |
| Requirements (from DOORS) | 31 |
| Satisfied and verified | 87 % |
| TODO lines left | 0 |

## Changes since the start of the port

### Added
- The full structure from drop v1.0, with level tags and the `@Procured` metadata (!4 to !21).
- Requirements from DOORS with their `doorsId`; satisfy and verify trace (!22 to !30).
- Hand-written SysML v2 for `BuoyModes`, `DailyUpload`, `EnergyBalance` and the observation-cycle
fork (!31 to !38).
- Drop v1.1: the Hygrometer and TW-SYS-017 (!61).

## Prime drops included

| Version | Delivered | Intake |
|---|---|---|
| v1.0 | 2026-10-02 | #5 |
| v1.1 | 2027-01-08 | #61 |

## Known gaps (accepted)

- [ ] TW-SYS-011 not verified: DR-2 (#9), prime due in a later drop
- [ ] TW-SYS-012 not satisfied: DR-1 (#8)
- [ ] TW-SYS-016 not in the Cameo model: DR-4 (#11)

## Evidence
```

```
- Tag pipeline #301 (`baseline` job: "baseline-1 rebuilds identically")
- Trace report: the `trace` job artifact of the release MR !70
```

GitLab Markdown: tools/gitlab/samples/12-wiki-decision.md

```
# Decision: requirements come from DOORS, relationships from Cameo

[[_TOC_]]

| | |
|---|---|
| Status | **Accepted** (Sprint 3, 2026-11-06) |
| Decided by | @jim |
| Spike | #14 |
| Affects | `model/requirements/`, `model/doors/`, the `trace` job |

## Context

Each prime drop carries the requirements twice: in DOORS (ReqIF, CSV) and inside the Cameo model
(XMI), synchronized by
the prime's DOORS plug-in. Both converters emit them, named by DOORS id (`TW_SYS_001`). The two
copies can drift: in
drop v1.0, TW-SYS-006 already differs[^1].

## Decision

1. **DOORS is the requirements authority.** `reqif2sysml.pl` output goes to
`model/doors/<module>/`.
2. **Cameo is the relationships authority.** Satisfy, verify and derive relationships come from
the XMI conversion in
   `model/requirements/` and `model/verification/`.
3. The `trace` job compares the two on every MR and reports drift; drift is a discrepancy report,
never a hand edit.

## Consequences

- No government edits to requirement text, ever: the vendor branch keeps both copies exactly as
delivered.
- The model has two definitions per requirement (one per package); analyses cite `model/doors/`.

```mermaid
flowchart LR
  D[DOORS ReqIF] -->|reqif2sysml| R[model/doors]
  C[Cameo XMI] -->|xmi2sysml| M[model/requirements, model/verification]
  R --> T{trace job}
  M --> T
  T -->|drift| DR[Discrepancy Report]
```

[^1]: DR-3 (#10): "twice a day" is in DOORS only.
```

A decision record in the wiki: status, who decided, the context, the decision, the consequences, a diagram and a footnote. It is the document era's memo for record, with the difference that it is versioned and linked from the issues it affects.

GitLab Markdown: tools/gitlab/samples/13-commit-messages.md

````
<!-- Commit messages from Sprints 1 to 6, as `git log` shows them. Subject in the imperative,
under 72 characters; a
     blank line; the why; trailers last. GitLab links "#19" and "!19"; the CHANGELOG is built
     from "Changelog:". -->

```text
Replace the Stations level-tag TODOs with @L tags

The converter reports v1 profile stereotypes as TODO lines. Each one
becomes the metadata tag defined in model/gov/Conventions.sysml, in
place, so the next drop's three-way merge keeps it.

Changelog: changed
Issue: #19
```

```text
Write state def BuoyModes from the v1 state machine

The converter does not carry state machines. Written from drop v1.0
(xmi:id kept in a comment) after the prime's modeler confirmed the XMI,
not the diagram, is current.

Changelog: added
Issue: #20
Reviewed-by: Jim <jim@example.invalid>
```

```text
Import prime drop v1.0 (MANIFEST sha256 4f1c…9a7e)

CDRL A003, delivered 2026-10-02. Converted with xmi2sysml.pl and
reqif2sysml.pl; raw files under imports/v1.0/. No hand edits on this
branch.

Issue: #5
```

```text
Merge branch 'feature/19-stations-level-tags' into 'develop'

Replace the Stations level-tag TODOs with @L tags

Closes #19

See merge request tidewater/model!19
```

```text
Fix the missing @L6Component tag on Cell

!31 resolved a merge conflict by taking the prime's text and did not
re-apply the government edit.

Changelog: fixed
Fixes: #34
````
````

```
```

Commit messages are not a GitLab field, but GitLab shows them everywhere and links their references. The format is the common git convention: a subject in the imperative under 72 characters, a blank line, the reason, and trailers (`Changelog:`, `Issue:`, `Reviewed-by:`) last.

I.7.5 The board and references

GitLab Markdown: tools/gitlab/samples/14-board-and-labels.md

```
# The board and the labels

## Board: "Sprint" (Plan > Issue boards)

Scope: milestone = the current sprint. Lists, left to right, are scoped labels, so moving a card
swaps the label:

| Open | ~"workflow::ready" | ~"workflow::in-progress" | ~"workflow::review" |
~"workflow::blocked" | Closed |
|---|---|---|---|---|---|
| not yet ready | can be pulled | someone is on it | an MR is open | see the last comment | done
|

Swimlanes (Premium): by epic. On Free, a second board "Epics" with one list per ~"epic::…" label.

## Board: "Prime" (discrepancy reports)

Scope: label ~"owner::prime". Lists: ~"severity::1", ~"severity::2", ~"severity::3". Jim reviews
it before each drop.

## Label legend

| Label | Meaning |
|---|---|
| ~"type::story" | fits in one sprint; several tasks |
| ~"type::task" | under a week (≤ 5 points); one MR |
| ~"type::epic" | under a semester; several stories (Free tier stand-in for a group epic) |
| ~"type::drop" | intake of one prime drop |
| ~"type::discrepancy" | a DR to the prime; closed by the drop that fixes it |
| ~"type::defect" | wrong in the government model or the pipeline |
| ~"tome::port" | year-scale grouping: the port |
| ~"epic::structure" | semester-scale grouping: the structure port |
| ~"owner::prime" / ~"owner::gov" | who fixes it |
| ~"area::stations" | which package; matches a CODEOWNERS section |
| ~"area::interface" | a defect between packages (status metric H) |
| ~"phase::port" | found during the port |
| ~"priority::1" | do first |
```

GitLab Markdown: tools/gitlab/samples/15-snippet-and-mentions.md

```
# References and mentions, in any Markdown field

| You type | GitLab shows | Meaning |
```

```
|---|---|---|
| `#18` | a link to issue 18 with its title on hover | an issue in this project |
| `#18+` | the link plus the issue's title inline | same, expanded |
| `!19` | a link to merge request 19 | an MR |
| `%"Sprint 4"` | a link to the milestone | a sprint |
| `~"epic::structure"` | the label chip | a label |
| `@bob` | a mention; Bob is notified | a person |
| `@tidewater/model-team` | a group mention | everyone in the group |
| `tidewater/model#18` | a cross-project issue link | another project's issue |
| `3c9e1a2` | a link to the commit | a commit |
| `baseline-1` (in a release) | plain text; use `[baseline-1](../-/tags/baseline-1)` | a tag |
| `[[_TOC_]]` | a table of contents | in descriptions and wiki pages |
| `- [ ] item` | a checkbox, counted in the issue list ("2 of 5 checklist items") | a task list |
| `> [!note]` | a blue alert box | also `[!tip]`, `[!important]`, `[!caution]`, `[!warning]` |
| `<details><summary>…</summary>…</details>` | a collapsible section | long logs |
| ` ```mermaid ` | a rendered diagram | flowchart, gitGraph, gantt, sequenceDiagram |
| ` ```suggestion:-0+1 ` | an "Apply suggestion" button (MR diffs only) | a proposed change |
| `[^1]` and `[^1]: text` | a footnote | references in wiki pages |

Quick actions (a line of their own, in a description or comment):

```text
/label ~"type::story" ~"epic::structure"      /unlabel ~"workflow::blocked"
/milestone %"Sprint 4"                         /assign @bob   /assign_reviewer @jim
/estimate 2d    /spend 4h                      /relate #8 #11
/close    /reopen    /approve    /merge        /draft    /ready
```
```

I.8 Applying this to your project

- **Copy** the folder structure (`.gitlab/issue_templates/`, `.gitlab/merge_request_templates/`) and the Default MR template on day one; they cost nothing and shape every change from the start.
- **Adapt** the labels and the quick actions in each template to your own label set. Keep scoped labels for anything that has exactly one value at a time (type, workflow state, priority).
- **Decide locally** which fields your auditors will look for (the closure of a discrepancy report, the approval of a baseline) and make sure a template asks for each one.

I.9 Exercises

Exercise I.1: Write a story

In a scratch project, or on the emulator with `gitlab.pl --local /tmp/gl issue new … --template Story`, write a story for "Close the DataCenter TODO lines" with three tasks under five points each. Make every quick action run.

Exercise I.2: Find the features

Pick three samples and list every GitLab Markdown feature each uses, from the table in Section I.2, GitLab Markdown in fifteen features (p. 748). Which feature would you add to

the Discrepancy Report template, and why?

Exercise I.3: Write a discrepancy report

Run the trace on drop 1 (`reqif2sysml.pl --report --model`) and write the discrepancy report for one orphan requirement, using the template. What exactly do you ask the prime to change, and how will you verify it in the next drop?

CHAPTER

J

Checklists

From what you know →

You know

entry and exit criteria for reviews.

→ Here it is called

checklists for each step of the rhythm.

→ What is new

many items are confirmed by the pipeline instead of by a signature.

Every checklist in the book, in the order the work meets them. Print the ones you use; tick them in a GitLab issue comment (paste the lines as `- [ ]` items) when the record matters. Each names the chapter that explains it.

J.1 Week zero

From Chapter 3, The directive (p. 22) and Chapter 15, Setting up GitLab (p. 226).

Checklist: Before sprint 1

- ☐ Ask the enterprise GitLab administrators for the group, the project, the accounts and a runner (lead time: about a week)
- ☐ Book the security officer (ISSM) for a toolchain briefing in sprint 2
- ☐ Check every laptop: Git for Windows (Git Bash, Perl), Vim, the kit at the agreed tag; `perl tests/run.pl` passes
- ☐ Agree the scorecard with management: measures, how each is computed, targets, old-way baselines; signed
- ☐ Ask the prime for stable `xmi:id`s across drops and a `MANIFEST.txt` with checksums in each drop
- ☐ Log the first drop's delivery against its CDRL; store it on approved storage; do nothing else with it yet

- ☐ Put stand-ups, planning, refinement, review and retrospective in the calendar for the whole quarter
- ☐ Enter holidays and planned leave, so the first capacity is honest
- ☐ Make the journal project: `daily.pl init`, `daily.pl roster add …` for each person

J.2 Project set-up

From Chapter 15, Setting up GitLab (p. 226) and Chapter 17, The DevSecOps tools that earn their keep (p. 255).

Checklist: The GitLab project, first day

- ☐ `project.conf` written: settings, members, branches, tags, labels, sprints, approvals, variables (no secret values)
- ☐ `gitlab.pl --dry-run setup project.conf` reviewed, then applied
- ☐ Default branch is `develop`; merge method is merge commit; squash never; pipelines must succeed; discussions must be resolved
- ☐ `main` and `develop` protected: push allowed to no one, force push off, code owner approval on (Premium)
- ☐ `vendor/prime` protected: push by maintainers, merge by no one
- ☐ `baseline-*` tags protected: created by maintainers only
- ☐ `CODEOWNERS`, issue templates and MR templates committed through the first MR
- ☐ Runner registered on a dedicated machine (project runner, tags, service account without admin rights, `core.longpaths`, antivirus exclusion for the builds folder) and a first job has run
- ☐ `.gitlab-ci.yml` with grammar, lint and outline jobs; a deliberately broken MR has been seen to fail and be blocked
- ☐ Board with workflow columns; sprint milestones two sprints ahead
- ☐ Wiki: Home, Definition of Ready, Definition of Done, Glossary

J.3 The sprint

From Chapter 13, The operating model (p. 196) and Chapter 24, The rhythm: stand-ups, poker, refinement, kickoff, demo, retro (p. 377).

Checklist: Kickoff and planning (day 1)

- ☐ Last sprint's carryover discussed, not just moved
- ☐ Capacity per person from the calendar (holidays, leave, training); `cap` in the stand-up file
- ☐ The sprint goal written in one sentence, in the milestone description
- ☐ Stories pulled in priority order until about 85 to 95 percent of capacity
- ☐ Every pulled story meets the Definition of Ready; every task is 5 points or fewer
- ☐ Estimates settled by `#est` rounds in the chat; outliers explained before a re-vote

- ☐ `commit` lines in the stand-up file; issues moved to the sprint milestone and `workflow::ready`
- ☐ The board shows only this sprint's milestone

Checklist: Daily stand-up (15 minutes)

- ☐ First question: is `develop` green? If not, fixing it comes first
- ☐ Each person posts one terse line: Yesterday, Today, Blockers, with story ids
- ☐ `daily.pl lint` while people are still on the call; replies pasted back for any status it flags
- ☐ Blockers named, owners agreed; the discussion happens after the stand-up
- ☐ After the call: `daily.pl answers`, confirm the suggested lines, `daily.pl compile`, `daily.pl commit`

Checklist: Refinement (day 9)

- ☐ New work entered as stories (`new` lines and issues from the Story template)
- ☐ Each candidate story checked against the Definition of Ready; tasks cut to 5 points or fewer
- ☐ `#est` rounds for anything not yet sized
- ☐ Stale stories pruned with a reason
- ☐ Dependencies on the prime or another story linked
- ☐ The next sprint's top stories labelled `workflow::ready`

Checklist: Review and demo (day 10)

- ☐ Demonstrate from the real system: the merged MRs, a pipeline with its widgets, the model site, the trace
- ☐ Show the scorecard, every agreed measure, including the bad ones
- ☐ Record management's feedback as issues, not as notes
- ☐ Close the stories that meet the Definition of Done; the rest carry over with a reason

Checklist: Retrospective (day 10)

- ☐ The pulse survey (five questions, 1 to 5), anonymous
- ☐ Start, stop, continue: one change the team will actually make next sprint, written as a story or a rule
- ☐ The monthly pitfall check (Section 51.7, A monthly look in the mirror (p. 692)) once a month
- ☐ Last retrospective's change: did it happen?

Checklist: Weekly communication (Friday)

- ☐ `daily.pl quad` reviewed and sent: priorities, watch items, milestones, accomplishments
- ☐ The scorecard numbers refreshed from the export, not typed
- ☐ Anything management must decide is stated as a decision with a date

J.4 A change

From Chapter 12, Git from zero, for systems engineers (p. 171), Chapter 20, The pipeline, job by job (p. 302) and Appendix I, Templates and GitLab Markdown, field by field (p. 747).

Checklist: Author, for every task

- ☐ Branch from up-to-date `develop`: `gitlab.pl flow start feature IID-slug`
- ☐ Converter output edited in place and minimally; government-only content in files the converter never writes (`model/library/conventions/` and the like)
- ☐ `\mx` on each edited file, `\ml` on the model, `gitlab.pl ci --event merge_request_event --target develop` before pushing
- ☐ `docs/` regenerated (`\md`) and committed
- ☐ Commit messages: imperative subject under 72 characters, the why, a `Changelog:` trailer
- ☐ MR from the Default template: what and why, `Closes #task`, model changes, checks ticked
- ☐ Pipeline green on the merged result; every discussion answered

Checklist: Reviewer, for every MR

- ☐ The change matches the task; nothing unrelated is in it
- ☐ Every closed TODO line is replaced by the construct that closes it
- ☐ No requirement text changed outside a drop
- ☐ Pipeline green; JUnit and code quality widgets show no new failures; TODO count did not rise without a task
- ☐ Trace: no new orphan, unverified or drifted requirement, or a DR is linked
- ☐ Comments are resolvable and specific; suggestions used for small fixes
- ☐ Approve only what you would sign

J.5 A prime drop

From Chapter 43, Maintenance through the life cycle (p. 602) and Appendix D, Quick reference: SysML v1 to v2 (p. 719).

Checklist: Drop intake (one story, finished in the sprint it arrives)

- ☐ Files in a dated folder outside the repository; every sha256 matches `MANIFEST.txt`
- ☐ Prime Drop issue opened from the template in the current sprint

- ☐ Files uploaded to the package registry (`prime-drop`, the version)
- ☐ `gitlab.pl flow vendor DROPDIR VERSION`: `vendor/prime` committed and pushed, `feature/drop-VERSION` started
- ☐ `xmi2sysml.pl --delta OLD NEW` read; each *manual* line is a task
- ☐ Each conflict resolved by taking the vendor side and re-applying the government edit; listed in the MR
- ☐ No conflict markers left; grammar, lint 0/0 and outline pass
- ☐ Trace read: DRs fixed by this drop closed, new gaps raised as DRs
- ☐ MR from the Prime Drop template, approved by a code owner, merged; `CHANGELOG.md` updated
- ☐ Drop latency (delivery to merged) recorded

Checklist: Discrepancy report to the prime

- ☐ Facts only: what the model says, what DOORS says, what the trace or delta printed
- ☐ The v1 element's qualified name and `xmi:id`; the DOORS id
- ☐ One requested change, specific enough to verify in the next drop
- ☐ Severity set; sent through the contract channel; the prime's tracking number recorded
- ☐ Closed only by the intake issue of the drop that fixes it

J.6 A baseline and a hotfix

From Chapter 43, Maintenance through the life cycle (p. 602).

Checklist: Baseline

- ☐ Content decided: which drops and stories are in; the rest moved to the next milestone
- ☐ `gitlab.pl flow start release baseline-N`; only release fixes on the branch
- ☐ `CHANGELOG.md` [Unreleased] moved under the baseline heading; docs regenerated
- ☐ Release MR from the template: content, accepted gaps listed and accepted by the model lead
- ☐ Merged into `main`; `gitlab.pl flow released IID` tags `baseline-N` with the docs hash and opens the back-merge
- ☐ Tag pipeline: the `baseline` job reports that the baseline rebuilds identically
- ☐ Release notes published; project exported and stored; milestones closed
- ☐ Back-merge MR to `develop` merged

Checklist: Hotfix to a released baseline

- ☐ The defect is in the released baseline and someone depends on it now (otherwise it waits for the next baseline)
- ☐ `gitlab.pl flow start hotfix slug` from `main`; the smallest change that corrects it
- ☐ Hotfix MR to `main`; pipeline green; approved

- ☐ Tagged `baseline-N.1`; the users of baseline-N told
- ☐ The same fix back-merged to `develop`

J.7 Administration

From Chapter 42, Administration (p. 586); each is explained there.

Checklist: Onboarding one person

- ☐ Account, project role (the lowest that works), SSH key or token, `CODEOWNERS` if an owner
- ☐ `roster.txt` and the stand-up invitation
- ☐ Onboarding wiki page with a pairing partner; first MR merged within a week

Checklist: Offboarding one person (the same day)

- ☐ Removed from the project and the group; tokens revoked; readable CI/CD variables rotated
- ☐ Open issues and MRs reassigned; pipeline schedules taken over; `CODEOWNERS`, roster and invitation updated

Checklist: Monthly

- ☐ Token inventory checked; any token expiring within a month rotated
- ☐ Runner patched (operating system, gitlab-runner matching the server's version, Git for Windows with the laptops, the kit at its tag)
- ☐ Pipeline schedules owned by a current member and passing
- ☐ Container image rebuilt from a pinned base and scanned (only if the Docker image is used)
- ☐ Protection audit: the settings pages match `project.conf` (`setup` re-applied if not); every merge had a passing pipeline and a non-author approval
- ☐ Audit events reviewed (Premium)
- ☐ Project exported to the package registry
- ☐ GitLab release notes read for CI and API changes

Checklist: Quarterly

- ☐ Access review, inherited memberships included
- ☐ Restore test from the latest export into a scratch project

J.8 Before you call the port done

Checklist: The port's exit criteria

- ☐ TODO count zero, or every remaining TODO has an issue and an accepted reason
- ☐ No duplicate blocks, no package cycles; the clutter has DRs and the prime is removing it upstream
- ☐ Trace: every requirement linked; every orphan, unverified or drifted requirement has a DR
- ☐ Every outline waiver removed; all jobs blocking except those allowed to fail by design, with reasons
- ☐ A baseline tagged on `main` that rebuilds identically
- ☐ At least one prime drop taken through the full intake cycle after the first import
- ☐ Every runbook followed at least once by someone other than its author
- ☐ The scorecard reported every sprint, including the bad sprints

CHAPTER

K

Glossary

From what you know →

You know

the acronym list at the back of every SEMP.

→ Here it is called

the glossary: systems engineering, DevSecOps and SysML v2 side by side.

→ What is new

each new term is mapped to the one you already use.

Three vocabularies meet in this book: systems engineering and configuration management, which the reader already speaks; GitLab and DevSecOps, which Jim, Bob and Tom learned in twelve weeks; and SysML v2, which replaces the SysML v1 they knew from Cameo. The first table translates between the first two, the second defines the DevSecOps and GitLab words, the third maps SysML v1 terms to v2, and the last two cover the agile words and the terms this book and its kit coined. The project wiki's Glossary page (`tools/gitlab/templates/wiki/Glossary.md`) uses the same translations.

Warning: CI means two things

In configuration management a **CI** is a *configuration item*. In GitLab, **CI** means *continuous integration*, the automatic checks. This book writes "configuration item" in full and uses CI only for the checks.

K.1 What you already do, and what GitLab calls it

Table K.1: Systems engineering terms and their GitLab counterparts

Systems engineering or CM term	GitLab or DevSecOps term	Same	Different
Configuration item	a folder or file under `model/`	under change control, with an owner	many small items; changes recorded line by line

Systems engineering or CM term	GitLab or DevSecOps term	Same	Different
Controlled master copy	the `develop` branch	the copy everyone works from	nobody edits it directly; changes arrive by merge request
Released baseline	a protected tag `baseline-N` on `main`, with a release	frozen, citable	the server forbids moving it; the pipeline proves it rebuilds
Engineering change proposal, change request	merge request (MR)	proposes, shows, is reviewed and approved	days instead of months; the exact lines are shown
CCB approval	approval by the code owners of the changed parts	the right authority approves before incorporation	per change, online; no waiting for the board
Redlines, markups	review comments and suggestions on MR lines	comments pinned to the text	a suggestion is applied with one click
Configuration audit (FCA, PCA)	the pipeline	checks the configuration against its rules	minutes, on every change
Configuration status accounting	git history, the merge log, `gitlab.pl export`	who changed what, when, under which change	complete and exact, typed by nobody
Revision (Rev A, Rev B)	commit	one recorded change with an author and a date	many and small; the commit id is the revision number
Revision history block	`git log`, `CHANGELOG.md`	the list of changes	generated from the work
Working draft	feature branch	your copy while you work	lives days; git brings the changes back
Incorporating an approved change	merge	the change joins the master copy	line by line; stops only at a conflict
CDRL delivery from the contractor	a prime drop, committed on `vendor/prime`	the deliverable, received and checked	converted and merged, keeping the government's edits
Discrepancy or deficiency report	issue labelled `type::discrepancy`	a written finding against a deliverable	tracked on a board until the fixing drop closes it
Action item	issue (task or story)	owner, due date, status	visible, linked to the change that closes it
Action item tracker	issue board	one row per action, a status column	status is a label; the board updates itself
WBS element	tome and epic (labels on Free, epics on Premium)	groups work in a hierarchy	sized by time: epic within a semester, tome within a year
IMS milestone	release, baseline	a dated program event	
A two-week work period	GitLab milestone `Sprint N`	a time box with a list of work	GitLab calls any dated time box a milestone
Data item description, document template	issue and MR templates	say what goes in each section	filled in online; offered when the item is created
SEMP, process documents	the wiki	how we work	short, versioned pages
Requirements traceability matrix	the trace job, `docs/Traceability.md`	which requirement is satisfied and verified by what	rebuilt on every change from the model
Verification cross-reference matrix	`verify` relationships in `model/verification/`	which test verifies which requirement	gaps checked automatically
Peer review, desk check	MR review	a second engineer reads the change	small changes; minutes
Signature, sign-off date	approval, merged-at time	who approved, when	recorded by the server; cannot be backdated
Distribution list	mentions and notifications	who is told	about what concerns them, when it happens

Systems engineering or CM term	GitLab or DevSecOps term	Same	Different
Weekly status chart	status metrics, the weekly quad	progress and problems	computed from the work

K.2 DevSecOps and GitLab words

Table K.2: DevSecOps and GitLab words

Word	Meaning in this book
Allow failure	a job setting: its failure shows yellow and does not block the merge
Approval	a reviewer's recorded "yes" on an MR
Artifact	a file a job keeps after it runs (a report, a log, a count)
Back-merge	merging `main` into `develop` after a release or hotfix, so nothing is lost
Board	issues shown as cards in columns, one column per workflow label
Branch	a line of work; `develop` is the master copy, feature branches are drafts
Cache	files kept between jobs to save time; may be empty; never evidence
CI/CD	continuous integration and continuous delivery: every change checked and its products produced automatically
Clone	your full copy of the repository, with its history
Code owner	the person who must approve changes to a path, named in `CODEOWNERS`
Commit	one recorded change: what, who, when and why
Conflict	git cannot merge because both sides changed the same lines; a person chooses
Default branch	the branch GitLab shows first and closes issues on (`develop` here)
Deploy token	a read-only credential for cloning or downloading from outside
DevSecOps	development, security and operations done together, in small changes with automated checks
Environment	a named place a job publishes to (the model site); each successful publish is a deployment
Fast-forward	moving a branch to a newer commit without a merge commit
Force push	overwriting a branch's history on the server; forbidden on protected branches
Gitflow	the branching scheme used here: `main`, `develop`, feature, release, hotfix, plus `vendor/prime`
GitLab flow	a simpler alternative scheme: feature branches into `main`, promotion by environment or release branches
Issue	a work item: story, task, defect, discrepancy report, drop intake
Job	one step of a pipeline, such as `lint`
Job token (`CI_JOB_TOKEN`)	a credential that exists only while a job runs
Label, scoped label	a tag on an issue or MR; a scoped label `key::value` allows one value per key (Premium)

Word	Meaning in this book
Lint	an automatic structural check of the model
Masked variable	a CI/CD variable hidden as `[MASKED]` in job logs
Merge request (MR)	a request to merge a branch, with its changes, checks, discussion and approvals
Merged-results pipeline	an MR pipeline run on the result of merging the branch into its target
Milestone	a dated time box; here, one sprint
Package registry	storage for files with versions and checksums (the prime drops, baseline bundles)
Pages	a static website published from a job's `public/` folder (the model site)
Personal access token	a credential for one person's scripts
Pipeline	the set of jobs GitLab runs for a change: the automatic configuration audit
Project access token	a credential belonging to a bot user of one project, for automation
Protected branch, protected tag	a branch or tag only certain roles may change, and none may force-push or delete
Protected variable	a CI/CD variable given only to pipelines on protected branches and tags
Push, pull, fetch	send commits to the server; bring them and merge; bring them without merging
Quick action	a `/command` line in a comment that changes the issue or MR
Rebase	rewriting a branch's commits on top of another; never on shared branches
Release	a tag with release notes on GitLab's Releases page
Repository	the files under version control with their whole history
Rules	the conditions that decide whether a job runs in a pipeline
Runner	the machine that executes pipeline jobs
Schedule	a pipeline started by the clock (the nightly status)
Secret detection	a job that looks for passwords, keys and tokens in committed files
Shift left	finding a problem earlier: at merge rather than at integration
Squash	combining a branch's commits into one at merge; never used here
Stage	a column of jobs; stages run in order
Tag, annotated tag	a permanent name for one state of the repository; annotated tags carry a message
Vendor branch	`vendor/prime`: exactly what the converters make of each drop, never edited
Wiki	the project's versioned pages of procedures and decisions
YAML	the indented text format of `.gitlab-ci.yml`

K.3 SysML v1 and SysML v2

Table K.3: SysML v1 and v2 terms

SysML v1 (Cameo)	SysML v2 text	Note
Model (the project file)	a set of `.sysml` text files	readable, comparable, mergeable
Package	`package`	one top-level package per file here
Block	`part def`	a *definition*: the kind of thing
Part property	`part x : T;`	a *usage*: one occurrence of a definition
Reference property	`ref part`	
Value property, value type	`attribute`, `attribute def`	units by specializing ISQ quantities
Interface block, flow property	`port def` with directed features	
Port (proxy, full, flow)	`port p : P;`, conjugated `~P`	
Connector, binding connector	`connect a to b;`, `bind a = b;`	
Interface (connector typed by an association block)	`interface def`, `interface … connect`	
Activity, action, control flow	`action def`, `action`, `first a then b`	
Object flow, pin	`flow a.x to b.y`, action parameters	not converted; written by hand
Fork, join, decision, merge	`fork`, `join`, `decide`, `merge` nodes	not converted; written by hand
State machine	`state def`, `state`, `transition`	not converted; written by hand
Sequence diagram (interaction)	`message … from a to b`, occurrences	not converted; written by hand
Use case, include	`use case def`, `include use case`	include written by hand
Constraint block	`constraint def` with an expression	expression written by hand
Requirement (Id, Text)	`requirement` with `doc` and `doorsId`	named from the DOORS id
Satisfy, verify, deriveReqt	`satisfy R by X`, `verify R`, derivation connection	
Test case	`verification def`	
Allocate	`allocate A to B`	
Stereotype from a profile	`metadata def` and `@Name` annotations	project profiles become metadata in `model/library/conventions/`
Comment	`doc /* … */`	
Diagram	a *view*, drawn from the text by a tool	not stored; regenerated
`xmi:id`	kept in TODO lines and the delta	the key that links a v2 line to its v1 element
Definition and usage	the central v2 idea: `part def Battery` is the kind; `part battery : Battery` is one	

K.4 Agile words

Table K.4: Agile words

Word	Meaning in this book
Project	the whole effort, years long: here, eighteen months of port and maintenance
Tome	a collection of epics finished within a year (two semesters)
Epic	a collection of stories finished within a semester (13 sprints)
Story	several tasks delivering something useful within one sprint; a journal item
Task	work of less than a week, 5 points or fewer; a GitLab issue, usually one MR
Point	the size unit: one ideal person-day
Sprint	two weeks with a goal, from kickoff to review
Capacity	the points a person or team can work in a sprint, after holidays and leave
Velocity	points done per sprint, averaged
Carryover	committed work not done by the sprint's end, moved to the next
Kickoff, planning	day 1: goal, capacity, commitment
Stand-up	15 minutes daily: yesterday, today, blockers
Refinement	mid-sprint session preparing the next stories
Planning poker	estimating by simultaneous votes (`#est` rounds in the chat), discussing outliers
Review, demo	the last day: show the work to stakeholders
Retrospective	the last day: the team improves its own process
Definition of Ready	the checklist a story must meet before a sprint takes it
Definition of Done	the checklist work must meet to count as done
Pulse survey	five questions, 1 to 5, at each retrospective (scorecard measure 12)

K.5 Terms of this book and its kit

Table K.5: Terms of this book

Term	Meaning
Tidewater	the fictional coastal weather-observation network of the running example
The prime	the fictional prime contractor, Northwind Marine Systems, which maintains the v1 model
Prime drop	one delivery of the v1 model: XMI, ReqIF, CSV and `MANIFEST.txt`, under CDRL A003
DR	discrepancy report: a finding against a drop, sent to the prime
TODO line	a `// TODO` comment the converter writes for what it cannot carry; conversion debt
Conversion debt	the count of TODO lines left (scorecard measure 1)
Delta	the v1-to-v1 comparison of two drops (`xmi2sysml.pl --delta`), marking each change carried or manual
Manual (in a delta)	a v1 change the converter does not carry; invisible in the converted text
Drift	a requirement whose model text differs from DOORS
Orphan	a requirement nothing satisfies (measure F)
Unverified	a requirement no verification case verifies (measure G)
Trace completeness	percent of requirements both satisfied and verified (measure I)

Term	Meaning
Spider web	the tangled v1 model: duplicates, stale diagrams, package cycles, trace spaghetti
Scorecard	the twelve measures agreed before sprint 1
The kit	the plain-text toolkit this book uses: converters, model tools, journal, `gitlab.pl`
Journal	the kit's plain-text ledger of stories, sprints and stand-ups (`scrum.txt`)
Y/T/B	a terse stand-up status: Yesterday, Today, Blockers
Quad	the kit's one-page weekly report: priorities, watch items, milestones, accomplishments
Emulator	`gitlab.pl --local`: an offline stand-in for a GitLab server
`KIT`	the CI/CD variable naming where the kit is on the runner

CHAPTER

L

A twelve-week study plan per role

From what you know →

You know

a training plan with courses and dates.

→ Here it is called

a twelve-week study plan per role.

→ What is new

the learning time is budgeted on the team's board, not taken from evenings.

This book is long because it is a reference as well as a story. Nobody should read it front to back before doing anything. This appendix is the order the Tidewater team read it in, week by week, with the practice lessons and the exercises that go with each week, and the learning time the plan budgeted for it. Use it as a starting point for your own team: the roles matter more than the names.

The three roles:

- **The lead (Jim).** Owns the model, the backlog's tomes and epics, the change board and the conversation with management. Reads for decisions and for the case to make; types fewer commands than the others.
- **The modeler (Bob).** Does most of the model work: closing TODO lines, writing behavior by hand, reviewing. Reads for SysML v2 and Vim, and practises the most.
- **The pipeline owner (Tom).** A systems engineer who took on the GitLab project, the runner and the pipeline. Reads for setup, CI/CD and administration.

L.1 Where the hours come from

Learning time is planned work, not evenings and weekends (Chapter 41, For the lead: leading a reluctant team (p. 574), Chapter 14, Writing stories and tasks for model work (p. 214)). On Tidewater it is a `work::learning` story every sprint, with a task per person, sized like any other task (one point is one ideal day, about eight hours). The plan file is the source; print each person's learning tasks from it:

```
perl -e 'require "./sim/tidewater-plan.pl";
  for my $s (grep { $_->{n} <= 6 } @{ tw_plan() }) {
    for my $st (grep { $_->{work} eq "learning" } @{ $s->{stories} }) {
      printf "S%d %-4s %4.1f h  %s\n", $s->{n}, $_->{who}, 8 * $_->{p}, $_->{t} for @{
      $st->{tasks} } } }'
```

Table L.1: Learning hours per person per sprint, from the plan (printed by the block above)

Sprint	Jim	Bob	Tom
1	8 h	12 h	12 h
2	4 h	12 h	8 h
3	4 h	8 h	8 h
4	4 h	4 h	4 h
5	4 h	4 h	0 h
6	4 h	0 h	0 h

The learning share falls as competence rises: see the sprint-by-sprint split in Bottom line up front (p. iii). Reading this book's chapters and doing the practice lessons is what those hours buy. The practice lessons are offline, run on the laptop against the kit's GitLab stand-in, and take about forty-five minutes each (`perl agile.pl practice N`; `perl agile.pl practice check N` says what to do next).

L.2 Week zero: everyone

Before sprint 1, all three read the same short list, so the team starts with one vocabulary. The list climbs the ladder in the preface (From what you know to what you do not (p. xxxiv)): what you know first, then what is new.

- Bottom line up front (p. iii) and Preface (p. xxxii): what the twelve weeks are, and what they cost.
- Chapter 3, The directive (p. 22): what management asked for, and the scorecard that will measure it.
- Chapter 4, The Vee, kept: the systems-engineering V in MBSE DevSecOps (p. 34): the systems-engineering Vee you already know, and where each piece of it lives in GitLab.
- Chapter 5, DevSecOps in systems-engineering words (p. 46): every DevSecOps term in systems-engineering words.
- Chapter 9, The survival kit (p. 126), up to the end of the laptop checks; then `perl "$KIT/tools/gitlab/kickoff.pl" doctor` on each laptop.

The lead also reads Chapter 13, The operating model (p. 196) and Chapter 50, Your project in twelve weeks: a playbook (p. 672), because week zero is when the scorecard, the exchange agreement with the prime and the first meeting with the security officer happen.

L.3 Jim, the lead

Table L.2: The lead's twelve weeks

Week	Read	Do
1	Chapter 13, The operating model (p. 196), Chapter 14, Writing stories and tasks for model work (p. 214)	Practice lesson 10; write the Definition of Ready and Done; the exercises in Chapter 14, Writing stories and tasks for model work (p. 214)
2	Chapter 24, The rhythm: stand-ups, poker, refinement, kickoff, demo, retro (p. 377), Chapter 23, Stand-ups, the journal and the board (p. 367)	Practice lesson 11 (as the approver); run the first kickoff and stand-ups from the chapter
3	Chapter 11, Seeing the model: diagrams from text (p. 158), Chapter 15, Setting up GitLab (p. 226) (skim)	Practice lessons 12-13; the request-for-enablers memo (Chapter 37, Writing it down: DoD-style memos from Vim (p. 545))
4	Chapter 7, Requirements a pipeline can check: the recipe (p. 73), Chapter 45, The prime's model changes: receive, assess, disposition (p. 619) (the contract parts)	Draft the delivery interface with the prime; the exercises in Chapter 7, Requirements a pipeline can check: the recipe (p. 73)
5	Chapter 25, Talking about progress: helps for engineers who would rather not (p. 390), Chapter 22, Reviewing a model change (p. 357)	Draft a stand-up with `talk.pl` and edit it; review one of Bob's merge requests with the checklist
6	Chapter 32, Measuring with GitLab analytics (p. 513), Chapter 36, Telling management (p. 538)	Run `gitlab.pl analytics`; read the week's quad; write the sprint mail
7	Chapter 44, Verification as a closed loop (p. 613) (stage 1), Chapter 34, Proving the port is complete and correct (p. 527)	Write the trace page for the program office
8	Chapter 41, For the lead: leading a reluctant team (p. 574)	Plan the second semester's learning budget; the exercises in Chapter 41, For the lead: leading a reluctant team (p. 574)
9	Chapter 37, Writing it down: DoD-style memos from Vim (p. 545), Chapter 46, Systems engineering work, not pipeline work (p. 636)	Grade the requirements with the recipe; write the discrepancy report on the DOORS wording
10	Chapter 33, The twelve-week scorecard (p. 520)	Fill the scorecard from the evidence; check every number against its source
11	Chapter 35, The briefing (p. 532), Chapter 40, Documents still matter: review stages for generated documents (p. 570)	Rehearse the briefing on Day 9; move the briefing memo through the review stages
12	Chapter 38, How the team changed its mind (p. 557), Chapter 43, Maintenance through the life cycle (p. 602), Chapter 49, Scaling up: segments, blocks and a ladder of fidelity (p. 662) (skim)	Give the briefing; plan the next drop's intake and the next baseline; note which scaling lessons apply to the next block

L.4 Bob, the modeler

Table L.3: The modeler's twelve weeks

Week	Read	Do
1	Chapter 9, The survival kit (p. 126), Chapter 10, Vim for models (p. 143)	Practice lesson 10; set up Vim with the kit's SysML support; the exercises in Chapter 10, Vim for models (p. 143)
2	Chapter 6, SysML v2 for SysML v1 modelers (p. 57), Chapter 12, Git from zero, for systems engineers (p. 171)	Practice lesson 11; every exercise in Chapter 12, Git from zero, for systems engineers (p. 171), typed
3	Chapter 8, SysML v2 by example: a workbook (p. 92) (first half)	Practice lesson 12; the workbook's exercises up to verification cases
4	Chapter 8, SysML v2 by example: a workbook (p. 92) (second half), Chapter 11, Seeing the model: diagrams from text (p. 158)	Practice lesson 13; draw the four views of the drop-1 model
5	Chapter 22, Reviewing a model change (p. 357), Chapter 16, The model repository: what goes where (p. 245)	Review one of Tom's merge requests with the checklist; the exercises in Chapter 22, Reviewing a model change (p. 357)
6	Chapter 25, Talking about progress: helps for engineers who would rather not (p. 390)	Draft every stand-up with `talk.pl`, then edit it; the "so what?" check on a week of statuses
7	Appendix F, Model error catalog (p. 731), Appendix E, Rescue: when something goes wrong (p. 724)	Reproduce three errors from the catalog and fix them; one rescue recipe
8	Chapter 7, Requirements a pipeline can check: the recipe (p. 73)	Grade the Stations requirements; write two rewrites
9	Chapter 47, Keeping SysML v2 from becoming a spider web (p. 645), Appendix G, Modeling style guide (p. 739)	Read the package view; propose the next cycle break
10	Chapter 45, The prime's model changes: receive, assess, disposition (p. 619) (the drop loop)	Run the impact report on the next drop in a sandbox (Chapter 43, Maintenance through the life cycle (p. 602))
11	Chapter 44, Verification as a closed loop (p. 613)	Write a verification case for a requirement without one
12	Chapter 39, A day in the life, week twelve (p. 563)	Compare your own week with Bob's day in week twelve

L.5 Tom, the pipeline owner

Table L.4: The pipeline owner's twelve weeks

Week	Read	Do
1	Chapter 9, The survival kit (p. 126), Chapter 10, Vim for models (p. 143) (the checks and conflicts sections), Chapter 12, Git from zero, for systems engineers (p. 171), Chapter 15, Setting up GitLab (p. 226)	Practice lesson 10; `kickoff.pl doctor --fix`; request the group and the runner
2	Chapter 19, CI/CD from zero (p. 288), Chapter 21, Runners: where the pipeline runs (p. 347)	Practice lesson 11; register the runner; run the pipeline locally with `gitlab.pl ci`
3	Chapter 20, The pipeline, job by job (p. 302)	Practice lesson 12; the exercises in Chapter 20, The pipeline, job by job (p. 302); add one job and watch it in a merge request
4	Chapter 17, The DevSecOps tools that earn their keep (p. 255), Chapter 16, The model repository: what goes where (p. 245)	Practice lesson 13; the model site on Pages; the nightly schedule
5	Chapter 48, Security: protect the environment, engineer it into the system (p. 651) (the environment half)	Runner hardening with the security officer; the secrets job; the spillage tabletop
6	Chapter 42, Administration (p. 586)	The monthly drift check (`gitlab.pl setup --diff`); the access review
7	Chapter 45, The prime's model changes: receive, assess, disposition (p. 619) (Jenkins and the intake)	The delivery interface with the prime's pipeline; trigger and deploy tokens in a sandbox
8	Chapter 43, Maintenance through the life cycle (p. 602) (the runbook)	Run the drop intake runbook end to end in a sandbox
9	Chapter 25, Talking about progress: helps for engineers who would rather not (p. 390), Chapter 24, The rhythm: stand-ups, poker, refinement, kickoff, demo, retro (p. 377)	Narrate a rehearsed demo; rotate the driver
10	Appendix E, Rescue: when something goes wrong (p. 724)	Every rescue recipe, in a scratch repository
11	Chapter 43, Maintenance through the life cycle (p. 602) (baselines)	Cut a release and a hotfix in a sandbox
12	Chapter 32, Measuring with GitLab analytics (p. 513)	Pipeline success rate and duration for the briefing

L.6 Check yourself

At the end of each sprint, each person should be able to say yes to that sprint's "You can now..." list in the sprint chapter (Chapter 26, Sprint 1: A GitLab project the team can work in (p. 404) to Chapter 31, Sprint 6: Behavior by hand, a table for the program, and baseline-0 (p. 495)). If the answer is no for two sprints running, the learning budget is too small or the backlog too full; that is a conversation for the retrospective, not a private worry.

Exercise L.1: Build your own plan

Copy the three tables into your team's wiki, rename the roles to your people, and move rows to match your program (a team with no supplier drops can skip the drop-loop rows

until it has one). Then print your own plan's learning tasks the way the block above does, and check that the hours in the tables match what the backlog actually reserves.

CHAPTER

M

Further reading

From what you know →

You know

the applicable documents list.

→ Here it is called

further reading.

→ What is new

nothing; this is where to go deeper.

Titles only, deliberately: web addresses move, and an address printed in a book is wrong within a year. Every item below is easy to find by its title, and most are free. Check with your security office before downloading anything to a work laptop; several are available in the program's library.

M.1 SysML v2 and the language underneath

- **OMG Systems Modeling Language (SysML), Version 2.0**, the specification. Read Part 1 (the language) for the textual notation; dip into the rest as needed. The kit's grammar check is built from the official grammar that comes with it.
- **OMG Kernel Modeling Language (KerML), Version 1.0**, the foundation SysML v2 is defined on. Useful when a SysML construct behaves unexpectedly and you need the underlying rule.
- **OMG Systems Modeling Application Programming Interface (API) and Services**, the standard API for model repositories, if your program plans to publish models to an API server.
- **The SysML v2 Pilot Implementation**, the open-source reference implementation and its training material, including the textual notation examples. The kit's `model.pl validate` can call it for full semantic checking.
- **The SysML v2 release repository's training and examples folders**, the most direct way to see idiomatic v2 text.

M.2 Systems engineering

- **INCOSE Systems Engineering Handbook**, the shared vocabulary for the processes this book renames (configuration management, verification, technical reviews).
- **NASA Systems Engineering Handbook**, free, and a good second view of the same processes from a program that writes everything down.
- **The DoD Digital Engineering Strategy**, for the policy background behind the directive in Chapter 3, The directive (p. 22).

M.3 Git, GitLab and DevSecOps

- **Pro Git**, by Scott Chacon and Ben Straub, free online. Chapters 1 to 3 cover everything in Chapter 12, Git from zero, for systems engineers (p. 171) and more; chapter 7 is the rescue kit (Appendix E, Rescue: when something goes wrong (p. 724)) in depth.
- **The GitLab documentation**, especially the user guides for merge requests, protected branches, CODEOWNERS, CI/CD YAML syntax, runners, and the Generic Package Registry. GitLab's documentation is versioned: read the version that matches your instance.
- **The GitLab Runner documentation** on the shell and Docker executors, for Chapter 21, Runners: where the pipeline runs (p. 347).
- **The DoD Enterprise DevSecOps Reference Design** and its playbooks, the program-level picture of what this book does at team scale.
- **NIST SP 800-218, the Secure Software Development Framework**, as a checklist for the pipeline's security posture (Chapter 48, Security: protect the environment, engineer it into the system (p. 651)).

M.4 Security frameworks named in this book

- **NIST SP 800-53**, security and privacy controls, used under the Risk Management Framework (**NIST SP 800-37**).
- **NIST SP 800-171**, protecting controlled unclassified information, and the **CMMC** program built on it.
- Your program's security classification guide and marking instructions, which outrank everything else on this list.

M.5 Agile, for systems engineers

- **The Scrum Guide**, by Ken Schwaber and Jeff Sutherland, short and free. Tidewater's rhythm is a light version of it (Chapter 13, The operating model (p. 196)).
- **The Agile Manifesto and its twelve principles**, one page, worth reading for what it does *not* say.

M.6 Vim

- **Vim's own help**, which is better than any book: `:help user-manual` for the guided tour, `:help quickref` for the one-page summary, and `:help agile` for the kit's commands (Chapter 10, Vim for models (p. 143)).

- **vimtutor**, the thirty-minute interactive lesson that ships with Vim: run `vimtutor` in Git Bash.

M.7 The kit's own documents

These ship with the kit, in `docs/`, and need no download:

- `docs/GITLAB.html`, the GitLab guide written for this team's process.
- `docs/SYSML-PORT.html` and `docs/SYSML-PORT-QRG.pdf`, the port guide and its one-sheet card.
- `docs/VIM-CHEATSHEET.html`, one page for every Vim mode.
- `docs/OLLAMA.md`, installing a local model, approvals first (Chapter 18, A local AI assistant on the laptop (p. 269)).
- `docs/HELP.html`, the kit's help, the same text as `:help agile` and `perl agile.pl help`.

CHAPTER

N

References: DoD and Microsoft guidance

From what you know →

You know

the applicable documents list at the front of a SEMP, with each document's number and where to get it.

→ Here it is called

references: the DoD's software and DevSecOps guidance and Microsoft's Azure Government pages.

→ What is new

these documents are written for software teams; this appendix says which part a systems engineer needs.

Appendix M, Further reading (p. 809) lists titles only, because web addresses move. This appendix makes one exception: the official DoD and Microsoft pages a program office will ask about, with the addresses as they stood on 7 October 2026. Start from the two index pages (the DoD CIO library and Microsoft's Azure Government documentation); when an address below has moved, the document is still listed there under the same title. Check with your security office before downloading anything to a work laptop.

N.1 DoD software and DevSecOps guidance

The DoD Chief Information Officer (CIO) publishes the DevSecOps document set as a library: a fundamentals document for everyone, guidebooks and playbooks for practitioners, and reference designs for the teams that build a software factory. All of it is approved for public release.

Document	What to read it for
DoD CIO Library (the index) `https://dodcio.defense.gov/library/`	the current versions of everything in this section; start here

Document	What to read it for
DoD Enterprise DevSecOps Fundamentals `https://dodcio.defense.gov/Portals/0/Documents/Library/DoD%20Enterprise%20DevSecOps%20Fundamentals_DoD-CIO_20211019.pdf`	the DoD's definitions: the lifecycle, the software factory, culture and metrics. The vocabulary management will use; compare it with Chapter 5, DevSecOps in systems-engineering words (p. 46)
DevSecOps Fundamentals Guidebook: DevSecOps Tools and Activities `https://dodcio.defense.gov/Portals/0/Documents/Library/DevSecOps%20Fundamentals%20Guidebook-DevSecOps%20Tools%20and%20Activities_DoD-CIO_20211019.pdf`	the activities and tools by phase. Map the team's pipeline jobs (Chapter 20, The pipeline, job by job (p. 302)) onto its tables when an assessor asks "which activities do you perform?"
DoD Enterprise DevSecOps: Pathway to a Reference Design `https://dodcio.defense.gov/Portals/0/Documents/Library/DoD%20Enterprise%20DevSecOps-Pathway%20to%20a%20Reference%20Design_DoD-CIO_20211018.pdf`	how a reference design is built up; read before the Kubernetes reference design
DoD Enterprise DevSecOps Reference Design: CNCF Kubernetes (in the library)	the design the platform team follows when the runners run on Kubernetes; what a job pod may and may not do (Chapter 21, Runners: where the pipeline runs (p. 347))
DoD Enterprise DevSecOps Strategy Guide `https://dodcio.defense.gov/Portals/0/Documents/Library/DoDEnterpriseDevSecOpsStrategyGuide.pdf`	the why, for the program manager: the argument Chapter 3, The directive (p. 22) makes, in the DoD's words

N.2 Acquisition policy

Document	What to read it for
DoD Software Modernization Strategy (2022) `https://media.defense.gov/2022/Feb/03/2002932833/-1/-1/1/department-of-defense-software-modernization-strategy.pdf`	the department-level goals the directive cites: software factories, cloud, faster authorization
DoDI 5000.87, Operation of the Software Acquisition Pathway `https://www.esd.whs.mil/Portals/54/Documents/DD/issuances/dodi/500087p.PDF`	how a software program is bought and reported: iterations, value assessments, metrics. Read it to understand why management asks for the measures in Chapter 33, The twelve-week scorecard (p. 520) and Chapter 32, Measuring with GitLab analytics (p. 513); the Directives Division site `https://www.esd.whs.mil/DD/` has the current version

N.3 Microsoft: Azure Government for DoD

Document	What to read it for
Department of Defense (DoD) in Azure Government `https://learn.microsoft.com/en-us/azure/azure-government/documentation-government-overview-dod`	the DoD regions and which impact levels each supports; what "the runners are in Azure" means for where the model may live
Azure Government isolation guidelines for Impact Level 5 workloads `https://learn.microsoft.com/en-us/azure/azure-government/documentation-government-impact-level-5`	how IL5 workloads are isolated; for the platform team, and for the ISSM conversation in Chapter 48, Security: protect the environment, engineer it into the system (p. 651)
DevSecOps puts security at the heart of program development (Azure Government blog) `https://devblogs.microsoft.com/azuregov/devsecops-puts-security-at-the-heart-of-program-development/`	a short orientation to Microsoft's DevSecOps offerings for government

Tip: Which three to read first

For a systems engineer starting the twelve weeks: the **DevSecOps Fundamentals** (the words), the **Tools and Activities guidebook** (to map your pipeline), and **DoDI 5000.87** (to understand what management will measure). Leave the reference designs and the Azure pages to the platform team unless you are asked.

CHAPTER

O

Answers to the exercises

Every exercise in the book has an answer here, in book order. Where an exercise asks you to run something, the answer runs it, and the output shown is what the kit really prints; the blocks are tested like every other command in the book, so if your output differs, the difference is worth a look. Where an exercise asks for judgement, the answer is a model answer: one good response, the reasoning behind it, and what any good answer must contain. Your answer can differ from a model answer and still be right; it cannot leave out what the "must contain" lines name.

Many exercises ask about *your own* program. Those answers show what a good response looks like for Tidewater, so you can see the shape, and then say what to look for in yours.

The commands below run in three places. A scratch folder holds drops, small models and offline GitLab projects made for the answer. The `tidewater` folder is the simulation's state after sprint 6, the same one the sprint chapters use. A few answers read the kit itself. Each block says which.

O.1 Part I: where they start

Exercise 1.1 (p. 9): Count the copies

A typical result for one interface requirement on a document-era program: the system specification (owned by the systems engineering lead, updated at the last revision), the interface control document (owned by the interface lead, two revisions behind), the requirements database export (owned by the requirements manager, refreshed monthly), the verification matrix spreadsheet (owned by the test lead), and two briefings. Six places; on most programs two or three agree word for word.

You cannot find out without reading them all, which is the point of the exercise. A good answer names the one tool that would answer it mechanically: a comparison of the text with the same identifier in each place. In this book that is the trace job's *drift* list, which compares DOORS text with the model's text for every requirement on every change (Chapter 20, The pipeline, job by job (p. 302)).

Must contain: every place, an owner for each, a date for each, and an honest "I could not check this one" where that is true.

Exercise 1.2 (p. 9): Reconstruct a change

Lay the change out as dated events: request raised, impact assessment started and finished, board scheduled, board held, document updated, document re-baselined, downstream documents updated. Then mark each interval *deciding* (a person working on it) or *waiting* (in a queue, an inbox, a calendar). On most programs the waiting is well over half the elapsed time; the chapter's example was mostly waiting for the next board and for the document owners.

Must contain: the elapsed time, the decided time, the fraction waiting, and the one queue that held it longest. That queue is the first thing a merge request with automatic checks removes.

Exercise 1.3 (p. 9): The five virtues

A model answer for Tidewater's document era:

- **Readable by anyone:** a Word file opens everywhere. Cost: anyone can read it, nobody can check it mechanically.
- **Contractually meaningful:** the specification is a deliverable with a signature. Cost: changing it takes a board.
- **Reviewed:** every revision went through a review meeting. Cost: weeks of calendar time per review.
- **Baselined:** revision letters and a document register. Cost: the register and the documents drift apart.
- **Traced:** a verification matrix spreadsheet. Cost: two days to rebuild it before every review.

Keep the page. After Chapter 33, The twelve-week scorecard (p. 520), each line should have a GitLab counterpart: a text file anyone can read, a merge request with approvals, review in hours, a protected tag, a trace job.

Exercise 2.1 (p. 21): Measure it yourself

Generate drop 1 and search it:

```
perl "$KIT/sim/tidewater-v1.pl" --drop 1 drop1 > /dev/null
grep -n -o 'name="Battery[^"]*"' drop1/tidewater.xmi
```

There are two: `Battery` and `Battery_old`. The real battery is the one something uses. A block is real if another element has it as a type:

```
for n in Battery Battery_old; do
  id=$(grep -o "xmi:id=\"[^\"]*\" name=\"$n\"" drop1/tidewater.xmi | sed
  's/xmi:id="\([^"]*\)".*/\1/')
  echo "$n used $(grep -c "type=\"$id\"" drop1/tidewater.xmi) time(s)"
done
```

`Battery` is the type of the power system's battery part; `Battery_old` sits in the "Legacy Import" package and nothing refers to it. Deciding by use, not by name, is the habit to keep.

Exercise 2.2 (p. 21): Read the manifest

```
cd drop1
grep 'tidewater\.' MANIFEST.txt | tr -d '\r' | sed 's/^ *//' | sha256sum -c
```

The checksums match. If one did not, the file you hold is not the file the prime's build recorded: corrupted in transit, edited by someone, or from a different build. Do not import it. Call the prime's modeler (the contract's named technical contact) and ask for the drop to be sent again; record the event on the Prime Drop issue (Chapter 45, The prime's model changes: receive, assess, disposition (p. 619)).

Exercise 2.3 (p. 21): Belief and fact

A model answer sorts each belief into one of three columns:

- **About the language:** "SysML cannot express our constraints" (it can, with constraint blocks; the team never used them), "a requirement must be a diagram element" (it is an element; diagrams are views of it).
- **About the tool:** "the model is the diagrams", "you cannot merge two people's work", "you need a seat to read the model". These are Cameo's file format and licensing, not SysML.
- **About how the team was asked to work:** "the model must contain every document paragraph", "every slide needs a diagram", "nobody may delete anything". These came from how the port was ordered.

Must contain: at least one belief in each column. Most teams find the third column is the longest, which is good news: it is the one they can change.

Exercise 3.1 (p. 33): Read your own directive

Three columns: what it says (quote it), what it requires (the minimum that would satisfy an auditor), what it leaves open (everything else). For Tidewater's directive (Chapter 3, The directive (p. 22)), "adopt DevSecOps practices" *says* a lot and *requires* little: a version-controlled model, automated checks, a demonstration within twelve weeks. It leaves open the tools, the branching model, the measures and the team's pace.

The open items belong to the person who will be held to the result, which for an MBSE team is the model lead, with the program manager's concurrence. Write them down as decisions (Chapter 13, The operating model (p. 196)) so they stop being open.

Exercise 3.2 (p. 33): Write down the objections

The exercise is the method; a good result looks like Tidewater's list in Chapter 3, The directive (p. 22): nine objections in the speakers' words, each with a column "answered where, with what evidence". Typical entries: "we will lose the diagrams" (answered by generated views, Chapter 11, Seeing the model: diagrams from text (p. 158)); "nobody will read text" (answered by the model site the program office reads, Chapter 17, The DevSecOps tools that earn their keep (p. 255)); "this is a software process" (answered by the systems engineering share of the work, Bottom line up front (p. iii)).

Must contain: the speaker's own words, not a paraphrase, and a planned date for each answer. An objection without a date is a promise nobody is held to.

Exercise 3.3 (p. 33): Pick your measures

Five that work for most ports, each computable without judgement:

1. TODO lines left after conversion, from the pipeline's `todo` job; target zero; old-way baseline: none (nobody counted).
2. Requirements with a verification case, from the trace job; target: every requirement the government owns; old-way baseline: the last verification matrix, counted by hand.
3. Merge requests that went through a green pipeline and an approval, from GitLab; target: all of them after the first sprint; old-way baseline: changes made without a recorded review.
4. Change lead time, from GitLab analytics (Chapter 32, Measuring with GitLab analytics (p. 513)); target: days, not weeks; old-way baseline: the board cycle.
5. The team's pulse survey; target: rising; old-way baseline: the first retrospective.

The skeptic usually rejects the pulse survey ("feelings are not data") and the lead time ("you are comparing a typo fix with a board decision"). Both objections are fair. Answer the first by keeping the survey anonymous and the questions fixed; answer the second by comparing like with like, and by saying so on the slide.

Exercise 4.1 (p. 44): Map your program's Vee

A good answer names every rung on the right arm, puts "nobody" or "a person, before each review" against at least one of them, and circles a rung below system verification: those are the repetitive ones. Tidewater's answer circles the verification matrix, which used to take two people a week before each review.

Exercise 4.2 (p. 44): Find the three Vees

A merge request's Vee: any change into `develop`, with the ten jobs `gitlab.pl ci --list --event merge_request_event --target develop` shows. A baseline's Vee: a `baseline-N` tag, with the thirteen jobs `--event tag --tag baseline-N` shows, three of them the release jobs. The program's Vee: the next technical review, which reads the latest baseline's release package.

Exercise 4.3 (p. 44): Break and mend a horizontal line

The job reports `orphans 2 -> 3 (new: …)` and fails. Two sentences for the program manager: "The check stops a change that breaks a link between a requirement and the design that meets it. The nine gaps already in the model are the prime's, reported to the prime, and they are counted every day instead of blocking our work."

Exercise 4.4 (p. 44): The thread before the twin

Most programs moving from documents have none of the four on day one; a good answer says so and names the first step, usually stable ids from the prime and `doorsId` on every requirement. The decision a twin would change must come with a number, for example "how many buoys to recover before winter, worth one vessel day each".

Exercise 5.1 (p. 56): Make your own list

A starting list, with the systems engineering term beside each:

DevSecOps term	Closest SE / CM term
repository	controlled document library
commit	a change to a controlled item, with author and date
branch	a working copy for one change
merge request	engineering change request with its review
pipeline	configuration audit plus review checklist, run by machine
tag	baseline
release	published baseline with its release notes
runner	the machine the audit runs on (no equivalent)
artifact	review package, generated
code owner	approval authority for an item

"Runner" and "pipeline schedule" usually have no equivalent; that is fine. Compare with the translation table in Chapter 5, DevSecOps in systems-engineering words (p. 46) and keep the words your team already uses for the rest.

Exercise 5.2 (p. 56): Which idea first?

For Tidewater the time-saver was automatic checks on every change (the grammar and lint jobs found in minutes what review meetings found in weeks) and the most resisted was small changes reviewed one at a time ("we will drown in reviews"). They are usually *different* ideas, and that matters: lead with the time-saver, because it pays for the patience the resisted idea needs. A good answer names both and says which one you will introduce first.

Exercise 5.3 (p. 56): Follow a change

Write your change against the ten steps (issue, branch, edit, local check, commit, push, merge request, pipeline, review and approval, merge). In a document-era process, steps 1, 7 and 9 usually exist (a change request, a review, a signature), steps 4 and 8 do not exist

at all (no automatic check), and step 10 is a person re-typing the change into the master document. The time goes into the review queue and into re-typing. A good answer marks each step "existed / missing / done by hand" and puts a duration on each existing one.

Exercise 6.1 (p. 72): Convert and count

```
perl "$KIT/bin/xmi2sysml.pl" --report --out drop1-v2 drop1/tidewater.xmi > drop1-report.txt
grep -r "// TODO" drop1-v2 | wc -l
```

Fifty-two, which matches the table "Drop 1's conversion debt, by kind" in Chapter 6, SysML v2 for SysML v1 modelers (p. 57).

Exercise 6.2 (p. 72): Read a package

```
grep -n "part \|port \|connection" drop1-v2/datacenter/DataCenter.sysml
```

The data segment has four parts (ingest, quality control, archive, publisher), three external ports (observations in, health in, the public interface out) and five connections in a chain: observations into ingest, ingest to quality control, quality control to the archive, the archive to the publisher, the publisher out to the public port. Most readers draw it in a few minutes, faster than finding the right diagram among the forty-seven in the v1 model, and the text cannot be out of date with itself.

Exercise 6.3 (p. 72): Find a hidden error

```
grep -c "OpaqueExpression" drop1/tidewater.xmi
```

There is only one, the energy balance. A discrepancy report for it: "The constraint `balance` on `EnergyBalance` (xmi:id in the drop) uses `sunHours`, which is not a parameter of the constraint block. Please declare `sunHours` (with its unit) or state where its value comes from. Impact: the government's v2 model cannot evaluate the constraint until it is declared." Name the element by its id, say what is wrong, say what you need, say why it matters.

Exercise 6.4 (p. 72): Write a state machine

```
cat > modes.sysml <<'EOF'
package ModesExample {
    state def BuoyModes {
        entry; then observing;
        state observing;
        state degraded;
        state maintenance;
        state recovered;
```

```
        transition observing_to_degraded first observing accept LowBattery then degraded;
        transition degraded_to_observing first degraded accept Charged then observing;
        transition to_maintenance first observing accept CrewOnSite then maintenance;
        transition degraded_to_maintenance first degraded accept CrewOnSite then
        maintenance;
        transition back_to_service first maintenance accept CrewDeparts then observing;
        transition lifted first maintenance accept LiftedAboard then recovered;
    }
    attribute def LowBattery;
    attribute def Charged;
    attribute def CrewOnSite;
    attribute def CrewDeparts;
    attribute def LiftedAboard;
}
EOF
perl "$KIT/tools/sysml/sysml.pl" check modes.sysml
```

One new line, named so it reads as a sentence, reusing the existing `CrewOnSite` event.

Exercise 7.1 (p. 90): Decompose by hand, then check

By hand: classification (missing; Tidewater is unmarked, so optional here), state and trigger (missing), specification target "the power system", operative verb "shall", target verb "report", precise object "the battery state of charge", modifier and modification (missing), reference (missing), exceptions (missing), ID TW-SEG-003. All required parts are present. A test engineer needs the optional parts the sentence lacks: *to whom* it reports, *how often*, and *how accurately*. Without them the requirement cannot fail a test.

```
perl "$KIT/tools/sysml/req-recipe.pl" --only TW-SEG-003 drop1/tidewater.reqif
```

The checker agrees: every required part found, one minor finding, no threshold.

Exercise 7.2 (p. 90): Split TW-SYS-001

```
cat > split.txt <<'EOF'
(U) While in observing mode, each station shall measure wind speed, wind direction, air
pressure and air temperature at least once every 10 minutes. (TW-SYS-001a)

(U) While in observing mode, each moored buoy shall measure wave height and wave period at
least once every 30 minutes. (TW-SYS-001b)
EOF
perl "$KIT/tools/sysml/req-recipe.pl" --strict --require classification split.txt
```

What you had to decide: how often each quantity is measured (the original had no rate), whether waves are measured only by buoys (the original said "each buoy shall *also*"), and in which mode the requirement applies. The rates above are notional; on a real program they come from the stakeholder need, and asking for them is a discrepancy report to whoever owns the wording.

Exercise 7.3 (p. 91): A need that can be traced

"Mariners shall be able to obtain the latest marine bulletin and the latest observation from every station through a public service, at any time, without an account." It stays a stakeholder need (it names the mariner and the outcome, not a system element) and becomes verifiable by demonstration: the program office watches a mariner, with no account, read the latest bulletin and observations. The evidence is the demonstration record. The weak verb "be able to read" is replaced by an outcome someone can watch happen.

Exercise 7.4 (p. 91): Add a constraint

```
cat > recipe-seg.sysml <<'EOF'
package RecipeSeg {
    private import ISQ::*;
    private import SI::*;
    part def PowerSystem { attribute socReportPeriod : DurationValue; }
    requirement def StateOfChargeReporting {
        doc /* The power system reports the battery state of charge at least once per
        reporting period. */
        subject power : PowerSystem;
        attribute reportingPeriod : DurationValue;
        require constraint { power.socReportPeriod <= reportingPeriod }
    }
    requirement tw_seg_003 : StateOfChargeReporting {
        attribute doorsId = "TW-SEG-003";
        doc /* (U) While in observing mode, the power system shall report the battery state
        of charge to the data logger every 10 minutes with an accuracy of 5 % of rated
        capacity, except during maintenance mode. */
        attribute :>> reportingPeriod = 10 [min];
    }
}
EOF
perl "$KIT/tools/sysml/sysml.pl" check recipe-seg.sysml
```

```
perl "$KIT/tools/sysml/req-recipe.pl" recipe-seg.sysml
```

The sentence carries the recipe for people; the constraint carries the same rule for the tools.

Exercise 7.5 (p. 91): Write the DR

One screen: a title ("DR-6: thirteen DOORS requirements do not meet the requirement recipe"), a table of the thirteen IDs with one-line findings (TW-STK-001 weak verb "provide"; TW-STK-003 and TW-STK-004 "be able to"; TW-SYS-001 two operative verbs; TW-SYS-012 "be serviceable"; the rest "no measurable threshold"), and the suggested wording for the five in `recipe-rewrites.txt`.

Ask the prime to fix TW-SYS-001 and TW-SYS-012 first: TW-SYS-001 is two requirements in one sentence, so one of them cannot be verified separately, and TW-SYS-012 has no satisfy in the model and no verifiable verb, so it blocks both traceability and verification. The "no

threshold" findings matter too, but they can wait for the wording owner's next revision.

O.2 The SysML v2 workbook

Each answer below writes its snippet into the scratch folder and checks it. The snippets are complete files, so you can run any one of them on its own.

Exercise 8.1 (p. 94): Break it three ways

```
cat > fixed.sysml <<'EOF'
package Stations {
    part def Ballast;
    part def Hull { part ballast : Ballast; }
}
EOF
sed 's/part def Ballast;/part def Ballast/' fixed.sysml > m1.sysml
sed '$d' fixed.sysml > m2.sysml
sed 's/part ballast/prat ballast/' fixed.sysml > m3.sysml
for f in fixed m1 m2 m3; do echo "== $f.sysml"; perl "$KIT/tools/sysml/sysml.pl" check
$f.sysml | head -1; done
```

Only the misspelled `prat` is reported on the line that is actually wrong (line 3, at the word after `prat`, because the checker read `prat` as a name and then found another name). The missing `;` is reported one line *later*, where the next statement starts. The missing `}` is reported at the end of the file, wherever that is. The two rules from Chapter 8, SysML v2 by example: a workbook (p. 92) in action: fix the first error first, and look one line up.

Exercise 8.2 (p. 95): Narrow the imports

```
cat > packages.sysml <<'EOF'
package Stations {
    private import Types::StationMode;
    public import Interfaces::ObservationPort;
    part def Hull;
    part def Mooring { attribute mode : StationMode; }
}
package Types { attribute def StationMode; }
package Interfaces { port def ObservationPort; }
package Fourth {
    private import Stations::*;
    part def Check { part hull : Hull; port p : ObservationPort; }
}
EOF
perl "$KIT/tools/sysml/sysml.pl" check packages.sysml
```

`Fourth` sees `Hull` and `Mooring` (Stations' own public members) and `ObservationPort`, because Stations imports it *publicly* and so re-exports it. It does not see `StationMode`, which Stations imports privately. A public import is a promise to every package that imports you;

make imports private unless you mean that promise.

Exercise 8.3 (p. 96): Document a decision

```
cat > names.sysml <<'EOF'
package 'Data Center' {
    part def QualityControl {
        doc /* Checks each observation and sets its quality flag. */
    }
    comment about QualityControl
        /* Decision 2026-11: flag, do not delete. A deleted observation cannot be reviewed
        later, and the archive
           keeps every record with its flag (TW-SYS-005). */
}
EOF
perl "$KIT/tools/sysml/sysml.pl" check names.sysml
```

The forecaster's document wants the `doc`: what quality control *is*. The decision belongs in review and in the decision log; if it matters for years, it belongs in a decision record (Chapter 13, The operating model (p. 196)), and the comment points to it.

Exercise 8.4 (p. 98): A shore station

```
cat > parts.sysml <<'EOF'
package Parts {
    part def SensorSuite;
    part def Battery;
    part def Cell;
    part def Mast;
    part def ObservingStation {
        part sensors : SensorSuite;
        part battery : Battery { part cells : Cell[4]; }
    }
    part def MooredBuoy :> ObservingStation;
    part def ShoreStation :> ObservingStation { part mast : Mast; }
    part def StationSegment {
        part buoys : MooredBuoy[1..*];
        part shoreStations : ShoreStation[1..*];
    }
}
EOF
perl "$KIT/tools/sysml/sysml.pl" check parts.sysml
```

A cell in a buoy in the segment is four levels down from the segment: segment, buoy, battery, cell. The buoy inherits the battery from `ObservingStation`, so the depth is the same in a shore station.

Exercise 8.5 (p. 98): A shore-station suite

```
cat > redefinition.sysml <<'EOF'
package Redefinition {
    private import ISQ::*;
    private import SI::*;
    part def Sensor { attribute samplePeriod : DurationValue; }
    part def SensorSuite {
        part sensors : Sensor[1..*];
        attribute samplePeriod : DurationValue = 10 [min];
    }
    part def Barometer :> Sensor;
    part def ShoreSensorSuite :> SensorSuite {
        part barometer : Barometer :> sensors;
        attribute :>> samplePeriod = 5 [min];
    }
}
EOF
perl "$KIT/tools/sysml/sysml.pl" check redefinition.sysml
```

Now without the `:>>`, in a lint project:

```
mkdir -p lintdemo/model && printf 'title = Lint demo\n' > lintdemo/sysml.conf
sed 's/attribute :>> samplePeriod/attribute samplePeriod/' redefinition.sysml >
lintdemo/model/Redefinition.sysml
perl "$KIT/tools/sysml/model.pl" --root lintdemo lint
```

Lint says nothing, and that is the lesson. Without `:>>` the suite declares a *second* attribute with the same name that hides the inherited one; it is legal text and the kit's lint does not flag the shadowing. A full SysML v2 tool with semantic checks (the OMG pilot implementation, `model.pl validate` with Java) reports it. Until then, reviewers look for the same name declared twice in a specialization (Chapter 22, Reviewing a model change (p. 357)).

Exercise 8.6 (p. 100): A health interface

```
cat > ports.sysml <<'EOF'
package Ports {
    item def ObservationRecord;
    item def HealthReport;
    port def ObservationPort { out item observation : ObservationRecord; }
    port def HealthPort { out item health : HealthReport; }
    interface def ObservationLink { end transmitter : ObservationPort; end receiver :
    ~ObservationPort; }
    interface def HealthLink { end transmitter : HealthPort; end receiver : ~HealthPort; }
    part def TelemetryUnit { port obsOut : ObservationPort; port healthOut : HealthPort; }
    part def IngestService { port obsIn : ~ObservationPort; port healthIn : ~HealthPort; }
    part def Network {
        part telemetry : TelemetryUnit;
        part ingest : IngestService;
```

```
        interface link : ObservationLink connect transmitter ::> telemetry.obsOut to
        receiver ::> ingest.obsIn;
        interface health : HealthLink connect transmitter ::> telemetry.healthOut to
        receiver ::> ingest.healthIn;
    }
}
EOF
perl "$KIT/tools/sysml/sysml.pl" check ports.sysml
```

Exercise 8.7 (p. 101): Two flows

```
cat > flows.sysml <<'EOF'
package Flows {
    item def ObservationRecord;
    item def HealthReport;
    part def DataLogger { out item record : ObservationRecord; out item health :
    HealthReport; }
    part def TelemetryUnit { in item record : ObservationRecord; in item health :
    HealthReport; }
    part def Station {
        part logger : DataLogger;
        part telemetry : TelemetryUnit;
        connection toTelemetry connect logger to telemetry;
        flow of ObservationRecord from logger.record to telemetry.record;
        flow of HealthReport from logger.health to telemetry.health;
    }
}
EOF
perl "$KIT/tools/sysml/sysml.pl" check flows.sysml
```

One connection is enough: the connection says the two parts are joined, and each flow says what crosses. Two flows on one connection is usually clearer to a reviewer, because it mirrors the hardware (one cable, two kinds of traffic). A second connection is right only when there really are two physical paths.

Exercise 8.8 (p. 102): A power budget's inputs

```
cat > values.sysml <<'EOF'
package Values {
    private import ISQ::*;
    private import SI::*;
    private import ScalarValues::*;
    attribute def Power :> PowerValue;
    part def Battery {
        attribute stateOfCharge : Real;
        assert constraint socRange { stateOfCharge >= 0.0 and stateOfCharge <= 1.0 }
    }
    part def Station { attribute averageLoad : Power; }
}
EOF
perl "$KIT/tools/sysml/sysml.pl" check values.sysml
```

```
mkdir -p lintvalues/model && printf 'title = Lint demo\n' > lintvalues/sysml.conf && cp
values.sysml lintvalues/model/
perl "$KIT/tools/sysml/model.pl" --root lintvalues lint
```

A fraction from zero to one, constrained, is the simplest honest “percentage”. Reading the library turns up dimensionless ratio types too; use whichever your team agrees on, once, in the library package, so every model says it the same way.

Exercise 8.9 (p. 103): A reserve

```
cat > expressions.sysml <<'EOF'
package Expressions {
    private import ISQ::*;
    private import SI::*;
    private import ScalarValues::*;
    part def Battery {
        attribute capacity : EnergyValue = 400 [W*h];
        attribute stateOfCharge : Real = 0.8;
        attribute usable : EnergyValue = capacity * stateOfCharge;
    }
    part def Station {
        part batteries : Battery[2];
        attribute total : EnergyValue = sum(batteries.usable);
        attribute reserve : EnergyValue = total - 15 [W] * 24 [h];
        constraint reservePositive { reserve > 0 [W*h] }
    }
}
EOF
perl "$KIT/tools/sysml/sysml.pl" check expressions.sysml
```

The English: “The usable energy of the station’s batteries shall exceed one day of a 15 W load.” The text form is exact about units and arithmetic; the English is what a reviewer checks against intent. Keep both: the sentence in the `doc`, the expression in the constraint.

Exercise 8.10 (p. 105): The battery requirement

```
cat > requirements.sysml <<'EOF'
package Reqs {
    private import ISQ::*;
    private import SI::*;
    part def PowerSystem { attribute socReportPeriod : DurationValue; }
    requirement def StateOfChargeReporting {
        doc /* The power system reports the battery state of charge at least once per
        reporting period. */
        subject power : PowerSystem;
        attribute reportingPeriod : DurationValue;
        require constraint { power.socReportPeriod <= reportingPeriod }
    }
    requirement tw_seg_003 : StateOfChargeReporting {
        attribute doorsId = "TW-SEG-003";
```

```
        attribute :>> reportingPeriod = 10 [min];
    }
}
EOF
perl "$KIT/tools/sysml/sysml.pl" check requirements.sysml
```

Exercise 8.11 (p. 106): Trace a new requirement

```
cat > trace.sysml <<'EOF'
package Trace {
    private import RequirementDerivation::*;
    requirement tw_stk_002 { doc /* Timely observations. */ }
    requirement tw_sys_013 { doc /* Each station keeps observations it could not send and
    sends them later. */ }
    part def DataLogger;
    part def Station { part logger : DataLogger; }
    satisfy tw_sys_013 by Station::logger;
    #derivation connection {
        end #original ::> tw_stk_002;
        end #derive ::> tw_sys_013;
    }
}
EOF
perl "$KIT/tools/sysml/sysml.pl" check trace.sysml
```

The trace job would list TW-SYS-013 as *unverified*: something satisfies it, nothing verifies it. On a merge request the ratchet fails only if that is a new gap compared with the target branch (Chapter 20, The pipeline, job by job (p. 302)); adding a new requirement without its verification case is exactly that, so write the case in the same merge request.

Exercise 8.12 (p. 107): Inspect the vessel access

```
cat > verification.sysml <<'EOF'
package Verification {
    private import VerificationCases::*;
    requirement tw_sys_012 {
        attribute doorsId = "TW-SYS-012";
        attribute verificationMethod = "Inspection";
        doc /* Every station shall be serviceable from the service vessel. */
    }
    part def ServiceVessel;
    verification def InspectVesselAccess {
        doc /* A crew inspects each station type from the service vessel. Method: inspection.
        */
        subject vessel : ServiceVessel;
        objective { verify tw_sys_012; }
        return verdict : VerdictKind;
    }
}
EOF
```

```
perl "$KIT/tools/sysml/sysml.pl" check verification.sysml
```

Record the method on the requirement (`verificationMethod`, as the converters write it from DOORS), because the VCRM job reads it from there (Chapter 44, Verification as a closed loop (p. 613)); the verification case's `doc` repeats it for the human reader.

Exercise 8.13 (p. 109): The Storm mode

```
cat > states.sysml <<'EOF'
package Modes {
    attribute def LowBattery;
    attribute def CrewOnSite;
    attribute def StormWarning;
    attribute def StormOver;
    state def BuoyModes {
        entry; then observing;
        state observing;
        state degraded;
        state maintenance;
        state storm;
        transition first observing accept LowBattery then degraded;
        transition first degraded accept CrewOnSite then maintenance;
        transition first maintenance then observing;
        transition to_storm first observing accept StormWarning then storm;
        transition from_storm first storm accept StormOver then observing;
    }
    part def MooredBuoy { exhibit state modes : BuoyModes; }
}
EOF
perl "$KIT/tools/sysml/sysml.pl" check states.sysml
```

The prime's drop 3 adds the same state with triggers named `stormWarning` and `calm`. Your names may differ; when the drop lands, use the prime's names so the next delta stays clean, and say so in the merge request.

Exercise 8.14 (p. 110): Join and finish

```
cat > actions.sysml <<'EOF'
package Cycle {
    item def ObservationRecord;
    item def Bulletin;
    action def SampleSensors { out record : ObservationRecord; }
    action def CheckQuality { in record : ObservationRecord; out checked : ObservationRecord;
    }
    action def ArchiveObservations { in record : ObservationRecord; }
    action def PublishBulletin { in record : ObservationRecord; out bulletin : Bulletin; }
    action def LogBulletin;
    action def ObserveCoast {
        out bulletin : Bulletin;
        action sample : SampleSensors;
```

```
        action check : CheckQuality;
        action store : ArchiveObservations;
        action publish : PublishBulletin;
        action logBulletin : LogBulletin;
        flow sample.record to check.record;
        first start then sample;
        first sample then check;
        first check then afterCheck;
        fork afterCheck;
        first afterCheck then store;
        first afterCheck then publish;
        flow check.checked to store.record;
        flow check.checked to publish.record;
        join done;
        first store then done;
        first publish then done;
        first done then logBulletin;
        flow publish.bulletin to ObserveCoast::bulletin;
    }
}
EOF
perl "$KIT/tools/sysml/sysml.pl" check actions.sysml
```

Exercise 8.15 (p. 111): Use the calc

```
cat > analysis.sysml <<'EOF'
package Analysis {
    private import ISQ::*;
    private import SI::*;
    private import ScalarValues::*;
    calc def DailyEnergy {
        in power : PowerValue;
        in hours : Real;
        return energy : EnergyValue = power * hours * (1 [h]);
    }
    part def PowerSystem {
        attribute generation : PowerValue;
        attribute load : PowerValue;
        calc dailyLoad : DailyEnergy { in power = PowerSystem::load; in hours = 24.0; }
        calc dailyGeneration : DailyEnergy { in power = PowerSystem::generation; in hours =
        6.0; }
        constraint balance { dailyGeneration.energy >= dailyLoad.energy }
    }
}
EOF
perl "$KIT/tools/sysml/sysml.pl" check analysis.sysml
```

A reviewer checks this form more easily: each calc has a name that says what it computes, and the constraint reads “generation covers load”. The one-line expression is shorter but hides the units of `sunHours` and `24`.

Exercise 8.16 (p. 112): Allocate the cycle

```
cat > allocations.sysml <<'EOF'
package Allocations {
    action def CheckQuality;
    action def ArchiveObservations;
    part def QualityControl;
    part def Archive;
    allocate CheckQuality to QualityControl;
    allocate ArchiveObservations to Archive;
}
EOF
perl "$KIT/tools/sysml/sysml.pl" check allocations.sysml
```

Allocate `PublishBulletin` to the publisher, not to the data center: allocate to the lowest element that actually performs the function, but no lower (Chapter 47, Keeping SysML v2 from becoming a spider web (p. 645)). An allocation to the whole data center says nothing a reviewer can check, and the IDEF0 plates show the publisher as the mechanism, which is what the forecaster expects.

Exercise 8.17 (p. 114): A marking

```
cat > metadata.sysml <<'EOF'
package Conventions {
    private import ScalarValues::*;
    metadata def Marking { attribute level : String; }
}
package Stations {
    private import Conventions::*;
    @Marking { level = "U"; }
    part def MooredBuoy;
}
EOF
perl "$KIT/tools/sysml/sysml.pl" check metadata.sysml
```

On a real program the value is decided by the original classification authority, through the security classification guide; the team applies it, and the ISSM reviews it. The model records the decision; it does not make it (Chapter 48, Security: protect the environment, engineer it into the system (p. 651)).

Exercise 8.18 (p. 114): Read a bulletin

```
cat > usecases.sysml <<'EOF'
package UseCases {
    part def ObservationNetwork;
    use case def QueryArchive { doc /* Look up observations by station and time. */ }
    use case def ReadBulletin {
        doc /* A mariner reads the latest bulletin. */
        subject network : ObservationNetwork;
```

```
        actor mariner;
    }
}
EOF
perl "$KIT/tools/sysml/sysml.pl" check usecases.sysml
```

No include: reading the latest bulletin never requires a query of the archive; a mariner who wants history performs a different use case. Including it would make every bulletin read depend on the archive.

Exercise 8.19 (p. 115): A hybrid

```
cat > variants.sysml <<'EOF'
package Variants {
    part def Modem;
    part def CellularModem :> Modem;
    part def SatelliteModem :> Modem;
    part def HybridModem :> Modem;
    part def TelemetryUnit {
        variation part modem : Modem {
            variant part cellular : CellularModem;
            variant part satellite : SatelliteModem;
            variant part hybrid : HybridModem;
        }
    }
    part shoreTelemetry : TelemetryUnit { part :>> modem = modem::cellular; }
    part buoyTelemetry : TelemetryUnit { part :>> modem = modem::satellite; }
}
EOF
perl "$KIT/tools/sysml/sysml.pl" check variants.sysml
```

The trade study needs the same attributes on every variant, with units: cost, power draw, data rate, coverage at sea, and availability. Put them on `Modem` so each variant must give a value; a variant without one cannot be scored.

Exercise 8.20 (p. 116): A test view

```
cat > views.sysml <<'EOF'
package Views {
    private import VerificationCases::*;
    requirement tw_sys_002;
    verification def TestStationObservations { objective { verify tw_sys_002; } }
    viewpoint def TestViewpoint {
        doc /* Which requirements each verification case covers, and by which method. */
    }
    view def CoverageView {
        satisfy requirement vp : TestViewpoint;
    }
    view coverage : CoverageView {
        expose TestStationObservations;
```

```
        expose tw_sys_002;
    }
}
EOF
perl "$KIT/tools/sysml/sysml.pl" check views.sysml
```

Expose the verification package and the requirements packages, and nothing structural: the test organization needs the cases, what they verify and the methods. That view is, in effect, the VCRM (Chapter 44, Verification as a closed loop (p. 613)).

Exercise 8.21 (p. 117): Find three types

```
LIB="$KIT/tools/sysml/vendor/sysml-v2-release/sysml.library/Domain Libraries/Quantities and
Units"
grep -h "attribute def LengthValue\|attribute def ElectricCurrentValue\|attribute def
CelsiusTemperatureValue" "$LIB"/*.sysml
grep -h "attribute <m> metre\|attribute <'°C'>" "$LIB/SI.sysml"
```

```
cat > library.sysml <<'EOF'
package Instruments {
    private import ISQ::*;
    private import SI::*;
    part def Thermometer {
        attribute minTemperature : CelsiusTemperatureValue = -40 ['°C'];
        attribute maxTemperature : CelsiusTemperatureValue = 60 ['°C'];
        attribute mastHeight : LengthValue = 4 [m];
        attribute supplyCurrent : ElectricCurrentValue = 0.02 [A];
    }
}
EOF
perl "$KIT/tools/sysml/sysml.pl" check library.sysml
mkdir -p lintlib/model && printf 'title = Lint demo\n' > lintlib/sysml.conf && cp
library.sysml lintlib/model/
perl "$KIT/tools/sysml/model.pl" --root lintlib lint
```

The degree Celsius symbol is quoted (`'°C'`) because it is not a plain name. A temperature *range* is two values, minimum and maximum; a single attribute called `range` would hide which is which.

Exercise 8.22 (p. 118): Review a file in five passes

```
grep -n "import" drop1-v2/datacenter/DataCenter.sysml
grep -rl "import DataCenter" drop1-v2 | sort
```

```
grep -c "// TODO" drop1-v2/datacenter/DataCenter.sysml
```

DataCenter imports `Interfaces` and `Stations`; `Stations` imports it back, which is the Stations and DataCenter cycle (Chapter 47, Keeping SysML v2 from becoming a spider web (p. 645)). Its five TODO lines are all level stereotypes. One `doc` is not a description of the element: the second `doc` on `QualityControl` is pasted from a briefing ("From the Ops Concept briefing, slide 22 ..."), and the `dependency from QualityControl to SensorSuite` is what closes the cycle.

To the prime, as a discrepancy report: the pasted briefing text and the dependency, because both are in the prime's model and come back with every drop. The government's to fix: the level tags (TODO lines the government closes in its own copy).

Exercise 8.23 (p. 121): Grow the mini model

The workbook's mini model, rebuilt from the workbook's own text so this answer stays in step with it:

```
awk '/^cat > mini\//{p=1} p{print} /^EOF$/{p=0}'
"$KIT/docs/src/primer/06b-sysml-v2-workbook.txt" > make-mini.sh
mkdir -p mini/model && sh make-mini.sh
cd mini && perl "$KIT/tools/sysml/model.pl" lint
```

Adding the state machine to `Network.sysml` as the exercise says gives `Network` ten members, and the outline check fails: two to nine children per parent. The state machine is behavior, so the better home is `Behavior.sysml`, with `Network` importing it:

```
perl -0pi -e 's/(    action def Sample)/    attribute def LowBattery;\n    attribute def
CrewOnSite;\n    state def BuoyModes {\n        entry; then observing;\n        state
observing;\n        state degraded;\n        state maintenance;\n        transition first
observing accept LowBattery then degraded;\n        transition first degraded accept
CrewOnSite then maintenance;\n        transition first maintenance then observing;\n
}\n$1/' model/Behavior.sysml
perl -0pi -e 's/(    private import Interfaces::\*;\n)/$1    private import Behavior::*;\n/;
s/part def MooredBuoy \{ \@L3Station; \}/part def MooredBuoy { \@L3Station; exhibit state
modes : BuoyModes; }/' model/Network.sysml
perl -0pi -e 's/    satisfy tw_sys_002 by StationSegment;/    requirement tw_sys_005 {\n
   attribute doorsId = "TW-SYS-005";\n        doc \/* The data center shall archive every
observation together with its quality flag. *\/\n    }\n    satisfy tw_sys_002 by
StationSegment;\n    satisfy tw_sys_005 by Archive;/' model/Requirements.sysml
perl -0pi -e 's/(    verification def TestDataFlow \{)/    verification def
DemonstrateArchive {\n        subject archive : Archive;\n        objective { verify
tw_sys_005; }\n    }\n$1/' model/Verification.sysml
perl "$KIT/tools/sysml/sysml.pl" check model/*.sysml
perl "$KIT/tools/sysml/model.pl" lint
perl "$KIT/tools/sysml/model.pl" check | tail -1
perl "$KIT/tools/sysml/model.pl" trace
```

Now break the trace by removing the new verification case:

```
perl -0pi -e 's/    verification def DemonstrateArchive \{\n        subject archive :
Archive;\n        objective \{ verify tw_sys_005; \}\n    \}\n//' model/Verification.sysml
perl "$KIT/tools/sysml/model.pl" trace
```

On a merge request the pipeline's trace job would fail: the target branch had no unverified requirement and this change adds one (Chapter 20, The pipeline, job by job (p. 302)). That is the ratchet: existing gaps are reported, new ones stop the merge.

Exercise 8.24 (p. 123): Your first TODO

The TODO line on `ObservationNetwork` reads `// TODO stereotype Tidewater_Profile:L1 Network on this element (xmi:id …) not converted`. It is replaced by the one line `@L1 Network;`, and the package gains `private import Conventions::*;`, where the `Conventions` package (in `model/library/conventions/`) defines the level tags.

O.3 Part II: week zero

Exercise 13.1 (p. 212): Size the tasks

A model answer for "write the interface section of the ICD for the observation link": stories "Observation link defined in the model" (port def, interface def, the item and its fields; 5 points) and "Observation link reviewed by both owners" (review, comments, merge; 2 points). Tasks: "write the item def and its fields" (2), "write the port and interface defs" (2), "connect it in the network" (1), "review with the data center owner" (1), "answer the review" (1).

The hardest split is usually the review: it waits on someone else. Rule 2 of Section 13.3.3, Splitting work that is too big (p. 200), *pull out the waiting*, is the one that helps: the review becomes its own task, so the modeling finishes on time even when the reviewer is busy.

Exercise 13.2 (p. 212): Split a story

Split by result, one construct per story, each ending in something that can be shown:

1. "BuoyModes state machine passes lint and appears on the model site" (3 points).
2. "DailyUpload interaction written and grammar-checked" (2 points).
3. "ObserveCoast fork, pins and object flows written; the IDEF0 plate shows them" (3 points).
4. "EnergyBalance constraint written with declared parameters" (3 points).

Eleven points become four stories of at most three, and each one's end state can be demonstrated: open the model site, show the plate, run the check.

Exercise 13.3 (p. 212): Compute a sprint's capacity

Sprint 4 has nine working days (Thanksgiving is a holiday). Prorate each standing capacity by the days each person is there:

Person	Standing (10 days)	Days present	Capacity
Jim	5	8 (holiday and Friday's leave)	$5 \times 8/10 = 4.0$
Bob	8	9	$8 \times 9/10 = 7.2$
Tom	8	9	$8 \times 9/10 = 7.2$
Team			**18.4**

Commit 85 to 95 percent of it: about 15.6 to 17.5 points. The team's real sprint 4 kickoff recorded the same capacity and committed seventeen:

docs/tidewater/sprints/04/kickoff.txt

```
Sprint 4 kickoff, 2026-11-16 09:00 (90 minutes)

Goal: Stations levels and Types closed, one cycle broken, gaps reported; the model site for
management
Capacity: 18.4 points (Jim 5, Bob 8, Tom 8 per full sprint, less holidays and leave)
Committed: 17.0 points in 7 stories (92% of capacity): engineering 12.5, platform 3.0 (17%),
learning 1.5 (8%)
```

Exercise 13.4 (p. 213): Write a decision record

A model record, in the template's shape:

```
# 0015. Model element names are UpperCamelCase for definitions, lowerCamelCase for usages

Status: accepted, 2026-11-04 (Jim)

## Context
Names in the converted model mix styles because the v1 model did. Reviews keep arguing about
it, and the
generated documents split names into words by case.

## Decision
Definitions (part def, port def, ...) use UpperCamelCase; usages use lowerCamelCase.
Converted files keep the
prime's names; the rule applies to government-written elements.

## Consequences
Reviews stop debating style. The prime's names may differ; we do not rename them, because
the next drop would
undo it.

## Revisit when
The prime adopts a naming rule, or a second team joins the model.
```

Must contain: the context in one paragraph, the decision in one sentence, the consequences including the cost, and a condition that would reopen it.

Exercise 13.5 (p. 213): Enforce a RACI cell

Three typical cells:

- "The interface lead approves every interface change": enforceable. A CODEOWNERS entry for the interfaces folder plus "code owner approval required" on `develop` (Chapter 15, Setting up GitLab (p. 226)).
- "Only the CM lead creates baselines": enforceable. A protected tag rule `baseline-*` with create:maintainer, and only the CM lead is a maintainer for that purpose.
- "The model lead is consulted on scope changes": not enforceable by a setting. It is a ceremony: the change goes on the board as a story, and refinement is where the lead is consulted (Chapter 24, The rhythm: stand-ups, poker, refinement, kickoff, demo, retro (p. 377)).

The pattern: approvals and permissions are settings; consultation and judgement are stories and meetings.

Exercise 9.1 (p. 142): Find a requirement

```
grep -c ',"Inspection",' drop1/tidewater.csv
grep -rl "TW-SYS-012" drop1-v2
```

Five requirements are verified by inspection (TW-SYS-010, TW-SYS-012, TW-SYS-015, TW-SEG-001 and TW-SEG-008), and TW-SYS-012 appears in the converted requirements package of the Cameo model.

Exercise 9.2 (p. 142): Count the debt by package

```
grep -c "// TODO" drop1-v2/*/*.sysml | sort -t: -k2 -n -r
```

Stations holds 29 of the 52, because it is the biggest package: most of the model's part definitions live there, and each one carries a level stereotype the converter could not carry (`L3 Station` down to `L6Component`). Most of the debt is that one mechanical kind, which is why it was closed first (Chapter 10, Vim for models (p. 143)).

Exercise 9.3 (p. 142): Break a checksum

```
cp -r drop1 drop1-broken
sed -i 's/Coastal coverage/Coastal coverag3/' drop1-broken/tidewater.csv
cd drop1-broken
grep 'tidewater\.' MANIFEST.txt | tr -d '\r' | sed 's/^ *//' | sha256sum -c 2>&1 || echo
"exit status $?"
```

`sha256sum -c` prints `FAILED` for the changed file and a warning that one checksum did not match. With a real delivery: stop, do not import, record it on the Prime Drop issue, and ask

the prime to send the drop again (Chapter 45, The prime's model changes: receive, assess, disposition (p. 619)).

Exercise 9.4 (p. 142): Make each mistake on purpose

```
sl 2>&1 || true
ls no-such-folder 2>&1 || true
git -C / status 2>&1 || true
```

Read each one aloud: "command not found" (the name is wrong, or the program is not installed), "No such file or directory" (the path is wrong; check where you are with `pwd`), "not a git repository" (you are outside the clone; `cd` into it). None of them changed anything.

Exercise 12.1 (p. 195): The practice afternoon

The recoveries, each as one command:

Mistake	Recovery
broke a file you had not committed	`git restore FILE`
staged a file by mistake	`git restore --staged FILE`
wrong commit message on the last commit	`git commit --amend -m "…"` (only before pushing)
a commit that should not have happened, already shared	`git revert COMMIT`
a merge that went wrong, still in progress	`git merge --abort`

The one to remember: on anything already pushed, use `revert`, never `amend` or `reset`. Appendix E, Rescue: when something goes wrong (p. 724) has each one played out.

Exercise 12.2 (p. 195): A conflict with three sides

Two conflicts, one per merge: the first branch conflicts with the change on `main`, and after you resolve it the second branch conflicts with the result. `git log --oneline --all --graph` shows two merge commits on `main`, each with two parents. Resolve each so the line says what you mean, not by keeping both versions side by side; with `merge.conflictstyle diff3` set, the middle section shows the original line, which tells you what each side changed.

Exercise 12.3 (p. 195): The auditor's questions

```
git blame -L 3,3 model.sysml        # who last changed line 3, and in which commit
git show COMMIT                     # what that commit changed
git log -1 --skip=3 --format=%s     # the message of the commit three before the latest
```

Exercise 12.4 (p. 195): A vendor branch of your own

```
git init -q -b main vendor-demo && cd vendor-demo
git config user.name Bob && git config user.email bob@example.invalid
printf 'package S {\n    part def Hull;\n    part def Mast;\n}\n' > S.sysml
git add . && git commit -qm "drop 1"
git branch vendor/prime
git switch -q -c develop
sed -i 's|part def Hull;|part def Hull { doc /* The floating body. */ }|' S.sysml && git
commit -qam "doc on Hull"
git switch -q vendor/prime
sed -i '/Hull/d' S.sysml && git commit -qam "drop 4: the prime deletes Hull"
git switch -q develop
git merge vendor/prime 2>&1 || true
cat S.sysml
```

A change on one side and a deletion on the other is a conflict: git cannot know whether the deletion or the new `doc` should win, so it shows the government's version against an empty prime side. The team accepts the prime's deletion, because the prime owns the structure, and checks first whether anything still uses `Hull`. If the `doc` mattered, it goes to the prime as a discrepancy report so it survives the next drop.

Exercise 12.5 (p. 195): Choose a strategy

For Tidewater the facts are: formal baselines (yes), others depending on specific versions (yes, the program office and the prime), released versions corrected (yes, a hotfix in sprint 24), publication (every change, through the model site). Those point to gitflow: `main` holds baselines, `develop` holds integration, hotfixes are possible.

A program with no formal baselines, one team and continuous publication points to GitLab flow. The decision record says which facts decided it, so the decision is revisited when the facts change.

Exercise 10.1 (p. 156): The debt as a jump list

Grouped by kind, drop 1's fifty-two TODO lines are:

```
grep -rh "// TODO" drop1-v2 | perl -ne '
  if (/stereotype Tidewater_Profile:L\d/) { $k = "level stereotype" }
  elsif (/Procured/) { $k = "procured part" }
  elsif (/InstanceSpecification/) { $k = "unit" }
  elsif (/ForkNode|InputPin|ObjectFlow|ControlFlow/) { $k = "activity construct" }
  else { $k = "other" }
  $c{$k}++; END { print "$c{$_} $_\n" for sort { $c{$b} <=> $c{$a} || $a cmp $b } keys %c }'
```

Thirty-six level stereotypes, four activity constructs, three procured parts, three units and six others (the state machine, the interaction, the include, the constraint expression twice, the profile). Close the level stereotypes first: they are mechanical, one substitution per package, and they move the burn-down the most.

Exercise 10.2 (p. 156): Close the level tags in one package

In Stations, the level tags are 25 of the 29 TODO lines; after the substitution four remain:

```
grep "// TODO" drop1-v2/stations/Stations.sysml | grep -v "Tidewater_Profile:L[0-9]" | sed
's/(xmi:id.*//' | perl -pe 's/^\s+\/\/ TODO //; s/ on this element//; s/\s+$/\n/' | sort |
uniq -c
```

Three procured parts (metadata to write) and the BuoyModes state machine (a design task for later).

Exercise 10.3 (p. 156): Break it three ways

- **Misspelled type:** the grammar check passes; lint reports `unresolved type` on the line of the usage.
- **Missing semicolon:** the grammar check reports it, on the *next* line, where the following statement starts.
- **Level tag on the wrong level:** the grammar check and lint both pass; `model.pl check` reports a level-tag mismatch on the part def. A misspelled tag name such as `@L1 Netwrk;` passes all three, which is why the practice lesson's own check exists (Chapter 10, Vim for models (p. 143)).

Exercise 10.4 (p. 156): Resolve with the ancestor

For a single conflict, the marker search (`/^<<<<<<<` then `]c`-style editing) is faster: you see the three versions in place and edit one line. For ten conflicts, `git mergetool` with Vim's three-way diff is better, because it walks you from conflict to conflict and shows the base beside each side. Either way, the `diff3` middle section (the ancestor) is what tells you who changed what.

Exercise 10.5 (p. 157): Write a merge request description in Vim

A good description says what changed (the Stations level tags, 25 TODO lines closed), why (the structure epic, story id), how it was checked (`\mx` and `\ml` clean, the outline count unchanged), and what the reviewer should look at (that each tag matches the part's level in the tree). With `git diff --stat` inserted, a reviewer can approve from the description and the diff: the change is mechanical and the checks prove the mechanics; the reviewer checks the levels.

Exercise 14.1 (p. 219): Rewrite three criteria

- (a) "The `todo` job reports zero TODO lines in `model/logical/`, and the IDEF0 plates regenerate without warnings."
- (b) "The trace job reports zero unverified segment requirements, or each one has an open discrepancy report linked from this story."
- (c) "A Discrepancy Report issue exists for each finding, labelled `owner::prime`, and

the transmittal memo is recorded on it."

Each names a job, a number or a record someone can check.

Exercise 14.2 (p. 225): Split a story

By requirement group and by investigation versus result (Section 14.8, Splitting patterns for model work (p. 219)):

1. "Stakeholder needs in the model" (the DOORS stakeholder module converted and merged): the `grammar` and `lint` jobs pass and the trace links all stakeholder needs.
2. "System requirements in the model": the trace job shows every system requirement linked.
3. "Segment requirements in the model": the same for the segment module.
4. "Drift and gaps reported" (investigation): one Discrepancy Report issue per drift or gap the trace lists.

Exercise 14.3 (p. 225): Budget your own learning

Sprint 1, about a fifth of capacity: "Learn the tools: survival kit, git, Vim" (2 points each; done when practice lessons 10 and 11 check as done), "Laptops ready" (1 point each, plus the software request's wait; done when `kickoff.pl doctor` passes). Sprint 2, about a sixth: "Learn the pipeline and a prime drop" (2 points each; done when lessons 12 and 13 check as done and each person has merged one real change). Label them `work::learning` so the reserve is visible (Chapter 14, Writing stories and tasks for model work (p. 214), Bottom line up front (p. iii)).

Exercise 11.1 (p. 169): Find a cycle

The Stations and DataCenter cycle is closed by two lines in the prime's files: `dependency from QualityControl to SensorSuite` in DataCenter, and `dependency from DataLogger to IngestService` in Stations (plus the imports they need). The shared element is the observation record, which already belongs in the library's interfaces; with both segments depending only on the library, the cycle disappears. Both dependencies are in the prime's model, so the fix is a discrepancy report, not a government edit (Chapter 29, Sprint 4: the model site, a cycle broken, and the demo that landed (p. 459)).

Exercise 11.2 (p. 170): Read a plate

For the Check step, from the plates of the converted model:

docs/tidewater/views/tidewater-idef0.md

```
a# Check Quality
  ## Range, step and consistency checks; set the flag and the reason.
  i# Ingest Observation Record
  o# Check Observation Record
  m# Quality Control
```

Input: the ingested observation record. Output: the checked observation record. Mechanism: Quality Control, supplied by `allocate CheckQuality to QualityControl;` in the allocations package.

Exercise 15.1 (p. 243): Build the project

```
gl() { perl "$KIT/bin/gitlab.pl" --local gl --project tidewater/model "$@"; }
for u in jim bob tom; do gl user $u > /dev/null; done
gl --as tom setup "$KIT/tools/gitlab/templates/project.conf" > setup.txt
echo "labels: $(grep -c '^label ' setup.txt)"
echo "milestones: $(grep -c '^milestone ' setup.txt)"
```

Setup created seventy-two labels (the `[labels]` section, one line each) and thirty-nine milestones (the one `sprints = …` line in `[milestones]`, which expands to a milestone per sprint).

Exercise 15.2 (p. 243): Make it refuse you

```
G="$PWD/gl"
gl() { perl "$KIT/bin/gitlab.pl" --local "$G" --project tidewater/model "$@"; }
git clone -q gl/repos/tidewater/model.git refuse-demo 2> /dev/null && cd refuse-demo
git config user.name Tom && git config user.email tom@example.invalid
git switch -q -c main && echo "Tidewater" > README.md && git add . && git commit -qm "first
commit"
GITLAB_USER_LOGIN=tom git push -q origin main
gl --as tom flow init > /dev/null && git fetch -q && git switch -q develop
echo "a direct change" >> README.md && git commit -qam "direct"
GITLAB_USER_LOGIN=bob git push origin develop 2>&1 | grep -i "not allowed\|->" || true
GITLAB_USER_LOGIN=bob git push origin develop:main 2>&1 | grep -i "not allowed\|->" || true
gl --as bob tag baseline-0 main 2>&1 || true
```

Each refusal comes from one line of `project.conf`: `develop = push:none …` and `main = push:none …` in `[branches]` (nobody pushes; changes arrive by merge request), and `baseline-* = create:maintainer` in `[tags]` (Bob is a developer, so cannot create a baseline).

Exercise 15.3 (p. 244): Redraw the signature page

A specification's signature page usually reads: prepared by (author), reviewed by (subsystem leads), approved by (chief engineer), CM release (CM lead). As a CODEOWNERS file over a folder of text files:

```
[Systems lead]
/model/requirements/   @jim
[Interfaces] @jim
/model/library/Interfaces.sysml
[Segments]
/model/physical/stations/     @bob
```

```
/model/physical/datacenter/   @bob @tom
```

The CM release is not a code owner: it is the protected `baseline-*` tag. Signatures with no clear owner are usually "reviewed by" lines that list a role nobody holds; those become an open question for the lead.

Exercise 15.4 (p. 244): Defend a branching strategy

GitLab flow: `main` is the integration branch and the published model, releases are tags on `main`, and there is no `develop`. In `project.conf`: set `default_branch = main`, remove the `develop` line from `[branches]` (and make `main` `merge:developer`), and drop the `vendor/prime` line, since there are no drops. Monthly versions become tags without a release branch.

Exercise 16.1 (p. 253): Place a new package

With Tidewater's map, an unlisted package lands under `unsorted/` (the map's `* = unsorted/` line), and the converter says so in its report: "layout map: Drifters is not listed; written to unsorted/...". The `delta` job's report on the drop merge request shows the same. In one merge request, before anyone edits it: add the package to the layout map in the right layer, move the file there on the vendor branch by re-importing the drop with the new map, and add the folder to CODEOWNERS (and to `layers` in `sysml.conf` if it is a new layer).

Exercise 16.2 (p. 253): Spot the upward import

It breaks the layering rule: requirements are a lower layer than the physical architecture, and lower layers never import higher ones. It is a smell because a requirement should not depend on a design; that is how the spider web starts. The file is generated from the prime's export, so do not hand-edit it: the complexity job reports upward imports in generated files instead of failing on them, and the fix goes to the prime (or to the converter's layout, if the import is the converter's own) as a discrepancy report (Chapter 47, Keeping SysML v2 from becoming a spider web (p. 645)).

Exercise 17.1 (p. 268): Apply the rule

A model answer:

Problem today	GitLab tool	The problem it creates if turned on too early
nobody knows which version of the ICD is current	protected tags and releases	ceremony around every small change before anyone has felt the version problem
reviews happen in e-mail threads nobody can find	merge requests with approvals	approvals queue up behind one reviewer if code owners are drawn too narrowly
status slides take a day each week	the model site on Pages, analytics	a dashboard nobody asked for, numbers nobody trusts yet

The rule in Chapter 17, The DevSecOps tools that earn their keep (p. 255): a tool appears when the team has felt the pain it removes.

Exercise 17.2 (p. 268): Read a widget

The JUnit report (`lint.xml`) has one `testcase` per check with a `failure` element carrying the message; GitLab shows that message in the merge request's test widget as one failed test. The Code Quality report (`lint-cq.json`) has one finding with `description`, `location.path` and `location.lines.begin`; GitLab shows it in the code quality widget and on the diff line. The line to look for in both: the `unresolved type` message with its file and line.

Exercise 17.3 (p. 268): Choose the token

- (a) A personal access token with `read_api`, owned by the person who runs the script, with an expiry date.
- (b) No stored token at all: the pipeline's `CI_JOB_TOKEN`, which exists only while the job runs.
- (c) A project access token with `read_api` only, stored as a masked, protected CI/CD variable, with an owner and an expiry in the token inventory (Chapter 42, Administration (p. 586)).

Exercise 17.4 (p. 268): Argue for one left-off tool

For merge trains: "Twice this month two merge requests were green alone and red together on `develop`, and `develop` was red for half a day each time. A merge train tests each merge request on top of the ones queued before it, so `develop` stays green." The evidence needed: the count of red `develop` pipelines caused by two green merge requests, from analytics (Chapter 32, Measuring with GitLab analytics (p. 513)). Without that evidence, Tom is right to leave it off.

Exercise 18.1 (p. 286): Write the approval request

A complete request names one runtime (Ollama, a specific version), one model family and size from an allowed developer, where it runs (three laptops, or one approved server), the data rule ("only information our marking rules allow on that machine"), and that the model's origin and license were checked. The office that approves model origin is usually the one that approves software intake, with the ISSM concurring; find out by asking both, in writing.

Exercise 18.2 (p. 286): Run a three-task bake-off

The answer is your scoresheet. A good one has, per task: the prompt written in advance, the number of tries, whether the result passed the gates (grammar, lint, the recipe for text), and the minutes to a mergeable result, with and without the model. A typical finding: the model helps on the helper script and the discrepancy report text, and loses on SysML v2 text, where its first draft fails the gates.

Exercise 18.3 (p. 286): Make the gates catch a hallucination

The grammar check passes: invented names such as `ISQ::SolarPowerValue` are perfectly good grammar. Lint catches them, as `unresolved type` errors on the lines that use them, because lint resolves names against the model and the library and the grammar check does not. With the model's own `Power` and `Energy` types both gates pass. Section 18.10.6, 6. SysML v2 text: the EnergyBalance constraint (p. 283) shows the real runs.

Exercise 18.4 (p. 286): The scratch-buffer loop in Vim

With `cat` standing in for the model, the loop is: select the TODO line and the XMI fragment, `:'<,'>w !cat > /tmp/prompt`, `:new`, `:r !cat /tmp/prompt`, `:w answer.sysml`, `\mx`. Most people time it at a minute or two once the keys are familiar; the time is in reading the answer, not in the keys.

Exercise 18.5 (p. 287): Check a draft for added facts

Typical added facts: a date the defect was found, a test that "failed", a section number in a document the prompt never named, and a severity. The one that would embarrass you in front of the prime is any invented reference, such as "per SRS paragraph 3.2.1": the prime will look it up. Delete every added fact or verify it before sending.

Exercise 20.1 (p. 346): Which jobs, which event

Running `gitlab.pl ci --list` for each event reproduces the table in Chapter 20, The pipeline, job by job (p. 302). The rule that usually surprises people: a job with no `rules:` does not join merge request pipelines at all, on GitLab and locally. Every check job in the template has an explicit merge-request rule for that reason.

Exercise 20.2 (p. 346): Turn the ratchet

After fixing one violation the `outline` job passes and says "fewer violations than allowed: lower .ci/outline-max to N in this MR". Lower the number in `.ci/outline-max` in the same merge request and run it again: it passes with the new allowance, and nobody can add a violation back without the job failing.

Exercise 20.3 (p. 346): Find your ratchet

Example: "every requirement has a verification method". Counted: requirements with an empty method in the DOORS export. The allowed number lives in a one-line file in the repository, like `.ci/outline-max`. Anyone may lower it in a merge request that fixes some; only the lead may raise it, and only with a recorded reason, enforced by a CODEOWNERS line for that file.

Exercise 20.4 (p. 346): Stale documents

The `docs` job fails: it regenerates the documents and finds the committed ones differ ("docs are stale"). Fix it by running `model.pl docs` and committing the result. The team commits generated files so a reviewer sees, in the merge request diff, what the change does to the documents people read, and so a baseline's checksum covers the documents as well as the model.

Exercise 20.5 (p. 346): Review a pipeline change

"Request changes. `allow_failure` on lint means a model with unresolved names can merge, and nobody will notice until the next release. If lint is noisy, that is information: which messages are noise? Let's fix those (or ratchet the count, as we do for the outline) and keep lint blocking." The alternative: a ratchet on the lint warning count, with errors still blocking.

Exercise 19.1 (p. 300): Break it and watch it fail

The `lint` job fails and its log names the file and line with `unresolved type 'NAME'`. In Vim, `:make lint` fills the quickfix list from the same message, `:cnext` jumps to the line, the fix is the correct type name, and the job passes on the next run.

Exercise 19.2 (p. 301): One line up

The error names the line *after* the one missing the semicolon (the workbook answer above shows it: `m1.sysml:3:5`, when the semicolon was missing on line 2). In one sentence: "The checker only notices a statement did not end when the next one starts, so it points at the start of the next statement."

Exercise 19.3 (p. 301): Add a job

```
todo:
  stage: check
  rules:
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'
    - if: '$CI_COMMIT_BRANCH'
  script:
    - perl -e 'my $n = 0; for my $f (@ARGV) { open my $h, "<", $f or next; $n += grep { m{//
    TODO} } <$h> } print "todo_lines $n\n"' model/*/*.sysml
```

To show the count on the merge request page, write it to a metrics file and declare it as `artifacts: reports: metrics:`; GitLab then shows it in the metrics widget, with the change since the target branch (Chapter 17, The DevSecOps tools that earn their keep (p. 255)).

Exercise 19.4 (p. 301): Allowed to fail

The pipeline becomes "passed with warnings" and the merge request can merge. That is a bad idea for lint because lint errors are real defects (unresolved names), and a warning icon is easy to ignore; a broken model would reach `develop`. The team first made `trace` allowed to fail because its gaps were the prime's, inherited and outside the team's control, so failing on them would block every merge. The ratchet replaced it because it separates what the team inherited (reported) from what a change adds (blocked).

Exercise 19.5 (p. 301): The manager's question

On a merge request: "The automated checks found a problem in a proposed change before it was accepted. Nothing in the model is affected. The author is fixing it today."

On the integration branch: "The accepted model currently fails a check, so every new change is on hold. We know the cause and expect it green by this afternoon. The last good baseline is unaffected."

Exercise 21.1 (p. 355): Make a job pend

GitLab shows the job as *pending* with "This job is stuck because you don't have any active runners online or available with any of these tags assigned to them: does-not-exist". Locally, `gitlab.pl ci --list` still lists the job (the local runner ignores tags), which is itself the lesson: tags are a property of the server's runners. Fix it by removing the tag, or by registering (or editing) a runner with that tag.

Exercise 21.2 (p. 356): Translate a line

In PowerShell: `if (-not $env:KIT) { Write-Output "KIT is not set"; exit 1 }`. The team chose `perl ci/job.pl NAME` because a Perl command line is the same in PowerShell, cmd, Git Bash and Docker, so one pipeline file works on every runner, and the logic lives in one script that has its own tests instead of two shell dialects to keep in step.

Exercise 21.3 (p. 356): Write the runner's security description

One page, in this order: the machine (a dedicated Windows VM, its name and network segment), the account (a service account with no interactive login and no administrator rights), what it can reach (the GitLab server only, outbound), the credentials on it (the runner token, stored by the runner service; no personal tokens), what runs on it (the model project's pipeline jobs only, through a project runner with tags), who maintains it (Tom, patched monthly with the runner version kept in step with GitLab's), and how it is cleaned (build folders removed after each job). Section 21.8, The service account: least privilege (p. 353) is the model.

Exercise 22.1 (p. 366): Review a change

The pipeline catches the removed import: lint fails with unresolved names (`Observation` `IF`, `HealthIF`) on the port lines. Only a reviewer catches the `doc` change, because it is valid text with a different meaning; the generated documents in the diff make it visible, and the review checklist asks whether each changed `doc` still says what the element is (Chapter 22, Reviewing a model change (p. 357)).

Exercise 22.2 (p. 366): Rewrite a finding

- (a) "This satisfies TW-SYS-003 with the archive, but the requirement is about ingest. Should it be `IngestService`?"
- (b) "TW-SYS-013 has no verification case after this change; the trace job will flag it. Can you add the case here, or link the story that will?"
- (c) "The package has two cycles and a duplicated block (see the pkg view). Could we take the cycle in this merge request and open a story for the duplicate?"

Each says what, where, why it matters, and asks for a specific action.

Exercise 23.1 (p. 376): Make a journal

```
mkdir -p journal && cd journal
perl "$KIT/bin/daily.pl" init . > /dev/null
perl -pi -e 's/^default_team\s*=\s*/default_team  = Model   /' scrum.conf
for who in Jim Bob Tom; do perl "$KIT/bin/daily.pl" roster add "$who" $(echo $who | tr A-Z
a-z)@example.invalid Model Engineer Gov > /dev/null; done
cat > standups/2026-10-05.txt <<'PLAN'
2026-10-05
sprint 1
== Model
cap 19
new TW-1 4 Learn the tools o:Jim
new TW-6 3 Convert drop 1 and assess it o:Bob
new TW-54 2 Runner service account and hardening checklist o:Tom
commit TW-1 Jim
commit TW-6 Bob
commit TW-54 Tom
PLAN
perl "$KIT/bin/daily.pl" compile
perl "$KIT/bin/daily.pl" sprint 1
```

Exercise 23.2 (p. 376): Break the compile

```
before=$(sha256sum scrum.txt)
printf '2026-10-06\nsprint 1\n== Model\ndone XX-99\n' > standups/2026-10-06.txt
perl "$KIT/bin/daily.pl" compile 2>&1 || true
[ "$before" = "$(sha256sum scrum.txt)" ] && echo "journal unchanged"
rm standups/2026-10-06.txt
```

The kit names the unknown task and writes nothing: a stand-up file with any error is rejected whole, never half applied.

Exercise 23.3 (p. 376): Where does it go?

- (a) GitLab: a task, under one week, closed by a merge request.
- (b) The journal *and* GitLab: a story in the journal (points, sprint), its tasks in GitLab.
- (c) The journal: an epic, as the `e:` tag on its stories.
- (d) Neither: hours stay in the team's own time records (GitLab's `/spend` if the team uses it); the journal never holds hours.
- (e) The journal (a `block` line on the story, so the quad shows it) and GitLab (the `workflow::blocked` label on the task, with the reason in a comment).

Exercise 23.4 (p. 376): Re-estimate

```
printf '2026-10-07\nsprint 1\n== Model\nest TW-6 5\n' > standups/2026-10-07.txt
perl "$KIT/bin/daily.pl" compile > /dev/null
grep -A3 "2026-10-07 Standup" scrum.txt
```

The kit posts the difference (two more points) instead of replacing the three, because the journal is a ledger: history is never rewritten, so the original estimate and the change both stay visible, and velocity can be computed honestly afterwards.

Exercise 24.1 (p. 388): Lint your own status

```
cat > standups/2026-10-06-chat.txt <<'CHAT'
[9:01 AM] Tom
Y TW-54 sent the hardening checklist T TW-54 wait for the ISSM B
CHAT
perl "$KIT/bin/daily.pl" --today 2026-10-06 lint standups/2026-10-06-chat.txt || echo "exit
status $?"
perl "$KIT/bin/daily.pl" --today 2026-10-06 lint standups/2026-10-06-chat.txt --reply ||
true
```

```
cat > standups/2026-10-06-chat.txt <<'CHAT'
[9:01 AM] Tom
Y TW-54 sent the hardening checklist to the ISSM T TW-54 answer the ISSM's questions B TW-54
waiting on the ISSM's signature, no date -- need: Jim to ask for a date today
CHAT
perl "$KIT/bin/daily.pl" --today 2026-10-06 lint standups/2026-10-06-chat.txt
```

Exercise 24.2 (p. 388): Play a round

The tally shows the spread and the outlier for each round. The spread usually comes from one person knowing something the others do not (Bob's high vote on TW-3 in sprint 1 came from experience of the export). Discussion should change the number when it surfaces that knowledge; if it does not, take the higher of the converged votes and note the risk on the story.

Exercise 24.3 (p. 389): Apply the Definition of Ready

The first item that usually fails is "has acceptance criteria a pipeline job or a record can check". Making it ready means writing those criteria (as in the stories exercises above), sizing it under a sprint, and naming the owner. If a dependency outside the team exists, it becomes its own task first.

Exercise 24.4 (p. 389): Plan a demo without slides

Twenty minutes: the question the two weeks answered (two minutes), the model site's front page with the four headline numbers (three), one merge request from red to green, opened live (five), one before-and-after with a number (three), one safe live change (five), and the ask with a date (two). The question for the audience: "Is this the evidence you need for the next milestone, and if not, what is missing?"

Exercise 25.1 (p. 401): Rewrite five statuses

For the three at the start of Chapter 25, Talking about progress: helps for engineers who would rather not (p. 390), outcome first with one number:

- "Y TW-12 Stations level tags done: 9 TODO lines closed, 43 left. T TW-15 decide who owns ObservationRecord, by Thursday. B none"
- "Y TW-22 the trace found 2 requirements nothing satisfies; they are the prime's, report goes Friday. T TW-22 draft the report. B none"
- "Y TW-3 runner request sent. T TW-4 pipeline draft. B TW-3 waiting on IT approval of the runner machine – need: Jim to escalate today"

The hardest is usually the blocker: it must name what you need and from whom, which feels like complaining. It is not; it is the one sentence management can act on.

Exercise 25.2 (p. 401): Climb the ladder

For "the model site is live": one sentence, "The program office can now see the current model, its checks and its numbers on a web page that updates on every change." Thirty seconds: add what is on the page and the one number that moved. Five minutes: open the page, show the part tree, the trace page and the burn-down, and say what it replaces. If the colleague cannot repeat the one sentence, it had a mechanism in it ("Pages", "pipeline"); take it out.

Exercise 25.3 (p. 402): Your stakeholder map

For Tidewater: the program manager (schedule and risk; the one number is TODO lines left against the plan), the deputy program manager (change control; the open prime changes), the ISSM (security; open security findings), the prime's modeler (the drop loop; open discrepancy reports), the test organization (verification; requirements with a verification case). The pipeline produces all of these except the program manager's plan comparison, which comes from the journal.

Exercise 25.4 (p. 402): Draft from GitLab

`talk.pl standup --as bob` drafts the closed task as done and the next open task as today, with a number. The edit that matters turns "closed #N" into what it means: "Stations tags done: the structure epic is 60% through, and the model site shows the new tree."

Exercise 25.5 (p. 402): Run a dry run

The narrator usually stumbles at the transitions: from the question to the first screen, and from the live change to the ask. Fix the script by writing the transition sentence into it ("That is what changed; here is what it means for the review"), and by moving any screen that needs explanation before it makes sense to the end or out.

O.4 Part III: twelve weeks

Exercise 26.1 (p. 422): The first status

Bob's Tuesday lines become one line per question, each naming the story (TW-6, *Convert drop 1 and assess it*). On the kickoff Monday nothing came before the sprint, so the honest yesterday is `none`. In the scratch journal from the journal exercises above:

```
cat > standups/2026-10-05-chat.txt <<'CHAT'
[9:02 AM] Bob
Y: none
T: TW-6 run both converters on drop 1 with --report
B: none
CHAT
cat > standups/2026-10-06-chat.txt <<'CHAT'
[9:02 AM] Bob
Y: TW-6 read the converter reports
T: TW-6 TODO inventory by package
B: none
CHAT
perl "$KIT/bin/daily.pl" --today 2026-10-05 lint standups/2026-10-05-chat.txt
perl "$KIT/bin/daily.pl" --today 2026-10-06 lint standups/2026-10-06-chat.txt
```

Exercise 26.2 (p. 422): Read the drop yourself

```
perl "$KIT/bin/xmi2sysml.pl" --out s1/model --layout-map
"$KIT/examples/tidewater-v1/layout.map" drops/v1.0/tidewater.xmi > /dev/null
grep -n "TODO.*BuoyModes" s1/model/physical/stations/Stations.sysml
grep -o '<[^>]*xmi:id="_22_1_7f3a0c1_7a13eb20b93a"[^>]*>' drops/v1.0/tidewater.xmi
```

The id points at the `uml:StateMachine` named BuoyModes, owned as a behavior in the v1 model. In Vim, `/TODO.*BuoyModes` finds the line, and the id is what you search for in the XMI (or quote in a discrepancy report) to find the original.

Exercise 26.3 (p. 422): A spread worth having

The rounds went 2, 5, 3 and then 3, 3, 3:

docs/tidewater/sprints/01/poker-2026-10-05.txt

```
ESTIMATE TW-3 Create the GitLab group and model project  (3 answers at 10:07)
  Bob            5
  Jim            2
  Tom            3
  no consensus: median 3, mode 2, range 2-5

ESTIMATE TW-3 Create the GitLab group and model project  (3 answers at 10:09)
  Bob            3
  Jim            3
  Tom            3
  consensus: 3
```

Bob's sentence, a model answer: "Creating the project is an hour; proving the protections refuse a push, and getting the runner requested, is the rest." The note on the story: "Includes proving the branch protection refuses a direct push (task #15) and the runner request; the runner's approval is outside the team." Both moved into the plan: task #15 is "Prove a direct push to develop is refused".

Exercise 26.4 (p. 422): Your week zero

Tidewater's own record, for comparison:

```
tail -n +4 sprints/00/week-zero.txt
```

A model answer lists the same kinds of hours: the directive meeting, the kickoff with management, software requests, the repository skeleton, the first security conversation, the decisions, the practice repository, and the delivery agreement with the prime. The ones people are tempted to hide in sprint 1 are the waits and the security meeting, because they produce nothing on a board. Do not: hidden hours make sprint 1's velocity look worse than it was and the next team's plan too optimistic, and the waits are exactly what management can

shorten if it sees them.

Exercise 27.1 (p. 438): Translate the failure

What a systems engineer needs: "The runner reads each job's script as PowerShell, not as the Bash we tested on our laptops, so `$(…)` and `||` mean nothing to it and the job dies on line one." What Tom changed: "Each job's script became one line, `perl ci/job.pl NAME`, which reads the same in every shell, with the logic moved into a Perl script in the repository."

Exercise 27.2 (p. 438): Re-run the demo

A five-minute model script:

1. **The question** (30 seconds): "Can the program trust that every change to the model is checked before it is accepted?"
2. **The output** (1 minute): the merge request list, six of six changes merged with green checks, and the model now on `develop`.
3. **One before-and-after** (1 minute): "Review turnaround is 3.3 hours, median. Before, a model change waited for the next review board."
4. **One safe live change** (2 minutes): on a scratch branch, misspell one type; the pipeline goes red with the file and line; fix it; green.
5. **The ask** (30 seconds): "We need the runner machine approved by 13 November, or the checks stay on our laptops."

Exercise 27.3 (p. 438): Read the integration branch

```
git -C work/bob log --oneline --first-parent --before=2026-10-31 origin/develop
```

The merges, oldest first (the log lists them newest first; your commit ids will differ, the branch names will not):

Merge	Story	Kind
feature/16	TW-3, CODEOWNERS and templates	machinery
feature/37	TW-4, the first pipeline	machinery
feature/43	the government conventions package	model (library)
feature/44	level tags in `sysml.conf`	machinery
feature/46	decision records 0001 to 0014	records
feature/drop-v1.0	drop 1 import	**the model**
feature/42	the `todo` burn-down job	machinery

The drop-v1.0 merge brought the model in. CODEOWNERS, the pipeline, the level tags and the burn-down job changed only the machinery; the conventions package is the team's first model text, and the decision records are documentation.

Exercise 27.4 (p. 438): A layout for your model

A model answer for a typical model: the library gets value types, units, port and interface definitions and the conventions; requirements get the DOORS modules; logical gets the functional architecture and the activities; physical gets one folder per segment; analysis gets constraints and trade studies; verification gets the cases. The least sure package is usually the one that mixes interfaces and structure (Tidewater's `Interfaces`). After the third drop, moving it costs one layout-map line, one re-import on the vendor branch, a CODEOWNERS change, and a merge that touches every file importing it; the converter's `--delta` shows how many.

Exercise 28.1 (p. 457): Make the grammar check say no

The lesson's checks say when it is done (`perl agile.pl practice check 11`). A model stand-up line for the next morning: "Y TW-61 Stations port merged after a red grammar check: 1 syntax error found by the pipeline, 0 by the reviewer. T TW-62 ... B none".

Exercise 28.2 (p. 457): Read the trace report

```
perl "$KIT/bin/gitlab.pl" --local gitlab --project tidewater/model pipeline show 29 | grep
trace
grep "^orphans\|^unverified\|^drift" gitlab/pipelines/tidewater-model/29/trace.log
```

- **TW-SYS-012, orphan:** the prime's. Nothing in the prime's model satisfies it; adding a `satisfy` would put a design decision in a file the next drop overwrites.
- **TW-SYS-016, orphan:** the prime's, for the same reason; it is also unverified, so one report covers both gaps.
- **TW-SYS-006, drift:** the prime's. The model's text no longer matches DOORS, and DOORS is the prime's master, so the prime decides which wording is right.

All three become discrepancy reports, not government edits.

Exercise 28.3 (p. 457): Rewrite Tom's stand-up

On 9 November the wait had already started; the rewrite names it in the B line with what Tom needs from whom:

```
cat > standups/2026-10-07-chat.txt <<'CHAT'
[9:03 AM] Tom
Y: TW-54 hardening checklist sent to the ISSM
T: TW-54 answer the ISSM's questions; the runner request is ready to submit
B: TW-54 runner request waiting on the ISSM's review, no date -- need: Jim to ask the ISSM
for a date today
CHAT
perl "$KIT/bin/daily.pl" --today 2026-10-07 lint standups/2026-10-07-chat.txt
```

(The scratch journal's dates differ from the story; the lint checks the form, which is what

matters here.) A blocker in the B line goes into the quad's watch items the same day, so Jim would have seen it on the first morning, not the third.

Exercise 29.1 (p. 474): Find a cycle

```
perl "$KIT/tools/sysml/model.pl" draw pkg -o ../../pkg.svg 2>&1 | grep "cycle"
```

The remaining Stations and DataCenter cycle is made by two dependencies: `DataLogger` to `IngestService` in Stations and `QualityControl` to `SensorSuite` in DataCenter. Remove one by moving the element both sides share (the observation record) into the library, so both segments depend downward. That is a design decision because it decides which segment owns the data's definition and changes the interface both owners review; it also touches the prime's files, so it goes through a discrepancy report.

Exercise 29.2 (p. 474): Write a discrepancy report

For the six unverified stakeholder needs (TW-STK-001 to TW-STK-006):

```
# DR: stakeholder needs have no verification

## What we found
Drop v1.1: TW-STK-001 to TW-STK-006 have no verification case (trace job, pipeline #29).

## Why it matters
Without a case, the VCRM cannot show the needs met at acceptance.

## What we ask
Add a case per need, or state that each need is verified through the system requirements
derived from it, by DOORS id.

## Evidence
trace.log of pipeline #29; v1 ids of the six requirements.
```

Open it on the emulator with `gitlab.pl issue new "Stakeholder needs have no verification" --template "Discrepancy Report" --label owner::prime`.

Exercise 29.3 (p. 474): Rehearse a demo

The draft opens with a generic question and lists four merge requests. A rewrite: open with "Can the program office read the model without Cameo?", show the model site first (the part tree and the trace page), give one before-and-after ("TODO lines: 50 to 22 this sprint"), make the safe live change already in the draft (delete one `satisfy` on a scratch branch and watch the trace name the requirement), and close with an ask and a date: "the tier decision by 11 December, so the merge-request approvals can be enforced before the baseline."

Exercise 30.1 (p. 494): Grade a module

```
mkdir -p seg && git -C work/jim archive origin/develop model/requirements/segment | tar -xm
-C seg
perl "$KIT/tools/sysml/req-recipe.pl" seg/model/requirements/segment 2>&1 | grep
"requirements=\|score 9[0-9] "
```

The two lowest are TW-SEG-003 and TW-SEG-004. Rewrites that pass the strict check:

```
cat > seg-rewrite.txt <<'REQ'
(U) While in observing mode, the power system shall report the battery state of charge to
the data logger every 10 minutes with an accuracy of 5 % of rated capacity, except during
maintenance mode. (TW-SEG-003)

(U) Upon a failed transmission, the telemetry unit shall retry the transmission at least 3
times within 10 minutes, except during maintenance mode. (TW-SEG-004)
REQ
perl "$KIT/tools/sysml/req-recipe.pl" --strict seg-rewrite.txt 2>&1 | tail -1
```

Exercise 30.2 (p. 494): Break the trace on purpose

```
back=$(git rev-parse --abbrev-ref HEAD)
git switch -q -c try-ratchet origin/develop
f=$(grep -rl "satisfy TW_SYS_003 by IngestService;" model)
sed -i '/satisfy TW_SYS_003 by IngestService;/d' "$f" && git commit -qam "drop one satisfy"
CI_PIPELINE_SOURCE=merge_request_event CI_MERGE_REQUEST_TARGET_BRANCH_NAME=develop perl
ci/job.pl trace 2>&1 | tail -1
git switch -q "$back" && git branch -q -D try-ratchet
```

The line names TW-SYS-003. Note that the job also needs `CI_PIPELINE_SOURCE=merge_request_event`: with only the target branch set it treats the run as a branch pipeline and says “not a merge request: reported only”.

Exercise 30.3 (p. 494): Write a discrepancy report

For TW-SEG-004 (no time window on the retry), in under a hundred words:

```
# DR: TW-SEG-004 retry has no time bound

What we found: TW-SEG-004 says the telemetry unit shall retry a failed transmission, with no
count or time
window (recipe finding, recipe-doors.txt).
Why it matters: a test cannot pass or fail it; any retry, however late, complies.
What we ask: state the count and the window, for example "at least 3 times within 10
minutes", and the mode in
which it applies.
```

Exercise 30.4 (p. 494): Read the record of a tabletop

```
perl "$KIT/bin/gitlab.pl" --local gitlab --project tidewater/model issue show 108 | sed -n
'1p;$p'
```

The record says only that it was done. The three bullets, as a model answer:

- **What we did:** pushed a file marked above the repository's level to a feature branch, then ran the spillage procedure: stop, call the ISSM, do not delete or rewrite anything ourselves.
- **What surprised us:** the copy was already in three places (the runner's build folder, a job artifact and a clone) before anyone noticed.
- **What we would change:** a marking check in the pipeline's first stage, and the ISSM's phone number in the README.

Exercise 31.1 (p. 510): Audit baseline-0

```
back=$(git rev-parse --abbrev-ref HEAD)
git checkout -q baseline-0
CI_COMMIT_TAG=baseline-0 perl ci/job.pl baseline 2>&1 | grep "rebuild"
f=$(git ls-files docs | head -1) && echo "x" >> "$f"
echo "after a docs edit: $(CI_COMMIT_TAG=baseline-0 perl ci/job.pl baseline 2>&1 | grep
rebuild)"
git checkout -q -- . && sed -i 's/\bThermometer\b/AirThermometer/g'
model/physical/stations/Stations.sysml
echo "after a model edit: $(CI_COMMIT_TAG=baseline-0 perl ci/job.pl baseline 2>&1 | grep
rebuild)"
git checkout -q -- . && git checkout -q "$back"
```

A changed character under `docs/` does *not* trip the audit: the job regenerates the documents from the model and hashes what it built, so the hand edit is overwritten. A change to the model does trip it, because the documents it generates no longer match the tag. The exercise's docs edit therefore shows the opposite of what it implies; edit a model file to see the failure.

Exercise 31.2 (p. 510): Two measures, one slide

- "Verify coverage: 21 of 30 requirements (70%) have a verification case that claims them. Unchanged since drop 1; the nine gaps are the prime's."
- "Verification status (VCRM): 2 of 30 requirements have a recorded verdict, both by inspection. Expected: the program has not reached test."

An outside reader should say "most requirements have a plan to verify them, and two are verified so far". If they say "verification dropped", the slide still needs work.

Exercise 31.3 (p. 511): Fix the plan, not the sprint

Split TW-112 so its first task fits before Bob's leave: "BuoyModes states and transitions written from the v1 diagram" (1 point, Bob, 14 to 17 December) inside the sprint, and "BuoyModes triggers, guards and review" (2 points) in sprint 7. Done for the first task, as a pipeline check: the `grammar` and `lint` jobs pass on the merge request, the BuoyModes TODO line is gone (the `todo` job's count drops by one), and the outline job reports no new violation.

Exercise 31.4 (p. 511): An assumption to the prime

```
# DR: confirm when a station's health report is "late"

What we found: TW-SYS-009 does not define when a health report is late. The data exchange
table (baseline-0)
carries our assumption, written in Interfaces::health: late = two missed transmissions.
Why it matters: the data center's station-down alarm, and its verification case, depend on
the number.
What we ask: confirm the assumption, or give the period and the count, in the next drop.
```

If the prime disagrees, the `health` flow's doc text changes, the data exchange table regenerates, and the alarm logic and verification case in DataCenter change with it; the `prime-tbd` label closes with the drop that settles it.

O.5 Part IV: making the case

Exercise 32.1 (p. 518): Run the report yourself

```
perl "$KIT/bin/gitlab.pl" --local gitlab --project tidewater/model analytics --since
2026-11-02 --until 2026-11-13 | sed -n '/Value stream/,/^$/p;/^3\. CI/p'
perl "$KIT/bin/gitlab.pl" --local gitlab --project tidewater/model analytics --since
2026-12-14 --until 2026-12-24 --by week | grep "week of" | head -2
```

The longest stage is *issue created to first commit*, about four days at the median. That is not a problem: the team files the sprint's issues at the kickoff, so each one waits for its turn by design. The stages after the first commit (opening the merge request, review, merge) take hours, and they are the ones that measure the flow.

Exercise 32.2 (p. 518): Translate a DORA measure

- **Deployment frequency:** "How often the published model is updated: the model site rebuilt, or a baseline released."
- **Lead time for changes:** "How long a change takes from the first edit to being part of the accepted model."
- **Change failure rate:** "How often an accepted change had to be corrected or undone."
- **Time to restore:** "When the accepted model fails its checks, how long until it passes

again."

Leave out deployment frequency, because it mostly counts the nightly schedule, and change failure rate, because at zero it says nothing yet. Keep lead time and time to restore: they answer "how fast" and "how safe".

Exercise 32.3 (p. 518): Find the trap

The final version is right. The draft's line, "6.7% verified (VCRM) (down 63.3 from 70)", compares two different measures: the VCRM's *verdict recorded* against the trace's *case exists*. Read out, it would have told the program manager that verification collapsed in sprint 6, when nothing got worse: a stricter measure appeared. The review record keeps verify coverage at 70% and reports the VCRM separately.

Exercise 33.1 (p. 525): Read it like a skeptic

A model answer:

Measure	The question	The evidence
2, change lead time	"Is 3.5 hours a real change, or only typo fixes?"	the merge request list in the analytics output, per author, and each MR's diff
4, trace answer time	"Who timed 22 seconds, and on what?"	the trace job's duration in the pipeline logs (`gitlab/pipelines/…/trace.log`)
12, morale	"Is three people's anonymous answer meaningful?"	`sprints/NN/morale.json`, with the question wording

The skeptic is right to ask: with three people, morale is a conversation starter, not a statistic, and the scorecard should say so.

Exercise 33.2 (p. 525): The measure that missed

"We set out to remove both package cycles inherited from the old model; we removed one, and the other is still there because a second reference between DataCenter and Stations remains. Finding and removing that last reference is planned for sprint 7, and the complexity job reports the count on every change, so you will see when it reaches zero." The briefing's own sentence is shorter and says the same thing (Section 35.5, Slide 3: the scorecard, sprint by sprint (p. 534)).

Exercise 33.3 (p. 525): Your own baseline

The answer is your program's numbers. A good one says, for each: lead time (a change board log: days from a change request to its approval, *measured* if the log has dates), trace answer time (how long the last "which requirements are unverified?" question took, usually *remembered*), and review turnaround (comment-matrix dates, *measured* if the matrices are filed). Mark remembered numbers as such on every slide that uses them.

Exercise 34.1 (p. 531): Draw your own sample

```
perl "$KIT/bin/xmi2sysml.pl" --report --layout-map "$KIT/examples/tidewater-v1/layout.map"
--map map15.csv --out v1check15 drops/v1.0/tidewater.xmi > /dev/null
perl -ne 'next if $. == 1; my @f = split /,/; print "$f[2] -> $f[4]\n" if $f[4] =~ /^(part
def|requirement|port|connection|action def|satisfy)/ && $. % 15 == 0' map15.csv
git clone -q -b develop gitlab/repos/tidewater/model.git port15
grep -n 'name="IngestService"' drops/v1.0/tidewater.xmi
grep -n -A2 "part def IngestService" port15/model/physical/datacenter/DataCenter.sysml
grep '"TW-STK-005"' drops/v1.0/tidewater.csv
grep -h "TW_STK_005 {" port15/model/requirements/stakeholder/*.sysml
```

- **IngestService:** same. The class became a part def of the same name, with its documentation and ports.
- **Observation history (TW-STK-005):** same, reworded. The v1 class was named by its title; the v2 requirement is named by its DOORS id, keeps the title as an attribute, and carries the DOORS text in its `doc`.

Exercise 34.2 (p. 531): Explain a difference

Two folds from drop 1's map:

- **The multiplicity of a part** (a `LiteralInteger` and a `LiteralUnlimitedNatural`, two v1 elements) folds into the `[4]` on the usage: v2 writes the bounds inline, so nothing is lost; there are just fewer elements.
- **A generalization** folds into the `:>` of the definition, for example `attribute def Power :> PowerValue`. v1 stores the inheritance as its own element; v2 writes it as part of the declaration.

The test of a fold is the same each time: can you point at the v2 text that carries the information? If so, it is not a loss.

Exercise 35.1 (p. 537): Rewrite a slide headline

- "Every model change is now checked automatically before anyone accepts it."
- "The model answers in seconds questions that used to take days."
- "Twelve weeks in, two thirds of the team's work is systems engineering again."

A program manager remembers the second: it names a pain they have felt, and a number.

Exercise 35.2 (p. 537): Rehearse the demo

```
git clone -q -b develop gitlab/repos/tidewater/model.git demo-verify
cd demo-verify && git checkout -q -b scratch/demo-verify
perl -0pi -e 's/^(\s*)verify TW_SYS_003;\n//m' model/verification/Verification.sysml
git commit -qam "demo: drop one verify"
CI_PIPELINE_SOURCE=merge_request_event CI_MERGE_REQUEST_TARGET_BRANCH_NAME=develop perl
ci/job.pl trace 2>&1 | tail -1
```

In one sentence: "This change removed the only test for TW-SYS-003, and the check caught it before anyone accepted the change."

Exercise 35.3 (p. 537): Find your ask

A model ask: "Approve the runner machine request by 13 November. Without it, the automated checks run only on our laptops, nothing stops an unchecked change, and the December baseline cannot be rebuilt by anyone but us." One decision, one date, one consequence the decision-maker cares about.

Exercise 36.1 (p. 544): Read a quad

docs/tidewater/sprints/04/quad-2026-11-27.txt

```
WATCH ITEMS / PM HELP NEEDED
  (WI) TW-67      carried to next sprint
```

The one watch item is TW-67, the Stations level tags story, carried into the next sprint. `(WI)` marks a watch item (for awareness); `(PM)` would ask for the program manager's help. The program manager should note it and do nothing: the team owns it, and the story finishes early the next sprint.

Exercise 36.2 (p. 544): Write the note

```
perl "$KIT/tools/gitlab/talk.pl" --local gitlab --project tidewater/model weekly --date
2026-11-13
```

A note to send:

> *On track. Three stories finished this week; 50 converter TODO lines remain; 21 of 30 requirements have a verification case (70%); the nine without are the prime's and are reported. One risk: the runner's security review has waited three days for the ISSM. Ask: a date from the ISSM by Tuesday.*

The draft lists no risk, because the ISSM wait was in stand-up chat, not in GitLab. Jim adds what only people know.

Exercise 36.3 (p. 544): Map your own template

A typical program weekly template: accomplishments (merged MRs and closed stories), next week (the open milestone's issues), schedule (the quad's milestones, from the journal), risks (blocked issues and old discrepancy reports), metrics (the pipeline's scorecard), staffing (no source), budget (no source). Staffing and budget have no source in the system;

the lead writes them, or they come from the finance system, and the note says which.

Exercise 37.1 (p. 555): Ask for what you need

The finished memo passes the check silently and builds:

```
mkdir -p docs/memos
cp "$KIT/docs/tidewater/memos/2026-10-21-request-for-enablers.md" docs/memos/
perl "$KIT/tools/memo/memo-build.pl" --check docs/memos/2026-10-21-request-for-enablers.md
&& echo "check: exit 0"
perl "$KIT/tools/memo/memo-build.pl" docs/memos/2026-10-21-request-for-enablers.md
```

Your own asks, as a table row each: "Training time, 4 points per person in sprint 1 and 2 in sprint 2; so the team learns on the board, not on weekends; by the sprint 1 kickoff; if not, the transition takes twice as long." Do the same for the runner (by sprint 2, or checks stay on laptops), the tool approval, and the GitLab tier decision.

Exercise 37.2 (p. 555): Record a decision

The memo for record is signed, dated and addressed to a reader outside the team: it is what an auditor or the contracting officer files. The ADR lives beside the model, is versioned with it, and is read by the next engineer. The memo carries authority; the ADR carries context and is easy to supersede. Keep both for decisions that bind the program (the branching model), and only the ADR for decisions that bind the team.

Exercise 37.3 (p. 555): Build from Vim

Yes: the `.txt` diff shows the changed paragraph as a few changed lines, with the rest unchanged, because the build wraps text the same way every time. The PDF diff is binary and unreadable. That is why the team commits the `.txt` beside the PDF.

Exercise 37.4 (p. 555): Draft the progress memo from evidence

From the pipeline: TODO lines, trace numbers, pipelines passing. From GitLab: stories done, merge requests, review turnaround. From you: the plan comparison, the risks only people know (the ISSM's wait), and the document-era baselines. A memo with more of the third kind than the first two needs more evidence before it goes out.

Exercise 38.1 (p. 562): Run the pulse on your own team

The answer is your team's data. Tidewater's dip came in sprint 3, when the new tools were in use and the old shortcuts were gone; most teams predict sprint 2. A prediction made in advance and checked afterwards is the point: it tests whether you know your team.

Exercise 38.2 (p. 562): Write your team's objections down

The answer is your team's list. The objections still open after six sprints are usually about the supplier ("the prime will never change its model") or the organization ("the CCB will not accept a merge request"); the tool objections ("git is too hard") are usually answered by sprint 3, with evidence.

Exercise 39.1 (p. 568): Write your own two-column day

Rows that change most: waiting for a review (hours instead of weeks), finding the current version (the branch, not an e-mail hunt), and building status slides (generated). Rows that do not change: thinking about the design, talking to the people who own the requirements, and meetings with the supplier.

Exercise 40.1 (p. 573): Grade a document both ways

The pipeline is right about what it can check (references resolve, nothing is missing, the text matches DOORS); the hand grade is right about what it cannot (whether the design is good, whether the reader understands it). Where they disagree on something the pipeline checks, trust the pipeline and find out why the hand grade missed it.

Exercise 41.1 (p. 583): Budget the learning

A model plan: a fifth of sprint 1, a sixth of sprint 2, a tenth of sprint 3, as `work::learning` stories with practice lessons as acceptance criteria. The engineering scope that moves is usually the second package of conversion work: with learning on the board, sprint 2 holds one package, not two (Section 50.6, The learning budget (p. 677)).

Exercise 41.2 (p. 583): Choose the first win

Three typical questions: "which requirements have no verification?" (the trace job), "what changed in the supplier's last drop?" (the converter's delta), "who changed this requirement and when?" (git blame on the requirements file). The trace is the one for sprint 1: it needs only the drop and the converter, and the skeptic who has answered it by hand for years is the right person to run it.

Exercise 41.3 (p. 583): Say no with a date

- "Not this sprint: the special report needs the trace job, which is ready in sprint 3. Today I can send last review's numbers."
- "Not today, unless it replaces something: the model change goes on the board for Wednesday's refinement. If it is urgent, tell me which story it displaces."
- "Not until the sprint ends on the 23rd: Bob is committed to the drop import. From the 26th, for two weeks."

Exercise 41.4 (p. 584): Read the signs

The answer is your team's. The common signs: stand-up lines that list activities with no outcome, merge requests open for days, a red `develop` nobody mentions. Change one practice per sprint, and tell management the sign and the fix in the same sentence.

O.6 Part V: living with it

Exercise 42.1 (p. 601): Assign the roles

- **Model lead: Maintainer.** Owns protections and baselines; the only one who can change who approves what.
- **Two modelers: Developer.** They push branches and open merge requests, but cannot merge into protected branches alone or change the rules that check them.
- **CM engineer: Maintainer**, for tags and releases, with the protections still requiring another person's approval on merges, so running the pipeline does not mean approving content.
- **Contractor liaison: Reporter.** Reads the model and comments on issues, cannot push: the supplier's view never becomes the supplier's write access.

Exercise 42.2 (p. 601): Choose the token

- (a) The job's own `CI_JOB_TOKEN`, which can publish to the project's package registry; no stored token.
- (b) A project access token with `read_api`, stored as a masked, protected variable, with an owner and expiry.
- (c) Jim's personal access token with `api`, short expiry, kept out of any file; or a project access token with the Maintainer role if the script becomes routine.
- (d) A deploy token with `read_repository` only.

Exercise 42.3 (p. 601): Find the drift

```
cp "$KIT/tools/gitlab/templates/project.conf" drift.conf
ga() { perl "$KIT/bin/gitlab.pl" --local drift --project tidewater/model "$@"; }
for u in jim bob tom; do ga user $u > /dev/null; done
ga --as tom setup drift.conf > /dev/null 2>&1
sed -i 's/^\(develop = .*\)force:no/\1force:yes/' drift.conf
ga --as tom setup drift.conf 2>&1 | grep "protected branch develop"
perl "$KIT/bin/gitlab.pl" --url https://gitlab.example.com --project tidewater/model
--dry-run setup drift.conf | grep '"name":"develop"'
sed -i 's/^\(develop = .*\)force:yes/\1force:no/' drift.conf
ga setup --diff drift.conf 2>&1 | grep "^\[branches\]"
```

Setup prints the protected branch with `force:yes`. The preview lists every request because a dry run reads nothing from the server: it cannot know what is already set, so it shows the whole file. A change made in the web pages shows up with `setup --diff`, which compares

the file with the live project (the last line above, after the file was put back); re-running `setup` puts the setting back. Two corrections to the book: `--dry-run` now works with `--local` too (it prints and applies nothing), and the read-only comparison the admin chapter says the kit lacks is `setup --diff`.

Exercise 42.4 (p. 601): Plan a restore test

The checklist: import the export into a scratch group; check the repository, branches and tags; issues, merge requests and comments; labels and milestones; the wiki; then re-create what an export leaves out on purpose. An export carries no CI/CD variables or secrets, no runners, no protected-branch and tag rules in some versions, no group-level settings, and no container or package registry contents. Those items fail on any server, by design; they must be re-applied from `project.conf` and the token inventory.

Exercise 42.5 (p. 601): Make the tier case

"On Free, the rule that a model change needs another engineer's approval is a habit we record; on Premium, GitLab refuses the merge without it, and the code owner of the interface files must be one of the approvers. For a model the government will baseline under contract, an approval the server enforces is the evidence an auditor needs. The cost is a few seats a year against one finding at a configuration audit."

Exercise 43.1 (p. 611): Run a drop through the loop yourself

The delta from drop 2 to drop 3 marks five elements manual, all of the storm mode:

```
for n in 2 3 4; do perl "$KIT/sim/tidewater-v1.pl" --drop $n pd/d$n > /dev/null; done
perl "$KIT/bin/xmi2sysml.pl" --delta pd/d2/tidewater.xmi pd/d3/tidewater.xmi | grep
"^manual"
perl "$KIT/bin/xmi2sysml.pl" --delta pd/d2/tidewater.xmi pd/d3/tidewater.xmi | grep -B1
"manual:" | grep -v "^--" | head -3
```

NWM-CR-0015's new state, two transitions and two triggers (`stormWarning`, `calm`) sit inside the BuoyModes state machine, which the converter leaves as a TODO line: there is no v2 text for them to land in, so an engineer adds them to the hand-written state machine. The fuse, the deleted requirement, the new satisfy and the deleted duplicate block are all carried.

Exercise 43.2 (p. 611): Cut the next baseline

The changelog section names the baseline and date, the drop merged (v1.2) and its change requests (NWM-CR-0015 to 0019, with the storm mode marked "reworked by hand"), and the discrepancy reports the drop closed. The release links to the package (model and documents tarball, VCRM, toolchain inventory, checksum file) and names the sprint milestones it closes; the tag message records the documents' checksum.

Exercise 43.3 (p. 611): A hotfix that the prime must also fix

"In baseline-1 the requirement TW-SYS-007 is satisfied by BulletinPublisher, which does not provide the public interface the requirement is about; we corrected it to DataSegment::publicOut in baseline-1.1. That relationship is generated from your v1 model on every drop, so the next drop will put the old satisfy back and our correction will appear as a hand edit to redo. Please change the satisfy in your model to the public interface, and tell us which drop carries it."

Exercise 44.1 (p. 618): Read the matrix

```
git checkout -q develop
perl ci/job.pl vcrm > /dev/null && grep ",no case," vcrm.csv
grep "^unverified:" ../../gitlab/pipelines/tidewater-model/29/trace.log
```

TW-SYS-011, for example, has an analysis method and no case; the trace lists it as unverified, and it has no satisfy gap. The case is the prime's to write, because the prime owns the system model's verification; ask through a discrepancy report naming the requirement, its method, and the drop you need the case in.

Exercise 44.2 (p. 618): Map your program to the stages

The answer is your program's table. The five stages are traceability first (government, now), automated result capture (the prime or test organization, once their harness can write the format), a parallel pilot on one subsystem (joint, between campaigns), shifting checks left (government and prime, after the pilot), and scaling and retiring legacy artifacts (with the customer's agreement). A good answer names the campaign each one must not interrupt, such as a qualification test series.

Exercise 45.1 (p. 635): Read a drop like a change board

```
perl "$KIT/bin/xmi2sysml.pl" --out g3/model --layout-map
"$KIT/examples/tidewater-v1/layout.map" --map m3.csv pd/d3/tidewater.xmi > /dev/null
perl "$KIT/bin/xmi2sysml.pl" --delta pd/d3/tidewater.xmi pd/d4/tidewater.xmi --impact
g3/model --map m3.csv | sed -n '/^== NWM/p;/impact summary/,$p'
```

Change request	Disposition	Why
NWM-CR-0020 salinity	accept	a new field in the observation record, with a reason; the converter carries it
NWM-CR-0021 drifting buoys	accept with impact	a new part in the station segment that touches both network interfaces
NWM-CR-0022 TW-SYS-016	accept	it answers the government's own discrepancy report
NWM-CR-0023 TW-SYS-002 interval	accept with impact	the requirement changed, so its verification case must change

The drifting buoys add government work: the observation and health links now carry a new kind of station, so the interface owners review the links, the data exchange table regenerates, and the verification of the links needs a drifting-buoy case.

Exercise 45.2 (p. 635): Write the clarification

```
# DR: NWM-CR-0025 renames PublishBulletin with no rationale

What we need: the reason for renaming the activity PublishBulletin to IssueBulletin, or a
withdrawal of the rename.
Why it matters: the name is referenced by the government's hand-written behavior, by test
procedures and by
documents outside the model. Accepting it means changing all of them.
By when: with drop v1.5, so the change board can disposition both together.
Disposition until then: reject, clarify.
```

Exercise 45.3 (p. 635): Break the intake on purpose

```
perl "$KIT/tools/gitlab/kickoff.pl" scaffold model --project tidewater/model --title
Tidewater > /dev/null
cp -r pd/d4 pd/d4-copy && sed -i '0,/TW-SYS-001/s/TW-SYS-001/TW-SYS-00l/'
pd/d4-copy/tidewater.reqif
cd model
perl "$KIT/bin/gitlab.pl" ci --event trigger --branch develop --job intake --var
DROP_VERSION=v1.3 \
    --var INTAKE_DROP_DIR="$PWD/../pd/d4-copy" --logs ../intake-1 > /dev/null 2>&1 || echo
    "exit status $?"
grep -E "^(MISMATCH|REJECTED)" ../intake-1/intake.log
perl "$KIT/sim/tidewater-v1.pl" --drop 4 ../pd/d4-copy > /dev/null
perl "$KIT/bin/gitlab.pl" ci --event trigger --branch develop --job intake --var
DROP_VERSION=v1.3 \
    --var INTAKE_DROP_DIR="$PWD/../pd/d4-copy" --logs ../intake-2 > /dev/null 2>&1
grep "^ACCEPTED" ../intake-2/intake.log
```

The first run rejects the drop and names the file with both checksums; the second accepts it. Regenerating is the only acceptable fix because the manifest's checksums describe what the prime built: editing the file back by hand might match by luck or not at all, and editing the manifest would make the check worthless. On a real program the "regenerate" step is the prime's build sending the drop again.

Exercise 45.4 (p. 635): Choose a strategy

In `model/external/prime/`, generated and never edited: the system and segment requirements allocated to the data center, the port and interface definitions the data center connects to (`ObservationIF`, `HealthIF`, the observation record), and the parameters the data center depends on (reporting intervals). The data center team owns its own design: IngestService, QualityControl, the archive and the publisher, their behavior, the satisfy links to the generated requirements and the verification cases.

Exercise 46.1 (p. 643): Label your backlog

Tidewater's sprint 7 plan: "Break the Stations and DataCenter cycle", "The DailyUpload interaction by hand", "The EnergyBalance constraint expression" and "Retire the Tidewater Profile TODO" are engineering. "Pin the kit version" is platform, and makes every later engineering story cheaper to verify: a baseline rebuilt next year uses the same checks as the one cut today. If a platform story cannot name the engineering it speeds up, it can probably wait.

Exercise 46.2 (p. 644): Split along the decisions

1. Find every reference between Stations and DataCenter and decide which are real design dependencies (2 points).
2. Decide where the shared element belongs (the library, or one segment) and move it (2 points).
3. Remove or redirect the remaining accidental references, and confirm with the complexity job that the cycle count is zero (1 point).

The plan has one task, "Remove the QualityControl to SensorSuite dependency" (2 points). That is the split along a command, and it is why the cycle survived: it removed the dependency it was written against, and nobody had looked for the other reference first.

Exercise 47.1 (p. 649): Measure a model you own

On Tidewater, as a stand-in:

```
git checkout -q develop
perl ci/job.pl complexity 2>&1 | sed -n '/^cycles:/,/^upward/p;/^max_fan_out/p'
```

The DataCenter and Stations cycle is an accident: two trace-style dependencies drawn on v1 diagrams, not a design need, since the two segments exchange data through the network's interfaces. The largest fan-out (3) is the network package, which is a real dependency: the network is what connects the segments.

Exercise 47.2 (p. 650): Mark the twenty practices for your project

The answer is your table. For most teams the three with the biggest effect in the next sprint are layering with a one-way rule (1), small reviewable changes (17), and generated views (12), because each one prevents new tangle without needing the old tangle fixed first.

Exercise 48.1 (p. 660): Close a boundary crossing

Rebuild the worked example, add the laptop and its interface, and run the gate:

```
mkdir -p secgate && cd secgate
awk '/expect="5 package"/ || /expect="0 error.s., 0 warning.s. -> PASS"/ {p=1; next} p &&
/^```$/ {p=0} p' "$KIT/docs/src/primer/26e-security.txt" > build.sh
sh build.sh > /dev/null 2>&1
```

```
f=model/physical/Network.sysml
sed -i 's#^    port def ObservationPort .*#&\n    port def MaintenancePort { doc /* A
maintenance session into the data center. */ }#' $f
sed -i 's#^        part publicSite : PublicSite;#&\n        part laptop :
MaintenanceLaptop;#' $f
sed -i 's#^        satisfy secR01 by dataCenter;#        interface ifcMaintenance { end ::>
laptop.service; end ::> dataCenter.maintIn; attribute fromZone = "untrusted"; attribute
toZone = "trusted"; attribute secReq = ""; }\n&#' $f
sed -i 's#port publicOut : ObservationPort; }#port publicOut : ObservationPort; port maintIn
: ~MaintenancePort; }#' $f
sed -i 's#^}$#    part def MaintenanceLaptop { @TrustZone { zone = Zone::untrusted; } doc /*
A technician laptop, brought to the data center. */ port service : MaintenancePort; }\n}#'
$f
perl "$KIT/tools/sysml/model.pl" gate 2>&1 | grep -E "error: interface
.ifcMaintenance.|FAIL" || true
perl "$KIT/tools/sysml/model.pl" gate > /dev/null 2>&1 || echo "exit status $?"
```

Then the requirement, the satisfy and the verification:

```
sed -i 's/attribute toZone = "trusted"; attribute secReq = ""; }/attribute toZone =
"trusted"; attribute secReq = "secR03"; }/' model/physical/Network.sysml
sed -i 's/^        satisfy secR02 by dataCenter;/&\n        satisfy secR03 by dataCenter;/'
model/physical/Network.sysml
sed -i 's#^}$#    requirement secR03 { attribute doorsId = "TW-SEC-003";
@SecurityRequirement; doc /* The data center shall accept a maintenance session only from a
laptop the program issued, authenticated by its hardware token. */ }\n}#'
model/requirements/SecurityRequirements.sysml
sed -i 's#^}$#    verification def TestMaintenanceAccess {\n        doc /* Connect an
unissued laptop and an issued one; confirm only the issued one gets a session. */\n
subject network : ObservationNetwork;\n        objective { verify secR03; }\n    }\n}#'
model/verification/SecurityVerification.sysml
perl "$KIT/tools/sysml/model.pl" gate | tail -1
```

The engineering decisions: which zone a maintenance laptop belongs in (untrusted, because it travels), what the data center should trust about it (an issued device and a token, not a password), whether the requirement also mitigates a threat worth adding to the register (an unauthorized maintenance session), and how to verify it (a test with a laptop that should be refused). The `sed` lines are the typing.

Exercise 48.2 (p. 661): Your twenty-six

The answer is your table. Most ISSMs ask first about point 1 (what the model contains, and at what level), point 4 (separating repositories by sensitivity) and point 9 (runners as privileged machines), because those decide whether the system can be authorized at all. Every LATER needs a story or epic name; "later" without one is the pattern the chapter warns about.

Exercise 49.1 (p. 670): Name the cadences

A good answer separates the segments by what limits them, not by who builds them. Hardware is limited by lead times, qualification and field access; software by the pipeline. The baselines fall where the slowest segment can freeze: for Tidewater, the station segment's field seasons set them, and the data center's sprints line up underneath.

Exercise 49.2 (p. 670): One copy of the interface

It is defined once in model/library/Interfaces.sysml and used in the stations, the data center and the observation cycle's behavior (model/physical/stations, model/physical/datacenter, model/logical). CODEOWNERS gives that file two sections, both of which must approve: the systems lead (Jim) and one of the people who build on both sides (Bob or Tom, whoever did not write the change). If a segment could change it alone, the other segments would find out at integration; with one copy and system-level owners, they find out in the merge request.

Exercise 49.3 (p. 671): Find the next block in the drops

Drop 6 (v1.5): its CHANGES.csv carries NWM-CR-0026, "Add partner sharing TW-SYS-019", with the rationale of a data-sharing agreement with partner agencies; it adds TW-SYS-019, the data center sharing observations with partner agencies. It is Block 3 in the picture, starting while Block 2 is still in verification.

Exercise 49.4 (p. 671): Put a check on its rung

The missing `verify` is a model check, on every merge request. The link model belongs to simulation. The storm power loss is a sea-trial finding that then calibrates the simulation. The late bulletin is a simulation result (load) that a harbor test should confirm end to end. The modem reset belongs on the bench, with the real modem and logger.

Exercise 50.1 (p. 683): Your twelve-week table

The answer is your program's table. A good row reads like Tidewater's: "Sprint 2: the first drop on the integration branch, every change gated. Done when: the drop's merge request merged with a green pipeline, and a direct push to `develop` is refused." Each goal names an outcome a manager recognizes; each "done when" is a pipeline result, a record or a merged change.

Exercise 50.2 (p. 683): Your decision records

Each ADR has the template's four parts: context, decision, consequences and when to revisit (Exercise 13.4 (p. 213) has a model). The merge-method record is the one most teams get wrong: if you will measure lead time or review turnaround from history, choose merge commits, and say so as the reason.

Exercise 50.3 (p. 683): Your sprint 1

```
perl -MText::ParseWords -ne 'next if $. == 1; s/\r//; my @c = parse_line(",", 0, $_);
$p{$c[6]} += $c[4] if $c[6] =~ /^[12]$/;
  END { print "sprint $_: $p{$_}\n" for sort keys %p }'
  "$KIT/tools/gitlab/templates/backlog.csv"
```

Ten and twelve points as written, before multiplying the "one per person" tasks by the team's size (for three people, sprint 1 grows by five points). Against a three-person capacity of about nineteen with a fifth reserved for learning, it fits. What to cut first: the stories your program does not need (no supplier, no drop intake), then anything that is not on the path to the first gated change.

Exercise 50.4 (p. 683): Run week zero offline

```
mkdir -p harbor && cd harbor
perl "$KIT/sim/tidewater-v1.pl" --drop 1 first-drop > /dev/null
perl "$KIT/tools/gitlab/kickoff.pl" start --yes --local gitlab --project harbor/model
--title Harbor \
    --drop first-drop --version v1.0 > start.log 2>&1 || echo "exit status $?"
grep "week zero is set up" start.log
perl "$KIT/bin/gitlab.pl" --local gitlab --project harbor/model issue list
```

Five stories: the protected project, the first pipeline, receiving the drop, converting it, and the working agreements. Programs without a supplier rewrite the two drop stories; most programs rewrite the conversion story's acceptance criteria to name their own packages.

Exercise 51.1 (p. 693): Diagnose the symptom

- (a) **Reformatting the converter's output** (Section 51.2.2, Reformatting the converter's output (p. 685)). Diff the government's file against the vendor branch with whitespace ignored, and revert everything but the level tags.
- (b) **Changing requirement text in the model** (Section 51.2.4, Changing requirement text in the model (p. 686)). Find the merge request in the drift report and restore the DOORS wording; send the improvement to the prime as a discrepancy report.
- (c) **Trusting the TODO count alone** (Section 51.2.5, Trusting the TODO count alone (p. 686)). Run the converter's `--delta` on the drop and rework the manual lines by hand.
- (d) **Red is normal** (Section 51.4.1, Red is normal (p. 688)). Stop merging, name an owner for the red pipeline today, and fix it before anything else.

Exercise 51.2 (p. 693): Your two pitfalls

The answer is your organization's. A good one names a symptom you can observe in your own tools ("the drift count rose after our MR"), not a feeling, and a rule that a setting or a

check enforces, not a reminder.

Exercise 51.3 (p. 693): Prove the TODO count blind spot

```
for n in 2 3; do perl "$KIT/bin/xmi2sysml.pl" --out conv$n pd/d$n/tidewater.xmi > /dev/null;
done
diff conv2/stations/Stations.sysml conv3/stations/Stations.sysml || true
echo "storm lines in the converted drop 3: $(grep -ci storm conv3/stations/Stations.sysml)"
perl "$KIT/bin/xmi2sysml.pl" --delta pd/d2/tidewater.xmi pd/d3/tidewater.xmi | grep
"BuoyModes::Storm$"
```

The converted files differ only by the fuse. The storm mode appears only in the delta: it is inside the BuoyModes state machine, which the converter writes as one TODO line in both drops, so the TODO count does not change while the content behind it does.

O.7 Appendices

Exercise I.1 (p. 787): Write a story

```
gs() { perl "$KIT/bin/gitlab.pl" --local story-gl --project tidewater/model "$@"; }
for u in jim bob tom; do gs user $u > /dev/null; done
gs --as tom setup "$KIT/tools/gitlab/templates/project.conf" > /dev/null 2>&1
gs --as jim issue new "Close the DataCenter TODO lines" --template
"$KIT/tools/gitlab/templates/.gitlab/issue_templates/Story.md" \
   --label type::story --milestone "Sprint 5" --assign bob > /dev/null
for t in "Level tags in DataCenter|1" "Units on the DataCenter attributes|2" "Review and
merge|1"; do
  gs --as jim issue new "${t%|*}" --type task --description "Part of #1
/estimate ${t#*|}d
/assign @bob
/relate #1" > /dev/null
done
gs issue list
```

Each task description's quick actions ran: the estimate, the assignee and the link to the story. The story's own template carries the label and milestone lines.

Exercise I.2 (p. 787): Find the features

A model answer for three samples: the merge request sample uses headings, a table, task lists, an issue reference and a closing keyword; the review sample uses mentions and a suggestion block; the release sample uses headings, milestone references and an alert. For the Discrepancy Report template, add a collapsible section for the evidence (the trace log excerpt), so the report stays short and the evidence travels with it.

Exercise I.3 (p. 788): Write a discrepancy report

```
perl "$KIT/bin/reqif2sysml.pl" --report --model drop1-v2 drop1/tidewater.reqif 2>&1 | grep
"^orphans"
```

```
# DR: TW-SYS-012 has nothing that satisfies it

What we found: in drop v1.0, no element satisfies TW-SYS-012 (trace report, orphans).
What we ask: add the satisfy relationship to the element that meets it in your model, or
tell us that the
requirement is allocated outside the model.
How we verify: the trace job on the next drop's import merge request no longer lists
TW-SYS-012 as an orphan.
```

The prime answered it in drop v1.2 with NWM-CR-0018, satisfying it by the service vessel.

Exercise L.1 (p. 807): Build your own plan

```
perl -e 'require "./sim/tidewater-plan.pl";
  my %h; for my $s (grep { $_->{n} <= 6 } @{ tw_plan() }) {
    for my $st (grep { $_->{work} eq "learning" } @{ $s->{stories} }) {
      for (@{ $st->{tasks} }) { $h{"S$s->{n} $_->{who}"} += 8 * $_->{p}; printf "S%d %-4s
      %4.1f h  %s\n", $s->{n}, $_->{who}, 8 * $_->{p}, $_->{t} } } }
  print "$_: $h{$_} h\n" for sort keys %h'
```

Sum the hours per person and sprint, as the last lines do, and compare them with the tables. A difference means the backlog reserves less learning than the plan promises; fix the backlog, not the table.